

Ada Reference Manual

ISO/IEC 8652:1995(E)
with Technical Corrigendum 1
and Amendment 1
and Amendment 2 (Draft 9)

Language and Standard Libraries

Copyright © 1992, 1993, 1994, 1995 Intermetrics, Inc.

Copyright © 2000 The MITRE Corporation, Inc.

Copyright © 2004, 2005, 2006 AXE Consultants

Copyright © 2004, 2005, 2006 Ada-Europe

Copyright © 2008, 2009, 2010 AXE Consultants

Ada Reference Manual - Language and Standard Libraries

Copyright © 1992, 1993, 1994, 1995, Intermetrics, Inc.

This copyright is assigned to the U.S. Government. All rights reserved.

This document may be copied, in whole or in part, in any form or by any means, as is or with alterations, provided that (1) alterations are clearly marked as alterations and (2) this copyright notice is included unmodified in any copy. Compiled copies of standard library units and examples need not contain this copyright notice so long as the notice is included in all copies of source code and documentation.

Technical Corrigendum 1

Copyright © 2000, The MITRE Corporation. All Rights Reserved.

This document may be copied, in whole or in part, in any form or by any means, as is, or with alterations, provided that (1) alterations are clearly marked as alterations and (2) this copyright notice is included unmodified in any copy. Any other use or distribution of this document is prohibited without the prior express permission of MITRE.

You use this document on the condition that you indemnify and hold harmless MITRE, its Board of Trustees, officers, agents, and employees, from any and all liability or damages to yourself or your hardware or software, or third parties, including attorneys' fees, court costs, and other related costs and expenses, arising out of your use of this document irrespective of the cause of said liability.

MITRE MAKES THIS DOCUMENT AVAILABLE ON AN "AS IS" BASIS AND MAKES NO WARRANTY, EXPRESS OR IMPLIED, AS TO THE ACCURACY, CAPABILITY, EFFICIENCY, MERCHANTABILITY, OR FUNCTIONING OF THIS DOCUMENT. IN NO EVENT WILL MITRE BE LIABLE FOR ANY GENERAL, CONSEQUENTIAL, INDIRECT, INCIDENTAL, EXEMPLARY, OR SPECIAL DAMAGES, EVEN IF MITRE HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Amendment 1

Copyright © 2004, 2005, 2006, 2007, AXE Consultants. All Rights Reserved.

This document may be copied, in whole or in part, in any form or by any means, as is, or with alterations, provided that (1) alterations are clearly marked as alterations and (2) this copyright notice is included unmodified in any copy. Any other use or distribution of this document is prohibited without the prior express permission of AXE.

You use this document on the condition that you indemnify and hold harmless AXE, its board, officers, agents, and employees, from any and all liability or damages to yourself or your hardware or software, or

third parties, including attorneys' fees, court costs, and other related costs and expenses, arising out of your use of this document irrespective of the cause of said liability.

AXE MAKES THIS DOCUMENT AVAILABLE ON AN "AS IS" BASIS AND MAKES NO WARRANTY, EXPRESS OR IMPLIED, AS TO THE ACCURACY, CAPABILITY, EFFICIENCY MERCHANTABILITY, OR FUNCTIONING OF THIS DOCUMENT. IN NO EVENT WILL AXE BE LIABLE FOR ANY GENERAL, CONSEQUENTIAL, INDIRECT, INCIDENTAL, EXEMPLARY, OR SPECIAL DAMAGES, EVEN IF AXE HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Amendment 2

Copyright © 2008, 2009, 2010, AXE Consultants. All Rights Reserved.

This document may be copied, in whole or in part, in any form or by any means, as is, or with alterations, provided that (1) alterations are clearly marked as alterations and (2) this copyright notice is included unmodified in any copy. Any other use or distribution of this document is prohibited without the prior express permission of AXE.

You use this document on the condition that you indemnify and hold harmless AXE, its board, officers, agents, and employees, from any and all liability or damages to yourself or your hardware or software, or third parties, including attorneys' fees, court costs, and other related costs and expenses, arising out of your use of this document irrespective of the cause of said liability.

AXE MAKES THIS DOCUMENT AVAILABLE ON AN "AS IS" BASIS AND MAKES NO WARRANTY, EXPRESS OR IMPLIED, AS TO THE ACCURACY, CAPABILITY, EFFICIENCY MERCHANTABILITY, OR FUNCTIONING OF THIS DOCUMENT. IN NO EVENT WILL AXE BE LIABLE FOR ANY GENERAL, CONSEQUENTIAL, INDIRECT, INCIDENTAL, EXEMPLARY, OR SPECIAL DAMAGES, EVEN IF AXE HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Ada 2005 Consolidated Standard

Copyright © 2004, 2005, 2006, Ada-Europe.

This document may be copied, in whole or in part, in any form or by any means, as is, or with alterations, provided that (1) alterations are clearly marked as alterations and (2) this copyright notice is included unmodified in any copy. Any other use or distribution of this document is prohibited without the prior express permission of Ada-Europe.

You use this document on the condition that you indemnify and hold harmless Ada-Europe and its Board from any and all liability or damages to yourself or your hardware or software, or third parties, including attorneys' fees, court costs, and other related costs and expenses, arising out of your use of this document irrespective of the cause of said liability.

ADA-EUROPE MAKES THIS DOCUMENT AVAILABLE ON AN "AS IS" BASIS AND MAKES NO WARRANTY, EXPRESS OR IMPLIED, AS TO THE ACCURACY, CAPABILITY, EFFICIENCY MERCHANTABILITY, OR FUNCTIONING OF THIS DOCUMENT. IN NO EVENT WILL ADA-EUROPE BE LIABLE FOR ANY GENERAL, CONSEQUENTIAL, INDIRECT, INCIDENTAL, EXEMPLARY, OR SPECIAL DAMAGES, EVEN IF ADA-EUROPE HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Table of Contents

Table of Contents	i
Foreword to this version of the Ada Reference Manual	xi
Foreword	xii
Introduction	xiii
Section 1: General.....	1
1.1 Scope	1
1.1.1 Extent	1
1.1.2 Structure	2
1.1.3 Conformity of an Implementation with the Standard.....	4
1.1.4 Method of Description and Syntax Notation.....	5
1.1.5 Classification of Errors	7
1.2 Normative References	8
1.3 Definitions.....	8
Section 2: Lexical Elements.....	9
2.1 Character Set.....	9
2.2 Lexical Elements, Separators, and Delimiters	11
2.3 Identifiers	12
2.4 Numeric Literals	13
2.4.1 Decimal Literals	13
2.4.2 Based Literals	14
2.5 Character Literals	14
2.6 String Literals	15
2.7 Comments.....	15
2.8 Pragmas	16
2.9 Reserved Words.....	18
Section 3: Declarations and Types	19
3.1 Declarations.....	19
3.2 Types and Subtypes	20
3.2.1 Type Declarations	22
3.2.2 Subtype Declarations	23
3.2.3 Classification of Operations.....	24
3.3 Objects and Named Numbers.....	25
3.3.1 Object Declarations	27
3.3.2 Number Declarations.....	29
3.4 Derived Types and Classes	30
3.4.1 Derivation Classes.....	33
3.5 Scalar Types	34
3.5.1 Enumeration Types	39
3.5.2 Character Types.....	40
3.5.3 Boolean Types	41
3.5.4 Integer Types	41
3.5.5 Operations of Discrete Types.....	44
3.5.6 Real Types	45
3.5.7 Floating Point Types	46
3.5.8 Operations of Floating Point Types.....	47

3.5.9 Fixed Point Types.....	48
3.5.10 Operations of Fixed Point Types.....	50
3.6 Array Types	51
3.6.1 Index Constraints and Discrete Ranges	53
3.6.2 Operations of Array Types	54
3.6.3 String Types.....	55
3.7 Discriminants	56
3.7.1 Discriminant Constraints	58
3.7.2 Operations of Discriminated Types	60
3.8 Record Types	60
3.8.1 Variant Parts and Discrete Choices	62
3.9 Tagged Types and Type Extensions	64
3.9.1 Type Extensions.....	68
3.9.2 Dispatching Operations of Tagged Types.....	69
3.9.3 Abstract Types and Subprograms	72
3.9.4 Interface Types	73
3.10 Access Types.....	76
3.10.1 Incomplete Type Declarations	78
3.10.2 Operations of Access Types.....	81
3.11 Declarative Parts	85
3.11.1 Completions of Declarations	86
Section 4: Names and Expressions	89
4.1 Names	89
4.1.1 Indexed Components.....	90
4.1.2 Slices	91
4.1.3 Selected Components	92
4.1.4 Attributes.....	94
4.2 Literals	95
4.3 Aggregates	96
4.3.1 Record Aggregates	97
4.3.2 Extension Aggregates	100
4.3.3 Array Aggregates	101
4.4 Expressions	104
4.5 Operators and Expression Evaluation	105
4.5.1 Logical Operators and Short-circuit Control Forms	106
4.5.2 Relational Operators and Membership Tests	107
4.5.3 Binary Adding Operators	111
4.5.4 Unary Adding Operators	112
4.5.5 Multiplying Operators	112
4.5.6 Highest Precedence Operators	114
4.5.7 Quantified Expressions	115
4.6 Type Conversions	116
4.7 Qualified Expressions.....	121
4.8 Allocators	121
4.9 Static Expressions and Static Subtypes.....	123
4.9.1 Statically Matching Constraints and Subtypes.....	126
Section 5: Statements	127
5.1 Simple and Compound Statements - Sequences of Statements	127
5.2 Assignment Statements.....	128
5.3 If Statements	130
5.4 Case Statements.....	131
5.5 Loop Statements	132

5.6 Block Statements	134
5.7 Exit Statements	134
5.8 Goto Statements	135
Section 6: Subprograms	137
6.1 Subprogram Declarations	137
6.2 Formal Parameter Modes	139
6.3 Subprogram Bodies.....	140
6.3.1 Conformance Rules.....	141
6.3.2 Inline Expansion of Subprograms	143
6.4 Subprogram Calls	144
6.4.1 Parameter Associations.....	146
6.5 Return Statements	148
6.5.1 Pragma No_Return	150
6.6 Overloading of Operators	151
6.7 Null Procedures	153
Section 7: Packages	155
7.1 Package Specifications and Declarations.....	155
7.2 Package Bodies	156
7.3 Private Types and Private Extensions	157
7.3.1 Private Operations.....	160
7.4 Deferred Constants.....	162
7.5 Limited Types	163
7.6 Assignment and Finalization	165
7.6.1 Completion and Finalization.....	167
Section 8: Visibility Rules	171
8.1 Declarative Region.....	171
8.2 Scope of Declarations	172
8.3 Visibility.....	173
8.3.1 Overriding Indicators	176
8.4 Use Clauses.....	177
8.5 Renaming Declarations	178
8.5.1 Object Renaming Declarations	178
8.5.2 Exception Renaming Declarations	179
8.5.3 Package Renaming Declarations	180
8.5.4 Subprogram Renaming Declarations	180
8.5.5 Generic Renaming Declarations	182
8.6 The Context of Overload Resolution	183
Section 9: Tasks and Synchronization	187
9.1 Task Units and Task Objects	187
9.2 Task Execution - Task Activation.....	190
9.3 Task Dependence - Termination of Tasks.....	191
9.4 Protected Units and Protected Objects	193
9.5 Intertask Communication.....	196
9.5.1 Protected Subprograms and Protected Actions	197
9.5.2 Entries and Accept Statements.....	199
9.5.3 Entry Calls	202
9.5.4 Requeue Statements	204
9.6 Delay Statements, Duration, and Time	206
9.6.1 Formatting, Time Zones, and other operations for Time	209
9.7 Select Statements	215

9.7.1 Selective Accept	215
9.7.2 Timed Entry Calls	217
9.7.3 Conditional Entry Calls	218
9.7.4 Asynchronous Transfer of Control	220
9.8 Abort of a Task - Abort of a Sequence of Statements	221
9.9 Task and Entry Attributes	222
9.10 Shared Variables	223
9.11 Example of Tasking and Synchronization	224
Section 10: Program Structure and Compilation Issues	227
10.1 Separate Compilation	227
10.1.1 Compilation Units - Library Units	227
10.1.2 Context Clauses - With Clauses	230
10.1.3 Subunits of Compilation Units	232
10.1.4 The Compilation Process	234
10.1.5 Pragmas and Program Units	235
10.1.6 Environment-Level Visibility Rules	236
10.2 Program Execution	237
10.2.1 Elaboration Control	239
Section 11: Exceptions	243
11.1 Exception Declarations	243
11.2 Exception Handlers	243
11.3 Raise Statements	244
11.4 Exception Handling	245
11.4.1 The Package Exceptions	246
11.4.2 Pragmas Assert and Assertion_Policy	248
11.4.3 Example of Exception Handling	249
11.5 Suppressing Checks	250
11.6 Exceptions and Optimization	253
Section 12: Generic Units	255
12.1 Generic Declarations	255
12.2 Generic Bodies	257
12.3 Generic Instantiation	258
12.4 Formal Objects	260
12.5 Formal Types	262
12.5.1 Formal Private and Derived Types	263
12.5.2 Formal Scalar Types	265
12.5.3 Formal Array Types	266
12.5.4 Formal Access Types	267
12.5.5 Formal Interface Types	267
12.6 Formal Subprograms	268
12.7 Formal Packages	270
12.8 Example of a Generic Package	273
Section 13: Representation Issues	275
13.1 Operational and Representation Items	275
13.2 Pragma Pack	278
13.3 Operational and Representation Attributes	279
13.4 Enumeration Representation Clauses	285
13.5 Record Layout	286
13.5.1 Record Representation Clauses	286
13.5.2 Storage Place Attributes	289

13.5.3 Bit Ordering	289
13.6 Change of Representation	291
13.7 The Package System	292
13.7.1 The Package System.Storage_Elements	294
13.7.2 The Package System.Address_To_Access_Conversions	295
13.8 Machine Code Insertions	295
13.9 Unchecked Type Conversions	296
13.9.1 Data Validity	297
13.9.2 The Valid Attribute	298
13.10 Unchecked Access Value Creation	299
13.11 Storage Management	299
13.11.1 Storage Allocation Attributes	303
13.11.2 Unchecked Storage Deallocation	304
13.11.3 Pragma Controlled	305
13.12 Pragma Restrictions	305
13.12.1 Language-Defined Restrictions	307
13.13 Streams	307
13.13.1 The Package Streams	307
13.13.2 Stream-Oriented Attributes	308
13.14 Freezing Rules	314
The Standard Libraries	317
Annex A (normative) Predefined Language Environment	319
A.1 The Package Standard	323
A.2 The Package Ada	327
A.3 Character Handling	327
A.3.1 The Packages Characters, Wide_Characters, and Wide_Wide_Characters	328
A.3.2 The Package Characters.Handling	328
A.3.3 The Package Characters.Latin_1	331
A.3.4 The Package Characters.Conversions	336
A.4 String Handling	338
A.4.1 The Package Strings	338
A.4.2 The Package Strings.Maps	338
A.4.3 Fixed-Length String Handling	341
A.4.4 Bounded-Length String Handling	350
A.4.5 Unbounded-Length String Handling	357
A.4.6 String-Handling Sets and Mappings	362
A.4.7 Wide_String Handling	363
A.4.8 Wide_Wide_String Handling	365
A.4.9 String Hashing	368
A.4.10 String Comparison	369
A.4.11 String Encoding	370
A.5 The Numerics Packages	374
A.5.1 Elementary Functions	375
A.5.2 Random Number Generation	378
A.5.3 Attributes of Floating Point Types	383
A.5.4 Attributes of Fixed Point Types	387
A.6 Input-Output	388
A.7 External Files and File Objects	388
A.8 Sequential and Direct Files	389
A.8.1 The Generic Package Sequential_IO	390
A.8.2 File Management	391
A.8.3 Sequential Input-Output Operations	393

A.8.4 The Generic Package Direct_IO	393
A.8.5 Direct Input-Output Operations.....	394
A.9 The Generic Package Storage_IO	395
A.10 Text Input-Output.....	396
A.10.1 The Package Text_IO	397
A.10.2 Text File Management	402
A.10.3 Default Input, Output, and Error Files	403
A.10.4 Specification of Line and Page Lengths	404
A.10.5 Operations on Columns, Lines, and Pages	405
A.10.6 Get and Put Procedures.....	408
A.10.7 Input-Output of Characters and Strings.....	410
A.10.8 Input-Output for Integer Types.....	412
A.10.9 Input-Output for Real Types	413
A.10.10 Input-Output for Enumeration Types	416
A.10.11 Input-Output for Bounded Strings.....	417
A.10.12 Input-Output for Unbounded Strings	419
A.11 Wide Text Input-Output and Wide Wide Text Input-Output.....	420
A.12 Stream Input-Output.....	421
A.12.1 The Package Streams.Stream_IO	421
A.12.2 The Package Text_IO.Text_Streams.....	423
A.12.3 The Package Wide_Text_IO.Text_Streams.....	424
A.12.4 The Package Wide_Wide_Text_IO.Text_Streams	424
A.13 Exceptions in Input-Output.....	424
A.14 File Sharing.....	426
A.15 The Package Command_Line.....	426
A.16 The Package Directories	428
A.16.1 The Package Directories.Hierarchical_File_Names.....	435
A.17 The Package Environment_Variables.....	437
A.18 Containers	440
A.18.1 The Package Containers.....	440
A.18.2 The Package Containers.Vectors	440
A.18.3 The Package Containers.Doubly_Linked_Lists	456
A.18.4 Maps	465
A.18.5 The Package Containers.Hashed_Maps	470
A.18.6 The Package Containers.Ordered_Maps	474
A.18.7 Sets	477
A.18.8 The Package Containers.Hashed_Sets	484
A.18.9 The Package Containers.Ordered_Sets	488
A.18.10 The Package Containers.Multiway_Trees.....	493
A.18.11 The Package Containers.Indefinite_Vectors	506
A.18.12 The Package Containers.Indefinite_Doubly_Linked_Lists	506
A.18.13 The Package Containers.Indefinite_Hashed_Maps	507
A.18.14 The Package Containers.Indefinite_Ordered_Maps	507
A.18.15 The Package Containers.Indefinite_Hashed_Sets.....	508
A.18.16 The Package Containers.Indefinite_Ordered_Sets.....	508
A.18.17 The Package Containers.Indefinite_Multiway_Trees	508
A.18.18 The Package Containers.Indefinite_Holders	509
A.18.19 The Package Containers.Bounded_Vectors.....	512
A.18.20 The Package Containers.Bounded_Doubly_Linked_Lists	512
A.18.21 The Package Containers.Bounded_Hashed_Maps	513
A.18.22 The Package Containers.Bounded_Ordered_Maps	514
A.18.23 The Package Containers.Bounded_Hashed_Sets	515
A.18.24 The Package Containers.Bounded_Ordered_Sets	516

A.18.25 The Package Containers.Bounded_Multiway_Trees	517
A.18.26 Array Sorting	517
Annex B (normative) Interface to Other Languages	521
B.1 Interfacing Pragmas	521
B.2 The Package Interfaces	524
B.3 Interfacing with C and C++	526
B.3.1 The Package Interfaces.C.Strings	532
B.3.2 The Generic Package Interfaces.C.Pointers	535
B.3.3 Pragma Unchecked_Union	538
B.4 Interfacing with COBOL	541
B.5 Interfacing with Fortran	547
Annex C (normative) Systems Programming	551
C.1 Access to Machine Operations	551
C.2 Required Representation Support	552
C.3 Interrupt Support	552
C.3.1 Protected Procedure Handlers	554
C.3.2 The Package Interrupts	556
C.4 Preelaboration Requirements	558
C.5 Pragma Discard_Names	559
C.6 Shared Variable Control	559
C.7 Task Information	561
C.7.1 The Package Task_Identification	562
C.7.2 The Package Task_Attributes	563
C.7.3 The Package Task_Termination	565
Annex D (normative) Real-Time Systems	567
D.1 Task Priorities	567
D.2 Priority Scheduling	569
D.2.1 The Task Dispatching Model	569
D.2.2 Task Dispatching Pragmas	571
D.2.3 Preemptive Dispatching	572
D.2.4 Non-Preemptive Dispatching	573
D.2.5 Round Robin Dispatching	574
D.2.6 Earliest Deadline First Dispatching	577
D.3 Priority Ceiling Locking	579
D.4 Entry Queuing Policies	581
D.5 Dynamic Priorities	582
D.5.1 Dynamic Priorities for Tasks	582
D.5.2 Dynamic Priorities for Protected Objects	584
D.6 Preemptive Abort	585
D.7 Tasking Restrictions	586
D.8 Monotonic Time	588
D.9 Delay Accuracy	591
D.10 Synchronous Task Control	592
D.11 Asynchronous Task Control	593
D.12 Other Optimizations and Determinism Rules	594
D.13 Run-time Profiles	595
D.13.1 The Ravenscar Profile	597
D.14 Execution Time	598
D.14.1 Execution Time Timers	600
D.14.2 Group Execution Time Budgets	602
D.15 Timing Events	604

Annex E (normative) Distributed Systems	607
E.1 Partitions	607
E.2 Categorization of Library Units	608
E.2.1 Shared Passive Library Units	609
E.2.2 Remote Types Library Units	610
E.2.3 Remote Call Interface Library Units	611
E.3 Consistency of a Distributed System	612
E.4 Remote Subprogram Calls	613
E.4.1 Pragma Asynchronous	615
E.4.2 Example of Use of a Remote Access-to-Class-Wide Type	616
E.5 Partition Communication Subsystem	617
Annex F (normative) Information Systems	621
F.1 Machine_Radix Attribute Definition Clause	621
F.2 The Package Decimal	622
F.3 Edited Output for Decimal Types	623
F.3.1 Picture String Formation	624
F.3.2 Edited Output Generation	628
F.3.3 The Package Text_IO.Editing	632
F.3.4 The Package Wide_Text_IO.Editing	635
F.3.5 The Package Wide_Wide_Text_IO.Editing	635
Annex G (normative) Numerics	637
G.1 Complex Arithmetic	637
G.1.1 Complex Types	637
G.1.2 Complex Elementary Functions	642
G.1.3 Complex Input-Output	645
G.1.4 The Package Wide_Text_IO.Complex_IO	648
G.1.5 The Package Wide_Wide_Text_IO.Complex_IO	648
G.2 Numeric Performance Requirements	648
G.2.1 Model of Floating Point Arithmetic	649
G.2.2 Model-Oriented Attributes of Floating Point Types	650
G.2.3 Model of Fixed Point Arithmetic	651
G.2.4 Accuracy Requirements for the Elementary Functions	653
G.2.5 Performance Requirements for Random Number Generation	655
G.2.6 Accuracy Requirements for Complex Arithmetic	655
G.3 Vector and Matrix Manipulation	657
G.3.1 Real Vectors and Matrices	657
G.3.2 Complex Vectors and Matrices	662
Annex H (normative) High Integrity Systems	675
H.1 Pragma Normalize_Scalars	675
H.2 Documentation of Implementation Decisions	676
H.3 Reviewable Object Code	676
H.3.1 Pragma Reviewable	676
H.3.2 Pragma Inspection_Point	677
H.4 High Integrity Restrictions	678
H.5 Pragma Detect_Blocking	680
H.6 Pragma Partition_Elaboration_Policy	681
Annex J (normative) Obsolescent Features	683
J.1 Renamings of Library Units	683
J.2 Allowed Replacements of Characters	683
J.3 Reduced Accuracy Subtypes	684

J.4 The Constrained Attribute	685
J.5 ASCII	685
J.6 Numeric_Error	686
J.7 At Clauses	686
J.7.1 Interrupt Entries	686
J.8 Mod Clauses	688
J.9 The Storage_Size Attribute	688
J.10 Specific Suppression of Checks	688
J.11 The Class Attribute of Untagged Incomplete Types	689
J.12 Pragma Interface	689
J.13 Dependence Restriction Identifiers	689
J.14 Character and Wide_Character Conversion Functions	690
Annex K (informative) Language-Defined Attributes	691
Annex L (informative) Language-Defined Pragmas	707
Annex M (informative) Summary of Documentation Requirements	709
M.1 Specific Documentation Requirements	709
M.2 Implementation-Defined Characteristics	711
M.3 Implementation Advice	717
Annex N (informative) Glossary	727
Annex P (informative) Syntax Summary	733
Annex Q (informative) Language-Defined Entities	763
Q.1 Language-Defined Packages	763
Q.2 Language-Defined Types and Subtypes	765
Q.3 Language-Defined Subprograms	770
Q.4 Language-Defined Exceptions	780
Q.5 Language-Defined Objects	781
Index	787

Foreword to this version of the Ada Reference Manual

The International Standard for the programming language Ada is ISO/IEC 8652:1995(E).	0.1/1
The Ada Working Group ISO/IEC JTC 1/SC 22/WG 9 is tasked by ISO with the work item to interpret and maintain the International Standard and to produce Technical Corrigenda, as appropriate. The technical work on the International Standard is performed by the Ada Rapporteur Group (ARG) of WG 9. In September 2000, WG 9 approved and forwarded Technical Corrigendum 1 to SC 22 for ISO approval, which was granted in February 2001. Technical Corrigendum 1 was published in June 2001.	0.2/1
In October 2002, WG 9 approved a schedule and guidelines for the preparation of an Amendment to the International Standard. WG 9 approved the scope of the Amendment in June 2004. In April 2006, WG 9 approved and forwarded the Amendment to SC 22 for approval, which was granted in August 2006. Final ISO/IEC approval came in January 2007, and the Amendment was published as ISO/IEC 8652:1995/Amd 1:2007(E) in March 2007.	0.3/3
In November 2008, WG 9 requested the preparation of a second Amendment to the International Standard. The completion of Amendment 2 is expected at the end of 2010, with final approval in 2011.	0.4/3
The Technical Corrigendum lists the individual changes that need to be made to the text of the International Standard to correct errors, omissions or inconsistencies. The corrections specified in Technical Corrigendum 1 are part of the International Standard ISO/IEC 8652:1995(E).	0.5/1
Similarly, Amendment 1 and Amendment 2 list the individual changes that need to be made to the text of the International Standard to add new features as well as correct errors.	0.6/3
When ISO published Technical Corrigendum 1, it did not also publish a document that merges the changes from the Technical Corrigendum into the text of the International Standard. Similarly, ISO did not publish a document that merges the changes from Technical Corrigendum and Amendment 1 into the text of the International Standard. It is not known whether ISO will publish a document that merges the changes from Technical Corrigendum 1, Amendment 1, and Amendment 2 into the text of the International Standard. However, ISO rules require that the project editor for the International Standard be able to produce such a document on demand.	0.7/3
This version of the Ada Reference Manual is what the project editor would provide to ISO in response to such a request. It incorporates the changes specified in the Technical Corrigendum and Amendments 1 and 2 into the text of ISO/IEC 8652:1995(E). It should be understood that the publication of any ISO document involves changes in general format, boilerplate, headers, etc., as well as a review by professional editors that may introduce editorial changes to the text. This version of the Ada Reference Manual is therefore neither an official ISO document, nor a version guaranteed to be identical to an official ISO document, should ISO decide to reprint the International Standard incorporating an approved Technical Corrigendum and Amendments. It is nevertheless a best effort to be as close as possible to the technical content of such an updated document. In the case of a conflict between this document and Amendment 2 as approved by ISO (or between this document and Amendment 1 in the case of paragraphs not changed by Amendment 2; or between this document and Technical Corrigendum 1 in the case of paragraphs not changed by either Amendment; or between this document and the original 8652:1995 in the case of paragraphs not changed by either Amendment or Technical Corrigendum 1), the other documents contain the official text of the International Standard ISO/IEC 8652:1995(E) and its Amendments.	0.8/3
As it is very inconvenient to have the Reference Manual for Ada specified in four documents, this consolidated version of the Ada Reference Manual is made available to the public.	0.9/3

Foreword

- 1 ISO (the International Organization for Standardization) and IEC (the International Electrotechnical Commission) form the specialized system for worldwide standardization. National bodies that are members of ISO or IEC participate in the development of International Standards through technical committees established by the respective organization to deal with particular fields of technical activity. ISO and IEC technical committees collaborate in fields of mutual interest. Other international organizations, governmental and non-governmental, in liaison with ISO and IEC, also take part in the work.
- 2 In the field of information technology, ISO and IEC have established a joint technical committee, ISO/IEC JTC 1. Draft International Standards adopted by the joint technical committee are circulated to national bodies for voting. Publication as an International Standard requires approval by at least 75 % of the national bodies casting a vote.
- 3 International Standard ISO/IEC 8652 was prepared by Joint Technical Committee ISO/IEC JTC 1, *Information Technology*.
- 4/2 This consolidated edition updates the second edition (ISO 8652:1995).
- 5/2 Annexes A to J form an integral part of this International Standard. Annexes K to Q are for information only.

Introduction

This is the Ada Reference Manual.	1
Other available Ada documents include:	2
Ada 95 Rationale. This gives an introduction to the new features of Ada incorporated in the 1995 edition of this Standard, and explains the rationale behind them. Programmers unfamiliar with Ada 95 should read this first.	3/2
Ada 2005 Rationale. This gives an introduction to the changes and new features in Ada 2005 (compared with the 1995 edition), and explains the rationale behind them. Programmers should read this rationale before reading this Standard in depth.	3.1/2
<i>This paragraph was deleted.</i>	4/1
The Annotated Ada Reference Manual (AARM). The AARM contains all of the text in the consolidated Ada Reference Manual, plus various annotations. It is intended primarily for compiler writers, validation test writers, and others who wish to study the fine details. The annotations include detailed rationale for individual rules and explanations of some of the more arcane interactions among the rules.	5/2

Design Goals

Ada was originally designed with three overriding concerns: program reliability and maintenance, programming as a human activity, and efficiency. The 1995 revision to the language was designed to provide greater flexibility and extensibility, additional control over storage management and synchronization, and standardized packages oriented toward supporting important application areas, while at the same time retaining the original emphasis on reliability, maintainability, and efficiency. This amended version provides further flexibility and adds more standardized packages within the framework provided by the 1995 revision.

The need for languages that promote reliability and simplify maintenance is well established. Hence emphasis was placed on program readability over ease of writing. For example, the rules of the language require that program variables be explicitly declared and that their type be specified. Since the type of a variable is invariant, compilers can ensure that operations on variables are compatible with the properties intended for objects of the type. Furthermore, error-prone notations have been avoided, and the syntax of the language avoids the use of encoded forms in favor of more English-like constructs. Finally, the language offers support for separate compilation of program units in a way that facilitates program development and maintenance, and which provides the same degree of checking between units as within a unit.

Concern for the human programmer was also stressed during the design. Above all, an attempt was made to keep to a relatively small number of underlying concepts integrated in a consistent and systematic way while continuing to avoid the pitfalls of excessive involution. The design especially aims to provide language constructs that correspond intuitively to the normal expectations of users.

Like many other human activities, the development of programs is becoming ever more decentralized and distributed. Consequently, the ability to assemble a program from independently produced software components continues to be a central idea in the design. The concepts of packages, of private types, and of generic units are directly related to this idea, which has ramifications in many other aspects of the language. An allied concern is the maintenance of programs to match changing requirements; type extension and the hierarchical library enable a program to be modified while minimizing disturbance to existing tested and trusted components.

- 10 No language can avoid the problem of efficiency. Languages that require over-elaborate compilers, or that lead to the inefficient use of storage or execution time, force these inefficiencies on all machines and on all programs. Every construct of the language was examined in the light of present implementation techniques. Any proposed construct whose implementation was unclear or that required excessive machine resources was rejected.

Language Summary

- 11 An Ada program is composed of one or more program units. Program units may be subprograms (which define executable algorithms), packages (which define collections of entities), task units (which define concurrent computations), protected units (which define operations for the coordinated sharing of data between tasks), or generic units (which define parameterized forms of packages and subprograms). Each program unit normally consists of two parts: a specification, containing the information that must be visible to other units, and a body, containing the implementation details, which need not be visible to other units. Most program units can be compiled separately.
- 12 This distinction of the specification and body, and the ability to compile units separately, allows a program to be designed, written, and tested as a set of largely independent software components.
- 13 An Ada program will normally make use of a library of program units of general utility. The language provides means whereby individual organizations can construct their own libraries. All libraries are structured in a hierarchical manner; this enables the logical decomposition of a subsystem into individual components. The text of a separately compiled program unit must name the library units it requires.
- 14 *Program Units*
- 15 A subprogram is the basic unit for expressing an algorithm. There are two kinds of subprograms: procedures and functions. A procedure is the means of invoking a series of actions. For example, it may read data, update variables, or produce some output. It may have parameters, to provide a controlled means of passing information between the procedure and the point of call. A function is the means of invoking the computation of a value. It is similar to a procedure, but in addition will return a result.
- 16 A package is the basic unit for defining a collection of logically related entities. For example, a package can be used to define a set of type declarations and associated operations. Portions of a package can be hidden from the user, thus allowing access only to the logical properties expressed by the package specification.
- 17 Subprogram and package units may be compiled separately and arranged in hierarchies of parent and child units giving fine control over visibility of the logical properties and their detailed implementation.
- 18 A task unit is the basic unit for defining a task whose sequence of actions may be executed concurrently with those of other tasks. Such tasks may be implemented on multicomputers, multiprocessors, or with interleaved execution on a single processor. A task unit may define either a single executing task or a task type permitting the creation of any number of similar tasks.
- 19/2 A protected unit is the basic unit for defining protected operations for the coordinated use of data shared between tasks. Simple mutual exclusion is provided automatically, and more elaborate sharing protocols can be defined. A protected operation can either be a subprogram or an entry. A protected entry specifies a Boolean expression (an entry barrier) that must be True before the body of the entry is executed. A protected unit may define a single protected object or a protected type permitting the creation of several similar objects.

Declarations and Statements

20

The body of a program unit generally contains two parts: a declarative part, which defines the logical entities to be used in the program unit, and a sequence of statements, which defines the execution of the program unit.

21

The declarative part associates names with declared entities. For example, a name may denote a type, a constant, a variable, or an exception. A declarative part also introduces the names and parameters of other nested subprograms, packages, task units, protected units, and generic units to be used in the program unit.

22

The sequence of statements describes a sequence of actions that are to be performed. The statements are executed in succession (unless a transfer of control causes execution to continue from another place).

23

An assignment statement changes the value of a variable. A procedure call invokes execution of a procedure after associating any actual parameters provided at the call with the corresponding formal parameters.

24

Case statements and if statements allow the selection of an enclosed sequence of statements based on the value of an expression or on the value of a condition.

25

The loop statement provides the basic iterative mechanism in the language. A loop statement specifies that a sequence of statements is to be executed repeatedly as directed by an iteration scheme, or until an exit statement is encountered.

26

A block statement comprises a sequence of statements preceded by the declaration of local entities used by the statements.

27

Certain statements are associated with concurrent execution. A delay statement delays the execution of a task for a specified duration or until a specified time. An entry call statement is written as a procedure call statement; it requests an operation on a task or on a protected object, blocking the caller until the operation can be performed. A called task may accept an entry call by executing a corresponding accept statement, which specifies the actions then to be performed as part of the rendezvous with the calling task. An entry call on a protected object is processed when the corresponding entry barrier evaluates to true, whereupon the body of the entry is executed. The requeue statement permits the provision of a service as a number of related activities with preference control. One form of the select statement allows a selective wait for one of several alternative rendezvous. Other forms of the select statement allow conditional or timed entry calls and the asynchronous transfer of control in response to some triggering event.

28

Execution of a program unit may encounter error situations in which normal program execution cannot continue. For example, an arithmetic computation may exceed the maximum allowed value of a number, or an attempt may be made to access an array component by using an incorrect index value. To deal with such error situations, the statements of a program unit can be textually followed by exception handlers that specify the actions to be taken when the error situation arises. Exceptions can be raised explicitly by a raise statement.

29

Data Types

30

Every object in the language has a type, which characterizes a set of values and a set of applicable operations. The main classes of types are elementary types (comprising enumeration, numeric, and access types) and composite types (including array and record types).

31

An enumeration type defines an ordered set of distinct enumeration literals, for example a list of states or an alphabet of characters. The enumeration types Boolean, Character, Wide_Character, and Wide_Wide_Character are predefined.

32/2

- 33 Numeric types provide a means of performing exact or approximate numerical computations. Exact computations use integer types, which denote sets of consecutive integers. Approximate computations use either fixed point types, with absolute bounds on the error, or floating point types, with relative bounds on the error. The numeric types Integer, Float, and Duration are predefined.
- 34/2 Composite types allow definitions of structured objects with related components. The composite types in the language include arrays and records. An array is an object with indexed components of the same type. A record is an object with named components of possibly different types. Task and protected types are also forms of composite types. The array types String, Wide_String, and Wide_Wide_String are predefined.
- 35 Record, task, and protected types may have special components called discriminants which parameterize the type. Variant record structures that depend on the values of discriminants can be defined within a record type.
- 36 Access types allow the construction of linked data structures. A value of an access type represents a reference to an object declared as aliased or to an object created by the evaluation of an allocator. Several variables of an access type may designate the same object, and components of one object may designate the same or other objects. Both the elements in such linked data structures and their relation to other elements can be altered during program execution. Access types also permit references to subprograms to be stored, passed as parameters, and ultimately dereferenced as part of an indirect call.
- 37 Private types permit restricted views of a type. A private type can be defined in a package so that only the logically necessary properties are made visible to the users of the type. The full structural details that are externally irrelevant are then only available within the package and any child units.
- 38 From any type a new type may be defined by derivation. A type, together with its derivatives (both direct and indirect) form a derivation class. Class-wide operations may be defined that accept as a parameter an operand of any type in a derivation class. For record and private types, the derivatives may be extensions of the parent type. Types that support these object-oriented capabilities of class-wide operations and type extension must be tagged, so that the specific type of an operand within a derivation class can be identified at run time. When an operation of a tagged type is applied to an operand whose specific type is not known until run time, implicit dispatching is performed based on the tag of the operand.
- 38.1/2 Interface types provide abstract models from which other interfaces and types may be composed and derived. This provides a reliable form of multiple inheritance. Interface types may also be implemented by task types and protected types thereby enabling concurrent programming and inheritance to be merged.
- 39 The concept of a type is further refined by the concept of a subtype, whereby a user can constrain the set of allowed values of a type. Subtypes can be used to define subranges of scalar types, arrays with a limited set of index values, and records and private types with particular discriminant values.
- 40 *Other Facilities*
- 41/2 Aspect clauses can be used to specify the mapping between types and features of an underlying machine. For example, the user can specify that objects of a given type must be represented with a given number of bits, or that the components of a record are to be represented using a given storage layout. Other features allow the controlled use of low level, nonportable, or implementation-dependent aspects, including the direct insertion of machine code.
- 42/2 The predefined environment of the language provides for input-output and other capabilities by means of standard library packages. Input-output is supported for values of user-defined as well as of predefined types. Standard means of representing values in display form are also provided.

The predefined standard library packages provide facilities such as string manipulation, containers of various kinds (vectors, lists, maps, etc.), mathematical functions, random number generation, and access to the execution environment. 42.1/2

The specialized annexes define further predefined library packages and facilities with emphasis on areas such as real-time scheduling, interrupt handling, distributed systems, numerical computation, and high-integrity systems. 42.2/2

Finally, the language provides a powerful means of parameterization of program units, called generic program units. The generic parameters can be types and subprograms (as well as objects and packages) and so allow general algorithms and data structures to be defined that are applicable to all types of a given class. 43

Language Changes

This amended International Standard updates the edition of 1995 which replaced the first edition of 1987. In the 1995 edition, the following major language changes were incorporated: 44/2

- Support for standard 8-bit and 16-bit characters was added. See clauses 2.1, 3.5.2, 3.6.3, A.1, A.3, and A.4. 45/2
- The type model was extended to include facilities for object-oriented programming with dynamic polymorphism. See the discussions of classes, derived types, tagged types, record extensions, and private extensions in clauses 3.4, 3.9, and 7.3. Additional forms of generic formal parameters were allowed as described in clauses 12.5.1 and 12.7. 46/2
- Access types were extended to allow an access value to designate a subprogram or an object declared by an object declaration as opposed to just an object allocated on a heap. See clause 3.10. 47/2
- Efficient data-oriented synchronization was provided by the introduction of protected types. See clause 9.4. 48/2
- The library structure was extended to allow library units to be organized into a hierarchy of parent and child units. See clause 10.1. 49/2
- Additional support was added for interfacing to other languages. See Annex B. 50/2
- The Specialized Needs Annexes were added to provide specific support for certain application areas: 51/2
 - Annex C, “Systems Programming” 52
 - Annex D, “Real-Time Systems” 53
 - Annex E, “Distributed Systems” 54
 - Annex F, “Information Systems” 55
 - Annex G, “Numerics” 56
 - Annex H, “High Integrity Systems” 57

Amendment 1 modifies the 1995 International Standard by making changes and additions that improve the capability of the language and the reliability of programs written in the language. In particular the changes were designed to improve the portability of programs, interfacing to other languages, and both the object-oriented and real-time capabilities. 57.1/2

The following significant changes with respect to the 1995 edition are incorporated: 57.2/2

- 57.3/2 • Support for program text is extended to cover the entire ISO/IEC 10646:2003 repertoire. Execution support now includes the 32-bit character set. See clauses 2.1, 3.5.2, 3.6.3, A.1, A.3, and A.4.
- 57.4/2 • The object-oriented model has been improved by the addition of an interface facility which provides multiple inheritance and additional flexibility for type extensions. See clauses 3.4, 3.9, and 7.3. An alternative notation for calling operations more akin to that used in other languages has also been added. See clause 4.1.3.
- 57.5/2 • Access types have been further extended to unify properties such as the ability to access constants and to exclude null values. See clause 3.10. Anonymous access types are now permitted more freely and anonymous access-to-subprogram types are introduced. See clauses 3.3, 3.6, 3.10, and 8.5.1.
- 57.6/2 • The control of structure and visibility has been enhanced to permit mutually dependent references between units and finer control over access from the private part of a package. See clauses 3.10.1 and 10.1.2. In addition, limited types have been made more useful by the provision of aggregates, constants, and constructor functions. See clauses 4.3, 6.5, and 7.5.
- 57.7/2 • The predefined environment has been extended to include additional time and calendar operations, improved string handling, a comprehensive container library, file and directory management, and access to environment variables. See clauses 9.6.1, A.4, A.16, A.17, and A.18.
- 57.8/2 • Two of the Specialized Needs Annexes have been considerably enhanced:
 - 57.9/2 • The Real-Time Systems Annex now includes the Ravenscar profile for high-integrity systems, further dispatching policies such as Round Robin and Earliest Deadline First, support for timing events, and support for control of CPU time utilization. See clauses D.2, D.13, D.14, and D.15.
 - 57.10/2 • The Numerics Annex now includes support for real and complex vectors and matrices as previously defined in ISO/IEC 13813:1997 plus further basic operations for linear algebra. See clause G.3.
- 57.11/2 • The overall reliability of the language has been enhanced by a number of improvements. These include new syntax which detects accidental overloading, as well as pragmas for making assertions and giving better control over the suppression of checks. See clauses 6.1, 11.4.2, and 11.5.
- 57.12/3 Amendment 2 modifies the 1995 International Standard by making changes and additions that improve the capability of the language and the reliability of programs written in the language. In particular blah blah blah.
- 57.13/3 ** List of new good stuff TBD **

Instructions for Comment Submission

Informal comments on this International Standard may be sent via e-mail to **ada-comment@ada-auth.org**. If appropriate, the Project Editor will initiate the defect correction procedure. 58/1

Comments should use the following format: 59

!topic *Title summarizing comment* 60/2
!reference *Ada 2005 RMss.ss(pp)*
!from *Author Name yy-mm-dd*
!keywords *keywords related to topic*
!discussion

text of discussion

where *ss.ss* is the section, clause or subclause number, *pp* is the paragraph number where applicable, and *yy-mm-dd* is the date the comment was sent. The date is optional, as is the **!keywords** line. 61

Please use a descriptive “Subject” in your e-mail message, and limit each message to a single comment. 62/1

When correcting typographical errors or making minor wording suggestions, please put the correction directly as the topic of the comment; use square brackets [] to indicate text to be omitted and curly braces { } to indicate text to be added, and provide enough context to make the nature of the suggestion self-evident or put additional information in the body of the comment, for example: 63

!topic [c]{C}haracter 64
!topic it[']s meaning is not defined

Formal requests for interpretations and for reporting defects in this International Standard may be made in accordance with the ISO/IEC JTC 1 Directives and the ISO/IEC JTC 1/SC 22 policy for interpretations. National Bodies may submit a Defect Report to ISO/IEC JTC 1/SC 22 for resolution under the JTC 1 procedures. A response will be provided and, if appropriate, a Technical Corrigendum will be issued in accordance with the procedures. 65

Acknowledgements for the Ada 83 edition of the Ada Reference Manual

- 65.1/3 Ada is the result of a collective effort to design a common language for programming large scale and real-time systems.
- 65.2/3 The common high order language program began in 1974. The requirements of the United States Department of Defense were formalized in a series of documents which were extensively reviewed by the Services, industrial organizations, universities, and foreign military departments. The Ada language was designed in accordance with the final (1978) form of these requirements, embodied in the Steelman specification.
- 65.3/3 The Ada design team was led by Jean D. Ichbiah and has included Bernd Krieg-Brueckner, Brian A. Wichmann, Henry F. Ledgard, Jean-Claude Heliard, Jean-Loup Gailly, Jean-Raymond Abrial, John G.P. Barnes, Mike Woodger, Olivier Roubine, Paul N. Hilfinger, and Robert Firth.
- 65.4/3 At various stages of the project, several people closely associated with the design team made major contributions. They include J.B. Goodenough, R.F. Brender, M.W. Davis, G. Ferran, K. Lester, L. MacLaren, E. Morel, I.R. Nassi, I.C. Pyle, S.A. Schuman, and S.C. Vestal.
- 65.5/3 Two parallel efforts that were started in the second phase of this design had a deep influence on the language. One was the development of a formal definition using denotational semantics, with the participation of V. Donzeau-Gouge, G. Kahn, and B. Lang. The other was the design of a test translator with the participation of K. Ripken, P. Boullier, P. Cadiou, J. Holden, J.F. Hueras, R.G. Lange, and D.T. Cornhill. The entire effort benefitted from the dedicated assistance of Lyn Churchill and Marion Myers, and the effective technical support of B. Gravem, W.L. Heimerdinger, and P. Cleve. H.G. Schmitz served as program manager.
- 65.6/3 Over the five years spent on this project, several intense week-long design reviews were conducted, with the participation of P. Belmont, B. Brosgol, P. Cohen, R. Dewar, A. Evans, G. Fisher, H. Harte, A.L. Hisgen, P. Knueven, M. Kronental, N. Lomuto, E. Ploedereder, G. Seegmueller, V. Stenning, D. Taffs, and also F. Belz, R. Converse, K. Correll, A.N. Habermann, J. Sammet, S. Squires, J. Teller, P. Wegner, and P.R. Wetherall.
- 65.7/3 Several persons had a constructive influence with their comments, criticisms and suggestions. They include P. Brinch Hansen, G. Goos, C.A.R. Hoare, Mark Rain, W.A. Wulf, and also E. Boebert, P. Bonnard, H. Clausen, M. Cox, G. Dismukes, R. Eachus, T. Froggatt, H. Ganzinger, C. Hewitt, S. Kamin, R. Kotler, O. Lecarme, J.A.N. Lee, J.L. Mansion, F. Minel, T. Phinney, J. Roehrich, V. Schneider, A. Singer, D. Slosberg, I.C. Wand, the reviewers of Ada-Europe, AdaTech, Afcet, those of the LMSC review team, and those of the Ada Tokyo Study Group.
- 65.8/3 These reviews and comments, the numerous evaluation reports received at the end of the first and second phase, the nine hundred language issue reports and test and evaluation reports received from fifteen different countries during the third phase of the project, the thousands of comments received during the ANSI Canvass, and the on-going work of the IFIP Working Group 2.4 on system implementation languages and that of the Purdue Europe LTPL-E committee, all had a substantial influence on the final definition of Ada.
- 65.9/3 The Military Departments and Agencies have provided a broad base of support including funding, extensive reviews, and countless individual contributions by the members of the High Order Language Working Group and other interested personnel. In particular, William A. Whitaker provided leadership for the program during the formative stages. David A. Fisher was responsible for the successful development and refinement of the language requirement documents that led to the Steelman specification.

The Ada 83 language definition was developed by Cii Honeywell Bull and later Alsys, and by Honeywell Systems and Research Center, under contract to the United States Department of Defense. William E. Carlson and later Larry E. Druffel served as the technical representatives of the United States Government and effectively coordinated the efforts of all participants in the Ada program. 65.10/3

Acknowledgements for the Ada 95 edition of the Ada Reference Manual

This International Standard was prepared by the Ada 9X Mapping/Revision Team based at Intermetrics, Inc., which has included: W. Carlson, Program Manager; T. Taft, Technical Director; J. Barnes (consultant); B. Brosgol (consultant); R. Duff (Oak Tree Software); M. Edwards; C. Garrity; R. Hilliard; O. Pazy (consultant); D. Rosenfeld; L. Shafer; W. White; M. Woodger. 66

The following consultants to the Ada 9X Project contributed to the Specialized Needs Annexes: T. Baker (Real-Time/Systems Programming — SEI, FSU); K. Dritz (Numerics — Argonne National Laboratory); A. Gargaro (Distributed Systems — Computer Sciences); J. Goodenough (Real-Time/Systems Programming — SEI); J. McHugh (Secure Systems — consultant); B. Wichmann (Safety-Critical Systems — NPL: UK). 67

This work was regularly reviewed by the Ada 9X Distinguished Reviewers and the members of the Ada 9X Rapporteur Group (XRG): E. Ploedereder, Chairman of DRs and XRG (University of Stuttgart: Germany); B. Bardin (Hughes); J. Barnes (consultant: UK); B. Brett (DEC); B. Brosgol (consultant); R. Brukardt (RR Software); N. Cohen (IBM); R. Dewar (NYU); G. Dismukes (TeleSoft); A. Evans (consultant); A. Gargaro (Computer Sciences); M. Gerhardt (ESL); J. Goodenough (SEI); S. Heilbrunner (University of Salzburg: Austria); P. Hilfinger (UC/Berkeley); B. Källberg (CelsiusTech: Sweden); M. Kamrad II (Unisys); J. van Katwijk (Delft University of Technology: The Netherlands); V. Kaufman (Russia); P. Kruchten (Rational); R. Landwehr (CCI: Germany); C. Lester (Portsmouth Polytechnic: UK); L. Månsson (TELIA Research: Sweden); S. Michell (Multiprocessor Toolsmiths: Canada); M. Mills (US Air Force); D. Pogge (US Navy); K. Power (Boeing); O. Roubine (Verdix: France); A. Strohmeier (Swiss Fed Inst of Technology: Switzerland); W. Taylor (consultant: UK); J. Tokar (Tartan); E. Vasilescu (Grumman); J. Vladik (Prospeks s.r.o.: Czech Republic); S. Van Vlierberghe (OFFIS: Belgium). 68

Other valuable feedback influencing the revision process was provided by the Ada 9X Language Precision Team (Odyssey Research Associates), the Ada 9X User/Implementer Teams (AETECH, Tartan, TeleSoft), the Ada 9X Implementation Analysis Team (New York University) and the Ada community-at-large. 69

Special thanks go to R. Mathis, Convenor of ISO/IEC JTC 1/SC 22 Working Group 9. 70

The Ada 9X Project was sponsored by the Ada Joint Program Office. Christine M. Anderson at the Air Force Phillips Laboratory (Kirtland AFB, NM) was the project manager. 71

Acknowledgements for the Corrigendum version of the Ada Reference Manual

The editor [R. Brukardt (USA)] would like to thank the many people whose hard work and assistance has made this revision possible. 71.1/1

Thanks go out to all of the members of the ISO/IEC JTC 1/SC 22/WG 9 Ada Rapporteur Group, whose work on creating and editing the wording corrections was critical to the entire process. Especially valuable contributions came from the chairman of the ARG, E. Ploedereder (Germany), who kept the process moving; J. Barnes (UK) and K. Ishihata (Japan), whose extremely detailed reviews kept the editor on his toes; G. Dismukes (USA), M. Kamrad (USA), P. Leroy (France), S. Michell (Canada), T. Taft (USA), J. Tokar (USA), and other members too numerous to mention. 71.2/1

- 71.3/1 Special thanks go to R. Duff (USA) for his explanations of the previous system of formatting of these documents during the tedious conversion to more modern formats. Special thanks also go to the convener of ISO/IEC JTC 1/SC 22/WG 9, J. Moore (USA), without whose help and support the corrigendum and this consolidated reference manual would not have been possible.

Acknowledgements for the Amendment 1 version of the Ada Reference Manual

- 71.4/2 The editor [R. Brukardt (USA)] would like to thank the many people whose hard work and assistance has made this revision possible.
- 71.5/2 Thanks go out to all of the members of the ISO/IEC JTC 1/SC 22/WG 9 Ada Rapporteur Group, whose work on creating and editing the wording corrections was critical to the entire process. Especially valuable contributions came from the chairman of the ARG, P. Leroy (France), who kept the process on schedule; J. Barnes (UK) whose careful reviews found many typographical errors; T. Taft (USA), who always seemed to have a suggestion when we were stuck, and who also was usually able to provide the valuable service of explaining why things were as they are; S. Baird (USA), who found many obscure problems with the proposals; and A. Burns (UK), who pushed many of the real-time proposals to completion. Other ARG members who contributed were: R. Dewar (USA), G. Dismukes (USA), R. Duff (USA), K. Ishihata (Japan), S. Michell (Canada), E. Ploedereder (Germany), J.P. Rosen (France), E. Schonberg (USA), J. Tokar (USA), and T. Vardanega (Italy).
- 71.6/2 Special thanks go to Ada-Europe and the Ada Resource Association, without whose help and support the Amendment and this consolidated reference manual would not have been possible. M. Heaney (USA) requires special thanks for his tireless work on the containers packages. Finally, special thanks go to the convener of ISO/IEC JTC 1/SC 22/WG 9, J. Moore (USA), who guided the document through the standardization process.

Acknowledgements for the Amendment 2 version of the Ada Reference Manual

- 71.7/3 The editor [R. Brukardt (USA)] would like to thank the many people whose hard work and assistance has made this revision possible.
- 71.8/3 ** List of people to thank TBD **

Changes

The International Standard is the same as this version of the Reference Manual, except:	72
This list of Changes is not included in the International Standard.	73
The “Acknowledgements” page is not included in the International Standard.	74
The text in the running headers and footers on each page is slightly different in the International Standard.	75
The title page(s) are different in the International Standard.	76
This document is formatted for 8.5-by-11-inch paper, whereas the International Standard is formatted for A4 paper (210-by-297mm); thus, the page breaks are in different places.	77
The “Foreword to this version of the Ada Reference Manual” clause is not included in the International Standard.	77.1/1
The “Using this version of the Ada Reference Manual” clause is not included in the International Standard.	77.2/2

Using this version of the Ada Reference Manual

This document has been revised with the corrections specified in Technical Corrigendum 1 (ISO/IEC 8652:1995/COR.1:2001), Amendment 1 (ISO/IEC 8652/AMD 1:2007), and Amendment 2 (ISO/IEC 8652/AMD 2:201z. In addition, a variety of editorial errors have been corrected. 77.3/3

Changes to the original 8652:1995 can be identified by the version number following the paragraph number. Paragraphs with a version number of /1 were changed by Technical Corrigendum 1 or were editorial corrections at that time, while paragraphs with a version number of /2 were changed by Amendment 1 or were more recent editorial corrections, and paragraphs with a version number of /3 were changed by Amendment 2 or were still more recent editorial corrections. Paragraphs not so marked are unchanged by Amendment 1, Amendment 2, Technical Corrigendum 1, or editorial corrections. Paragraph numbers of unchanged paragraphs are the same as in the original Ada Reference Manual. In addition, some versions of this document include revision bars near the paragraph numbers. Where paragraphs are inserted, the paragraph numbers are of the form pp.nn, where pp is the number of the preceding paragraph, and nn is an insertion number. For instance, the first paragraph inserted after paragraph 8 is numbered 8.1, the second paragraph inserted is numbered 8.2, and so on. Deleted paragraphs are indicated by the text *This paragraph was deleted*. Deleted paragraphs include empty paragraphs that were numbered in the original Ada Reference Manual. 77.4/3

Information technology — Programming Languages — Ada

Section 1: General

Ada is a programming language designed to support the construction of long-lived, highly reliable software systems. The language includes facilities to define packages of related types, objects, and operations. The packages may be parameterized and the types may be extended to support the construction of libraries of reusable, adaptable software components. The operations may be implemented as subprograms using conventional sequential control structures, or as entries that include synchronization of concurrent threads of control as part of their invocation. The language treats modularity in the physical sense as well, with a facility to support separate compilation.

The language includes a complete facility for the support of real-time, concurrent programming. Errors can be signaled as exceptions and handled explicitly. The language also covers systems programming; this requires precise control over the representation of data and access to system-dependent properties. Finally, a predefined environment of standard packages is provided, including facilities for, among others, input-output, string manipulation, numeric elementary functions, and random number generation.

1.1 Scope

This International Standard specifies the form and meaning of programs written in Ada. Its purpose is to promote the portability of Ada programs to a variety of data processing systems.

1.1.1 Extent

This International Standard specifies:

- The form of a program written in Ada;
- The effect of translating and executing such a program;
- The manner in which program units may be combined to form Ada programs;
- The language-defined library units that a conforming implementation is required to supply;

- The permissible variations within the standard, and the manner in which they are to be documented;
- Those violations of the standard that a conforming implementation is required to detect, and the effect of attempting to translate or execute a program containing such violations;
- Those violations of the standard that a conforming implementation is not required to detect.

This International Standard does not specify:

- The means whereby a program written in Ada is transformed into object code executable by a processor;
- The means whereby translation or execution of programs is invoked and the executing units are controlled;
- The size or speed of the object code, or the relative execution speed of different language constructs;
- The form or contents of any listings produced by implementations; in particular, the form or contents of error or warning messages;
- The effect of unspecified execution.
- The size of a program or program unit that will exceed the capacity of a particular conforming implementation.

1.1.2 Structure

This International Standard contains thirteen sections, fourteen annexes, and an index.

The *core* of the Ada language consists of:

- Sections 1 through 13
- Annex A, “Predefined Language Environment”
- Annex B, “Interface to Other Languages”
- Annex J, “Obsolescent Features”

The following *Specialized Needs Annexes* define features that are needed by certain application areas:

- Annex C, “Systems Programming”
- Annex D, “Real-Time Systems”
- Annex E, “Distributed Systems”
- Annex F, “Information Systems”
- Annex G, “Numerics”
- Annex H, “High Integrity Systems”

The core language and the Specialized Needs Annexes are normative, except that the material in each of the items listed below is informative:

- Text under a NOTES or Examples heading.
- Each clause or subclause whose title starts with the word “Example” or “Examples”.

All implementations shall conform to the core language. In addition, an implementation may conform separately to one or more Specialized Needs Annexes.	17
The following Annexes are informative:	18
• Annex K, “Language-Defined Attributes”	19
• Annex L, “Language-Defined Pragmas”	20
• Annex M, “Summary of Documentation Requirements”	21/3
• Annex N, “Glossary”	22
• Annex P, “Syntax Summary”	23
Each section is divided into clauses and subclauses that have a common structure. Each section, clause, and subclause first introduces its subject. After the introductory text, text is labeled with the following headings:	24
<i>Syntax</i>	
Syntax rules (indented).	25
<i>Name Resolution Rules</i>	
Compile-time rules that are used in name resolution, including overload resolution.	26
<i>Legality Rules</i>	
Rules that are enforced at compile time. A construct is <i>legal</i> if it obeys all of the Legality Rules.	27
<i>Static Semantics</i>	
A definition of the compile-time effect of each construct.	28
<i>Post-Compilation Rules</i>	
Rules that are enforced before running a partition. A partition is legal if its compilation units are legal and it obeys all of the Post-Compilation Rules.	29
<i>Dynamic Semantics</i>	
A definition of the run-time effect of each construct.	30
<i>Bounded (Run-Time) Errors</i>	
Situations that result in bounded (run-time) errors (see 1.1.5).	31
<i>Erroneous Execution</i>	
Situations that result in erroneous execution (see 1.1.5).	32
<i>Implementation Requirements</i>	
Additional requirements for conforming implementations.	33
<i>Documentation Requirements</i>	
Documentation requirements for conforming implementations.	34
<i>Metrics</i>	
Metrics that are specified for the time/space properties of the execution of certain language constructs.	35

Implementation Permissions

36 Additional permissions given to the implementer.

Implementation Advice

37 Optional advice given to the implementer. The word “should” is used to indicate that the advice is a recommendation, not a requirement. It is implementation defined whether or not a given recommendation is obeyed.

NOTES

38 1 Notes emphasize consequences of the rules described in the (sub)clause or elsewhere. This material is informative.

Examples

39 Examples illustrate the possible forms of the constructs described. This material is informative.

1.1.3 Conformity of an Implementation with the Standard

Implementation Requirements

1 A conforming implementation shall:

- 2 • Translate and correctly execute legal programs written in Ada, provided that they are not so large as to exceed the capacity of the implementation;
- 3 • Identify all programs or program units that are so large as to exceed the capacity of the implementation (or raise an appropriate exception at run time);
- 4 • Identify all programs or program units that contain errors whose detection is required by this International Standard;
- 5 • Supply all language-defined library units required by this International Standard;
- 6 • Contain no variations except those explicitly permitted by this International Standard, or those that are impossible or impractical to avoid given the implementation's execution environment;
- 7 • Specify all such variations in the manner prescribed by this International Standard.

8 The *external effect* of the execution of an Ada program is defined in terms of its interactions with its external environment. The following are defined as *external interactions*:

- 9 • Any interaction with an external file (see A.7);
- 10 • The execution of certain `code_statements` (see 13.8); which `code_statements` cause external interactions is implementation defined.
- 11 • Any call on an imported subprogram (see Annex B), including any parameters passed to it;
- 12 • Any result returned or exception propagated from a main subprogram (see 10.2) or an exported subprogram (see Annex B) to an external caller;
- 13 • Any read or update of an atomic or volatile object (see C.6);
- 14 • The values of imported and exported objects (see Annex B) at the time of any other interaction with the external environment.

15 A conforming implementation of this International Standard shall produce for the execution of a given Ada program a set of interactions with the external environment whose order and timing are consistent with the definitions and requirements of this International Standard for the semantics of the given program.

An implementation that conforms to this Standard shall support each capability required by the core language as specified. In addition, an implementation that conforms to this Standard may conform to one or more Specialized Needs Annexes (or to none). Conformance to a Specialized Needs Annex means that each capability required by the Annex is provided as specified. 16

An implementation conforming to this International Standard may provide additional attributes, library units, and pragmas. However, it shall not provide any attribute, library unit, or pragma having the same name as an attribute, library unit, or pragma (respectively) specified in a Specialized Needs Annex unless the provided construct is either as specified in the Specialized Needs Annex or is more limited in capability than that required by the Annex. A program that attempts to use an unsupported capability of an Annex shall either be identified by the implementation before run time or shall raise an exception at run time. 17

Documentation Requirements

Certain aspects of the semantics are defined to be either *implementation defined* or *unspecified*. In such cases, the set of possible effects is specified, and the implementation may choose any effect in the set. Implementations shall document their behavior in implementation-defined situations, but documentation is not required for unspecified situations. The implementation-defined characteristics are summarized in M.2. 18

The implementation may choose to document implementation-defined behavior either by documenting what happens in general, or by providing some mechanism for the user to determine what happens in a particular case. 19

Implementation Advice

If an implementation detects the use of an unsupported Specialized Needs Annex feature at run time, it should raise `Program_Error` if feasible. 20

If an implementation wishes to provide implementation-defined extensions to the functionality of a language-defined library unit, it should normally do so by adding children to the library unit. 21

NOTES

2 The above requirements imply that an implementation conforming to this Standard may support some of the capabilities required by a Specialized Needs Annex without supporting all required capabilities. 22

1.1.4 Method of Description and Syntax Notation

The form of an Ada program is described by means of a context-free syntax together with context-dependent requirements expressed by narrative rules. 1

The meaning of Ada programs is described by means of narrative rules defining both the effects of each construct and the composition rules for constructs. 2

The context-free syntax of the language is described using a simple variant of Backus-Naur Form. In particular: 3

- Lower case words in a sans-serif font, some containing embedded underlines, are used to denote syntactic categories, for example: 4

`case_statement` 5

- Boldface words are used to denote reserved words, for example: 6

array 7

- 8 • Square brackets enclose optional items. Thus the two following rules are equivalent.
- 9/2 `simple_return_statement ::= return [expression];`
 `simple_return_statement ::= return; | return expression;`
- 10 • Curly brackets enclose a repeated item. The item may appear zero or more times; the repetitions occur from left to right as with an equivalent left-recursive rule. Thus the two following rules are equivalent.
- 11 `term ::= factor {multiplying_operator factor}`
 `term ::= factor | term multiplying_operator factor`
- 12/3 • A vertical line separates alternative items unless it occurs immediately after an opening curly bracket or a `loop_parameter_specification`, in which case it stands for itself:
- 13 `constraint ::= scalar_constraint | composite_constraint`
 `discrete_choice_list ::= discrete_choice {| discrete_choice}`
- 14 • If the name of any syntactic category starts with an italicized part, it is equivalent to the category name without the italicized part. The italicized part is intended to convey some semantic information. For example *subtype_name* and *task_name* are both equivalent to *name* alone.
- 14.1/3 The delimiters, compound delimiters, reserved words, and `numeric_literals` are exclusively made of the characters whose code position is between 16#20# and 16#7E#, inclusively. The special characters for which names are defined in this International Standard (see 2.1) belong to the same range. For example, the character E in the definition of `exponent` is the character whose name is “LATIN CAPITAL LETTER E”, not “GREEK CAPITAL LETTER EPSILON”.
- 14.2/2 When this International Standard mentions the conversion of some character or sequence of characters to upper case, it means the character or sequence of characters obtained by using locale-independent full case folding, as defined by documents referenced in the note in section 1 of ISO/IEC 10646:2003.
- 15 A *syntactic category* is a nonterminal in the grammar defined in BNF under “Syntax.” Names of syntactic categories are set in a different font, like *this*.
- 16 A *construct* is a piece of text (explicit or implicit) that is an instance of a syntactic category defined under “Syntax”.
- 17 A *constituent* of a construct is the construct itself, or any construct appearing within it.
- 18 Whenever the run-time semantics defines certain actions to happen in an *arbitrary order*, this means that the implementation shall arrange for these actions to occur in a way that is equivalent to some sequential order, following the rules that result from that sequential order. When evaluations are defined to happen in an arbitrary order, with conversion of the results to some subtypes, or with some run-time checks, the evaluations, conversions, and checks may be arbitrarily interspersed, so long as each expression is evaluated before converting or checking its value. Note that the effect of a program can depend on the order chosen by the implementation. This can happen, for example, if two actual parameters of a given call have side effects.
- 19 NOTES
 3 The syntax rules describing structured constructs are presented in a form that corresponds to the recommended paragraphing. For example, an `if_statement` is defined as:

```

if_statement ::=
    if condition then
        sequence_of_statements
    {elsif condition then
        sequence_of_statements}
    [else
        sequence_of_statements]
    end if;

```

20

4 The line breaks and indentation in the syntax rules indicate the recommended line breaks and indentation in the corresponding constructs. The preferred places for other line breaks are after semicolons. 21

1.1.5 Classification of Errors

Implementation Requirements

The language definition classifies errors into several different categories: 1

- Errors that are required to be detected prior to run time by every Ada implementation; 2

These errors correspond to any violation of a rule given in this International Standard, other than those listed below. In particular, violation of any rule that uses the terms shall, allowed, permitted, legal, or illegal belongs to this category. Any program that contains such an error is not a legal Ada program; on the other hand, the fact that a program is legal does not mean, *per se*, that the program is free from other forms of error. 3

The rules are further classified as either compile time rules, or post compilation rules, depending on whether a violation has to be detected at the time a compilation unit is submitted to the compiler, or may be postponed until the time a compilation unit is incorporated into a partition of a program. 4

- Errors that are required to be detected at run time by the execution of an Ada program; 5

The corresponding error situations are associated with the names of the predefined exceptions. Every Ada compiler is required to generate code that raises the corresponding exception if such an error situation arises during program execution. If such an error situation is certain to arise in every execution of a construct, then an implementation is allowed (although not required) to report this fact at compilation time. 6

- Bounded errors; 7

The language rules define certain kinds of errors that need not be detected either prior to or during run time, but if not detected, the range of possible effects shall be bounded. The errors of this category are called *bounded errors*. The possible effects of a given bounded error are specified for each such error, but in any case one possible effect of a bounded error is the raising of the exception Program_Error. 8

- Erroneous execution. 9

In addition to bounded errors, the language rules define certain kinds of errors as leading to *erroneous execution*. Like bounded errors, the implementation need not detect such errors either prior to or during run time. Unlike bounded errors, there is no language-specified bound on the possible effect of erroneous execution; the effect is in general not predictable. 10

Implementation Permissions

An implementation may provide *nonstandard modes* of operation. Typically these modes would be selected by a `pragma` or by a command line switch when the compiler is invoked. When operating in a nonstandard mode, the implementation may reject `compilation_units` that do not conform to additional requirements associated with the mode, such as an excessive number of warnings or violation of coding style guidelines. Similarly, in a nonstandard mode, the implementation may apply special optimizations or 11

alternative algorithms that are only meaningful for programs that satisfy certain criteria specified by the implementation. In any case, an implementation shall support a *standard* mode that conforms to the requirements of this International Standard; in particular, in the standard mode, all legal compilation_units shall be accepted.

Implementation Advice

- 12 If an implementation detects a bounded error or erroneous execution, it should raise Program_Error.

1.2 Normative References

- 1 The following standards contain provisions which, through reference in this text, constitute provisions of this International Standard. At the time of publication, the editions indicated were valid. All standards are subject to revision, and parties to agreements based on this International Standard are encouraged to investigate the possibility of applying the most recent editions of the standards indicated below. Members of IEC and ISO maintain registers of currently valid International Standards.
- 2 ISO/IEC 646:1991, *Information technology — ISO 7-bit coded character set for information interchange*.
- 3/2 ISO/IEC 1539-1:2004, *Information technology — Programming languages — Fortran — Part 1: Base language*.
- 4/2 ISO/IEC 1989:2002, *Information technology — Programming languages — COBOL*.
- 5 ISO/IEC 6429:1992, *Information technology — Control functions for coded graphic character sets*.
- 5.1/2 ISO 8601:2004, *Data elements and interchange formats — Information interchange — Representation of dates and times*.
- 6 ISO/IEC 8859-1:1987, *Information processing — 8-bit single-byte coded character sets — Part 1: Latin alphabet No. 1*.
- 7/2 ISO/IEC 9899:1999, *Programming languages — C*, supplemented by Technical Corrigendum 1:2001 and Technical Corrigendum 2:2004.
- 8/2 ISO/IEC 10646:2003, *Information technology — Universal Multiple-Octet Coded Character Set (UCS)*.
- 9/2 ISO/IEC 14882:2003, *Programming languages — C++*.
- 10/2 ISO/IEC TR 19769:2004, *Information technology — Programming languages, their environments and system software interfaces — Extensions for the programming language C to support new character data types*.

1.3 Definitions

- 1/2 Terms are defined throughout this International Standard, indicated by *italic* type. Terms explicitly defined in this International Standard are not to be presumed to refer implicitly to similar terms defined elsewhere. Mathematical terms not defined in this International Standard are to be interpreted according to the *CRC Concise Encyclopedia of Mathematics, Second Edition*. Other terms not defined in this International Standard are to be interpreted according to the *Webster's Third New International Dictionary of the English Language*. Informal descriptions of some terms are also given in Annex N, “Glossary”.

Section 2: Lexical Elements

The text of a program consists of the texts of one or more compilations. The text of a compilation is a sequence of lexical elements, each composed of characters; the rules of composition are given in this section. Pragmas, which provide certain information for the compiler, are also described in this section. 1

2.1 Character Set

The character repertoire for the text of an Ada program consists of the entire coding space described by the ISO/IEC 10646:2003 Universal Multiple-Octet Coded Character Set. This coding space is organized in *planes*, each plane comprising 65536 characters. 1/2

Syntax

Paragraphs 2 and 3 were deleted.

A **character** is defined by this International Standard for each cell in the coding space described by ISO/IEC 10646:2003, regardless of whether or not ISO/IEC 10646:2003 allocates a character to that cell. 3.1/2

Static Semantics

The coded representation for characters is implementation defined (it need not be a representation defined within ISO/IEC 10646:2003). A character whose relative code position in its plane is 16#FFFE# or 16#FFFF# is not allowed anywhere in the text of a program. The only characters allowed outside of comments are those in categories `other_format`, `format_effector`, and `graphic_character`. 4/3

The semantics of an Ada program whose text is not in Normalization Form KC (as defined by section 24 of ISO/IEC 10646:2003) is implementation defined. 4.1/2

The description of the language definition in this International Standard uses the character properties General Category, Simple Uppercase Mapping, Uppercase Mapping, and Special Case Condition of the documents referenced by the note in section 1 of ISO/IEC 10646:2003. The actual set of graphic symbols used by an implementation for the visual representation of the text of an Ada program is not specified. 5/2

Characters are categorized as follows: 6/2

This paragraph was deleted. 7/2

letter_uppercase 8/2

Any character whose General Category is defined to be “Letter, Uppercase”.

letter_lowercase 9/2

Any character whose General Category is defined to be “Letter, Lowercase”.

letter_titlecase 9.1/2

Any character whose General Category is defined to be “Letter, Titlecase”.

letter_modifier 9.2/2

Any character whose General Category is defined to be “Letter, Modifier”.

letter_other 9.3/2

Any character whose General Category is defined to be “Letter, Other”.

mark_non_spacing 9.4/2

Any character whose General Category is defined to be “Mark, Non-Spacing”.

- 9.5/2 **mark_spacing_combining**
Any character whose General Category is defined to be “Mark, Spacing Combining”.
- 10/2 **number_decimal**
Any character whose General Category is defined to be “Number, Decimal”.
- 10.1/2 **number_letter**
Any character whose General Category is defined to be “Number, Letter”.
- 10.2/2 **punctuation_connector**
Any character whose General Category is defined to be “Punctuation, Connector”.
- 10.3/2 **other_format**
Any character whose General Category is defined to be “Other, Format”.
- 11/2 **separator_space**
Any character whose General Category is defined to be “Separator, Space”.
- 12/2 **separator_line**
Any character whose General Category is defined to be “Separator, Line”.
- 12.1/2 **separator_paragraph**
Any character whose General Category is defined to be “Separator, Paragraph”.
- 13/2 **format_effector**
The characters whose code positions are 16#09# (CHARACTER TABULATION), 16#0A# (LINE FEED), 16#0B# (LINE TABULATION), 16#0C# (FORM FEED), 16#0D# (CARRIAGE RETURN), 16#85# (NEXT LINE), and the characters in categories **separator_line** and **separator_paragraph**.
- 13.1/2 **other_control**
Any character whose General Category is defined to be “Other, Control”, and which is not defined to be a **format_effector**.
- 13.2/2 **other_private_use**
Any character whose General Category is defined to be “Other, Private Use”.
- 13.3/2 **other_surrogate**
Any character whose General Category is defined to be “Other, Surrogate”.
- 14/2 **graphic_character**
Any character that is not in the categories **other_control**, **other_private_use**, **other_surrogate**, **format_effector**, and whose relative code position in its plane is neither 16#FFFE# nor 16#FFFF#.
- 15/2 The following names are used when referring to certain characters (the first name is that given in ISO/IEC 10646:2003):

graphic symbol	name	graphic symbol	name
"	quotation mark	:	colon
#	number sign	;	semicolon
&	ampersand	<	less-than sign
'	apostrophe, tick	=	equals sign
(	left parenthesis	>	greater-than sign
)	right parenthesis	—	low line, underline
*	asterisk, multiply		vertical line
+	plus sign	/	solidus, divide
,	comma	!	exclamation point
—	hyphen-minus, minus	%	percent sign
.	full stop, dot, point		

Implementation Permissions

This paragraph was deleted. 16/2

- NOTES
- 1 The characters in categories `other_control`, `other_private_use`, and `other_surrogate` are only allowed in comments. 17/2
- 2 The language does not specify the source representation of programs. 18

2.2 Lexical Elements, Separators, and Delimiters

Static Semantics

The text of a program consists of the texts of one or more compilations. The text of each compilation is a sequence of separate *lexical elements*. Each lexical element is formed from a sequence of characters, and is either a delimiter, an identifier, a reserved word, a `numeric_literal`, a `character_literal`, a `string_literal`, or a comment. The meaning of a program depends only on the particular sequences of lexical elements that form its compilations, excluding comments. 1

The text of a compilation is divided into *lines*. In general, the representation for an end of line is implementation defined. However, a sequence of one or more `format_effectors` other than the character whose code position is 16#09# (CHARACTER TABULATION) signifies at least one end of line. 2/2

In some cases an explicit *separator* is required to separate adjacent lexical elements. A separator is any of a `separator_space`, a `format_effector`, or the end of a line, as follows: 3/2

- A `separator_space` is a separator except within a comment, a `string_literal`, or a `character_literal`. 4/2
- The character whose code position is 16#09# (CHARACTER TABULATION) is a separator except within a comment. 5/2
- The end of a line is always a separator. 6

One or more separators are allowed between any two adjacent lexical elements, before the first of each compilation, or after the last. At least one separator is required between an identifier, a reserved word, or a `numeric_literal` and an adjacent identifier, reserved word, or `numeric_literal`. 7

One of more `other_format` characters are allowed anywhere that a separator is; any such characters have no effect on the meaning of an Ada program. 7.1/3

8/2 A *delimiter* is either one of the following characters:

9 & ' () * + , - . / : ; < = > |

10 or one of the following *compound delimiters* each composed of two adjacent special characters

11 => .. ** := /= >= <= << >> <>

12 Each of the special characters listed for single character delimiters is a single delimiter except if this character is used as a character of a compound delimiter, or as a character of a *comment*, *string_literal*, *character_literal*, or *numeric_literal*.

13 The following names are used when referring to compound delimiters:

delimiter	name
=>	arrow
..	double dot
**	double star, exponentiate
:=	assignment (pronounced: “becomes”)
/=	inequality (pronounced: “not equal”)
>=	greater than or equal
<=	less than or equal
<<	left label bracket
>>	right label bracket
<>	box

Implementation Requirements

14 An implementation shall support lines of at least 200 characters in length, not counting any characters used to signify the end of a line. An implementation shall support lexical elements of at least 200 characters in length. The maximum supported line length and lexical element length are implementation defined.

2.3 Identifiers

1 Identifiers are used as names.

Syntax

2/2 identifier ::=
 identifier_start {identifier_start | identifier_extend}

3/2 identifier_start ::=
 letter_uppercase
 | letter_lowercase
 | letter_titlecase
 | letter_modifier
 | letter_other
 | number_letter

identifier_extend ::= 3.1/3
 mark_non_spacing
 | mark_spacing_combining
 | number_decimal
 | punctuation_connector

An identifier shall not contain two consecutive characters in category punctuation_connector, or end with a character in that category. 4/3

Static Semantics

Two identifiers are considered the same if they consist of the same sequence of characters after converting the characters to upper case. 5/3

- *This paragraph was deleted.* 5.1/3
- *This paragraph was deleted.* 5.2/3

After converting to upper case, an identifier shall not be identical to a reserved word (in upper case). 5.3/3

Implementation Permissions

In a nonstandard mode, an implementation may support other upper/lower case equivalence rules for identifiers, to accommodate local conventions. 6

NOTES

- 3 Identifiers differing only in the use of corresponding upper and lower case letters are considered the same. 6.1/2

Examples

Examples of identifiers: 7

Count	X	Get_Symbol	Ethelyn	Marion	8/2
Snobol_4	X1	Page_Count	Store_Next_Item		
Πλάτων	--	Plato			
Чайковский	--	Tchaikovsky			
θ φ	--	Angles			

2.4 Numeric Literals

There are two kinds of numeric_literals, *real literals* and *integer literals*. A real literal is a numeric_literal that includes a point; an integer literal is a numeric_literal without a point. 1

Syntax

numeric_literal ::= decimal_literal | based_literal 2

NOTES

- 4 The type of an integer literal is *universal_integer*. The type of a real literal is *universal_real*. 3

2.4.1 Decimal Literals

A decimal_literal is a numeric_literal in the conventional decimal notation (that is, the base is ten). 1

Syntax

decimal_literal ::= numeral [.numeral] [exponent] 2

numeral ::= digit {[underline] digit} 3

exponent ::= E [+] numeral | E – numeral 4

digit ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 4.1/2

An exponent for an integer literal shall not have a minus sign.

Static Semantics

An underline character in a `numeric_literal` does not affect its meaning. The letter E of an exponent can be written either in lower case or in upper case, with the same meaning.

An exponent indicates the power of ten by which the value of the `decimal_literal` without the exponent is to be multiplied to obtain the value of the `decimal_literal` with the exponent.

Examples

Examples of decimal literals:

```
12          0          1E6      123_456      -- integer literals
12.0        0.0        0.456    3.14159_26    -- real literals
```

2.4.2 Based Literals

A `based_literal` is a `numeric_literal` expressed in a form that specifies the base explicitly.

Syntax

```
based_literal ::=
  base # based_numeral [.based_numeral] # [exponent]
base ::= numeral
based_numeral ::=
  extended_digit {[underline] extended_digit}
extended_digit ::= digit | A | B | C | D | E | F
```

Legality Rules

The *base* (the numeric value of the decimal numeral preceding the first #) shall be at least two and at most sixteen. The `extended_digits` A through F represent the digits ten through fifteen, respectively. The value of each `extended_digit` of a `based_literal` shall be less than the base.

Static Semantics

The conventional meaning of based notation is assumed. An exponent indicates the power of the base by which the value of the `based_literal` without the exponent is to be multiplied to obtain the value of the `based_literal` with the exponent. The base and the exponent, if any, are in decimal notation.

The `extended_digits` A through F can be written either in lower case or in upper case, with the same meaning.

Examples

Examples of based literals:

```
2#1111_1111#  16#FF#          016#0ff#      -- integer literals of value 255
16#E#E1       2#1110_0000#      -- integer literals of value 224
16#F.FF#E+2   2#1.1111_1111_1110#E11    -- real literals of value 4095.0
```

2.5 Character Literals

A `character_literal` is formed by enclosing a graphic character between two apostrophe characters.

Syntax

`character_literal ::= 'graphic_character'`

2

NOTES

5 A `character_literal` is an enumeration literal of a character type. See 3.5.2.

3

Examples

Examples of character literals:

4

'A' '*' ' ' ' ' -- Various els.
'L' 'J' 'Δ' -- Big numbers - infinity and aleph.
'∞' 'ℵ'

5/2

2.6 String Literals

A `string_literal` is formed by a sequence of graphic characters (possibly none) enclosed between two quotation marks used as string brackets. They are used to represent `operator_symbols` (see 6.1), values of a string type (see 4.2), and array subaggregates (see 4.3.3).

1

Syntax

`string_literal ::= "{string_element}"`

2

`string_element ::= "" | non_quotation_mark_graphic_character`

3

A `string_element` is either a pair of quotation marks (""), or a single `graphic_character` other than a quotation mark.

4

Static Semantics

The *sequence of characters* of a `string_literal` is formed from the sequence of `string_elements` between the bracketing quotation marks, in the given order, with a `string_element` that is "" becoming a single quotation mark in the sequence of characters, and any other `string_element` being reproduced in the sequence.

5

A *null string literal* is a `string_literal` with no `string_elements` between the quotation marks.

6

NOTES

6 An end of line cannot appear in a `string_literal`.

7

7 No transformation is performed on the sequence of characters of a `string_literal`.

7.1/2

Examples

Examples of string literals:

8

"Message of the day:"

9/2

" -- a null string literal
" " "A" "" -- three string literals of length 1

"Characters such as \$, %, and } are allowed in string literals"
"Archimedes said "Εὐρηκα"
"Volume of cylinder ($\pi r^2 h$) = "

2.7 Comments

A comment starts with two adjacent hyphens and extends up to the end of the line.

1

Syntax

2 comment ::= --{*non_end_of_line_character*}

3 A comment may appear on any line of a program.

Static Semantics

4 The presence or absence of comments has no influence on whether a program is legal or illegal. Furthermore, comments do not influence the meaning of a program; their sole purpose is the enlightenment of the human reader.

Examples

5 *Examples of comments:*

6 -- the last sentence above echoes the Algol 68 report

end; -- processing of Line is complete

 -- a long comment may be split onto

 -- two or more consecutive lines

 ----- the first two hyphens start the comment

2.8 Pragmas

1 A pragma is a compiler directive. There are language-defined pragmas that give instructions for optimization, listing control, etc. An implementation may support additional (implementation-defined) pragmas.

Syntax

2 pragma ::=

pragma identifier [(pragma_argument_association {, pragma_argument_association})];

3 pragma_argument_association ::=

 [*pragma_argument_identifier* =>] name

 | [*pragma_argument_identifier* =>] expression

4 In a pragma, any *pragma_argument_associations* without a *pragma_argument_identifier* shall precede any associations with a *pragma_argument_identifier*.

5 Pragmas are only allowed at the following places in a program:

- 6 • After a semicolon delimiter, but not within a *formal_part* or *discriminant_part*.
- 7/3 • At any place where the syntax rules allow a construct defined by a syntactic category whose name ends with “declaration”, “statement”, “clause”, or “alternative”, or one of the syntactic categories *variant* or *exception_handler*; but not in place of such a construct if the construct is required, or is part of a list that is required to have at least one such construct, as in a *sequence_of_statements*.
- 7.1/3 • At any place where a *compilation_unit* is allowed.

8 Additional syntax rules and placement restrictions exist for specific pragmas.

9 The *name* of a pragma is the identifier following the reserved word **pragma**. The name or expression of a *pragma_argument_association* is a *pragma_argument*.

10 An *identifier specific to a pragma* is an identifier that is used in a pragma argument with special meaning for that pragma.

Static Semantics

If an implementation does not recognize the name of a **pragma**, then it has no effect on the semantics of the program. Inside such a **pragma**, the only rules that apply are the Syntax Rules. 11

Dynamic Semantics

Any **pragma** that appears at the place of an executable construct is executed. Unless otherwise specified for a particular **pragma**, this execution consists of the evaluation of each evaluable **pragma** argument in an arbitrary order. 12

Implementation Requirements

The implementation shall give a warning message for an unrecognized **pragma** name. 13

Implementation Permissions

An implementation may provide implementation-defined **pragmas**; the name of an implementation-defined **pragma** shall differ from those of the language-defined **pragmas**. 14

An implementation may ignore an unrecognized **pragma** even if it violates some of the Syntax Rules, if detecting the syntax error is too complex. 15

Implementation Advice

Normally, implementation-defined **pragmas** should have no semantic effect for error-free programs; that is, if the implementation-defined **pragmas** are removed from a working program, the program should still be legal, and should still have the same semantics. 16

Normally, an implementation should not define **pragmas** that can make an illegal program legal, except as follows: 17

- A **pragma** used to complete a declaration, such as a **pragma Import**; 18
- A **pragma** used to configure the environment by adding, removing, or replacing **library_items**. 19

Syntax

The forms of **List**, **Page**, and **Optimize** **pragmas** are as follows: 20

pragma List(identifier); 21

pragma Page; 22

pragma Optimize(identifier); 23

Other **pragmas** are defined throughout this International Standard, and are summarized in Annex L. 24

Static Semantics

A **pragma List** takes one of the identifiers **On** or **Off** as the single argument. This **pragma** is allowed anywhere a **pragma** is allowed. It specifies that listing of the compilation is to be continued or suspended until a **List pragma** with the opposite argument is given within the same compilation. The **pragma** itself is always listed if the compiler is producing a listing. 25

A **pragma Page** is allowed anywhere a **pragma** is allowed. It specifies that the program text which follows the **pragma** should start on a new page (if the compiler is currently producing a listing). 26

A **pragma Optimize** takes one of the identifiers **Time**, **Space**, or **Off** as the single argument. This **pragma** is allowed anywhere a **pragma** is allowed, and it applies until the end of the immediately enclosing declarative region, or for a **pragma** at the place of a **compilation_unit**, to the end of the compilation. It 27

gives advice to the implementation as to whether time or space is the primary optimization criterion, or that optional optimizations should be turned off. It is implementation defined how this advice is followed.

Examples

28 *Examples of pragmas:*

29/2 **pragma** List (Off) ; -- turn off listing generation
pragma Optimize (Off) ; -- turn off optional optimizations
pragma Inline (Set_Mask) ; -- generate code for Set_Mask inline
pragma Import (C, Put_Char, External_Name => "putchar") ; -- import C putchar function

2.9 Reserved Words

Syntax

1/1 *This paragraph was deleted.*

2/3 The following are the *reserved words*. Within a program, some or all of the letters of a reserved word may be in upper case.

abort	else	new	return
abs	elsif	not	reverse
abstract	end	null	
accept	entry		select
access	exception	of	separate
aliased	exit	or	subtype
all		others	synchronized
and	for	out	tagged
array	function	overriding	task
at	generic	package	terminate
	goto	pragma	then
begin		private	type
body	if	procedure	
	in	protected	until
case	interface		use
constant	is	raise	
		range	when
declare	limited	record	while
delay	loop	rem	with
delta		renames	
digits	mod	requeue	xor
do			

NOTES

3 8 The reserved words appear in **lower case boldface** in this International Standard, except when used in the designator of an attribute (see 4.1.4). Lower case boldface is also used for a reserved word in a `string_literal` used as an `operator_symbol`. This is merely a convention — programs may be written in whatever typeface is desired and available.

Section 3: Declarations and Types

This section describes the types in the language and the rules for declaring constants, variables, and named numbers. 1

3.1 Declarations

The language defines several kinds of named *entities* that are declared by declarations. The entity's *name* is defined by the declaration, usually by a *defining_identifier*, but sometimes by a *defining_character_literal* or *defining_operator_symbol*. 1

There are several forms of declaration. A *basic_declaration* is a form of declaration defined as follows. 2

Syntax

```
basic_declaration ::=
    type_declaration           | subtype_declaration
    | object_declaration       | number_declaration
    | subprogram_declaration   | abstract_subprogram_declaration
    | null_procedure_declaration | package_declaration
    | renaming_declaration     | exception_declaration
    | generic_declaration      | generic_instantiation

defining_identifier ::= identifier
```

3/2

4

Static Semantics

A *declaration* is a language construct that associates a name with (a view of) an entity. A declaration may appear explicitly in the program text (an *explicit* declaration), or may be supposed to occur at a given place in the text as a consequence of the semantics of another construct (an *implicit* declaration). 5

Each of the following is defined to be a declaration: any *basic_declaration*; an *enumeration_literal_specification*; a *discriminant_specification*; a *component_declaration*; a *loop_parameter_specification*; a *parameter_specification*; a *subprogram_body*; an *entry_declaration*; an *entry_index_specification*; a *choice_parameter_specification*; a *generic_formal_parameter_declaration*. In addition, an *extended_return_statement* is a declaration of its *defining_identifier*. 6/2

All declarations contain a *definition* for a *view* of an entity. A view consists of an identification of the entity (the entity *of* the view), plus view-specific characteristics that affect the use of the entity through that view (such as mode of access to an object, formal parameter names and defaults for a subprogram, or visibility to components of a type). In most cases, a declaration also contains the definition for the entity itself (a *renaming_declaration* is an example of a declaration that does not define a new entity, but instead defines a view of an existing entity (see 8.5)). 7

When it is clear from context, the term *object* is used in place of *view of an object*. Similarly, the terms *type* and *subtype* are used in place of *view of a type* and *view of a subtype*, respectively. 7.1/3

For each declaration, the language rules define a certain region of text called the *scope* of the declaration (see 8.2). Most declarations associate an *identifier* with a declared entity. Within its scope, and only there, there are places where it is possible to use the *identifier* to refer to the declaration, the view it defines, and the associated entity; these places are defined by the visibility rules (see 8.3). At such places the *identifier* is said to be a *name* of the entity (the *direct_name* or *selector_name*); the name is said to *denote* the 8

declaration, the view, and the associated entity (see 8.6). The declaration is said to *declare* the name, the view, and in most cases, the entity itself.

- 9 As an alternative to an *identifier*, an enumeration literal can be declared with a *character_literal* as its name (see 3.5.1), and a function can be declared with an *operator_symbol* as its name (see 6.1).
- 10 The syntax rules use the terms *defining_identifier*, *defining_character_literal*, and *defining_operator_symbol* for the defining occurrence of a name; these are collectively called *defining names*. The terms *direct_name* and *selector_name* are used for usage occurrences of *identifiers*, *character_literals*, and *operator_symbols*. These are collectively called *usage names*.

Dynamic Semantics

- 11 The process by which a construct achieves its run-time effect is called *execution*. This process is also called *elaboration* for declarations and *evaluation* for expressions. One of the terms execution, elaboration, or evaluation is defined by this International Standard for each construct that has a run-time effect.

NOTES

- 12 1 At compile time, the declaration of an entity *declares* the entity. At run time, the elaboration of the declaration *creates* the entity.

3.2 Types and Subtypes

Static Semantics

- 1 A *type* is characterized by a set of values, and a set of *primitive operations* which implement the fundamental aspects of its semantics. An *object* of a given type is a run-time entity that contains (has) a value of the type.
- 2/2 Types are grouped into *categories* of types. There exist several *language-defined categories* of types (see NOTES below), reflecting the similarity of their values and primitive operations. Most categories of types form *classes* of types. *Elementary* types are those whose values are logically indivisible; *composite* types are those whose values are composed of *component* values.
- 3 The elementary types are the *scalar* types (*discrete* and *real*) and the *access* types (whose values provide access to objects or subprograms). Discrete types are either *integer* types or are defined by enumeration of their values (*enumeration* types). Real types are either *floating point* types or *fixed point* types.
- 4/2 The composite types are the *record* types, *record extensions*, *array* types, *interface* types, *task* types, and *protected* types.
- 4.1/2 There can be multiple views of a type with varying sets of operations. An *incomplete* type represents an incomplete view (see 3.10.1) of a type with a very restricted usage, providing support for recursive data structures. A *private* type or *private extension* represents a partial view (see 7.3) of a type, providing support for data abstraction. The full view (see 3.2.1) of a type represents its complete definition. An incomplete or partial view is considered a composite type, even if the full view is not.
- 5/2 Certain composite types (and views thereof) have special components called *discriminants* whose values affect the presence, constraints, or initialization of other components. Discriminants can be thought of as parameters of the type.
- 6/2 The term *subcomponent* is used in this International Standard in place of the term component to indicate either a component, or a component of another subcomponent. Where other subcomponents are excluded,

the term component is used instead. Similarly, a *part* of an object or value is used to mean the whole object or value, or any set of its subcomponents. The terms component, subcomponent, and part are also applied to a type meaning the component, subcomponent, or part of objects and values of the type.

The set of possible values for an object of a given type can be subjected to a condition that is called a *constraint* (the case of a *null constraint* that specifies no restriction is also included); the rules for which values satisfy a given kind of constraint are given in 3.5 for *range_constraints*, 3.6.1 for *index_constraints*, and 3.7.1 for *discriminant_constraints*. The set of possible values for an object of an access type can also be subjected to a condition that excludes the null value (see 3.10). 7/2

A *subtype* of a given type is a combination of the type, a constraint on values of the type, and certain attributes specific to the subtype. The given type is called the *type of the subtype*. Similarly, the associated constraint is called the *constraint of the subtype*. The set of values of a subtype consists of the values of its type that satisfy its constraint and any exclusion of the null value. Such values *belong* to the subtype. 8/2

A subtype is called an *unconstrained* subtype if its type has unknown discriminants, or if its type allows range, index, or discriminant constraints, but the subtype does not impose such a constraint; otherwise, the subtype is called a *constrained* subtype (since it has no unconstrained characteristics). 9

NOTES

2 Any set of types can be called a “category” of types, and any set of types that is closed under derivation (see 3.4) can be called a “class” of types. However, only certain categories and classes are used in the description of the rules of the language — generally those that have their own particular set of primitive operations (see 3.2.3), or that correspond to a set of types that are matched by a given kind of generic formal type (see 12.5). The following are examples of “interesting” *language-defined classes*: elementary, scalar, discrete, enumeration, character, boolean, integer, signed integer, modular, real, floating point, fixed point, ordinary fixed point, decimal fixed point, numeric, access, access-to-object, access-to-subprogram, composite, array, string, (untagged) record, tagged, task, protected, unlimited. Special syntax is provided to define types in each of these classes. In addition to these classes, the following are examples of “interesting” *language-defined categories*: abstract, incomplete, interface, limited, private, record. 10/2

These language-defined categories are organized like this: 11/2

```

all types
  elementary
    scalar
      discrete
        enumeration
          character
          boolean
          other enumeration
        integer
          signed integer
          modular integer
      real
        floating point
        fixed point
          ordinary fixed point
          decimal fixed point
      access
        access-to-object
        access-to-subprogram
    composite
      untagged
      array
  
```

12/2

string
 other array
 record
 task
 protected
 tagged (including interfaces)
 nonlimited tagged record
 limited tagged
 limited tagged record
 synchronized tagged
 tagged task
 tagged protected

13/2 There are other categories, such as “numeric” and “discriminated”, which represent other categorization dimensions, but do not fit into the above strictly hierarchical picture.

3.2.1 Type Declarations

1 A `type_declaration` declares a type and its first subtype.

Syntax

```

2  type_declaration ::= full_type_declaration
    | incomplete_type_declaration
    | private_type_declaration
    | private_extension_declaration
3  full_type_declaration ::=
    type defining_identifier [known_discriminant_part] is type_definition;
    | task_type_declaration
    | protected_type_declaration
4/2 type_definition ::=
    enumeration_type_definition | integer_type_definition
    | real_type_definition      | array_type_definition
    | record_type_definition    | access_type_definition
    | derived_type_definition   | interface_type_definition
  
```

Legality Rules

5 A given type shall not have a subcomponent whose type is the given type itself.

Static Semantics

6 The `defining_identifier` of a `type_declaration` denotes the *first subtype* of the type. The `known_discriminant_part`, if any, defines the discriminants of the type (see 3.7, “Discriminants”). The remainder of the `type_declaration` defines the remaining characteristics of (the view of) the type.

7/2 A type defined by a `type_declaration` is a *named* type; such a type has one or more nameable subtypes. Certain other forms of declaration also include type definitions as part of the declaration for an object. The type defined by such a declaration is *anonymous* — it has no nameable subtypes. For explanatory purposes, this International Standard sometimes refers to an anonymous type by a pseudo-name, written in italics, and uses such pseudo-names at places where the syntax normally requires an `identifier`. For a named type whose first subtype is T, this International Standard sometimes refers to the type of T as simply “the type T”.

A named type that is declared by a `full_type_declaration`, or an anonymous type that is defined by an `access_definition` or as part of declaring an object of the type, is called a *full type*. The declaration of a full type also declares the *full view* of the type. The `type_definition`, `task_definition`, `protected_definition`, or `access_definition` that defines a full type is called a *full type definition*. Types declared by other forms of `type_declaration` are not separate types; they are partial or incomplete views of some full type. 8/2

The definition of a type implicitly declares certain *predefined operators* that operate on the type, according to what classes the type belongs to, as specified in 4.5, “Operators and Expression Evaluation”. 9

The *predefined types* (for example the types Boolean, Wide_Character, Integer, *root_integer*, and *universal_integer*) are the types that are defined in a predefined library package called Standard; this package also includes the (implicit) declarations of their predefined operators. The package Standard is described in A.1. 10

Dynamic Semantics

The elaboration of a `full_type_declaration` consists of the elaboration of the full type definition. Each elaboration of a full type definition creates a distinct type and its first subtype. 11

Examples

Examples of type definitions: 12

```
(White, Red, Yellow, Green, Blue, Brown, Black)
range 1 .. 72
array(1 .. 10) of Integer
```

 13

Examples of type declarations: 14

```
type Color is (White, Red, Yellow, Green, Blue, Brown, Black);
type Column is range 1 .. 72;
type Table is array(1 .. 10) of Integer;
```

 15

NOTES

3 Each of the above examples declares a named type. The identifier given denotes the first subtype of the type. Other named subtypes of the type can be declared with `subtype_declarations` (see 3.2.2). Although names do not directly denote types, a phrase like “the type Column” is sometimes used in this International Standard to refer to the type of Column, where Column denotes the first subtype of the type. For an example of the definition of an anonymous type, see the declaration of the array `Color_Table` in 3.3.1; its type is anonymous — it has no nameable subtypes. 16

3.2.2 Subtype Declarations

A `subtype_declaration` declares a subtype of some previously declared type, as defined by a `subtype_indication`. 1

Syntax

```
subtype_declaration ::=
    subtype defining_identifier is subtype_indication; 2

subtype_indication ::= [null_exclusion] subtype_mark [constraint] 3/2

subtype_mark ::= subtype_name 4

constraint ::= scalar_constraint | composite_constraint 5

scalar_constraint ::= 6
    range_constraint | digits_constraint | delta_constraint

composite_constraint ::= 7
    index_constraint | discriminant_constraint
```

Name Resolution Rules

- 8 A `subtype_mark` shall resolve to denote a subtype. The type *determined by* a `subtype_mark` is the type of the subtype denoted by the `subtype_mark`.

Dynamic Semantics

- 9 The elaboration of a `subtype_declaration` consists of the elaboration of the `subtype_indication`. The elaboration of a `subtype_indication` creates a new subtype. If the `subtype_indication` does not include a constraint, the new subtype has the same (possibly null) constraint as that denoted by the `subtype_mark`. The elaboration of a `subtype_indication` that includes a constraint proceeds as follows:
- 10 • The constraint is first elaborated.
- 11 • A check is then made that the constraint is *compatible* with the subtype denoted by the `subtype_mark`.
- 12 The condition imposed by a constraint is the condition obtained after elaboration of the constraint. The rules defining compatibility are given for each form of constraint in the appropriate subclause. These rules are such that if a constraint is *compatible* with a subtype, then the condition imposed by the constraint cannot contradict any condition already imposed by the subtype on its values. The exception `Constraint_Error` is raised if any check of compatibility fails.

NOTES

- 13 4 A `scalar_constraint` may be applied to a subtype of an appropriate scalar type (see 3.5, 3.5.9, and J.3), even if the subtype is already constrained. On the other hand, a `composite_constraint` may be applied to a composite subtype (or an access-to-composite subtype) only if the composite subtype is unconstrained (see 3.6.1 and 3.7.1).

Examples

- 14 *Examples of subtype declarations:*

```
15/2  subtype Rainbow    is Color range Red .. Blue;           -- see 3.2.1
      subtype Red_Blue  is Rainbow;
      subtype Int       is Integer;
      subtype Small_Int is Integer range -10 .. 10;
      subtype Up_To_K   is Column range 1 .. K;              -- see 3.2.1
      subtype Square    is Matrix(1 .. 10, 1 .. 10);         -- see 3.6
      subtype Male      is Person(Sex => M);                 -- see 3.10.1
      subtype Binop_Ref is not null Binop_Ptr;               -- see 3.10
```

3.2.3 Classification of Operations

Static Semantics

- 1/2 An operation *operates on a type* *T* if it yields a value of type *T*, if it has an operand whose expected type (see 8.6) is *T*, or if it has an access parameter or access result type (see 6.1) designating *T*. A predefined operator, or other language-defined operation such as assignment or a membership test, that operates on a type, is called a *predefined operation* of the type. The *primitive operations* of a type are the predefined operations of the type, plus any user-defined primitive subprograms.
- 2 The *primitive subprograms* of a specific type are defined as follows:
- 3 • The predefined operators of the type (see 4.5);
- 4 • For a derived type, the inherited (see 3.4) user-defined subprograms;
- 5 • For an enumeration type, the enumeration literals (which are considered parameterless functions — see 3.5.1);

- For a specific type declared immediately within a `package_specification`, any subprograms (in addition to the enumeration literals) that are explicitly declared immediately within the same `package_specification` and that operate on the type; 6
- For a specific type with an explicitly declared primitive `"=` operator whose result type is Boolean, the corresponding `"/=` operator (see 6.6); 6.1/3
- For a nonformal type, any subprograms not covered above that are explicitly declared immediately within the same declarative region as the type and that override (see 8.3) other implicitly declared primitive subprograms of the type. 7/2

A primitive subprogram whose designator is an `operator_symbol` is called a *primitive operator*. 8

3.3 Objects and Named Numbers

Objects are created at run time and contain a value of a given type. An object can be created and initialized as part of elaborating a declaration, evaluating an `allocator`, `aggregate`, or `function_call`, or passing a parameter by copy. Prior to reclaiming the storage for an object, it is finalized if necessary (see 7.6.1). 1

Static Semantics

All of the following are objects: 2

- the entity declared by an `object_declaration`; 3
- a formal parameter of a subprogram, entry, or generic subprogram; 4
- a generic formal object; 5
- a loop parameter; 6
- a choice parameter of an `exception_handler`; 7
- an entry index of an `entry_body`; 8
- the result of dereferencing an access-to-object value (see 4.1); 9
- the return object of a function; 10/3
- the result of evaluating an `aggregate`; 11
- a `qualified_expression` whose operand denotes an object; 11.1/3
- a component, slice, or view conversion of another object. 12

An object is either a *constant* object or a *variable* object. Similarly, a view of an object is either a *constant* or a *variable*. All views of a constant elementary object are constant. All views of a constant composite object are constant, except for parts that are of controlled or immutably limited types; variable views of those parts and their subcomponents may exist. In this sense, objects of controlled and immutably limited types are *inherently mutable*. A constant view of an object cannot be used to modify its value. The terms constant and variable by themselves refer to constant and variable views of objects. 13/3

The value of an object is *read* when the value of any part of the object is evaluated, or when the value of an enclosing object is evaluated. The value of a variable is *updated* when an assignment is performed to any part of the variable, or when an assignment is performed to an enclosing object. 14

Whether a view of an object is constant or variable is determined by the definition of the view. The following (and no others) represent constants: 15

- an object declared by an `object_declaration` with the reserved word **constant**; 16

- 17 • a formal parameter or generic formal object of mode **in**;
 - 18 • a discriminant;
 - 19 • a loop parameter, choice parameter, or entry index;
 - 20 • the dereference of an access-to-constant value;
 - 20.1/3 • the return object declared by an `extended_return_statement` with the reserved word **constant**;
 - 21/3 • the object denoted by a `function_call` or an `aggregate`;
 - 21.1/3 • the result of evaluating a `qualified_expression`;
 - 21.2/3 • within the body of a protected function (or a function declared immediately within a `protected_body`), the current instance of the enclosing protected unit;
 - 22 • a `selected_component`, `indexed_component`, `slice`, or view conversion of a constant.
- 23 At the place where a view of an object is defined, a *nominal subtype* is associated with the view. The object's *actual subtype* (that is, its subtype) can be more restrictive than the nominal subtype of the view; it always is if the nominal subtype is an *indefinite subtype*. A subtype is an indefinite subtype if it is an unconstrained array subtype, or if it has unknown discriminants or unconstrained discriminants without defaults (see 3.7); otherwise the subtype is a *definite subtype* (all elementary subtypes are definite subtypes). A class-wide subtype is defined to have unknown discriminants, and is therefore an indefinite subtype. An indefinite subtype does not by itself provide enough information to create an object; an additional **constraint** or explicit initialization **expression** is necessary (see 3.3.1). A component cannot have an indefinite nominal subtype.
- 23.1/3 A view of a composite object is *known to be constrained* if:
- 23.2/3 • its nominal subtype is constrained, and is not an untagged partial view; or
 - 23.3/3 • its nominal subtype is indefinite; or
 - 23.4/3 • its type is immutably limited (see 7.5); or
 - 23.5/3 • it is part of a stand-alone constant (including a generic formal object of mode **in**); or
 - 23.6/3 • it is part of a formal parameter of mode **in**; or
 - 23.7/3 • it is part of the object denoted by a `function_call` or `aggregate`; or
 - 23.8/3 • it is part of a constant return object of an `extended_return_statement`; or
 - 23.9/3 • it is a dereference of a pool-specific access type, and there is no ancestor of its type that has a constrained partial view.
- 23.10/3 For the purposes of determining within a generic body whether an object is known to be constrained:
- 23.11/3 • if a subtype is a descendant of an untagged generic formal private or derived type, and the subtype is not an unconstrained array subtype, it is not considered indefinite and is considered to have a constrained partial view;
 - 23.12/3 • if a subtype is a descendant of a formal access type, it is not considered pool-specific.
- 24 A *named number* provides a name for a numeric value known at compile time. It is declared by a `number_declaration`.
- NOTES
- 25 5 A constant cannot be the target of an assignment operation, nor be passed as an **in out** or **out** parameter, between its initialization and finalization, if any.
 - 25.1/3 6 The value of a constant object cannot be changed after its initialization, except in some cases where the object has a controlled or immutably limited part (see 7.5, 7.6, and 13.9.1).

7 The nominal and actual subtypes of an elementary object are always the same. For a discriminated or array object, if the nominal subtype is constrained then so is the actual subtype. 26

3.3.1 Object Declarations

An `object_declaration` declares a *stand-alone* object with a given nominal subtype and, optionally, an explicit initial value given by an initialization expression. For an array, task, or protected object, the `object_declaration` may include the definition of the (anonymous) type of the object. 1

Syntax

```
object_declaration ::=
    defining_identifier_list : [aliased] [constant] subtype_indication [:= expression];
    | defining_identifier_list : [aliased] [constant] access_definition [:= expression];
    | defining_identifier_list : [aliased] [constant] array_type_definition [:= expression];
    | single_task_declaration
    | single_protected_declaration
    defining_identifier_list ::=
    defining_identifier {, defining_identifier} 2/2
```

3

Name Resolution Rules

For an `object_declaration` with an expression following the compound delimiter `:=`, the type expected for the expression is that of the object. This expression is called the *initialization expression*. 4

Legality Rules

An `object_declaration` without the reserved word **constant** declares a variable object. If it has a `subtype_indication` or an `array_type_definition` that defines an indefinite subtype, then there shall be an initialization expression. 5/2

Static Semantics

An `object_declaration` with the reserved word **constant** declares a constant object. If it has an initialization expression, then it is called a *full constant declaration*. Otherwise it is called a *deferred constant declaration*. The rules for deferred constant declarations are given in clause 7.4. The rules for full constant declarations are given in this subclause. 6

Any declaration that includes a `defining_identifier_list` with more than one `defining_identifier` is equivalent to a series of declarations each containing one `defining_identifier` from the list, with the rest of the text of the declaration copied for each declaration in the series, in the same order as the list. The remainder of this International Standard relies on this equivalence; explanations are given for declarations with a single `defining_identifier`. 7

The `subtype_indication`, `access_definition`, or full type definition of an `object_declaration` defines the nominal subtype of the object. The `object_declaration` declares an object of the type of the nominal subtype. 8/2

A component of an object is said to *require late initialization* if it has an access discriminant value constrained by a per-object expression, or if it has an initialization expression that includes a name denoting the current instance of the type or denoting an access discriminant. 8.1/2

Dynamic Semantics

If a composite object declared by an `object_declaration` has an unconstrained nominal subtype, then if this subtype is indefinite or the object is constant the actual subtype of this object is constrained. The 9/2

constraint is determined by the bounds or discriminants (if any) of its initial value; the object is said to be *constrained by its initial value*. When not constrained by its initial value, the actual and nominal subtypes of the object are the same. If its actual subtype is constrained, the object is called a *constrained object*.

10 For an `object_declaration` without an initialization expression, any initial values for the object or its subcomponents are determined by the *implicit initial values* defined for its nominal subtype, as follows:

- 11 • The implicit initial value for an access subtype is the null value of the access type.
- 12 • The implicit initial (and only) value for each discriminant of a constrained discriminated subtype is defined by the subtype.
- 13 • For a (definite) composite subtype, the implicit initial value of each component with a `default_expression` is obtained by evaluation of this expression and conversion to the component's nominal subtype (which might raise `Constraint_Error` — see 4.6, “Type Conversions”), unless the component is a discriminant of a constrained subtype (the previous case), or is in an excluded variant (see 3.8.1). For each component that does not have a `default_expression`, any implicit initial values are those determined by the component's nominal subtype.
- 14 • For a protected or task subtype, there is an implicit component (an entry queue) corresponding to each entry, with its implicit initial value being an empty queue.

15 The elaboration of an `object_declaration` proceeds in the following sequence of steps:

- 16/2 1. The `subtype_indication`, `access_definition`, `array_type_definition`, `single_task_declaration`, or `single_protected_declaration` is first elaborated. This creates the nominal subtype (and the anonymous type in the last four cases).
- 17 2. If the `object_declaration` includes an initialization expression, the (explicit) initial value is obtained by evaluating the expression and converting it to the nominal subtype (which might raise `Constraint_Error` — see 4.6).
- 18/2 3. The object is created, and, if there is not an initialization expression, the object is *initialized by default*. When an object is initialized by default, any per-object constraints (see 3.8) are elaborated and any implicit initial values for the object or for its subcomponents are obtained as determined by the nominal subtype. Any initial values (whether explicit or implicit) are assigned to the object or to the corresponding subcomponents. As described in 5.2 and 7.6, `Initialize` and `Adjust` procedures can be called.

19/2 *This paragraph was deleted.*

20/2 For the third step above, evaluations and assignments are performed in an arbitrary order subject to the following restrictions:

- 20.1/2 • Assignment to any part of the object is preceded by the evaluation of the value that is to be assigned.
- 20.2/2 • The evaluation of a `default_expression` that includes the name of a discriminant is preceded by the assignment to that discriminant.
- 20.3/2 • The evaluation of the `default_expression` for any component that depends on a discriminant is preceded by the assignment to that discriminant.
- 20.4/3 • The assignments to any components, including implicit components, not requiring late initialization precede the initial value evaluations for any components requiring late initialization; if two components both require late initialization, then assignments to parts of the component occurring earlier in the order of the component declarations precede the initial value evaluations of the component occurring later.

There is no implicit initial value defined for a scalar subtype. In the absence of an explicit initialization, a newly created scalar object might have a value that does not belong to its subtype (see 13.9.1 and H.1).

NOTES

8 Implicit initial values are not defined for an indefinite subtype, because if an object's nominal subtype is indefinite, an explicit initial value is required.

9 As indicated above, a stand-alone object is an object declared by an `object_declaration`. Similar definitions apply to “stand-alone constant” and “stand-alone variable.” A subcomponent of an object is not a stand-alone object, nor is an object that is created by an allocator. An object declared by a `loop_parameter_specification`, `parameter_specification`, `entry_index_specification`, `choice_parameter_specification`, `extended_return_statement`, or a `formal_object_declaration` of mode **in out** is not considered a stand-alone object.

10 or a `formal_object_declaration` is not called a stand-alone object.

11 The type of a stand-alone object cannot be abstract (see 3.9.3).

Examples

Example of a multiple object declaration:

-- the multiple object declaration

John, Paul : **not null** Person_Name := **new** Person(Sex => M); -- see 3.10.1

-- is equivalent to the two single object declarations in the order given

John : **not null** Person_Name := **new** Person(Sex => M);

Paul : **not null** Person_Name := **new** Person(Sex => M);

Examples of variable declarations:

```
Count, Sum : Integer;
Size       : Integer range 0 .. 10_000 := 0;
Sorted     : Boolean := False;
Color_Table : array(1 .. Max) of Color;
Option     : Bit_Vector(1 .. 10) := (others => True);
Hello      : aliased String := "Hi, world.";
θ, φ       : Float range -π .. +π;
```

Examples of constant declarations:

```
Limit      : constant Integer := 10_000;
Low_Limit  : constant Integer := Limit/10;
Tolerance  : constant Real := Dispersion(1.15);
Hello_Msg  : constant access String := Hello'Access; -- see 3.10.2
```

3.3.2 Number Declarations

A `number_declaration` declares a named number.

Syntax

```
number_declaration ::=
    defining_identifier_list : constant := static_expression;
```

Name Resolution Rules

The `static_expression` given for a `number_declaration` is expected to be of any numeric type.

Legality Rules

The `static_expression` given for a number declaration shall be a static expression, as defined by clause 4.9.

Static Semantics

- 5 The named number denotes a value of type *universal_integer* if the type of the *static_expression* is an integer type. The named number denotes a value of type *universal_real* if the type of the *static_expression* is a real type.
- 6 The value denoted by the named number is the value of the *static_expression*, converted to the corresponding universal type.

Dynamic Semantics

- 7 The elaboration of a *number_declaration* has no effect.

Examples

8 *Examples of number declarations:*

```

9      Two_Pi      : constant := 2.0*Ada.Numerics.Pi;    -- a real number (see A.5)
10/2    Max        : constant := 500;                  -- an integer number
      Max_Line_Size : constant := Max/6;                -- the integer 83
      Power_16     : constant := 2**16;                 -- the integer 65_536
      One, Un, Eins : constant := 1;                    -- three different names for 1
```

3.4 Derived Types and Classes

- 1/2 A *derived_type_definition* defines a *derived type* (and its first subtype) whose characteristics are *derived* from those of a parent type, and possibly from progenitor types.
- 1.1/2 A *class of types* is a set of types that is closed under derivation; that is, if the parent or a progenitor type of a derived type belongs to a class, then so does the derived type. By saying that a particular group of types forms a class, we are saying that all derivatives of a type in the set inherit the characteristics that define that set. The more general term *category of types* is used for a set of types whose defining characteristics are not necessarily inherited by derivatives; for example, limited, abstract, and interface are all categories of types, but not classes of types.

Syntax

- 2/2 **derived_type_definition** ::=
 [**abstract**] [**limited**] **new** *parent_subtype_indication* [[**and** *interface_list*] *record_extension_part*]

Legality Rules

- 3/2 The *parent_subtype_indication* defines the *parent subtype*; its type is the *parent type*. The *interface_list* defines the progenitor types (see 3.9.4). A derived type has one parent type and zero or more progenitor types.
- 4 A type shall be completely defined (see 3.11.1) prior to being specified as the parent type in a *derived_type_definition* — the *full_type_declarations* for the parent type and any of its subcomponents have to precede the *derived_type_definition*.
- 5/2 If there is a *record_extension_part*, the derived type is called a *record extension* of the parent type. A *record_extension_part* shall be provided if and only if the parent type is a tagged type. An *interface_list* shall be provided only if the parent type is a tagged type.
- 5.1/3 If the reserved word **limited** appears in a *derived_type_definition*, the parent type shall be a limited type. If the parent type is a tagged formal type, then in addition to the places where Legality Rules normally apply (see 12.3), this rule applies also in the private part of an instance of a generic unit.

Static Semantics

The first subtype of the derived type is unconstrained if a `known_discriminant_part` is provided in the declaration of the derived type, or if the parent subtype is unconstrained. Otherwise, the constraint of the first subtype *corresponds* to that of the parent subtype in the following sense: it is the same as that of the parent subtype except that for a range constraint (implicit or explicit), the value of each bound of its range is replaced by the corresponding value of the derived type. 6

The first subtype of the derived type excludes null (see 3.10) if and only if the parent subtype excludes null. 6.1/2

The characteristics of the derived type are defined as follows: 7

- If the parent type or a progenitor type belongs to a class of types, then the derived type also belongs to that class. The following sets of types, as well as any higher-level sets composed from them, are classes in this sense, and hence the characteristics defining these classes are inherited by derived types from their parent or progenitor types: signed integer, modular integer, ordinary fixed, decimal fixed, floating point, enumeration, boolean, character, access-to-constant, general access-to-variable, pool-specific access-to-variable, access-to-subprogram, array, string, non-array composite, nonlimited, untagged record, tagged, task, protected, and synchronized tagged. 8/2
- If the parent type is an elementary type or an array type, then the set of possible values of the derived type is a copy of the set of possible values of the parent type. For a scalar type, the base range of the derived type is the same as that of the parent type. 9
- If the parent type is a composite type other than an array type, then the components, protected subprograms, and entries that are declared for the derived type are as follows: 10
 - The discriminants specified by a new `known_discriminant_part`, if there is one; otherwise, each discriminant of the parent type (implicitly declared in the same order with the same specifications) — in the latter case, the discriminants are said to be *inherited*, or if unknown in the parent, are also unknown in the derived type; 11
 - Each nondiscriminant component, entry, and protected subprogram of the parent type, implicitly declared in the same order with the same declarations; these components, entries, and protected subprograms are said to be *inherited*; 12
 - Each component declared in a `record_extension_part`, if any. 13

Declarations of components, protected subprograms, and entries, whether implicit or explicit, occur immediately within the declarative region of the type, in the order indicated above, following the parent `subtype_indication`. 14

- *This paragraph was deleted.* 15/2
- For each predefined operator of the parent type, there is a corresponding predefined operator of the derived type. 16
- For each user-defined primitive subprogram (other than a user-defined equality operator — see below) of the parent type or of a progenitor type that already exists at the place of the `derived_type_definition`, there exists a corresponding *inherited* primitive subprogram of the derived type with the same defining name. Primitive user-defined equality operators of the parent type and any progenitor types are also inherited by the derived type, except when the derived type is a nonlimited record extension, and the inherited operator would have a profile that is type conformant with the profile of the corresponding predefined equality operator; in this case, the user-defined equality operator is not inherited, but is rather incorporated into the implementation of the predefined equality operator of the record extension (see 4.5.2). 17/2

- 18/3 The profile of an inherited subprogram (including an inherited enumeration literal) is obtained from the profile of the corresponding (user-defined) primitive subprogram of the parent or progenitor type, after systematic replacement of each subtype of its profile (see 6.1), other than those found in the designated profile of an `access_definition`, that is of the parent or progenitor type with a *corresponding subtype* of the derived type. For a given subtype of the parent or progenitor type, the corresponding subtype of the derived type is defined as follows:
- 19 • If the declaration of the derived type has neither a `known_discriminant_part` nor a `record_extension_part`, then the corresponding subtype has a constraint that corresponds (as defined above for the first subtype of the derived type) to that of the given subtype.
 - 20 • If the derived type is a record extension, then the corresponding subtype is the first subtype of the derived type.
 - 21 • If the derived type has a new `known_discriminant_part` but is not a record extension, then the corresponding subtype is constrained to those values that when converted to the parent type belong to the given subtype (see 4.6).
- 22/2 The same formal parameters have `default_expressions` in the profile of the inherited subprogram. Any type mismatch due to the systematic replacement of the parent or progenitor type by the derived type is handled as part of the normal type conversion associated with parameter passing — see 6.4.1.
- 23/2 If a primitive subprogram of the parent or progenitor type is visible at the place of the `derived_type_definition`, then the corresponding inherited subprogram is implicitly declared immediately after the `derived_type_definition`. Otherwise, the inherited subprogram is implicitly declared later or not at all, as explained in 7.3.1.
- 24 A derived type can also be defined by a `private_extension_declaration` (see 7.3) or a `formal_derived_type_definition` (see 12.5.1). Such a derived type is a partial view of the corresponding full or actual type.
- 25 All numeric types are derived types, in that they are implicitly derived from a corresponding root numeric type (see 3.5.4 and 3.5.6).

Dynamic Semantics

- 26 The elaboration of a `derived_type_definition` creates the derived type and its first subtype, and consists of the elaboration of the `subtype_indication` and the `record_extension_part`, if any. If the `subtype_indication` depends on a discriminant, then only those expressions that do not depend on a discriminant are evaluated.
- 27/2 For the execution of a call on an inherited subprogram, a call on the corresponding primitive subprogram of the parent or progenitor type is performed; the normal conversion of each actual parameter to the subtype of the corresponding formal parameter (see 6.4.1) performs any necessary type conversion as well. If the result type of the inherited subprogram is the derived type, the result of calling the subprogram of the parent or progenitor is converted to the derived type, or in the case of a null extension, extended to the derived type using the equivalent of an `extension_aggregate` with the original result as the `ancestor_part` and **null record** as the `record_component_association_list`.

NOTES

- 28 12 Classes are closed under derivation — any class that contains a type also contains its derivatives. Operations available for a given class of types are available for the derived types in that class.
- 29 13 Evaluating an inherited enumeration literal is equivalent to evaluating the corresponding enumeration literal of the parent type, and then converting the result to the derived type. This follows from their equivalence to parameterless functions.

- 14 A generic subprogram is not a subprogram, and hence cannot be a primitive subprogram and cannot be inherited by a derived type. On the other hand, an instance of a generic subprogram can be a primitive subprogram, and hence can be inherited. 30
- 15 If the parent type is an access type, then the parent and the derived type share the same storage pool; there is a **null** access value for the derived type and it is the implicit initial value for the type. See 3.10. 31
- 16 If the parent type is a boolean type, the predefined relational operators of the derived type deliver a result of the predefined type Boolean (see 4.5.2). If the parent type is an integer type, the right operand of the predefined exponentiation operator is of the predefined type Integer (see 4.5.6). 32
- 17 Any discriminants of the parent type are either all inherited, or completely replaced with a new set of discriminants. 33
- 18 For an inherited subprogram, the subtype of a formal parameter of the derived type need not have any value in common with the first subtype of the derived type. 34
- 19 If the reserved word **abstract** is given in the declaration of a type, the type is abstract (see 3.9.3). 35
- 20 An interface type that has a progenitor type “is derived from” that type. A `derived_type_definition`, however, never defines an interface type. 35.1/2
- 21 It is illegal for the parent type of a `derived_type_definition` to be a synchronized tagged type. 35.2/2

Examples

Examples of derived type declarations:

```

type Local_Coordinate is new Coordinate;    -- two different types
type Midweek is new Day range Tue .. Thu;   -- see 3.5.1
type Counter is new Positive;               -- same range as Positive
type Special_Key is new Key_Manager.Key;    -- see 7.3.1
-- the inherited subprograms have the following specifications:
--   procedure Get_Key(K : out Special_Key);
--   function "<"(X,Y : Special_Key) return Boolean;
```

3.4.1 Derivation Classes

In addition to the various language-defined classes of types, types can be grouped into *derivation classes*.

Static Semantics

A derived type is *derived from* its parent type *directly*; it is derived *indirectly* from any type from which its parent type is derived. A derived type, interface type, type extension, task type, protected type, or formal derived type is also derived from every ancestor of each of its progenitor types, if any. The derivation class of types for a type *T* (also called the class *rooted at T*) is the set consisting of *T* (the *root type* of the class) and all types derived from *T* (directly or indirectly) plus any associated universal or class-wide types (defined below).

Every type is either a *specific* type, a *class-wide* type, or a *universal* type. A specific type is one defined by a `type_declaration`, a `formal_type_declaration`, or a full type definition embedded in another construct. Class-wide and universal types are implicitly defined, to act as representatives for an entire class of types, as follows:

Class-wide types

Class-wide types are defined for (and belong to) each derivation class rooted at a tagged type (see 3.9). Given a subtype *S* of a tagged type *T*, *S*'Class is the `subtype_mark` for a corresponding subtype of the tagged class-wide type *T*'Class. Such types are called “class-wide” because when a formal parameter is defined to be of a class-wide type *T*'Class, an actual parameter of any type in the derivation class rooted at *T* is acceptable (see 8.6).

The set of values for a class-wide type *T*'Class is the discriminated union of the set of values of each specific type in the derivation class rooted at *T* (the tag acts as the implicit

discriminant — see 3.9). Class-wide types have no primitive subprograms of their own. However, as explained in 3.9.2, operands of a class-wide type *T*'Class can be used as part of a dispatching call on a primitive subprogram of the type *T*. The only components (including discriminants) of *T*'Class that are visible are those of *T*. If *S* is a first subtype, then *S*'Class is a first subtype.

6/2 Universal types

Universal types are defined for (and belong to) the integer, real, fixed point, and access classes, and are referred to in this standard as respectively, *universal_integer*, *universal_real*, *universal_fixed*, and *universal_access*. These are analogous to class-wide types for these language-defined elementary classes. As with class-wide types, if a formal parameter is of a universal type, then an actual parameter of any type in the corresponding class is acceptable. In addition, a value of a universal type (including an integer or real numeric_literal, or the literal **null**) is “universal” in that it is acceptable where some particular type in the class is expected (see 8.6).

- 7 The set of values of a universal type is the undiscriminated union of the set of values possible for any definable type in the associated class. Like class-wide types, universal types have no primitive subprograms of their own. However, their “universality” allows them to be used as operands with the primitive subprograms of any type in the corresponding class.
- 8 The integer and real numeric classes each have a specific root type in addition to their universal type, named respectively *root_integer* and *root_real*.
- 9 A class-wide or universal type is said to *cover* all of the types in its class. A specific type covers only itself.
- 10/2 A specific type *T2* is defined to be a *descendant* of a type *T1* if *T2* is the same as *T1*, or if *T2* is derived (directly or indirectly) from *T1*. A class-wide type *T2*'Class is defined to be a descendant of type *T1* if *T2* is a descendant of *T1*. Similarly, the numeric universal types are defined to be descendants of the root types of their classes. If a type *T2* is a descendant of a type *T1*, then *T1* is called an *ancestor* of *T2*. An *ultimate ancestor* of a type is an ancestor of that type that is not itself a descendant of any other type. Every untagged type has a unique ultimate ancestor.
- 11 An inherited component (including an inherited discriminant) of a derived type is inherited *from* a given ancestor of the type if the corresponding component was inherited by each derived type in the chain of derivations going back to the given ancestor.

NOTES

- 12 22 Because operands of a universal type are acceptable to the predefined operators of any type in their class, ambiguity can result. For *universal_integer* and *universal_real*, this potential ambiguity is resolved by giving a preference (see 8.6) to the predefined operators of the corresponding root types (*root_integer* and *root_real*, respectively). Hence, in an apparently ambiguous expression like
- 13 $1 + 4 < 7$
- 14 where each of the literals is of type *universal_integer*, the predefined operators of *root_integer* will be preferred over those of other specific integer types, thereby resolving the ambiguity.

3.5 Scalar Types

- 1 *Scalar* types comprise enumeration types, integer types, and real types. Enumeration types and integer types are called *discrete* types; each value of a discrete type has a *position number* which is an integer value. Integer types and real types are called *numeric* types. All scalar types are ordered, that is, all relational operators are predefined for their values.

Syntax

range_constraint ::= **range** range 2

range ::= range_attribute_reference 3

| simple_expression .. simple_expression

A *range* has a *lower bound* and an *upper bound* and specifies a subset of the values of some scalar type (the *type of the range*). A range with lower bound L and upper bound R is described by “L .. R”. If R is less than L, then the range is a *null range*, and specifies an empty set of values. Otherwise, the range specifies the values of the type from the lower bound to the upper bound, inclusive. A value *belongs* to a range if it is of the type of the range, and is in the subset of values specified by the range. A value *satisfies* a range constraint if it belongs to the associated range. One range is *included* in another if all values that belong to the first range also belong to the second.

Name Resolution Rules

For a subtype_indication containing a range_constraint, either directly or as part of some other scalar_constraint, the type of the range shall resolve to that of the type determined by the subtype_mark of the subtype_indication. For a range of a given type, the simple_expressions of the range (likewise, the simple_expressions of the equivalent range for a range_attribute_reference) are expected to be of the type of the range. 5

Static Semantics

The *base range* of a scalar type is the range of finite values of the type that can be represented in every unconstrained object of the type; it is also the range supported at a minimum for intermediate values during the evaluation of expressions involving predefined operators of the type. 6

A constrained scalar subtype is one to which a range constraint applies. The *range* of a constrained scalar subtype is the range associated with the range constraint of the subtype. The *range* of an unconstrained scalar subtype is the base range of its type. 7

Dynamic Semantics

A range is *compatible* with a scalar subtype if and only if it is either a null range or each bound of the range belongs to the range of the subtype. A range_constraint is *compatible* with a scalar subtype if and only if its range is compatible with the subtype. 8

The elaboration of a range_constraint consists of the evaluation of the range. The evaluation of a range determines a lower bound and an upper bound. If simple_expressions are given to specify bounds, the evaluation of the range evaluates these simple_expressions in an arbitrary order, and converts them to the type of the range. If a range_attribute_reference is given, the evaluation of the range consists of the evaluation of the range_attribute_reference. 9

Attributes

For every scalar subtype S, the following attributes are defined: 10

S'First S'First denotes the lower bound of the range of S. The value of this attribute is of the type of S. 12

S'Last S'Last denotes the upper bound of the range of S. The value of this attribute is of the type of S. 13

S'Range S'Range is equivalent to the range S'First .. S'Last. 14

S'Base S'Base denotes an unconstrained subtype of the type of S. This unconstrained subtype is called the *base subtype* of the type. 15

16	S'Min	S'Min denotes a function with the following specification:
17		function S'Min(<i>Left</i> , <i>Right</i> : S'Base) return S'Base
18		The function returns the lesser of the values of the two parameters.
19	S'Max	S'Max denotes a function with the following specification:
20		function S'Max(<i>Left</i> , <i>Right</i> : S'Base) return S'Base
21		The function returns the greater of the values of the two parameters.
22	S'Succ	S'Succ denotes a function with the following specification:
23		function S'Succ(<i>Arg</i> : S'Base) return S'Base
24		For an enumeration type, the function returns the value whose position number is one more than that of the value of <i>Arg</i> ; Constraint_Error is raised if there is no such value of the type. For an integer type, the function returns the result of adding one to the value of <i>Arg</i> . For a fixed point type, the function returns the result of adding <i>small</i> to the value of <i>Arg</i> . For a floating point type, the function returns the machine number (as defined in 3.5.7) immediately above the value of <i>Arg</i> ; Constraint_Error is raised if there is no such machine number.
25	S'Pred	S'Pred denotes a function with the following specification:
26		function S'Pred(<i>Arg</i> : S'Base) return S'Base
27		For an enumeration type, the function returns the value whose position number is one less than that of the value of <i>Arg</i> ; Constraint_Error is raised if there is no such value of the type. For an integer type, the function returns the result of subtracting one from the value of <i>Arg</i> . For a fixed point type, the function returns the result of subtracting <i>small</i> from the value of <i>Arg</i> . For a floating point type, the function returns the machine number (as defined in 3.5.7) immediately below the value of <i>Arg</i> ; Constraint_Error is raised if there is no such machine number.
27.1/2	S'Wide_Wide_Image	S'Wide_Wide_Image denotes a function with the following specification:
27.2/2		function S'Wide_Wide_Image(<i>Arg</i> : S'Base) return Wide_Wide_String
27.3/2		The function returns an <i>image</i> of the value of <i>Arg</i> , that is, a sequence of characters representing the value in display form. The lower bound of the result is one.
27.4/2		The image of an integer value is the corresponding decimal literal, without underlines, leading zeros, exponent, or trailing spaces, but with a single leading character that is either a minus sign or a space.
27.5/2		The image of an enumeration value is either the corresponding identifier in upper case or the corresponding character literal (including the two apostrophes); neither leading nor trailing spaces are included. For a <i>nongraphic character</i> (a value of a character type that has no enumeration literal associated with it), the result is a corresponding language-defined name in upper case (for example, the image of the nongraphic character identified as <i>nul</i> is “NUL” — the quotes are not part of the image).
27.6/2		The image of a floating point value is a decimal real literal best approximating the value (rounded away from zero if halfway between) with a single leading character that is either a minus sign or a space, a single digit (that is nonzero unless the value is zero), a decimal point, S'Digits-1 (see 3.5.8) digits after the decimal point (but one if S'Digits is one), an upper case E, the sign of the exponent (either + or -), and two or more digits (with leading

zeros if necessary) representing the exponent. If `S'Signed_Zeros` is `True`, then the leading character is a minus sign for a negatively signed zero.

The image of a fixed point value is a decimal real literal best approximating the value (rounded away from zero if halfway between) with a single leading character that is either a minus sign or a space, one or more digits before the decimal point (with no redundant leading zeros), a decimal point, and `S'Aft` (see 3.5.10) digits after the decimal point. 27.7/2

`S'Wide_Image` `S'Wide_Image` denotes a function with the following specification: 28

```
function S'Wide_Image (Arg : S'Base)
return Wide_String
```

 29

The function returns an image of the value of `Arg` as a `Wide_String`. The lower bound of the result is one. The image has the same sequence of character as defined for `S'Wide_Wide_Image` if all the graphic characters are defined in `Wide_Character`; otherwise the sequence of characters is implementation defined (but no shorter than that of `S'Wide_Wide_Image` for the same value of `Arg`). 30/2

Paragraphs 31 through 34 were moved to `Wide_Wide_Image`.

`S'Image` `S'Image` denotes a function with the following specification: 35

```
function S'Image (Arg : S'Base)
return String
```

 36

The function returns an image of the value of `Arg` as a `String`. The lower bound of the result is one. The image has the same sequence of graphic characters as that defined for `S'Wide_Wide_Image` if all the graphic characters are defined in `Character`; otherwise the sequence of characters is implementation defined (but no shorter than that of `S'Wide_Wide_Image` for the same value of `Arg`). 37/2

`S'Wide_Wide_Width` 37.1/2

`S'Wide_Wide_Width` denotes the maximum length of a `Wide_Wide_String` returned by `S'Wide_Wide_Image` over all values of the subtype `S`. It denotes zero for a subtype that has a null range. Its type is *universal_integer*.

`S'Wide_Width` 38

`S'Wide_Width` denotes the maximum length of a `Wide_String` returned by `S'Wide_Image` over all values of the subtype `S`. It denotes zero for a subtype that has a null range. Its type is *universal_integer*.

`S'Width` `S'Width` denotes the maximum length of a `String` returned by `S'Image` over all values of the subtype `S`. It denotes zero for a subtype that has a null range. Its type is *universal_integer*. 39

`S'Wide_Wide_Value` 39.1/2

`S'Wide_Wide_Value` denotes a function with the following specification:

```
function S'Wide_Wide_Value (Arg : Wide_Wide_String)
return S'Base
```

 39.2/2

This function returns a value given an image of the value as a `Wide_Wide_String`, ignoring any leading or trailing spaces. 39.3/2

For the evaluation of a call on `S'Wide_Wide_Value` for an enumeration subtype `S`, if the sequence of characters of the parameter (ignoring leading and trailing spaces) has the syntax of an enumeration literal and if it corresponds to a literal of the type of `S` (or corresponds to the result of `S'Wide_Wide_Image` for a nongraphic character of the type), the result is the corresponding enumeration value; otherwise `Constraint_Error` is raised. 39.4/2

For the evaluation of a call on `S'Wide_Wide_Value` for an integer subtype `S`, if the sequence of characters of the parameter (ignoring leading and trailing spaces) has the syntax of an integer literal, with an optional leading sign character (plus or minus for a signed type; only plus for a modular type), and the corresponding numeric value belongs to 39.5/2

the base range of the type of *S*, then that value is the result; otherwise *Constraint_Error* is raised.

39.6/2 For the evaluation of a call on *S'Wide_Wide_Value* for a real subtype *S*, if the sequence of characters of the parameter (ignoring leading and trailing spaces) has the syntax of one of the following:

- 39.7/2 • *numeric_literal*
- 39.8/2 • *numeral*[*exponent*]
- 39.9/2 • *.numeral*[*exponent*]
- 39.10/2 • *base#based_numeral*#[*exponent*]
- 39.11/2 • *base#.based_numeral*#[*exponent*]

39.12/2 with an optional leading sign character (plus or minus), and if the corresponding numeric value belongs to the base range of the type of *S*, then that value is the result; otherwise *Constraint_Error* is raised. The sign of a zero value is preserved (positive if none has been specified) if *S'Signed_Zeros* is *True*.

40 *S'Wide_Value*

S'Wide_Value denotes a function with the following specification:

```
41      function S'Wide_Value (Arg : Wide_String)
      return S'Base
```

42 This function returns a value given an image of the value as a *Wide_String*, ignoring any leading or trailing spaces.

43/2 For the evaluation of a call on *S'Wide_Value* for an enumeration subtype *S*, if the sequence of characters of the parameter (ignoring leading and trailing spaces) has the syntax of an enumeration literal and if it corresponds to a literal of the type of *S* (or corresponds to the result of *S'Wide_Image* for a value of the type), the result is the corresponding enumeration value; otherwise *Constraint_Error* is raised. For a numeric subtype *S*, the evaluation of a call on *S'Wide_Value* with *Arg* of type *Wide_String* is equivalent to a call on *S'Wide_Wide_Value* for a corresponding *Arg* of type *Wide_Wide_String*.

Paragraphs 44 through 51 were moved to Wide_Wide_Value.

52 *S'Value*

S'Value denotes a function with the following specification:

```
53      function S'Value (Arg : String)
      return S'Base
```

54 This function returns a value given an image of the value as a *String*, ignoring any leading or trailing spaces.

55/2 For the evaluation of a call on *S'Value* for an enumeration subtype *S*, if the sequence of characters of the parameter (ignoring leading and trailing spaces) has the syntax of an enumeration literal and if it corresponds to a literal of the type of *S* (or corresponds to the result of *S'Image* for a value of the type), the result is the corresponding enumeration value; otherwise *Constraint_Error* is raised. For a numeric subtype *S*, the evaluation of a call on *S'Value* with *Arg* of type *String* is equivalent to a call on *S'Wide_Wide_Value* for a corresponding *Arg* of type *Wide_Wide_String*.

Implementation Permissions

56/2 An implementation may extend the *Wide_Wide_Value*, *Wide_Value*, *Value*, *Wide_Wide_Image*, *Wide_Image*, and *Image* attributes of a floating point type to support special values such as infinities and NaNs.

NOTES

- 23 The evaluation of S'First or S'Last never raises an exception. If a scalar subtype S has a nonnull range, S'First and S'Last belong to this range. These values can, for example, always be assigned to a variable of subtype S. 57
- 24 For a subtype of a scalar type, the result delivered by the attributes Succ, Pred, and Value might not belong to the subtype; similarly, the actual parameters of the attributes Succ, Pred, and Image need not belong to the subtype. 58
- 25 For any value V (including any nongraphic character) of an enumeration subtype S, S'Value(S'Image(V)) equals V, as do S'Wide_Value(S'Wide_Image(V)) and S'Wide_Wide_Value(S'Wide_Wide_Image(V)). None of these expressions ever raise Constraint_Error. 59

*Examples**Examples of ranges:*

```
-10 .. 10
X .. X + 1
0.0 .. 2.0*Pi
Red .. Green      -- see 3.5.1
1 .. 0            -- a null range
Table'Range       -- a range attribute reference (see 3.6)
```

Examples of range constraints:

```
range -999.0 .. +999.0
range S'First+1 .. S'Last-1
```

3.5.1 Enumeration Types

An enumeration_type_definition defines an enumeration type.

Syntax

```
enumeration_type_definition ::=
  (enumeration_literal_specification {, enumeration_literal_specification})
enumeration_literal_specification ::= defining_identifier | defining_character_literal
defining_character_literal ::= character_literal
```

Legality Rules

The defining_identifiers and defining_character_literals listed in an enumeration_type_definition shall be distinct.

Static Semantics

Each enumeration_literal_specification is the explicit declaration of the corresponding *enumeration literal*: it declares a parameterless function, whose defining name is the defining_identifier or defining_character_literal, and whose result subtype is the base subtype of the enumeration type.

Each enumeration literal corresponds to a distinct value of the enumeration type, and to a distinct position number. The position number of the value of the first listed enumeration literal is zero; the position number of the value of each subsequent enumeration literal is one more than that of its predecessor in the list.

The predefined order relations between values of the enumeration type follow the order of corresponding position numbers.

If the same defining_identifier or defining_character_literal is specified in more than one enumeration_type_definition, the corresponding enumeration literals are said to be *overloaded*. At any place where an overloaded enumeration literal occurs in the text of a program, the type of the enumeration literal has to be determinable from the context (see 8.6).

Dynamic Semantics

- 10 The elaboration of an `enumeration_type_definition` creates the enumeration type and its first subtype, which is constrained to the base range of the type.
- 11 When called, the parameterless function associated with an enumeration literal returns the corresponding value of the enumeration type.

NOTES

- 12 26 If an enumeration literal occurs in a context that does not otherwise suffice to determine the type of the literal, then qualification by the name of the enumeration type is one way to resolve the ambiguity (see 4.7).

Examples

- 13 *Examples of enumeration types and subtypes:*

```

14     type Day      is (Mon, Tue, Wed, Thu, Fri, Sat, Sun);
     type Suit     is (Clubs, Diamonds, Hearts, Spades);
     type Gender   is (M, F);
     type Level    is (Low, Medium, Urgent);
     type Color    is (White, Red, Yellow, Green, Blue, Brown, Black);
     type Light    is (Red, Amber, Green); -- Red and Green are overloaded
15     type Hexa    is ('A', 'B', 'C', 'D', 'E', 'F');
     type Mixed    is ('A', 'B', '*', B, None, '?', '%');
16     subtype Weekday is Day      range Mon .. Fri;
     subtype Major   is Suit     range Hearts .. Spades;
     subtype Rainbow is Color    range Red .. Blue; -- the Color Red, not the Light

```

3.5.2 Character Types

Static Semantics

- 1 An enumeration type is said to be a *character type* if at least one of its enumeration literals is a `character_literal`.
- 2/3 The predefined type `Character` is a character type whose values correspond to the 256 code positions of Row 00 (also known as Latin-1) of the ISO/IEC 10646:2003 Basic Multilingual Plane (BMP). Each of the graphic characters of Row 00 of the BMP has a corresponding `character_literal` in `Character`. Each of the nongraphic positions of Row 00 has a corresponding language-defined name, which is not usable as an enumeration literal, but which is usable with the attributes `Image`, `Wide_Image`, `Wide_Wide_Image`, `Value`, `Wide_Value`, and `Wide_Wide_Value`; these names are given in the definition of type `Character` in A.1, “The Package Standard”, but are set in *italics*.
- 3/2 The predefined type `Wide_Character` is a character type whose values correspond to the 65536 code positions of the ISO/IEC 10646:2003 Basic Multilingual Plane (BMP). Each of the graphic characters of the BMP has a corresponding `character_literal` in `Wide_Character`. The first 256 values of `Wide_Character` have the same `character_literal` or language-defined name as defined for `Character`. Each of the `graphic_characters` has a corresponding `character_literal`.
- 3.1/2 The predefined type `Wide_Wide_Character` is a character type whose values correspond to the 2147483648 code positions of the ISO/IEC 10646:2003 character set. Each of the `graphic_characters` has a corresponding `character_literal` in `Wide_Wide_Character`. The first 65536 values of `Wide_Wide_Character` have the same `character_literal` or language-defined name as defined for `Wide_Character`.
- 3.2/2 The characters whose code position is larger than 16#FF# and which are not `graphic_characters` have language-defined names which are formed by appending to the string “Hex_” the representation of their

code position in hexadecimal as eight extended digits. As with other language-defined names, these names are usable only with the attributes (Wide_)Wide_Image and (Wide_)Wide_Value; they are not usable as enumeration literals.

Implementation Permissions

This paragraph was deleted.

4/2

Implementation Advice

This paragraph was deleted.

5/2

NOTES

27 The language-defined library package Characters.Latin_1 (see A.3.3) includes the declaration of constants denoting control characters, lower case characters, and special characters of the predefined type Character.

6

28 A conventional character set such as *EBCDIC* can be declared as a character type; the internal codes of the characters can be specified by an `enumeration_representation_clause` as explained in clause 13.4.

7

Examples

Example of a character type:

8

```
type Roman_Digit is ('I', 'V', 'X', 'L', 'C', 'D', 'M');
```

9

3.5.3 Boolean Types

Static Semantics

There is a predefined enumeration type named Boolean, declared in the visible part of package Standard. It has the two enumeration literals False and True ordered with the relation False < True. Any descendant of the predefined type Boolean is called a *boolean* type.

1

3.5.4 Integer Types

An `integer_type_definition` defines an integer type; it defines either a *signed* integer type, or a *modular* integer type. The base range of a signed integer type includes at least the values of the specified range. A modular type is an integer type with all arithmetic modulo a specified positive *modulus*; such a type corresponds to an unsigned type with wrap-around semantics.

1

Syntax

```
integer_type_definition ::= signed_integer_type_definition | modular_type_definition
```

2

```
signed_integer_type_definition ::= range static_simple_expression .. static_simple_expression
```

3

```
modular_type_definition ::= mod static_expression
```

4

Name Resolution Rules

Each `static_simple_expression` in a `signed_integer_type_definition` is expected to be of any integer type; they need not be of the same type. The expression in a `modular_type_definition` is likewise expected to be of any integer type.

5

Legality Rules

The `static_expressions` of a `signed_integer_type_definition` shall be static, and their values shall be in the range System.Min_Int .. System.Max_Int.

6

- 7 The expression of a `modular_type_definition` shall be static, and its value (the *modulus*) shall be positive, and shall be no greater than `System.Max_Binary_Modulus` if a power of 2, or no greater than `System.Max_Nonbinary_Modulus` if not.

Static Semantics

- 8 The set of values for a signed integer type is the (infinite) set of mathematical integers, though only values of the base range of the type are fully supported for run-time operations. The set of values for a modular integer type are the values from 0 to one less than the modulus, inclusive.
- 9 A `signed_integer_type_definition` defines an integer type whose base range includes at least the values of the `simple_expressions` and is symmetric about zero, excepting possibly an extra negative value. A `signed_integer_type_definition` also defines a constrained first subtype of the type, with a range whose bounds are given by the values of the `simple_expressions`, converted to the type being defined.
- 10 A `modular_type_definition` defines a modular type whose base range is from zero to one less than the given modulus. A `modular_type_definition` also defines a constrained first subtype of the type with a range that is the same as the base range of the type.
- 11 There is a predefined signed integer subtype named `Integer`, declared in the visible part of package `Standard`. It is constrained to the base range of its type.
- 12 `Integer` has two predefined subtypes, declared in the visible part of package `Standard`:
- 13

```
subtype Natural is Integer range 0 .. Integer'Last;
subtype Positive is Integer range 1 .. Integer'Last;
```
- 14 A type defined by an `integer_type_definition` is implicitly derived from *root_integer*, an anonymous predefined (specific) integer type, whose base range is `System.Min_Int .. System.Max_Int`. However, the base range of the new type is not inherited from *root_integer*, but is instead determined by the range or modulus specified by the `integer_type_definition`. Integer literals are all of the type *universal_integer*, the universal type (see 3.4.1) for the class rooted at *root_integer*, allowing their use with the operations of any integer type.
- 15 The *position number* of an integer value is equal to the value.

- 16/2 For every modular subtype `S`, the following attributes are defined:
- 16.1/2 `S'Mod` `S'Mod` denotes a function with the following specification:
- 16.2/2

```
function S'Mod (Arg : universal_integer)
return S'Base
```
- 16.3/2 This function returns `Arg mod S'Modulus`, as a value of the type of `S`.
- 17 `S'Modulus` `S'Modulus` yields the modulus of the type of `S`, as a value of the type *universal_integer*.

Dynamic Semantics

- 18 The elaboration of an `integer_type_definition` creates the integer type and its first subtype.
- 19 For a modular type, if the result of the execution of a predefined operator (see 4.5) is outside the base range of the type, the result is reduced modulo the modulus of the type to a value that is within the base range of the type.
- 20 For a signed integer type, the exception `Constraint_Error` is raised by the execution of an operation that cannot deliver the correct result because it is outside the base range of the type. For any integer type, `Constraint_Error` is raised by the operators `"/`, `"rem`", and `"mod`" if the right operand is zero.

Implementation Requirements

- In an implementation, the range of Integer shall include the range $-2^{15}+1 .. +2^{15}-1$. 21
- If Long_Integer is predefined for an implementation, then its range shall include the range $-2^{31}+1 .. +2^{31}-1$. 22
- System.Max_Binary_Modulus shall be at least 2^{16} . 23

Implementation Permissions

- For the execution of a predefined operation of a signed integer type, the implementation need not raise Constraint_Error if the result is outside the base range of the type, so long as the correct result is produced. 24
- An implementation may provide additional predefined signed integer types, declared in the visible part of Standard, whose first subtypes have names of the form Short_Integer, Long_Integer, Short_Short_Integer, Long_Long_Integer, etc. Different predefined integer types are allowed to have the same base range. However, the range of Integer should be no wider than that of Long_Integer. Similarly, the range of Short_Integer (if provided) should be no wider than Integer. Corresponding recommendations apply to any other predefined integer types. There need not be a named integer type corresponding to each distinct base range supported by an implementation. The range of each first subtype should be the base range of its type. 25
- An implementation may provide *nonstandard integer types*, descendants of *root_integer* that are declared outside of the specification of package Standard, which need not have all the standard characteristics of a type defined by an *integer_type_definition*. For example, a nonstandard integer type might have an asymmetric base range or it might not be allowed as an array or loop index (a very long integer). Any type descended from a nonstandard integer type is also nonstandard. An implementation may place arbitrary restrictions on the use of such types; it is implementation defined whether operators that are predefined for “any integer type” are defined for a particular nonstandard integer type. In any case, such types are not permitted as *explicit_generic_actual_parameters* for formal scalar types — see 12.5.2. 26
- For a one's complement machine, the high bound of the base range of a modular type whose modulus is one less than a power of 2 may be equal to the modulus, rather than one less than the modulus. It is implementation defined for which powers of 2, if any, this permission is exercised. 27
- For a one's complement machine, implementations may support non-binary modulus values greater than System.Max_Nonbinary_Modulus. It is implementation defined which specific values greater than System.Max_Nonbinary_Modulus, if any, are supported. 27.1/1

Implementation Advice

- An implementation should support Long_Integer in addition to Integer if the target machine supports 32-bit (or longer) arithmetic. No other named integer subtypes are recommended for package Standard. Instead, appropriate named integer subtypes should be provided in the library package Interfaces (see B.2). 28
- An implementation for a two's complement machine should support modular types with a binary modulus up to System.Max_Int*2+2. An implementation should support a nonbinary modulus up to Integer'Last. 29

NOTES

- 29 Integer literals are of the anonymous predefined integer type *universal_integer*. Other integer types have no literals. However, the overload resolution rules (see 8.6, “The Context of Overload Resolution”) allow expressions of the type *universal_integer* whenever an integer type is expected. 30
- 30 The same arithmetic operators are predefined for all signed integer types defined by a *signed_integer_type_definition* (see 4.5, “Operators and Expression Evaluation”). For modular types, these same operators are predefined, plus bit-wise 31

logical operators (**and**, **or**, **xor**, and **not**). In addition, for the unsigned types declared in the language-defined package Interfaces (see B.2), functions are defined that provide bit-wise shifting and rotating.

31 Modular types match a `generic_formal_parameter_declaration` of the form "**type T is mod** Δ ;" signed integer types match "**type T is range** Δ ;" (see 12.5.2).

Examples

Examples of integer types and subtypes:

```

type Page_Num is range 1 .. 2_000;
type Line_Size is range 1 .. Max_Line_Size;

subtype Small_Int is Integer range -10 .. 10;
subtype Column_Ptr is Line_Size range 1 .. 10;
subtype Buffer_Size is Integer range 0 .. Max;

type Byte is mod 256; -- an unsigned byte
type Hash_Index is mod 97; -- modulus is prime

```

3.5.5 Operations of Discrete Types

Static Semantics

For every discrete subtype S, the following attributes are defined:

S'Pos S'Pos denotes a function with the following specification:

```

function S'Pos (Arg : S'Base)
return universal_integer

```

This function returns the position number of the value of *Arg*, as a value of type *universal_integer*.

S'Val S'Val denotes a function with the following specification:

```

function S'Val (Arg : universal_integer)
return S'Base

```

This function returns a value of the type of S whose position number equals the value of *Arg*. For the evaluation of a call on S'Val, if there is no value in the base range of its type with the given position number, Constraint_Error is raised.

Implementation Advice

For the evaluation of a call on S'Pos for an enumeration subtype, if the value of the operand does not correspond to the internal code for any enumeration literal of its type (perhaps due to an uninitialized variable), then the implementation should raise Program_Error. This is particularly important for enumeration types with noncontiguous internal codes specified by an `enumeration_representation_clause`.

NOTES

32 Indexing and loop iteration use values of discrete types.

33 The predefined operations of a discrete type include the assignment operation, qualification, the membership tests, and the relational operators; for a boolean type they include the short-circuit control forms and the logical operators; for an integer type they include type conversion to and from other numeric types, as well as the binary and unary adding operators – and +, the multiplying operators, the unary operator **abs**, and the exponentiation operator. The assignment operation is described in 5.2. The other predefined operations are described in Section 4.

34 As for all types, objects of a discrete type have Size and Address attributes (see 13.3).

35 For a subtype of a discrete type, the result delivered by the attribute Val might not belong to the subtype; similarly, the actual parameter of the attribute Pos need not belong to the subtype. The following relations are satisfied (in the absence of an exception) by these attributes:

```

S'Val (S'Pos (X)) = X
S'Pos (S'Val (N)) = N

```


*Examples**Examples of attributes of discrete subtypes:*

```
-- For the types and subtypes declared in subclause 3.5.1 the following hold:
-- Color'First   = White,      Color'Last   = Black
-- Rainbow'First = Red,       Rainbow'Last = Blue
-- Color'Succ(Blue) = Rainbow'Succ(Blue) = Brown
-- Color'Pos(Blue)  = Rainbow'Pos(Blue)  = 4
-- Color'Val(0)     = Rainbow'Val(0)     = White
```

3.5.6 Real Types

Real types provide approximations to the real numbers, with relative bounds on errors for floating point types, and with absolute bounds for fixed point types.

Syntax

```
real_type_definition ::=
  floating_point_definition | fixed_point_definition
```

Static Semantics

A type defined by a *real_type_definition* is implicitly derived from *root_real*, an anonymous predefined (specific) real type. Hence, all real types, whether floating point or fixed point, are in the derivation class rooted at *root_real*.

Real literals are all of the type *universal_real*, the universal type (see 3.4.1) for the class rooted at *root_real*, allowing their use with the operations of any real type. Certain multiplying operators have a result type of *universal_fixed* (see 4.5.5), the universal type for the class of fixed point types, allowing the result of the multiplication or division to be used where any specific fixed point type is expected.

Dynamic Semantics

The elaboration of a *real_type_definition* consists of the elaboration of the *floating_point_definition* or the *fixed_point_definition*.

Implementation Requirements

An implementation shall perform the run-time evaluation of a use of a predefined operator of *root_real* with an accuracy at least as great as that of any floating point type definable by a *floating_point_definition*.

Implementation Permissions

For the execution of a predefined operation of a real type, the implementation need not raise *Constraint_Error* if the result is outside the base range of the type, so long as the correct result is produced, or the *Machine_Overflows* attribute of the type is *False* (see G.2).

An implementation may provide *nonstandard real types*, descendants of *root_real* that are declared outside of the specification of package *Standard*, which need not have all the standard characteristics of a type defined by a *real_type_definition*. For example, a nonstandard real type might have an asymmetric or unsigned base range, or its predefined operations might wrap around or “saturate” rather than overflow (modular or saturating arithmetic), or it might not conform to the accuracy model (see G.2). Any type descended from a nonstandard real type is also nonstandard. An implementation may place arbitrary restrictions on the use of such types; it is implementation defined whether operators that are predefined for “any real type” are defined for a particular nonstandard real type. In any case, such types are not permitted as *explicit_generic_actual_parameters* for formal scalar types — see 12.5.2.

NOTES

- 9 36 As stated, real literals are of the anonymous predefined real type *universal_real*. Other real types have no literals. However, the overload resolution rules (see 8.6) allow expressions of the type *universal_real* whenever a real type is expected.

3.5.7 Floating Point Types

- 1 For floating point types, the error bound is specified as a relative precision by giving the required minimum number of significant decimal digits.

Syntax

- 2 floating_point_definition ::=
 digits static_expression [real_range_specification]
 3 real_range_specification ::=
 range static_simple_expression .. static_simple_expression

Name Resolution Rules

- 4 The *requested decimal precision*, which is the minimum number of significant decimal digits required for the floating point type, is specified by the value of the expression given after the reserved word **digits**. This expression is expected to be of any integer type.
- 5 Each simple_expression of a real_range_specification is expected to be of any real type; the types need not be the same.

Legality Rules

- 6 The requested decimal precision shall be specified by a static expression whose value is positive and no greater than System.Max_Base_Digits. Each simple_expression of a real_range_specification shall also be static. If the real_range_specification is omitted, the requested decimal precision shall be no greater than System.Max_Digits.
- 7 A floating_point_definition is illegal if the implementation does not support a floating point type that satisfies the requested decimal precision and range.

Static Semantics

- 8 The set of values for a floating point type is the (infinite) set of rational numbers. The *machine numbers* of a floating point type are the values of the type that can be represented exactly in every unconstrained variable of the type. The base range (see 3.5) of a floating point type is symmetric around zero, except that it can include some extra negative values in some implementations.
- 9 The *base decimal precision* of a floating point type is the number of decimal digits of precision representable in objects of the type. The *safe range* of a floating point type is that part of its base range for which the accuracy corresponding to the base decimal precision is preserved by all predefined operations.
- 10 A floating_point_definition defines a floating point type whose base decimal precision is no less than the requested decimal precision. If a real_range_specification is given, the safe range of the floating point type (and hence, also its base range) includes at least the values of the simple expressions given in the real_range_specification. If a real_range_specification is not given, the safe (and base) range of the type includes at least the values of the range $-10.0^{*(4*D)}$.. $+10.0^{*(4*D)}$ where D is the requested decimal precision. The safe range might include other values as well. The attributes Safe_First and Safe_Last give the actual bounds of the safe range.

A `floating_point_definition` also defines a first subtype of the type. If a `real_range_specification` is given, then the subtype is constrained to a range whose bounds are given by a conversion of the values of the `simple_expressions` of the `real_range_specification` to the type being defined. Otherwise, the subtype is unconstrained. 11

There is a predefined, unconstrained, floating point subtype named `Float`, declared in the visible part of package `Standard`. 12

Dynamic Semantics

The elaboration of a `floating_point_definition` creates the floating point type and its first subtype. 13

Implementation Requirements

In an implementation that supports floating point types with 6 or more digits of precision, the requested decimal precision for `Float` shall be at least 6. 14

If `Long_Float` is predefined for an implementation, then its requested decimal precision shall be at least 11. 15

Implementation Permissions

An implementation is allowed to provide additional predefined floating point types, declared in the visible part of `Standard`, whose (unconstrained) first subtypes have names of the form `Short_Float`, `Long_Float`, `Short_Short_Float`, `Long_Long_Float`, etc. Different predefined floating point types are allowed to have the same base decimal precision. However, the precision of `Float` should be no greater than that of `Long_Float`. Similarly, the precision of `Short_Float` (if provided) should be no greater than `Float`. Corresponding recommendations apply to any other predefined floating point types. There need not be a named floating point type corresponding to each distinct base decimal precision supported by an implementation. 16

Implementation Advice

An implementation should support `Long_Float` in addition to `Float` if the target machine supports 11 or more digits of precision. No other named floating point subtypes are recommended for package `Standard`. Instead, appropriate named floating point subtypes should be provided in the library package `Interfaces` (see B.2). 17

NOTES

37 If a floating point subtype is unconstrained, then assignments to variables of the subtype involve only `Overflow_Checks`, never `Range_Checks`. 18

Examples

Examples of floating point types and subtypes: 19

```
type Coefficient is digits 10 range -1.0 .. 1.0; 20
```

```
type Real is digits 8; 21
```

```
type Mass is digits 7 range 0.0 .. 1.0E35; 22
```

```
subtype Probability is Real range 0.0 .. 1.0;  -- a subtype with a smaller range
```

3.5.8 Operations of Floating Point Types

Static Semantics

The following attribute is defined for every floating point subtype `S`: 1

- 2/1 S'Digits S'Digits denotes the requested decimal precision for the subtype S. The value of this attribute is of the type *universal integer*. The requested decimal precision of the base subtype of a floating point type *T* is defined to be the largest value of *d* for which $\text{ceiling}(d * \log(10) / \log(T'Machine_Radix)) + g \leq T'Model_Mantissa$ where *g* is 0 if Machine_Radix is a positive power of 10 and 1 otherwise.

NOTES

- 3 38 The predefined operations of a floating point type include the assignment operation, qualification, the membership tests, and explicit conversion to and from other numeric types. They also include the relational operators and the following predefined arithmetic operators: the binary and unary adding operators – and +, certain multiplying operators, the unary operator **abs**, and the exponentiation operator.
- 4 39 As for all types, objects of a floating point type have Size and Address attributes (see 13.3). Other attributes of floating point types are defined in A.5.3.

3.5.9 Fixed Point Types

- 1 A fixed point type is either an ordinary fixed point type, or a decimal fixed point type. The error bound of a fixed point type is specified as an absolute value, called the *delta* of the fixed point type.

Syntax

- 2 `fixed_point_definition ::= ordinary_fixed_point_definition | decimal_fixed_point_definition`
- 3 `ordinary_fixed_point_definition ::=`
 `delta static_expression real_range_specification`
- 4 `decimal_fixed_point_definition ::=`
 `delta static_expression digits static_expression [real_range_specification]`
- 5 `digits_constraint ::=`
 `digits static_expression [range_constraint]`

Name Resolution Rules

- 6 For a type defined by a `fixed_point_definition`, the *delta* of the type is specified by the value of the expression given after the reserved word **delta**; this expression is expected to be of any real type. For a type defined by a `decimal_fixed_point_definition` (a *decimal* fixed point type), the number of significant decimal digits for its first subtype (the *digits* of the first subtype) is specified by the expression given after the reserved word **digits**; this expression is expected to be of any integer type.

Legality Rules

- 7 In a `fixed_point_definition` or `digits_constraint`, the expressions given after the reserved words **delta** and **digits** shall be static; their values shall be positive.
- 8/2 The set of values of a fixed point type comprise the integral multiples of a number called the *small* of the type. The *machine numbers* of a fixed point type are the values of the type that can be represented exactly in every unconstrained variable of the type. For a type defined by an `ordinary_fixed_point_definition` (an *ordinary* fixed point type), the *small* may be specified by an `attribute_definition_clause` (see 13.3); if so specified, it shall be no greater than the *delta* of the type. If not specified, the *small* of an ordinary fixed point type is an implementation-defined power of two less than or equal to the *delta*.
- 9 For a decimal fixed point type, the *small* equals the *delta*; the *delta* shall be a power of 10. If a `real_range_specification` is given, both bounds of the range shall be in the range $-(10^{**digits}-1)*delta .. +(10^{**digits}-1)*delta$.
- 10 A `fixed_point_definition` is illegal if the implementation does not support a fixed point type with the given *small* and specified range or *digits*.

For a `subtype_indication` with a `digits_constraint`, the `subtype_mark` shall denote a decimal fixed point subtype. 11

Static Semantics

The base range (see 3.5) of a fixed point type is symmetric around zero, except possibly for an extra negative value in some implementations. 12

An `ordinary_fixed_point_definition` defines an ordinary fixed point type whose base range includes at least all multiples of *small* that are between the bounds specified in the `real_range_specification`. The base range of the type does not necessarily include the specified bounds themselves. An `ordinary_fixed_point_definition` also defines a constrained first subtype of the type, with each bound of its range given by the closer to zero of: 13

- the value of the conversion to the fixed point type of the corresponding expression of the `real_range_specification`; 14
- the corresponding bound of the base range. 15

A `decimal_fixed_point_definition` defines a decimal fixed point type whose base range includes at least the range $-(10^{**}digits-1)*delta .. +(10^{**}digits-1)*delta$. A `decimal_fixed_point_definition` also defines a constrained first subtype of the type. If a `real_range_specification` is given, the bounds of the first subtype are given by a conversion of the values of the expressions of the `real_range_specification`. Otherwise, the range of the first subtype is $-(10^{**}digits-1)*delta .. +(10^{**}digits-1)*delta$. 16

Dynamic Semantics

The elaboration of a `fixed_point_definition` creates the fixed point type and its first subtype. 17

For a `digits_constraint` on a decimal fixed point subtype with a given *delta*, if it does not have a `range_constraint`, then it specifies an implicit range $-(10^{**}D-1)*delta .. +(10^{**}D-1)*delta$, where *D* is the value of the expression. A `digits_constraint` is *compatible* with a decimal fixed point subtype if the value of the expression is no greater than the *digits* of the subtype, and if it specifies (explicitly or implicitly) a range that is compatible with the subtype. 18

The elaboration of a `digits_constraint` consists of the elaboration of the `range_constraint`, if any. If a `range_constraint` is given, a check is made that the bounds of the range are both in the range $-(10^{**}D-1)*delta .. +(10^{**}D-1)*delta$, where *D* is the value of the (static) expression given after the reserved word **digits**. If this check fails, `Constraint_Error` is raised. 19

Implementation Requirements

The implementation shall support at least 24 bits of precision (including the sign bit) for fixed point types. 20

Implementation Permissions

Implementations are permitted to support only *smalls* that are a power of two. In particular, all decimal fixed point type declarations can be disallowed. Note however that conformance with the Information Systems Annex requires support for decimal *smalls*, and decimal fixed point type declarations with *digits* up to at least 18. 21

NOTES

40 The base range of an ordinary fixed point type need not include the specified bounds themselves so that the range specification can be given in a natural way, such as: 22

```
type Fraction is delta 2.0**(-15) range -1.0 .. 1.0; 23
```

- 24 With 2's complement hardware, such a type could have a signed 16-bit representation, using 1 bit for the sign and 15 bits for fraction, resulting in a base range of $-1.0 \dots 1.0 - 2.0^{**}(-15)$.

Examples

25 *Examples of fixed point types and subtypes:*

```

26   type Volt is delta 0.125 range 0.0 .. 255.0;
27   -- A pure fraction which requires all the available
27   -- space in a word can be declared as the type Fraction:
   type Fraction is delta System.Fine_Delta range -1.0 .. 1.0;
   -- Fraction'Last = 1.0 - System.Fine_Delta
28   type Money is delta 0.01 digits 15; -- decimal fixed point
   subtype Salary is Money digits 10;
   -- Money'Last = 10.0**13 - 0.01, Salary'Last = 10.0**8 - 0.01

```

3.5.10 Operations of Fixed Point Types

Static Semantics

1 The following attributes are defined for every fixed point subtype S:

- | | | |
|-----|---------|--|
| 2/1 | S'Small | S'Small denotes the <i>small</i> of the type of S. The value of this attribute is of the type <i>universal_real</i> . Small may be specified for nonderived ordinary fixed point types via an <i>attribute_definition_clause</i> (see 13.3); the expression of such a clause shall be static. |
| 3 | S'Delta | S'Delta denotes the <i>delta</i> of the fixed point subtype S. The value of this attribute is of the type <i>universal_real</i> . |
| 4 | S'Fore | S'Fore yields the minimum number of characters needed before the decimal point for the decimal representation of any value of the subtype S, assuming that the representation does not include an exponent, but includes a one-character prefix that is either a minus sign or a space. (This minimum number does not include superfluous zeros or underlines, and is at least 2.) The value of this attribute is of the type <i>universal_integer</i> . |
| 5 | S'Aft | S'Aft yields the number of decimal digits needed after the decimal point to accommodate the <i>delta</i> of the subtype S, unless the <i>delta</i> of the subtype S is greater than 0.1, in which case the attribute yields the value one. (S'Aft is the smallest positive integer N for which $(10^{**}N) * S'Delta$ is greater than or equal to one.) The value of this attribute is of the type <i>universal_integer</i> . |

6 The following additional attributes are defined for every decimal fixed point subtype S:

- | | | |
|----|----------|--|
| 7 | S'Digits | S'Digits denotes the <i>digits</i> of the decimal fixed point subtype S, which corresponds to the number of decimal digits that are representable in objects of the subtype. The value of this attribute is of the type <i>universal_integer</i> . Its value is determined as follows: |
| 8 | | <ul style="list-style-type: none"> • For a first subtype or a subtype defined by a <i>subtype_indication</i> with a <i>digits_constraint</i>, the digits is the value of the expression given after the reserved word digits; |
| 9 | | <ul style="list-style-type: none"> • For a subtype defined by a <i>subtype_indication</i> without a <i>digits_constraint</i>, the digits of the subtype is the same as that of the subtype denoted by the <i>subtype_mark</i> in the <i>subtype_indication</i>. |
| 10 | | <ul style="list-style-type: none"> • The digits of a base subtype is the largest integer <i>D</i> such that the range $-(10^{**}D - 1) * delta \dots +(10^{**}D - 1) * delta$ is included in the base range of the type. |

S'Scale	S'Scale denotes the <i>scale</i> of the subtype S, defined as the value N such that S'Delta = 10.0*(-N). The scale indicates the position of the point relative to the rightmost significant digits of values of subtype S. The value of this attribute is of the type <i>universal_integer</i> .	11
S'Round	S'Round denotes a function with the following specification:	12
	function S'Round (<i>X</i> : <i>universal_real</i>)	13
	return S'Base	
	The function returns the value obtained by rounding X (away from 0, if X is midway between two values of the type of S).	14
NOTES		
41	All subtypes of a fixed point type will have the same value for the Delta attribute, in the absence of delta_constraints (see J.3).	15
42	S'Scale is not always the same as S'Aft for a decimal subtype; for example, if S'Delta = 1.0 then S'Aft is 1 while S'Scale is 0.	16
43	The predefined operations of a fixed point type include the assignment operation, qualification, the membership tests, and explicit conversion to and from other numeric types. They also include the relational operators and the following predefined arithmetic operators: the binary and unary adding operators – and +, multiplying operators, and the unary operator abs .	17
44	As for all types, objects of a fixed point type have Size and Address attributes (see 13.3). Other attributes of fixed point types are defined in A.5.4.	18

3.6 Array Types

An *array* object is a composite object consisting of components which all have the same subtype. The name for a component of an array uses one or more index values belonging to specified discrete types. The value of an array object is a composite value consisting of the values of the components.

Syntax

array_type_definition ::=	2
unconstrained_array_definition constrained_array_definition	
unconstrained_array_definition ::=	3
array (index_subtype_definition {, index_subtype_definition}) of component_definition	
index_subtype_definition ::= subtype_mark range <>	4
constrained_array_definition ::=	5
array (discrete_subtype_definition {, discrete_subtype_definition}) of component_definition	
discrete_subtype_definition ::= <i>discrete_subtype_indication</i> range	6
component_definition ::=	7/2
[aliased] subtype_indication	
[aliased] access_definition	

Name Resolution Rules

For a *discrete_subtype_definition* that is a *range*, the *range* shall resolve to be of some specific discrete type; which discrete type shall be determined without using any context other than the bounds of the *range* itself (plus the preference for *root_integer* — see 8.6).

Legality Rules

Each *index_subtype_definition* or *discrete_subtype_definition* in an *array_type_definition* defines an *index subtype*; its type (the *index type*) shall be discrete.

- 10 The subtype defined by the `subtype_indication` of a `component_definition` (the *component subtype*) shall be a definite subtype.

11/2 *This paragraph was deleted.*

Static Semantics

- 12 An array is characterized by the number of indices (the *dimensionality* of the array), the type and position of each index, the lower and upper bounds for each index, and the subtype of the components. The order of the indices is significant.
- 13 A one-dimensional array has a distinct component for each possible index value. A multidimensional array has a distinct component for each possible sequence of index values that can be formed by selecting one value for each index position (in the given order). The possible values for a given index are all the values between the lower and upper bounds, inclusive; this range of values is called the *index range*. The *bounds* of an array are the bounds of its index ranges. The *length* of a dimension of an array is the number of values of the index range of the dimension (zero for a null range). The *length* of a one-dimensional array is the length of its only dimension.
- 14 An `array_type_definition` defines an array type and its first subtype. For each object of this array type, the number of indices, the type and position of each index, and the subtype of the components are as in the type definition; the values of the lower and upper bounds for each index belong to the corresponding index subtype of its type, except for null arrays (see 3.6.1).
- 15 An `unconstrained_array_definition` defines an array type with an unconstrained first subtype. Each `index_subtype_definition` defines the corresponding index subtype to be the subtype denoted by the `subtype_mark`. The compound delimiter `<>` (called a *box*) of an `index_subtype_definition` stands for an undefined range (different objects of the type need not have the same bounds).
- 16 A `constrained_array_definition` defines an array type with a constrained first subtype. Each `discrete_subtype_definition` defines the corresponding index subtype, as well as the corresponding index range for the constrained first subtype. The *constraint* of the first subtype consists of the bounds of the index ranges.
- 17 The discrete subtype defined by a `discrete_subtype_definition` is either that defined by the `subtype_indication`, or a subtype determined by the *range* as follows:
- 18 • If the type of the *range* resolves to *root_integer*, then the `discrete_subtype_definition` defines a subtype of the predefined type Integer with bounds given by a conversion to Integer of the bounds of the *range*;
 - 19 • Otherwise, the `discrete_subtype_definition` defines a subtype of the type of the *range*, with the bounds given by the *range*.
- 20 The `component_definition` of an `array_type_definition` defines the nominal subtype of the components. If the reserved word **aliased** appears in the `component_definition`, then each component of the array is aliased (see 3.10).

Dynamic Semantics

- 21 The elaboration of an `array_type_definition` creates the array type and its first subtype, and consists of the elaboration of any `discrete_subtype_definitions` and the `component_definition`.
- 22/2 The elaboration of a `discrete_subtype_definition` that does not contain any per-object expressions creates the discrete subtype, and consists of the elaboration of the `subtype_indication` or the evaluation of the *range*. The elaboration of a `discrete_subtype_definition` that contains one or more per-object expressions is defined in 3.8. The elaboration of a `component_definition` in an `array_type_definition` consists of the

elaboration of the `subtype_indication` or `access_definition`. The elaboration of any `discrete_subtype_definitions` and the elaboration of the `component_definition` are performed in an arbitrary order.

NOTES

45 All components of an array have the same subtype. In particular, for an array of components that are one-dimensional arrays, this means that all components have the same bounds and hence the same length.

46 Each elaboration of an `array_type_definition` creates a distinct array type. A consequence of this is that each object whose `object_declaration` contains an `array_type_definition` is of its own unique type.

Examples

Examples of type declarations with unconstrained array definitions:

```

type Vector      is array(Integer range <>) of Real;
type Matrix      is array(Integer range <>, Integer range <>) of Real;
type Bit_Vector  is array(Integer range <>) of Boolean;
type Roman       is array(Positive range <>) of Roman_Digit; -- see 3.5.2

```

Examples of type declarations with constrained array definitions:

```

type Table       is array(1 .. 10) of Integer;
type Schedule    is array(Day) of Boolean;
type Line        is array(1 .. Max_Line_Size) of Character;

```

Examples of object declarations with array type definitions:

```

Grid           : array(1 .. 80, 1 .. 100) of Boolean;
Mix            : array(Color range Red .. Green) of Boolean;
Msg_Table      : constant array(Error_Code) of access constant String :=
  (Too_Big => new String("Result too big"), Too_Small => ...);
Page           : array(Positive range <>) of Line := -- an array of arrays
  (1 | 50 => Line'(1 | Line'Last => '+', others => '-'), -- see 4.3.3
   2 .. 49 => Line'(1 | Line'Last => '|', others => ' '));
-- Page is constrained by its initial value to (1..50)

```

3.6.1 Index Constraints and Discrete Ranges

An `index_constraint` determines the range of possible values for every index of an array subtype, and thereby the corresponding array bounds.

Syntax

`index_constraint ::= (discrete_range {, discrete_range})`

`discrete_range ::= discrete_subtype_indication | range`

Name Resolution Rules

The type of a `discrete_range` is the type of the subtype defined by the `subtype_indication`, or the type of the range. For an `index_constraint`, each `discrete_range` shall resolve to be of the type of the corresponding index.

Legality Rules

An `index_constraint` shall appear only in a `subtype_indication` whose `subtype_mark` denotes either an unconstrained array subtype, or an unconstrained access subtype whose designated subtype is an unconstrained array subtype; in either case, the `index_constraint` shall provide a `discrete_range` for each index of the array type.

Static Semantics

- 6 A `discrete_range` defines a range whose bounds are given by the `range`, or by the range of the subtype defined by the `subtype_indication`.

Dynamic Semantics

- 7 An `index_constraint` is *compatible* with an unconstrained array subtype if and only if the index range defined by each `discrete_range` is compatible (see 3.5) with the corresponding index subtype. If any of the `discrete_ranges` defines a null range, any array thus constrained is a *null array*, having no components. An array value *satisfies* an `index_constraint` if at each index position the array value and the `index_constraint` have the same index bounds.
- 8 The elaboration of an `index_constraint` consists of the evaluation of the `discrete_range(s)`, in an arbitrary order. The evaluation of a `discrete_range` consists of the elaboration of the `subtype_indication` or the evaluation of the `range`.

NOTES

- 9 47 The elaboration of a `subtype_indication` consisting of a `subtype_mark` followed by an `index_constraint` checks the compatibility of the `index_constraint` with the `subtype_mark` (see 3.2.2).
- 10 48 Even if an array value does not satisfy the index constraint of an array subtype, `Constraint_Error` is not raised on conversion to the array subtype, so long as the length of each dimension of the array value and the array subtype match. See 4.6.

Examples

- 11 *Examples of array declarations including an index constraint:*

```
12   Board      : Matrix(1 .. 8, 1 .. 8); -- see 3.6
13   Rectangle  : Matrix(1 .. 20, 1 .. 30);
14   Inverse    : Matrix(1 .. N, 1 .. N); -- N need not be static
15   Filter     : Bit_Vector(0 .. 31);
```

- 16 *Example of array declaration with a constrained array subtype:*

```
17   My_Schedule : Schedule; -- all arrays of type Schedule have the same bounds
```

- 18 *Example of record type with a component that is an array:*

```
19   type Var_Line(Length : Natural) is
20     record
21       Image : String(1 .. Length);
22     end record;
23   Null_Line : Var_Line(0); -- Null_Line.Image is a null array
```

3.6.2 Operations of Array Types

Legality Rules

- 1 The argument `N` used in the `attribute_designators` for the `N`-th dimension of an array shall be a static expression of some integer type. The value of `N` shall be positive (nonzero) and no greater than the dimensionality of the array.

Static Semantics

- 2/1 The following attributes are defined for a `prefix A` that is of an array type (after any implicit dereference), or denotes a constrained array subtype:

- 3 `A'First` `A'First` denotes the lower bound of the first index range; its type is the corresponding index type.

A'First(N)	A'First(N) denotes the lower bound of the N-th index range; its type is the corresponding index type.	4
A'Last	A'Last denotes the upper bound of the first index range; its type is the corresponding index type.	5
A'Last(N)	A'Last(N) denotes the upper bound of the N-th index range; its type is the corresponding index type.	6
A'Range	A'Range is equivalent to the range A'First .. A'Last, except that the prefix A is only evaluated once.	7
A'Range(N)	A'Range(N) is equivalent to the range A'First(N) .. A'Last(N), except that the prefix A is only evaluated once.	8
A'Length	A'Length denotes the number of values of the first index range (zero for a null range); its type is <i>universal_integer</i> .	9
A'Length(N)	A'Length(N) denotes the number of values of the N-th index range (zero for a null range); its type is <i>universal_integer</i> .	10

Implementation Advice

An implementation should normally represent multidimensional arrays in row-major order, consistent with the notation used for multidimensional array aggregates (see 4.3.3). However, if a **pragma Convention**(Fortran, ...) applies to a multidimensional array type, then column-major order should be used instead (see B.5, “Interfacing with Fortran”).

NOTES

49 The attribute_references A'First and A'First(1) denote the same value. A similar relation exists for the attribute_references A'Last, A'Range, and A'Length. The following relation is satisfied (except for a null array) by the above attributes if the index type is an integer type:

$$A'Length(N) = A'Last(N) - A'First(N) + 1$$

50 An array type is limited if its component type is limited (see 7.5).

51 The predefined operations of an array type include the membership tests, qualification, and explicit conversion. If the array type is not limited, they also include assignment and the predefined equality operators. For a one-dimensional array type, they include the predefined concatenation operators (if nonlimited) and, if the component type is discrete, the predefined relational operators; if the component type is boolean, the predefined logical operators are also included.

52 A component of an array can be named with an indexed_component. A value of an array type can be specified with an array_aggregate. For a one-dimensional array type, a slice of the array can be named; also, string literals are defined if the component type is a character type.

Examples

Examples (using arrays declared in the examples of subclause 3.6.1):

```
-- Filter'First      = 0      Filter'Last      = 31      Filter'Length = 32
-- Rectangle'Last(1) = 20     Rectangle'Last(2) = 30
```

3.6.3 String Types

Static Semantics

A one-dimensional array type whose component type is a character type is called a *string* type.

There are three predefined string types, String, Wide_String, and Wide_Wide_String, each indexed by values of the predefined subtype Positive; these are declared in the visible part of package Standard:

```
subtype Positive is Integer range 1 .. Integer'Last;
```

```

4/2  type String is array(Positive range <>) of Character;
      type Wide_String is array(Positive range <>) of Wide_Character;
      type Wide_Wide_String is array(Positive range <>) of Wide_Wide_Character;

```

NOTES

- 5 53 String literals (see 2.6 and 4.2) are defined for all string types. The concatenation operator & is predefined for string types, as for all nonlimited one-dimensional array types. The ordering operators <, <=, >, and >= are predefined for string types, as for all one-dimensional discrete array types; these ordering operators correspond to lexicographic order (see 4.5.2).

Examples

6 *Examples of string objects:*

```

7  Stars      : String(1 .. 120) := (1 .. 120 => '*' );
  Question    : constant String := "How many characters?";
                                     -- Question'First = 1, Question'Last = 20
                                     -- Question'Length = 20 (the number of
                                     characters)

8  Ask_Twice   : String := Question & Question;      -- constrained to (1..40)
  Ninety_Six   : constant Roman := "XCVI"; -- see 3.5.2 and 3.6

```

3.7 Discriminants

- 1/2 A composite type (other than an array or interface type) can have discriminants, which parameterize the type. A `known_discriminant_part` specifies the discriminants of a composite type. A discriminant of an object is a component of the object, and is either of a discrete type or an access type. An `unknown_discriminant_part` in the declaration of a view of a type specifies that the discriminants of the type are unknown for the given view; all subtypes of such a view are indefinite subtypes.

Syntax

```

2/2  discriminant_part ::= unknown_discriminant_part | known_discriminant_part
3    unknown_discriminant_part ::= (<>)
4    known_discriminant_part ::=
      (discriminant_specification {; discriminant_specification})
5/2  discriminant_specification ::=
      defining_identifier_list : [null_exclusion] subtype_mark [:= default_expression]
      | defining_identifier_list : access_definition [:= default_expression]
6    default_expression ::= expression

```

Name Resolution Rules

- 7 The expected type for the `default_expression` of a `discriminant_specification` is that of the corresponding discriminant.

Legality Rules

- 8/2 A `discriminant_part` is only permitted in a declaration for a composite type that is not an array or interface type (this includes generic formal types). A type declared with a `known_discriminant_part` is called a *discriminated type*, as is a type that inherits (known) discriminants.
- 9/2 The subtype of a discriminant may be defined by an optional `null_exclusion` and a `subtype_mark`, in which case the `subtype_mark` shall denote a discrete or access subtype, or it may be defined by an `access_definition`. A discriminant that is defined by an `access_definition` is called an *access discriminant* and is of an anonymous access type.

Default_expressions shall be provided either for all or for none of the discriminants of a known_discriminant_part. No default_expressions are permitted in a known_discriminant_part in a declaration of a tagged type or a generic formal type. 9.1/2

A discriminant_specification for an access discriminant may have a default_expression only in the declaration for an immutably limited type (see 7.5). In addition to the places where Legality Rules normally apply (see 12.3), this rule applies also in the private part of an instance of a generic unit. 10/3

This paragraph was deleted. 11/2

For a type defined by a derived_type_definition, if a known_discriminant_part is provided in its declaration, then: 12

- The parent subtype shall be constrained; 13
- If the parent type is not a tagged type, then each discriminant of the derived type shall be used in the constraint defining the parent subtype; 14
- If a discriminant is used in the constraint defining the parent subtype, the subtype of the discriminant shall be statically compatible (see 4.9.1) with the subtype of the corresponding parent discriminant. 15

This paragraph was deleted. 16/3

Static Semantics

A discriminant_specification declares a discriminant; the subtype_mark denotes its subtype unless it is an access discriminant, in which case the discriminant's subtype is the anonymous access-to-variable subtype defined by the access_definition. 17

For a type defined by a derived_type_definition, each discriminant of the parent type is either inherited, constrained to equal some new discriminant of the derived type, or constrained to the value of an expression. When inherited or constrained to equal some new discriminant, the parent discriminant and the discriminant of the derived type are said to *correspond*. Two discriminants also correspond if there is some common discriminant to which they both correspond. A discriminant corresponds to itself as well. If a discriminant of a parent type is constrained to a specific value by a derived_type_definition, then that discriminant is said to be *specified* by that derived_type_definition. 18

A constraint that appears within the definition of a discriminated type *depends on a discriminant* of the type if it names the discriminant as a bound or discriminant value. A component_definition depends on a discriminant if its constraint depends on the discriminant, or on a discriminant that corresponds to it. 19

A component *depends on a discriminant* if: 20

- Its component_definition depends on the discriminant; or 21
- It is declared in a variant_part that is governed by the discriminant; or 22
- It is a component inherited as part of a derived_type_definition, and the constraint of the parent_subtype_indication depends on the discriminant; or 23
- It is a subcomponent of a component that depends on the discriminant. 24

Each value of a discriminated type includes a value for each component of the type that does not depend on a discriminant; this includes the discriminants themselves. The values of discriminants determine which other component values are present in the value of the discriminated type. 25

A type declared with a known_discriminant_part is said to have *known discriminants*; its first subtype is unconstrained. A type declared with an unknown_discriminant_part is said to have *unknown* 26

discriminants. A type declared without a `discriminant_part` has no discriminants, unless it is a derived type; if derived, such a type has the same sort of discriminants (known, unknown, or none) as its parent (or ancestor) type. A tagged class-wide type also has unknown discriminants. Any subtype of a type with unknown discriminants is an unconstrained and indefinite subtype (see 3.2 and 3.3).

Dynamic Semantics

27/2 For an access discriminant, its `access_definition` is elaborated when the value of the access discriminant is defined: by evaluation of its `default_expression`, by elaboration of a `discriminant_constraint`, or by an assignment that initializes the enclosing object.

NOTES

- 28 54 If a discriminated type has `default_expressions` for its discriminants, then unconstrained variables of the type are permitted, and the values of the discriminants can be changed by an assignment to such a variable. If defaults are not provided for the discriminants, then all variables of the type are constrained, either by explicit constraint or by their initial value; the values of the discriminants of such a variable cannot be changed after initialization.
- 29 55 The `default_expression` for a discriminant of a type is evaluated when an object of an unconstrained subtype of the type is created.
- 30 56 Assignment to a discriminant of an object (after its initialization) is not allowed, since the name of a discriminant is a constant; neither `assignment_statements` nor assignments inherent in passing as an **in out** or **out** parameter are allowed. Note however that the value of a discriminant can be changed by assigning to the enclosing object, presuming it is an unconstrained variable.
- 31 57 A discriminant that is of a named access type is not called an access discriminant; that term is used only for discriminants defined by an `access_definition`.

Examples

32 *Examples of discriminated types:*

```

33   type Buffer(Size : Buffer_Size := 100) is           -- see 3.5.4
      record
        Pos      : Buffer_Size := 0;
        Value    : String(1 .. Size);
      end record;

34   type Matrix_Rec(Rows, Columns : Integer) is
      record
        Mat : Matrix(1 .. Rows, 1 .. Columns);        -- see 3.6
      end record;

35   type Square(Side : Integer) is new
      Matrix_Rec(Rows => Side, Columns => Side);

36   type Double_Square(Number : Integer) is
      record
        Left  : Square(Number);
        Right : Square(Number);
      end record;

37/2 task type Worker(Prio : System.Priority; Buf : access Buffer) is
      -- discriminants used to parameterize the task type (see 9.1)
      pragma Priority(Prio); -- see D.1
      entry Fill;
      entry Drain;
    end Worker;
```

3.7.1 Discriminant Constraints

- 1 A `discriminant_constraint` specifies the values of the discriminants for a given discriminated type.

Syntax

discriminant_constraint ::= (discriminant_association {, discriminant_association}) 2

discriminant_association ::= [discriminant_selector_name { | discriminant_selector_name } =>] expression 3

A discriminant_association is said to be *named* if it has one or more *discriminant_selector_names*; it is otherwise said to be *positional*. In a discriminant_constraint, any positional associations shall precede any named associations. 4

Name Resolution Rules

Each selector_name of a named discriminant_association shall resolve to denote a discriminant of the subtype being constrained; the discriminants so named are the *associated discriminants* of the named association. For a positional association, the *associated discriminant* is the one whose discriminant_specification occurred in the corresponding position in the known_discriminant_part that defined the discriminants of the subtype being constrained. 5

The expected type for the expression in a discriminant_association is that of the associated discriminant(s). 6

Legality Rules

A discriminant_constraint is only allowed in a subtype_indication whose subtype_mark denotes either an unconstrained discriminated subtype, or an unconstrained access subtype whose designated subtype is an unconstrained discriminated subtype. However, in the case of an access subtype, a discriminant_constraint is legal only if any dereference of a value of the access type is known to be constrained (see 3.3). In addition to the places where Legality Rules normally apply (see 12.3), these rules apply also in the private part of an instance of a generic unit. 7/3

A named discriminant_association with more than one selector_name is allowed only if the named discriminants are all of the same type. A discriminant_constraint shall provide exactly one value for each discriminant of the subtype being constrained. 8

This paragraph was deleted. 9/3

Dynamic Semantics

A discriminant_constraint is *compatible* with an unconstrained discriminated subtype if each discriminant value belongs to the subtype of the corresponding discriminant. 10

A composite value *satisfies* a discriminant constraint if and only if each discriminant of the composite value has the value imposed by the discriminant constraint. 11

For the elaboration of a discriminant_constraint, the expressions in the discriminant_associations are evaluated in an arbitrary order and converted to the type of the associated discriminant (which might raise Constraint_Error — see 4.6); the expression of a named association is evaluated (and converted) once for each associated discriminant. The result of each evaluation and conversion is the value imposed by the constraint for the associated discriminant. 12

NOTES

58 The rules of the language ensure that a discriminant of an object always has a value, either from explicit or implicit initialization. 13

Examples

14 *Examples (using types declared above in clause 3.7):*

```

15      Large   : Buffer(200);  -- constrained, always 200 characters
                                -- (explicit discriminant value)
      Message : Buffer;       -- unconstrained, initially 100 characters
                                -- (default discriminant value)
      Basis   : Square(5);    -- constrained, always 5 by 5
      Illegal : Square;       -- illegal, a Square has to be constrained

```

3.7.2 Operations of Discriminated Types

- 1 If a discriminated type has **default_expressions** for its discriminants, then unconstrained variables of the type are permitted, and the discriminants of such a variable can be changed by assignment to the variable. For a formal parameter of such a type, an attribute is provided to determine whether the corresponding actual parameter is constrained or unconstrained.

Static Semantics

- 2 For a prefix **A** that is of a discriminated type (after any implicit dereference), the following attribute is defined:
- 3 **A'Constrained**
 Yields the value **True** if **A** denotes a constant, a value, or a constrained variable, and **False** otherwise.

Erroneous Execution

- 4 The execution of a construct is erroneous if the construct has a constituent that is a **name** denoting a subcomponent that depends on discriminants, and the value of any of these discriminants is changed by this execution between evaluating the **name** and the last use (within this execution) of the subcomponent denoted by the **name**.

3.8 Record Types

- 1 A record object is a composite object consisting of named components. The value of a record object is a composite value consisting of the values of the components.

Syntax

```

2      record_type_definition ::= [[abstract] tagged] [limited] record_definition
3      record_definition ::=
          record
            component_list
          end record
        | null record
4      component_list ::=
          component_item {component_item}
        | {component_item} variant_part
        | null;
5/1      component_item ::= component_declaration | aspect_clause
6      component_declaration ::=
          defining_identifier_list : component_definition [:= default_expression];

```


Name Resolution Rules

The expected type for the `default_expression`, if any, in a `component_declaration` is the type of the component. 7

Legality Rules

This paragraph was deleted. 8/2

Each `component_declaration` declares a component of the record type. Besides components declared by `component_declarations`, the components of a record type include any components declared by `discriminant_specifications` of the record type declaration. The identifiers of all components of a record type shall be distinct. 9/2

Within a `type_declaration`, a name that denotes a component, protected subprogram, or entry of the type is allowed only in the following cases: 10

- A name that denotes any component, protected subprogram, or entry is allowed within an operational or representation item that occurs within the declaration of the composite type. 11/3
- A name that denotes a noninherited discriminant is allowed within the declaration of the type, but not within the `discriminant_part`. If the discriminant is used to define the constraint of a component, the bounds of an entry family, or the constraint of the parent subtype in a `derived_type_definition` then its name shall appear alone as a `direct_name` (not as part of a larger expression or expanded name). A discriminant shall not be used to define the constraint of a scalar component. 12

If the name of the current instance of a type (see 8.6) is used to define the constraint of a component, then it shall appear as a `direct_name` that is the prefix of an `attribute_reference` whose result is of an access type, and the `attribute_reference` shall appear alone. 13

Static Semantics

If a `record_type_definition` includes the reserved word **limited**, the type is called an *explicitly limited record* type. 13.1/3

The `component_definition` of a `component_declaration` defines the (nominal) subtype of the component. If the reserved word **aliased** appears in the `component_definition`, then the component is aliased (see 3.10). 14

If the `component_list` of a record type is defined by the reserved word **null** and there are no discriminants, then the record type has no components and all records of the type are *null records*. A `record_definition` of **null record** is equivalent to **record null; end record**. 15

Dynamic Semantics

The elaboration of a `record_type_definition` creates the record type and its first subtype, and consists of the elaboration of the `record_definition`. The elaboration of a `record_definition` consists of the elaboration of its `component_list`, if any. 16

The elaboration of a `component_list` consists of the elaboration of the `component_items` and `variant_part`, if any, in the order in which they appear. The elaboration of a `component_declaration` consists of the elaboration of the `component_definition`. 17

Within the definition of a composite type, if a `component_definition` or `discrete_subtype_definition` (see 9.5.2) includes a name that denotes a discriminant of the type, or that is an `attribute_reference` whose prefix denotes the current instance of the type, the expression containing the name is called a *per-object expression*, and the constraint or range being defined is called a *per-object constraint*. For the elaboration 18/2

of a `component_definition` of a `component_declaration` or the `discrete_subtype_definition` of an `entry_declaration` for an entry family (see 9.5.2), if the component subtype is defined by an `access_definition` or if the `constraint` or `range` of the `subtype_indication` or `discrete_subtype_definition` is not a per-object constraint, then the `access_definition`, `subtype_indication`, or `discrete_subtype_definition` is elaborated. On the other hand, if the `constraint` or `range` is a per-object constraint, then the elaboration consists of the evaluation of any included expression that is not part of a per-object expression. Each such expression is evaluated once unless it is part of a named association in a discriminant constraint, in which case it is evaluated once for each associated discriminant.

- 18.1/1 When a per-object constraint is elaborated (as part of creating an object), each per-object expression of the constraint is evaluated. For other expressions, the values determined during the elaboration of the `component_definition` or `entry_declaration` are used. Any checks associated with the enclosing `subtype_indication` or `discrete_subtype_definition` are performed, including the subtype compatibility check (see 3.2.2), and the associated subtype is created.

NOTES

- 19 59 A `component_declaration` with several identifiers is equivalent to a sequence of single `component_declarations`, as explained in 3.3.1.
- 20 60 The `default_expression` of a record component is only evaluated upon the creation of a default-initialized object of the record type (presuming the object has the component, if it is in a `variant_part` — see 3.3.1).
- 21 61 The subtype defined by a `component_definition` (see 3.6) has to be a definite subtype.
- 22 62 If a record type does not have a `variant_part`, then the same components are present in all values of the type.
- 23 63 A record type is limited if it has the reserved word **limited** in its definition, or if any of its components are limited (see 7.5).
- 24 64 The predefined operations of a record type include membership tests, qualification, and explicit conversion. If the record type is nonlimited, they also include assignment and the predefined equality operators.
- 25/2 65 A component of a record can be named with a `selected_component`. A value of a record can be specified with a `record_aggregate`.

Examples

26 *Examples of record type declarations:*

```
27     type Date is
        record
            Day   : Integer range 1 .. 31;
            Month : Month_Name;
            Year   : Integer range 0 .. 4000;
        end record;
28     type Complex is
        record
            Re : Real := 0.0;
            Im : Real := 0.0;
        end record;
```

29 *Examples of record variables:*

```
30     Tomorrow, Yesterday : Date;
     A, B, C : Complex;
31     -- both components of A, B, and C are implicitly initialized to zero
```

3.8.1 Variant Parts and Discrete Choices

- 1 A record type with a `variant_part` specifies alternative lists of components. Each `variant` defines the components for the value or values of the discriminant covered by its `discrete_choice_list`.

Syntax

```

variant_part ::=
    case discriminant_direct_name is
        variant
        {variant}
    end case;
variant ::=
    when discrete_choice_list =>
        component_list
discrete_choice_list ::= discrete_choice { | discrete_choice }
discrete_choice ::= expression | discrete_range | others

```

Name Resolution Rules

The *discriminant_direct_name* shall resolve to denote a discriminant (called the *discriminant of the variant_part*) specified in the known_discriminant_part of the full_type_declaration that contains the variant_part. The expected type for each discrete_choice in a variant is the type of the discriminant of the variant_part.

Legality Rules

The discriminant of the variant_part shall be of a discrete type.

The expressions and discrete_ranges given as discrete_choices in a variant_part shall be static. The discrete_choice **others** shall appear alone in a discrete_choice_list, and such a discrete_choice_list, if it appears, shall be the last one in the enclosing construct.

A discrete_choice is defined to *cover a value* in the following cases:

- A discrete_choice that is an expression covers a value if the value equals the value of the expression converted to the expected type.
- A discrete_choice that is a discrete_range covers all values (possibly none) that belong to the range.
- The discrete_choice **others** covers all values of its expected type that are not covered by previous discrete_choice_lists of the same construct.

A discrete_choice_list covers a value if one of its discrete_choices covers the value.

The possible values of the discriminant of a variant_part shall be covered as follows:

- If the discriminant is of a static constrained scalar subtype, then each non-**others** discrete_choice shall cover only values in that subtype, and each value of that subtype shall be covered by some discrete_choice (either explicitly or by **others**);
- If the type of the discriminant is a descendant of a generic formal scalar type then the variant_part shall have an **others** discrete_choice;
- Otherwise, each value of the base range of the type of the discriminant shall be covered (either explicitly or by **others**).

Two distinct discrete_choices of a variant_part shall not cover the same value.

Static Semantics

If the component_list of a variant is specified by **null**, the variant has no components.

- 20 The discriminant of a `variant_part` is said to *govern* the `variant_part` and its `variants`. In addition, the discriminant of a derived type governs a `variant_part` and its `variants` if it corresponds (see 3.7) to the discriminant of the `variant_part`.

Dynamic Semantics

- 21 A record value contains the values of the components of a particular `variant` only if the value of the discriminant governing the `variant` is covered by the `discrete_choice_list` of the `variant`. This rule applies in turn to any further `variant` that is, itself, included in the `component_list` of the given `variant`.
- 22 The elaboration of a `variant_part` consists of the elaboration of the `component_list` of each `variant` in the order in which they appear.

Examples

- 23 *Example of record type with a variant part:*

```

24     type Device is (Printer, Disk, Drum);
25     type State is (Open, Closed);
26     type Peripheral(Unit : Device := Disk) is
       record
         Status : State;
         case Unit is
           when Printer =>
             Line_Count : Integer range 1 .. Page_Size;
           when others =>
             Cylinder : Cylinder_Index;
             Track : Track_Number;
           end case;
       end record;

```

- 26 *Examples of record subtypes:*

```

27     subtype Drum_Unit is Peripheral(Drum);
28     subtype Disk_Unit is Peripheral(Disk);

```

- 28 *Examples of constrained record variables:*

```

29     Writer : Peripheral(Unit => Printer);
30     Archive : Disk_Unit;

```

3.9 Tagged Types and Type Extensions

- 1 Tagged types and type extensions support object-oriented programming, based on inheritance with extension and run-time polymorphism via *dispatching operations*.

Static Semantics

- 2/2 A record type or private type that has the reserved word **tagged** in its declaration is called a *tagged type*. In addition, an interface type is a tagged type, as is a task or protected type derived from an interface (see 3.9.4). When deriving from a tagged type, as for any derived type, additional primitive subprograms may be defined, and inherited primitive subprograms may be overridden. The derived type is called an *extension* of its ancestor types, or simply a *type extension*.
- 2.1/2 Every type extension is also a tagged type, and is a *record extension* or a *private extension* of some other tagged type, or a non-interface synchronized tagged type (see 3.9.4). A record extension is defined by a *derived_type_definition* with a *record_extension_part* (see 3.9.1), which may include the definition of additional components. A private extension, which is a partial view of a record extension or of a

synchronized tagged type, can be declared in the visible part of a package (see 7.3) or in a generic formal part (see 12.5.1).

An object of a tagged type has an associated (run-time) *tag* that identifies the specific tagged type used to create the object originally. The tag of an operand of a class-wide tagged type *TClass* controls which subprogram body is to be executed when a primitive subprogram of type *T* is applied to the operand (see 3.9.2); using a tag to control which body to execute is called *dispatching*. 3

The tag of a specific tagged type identifies the `full_type_declaration` of the type, and for a type extension, is sufficient to uniquely identify the type among all descendants of the same ancestor. If a declaration for a tagged type occurs within a `generic_package_declaration`, then the corresponding type declarations in distinct instances of the generic package are associated with distinct tags. For a tagged type that is local to a generic package body and with all of its ancestors (if any) also local to the generic body, the language does not specify whether repeated instantiations of the generic body result in distinct tags. 4/2

The following language-defined library package exists: 5

```

package Ada.Tags is
  pragma Preelaborate(Tags);
  type Tag is private;
  pragma Preelaborable_Initialization(Tag);
  No_Tag : constant Tag;
  function Expanded_Name(T : Tag) return String;
  function Wide_Expanded_Name(T : Tag) return Wide_String;
  function Wide_Wide_Expanded_Name(T : Tag) return Wide_Wide_String;
  function External_Tag(T : Tag) return String;
  function Internal_Tag(External : String) return Tag;
  function Descendant_Tag(External : String; Ancestor : Tag) return Tag;
  function Is_Descendant_At_Same_Level(Descendant, Ancestor : Tag)
    return Boolean;
  function Parent_Tag (T : Tag) return Tag;
  type Tag_Array is array (Positive range <>) of Tag;
  function Interface_Ancestor_Tags (T : Tag) return Tag_Array;
  Tag_Error : exception;
private
  ... -- not specified by the language
end Ada.Tags;

```

No_Tag is the default initial value of type Tag. 9.1/2

The function `Wide_Wide_Expanded_Name` returns the full expanded name of the first subtype of the specific type identified by the tag, in upper case, starting with a root library unit. The result is implementation defined if the type is declared within an unnamed block_statement. 10/2

The function `Expanded_Name` (respectively, `Wide_Expanded_Name`) returns the same sequence of graphic characters as that defined for `Wide_Wide_Expanded_Name`, if all the graphic characters are defined in `Character` (respectively, `Wide_Character`); otherwise, the sequence of characters is implementation defined, but no shorter than that returned by `Wide_Wide_Expanded_Name` for the same value of the argument. 10.1/2

The function `External_Tag` returns a string to be used in an external representation for the given tag. The call `External_Tag(S'Tag)` is equivalent to the attribute_reference `S'External_Tag` (see 13.3). 11

The string returned by the functions `Expanded_Name`, `Wide_Expanded_Name`, `Wide_Wide_Expanded_Name`, and `External_Tag` has lower bound 1. 11.1/2

- 12/2 The function `Internal_Tag` returns a tag that corresponds to the given external tag, or raises `Tag_Error` if the given string is not the external tag for any specific type of the partition. `Tag_Error` is also raised if the specific type identified is a library-level type whose tag has not yet been created (see 13.14).
- 12.1/3 The function `Descendant_Tag` returns the (internal) tag for the type that corresponds to the given external tag and is both a descendant of the type identified by the `Ancestor` tag and has the same accessibility level as the identified ancestor. `Tag_Error` is raised if `External` is not the external tag for such a type. `Tag_Error` is also raised if the specific type identified is a library-level type whose tag has not yet been created, or if the given external tag identifies more than one type that has the appropriate `Ancestor` and accessibility level.
- 12.2/2 The function `Is_Descendant_At_Same_Level` returns `True` if the `Descendant` tag identifies a type that is both a descendant of the type identified by `Ancestor` and at the same accessibility level. If not, it returns `False`.
- 12.3/2 The function `Parent_Tag` returns the tag of the parent type of the type whose tag is `T`. If the type does not have a parent type (that is, it was not declared by a `derived_type_declaration`), then `No_Tag` is returned.
- 12.4/2 The function `Interface_Ancestor_Tags` returns an array containing the tag of each interface ancestor type of the type whose tag is `T`, other than `T` itself. The lower bound of the returned array is 1, and the order of the returned tags is unspecified. Each tag appears in the result exactly once. If the type whose tag is `T` has no interface ancestors, a null array is returned.
- 13 For every subtype `S` of a tagged type `T` (specific or class-wide), the following attributes are defined:
- 14 `S'Class` `S'Class` denotes a subtype of the class-wide type (called `T'Class` in this International Standard) for the class rooted at `T` (or if `S` already denotes a class-wide subtype, then `S'Class` is the same as `S`).
- 15 `S'Class` is unconstrained. However, if `S` is constrained, then the values of `S'Class` are only those that when converted to the type `T` belong to `S`.
- 16 `S'Tag` `S'Tag` denotes the tag of the type `T` (or if `T` is class-wide, the tag of the root type of the corresponding class). The value of this attribute is of type `Tag`.
- 17 Given a prefix `X` that is of a class-wide tagged type (after any implicit dereference), the following attribute is defined:
- 18 `X'Tag` `X'Tag` denotes the tag of `X`. The value of this attribute is of type `Tag`.
- 18.1/2 The following language-defined generic function exists:
- 18.2/2
- ```

generic
 type T (<>) is abstract tagged limited private;
 type Parameters (<>) is limited private;
 with function Constructor (Params : not null access Parameters)
 return T is abstract;
function Ada.Tags.Generic_Dispatching_Constructor
 (The_Tag : Tag;
 Params : not null access Parameters) return T'Class;
pragma Preelaborate(Generic_Dispatching_Constructor);
pragma Convention(Intrinsic, Generic_Dispatching_Constructor);

```
- 18.3/2 `Tags.Generic_Dispatching_Constructor` provides a mechanism to create an object of an appropriate type from just a tag value. The function `Constructor` is expected to create the object given a reference to an object of type `Parameters`.

*Dynamic Semantics*

- The tag associated with an object of a tagged type is determined as follows: 19
- The tag of a stand-alone object, a component, or an aggregate of a specific tagged type *T* identifies *T*. 20
  - The tag of an object created by an allocator for an access type with a specific designated tagged type *T*, identifies *T*. 21
  - The tag of an object of a class-wide tagged type is that of its initialization expression. 22
  - The tag of the result returned by a function whose result type is a specific tagged type *T* identifies *T*. 23
  - The tag of the result returned by a function with a class-wide result type is that of the return object. 24/2
- The tag is preserved by type conversion and by parameter passing. The tag of a value is the tag of the associated object (see 6.2). 25

Tag\_Error is raised by a call of Descendant\_Tag, Expanded\_Name, External\_Tag, Interface\_Ancessor\_Tags, Is\_Descendant\_At\_Same\_Level, Parent\_Tag, Wide\_Expanded\_Name, or Wide\_Wide\_Expanded\_Name if any tag passed is No\_Tag. 25.1/3

An instance of Tags.Generic\_Dispatching\_Constructor raises Tag\_Error if The\_Tag does not represent a concrete descendant of *T* or if the innermost master (see 7.6.1) of this descendant is not also a master of the instance. Otherwise, it dispatches to the primitive function denoted by the formal Constructor for the type identified by The\_Tag, passing Params, and returns the result. Any exception raised by the function is propagated. 25.2/2

*Erroneous Execution*

If an internal tag provided to an instance of Tags.Generic\_Dispatching\_Constructor or to any subprogram declared in package Tags identifies either a type that is not library-level and whose tag has not been created (see 13.14), or a type that does not exist in the partition at the time of the call, then execution is erroneous. 25.3/2

*Implementation Permissions*

The implementation of Internal\_Tag and Descendant\_Tag may raise Tag\_Error if no specific type corresponding to the string External passed as a parameter exists in the partition at the time the function is called, or if there is no such type whose innermost master is a master of the point of the function call. 26/2

*Implementation Advice*

Internal\_Tag should return the tag of a type, if one exists, whose innermost master is a master of the point of the function call. 26.1/3

## NOTES

66 A type declared with the reserved word **tagged** should normally be declared in a package\_specification, so that new primitive subprograms can be declared for it. 27

67 Once an object has been created, its tag never changes. 28

68 Class-wide types are defined to have unknown discriminants (see 3.7). This means that objects of a class-wide type have to be explicitly initialized (whether created by an object\_declaration or an allocator), and that aggregates have to be explicitly qualified with a specific type when their expected type is class-wide. 29

*This paragraph was deleted.* 30/2

69 The capability provided by Tags.Generic\_Dispatching\_Constructor is sometimes known as a *factory*. 30.1/2

*Examples*

31 *Examples of tagged record types:*

```
32 type Point is tagged
 record
 X, Y : Real := 0.0;
 end record;
33 type Expression is tagged null record;
 -- Components will be added by each extension
```

### 3.9.1 Type Extensions

1/2 Every type extension is a tagged type, and is a *record extension* or a *private extension* of some other tagged type, or a non-interface synchronized tagged type.

*Syntax*

2 `record_extension_part ::= with record_definition`

*Legality Rules*

3/2 The parent type of a record extension shall not be a class-wide type nor shall it be a synchronized tagged type (see 3.9.4). If the parent type or any progenitor is nonlimited, then each of the components of the `record_extension_part` shall be nonlimited. In addition to the places where Legality Rules normally apply (see 12.3), these rules apply also in the private part of an instance of a generic unit.

4/2 Within the body of a generic unit, or the body of any of its descendant library units, a tagged type shall not be declared as a descendant of a formal type declared within the formal part of the generic unit.

*Static Semantics*

4.1/2 A record extension is a *null extension* if its declaration has no `known_discriminant_part` and its `record_extension_part` includes no `component_declarations`.

*Dynamic Semantics*

5 The elaboration of a `record_extension_part` consists of the elaboration of the `record_definition`.

## NOTES

6 70 The term “type extension” refers to a type as a whole. The term “extension part” refers to the piece of text that defines the additional components (if any) the type extension has relative to its specified ancestor type.

7/2 71 When an extension is declared immediately within a body, primitive subprograms are inherited and are overridable, but new primitive subprograms cannot be added.

8 72 A name that denotes a component (including a discriminant) of the parent type is not allowed within the `record_extension_part`. Similarly, a name that denotes a component defined within the `record_extension_part` is not allowed within the `record_extension_part`. It is permissible to use a name that denotes a discriminant of the record extension, providing there is a new `known_discriminant_part` in the enclosing type declaration. (The full rule is given in 3.8.)

9 73 Each visible component of a record extension has to have a unique name, whether the component is (visibly) inherited from the parent type or declared in the `record_extension_part` (see 8.3).

*Examples*

10 *Examples of record extensions (of types defined above in 3.9):*

```
11 type Painted_Point is new Point with
 record
 Paint : Color := White;
 end record;
 -- Components X and Y are inherited
```



```

Origin : constant Painted_Point := (X | Y => 0.0, Paint => Black);
type Literal is new Expression with
 record
 Value : Real;
 end record;
type Expr_Ptr is access all Expression'Class;
type Binary_Operation is new Expression with
 record
 Left, Right : Expr_Ptr;
 end record;
type Addition is new Binary_Operation with null record;
type Subtraction is new Binary_Operation with null record;
-- No additional components needed for these extensions
Tree : Expr_Ptr :=
 new Addition'(
 Left => new Literal'(Value => 5.0),
 Right => new Subtraction'(
 Left => new Literal'(Value => 13.0),
 Right => new Literal'(Value => 7.0)));

```

### 3.9.2 Dispatching Operations of Tagged Types

The primitive subprograms of a tagged type, the subprograms declared by `formal_abstract_subprogram_declarations`, and the stream attributes of a specific tagged type that are available (see 13.13.2) at the end of the declaration list where the type is declared are called *dispatching operations*. A dispatching operation can be called using a statically determined *controlling* tag, in which case the body to be executed is determined at compile time. Alternatively, the controlling tag can be dynamically determined, in which case the call *dispatches* to a body that is determined at run time; such a call is termed a *dispatching call*. As explained below, the properties of the operands and the context of a particular call on a dispatching operation determine how the controlling tag is determined, and hence whether or not the call is a dispatching call. Run-time polymorphism is achieved when a dispatching operation is called by a dispatching call.

#### Static Semantics

A *call on a dispatching operation* is a call whose **name** or **prefix** denotes the declaration of a dispatching operation. A *controlling operand* in a call on a dispatching operation of a tagged type *T* is one whose corresponding formal parameter is of type *T* or is of an anonymous access type with designated type *T*; the corresponding formal parameter is called a *controlling formal parameter*. If the controlling formal parameter is an access parameter, the controlling operand is the object designated by the actual parameter, rather than the actual parameter itself. If the call is to a (primitive) function with result type *T* (a *function with a controlling result*), then the call has a *controlling result* — the context of the call can control the dispatching. Similarly, if the call is to a function with access result type designating *T* (a *function with a controlling access result*), then the call has a *controlling access result*, and the context can similarly control dispatching.

A **name** or expression of a tagged type is either *statically* tagged, *dynamically* tagged, or *tag indeterminate*, according to whether, when used as a controlling operand, the tag that controls dispatching is determined statically by the operand's (specific) type, dynamically by its tag at run time, or from context. A `qualified_expression` or parenthesized expression is statically, dynamically, or indeterminately tagged according to its operand. For other kinds of **names** and expressions, this is determined as follows:

- 4/2     • The **name** or expression is *statically tagged* if it is of a specific tagged type and, if it is a call with a controlling result or controlling access result, it has at least one statically tagged controlling operand;
- 5/2     • The **name** or expression is *dynamically tagged* if it is of a class-wide type, or it is a call with a controlling result or controlling access result and at least one dynamically tagged controlling operand;
- 6/2     • The **name** or expression is *tag indeterminate* if it is a call with a controlling result or controlling access result, all of whose controlling operands (if any) are tag indeterminate.
- 7/1     A **type\_conversion** is statically or dynamically tagged according to whether the type determined by the **subtype\_mark** is specific or class-wide, respectively. For an object that is designated by an expression whose expected type is an anonymous access-to-specific tagged type, the object is dynamically tagged if the expression, ignoring enclosing parentheses, is of the form *X'Access*, where *X* is of a class-wide type, or is of the form **new** *T'(...)*, where *T* denotes a class-wide subtype. Otherwise, the object is statically or dynamically tagged according to whether the designated type of the type of the expression is specific or class-wide, respectively.

#### *Legality Rules*

- 8       A call on a dispatching operation shall not have both dynamically tagged and statically tagged controlling operands.
- 9/1     If the expected type for an expression or **name** is some specific tagged type, then the expression or **name** shall not be dynamically tagged unless it is a controlling operand in a call on a dispatching operation. Similarly, if the expected type for an expression is an anonymous access-to-specific tagged type, then the object designated by the expression shall not be dynamically tagged unless it is a controlling operand in a call on a dispatching operation.
- 10/2    In the declaration of a dispatching operation of a tagged type, everywhere a subtype of the tagged type appears as a subtype of the profile (see 6.1), it shall statically match the first subtype of the tagged type. If the dispatching operation overrides an inherited subprogram, it shall be subtype conformant with the inherited subprogram. The convention of an inherited dispatching operation is the convention of the corresponding primitive operation of the parent or progenitor type. The default convention of a dispatching operation that overrides an inherited primitive operation is the convention of the inherited operation; if the operation overrides multiple inherited operations, then they shall all have the same convention. An explicitly declared dispatching operation shall not be of convention *Intrinsic*.
- 11/2    The **default\_expression** for a controlling formal parameter of a dispatching operation shall be tag indeterminate.
- 11.1/2 If a dispatching operation is defined by a **subprogram\_renaming\_declaration** or the instantiation of a generic subprogram, any access parameter of the renamed subprogram or the generic subprogram that corresponds to a controlling access parameter of the dispatching operation, shall have a subtype that excludes null.
- 12       A given subprogram shall not be a dispatching operation of two or more distinct tagged types.
- 13       The explicit declaration of a primitive subprogram of a tagged type shall occur before the type is frozen (see 13.14). For example, new dispatching operations cannot be added after objects or values of the type exist, nor after deriving a record extension from it, nor after a body.

*Dynamic Semantics*

For the execution of a call on a dispatching operation of a type *T*, the *controlling tag value* determines which subprogram body is executed. The controlling tag value is defined as follows: 14

- If one or more controlling operands are statically tagged, then the controlling tag value is *statically determined* to be the tag of *T*. 15
- If one or more controlling operands are dynamically tagged, then the controlling tag value is not statically determined, but is rather determined by the tags of the controlling operands. If there is more than one dynamically tagged controlling operand, a check is made that they all have the same tag. If this check fails, *Constraint\_Error* is raised unless the call is a *function\_call* whose *name* denotes the declaration of an equality operator (predefined or user defined) that returns Boolean, in which case the result of the call is defined to indicate inequality, and no *subprogram\_body* is executed. This check is performed prior to evaluating any tag-indeterminate controlling operands. 16
- If all of the controlling operands (if any) are tag-indeterminate, then: 17/2
  - If the call has a controlling result or controlling access result and is itself, or designates, a (possibly parenthesized or qualified) controlling operand of an enclosing call on a dispatching operation of a descendant of type *T*, then its controlling tag value is determined by the controlling tag value of this enclosing call; 18/2
  - If the call has a controlling result or controlling access result and (possibly parenthesized, qualified, or dereferenced) is the expression of an *assignment\_statement* whose target is of a class-wide type, then its controlling tag value is determined by the target; 18.1/2
  - Otherwise, the controlling tag value is statically determined to be the tag of type *T*. 19

For the execution of a call on a dispatching operation, the action performed is determined by the properties of the corresponding dispatching operation of the specific type identified by the controlling tag value: 20/3

- if the corresponding operation is explicitly declared for this type, even if the declaration occurs in a private part, then the action comprises an invocation of the explicit body for the operation; 20.1/3
- if the operation is implicitly declared for this type and is implemented by an entry or protected subprogram (see 9.1 and 9.4), then the action comprises a call on this entry or protected subprogram, with the target object being given by the first actual parameter of the call, and the actual parameters of the entry or protected subprogram being given by the remaining actual parameters of the call, if any; 20.2/3
- otherwise, the action is the same as the action for the corresponding operation of the parent type or progenitor type from which the operation was inherited. 20.3/3

## NOTES

74 The body to be executed for a call on a dispatching operation is determined by the tag; it does not matter whether that tag is determined statically or dynamically, and it does not matter whether the subprogram's declaration is visible at the place of the call. 21

75 This subclause covers calls on dispatching subprograms of a tagged type. Rules for tagged type membership tests are described in 4.5.2. Controlling tag determination for an *assignment\_statement* is described in 5.2. 22/2

76 A dispatching call can dispatch to a body whose declaration is not visible at the place of the call. 23

77 A call through an access-to-subprogram value is never a dispatching call, even if the access value designates a dispatching operation. Similarly a call whose *prefix* denotes a *subprogram\_renaming\_declaration* cannot be a dispatching call unless the renaming itself is the declaration of a primitive subprogram. 24

### 3.9.3 Abstract Types and Subprograms

- 1/2 An *abstract type* is a tagged type intended for use as an ancestor of other types, but which is not allowed to have objects of its own. An *abstract subprogram* is a subprogram that has no body, but is intended to be overridden at some point when inherited. Because objects of an abstract type cannot be created, a dispatching call to an abstract subprogram always dispatches to some overriding body.

#### Syntax

- 1.1/2 `abstract_subprogram_declaration ::=`  
`[overriding_indicator]`  
`subprogram_specification is abstract;`

#### Static Semantics

- 1.2/2 Interface types (see 3.9.4) are abstract types. In addition, a tagged type that has the reserved word **abstract** in its declaration is an abstract type. The class-wide type (see 3.4.1) rooted at an abstract type is not itself an abstract type.

#### Legality Rules

- 2/2 Only a tagged type shall have the reserved word **abstract** in its declaration.
- 3/2 A subprogram declared by an `abstract_subprogram_declaration` or a `formal_abstract_subprogram_declaration` (see 12.6) is an *abstract subprogram*. If it is a primitive subprogram of a tagged type, then the tagged type shall be abstract.
- 4/3 If a type has an implicitly declared primitive subprogram that is inherited or is the predefined equality operator, and the corresponding primitive subprogram of the parent or ancestor type is abstract or is a function with a controlling access result, or if a type other than a non-abstract null extension inherits a function with a controlling result, then:
- 5/2 • If the type is abstract or untagged, the implicitly declared subprogram is *abstract*.
  - 6/2 • Otherwise, the subprogram shall be overridden with a nonabstract subprogram or, in the case of a private extension inheriting a function with a controlling result, have a full type that is a null extension; for a type declared in the visible part of a package, the overriding may be either in the visible or the private part. Such a subprogram is said to *require overriding*. However, if the type is a generic formal type, the subprogram need not be overridden for the formal type itself; a nonabstract version will necessarily be provided by the actual type.
- 7 A call on an abstract subprogram shall be a dispatching call; nondispatching calls to an abstract subprogram are not allowed.
- 8/3 The type of an *aggregate*, or of an object created by an `object_declaration` or an `allocator`, or a generic formal object of mode **in**, shall not be abstract. The type of the target of an assignment operation (see 5.2) shall not be abstract. The type of a component shall not be abstract. If the result type of a function is abstract, then the function shall be abstract. If a function has an access result type designating an abstract type, then the function shall be abstract. The type denoted by a `return_subtype_indication` (see 6.5) shall not be abstract. A generic function shall not have an abstract result type or an access result type designating an abstract type.
- 9 If a partial view is not abstract, the corresponding full view shall not be abstract. If a generic formal type is abstract, then for each primitive subprogram of the formal that is not abstract, the corresponding primitive subprogram of the actual shall not be abstract.

For an abstract type declared in a visible part, an abstract primitive subprogram shall not be declared in the private part, unless it is overriding an abstract subprogram implicitly declared in the visible part. For a tagged type declared in a visible part, a primitive function with a controlling result or a controlling access result shall not be declared in the private part, unless it is overriding a function implicitly declared in the visible part. 10/3

A generic actual subprogram shall not be an abstract subprogram unless the generic formal subprogram is declared by a `formal_abstract_subprogram_declaration`. The prefix of an `attribute_reference` for the `Access`, `Unchecked_Access`, or `Address` attributes shall not denote an abstract subprogram. 11/2

#### Dynamic Semantics

The elaboration of an `abstract_subprogram_declaration` has no effect. 11.1/2

#### NOTES

78 Abstractness is not inherited; to declare an abstract type, the reserved word **abstract** has to be used in the declaration of the type extension. 12

79 A class-wide type is never abstract. Even if a class is rooted at an abstract type, the class-wide type for the class is not abstract, and an object of the class-wide type can be created; the tag of such an object will identify some nonabstract type in the class. 13

#### Examples

*Example of an abstract type representing a set of natural numbers:* 14

```
package Sets is
 subtype Element_Type is Natural;
 type Set is abstract tagged null record;
 function Empty return Set is abstract;
 function Union(Left, Right : Set) return Set is abstract;
 function Intersection(Left, Right : Set) return Set is abstract;
 function Unit_Set(Element : Element_Type) return Set is abstract;
 procedure Take(Element : out Element_Type;
 From : in out Set) is abstract;
end Sets;
```

15

#### NOTES

80 *Notes on the example:* Given the above abstract type, one could then derive various (nonabstract) extensions of the type, representing alternative implementations of a set. One might use a bit vector, but impose an upper bound on the largest element representable, while another might use a hash table, trading off space for flexibility. 16

## 3.9.4 Interface Types

An interface type is an abstract tagged type that provides a restricted form of multiple inheritance. A tagged type, task type, or protected type may have one or more interface types as ancestors. 1/2

#### Syntax

```
interface_type_definition ::=
 [limited | task | protected | synchronized] interface [and interface_list]
interface_list ::= interface_subtype_mark {and interface_subtype_mark}
```

2/2  
3/2

#### Static Semantics

An interface type (also called an *interface*) is a specific abstract tagged type that is defined by an `interface_type_definition`. 4/2

An interface with the reserved word **limited**, **task**, **protected**, or **synchronized** in its definition is termed, respectively, a *limited interface*, a *task interface*, a *protected interface*, or a *synchronized interface*. In 5/2

addition, all task and protected interfaces are synchronized interfaces, and all synchronized interfaces are limited interfaces.

- 6/2 A task or protected type derived from an interface is a tagged type. Such a tagged type is called a *synchronized* tagged type, as are synchronized interfaces and private extensions whose declaration includes the reserved word **synchronized**.
- 7/2 A task interface is an abstract task type. A protected interface is an abstract protected type.
- 8/2 An interface type has no components.
- 9/2 An *interface\_subtype\_mark* in an *interface\_list* names a *progenitor subtype*; its type is the *progenitor type*. An interface type inherits user-defined primitive subprograms from each progenitor type in the same way that a derived type inherits user-defined primitive subprograms from its progenitor types (see 3.4).

#### *Legality Rules*

- 10/2 All user-defined primitive subprograms of an interface type shall be abstract subprograms or null procedures.
- 11/2 The type of a subtype named in an *interface\_list* shall be an interface type.
- 12/2 A type derived from a nonlimited interface shall be nonlimited.
- 13/2 An interface derived from a task interface shall include the reserved word **task** in its definition; any other type derived from a task interface shall be a private extension or a task type declared by a task declaration (see 9.1).
- 14/2 An interface derived from a protected interface shall include the reserved word **protected** in its definition; any other type derived from a protected interface shall be a private extension or a protected type declared by a protected declaration (see 9.4).
- 15/2 An interface derived from a synchronized interface shall include one of the reserved words **task**, **protected**, or **synchronized** in its definition; any other type derived from a synchronized interface shall be a private extension, a task type declared by a task declaration, or a protected type declared by a protected declaration.
- 16/2 No type shall be derived from both a task interface and a protected interface.
- 17/2 In addition to the places where Legality Rules normally apply (see 12.3), these rules apply also in the private part of an instance of a generic unit.

#### *Dynamic Semantics*

- 18/3 The elaboration of an *interface\_type\_definition* creates the interface type and its first subtype.
- NOTES
- 19/2 81 Nonlimited interface types have predefined nonabstract equality operators. These may be overridden with user-defined abstract equality operators. Such operators will then require an explicit overriding for any nonabstract descendant of the interface.

#### *Examples*

- 20/2 *Example of a limited interface and a synchronized interface extending it:*

```

type Queue is limited interface;
procedure Append(Q : in out Queue; Person : in Person_Name) is abstract;
procedure Remove_First(Q : in out Queue;
 Person : out Person_Name) is abstract;
function Cur_Count(Q : in Queue) return Natural is abstract;
function Max_Count(Q : in Queue) return Natural is abstract;
-- See 3.10.1 for Person_Name.

Queue_Error : exception;
-- Append raises Queue_Error if Cur_Count(Q) = Max_Count(Q)
-- Remove_First raises Queue_Error if Cur_Count(Q) = 0

type Synchronized_Queue is synchronized interface and Queue; -- see 9.11
procedure Append_Wait(Q : in out Synchronized_Queue;
 Person : in Person_Name) is abstract;
procedure Remove_First_Wait(Q : in out Synchronized_Queue;
 Person : out Person_Name) is abstract;

...

procedure Transfer(From : in out Queue'Class;
 To : in out Queue'Class;
 Number : in Natural := 1) is
 Person : Person_Name;
begin
 for I in 1..Number loop
 Remove_First(From, Person);
 Append(To, Person);
 end loop;
end Transfer;

```

This defines a Queue interface defining a queue of people. (A similar design could be created to define any kind of queue simply by replacing Person\_Name by an appropriate type.) The Queue interface has four dispatching operations, Append, Remove\_First, Cur\_Count, and Max\_Count. The body of a class-wide operation, Transfer is also shown. Every non-abstract extension of Queue must provide implementations for at least its four dispatching operations, as they are abstract. Any object of a type derived from Queue may be passed to Transfer as either the From or the To operand. The two operands need not be of the same type in any given call.

The Synchronized\_Queue interface inherits the four dispatching operations from Queue and adds two additional dispatching operations, which wait if necessary rather than raising the Queue\_Error exception. This synchronized interface may only be implemented by a task or protected type, and as such ensures safe concurrent access.

*Example use of the interface:*

```

type Fast_Food_Queue is new Queue with record ...;
procedure Append(Q : in out Fast_Food_Queue; Person : in Person_Name);
procedure Remove_First(Q : in out Fast_Food_Queue; Person : out
Person_Name);
function Cur_Count(Q : in Fast_Food_Queue) return Natural;
function Max_Count(Q : in Fast_Food_Queue) return Natural;

...

Cashier, Counter : Fast_Food_Queue;

...
-- Add George (see 3.10.1) to the cashier's queue:
Append (Cashier, George);
-- After payment, move George to the sandwich counter queue:
Transfer (Cashier, Counter);
...

```

An interface such as Queue can be used directly as the parent of a new type (as shown here), or can be used as a progenitor when a type is derived. In either case, the primitive operations of the interface are inherited. For Queue, the implementation of the four inherited routines must be provided. Inside the call of

Transfer, calls will dispatch to the implementations of Append and Remove\_First for type Fast\_Food\_Queue.

34/2 *Example of a task interface:*

```
35/2 type Serial_Device is task interface; -- see 9.1
 procedure Read (Dev : in Serial_Device; C : out Character) is abstract;
 procedure Write (Dev : in Serial_Device; C : in Character) is abstract;
```

36/2 The Serial\_Device interface has two dispatching operations which are intended to be implemented by task entries (see 9.1).

## 3.10 Access Types

- 1 A value of an access type (an *access value*) provides indirect access to the object or subprogram it *designates*. Depending on its type, an access value can designate either subprograms, objects created by allocators (see 4.8), or more generally *aliased* objects of an appropriate type.

### Syntax

```
2/2 access_type_definition ::=
 [null_exclusion] access_to_object_definition
 | [null_exclusion] access_to_subprogram_definition
3 access_to_object_definition ::=
 access [general_access_modifier] subtype_indication
4 general_access_modifier ::= all | constant
5 access_to_subprogram_definition ::=
 access [protected] procedure parameter_profile
 | access [protected] function parameter_and_result_profile
5.1/2 null_exclusion ::= not null
6/2 access_definition ::=
 [null_exclusion] access [constant] subtype_mark
 | [null_exclusion] access [protected] procedure parameter_profile
 | [null_exclusion] access [protected] function parameter_and_result_profile
```

### Static Semantics

- 7/1 There are two kinds of access types, *access-to-object* types, whose values designate objects, and *access-to-subprogram* types, whose values designate subprograms. Associated with an access-to-object type is a *storage pool*; several access types may share the same storage pool. All descendants of an access type share the same storage pool. A storage pool is an area of storage used to hold dynamically allocated objects (called *pool elements*) created by allocators; storage pools are described further in 13.11, “Storage Management”.
- 8 Access-to-object types are further subdivided into *pool-specific* access types, whose values can designate only the elements of their associated storage pool, and *general* access types, whose values can designate the elements of any storage pool, as well as aliased objects created by declarations rather than allocators, and aliased subcomponents of other objects.
- 9/3 A view of an object is defined to be *aliased* if it is defined by an `object_declaration` or `component_definition` with the reserved word **aliased**, or by a renaming of an aliased view. In addition, the dereference of an access-to-object value denotes an aliased view, as does a view conversion (see 4.6) of an aliased view. The current instance of an immutably limited type (see 7.5) is defined to be aliased, as is the return



object of an `extended_return_statement` (see 6.5) that is of an immutably limited type. Finally, a formal parameter or generic formal object of a tagged type is defined to be aliased. Aliased views are the ones that can be designated by an access value.

An `access_to_object_definition` defines an access-to-object type and its first subtype; the `subtype_indication` defines the *designated subtype* of the access type. If a `general_access_modifier` appears, then the access type is a general access type. If the modifier is the reserved word **constant**, then the type is an *access-to-constant type*; a designated object cannot be updated through a value of such a type. If the modifier is the reserved word **all**, then the type is an *access-to-variable type*; a designated object can be both read and updated through a value of such a type. If no `general_access_modifier` appears in the `access_to_object_definition`, the access type is a pool-specific access-to-variable type. 10

An `access_to_subprogram_definition` defines an access-to-subprogram type and its first subtype; the `parameter_profile` or `parameter_and_result_profile` defines the *designated profile* of the access type. There is a *calling convention* associated with the designated profile; only subprograms with this calling convention can be designated by values of the access type. By default, the calling convention is “*protected*” if the reserved word **protected** appears, and “Ada” otherwise. See Annex B for how to override this default. 11

An `access_definition` defines an anonymous general access type or an anonymous access-to-subprogram type. For a general access type, the `subtype_mark` denotes its *designated subtype*; if the `general_access_modifier` **constant** appears, the type is an access-to-constant type; otherwise it is an access-to-variable type. For an access-to-subprogram type, the `parameter_profile` or `parameter_and_result_profile` denotes its *designated profile*. 12/2

For each access type, there is a null access value designating no entity at all, which can be obtained by (implicitly) converting the literal **null** to the access type. The null value of an access type is the default initial value of the type. Non-null values of an access-to-object type are obtained by evaluating an `allocator`, which returns an access value designating a newly created object (see 3.10.2), or in the case of a general access-to-object type, evaluating an `attribute_reference` for the `Access` or `Unchecked_Access` attribute of an aliased view of an object. Non-null values of an access-to-subprogram type are obtained by evaluating an `attribute_reference` for the `Access` attribute of a non-intrinsic subprogram.. 13/2

A `null_exclusion` in a construct specifies that the null value does not belong to the access subtype defined by the construct, that is, the access subtype *excludes null*. In addition, the anonymous access subtype defined by the `access_definition` for a controlling access parameter (see 3.9.2) excludes null. Finally, for a `subtype_indication` without a `null_exclusion`, the subtype denoted by the `subtype_indication` excludes null if and only if the subtype denoted by the `subtype_mark` in the `subtype_indication` excludes null. 13.1/2

All subtypes of an access-to-subprogram type are constrained. The first subtype of a type defined by an `access_definition` or an `access_to_object_definition` is unconstrained if the designated subtype is an unconstrained array or discriminated subtype; otherwise it is constrained. 14/1

#### Legality Rules

If a `subtype_indication`, `discriminant_specification`, `parameter_specification`, `parameter_and_result_profile`, `object_renaming_declaration`, or `formal_object_declaration` has a `null_exclusion`, the `subtype_mark` in that construct shall denote an access subtype that does not exclude null. 14.1/2

#### Dynamic Semantics

A `composite_constraint` is *compatible* with an unconstrained access subtype if it is compatible with the designated subtype. A `null_exclusion` is compatible with any access subtype that does not exclude null. An 15/2

access value *satisfies* a *composite\_constraint* of an access subtype if it equals the null value of its type or if it designates an object whose value satisfies the constraint. An access value satisfies an exclusion of the null value if it does not equal the null value of its type.

16 The elaboration of an *access\_type\_definition* creates the access type and its first subtype. For an access-to-object type, this elaboration includes the elaboration of the *subtype\_indication*, which creates the designated subtype.

17/2 The elaboration of an *access\_definition* creates an anonymous access type.

#### NOTES

18 82 Access values are called “pointers” or “references” in some other languages.

19 83 Each access-to-object type has an associated storage pool; several access types can share the same pool. An object can be created in the storage pool of an access type by an *allocator* (see 4.8) for the access type. A storage pool (roughly) corresponds to what some other languages call a “heap.” See 13.11 for a discussion of pools.

20 84 Only *index\_constraints* and *discriminant\_constraints* can be applied to access types (see 3.6.1 and 3.7.1).

#### Examples

21 *Examples of access-to-object types:*

```
22/2 type Peripheral_Ref is not null access Peripheral; -- see 3.8.1
 type Binop_Ptr is access all Binary_Operation'Class;
 -- general access-to-class-wide, see 3.9.1
```

23 *Example of an access subtype:*

```
24 subtype Drum_Ref is Peripheral_Ref(Drum); -- see 3.8.1
```

25 *Example of an access-to-subprogram type:*

```
26 type Message_Procedure is access procedure (M : in String := "Error!");
 procedure Default_Message_Procedure(M : in String);
 Give_Message : Message_Procedure := Default_Message_Procedure'Access;
 ...
 procedure Other_Procedure(M : in String);
 ...
 Give_Message := Other_Procedure'Access;
 ...
 Give_Message("File not found."); -- call with parameter (.all is optional)
 Give_Message.all; -- call with no parameters
```

## 3.10.1 Incomplete Type Declarations

1 There are no particular limitations on the designated type of an access type. In particular, the type of a component of the designated type can be another access type, or even the same access type. This permits mutually dependent and recursive access types. An *incomplete\_type\_declaration* can be used to introduce a type to be used as a designated type, while deferring its full definition to a subsequent *full\_type\_declaration*.

#### Syntax

```
2/2 incomplete_type_declaration ::= type defining_identifier [discriminant_part] [is tagged];
```

#### Static Semantics

2.1/2 An *incomplete\_type\_declaration* declares an *incomplete view* of a type and its first subtype; the first subtype is unconstrained if a *discriminant\_part* appears. If the *incomplete\_type\_declaration* includes the reserved word **tagged**, it declares a *tagged incomplete view*. An incomplete view of a type is a limited view of the type (see 7.5).

Given an access type *A* whose designated type *T* is an incomplete view, a dereference of a value of type *A* also has this incomplete view except when: 2.2/2

- it occurs within the immediate scope of the completion of *T*, or 2.3/2
- it occurs within the scope of a `nonlimited_with_clause` that mentions a library package in whose visible part the completion of *T* is declared, or 2.4/3
- it occurs within the scope of the completion of *T* and *T* is an incomplete view declared by an `incomplete_type_declaration`. 2.5/3

In these cases, the dereference has the view of *T* visible at the point of the dereference. 2.6/3

Similarly, if a `subtype_mark` denotes a `subtype_declaration` defining a subtype of an incomplete view *T*, the `subtype_mark` denotes an incomplete view except under the same three circumstances given above, in which case it denotes the view of *T* visible at the point of the `subtype_mark`. 2.7/3

#### Legality Rules

An `incomplete_type_declaration` requires a completion, which shall be a `type_declaration` other than an `incomplete_type_declaration`. If the `incomplete_type_declaration` occurs immediately within either the visible part of a `package_specification` or a `declarative_part`, then the `type_declaration` shall occur later and immediately within this visible part or `declarative_part`. If the `incomplete_type_declaration` occurs immediately within the private part of a given `package_specification`, then the `type_declaration` shall occur later and immediately within either the private part itself, or the `declarative_part` of the corresponding `package_body`. 3/3

If an `incomplete_type_declaration` includes the reserved word **tagged**, then a `type_declaration` that completes it shall declare a tagged type. If an `incomplete_type_declaration` has a `known_discriminant_part`, then a `type_declaration` that completes it shall have a fully conforming (explicit) `known_discriminant_part` (see 6.3.1). If an `incomplete_type_declaration` has no `discriminant_part` (or an `unknown_discriminant_part`), then a corresponding `type_declaration` is nevertheless allowed to have discriminants, either explicitly, or inherited via derivation. 4/3

A name that denotes an incomplete view of a type may be used as follows: 5/2

- as the `subtype_mark` in the `subtype_indication` of an `access_to_object_definition`; the only form of constraint allowed in this `subtype_indication` is a `discriminant_constraint` (a `null_exclusion` is not allowed); 6/3
- as the `subtype_mark` in the `subtype_indication` of a `subtype_declaration`; the `subtype_indication` shall not have a `null_exclusion` or a constraint; 7/2
- as the `subtype_mark` in an `access_definition`; 8/3
- as the `subtype_mark` defining the subtype of a parameter or result in a profile occurring within a `basic_declaration`. 8.1/3

If such a name denotes a tagged incomplete view, it may also be used: 8.2/2

- as the `subtype_mark` defining the subtype of a parameter in the profile for a `subprogram_body`, `entry_body`, or `accept_statement`; 8.3/3
- as the prefix of an `attribute_reference` whose `attribute_designator` is `Class`; such an `attribute_reference` is restricted to the uses allowed here; it denotes a tagged incomplete view. 9/2

*This paragraph was deleted.* 9.1/3

- *This paragraph was deleted.* 9.2/3

9.3/2 If any of the above uses occurs as part of the declaration of a primitive subprogram of the incomplete view, and the declaration occurs immediately within the private part of a package, then the completion of the incomplete view shall also occur immediately within the private part; it shall not be deferred to the package body.

9.4/2 No other uses of a **name** that denotes an incomplete view of a type are allowed.

10/3 A **prefix** that denotes an object shall not be of an incomplete view. An actual parameter in a call shall not be of an untagged incomplete view. The result object of a function call shall not be of an incomplete view. A **prefix** shall not denote a subprogram having a formal parameter of an untagged incomplete view, nor a return type that is an incomplete view.

*Static Semantics*

11/2 *This paragraph was deleted.*

*Dynamic Semantics*

12 The elaboration of an **incomplete\_type\_declaration** has no effect.

NOTES

13 85 Within a **declarative\_part**, an **incomplete\_type\_declaration** and a corresponding **full\_type\_declaration** cannot be separated by an intervening body. This is because a type has to be completely defined before it is frozen, and a body freezes all types declared prior to it in the same **declarative\_part** (see 13.14).

13.1/3 86 A **name** that denotes an object of an incomplete view is defined to be of a limited type. Hence, the target of an assignment statement shall not be of an incomplete view.

*Examples*

14 *Example of a recursive type:*

```

15 type Cell; -- incomplete type declaration
16 type Link is access Cell;
17 type Cell is
18 record
19 Value : Integer;
20 Succ : Link;
21 Pred : Link;
22 end record;
23 Head : Link := new Cell'(0, null, null);
24 Next : Link := Head.Succ;
```

18 *Examples of mutually dependent access types:*

```

19/2 type Person(<>); -- incomplete type declaration
20/2 type Car is tagged; -- incomplete type declaration
21/2 type Person_Name is access Person;
22/2 type Car_Name is access all Car'Class;
23/2 type Car is tagged
24/2 record
25/2 Number : Integer;
26/2 Owner : Person_Name;
27/2 end record;
```

```

type Person(Sex : Gender) is
 record
 Name : String(1 .. 20);
 Birth : Date;
 Age : Integer range 0 .. 130;
 Vehicle : Car_Name;
 case Sex is
 when M => Wife : Person_Name(Sex => F);
 when F => Husband : Person_Name(Sex => M);
 end case;
 end record;

My_Car, Your_Car, Next_Car : Car_Name := new Car; -- see 4.8
George : Person_Name := new Person(M);
...
George.Vehicle := Your_Car;

```

### 3.10.2 Operations of Access Types

The attribute `Access` is used to create access values designating aliased objects and non-intrinsic subprograms. The “accessibility” rules prevent dangling references (in the absence of uses of certain unchecked features — see Section 13).

#### *Name Resolution Rules*

For an `attribute_reference` with `attribute_designator` `Access` (or `Unchecked_Access` — see 13.10), the expected type shall be a single access type *A* such that:

- *A* is an access-to-object type with designated type *D* and the type of the prefix is *D*'Class or is covered by *D*, or
- *A* is an access-to-subprogram type whose designated profile is type conformant with that of the prefix.

The prefix of such an `attribute_reference` is never interpreted as an `implicit_dereference` or a parameterless `function_call` (see 4.1.4). The designated type or profile of the expected type of the `attribute_reference` is the expected type or profile for the prefix.

#### *Static Semantics*

The accessibility rules, which prevent dangling references, are written in terms of *accessibility levels*, which reflect the run-time nesting of *masters*. As explained in 7.6.1, a master is the execution of a certain construct, such as a `subprogram_body`. An accessibility level is *deeper than* another if it is more deeply nested at run time. For example, an object declared local to a called subprogram has a deeper accessibility level than an object declared local to the calling subprogram. The accessibility rules for access types require that the accessibility level of an object designated by an access value be no deeper than that of the access type. This ensures that the object will live at least as long as the access type, which in turn ensures that the access value cannot later designate an object that no longer exists. The `Unchecked_Access` attribute may be used to circumvent the accessibility rules.

A given accessibility level is said to be *statically deeper* than another if the given level is known at compile time (as defined below) to be deeper than the other for all possible executions. In most cases, accessibility is enforced at compile time by Legality Rules. Run-time accessibility checks are also used, since the Legality Rules do not cover certain cases involving access parameters and generic packages.

- 5 Each master, and each entity and view created by it, has an accessibility level:
- 6 • The accessibility level of a given master is deeper than that of each dynamically enclosing master, and deeper than that of each master upon which the task executing the given master directly depends (see 9.3).
  - 7/2 • An entity or view defined by a declaration and created as part of its elaboration has the same accessibility level as the innermost master of the declaration except in the cases of renaming and derived access types described below. A parameter of a master has the same accessibility level as the master.
  - 8 • The accessibility level of a view of an object or subprogram defined by a `renaming_declaration` is the same as that of the renamed view.
  - 9/2 • The accessibility level of a view conversion, `qualified_expression`, or parenthesized expression, is the same as that of the operand.
  - 10/2 • The accessibility level of an **aggregate** or the result of a function call (or equivalent use of an operator) that is used (in its entirety) to directly initialize part of an object is that of the object being initialized. In other contexts, the accessibility level of an **aggregate** or the result of a function call is that of the innermost master that evaluates the **aggregate** or function call.
  - 10.1/2 • Within a return statement, the accessibility level of the return object is that of the execution of the return statement. If the return statement completes normally by returning from the function, then prior to leaving the function, the accessibility level of the return object changes to be a level determined by the point of call, as does the level of any coextensions (see below) of the return object.
  - 11 • The accessibility level of a derived access type is the same as that of its ultimate ancestor.
  - 11.1/2 • The accessibility level of the anonymous access type defined by an `access_definition` of an `object_renaming_declaration` is the same as that of the renamed view.
  - 12/2 • The accessibility level of the anonymous access type of an access discriminant in the `subtype_indication` or `qualified_expression` of an `allocator`, or in the `expression` or `return_subtype_indication` of a return statement is determined as follows:
    - 12.1/2 • If the value of the access discriminant is determined by a `discriminant_association` in a `subtype_indication`, the accessibility level of the object or subprogram designated by the associated value (or library level if the value is null);
    - 12.2/3 • If the value of the access discriminant is determined by a `record_component_association` in an **aggregate**, the accessibility level of the object or subprogram designated by the associated value (or library level if the value is null);
    - 12.3/2 • In other cases, where the value of the access discriminant is determined by an object with an unconstrained nominal subtype, the accessibility level of the object.
  - 12.4/2 • The accessibility level of the anonymous access type of an access discriminant in any other context is that of the enclosing object.
  - 13/2 • The accessibility level of the anonymous access type of an access parameter specifying an access-to-object type is the same as that of the view designated by the actual.
  - 13.1/2 • The accessibility level of the anonymous access type of an access parameter specifying an access-to-subprogram type is deeper than that of any master; all such anonymous access types have this same level.
  - 13.2/3 • The accessibility level of the type of a stand-alone object of an anonymous access-to-object type is the same as the accessibility level of the type of the access value most recently assigned to the object, but is never deeper than that of the declaration of the stand-alone object.

- The accessibility level of an object created by an **allocator** is the same as that of the access type, except for an **allocator** of an anonymous access type that defines the value of an access parameter or an access discriminant. For an **allocator** defining the value of an access parameter, the accessibility level is that of the innermost master of the call. For one defining an access discriminant, the accessibility level is determined as follows: 14/2
  - for an **allocator** used to define the discriminant of an object, the level of the object; 14.1/3
  - for an **allocator** used to define the constraint in a **subtype\_indication** in any other context, the level of the master that elaborates the **subtype\_indication**. 14.2/3
  - *This paragraph was deleted.* 14.3/3

In the first case, the allocated object is said to be a *coextension* of the object whose discriminant designates it, as well as of any object of which the discriminated object is itself a coextension or subcomponent. If the allocated object is a coextension of an anonymous object representing the result of an aggregate or function call that is used (in its entirety) to directly initialize a part of an object, after the result is assigned, the coextension becomes a coextension of the object being initialized and is no longer considered a coextension of the anonymous object. All coextensions of an object (which have not thus been transferred by such an initialization) are finalized when the object is finalized (see 7.6.1). 14.4/3

- The accessibility level of a view of an object or subprogram designated by an access value is the same as that of the access type. 15/3
- The accessibility level of a component, protected subprogram, or entry of (a view of) a composite object is the same as that of (the view of) the composite object. 16

In the above rules, the operand of a view conversion, parenthesized expression or **qualified\_expression** is considered to be used in a context if the view conversion, parenthesized expression or **qualified\_expression** itself is used in that context. 16.1/2

One accessibility level is defined to be *statically deeper* than another in the following cases: 17

- For a master that is statically nested within another master, the accessibility level of the inner master is statically deeper than that of the outer master. 18
- The accessibility level of the anonymous access type of an access parameter specifying an access-to-subprogram type is statically deeper than that of any master; all such anonymous access types have this same level. 18.1/2
- The statically deeper relationship does not apply to the accessibility level of the anonymous type of an access parameter specifying an access-to-object type nor does it apply to a descendant of a generic formal type; that is, such an accessibility level is not considered to be statically deeper, nor statically shallower, than any other. 19/3
- The statically deeper relationship does not apply to the accessibility level of the type of a stand-alone object of an anonymous access-to-object type; that is, such an accessibility level is not considered to be statically deeper, nor statically shallower, than any other. 19.1/3
- For determining whether one level is statically deeper than another when within a generic package body, the generic package is presumed to be instantiated at the same level as where it was declared; run-time checks are needed in the case of more deeply nested instantiations. 20
- For determining whether one level is statically deeper than another when within the declarative region of a **type\_declaration**, the current instance of the type is presumed to be an object created at a deeper level than that of the type. 21

The accessibility level of all library units is called the *library level*; a library-level declaration or entity is one whose accessibility level is the library level. 22

The following attribute is defined for a **prefix** *X* that denotes an aliased view of an object:

|        |                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 23     |                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 24/1   | <b>X'Access</b> | <i>X'Access</i> yields an access value that designates the object denoted by <i>X</i> . The type of <i>X'Access</i> is an access-to-object type, as determined by the expected type. The expected type shall be a general access type. <i>X</i> shall denote an aliased view of an object, including possibly the current instance (see 8.6) of a limited type within its definition, or a formal parameter or generic formal object of a tagged type. The view denoted by the <b>prefix</b> <i>X</i> shall satisfy the following additional requirements, presuming the expected type for <i>X'Access</i> is the general access type <i>A</i> with designated type <i>D</i> :                                                                                                                                                                                                                                                                                                                               |
| 25     |                 | <ul style="list-style-type: none"> <li>• If <i>A</i> is an access-to-variable type, then the view shall be a variable; on the other hand, if <i>A</i> is an access-to-constant type, the view may be either a constant or a variable.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 26/3   |                 | <ul style="list-style-type: none"> <li>• The view shall not be a subcomponent that depends on discriminants of an object unless the object is known to be constrained.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 27/2   |                 | <ul style="list-style-type: none"> <li>• If <i>A</i> is a named access type and <i>D</i> is a tagged type, then the type of the view shall be covered by <i>D</i>; if <i>A</i> is anonymous and <i>D</i> is tagged, then the type of the view shall be either <i>D'Class</i> or a type covered by <i>D</i>; if <i>D</i> is untagged, then the type of the view shall be <i>D</i>, and either: <ul style="list-style-type: none"> <li>• the designated subtype of <i>A</i> shall statically match the nominal subtype of the view; or</li> <li>• <i>D</i> shall be discriminated in its full view and unconstrained in any partial view, and the designated subtype of <i>A</i> shall be unconstrained. For the purposes of determining within a generic body whether <i>D</i> is unconstrained in any partial view, a discriminated subtype is considered to have a constrained partial view if it is a descendant of an untagged generic formal private or derived type.</li> </ul> </li> </ul>             |
| 27.1/2 |                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 27.2/3 |                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 28/3   |                 | <ul style="list-style-type: none"> <li>• The accessibility level of the view shall not be statically deeper than that of the access type <i>A</i>.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 28.1/3 |                 | In addition to the places where Legality Rules normally apply (see 12.3), these requirements apply also in the private part of an instance of a generic unit.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 29     |                 | A check is made that the accessibility level of <i>X</i> is not deeper than that of the access type <i>A</i> . If this check fails, <code>Program_Error</code> is raised.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 30     |                 | If the nominal subtype of <i>X</i> does not statically match the designated subtype of <i>A</i> , a view conversion of <i>X</i> to the designated subtype is evaluated (which might raise <code>Constraint_Error</code> — see 4.6) and the value of <i>X'Access</i> designates that view.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 31     |                 | The following attribute is defined for a <b>prefix</b> <i>P</i> that denotes a subprogram:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 32/2   | <b>P'Access</b> | <i>P'Access</i> yields an access value that designates the subprogram denoted by <i>P</i> . The type of <i>P'Access</i> is an access-to-subprogram type ( <i>S</i> ), as determined by the expected type. The accessibility level of <i>P</i> shall not be statically deeper than that of <i>S</i> . In addition to the places where Legality Rules normally apply (see 12.3), this rule applies also in the private part of an instance of a generic unit. The profile of <i>P</i> shall be subtype-conformant with the designated profile of <i>S</i> , and shall not be Intrinsic. If the subprogram denoted by <i>P</i> is declared within a generic unit, and the expression <i>P'Access</i> occurs within the body of that generic unit or within the body of a generic unit declared within the declarative region of the generic unit, then the ultimate ancestor of <i>S</i> shall be either a non-formal type declared within the generic unit or an anonymous access type of an access parameter. |



## NOTES

- 87 The `Unchecked_Access` attribute yields the same result as the `Access` attribute for objects, but has fewer restrictions (see 13.10). There are other predefined operations that yield access values: an `allocator` can be used to create an object, and return an access value that designates it (see 4.8); evaluating the literal `null` yields a null access value that designates no entity at all (see 4.2). 33
- 88 The predefined operations of an access type also include the assignment operation, qualification, and membership tests. Explicit conversion is allowed between general access types with matching designated subtypes; explicit conversion is allowed between access-to-subprogram types with subtype conformant profiles (see 4.6). Named access types have predefined equality operators; anonymous access types do not, but they can use the predefined equality operators for *universal\_access* (see 4.5.2). 34/2
- 89 The object or subprogram designated by an access value can be named with a dereference, either an `explicit_dereference` or an `implicit_dereference`. See 4.1. 35
- 90 A call through the dereference of an access-to-subprogram value is never a dispatching call. 36
- 91 The `Access` attribute for subprograms and parameters of an anonymous access-to-subprogram type may together be used to implement “downward closures” — that is, to pass a more-nested subprogram as a parameter to a less-nested subprogram, as might be appropriate for an iterator abstraction or numerical integration. Downward closures can also be implemented using generic formal subprograms (see 12.6). Note that `Unchecked_Access` is not allowed for subprograms. 37/2
- 92 Note that using an access-to-class-wide tagged type with a dispatching operation is a potentially more structured alternative to using an access-to-subprogram type. 38
- 93 An implementation may consider two access-to-subprogram values to be unequal, even though they designate the same subprogram. This might be because one points directly to the subprogram, while the other points to a special prologue that performs an `Elaboration_Check` and then jumps to the subprogram. See 4.5.2. 39

*Examples**Example of use of the Access attribute:*

```

Martha : Person_Name := new Person(F); -- see 3.10.1
Cars : array (1..2) of aliased Car;
...
Martha.Vehicle := Cars(1)'Access;
George.Vehicle := Cars(2)'Access;
```

### 3.11 Declarative Parts

A `declarative_part` contains `declarative_items` (possibly none).*Syntax*

```

declarative_part ::= {declarative_item} 2
declarative_item ::= 3
 basic_declarative_item | body
basic_declarative_item ::= 4/1
 basic_declaration | aspect_clause | use_clause
body ::= proper_body | body_stub 5
proper_body ::= 6
 subprogram_body | package_body | task_body | protected_body
```

*Static Semantics*The list of `declarative_items` of a `declarative_part` is called the *declaration list* of the `declarative_part`. 6.1/2

*Dynamic Semantics*

- 7 The elaboration of a **declarative\_part** consists of the elaboration of the **declarative\_items**, if any, in the order in which they are given in the **declarative\_part**.
- 8 An elaborable construct is in the *elaborated* state after the normal completion of its elaboration. Prior to that, it is *not yet elaborated*.
- 9 For a construct that attempts to use a body, a check (**Elaboration\_Check**) is performed, as follows:
- 10/1
- For a call to a (non-protected) subprogram that has an explicit body, a check is made that the body is already elaborated. This check and the evaluations of any actual parameters of the call are done in an arbitrary order.
  - 11 • For a call to a protected operation of a protected type (that has a body — no check is performed if a **pragma** **Import** applies to the protected type), a check is made that the **protected\_body** is already elaborated. This check and the evaluations of any actual parameters of the call are done in an arbitrary order.
  - 12 • For the activation of a task, a check is made by the activator that the **task\_body** is already elaborated. If two or more tasks are being activated together (see 9.2), as the result of the elaboration of a **declarative\_part** or the initialization for the object created by an allocator, this check is done for all of them before activating any of them.
  - 13 • For the instantiation of a generic unit that has a body, a check is made that this body is already elaborated. This check and the evaluation of any **explicit\_generic\_actual\_parameters** of the instantiation are done in an arbitrary order.
- 14 The exception **Program\_Error** is raised if any of these checks fails.

**3.11.1 Completions of Declarations**

- 1/1 Declarations sometimes come in two parts. A declaration that requires a second part is said to *require completion*. The second part is called the *completion* of the declaration (and of the entity declared), and is either another declaration, a body, or a **pragma**. A *body* is a body, an **entry\_body**, or a renaming-as-body (see 8.5.4).

*Name Resolution Rules*

- 2 A construct that can be a completion is interpreted as the completion of a prior declaration only if:
- 3
- The declaration and the completion occur immediately within the same declarative region;
  - 4 • The defining name or **defining\_program\_unit\_name** in the completion is the same as in the declaration, or in the case of a **pragma**, the **pragma** applies to the declaration;
  - 5 • If the declaration is overloadable, then the completion either has a type-conformant profile, or is a **pragma**.

*Legality Rules*

- 6 An implicit declaration shall not have a completion. For any explicit declaration that is specified to *require completion*, there shall be a corresponding explicit completion.
- 7 At most one completion is allowed for a given declaration. Additional requirements on completions appear where each kind of completion is defined.

A type is *completely defined* at a place that is after its full type definition (if it has one) and after all of its subcomponent types are completely defined. A type shall be completely defined before it is frozen (see 13.14 and 7.3). 8

#### NOTES

94 Completions are in principle allowed for any kind of explicit declaration. However, for some kinds of declaration, the only allowed completion is a `pragma Import`, and implementations are not required to support `pragma Import` for every kind of entity. 9

95 There are rules that prevent premature uses of declarations that have a corresponding completion. The Elaboration\_Checks of 3.11 prevent such uses at run time for subprograms, protected operations, tasks, and generic units. The rules of 13.14, “Freezing Rules” prevent, at compile time, premature uses of other entities such as private types and deferred constants. 10



## Section 4: Names and Expressions

The rules applicable to the different forms of name and expression, and to their evaluation, are given in this section.

### 4.1 Names

Names can denote declared entities, whether declared explicitly or implicitly (see 3.1). Names can also denote objects or subprograms designated by access values; the results of type\_conversions or function\_calls; subcomponents and slices of objects and values; protected subprograms, single entries, entry families, and entries in families of entries. Finally, names can denote attributes of any of the foregoing.

#### Syntax

```

name ::=
 direct_name | explicit_dereference
 | indexed_component | slice
 | selected_component | attribute_reference
 | type_conversion | function_call
 | character_literal | qualified_expression

direct_name ::= identifier | operator_symbol

prefix ::= name | implicit_dereference

explicit_dereference ::= name.all

implicit_dereference ::= name

```

Certain forms of name (indexed\_components, selected\_components, slices, and attribute\_references) include a prefix that is either itself a name that denotes some related entity, or an implicit\_dereference of an access value that designates some related entity.

#### Name Resolution Rules

The name in a dereference (either an implicit\_dereference or an explicit\_dereference) is expected to be of any access type.

#### Static Semantics

If the type of the name in a dereference is some access-to-object type *T*, then the dereference denotes a view of an object, the *nominal subtype* of the view being the designated subtype of *T*. If the designated subtype has unconstrained discriminants, the (actual) subtype of the view is constrained by the values of the discriminants of the designated object, except when there is a partial view of the type of the designated subtype that does not have discriminants, in which case the dereference is not constrained by its discriminant values.

If the type of the name in a dereference is some access-to-subprogram type *S*, then the dereference denotes a view of a subprogram, the *profile* of the view being the designated profile of *S*.

#### Dynamic Semantics

The evaluation of a name determines the entity denoted by the name. This evaluation has no other effect for a name that is a direct\_name or a character\_literal.

- 12 The evaluation of a **name** that has a **prefix** includes the evaluation of the **prefix**. The evaluation of a **prefix** consists of the evaluation of the **name** or the **implicit\_dereference**. The **prefix** denotes the entity denoted by the **name** or the **implicit\_dereference**.
- 13 The evaluation of a **dereference** consists of the evaluation of the **name** and the determination of the object or subprogram that is designated by the value of the **name**. A check is made that the value of the **name** is not the null access value. **Constraint\_Error** is raised if this check fails. The **dereference** denotes the object or subprogram designated by the value of the **name**.

#### Examples

##### 14 Examples of direct names:

- |    |               |                                         |             |
|----|---------------|-----------------------------------------|-------------|
| 15 | <b>Pi</b>     | -- the direct name of a number          | (see 3.3.2) |
|    | <b>Limit</b>  | -- the direct name of a constant        | (see 3.3.1) |
|    | <b>Count</b>  | -- the direct name of a scalar variable | (see 3.3.1) |
|    | <b>Board</b>  | -- the direct name of an array variable | (see 3.6.1) |
|    | <b>Matrix</b> | -- the direct name of a type            | (see 3.6)   |
|    | <b>Random</b> | -- the direct name of a function        | (see 6.1)   |
|    | <b>Error</b>  | -- the direct name of an exception      | (see 11.1)  |

##### 16 Examples of dereferences:

- |    |                       |                                                           |
|----|-----------------------|-----------------------------------------------------------|
| 17 | <b>Next_Car.all</b>   | -- explicit dereference denoting the object designated by |
|    |                       | -- the access variable <b>Next_Car</b> (see 3.10.1)       |
|    | <b>Next_Car.Owner</b> | -- selected component with implicit dereference;          |
|    |                       | -- same as <b>Next_Car.all.Owner</b>                      |

## 4.1.1 Indexed Components

- 1 An **indexed\_component** denotes either a component of an array or an entry in a family of entries.

#### Syntax

- 2 **indexed\_component** ::= **prefix**(**expression** {, **expression**} )

#### Name Resolution Rules

- 3 The **prefix** of an **indexed\_component** with a given number of **expressions** shall resolve to denote an array (after any implicit dereference) with the corresponding number of index positions, or shall resolve to denote an entry family of a task or protected object (in which case there shall be only one **expression**).
- 4 The expected type for each **expression** is the corresponding index type.

#### Static Semantics

- 5 When the **prefix** denotes an array, the **indexed\_component** denotes the component of the array with the specified index value(s). The nominal subtype of the **indexed\_component** is the component subtype of the array type.
- 6 When the **prefix** denotes an entry family, the **indexed\_component** denotes the individual entry of the entry family with the specified index value.

#### Dynamic Semantics

- 7 For the evaluation of an **indexed\_component**, the **prefix** and the **expressions** are evaluated in an arbitrary order. The value of each **expression** is converted to the corresponding index type. A check is made that each index value belongs to the corresponding index range of the array or entry family denoted by the **prefix**. **Constraint\_Error** is raised if this check fails.

*Examples**Examples of indexed components:*

|                       |    |                                        |             |   |
|-----------------------|----|----------------------------------------|-------------|---|
| My_Schedule (Sat)     | -- | a component of a one-dimensional array | (see 3.6.1) | 8 |
| Page (10)             | -- | a component of a one-dimensional array | (see 3.6)   | 9 |
| Board (M, J + 1)      | -- | a component of a two-dimensional array | (see 3.6.1) |   |
| Page (10) (20)        | -- | a component of a component             | (see 3.6)   |   |
| Request (Medium)      | -- | an entry in a family of entries        | (see 9.1)   |   |
| Next_Frame (L) (M, N) | -- | a component of a function call         | (see 6.1)   |   |

**NOTES**

1 *Notes on the examples:* Distinct notations are used for components of multidimensional arrays (such as Board) and arrays of arrays (such as Page). The components of an array of arrays are arrays and can therefore be indexed. Thus Page(10)(20) denotes the 20th component of Page(10). In the last example Next\_Frame(L) is a function call returning an access value that designates a two-dimensional array.

**4.1.2 Slices**

A **slice** denotes a one-dimensional array formed by a sequence of consecutive components of a one-dimensional array. A **slice** of a variable is a variable; a **slice** of a constant is a constant; a **slice** of a value is a value.

*Syntax*

**slice** ::= prefix(discrete\_range)

*Name Resolution Rules*

The **prefix** of a **slice** shall resolve to denote a one-dimensional array (after any implicit dereference).

The expected type for the **discrete\_range** of a **slice** is the index type of the array type.

*Static Semantics*

A **slice** denotes a one-dimensional array formed by the sequence of consecutive components of the array denoted by the **prefix**, corresponding to the range of values of the index given by the **discrete\_range**.

The type of the **slice** is that of the **prefix**. Its bounds are those defined by the **discrete\_range**.

*Dynamic Semantics*

For the evaluation of a **slice**, the **prefix** and the **discrete\_range** are evaluated in an arbitrary order. If the **slice** is not a *null slice* (a **slice** where the **discrete\_range** is a null range), then a check is made that the bounds of the **discrete\_range** belong to the index range of the array denoted by the **prefix**. Constraint\_Error is raised if this check fails.

**NOTES**

2 A **slice** is not permitted as the **prefix** of an Access attribute\_reference, even if the components or the array as a whole are aliased. See 3.10.2.

3 For a one-dimensional array A, the **slice** A(N .. N) denotes an array that has only one component; its type is the type of A. On the other hand, A(N) denotes a component of the array A and has the corresponding component type.

## Examples

10 *Examples of slices:*

```

11 Stars(1 .. 15) -- a slice of 15 characters (see 3.6.3)
 Page(10 .. 10 + Size) -- a slice of 1 + Size components (see 3.6)
 Page(L) (A .. B) -- a slice of the array Page(L) (see 3.6)
 Stars(1 .. 0) -- a null slice (see 3.6.3)
 My_Schedule(Weekday) -- bounds given by subtype (see 3.6.1 and 3.5.1)
 Stars(5 .. 15) (K) -- same as Stars(K) (see 3.6.3)
 -- provided that K is in 5 .. 15

```

### 4.1.3 Selected Components

1 Selected\_components are used to denote components (including discriminants), entries, entry families, and protected subprograms; they are also used as expanded names as described below.

## Syntax

```

2 selected_component ::= prefix . selector_name
3 selector_name ::= identifier | character_literal | operator_symbol

```

## Name Resolution Rules

4 A **selected\_component** is called an *expanded name* if, according to the visibility rules, at least one possible interpretation of its **prefix** denotes a package or an enclosing named construct (directly, not through a **subprogram\_renaming\_declaration** or **generic\_renaming\_declaration**).

5 A **selected\_component** that is not an expanded name shall resolve to denote one of the following:

- 6 • A component (including a discriminant):
  - 7 The **prefix** shall resolve to denote an object or value of some non-array composite type (after any implicit dereference). The **selector\_name** shall resolve to denote a **discriminant\_specification** of the type, or, unless the type is a protected type, a **component\_declaration** of the type. The **selected\_component** denotes the corresponding component of the object or value.
- 8 • A single entry, an entry family, or a protected subprogram:
  - 9 The **prefix** shall resolve to denote an object or value of some task or protected type (after any implicit dereference). The **selector\_name** shall resolve to denote an **entry\_declaration** or **subprogram\_declaration** occurring (implicitly or explicitly) within the visible part of that type. The **selected\_component** denotes the corresponding entry, entry family, or protected subprogram.
- 9.1/2 • A view of a subprogram whose first formal parameter is of a tagged type or is an access parameter whose designated type is tagged:
  - 9.2/3 The **prefix** (after any implicit dereference) shall resolve to denote an object or value of a specific tagged type *T* or class-wide type *T*Class. The **selector\_name** shall resolve to denote a view of a subprogram declared immediately within the declarative region in which an ancestor of the type *T* is declared. The first formal parameter of the subprogram shall be of type *T*, or a class-wide type that covers *T*, or an access parameter designating one of these types. The designator of the subprogram shall not be the same as that of a component of the tagged type visible at the point of the **selected\_component**. The subprogram shall not be an implicitly declared primitive operation of type *T* that overrides an inherited subprogram implemented by an entry or protected subprogram visible at the point of the selected component. The **selected\_component** denotes a view of this subprogram that omits the first formal parameter. This view is called a *prefixed view* of the subprogram, and the **prefix** of the **selected\_component** (after any implicit dereference) is called the *prefix* of the prefixed view.



An expanded name shall resolve to denote a declaration that occurs immediately within a named declarative region, as follows:

- The `prefix` shall resolve to denote either a package (including the current instance of a generic package, or a rename of a package), or an enclosing named construct.
- The `selector_name` shall resolve to denote a declaration that occurs immediately within the declarative region of the package or enclosing construct (the declaration shall be visible at the place of the expanded name — see 8.3). The expanded name denotes that declaration.
- If the `prefix` does not denote a package, then it shall be a `direct_name` or an expanded name, and it shall resolve to denote a program unit (other than a package), the current instance of a type, a block\_statement, a loop\_statement, or an `accept_statement` (in the case of an `accept_statement` or `entry_body`, no family index is allowed); the expanded name shall occur within the declarative region of this construct. Further, if this construct is a callable construct and the `prefix` denotes more than one such enclosing callable construct, then the expanded name is ambiguous, independently of the `selector_name`.

#### Legality Rules

For a subprogram whose first parameter is an access parameter, the prefix of any prefixed view shall denote an aliased view of an object.

For a subprogram whose first parameter is of mode **in out** or **out**, or of an anonymous access-to-variable type, the prefix of any prefixed view shall denote a variable.

#### Dynamic Semantics

The evaluation of a `selected_component` includes the evaluation of the prefix.

For a `selected_component` that denotes a component of a `variant`, a check is made that the values of the discriminants are such that the value or object denoted by the prefix has this component. The exception `Constraint_Error` is raised if this check fails.

#### Examples

##### Examples of selected components:

|                                 |                                                         |              |
|---------------------------------|---------------------------------------------------------|--------------|
| <code>Tomorrow.Month</code>     | -- a record component                                   | (see 3.8)    |
| <code>Next_Car.Owner</code>     | -- a record component                                   | (see 3.10.1) |
| <code>Next_Car.Owner.Age</code> | -- a record component                                   | (see 3.10.1) |
|                                 | -- the previous two lines involve implicit dereferences |              |
| <code>Writer.Unit</code>        | -- a record component (a discriminant)                  | (see 3.8.1)  |
| <code>Min_Cell(H).Value</code>  | -- a record component of the result                     | (see 6.1)    |
|                                 | -- of the function call <code>Min_Cell(H)</code>        |              |
| <code>Cashier.Append</code>     | -- a prefixed view of a procedure                       | (see 3.9.4)  |
| <code>Control.Seize</code>      | -- an entry of a protected object                       | (see 9.4)    |
| <code>Pool(K).Write</code>      | -- an entry of the task <code>Pool(K)</code>            | (see 9.4)    |

##### Examples of expanded names:

|                                 |                                                 |             |
|---------------------------------|-------------------------------------------------|-------------|
| <code>Key_Manager."&lt;"</code> | -- an operator of the visible part of a package | (see 7.3.1) |
| <code>Dot_Product.Sum</code>    | -- a variable declared in a function body       | (see 6.1)   |
| <code>Buffer.Pool</code>        | -- a variable declared in a protected unit      | (see 9.11)  |
| <code>Buffer.Read</code>        | -- an entry of a protected unit                 | (see 9.11)  |
| <code>Swap.Temp</code>          | -- a variable declared in a block statement     | (see 5.6)   |
| <code>Standard.Boolean</code>   | -- the name of a predefined type                | (see A.1)   |

#### 4.1.4 Attributes

- 1 An *attribute* is a characteristic of an entity that can be queried via an *attribute\_reference* or a *range\_attribute\_reference*.

##### *Syntax*

- 2 *attribute\_reference* ::= prefix'*attribute\_designator*
- 3/2 *attribute\_designator* ::=  
 identifier[(*static\_expression*)]  
 | Access | Delta | Digits | Mod
- 4 *range\_attribute\_reference* ::= prefix'*range\_attribute\_designator*
- 5 *range\_attribute\_designator* ::= Range[(*static\_expression*)]

##### *Name Resolution Rules*

- 6 In an *attribute\_reference*, if the *attribute\_designator* is for an attribute defined for (at least some) objects of an access type, then the *prefix* is never interpreted as an *implicit\_dereference*; otherwise (and for all *range\_attribute\_references*), if the type of the name within the *prefix* is of an access type, the *prefix* is interpreted as an *implicit\_dereference*. Similarly, if the *attribute\_designator* is for an attribute defined for (at least some) functions, then the *prefix* is never interpreted as a *parameterless\_function\_call*; otherwise (and for all *range\_attribute\_references*), if the *prefix* consists of a name that denotes a function, it is interpreted as a *parameterless\_function\_call*.
- 7 The expression, if any, in an *attribute\_designator* or *range\_attribute\_designator* is expected to be of any integer type.

##### *Legality Rules*

- 8 The expression, if any, in an *attribute\_designator* or *range\_attribute\_designator* shall be static.

##### *Static Semantics*

- 9/3 An *attribute\_reference* denotes a value, an object, a subprogram, or some other kind of program entity. For an *attribute\_reference* that denotes a value or an object, if its type is scalar, then its nominal subtype is the base subtype of the type; if its type is tagged, its nominal subtype is the first subtype of the type; otherwise, its nominal subtype is a subtype of the type without any constraint or *null\_exclusion*. Similarly, unless explicitly specified otherwise, for an *attribute\_reference* that denotes a function, when its result type is scalar, its result subtype is the base subtype of the type, when its result type is tagged, the result subtype is the first subtype of the type, and when the result type is some other type, the result subtype is a subtype of the type without any constraint or *null\_exclusion*.
- 10 A *range\_attribute\_reference* X'Range(N) is equivalent to the *range* X'First(N) .. X'Last(N), except that the *prefix* is only evaluated once. Similarly, X'Range is equivalent to X'First .. X'Last, except that the *prefix* is only evaluated once.

##### *Dynamic Semantics*

- 11 The evaluation of an *attribute\_reference* (or *range\_attribute\_reference*) consists of the evaluation of the *prefix*.

*Implementation Permissions*

An implementation may provide implementation-defined attributes; the identifier for an implementation-defined attribute shall differ from those of the language-defined attributes unless supplied for compatibility with a previous edition of this International Standard. 12/1

**NOTES**

4 Attributes are defined throughout this International Standard, and are summarized in Annex K. 13

5 In general, the name in a prefix of an `attribute_reference` (or a `range_attribute_reference`) has to be resolved without using any context. However, in the case of the Access attribute, the expected type for the `attribute_reference` has to be a single access type, and the resolution of the name can use the fact that the type of the object or the profile of the callable entity denoted by the prefix has to match the designated type or be type conformant with the designated profile of the access type. 14/2

*Examples*

*Examples of attributes:* 15

|                      |                                                 |             |    |
|----------------------|-------------------------------------------------|-------------|----|
| Color'First          | -- minimum value of the enumeration type Color  | (see 3.5.1) | 16 |
| Rainbow'Base'First   | -- same as Color'First                          | (see 3.5.1) |    |
| Real'Digits          | -- precision of the type Real                   | (see 3.5.7) |    |
| Board'Last (2)       | -- upper bound of the second dimension of Board | (see 3.6.1) |    |
| Board'Range (1)      | -- index range of the first dimension of Board  | (see 3.6.1) |    |
| Pool (K) 'Terminated | -- True if task Pool(K) is terminated           | (see 9.1)   |    |
| Date'Size            | -- number of bits for records of type Date      | (see 3.8)   |    |
| Message'Address      | -- address of the record variable Message       | (see 3.7.1) |    |

## 4.2 Literals

A *literal* represents a value literally, that is, by means of notation suited to its kind. A literal is either a numeric\_literal, a character\_literal, the literal **null**, or a string\_literal. 1

*Name Resolution Rules*

*This paragraph was deleted.* 2/2

For a name that consists of a `character_literal`, either its expected type shall be a single character type, in which case it is interpreted as a parameterless `function_call` that yields the corresponding value of the character type, or its expected profile shall correspond to a parameterless function with a character result type, in which case it is interpreted as the name of the corresponding parameterless function declared as part of the character type's definition (see 3.5.1). In either case, the `character_literal` denotes the `enumeration_literal_specification`. 3

The expected type for a primary that is a `string_literal` shall be a single string type. 4

*Legality Rules*

A `character_literal` that is a name shall correspond to a `defining_character_literal` of the expected type, or of the result type of the expected profile. 5

For each character of a `string_literal` with a given expected string type, there shall be a corresponding `defining_character_literal` of the component type of the expected string type. 6

*This paragraph was deleted.* 7/2

*Static Semantics*

An integer literal is of type *universal\_integer*. A real literal is of type *universal\_real*. The literal **null** is of type *universal\_access*. 8/2

*Dynamic Semantics*

- 9 The evaluation of a numeric literal, or the literal **null**, yields the represented value.
- 10 The evaluation of a **string\_literal** that is a **primary** yields an array value containing the value of each character of the sequence of characters of the **string\_literal**, as defined in 2.6. The bounds of this array value are determined according to the rules for **positional\_array\_aggregates** (see 4.3.3), except that for a null string literal, the upper bound is the predecessor of the lower bound.
- 11 For the evaluation of a **string\_literal** of type *T*, a check is made that the value of each character of the **string\_literal** belongs to the component subtype of *T*. For the evaluation of a null string literal, a check is made that its lower bound is greater than the lower bound of the base range of the index type. The exception **Constraint\_Error** is raised if either of these checks fails.

## NOTES

- 12 6 Enumeration literals that are **identifiers** rather than **character\_literals** follow the normal rules for **identifiers** when used in a name (see 4.1 and 4.1.3). **Character\_literals** used as **selector\_names** follow the normal rules for expanded names (see 4.1.3).

*Examples*

- 13 *Examples of literals:*

- 14     3.14159\_26536        -- a real literal  
       1\_345               -- an integer literal  
       'A'                 -- a character literal  
       "Some Text"        -- a string literal

## 4.3 Aggregates

- 1 An *aggregate* combines component values into a composite value of an array type, record type, or record extension.

*Syntax*

- 2     **aggregate** ::= **record\_aggregate** | **extension\_aggregate** | **array\_aggregate**

*Name Resolution Rules*

- 3/2 The expected type for an **aggregate** shall be a single array type, record type, or record extension.

*Legality Rules*

- 4 An **aggregate** shall not be of a class-wide type.

*Dynamic Semantics*

- 5 For the evaluation of an **aggregate**, an anonymous object is created and values for the components or ancestor part are obtained (as described in the subsequent subclause for each kind of the **aggregate**) and assigned into the corresponding components or ancestor part of the anonymous object. Obtaining the values and the assignments occur in an arbitrary order. The value of the **aggregate** is the value of this object.
- 6 If an **aggregate** is of a tagged type, a check is made that its value belongs to the first subtype of the type. **Constraint\_Error** is raised if this check fails.

### 4.3.1 Record Aggregates

In a `record_aggregate`, a value is specified for each component of the record or record extension value, using either a named or a positional association. 1

#### Syntax

```

record_aggregate ::= (record_component_association_list) 2
record_component_association_list ::= 3
 record_component_association {, record_component_association}
 | null record
record_component_association ::= 4/2
 [component_choice_list =>] expression
 | component_choice_list => <>
component_choice_list ::= 5
 component_selector_name { | component_selector_name }
 | others

```

A `record_component_association` is a *named component association* if it has a `component_choice_list`; otherwise, it is a *positional component association*. Any positional component associations shall precede any named component associations. If there is a named association with a `component_choice_list` of **others**, it shall come last. 6

In the `record_component_association_list` for a `record_aggregate`, if there is only one association, it shall be a named association. 7

#### Name Resolution Rules

The expected type for a `record_aggregate` shall be a single record type or record extension. 8/2

For the `record_component_association_list` of a `record_aggregate`, all components of the composite value defined by the aggregate are *needed*; for the association list of an `extension_aggregate`, only those components not determined by the ancestor expression or subtype are needed (see 4.3.2). Each `selector_name` in a `record_component_association` shall denote a needed component (including possibly a discriminant). 9

The expected type for the expression of a `record_component_association` is the type of the *associated* component(s); the associated component(s) are as follows: 10

- For a positional association, the component (including possibly a discriminant) in the corresponding relative position (in the declarative region of the type), counting only the needed components; 11
- For a named association with one or more `component_selector_names`, the named component(s); 12
- For a named association with the reserved word **others**, all needed components that are not associated with some previous association. 13

#### Legality Rules

If the type of a `record_aggregate` is a record extension, then it shall be a descendant of a record type, through one or more record extensions (and no private extensions). 14

The reserved words **null record** may appear only if there are no components needed in a given `record_component_association_list`. 15/3

- 16/2 Each `record_component_association` other than an **others** choice with a  $\triangleleft$  shall have at least one associated component, and each needed component shall be associated with exactly one `record_component_association`. If a `record_component_association` with an `expression` has two or more associated components, all of them shall be of the same type.
- 17 If the components of a `variant_part` are needed, then the value of a discriminant that governs the `variant_part` shall be given by a static expression.
- 17.1/2 A `record_component_association` for a discriminant without a `default_expression` shall have an `expression` rather than  $\triangleleft$ .

*Dynamic Semantics*

- 18 The evaluation of a `record_aggregate` consists of the evaluation of the `record_component_association_list`.
- 19 For the evaluation of a `record_component_association_list`, any per-object constraints (see 3.8) for components specified in the association list are elaborated and any `expressions` are evaluated and converted to the subtype of the associated component. Any constraint elaborations and `expression` evaluations (and conversions) occur in an arbitrary order, except that the `expression` for a discriminant is evaluated (and converted) prior to the elaboration of any per-object constraint that depends on it, which in turn occurs prior to the evaluation and conversion of the `expression` for the component with the per-object constraint.
- 19.1/2 For a `record_component_association` with an `expression`, the `expression` defines the value for the associated component(s). For a `record_component_association` with  $\triangleleft$ , if the `component_declaration` has a `default_expression`, that `default_expression` defines the value for the associated component(s); otherwise, the associated component(s) are initialized by default as for a stand-alone object of the component subtype (see 3.3.1).
- 20 The `expression` of a `record_component_association` is evaluated (and converted) once for each associated component.

## NOTES

- 21 7 For a `record_aggregate` with positional associations, expressions specifying discriminant values appear first since the `known_discriminant_part` is given first in the declaration of the type; they have to be in the same order as in the `known_discriminant_part`.

*Examples*

- 22 *Example of a record aggregate with positional associations:*

23     (4, July, 1776) -- see 3.8

- 24 *Examples of record aggregates with named associations:*

25     (Day => 4, Month => July, Year => 1776)  
       (Month => July, Day => 4, Year => 1776)

26     (Disk, Closed, Track => 5, Cylinder => 12) -- see 3.8.1  
       (Unit => Disk, Status => Closed, Cylinder => 9, Track => 1)

- 27/2 *Examples of component associations with several choices:*

28     (Value => 0, Succ|Pred => new Cell'(0, null, null)) -- see 3.10.1  
   29     -- The allocator is evaluated twice: Succ and Pred designate different cells

29.1/2     (Value => 0, Succ|Pred =>  $\triangleleft$ ) -- see 3.10.1

29.2/2     -- Succ and Pred will be set to **null**

*Examples of record aggregates for tagged types (see 3.9 and 3.9.1):*

Expression' (**null record**)

Literal' (Value => 0.0)

Painted\_Point' (0.0, Pi/2.0, Paint => Red)

30

31

### 4.3.2 Extension Aggregates

- 1 An `extension_aggregate` specifies a value for a type that is a record extension by specifying a value or subtype for an ancestor of the type, followed by associations for any components not determined by the `ancestor_part`.

#### *Syntax*

- 2       `extension_aggregate ::=`  
           `(ancestor_part with record_component_association_list)`  
 3       `ancestor_part ::= expression | subtype_mark`

#### *Name Resolution Rules*

- 4/2 The expected type for an `extension_aggregate` shall be a single type that is a record extension. If the `ancestor_part` is an expression, it is expected to be of any tagged type.

#### *Legality Rules*

- 5/2 If the `ancestor_part` is a `subtype_mark`, it shall denote a specific tagged subtype. If the `ancestor_part` is an expression, it shall not be dynamically tagged. The type of the `extension_aggregate` shall be derived from the type of the `ancestor_part`, through one or more record extensions (and no private extensions).
- 5.1/3 If the `ancestor_part` is a function call and the type of the `ancestor_part` is limited, then the `ancestor_part` shall have a constrained nominal subtype unless there are no components needed in the `record_component_association_list`.

#### *Static Semantics*

- 6 For the `record_component_association_list` of an `extension_aggregate`, the only components *needed* are those of the composite value defined by the aggregate that are not inherited from the type of the `ancestor_part`, plus any inherited discriminants if the `ancestor_part` is a `subtype_mark` that denotes an unconstrained subtype.

#### *Dynamic Semantics*

- 7 For the evaluation of an `extension_aggregate`, the `record_component_association_list` is evaluated. If the `ancestor_part` is an expression, it is also evaluated; if the `ancestor_part` is a `subtype_mark`, the components of the value of the aggregate not given by the `record_component_association_list` are initialized by default as for an object of the ancestor type. Any implicit initializations or evaluations are performed in an arbitrary order, except that the expression for a discriminant is evaluated prior to any other evaluation or initialization that depends on it.
- 8 If the type of the `ancestor_part` has discriminants that are not inherited by the type of the `extension_aggregate`, then, unless the `ancestor_part` is a `subtype_mark` that denotes an unconstrained subtype, a check is made that each discriminant of the ancestor has the value specified for a corresponding discriminant, either in the `record_component_association_list`, or in the `derived_type_definition` for some ancestor of the type of the `extension_aggregate`. `Constraint_Error` is raised if this check fails.

#### NOTES

- 9 8 If all components of the value of the `extension_aggregate` are determined by the `ancestor_part`, then the `record_component_association_list` is required to be simply **null record**.
- 10 9 If the `ancestor_part` is a `subtype_mark`, then its type can be abstract. If its type is controlled, then as the last step of evaluating the aggregate, the Initialize procedure of the ancestor type is called, unless the Initialize procedure is abstract (see 7.6).



*Examples**Examples of extension aggregates (for types defined in 3.9.1):*

```

Painted_Point'(Point with Red)
(Point'(P) with Paint => Black)

(Expression with Left => 1.2, Right => 3.4)
Addition'(Binop with null record)
-- presuming Binop is of type Binary_Operation

```

**4.3.3 Array Aggregates**

In an `array_aggregate`, a value is specified for each component of an array, either positionally or by its index. For a `positional_array_aggregate`, the components are given in increasing-index order, with a final **others**, if any, representing any remaining components. For a `named_array_aggregate`, the components are identified by the values covered by the `discrete_choices`.

*Syntax*

```

array_aggregate ::=
 positional_array_aggregate | named_array_aggregate

positional_array_aggregate ::=
 (expression, expression {, expression})
 | (expression {, expression}, others => expression)
 | (expression {, expression}, others => <>)

named_array_aggregate ::=
 (array_component_association {, array_component_association})

array_component_association ::=
 discrete_choice_list => expression
 | discrete_choice_list => <>

```

An *n*-dimensional `array_aggregate` is one that is written as *n* levels of nested `array_aggregates` (or at the bottom level, equivalent `string_literals`). For the multidimensional case (*n* ≥ 2) the `array_aggregates` (or equivalent `string_literals`) at the *n*−1 lower levels are called *subaggregates* of the enclosing *n*-dimensional `array_aggregate`. The expressions of the bottom level subaggregates (or of the `array_aggregate` itself if one-dimensional) are called the *array component expressions* of the enclosing *n*-dimensional `array_aggregate`.

*Name Resolution Rules*

The expected type for an `array_aggregate` (that is not a subaggregate) shall be a single array type. The component type of this array type is the expected type for each array component expression of the `array_aggregate`.

The expected type for each `discrete_choice` in any `discrete_choice_list` of a `named_array_aggregate` is the type of the *corresponding index*; the corresponding index for an `array_aggregate` that is not a subaggregate is the first index of its type; for an (*n*−*m*)-dimensional subaggregate within an `array_aggregate` of an *n*-dimensional type, the corresponding index is the index in position *m*+1.

*Legality Rules*

An `array_aggregate` of an *n*-dimensional array type shall be written as an *n*-dimensional `array_aggregate`.

An **others** choice is allowed for an `array_aggregate` only if an *applicable index constraint* applies to the `array_aggregate`. An applicable index constraint is a constraint provided by certain contexts where an

`array_aggregate` is permitted that can be used to determine the bounds of the array value specified by the aggregate. Each of the following contexts (and none other) defines an applicable index constraint:

- 11/2 • For an `explicit_actual_parameter`, an `explicit_generic_actual_parameter`, the expression of a return statement, the initialization expression in an `object_declaration`, or a `default_expression` (for a parameter or a component), when the nominal subtype of the corresponding formal parameter, generic formal parameter, function return object, object, or component is a constrained array subtype, the applicable index constraint is the constraint of the subtype;
- 12 • For the expression of an `assignment_statement` where the name denotes an array variable, the applicable index constraint is the constraint of the array variable;
- 13 • For the operand of a `qualified_expression` whose `subtype_mark` denotes a constrained array subtype, the applicable index constraint is the constraint of the subtype;
- 14 • For a component expression in an `aggregate`, if the component's nominal subtype is a constrained array subtype, the applicable index constraint is the constraint of the subtype;
- 15 • For a parenthesized expression, the applicable index constraint is that, if any, defined for the expression.
- 16 The applicable index constraint *applies* to an `array_aggregate` that appears in such a context, as well as to any subaggregates thereof. In the case of an `explicit_actual_parameter` (or `default_expression`) for a call on a generic formal subprogram, no applicable index constraint is defined.
- 17 The `discrete_choice_list` of an `array_component_association` is allowed to have a `discrete_choice` that is a nonstatic expression or that is a `discrete_range` that defines a nonstatic or null range, only if it is the single `discrete_choice` of its `discrete_choice_list`, and there is only one `array_component_association` in the `array_aggregate`.
- 18 In a `named_array_aggregate` with more than one `discrete_choice`, no two `discrete_choices` are allowed to cover the same value (see 3.8.1); if there is no **others** choice, the `discrete_choices` taken together shall exactly cover a contiguous sequence of values of the corresponding index type.
- 19 A bottom level subaggregate of a multidimensional `array_aggregate` of a given array type is allowed to be a `string_literal` only if the component type of the array type is a character type; each character of such a `string_literal` shall correspond to a `defining_character_literal` of the component type.

#### *Static Semantics*

- 20 A subaggregate that is a `string_literal` is equivalent to one that is a `positional_array_aggregate` of the same length, with each expression being the `character_literal` for the corresponding character of the `string_literal`.

#### *Dynamic Semantics*

- 21 The evaluation of an `array_aggregate` of a given array type proceeds in two steps:
- 22 1. Any `discrete_choices` of this aggregate and of its subaggregates are evaluated in an arbitrary order, and converted to the corresponding index type;
- 23 2. The array component expressions of the aggregate are evaluated in an arbitrary order and their values are converted to the component subtype of the array type; an array component expression is evaluated once for each associated component.
- 23.1/2 Each expression in an `array_component_association` defines the value for the associated component(s). For an `array_component_association` with  $\diamond$ , the associated component(s) are initialized by default as for a stand-alone object of the component subtype (see 3.3.1).

The bounds of the index range of an `array_aggregate` (including a subaggregate) are determined as follows:

- For an `array_aggregate` with an **others** choice, the bounds are those of the corresponding index range from the applicable index constraint;
- For a `positional_array_aggregate` (or equivalent `string_literal`) without an **others** choice, the lower bound is that of the corresponding index range in the applicable index constraint, if defined, or that of the corresponding index subtype, if not; in either case, the upper bound is determined from the lower bound and the number of expressions (or the length of the `string_literal`);
- For a `named_array_aggregate` without an **others** choice, the bounds are determined by the smallest and largest index values covered by any `discrete_choice_list`.

For an `array_aggregate`, a check is made that the index range defined by its bounds is compatible with the corresponding index subtype.

For an `array_aggregate` with an **others** choice, a check is made that no expression or  $\langle \rangle$  is specified for an index value outside the bounds determined by the applicable index constraint.

For a multidimensional `array_aggregate`, a check is made that all subaggregates that correspond to the same index have the same bounds.

The exception `Constraint_Error` is raised if any of the above checks fail.

#### NOTES

10 In an `array_aggregate`, positional notation may only be used with two or more expressions; a single expression in parentheses is interpreted as a parenthesized expression. A `named_array_aggregate`, such as  $(1 \Rightarrow X)$ , may be used to specify an array with a single component.

#### Examples

*Examples of array aggregates with positional associations:*

```
(7, 9, 5, 1, 3, 2, 4, 8, 6, 0)
Table'(5, 8, 4, 1, others => 0) -- see 3.6
```

*Examples of array aggregates with named associations:*

```
(1 .. 5 => (1 .. 8 => 0.0)) -- two-dimensional
(1 .. N => new Cell) -- N new cells, in particular for N = 0

Table'(2 | 4 | 10 => 1, others => 0)
Schedule'(Mon .. Fri => True, others => False) -- see 3.6
Schedule'(Wed | Sun => False, others => True)
Vector'(1 => 2.5) -- single-component vector
```

*Examples of two-dimensional array aggregates:*

```
-- Three aggregates for the same value of subtype Matrix(1..2,1..3) (see 3.6):
((1.1, 1.2, 1.3), (2.1, 2.2, 2.3))
(1 => (1.1, 1.2, 1.3), 2 => (2.1, 2.2, 2.3))
(1 => (1 => 1.1, 2 => 1.2, 3 => 1.3), 2 => (1 => 2.1, 2 => 2.2, 3 => 2.3))
```

*Examples of aggregates as initial values:*

```
A : Table := (7, 9, 5, 1, 3, 2, 4, 8, 6, 0); -- A(1)=7, A(10)=0
B : Table := (2 | 4 | 10 => 1, others => 0); -- B(1)=0, B(10)=1
C : constant Matrix := (1 .. 5 => (1 .. 8 => 0.0)); -- C'Last(1)=5, C'Last(2)=8
D : Bit_Vector(M .. N) := (M .. N => True); -- see 3.6
E : Bit_Vector(M .. N) := (others => True);
F : String(1 .. 1) := (1 => 'F'); -- a one component aggregate: same as "F"
```

44/2 *Example of an array aggregate with defaulted others choice and with an applicable index constraint provided by an enclosing record aggregate:*

45/2     Buffer'(Size => 50, Pos => 1, Value => String('x', **others** => <>))    -- see 3.7

## 4.4 Expressions

- 1 An *expression* is a formula that defines the computation or retrieval of a value. In this International Standard, the term “expression” refers to a construct of the syntactic category **expression** or of any of the other five syntactic categories defined below.

### *Syntax*

```

2 expression ::=
 relation {and relation} | relation {and then relation}
 | relation {or relation} | relation {or else relation}
 | relation {xor relation}
3 relation ::=
 simple_expression [relational_operator simple_expression]
 | simple_expression [not] in range
 | simple_expression [not] in subtype_mark
4 simple_expression ::= [unary_adding_operator] term {binary_adding_operator term}
5 term ::= factor {multiplying_operator factor}
6 factor ::= primary [** primary] | abs primary | not primary
7/3 primary ::=
 numeric_literal | null | string_literal | aggregate
 | name | allocator | (expression)
 | (quantified_expression)

```

### *Name Resolution Rules*

- 8 A name used as a primary shall resolve to denote an object or a value.

### *Static Semantics*

- 9 Each expression has a type; it specifies the computation or retrieval of a value of that type.

### *Dynamic Semantics*

- 10 The value of a primary that is a name denoting an object is the value of the object.

### *Implementation Permissions*

- 11 For the evaluation of a primary that is a name denoting an object of an unconstrained numeric subtype, if the value of the object is outside the base range of its type, the implementation may either raise **Constraint\_Error** or return the value of the object.

*Examples**Examples of primaries:*

|                   |                             |    |
|-------------------|-----------------------------|----|
| 4.0               | -- real literal             | 12 |
| Pi                | -- named number             | 13 |
| (1 .. 10 => 0)    | -- array aggregate          |    |
| Sum               | -- variable                 |    |
| Integer'Last      | -- attribute                |    |
| Sine(X)           | -- function call            |    |
| Color'(Blue)      | -- qualified expression     |    |
| Real(M*N)         | -- conversion               |    |
| (Line_Count + 10) | -- parenthesized expression |    |

*Examples of expressions:*

|                                        |                                          |      |
|----------------------------------------|------------------------------------------|------|
| Volume                                 | -- primary                               | 14   |
| <b>not</b> Destroyed                   | -- factor                                | 15/2 |
| 2*Line_Count                           | -- term                                  |      |
| -4.0                                   | -- simple expression                     |      |
| -4.0 + A                               | -- simple expression                     |      |
| B**2 - 4.0*A*C                         | -- simple expression                     |      |
| R*Sin(θ)*Cos(φ)                        | -- simple expression                     |      |
| Password(1 .. 3) = "Bwv"               | -- relation                              |      |
| Count <b>in</b> Small_Int              | -- relation                              |      |
| Count <b>not in</b> Small_Int          | -- relation                              |      |
| Index = 0 <b>or</b> Item_Hit           | -- expression                            |      |
| (Cold <b>and</b> Sunny) <b>or</b> Warm | -- expression (parentheses are required) |      |
| A**(B**C)                              | -- expression (parentheses are required) |      |

## 4.5 Operators and Expression Evaluation

The language defines the following six categories of operators (given in order of increasing precedence). The corresponding `operator_symbols`, and only those, can be used as `designators` in declarations of functions for user-defined operators. See 6.6, “Overloading of Operators”.

*Syntax*

|                                 |                                     |   |
|---------------------------------|-------------------------------------|---|
| logical_operator ::=            | <b>and</b>   <b>or</b>   <b>xor</b> | 2 |
| relational_operator ::=         | =   /=   <   <=   >   >=            | 3 |
| binary_adding_operator ::=      | +   -   &                           | 4 |
| unary_adding_operator ::=       | +   -                               | 5 |
| multiplying_operator ::=        | *   /   <b>mod</b>   <b>rem</b>     | 6 |
| highest_precedence_operator ::= | **   <b>abs</b>   <b>not</b>        | 7 |

*Static Semantics*

For a sequence of operators of the same precedence level, the operators are associated with their operands in textual order from left to right. Parentheses can be used to impose specific associations.

For each form of type definition, certain of the above operators are *predefined*; that is, they are implicitly declared immediately after the type definition. For each such implicit operator declaration, the parameters are called Left and Right for *binary* operators; the single parameter is called Right for *unary* operators. An expression of the form X op Y, where op is a binary operator, is equivalent to a `function_call` of the form "op"(X, Y). An expression of the form op Y, where op is a unary operator, is equivalent to a `function_call` of the form "op"(Y). The predefined operators and their effects are described in subclauses 4.5.1 through 4.5.6.

*Dynamic Semantics*

- 10 The predefined operations on integer types either yield the mathematically correct result or raise the exception `Constraint_Error`. For implementations that support the Numerics Annex, the predefined operations on real types yield results whose accuracy is defined in Annex G, or raise the exception `Constraint_Error`.

*Implementation Requirements*

- 11 The implementation of a predefined operator that delivers a result of an integer or fixed point type may raise `Constraint_Error` only if the result is outside the base range of the result type.
- 12 The implementation of a predefined operator that delivers a result of a floating point type may raise `Constraint_Error` only if the result is outside the safe range of the result type.

*Implementation Permissions*

- 13 For a sequence of predefined operators of the same precedence level (and in the absence of parentheses imposing a specific association), an implementation may impose any association of the operators with operands so long as the result produced is an allowed result for the left-to-right association, but ignoring the potential for failure of language-defined checks in either the left-to-right or chosen order of association.

## NOTES

- 14 11 The two operands of an expression of the form `X op Y`, where `op` is a binary operator, are evaluated in an arbitrary order, as for any `function_call` (see 6.4).

*Examples*15 *Examples of precedence:*

- 16     `not` Sunny `or` Warm     -- same as (not Sunny) or Warm  
    `X > 4.0 and Y > 0.0`     -- same as (X > 4.0) and (Y > 0.0)
- 17     `-4.0*A**2`             -- same as  $-(4.0 * (A**2))$   
    `abs (1 + A) + B`        -- same as  $(abs (1 + A)) + B$   
    `Y**(-3)`                -- parentheses are necessary  
    `A / B * C`               -- same as  $(A/B)*C$   
    `A + (B + C)`            -- evaluate B + C before adding it to A

## 4.5.1 Logical Operators and Short-circuit Control Forms

*Name Resolution Rules*

- 1 An expression consisting of two relations connected by **and then** or **or else** (a *short-circuit control form*) shall resolve to be of some boolean type; the expected type for both relations is that same boolean type.

*Static Semantics*

- 2 The following logical operators are predefined for every boolean type *T*, for every modular type *T*, and for every one-dimensional array type *T* whose component type is a boolean type:

- 3     **function** "and" (Left, Right : *T*) **return** *T*  
       **function** "or" (Left, Right : *T*) **return** *T*  
       **function** "xor" (Left, Right : *T*) **return** *T*

- 4 For boolean types, the predefined logical operators **and**, **or**, and **xor** perform the conventional operations of conjunction, inclusive disjunction, and exclusive disjunction, respectively.

- 5 For modular types, the predefined logical operators are defined on a bit-by-bit basis, using the binary representation of the value of the operands to yield a binary representation for the result, where zero

represents False and one represents True. If this result is outside the base range of the type, a final subtraction by the modulus is performed to bring the result into the base range of the type.

The logical operators on arrays are performed on a component-by-component basis on matching components (as for equality — see 4.5.2), using the predefined logical operator for the component type. The bounds of the resulting array are those of the left operand.

#### Dynamic Semantics

The short-circuit control forms **and then** and **or else** deliver the same result as the corresponding predefined **and** and **or** operators for boolean types, except that the left operand is always evaluated first, and the right operand is not evaluated if the value of the left operand determines the result.

For the logical operators on arrays, a check is made that for each component of the left operand there is a matching component of the right operand, and vice versa. Also, a check is made that each component of the result belongs to the component subtype. The exception `Constraint_Error` is raised if either of the above checks fails.

#### NOTES

12 The conventional meaning of the logical operators is given by the following truth table:

| A     | B     | (A and B) | (A or B) | (A xor B) |
|-------|-------|-----------|----------|-----------|
| True  | True  | True      | True     | False     |
| True  | False | False     | True     | True      |
| False | True  | False     | True     | True      |
| False | False | False     | False    | False     |

#### Examples

*Examples of logical operators:*

```
Sunny or Warm
Filter(1 .. 10) and Filter(15 .. 24) -- see 3.6.1
```

*Examples of short-circuit control forms:*

```
Next_Car.Owner /= null and then Next_Car.Owner.Age > 25 -- see 3.10.1
N = 0 or else A(N) = Hit_Value
```

## 4.5.2 Relational Operators and Membership Tests

The *equality operators* `=` (equals) and `/=` (not equals) are predefined for nonlimited types. The other *relational operators* are the *ordering operators* `<` (less than), `<=` (less than or equal), `>` (greater than), and `>=` (greater than or equal). The ordering operators are predefined for scalar types, and for *discrete array types*, that is, one-dimensional array types whose components are of a discrete type.

A *membership test*, using **in** or **not in**, determines whether or not a value belongs to a given subtype or range, or has a tag that identifies a type that is covered by a given type. Membership tests are allowed for all types.

#### Name Resolution Rules

The *tested type* of a membership test is the type of the `range` or the type determined by the `subtype_mark`. If the tested type is tagged, then the `simple_expression` shall resolve to be of a type that is convertible (see 4.6) to the tested type; if untagged, the expected type for the `simple_expression` is the tested type.

*Legality Rules*

- 4 For a membership test, if the `simple_expression` is of a tagged class-wide type, then the tested type shall be (visibly) tagged.

*Static Semantics*

- 5 The result type of a membership test is the predefined type `Boolean`.
- 6 The equality operators are predefined for every specific type *T* that is not limited, and not an anonymous access type, with the following specifications:

```
7 function "=" (Left, Right : T) return Boolean
 function "/=" (Left, Right : T) return Boolean
```

- 7.1/2 The following additional equality operators for the *universal\_access* type are declared in package `Standard` for use with anonymous access types:

```
7.2/2 function "=" (Left, Right : universal_access) return Boolean
 function "/=" (Left, Right : universal_access) return Boolean
```

- 8 The ordering operators are predefined for every specific scalar type *T*, and for every discrete array type *T*, with the following specifications:

```
9 function "<" (Left, Right : T) return Boolean
 function "<=" (Left, Right : T) return Boolean
 function ">" (Left, Right : T) return Boolean
 function ">=" (Left, Right : T) return Boolean
```

*Name Resolution Rules*

- 9.1/2 At least one of the operands of an equality operator for *universal\_access* shall be of a specific anonymous access type. Unless the predefined equality operator is identified using an expanded name with prefix denoting the package `Standard`, neither operand shall be of an access-to-object type whose designated type is *D* or *D*'Class, where *D* has a user-defined primitive equality operator such that:
- 9.2/2 • its result type is `Boolean`;
- 9.3/3 • it is declared immediately within the same declaration list as *D* or any partial or incomplete view of *D*; and
- 9.4/2 • at least one of its operands is an access parameter with designated type *D*.

*Legality Rules*

- 9.5/2 At least one of the operands of the equality operators for *universal\_access* shall be of type *universal\_access*, or both shall be of access-to-object types, or both shall be of access-to-subprogram types. Further:
- 9.6/2 • When both are of access-to-object types, the designated types shall be the same or one shall cover the other, and if the designated types are elementary or array types, then the designated subtypes shall statically match;
- 9.7/2 • When both are of access-to-subprogram types, the designated profiles shall be subtype conformant.
- 9.8/3 The explicit declaration of a primitive equality operator of an untagged record type, if its profile is type conformant with that of the corresponding predefined equality operator, shall occur before the type is frozen. If the untagged record type has a nonlimited partial view, then the declaration shall occur in the visible part of the enclosing package. In addition to the places where Legality Rules normally apply (see 12.3), this rule applies also in the private part of an instance of a generic unit.



*Dynamic Semantics*

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |        |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| For discrete types, the predefined relational operators are defined in terms of corresponding mathematical operations on the position numbers of the values of the operands.                                                                                                                                                                                                                                                                                                        | 10     |
| For real types, the predefined relational operators are defined in terms of the corresponding mathematical operations on the values of the operands, subject to the accuracy of the type.                                                                                                                                                                                                                                                                                           | 11     |
| Two access-to-object values are equal if they designate the same object, or if both are equal to the null value of the access type.                                                                                                                                                                                                                                                                                                                                                 | 12     |
| Two access-to-subprogram values are equal if they are the result of the same evaluation of an Access <code>attribute_reference</code> , or if both are equal to the null value of the access type. Two access-to-subprogram values are unequal if they designate different subprograms. It is unspecified whether two access values that designate the same subprogram but are the result of distinct evaluations of Access <code>attribute_references</code> are equal or unequal. | 13     |
| For a type extension, predefined equality is defined in terms of the primitive (possibly user-defined) equals operator for the parent type and for any components that have a record type in the extension part, and predefined equality for any other components not inherited from the parent type.                                                                                                                                                                               | 14/3   |
| For a derived type whose parent is an untagged record type, predefined equality is defined in terms of the primitive (possibly user-defined) equals operator of the parent type.                                                                                                                                                                                                                                                                                                    | 14.1/3 |
| For a private type, if its full type is a record type, predefined equality is defined in terms of the primitive equals operator of the full type; if the full type is untagged, predefined equality for the private type is that of its full type.                                                                                                                                                                                                                                  | 15/3   |
| For other composite types, the predefined equality operators (and certain other predefined operations on composite types — see 4.5.1 and 4.6) are defined in terms of the corresponding operation on <i>matching components</i> , defined as follows:                                                                                                                                                                                                                               | 16     |
| <ul style="list-style-type: none"> <li>For two composite objects or values of the same non-array type, matching components are those that correspond to the same <code>component_declaration</code> or <code>discriminant_specification</code>;</li> </ul>                                                                                                                                                                                                                          | 17     |
| <ul style="list-style-type: none"> <li>For two one-dimensional arrays of the same type, matching components are those (if any) whose index values match in the following sense: the lower bounds of the index ranges are defined to match, and the successors of matching indices are defined to match;</li> </ul>                                                                                                                                                                  | 18     |
| <ul style="list-style-type: none"> <li>For two multidimensional arrays of the same type, matching components are those whose index values match in successive index positions.</li> </ul>                                                                                                                                                                                                                                                                                           | 19     |
| The analogous definitions apply if the types of the two objects or values are convertible, rather than being the same.                                                                                                                                                                                                                                                                                                                                                              | 20     |
| Given the above definition of matching components, the result of the predefined equals operator for composite types (other than for those composite types covered earlier) is defined as follows:                                                                                                                                                                                                                                                                                   | 21     |
| <ul style="list-style-type: none"> <li>If there are no components, the result is defined to be True;</li> </ul>                                                                                                                                                                                                                                                                                                                                                                     | 22     |
| <ul style="list-style-type: none"> <li>If there are unmatched components, the result is defined to be False;</li> </ul>                                                                                                                                                                                                                                                                                                                                                             | 23     |
| <ul style="list-style-type: none"> <li>Otherwise, the result is defined in terms of the primitive equals operator for any matching components that are records, and the predefined equals for any other matching components.</li> </ul>                                                                                                                                                                                                                                             | 24/3   |
| If the primitive equals operator for an untagged record type is abstract, then <code>Program_Error</code> is raised at the point of any (implicit) call to that abstract subprogram.                                                                                                                                                                                                                                                                                                | 24.1/3 |

- 24.2/1 For any composite type, the order in which "=" is called for components is unspecified. Furthermore, if the result can be determined before calling "=" on some components, it is unspecified whether "=" is called on those components.
- 25 The predefined "/=" operator gives the complementary result to the predefined "=" operator.
- 26 For a discrete array type, the predefined ordering operators correspond to *lexicographic order* using the predefined order relation of the component type: A null array is lexicographically less than any array having at least one component. In the case of nonnull arrays, the left operand is lexicographically less than the right operand if the first component of the left operand is less than that of the right; otherwise the left operand is lexicographically less than the right operand only if their first components are equal and the tail of the left operand is lexicographically less than that of the right (the *tail* consists of the remaining components beyond the first and can be null).
- 27 For the evaluation of a membership test, the `simple_expression` and the `range` (if any) are evaluated in an arbitrary order.
- 28 A membership test using `in` yields the result `True` if:
- 29 • The tested type is scalar, and the value of the `simple_expression` belongs to the given range, or the range of the named subtype; or
  - 30/2 • The tested type is not scalar, and the value of the `simple_expression` satisfies any constraints of the named subtype, and:
    - 30.1/2 • if the type of the `simple_expression` is class-wide, the value has a tag that identifies a type covered by the tested type;
    - 30.2/3 • if the tested type is an access type and the named subtype excludes null, the value of the `simple_expression` is not null;
    - 30.3/3 • if the tested type is a general access-to-object type, the type of the `simple_expression` is convertible to the tested type and its accessibility level is no deeper than that of the tested type; further, if the designated type is tagged and the `simple_expression` is non-null, the tag of the object designated by the value of the `simple_expression` is covered by the designated type of the tested type.
- 31 Otherwise the test yields the result `False`.
- 32 A membership test using `not in` gives the complementary result to the corresponding membership test using `in`.

#### *Implementation Requirements*

- 32.1/1 For all nonlimited types declared in language-defined packages, the "=" and "/=" operators of the type shall behave as if they were the predefined equality operators for the purposes of the equality of composite types and generic formal types.

#### NOTES

- 33/2 *This paragraph was deleted.*
- 34 13 If a composite type has components that depend on discriminants, two values of this type have matching components if and only if their discriminants are equal. Two nonnull arrays have matching components if and only if the length of each dimension is the same for both.

#### *Examples*

- 35 *Examples of expressions involving relational operators and membership tests:*

36     `X /= Y`

```

" " < "A" and "A" < "Aa" -- True 37
"Aa" < "B" and "A" < "A " -- True
My_Car = null -- true if My_Car has been set to null (see 3.10.1)
My_Car = Your_Car -- true if we both share the same car
My_Car.all = Your_Car.all -- true if the two cars are identical
N not in 1 .. 10 -- range membership test
Today in Mon .. Fri -- range membership test
Today in Weekday -- subtype membership test (see 3.5.1)
Archive in Disk_Unit -- subtype membership test (see 3.8.1)
Tree.all in Addition'Class -- class membership test (see 3.9.1)

```

## 4.5.3 Binary Adding Operators

### Static Semantics

The binary adding operators + (addition) and – (subtraction) are predefined for every specific numeric type *T* with their conventional meaning. They have the following specifications:

```

function "+" (Left, Right : T) return T
function "-" (Left, Right : T) return T

```

The concatenation operators & are predefined for every nonlimited, one-dimensional array type *T* with component type *C*. They have the following specifications:

```

function "&" (Left : T; Right : T) return T
function "&" (Left : T; Right : C) return T
function "&" (Left : C; Right : T) return T
function "&" (Left : C; Right : C) return T

```

### Dynamic Semantics

For the evaluation of a concatenation with result type *T*, if both operands are of type *T*, the result of the concatenation is a one-dimensional array whose length is the sum of the lengths of its operands, and whose components comprise the components of the left operand followed by the components of the right operand. If the left operand is a null array, the result of the concatenation is the right operand. Otherwise, the lower bound of the result is determined as follows:

- If the ultimate ancestor of the array type was defined by a `constrained_array_definition`, then the lower bound of the result is that of the index subtype;
- If the ultimate ancestor of the array type was defined by an `unconstrained_array_definition`, then the lower bound of the result is that of the left operand.

The upper bound is determined by the lower bound and the length. A check is made that the upper bound of the result of the concatenation belongs to the range of the index subtype, unless the result is a null array. `Constraint_Error` is raised if this check fails.

If either operand is of the component type *C*, the result of the concatenation is given by the above rules, using in place of such an operand an array having this operand as its only component (converted to the component subtype) and having the lower bound of the index subtype of the array type as its lower bound.

The result of a concatenation is defined in terms of an assignment to an anonymous object, as for any function call (see 6.5).

### NOTES

14 As for all predefined operators on modular types, the binary adding operators + and – on modular types include a final reduction modulo the modulus if the result is outside the base range of the type.

*Examples*

*Examples of expressions involving binary adding operators:*

```

Z + 0.1 -- Z has to be of a real type
"A" & "BCD" -- concatenation of two string literals
'A' & "BCD" -- concatenation of a character literal and a string literal
'A' & 'A' -- concatenation of two character literals

```

## 4.5.4 Unary Adding Operators

*Static Semantics*

The unary adding operators + (identity) and – (negation) are predefined for every specific numeric type *T* with their conventional meaning. They have the following specifications:

```

function "+" (Right : T) return T
function "-" (Right : T) return T

```

**NOTES**

15 For modular integer types, the unary adding operator –, when given a nonzero operand, returns the result of subtracting the value of the operand from the modulus; for a zero operand, the result is zero.

## 4.5.5 Multiplying Operators

*Static Semantics*

The multiplying operators \* (multiplication), / (division), **mod** (modulus), and **rem** (remainder) are predefined for every specific integer type *T*:

```

function "*" (Left, Right : T) return T
function "/" (Left, Right : T) return T
function "mod" (Left, Right : T) return T
function "rem" (Left, Right : T) return T

```

Signed integer multiplication has its conventional meaning.

Signed integer division and remainder are defined by the relation:

$$A = (A/B) * B + (A \text{ rem } B)$$

where  $(A \text{ rem } B)$  has the sign of *A* and an absolute value less than the absolute value of *B*. Signed integer division satisfies the identity:

$$(-A) / B = -(A / B) = A / (-B)$$

The signed integer modulus operator is defined such that the result of  $A \text{ mod } B$  has the sign of *B* and an absolute value less than the absolute value of *B*; in addition, for some signed integer value *N*, this result satisfies the relation:

$$A = B * N + (A \text{ mod } B)$$

The multiplying operators on modular types are defined in terms of the corresponding signed integer operators, followed by a reduction modulo the modulus if the result is outside the base range of the type (which is only possible for the "\*" operator).

Multiplication and division operators are predefined for every specific floating point type *T*:

```

function "*" (Left, Right : T) return T
function "/" (Left, Right : T) return T

```

The following multiplication and division operators, with an operand of the predefined type Integer, are predefined for every specific fixed point type *T*:

```

function "*" (Left : T; Right : Integer) return T
function "*" (Left : Integer; Right : T) return T
function "/" (Left : T; Right : Integer) return T

```

All of the above multiplying operators are usable with an operand of an appropriate universal numeric type. The following additional multiplying operators for *root\_real* are predefined, and are usable when both operands are of an appropriate universal or root numeric type, and the result is allowed to be of type *root\_real*, as in a *number\_declaration*:

```

function "*" (Left, Right : root_real) return root_real
function "/" (Left, Right : root_real) return root_real
function "*" (Left : root_real; Right : root_integer) return root_real
function "*" (Left : root_integer; Right : root_real) return root_real
function "/" (Left : root_real; Right : root_integer) return root_real

```

Multiplication and division between any two fixed point types are provided by the following two predefined operators:

```

function "*" (Left, Right : universal_fixed) return universal_fixed
function "/" (Left, Right : universal_fixed) return universal_fixed

```

#### *Name Resolution Rules*

The above two fixed-fixed multiplying operators shall not be used in a context where the expected type for the result is itself *universal\_fixed* — the context has to identify some other numeric type to which the result is to be converted, either explicitly or implicitly. Unless the predefined universal operator is identified using an expanded name with prefix denoting the package Standard, an explicit conversion is required on the result when using the above fixed-fixed multiplication operator if either operand is of a type having a user-defined primitive multiplication operator such that:

- it is declared immediately within the same declaration list as the type or any partial view thereof; and
- both of its formal parameters are of a fixed-point type.

A corresponding requirement applies to the universal fixed-fixed division operator.

#### *Legality Rules*

*This paragraph was deleted.*

#### *Dynamic Semantics*

The multiplication and division operators for real types have their conventional meaning. For floating point types, the accuracy of the result is determined by the precision of the result type. For decimal fixed point types, the result is truncated toward zero if the mathematical result is between two multiples of the *small* of the specific result type (possibly determined by context); for ordinary fixed point types, if the mathematical result is between two multiples of the *small*, it is unspecified which of the two is the result.

The exception *Constraint\_Error* is raised by integer division, **rem**, and **mod** if the right operand is zero. Similarly, for a real type *T* with *T*Machine\_Overflows True, division by zero raises *Constraint\_Error*.

#### NOTES

16 For positive A and B, A/B is the quotient and A **rem** B is the remainder when A is divided by B. The following relations are satisfied by the rem operator:

```

A rem (-B) = A rem B
(-A) rem B = -(A rem B)

```

17 For any signed integer K, the following identity holds:

```

A mod B = (A + K*B) mod B

```

27 The relations between signed integer division, remainder, and modulus are illustrated by the following table:

| 28 | A  | B  | A/B | A <b>rem</b> B | A <b>mod</b> B | A   | B  | A/B | A <b>rem</b> B | A <b>mod</b> B |
|----|----|----|-----|----------------|----------------|-----|----|-----|----------------|----------------|
| 29 | 10 | 5  | 2   | 0              | 0              | -10 | 5  | -2  | 0              | 0              |
|    | 11 | 5  | 2   | 1              | 1              | -11 | 5  | -2  | -1             | 4              |
|    | 12 | 5  | 2   | 2              | 2              | -12 | 5  | -2  | -2             | 3              |
|    | 13 | 5  | 2   | 3              | 3              | -13 | 5  | -2  | -3             | 2              |
|    | 14 | 5  | 2   | 4              | 4              | -14 | 5  | -2  | -4             | 1              |
| 30 | A  | B  | A/B | A <b>rem</b> B | A <b>mod</b> B | A   | B  | A/B | A <b>rem</b> B | A <b>mod</b> B |
|    | 10 | -5 | -2  | 0              | 0              | -10 | -5 | 2   | 0              | 0              |
|    | 11 | -5 | -2  | 1              | -4             | -11 | -5 | 2   | -1             | -1             |
|    | 12 | -5 | -2  | 2              | -3             | -12 | -5 | 2   | -2             | -2             |
|    | 13 | -5 | -2  | 3              | -2             | -13 | -5 | 2   | -3             | -3             |
|    | 14 | -5 | -2  | 4              | -1             | -14 | -5 | 2   | -4             | -4             |

#### Examples

31 *Examples of expressions involving multiplying operators:*

```

32 I : Integer := 1;
 J : Integer := 2;
 K : Integer := 3;

33 X : Real := 1.0; -- see 3.5.7
 Y : Real := 2.0;

34 F : Fraction := 0.25; -- see 3.5.9
 G : Fraction := 0.5;
```

| 35 | Expression     | Value | Result Type                                                     |
|----|----------------|-------|-----------------------------------------------------------------|
|    | I*J            | 2     | same as I and J, that is, Integer                               |
|    | K/J            | 1     | same as K and J, that is, Integer                               |
|    | K <b>mod</b> J | 1     | same as K and J, that is, Integer                               |
|    | X/Y            | 0.5   | same as X and Y, that is, Real                                  |
|    | F/2            | 0.125 | same as F, that is, Fraction                                    |
|    | 3*F            | 0.75  | same as F, that is, Fraction                                    |
|    | 0.75*G         | 0.375 | universal_fixed, implicitly convertible to any fixed point type |
|    | Fraction(F*G)  | 0.125 | Fraction, as stated by the conversion                           |
|    | Real(J)*Y      | 4.0   | Real, the type of both operands after conversion of J           |

## 4.5.6 Highest Precedence Operators

### Static Semantics

1 The highest precedence unary operator **abs** (absolute value) is predefined for every specific numeric type *T*, with the following specification:

```
2 function "abs"(Right : T) return T
```

3 The highest precedence unary operator **not** (logical negation) is predefined for every boolean type *T*, every modular type *T*, and for every one-dimensional array type *T* whose components are of a boolean type, with the following specification:

```
4 function "not"(Right : T) return T
```

5 The result of the operator **not** for a modular type is defined as the difference between the high bound of the base range of the type and the value of the operand. For a binary modulus, this corresponds to a bit-wise complement of the binary representation of the value of the operand.

The operator **not** that applies to a one-dimensional array of boolean components yields a one-dimensional boolean array with the same bounds; each component of the result is obtained by logical negation of the corresponding component of the operand (that is, the component that has the same index value). A check is made that each component of the result belongs to the component subtype; the exception `Constraint_Error` is raised if this check fails.

The highest precedence *exponentiation* operator **\*\*** is predefined for every specific integer type *T* with the following specification:

```
function "*" (Left : T; Right : Natural) return T
```

Exponentiation is also predefined for every specific floating point type as well as *root\_real*, with the following specification (where *T* is *root\_real* or the floating point type):

```
function "*" (Left : T; Right : Integer'Base) return T
```

The right operand of an exponentiation is the *exponent*. The value of  $X^{**}N$  with the value of the exponent *N* positive is the same as the value of  $X * X * \dots * X$  (with *N*–1 multiplications) except that the multiplications are associated in an arbitrary order. With *N* equal to zero, the result is one. With the value of *N* negative (only defined for a floating point operand), the result is the reciprocal of the result using the absolute value of *N* as the exponent.

#### Implementation Permissions

The implementation of exponentiation for the case of a negative exponent is allowed to raise `Constraint_Error` if the intermediate result of the repeated multiplications is outside the safe range of the type, even though the final result (after taking the reciprocal) would not be. (The best machine approximation to the final result in this case would generally be 0.0.)

#### NOTES

18 As implied by the specification given above for exponentiation of an integer type, a check is made that the exponent is not negative. `Constraint_Error` is raised if this check fails.

## 4.5.7 Quantified Expressions

#### Syntax

```
quantified_expression ::= for quantifier loop_parameter_specification | predicate
```

```
quantifier ::= all | Some
```

```
predicate ::= boolean_expression
```

Wherever the Syntax Rules allow an *expression*, a *quantified\_expression* may be used in place of the *expression*, so long as it is immediately surrounded by parentheses.

#### Name Resolution Rules

The expected type of a *quantified\_expression* is any Boolean type. The *predicate* in a *quantified\_expression* is expected to be of the same type.

#### Dynamic Semantics

For the evaluation of a *quantified\_expression*, the *loop\_parameter\_specification* is first elaborated. The evaluation of a *quantified\_expression* then evaluates the *predicate* for each value of the loop parameter. These values are examined in the order specified by the *loop\_parameter\_specification* (see 5.5).

The value of the *quantified\_expression* is determined as follows:

- 8/3 • If the **quantifier** is **all**, the expression is True if the evaluation of the **predicate** yields True for each value of the loop parameter. It is False otherwise. Evaluation of the **quantified\_expression** stops when all values of the domain have been examined, or when the **predicate** yields False for a given value. Any exception raised by evaluation of the **predicate** is propagated.
- 9/3 • If the **quantifier** is **Some**, the expression is True if the evaluation of the **predicate** yields True for some value of the loop parameter. It is False otherwise. Evaluation of the **quantified\_expression** stops when all values of the domain have been examined, or when the **predicate** yields True for a given value. Any exception raised by evaluation of the **predicate** is propagated.
- NOTES
- 10/3 19 Some is not a reserved word.

#### Examples

- 11/3 The postcondition for a sorting routine on an array A with an index subtype T can be written:
- 12/3 `Post => (for all I in A'First .. T'Pred(A'Last) | A (I) <= A (T'Succ (I)))`
- 13/3 The assertion that a positive number is composite (as opposed to prime) can be written:
- 14/3 `pragma Assert (for some X in 2 .. N / 2 | N mod X = 0);`

## 4.6 Type Conversions

- 1 Explicit type conversions, both value conversions and view conversions, are allowed between closely related types as defined below. This clause also defines rules for value and view conversions to a particular subtype of a type, both explicit ones and those implicit in other constructs.

#### Syntax

- 2 `type_conversion ::=`  
`subtype_mark(expression)`  
`| subtype_mark(name)`
- 3 The *target subtype* of a **type\_conversion** is the subtype denoted by the **subtype\_mark**. The *operand* of a **type\_conversion** is the expression or name within the parentheses; its type is the *operand type*.
- 4 One type is *convertible* to a second type if a **type\_conversion** with the first type as operand type and the second type as target type is legal according to the rules of this clause. Two types are convertible if each is convertible to the other.
- 5/2 A **type\_conversion** whose operand is the name of an object is called a *view conversion* if both its target type and operand type are tagged, or if it appears in a call as an actual parameter of mode **out** or **in out**; other **type\_conversions** are called *value conversions*.

#### Name Resolution Rules

- 6 The operand of a **type\_conversion** is expected to be of any type.
- 7 The operand of a view conversion is interpreted only as a **name**; the operand of a value conversion is interpreted as an **expression**.

#### Legality Rules

- 8/2 In a view conversion for an untagged type, the target type shall be convertible (back) to the operand type.

*Paragraphs 9 through 20 were reorganized and moved below.*



- If there is a type that is an ancestor of both the target type and the operand type, or both types are class-wide types, then at least one of the following rules shall apply: 21/2
- The target type shall be untagged; or 21.1/2
  - The operand type shall be covered by or descended from the target type; or 22
  - The operand type shall be a class-wide type that covers the target type; or 23/2
  - The operand and target types shall both be class-wide types and the specific type associated with at least one of them shall be an interface type. 23.1/2
- If there is no type that is the ancestor of both the target type and the operand type, and they are not both class-wide types, one of the following rules shall apply: 24/2
- If the target type is a numeric type, then the operand type shall be a numeric type. 24.1/2
  - If the target type is an array type, then the operand type shall be an array type. Further: 24.2/2
    - The types shall have the same dimensionality; 24.3/2
    - Corresponding index types shall be convertible; 24.4/2
    - The component subtypes shall statically match; 24.5/2
    - If the component types are anonymous access types, then the accessibility level of the operand type shall not be statically deeper than that of the target type; 24.6/2
    - Neither the target type nor the operand type shall be limited; 24.7/2
    - If the target type of a view conversion has aliased components, then so shall the operand type; and 24.8/2
    - The operand type of a view conversion shall not have a tagged, private, or volatile subcomponent. 24.9/2
  - If the target type is *universal\_access*, then the operand type shall be an access type. 24.10/2
  - If the target type is a general access-to-object type, then the operand type shall be *universal\_access* or an access-to-object type. Further, if the operand type is not *universal\_access*: 24.11/2
    - If the target type is an access-to-variable type, then the operand type shall be an access-to-variable type; 24.12/2
    - If the target designated type is tagged, then the operand designated type shall be convertible to the target designated type; 24.13/2
    - If the target designated type is not tagged, then the designated types shall be the same, and either: 24.14/2
      - the designated subtypes shall statically match; or 24.15/2
      - the designated type shall be discriminated in its full view and unconstrained in any partial view, and one of the designated subtypes shall be unconstrained; 24.16/2
    - The accessibility level of the operand type shall not be statically deeper than that of the target type, unless the target is a stand-alone object of an anonymous access type. If the target is such a stand-alone object, the accessibility level of the operand type shall not be statically deeper than that of the declaration of the stand-alone object. In addition to the places where Legality Rules normally apply (see 12.3), this rule applies also in the private part of an instance of a generic unit. 24.17/3
  - If the target type is a pool-specific access-to-object type, then the operand type shall be *universal\_access*. 24.18/2

- 24.19/2 • If the target type is an access-to-subprogram type, then the operand type shall be *universal\_access* or an access-to-subprogram type. Further, if the operand type is not *universal\_access*:
- 24.20/2 • The designated profiles shall be subtype-conformant.
- 24.21/2 • The accessibility level of the operand type shall not be statically deeper than that of the target type. In addition to the places where Legality Rules normally apply (see 12.3), this rule applies also in the private part of an instance of a generic unit. If the operand type is declared within a generic body, the target type shall be declared within the generic body.

*Static Semantics*

- 25 A *type\_conversion* that is a value conversion denotes the value that is the result of converting the value of the operand to the target subtype.
- 26 A *type\_conversion* that is a view conversion denotes a view of the object denoted by the operand. This view is a variable of the target type if the operand denotes a variable; otherwise it is a constant of the target type.
- 27 The nominal subtype of a *type\_conversion* is its target subtype.

*Dynamic Semantics*

- 28 For the evaluation of a *type\_conversion* that is a value conversion, the operand is evaluated, and then the value of the operand is *converted* to a *corresponding* value of the target type, if any. If there is no value of the target type that corresponds to the operand value, *Constraint\_Error* is raised; this can only happen on conversion to a modular type, and only when the operand value is outside the base range of the modular type. Additional rules follow:
- 29 • **Numeric Type Conversion**
    - 30 • If the target and the operand types are both integer types, then the result is the value of the target type that corresponds to the same mathematical integer as the operand.
    - 31 • If the target type is a decimal fixed point type, then the result is truncated (toward 0) if the value of the operand is not a multiple of the *small* of the target type.
    - 32 • If the target type is some other real type, then the result is within the accuracy of the target type (see G.2, “Numeric Performance Requirements”, for implementations that support the Numerics Annex).
    - 33 • If the target type is an integer type and the operand type is real, the result is rounded to the nearest integer (away from zero if exactly halfway between two integers).
  - 34 • **Enumeration Type Conversion**
    - 35 • The result is the value of the target type with the same position number as that of the operand value.
  - 36 • **Array Type Conversion**
    - 37 • If the target subtype is a constrained array subtype, then a check is made that the length of each dimension of the value of the operand equals the length of the corresponding dimension of the target subtype. The bounds of the result are those of the target subtype.
    - 38 • If the target subtype is an unconstrained array subtype, then the bounds of the result are obtained by converting each bound of the value of the operand to the corresponding index type of the target type. For each nonnull index range, a check is made that the bounds of the range belong to the corresponding index subtype.
    - 39 • In either array case, the value of each component of the result is that of the matching component of the operand value (see 4.5.2).

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |        |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| <ul style="list-style-type: none"> <li>If the component types of the array types are anonymous access types, then a check is made that the accessibility level of the operand type is not deeper than that of the target type.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                             | 39.1/2 |
| <ul style="list-style-type: none"> <li>Composite (Non-Array) Type Conversion</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | 40     |
| <ul style="list-style-type: none"> <li> <ul style="list-style-type: none"> <li>The value of each nondiscriminant component of the result is that of the matching component of the operand value.</li> </ul> </li> </ul>                                                                                                                                                                                                                                                                                                                                                                                               | 41     |
| <ul style="list-style-type: none"> <li> <ul style="list-style-type: none"> <li>The tag of the result is that of the operand. If the operand type is class-wide, a check is made that the tag of the operand identifies a (specific) type that is covered by or descended from the target type.</li> </ul> </li> </ul>                                                                                                                                                                                                                                                                                                 | 42     |
| <ul style="list-style-type: none"> <li> <ul style="list-style-type: none"> <li>For each discriminant of the target type that corresponds to a discriminant of the operand type, its value is that of the corresponding discriminant of the operand value; if it corresponds to more than one discriminant of the operand type, a check is made that all these discriminants are equal in the operand value.</li> </ul> </li> </ul>                                                                                                                                                                                    | 43     |
| <ul style="list-style-type: none"> <li> <ul style="list-style-type: none"> <li>For each discriminant of the target type that corresponds to a discriminant that is specified by the <code>derived_type_definition</code> for some ancestor of the operand type (or if class-wide, some ancestor of the specific type identified by the tag of the operand), its value in the result is that specified by the <code>derived_type_definition</code>.</li> </ul> </li> </ul>                                                                                                                                             | 44     |
| <ul style="list-style-type: none"> <li> <ul style="list-style-type: none"> <li>For each discriminant of the operand type that corresponds to a discriminant that is specified by the <code>derived_type_definition</code> for some ancestor of the target type, a check is made that in the operand value it equals the value specified for it.</li> </ul> </li> </ul>                                                                                                                                                                                                                                                | 45     |
| <ul style="list-style-type: none"> <li> <ul style="list-style-type: none"> <li>For each discriminant of the result, a check is made that its value belongs to its subtype.</li> </ul> </li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                     | 46     |
| <ul style="list-style-type: none"> <li>Access Type Conversion</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | 47     |
| <ul style="list-style-type: none"> <li> <ul style="list-style-type: none"> <li>For an access-to-object type, a check is made that the accessibility level of the operand type is not deeper than that of the target type, unless the target is a stand-alone object of an anonymous access type. If the target is such a stand-alone object, a check is made that the accessibility level of the operand type is not deeper than that of the declaration of the stand-alone object ; then if the check succeeds, the accessibility level of the target type becomes that of the operand type..</li> </ul> </li> </ul> | 48/3   |
| <ul style="list-style-type: none"> <li> <ul style="list-style-type: none"> <li>If the operand value is null, the result of the conversion is the null value of the target type.</li> </ul> </li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                | 49/2   |
| <ul style="list-style-type: none"> <li> <ul style="list-style-type: none"> <li>If the operand value is not null, then the result designates the same object (or subprogram) as is designated by the operand value, but viewed as being of the target designated subtype (or profile); any checks associated with evaluating a conversion to the target designated subtype are performed.</li> </ul> </li> </ul>                                                                                                                                                                                                       | 50     |
| <p>After conversion of the value to the target type, if the target subtype is constrained, a check is performed that the value satisfies this constraint. If the target subtype excludes null, then a check is made that the value is not null.</p>                                                                                                                                                                                                                                                                                                                                                                   | 51/2   |
| <p>For the evaluation of a view conversion, the operand <code>name</code> is evaluated, and a new view of the object denoted by the operand is created, whose type is the target type; if the target type is composite, checks are performed as above for a value conversion.</p>                                                                                                                                                                                                                                                                                                                                     | 52     |
| <p>The properties of this new view are as follows:</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 53     |
| <ul style="list-style-type: none"> <li>If the target type is composite, the bounds or discriminants (if any) of the view are as defined above for a value conversion; each nondiscriminant component of the view denotes the matching component of the operand object; the subtype of the view is constrained if either the target subtype or the operand object is constrained, or if the target subtype is indefinite, or if the operand type is a descendant of the target type and has discriminants that were not inherited from the target type;</li> </ul>                                                     | 54/1   |

- 55 • If the target type is tagged, then an assignment to the view assigns to the corresponding part of the object denoted by the operand; otherwise, an assignment to the view assigns to the object, after converting the assigned value to the subtype of the object (which might raise `Constraint_Error`);
- 56 • Reading the value of the view yields the result of converting the value of the operand object to the target subtype (which might raise `Constraint_Error`), except if the object is of an access type and the view conversion is passed as an **out** parameter; in this latter case, the value of the operand object is used to initialize the formal parameter without checking against any constraint of the target subtype (see 6.4.1).
- 57 If an `Accessibility_Check` fails, `Program_Error` is raised. Any other check associated with a conversion raises `Constraint_Error` if it fails.
- 58 Conversion to a type is the same as conversion to an unconstrained subtype of the type.

## NOTES

- 59 20 In addition to explicit `type_conversions`, type conversions are performed implicitly in situations where the expected type and the actual type of a construct differ, as is permitted by the type resolution rules (see 8.6). For example, an integer literal is of the type *universal\_integer*, and is implicitly converted when assigned to a target of some specific integer type. Similarly, an actual parameter of a specific tagged type is implicitly converted when the corresponding formal parameter is of a class-wide type.
- 60 Even when the expected and actual types are the same, implicit subtype conversions are performed to adjust the array bounds (if any) of an operand to match the desired target subtype, or to raise `Constraint_Error` if the (possibly adjusted) value does not satisfy the constraints of the target subtype.
- 61/2 21 A ramification of the overload resolution rules is that the operand of an (explicit) `type_conversion` cannot be an allocator, an aggregate, a `string_literal`, a `character_literal`, or an `attribute_reference` for an `Access` or `Unchecked_Access` attribute. Similarly, such an expression enclosed by parentheses is not allowed. A `qualified_expression` (see 4.7) can be used instead of such a `type_conversion`.
- 62 22 The constraint of the target subtype has no effect for a `type_conversion` of an elementary type passed as an **out** parameter. Hence, it is recommended that the first subtype be specified as the target to minimize confusion (a similar recommendation applies to renaming and generic formal **in out** objects).

*Examples*

63 *Examples of numeric type conversion:*

```
64 Real(2*J) -- value is converted to floating point
 Integer(1.6) -- value is 2
 Integer(-0.4) -- value is 0
```

65 *Example of conversion between derived types:*

```
66 type A_Form is new B_Form;
67 X : A_Form;
 Y : B_Form;
68 X := A_Form(Y);
 Y := B_Form(X); -- the reverse conversion
```

69 *Examples of conversions between array types:*

```
70 type Sequence is array (Integer range <>) of Integer;
 subtype Dozen is Sequence(1 .. 12);
 Ledger : array(1 .. 100) of Integer;
71 Sequence(Ledger) -- bounds are those of Ledger
 Sequence(Ledger(31 .. 42)) -- bounds are 31 and 42
 Dozen(Ledger(31 .. 42)) -- bounds are those of Dozen
```

## 4.7 Qualified Expressions

A `qualified_expression` is used to state explicitly the type, and to verify the subtype, of an operand that is either an expression or an aggregate.

### Syntax

```
qualified_expression ::=
 subtype_mark'(expression) | subtype_mark'aggregate
```

### Name Resolution Rules

The *operand* (the expression or aggregate) shall resolve to be of the type determined by the `subtype_mark`, or a universal type that covers it.

### Static Semantics

If the operand of a `qualified_expression` denotes an object, the `qualified_expression` denotes a constant view of that object. The nominal subtype of a `qualified_expression` is the subtype denoted by the `subtype_mark`.

### Dynamic Semantics

The evaluation of a `qualified_expression` evaluates the operand (and if of a universal type, converts it to the type determined by the `subtype_mark`) and checks that its value belongs to the subtype denoted by the `subtype_mark`. The exception `Constraint_Error` is raised if this check fails.

### NOTES

23 When a given context does not uniquely identify an expected type, a `qualified_expression` can be used to do so. In particular, if an overloaded name or aggregate is passed to an overloaded subprogram, it might be necessary to qualify the operand to resolve its type.

### Examples

*Examples of disambiguating expressions using qualification:*

```
type Mask is (Fix, Dec, Exp, Signif);
type Code is (Fix, Cla, Dec, Tnz, Sub);

Print (Mask'(Dec)); -- Dec is of type Mask
Print (Code'(Dec)); -- Dec is of type Code

for J in Code'(Fix) .. Code'(Dec) loop ... -- qualification needed for either Fix or Dec
for J in Code range Fix .. Dec loop ... -- qualification unnecessary
for J in Code'(Fix) .. Dec loop ... -- qualification unnecessary for Dec

Dozen'(1 | 3 | 5 | 7 => 2, others => 0) -- see 4.6
```

## 4.8 Allocators

The evaluation of an allocator creates an object and yields an access value that designates the object.

### Syntax

```
allocator ::=
 new subtype_indication | new qualified_expression
```

For an allocator with a `subtype_indication`, the `subtype_indication` shall not specify a `null_exclusion`.

*Name Resolution Rules*

- 3/1 The expected type for an **allocator** shall be a single access-to-object type with designated type *D* such that either *D* covers the type determined by the **subtype\_mark** of the **subtype\_indication** or **qualified\_expression**, or the expected type is anonymous and the determined type is *D*'Class.

*Legality Rules*

- 4 An *initialized* allocator is an **allocator** with a **qualified\_expression**. An *uninitialized* allocator is one with a **subtype\_indication**. In the **subtype\_indication** of an uninitialized allocator, a **constraint** is permitted only if the **subtype\_mark** denotes an unconstrained composite subtype; if there is no **constraint**, then the **subtype\_mark** shall denote a definite subtype.
- 5/2 If the type of the **allocator** is an access-to-constant type, the **allocator** shall be an initialized allocator.
- 5.1/2 If the designated type of the type of the **allocator** is class-wide, the accessibility level of the type determined by the **subtype\_indication** or **qualified\_expression** shall not be statically deeper than that of the type of the **allocator**.
- 5.2/2 If the designated subtype of the type of the **allocator** has one or more unconstrained access discriminants, then the accessibility level of the anonymous access type of each access discriminant, as determined by the **subtype\_indication** or **qualified\_expression** of the **allocator**, shall not be statically deeper than that of the type of the **allocator** (see 3.10.2).
- 5.3/3 An **allocator** shall not be of an access type for which the **Storage\_Size** has been specified by a static expression with value zero or is defined by the language to be zero.
- 5.4/3 If the designated type of the type of the **allocator** is limited, then the **allocator** shall not be used to define the value of an access discriminant, unless the discriminated type is immutably limited (see 7.5).
- 5.5/3 In addition to the places where Legality Rules normally apply (see 12.3), these rules apply also in the private part of an instance of a generic unit.

*Static Semantics*

- 6/3 If the designated type of the type of the **allocator** is elementary, then the subtype of the created object is the designated subtype. If the designated type is composite, then the subtype of the created object is the designated subtype when the designated subtype is constrained or there is an ancestor of the designated type that has a constrained partial view; otherwise, the created object is constrained by its initial value (even if the designated subtype is unconstrained with defaults).

*Dynamic Semantics*

- 7/2 For the evaluation of an initialized allocator, the evaluation of the **qualified\_expression** is performed first. An object of the designated type is created and the value of the **qualified\_expression** is converted to the designated subtype and assigned to the object.
- 8 For the evaluation of an uninitialized allocator, the elaboration of the **subtype\_indication** is performed first. Then:
- 9/2
- If the designated type is elementary, an object of the designated subtype is created and any implicit initial value is assigned;
- 10/2
- If the designated type is composite, an object of the designated type is created with tag, if any, determined by the **subtype\_mark** of the **subtype\_indication**. This object is then initialized by default (see 3.3.1) using the **subtype\_indication** to determine its nominal subtype. A check is made that the value of the object belongs to the designated subtype. **Constraint\_Error** is raised if

this check fails. This check and the initialization of the object are performed in an arbitrary order.

For any `allocator`, if the designated type of the type of the `allocator` is class-wide, then a check is made that the master of the type determined by the `subtype_indication`, or by the tag of the value of the `qualified_expression`, includes the elaboration of the type of the `allocator`. If the designated subtype of the `allocator` has one or more unconstrained access discriminants, then a check is made that the accessibility level of the anonymous access type of each access discriminant is not deeper than that of the type of the `allocator`. `Program_Error` is raised if either such check fails. 10.1/3

If the object to be created by an `allocator` has a controlled or protected part, and the finalization of the collection of the type of the `allocator` (see 7.6.1) has started, `Program_Error` is raised. 10.2/2

If the object to be created by an `allocator` contains any tasks, and the master of the type of the `allocator` is completed, and all of the dependent tasks of the master are terminated (see 9.3), then `Program_Error` is raised. 10.3/2

If the created object contains any tasks, they are activated (see 9.2). Finally, an access value that designates the created object is returned. 11

#### *Bounded (Run-Time) Errors*

It is a bounded error if the finalization of the collection of the type (see 7.6.1) of the `allocator` has started. If the error is detected, `Program_Error` is raised. Otherwise, the allocation proceeds normally. 11.1/2

#### NOTES

24 Allocators cannot create objects of an abstract type. See 3.9.3. 12

25 If any part of the created object is controlled, the initialization includes calls on corresponding `Initialize` or `Adjust` procedures. See 7.6. 13

26 As explained in 13.11, “Storage Management”, the storage for an object allocated by an `allocator` comes from a storage pool (possibly user defined). The exception `Storage_Error` is raised by an `allocator` if there is not enough storage. Instances of `Unchecked_Deallocation` may be used to explicitly reclaim storage. 14

27 Implementations are permitted, but not required, to provide garbage collection (see 13.11.3). 15

#### *Examples*

*Examples of allocators:*

```
new Cell'(0, null, null) -- initialized explicitly, see 3.10.1
new Cell'(Value => 0, Succ => null, Pred => null) -- initialized explicitly
new Cell -- not initialized
new Matrix(1 .. 10, 1 .. 20) -- the bounds only are given
new Matrix'(1 .. 10 => (1 .. 20 => 0.0)) -- initialized explicitly
new Buffer(100) -- the discriminant only is given
new Buffer'(Size => 80, Pos => 0, Value => (1 .. 80 => 'A')) -- initialized explicitly
Expr_Ptr'(new Literal) -- allocator for access-to-class-wide type, see 3.9.1
Expr_Ptr'(new Literal'(Expression with 3.5)) -- initialized explicitly
```

## 4.9 Static Expressions and Static Subtypes

Certain expressions of a scalar or string type are defined to be static. Similarly, certain discrete ranges are defined to be static, and certain scalar and string subtypes are defined to be static subtypes. *Static* means determinable at compile time, using the declared properties or values of the program entities. 1

A static expression is a scalar or string expression that is one of the following: 2

- 3 • a `numeric_literal`;
- 4 • a `string_literal` of a static string subtype;
- 5 • a `name` that denotes the declaration of a named number or a static constant;
- 6 • a `function_call` whose *function\_name* or *function\_prefix* statically denotes a static function, and whose actual parameters, if any (whether given explicitly or by default), are all static expressions;
- 7 • an `attribute_reference` that denotes a scalar value, and whose `prefix` denotes a static scalar subtype;
- 8 • an `attribute_reference` whose `prefix` statically denotes a statically constrained array object or array subtype, and whose `attribute_designator` is `First`, `Last`, or `Length`, with an optional dimension;
- 9 • a `type_conversion` whose `subtype_mark` denotes a static scalar subtype, and whose operand is a static expression;
- 10 • a `qualified_expression` whose `subtype_mark` denotes a static (scalar or string) subtype, and whose operand is a static expression;
- 11 • a membership test whose `simple_expression` is a static expression, and whose `range` is a static range or whose `subtype_mark` denotes a static (scalar or string) subtype;
- 12 • a short-circuit control form both of whose `relations` are static expressions;
- 13 • a static expression enclosed in parentheses.

14 A *name* *statically denotes* an entity if it denotes the entity and:

- 15 • It is a `direct_name`, expanded name, or `character_literal`, and it denotes a declaration other than a `renaming_declaration`; or
- 16 • It is an `attribute_reference` whose `prefix` statically denotes some entity; or
- 17 • It denotes a `renaming_declaration` with a `name` that statically denotes the renamed entity.

18 A *static function* is one of the following:

- 19 • a predefined operator whose parameter and result types are all scalar types none of which are descendants of formal scalar types;
- 20 • a predefined concatenation operator whose result type is a string type;
- 21 • an enumeration literal;
- 22 • a language-defined attribute that is a function, if the `prefix` denotes a static scalar subtype, and if the parameter and result types are scalar.

23 In any case, a generic formal subprogram is not a static function.

24 A *static constant* is a constant view declared by a full constant declaration or an `object_renaming_declaration` with a static nominal subtype, having a value defined by a static scalar expression or by a static string expression whose value has a length not exceeding the maximum length of a `string_literal` in the implementation.

25 A *static range* is a `range` whose bounds are static expressions, or a `range_attribute_reference` that is equivalent to such a `range`. A *static discrete range* is one that is a static range or is a `subtype_indication` that defines a static scalar subtype. The base range of a scalar type is a static range, unless the type is a descendant of a formal scalar type.



A *static subtype* is either a *static scalar subtype* or a *static string subtype*. A static scalar subtype is an unconstrained scalar subtype whose type is not a descendant of a formal type, or a constrained scalar subtype formed by imposing a compatible static constraint on a static scalar subtype. A static string subtype is an unconstrained string subtype whose index subtype and component subtype are static, or a constrained string subtype formed by imposing a compatible static constraint on a static string subtype. In any case, the subtype of a generic formal object of mode **in out**, and the result subtype of a generic formal function, are not static. 26/2

The different kinds of *static constraint* are defined as follows: 27

- A null constraint is always static; 28
- A scalar constraint is static if it has no `range_constraint`, or one with a static range; 29
- An index constraint is static if each `discrete_range` is static, and each index subtype of the corresponding array type is static; 30
- A discriminant constraint is static if each `expression` of the constraint is static, and the subtype of each discriminant is static. 31

In any case, the constraint of the first subtype of a scalar formal type is neither static nor null. 31.1/2

A subtype is *statically constrained* if it is constrained, and its constraint is static. An object is *statically constrained* if its nominal subtype is statically constrained, or if it is a static string constant. 32

#### Legality Rules

A static expression is evaluated at compile time except when it is part of the right operand of a static short-circuit control form whose value is determined by its left operand. This evaluation is performed exactly, without performing `Overflow_Checks`. For a static expression that is evaluated: 33

- The expression is illegal if its evaluation fails a language-defined check other than `Overflow_Check`. 34
- If the expression is not part of a larger static expression and the expression is expected to be of a single specific type, then its value shall be within the base range of its expected type. Otherwise, the value may be arbitrarily large or small. 35/2
- If the expression is of type *universal\_real* and its expected type is a decimal fixed point type, then its value shall be a multiple of the *small* of the decimal type. This restriction does not apply if the expected type is a descendant of a formal scalar type (or a corresponding actual type in an instance). 36/2

In addition to the places where Legality Rules normally apply (see 12.3), the above restrictions also apply in the private part of an instance of a generic unit. 37/2

#### Implementation Requirements

For a real static expression that is not part of a larger static expression, and whose expected type is not a descendant of a formal type, the implementation shall round or truncate the value (according to the `Machine_Rounds` attribute of the expected type) to the nearest machine number of the expected type; if the value is exactly half-way between two machine numbers, the rounding performed is implementation-defined. If the expected type is a descendant of a formal type, or if the static expression appears in the body of an instance of a generic unit and the corresponding expression is nonstatic in the corresponding generic body, then no special rounding or truncating is required — normal accuracy rules apply (see Annex G). 38/2

*Implementation Advice*

- 38.1/2 For a real static expression that is not part of a larger static expression, and whose expected type is not a descendant of a formal type, the rounding should be the same as the default rounding for the target system.

## NOTES

- 39 28 An expression can be static even if it occurs in a context where staticness is not required.
- 40 29 A static (or run-time) `type_conversion` from a real type to an integer type performs rounding. If the operand value is exactly half-way between two integers, the rounding is performed away from zero.

*Examples*

- 41 *Examples of static expressions:*

```

42 1 + 1 -- 2
 abs(-10)*3 -- 30

43 Kilo : constant := 1000;
 Mega : constant := Kilo*Kilo; -- 1_000_000
 Long : constant := Float'Digits*2;

44 Half_Pi : constant := Pi/2; -- see 3.3.2
 Deg_To_Rad : constant := Half_Pi/90;
 Rad_To_Deg : constant := 1.0/Deg_To_Rad; -- equivalent to 1.0/((3.14159_26536/2)/90)

```

**4.9.1 Statically Matching Constraints and Subtypes***Static Semantics*

- 1/2 A constraint *statically matches* another constraint if:
- 1.1/2 • both are null constraints;
  - 1.2/2 • both are static and have equal corresponding bounds or discriminant values;
  - 1.3/2 • both are nonstatic and result from the same elaboration of a constraint of a subtype\_indication or the same evaluation of a range of a discrete\_subtype\_definition; or
  - 1.4/2 • both are nonstatic and come from the same formal\_type\_declaration.
- 2/2 A subtype *statically matches* another subtype of the same type if they have statically matching constraints, and, for access subtypes, either both or neither exclude null. Two anonymous access-to-object subtypes statically match if their designated subtypes statically match, and either both or neither exclude null, and either both or neither are access-to-constant. Two anonymous access-to-subprogram subtypes statically match if their designated profiles are subtype conformant, and either both or neither exclude null.
- 3 Two ranges of the same type *statically match* if both result from the same evaluation of a range, or if both are static and have equal corresponding bounds.
- 4/3 A constraint is *statically compatible* with a scalar subtype if it statically matches the constraint of the subtype, or if both are static and the constraint is compatible with the subtype. A constraint is *statically compatible* with an access or composite subtype if it statically matches the constraint of the subtype, or if the subtype is unconstrained. One subtype is *statically compatible* with a second subtype if the constraint of the first is statically compatible with the second subtype, and in the case of an access type, if the second subtype excludes null, then so does the first.

## Section 5: Statements

A **statement** defines an action to be performed upon its execution.

1

This section describes the general rules applicable to all **statements**. Some **statements** are discussed in later sections: **Procedure\_call\_statements** and **return statements** are described in 6, “Subprograms”. **Entry\_call\_statements**, **requeue\_statements**, **delay\_statements**, **accept\_statements**, **select\_statements**, and **abort\_statements** are described in 9, “Tasks and Synchronization”. **Raise\_statements** are described in 11, “Exceptions”, and **code\_statements** in 13. The remaining forms of **statements** are presented in this section.

2/2

### 5.1 Simple and Compound Statements - Sequences of Statements

A **statement** is either simple or compound. A **simple\_statement** encloses no other **statement**. A **compound\_statement** can enclose **simple\_statements** and other **compound\_statements**.

1

#### *Syntax*

**sequence\_of\_statements** ::= **statement** {**statement**} {**label**}

2/3

**statement** ::=

3

{**label**} **simple\_statement** | {**label**} **compound\_statement**

**simple\_statement** ::= **null\_statement**

4/2

|                                |                                 |
|--------------------------------|---------------------------------|
| <b>assignment_statement</b>    | <b>exit_statement</b>           |
| <b>goto_statement</b>          | <b>procedure_call_statement</b> |
| <b>simple_return_statement</b> | <b>entry_call_statement</b>     |
| <b>requeue_statement</b>       | <b>delay_statement</b>          |
| <b>abort_statement</b>         | <b>raise_statement</b>          |
| <b>code_statement</b>          |                                 |

**compound\_statement** ::=

5/2

|                                  |                         |
|----------------------------------|-------------------------|
| <b>if_statement</b>              | <b>case_statement</b>   |
| <b>loop_statement</b>            | <b>block_statement</b>  |
| <b>extended_return_statement</b> |                         |
| <b>accept_statement</b>          | <b>select_statement</b> |

**null\_statement** ::= **null**;

6

**label** ::= <<**label\_statement\_identifier**>>

7

**statement\_identifier** ::= **direct\_name**

8

The **direct\_name** of a **statement\_identifier** shall be an **identifier** (not an **operator\_symbol**).

9

#### *Name Resolution Rules*

The **direct\_name** of a **statement\_identifier** shall resolve to denote its corresponding implicit declaration (see below).

10

#### *Legality Rules*

Distinct **identifiers** shall be used for all **statement\_identifiers** that appear in the same body, including inner **block\_statements** but excluding inner program units.

11

*Static Semantics*

- 12 For each `statement_identifier`, there is an implicit declaration (with the specified `identifier`) at the end of the `declarative_part` of the innermost `block_statement` or body that encloses the `statement_identifier`. The implicit declarations occur in the same order as the `statement_identifiers` occur in the source text. If a usage name denotes such an implicit declaration, the entity it denotes is the `label`, `loop_statement`, or `block_statement` with the given `statement_identifier`.
- 12.1/3 If one or more `labels` end a `sequence_of_statements`, an implicit `null_statement` follows the `labels` before any following constructs.

*Dynamic Semantics*

- 13 The execution of a `null_statement` has no effect.
- 14/2 A *transfer of control* is the run-time action of an `exit_statement`, `return statement`, `goto_statement`, or `requeue_statement`, selection of a `terminate_alternative`, raising of an exception, or an abort, which causes the next action performed to be one other than what would normally be expected from the other rules of the language. As explained in 7.6.1, a transfer of control can cause the execution of constructs to be completed and then left, which may trigger finalization.
- 15 The execution of a `sequence_of_statements` consists of the execution of the individual `statements` in succession until the `sequence_` is completed.

## NOTES

- 16 1 A `statement_identifier` that appears immediately within the declarative region of a named `loop_statement` or an `accept_statement` is nevertheless implicitly declared immediately within the declarative region of the innermost enclosing body or `block_statement`; in other words, the expanded name for a named statement is not affected by whether the statement occurs inside or outside a named loop or an `accept_statement` — only nesting within `block_statements` is relevant to the form of its expanded name.

*Examples*

- 17 *Examples of labeled statements:*
- 18     <<Here>> <<Ici>> <<Aqui>> <<Hier>> **null**;
- 19     <<After>> X := 1;

## 5.2 Assignment Statements

- 1 An `assignment_statement` replaces the current value of a variable with the result of evaluating an expression.

*Syntax*

- 2     `assignment_statement ::=`  
       `variable_name := expression;`
- 3 The execution of an `assignment_statement` includes the evaluation of the `expression` and the *assignment* of the value of the `expression` into the *target*. An assignment operation (as opposed to an `assignment_statement`) is performed in other contexts as well, including object initialization and by-copy parameter passing. The *target* of an assignment operation is the view of the object to which a value is being assigned; the target of an `assignment_statement` is the variable denoted by the `variable_name`.

*Name Resolution Rules*

- 4/2 The `variable_name` of an `assignment_statement` is expected to be of any type. The expected type for the `expression` is the type of the target.

*Legality Rules*

The target denoted by the *variable\_name* shall be a variable of a nonlimited type. 5/2

If the target is of a tagged class-wide type *TClass*, then the *expression* shall either be dynamically tagged, or of type *T* and tag-indeterminate (see 3.9.2). 6

*Dynamic Semantics*

For the execution of an *assignment\_statement*, the *variable\_name* and the *expression* are first evaluated in an arbitrary order. 7

When the type of the target is class-wide: 8

- If the *expression* is tag-indeterminate (see 3.9.2), then the controlling tag value for the *expression* is the tag of the target; 9
- Otherwise (the *expression* is dynamically tagged), a check is made that the tag of the value of the *expression* is the same as that of the target; if this check fails, *Constraint\_Error* is raised. 10

The value of the *expression* is converted to the subtype of the target. The conversion might raise an exception (see 4.6). 11

In cases involving controlled types, the target is finalized, and an anonymous object might be used as an intermediate in the assignment, as described in 7.6.1, “Completion and Finalization”. In any case, the converted value of the *expression* is then *assigned* to the target, which consists of the following two steps: 12

- The value of the target becomes the converted value. 13
- If any part of the target is controlled, its value is adjusted as explained in clause 7.6. 14

## NOTES

2 The tag of an object never changes; in particular, an *assignment\_statement* does not change the tag of the target. 15

*This paragraph was deleted.* 16/2

*Examples*

*Examples of assignment statements:* 17

```
Value := Max_Value - 1; 18
Shade := Blue;
Next_Frame(F)(M, N) := 2.5; -- see 4.1.1 19
U := Dot_Product(V, W); -- see 6.3
Writer := (Status => Open, Unit => Printer, Line_Count => 60); -- see 3.8.1 20
Next_Car.all := (72074, null); -- see 3.10.1
```

*Examples involving scalar subtype conversions:* 21

```
I, J : Integer range 1 .. 10 := 5; 22
K : Integer range 1 .. 20 := 15;
...
I := J; -- identical ranges 23
K := J; -- compatible ranges
J := K; -- will raise Constraint_Error if K > 10
```

*Examples involving array subtype conversions:* 24

```
A : String(1 .. 31); 25
B : String(3 .. 33);
...
A := B; -- same number of components 26
```

27     A(1 .. 9)     := "tar sauce";  
       A(4 .. 12) := A(1 .. 9);   -- A(1 .. 12) = "tartar sauce"

## NOTES

28     3 *Notes on the examples:* Assignment\_statements are allowed even in the case of overlapping slices of the same array, because the *variable\_name* and expression are both evaluated before copying the value into the variable. In the above example, an implementation yielding A(1 .. 12) = "tartartartar" would be incorrect.

## 5.3 If Statements

1     An *if\_statement* selects for execution at most one of the enclosed *sequences\_of\_statements*, depending on the (truth) value of one or more corresponding conditions.

*Syntax*

2     *if\_statement* ::=  
       **if** condition **then**  
         sequence\_of\_statements  
       { **elsif** condition **then**  
         sequence\_of\_statements }  
       [ **else**  
         sequence\_of\_statements ]  
       **end if**;  
       condition ::= *boolean\_expression*

*Name Resolution Rules*

4     A condition is expected to be of any boolean type.

*Dynamic Semantics*

5     For the execution of an *if\_statement*, the condition specified after **if**, and any conditions specified after **elsif**, are evaluated in succession (treating a final **else** as **elsif True then**), until one evaluates to True or all conditions are evaluated and yield False. If a condition evaluates to True, then the corresponding *sequence\_of\_statements* is executed; otherwise none of them is executed.

*Examples*

6     *Examples of if statements:*

7     **if** Month = December **and** Day = 31 **then**  
       Month := January;  
       Day   := 1;  
       Year  := Year + 1;  
     **end if**;  
       **if** Line\_Too\_Short **then**  
         **raise** Layout\_Error;  
       **elsif** Line\_Full **then**  
         New\_Line;  
         Put(Item);  
       **else**  
         Put(Item);  
       **end if**;  
       **if** My\_Car.Owner.Vehicle /= My\_Car **then**                   -- see 3.10.1  
         Report ("Incorrect data");  
       **end if**;

## 5.4 Case Statements

A `case_statement` selects for execution one of a number of alternative `sequences_of_statements`; the chosen alternative is defined by the value of an expression. 1

### *Syntax*

```

case_statement ::=
 case expression is
 case_statement_alternative
 {case_statement_alternative}
 end case;
case_statement_alternative ::=
 when discrete_choice_list =>
 sequence_of_statements

```

2 3

### *Name Resolution Rules*

The expression is expected to be of any discrete type. The expected type for each `discrete_choice` is the type of the expression. 4

### *Legality Rules*

The expressions and `discrete_ranges` given as `discrete_choices` of a `case_statement` shall be static. A `discrete_choice` **others**, if present, shall appear alone and in the last `discrete_choice_list`. 5

The possible values of the expression shall be covered as follows: 6

- If the expression is a name (including a `type_conversion`, a `qualified_expression`, or a `function_call`) having a static and constrained nominal subtype, then each non-**others** `discrete_choice` shall cover only values in that subtype, and each value of that subtype shall be covered by some `discrete_choice` (either explicitly or by **others**). 7/3
- If the type of the expression is *root\_integer*, *universal\_integer*, or a descendant of a formal scalar type, then the `case_statement` shall have an **others** `discrete_choice`. 8
- Otherwise, each value of the base range of the type of the expression shall be covered (either explicitly or by **others**). 9

Two distinct `discrete_choices` of a `case_statement` shall not cover the same value. 10

### *Dynamic Semantics*

For the execution of a `case_statement` the expression is first evaluated. 11

If the value of the expression is covered by the `discrete_choice_list` of some `case_statement_alternative`, then the `sequence_of_statements` of the `_alternative` is executed. 12

Otherwise (the value is not covered by any `discrete_choice_list`, perhaps due to being outside the base range), `Constraint_Error` is raised. 13

### NOTES

- 4 The execution of a `case_statement` chooses one and only one alternative. Qualification of the expression of a `case_statement` by a static subtype can often be used to limit the number of choices that need be given explicitly. 14

*Examples*

Examples of case statements:

```

case Sensor is
 when Elevation => Record_Elevation(Sensor_Value);
 when Azimuth => Record_Azimuth (Sensor_Value);
 when Distance => Record_Distance (Sensor_Value);
 when others => null;
end case;

case Today is
 when Mon => Compute_Initial_Balance;
 when Fri => Compute_Closing_Balance;
 when Tue .. Thu => Generate_Report(Today);
 when Sat .. Sun => null;
end case;

case Bin_Number(Count) is
 when 1 => Update_Bin(1);
 when 2 => Update_Bin(2);
 when 3 | 4 =>
 Empty_Bin(1);
 Empty_Bin(2);
 when others => raise Error;
end case;

```

## 5.5 Loop Statements

A loop\_statement includes a sequence\_of\_statements that is to be executed repeatedly, zero or more times.

*Syntax*

```

loop_statement ::=
 [loop_statement_identifier:]
 [iteration_scheme] loop
 sequence_of_statements
 end loop [loop_identifier];

iteration_scheme ::= while condition
 | for loop_parameter_specification

loop_parameter_specification ::=
 defining_identifier in [reverse] discrete_subtype_definition

```

If a loop\_statement has a loop\_statement\_identifier, then the identifier shall be repeated after the end loop; otherwise, there shall not be an identifier after the end loop.

*Static Semantics*

A loop\_parameter\_specification declares a loop parameter, which is an object whose subtype is that defined by the discrete\_subtype\_definition.

*Dynamic Semantics*

For the execution of a loop\_statement, the sequence\_of\_statements is executed repeatedly, zero or more times, until the loop\_statement is complete. The loop\_statement is complete when a transfer of control occurs that transfers control out of the loop, or, in the case of an iteration\_scheme, as specified below.

For the execution of a loop\_statement with a while iteration\_scheme, the condition is evaluated before each execution of the sequence\_of\_statements; if the value of the condition is True, the sequence\_of\_statements is executed; if False, the execution of the loop\_statement is complete.



For the execution of a `loop_statement` with a **for** `iteration_scheme`, the `loop_parameter_specification` is first elaborated. This elaboration creates the loop parameter and elaborates the `discrete_subtype_definition`. If the `discrete_subtype_definition` defines a subtype with a null range, the execution of the `loop_statement` is complete. Otherwise, the `sequence_of_statements` is executed once for each value of the discrete subtype defined by the `discrete_subtype_definition` (or until the loop is left as a consequence of a transfer of control). Prior to each such iteration, the corresponding value of the discrete subtype is assigned to the loop parameter. These values are assigned in increasing order unless the reserved word **reverse** is present, in which case the values are assigned in decreasing order.

## NOTES

5 A loop parameter is a constant; it cannot be updated within the `sequence_of_statements` of the loop (see 3.3).

6 An `object_declaration` should not be given for a loop parameter, since the loop parameter is automatically declared by the `loop_parameter_specification`. The scope of a loop parameter extends from the `loop_parameter_specification` to the end of the `loop_statement`, and the visibility rules are such that a loop parameter is only visible within the `sequence_of_statements` of the loop.

7 The `discrete_subtype_definition` of a for loop is elaborated just once. Use of the reserved word **reverse** does not alter the discrete subtype defined, so that the following `iteration_schemes` are not equivalent; the first has a null range.

```
for J in reverse 1 .. 0
for J in 0 .. 1
```

*Examples*

*Example of a loop statement without an iteration scheme:*

```
loop
 Get(Current_Character);
 exit when Current_Character = '*';
end loop;
```

*Example of a loop statement with a while iteration scheme:*

```
while Bid(N).Price < Cut_Off.Price loop
 Record_Bid(Bid(N).Price);
 N := N + 1;
end loop;
```

*Example of a loop statement with a for iteration scheme:*

```
for J in Buffer'Range loop -- works even with a null range
 if Buffer(J) /= Space then
 Put(Buffer(J));
 end if;
end loop;
```

*Example of a loop statement with a name:*

```
Summation:
 while Next /= Head loop -- see 3.10.1
 Sum := Sum + Next.Value;
 Next := Next.Succ;
 end loop Summation;
```

## 5.6 Block Statements

- 1 A `block_statement` encloses a `handled_sequence_of_statements` optionally preceded by a `declarative_part`.

### Syntax

- 2 `block_statement ::=`  
     `[block_statement_identifier:]`  
     `declare`  
         `declarative_part`  
     `begin`  
         `handled_sequence_of_statements`  
     `end [block_identifier];`

- 3 If a `block_statement` has a `block_statement_identifier`, then the identifier shall be repeated after the `end`; otherwise, there shall not be an identifier after the `end`.

### Static Semantics

- 4 A `block_statement` that has no explicit `declarative_part` has an implicit empty `declarative_part`.

### Dynamic Semantics

- 5 The execution of a `block_statement` consists of the elaboration of its `declarative_part` followed by the execution of its `handled_sequence_of_statements`.

### Examples

- 6 *Example of a block statement with a local variable:*

- 7 `Swap:`  
     `declare`  
         `Temp : Integer;`  
     `begin`  
         `Temp := V; V := U; U := Temp;`  
     `end Swap;`

## 5.7 Exit Statements

- 1 An `exit_statement` is used to complete the execution of an enclosing `loop_statement`; the completion is conditional if the `exit_statement` includes a condition.

### Syntax

- 2 `exit_statement ::=`  
     `exit [loop_name] [when condition];`

### Name Resolution Rules

- 3 The `loop_name`, if any, in an `exit_statement` shall resolve to denote a `loop_statement`.

### Legality Rules

- 4 Each `exit_statement` *applies to* a `loop_statement`; this is the `loop_statement` being exited. An `exit_statement` with a name is only allowed within the `loop_statement` denoted by the name, and applies to that `loop_statement`. An `exit_statement` without a name is only allowed within a `loop_statement`, and applies to the innermost enclosing one. An `exit_statement` that applies to a given `loop_statement` shall not

appear within a body or `accept_statement`, if this construct is itself enclosed by the given `loop_statement`.

#### *Dynamic Semantics*

For the execution of an `exit_statement`, the condition, if present, is first evaluated. If the value of the condition is True, or if there is no condition, a transfer of control is done to complete the `loop_statement`. If the value of the condition is False, no transfer of control takes place.

#### NOTES

8 Several nested loops can be exited by an `exit_statement` that names the outer loop.

#### *Examples*

*Examples of loops with exit statements:*

```

for N in 1 .. Max_Num_Items loop
 Get_New_Item(New_Item);
 Merge_Item(New_Item, Storage_File);
 exit when New_Item = Terminal_Item;
end loop;

Main_Cycle:
loop
 -- initial statements
 exit Main_Cycle when Found;
 -- final statements
end loop Main_Cycle;

```

## 5.8 Goto Statements

A `goto_statement` specifies an explicit transfer of control from this statement to a target statement with a given label.

#### *Syntax*

```
goto_statement ::= goto label_name;
```

#### *Name Resolution Rules*

The `label_name` shall resolve to denote a label; the statement with that label is the *target statement*.

#### *Legality Rules*

The innermost `sequence_of_statements` that encloses the target statement shall also enclose the `goto_statement`. Furthermore, if a `goto_statement` is enclosed by an `accept_statement` or a body, then the target statement shall not be outside this enclosing construct.

#### *Dynamic Semantics*

The execution of a `goto_statement` transfers control to the target statement, completing the execution of any `compound_statement` that encloses the `goto_statement` but does not enclose the target.

#### NOTES

9 The above rules allow transfer of control to a statement of an enclosing `sequence_of_statements` but not the reverse. Similarly, they prohibit transfers of control such as between alternatives of a `case_statement`, `if_statement`, or `select_statement`; between `exception_handlers`; or from an `exception_handler` of a `handled_sequence_of_statements` back to its `sequence_of_statements`.

*Examples*

7 *Example of a loop containing a goto statement:*

```
8 <<Sort>>
 for I in 1 .. N-1 loop
 if A(I) > A(I+1) then
 Exchange(A(I), A(I+1));
 goto Sort;
 end if;
 end loop;
```

## Section 6: Subprograms

A subprogram is a program unit or intrinsic operation whose execution is invoked by a subprogram call. There are two forms of subprogram: procedures and functions. A procedure call is a **statement**; a function call is an expression and returns a value. The definition of a subprogram can be given in two parts: a subprogram declaration defining its interface, and a **subprogram\_body** defining its execution. Operators and enumeration literals are functions.

A *callable entity* is a subprogram or entry (see Section 9). A callable entity is invoked by a *call*; that is, a subprogram call or entry call. A *callable construct* is a construct that defines the action of a call upon a callable entity: a **subprogram\_body**, **entry\_body**, or **accept\_statement**.

### 6.1 Subprogram Declarations

A **subprogram\_declaration** declares a procedure or function.

*Syntax*

```

subprogram_declaration ::=
 [overriding_indicator]
 subprogram_specification;
2/2

This paragraph was deleted.
3/2

subprogram_specification ::=
 procedure_specification
 | function_specification
4/2

procedure_specification ::= procedure defining_program_unit_name parameter_profile
4.1/2

function_specification ::= function defining_designator parameter_and_result_profile
4.2/2

designator ::= [parent_unit_name .] identifier | operator_symbol
5

defining_designator ::= defining_program_unit_name | defining_operator_symbol
6

defining_program_unit_name ::= [parent_unit_name .] defining_identifier
7

The optional parent_unit_name is only allowed for library units (see 10.1.1).
8

operator_symbol ::= string_literal
9

The sequence of characters in an operator_symbol shall form a reserved word, a delimiter, or
10/2
compound delimiter that corresponds to an operator belonging to one of the six categories of
operators defined in clause 4.5.

defining_operator_symbol ::= operator_symbol
11

parameter_profile ::= [formal_part]
12

parameter_and_result_profile ::=
 [formal_part] return [null_exclusion] subtype_mark
 | [formal_part] return access_definition
13/2

formal_part ::=
 (parameter_specification {; parameter_specification})
14

parameter_specification ::=
 defining_identifier_list : mode [null_exclusion] subtype_mark [:= default_expression]
 | defining_identifier_list : access_definition [:= default_expression]
15/2

mode ::= [in] | in out | out
16

```

*Name Resolution Rules*

- 17 A *formal parameter* is an object directly visible within a `subprogram_body` that represents the actual parameter passed to the subprogram in a call; it is declared by a `parameter_specification`. For a formal parameter, the expected type for its `default_expression`, if any, is that of the formal parameter.

*Legality Rules*

- 18/3 The *parameter mode* of a formal parameter conveys the direction of information transfer with the actual parameter: **in**, **in out**, or **out**. Mode **in** is the default, and is the mode of a parameter defined by an `access_definition`. The formal parameters of a function, if any, shall have the mode **in**.
- 19 A `default_expression` is only allowed in a `parameter_specification` for a formal parameter of mode **in**.
- 20/2 A `subprogram_declaration` or a `generic_subprogram_declaration` requires a completion: a body, a `renaming_declaration` (see 8.5), or a `pragma Import` (see B.1). A completion is not allowed for an `abstract_subprogram_declaration` (see 3.9.3) or a `null_procedure_declaration` (see 6.7).
- 21 A name that denotes a formal parameter is not allowed within the `formal_part` in which it is declared, nor within the `formal_part` of a corresponding body or `accept_statement`.

*Static Semantics*

- 22 The *profile* of (a view of) a callable entity is either a `parameter_profile` or `parameter_and_result_profile`; it embodies information about the interface to that entity — for example, the profile includes information about parameters passed to the callable entity. All callable entities have a profile — enumeration literals, other subprograms, and entries. An access-to-subprogram type has a designated profile. Associated with a profile is a calling convention. A `subprogram_declaration` declares a procedure or a function, as indicated by the initial reserved word, with name and profile as given by its specification.
- 23/2 The nominal subtype of a formal parameter is the subtype determined by the optional `null_exclusion` and the `subtype_mark`, or defined by the `access_definition`, in the `parameter_specification`. The nominal subtype of a function result is the subtype determined by the optional `null_exclusion` and the `subtype_mark`, or defined by the `access_definition`, in the `parameter_and_result_profile`.
- 24/2 An *access parameter* is a formal **in** parameter specified by an `access_definition`. An *access result type* is a function result type specified by an `access_definition`. An access parameter or result type is of an anonymous access type (see 3.10). Access parameters of an access-to-object type allow dispatching calls to be controlled by access values. Access parameters of an access-to-subprogram type permit calls to subprograms passed as parameters irrespective of their accessibility level.
- 25 The *subtypes of a profile* are:
- 26 • For any non-access parameters, the nominal subtype of the parameter.
  - 27/2 • For any access parameters of an access-to-object type, the designated subtype of the parameter type.
  - 27.1/3 • For any access parameters of an access-to-subprogram type, the subtypes of the designated profile of the parameter type.
  - 28/2 • For any non-access result, the nominal subtype of the function result.
  - 28.1/2 • For any access result type of an access-to-object type, the designated subtype of the result type.
  - 28.2/3 • For any access result type of an access-to-subprogram type, the subtypes of the designated profile of the result type.
- 29 The *types of a profile* are the types of those subtypes.

A subprogram declared by an `abstract_subprogram_declaration` is abstract; a subprogram declared by a `subprogram_declaration` is not. See 3.9.3, “Abstract Types and Subprograms”. Similarly, a procedure defined by a `null_procedure_declaration` is a null procedure; a procedure declared by a `subprogram_declaration` is not. See 6.7, “Null Procedures”.

An `overriding_indicator` is used to indicate whether overriding is intended. See 8.3.1, “Overriding Indicators”.

#### Dynamic Semantics

The elaboration of a `subprogram_declaration` has no effect.

#### NOTES

- 1 A `parameter_specification` with several identifiers is equivalent to a sequence of single `parameter_specifications`, as explained in 3.3.
- 2 Abstract subprograms do not have bodies, and cannot be used in a nondispatching call (see 3.9.3, “Abstract Types and Subprograms”).
- 3 The evaluation of `default_expressions` is caused by certain calls, as described in 6.4.1. They are not evaluated during the elaboration of the subprogram declaration.
- 4 Subprograms can be called recursively and can be called concurrently from multiple tasks.

#### Examples

*Examples of subprogram declarations:*

```

procedure Traverse_Tree;
procedure Increment(X : in out Integer);
procedure Right_Indent(Margin : out Line_Size);
procedure Switch(From, To : in out Link);
function Random return Probability;
function Min_Cell(X : Link) return Cell;
function Next_Frame(K : Positive) return Frame;
function Dot_Product(Left, Right : Vector) return Real;
function "*" (Left, Right : Matrix) return Matrix;

```

*Examples of `in` parameters with default expressions:*

```

procedure Print_Header(Pages : in Natural;
Header : in Line := (1 .. Line'Last => ' ');
Center : in Boolean := True);

```

## 6.2 Formal Parameter Modes

A `parameter_specification` declares a formal parameter of mode **in**, **in out**, or **out**.

#### Static Semantics

A parameter is passed either *by copy* or *by reference*. When a parameter is passed by copy, the formal parameter denotes a separate object from the actual parameter, and any information transfer between the two occurs only before and after executing the subprogram. When a parameter is passed by reference, the formal parameter denotes (a view of) the object denoted by the actual parameter; reads and updates of the formal parameter directly reference the actual parameter object.

A type is a *by-copy type* if it is an elementary type, or if it is a descendant of a private type whose full type is a by-copy type. A parameter of a by-copy type is passed by copy.

- 4 A type is a *by-reference type* if it is a descendant of one of the following:
- 5 • a tagged type;
  - 6 • a task or protected type;
  - 7/3 • an explicitly limited record type;
  - 8 • a composite type with a subcomponent of a by-reference type;
  - 9 • a private type whose full type is a by-reference type.
- 10 A parameter of a by-reference type is passed by reference. Each value of a by-reference type has an associated object. For a parenthesized expression, `qualified_expression`, or `type_conversion`, this object is the one associated with the operand.
- 11 For parameters of other types, it is unspecified whether the parameter is passed by copy or by reference.

*Bounded (Run-Time) Errors*

- 12 If one **name** denotes a part of a formal parameter, and a second **name** denotes a part of a distinct formal parameter or an object that is not part of a formal parameter, then the two **names** are considered *distinct access paths*. If an object is of a type for which the parameter passing mechanism is not specified, then it is a bounded error to assign to the object via one access path, and then read the value of the object via a distinct access path, unless the first access path denotes a part of a formal parameter that no longer exists at the point of the second access (due to leaving the corresponding callable construct). The possible consequences are that `Program_Error` is raised, or the newly assigned value is read, or some old value of the object is read.

NOTES

- 13 5 A formal parameter of mode **in** is a constant view (see 3.3); it cannot be updated within the `subprogram_body`.

## 6.3 Subprogram Bodies

- 1 A `subprogram_body` specifies the execution of a subprogram.

*Syntax*

2/2 `subprogram_body ::=`  
     `[overriding_indicator]`  
     `subprogram_specification is`  
         `declarative_part`  
     `begin`  
         `handled_sequence_of_statements`  
     `end [designator];`

- 3 If a **designator** appears at the end of a `subprogram_body`, it shall repeat the **defining\_designator** of the `subprogram_specification`.

*Legality Rules*

- 4 In contrast to other bodies, a `subprogram_body` need not be the completion of a previous declaration, in which case the body declares the subprogram. If the body is a completion, it shall be the completion of a `subprogram_declaration` or `generic_subprogram_declaration`. The profile of a `subprogram_body` that completes a declaration shall conform fully to that of the declaration.



*Static Semantics*

A subprogram\_body is considered a declaration. It can either complete a previous declaration, or itself be the initial declaration of the subprogram. 5

*Dynamic Semantics*

The elaboration of a non-generic subprogram\_body has no other effect than to establish that the subprogram can from then on be called without failing the Elaboration\_Check. 6

The execution of a subprogram\_body is invoked by a subprogram call. For this execution the declarative\_part is elaborated, and the handled\_sequence\_of\_statements is then executed. 7

*Examples*

*Example of procedure body:* 8

```

procedure Push(E : in Element_Type; S : in out Stack) is
begin
 if S.Index = S.Size then
 raise Stack_Overflow;
 else
 S.Index := S.Index + 1;
 S.Space(S.Index) := E;
 end if;
end Push;

```

9

*Example of a function body:* 10

```

function Dot_Product(Left, Right : Vector) return Real is
 Sum : Real := 0.0;
begin
 Check(Left'First = Right'First and Left'Last = Right'Last);
 for J in Left'Range loop
 Sum := Sum + Left(J)*Right(J);
 end loop;
 return Sum;
end Dot_Product;

```

11

### 6.3.1 Conformance Rules

When subprogram profiles are given in more than one place, they are required to conform in one of four ways: type conformance, mode conformance, subtype conformance, or full conformance. 1

*Static Semantics*

As explained in B.1, “Interfacing Pragmas”, a *convention* can be specified for an entity. Unless this International Standard states otherwise, the default convention of an entity is Ada. For a callable entity or access-to-subprogram type, the convention is called the *calling convention*. The following conventions are defined by the language: 2/1

- The default calling convention for any subprogram not listed below is *Ada*. A pragma Convention, Import, or Export may be used to override the default calling convention (see B.1). 3
- The *Intrinsic* calling convention represents subprograms that are “built in” to the compiler. The default calling convention is Intrinsic for the following: 4
  - an enumeration literal; 5
  - a “/=” operator declared implicitly due to the declaration of “=” (see 6.6); 6
  - any other implicitly declared subprogram unless it is a dispatching operation of a tagged type; 7

- 8           • an inherited subprogram of a generic formal tagged type with unknown discriminants;
- 9           • an attribute that is a subprogram;
- 10/2       • a subprogram declared immediately within a `protected_body`;
- 10.1/2     • any prefixed view of a subprogram (see 4.1.3).
- 11       The Access attribute is not allowed for Intrinsic subprograms.
- 12       • The default calling convention is *protected* for a protected subprogram, and for an access-to-subprogram type with the reserved word **protected** in its definition.
- 13       • The default calling convention is *entry* for an entry.
- 13.1/2     • The calling convention for an anonymous access-to-subprogram parameter or anonymous access-to-subprogram result is *protected* if the reserved word **protected** appears in its definition and otherwise is the convention of the subprogram that contains the parameter.
- 13.2/1     • If not specified above as Intrinsic, the calling convention for any inherited or overriding dispatching operation of a tagged type is that of the corresponding subprogram of the parent type. The default calling convention for a new dispatching operation of a tagged type is the convention of the type.
- 14       Of these four conventions, only Ada and Intrinsic are allowed as a *convention\_identifier* in a `pragma Convention`, `Import`, or `Export`.
- 15/2       Two profiles are *type conformant* if they have the same number of parameters, and both have a result if either does, and corresponding parameter and result types are the same, or, for access parameters or access results, corresponding designated types are the same, or corresponding designated profiles are type conformant.
- 16/3       Two profiles are *mode conformant* if they are type conformant, corresponding parameters have identical modes, and, for access parameters or access result types, the designated subtypes statically match and either both or neither are access-to-constant, or the designated profiles are subtype conformant.
- 17       Two profiles are *subtype conformant* if they are mode-conformant, corresponding subtypes of the profile statically match, and the associated calling conventions are the same. The profile of a generic formal subprogram is not subtype-conformant with any other profile.
- 18/3       Two profiles are *fully conformant* if they are subtype-conformant, if they have access-to-subprogram results whose designated profiles are fully conformant, and corresponding parameters have:
  - 18.1/3     • the same names; and
  - 18.2/3     • both or neither have `null_exclusions`; and
  - 18.3/3     • neither have `default_expressions`, or they both have `default_expressions` that are fully conformant with one another; and
  - 18.4/3     • for access-to-subprogram parameters, the designated profiles are fully conformant.
- 19       Two expressions are *fully conformant* if, after replacing each use of an operator with the equivalent `function_call`:
  - 20       • each constituent construct of one corresponds to an instance of the same syntactic category in the other, except that an expanded name may correspond to a `direct_name` (or `character_literal`) or to a different expanded name in the other; and
  - 21       • each `direct_name`, `character_literal`, and `selector_name` that is not part of the prefix of an expanded name in one denotes the same declaration as the corresponding `direct_name`, `character_literal`, or `selector_name` in the other; and

- each `attribute_designator` in one is the same as the corresponding `attribute_designator` in the other; and 21.1/3
- each `primary` that is a literal in one has the same value as the corresponding literal in the other. 22

Two `known_discriminant_parts` are *fully conformant* if they have the same number of discriminants, and discriminants in the same positions have the same names, statically matching subtypes, and `default_expressions` that are fully conformant with one another. 23

Two `discrete_subtype_definitions` are *fully conformant* if they are both `subtype_indications` or are both `ranges`, the `subtype_marks` (if any) denote the same subtype, and the corresponding `simple_expressions` of the `ranges` (if any) fully conform. 24

The *prefixed view profile* of a subprogram is the profile obtained by omitting the first parameter of that subprogram. There is no prefixed view profile for a parameterless subprogram. For the purposes of defining subtype and mode conformance, the convention of a prefixed view profile is considered to match that of either an entry or a protected operation. 24.1/2

#### *Implementation Permissions*

An implementation may declare an operator declared in a language-defined library unit to be intrinsic. 25

## 6.3.2 Inline Expansion of Subprograms

Subprograms may be expanded in line at the call site. 1

#### *Syntax*

The form of a `pragma Inline`, which is a program unit pragma (see 10.1.5), is as follows: 2

**pragma** Inline(name {, name}); 3

#### *Legality Rules*

The `pragma` shall apply to one or more callable entities or generic subprograms. 4

#### *Static Semantics*

If a `pragma Inline` applies to a callable entity, this indicates that inline expansion is desired for all calls to that entity. If a `pragma Inline` applies to a generic subprogram, this indicates that inline expansion is desired for all calls to all instances of that generic subprogram. 5

#### *Implementation Permissions*

For each call, an implementation is free to follow or to ignore the recommendation expressed by the `pragma`. 6

An implementation may allow a `pragma Inline` that has an argument which is a `direct_name` denoting a `subprogram_body` of the same `declarative_part`. 6.1/2

#### NOTES

6 The `name` in a `pragma Inline` can denote more than one entity in the case of overloading. Such a `pragma` applies to all of the denoted entities. 7

## 6.4 Subprogram Calls

- 1 A *subprogram call* is either a *procedure\_call\_statement* or a *function\_call*; it invokes the execution of the *subprogram\_body*. The call specifies the association of the actual parameters, if any, with formal parameters of the subprogram.

### Syntax

- 2 *procedure\_call\_statement* ::=  
     *procedure\_name*;  
     | *procedure\_prefix* *actual\_parameter\_part*;
- 3 *function\_call* ::=  
     *function\_name*  
     | *function\_prefix* *actual\_parameter\_part*
- 4 *actual\_parameter\_part* ::=  
     (*parameter\_association* {, *parameter\_association*})
- 5 *parameter\_association* ::=  
     [*formal\_parameter\_selector\_name* =>] *explicit\_actual\_parameter*
- 6 *explicit\_actual\_parameter* ::= *expression* | *variable\_name*
- 7 A *parameter\_association* is *named* or *positional* according to whether or not the *formal\_parameter\_selector\_name* is specified. Any positional associations shall precede any named associations. Named associations are not allowed if the *prefix* in a subprogram call is an *attribute\_reference*.

### Name Resolution Rules

- 8/2 The *name* or *prefix* given in a *procedure\_call\_statement* shall resolve to denote a callable entity that is a procedure, or an entry renamed as (viewed as) a procedure. The *name* or *prefix* given in a *function\_call* shall resolve to denote a callable entity that is a function. The *name* or *prefix* shall not resolve to denote an abstract subprogram unless it is also a dispatching subprogram. When there is an *actual\_parameter\_part*, the *prefix* can be an *implicit\_dereference* of an access-to-subprogram value.
- 9 A subprogram call shall contain at most one association for each formal parameter. Each formal parameter without an association shall have a *default\_expression* (in the profile of the view denoted by the *name* or *prefix*). This rule is an overloading rule (see 8.6).

### Dynamic Semantics

- 10/2 For the execution of a subprogram call, the *name* or *prefix* of the call is evaluated, and each *parameter\_association* is evaluated (see 6.4.1). If a *default\_expression* is used, an implicit *parameter\_association* is assumed for this rule. These evaluations are done in an arbitrary order. The *subprogram\_body* is then executed, or a call on an entry or protected subprogram is performed (see 3.9.2). Finally, if the subprogram completes normally, then after it is left, any necessary assigning back of formal to actual parameters occurs (see 6.4.1).
- 10.1/2 If the *name* or *prefix* of a subprogram call denotes a prefixed view (see 4.1.3), the subprogram call is equivalent to a call on the underlying subprogram, with the first actual parameter being provided by the *prefix* of the prefixed view (or the Access attribute of this *prefix* if the first formal parameter is an access parameter), and the remaining actual parameters given by the *actual\_parameter\_part*, if any.
- 11/2 The exception *Program\_Error* is raised at the point of a *function\_call* if the function completes normally without executing a return statement.

A `function_call` denotes a constant, as defined in 6.5; the nominal subtype of the constant is given by the nominal subtype of the function result. 12/2

#### Examples

##### Examples of procedure calls:

```

Traverse_Tree; -- see 6.1 13
Print_Header(128, Title, True); -- see 6.1 14
Switch(From => X, To => Next); -- see 6.1 15
Print_Header(128, Header => Title, Center => True); -- see 6.1
Print_Header(Header => Title, Center => True, Pages => 128); -- see 6.1

```

##### Examples of function calls:

```

Dot_Product(U, V) -- see 6.1 and 6.3 16
Clock -- see 9.6 17
F.all -- presuming F is of an access-to-subprogram type — see 3.10

```

##### Examples of procedures with default expressions:

```

procedure Activate(Process : in Process_Name;
 After : in Process_Name := No_Process;
 Wait : in Duration := 0.0;
 Prior : in Boolean := False);
procedure Pair(Left, Right : in Person_Name := new Person); -- see 3.10.1 20

```

##### Examples of their calls:

```

Activate(X); 21
Activate(X, After => Y); 22
Activate(X, Wait => 60.0, Prior => True);
Activate(X, Y, 10.0, False);
Pair; 23
Pair(Left => new Person, Right => new Person);

```

#### NOTES

7 If a `default_expression` is used for two or more parameters in a multiple parameter\_specification, the `default_expression` is evaluated once for each omitted parameter. Hence in the above examples, the two calls of `Pair` are equivalent. 24

#### Examples

##### Examples of overloaded subprograms:

```

procedure Put(X : in Integer); 25
procedure Put(X : in String); 26
procedure Set(Tint : in Color); 27
procedure Set(Signal : in Light);

```

##### Examples of their calls:

```

Put(28); 28
Put("no possible ambiguity here"); 29
Set(Tint => Red); 30
Set(Signal => Red);
Set(Color'(Red));
-- Set(Red) would be ambiguous since Red may
-- denote a value either of type Color or of type Light 31

```

## 6.4.1 Parameter Associations

- 1 A parameter association defines the association between an actual parameter and a formal parameter.

### *Name Resolution Rules*

- 2/3 The *formal parameter selector\_name* of a named *parameter\_association* shall resolve to denote a *parameter\_specification* of the view being called; this is the formal parameter of the association. The formal parameter for a positional *parameter\_association* is the parameter with the corresponding position in the formal part of the view being called.
- 3 The *actual parameter* is either the *explicit\_actual\_parameter* given in a *parameter\_association* for a given formal parameter, or the corresponding *default\_expression* if no *parameter\_association* is given for the formal parameter. The expected type for an actual parameter is the type of the corresponding formal parameter.
- 4 If the mode is **in**, the actual is interpreted as an **expression**; otherwise, the actual is interpreted only as a **name**, if possible.

### *Legality Rules*

- 5 If the mode is **in out** or **out**, the actual shall be a **name** that denotes a variable.
- 6/3 Two **names** are *known to denote the same object* if:
- 6.1/3 • both **names** statically denote the same stand-alone object or parameter; or
  - 6.2/3 • both **names** are **selected\_components**, their **prefixes** are known to denote the same object, and their **selector\_names** denote the same component; or
  - 6.3/3 • both **names** are dereferences (implicit or explicit) and the dereferenced **names** are known to denote the same object; or
  - 6.4/3 • both **names** are **indexed\_components**, their **prefixes** are known to denote the same object, and each of the pairs of corresponding index values are either static expressions with the same value or **names** that are known to denote the same object; or
  - 6.5/3 • both **names** are **slices**, their **prefixes** are known to denote the same object, and the two **slices** have statically matching index constraints; or
  - 6.6/3 • one of the two **names** statically denotes a renaming declaration whose renamed *object\_name* is known to denote the same object as the other, and every index **expression** for each **indexed\_component** which occurs within the renamed *object\_name* is a known to be unvarying **expression**, and the dereferenced name for each (implicit or explicit) dereference which occurs within the renamed *object\_name* is a known to be unvarying **name**.
- 6.7/3 A **name** or **expression** is *known to be unvarying* if it denotes:
- 6.8/3 • a static **expression**; or
  - 6.9/3 • a constant object; or
  - 6.10/3 • a non-aliased formal parameter of mode **in**; or
  - 6.11/3 • a **selected\_component** of a known to be unvarying **name**.
- 6.12/3 Two **names** are *known to refer to the same object* if
- 6.13/3 • The two **names** are known to denote the same object; or

- One of the names is a `selected_component`, `indexed_component`, or `slice` and its prefix is known to refer to the same object as the other name; or 6.14/3
- One of the two names statically denotes a renaming declaration whose renamed *object\_name* is known to refer to the same object as the other name. 6.15/3

If a call *C* has two or more parameters of mode **in out** or **out** that are of an elementary type, then the call is legal only if: 6.16/3

- For each name *N* that is passed as a parameter of mode **in out** or **out** to the call *C*, there is no other name among the other parameters of mode **in out** or **out** to *C* that is known to denote the same object. 6.17/3

If a construct *C* has two or more direct constituents that are names or expressions whose evaluation may occur in an arbitrary order, at least one of which contains a function call with an **in out** or **out** parameter, then the construct is legal only if: 6.18/3

- For each name *N* that is passed as a parameter of mode **in out** or **out** to some inner function call *C2* (not including the construct *C* itself), there is no other name anywhere within a direct constituent of the construct *C* other than the one containing *C2*, that is known to refer to the same object. 6.19/3

For the purposes of checking this rule: 6.20/3

- For an array `aggregate`, an expression associated with a `discrete_choice_list` that has two or more discrete choices, or that has a nonstatic range, is considered as two or more separate occurrences of the expression; 6.21/3
- For a record `aggregate`: 6.22/3
  - The expression of a `record_component_association` is considered to occur once for each associated component; and 6.23/3
  - The `default_expression` for each `record_component_association` with  $\diamond$  for which the associated component has a `default_expression` is considered part of the `aggregate`; 6.24/3
- For a call, any `default_expression` evaluated as part of the call is considered part of the call. 6.25/3

#### *Dynamic Semantics*

For the evaluation of a `parameter_association`: 7

- The actual parameter is first evaluated. 8
- For an access parameter, the `access_definition` is elaborated, which creates the anonymous access type. 9
- For a parameter (of any mode) that is passed by reference (see 6.2), a view conversion of the actual parameter to the nominal subtype of the formal parameter is evaluated, and the formal parameter denotes that conversion. 10
- For an **in** or **in out** parameter that is passed by copy (see 6.2), the formal parameter object is created, and the value of the actual parameter is converted to the nominal subtype of the formal parameter and assigned to the formal. 11
- For an **out** parameter that is passed by copy, the formal parameter object is created, and: 12
  - For an access type, the formal parameter is initialized from the value of the actual, without checking that the value satisfies any constraint or any exclusion of the null value; 13/3
  - For a composite type with discriminants or that has implicit initial values for any subcomponents (see 3.3.1), the behavior is as for an **in out** parameter passed by copy. 14

- 15
  - For any other type, the formal parameter is uninitialized. If composite, a view conversion of the actual parameter to the nominal subtype of the formal is evaluated (which might raise `Constraint_Error`), and the actual subtype of the formal is that of the view conversion. If elementary, the actual subtype of the formal is given by its nominal subtype.
- 16 A formal parameter of mode **in out** or **out** with discriminants is constrained if either its nominal subtype or the actual parameter is constrained.
- 17 After normal completion and leaving of a subprogram, for each **in out** or **out** parameter that is passed by copy, the value of the formal parameter is converted to the subtype of the variable given as the actual parameter and assigned to it. These conversions and assignments occur in an arbitrary order.

*Erroneous Execution*

- 17.1/3 If the nominal subtype of a formal parameter with discriminants is constrained or indefinite, and the parameter is passed by reference, then the execution of the call is erroneous if the value of any discriminant of the actual is changed while the formal parameter exists (that is, before leaving the corresponding callable construct).

## 6.5 Return Statements

- 1/2 A `simple_return_statement` or `extended_return_statement` (collectively called a *return statement*) is used to complete the execution of the innermost enclosing `subprogram_body`, `entry_body`, or `accept_statement`.

*Syntax*

- 2/2 `simple_return_statement ::= return [expression];`
- 2.1/3 `extended_return_statement ::=`  
`return defining_identifier : [constant] return_subtype_indication [:= expression] [do`  
`handled_sequence_of_statements`  
`end return];`
- 2.2/2 `return_subtype_indication ::= subtype_indication | access_definition`

*Name Resolution Rules*

- 3/2 The *result subtype* of a function is the subtype denoted by the `subtype_mark`, or defined by the `access_definition`, after the reserved word **return** in the profile of the function. The expected type for the expression, if any, of a `simple_return_statement` is the result type of the corresponding function. The expected type for the expression of an `extended_return_statement` is that of the `return_subtype_indication`.

*Legality Rules*

- 4/2 A return statement shall be within a callable construct, and it *applies to* the innermost callable construct or `extended_return_statement` that contains it. A return statement shall not be within a body that is within the construct to which the return statement applies.
- 5/3 A function body shall contain at least one return statement that applies to the function body, unless the function contains `code_statements`. A `simple_return_statement` shall include an expression if and only if it applies to a function body. An `extended_return_statement` shall apply to a function body. An `extended_return_statement` with the reserved word **constant** shall include an expression.
- 5.1/2 For an `extended_return_statement` that applies to a function body:



- If the result subtype of the function is defined by a `subtype_mark`, the `return_subtype_indication` shall be a `subtype_indication`. The type of the `subtype_indication` shall be covered by the result type of the function. The subtype defined by the `subtype_indication` shall be statically compatible with the result subtype of the function; if the result type of the function is elementary, the two subtypes shall statically match. If the result subtype of the function is indefinite, then the subtype defined by the `subtype_indication` shall be a definite subtype, or there shall be an `expression`. 5.2/3
- If the result subtype of the function is defined by an `access_definition`, the `return_subtype_indication` shall be an `access_definition`. The subtype defined by the `access_definition` shall statically match the result subtype of the function. The accessibility level of this anonymous access subtype is that of the result subtype. 5.3/2
- If the result subtype of the function is class-wide, the accessibility level of the type of the subtype defined by the `return_subtype_indication` shall not be statically deeper than that of the master that elaborated the function body. 5.4/3

For any return statement that applies to a function body: 5.5/3

- If the result subtype of the function is limited, then the `expression` of the return statement (if any) shall be an `aggregate`, a function call (or equivalent use of an operator), or a `qualified_expression` or parenthesized expression whose operand is one of these. 5.6/3
- If the result subtype of the function is class-wide, the accessibility level of the type of the `expression` (if any) of the return statement shall not be statically deeper than that of the master that elaborated the function body. If the result subtype has one or more unconstrained access discriminants, the accessibility level of the anonymous access type of each access discriminant, as determined by the `expression` of the `simple_return_statement` or the `return_subtype_indication`, shall not be statically deeper than that of the master that elaborated the function body. 5.7/3

#### *Static Semantics*

Within an `extended_return_statement`, the *return object* is declared with the given `defining_identifier`, with the nominal subtype defined by the `return_subtype_indication`. An `extended_return_statement` with the reserved word **constant** is a full constant declaration that declares the return object to be a constant object. 5.8/3

#### *Dynamic Semantics*

For the execution of an `extended_return_statement`, the `subtype_indication` or `access_definition` is elaborated. This creates the nominal subtype of the return object. If there is an `expression`, it is evaluated and converted to the nominal subtype (which might raise `Constraint_Error` — see 4.6); the return object is created and the converted value is assigned to the return object. Otherwise, the return object is created and initialized by default as for a stand-alone object of its nominal subtype (see 3.3.1). If the nominal subtype is indefinite, the return object is constrained by its initial value. A check is made that the value of the return object belongs to the function result subtype. `Constraint_Error` is raised if this check fails. 5.9/3

For the execution of a `simple_return_statement`, the `expression` (if any) is first evaluated, converted to the result subtype, and then is assigned to the anonymous *return object*. 6/2

If the return object has any parts that are tasks, the activation of those tasks does not occur until after the function returns (see 9.2). 7/2

If the result type of a function is a specific tagged type, the tag of the return object is that of the result type. If the result type is class-wide, the tag of the return object is that of the type of the `subtype_indication` if it is specific, or otherwise that of the value of the `expression`. A check is made that 8/3

the master of the type identified by the tag of the result includes the elaboration of the master that elaborated the function body. If this check fails, `Program_Error` is raised.

- 8.1/3 If the result subtype of the function is defined by an `access_definition` designating a specific tagged type *T*, a check is made that the result value is null or the tag of the object designated by the result value identifies *T*. `Constraint_Error` is raised if this check fails.

*Paragraphs 9 through 20 were deleted.*

- 21/2 If the result subtype of a function has one or more unconstrained access discriminants, a check is made that the accessibility level of the anonymous access type of each access discriminant, as determined by the expression or the `return_subtype_indication` of the function, is not deeper than that of the master that elaborated the function body. If this check fails, `Program_Error` is raised.
- 22/3 For the execution of an `extended_return_statement`, the `handled_sequence_of_statements` is executed. Within this `handled_sequence_of_statements`, the execution of a `simple_return_statement` that applies to the `extended_return_statement` causes a transfer of control that completes the `extended_return_statement`. Upon completion of a return statement that applies to a callable construct by the normal completion of a `simple_return_statement` or by reaching the **end return** of an `extended_return_statement`, a transfer of control is performed which completes the execution of the callable construct, and returns to the caller.
- 23/2 In the case of a function, the `function_call` denotes a constant view of the return object.

#### *Implementation Permissions*

- 24/3 For a function call used to initialize a composite object with a constrained nominal subtype or used to initialize a return object that is built in place into such an object:
- 24.1/3
- If the result subtype of the function is constrained, and conversion of an object of this subtype to the subtype of the object being initialized would raise `Constraint_Error`, then `Constraint_Error` may be raised before calling the function.
- 24.2/3
- If the result subtype of the function is unconstrained, and a return statement is executed such that the return object is known to be constrained, and conversion of the return object to the subtype of the object being initialized would raise `Constraint_Error`, then `Constraint_Error` may be raised at the point of the call (after abandoning the execution of the function body).

#### *Examples*

25 *Examples of return statements:*

```

26/2 return; -- in a procedure body, entry_body,
 -- accept_statement, or extended_return_statement

27 return Key_Value (Last_Index); -- in a function body

28/2 return Node : Cell do -- in a function body, see 3.10.1 for Cell
 Node.Value := Result;
 Node.Succ := Next_Node;
 end return;

```

## 6.5.1 Pragma No\_Return

- 1/2 A pragma `No_Return` indicates that a procedure cannot return normally; it may propagate an exception or loop forever.

#### *Syntax*

- 2/2 The form of a pragma `No_Return`, which is a representation pragma (see 13.1), is as follows:

**pragma** No\_Return(*procedure\_local\_name*{, *procedure\_local\_name*}); 3/2

*Legality Rules*

Each *procedure\_local\_name* shall denote one or more procedures or generic procedures; the denoted entities are *non-returning*. The *procedure\_local\_name* shall not denote a null procedure nor an instance of a generic unit. 4/2

A return statement shall not apply to a non-returning procedure or generic procedure. 5/2

A procedure shall be non-returning if it overrides a dispatching non-returning procedure. In addition to the places where Legality Rules normally apply (see 12.3), this rule applies also in the private part of an instance of a generic unit. 6/2

If a renaming-as-body completes a non-returning procedure declaration, then the renamed procedure shall be non-returning. 7/2

*Static Semantics*

If a generic procedure is non-returning, then so are its instances. If a procedure declared within a generic unit is non-returning, then so are the corresponding copies of that procedure in instances. 8/2

*Dynamic Semantics*

If the body of a non-returning procedure completes normally, Program\_Error is raised at the point of the call. 9/2

*Examples*

```
procedure Fail(Msg : String); -- raises Fatal_Error exception
pragma No_Return(Fail);
 -- Inform compiler and reader that procedure never returns normally
```

10/2

## 6.6 Overloading of Operators

An *operator* is a function whose *designator* is an *operator\_symbol*. Operators, like other functions, may be overloaded. 1

*Name Resolution Rules*

Each use of a unary or binary operator is equivalent to a *function\_call* with *function\_prefix* being the corresponding *operator\_symbol*, and with (respectively) one or two positional actual parameters being the operand(s) of the operator (in order). 2

*Legality Rules*

The *subprogram\_specification* of a unary or binary operator shall have one or two parameters, respectively. The parameters shall be of mode **in**. A generic function instantiation whose *designator* is an *operator\_symbol* is only allowed if the specification of the generic function has the corresponding number of parameters of mode **in** and no other parameters. 3/3

Default\_expressions are not allowed for the parameters of an operator (whether the operator is declared with an explicit *subprogram\_specification* or by a *generic\_instantiation*). 4

An explicit declaration of **"/="** shall not have a result type of the predefined type Boolean. 5

*Static Semantics*

- 6/3 An explicit declaration of "=" whose result type is Boolean implicitly declares an operator "/=" that gives the complementary result.

## NOTES

- 7 8 The operators "+" and "-" are both unary and binary operators, and hence may be overloaded with both one- and two-parameter functions.

*Examples*

- 8 *Examples of user-defined operators:*

- 9     **function** "+" (Left, Right : Matrix) **return** Matrix;  
       **function** "+" (Left, Right : Vector) **return** Vector;

-- assuming that A, B, and C are of the type Vector  
 -- the following two statements are equivalent:

A := B + C;  
 A := "+" (B, C) ;

## 6.7 Null Procedures

A `null_procedure_declaration` provides a shorthand to declare a procedure with an empty body.

1/2

### *Syntax*

```

null_procedure_declaration ::=
 [overriding_indicator]
 procedure_specification is null;

```

2/2

### *Static Semantics*

A `null_procedure_declaration` declares a *null procedure*. A completion is not allowed for a `null_procedure_declaration`.

3/2

### *Dynamic Semantics*

The execution of a null procedure is invoked by a subprogram call. For the execution of a subprogram call on a null procedure, the execution of the `subprogram_body` has no effect.

4/2

The elaboration of a `null_procedure_declaration` has no effect.

5/2

### *Examples*

```

procedure Simplify(Expr : in out Expression) is null; -- see 3.9
-- By default, Simplify does nothing, but it may be overridden in extensions of Expression

```

6/2



## Section 7: Packages

Packages are program units that allow the specification of groups of logically related entities. Typically, a package contains the declaration of a type (often a private type or private extension) along with the declarations of primitive subprograms of the type, which can be called from outside the package, while their inner workings remain hidden from outside users. 1

### 7.1 Package Specifications and Declarations

A package is generally provided in two parts: a `package_specification` and a `package_body`. Every package has a `package_specification`, but not all packages have a `package_body`. 1

#### Syntax

```
package_declaration ::= package_specification; 2
package_specification ::= 3
 package defining_program_unit_name is
 {basic_declarative_item}
 [private
 {basic_declarative_item}]
 end [[parent_unit_name.]identifier]
```

If an identifier or `parent_unit_name.identifier` appears at the end of a `package_specification`, then this sequence of lexical elements shall repeat the `defining_program_unit_name`. 4

#### Legality Rules

A `package_declaration` or `generic_package_declaration` requires a completion (a body) if it contains any `basic_declarative_item` that requires a completion, but whose completion is not in its `package_specification`. 5/2

#### Static Semantics

The first list of `basic_declarative_items` of a `package_specification` of a package other than a generic formal package is called the *visible part* of the package. The optional list of `basic_declarative_items` after the reserved word **private** (of any `package_specification`) is called the *private part* of the package. If the reserved word **private** does not appear, the package has an implicit empty private part. Each list of `basic_declarative_items` of a `package_specification` forms a *declaration list* of the package. 6/2

An entity declared in the private part of a package is visible only within the declarative region of the package itself (including any child units — see 10.1.1). In contrast, expanded names denoting entities declared in the visible part can be used even outside the package; furthermore, direct visibility of such entities can be achieved by means of `use_clauses` (see 4.1.3 and 8.4). 7

#### Dynamic Semantics

The elaboration of a `package_declaration` consists of the elaboration of its `basic_declarative_items` in the given order. 8

#### NOTES

- 1 The visible part of a package contains all the information that another program unit is able to know about the package. 9
- 2 If a declaration occurs immediately within the specification of a package, and the declaration has a corresponding completion that is a body, then that body has to occur immediately within the body of the package. 10

*Examples*

Example of a package declaration:

```

package Rational_Numbers is
 type Rational is
 record
 Numerator : Integer;
 Denominator : Positive;
 end record;
 function "=" (X,Y : Rational) return Boolean;
 function "/" (X,Y : Integer) return Rational; -- to construct a rational number
 function "+" (X,Y : Rational) return Rational;
 function "-" (X,Y : Rational) return Rational;
 function "*" (X,Y : Rational) return Rational;
 function "/" (X,Y : Rational) return Rational;
end Rational_Numbers;

```

There are also many examples of package declarations in the predefined language environment (see Annex A).

## 7.2 Package Bodies

In contrast to the entities declared in the visible part of a package, the entities declared in the `package_body` are visible only within the `package_body` itself. As a consequence, a package with a `package_body` can be used for the construction of a group of related subprograms in which the logical operations available to clients are clearly isolated from the internal entities.

*Syntax*

```

package_body ::=
 package body defining_program_unit_name is
 declarative_part
 [begin
 handled_sequence_of_statements]
 end [[parent_unit_name.]identifier];

```

If an identifier or `parent_unit_name.identifier` appears at the end of a `package_body`, then this sequence of lexical elements shall repeat the `defining_program_unit_name`.

*Legality Rules*

A `package_body` shall be the completion of a previous `package_declaration` or `generic_package_declaration`. A library `package_declaration` or library `generic_package_declaration` shall not have a body unless it requires a body; `pragma Elaborate_Body` can be used to require a `library_unit_declaration` to have a body (see 10.2.1) if it would not otherwise require one.

*Static Semantics*

In any `package_body` without statements there is an implicit `null_statement`. For any `package_declaration` without an explicit completion, there is an implicit `package_body` containing a single `null_statement`. For a noninstance, nonlibrary package, this body occurs at the end of the `declarative_part` of the innermost enclosing program unit or `block_statement`; if there are several such packages, the order of the implicit `package_bodies` is unspecified. (For an instance, the implicit `package_body` occurs at the place of the instantiation (see 12.3). For a library package, the place is partially determined by the elaboration dependences (see Section 10).)



*Dynamic Semantics*

For the elaboration of a nongeneric `package_body`, its `declarative_part` is first elaborated, and its `handled_sequence_of_statements` is then executed.

## NOTES

3 A variable declared in the body of a package is only visible within this body and, consequently, its value can only be changed within the `package_body`. In the absence of local tasks, the value of such a variable remains unchanged between calls issued from outside the package to subprograms declared in the visible part. The properties of such a variable are similar to those of a “static” variable of C.

4 The elaboration of the body of a subprogram explicitly declared in the visible part of a package is caused by the elaboration of the body of the package. Hence a call of such a subprogram by an outside program unit raises the exception `Program_Error` if the call takes place before the elaboration of the `package_body` (see 3.11).

*Examples*

*Example of a package body (see 7.1):*

```

package body Rational_Numbers is
 procedure Same_Denominator (X,Y : in out Rational) is
 begin
 -- reduces X and Y to the same denominator:
 ...
 end Same_Denominator;
 function "=" (X,Y : Rational) return Boolean is
 U : Rational := X;
 V : Rational := Y;
 begin
 Same_Denominator (U,V);
 return U.Numerator = V.Numerator;
 end "=";
 function "/" (X,Y : Integer) return Rational is
 begin
 if Y > 0 then
 return (Numerator => X, Denominator => Y);
 else
 return (Numerator => -X, Denominator => -Y);
 end if;
 end "/";
 function "+" (X,Y : Rational) return Rational is ... end "+";
 function "-" (X,Y : Rational) return Rational is ... end "-";
 function "*" (X,Y : Rational) return Rational is ... end "*";
 function "/" (X,Y : Rational) return Rational is ... end "/";
end Rational_Numbers;

```

## 7.3 Private Types and Private Extensions

The declaration (in the visible part of a package) of a type as a private type or private extension serves to separate the characteristics that can be used directly by outside program units (that is, the logical properties) from other characteristics whose direct use is confined to the package (the details of the definition of the type itself). See 3.9.1 for an overview of type extensions.

*Syntax*

```

private_type_declaration ::=
 type defining_identifier [discriminant_part] is [[abstract] tagged] [[limited] private];

```

3/2      `private_extension_declaration ::=`  
          `type defining_identifier [discriminant_part] is`  
              `[abstract] [limited | synchronized] new ancestor_subtype_indication`  
              `[and interface_list] with private;`

*Legality Rules*

- 4      A `private_type_declaration` or `private_extension_declaration` declares a *partial view* of the type; such a declaration is allowed only as a `declarative_item` of the visible part of a package, and it requires a completion, which shall be a `full_type_declaration` that occurs as a `declarative_item` of the private part of the package. The view of the type declared by the `full_type_declaration` is called the *full view*. A generic formal private type or a generic formal private extension is also a partial view.
- 5      A type shall be completely defined before it is frozen (see 3.11.1 and 13.14). Thus, neither the declaration of a variable of a partial view of a type, nor the creation by an `allocator` of an object of the partial view are allowed before the full declaration of the type. Similarly, before the full declaration, the name of the partial view cannot be used in a `generic_instantiation` or in a representation item.
- 6/2    A private type is limited if its declaration includes the reserved word **limited**; a private extension is limited if its ancestor type is a limited type that is not an interface type, or if the reserved word **limited** or **synchronized** appears in its definition. If the partial view is nonlimited, then the full view shall be nonlimited. If a tagged partial view is limited, then the full view shall be limited. On the other hand, if an untagged partial view is limited, the full view may be limited or nonlimited.
- 7      If the partial view is tagged, then the full view shall be tagged. On the other hand, if the partial view is untagged, then the full view may be tagged or untagged. In the case where the partial view is untagged and the full view is tagged, no derivatives of the partial view are allowed within the immediate scope of the partial view; derivatives of the full view are allowed.
- 7.1/2   If a full type has a partial view that is tagged, then:
  - 7.2/2    • the partial view shall be a synchronized tagged type (see 3.9.4) if and only if the full type is a synchronized tagged type;
  - 7.3/2    • the partial view shall be a descendant of an interface type (see 3.9.4) if and only if the full type is a descendant of the interface type.
- 8      The *ancestor\_subtype* of a `private_extension_declaration` is the subtype defined by the *ancestor\_subtype\_indication*; the ancestor type shall be a specific tagged type. The full view of a private extension shall be derived (directly or indirectly) from the ancestor type. In addition to the places where Legality Rules normally apply (see 12.3), the requirement that the ancestor be specific applies also in the private part of an instance of a generic unit.
- 8.1/2   If the reserved word **limited** appears in a `private_extension_declaration`, the ancestor type shall be a limited type. If the reserved word **synchronized** appears in a `private_extension_declaration`, the ancestor type shall be a limited interface.
- 9      If the declaration of a partial view includes a `known_discriminant_part`, then the `full_type_declaration` shall have a fully conforming (explicit) `known_discriminant_part` (see 6.3.1, “Conformance Rules”). The ancestor subtype may be unconstrained; the parent subtype of the full view is required to be constrained (see 3.7).
- 10     If a private extension inherits known discriminants from the ancestor subtype, then the full view shall also inherit its discriminants from the ancestor subtype, and the parent subtype of the full view shall be constrained if and only if the ancestor subtype is constrained.

If the `full_type_declaration` for a private extension includes a `derived_type_definition`, then the reserved word **limited** shall appear in the `full_type_declaration` if and only if it also appears in the `private_extension_declaration`. 10.1/3

If a partial view has unknown discriminants, then the `full_type_declaration` may define a definite or an indefinite subtype, with or without discriminants. 11

If a partial view has neither known nor unknown discriminants, then the `full_type_declaration` shall define a definite subtype. 12

If the ancestor subtype of a private extension has constrained discriminants, then the parent subtype of the full view shall impose a statically matching constraint on those discriminants. 13

#### *Static Semantics*

A `private_type_declaration` declares a private type and its first subtype. Similarly, a `private_extension_declaration` declares a private extension and its first subtype. 14

A declaration of a partial view and the corresponding `full_type_declaration` define two views of a single type. The declaration of a partial view together with the visible part define the operations that are available to outside program units; the declaration of the full view together with the private part define other operations whose direct use is possible only within the declarative region of the package itself. Moreover, within the scope of the declaration of the full view, the *characteristics* of the type are determined by the full view; in particular, within its scope, the full view determines the classes that include the type, which components, entries, and protected subprograms are visible, what attributes and other predefined operations are allowed, and whether the first subtype is static. See 7.3.1. 15

A private extension inherits components (including discriminants unless there is a new `discriminant_part` specified) and user-defined primitive subprograms from its ancestor type and its progenitor types (if any), in the same way that a record extension inherits components and user-defined primitive subprograms from its parent type and its progenitor types (see 3.4). 16/2

#### *Dynamic Semantics*

The elaboration of a `private_type_declaration` creates a partial view of a type. The elaboration of a `private_extension_declaration` elaborates the `ancestor_subtype_indication`, and creates a partial view of a type. 17

#### NOTES

5 The partial view of a type as declared by a `private_type_declaration` is defined to be a composite view (in 3.2). The full view of the type might or might not be composite. A private extension is also composite, as is its full view. 18

6 Declaring a private type with an `unknown_discriminant_part` is a way of preventing clients from creating uninitialized objects of the type; they are then forced to initialize each object by calling some operation declared in the visible part of the package. 19/2

7 The ancestor type specified in a `private_extension_declaration` and the parent type specified in the corresponding declaration of a record extension given in the private part need not be the same. If the ancestor type is not an interface type, the parent type of the full view can be any descendant of the ancestor type. In this case, for a primitive subprogram that is inherited from the ancestor type and not overridden, the formal parameter names and default expressions (if any) come from the corresponding primitive subprogram of the specified ancestor type, while the body comes from the corresponding primitive subprogram of the parent type of the full view. See 3.9.2. 20/2

8 If the ancestor type specified in a `private_extension_declaration` is an interface type, the parent type can be any type so long as the full view is a descendant of the ancestor type. The progenitor types specified in a `private_extension_declaration` and the progenitor types specified in the corresponding declaration of a record extension given in the private part need not be the same — the only requirement is that the private extension and the record extension be descended from the same set of interfaces. 20.1/2

*Examples*

21 *Examples of private type declarations:*

22     **type** Key **is private**;  
       **type** File\_Name **is limited private**;

23 *Example of a private extension declaration:*

24     **type** List **is new** Ada.Finalization.Controlled **with private**;

### 7.3.1 Private Operations

- 1 For a type declared in the visible part of a package or generic package, certain operations on the type do not become visible until later in the package — either in the private part or the body. Such *private operations* are available only inside the declarative region of the package or generic package.

*Static Semantics*

- 2 The predefined operators that exist for a given type are determined by the classes to which the type belongs. For example, an integer type has a predefined "+" operator. In most cases, the predefined operators of a type are declared immediately after the definition of the type; the exceptions are explained below. Inherited subprograms are also implicitly declared immediately after the definition of the type, except as stated below.
- 3/3 For a composite type, the characteristics (see 7.3) of the type are determined in part by the characteristics of its component types. At the place where the composite type is declared, the only characteristics of component types used are those characteristics visible at that place. If later immediately within the declarative region in which the composite type is declared additional characteristics become visible for a component type, then any corresponding characteristics become visible for the composite type. Any additional predefined operators are implicitly declared at that place. If there is no such place, then additional predefined operators are not declared at all, but they still exist.
- 4/1 The corresponding rule applies to a type defined by a **derived\_type\_definition**, if there is a place immediately within the declarative region in which the type is declared where additional characteristics of its parent type become visible.
- 5/1 For example, an array type whose component type is limited private becomes nonlimited if the full view of the component type is nonlimited and visible at some later place immediately within the declarative region in which the array type is declared. In such a case, the predefined "=" operator is implicitly declared at that place, and assignment is allowed after that place.
- 6/3 Inherited primitive subprograms follow a different rule. For a **derived\_type\_definition**, each inherited primitive subprogram is implicitly declared at the earliest place, if any, immediately within the declarative region in which the **type\_declaration** occurs, but after the **type\_declaration**, where the corresponding declaration from the parent is visible. If there is no such place, then the inherited subprogram is not declared at all, but it still exists. For a tagged type, it is possible to dispatch to an inherited subprogram that is not declared at all.
- 7 For a **private\_extension\_declaration**, each inherited subprogram is declared immediately after the **private\_extension\_declaration** if the corresponding declaration from the ancestor is visible at that place. Otherwise, the inherited subprogram is not declared for the private extension, though it might be for the full type.
- 8 The Class attribute is defined for tagged subtypes in 3.9. In addition, for every subtype S of an untagged private type whose full view is tagged, the following attribute is defined:

**S'Class** Denotes the class-wide subtype corresponding to the full view of S. This attribute is allowed only from the beginning of the private part in which the full view is declared, until the declaration of the full view. After the full view, the Class attribute of the full view can be used. 9

#### NOTES

9 Because a partial view and a full view are two different views of one and the same type, outside of the defining package the characteristics of the type are those defined by the visible part. Within these outside program units the type is just a private type or private extension, and any language rule that applies only to another class of types does not apply. The fact that the full declaration might implement a private type with a type of a particular class (for example, as an array type) is relevant only within the declarative region of the package itself including any child units. 10

The consequences of this actual implementation are, however, valid everywhere. For example: any default initialization of components takes place; the attribute Size provides the size of the full view; finalization is still done for controlled components of the full view; task dependence rules still apply to components that are task objects. 11

10 Partial views provide initialization, membership tests, selected components for the selection of discriminants and inherited components, qualification, and explicit conversion. Nonlimited partial views also allow use of assignment\_statements. 12/2

11 For a subtype S of a partial view, S'Size is defined (see 13.3). For an object A of a partial view, the attributes A'Size and A'Address are defined (see 13.3). The Position, First\_Bit, and Last\_Bit attributes are also defined for discriminants and inherited components. 13

#### Examples

*Example of a type with private operations:* 14

```

package Key_Manager is
 type Key is private;
 Null_Key : constant Key; -- a deferred constant declaration (see 7.4)
 procedure Get_Key(K : out Key);
 function "<" (X, Y : Key) return Boolean;
private
 type Key is new Natural;
 Null_Key : constant Key := Key'First;
end Key_Manager;

package body Key_Manager is
 Last_Key : Key := Null_Key;
 procedure Get_Key(K : out Key) is
 begin
 Last_Key := Last_Key + 1;
 K := Last_Key;
 end Get_Key;

 function "<" (X, Y : Key) return Boolean is
 begin
 return Natural(X) < Natural(Y);
 end "<";
end Key_Manager;

```

#### NOTES

12 *Notes on the example:* Outside of the package Key\_Manager, the operations available for objects of type Key include assignment, the comparison for equality or inequality, the procedure Get\_Key and the operator "<"; they do not include other relational operators such as ">=", or arithmetic operators. 18

The explicitly declared operator "<" hides the predefined operator "<" implicitly declared by the full\_type\_declaration. Within the body of the function, an explicit conversion of X and Y to the subtype Natural is necessary to invoke the "<" operator of the parent type. Alternatively, the result of the function could be written as not (X >= Y), since the operator ">=" is not redefined. 19

The value of the variable Last\_Key, declared in the package body, remains unchanged between calls of the procedure Get\_Key. (See also the NOTES of 7.2.) 20

## 7.4 Deferred Constants

- 1 Deferred constant declarations may be used to declare constants in the visible part of a package, but with the value of the constant given in the private part. They may also be used to declare constants imported from other languages (see Annex B).

### *Legality Rules*

- 2 A *deferred constant declaration* is an `object_declaration` with the reserved word **constant** but no initialization expression. The constant declared by a deferred constant declaration is called a *deferred constant*. A deferred constant declaration requires a completion, which shall be a full constant declaration (called the *full declaration* of the deferred constant), or a `pragma Import` (see Annex B).
- 3 A deferred constant declaration that is completed by a full constant declaration shall occur immediately within the visible part of a `package_specification`. For this case, the following additional rules apply to the corresponding full declaration:
- 4 • The full declaration shall occur immediately within the private part of the same package;
  - 5/2 • The deferred and full constants shall have the same type, or shall have statically matching anonymous access subtypes;
  - 6/3 • If the deferred constant declaration includes a `subtype_indication` that defines a constrained subtype, then the constraint defined by the `subtype_indication` in the full declaration shall match it statically. On the other hand, if the subtype of the deferred constant is unconstrained, then the full declaration is still allowed to impose a constraint. The constant itself will be constrained, like all constants;
  - 7/2 • If the deferred constant declaration includes the reserved word **aliased**, then the full declaration shall also;
  - 7.1/2 • If the subtype of the deferred constant declaration excludes null, the subtype of the full declaration shall also exclude null.
- 8 A deferred constant declaration that is completed by a `pragma Import` need not appear in the visible part of a `package_specification`, and has no full constant declaration.
- 9/2 The completion of a deferred constant declaration shall occur before the constant is frozen (see 13.14).

### *Dynamic Semantics*

- 10/3 The elaboration of a deferred constant declaration elaborates the `subtype_indication`, `access_definition`, or (only allowed in the case of an imported constant) the `array_type_definition`.

#### NOTES

- 11 13 The full constant declaration for a deferred constant that is of a given private type or private extension is not allowed before the corresponding `full_type_declaration`. This is a consequence of the freezing rules for types (see 13.14).

### *Examples*

- 12 *Examples of deferred constant declarations:*

```

13 Null_Key : constant Key; -- see 7.3.1
14 CPU_Identifier : constant String(1..8);
 pragma Import(Assembler, CPU_Identifier, Link_Name => "CPU_ID");
 -- see B.1

```

## 7.5 Limited Types

A limited type is (a view of) a type for which copying (such as for an `assignment_statement`) is not allowed. A nonlimited type is a (view of a) type for which copying is allowed. 1/2

### *Legality Rules*

If a tagged record type has any limited components, then the reserved word **limited** shall appear in its `record_type_definition`. If the reserved word **limited** appears in the definition of a `derived_type_definition`, its parent type and any progenitor interfaces shall be limited. 2/2

In the following contexts, an expression of a limited type is not permitted unless it is an **aggregate**, a `function_call`, or a parenthesized expression or `qualified_expression` whose operand is permitted by this rule: 2.1/2

- the initialization expression of an `object_declaration` (see 3.3.1) 2.2/2
- the `default_expression` of a `component_declaration` (see 3.8) 2.3/2
- the expression of a `record_component_association` (see 4.3.1) 2.4/2
- the expression for an `ancestor_part` of an `extension_aggregate` (see 4.3.2) 2.5/2
- an expression of a `positional_array_aggregate` or the expression of an `array_component_association` (see 4.3.3) 2.6/2
- the `qualified_expression` of an initialized allocator (see 4.8) 2.7/2
- the expression of a return statement (see 6.5) 2.8/2
- the `default_expression` or actual parameter for a formal object of mode **in** (see 12.4) 2.9/2

### *Static Semantics*

A view of a type is *limited* if it is one of the following: 3/3

- a type with the reserved word **limited**, **synchronized**, **task**, or **protected** in its definition; 4/2
- a class-wide type whose specific type is limited; 5/3
- a composite type with a limited component; 6/2
- an incomplete view; 6.1/3
- a derived type whose parent is limited and is not an interface. 6.2/2

Otherwise, the type is nonlimited. 7

There are no predefined equality operators for a limited type. 8

A type is *immutably limited* if it is one of the following: 8.1/3

- An explicitly limited record type; 8.2/3
- A non-formal limited private type that is tagged or has at least one access discriminant with a `default_expression`; 8.3/3
- A task type, a protected type, or a synchronized interface; 8.4/3
- A type derived from an immutably limited type. 8.5/3

- 8.6/3 A descendant of a generic formal limited private type is presumed to be immutably limited except within the body of a generic unit or a body declared within the declarative region of a generic unit, if the formal type is declared within the formal part of the generic unit.

#### Implementation Requirements

- 8.7/3 *This paragraph was deleted.*

#### NOTES

- 9/3 14 While it is allowed to write initializations of limited objects, such initializations never copy a limited object. The source of such an assignment operation must be an **aggregate** or **function\_call**, and such **aggregates** and **function\_calls** must be built directly in the target object (see 7.6).

*Paragraphs 10 through 15 were deleted.*

- 16 15 As illustrated in 7.3.1, an untagged limited type can become nonlimited under certain circumstances.

#### Examples

- 17 *Example of a package with a limited type:*

```

18 package IO_Package is
 type File_Name is limited private;
19 procedure Open (F : in out File_Name);
 procedure Close(F : in out File_Name);
 procedure Read (F : in File_Name; Item : out Integer);
 procedure Write(F : in File_Name; Item : in Integer);
private
 type File_Name is
 limited record
 Internal_Name : Integer := 0;
 end record;
end IO_Package;

20 package body IO_Package is
 Limit : constant := 200;
 type File_Descriptor is record ... end record;
 Directory : array (1 .. Limit) of File_Descriptor;
 ...
 procedure Open (F : in out File_Name) is ... end;
 procedure Close(F : in out File_Name) is ... end;
 procedure Read (F : in File_Name; Item : out Integer) is ... end;
 procedure Write(F : in File_Name; Item : in Integer) is ... end;
begin
 ...
end IO_Package;
```

#### NOTES

- 21 16 *Notes on the example:* In the example above, an outside subprogram making use of **IO\_Package** may obtain a file name by calling **Open** and later use it in calls to **Read** and **Write**. Thus, outside the package, a file name obtained from **Open** acts as a kind of password; its internal properties (such as containing a numeric value) are not known and no other operations (such as addition or comparison of internal names) can be performed on a file name. Most importantly, clients of the package cannot make copies of objects of type **File\_Name**.
- 22 This example is characteristic of any case where complete control over the operations of a type is desired. Such packages serve a dual purpose. They prevent a user from making use of the internal structure of the type. They also implement the notion of an encapsulated data type where the only operations on the type are those given in the package specification.
- 23/2 The fact that the full view of **File\_Name** is explicitly declared **limited** means that parameter passing will always be by reference and function results will always be built directly in the result object (see 6.2 and 6.5).



## 7.6 Assignment and Finalization

Three kinds of actions are fundamental to the manipulation of objects: initialization, finalization, and assignment. Every object is initialized, either explicitly or by default, after being created (for example, by an `object_declaration` or `allocator`). Every object is finalized before being destroyed (for example, by leaving a `subprogram_body` containing an `object_declaration`, or by a call to an instance of `Unchecked_Deallocation`). An assignment operation is used as part of `assignment_statements`, explicit initialization, parameter passing, and other operations.

Default definitions for these three fundamental operations are provided by the language, but a *controlled* type gives the user additional control over parts of these operations. In particular, the user can define, for a controlled type, an `Initialize` procedure which is invoked immediately after the normal default initialization of a controlled object, a `Finalize` procedure which is invoked immediately before finalization of any of the components of a controlled object, and an `Adjust` procedure which is invoked as the last step of an assignment to a (nonlimited) controlled object.

### Static Semantics

The following language-defined library package exists:

```

package Ada.Finalization is
 pragma Preelaborate(Finalization);
 pragma Remote_Types(Finalization);
 type Controlled is abstract tagged private;
 pragma Preelaborable_Initialization(Controlled);
 procedure Initialize (Object : in out Controlled) is null;
 procedure Adjust (Object : in out Controlled) is null;
 procedure Finalize (Object : in out Controlled) is null;
 type Limited_Controlled is abstract tagged limited private;
 pragma Preelaborable_Initialization(Limited_Controlled);
 procedure Initialize (Object : in out Limited_Controlled) is null;
 procedure Finalize (Object : in out Limited_Controlled) is null;
private
 ... -- not specified by the language
end Ada.Finalization;
```

A controlled type is a descendant of `Controlled` or `Limited_Controlled`. The predefined "=" operator of type `Controlled` always returns `True`, since this operator is incorporated into the implementation of the predefined equality operator of types derived from `Controlled`, as explained in 4.5.2. The type `Limited_Controlled` is like `Controlled`, except that it is limited and it lacks the primitive subprogram `Adjust`.

A type is said to *need finalization* if:

- it is a controlled type, a task type or a protected type; or
- it has a component that needs finalization; or
- it is a class-wide type; or
- it is a partial view whose full view needs finalization; or
- it is one of a number of language-defined types that are explicitly defined to need finalization.

### Dynamic Semantics

During the elaboration or evaluation of a construct that causes an object to be initialized by default, for every controlled subcomponent of the object that is not assigned an initial value (as defined in 3.3.1),

Initialize is called on that subcomponent. Similarly, if the object that is initialized by default as a whole is controlled, Initialize is called on the object.

- 11/2 For an **extension\_aggregate** whose **ancestor\_part** is a **subtype\_mark** denoting a controlled subtype, the Initialize procedure of the ancestor type is called, unless that Initialize procedure is abstract.
- 12 Initialize and other initialization operations are done in an arbitrary order, except as follows. Initialize is applied to an object after initialization of its subcomponents, if any (including both implicit initialization and Initialize calls). If an object has a component with an access discriminant constrained by a per-object expression, Initialize is applied to this component after any components that do not have such discriminants. For an object with several components with such a discriminant, Initialize is applied to them in order of their **component\_declarations**. For an **allocator**, any task activations follow all calls on Initialize.
- 13 When a target object with any controlled parts is assigned a value, either when created or in a subsequent **assignment\_statement**, the *assignment operation* proceeds as follows:
- 14 • The value of the target becomes the assigned value.
  - 15 • The value of the target is *adjusted*.
- 16/3 To adjust the value of a composite object, the values of the components of the object are first adjusted in an arbitrary order, and then, if the object is nonlimited controlled, Adjust is called. Adjusting the value of an elementary object has no effect, nor does adjusting the value of a composite object with no controlled parts.
- 17 For an **assignment\_statement**, after the name and expression have been evaluated, and any conversion (including constraint checking) has been done, an anonymous object is created, and the value is assigned into it; that is, the assignment operation is applied. (Assignment includes value adjustment.) The target of the **assignment\_statement** is then finalized. The value of the anonymous object is then assigned into the target of the **assignment\_statement**. Finally, the anonymous object is finalized. As explained below, the implementation may eliminate the intermediate anonymous object, so this description subsumes the one given in 5.2, “Assignment Statements”.
- 17.1/3 When a function call or **aggregate** is used to initialize an object, the result of the function call or **aggregate** is an anonymous object, which is assigned into the newly-created object. For such an assignment, the anonymous object might be built in place. Under certain circumstances, the anonymous object is required to be *built in place*, in which case the assignment does not involve any copying. In particular:
- 17.2/3 • If the full type of any part of the object is immutably limited, the anonymous object is built in place.
  - 17.3/3 • In the case of an **aggregate**, if the full type of any part of the newly-created object is controlled, the anonymous object is built in place.
  - 17.4/3 • In other cases, it is unspecified whether the anonymous object is built in place.
- 17.5/3 Notwithstanding what this International Standard says elsewhere, if an object is built in place:
- 17.6/3 • Upon successful completion of the return statement or **aggregate**, the anonymous object “mutates into” the newly-created object; that is, the anonymous object ceases to exist, and the newly-created object appears in its place.
  - 17.7/3 • Finalization is not performed on the anonymous object.
  - 17.8/3 • Adjustment is not performed on the newly-created object.

- All access values that designate parts of the anonymous object now designate the corresponding parts of the newly-created object. 17.9/3
- All renamings of parts of the anonymous object now denote views of the corresponding parts of the newly-created object. 17.10/3
- Coextensions of the anonymous object become coextensions of the newly-created object. 17.11/3

*Implementation Requirements*

*This paragraph was deleted.*

17.12/3

*Implementation Permissions*

An implementation is allowed to relax the above rules for **assignment\_statements** in the following ways: 18/3

- If an object is assigned the value of that same object, the implementation need not do anything. 19/3
- For assignment of a noncontrolled type, the implementation may finalize and assign each component of the variable separately (rather than finalizing the entire variable and assigning the entire new value) unless a discriminant of the variable is changed by the assignment. 20/3
- The implementation need not create an anonymous object if the value being assigned is the result of evaluating a **name** denoting an object (the source object) whose storage cannot overlap with the target. If the source object might overlap with the target object, then the implementation can avoid the need for an intermediary anonymous object by exercising one of the above permissions and perform the assignment one component at a time (for an overlapping array assignment), or not at all (for an assignment where the target and the source of the assignment are the same object). 21/3

Furthermore, an implementation is permitted to omit implicit Initialize, Adjust, and Finalize calls and associated assignment operations on an object of a nonlimited controlled type provided that: 22/2

- any omitted Initialize call is not a call on a user-defined Initialize procedure, and 23/2
- any usage of the value of the object after the implicit Initialize or Adjust call and before any subsequent Finalize call on the object does not change the external effect of the program, and 24/2
- after the omission of such calls and operations, any execution of the program that executes an Initialize or Adjust call on an object or initializes an object by an **aggregate** will also later execute a Finalize call on the object and will always do so prior to assigning a new value to the object, and 25/2
- the assignment operations associated with omitted Adjust calls are also omitted. 26/2

This permission applies to Adjust and Finalize calls even if the implicit calls have additional external effects. 27/2

## 7.6.1 Completion and Finalization

This subclause defines *completion* and *leaving* of the execution of constructs and entities. A *master* is the execution of a construct that includes finalization of local objects after it is complete (and after waiting for any local tasks — see 9.3), but before leaving. Other constructs and entities are left immediately upon completion. 1

*Dynamic Semantics*

The execution of a construct or entity is *complete* when the end of that execution has been reached, or when a transfer of control (see 5.1) causes it to be abandoned. Completion due to reaching the end of execution, or due to the transfer of control of an **exit\_statement**, **return statement**, **goto\_statement**, or 2/2

requeue\_statement or of the selection of a terminate\_alternative is *normal completion*. Completion is *abnormal* otherwise — when control is transferred out of a construct due to abort or the raising of an exception.

- 3/2 After execution of a construct or entity is complete, it is *left*, meaning that execution continues with the next action, as defined for the execution that is taking place. Leaving an execution happens immediately after its completion, except in the case of a *master*: the execution of a body other than a package\_body; the execution of a statement; or the evaluation of an expression, function\_call, or range that is not part of an enclosing expression, function\_call, range, or simple\_statement other than a simple\_return\_-statement. A master is finalized after it is complete, and before it is left.
- 4 For the *finalization* of a master, dependent tasks are first awaited, as explained in 9.3. Then each object whose accessibility level is the same as that of the master is finalized if the object was successfully initialized and still exists. These actions are performed whether the master is left by reaching the last statement or via a transfer of control. When a transfer of control causes completion of an execution, each included master is finalized in order, from innermost outward.
- 5 For the *finalization* of an object:
- 6/3 • If the full type of the object is an elementary type, finalization has no effect;
  - 7/3 • If the full type of the object is a tagged type, and the tag of the object identifies a controlled type, the Finalize procedure of that controlled type is called;
  - 8/3 • If the full type of the object is a protected type, or if the full type of the object is a tagged type and the tag of the object identifies a protected type, the actions defined in 9.4 are performed;
  - 9/3 • If the full type of the object is a composite type, then after performing the above actions, if any, every component of the object is finalized in an arbitrary order, except as follows: if the object has a component with an access discriminant constrained by a per-object expression, this component is finalized before any components that do not have such discriminants; for an object with several components with such a discriminant, they are finalized in the reverse of the order of their component\_declarations;
  - 9.1/2 • If the object has coextensions (see 3.10.2), each coextension is finalized after the object whose access discriminant designates it.
- 10 Immediately before an instance of Unchecked\_Deallocation reclaims the storage of an object, the object is finalized. If an instance of Unchecked\_Deallocation is never applied to an object created by an allocator, the object will still exist when the corresponding master completes, and it will be finalized then.
- 11/2 The order in which the finalization of a master performs finalization of objects is as follows: Objects created by declarations in the master are finalized in the reverse order of their creation. For objects that were created by allocators for an access type whose ultimate ancestor is declared in the master, this rule is applied as though each such object that still exists had been created in an arbitrary order at the first freezing point (see 13.14) of the ultimate ancestor type; the finalization of these objects is called the *finalization of the collection*. After the finalization of a master is complete, the objects finalized as part of its finalization cease to *exist*, as do any types and subtypes defined and created within the master.
- 12/2 The target of an assignment\_statement is finalized before copying in the new value, as explained in 7.6.
- 13/3 The master of an object is the master enclosing its creation whose accessibility level (see 3.10.2) is equal to that of the object, except in the case of an anonymous object representing the result of an aggregate or function call. The master of such an anonymous object is the innermost master enclosing the evaluation of the aggregate or function call, which may be the aggregate or function call itself.

In the case of an **expression** that is a master, finalization of any (anonymous) objects occurs after completing evaluation of the **expression** and use of the objects, prior to starting the execution of any subsequent construct. 13.1/3

#### *Bounded (Run-Time) Errors*

It is a bounded error for a call on Finalize or Adjust that occurs as part of object finalization or assignment to propagate an exception. The possible consequences depend on what action invoked the Finalize or Adjust operation: 14/1

- For a Finalize invoked as part of an **assignment\_statement**, **Program\_Error** is raised at that point. 15
- For an Adjust invoked as part of assignment operations other than those invoked as part of an **assignment\_statement**, other adjustments due to be performed might or might not be performed, and then **Program\_Error** is raised. During its propagation, finalization might or might not be applied to objects whose Adjust failed. For an Adjust invoked as part of an **assignment\_statement**, any other adjustments due to be performed are performed, and then **Program\_Error** is raised. 16/2
- For a Finalize invoked as part of a call on an instance of **Unchecked\_Deallocation**, any other finalizations due to be performed are performed, and then **Program\_Error** is raised. 17
- *This paragraph was deleted.* 17.1/3
- For a Finalize invoked due to reaching the end of the execution of a master, any other finalizations associated with the master are performed, and **Program\_Error** is raised immediately after leaving the master. 17.2/1
- For a Finalize invoked by the transfer of control of an **exit\_statement**, **return statement**, **goto\_statement**, or **requeue\_statement**, **Program\_Error** is raised no earlier than after the finalization of the master being finalized when the exception occurred, and no later than the point where normal execution would have continued. Any other finalizations due to be performed up to that point are performed before raising **Program\_Error**. 18/2
- For a Finalize invoked by a transfer of control that is due to raising an exception, any other finalizations due to be performed for the same master are performed; **Program\_Error** is raised immediately after leaving the master. 19
- For a Finalize invoked by a transfer of control due to an abort or selection of a terminate alternative, the exception is ignored; any other finalizations due to be performed are performed. 20

#### *Implementation Permissions*

If the execution of an **allocator** propagates an exception, any parts of the allocated object that were successfully initialized may be finalized as part of the finalization of the innermost master enclosing the **allocator**. 20.1/3

#### NOTES

- 17 The rules of Section 10 imply that immediately prior to partition termination, Finalize operations are applied to library-level controlled objects (including those created by **allocators** of library-level access types, except those already finalized). This occurs after waiting for library-level tasks to terminate. 21
- 18 A constant is only constant between its initialization and finalization. Both initialization and finalization are allowed to change the value of a constant. 22
- 19 Abort is deferred during certain operations related to controlled types, as explained in 9.8. Those rules prevent an abort from causing a controlled object to be left in an ill-defined state. 23
- 20 The Finalize procedure is called upon finalization of a controlled object, even if Finalize was called earlier, either explicitly or as part of an assignment; hence, if a controlled type is visibly controlled (implying that its Finalize primitive 24

is directly callable), or is nonlimited (implying that assignment is allowed), its Finalize procedure should be designed to have no ill effect if it is applied a second time to the same object.

## Section 8: Visibility Rules

The rules defining the scope of declarations and the rules defining which identifiers, `character_literals`, and `operator_symbols` are visible at (or from) various places in the text of the program are described in this section. The formulation of these rules uses the notion of a declarative region.

As explained in Section 3, a declaration declares a view of an entity and associates a defining name with that view. The view comprises an identification of the viewed entity, and possibly additional properties. A usage name denotes a declaration. It also denotes the view declared by that declaration, and denotes the entity of that view. Thus, two different usage names might denote two different views of the same entity; in this case they denote the same entity.

### 8.1 Declarative Region

#### *Static Semantics*

For each of the following constructs, there is a portion of the program text called its *declarative region*, within which nested declarations can occur:

- any declaration, other than that of an enumeration type, that is not a completion of a previous declaration;
- a `block_statement`;
- a `loop_statement`;
- an `extended_return_statement`;
- an `accept_statement`;
- an `exception_handler`.

The declarative region includes the text of the construct together with additional text determined (recursively), as follows:

- If a declaration is included, so is its completion, if any.
- If the declaration of a library unit (including Standard — see 10.1.1) is included, so are the declarations of any child units (and their completions, by the previous rule). The child declarations occur after the declaration.
- If a `body_stub` is included, so is the corresponding subunit.
- If a `type_declaration` is included, then so is a corresponding `record_representation_clause`, if any.

The declarative region of a declaration is also called the *declarative region* of any view or entity declared by the declaration.

A declaration occurs *immediately within* a declarative region if this region is the innermost declarative region that encloses the declaration (the *immediately enclosing* declarative region), not counting the declarative region (if any) associated with the declaration itself.

A declaration is *local* to a declarative region if the declaration occurs immediately within the declarative region. An entity is *local* to a declarative region if the entity is declared by a declaration that is local to the declarative region.

- 15 A declaration is *global* to a declarative region if the declaration occurs immediately within another declarative region that encloses the declarative region. An entity is *global* to a declarative region if the entity is declared by a declaration that is global to the declarative region.

## NOTES

- 16 1 The children of a parent library unit are inside the parent's declarative region, even though they do not occur inside the parent's declaration or body. This implies that one can use (for example) "P.Q" to refer to a child of P whose defining name is Q, and that after "use P;" Q can refer (directly) to that child.
- 17 2 As explained above and in 10.1.1, "Compilation Units - Library Units", all library units are descendants of Standard, and so are contained in the declarative region of Standard. They are *not* inside the declaration or body of Standard, but they *are* inside its declarative region.
- 18 3 For a declarative region that comes in multiple parts, the text of the declarative region does not contain any text that might appear between the parts. Thus, when a portion of a declarative region is said to extend from one place to another in the declarative region, the portion does not contain any text that might appear between the parts of the declarative region.

## 8.2 Scope of Declarations

- 1 For each declaration, the language rules define a certain portion of the program text called the *scope* of the declaration. The scope of a declaration is also called the scope of any view or entity declared by the declaration. Within the scope of an entity, and only there, there are places where it is legal to refer to the declared entity. These places are defined by the rules of visibility and overloading.

### Static Semantics

- 2 The *immediate scope* of a declaration is a portion of the declarative region immediately enclosing the declaration. The immediate scope starts at the beginning of the declaration, except in the case of an overloadable declaration, in which case the immediate scope starts just after the place where the profile of the callable entity is determined (which is at the end of the `_specification` for the callable entity, or at the end of the `generic_instantiation` if an instance). The immediate scope extends to the end of the declarative region, with the following exceptions:
- 3 • The immediate scope of a `library_item` includes only its semantic dependents.
- 4 • The immediate scope of a declaration in the private part of a library unit does not include the visible part of any public descendant of that library unit.
- 5 The *visible part* of (a view of) an entity is a portion of the text of its declaration containing declarations that are visible from outside. The *private part* of (a view of) an entity that has a visible part contains all declarations within the declaration of (the view of) the entity, except those in the visible part; these are not visible from outside. Visible and private parts are defined only for these kinds of entities: callable entities, other program units, and composite types.
- 6 • The visible part of a view of a callable entity is its profile.
- 7 • The visible part of a composite type other than a task or protected type consists of the declarations of all components declared (explicitly or implicitly) within the `type_declaration`.
- 8 • The visible part of a generic unit includes the `generic_formal_part`. For a generic package, it also includes the first list of `basic_declarative_items` of the `package_specification`. For a generic subprogram, it also includes the profile.
- 9 • The visible part of a package, task unit, or protected unit consists of declarations in the program unit's declaration other than those following the reserved word **private**, if any; see 7.1 and 12.7 for packages, 9.1 for task units, and 9.4 for protected units.



The scope of a declaration always contains the immediate scope of the declaration. In addition, for a given declaration that occurs immediately within the visible part of an outer declaration, or is a public child of an outer declaration, the scope of the given declaration extends to the end of the scope of the outer declaration, except that the scope of a `library_item` includes only its semantic dependents. 10

The scope of an `attribute_definition_clause` is identical to the scope of a declaration that would occur at the point of the `attribute_definition_clause`. 10.1/2

The immediate scope of a declaration is also the immediate scope of the entity or view declared by the declaration. Similarly, the scope of a declaration is also the scope of the entity or view declared by the declaration. 11

#### NOTES

4 There are notations for denoting visible declarations that are not directly visible. For example, `parameter_specifications` are in the visible part of a `subprogram_declaration` so that they can be used in named-notation calls appearing outside the called subprogram. For another example, declarations of the visible part of a package can be denoted by expanded names appearing outside the package, and can be made directly visible by a `use_clause`. 12

## 8.3 Visibility

The *visibility rules*, given below, determine which declarations are visible and directly visible at each place within a program. The visibility rules apply to both explicit and implicit declarations. 1

#### Static Semantics

A declaration is defined to be *directly visible* at places where a `name` consisting of only an `identifier` or `operator_symbol` is sufficient to denote the declaration; that is, no `selected_component` notation or special context (such as preceding `=>` in a named association) is necessary to denote the declaration. A declaration is defined to be *visible* wherever it is directly visible, as well as at other places where some `name` (such as a `selected_component`) can denote the declaration. 2

The syntactic category `direct_name` is used to indicate contexts where direct visibility is required. The syntactic category `selector_name` is used to indicate contexts where visibility, but not direct visibility, is required. 3

There are two kinds of direct visibility: *immediate visibility* and *use-visibility*. A declaration is immediately visible at a place if it is directly visible because the place is within its immediate scope. A declaration is use-visible if it is directly visible because of a `use_clause` (see 8.4). Both conditions can apply. 4

A declaration can be *hidden*, either from direct visibility, or from all visibility, within certain parts of its scope. Where *hidden from all visibility*, it is not visible at all (neither using a `direct_name` nor a `selector_name`). Where *hidden from direct visibility*, only direct visibility is lost; visibility using a `selector_name` is still possible. 5

Two or more declarations are *overloaded* if they all have the same defining name and there is a place where they are all directly visible. 6

The declarations of callable entities (including enumeration literals) are *overloadable*, meaning that overloading is allowed for them. 7

Two declarations are *homographs* if they have the same defining name, and, if both are overloadable, their profiles are type conformant. An inner declaration hides any outer homograph from direct visibility. 8

- 9/1 Two homographs are not generally allowed immediately within the same declarative region unless one *overrides* the other (see Legality Rules below). The only declarations that are *overridable* are the implicit declarations for predefined operators and inherited primitive subprograms. A declaration overrides another homograph that occurs immediately within the same declarative region in the following cases:
- 10/1 • A declaration that is not overridable overrides one that is overridable, regardless of which declaration occurs first;
  - 11 • The implicit declaration of an inherited operator overrides that of a predefined operator;
  - 12 • An implicit declaration of an inherited subprogram overrides a previous implicit declaration of an inherited subprogram.
  - 12.1/2 • If two or more homographs are implicitly declared at the same place:
    - 12.2/2 • If at least one is a subprogram that is neither a null procedure nor an abstract subprogram, and does not require overriding (see 3.9.3), then they override those that are null procedures, abstract subprograms, or require overriding. If more than one such homograph remains that is not thus overridden, then they are all hidden from all visibility.
    - 12.3/2 • Otherwise (all are null procedures, abstract subprograms, or require overriding), then any null procedure overrides all abstract subprograms and all subprograms that require overriding; if more than one such homograph remains that is not thus overridden, then if they are all fully conformant with one another, one is chosen arbitrarily; if not, they are all hidden from all visibility.
  - 13 • For an implicit declaration of a primitive subprogram in a generic unit, there is a copy of this declaration in an instance. However, a whole new set of primitive subprograms is implicitly declared for each type declared within the visible part of the instance. These new declarations occur immediately after the type declaration, and override the copied ones. The copied ones can be called only from within the instance; the new ones can be called only from outside the instance, although for tagged types, the body of a new one can be executed by a call to an old one.
- 14 A declaration is visible within its scope, except where hidden from all visibility, as follows:
- 15 • An overridden declaration is hidden from all visibility within the scope of the overriding declaration.
  - 16 • A declaration is hidden from all visibility until the end of the declaration, except:
    - 17 • For a record type or record extension, the declaration is hidden from all visibility only until the reserved word **record**;
    - 17.1/3 • For an **extended\_return\_statement**, the declaration is hidden from all visibility only until the reserved word **do** of the statement;
    - 18/2 • For a **package\_declaration**, **generic\_package\_declaration**, or **subprogram\_body**, the declaration is hidden from all visibility only until the reserved word **is** of the declaration;
    - 18.1/2 • For a task declaration or protected declaration, the declaration is hidden from all visibility only until the reserved word **with** of the declaration if there is one, or the reserved word **is** of the declaration if there is no **with**.
  - 19 • If the completion of a declaration is a declaration, then within the scope of the completion, the first declaration is hidden from all visibility. Similarly, a **discriminant\_specification** or **parameter\_specification** is hidden within the scope of a corresponding **discriminant\_specification** or **parameter\_specification** of a corresponding completion, or of a corresponding **accept\_statement**.

- The declaration of a library unit (including a `library_unit_renaming_declaration`) is hidden from all visibility at places outside its declarative region that are not within the scope of a `nonlimited_with_clause` that mentions it. The limited view of a library package is hidden from all visibility at places that are not within the scope of a `limited_with_clause` that mentions it; in addition, the limited view is hidden from all visibility within the declarative region of the package, as well as within the scope of any `nonlimited_with_clause` that mentions the package. Where the declaration of the limited view of a package is visible, any name that denotes the package denotes the limited view, including those provided by a package renaming. 20/2
- For each declaration or renaming of a generic unit as a child of some parent generic package, there is a corresponding declaration nested immediately within each instance of the parent. Such a nested declaration is hidden from all visibility except at places that are within the scope of a `with_clause` that mentions the child. 20.1/2

A declaration with a `defining_identifier` or `defining_operator_symbol` is immediately visible (and hence directly visible) within its immediate scope except where hidden from direct visibility, as follows: 21

- A declaration is hidden from direct visibility within the immediate scope of a homograph of the declaration, if the homograph occurs within an inner declarative region; 22
- A declaration is also hidden from direct visibility where hidden from all visibility. 23

An `attribute_definition_clause` is *visible* everywhere within its scope. 23.1/2

#### *Name Resolution Rules*

A `direct_name` shall resolve to denote a directly visible declaration whose defining name is the same as the `direct_name`. A `selector_name` shall resolve to denote a visible declaration whose defining name is the same as the `selector_name`. 24

These rules on visibility and direct visibility do not apply in a `context_clause`, a `parent_unit_name`, or a `pragma` that appears at the place of a `compilation_unit`. For those contexts, see the rules in 10.1.6, “Environment-Level Visibility Rules”. 25

#### *Legality Rules*

A non-overridable declaration is illegal if there is a homograph occurring immediately within the same declarative region that is visible at the place of the declaration, and is not hidden from all visibility by the non-overridable declaration. In addition, a type extension is illegal if somewhere within its immediate scope it has two visible components with the same name. Similarly, the `context_clause` for a compilation unit is illegal if it mentions (in a `with_clause`) some library unit, and there is a homograph of the library unit that is visible at the place of the compilation unit, and the homograph and the mentioned library unit are both declared immediately within the same declarative region. These rules also apply to dispatching operations declared in the visible part of an instance of a generic unit. However, they do not apply to other overloadable declarations in an instance; such declarations may have type conformant profiles in the instance, so long as the corresponding declarations in the generic were not type conformant. 26/2

#### NOTES

5 Visibility for compilation units follows from the definition of the environment in 10.1.4, except that it is necessary to apply a `with_clause` to obtain visibility to a `library_unit_declaration` or `library_unit_renaming_declaration`. 27

6 In addition to the visibility rules given above, the meaning of the occurrence of a `direct_name` or `selector_name` at a given place in the text can depend on the overloading rules (see 8.6). 28

7 Not all contexts where an `identifier`, `character_literal`, or `operator_symbol` are allowed require visibility of a corresponding declaration. Contexts where visibility is not required are identified by using one of these three syntactic categories directly in a syntax rule, rather than using `direct_name` or `selector_name`. 29

### 8.3.1 Overriding Indicators

- 1/2 An overriding\_indicator is used to declare that an operation is intended to override (or not override) an inherited operation.

#### *Syntax*

- 2/2 overriding\_indicator ::= [**not**] **overriding**

#### *Legality Rules*

- 3/2 If an abstract\_subprogram\_declaration, null\_procedure\_declaration, subprogram\_body, subprogram\_body\_stub, subprogram\_renaming\_declaration, generic\_instantiation of a subprogram, or subprogram\_declaration other than a protected subprogram has an overriding\_indicator, then:

- 4/2 • the operation shall be a primitive operation for some type;
- 5/2 • if the overriding\_indicator is **overriding**, then the operation shall override a homograph at the place of the declaration or body;
- 6/2 • if the overriding\_indicator is **not overriding**, then the operation shall not override any homograph (at any place).

- 7/2 In addition to the places where Legality Rules normally apply, these rules also apply in the private part of an instance of a generic unit.

#### NOTES

- 8/2 8 Rules for overriding\_indicators of task and protected entries and of protected subprograms are found in 9.5.2 and 9.4, respectively.

#### *Examples*

- 9/2 The use of overriding\_indicators allows the detection of errors at compile-time that otherwise might not be detected at all. For instance, we might declare a security queue derived from the Queue interface of 3.9.4 as:

- ```

10/2  type Security_Queue is new Queue with record ...;
11/2  overriding
12/2  procedure Append(Q : in out Security_Queue; Person : in Person_Name);
13/2  overriding
14/2  procedure Remove_First(Q : in out Security_Queue; Person : in Person_Name);
15/2  overriding
16/2  function Cur_Count(Q : in Security_Queue) return Natural;
17/2  overriding
18/2  function Max_Count(Q : in Security_Queue) return Natural;
19/2  not overriding
20/2  procedure Arrest(Q : in out Security_Queue; Person : in Person_Name);

```

- 21/2 The first four subprogram declarations guarantee that these subprograms will override the four subprograms inherited from the Queue interface. A misspelling in one of these subprograms will be detected by the implementation. Conversely, the declaration of Arrest guarantees that this is a new operation.

8.4 Use Clauses

A `use_package_clause` achieves direct visibility of declarations that appear in the visible part of a package; a `use_type_clause` achieves direct visibility of the primitive operators of a type. 1

Syntax

```

use_clause ::= use_package_clause | use_type_clause 2
use_package_clause ::= use package_name {, package_name}; 3
use_type_clause ::= use {all} type subtype_mark {, subtype_mark}; 4/3

```

Legality Rules

A `package_name` of a `use_package_clause` shall denote a nonlimited view of a package. 5/2

Static Semantics

For each `use_clause`, there is a certain region of text called the *scope* of the `use_clause`. For a `use_clause` within a `context_clause` of a `library_unit_declaration` or `library_unit_renaming_declaration`, the scope is the entire declarative region of the declaration. For a `use_clause` within a `context_clause` of a body, the scope is the entire body and any subunits (including multiply nested subunits). The scope does not include `context_clauses` themselves. 6

For a `use_clause` immediately within a declarative region, the scope is the portion of the declarative region starting just after the `use_clause` and extending to the end of the declarative region. However, the scope of a `use_clause` in the private part of a library unit does not include the visible part of any public descendant of that library unit. 7

A package is *named* in a `use_package_clause` if it is denoted by a `package_name` of that clause. A type is *named* in a `use_type_clause` if it is determined by a `subtype_mark` of that clause. 7.1/2

For each package named in a `use_package_clause` whose scope encloses a place, each declaration that occurs immediately within the declarative region of the package is *potentially use-visible* at this place if the declaration is visible at this place. For each type *T* or *T*Class named in a `use_type_clause` whose scope encloses a place, the declaration of each primitive operator of type *T* is potentially use-visible at this place if its declaration is visible at this place. If a `use_type_clause` whose scope encloses a place includes the reserved word **all**, then the following entities are also potentially use-visible at this place if the declaration of the entity is visible at this place: 8/3

- Each primitive subprogram of *T* including each enumeration literal (if any); 8.1/3
- Each subprogram that is declared immediately within the declarative region in which an ancestor type of *T* is declared and that operates on a class-wide type that covers *T*. 8.2/3

A declaration is *use-visible* if it is potentially use-visible, except in these naming-conflict cases: 9

- A potentially use-visible declaration is not use-visible if the place considered is within the immediate scope of a homograph of the declaration. 10
- Potentially use-visible declarations that have the same `identifier` are not use-visible unless each of them is an overloadable declaration. 11

Dynamic Semantics

The elaboration of a `use_clause` has no effect. 12

Examples

13 *Example of a use clause in a context clause:*

14 **with** Ada.Calendar; **use** Ada;

15 *Example of a use type clause:*

16 **use type** Rational_Numbers.Rational; -- see 7.1
 Two_Thirds: Rational_Numbers.Rational := 2/3;

8.5 Renaming Declarations

1 A *renaming_declaration* declares another name for an entity, such as an object, exception, package, subprogram, entry, or generic unit. Alternatively, a *subprogram_renaming_declaration* can be the completion of a previous *subprogram_declaration*.

Syntax

2 *renaming_declaration* ::=
 object_renaming_declaration
 | *exception_renaming_declaration*
 | *package_renaming_declaration*
 | *subprogram_renaming_declaration*
 | *generic_renaming_declaration*

Dynamic Semantics

3 The elaboration of a *renaming_declaration* evaluates the **name** that follows the reserved word **renames** and thereby determines the view and entity denoted by this name (the *renamed view* and *renamed entity*). A name that denotes the *renaming_declaration* denotes (a new view of) the renamed entity.

NOTES

4 9 Renaming may be used to resolve name conflicts and to act as a shorthand. Renaming with a different *identifier* or *operator_symbol* does not hide the old name; the new name and the old name need not be visible at the same places.

5 10 A task or protected object that is declared by an explicit *object_declaration* can be renamed as an object. However, a single task or protected object cannot be renamed since the corresponding type is anonymous (meaning it has no nameable subtypes). For similar reasons, an object of an anonymous array or access type cannot be renamed.

6 11 A subtype defined without any additional constraint can be used to achieve the effect of renaming another subtype (including a task or protected subtype) as in

7 **subtype** Mode **is** Ada.Text_IO.File_Mode;

8.5.1 Object Renaming Declarations

1 An *object_renaming_declaration* is used to rename an object.

Syntax

2/2 *object_renaming_declaration* ::=
 defining_identifier : [null_exclusion] *subtype_mark* **renames** *object_name*;
 | *defining_identifier* : *access_definition* **renames** *object_name*;

Name Resolution Rules

3/2 The type of the *object_name* shall resolve to the type determined by the *subtype_mark*, or in the case where the type is defined by an *access_definition*, to an anonymous access type. If the anonymous access type is an access-to-object type, the type of the *object_name* shall have the same designated type as that of

the `access_definition`. If the anonymous access type is an access-to-subprogram type, the type of the `object_name` shall have a designated profile that is type conformant with that of the `access_definition`.

Legality Rules

The renamed entity shall be an object.

4

In the case where the type is defined by an `access_definition`, the type of the renamed object and the type defined by the `access_definition`:

4.1/2

- shall both be access-to-object types with statically matching designated subtypes and with both or neither being access-to-constant types; or
- shall both be access-to-subprogram types with subtype conformant designated profiles.

4.2/2

4.3/2

For an `object_renaming_declaration` with a `null_exclusion` or an `access_definition` that has a `null_exclusion`:

4.4/2

- if the `object_name` denotes a generic formal object of a generic unit G , and the `object_renaming_declaration` occurs within the body of G or within the body of a generic unit declared within the declarative region of G , then the declaration of the formal object of G shall have a `null_exclusion`;
- otherwise, the subtype of the `object_name` shall exclude null. In addition to the places where Legality Rules normally apply (see 12.3), this rule applies also in the private part of an instance of a generic unit.

4.5/2

4.6/2

The renamed entity shall not be a subcomponent that depends on discriminants of an object whose nominal subtype is unconstrained unless the object is known to be constrained. A slice of an array shall not be renamed if this restriction disallows renaming of the array. In addition to the places where Legality Rules normally apply, these rules apply also in the private part of an instance of a generic unit.

5/3

Static Semantics

An `object_renaming_declaration` declares a new view of the renamed object whose properties are identical to those of the renamed view. Thus, the properties of the renamed object are not affected by the `renaming_declaration`. In particular, its value and whether or not it is a constant are unaffected; similarly, the null exclusion or constraints that apply to an object are not affected by renaming (any constraint implied by the `subtype_mark` or `access_definition` of the `object_renaming_declaration` is ignored).

6/2

Examples

Example of renaming an object:

7

```

declare
  L : Person renames Leftmost_Person; -- see 3.10.1
begin
  L.Age := L.Age + 1;
end;

```

8

8.5.2 Exception Renaming Declarations

An `exception_renaming_declaration` is used to rename an exception.

1

Syntax

`exception_renaming_declaration ::= defining_identifier : exception renames exception_name;`

2

Legality Rules

The renamed entity shall be an exception.

3

Static Semantics

- 4 An `exception_renaming_declaration` declares a new view of the renamed exception.

Examples

- 5 *Example of renaming an exception:*

6 `EOF : exception renames Ada.IO_Exceptions.End_Error; -- see A.13`

8.5.3 Package Renaming Declarations

- 1 A `package_renaming_declaration` is used to rename a package.

Syntax

2 `package_renaming_declaration ::=`
 3 `package defining_program_unit_name renames package_name;`

Legality Rules

- 3 The renamed entity shall be a package.
- 3.1/2 If the *package_name* of a `package_renaming_declaration` denotes a limited view of a package *P*, then a name that denotes the `package_renaming_declaration` shall occur only within the immediate scope of the renaming or the scope of a `with_clause` that mentions the package *P* or, if *P* is a nested package, the innermost library package enclosing *P*.

Static Semantics

- 4 A `package_renaming_declaration` declares a new view of the renamed package.
- 4.1/2 At places where the declaration of the limited view of the renamed package is visible, a name that denotes the `package_renaming_declaration` denotes a limited view of the package (see 10.1.1).

Examples

- 5 *Example of renaming a package:*

6 `package TM renames Table_Manager;`

8.5.4 Subprogram Renaming Declarations

- 1 A `subprogram_renaming_declaration` can serve as the completion of a `subprogram_declaration`; such a renaming_declaration is called a *renaming-as-body*. A `subprogram_renaming_declaration` that is not a completion is called a *renaming-as-declaration*, and is used to rename a subprogram (possibly an enumeration literal) or an entry.

Syntax

2/2 `subprogram_renaming_declaration ::=`
 [overriding_indicator]
`subprogram_specification renames callable_entity_name;`

Name Resolution Rules

- 3 The expected profile for the *callable_entity_name* is the profile given in the `subprogram_specification`.

Legality Rules

The profile of a renaming-as-declaration shall be mode-conformant with that of the renamed callable entity. 4

For a parameter or result subtype of the `subprogram_specification` that has an explicit `null_exclusion`: 4.1/2

- if the `callable_entity_name` denotes a generic formal subprogram of a generic unit *G*, and the `subprogram_renaming_declaration` occurs within the body of a generic unit *G* or within the body of a generic unit declared within the declarative region of the generic unit *G*, then the corresponding parameter or result subtype of the formal subprogram of *G* shall have a `null_exclusion`; 4.2/2
- otherwise, the subtype of the corresponding parameter or result type of the renamed callable entity shall exclude null. In addition to the places where Legality Rules normally apply (see 12.3), this rule applies also in the private part of an instance of a generic unit. 4.3/2

The profile of a renaming-as-body shall conform fully to that of the declaration it completes. If the renaming-as-body completes that declaration before the subprogram it declares is frozen, the profile shall be mode-conformant with that of the renamed callable entity and the subprogram it declares takes its convention from the renamed subprogram; otherwise, the profile shall be subtype-conformant with that of the renamed callable entity and the convention of the renamed subprogram shall not be Intrinsic. A renaming-as-body is illegal if the declaration occurs before the subprogram whose declaration it completes is frozen, and the renaming renames the subprogram itself, through one or more subprogram renaming declarations, none of whose subprograms has been frozen. 5/1

The `callable_entity_name` of a renaming shall not denote a subprogram that requires overriding (see 3.9.3). 5.1/2

The `callable_entity_name` of a renaming-as-body shall not denote an abstract subprogram. 5.2/2

A name that denotes a formal parameter of the `subprogram_specification` is not allowed within the `callable_entity_name`. 6

Static Semantics

A renaming-as-declaration declares a new view of the renamed entity. The profile of this new view takes its subtypes, parameter modes, and calling convention from the original profile of the callable entity, while taking the formal parameter names and `default_expressions` from the profile given in the `subprogram_renaming_declaration`. The new view is a function or procedure, never an entry. 7

Dynamic Semantics

For a call to a subprogram whose body is given as a renaming-as-body, the execution of the renaming-as-body is equivalent to the execution of a `subprogram_body` that simply calls the renamed subprogram with its formal parameters as the actual parameters and, if it is a function, returns the value of the call. 7.1/1

For a call on a renaming of a dispatching subprogram that is overridden, if the overriding occurred before the renaming, then the body executed is that of the overriding declaration, even if the overriding declaration is not visible at the place of the renaming; otherwise, the inherited or predefined subprogram is called. A corresponding rule applies to a call on a renaming of a predefined equality operator for an untagged record type. 8/3

Bounded (Run-Time) Errors

- 8.1/1 If a subprogram directly or indirectly renames itself, then it is a bounded error to call that subprogram. Possible consequences are that Program_Error or Storage_Error is raised, or that the call results in infinite recursion.

NOTES

- 9 12 A procedure can only be renamed as a procedure. A function whose `defining_designator` is either an identifier or an operator_symbol can be renamed with either an identifier or an operator_symbol; for renaming as an operator, the subprogram specification given in the `renaming_declaration` is subject to the rules given in 6.6 for operator declarations. Enumeration literals can be renamed as functions; similarly, `attribute_references` that denote functions (such as references to Succ and Pred) can be renamed as functions. An entry can only be renamed as a procedure; the new name is only allowed to appear in contexts that allow a procedure name. An entry of a family can be renamed, but an entry family cannot be renamed as a whole.
- 10 13 The operators of the root numeric types cannot be renamed because the types in the profile are anonymous, so the corresponding specifications cannot be written; the same holds for certain attributes, such as Pos.
- 11 14 Calls with the new name of a renamed entry are `procedure_call_statements` and are not allowed at places where the syntax requires an `entry_call_statement` in `conditional_` and `timed_entry_calls`, nor in an `asynchronous_select`; similarly, the Count attribute is not available for the new name.
- 12 15 The primitiveness of a renaming-as-declaration is determined by its profile, and by where it occurs, as for any declaration of (a view of) a subprogram; primitiveness is not determined by the renamed view. In order to perform a dispatching call, the subprogram name has to denote a primitive subprogram, not a non-primitive renaming of a primitive subprogram.

Examples

- 13 *Examples of subprogram renaming declarations:*

```

14 procedure My_Write(C : in Character) renames Pool(K).Write; -- see 4.1.3
15 function Real_Plus(Left, Right : Real ) return Real renames "+";
16 function Int_Plus (Left, Right : Integer) return Integer renames "+";
17 function Rouge return Color renames Red; -- see 3.5.1
18 function Rot return Color renames Red;
19 function Rosso return Color renames Rouge;
20 function Next(X : Color) return Color renames Color'Succ; -- see 3.5.1

```

- 18 *Example of a subprogram renaming declaration with new parameter names:*

```

19 function "*" (X,Y : Vector) return Real renames Dot_Product; -- see 6.1

```

- 20 *Example of a subprogram renaming declaration with a new default expression:*

```

21 function Minimum(L : Link := Head) return Cell renames Min_Cell; -- see 6.1

```

8.5.5 Generic Renaming Declarations

- 1 A `generic_renaming_declaration` is used to rename a generic unit.

Syntax

```

2 generic_renaming_declaration ::=
   | generic package      defining_program_unit_name renames generic_package_name;
   | generic procedure   defining_program_unit_name renames generic_procedure_name;
   | generic function    defining_program_unit_name renames generic_function_name;

```

Legality Rules

- 3 The renamed entity shall be a generic unit of the corresponding kind.

Static Semantics

A `generic_renaming_declaration` declares a new view of the renamed generic unit. 4

NOTES

16 Although the properties of the new view are the same as those of the renamed view, the place where the `generic_renaming_declaration` occurs may affect the legality of subsequent renamings and instantiations that denote the `generic_renaming_declaration`, in particular if the renamed generic unit is a library unit (see 10.1.1). 5

Examples

Example of renaming a generic unit: 6

```
generic package Enum_IO renames Ada.Text_IO Enumeration_IO;  -- see A.10.10 7
```

8.6 The Context of Overload Resolution

Because declarations can be overloaded, it is possible for an occurrence of a usage name to have more than one possible interpretation; in most cases, ambiguity is disallowed. This clause describes how the possible interpretations resolve to the actual interpretation. 1

Certain rules of the language (the Name Resolution Rules) are considered “overloading rules”. If a possible interpretation violates an overloading rule, it is assumed not to be the intended interpretation; some other possible interpretation is assumed to be the actual interpretation. On the other hand, violations of non-overloading rules do not affect which interpretation is chosen; instead, they cause the construct to be illegal. To be legal, there usually has to be exactly one acceptable interpretation of a construct that is a “complete context”, not counting any nested complete contexts. 2

The syntax rules of the language and the visibility rules given in 8.3 determine the possible interpretations. Most type checking rules (rules that require a particular type, or a particular class of types, for example) are overloading rules. Various rules for the matching of formal and actual parameters are overloading rules. 3

Name Resolution Rules

Overload resolution is applied separately to each *complete context*, not counting inner complete contexts. Each of the following constructs is a *complete context*: 4

- A `context_item`. 5
- A `declarative_item` or declaration. 6
- A `statement`. 7
- A `pragma_argument_association`. 8
- The expression of a `case_statement`. 9

An (overall) *interpretation* of a complete context embodies its meaning, and includes the following information about the constituents of the complete context, not including constituents of inner complete contexts: 10

- for each constituent of the complete context, to which syntactic categories it belongs, and by which syntax rules; and 11
- for each usage name, which declaration it denotes (and, therefore, which view and which entity it denotes); and 12
- for a complete context that is a `declarative_item`, whether or not it is a completion of a declaration, and (if so) which declaration it completes. 13

- 14 A *possible interpretation* is one that obeys the syntax rules and the visibility rules. An *acceptable interpretation* is a possible interpretation that obeys the *overloading rules*, that is, those rules that specify an expected type or expected profile, or specify how a construct shall *resolve* or be *interpreted*.
- 15 The *interpretation* of a constituent of a complete context is determined from the overall interpretation of the complete context as a whole. Thus, for example, “interpreted as a `function_call`,” means that the construct's interpretation says that it belongs to the syntactic category `function_call`.
- 16 Each occurrence of a usage name *denotes* the declaration determined by its interpretation. It also denotes the view declared by its denoted declaration, except in the following cases:
- 17/2
- If a usage name appears within the declarative region of a `type_declaration` and denotes that same `type_declaration`, then it denotes the *current instance* of the type (rather than the type itself); the current instance of a type is the object or value of the type that is associated with the execution that evaluates the usage name. This rule does not apply if the usage name appears within the `subtype_mark` of an `access_definition` for an access-to-object type, or within the subtype of a parameter or result of an access-to-subprogram type.
 - 18 • If a usage name appears within the declarative region of a `generic_declaration` (but not within its `generic_formal_part`) and it denotes that same `generic_declaration`, then it denotes the *current instance* of the generic unit (rather than the generic unit itself). See also 12.3.
- 19 A usage name that denotes a view also denotes the entity of that view.
- 20/2 The *expected type* for a given `expression`, `name`, or other construct determines, according to the *type resolution rules* given below, the types considered for the construct during overload resolution. The type resolution rules provide support for class-wide programming, universal literals, dispatching operations, and anonymous access types:
- 21
- If a construct is expected to be of any type in a class of types, or of the universal or class-wide type for a class, then the type of the construct shall resolve to a type in that class or to a universal type that covers the class.
 - 22 • If the expected type for a construct is a specific type *T*, then the type of the construct shall resolve either to *T*, or:
- 23
- to *T*Class; or
 - 24 • to a universal type that covers *T*; or
 - 25/2 • when *T* is a specific anonymous access-to-object type (see 3.10) with designated type *D*, to an access-to-object type whose designated type is *D*Class or is covered by *D*; or
 - 25.1/3 • when *T* is a named general access-to-object type (see 3.10) with designated type *D*, to an anonymous access-to-object type whose designated type covers or is covered by *D*; or
 - 25.2/2 • when *T* is an anonymous access-to-subprogram type (see 3.10), to an access-to-subprogram type whose designated profile is type-conformant with that of *T*.
- 26 In certain contexts, such as in a `subprogram_renaming_declaration`, the Name Resolution Rules define an *expected profile* for a given `name`; in such cases, the `name` shall resolve to the name of a callable entity whose profile is type conformant with the expected profile.

Legality Rules

- 27/2 When a construct is one that requires that its expected type be a *single* type in a given class, the type of the construct shall be determinable solely from the context in which the construct appears, excluding the construct itself, but using the requirement that it be in the given class. Furthermore, the context shall not

be one that expects any type in some class that contains types of the given class; in particular, the construct shall not be the operand of a `type_conversion`.

Other than for the `simple_expression` of a membership test, if the expected type for a `name` or `expression` is not the same as the actual type of the `name` or `expression`, the actual type shall be convertible to the expected type (see 4.6); further, if the expected type is a named access-to-object type with designated type *D1* and the actual type is an anonymous access-to-object type with designated type *D2*, then *D1* shall cover *D2*, and the `name` or `expression` shall denote a view with an accessibility level for which the statically deeper relationship applies; in particular it shall not denote an access parameter nor a standalone access object. 27.1/3

A complete context shall have at least one acceptable interpretation; if there is exactly one, then that one is chosen. 28

There is a *preference* for the primitive operators (and *ranges*) of the root numeric types *root_integer* and *root_real*. In particular, if two acceptable interpretations of a constituent of a complete context differ only in that one is for a primitive operator (or *range*) of the type *root_integer* or *root_real*, and the other is not, the interpretation using the primitive operator (or *range*) of the root numeric type is *preferred*. 29

Similarly, there is a preference for the equality operators of the *universal_access* type (see 4.5.2). If two acceptable interpretations of a constituent of a complete context differ only in that one is for an equality operator of the *universal_access* type, and the other is not, the interpretation using the equality operator of the *universal_access* type is preferred. 29.1/3

For a complete context, if there is exactly one overall acceptable interpretation where each constituent's interpretation is the same as or preferred (in the above sense) over those in all other overall acceptable interpretations, then that one overall acceptable interpretation is chosen. Otherwise, the complete context is *ambiguous*. 30

A complete context other than a `pragma_argument_association` shall not be ambiguous. 31

A complete context that is a `pragma_argument_association` is allowed to be ambiguous (unless otherwise specified for the particular pragma), but only if every acceptable interpretation of the pragma argument is as a `name` that statically denotes a callable entity. Such a `name` denotes all of the declarations determined by its interpretations, and all of the views declared by these declarations. 32

NOTES

17 If a usage name has only one acceptable interpretation, then it denotes the corresponding entity. However, this does not mean that the usage name is necessarily legal since other requirements exist which are not considered for overload resolution; for example, the fact that an expression is static, whether an object is constant, mode and subtype conformance rules, freezing rules, order of elaboration, and so on. 33

Similarly, subtypes are not considered for overload resolution (the violation of a constraint does not make a program illegal but raises an exception during program execution). 34

Section 9: Tasks and Synchronization

The execution of an Ada program consists of the execution of one or more *tasks*. Each task represents a separate thread of control that proceeds independently and concurrently between the points where it *interacts* with other tasks. The various forms of task interaction are described in this section, and include:

- the activation and termination of a task;
- a call on a protected subprogram of a *protected object*, providing exclusive read-write access, or concurrent read-only access to shared data;
- a call on an entry, either of another task, allowing for synchronous communication with that task, or of a protected object, allowing for asynchronous communication with one or more other tasks using that same protected object;
- a timed operation, including a simple delay statement, a timed entry call or accept, or a timed asynchronous select statement (see next item);
- an asynchronous transfer of control as part of an asynchronous select statement, where a task stops what it is doing and begins execution at a different point in response to the completion of an entry call or the expiration of a delay;
- an abort statement, allowing one task to cause the termination of another task.

In addition, tasks can communicate indirectly by reading and updating (unprotected) shared variables, presuming the access is properly synchronized through some other kind of task interaction.

Static Semantics

The properties of a task are defined by a corresponding task declaration and `task_body`, which together define a program unit called a *task unit*.

Dynamic Semantics

Over time, tasks proceed through various *states*. A task is initially *inactive*; upon activation, and prior to its *termination* it is either *blocked* (as part of some task interaction) or *ready* to run. While ready, a task competes for the available *execution resources* that it requires to run.

NOTES

1 Concurrent task execution may be implemented on multicomputers, multiprocessors, or with interleaved execution on a single physical processor. On the other hand, whenever an implementation can determine that the required semantic effects can be achieved when parts of the execution of a given task are performed by different physical processors acting in parallel, it may choose to perform them in this way.

9.1 Task Units and Task Objects

A task unit is declared by a *task declaration*, which has a corresponding `task_body`. A task declaration may be a *task_type_declaration*, in which case it declares a named task type; alternatively, it may be a *single_task_declaration*, in which case it defines an anonymous task type, as well as declaring a named task object of that type.

Syntax

```
task_type_declaration ::=
  task_type defining_identifier [known_discriminant_part] [is
    [new interface_list with]
    task_definition];
```

- 3/2 `single_task_declaration ::=`
 `task defining_identifier [is`
 `[new interface_list with]`
 `task_definition];`
- 4 `task_definition ::=`
 `{task_item}`
 `[ private`
 `{task_item}]`
 `end [task_identifier]`
- 5/1 `task_item ::= entry_declaration | aspect_clause`
- 6 `task_body ::=`
 `task body defining_identifier is`
 `declarative_part`
 `begin`
 `handled_sequence_of_statements`
 `end [task_identifier];`
- 7 If a *task* identifier appears at the end of a *task_definition* or *task_body*, it shall repeat the defining_identifier.

Legality Rules

- 8/2 *This paragraph was deleted.*

Static Semantics

- 9 A *task_definition* defines a task type and its first subtype. The first list of *task_items* of a *task_definition*, together with the *known_discriminant_part*, if any, is called the visible part of the task unit. The optional list of *task_items* after the reserved word **private** is called the private part of the task unit.
- 9.1/1 For a task declaration without a *task_definition*, a *task_definition* without *task_items* is assumed.
- 9.2/3 For a task declaration with an *interface_list*, the task type inherits user-defined primitive subprograms from each progenitor type (see 3.9.4), in the same way that a derived type inherits user-defined primitive subprograms from its progenitor types (see 3.4). If the first parameter of a primitive inherited subprogram is of the task type or an access parameter designating the task type, and there is an *entry_declaration* for a single entry with the same identifier within the task declaration, whose profile is type conformant with the prefixed view profile of the inherited subprogram, the inherited subprogram is said to be *implemented* by the conforming task entry using an implicitly declared non-abstract subprogram which has the same profile as the inherited subprogram and which overrides it.

Legality Rules

- 9.3/2 A task declaration requires a completion, which shall be a *task_body*, and every *task_body* shall be the completion of some task declaration.
- 9.4/2 Each *interface_subtype_mark* of an *interface_list* appearing within a task declaration shall denote a limited interface type that is not a protected interface.
- 9.5/3 The prefixed view profile of an explicitly declared primitive subprogram of a tagged task type shall not be type conformant with any entry of the task type, if the subprogram has the same defining name as the entry and the first parameter of the subprogram is of the task type or is an access parameter designating the task type.

For each primitive subprogram inherited by the type declared by a task declaration, at most one of the following shall apply: 9.6/2

- the inherited subprogram is overridden with a primitive subprogram of the task type, in which case the overriding subprogram shall be subtype conformant with the inherited subprogram and not abstract; or 9.7/2
- the inherited subprogram is implemented by a single entry of the task type; in which case its prefixed view profile shall be subtype conformant with that of the task entry. 9.8/2

If neither applies, the inherited subprogram shall be a null procedure. In addition to the places where Legality Rules normally apply (see 12.3), these rules also apply in the private part of an instance of a generic unit. 9.9/2

Dynamic Semantics

The elaboration of a task declaration elaborates the **task_definition**. The elaboration of a **single_task_declaration** also creates an object of an (anonymous) task type. 10

The elaboration of a **task_definition** creates the task type and its first subtype; it also includes the elaboration of the **entry_declarations** in the given order. 11

As part of the initialization of a task object, any **aspect_clauses** and any per-object constraints associated with **entry_declarations** of the corresponding **task_definition** are elaborated in the given order. 12/1

The elaboration of a **task_body** has no effect other than to establish that tasks of the type can from then on be activated without failing the **Elaboration_Check**. 13

The execution of a **task_body** is invoked by the activation of a task of the corresponding type (see 9.2). 14

The content of a task object of a given task type includes: 15

- The values of the discriminants of the task object, if any; 16
- An entry queue for each entry of the task object; 17
- A representation of the state of the associated task. 18

NOTES

2 Other than in an **access_definition**, the name of a task unit within the declaration or body of the task unit denotes the current instance of the unit (see 8.6), rather than the first subtype of the corresponding task type (and thus the name cannot be used as a **subtype_mark**). 19/2

3 The notation of a **selected_component** can be used to denote a discriminant of a task (see 4.1.3). Within a task unit, the name of a discriminant of the task type denotes the corresponding discriminant of the current instance of the unit. 20

4 A task type is a limited type (see 7.5), and hence precludes use of **assignment_statements** and predefined equality operators. If an application needs to store and exchange task identities, it can do so by defining an access type designating the corresponding task objects and by using access values for identification purposes. Assignment is available for such an access type as for any access type. Alternatively, if the implementation supports the Systems Programming Annex, the Identity attribute can be used for task identification (see C.7.1). 21/2

Examples

Examples of declarations of task types: 22

```
task type Server is
  entry Next_Work_Item(WI : in Work_Item);
  entry Shut_Down;
end Server; 23
```

```

24/2    task type Keyboard_Driver(ID : Keyboard_ID := New_ID) is
        new Serial_Device with -- see 3.9.4
        entry Read (C : out Character);
        entry Write(C : in Character);
    end Keyboard_Driver;

```

25 *Examples of declarations of single tasks:*

```

26    task Controller is
        entry Request(Level) (D : Item); -- a family of entries
    end Controller;

27    task Parser is
        entry Next_Lexeme(L : in Lexical_Element);
        entry Next_Action(A : out Parser_Action);
    end;

28    task User; -- has no entries

```

29 *Examples of task objects:*

```

30    Agent      : Server;
    Teletype    : Keyboard_Driver(TTY_ID);
    Pool        : array(1 .. 10) of Keyboard_Driver;

```

31 *Example of access type designating task objects:*

```

32    type Keyboard is access Keyboard_Driver;
    Terminal : Keyboard := new Keyboard_Driver(Term_ID);

```

9.2 Task Execution - Task Activation

Dynamic Semantics

- 1 The execution of a task of a given task type consists of the execution of the corresponding `task_body`. The initial part of this execution is called the *activation* of the task; it consists of the elaboration of the `declarative_part` of the `task_body`. Should an exception be propagated by the elaboration of its `declarative_part`, the activation of the task is defined to have *failed*, and it becomes a completed task.
- 2/2 A task object (which represents one task) can be a part of a stand-alone object, of an object created by an `allocator`, or of an anonymous object of a limited type, or a coextension of one of these. All tasks that are part or coextensions of any of the stand-alone objects created by the elaboration of `object_declarations` (or `generic_associations` of formal objects of mode **in**) of a single declarative region are activated together. All tasks that are part or coextensions of a single object that is not a stand-alone object are activated together.
- 3/2 For the tasks of a given declarative region, the activations are initiated within the context of the `handled_sequence_of_statements` (and its associated `exception_handlers` if any — see 11.2), just prior to executing the statements of the `handled_sequence_of_statements`. For a package without an explicit body or an explicit `handled_sequence_of_statements`, an implicit body or an implicit `null_statement` is assumed, as defined in 7.2.
- 4/2 For tasks that are part or coextensions of a single object that is not a stand-alone object, activations are initiated after completing any initialization of the outermost object enclosing these tasks, prior to performing any other operation on the outermost object. In particular, for tasks that are part or coextensions of the object created by the evaluation of an `allocator`, the activations are initiated as the last step of evaluating the `allocator`, prior to returning the new access value. For tasks that are part or coextensions of an object that is the result of a function call, the activations are not initiated until after the function returns.

The task that created the new tasks and initiated their activations (the *activator*) is blocked until all of these activations complete (successfully or not). Once all of these activations are complete, if the activation of any of the tasks has failed (due to the propagation of an exception), `Tasking_Error` is raised in the activator, at the place at which it initiated the activations. Otherwise, the activator proceeds with its execution normally. Any tasks that are aborted prior to completing their activation are ignored when determining whether to raise `Tasking_Error`.

If the master that directly encloses the point where the activation of a task *T* would be initiated, completes before the activation of *T* is initiated, *T* becomes terminated and is never activated. Furthermore, if a return statement is left such that the return object is not returned to the caller, any task that was created as a part of the return object or one of its coextensions immediately becomes terminated and is never activated.

NOTES

- 5 An entry of a task can be called before the task has been activated. 7
- 6 If several tasks are activated together, the execution of any of these tasks need not await the end of the activation of the other tasks. 8
- 7 A task can become completed during its activation either because of an exception or because it is aborted (see 9.8). 9

Examples

Example of task activation:

```

procedure P is
  A, B : Server;      -- elaborate the task objects A, B
  C    : Server;      -- elaborate the task object C
begin
  -- the tasks A, B, C are activated together before the first statement
  ...
end;

```

9.3 Task Dependence - Termination of Tasks

Dynamic Semantics

Each task (other than an environment task — see 10.2) *depends* on one or more masters (see 7.6.1), as follows:

- If the task is created by the evaluation of an `allocator` for a given access type, it depends on each master that includes the elaboration of the declaration of the ultimate ancestor of the given access type. 2
- If the task is created by the elaboration of an `object_declaration`, it depends on each master that includes this elaboration. 3
- Otherwise, the task depends on the master of the outermost object of which it is a part (as determined by the accessibility level of that object — see 3.10.2 and 7.6.1), as well as on any master whose execution includes that of the master of the outermost object. 3.1/2

Furthermore, if a task depends on a given master, it is defined to depend on the task that executes the master, and (recursively) on any master of that task. 4

A task is said to be *completed* when the execution of its corresponding `task_body` is completed. A task is said to be *terminated* when any finalization of the `task_body` has been performed (see 7.6.1). The first step of finalizing a master (including a `task_body`) is to wait for the termination of any tasks dependent on the master. The task executing the master is blocked until all the dependents have terminated. Any remaining finalization is then performed and the master is left. 5

Completion of a task (and the corresponding **task_body**) can occur when the task is blocked at a **select_statement** with an open **terminate_alternative** (see 9.7.1); the open **terminate_alternative** is selected if and only if the following conditions are satisfied:

- The task depends on some completed master; and
- Each task that depends on the master considered is either already terminated or similarly blocked at a **select_statement** with an open **terminate_alternative**.

When both conditions are satisfied, the task considered becomes completed, together with all tasks that depend on the master considered that are not yet completed.

NOTES

8 The full view of a limited private type can be a task type, or can have subcomponents of a task type. Creation of an object of such a type creates dependences according to the full type.

9 An **object_renaming_declaration** defines a new view of an existing entity and hence creates no further dependence.

10 The rules given for the collective completion of a group of tasks all blocked on **select_statements** with open **terminate_alternatives** ensure that the collective completion can occur only when there are no remaining active tasks that could call one of the tasks being collectively completed.

11 If two or more tasks are blocked on **select_statements** with open **terminate_alternatives**, and become completed collectively, their finalization actions proceed concurrently.

12 The completion of a task can occur due to any of the following:

- the raising of an exception during the elaboration of the **declarative_part** of the corresponding **task_body**;
- the completion of the **handled_sequence_of_statements** of the corresponding **task_body**;
- the selection of an open **terminate_alternative** of a **select_statement** in the corresponding **task_body**;
- the abort of the task.

Examples

Example of task dependence:

```

declare
  type Global is access Server;           -- see 9.1
  A, B : Server;
  G    : Global;
begin
  -- activation of A and B
  declare
    type Local is access Server;
    X : Global := new Server;  -- activation of X.all
    L : Local  := new Server;  -- activation of L.all
    C : Server;
  begin
    -- activation of C
    G := X;  -- both G and X designate the same task object
    ...
  end;  -- await termination of C and L.all (but not X.all)
  ...
end;  -- await termination of A, B, and G.all

```

9.4 Protected Units and Protected Objects

A *protected object* provides coordinated access to shared data, through calls on its visible *protected operations*, which can be *protected subprograms* or *protected entries*. A *protected unit* is declared by a *protected declaration*, which has a corresponding *protected_body*. A protected declaration may be a *protected_type_declaration*, in which case it declares a named protected type; alternatively, it may be a *single_protected_declaration*, in which case it defines an anonymous protected type, as well as declaring a named protected object of that type.

Syntax

```

protected_type_declaration ::=                                2/2
    protected_type defining_identifier [known_discriminant_part] is
        [new interface_list with]
        protected_definition;

single_protected_declaration ::=                              3/2
    protected defining_identifier is
        [new interface_list with]
        protected_definition;

protected_definition ::=                                     4
    { protected_operation_declaration }
    [ private
        { protected_element_declaration } ]
    end [protected_identifier]

protected_operation_declaration ::= subprogram_declaration  5/1
    | entry_declaration
    | aspect_clause

protected_element_declaration ::= protected_operation_declaration  6
    | component_declaration

protected_body ::=                                           7
    protected body defining_identifier is
        { protected_operation_item }
    end [protected_identifier];

protected_operation_item ::= subprogram_declaration          8/1
    | subprogram_body
    | entry_body
    | aspect_clause

If a protected_identifier appears at the end of a protected_definition or protected_body, it shall  9
repeat the defining_identifier.

```

Legality Rules

This paragraph was deleted. 10/2

Static Semantics

A *protected_definition* defines a protected type and its first subtype. The list of *protected_operation_declarations* of a *protected_definition*, together with the *known_discriminant_part*, if any, is called the visible part of the protected unit. The optional list of *protected_element_declarations* after the reserved word **private** is called the private part of the protected unit. 11/2

- 11.1/3 For a protected declaration with an `interface_list`, the protected type inherits user-defined primitive subprograms from each progenitor type (see 3.9.4), in the same way that a derived type inherits user-defined primitive subprograms from its progenitor types (see 3.4). If the first parameter of a primitive inherited subprogram is of the protected type or an access parameter designating the protected type, and there is a `protected_operation_declaration` for a protected subprogram or single entry with the same identifier within the protected declaration, whose profile is type conformant with the prefixed view profile of the inherited subprogram, the inherited subprogram is said to be *implemented* by the conforming protected subprogram or entry using an implicitly declared non-abstract subprogram which has the same profile as the inherited subprogram and which overrides it.

Legality Rules

- 11.2/2 A protected declaration requires a completion, which shall be a `protected_body`, and every `protected_body` shall be the completion of some protected declaration.
- 11.3/2 Each `interface_subtype_mark` of an `interface_list` appearing within a protected declaration shall denote a limited interface type that is not a task interface.
- 11.4/3 The prefixed view profile of an explicitly declared primitive subprogram of a tagged protected type shall not be type conformant with any protected operation of the protected type, if the subprogram has the same defining name as the protected operation and the first parameter of the subprogram is of the protected type or is an access parameter designating the protected type.
- 11.5/2 For each primitive subprogram inherited by the type declared by a protected declaration, at most one of the following shall apply:
- 11.6/2 • the inherited subprogram is overridden with a primitive subprogram of the protected type, in which case the overriding subprogram shall be subtype conformant with the inherited subprogram and not abstract; or
 - 11.7/2 • the inherited subprogram is implemented by a protected subprogram or single entry of the protected type, in which case its prefixed view profile shall be subtype conformant with that of the protected subprogram or entry.
- 11.8/2 If neither applies, the inherited subprogram shall be a null procedure. In addition to the places where Legality Rules normally apply (see 12.3), these rules also apply in the private part of an instance of a generic unit.
- 11.9/2 If an inherited subprogram is implemented by a protected procedure or an entry, then the first parameter of the inherited subprogram shall be of mode **out** or **in out**, or an access-to-variable parameter.
- 11.10/2 If a protected subprogram declaration has an `overriding_indicator`, then at the point of the declaration:
- 11.11/2 • if the `overriding_indicator` is **overriding**, then the subprogram shall implement an inherited subprogram;
 - 11.12/2 • if the `overriding_indicator` is **not overriding**, then the subprogram shall not implement any inherited subprogram.
- 11.13/2 In addition to the places where Legality Rules normally apply (see 12.3), these rules also apply in the private part of an instance of a generic unit.

Dynamic Semantics

- 12 The elaboration of a protected declaration elaborates the `protected_definition`. The elaboration of a `single_protected_declaration` also creates an object of an (anonymous) protected type.

The elaboration of a `protected_definition` creates the protected type and its first subtype; it also includes the elaboration of the `component_declarations` and `protected_operation_declarations` in the given order. 13

As part of the initialization of a protected object, any per-object constraints (see 3.8) are elaborated. 14

The elaboration of a `protected_body` has no other effect than to establish that protected operations of the type can from then on be called without failing the `Elaboration_Check`. 15

The content of an object of a given protected type includes: 16

- The values of the components of the protected object, including (implicitly) an entry queue for each entry declared for the protected object; 17
- A representation of the state of the execution resource *associated* with the protected object (one such resource is associated with each protected object). 18

The execution resource associated with a protected object has to be acquired to read or update any components of the protected object; it can be acquired (as part of a protected action — see 9.5.1) either for concurrent read-only access, or for exclusive read-write access. 19

As the first step of the *finalization* of a protected object, each call remaining on any entry queue of the object is removed from its queue and `Program_Error` is raised at the place of the corresponding `entry_call_statement`. 20

Bounded (Run-Time) Errors

It is a bounded error to call an entry or subprogram of a protected object after that object is finalized. If the error is detected, `Program_Error` is raised. Otherwise, the call proceeds normally, which may leave a task queued forever. 20.1/2

NOTES

13 Within the declaration or body of a protected unit other than in an `access_definition`, the name of the protected unit denotes the current instance of the unit (see 8.6), rather than the first subtype of the corresponding protected type (and thus the name cannot be used as a `subtype_mark`). 21/2

14 A `selected_component` can be used to denote a discriminant of a protected object (see 4.1.3). Within a protected unit, the name of a discriminant of the protected type denotes the corresponding discriminant of the current instance of the unit. 22

15 A protected type is a limited type (see 7.5), and hence precludes use of `assignment_statements` and predefined equality operators. 23/2

16 The bodies of the protected operations given in the `protected_body` define the actions that take place upon calls to the protected operations. 24

17 The declarations in the private part are only visible within the private part and the body of the protected unit. 25

Examples

Example of declaration of protected type and corresponding body: 26

```

protected type Resource is
    entry Seize;
    procedure Release;
private
    Busy : Boolean := False;
end Resource;
protected body Resource is
    entry Seize when not Busy is
    begin
        Busy := True;
    end Seize;
  
```

```

29     procedure Release is
        begin
            Busy := False;
        end Release;
    end Resource;

```

30 *Example of a single protected declaration and corresponding body:*

```

31     protected Shared_Array is
        -- Index, Item, and Item_Array are global types
        function Component (N : in Index) return Item;
        procedure Set_Component (N : in Index; E : in Item);
    private
        Table : Item_Array(Index) := (others => Null_Item);
    end Shared_Array;

32     protected body Shared_Array is
        function Component (N : in Index) return Item is
            begin
                return Table(N);
            end Component;

33     procedure Set_Component (N : in Index; E : in Item) is
        begin
            Table(N) := E;
        end Set_Component;
    end Shared_Array;

```

34 *Examples of protected objects:*

```

35     Control : Resource;
     Flags : array(1 .. 100) of Resource;

```

9.5 Intertask Communication

1 The primary means for intertask communication is provided by calls on entries and protected subprograms. Calls on protected subprograms allow coordinated access to shared data objects. Entry calls allow for blocking the caller until a given condition is satisfied (namely, that the corresponding entry is open — see 9.5.3), and then communicating data or control information directly with another task or indirectly via a shared protected object.

Static Semantics

2 Any call on an entry or on a protected subprogram identifies a *target object* for the operation, which is either a task (for an entry call) or a protected object (for an entry call or a protected subprogram call). The target object is considered an implicit parameter to the operation, and is determined by the operation name (or prefix) used in the call on the operation, as follows:

- 3 • If it is a *direct_name* or expanded name that denotes the declaration (or body) of the operation, then the target object is implicitly specified to be the current instance of the task or protected unit immediately enclosing the operation; such a call is defined to be an *internal call*;
- 4 • If it is a *selected_component* that is not an expanded name, then the target object is explicitly specified to be the task or protected object denoted by the prefix of the name; such a call is defined to be an *external call*;
- 5 • If the name or prefix is a dereference (implicit or explicit) of an access-to-protected-subprogram value, then the target object is determined by the prefix of the Access attribute_reference that produced the access value originally, and the call is defined to be an *external call*;
- 6 • If the name or prefix denotes a *subprogram_renaming_declaration*, then the target object is as determined by the name of the renamed entity.

A corresponding definition of target object applies to a `requeue_statement` (see 9.5.4), with a corresponding distinction between an *internal requeue* and an *external requeue*. 7

Legality Rules

The view of the target protected object associated with a call of a protected procedure or entry shall be a variable. 7.1/2

Dynamic Semantics

Within the body of a protected operation, the current instance (see 8.6) of the immediately enclosing protected unit is determined by the target object specified (implicitly or explicitly) in the call (or requeue) on the protected operation. 8

Any call on a protected procedure or entry of a target protected object is defined to be an update to the object, as is a requeue on such an entry. 9

Syntax

The form of a `pragma Implemented` is as follows: 10/3

pragma Implemented(*procedure_local_name*, *implementation_kind*); 11/3

implementation_kind ::= By_Entry | By_Protected_Procedure | By_Any 12/3

Legality Rules

The *procedure_local_name* of a `pragma Implemented` shall denote a primitive procedure of a synchronized tagged type. 13/3

A `pragma Implemented` with *implementation_kind* `By_Protected_Procedure` shall not be applied to a primitive procedure of a task interface. 14/3

A procedure for which the *implementation_kind* is specified as `By_Entry` shall be implemented by an entry. A procedure for which the *implementation_kind* is specified as `By_Protected_Procedure` shall be implemented by a protected procedure. 15/3

If a primitive procedure overrides an inherited operation to which a `pragma Implemented` applies then any `pragma Implemented` applied to the overriding operation shall have the same *implementation_kind* or the inherited *implementation_kind* shall be `By_Any`. 16/3

In addition to the places where Legality Rules normally apply (see 12.3), these rules also apply in the private part of an instance of a generic unit. 17/3

Static Semantics

A `pragma Implemented` is said to *apply* to the procedure denoted by its *procedure_local_name*. If an overriding operation does not have a `pragma Implemented` then any `pragma Implemented` applying to the inherited operation applies to the overriding operation. 18/3

NOTES

18 The *implementation_kind* `By_Protected_Procedure` implies that the operation will not block. 19/3

9.5.1 Protected Subprograms and Protected Actions

A *protected subprogram* is a subprogram declared immediately within a `protected_definition`. Protected procedures provide exclusive read-write access to the data of a protected object; protected functions provide concurrent read-only access to the data. 1

Static Semantics

- 2 Within the body of a protected function (or a function declared immediately within a `protected_body`), the current instance of the enclosing protected unit is defined to be a constant (that is, its subcomponents may be read but not updated). Within the body of a protected procedure (or a procedure declared immediately within a `protected_body`), and within an `entry_body`, the current instance is defined to be a variable (updating is permitted).

Dynamic Semantics

- 3 For the execution of a call on a protected subprogram, the evaluation of the `name` or `prefix` and of the parameter associations, and any assigning back of **in out** or **out** parameters, proceeds as for a normal subprogram call (see 6.4). If the call is an internal call (see 9.5), the body of the subprogram is executed as for a normal subprogram call. If the call is an external call, then the body of the subprogram is executed as part of a new *protected action* on the target protected object; the protected action completes after the body of the subprogram is executed. A protected action can also be started by an entry call (see 9.5.3).
- 4 A new protected action is not started on a protected object while another protected action on the same protected object is underway, unless both actions are the result of a call on a protected function. This rule is expressible in terms of the execution resource associated with the protected object:
- 5 • *Starting* a protected action on a protected object corresponds to *acquiring* the execution resource associated with the protected object, either for concurrent read-only access if the protected action is for a call on a protected function, or for exclusive read-write access otherwise;
- 6 • *Completing* the protected action corresponds to *releasing* the associated execution resource.
- 7 After performing an operation on a protected object other than a call on a protected function, but prior to completing the associated protected action, the entry queues (if any) of the protected object are serviced (see 9.5.3).

Bounded (Run-Time) Errors

- 8 During a protected action, it is a bounded error to invoke an operation that is *potentially blocking*. The following are defined to be potentially blocking operations:
- 9 • a `select_statement`;
- 10 • an `accept_statement`;
- 11 • an `entry_call_statement`;
- 12 • a `delay_statement`;
- 13 • an `abort_statement`;
- 14 • task creation or activation;
- 15 • an external call on a protected subprogram (or an external requeue) with the same target object as that of the protected action;
- 16 • a call on a subprogram whose body contains a potentially blocking operation.
- 17 If the bounded error is detected, `Program_Error` is raised. If not detected, the bounded error might result in deadlock or a (nested) protected action on the same target object.
- 18 Certain language-defined subprograms are potentially blocking. In particular, the subprograms of the language-defined input-output packages that manipulate files (implicitly or explicitly) are potentially blocking. Other potentially blocking subprograms are identified where they are defined. When not specified as potentially blocking, a language-defined subprogram is nonblocking.

NOTES

- 19 If two tasks both try to start a protected action on a protected object, and at most one is calling a protected function, then only one of the tasks can proceed. Although the other task cannot proceed, it is not considered blocked, and it might be consuming processing resources while it awaits its turn. There is no language-defined ordering or queuing presumed for tasks competing to start a protected action — on a multiprocessor such tasks might use busy-waiting; for monoprocessor considerations, see D.3, “Priority Ceiling Locking”. 19
- 20 The body of a protected unit may contain declarations and bodies for local subprograms. These are not visible outside the protected unit. 20
- 21 The body of a protected function can contain internal calls on other protected functions, but not protected procedures, because the current instance is a constant. On the other hand, the body of a protected procedure can contain internal calls on both protected functions and procedures. 21
- 22 From within a protected action, an internal call on a protected subprogram, or an external call on a protected subprogram with a different target object is not considered a potentially blocking operation. 22
- 23 The `pragma Detect_Blocking` may be used to ensure that all executions of potentially blocking operations during a protected action raise `Program_Error`. See H.5. 22.1/2

Examples

Examples of protected subprogram calls (see 9.4): 23

```
Shared_Array.Set_Component (N, E); 24
E := Shared_Array.Component (M);
Control.Release;
```

9.5.2 Entries and Accept Statements

Entry_declarations, with the corresponding entry_bodies or accept_statements, are used to define potentially queued operations on tasks and protected objects. 1

Syntax

```
entry_declaration ::= 2/2
  [overriding_indicator]
  entry_defining_identifier [(discrete_subtype_definition)] parameter_profile;

accept_statement ::= 3
  accept entry_direct_name [(entry_index)] parameter_profile [do
    handled_sequence_of_statements
  end [entry_identifier]];

entry_index ::= expression 4

entry_body ::= 5
  entry_defining_identifier entry_body_formal_part entry_barrier is
    declarative_part
  begin
    handled_sequence_of_statements
  end [entry_identifier];

entry_body_formal_part ::= [(entry_index_specification)] parameter_profile 6

entry_barrier ::= when condition 7

entry_index_specification ::= for defining_identifier in discrete_subtype_definition 8

If an entry_identifier appears at the end of an accept_statement, it shall repeat the entry_direct_ 9
name. If an entry_identifier appears at the end of an entry_body, it shall repeat the defining_
identifier.

An entry_declaration is allowed only in a protected or task declaration. 10
```

- 10.1/2 An `overriding_indicator` is not allowed in an `entry_declaration` that includes a `discrete_subtype_definition`.

Name Resolution Rules

- 11 In an `accept_statement`, the expected profile for the `entry_direct_name` is that of the `entry_declaration`; the expected type for an `entry_index` is that of the subtype defined by the `discrete_subtype_definition` of the corresponding `entry_declaration`.
- 12 Within the `handled_sequence_of_statements` of an `accept_statement`, if a `selected_component` has a prefix that denotes the corresponding `entry_declaration`, then the entity denoted by the prefix is the `accept_statement`, and the `selected_component` is interpreted as an expanded name (see 4.1.3); the `selector_name` of the `selected_component` has to be the identifier for some formal parameter of the `accept_statement`.

Legality Rules

- 13 An `entry_declaration` in a task declaration shall not contain a specification for an access parameter (see 3.10).
- 13.1/2 If an `entry_declaration` has an `overriding_indicator`, then at the point of the declaration:
- 13.2/2 • if the `overriding_indicator` is **overriding**, then the entry shall implement an inherited subprogram;
- 13.3/2 • if the `overriding_indicator` is **not overriding**, then the entry shall not implement any inherited subprogram.
- 13.4/2 In addition to the places where Legality Rules normally apply (see 12.3), these rules also apply in the private part of an instance of a generic unit.
- 14 For an `accept_statement`, the innermost enclosing body shall be a `task_body`, and the `entry_direct_name` shall denote an `entry_declaration` in the corresponding task declaration; the profile of the `accept_statement` shall conform fully to that of the corresponding `entry_declaration`. An `accept_statement` shall have a parenthesized `entry_index` if and only if the corresponding `entry_declaration` has a `discrete_subtype_definition`.
- 15 An `accept_statement` shall not be within another `accept_statement` that corresponds to the same `entry_declaration`, nor within an `asynchronous_select` inner to the enclosing `task_body`.
- 16 An `entry_declaration` of a protected unit requires a completion, which shall be an `entry_body`, and every `entry_body` shall be the completion of an `entry_declaration` of a protected unit. The profile of the `entry_body` shall conform fully to that of the corresponding declaration.
- 17 An `entry_body_formal_part` shall have an `entry_index_specification` if and only if the corresponding `entry_declaration` has a `discrete_subtype_definition`. In this case, the `discrete_subtype_definitions` of the `entry_declaration` and the `entry_index_specification` shall fully conform to one another (see 6.3.1).
- 18 A name that denotes a formal parameter of an `entry_body` is not allowed within the `entry_barrier` of the `entry_body`.

Static Semantics

- 19 The parameter modes defined for parameters in the `parameter_profile` of an `entry_declaration` are the same as for a `subprogram_declaration` and have the same meaning (see 6.2).
- 20 An `entry_declaration` with a `discrete_subtype_definition` (see 3.6) declares a *family* of distinct entries having the same profile, with one such entry for each value of the *entry_index subtype* defined by the

`discrete_subtype_definition`. A name for an entry of a family takes the form of an `indexed_component`, where the prefix denotes the `entry_declaration` for the family, and the index value identifies the entry within the family. The term *single entry* is used to refer to any entry other than an entry of an entry family.

In the `entry_body` for an entry family, the `entry_index_specification` declares a named constant whose subtype is the entry index subtype defined by the corresponding `entry_declaration`; the value of the *named entry index* identifies which entry of the family was called. 21

Dynamic Semantics

The elaboration of an `entry_declaration` for an entry family consists of the elaboration of the `discrete_subtype_definition`, as described in 3.8. The elaboration of an `entry_declaration` for a single entry has no effect. 22/1

The actions to be performed when an entry is called are specified by the corresponding `accept_statements` (if any) for an entry of a task unit, and by the corresponding `entry_body` for an entry of a protected unit. 23

For the execution of an `accept_statement`, the `entry_index`, if any, is first evaluated and converted to the entry index subtype; this index value identifies which entry of the family is to be accepted. Further execution of the `accept_statement` is then blocked until a caller of the corresponding entry is selected (see 9.5.3), whereupon the `handled_sequence_of_statements`, if any, of the `accept_statement` is executed, with the formal parameters associated with the corresponding actual parameters of the selected entry call. Upon completion of the `handled_sequence_of_statements`, the `accept_statement` completes and is left. When an exception is propagated from the `handled_sequence_of_statements` of an `accept_statement`, the same exception is also raised by the execution of the corresponding `entry_call_statement`. 24

The above interaction between a calling task and an accepting task is called a *rendezvous*. After a rendezvous, the two tasks continue their execution independently. 25

An `entry_body` is executed when the condition of the `entry_barrier` evaluates to True and a caller of the corresponding single entry, or entry of the corresponding entry family, has been selected (see 9.5.3). For the execution of the `entry_body`, the `declarative_part` of the `entry_body` is elaborated, and the `handled_sequence_of_statements` of the body is executed, as for the execution of a `subprogram_body`. The value of the named entry index, if any, is determined by the value of the entry index specified in the `entry_name` of the selected entry call (or intermediate `requeue_statement` — see 9.5.4). 26

NOTES

24 A task entry has corresponding `accept_statements` (zero or more), whereas a protected entry has a corresponding `entry_body` (exactly one). 27

25 A consequence of the rule regarding the allowed placements of `accept_statements` is that a task can execute `accept_statements` only for its own entries. 28

26 A return statement (see 6.5) or a `requeue_statement` (see 9.5.4) may be used to complete the execution of an `accept_statement` or an `entry_body`. 29/2

27 The condition in the `entry_barrier` may reference anything visible except the formal parameters of the entry. This includes the entry index (if any), the components (including discriminants) of the protected object, the Count attribute of an entry of that protected object, and data global to the protected unit. 30

The restriction against referencing the formal parameters within an `entry_barrier` ensures that all calls of the same entry see the same barrier value. If it is necessary to look at the parameters of an entry call before deciding whether to handle it, the `entry_barrier` can be “**when** True” and the caller can be requeued (on some private entry) when its parameters indicate that it cannot be handled immediately. 31

*Examples**Examples of entry declarations:*

```

entry Read(V : out Item);
entry Seize;
entry Request(Level) (D : Item);  -- a family of entries

```

Examples of accept statements:

```

accept Shut_Down;
accept Read(V : out Item) do
    V := Local_Item;
end Read;
accept Request (Low) (D : Item) do
    ...
end Request;

```

9.5.3 Entry Calls

An *entry_call_statement* (an *entry call*) can appear in various contexts. A *simple* entry call is a stand-alone statement that represents an unconditional call on an entry of a target task or a protected object. Entry calls can also appear as part of *select_statements* (see 9.7).

Syntax

```

entry_call_statement ::= entry_name [actual_parameter_part];

```

Name Resolution Rules

The *entry_name* given in an *entry_call_statement* shall resolve to denote an entry. The rules for parameter associations are the same as for subprogram calls (see 6.4 and 6.4.1).

Static Semantics

The *entry_name* of an *entry_call_statement* specifies (explicitly or implicitly) the target object of the call, the entry or entry family, and the entry index, if any (see 9.5).

Dynamic Semantics

Under certain circumstances (detailed below), an entry of a task or protected object is checked to see whether it is *open* or *closed*:

- An entry of a task is open if the task is blocked on an *accept_statement* that corresponds to the entry (see 9.5.2), or on a *selective_accept* (see 9.7.1) with an open *accept_alternative* that corresponds to the entry; otherwise it is closed.
- An entry of a protected object is open if the condition of the *entry_barrier* of the corresponding *entry_body* evaluates to True; otherwise it is closed. If the evaluation of the condition propagates an exception, the exception *Program_Error* is propagated to all current callers of all entries of the protected object.

For the execution of an *entry_call_statement*, evaluation of the name and of the parameter associations is as for a subprogram call (see 6.4). The entry call is then *issued*: For a call on an entry of a protected object, a new protected action is started on the object (see 9.5.1). The named entry is checked to see if it is open; if open, the entry call is said to be *selected immediately*, and the execution of the call proceeds as follows:

- For a call on an open entry of a task, the accepting task becomes ready and continues the execution of the corresponding *accept_statement* (see 9.5.2).

- For a call on an open entry of a protected object, the corresponding `entry_body` is executed (see 9.5.2) as part of the protected action. 10

If the `accept_statement` or `entry_body` completes other than by a requeue (see 9.5.4), return is made to the caller (after servicing the entry queues — see below); any necessary assigning back of formal to actual parameters occurs, as for a subprogram call (see 6.4.1); such assignments take place outside of any protected action. 11

If the named entry is closed, the entry call is added to an *entry queue* (as part of the protected action, for a call on a protected entry), and the call remains queued until it is selected or cancelled; there is a separate (logical) entry queue for each entry of a given task or protected object (including each entry of an entry family). 12

When a queued call is *selected*, it is removed from its entry queue. Selecting a queued call from a particular entry queue is called *servicing* the entry queue. An entry with queued calls can be serviced under the following circumstances: 13

- When the associated task reaches a corresponding `accept_statement`, or a `selective_accept` with a corresponding open `accept_alternative`; 14
- If after performing, as part of a protected action on the associated protected object, an operation on the object other than a call on a protected function, the entry is checked and found to be open. 15

If there is at least one call on a queue corresponding to an open entry, then one such call is selected according to the *entry queuing policy* in effect (see below), and the corresponding `accept_statement` or `entry_body` is executed as above for an entry call that is selected immediately. 16

The entry queuing policy controls selection among queued calls both for task and protected entry queues. The default entry queuing policy is to select calls on a given entry queue in order of arrival. If calls from two or more queues are simultaneously eligible for selection, the default entry queuing policy does not specify which queue is serviced first. Other entry queuing policies can be specified by `pragmas` (see D.4). 17

For a protected object, the above servicing of entry queues continues until there are no open entries with queued calls, at which point the protected action completes. 18

For an entry call that is added to a queue, and that is not the `triggering_statement` of an `asynchronous_select` (see 9.7.4), the calling task is blocked until the call is cancelled, or the call is selected and a corresponding `accept_statement` or `entry_body` completes without requeuing. In addition, the calling task is blocked during a rendezvous. 19

An attempt can be made to cancel an entry call upon an abort (see 9.8) and as part of certain forms of `select_statement` (see 9.7.2, 9.7.3, and 9.7.4). The cancellation does not take place until a point (if any) when the call is on some entry queue, and not protected from cancellation as part of a requeue (see 9.5.4); at such a point, the call is removed from the entry queue and the call completes due to the cancellation. The cancellation of a call on an entry of a protected object is a protected action, and as such cannot take place while any other protected action is occurring on the protected object. Like any protected action, it includes servicing of the entry queues (in case some entry barrier depends on a Count attribute). 20

A call on an entry of a task that has already completed its execution raises the exception `Tasking_Error` at the point of the call; similarly, this exception is raised at the point of the call if the called task completes its execution or becomes abnormal before accepting the call or completing the rendezvous (see 9.8). This applies equally to a simple entry call and to an entry call as part of a `select_statement`. 21

Implementation Permissions

- 22 An implementation may perform the sequence of steps of a protected action using any thread of control; it need not be that of the task that started the protected action. If an `entry_body` completes without requeuing, then the corresponding calling task may be made ready without waiting for the entire protected action to complete.
- 23 When the entry of a protected object is checked to see whether it is open, the implementation need not reevaluate the condition of the corresponding `entry_barrier` if no variable or attribute referenced by the condition (directly or indirectly) has been altered by the execution (or cancellation) of a protected procedure or entry call on the object since the condition was last evaluated.
- 24 An implementation may evaluate the conditions of all `entry_barriers` of a given protected object any time any entry of the object is checked to see if it is open.
- 25 When an attempt is made to cancel an entry call, the implementation need not make the attempt using the thread of control of the task (or interrupt) that initiated the cancellation; in particular, it may use the thread of control of the caller itself to attempt the cancellation, even if this might allow the entry call to be selected in the interim.

NOTES

- 26 28 If an exception is raised during the execution of an `entry_body`, it is propagated to the corresponding caller (see 11.4).
- 27 29 For a call on a protected entry, the entry is checked to see if it is open prior to queuing the call, and again thereafter if its Count attribute (see 9.9) is referenced in some entry barrier.
- 28 30 In addition to simple entry calls, the language permits timed, conditional, and asynchronous entry calls (see 9.7.2, 9.7.3, and see 9.7.4).
- 29 31 The condition of an `entry_barrier` is allowed to be evaluated by an implementation more often than strictly necessary, even if the evaluation might have side effects. On the other hand, an implementation need not reevaluate the condition if nothing it references was updated by an intervening protected action on the protected object, even if the condition references some global variable that might have been updated by an action performed from outside of a protected action.

Examples

30 *Examples of entry calls:*

- 31 `Agent.Shut_Down;` -- *see 9.1*
 `Parser.Next_Lexeme(E);` -- *see 9.1*
 `Pool(5).Read(Next_Char);` -- *see 9.1*
 `Controller.Request(Low)(Some_Item);` -- *see 9.1*
 `Flags(3).Seize;` -- *see 9.4*

9.5.4 Requeue Statements

- 1 A `requeue_statement` can be used to complete an `accept_statement` or `entry_body`, while redirecting the corresponding entry call to a new (or the same) entry queue. Such a *requeue* can be performed with or without allowing an intermediate cancellation of the call, due to an abort or the expiration of a delay.

Syntax

- 2/3 `requeue_statement ::= requeue procedure_or_entry_name [with abort];`

Name Resolution Rules

- 3/3 The *procedure_or_entry_name* of a `requeue_statement` shall resolve to denote an entry (the *requeue target*) that either has no parameters, or that has a profile that is type conformant (see 6.3.1) with the profile of the innermost enclosing `entry_body` or `accept_statement`.

Legality Rules

A `requeue_statement` shall be within a callable construct that is either an `entry_body` or an `accept_statement`, and this construct shall be the innermost enclosing body or callable construct. 4

If the `requeue` target has parameters, then its profile shall be subtype conformant with the profile of the innermost enclosing callable construct. 5/3

If the target is a procedure, the name shall denote a rename of an entry, or shall denote a prefixed view of a primitive subprogram of a synchronized interface, where the first parameter of the unprefix view of the primitive subprogram shall be a controlling parameter, and a pragma `Implemented with implementation_kind By_Entry` shall apply to the primitive subprogram. 5.1/3

In a `requeue_statement` of an `accept_statement` of some task unit, either the target object shall be a part of a formal parameter of the `accept_statement`, or the accessibility level of the target object shall not be equal to or statically deeper than any enclosing `accept_statement` of the task unit. In a `requeue_statement` of an `entry_body` of some protected unit, either the target object shall be a part of a formal parameter of the `entry_body`, or the accessibility level of the target object shall not be statically deeper than that of the `entry_declaration` for the `entry_body`. 6/3

Dynamic Semantics

The execution of a `requeue_statement` proceeds by first evaluating the *procedure_or_entry_name*, including the prefix identifying the target task or protected object and the expression identifying the entry within an entry family, if any. The `entry_body` or `accept_statement` enclosing the `requeue_statement` is then completed, finalized, and left (see 7.6.1). 7/3

For the execution of a `requeue` on an entry of a target task, after leaving the enclosing callable construct, the named entry is checked to see if it is open and the `requeued` call is either selected immediately or queued, as for a normal entry call (see 9.5.3). 8

For the execution of a `requeue` on an entry of a target protected object, after leaving the enclosing callable construct: 9

- if the `requeue` is an internal `requeue` (that is, the `requeue` is back on an entry of the same protected object — see 9.5), the call is added to the queue of the named entry and the ongoing protected action continues (see 9.5.1); 10
- if the `requeue` is an external `requeue` (that is, the target protected object is not implicitly the same as the current object — see 9.5), a protected action is started on the target object and proceeds as for a normal entry call (see 9.5.3). 11

If the `requeue` target named in the `requeue_statement` has formal parameters, then during the execution of the `accept_statement` or `entry_body` corresponding to the new entry, the formal parameters denote the same objects as did the corresponding formal parameters of the callable construct completed by the `requeue`. In any case, no parameters are specified in a `requeue_statement`; any parameter passing is implicit. 12/3

If the `requeue_statement` includes the reserved words **with abort** (it is a *requeue-with-abort*), then: 13

- if the original entry call has been aborted (see 9.8), then the `requeue` acts as an abort completion point for the call, and the call is cancelled and no `requeue` is performed; 14
- if the original entry call was timed (or conditional), then the original expiration time is the expiration time for the `requeued` call. 15

If the reserved words **with abort** do not appear, then the call remains protected against cancellation while queued as the result of the `requeue_statement`. 16

NOTES

- 17 32 A *requeue* is permitted from a single entry to an entry of an entry family, or vice-versa. The entry index, if any, plays no part in the subtype conformance check between the profiles of the two entries; an entry index is part of the *entry_name* for an entry of a family.

Examples

18 *Examples of requeue statements:*

- 19 **requeue** Request (Medium) **with abort**;
 -- requeue on a member of an entry family of the current task, see 9.1
- 20 **requeue** Flags (I) .Seize;
 -- requeue on an entry of an array component, see 9.4

9.6 Delay Statements, Duration, and Time

- 1 A *delay_statement* is used to block further execution until a specified *expiration time* is reached. The expiration time can be specified either as a particular point in time (in a *delay_until_statement*), or in seconds from the current time (in a *delay_relative_statement*). The language-defined package *Calendar* provides definitions for a type *Time* and associated operations, including a function *Clock* that returns the current time.

Syntax

- 2 *delay_statement* ::= *delay_until_statement* | *delay_relative_statement*
- 3 *delay_until_statement* ::= **delay until** *delay_expression*;
- 4 *delay_relative_statement* ::= **delay** *delay_expression*;

Name Resolution Rules

- 5 The expected type for the *delay_expression* in a *delay_relative_statement* is the predefined type *Duration*. The *delay_expression* in a *delay_until_statement* is expected to be of any nonlimited type.

Legality Rules

- 6 There can be multiple time bases, each with a corresponding clock, and a corresponding *time type*. The type of the *delay_expression* in a *delay_until_statement* shall be a time type — either the type *Time* defined in the language-defined package *Calendar* (see below), or some other implementation-defined time type (see D.8).

Static Semantics

- 7 There is a predefined fixed point type named *Duration*, declared in the visible part of package *Standard*; a value of type *Duration* is used to represent the length of an interval of time, expressed in seconds. The type *Duration* is not specific to a particular time base, but can be used with any time base.
- 8 A value of the type *Time* in package *Calendar*, or of some other implementation-defined time type, represents a time as reported by a corresponding clock.
- 9 The following language-defined library package exists:

- 10 **package** Ada.Calendar **is**
- type** Time **is private**;
- 11/2 **subtype** Year_Number **is** Integer **range** 1901 .. 2399;
- subtype** Month_Number **is** Integer **range** 1 .. 12;
- subtype** Day_Number **is** Integer **range** 1 .. 31;
- subtype** Day_Duration **is** Duration **range** 0.0 .. 86_400.0;

```

function Clock return Time;
function Year   (Date : Time) return Year_Number;
function Month  (Date : Time) return Month_Number;
function Day    (Date : Time) return Day_Number;
function Seconds(Date : Time) return Day_Duration;

procedure Split (Date  : in Time;
                 Year   : out Year_Number;
                 Month  : out Month_Number;
                 Day    : out Day_Number;
                 Seconds : out Day_Duration);

function Time_Of (Year  : Year_Number;
                  Month  : Month_Number;
                  Day    : Day_Number;
                  Seconds : Day_Duration := 0.0)

    return Time;

function "+" (Left : Time;   Right : Duration) return Time;
function "+" (Left : Duration; Right : Time) return Time;
function "-" (Left : Time;   Right : Duration) return Time;
function "-" (Left : Time;   Right : Time) return Duration;

function "<" (Left, Right : Time) return Boolean;
function "<=" (Left, Right : Time) return Boolean;
function ">" (Left, Right : Time) return Boolean;
function ">=" (Left, Right : Time) return Boolean;

Time_Error : exception;

private
    ... -- not specified by the language
end Ada.Calendar;

```

Dynamic Semantics

For the execution of a **delay_statement**, the *delay_expression* is first evaluated. For a **delay_until_statement**, the expiration time for the delay is the value of the *delay_expression*, in the time base associated with the type of the *expression*. For a **delay_relative_statement**, the expiration time is defined as the current time, in the time base associated with relative delays, plus the value of the *delay_expression* converted to the type *Duration*, and then rounded up to the next clock tick. The time base associated with relative delays is as defined in D.9, “Delay Accuracy” or is implementation defined.

The task executing a **delay_statement** is blocked until the expiration time is reached, at which point it becomes ready again. If the expiration time has already passed, the task is not blocked.

If an attempt is made to *cancel* the **delay_statement** (as part of an **asynchronous_select** or abort — see 9.7.4 and 9.8), the statement is cancelled if the expiration time has not yet passed, thereby completing the **delay_statement**.

The time base associated with the type *Time* of package *Calendar* is implementation defined. The function *Clock* of package *Calendar* returns a value representing the current time for this time base. The implementation-defined value of the named number *System.Tick* (see 13.7) is an approximation of the length of the real-time interval during which the value of *Calendar.Clock* remains constant.

The functions *Year*, *Month*, *Day*, and *Seconds* return the corresponding values for a given value of the type *Time*, as appropriate to an implementation-defined time zone; the procedure *Split* returns all four corresponding values. Conversely, the function *Time_Of* combines a year number, a month number, a day number, and a duration, into a value of type *Time*. The operators “+” and “-” for addition and subtraction of times and durations, and the relational operators for times, have the conventional meaning.

- 25 If `Time_Of` is called with a seconds value of `86_400.0`, the value returned is equal to the value of `Time_Of` for the next day with a seconds value of `0.0`. The value returned by the function `Seconds` or through the `Seconds` parameter of the procedure `Split` is always less than `86_400.0`.
- 26/1 The exception `Time_Error` is raised by the function `Time_Of` if the actual parameters do not form a proper date. This exception is also raised by the operators `"+"` and `"-"` if the result is not representable in the type `Time` or `Duration`, as appropriate. This exception is also raised by the functions `Year`, `Month`, `Day`, and `Seconds` and the procedure `Split` if the year number of the given date is outside of the range of the subtype `Year_Number`.

Implementation Requirements

- 27 The implementation of the type `Duration` shall allow representation of time intervals (both positive and negative) up to at least 86400 seconds (one day); `Duration'Small` shall not be greater than twenty milliseconds. The implementation of the type `Time` shall allow representation of all dates with year numbers in the range of `Year_Number`; it may allow representation of other dates as well (both earlier and later).

Implementation Permissions

- 28 An implementation may define additional time types (see D.8).
- 29 An implementation may raise `Time_Error` if the value of a *delay_expression* in a *delay_until_statement* of a *select_statement* represents a time more than 90 days past the current time. The actual limit, if any, is implementation-defined.

Implementation Advice

- 30 Whenever possible in an implementation, the value of `Duration'Small` should be no greater than 100 microseconds.
- 31 The time base for *delay_relative_statements* should be monotonic; it need not be the same time base as used for `Calendar.Clock`.

NOTES

- 32 33 A *delay_relative_statement* with a negative value of the *delay_expression* is equivalent to one with a zero value.
- 33 34 A *delay_statement* may be executed by the environment task; consequently *delay_statements* may be executed as part of the elaboration of a *library_item* or the execution of the main subprogram. Such statements delay the environment task (see 10.2).
- 34 35 A *delay_statement* is an abort completion point and a potentially blocking operation, even if the task is not actually blocked.
- 35 36 There is no necessary relationship between `System.Tick` (the resolution of the clock of package `Calendar`) and `Duration'Small` (the *small* of type `Duration`).
- 36 37 Additional requirements associated with *delay_statements* are given in D.9, "Delay Accuracy".

Examples

- 37 *Example of a relative delay statement:*
- 38 **delay** 3.0; -- *delay 3.0 seconds*

Example of a periodic task:

```

declare
  use Ada.Calendar;
  Next_Time : Time := Clock + Period;
                                -- Period is a global constant of type Duration
begin
  loop                          -- repeated every Period seconds
    delay until Next_Time;
    ... -- perform some actions
    Next_Time := Next_Time + Period;
  end loop;
end;

```

39
40

9.6.1 Formatting, Time Zones, and other operations for Time

Static Semantics

The following language-defined library packages exist:

```

package Ada.Calendar.Time_Zones is
  -- Time zone manipulation:
  type Time_Offset is range -28*60 .. 28*60;
  Unknown_Zone_Error : exception;
  function UTC_Time_Offset (Date : Time := Clock) return Time_Offset;
end Ada.Calendar.Time_Zones;

package Ada.Calendar.Arithmetic is
  -- Arithmetic on days:
  type Day_Count is range
    -366*(1+Year_Number'Last - Year_Number'First)
    ..
    366*(1+Year_Number'Last - Year_Number'First);
  subtype Leap_Seconds_Count is Integer range -2047 .. 2047;
  procedure Difference (Left, Right : in Time;
    Days : out Day_Count;
    Seconds : out Duration;
    Leap_Seconds : out Leap_Seconds_Count);
  function "+" (Left : Time; Right : Day_Count) return Time;
  function "+" (Left : Day_Count; Right : Time) return Time;
  function "-" (Left : Time; Right : Day_Count) return Time;
  function "-" (Left, Right : Time) return Day_Count;
end Ada.Calendar.Arithmetic;

with Ada.Calendar.Time_Zones;
package Ada.Calendar.Formatting is
  -- Day of the week:
  type Day_Name is (Monday, Tuesday, Wednesday, Thursday,
    Friday, Saturday, Sunday);
  function Day_of_Week (Date : Time) return Day_Name;
  -- Hours:Minutes:Seconds access:
  subtype Hour_Number is Natural range 0 .. 23;
  subtype Minute_Number is Natural range 0 .. 59;
  subtype Second_Number is Natural range 0 .. 59;
  subtype Second_Duration is Day_Duration range 0.0 .. 1.0;

```

```

21/2      function Year      (Date : Time;
                             Time_Zone : Time_Zones.Time_Offset := 0)
                             return Year_Number;

22/2      function Month    (Date : Time;
                             Time_Zone : Time_Zones.Time_Offset := 0)
                             return Month_Number;

23/2      function Day      (Date : Time;
                             Time_Zone : Time_Zones.Time_Offset := 0)
                             return Day_Number;

24/2      function Hour     (Date : Time;
                             Time_Zone : Time_Zones.Time_Offset := 0)
                             return Hour_Number;

25/2      function Minute   (Date : Time;
                             Time_Zone : Time_Zones.Time_Offset := 0)
                             return Minute_Number;

26/2      function Second   (Date : Time)
                             return Second_Number;

27/2      function Sub_Second (Date : Time)
                             return Second_Duration;

28/2      function Seconds_Of (Hour   : Hour_Number;
                             Minute  : Minute_Number;
                             Second   : Second_Number := 0;
                             Sub_Second : Second_Duration := 0.0)
                             return Day_Duration;

29/2      procedure Split   (Seconds   : in Day_Duration;
                             Hour       : out Hour_Number;
                             Minute    : out Minute_Number;
                             Second     : out Second_Number;
                             Sub_Second : out Second_Duration);

30/2      function Time_Of  (Year      : Year_Number;
                             Month     : Month_Number;
                             Day       : Day_Number;
                             Hour      : Hour_Number;
                             Minute    : Minute_Number;
                             Second     : Second_Number;
                             Sub_Second : Second_Duration := 0.0;
                             Leap_Second: Boolean := False;
                             Time_Zone  : Time_Zones.Time_Offset := 0)
                             return Time;

31/2      function Time_Of  (Year      : Year_Number;
                             Month     : Month_Number;
                             Day       : Day_Number;
                             Seconds    : Day_Duration := 0.0;
                             Leap_Second: Boolean := False;
                             Time_Zone  : Time_Zones.Time_Offset := 0)
                             return Time;

32/2      procedure Split   (Date      : in Time;
                             Year       : out Year_Number;
                             Month     : out Month_Number;
                             Day       : out Day_Number;
                             Hour      : out Hour_Number;
                             Minute    : out Minute_Number;
                             Second     : out Second_Number;
                             Sub_Second : out Second_Duration;
                             Time_Zone  : in Time_Zones.Time_Offset := 0);

```

procedure Split (Date : in Time; Year : out Year_Number; Month : out Month_Number; Day : out Day_Number; Hour : out Hour_Number; Minute : out Minute_Number; Second : out Second_Number; Sub_Second : out Second_Duration; Leap_Second : out Boolean; Time_Zone : in Time_Zones.Time_Offset := 0);	33/2
procedure Split (Date : in Time; Year : out Year_Number; Month : out Month_Number; Day : out Day_Number; Seconds : out Day_Duration; Leap_Second : out Boolean; Time_Zone : in Time_Zones.Time_Offset := 0);	34/2
-- Simple image and value:	35/2
function Image (Date : Time; Include_Time_Fraction : Boolean := False; Time_Zone : Time_Zones.Time_Offset := 0) return String;	
function Value (Date : String; Time_Zone : Time_Zones.Time_Offset := 0) return Time;	36/2
function Image (Elapsed_Time : Duration; Include_Time_Fraction : Boolean := False) return String;	37/2
function Value (Elapsed_Time : String) return Duration;	38/2
end Ada.Calendar.Formatting;	39/2
Type Time_Offset represents the number of minutes difference between the implementation-defined time zone used by Calendar and another time zone.	40/2
function UTC_Time_Offset (Date : Time := Clock) return Time_Offset;	41/2
Returns, as a number of minutes, the difference between the implementation-defined time zone of Calendar, and UTC time, at the time Date. If the time zone of the Calendar implementation is unknown, then Unknown_Zone_Error is raised.	42/2
procedure Difference (Left, Right : in Time; Days : out Day_Count; Seconds : out Duration; Leap_Seconds : out Leap_Seconds_Count);	43/2
Returns the difference between Left and Right. Days is the number of days of difference, Seconds is the remainder seconds of difference excluding leap seconds, and Leap_Seconds is the number of leap seconds. If Left < Right, then Seconds <= 0.0, Days <= 0, and Leap_Seconds <= 0. Otherwise, all values are nonnegative. The absolute value of Seconds is always less than 86_400.0. For the returned values, if Days = 0, then Seconds + Duration(Leap_Seconds) = Calendar."-" (Left, Right).	44/2
function "+" (Left : Time; Right : Day_Count) return Time;	45/2
function "+" (Left : Day_Count; Right : Time) return Time;	
Adds a number of days to a time value. Time_Error is raised if the result is not representable as a value of type Time.	46/2
function "-" (Left : Time; Right : Day_Count) return Time;	47/2
Subtracts a number of days from a time value. Time_Error is raised if the result is not representable as a value of type Time.	48/2

49/2 **function** "-" (Left, Right : Time) **return** Day_Count;

50/2 Subtracts two time values, and returns the number of days between them. This is the same value that Difference would return in Days.

51/2 **function** Day_of_Week (Date : Time) **return** Day_Name;

52/2 Returns the day of the week for Time. This is based on the Year, Month, and Day values of Time.

53/2 **function** Year (Date : Time;
 Time_Zone : Time_Zones.Time_Offset := 0)
 return Year_Number;

54/2 Returns the year for Date, as appropriate for the specified time zone offset.

55/2 **function** Month (Date : Time;
 Time_Zone : Time_Zones.Time_Offset := 0)
 return Month_Number;

56/2 Returns the month for Date, as appropriate for the specified time zone offset.

57/2 **function** Day (Date : Time;
 Time_Zone : Time_Zones.Time_Offset := 0)
 return Day_Number;

58/2 Returns the day number for Date, as appropriate for the specified time zone offset.

59/2 **function** Hour (Date : Time;
 Time_Zone : Time_Zones.Time_Offset := 0)
 return Hour_Number;

60/2 Returns the hour for Date, as appropriate for the specified time zone offset.

61/2 **function** Minute (Date : Time;
 Time_Zone : Time_Zones.Time_Offset := 0)
 return Minute_Number;

62/2 Returns the minute within the hour for Date, as appropriate for the specified time zone offset.

63/2 **function** Second (Date : Time)
 return Second_Number;

64/2 Returns the second within the hour and minute for Date.

65/2 **function** Sub_Second (Date : Time)
 return Second_Duration;

66/2 Returns the fraction of second for Date (this has the same accuracy as Day_Duration). The value returned is always less than 1.0.

67/2 **function** Seconds_Of (Hour : Hour_Number;
 Minute : Minute_Number;
 Second : Second_Number := 0;
 Sub_Second : Second_Duration := 0.0)
 return Day_Duration;

68/2 Returns a Day_Duration value for the combination of the given Hour, Minute, Second, and Sub_Second. This value can be used in Calendar.Time_Of as well as the argument to Calendar."+" and Calendar."-". If Seconds_Of is called with a Sub_Second value of 1.0, the value returned is equal to the value of Seconds_Of for the next second with a Sub_Second value of 0.0.


```

procedure Split (Seconds      : in Day_Duration;
                  Hour         : out Hour_Number;
                  Minute       : out Minute_Number;
                  Second       : out Second_Number;
                  Sub_Second   : out Second_Duration);

```

Splits Seconds into Hour, Minute, Second and Sub_Second in such a way that the resulting values all belong to their respective subtypes. The value returned in the Sub_Second parameter is always less than 1.0. 70/2

```

function Time_Of (Year        : Year_Number;
                  Month       : Month_Number;
                  Day         : Day_Number;
                  Hour        : Hour_Number;
                  Minute      : Minute_Number;
                  Second      : Second_Number;
                  Sub_Second  : Second_Duration := 0.0;
                  Leap_Second: Boolean := False;
                  Time_Zone   : Time_Zones.Time_Offset := 0)
return Time;

```

If Leap_Second is False, returns a Time built from the date and time values, relative to the specified time zone offset. If Leap_Second is True, returns the Time that represents the time within the leap second that is one second later than the time specified by the other parameters. Time_Error is raised if the parameters do not form a proper date or time. If Time_Of is called with a Sub_Second value of 1.0, the value returned is equal to the value of Time_Of for the next second with a Sub_Second value of 0.0. 72/2

```

function Time_Of (Year        : Year_Number;
                  Month       : Month_Number;
                  Day         : Day_Number;
                  Seconds     : Day_Duration := 0.0;
                  Leap_Second: Boolean := False;
                  Time_Zone   : Time_Zones.Time_Offset := 0)
return Time;

```

If Leap_Second is False, returns a Time built from the date and time values, relative to the specified time zone offset. If Leap_Second is True, returns the Time that represents the time within the leap second that is one second later than the time specified by the other parameters. Time_Error is raised if the parameters do not form a proper date or time. If Time_Of is called with a Seconds value of 86_400.0, the value returned is equal to the value of Time_Of for the next day with a Seconds value of 0.0. 74/2

```

procedure Split (Date      : in Time;
                  Year      : out Year_Number;
                  Month     : out Month_Number;
                  Day       : out Day_Number;
                  Hour      : out Hour_Number;
                  Minute    : out Minute_Number;
                  Second     : out Second_Number;
                  Sub_Second : out Second_Duration;
                  Leap_Second: out Boolean;
                  Time_Zone  : in Time_Zones.Time_Offset := 0);

```

If Date does not represent a time within a leap second, splits Date into its constituent parts (Year, Month, Day, Hour, Minute, Second, Sub_Second), relative to the specified time zone offset, and sets Leap_Second to False. If Date represents a time within a leap second, set the constituent parts to values corresponding to a time one second earlier than that given by Date, relative to the specified time zone offset, and sets Leap_Seconds to True. The value returned in the Sub_Second parameter is always less than 1.0. 76/2

- 77/2 **procedure** Split (Date : **in** Time;
 Year : **out** Year_Number;
 Month : **out** Month_Number;
 Day : **out** Day_Number;
 Hour : **out** Hour_Number;
 Minute : **out** Minute_Number;
 Second : **out** Second_Number;
 Sub_Second : **out** Second_Duration;
 Time_Zone : **in** Time_Zones.Time_Offset := 0);
- 78/2 Splits Date into its constituent parts (Year, Month, Day, Hour, Minute, Second, Sub_Second), relative to the specified time zone offset. The value returned in the Sub_Second parameter is always less than 1.0.
- 79/2 **procedure** Split (Date : **in** Time;
 Year : **out** Year_Number;
 Month : **out** Month_Number;
 Day : **out** Day_Number;
 Seconds : **out** Day_Duration;
 Leap_Second : **out** Boolean;
 Time_Zone : **in** Time_Zones.Time_Offset := 0);
- 80/2 If Date does not represent a time within a leap second, splits Date into its constituent parts (Year, Month, Day, Seconds), relative to the specified time zone offset, and sets Leap_Second to False. If Date represents a time within a leap second, set the constituent parts to values corresponding to a time one second earlier than that given by Date, relative to the specified time zone offset, and sets Leap_Seconds to True. The value returned in the Seconds parameter is always less than 86_400.0.
- 81/2 **function** Image (Date : Time;
 Include_Time_Fraction : Boolean := False;
 Time_Zone : Time_Zones.Time_Offset := 0) **return** String;
- 82/2 Returns a string form of the Date relative to the given Time_Zone. The format is "Year-Month-Day Hour:Minute:Second", where the Year is a 4-digit value, and all others are 2-digit values, of the functions defined in Calendar and Calendar.Formatting, including a leading zero, if needed. The separators between the values are a minus, another minus, a colon, and a single space between the Day and Hour. If Include_Time_Fraction is True, the integer part of Sub_Seconds*100 is suffixed to the string as a point followed by a 2-digit value.
- 83/2 **function** Value (Date : String;
 Time_Zone : Time_Zones.Time_Offset := 0) **return** Time;
- 84/2 Returns a Time value for the image given as Date, relative to the given time zone. Constraint_Error is raised if the string is not formatted as described for Image, or the function cannot interpret the given string as a Time value.
- 85/2 **function** Image (Elapsed_Time : Duration;
 Include_Time_Fraction : Boolean := False) **return** String;
- 86/2 Returns a string form of the Elapsed_Time. The format is "Hour:Minute:Second", where all values are 2-digit values, including a leading zero, if needed. The separators between the values are colons. If Include_Time_Fraction is True, the integer part of Sub_Seconds*100 is suffixed to the string as a point followed by a 2-digit value. If Elapsed_Time < 0.0, the result is Image (**abs** Elapsed_Time, Include_Time_Fraction) prefixed with a minus sign. If **abs** Elapsed_Time represents 100 hours or more, the result is implementation-defined.

function Value (Elapsed_Time : String) **return** Duration; 87/2

Returns a Duration value for the image given as Elapsed_Time. Constraint_Error is raised if the string is not formatted as described for Image, or the function cannot interpret the given string as a Duration value. 88/2

Implementation Advice

An implementation should support leap seconds if the target system supports them. If leap seconds are not supported, Difference should return zero for Leap_Seconds, Split should return False for Leap_Second, and Time_Of should raise Time_Error if Leap_Second is True. 89/2

NOTES

38 The implementation-defined time zone of package Calendar may, but need not, be the local time zone. UTC_Time_Offset always returns the difference relative to the implementation-defined time zone of package Calendar. If UTC_Time_Offset does not raise Unknown_Zone_Error, UTC time can be safely calculated (within the accuracy of the underlying time-base). 90/2

39 Calling Split on the results of subtracting Duration(UTC_Time_Offset*60) from Clock provides the components (hours, minutes, and so on) of the UTC time. In the United States, for example, UTC_Time_Offset will generally be negative. 91/2

9.7 Select Statements

There are four forms of the **select_statement**. One form provides a selective wait for one or more **select_alternatives**. Two provide timed and conditional entry calls. The fourth provides asynchronous transfer of control. 1

Syntax

```
select_statement ::=
  selective_accept
  | timed_entry_call
  | conditional_entry_call
  | asynchronous_select
```

2

Examples

Example of a select statement: 3

```
select
  accept Driver_Awake_Signal;
or
  delay 30.0*Seconds;
  Stop_The_Train;
end select;
```

4

9.7.1 Selective Accept

This form of the **select_statement** allows a combination of waiting for, and selecting from, one or more alternatives. The selection may depend on conditions associated with each alternative of the **selective_accept**. 1

Syntax

```

2      selective_accept ::=
      select
      [guard]
      select_alternative
    { or
      [guard]
      select_alternative }
    [ else
      sequence_of_statements ]
      end select;

```

```

3      guard ::= when condition =>

```

```

4      select_alternative ::=
      accept_alternative
      | delay_alternative
      | terminate_alternative

```

```

5      accept_alternative ::=
      accept_statement [sequence_of_statements]

```

```

6      delay_alternative ::=
      delay_statement [sequence_of_statements]

```

```

7      terminate_alternative ::= terminate;

```

8 A **selective_accept** shall contain at least one **accept_alternative**. In addition, it can contain:

- 9 • a **terminate_alternative** (only one); or
- 10 • one or more **delay_alternatives**; or
- 11 • an *else part* (the reserved word **else** followed by a **sequence_of_statements**).

12 These three possibilities are mutually exclusive.

Legality Rules

13 If a **selective_accept** contains more than one **delay_alternative**, then all shall be **delay_relative_statements**, or all shall be **delay_until_statements** for the same time type.

Dynamic Semantics

14 A **select_alternative** is said to be *open* if it is not immediately preceded by a **guard**, or if the condition of its **guard** evaluates to True. It is said to be *closed* otherwise.

15 For the execution of a **selective_accept**, any **guard conditions** are evaluated; open alternatives are thus determined. For an open **delay_alternative**, the *delay expression* is also evaluated. Similarly, for an open **accept_alternative** for an entry of a family, the *entry_index* is also evaluated. These evaluations are performed in an arbitrary order, except that a *delay expression* or *entry_index* is not evaluated until after evaluating the corresponding condition, if any. Selection and execution of one open alternative, or of the else part, then completes the execution of the **selective_accept**; the rules for this selection are described below.

16 Open **accept_alternatives** are first considered. Selection of one such alternative takes place immediately if the corresponding entry already has queued calls. If several alternatives can thus be selected, one of them is selected according to the entry queuing policy in effect (see 9.5.3 and D.4). When such an alternative is selected, the selected call is removed from its entry queue and the **handled_sequence_of_statements** (if any) of the corresponding **accept_statement** is executed; after the rendezvous completes any subsequent

`sequence_of_statements` of the alternative is executed. If no selection is immediately possible (in the above sense) and there is no else part, the task blocks until an open alternative can be selected.

Selection of the other forms of alternative or of an else part is performed as follows:

- An open `delay_alternative` is selected when its expiration time is reached if no `accept_alternative` or other `delay_alternative` can be selected prior to the expiration time. If several `delay_alternatives` have this same expiration time, one of them is selected according to the queuing policy in effect (see D.4); the default queuing policy chooses arbitrarily among the `delay_alternatives` whose expiration time has passed.
- The else part is selected and its `sequence_of_statements` is executed if no `accept_alternative` can immediately be selected; in particular, if all alternatives are closed.
- An open `terminate_alternative` is selected if the conditions stated at the end of clause 9.3 are satisfied.

The exception `Program_Error` is raised if all alternatives are closed and there is no else part.

NOTES

40 A `selective_accept` is allowed to have several open `delay_alternatives`. A `selective_accept` is allowed to have several open `accept_alternatives` for the same entry.

Examples

Example of a task body with a selective accept:

```
task body Server is
  Current_Work_Item : Work_Item;
begin
  loop
    select
      accept Next_Work_Item(WI : in Work_Item) do
        Current_Work_Item := WI;
      end;
      Process_Work_Item(Current_Work_Item);
    or
      accept Shut_Down;
      exit; -- Premature shut down requested
    or
      terminate; -- Normal shutdown at end of scope
    end select;
  end loop;
end Server;
```

9.7.2 Timed Entry Calls

A `timed_entry_call` issues an entry call that is cancelled if the call (or a requeue-with-abort of the call) is not selected before the expiration time is reached. A procedure call may appear rather than an entry call for cases where the procedure might be implemented by an entry.

Syntax

```
timed_entry_call ::=
  select
    entry_call_alternative
  or
    delay_alternative
  end select;

entry_call_alternative ::=
  procedure_or_entry_call [sequence_of_statements]
```

3.1/2 `procedure_or_entry_call ::=`
 `procedure_call_statement | entry_call_statement`

Legality Rules

3.2/2 If a `procedure_call_statement` is used for a `procedure_or_entry_call`, the *procedure_name* or *procedure_prefix* of the `procedure_call_statement` shall statically denote an entry renamed as a procedure or (a view of) a primitive subprogram of a limited interface whose first parameter is a controlling parameter (see 3.9.2).

Static Semantics

3.3/2 If a `procedure_call_statement` is used for a `procedure_or_entry_call`, and the procedure is implemented by an entry, then the *procedure_name*, or *procedure_prefix* and possibly the first parameter of the `procedure_call_statement`, determine the target object of the call and the entry to be called.

Dynamic Semantics

- 4/2 For the execution of a `timed_entry_call`, the *entry_name*, *procedure_name*, or *procedure_prefix*, and any actual parameters are evaluated, as for a simple entry call (see 9.5.3) or procedure call (see 6.4). The expiration time (see 9.6) for the call is determined by evaluating the *delay_expression* of the *delay_alternative*. If the call is an entry call or a call on a procedure implemented by an entry, the entry call is then issued. Otherwise, the call proceeds as described in 6.4 for a procedure call, followed by the *sequence_of_statements* of the *entry_call_alternative*; the *sequence_of_statements* of the *delay_alternative* is ignored.
- 5 If the call is queued (including due to a requeue-with-abort), and not selected before the expiration time is reached, an attempt to cancel the call is made. If the call completes due to the cancellation, the optional *sequence_of_statements* of the *delay_alternative* is executed; if the entry call completes normally, the optional *sequence_of_statements* of the *entry_call_alternative* is executed.

Examples

6 *Example of a timed entry call:*

7 `select`
 `Controller.Request (Medium) (Some_Item) ;`
 `or`
 `delay 45.0 ;`
 `-- controller too busy, try something else`
 `end select;`

9.7.3 Conditional Entry Calls

1/2 A `conditional_entry_call` issues an entry call that is then cancelled if it is not selected immediately (or if a requeue-with-abort of the call is not selected immediately). A procedure call may appear rather than an entry call for cases where the procedure might be implemented by an entry.

Syntax

2 `conditional_entry_call ::=`
 `select`
 `entry_call_alternative`
 `else`
 `sequence_of_statements`
 `end select;`

Dynamic Semantics

The execution of a `conditional_entry_call` is defined to be equivalent to the execution of a `timed_entry_call` with a `delay_alternative` specifying an immediate expiration time and the same `sequence_of_statements` as given after the reserved word **else**. 3

NOTES

41 A `conditional_entry_call` may briefly increase the Count attribute of the entry, even if the conditional call is not selected. 4

Examples

Example of a conditional entry call: 5

```

procedure Spin(R : in Resource) is
begin
  loop
    select
      R.Seize;
      return;
    else
      null; -- busy waiting
    end select;
  end loop;
end;

```

6

9.7.4 Asynchronous Transfer of Control

- 1 An asynchronous `select_statement` provides asynchronous transfer of control upon completion of an entry call or the expiration of a delay.

Syntax

```

2 asynchronous_select ::=
    select
      triggering_alternative
    then abort
      abortable_part
    end select;
3 triggering_alternative ::= triggering_statement [sequence_of_statements]
4/2 triggering_statement ::= procedure_or_entry_call | delay_statement
5 abortable_part ::= sequence_of_statements

```

Dynamic Semantics

- 6/2 For the execution of an `asynchronous_select` whose `triggering_statement` is a `procedure_or_entry_call`, the `entry_name`, `procedure_name`, or `procedure_prefix`, and actual parameters are evaluated as for a simple entry call (see 9.5.3) or procedure call (see 6.4). If the call is an entry call or a call on a procedure implemented by an entry, the entry call is issued. If the entry call is queued (or requeued-with-abort), then the `abortable_part` is executed. If the entry call is selected immediately, and never requeued-with-abort, then the `abortable_part` is never started. If the call is on a procedure that is not implemented by an entry, the call proceeds as described in 6.4, followed by the `sequence_of_statements` of the `triggering_alternative`; the `abortable_part` is never started.
- 7 For the execution of an `asynchronous_select` whose `triggering_statement` is a `delay_statement`, the `delay_expression` is evaluated and the expiration time is determined, as for a normal `delay_statement`. If the expiration time has not already passed, the `abortable_part` is executed.
- 8 If the `abortable_part` completes and is left prior to completion of the `triggering_statement`, an attempt to cancel the `triggering_statement` is made. If the attempt to cancel succeeds (see 9.5.3 and 9.6), the `asynchronous_select` is complete.
- 9 If the `triggering_statement` completes other than due to cancellation, the `abortable_part` is aborted (if started but not yet completed — see 9.8). If the `triggering_statement` completes normally, the optional `sequence_of_statements` of the `triggering_alternative` is executed after the `abortable_part` is left.

Examples

- 10 *Example of a main command loop for a command interpreter:*

```

11 loop
    select
      Terminal.Wait_For_Interrupt;
      Put_Line("Interrupted");
    then abort
      -- This will be abandoned upon terminal interrupt
      Put_Line("-> ");
      Get_Line(Command, Last);
      Process_Command(Command(1..Last));
    end select;
end loop;

```


Example of a time-limited calculation:

```

select
  delay 5.0;
  Put_Line("Calculation does not converge");
then abort
  -- This calculation should finish in 5.0 seconds;
  -- if not, it is assumed to diverge.
  Horribly_Complicated_Recursive_Function(X, Y);
end select;

```

12
13

9.8 Abort of a Task - Abort of a Sequence of Statements

An `abort_statement` causes one or more tasks to become abnormal, thus preventing any further interaction with such tasks. The completion of the `triggering_statement` of an `asynchronous_select` causes a `sequence_of_statements` to be aborted.

1

Syntax

`abort_statement ::= abort task_name {, task_name};`

2

Name Resolution Rules

Each `task_name` is expected to be of any task type; they need not all be of the same task type.

3

Dynamic Semantics

For the execution of an `abort_statement`, the given `task_names` are evaluated in an arbitrary order. Each named task is then *aborted*, which consists of making the task *abnormal* and aborting the execution of the corresponding `task_body`, unless it is already completed.

4

When the execution of a construct is *aborted* (including that of a `task_body` or of a `sequence_of_statements`), the execution of every construct included within the aborted execution is also aborted, except for executions included within the execution of an *abort-deferred* operation; the execution of an abort-deferred operation continues to completion without being affected by the abort; the following are the abort-deferred operations:

5

- a protected action;
- waiting for an entry call to complete (after having initiated the attempt to cancel it — see below);
- waiting for the termination of dependent tasks;
- the execution of an Initialize procedure as the last step of the default initialization of a controlled object;
- the execution of a Finalize procedure as part of the finalization of a controlled object;
- an assignment operation to an object with a controlled part.

6

7

8

9

10

11

The last three of these are discussed further in 7.6.

12

When a master is aborted, all tasks that depend on that master are aborted.

13

The order in which tasks become abnormal as the result of an `abort_statement` or the abort of a `sequence_of_statements` is not specified by the language.

14

If the execution of an entry call is aborted, an immediate attempt is made to cancel the entry call (see 9.5.3). If the execution of a construct is aborted at a time when the execution is blocked, other than for an entry call, at a point that is outside the execution of an abort-deferred operation, then the execution of the

15

construct completes immediately. For an abort due to an **abort_statement**, these immediate effects occur before the execution of the **abort_statement** completes. Other than for these immediate cases, the execution of a construct that is aborted does not necessarily complete before the **abort_statement** completes. However, the execution of the aborted construct completes no later than its next *abort completion point* (if any) that occurs outside of an abort-deferred operation; the following are abort completion points for an execution:

- 16 • the point where the execution initiates the activation of another task;
- 17 • the end of the activation of a task;
- 18 • the start or end of the execution of an entry call, **accept_statement**, **delay_statement**, or **abort_statement**;
- 19 • the start of the execution of a **select_statement**, or of the **sequence_of_statements** of an **exception_handler**.

Bounded (Run-Time) Errors

- 20 An attempt to execute an **asynchronous_select** as part of the execution of an abort-deferred operation is a bounded error. Similarly, an attempt to create a task that depends on a master that is included entirely within the execution of an abort-deferred operation is a bounded error. In both cases, **Program_Error** is raised if the error is detected by the implementation; otherwise the operations proceed as they would outside an abort-deferred operation, except that an abort of the **abortable_part** or the created task might or might not have an effect.

Erroneous Execution

- 21 If an assignment operation completes prematurely due to an abort, the assignment is said to be *disrupted*; the target of the assignment or its parts can become abnormal, and certain subsequent uses of the object can be erroneous, as explained in 13.9.1.

NOTES

- 22 42 An **abort_statement** should be used only in situations requiring unconditional termination.
- 23 43 A task is allowed to abort any task it can name, including itself.
- 24 44 Additional requirements associated with abort are given in D.6, “Preemptive Abort”.

9.9 Task and Entry Attributes

Dynamic Semantics

- 1 For a prefix **T** that is of a task type (after any implicit dereference), the following attributes are defined:
- 2 **T'Callable** Yields the value **True** when the task denoted by **T** is *callable*, and **False** otherwise; a task is callable unless it is completed or abnormal. The value of this attribute is of the predefined type **Boolean**.
- 3 **T'Terminated** Yields the value **True** if the task denoted by **T** is terminated, and **False** otherwise. The value of this attribute is of the predefined type **Boolean**.
- 4 For a prefix **E** that denotes an entry of a task or protected unit, the following attribute is defined. This attribute is only allowed within the body of the task or protected unit, but excluding, in the case of an entry of a task unit, within any program unit that is, itself, inner to the body of the task unit.
- 5 **E'Count** Yields the number of calls presently queued on the entry **E** of the current instance of the unit. The value of this attribute is of the type *universal_integer*.

NOTES

- 45 For the Count attribute, the entry can be either a single entry or an entry of a family. The name of the entry or entry family can be either a `direct_name` or an expanded name. 6
- 46 Within task units, algorithms interrogating the attribute E'Count should take precautions to allow for the increase of the value of this attribute for incoming entry calls, and its decrease, for example with `timed_entry_calls`. Also, a `conditional_entry_call` may briefly increase this value, even if the conditional call is not accepted. 7
- 47 Within protected units, algorithms interrogating the attribute E'Count in the `entry_barrier` for the entry E should take precautions to allow for the evaluation of the condition of the barrier both before and after queuing a given caller. 8

9.10 Shared Variables

Static Semantics

If two different objects, including nonoverlapping parts of the same object, are *independently addressable*, they can be manipulated concurrently by two different tasks without synchronization. Any two nonoverlapping objects are independently addressable if either object is specified as independently addressable (see C.6). Otherwise, two nonoverlapping objects are independently addressable except when they are both parts of a composite object for which a non-confirming representation item is used to specify packing, record layout, `Component_Size`, or convention, in which case it is unspecified whether the parts are independently addressable. 1/3

Dynamic Semantics

Separate tasks normally proceed independently and concurrently with one another. However, task interactions can be used to synchronize the actions of two or more tasks to allow, for example, meaningful communication by the direct updating and reading of variables shared between the tasks. The actions of two different tasks are synchronized in this sense when an action of one task *signals* an action of the other task; an action A1 is defined to signal an action A2 under the following circumstances: 2

- If A1 and A2 are part of the execution of the same task, and the language rules require A1 to be performed before A2; 3
- If A1 is the action of an activator that initiates the activation of a task, and A2 is part of the execution of the task that is activated; 4
- If A1 is part of the activation of a task, and A2 is the action of waiting for completion of the activation; 5
- If A1 is part of the execution of a task, and A2 is the action of waiting for the termination of the task; 6
- If A1 is the termination of a task T, and A2 is either an evaluation of the expression T'Terminated that results in True, or a call to `Ada.Task_Identification.Is_Terminated` with an actual parameter that identifies T and a result of True (see C.7.1); 6.1/3
- If A1 is the action of issuing an entry call, and A2 is part of the corresponding execution of the appropriate `entry_body` or `accept_statement`. 7
- If A1 is part of the execution of an `accept_statement` or `entry_body`, and A2 is the action of returning from the corresponding entry call; 8
- If A1 is part of the execution of a protected procedure body or `entry_body` for a given protected object, and A2 is part of a later execution of an `entry_body` for the same protected object; 9
- If A1 signals some action that in turn signals A2. 10

Erroneous Execution

- 11 Given an action of assigning to an object, and an action of reading or updating a part of the same object (or of a neighboring object if the two are not independently addressable), then the execution of the actions is erroneous unless the actions are *sequential*. Two actions are sequential if one of the following is true:
- 12 • One action signals the other;
 - 13 • Both actions occur as part of the execution of the same task;
 - 14 • Both actions occur as part of protected actions on the same protected object, and at most one of the actions is part of a call on a protected function of the protected object.
- 15 A `pragma Atomic` or `Atomic_Components` may also be used to ensure that certain reads and updates are sequential — see C.6.

9.11 Example of Tasking and Synchronization

Examples

- 1 The following example defines a buffer protected object to smooth variations between the speed of output of a producing task and the speed of input of some consuming task. For instance, the producing task might have the following structure:

```

2      task Producer;
3/2    task body Producer is
        Person : Person_Name; -- see 3.10.1
    begin
        loop
            ... -- simulate arrival of the next customer
            Buffer.Append_Wait(Person);
            exit when Person = null;
        end loop;
    end Producer;
```

- 4 and the consuming task might have the following structure:

```

5      task Consumer;
6/2    task body Consumer is
        Person : Person_Name;
    begin
        loop
            Buffer.Remove_First_Wait(Person);
            exit when Person = null;
            ... -- simulate serving a customer
        end loop;
    end Consumer;
```

- 7/2 The buffer object contains an internal array of person names managed in a round-robin fashion. The array has two indices, an `In_Index` denoting the index for the next input person name and an `Out_Index` denoting the index for the next output person name.
- 7.1/2 The `Buffer` is defined as an extension of the `Synchronized_Queue` interface (see 3.9.4), and as such promises to implement the abstraction defined by that interface. By doing so, the `Buffer` can be passed to the `Transfer` class-wide operation defined for objects of a type covered by `Queue'Class`.

```

protected Buffer is new Synchronized_Queue with -- see 3.9.4
  entry Append_Wait(Person : in Person_Name);
  entry Remove_First_Wait(Person : out Person_Name);
  function Cur_Count return Natural;
  function Max_Count return Natural;
  procedure Append(Person : in Person_Name);
  procedure Remove_First(Person : out Person_Name);
private
  Pool      : Person_Name Array(1 .. 100);
  Count     : Natural := 0;
  In_Index, Out_Index : Positive := 1;
end Buffer;

protected body Buffer is
  entry Append_Wait(Person : in Person_Name)
    when Count < Pool'Length is
  begin
    Append(Person);
  end Append_Wait;
  procedure Append(Person : in Person_Name) is
  begin
    if Count = Pool'Length then
      raise Queue_Error with "Buffer Full"; -- see 11.3
    end if;
    Pool(In_Index) := Person;
    In_Index       := (In_Index mod Pool'Length) + 1;
    Count          := Count + 1;
  end Append;
  entry Remove_First_Wait(Person : out Person_Name)
    when Count > 0 is
  begin
    Remove_First(Person);
  end Remove_First_Wait;
  procedure Remove_First(Person : out Person_Name) is
  begin
    if Count = 0 then
      raise Queue_Error with "Buffer Empty"; -- see 11.3
    end if;
    Person := Pool(Out_Index);
    Out_Index := (Out_Index mod Pool'Length) + 1;
    Count := Count - 1;
  end Remove_First;
  function Cur_Count return Natural is
  begin
    return Buffer.Count;
  end Cur_Count;
  function Max_Count return Natural is
  begin
    return Pool'Length;
  end Max_Count;
end Buffer;

```


Section 10: Program Structure and Compilation Issues

The overall structure of programs and the facilities for separate compilation are described in this section. A *program* is a set of *partitions*, each of which may execute in a separate address space, possibly on a separate computer.

As explained below, a partition is constructed from *library units*. Syntactically, the declaration of a library unit is a *library_item*, as is the body of a library unit. An implementation may support a concept of a *program library* (or simply, a “library”), which contains *library_items* and their subunits. Library units may be organized into a hierarchy of children, grandchildren, and so on.

This section has two clauses: 10.1, “Separate Compilation” discusses compile-time issues related to separate compilation. 10.2, “Program Execution” discusses issues related to what is traditionally known as “link time” and “run time” — building and executing partitions.

10.1 Separate Compilation

A *program unit* is either a package, a task unit, a protected unit, a protected entry, a generic unit, or an explicitly declared subprogram other than an enumeration literal. Certain kinds of program units can be separately compiled. Alternatively, they can appear physically nested within other program units.

The text of a program can be submitted to the compiler in one or more compilations. Each compilation is a succession of *compilation_units*. A *compilation_unit* contains either the declaration, the body, or a renaming of a program unit. The representation for a compilation is implementation-defined.

A library unit is a separately compiled program unit, and is always a package, subprogram, or generic unit. Library units may have other (logically nested) library units as children, and may have other program units physically nested within them. A root library unit, together with its children and grandchildren and so on, form a *subsystem*.

Implementation Permissions

An implementation may impose implementation-defined restrictions on compilations that contain multiple *compilation_units*.

10.1.1 Compilation Units - Library Units

A *library_item* is a compilation unit that is the declaration, body, or renaming of a library unit. Each library unit (except Standard) has a *parent unit*, which is a library package or generic library package. A library unit is a *child* of its parent unit. The *root* library units are the children of the predefined library package Standard.

Syntax

```
compilation ::= {compilation_unit}
```

```
compilation_unit ::=  
  context_clause library_item  
  | context_clause subunit
```

```
library_item ::= [private] library_unit_declaration  
  | library_unit_body  
  | [private] library_unit_renaming_declaration
```

- 5 `library_unit_declaration ::=`
 `subprogram_declaration | package_declaration`
 `| generic_declaration | generic_instantiation`
- 6 `library_unit_renaming_declaration ::=`
 `package_renaming_declaration`
 `| generic_renaming_declaration`
 `| subprogram_renaming_declaration`
- 7 `library_unit_body ::= subprogram_body | package_body`
- 8 `parent_unit_name ::= name`
- 8.1/2 An overriding_indicator is not allowed in a subprogram_declaration, generic_instantiation, or subprogram_renaming_declaration that declares a library unit.
- 9 A *library unit* is a program unit that is declared by a `library_item`. When a program unit is a library unit, the prefix “library” is used to refer to it (or “generic library” if generic), as well as to its declaration and body, as in “library procedure”, “library package_body”, or “generic library package”. The term *compilation unit* is used to refer to a `compilation_unit`. When the meaning is clear from context, the term is also used to refer to the `library_item` of a `compilation_unit` or to the `proper_body` of a subunit (that is, the `compilation_unit` without the `context_clause` and the **separate** (`parent_unit_name`)).
- 10 The *parent declaration* of a `library_item` (and of the library unit) is the declaration denoted by the `parent_unit_name`, if any, of the `defining_program_unit_name` of the `library_item`. If there is no `parent_unit_name`, the parent declaration is the declaration of Standard, the `library_item` is a *root* `library_item`, and the library unit (renaming) is a *root* library unit (renaming). The declaration and body of Standard itself have no parent declaration. The *parent unit* of a `library_item` or library unit is the library unit declared by its parent declaration.
- 11 The children of a library unit occur immediately within the declarative region of the declaration of the library unit. The *ancestors* of a library unit are itself, its parent, its parent's parent, and so on. (Standard is an ancestor of every library unit.) The *descendant* relation is the inverse of the ancestor relation.
- 12 A `library_unit_declaration` or a `library_unit_renaming_declaration` is *private* if the declaration is immediately preceded by the reserved word **private**; it is otherwise *public*. A library unit is private or public according to its declaration. The *public descendants* of a library unit are the library unit itself, and the public descendants of its public children. Its other descendants are *private descendants*.
- 12.1/2 For each library `package_declaration` in the environment, there is an implicit declaration of a *limited view* of that library package. The limited view of a package contains:
- 12.2/3 • For each nested `package_declaration` directly in the visible part, a declaration of the limited view of that package, with the same `defining_program_unit_name`.
- 12.3/3 • For each `type_declaration` directly in the visible part that is not an `incomplete_type_declaration`, an incomplete view of the type with no `discriminant_part`; if the `type_declaration` is tagged, then the view is a tagged incomplete view.
- 12.4/2 The limited view of a library `package_declaration` is private if that library `package_declaration` is immediately preceded by the reserved word **private**.
- 12.5/2 There is no syntax for declaring limited views of packages, because they are always implicit. The implicit declaration of a limited view of a library package is not the declaration of a library unit (the `library_package_declaration` is); nonetheless, it is a `library_item`. The implicit declaration of the limited view of a library package forms an (implicit) compilation unit whose `context_clause` is empty.
- 12.6/2 A library `package_declaration` is the completion of the declaration of its limited view.

Legality Rules

- The parent unit of a `library_item` shall be a library package or generic library package. 13
- If a `defining_program_unit_name` of a given declaration or body has a `parent_unit_name`, then the given declaration or body shall be a `library_item`. The body of a program unit shall be a `library_item` if and only if the declaration of the program unit is a `library_item`. In a `library_unit_renaming_declaration`, the (old) name shall denote a `library_item`. 14
- A `parent_unit_name` (which can be used within a `defining_program_unit_name` of a `library_item` and in the **separate** clause of a subunit), and each of its prefixes, shall not denote a `renaming_declaration`. On the other hand, a name that denotes a `library_unit_renaming_declaration` is allowed in a `nonlimited_with_clause` and other places where the name of a library unit is allowed. 15/2
- If a library package is an instance of a generic package, then every child of the library package shall either be itself an instance or be a renaming of a library unit. 16
- A child of a generic library package shall either be itself a generic unit or be a renaming of some other child of the same generic unit. 17/3
- A child of a parent generic package shall be instantiated or renamed only within the declarative region of the parent generic. 18
- For each child *C* of some parent generic package *P*, there is a corresponding declaration *C* nested immediately within each instance of *P*. For the purposes of this rule, if a child *C* itself has a child *D*, each corresponding declaration for *C* has a corresponding child *D*. The corresponding declaration for a child within an instance is visible only within the scope of a `with_clause` that mentions the (original) child generic unit. 19/2
- A library subprogram shall not override a primitive subprogram. 20
- The defining name of a function that is a compilation unit shall not be an `operator_symbol`. 21

Static Semantics

- A `subprogram_renaming_declaration` that is a `library_unit_renaming_declaration` is a renaming-as-declaration, not a renaming-as-body. 22
- There are two kinds of dependences among compilation units: 23
- The *semantic dependences* (see below) are the ones needed to check the compile-time rules across compilation unit boundaries; a compilation unit depends semantically on the other compilation units needed to determine its legality. The visibility rules are based on the semantic dependences. 24
 - The *elaboration dependences* (see 10.2) determine the order of elaboration of `library_items`. 25
- A `library_item` depends semantically upon its parent declaration. A subunit depends semantically upon its parent body. A `library_unit_body` depends semantically upon the corresponding `library_unit_declaration`, if any. The declaration of the limited view of a library package depends semantically upon the declaration of the limited view of its parent. The declaration of a library package depends semantically upon the declaration of its limited view. A compilation unit depends semantically upon each `library_item` mentioned in a `with_clause` of the compilation unit. In addition, if a given compilation unit contains an `attribute_reference` of a type defined in another compilation unit, then the given compilation unit depends semantically upon the other compilation unit. The semantic dependence relationship is transitive. 26/2

Dynamic Semantics

26.1/2 The elaboration of the declaration of the limited view of a package has no effect.

NOTES

- 27 1 A simple program may consist of a single compilation unit. A compilation need not have any compilation units; for example, its text can consist of **pragmas**.
- 28 2 The designator of a library function cannot be an **operator_symbol**, but a nonlibrary **renaming_declaration** is allowed to rename a library function as an operator. Within a partition, two library subprograms are required to have distinct names and hence cannot overload each other. However, **renaming_declarations** are allowed to define overloaded names for such subprograms, and a locally declared subprogram is allowed to overload a library subprogram. The expanded name **Standard.L** can be used to denote a root library unit **L** (unless the declaration of **Standard** is hidden) since root library unit declarations occur immediately within the declarative region of package **Standard**.

Examples

29 *Examples of library units:*

```

30  package Rational_Numbers.IO is -- public child of Rational_Numbers, see 7.1
      procedure Put(R : in Rational);
      procedure Get(R : out Rational);
    end Rational_Numbers.IO;

31  private procedure Rational_Numbers.Reduce(R : in out Rational);
      -- private child of Rational_Numbers

32  with Rational_Numbers.Reduce; -- refer to a private child
    package body Rational_Numbers is
      ...
    end Rational_Numbers;

33  with Rational_Numbers.IO; use Rational_Numbers;
    with Ada.Text_IO; -- see A.10
    procedure Main is -- a root library procedure
      R : Rational;
    begin
      R := 5/3; -- construct a rational number, see 7.1
      Ada.Text_IO.Put("The answer is: ");
      IO.Put(R);
      Ada.Text_IO.New_Line;
    end Main;

34  with Rational_Numbers.IO;
    package Rational_IO renames Rational_Numbers.IO;
      -- a library unit renaming declaration

```

35 Each of the above library_items can be submitted to the compiler separately.

10.1.2 Context Clauses - With Clauses

1 A context_clause is used to specify the library_items whose names are needed within a compilation unit.

Syntax

```

2  context_clause ::= {context_item}
3  context_item ::= with_clause | use_clause
4/2  with_clause ::= limited_with_clause | nonlimited_with_clause
4.1/2  limited_with_clause ::= limited [private] with library_unit_name {, library_unit_name};
4.2/2  nonlimited_with_clause ::= [private] with library_unit_name {, library_unit_name};

```

Name Resolution Rules

The *scope* of a `with_clause` that appears on a `library_unit_declaration` or `library_unit_renaming_declaration` consists of the entire declarative region of the declaration, which includes all children and subunits. The scope of a `with_clause` that appears on a body consists of the body, which includes all subunits. 5

A `library_item` (and the corresponding library unit) is *named* in a `with_clause` if it is denoted by a `library_unit_name` in the `with_clause`. A `library_item` (and the corresponding library unit) is *mentioned* in a `with_clause` if it is named in the `with_clause` or if it is denoted by a prefix in the `with_clause`. 6/2

Outside its own declarative region, the declaration or renaming of a library unit can be visible only within the scope of a `with_clause` that mentions it. The visibility of the declaration or renaming of a library unit otherwise follows from its placement in the environment. 7

Legality Rules

If a `with_clause` of a given `compilation_unit` mentions a private child of some library unit, then the given `compilation_unit` shall be one of: 8/2

- the declaration, body, or subunit of a private descendant of that library unit; 9/2
- the body or subunit of a public descendant of that library unit, but not a subprogram body acting as a subprogram declaration (see 10.1.4); or 10/2
- the declaration of a public descendant of that library unit, in which case the `with_clause` shall include the reserved word **private**. 11/2

A name denoting a `library_item` that is visible only due to being mentioned in one or more `with_clauses` that include the reserved word **private** shall appear only within: 12/3

- a private part; 13/2
- a body, but not within the `subprogram_specification` of a library subprogram body; 14/2
- a private descendant of the unit on which one of these `with_clauses` appear; or 15/2
- a pragma within a context clause. 16/2

A `library_item` mentioned in a `limited_with_clause` shall be the implicit declaration of the limited view of a library package, not the declaration of a subprogram, generic unit, generic instance, or a renaming. 17/2

A `limited_with_clause` shall not appear on a `library_unit_body`, subunit, or `library_unit_renaming_declaration`. 18/2

A `limited_with_clause` that names a library package shall not appear: 19/2

- in the `context_clause` for the explicit declaration of the named library package or any of its descendants; 20/3
- within a `context_clause` for a `library_item` which is within the scope of a `nonlimited_with_clause` that mentions the same library package; or 21/3
- within a `context_clause` for a `library_item` which is within the scope of a `use_clause` that names an entity declared within the declarative region of the library package. 22/3

NOTES

3 A `library_item` mentioned in a `nonlimited_with_clause` of a compilation unit is visible within the compilation unit and hence acts just like an ordinary declaration. Thus, within a compilation unit that mentions its declaration, the name of a library package can be given in `use_clauses` and can be used to form expanded names, a library subprogram can be called, and instances of a generic library unit can be declared. If a child of a parent generic package is mentioned in a `nonlimited_with_clause`, then the corresponding declaration nested within each visible instance is visible within the 23/2

compilation unit. Similarly, a `library_item` mentioned in a `limited_with_clause` of a compilation unit is visible within the compilation unit and thus can be used to form expanded names.

Examples

```

24/2  package Office is
      end Office;

25/2  with Ada.Strings.Unbounded;
      package Office.Locations is
          type Location is new Ada.Strings.Unbounded.Unbounded_String;
      end Office.Locations;

26/2  limited with Office.Departments; -- types are incomplete
      private with Office.Locations; -- only visible in private part
      package Office.Employees is
          type Employee is private;

27/2      function Dept_Of(Emp : Employee) return access Departments.Department;
          procedure Assign_Dept(Emp : in out Employee;
                                Dept : access Departments.Department);

28/2  ...
      private
          type Employee is
              record
                  Dept : access Departments.Department;
                  Loc : Locations.Location;
                  ...
              end record;
      end Office.Employees;

29/2  limited with Office.Employees;
      package Office.Departments is
          type Department is private;

30/2      function Manager_Of(Dept : Department) return access Employees.Employee;
          procedure Assign_Manager(Dept : in out Department;
                                   Mgr : access Employees.Employee);
      ...
      end Office.Departments;

```

31/2 The `limited_with_clause` may be used to support mutually dependent abstractions that are split across multiple packages. In this case, an employee is assigned to a department, and a department has a manager who is an employee. If a `with_clause` with the reserved word **private** appears on one library unit and mentions a second library unit, it provides visibility to the second library unit, but restricts that visibility to the private part and body of the first unit. The compiler checks that no use is made of the second unit in the visible part of the first unit.

10.1.3 Subunits of Compilation Units

1 Subunits are like child units, with these (important) differences: subunits support the separate compilation of bodies only (not declarations); the parent contains a `body_stub` to indicate the existence and place of each of its subunits; declarations appearing in the parent's body can be visible within the subunits.

Syntax

```

2  body_stub ::=
   subprogram_body_stub | package_body_stub | task_body_stub | protected_body_stub

3/2 subprogram_body_stub ::=
   [overriding_indicator]
   subprogram_specification is separate;

4  package_body_stub ::= package body defining_identifier is separate;

```

<code>task_body_stub ::= task body defining_identifier is separate;</code>	5
<code>protected_body_stub ::= protected body defining_identifier is separate;</code>	6
<code>subunit ::= separate (parent_unit_name) proper_body</code>	7

Legality Rules

The *parent body* of a subunit is the body of the program unit denoted by its `parent_unit_name`. The term *subunit* is used to refer to a subunit and also to the `proper_body` of a subunit. The *subunits of a program unit* include any subunit that names that program unit as its parent, as well as any subunit that names such a subunit as its parent (recursively).

The parent body of a subunit shall be present in the current environment, and shall contain a corresponding `body_stub` with the same `defining_identifier` as the subunit.

A `package_body_stub` shall be the completion of a `package_declaration` or `generic_package_declaration`; a `task_body_stub` shall be the completion of a task declaration; a `protected_body_stub` shall be the completion of a protected declaration.

In contrast, a `subprogram_body_stub` need not be the completion of a previous declaration, in which case the `_stub` declares the subprogram. If the `_stub` is a completion, it shall be the completion of a `subprogram_declaration` or `generic_subprogram_declaration`. The profile of a `subprogram_body_stub` that completes a declaration shall conform fully to that of the declaration.

A subunit that corresponds to a `body_stub` shall be of the same kind (`package_`, `subprogram_`, `task_`, or `protected_`) as the `body_stub`. The profile of a `subprogram_body` subunit shall be fully conformant to that of the corresponding `body_stub`.

A `body_stub` shall appear immediately within the `declarative_part` of a compilation unit body. This rule does not apply within an instance of a generic unit.

The `defining_identifiers` of all `body_stubs` that appear immediately within a particular `declarative_part` shall be distinct.

Post-Compilation Rules

For each `body_stub`, there shall be a subunit containing the corresponding `proper_body`.

NOTES

4 The rules in 10.1.4, “The Compilation Process” say that a `body_stub` is equivalent to the corresponding `proper_body`. This implies:

- Visibility within a subunit is the visibility that would be obtained at the place of the corresponding `body_stub` (within the parent body) if the `context_clause` of the subunit were appended to that of the parent body.
- The effect of the elaboration of a `body_stub` is to elaborate the subunit.

Examples

The package Parent is first written without subunits:

```
package Parent is
    procedure Inner;
end Parent;
```

```

21  with Ada.Text_IO;
    package body Parent is
        Variable : String := "Hello, there.";
        procedure Inner is
            begin
                Ada.Text_IO.Put_Line(Variable);
            end Inner;
    end Parent;

```

22 The body of procedure Inner may be turned into a subunit by rewriting the package body as follows (with the declaration of Parent remaining the same):

```

23  package body Parent is
        Variable : String := "Hello, there.";
        procedure Inner is separate;
    end Parent;

24  with Ada.Text_IO;
    separate(Parent)
    procedure Inner is
        begin
            Ada.Text_IO.Put_Line(Variable);
        end Inner;

```

10.1.4 The Compilation Process

- 1 Each compilation unit submitted to the compiler is compiled in the context of an *environment declarative_part* (or simply, an *environment*), which is a conceptual *declarative_part* that forms the outermost declarative region of the context of any compilation. At run time, an environment forms the *declarative_part* of the body of the environment task of a partition (see 10.2, “Program Execution”).
- 2 The *declarative_items* of the environment are *library_items* appearing in an order such that there are no forward semantic dependences. Each included subunit occurs in place of the corresponding stub. The visibility rules apply as if the environment were the outermost declarative region, except that *with_clauses* are needed to make declarations of library units visible (see 10.1.2).
- 3/2 The mechanisms for creating an environment and for adding and replacing compilation units within an environment are implementation defined. The mechanisms for adding a compilation unit mentioned in a *limited_with_clause* to an environment are implementation defined.

Name Resolution Rules

- 4/1 If a *library_unit_body* that is a *subprogram_body* is submitted to the compiler, it is interpreted only as a completion if a *library_unit_declaration* with the same *defining_program_unit_name* already exists in the environment for a subprogram other than an instance of a generic subprogram or for a generic subprogram (even if the profile of the body is not type conformant with that of the declaration); otherwise the *subprogram_body* is interpreted as both the declaration and body of a library subprogram.

Legality Rules

- 5 When a compilation unit is compiled, all compilation units upon which it depends semantically shall already exist in the environment; the set of these compilation units shall be *consistent* in the sense that the new compilation unit shall not semantically depend (directly or indirectly) on two different versions of the same compilation unit, nor on an earlier version of itself.

Implementation Permissions

- 6/2 The implementation may require that a compilation unit be legal before it can be mentioned in a *limited_with_clause* or it can be inserted into the environment.

When a compilation unit that declares or renames a library unit is added to the environment, the implementation may remove from the environment any preexisting `library_item` or subunit with the same full expanded name. When a compilation unit that is a subunit or the body of a library unit is added to the environment, the implementation may remove from the environment any preexisting version of the same compilation unit. When a compilation unit that contains a `body_stub` is added to the environment, the implementation may remove any preexisting `library_item` or subunit with the same full expanded name as the `body_stub`. When a given compilation unit is removed from the environment, the implementation may also remove any compilation unit that depends semantically upon the given one. If the given compilation unit contains the body of a subprogram to which a `pragma Inline` applies, the implementation may also remove any compilation unit containing a call to that subprogram. 7/2

NOTES

- 5 The rules of the language are enforced across compilation and compilation unit boundaries, just as they are enforced within a single compilation unit. 8
- 6 An implementation may support a concept of a *library*, which contains `library_items`. If multiple libraries are supported, the implementation has to define how a single environment is constructed when a compilation unit is submitted to the compiler. Naming conflicts between different libraries might be resolved by treating each library as the root of a hierarchy of child library units. 9
- 7 A compilation unit containing an instantiation of a separately compiled generic unit does not semantically depend on the body of the generic unit. Therefore, replacing the generic body in the environment does not result in the removal of the compilation unit containing the instantiation. 10

10.1.5 Pragmas and Program Units

This subclause discusses pragmas related to program units, library units, and compilations. 1

Name Resolution Rules

Certain pragmas are defined to be *program unit pragmas*. A name given as the argument of a program unit pragma shall resolve to denote the declarations or renamings of one or more program units that occur immediately within the declarative region or compilation in which the pragma immediately occurs, or it shall resolve to denote the declaration of the immediately enclosing program unit (if any); the pragma applies to the denoted program unit(s). If there are no names given as arguments, the pragma applies to the immediately enclosing program unit. 2

Legality Rules

A program unit pragma shall appear in one of these places: 3

- At the place of a `compilation_unit`, in which case the pragma shall immediately follow in the same compilation (except for other pragmas) a `library_unit_declaration` that is a `subprogram_declaration`, `generic_subprogram_declaration`, or `generic_instantiation`, and the pragma shall have an argument that is a name denoting that declaration. 4
- Immediately within the visible part of a program unit and before any nested declaration (but not within a generic formal part), in which case the argument, if any, shall be a `direct_name` that denotes the immediately enclosing program unit declaration. 5/1
- At the place of a declaration other than the first, of a `declarative_part` or program unit declaration, in which case the pragma shall have an argument, which shall be a `direct_name` that denotes one or more of the following (and nothing else): a `subprogram_declaration`, a `generic_subprogram_declaration`, or a `generic_instantiation`, of the same `declarative_part` or program unit declaration. 6

- 7/3 Certain program unit pragmas are defined to be *library unit pragmas*. If a library unit pragma applies to a program unit, the program unit shall be a library unit.

Static Semantics

- 7.1/1 A library unit pragma that applies to a generic unit does not apply to its instances, unless a specific rule for the pragma specifies the contrary.

Post-Compilation Rules

- 8 Certain pragmas are defined to be *configuration pragmas*; they shall appear before the first `compilation_unit` of a compilation. They are generally used to select a partition-wide or system-wide option. The pragma applies to all `compilation_units` appearing in the compilation, unless there are none, in which case it applies to all future `compilation_units` compiled into the same environment.

Implementation Permissions

- 9/2 An implementation may require that configuration pragmas that select partition-wide or system-wide options be compiled when the environment contains no `library_items` other than those of the predefined environment. In this case, the implementation shall still accept configuration pragmas in individual compilations that confirm the initially selected partition-wide or system-wide options.

Implementation Advice

- 10/1 When applied to a generic unit, a program unit pragma that is not a library unit pragma should apply to each instance of the generic unit for which there is not an overriding pragma applied directly to the instance.

10.1.6 Environment-Level Visibility Rules

- 1 The normal visibility rules do not apply within a `parent_unit_name` or a `context_clause`, nor within a pragma that appears at the place of a compilation unit. The special visibility rules for those contexts are given here.

Static Semantics

- 2/2 Within the `parent_unit_name` at the beginning of an explicit `library_item`, and within a `nonlimited_with_clause`, the only declarations that are visible are those that are explicit `library_items` of the environment, and the only declarations that are directly visible are those that are explicit root `library_items` of the environment. Within a `limited_with_clause`, the only declarations that are visible are those that are the implicit declaration of the limited view of a library package of the environment, and the only declarations that are directly visible are those that are the implicit declaration of the limited view of a root library package.
- 3 Within a `use_clause` or pragma that is within a `context_clause`, each `library_item` mentioned in a previous `with_clause` of the same `context_clause` is visible, and each root `library_item` so mentioned is directly visible. In addition, within such a `use_clause`, if a given declaration is visible or directly visible, each declaration that occurs immediately within the given declaration's visible part is also visible. No other declarations are visible or directly visible.
- 4 Within the `parent_unit_name` of a subunit, `library_items` are visible as they are in the `parent_unit_name` of a `library_item`; in addition, the declaration corresponding to each `body_stub` in the environment is also visible.

Within a **pragma** that appears at the place of a compilation unit, the immediately preceding **library_item** and each of its ancestors is visible. The ancestor root **library_item** is directly visible. 5

Notwithstanding the rules of 4.1.3, an expanded name in a **with_clause**, a **pragma** in a **context_clause**, or a **pragma** that appears at the place of a compilation unit may consist of a **prefix** that denotes a generic package and a **selector_name** that denotes a child of that generic package. (The child is necessarily a generic unit; see 10.1.1.) 6/2

10.2 Program Execution

An Ada *program* consists of a set of *partitions*, which can execute in parallel with one another, possibly in a separate address space, and possibly on a separate computer. 1

Post-Compilation Rules

A partition is a program or part of a program that can be invoked from outside the Ada implementation. For example, on many systems, a partition might be an executable file generated by the system linker. The user can *explicitly assign* library units to a partition. The assignment is done in an implementation-defined manner. The compilation units included in a partition are those of the explicitly assigned library units, as well as other compilation units *needed by* those library units. The compilation units needed by a given compilation unit are determined as follows (unless specified otherwise via an implementation-defined **pragma**, or by some other implementation-defined means): 2

- A compilation unit needs itself; 3
- If a compilation unit is needed, then so are any compilation units upon which it depends semantically; 4
- If a **library_unit_declaration** is needed, then so is any corresponding **library_unit_body**; 5
- If a compilation unit with stubs is needed, then so are any corresponding subunits; 6/2
- If the (implicit) declaration of the limited view of a library package is needed, then so is the explicit declaration of the library package. 6.1/2

The user can optionally designate (in an implementation-defined manner) one subprogram as the *main subprogram* for the partition. A main subprogram, if specified, shall be a subprogram. 7

Each partition has an anonymous *environment task*, which is an implicit outermost task whose execution elaborates the **library_items** of the environment **declarative_part**, and then calls the main subprogram, if there is one. A partition's execution is that of its tasks. 8

The order of elaboration of library units is determined primarily by the *elaboration dependences*. There is an elaboration dependence of a given **library_item** upon another if the given **library_item** or any of its subunits depends semantically on the other **library_item**. In addition, if a given **library_item** or any of its subunits has a **pragma Elaborate** or **Elaborate_All** that names another library unit, then there is an elaboration dependence of the given **library_item** upon the body of the other library unit, and, for **Elaborate_All** only, upon each **library_item** needed by the declaration of the other library unit. 9

The environment task for a partition has the following structure: 10

```
task Environment_Task; 11
```

```

12/2  task body Environment_Task is
      ... (1) -- The environment declarative_part
           -- (that is, the sequence of library_items) goes here.
      begin
      ... (2) -- Call the main subprogram, if there is one.
      end Environment_Task;

```

13 The environment `declarative_part` at (1) is a sequence of `declarative_items` consisting of copies of the `library_items` included in the partition. The order of elaboration of `library_items` is the order in which they appear in the environment `declarative_part`:

- 14 • The order of all included `library_items` is such that there are no forward elaboration dependencies.
- 15 • Any included `library_unit_declaration` to which a `pragma Elaborate_Body` applies is immediately followed by its `library_unit_body`, if included.
- 16 • All `library_items` declared pure occur before any that are not declared pure.
- 17 • All preelaborated `library_items` occur before any that are not preelaborated.

18 There shall be a total order of the `library_items` that obeys the above rules. The order is otherwise implementation defined.

19 The full expanded names of the library units and subunits included in a given partition shall be distinct.

20 The `sequence_of_statements` of the environment task (see (2) above) consists of either:

- 21 • A call to the main subprogram, if the partition has one. If the main subprogram has parameters, they are passed; where the actuals come from is implementation defined. What happens to the result of a main function is also implementation defined.

22 or:

- 23 • A `null_statement`, if there is no main subprogram.

24 The mechanisms for building and running partitions are implementation defined. These might be combined into one operation, as, for example, in dynamic linking, or “load-and-go” systems.

Dynamic Semantics

25 The execution of a program consists of the execution of a set of partitions. Further details are implementation defined. The execution of a partition starts with the execution of its environment task, ends when the environment task terminates, and includes the executions of all tasks of the partition. The execution of the (implicit) `task_body` of the environment task acts as a master for all other tasks created as part of the execution of the partition. When the environment task completes (normally or abnormally), it waits for the termination of all such tasks, and then finalizes any remaining objects of the partition.

Bounded (Run-Time) Errors

26 Once the environment task has awaited the termination of all other tasks of the partition, any further attempt to create a task (during finalization) is a bounded error, and may result in the raising of `Program_Error` either upon creation or activation of the task. If such a task is activated, it is not specified whether the task is awaited prior to termination of the environment task.

Implementation Requirements

27 The implementation shall ensure that all compilation units included in a partition are consistent with one another, and are legal according to the rules of the language.

Implementation Permissions

The kind of partition described in this clause is known as an *active* partition. An implementation is allowed to support other kinds of partitions, with implementation-defined semantics. 28

An implementation may restrict the kinds of subprograms it supports as main subprograms. However, an implementation is required to support all main subprograms that are public parameterless library procedures. 29

If the environment task completes abnormally, the implementation may abort any dependent tasks. 30

NOTES

8 An implementation may provide inter-partition communication mechanism(s) via special packages and pragmas. Standard pragmas for distribution and methods for specifying inter-partition communication are defined in Annex E, “Distributed Systems”. If no such mechanisms are provided, then each partition is isolated from all others, and behaves as a program in and of itself. 31

9 Partitions are not required to run in separate address spaces. For example, an implementation might support dynamic linking via the partition concept. 32

10 An order of elaboration of `library_items` that is consistent with the partial ordering defined above does not always ensure that each `library_unit_body` is elaborated before any other compilation unit whose elaboration necessitates that the `library_unit_body` be already elaborated. (In particular, there is no requirement that the body of a library unit be elaborated as soon as possible after the `library_unit_declaration` is elaborated, unless the pragmas in subclause 10.2.1 are used.) 33

11 A partition (active or otherwise) need not have a main subprogram. In such a case, all the work done by the partition would be done by elaboration of various `library_items`, and by tasks created by that elaboration. Passive partitions, which cannot have main subprograms, are defined in Annex E, “Distributed Systems”. 34

10.2.1 Elaboration Control

This subclause defines pragmas that help control the elaboration order of `library_items`. 1

Syntax

The form of a `pragma Preelaborate` is as follows: 2

pragma Preelaborate[(*library_unit_name*)]; 3

A `pragma Preelaborate` is a library unit pragma. 4

The form of a `pragma Preelaborable_Initialization` is as follows: 4.1/2

pragma Preelaborable_Initialization(*direct_name*); 4.2/2

Legality Rules

An elaborable construct is preelaborable unless its elaboration performs any of the following actions: 5

- The execution of a **statement** other than a **null_statement**. 6
- A call to a subprogram other than a static function. 7
- The evaluation of a **primary** that is a **name** of an object, unless the **name** is a static expression, or statically denotes a discriminant of an enclosing type. 8
- The creation of an object (including a component) that is initialized by default, if its type does not have preelaborable initialization. Similarly, the evaluation of an **extension_aggregate** with an ancestor **subtype_mark** denoting a subtype of such a type. 9/3

A generic body is preelaborable only if elaboration of a corresponding instance body would not perform any such actions, presuming that: 10/2

- the actual for each discriminated formal derived type, formal private type, or formal private extension declared within the formal part of the generic unit is a private type (or extension) that 10.1/3

does not have preelaborable initialization, unless `pragma Preelaborable_Initialization` has been applied to the formal type;

- 10.2/2 • the actual for each formal type is nonstatic;
 - 10.3/2 • the actual for each formal object is nonstatic; and
 - 10.4/2 • the actual for each formal subprogram is a user-defined subprogram.
- 11/3 If a `pragma Preelaborate` (or `pragma Pure` — see below) applies to a library unit, then it is *preelaborated*. The declaration and body of a preelaborated library unit, and all subunits that are elaborated as part of elaborating the library unit, shall be preelaborable. In addition, the limited view of a library package is preelaborated. All compilation units of a preelaborated library unit shall depend semantically only on compilation units of other preelaborated library units. In addition to the places where Legality Rules normally apply (see 12.3), these rules also apply in the private part of an instance of a generic unit. If a library unit is preelaborated, then its declaration, if any, and body, if any, are elaborated prior to all non-preelaborated `library_items` of the partition.
- 11.1/2 The following rules specify which entities have *preelaborable initialization*:
- 11.2/3 • The partial view of a private type or private extension, a protected type without `entry_declarations`, a generic formal private type, or a generic formal derived type, has preelaborable initialization if and only if the `pragma Preelaborable_Initialization` has been applied to them. A protected type with `entry_declarations` or a task type never has preelaborable initialization.
 - 11.3/2 • A component (including a discriminant) of a record or protected type has preelaborable initialization if its declaration includes a `default_expression` whose execution does not perform any actions prohibited in preelaborable constructs as described above, or if its declaration does not include a default expression and its type has preelaborable initialization.
 - 11.4/3 • A derived type has preelaborable initialization if its parent type has preelaborable initialization and (in the case of a derived record extension) if the non-inherited components all have preelaborable initialization. However, a controlled type with an `Initialize` procedure that is not a null procedure does not have preelaborable initialization.
 - 11.5/2 • A view of a type has preelaborable initialization if it is an elementary type, an array type whose component type has preelaborable initialization, a record type whose components all have preelaborable initialization, or an interface type.
- 11.6/2 A `pragma Preelaborable_Initialization` specifies that a type has preelaborable initialization. This `pragma` shall appear in the visible part of a package or generic package.
- 11.7/3 If the `pragma` appears in the first list of `basic_declarative_items` of a `package_specification`, then the `direct_name` shall denote the first subtype of a composite type, and the type shall be declared immediately within the same package as the `pragma`. If the `pragma` is applied to a private type or a private extension, the full view of the type shall have preelaborable initialization. If the `pragma` is applied to a protected type, the protected type shall not have entries, and each component of the protected type shall have preelaborable initialization. For any other composite type, the type shall have preelaborable initialization. In addition to the places where Legality Rules normally apply (see 12.3), these rules apply also in the private part of an instance of a generic unit.
- 11.8/2 If the `pragma` appears in a `generic_formal_part`, then the `direct_name` shall denote a generic formal private type or a generic formal derived type declared in the same `generic_formal_part` as the `pragma`. In a `generic_instantiation` the corresponding actual type shall have preelaborable initialization.

Implementation Advice

In an implementation, a type declared in a preelaborated package should have the same representation in every elaboration of a given version of the package, whether the elaborations occur in distinct executions of the same program, or in executions of distinct programs or partitions that include the given version. 12

Syntax

The form of a **pragma Pure** is as follows: 13

pragma Pure[(*library_unit_name*)]; 14

A **pragma Pure** is a library unit pragma. 15

Static Semantics

A *pure* compilation unit is a preelaborable compilation unit whose elaboration does not perform any of the following actions: 15.1/3

- the elaboration of a variable declaration; 15.2/2
- the evaluation of an **allocator** of an access-to-variable type; for the purposes of this rule, the partial view of a type is presumed to have non-visible components whose default initialization evaluates such an **allocator**; 15.3/2
- the elaboration of the declaration of a non-derived named access-to-variable type unless the *Storage_Size* of the type has been specified by a static expression with value zero or is defined by the language to be zero; 15.4/3
- the elaboration of the declaration of a non-derived named access-to-constant type for which the *Storage_Size* has been specified by an expression other than a static expression with value zero. 15.5/3

A generic body is pure only if elaboration of a corresponding instance body would not perform any such actions presuming any composite formal types have non-visible components whose default initialization evaluates an **allocator** of an access-to-variable type. 15.6/3

The *Storage_Size* for an anonymous access-to-variable type declared at library level in a library unit that is declared pure is defined to be zero. 15.7/2

Legality Rules

This paragraph was deleted. 16/2

A **pragma Pure** is used to declare that a library unit is pure. The declaration and body of a declared pure library unit, and all subunits that are elaborated as part of elaborating the library unit, shall be pure. In addition, the limited view of a library package is declared pure. All compilation units of a declared pure library unit shall depend semantically only on compilation units of other library units that are declared pure. In addition to the places where Legality Rules normally apply (see 12.3), this rule also applies in the private part of an instance of a generic unit. Furthermore, the full view of any partial view declared in the visible part of the library unit that has any available stream attributes shall support external streaming (see 13.13.2). 17/3

Implementation Permissions

If a library unit is declared pure, then the implementation is permitted to omit a call on a library-level subprogram of the library unit if the results are not needed after the call. In addition, the implementation may omit a call on such a subprogram and simply reuse the results produced by an earlier call on the same subprogram, provided that none of the parameters nor any object accessible via access values from the parameters are of a limited type, and the addresses and values of all by-reference actual parameters, the values of all by-copy-in actual parameters, and the values of all objects accessible via access values from 18/2

the parameters, are the same as they were at the earlier call. This permission applies even if the subprogram produces other side effects when called.

Syntax

19 The form of a **pragma** Elaborate, Elaborate_All, or Elaborate_Body is as follows:

20 **pragma** Elaborate(*library_unit_name*{, *library_unit_name*});

21 **pragma** Elaborate_All(*library_unit_name*{, *library_unit_name*});

22 **pragma** Elaborate_Body[(*library_unit_name*)];

23 A **pragma** Elaborate or Elaborate_All is only allowed within a *context_clause*.

24 A **pragma** Elaborate_Body is a library unit pragma.

Legality Rules

25 If a **pragma** Elaborate_Body applies to a declaration, then the declaration requires a completion (a body).

25.1/2 The *library_unit_name* of a **pragma** Elaborate or Elaborate_All shall denote a nonlimited view of a library unit.

Static Semantics

26 A **pragma** Elaborate specifies that the body of the named library unit is elaborated before the current *library_item*. A **pragma** Elaborate_All specifies that each *library_item* that is needed by the named library unit declaration is elaborated before the current *library_item*. A **pragma** Elaborate_Body specifies that the body of the library unit is elaborated immediately after its declaration.

NOTES

27 12 A preelaborated library unit is allowed to have non-preelaborable children.

28 13 A library unit that is declared pure is allowed to have impure children.

Section 11: Exceptions

This section defines the facilities for dealing with errors or other exceptional situations that arise during program execution. An *exception* represents a kind of exceptional situation; an occurrence of such a situation (at run time) is called an *exception occurrence*. To *raise* an exception is to abandon normal program execution so as to draw attention to the fact that the corresponding situation has arisen. Performing some actions in response to the arising of an exception is called *handling* the exception. 1

An `exception_declaration` declares a name for an exception. An exception is raised initially by a `raise_statement`, by a call to `Raise_Exception`, or by the failure of a language-defined check. When an exception arises, control can be transferred to a user-provided `exception_handler` at the end of a `handled_sequence_of_statements`, or it can be propagated to a dynamically enclosing execution. 2/3

11.1 Exception Declarations

An `exception_declaration` declares a name for an exception. 1

Syntax

```
exception_declaration ::= defining_identifier_list : exception;
```

 2

Static Semantics

Each single `exception_declaration` declares a name for a different exception. If a generic unit includes an `exception_declaration`, the `exception_declarations` implicitly generated by different instantiations of the generic unit refer to distinct exceptions (but all have the same `defining_identifier`). The particular exception denoted by an exception name is determined at compilation time and is the same regardless of how many times the `exception_declaration` is elaborated. 3

The *predefined* exceptions are the ones declared in the declaration of package `Standard`: `Constraint_Error`, `Program_Error`, `Storage_Error`, and `Tasking_Error`; one of them is raised when a language-defined check fails. 4

Dynamic Semantics

The elaboration of an `exception_declaration` has no effect. 5

The execution of any construct raises `Storage_Error` if there is insufficient storage for that execution. The amount of storage needed for the execution of constructs is unspecified. 6

Examples

Examples of user-defined exception declarations: 7

```
Singular : exception;  
Error    : exception;  
Overflow, Underflow : exception;
```

 8

11.2 Exception Handlers

The response to one or more exceptions is specified by an `exception_handler`. 1

Syntax

```

2      handled_sequence_of_statements ::=
        sequence_of_statements
        [exception
         exception_handler
         {exception_handler}]
3      exception_handler ::=
        when [choice_parameter_specification:] exception_choice { | exception_choice } =>
          sequence_of_statements
4      choice_parameter_specification ::= defining_identifier
5      exception_choice ::= exception_name | others

```

Legality Rules

- 6 A choice with an *exception_name* covers the named exception. A choice with **others** covers all exceptions not named by previous choices of the same handled_sequence_of_statements. Two choices in different exception_handlers of the same handled_sequence_of_statements shall not cover the same exception.
- 7 A choice with **others** is allowed only for the last handler of a handled_sequence_of_statements and as the only choice of that handler.
- 8 An *exception_name* of a choice shall not denote an exception declared in a generic formal package.

Static Semantics

- 9 A choice_parameter_specification declares a *choice parameter*, which is a constant object of type Exception_Occurrence (see 11.4.1). During the handling of an exception occurrence, the choice parameter, if any, of the handler represents the exception occurrence that is being handled.

Dynamic Semantics

- 10 The execution of a handled_sequence_of_statements consists of the execution of the sequence_of_statements. The optional handlers are used to handle any exceptions that are propagated by the sequence_of_statements.

Examples

- 11 *Example of an exception handler:*

```

12      begin
        Open(File, In_File, "input.txt");    -- see A.8.2
      exception
        when E : Name_Error =>
          Put("Cannot open input file : ");
          Put_Line(Exception_Message(E));    -- see 11.4.1
          raise;
      end;

```

11.3 Raise Statements

- 1 A raise_statement raises an exception.

Syntax

```

2/2      raise_statement ::= raise;
        | raise exception_name [with string_expression];

```


Legality Rules

The name, if any, in a `raise_statement` shall denote an exception. A `raise_statement` with no *exception_name* (that is, a *re-raise statement*) shall be within a handler, but not within a body enclosed by that handler. 3

Name Resolution Rules

The expression, if any, in a `raise_statement`, is expected to be of type `String`. 3.1/2

Dynamic Semantics

To *raise an exception* is to raise a new occurrence of that exception, as explained in 11.4. For the execution of a `raise_statement` with an *exception_name*, the named exception is raised. If a *string_expression* is present, the expression is evaluated and its value is associated with the exception occurrence. For the execution of a re-raise statement, the exception occurrence that caused transfer of control to the innermost enclosing handler is raised again. 4/2

Examples

Examples of raise statements: 5

```
raise Ada.IO_Exceptions.Name_Error; -- see A.13 6/2
raise Queue_Error with "Buffer Full"; -- see 9.11
raise; -- re-raise the current exception 7
```

11.4 Exception Handling

When an exception occurrence is raised, normal program execution is abandoned and control is transferred to an applicable `exception_handler`, if any. To *handle* an exception occurrence is to respond to the exceptional event. To *propagate* an exception occurrence is to raise it again in another context; that is, to fail to respond to the exceptional event in the present context. 1

Dynamic Semantics

Within a given task, if the execution of construct *a* is defined by this International Standard to consist (in part) of the execution of construct *b*, then while *b* is executing, the execution of *a* is said to *dynamically enclose* the execution of *b*. The *innermost dynamically enclosing* execution of a given execution is the dynamically enclosing execution that started most recently. 2

When an exception occurrence is raised by the execution of a given construct, the rest of the execution of that construct is *abandoned*; that is, any portions of the execution that have not yet taken place are not performed. The construct is first completed, and then left, as explained in 7.6.1. Then: 3

- If the construct is a `task_body`, the exception does not propagate further; 4
- If the construct is the `sequence_of_statements` of a `handled_sequence_of_statements` that has a handler with a choice covering the exception, the occurrence is handled by that handler; 5
- Otherwise, the occurrence is *propagated* to the innermost dynamically enclosing execution, which means that the occurrence is raised again in that context. 6

When an occurrence is *handled* by a given handler, the `choice_parameter_specification`, if any, is first elaborated, which creates the choice parameter and initializes it to the occurrence. Then, the `sequence_of_statements` of the handler is executed; this execution replaces the abandoned portion of the execution of the `sequence_of_statements`. 7

NOTES

- 8 1 Note that exceptions raised in a `declarative_part` of a body are not handled by the handlers of the `handled_sequence_of_statements` of that body.

11.4.1 The Package Exceptions

Static Semantics

- 1 The following language-defined library package exists:

```

2/2  with Ada.Streams;
    package Ada.Exceptions is
        pragma Preelaborate(Exceptions);
        type Exception_Id is private;
        pragma Preelaborable Initialization(Exception_Id);
        Null_Id : constant Exception_Id;
        function Exception_Name(Id : Exception_Id) return String;
        function Wide_Exception_Name(Id : Exception_Id) return Wide_String;
        function Wide_Wide_Exception_Name(Id : Exception_Id)
            return Wide_Wide_String;

3/2  type Exception_Occurrence is limited private;
        pragma Preelaborable Initialization(Exception_Occurrence);
        type Exception_Occurrence_Access is access all Exception_Occurrence;
        Null_Occurrence : constant Exception_Occurrence;

4/2  procedure Raise_Exception(E : in Exception_Id;
                               Message : in String := "");
        pragma No_Return(Raise_Exception);
        function Exception_Message(X : Exception_Occurrence) return String;
        procedure Reraise_Occurrence(X : in Exception_Occurrence);

5/2  function Exception_Identity(X : Exception_Occurrence)
        return Exception_Id;
        function Exception_Name(X : Exception_Occurrence) return String;
        -- Same as Exception_Name(Exception_Identity(X)).
        function Wide_Exception_Name(X : Exception_Occurrence)
        return Wide_String;
        -- Same as Wide_Exception_Name(Exception_Identity(X)).
        function Wide_Wide_Exception_Name(X : Exception_Occurrence)
        return Wide_Wide_String;
        -- Same as Wide_Wide_Exception_Name(Exception_Identity(X)).
        function Exception_Information(X : Exception_Occurrence) return String;

6/2  procedure Save_Occurrence(Target : out Exception_Occurrence;
                               Source : in Exception_Occurrence);
        function Save_Occurrence(Source : Exception_Occurrence)
        return Exception_Occurrence_Access;

6.1/2 procedure Read_Exception_Occurrence
        (Stream : not null access Ada.Streams.Root_Stream_Type'Class;
         Item   : out Exception_Occurrence);
        procedure Write_Exception_Occurrence
        (Stream : not null access Ada.Streams.Root_Stream_Type'Class;
         Item   : in Exception_Occurrence);

6.2/2 for Exception_Occurrence'Read use Read_Exception_Occurrence;
        for Exception_Occurrence'Write use Write_Exception_Occurrence;

6.3/2 private
        ... -- not specified by the language
    end Ada.Exceptions;

```

- 7 Each distinct exception is represented by a distinct value of type `Exception_Id`. `Null_Id` does not represent any exception, and is the default initial value of type `Exception_Id`. Each occurrence of an exception is represented by a value of type `Exception_Occurrence`. `Null_Occurrence` does not represent any exception occurrence, and is the default initial value of type `Exception_Occurrence`.

For a prefix E that denotes an exception, the following attribute is defined:	8/1
E'Identity E'Identity returns the unique identity of the exception. The type of this attribute is Exception_Id.	9
Raise_Exception raises a new occurrence of the identified exception.	10/2
Exception_Message returns the message associated with the given Exception_Occurrence. For an occurrence raised by a call to Raise_Exception, the message is the Message parameter passed to Raise_Exception. For the occurrence raised by a raise_statement with an exception_name and a string_expression, the message is the string_expression. For the occurrence raised by a raise_statement with an exception_name but without a string_expression, the message is a string giving implementation-defined information about the exception occurrence. For an occurrence originally raised for some other reason, the message is an unspecified string. In all cases, Exception_Message returns a string with lower bound 1.	10.1/3
Reraise_Occurrence reraises the specified exception occurrence.	10.2/2
Exception_Identity returns the identity of the exception of the occurrence.	11
The Wide_Wide_Exception_Name functions return the full expanded name of the exception, in upper case, starting with a root library unit. For an exception declared immediately within package Standard, the defining_identifier is returned. The result is implementation defined if the exception is declared within an unnamed block_statement.	12/2
The Exception_Name functions (respectively, Wide_Exception_Name) return the same sequence of graphic characters as that defined for Wide_Wide_Exception_Name, if all the graphic characters are defined in Character (respectively, Wide_Character); otherwise, the sequence of characters is implementation defined, but no shorter than that returned by Wide_Wide_Exception_Name for the same value of the argument.	12.1/2
The string returned by the Exception_Name, Wide_Exception_Name, and Wide_Wide_Exception_Name functions has lower bound 1.	12.2/2
Exception_Information returns implementation-defined information about the exception occurrence. The returned string has lower bound 1.	13/2
Reraise_Occurrence has no effect in the case of Null_Occurrence. Raise_Exception and Exception_Name raise Constraint_Error for a Null_Id. Exception_Message, Exception_Name, and Exception_Information raise Constraint_Error for a Null_Occurrence. Exception_Identity applied to Null_Occurrence returns Null_Id.	14/2
The Save_Occurrence procedure copies the Source to the Target. The Save_Occurrence function uses an allocator of type Exception_Occurrence_Access to create a new object, copies the Source to this new object, and returns an access value designating this new object; the result may be deallocated using an instance of Unchecked_Deallocation.	15
Write_Exception_Occurrence writes a representation of an exception occurrence to a stream; Read_Exception_Occurrence reconstructs an exception occurrence from a stream (including one written in a different partition).	15.1/2

Implementation Requirements

This paragraph was deleted. 16/2

Implementation Permissions

- 17 An implementation of `Exception_Name` in a space-constrained environment may return the `defining_`-`identifier` instead of the full expanded name.
- 18 The string returned by `Exception_Message` may be truncated (to no less than 200 characters) by the `Save_Occurrence` procedure (not the function), the `Reraise_Occurrence` procedure, and the re-raise statement.

Implementation Advice

- 19 `Exception_Message` (by default) and `Exception_Information` should produce information useful for debugging. `Exception_Message` should be short (about one line), whereas `Exception_Information` can be long. `Exception_Message` should not include the `Exception_Name`. `Exception_Information` should include both the `Exception_Name` and the `Exception_Message`.

11.4.2 Pragmas Assert and Assertion_Policy

- 1/2 Pragma Assert is used to assert the truth of a Boolean expression at any point within a sequence of declarations or statements. Pragma Assertion_Policy is used to control whether such assertions are to be ignored by the implementation, checked at run-time, or handled in some implementation-defined manner.

Syntax

- 2/2 The form of a pragma Assert is as follows:
- 3/2 **pragma** Assert([Check =>] *boolean_expression* [, [Message =>] *string_expression*]);
- 4/2 A pragma Assert is allowed at the place where a `declarative_item` or a `statement` is allowed.
- 5/2 The form of a pragma Assertion_Policy is as follows:
- 6/2 **pragma** Assertion_Policy(*policy_identifier*);
- 7/2 A pragma Assertion_Policy is a configuration pragma.

Name Resolution Rules

- 8/2 The expected type for the *boolean_expression* of a pragma Assert is any boolean type. The expected type for the *string_expression* of a pragma Assert is type String.

Legality Rules

- 9/2 The *policy_identifier* of a pragma Assertion_Policy shall be either Check, Ignore, or an implementation-defined identifier.

Static Semantics

- 10/2 A pragma Assertion_Policy is a configuration pragma that specifies the assertion policy in effect for the compilation units to which it applies. Different policies may apply to different compilation units within the same partition. The default assertion policy is implementation-defined.
- 11/2 The following language-defined library package exists:
- 12/2 **package** Ada.Assertions **is**
 pragma Pure (Assertions);
 Assertion_Error : **exception**;
 procedure Assert (Check : **in** Boolean);
 procedure Assert (Check : **in** Boolean; Message : **in** String);
 15/2 **end** Ada.Assertions;

A compilation unit containing a **pragma Assert** has a semantic dependence on the Assertions library unit. 16/2

The assertion policy that applies to a generic unit also applies to all its instances. 17/2

Dynamic Semantics

An assertion policy specifies how a **pragma Assert** is interpreted by the implementation. If the assertion policy is **Ignore** at the point of a **pragma Assert**, the pragma is ignored. If the assertion policy is **Check** at the point of a **pragma Assert**, the elaboration of the pragma consists of evaluating the boolean expression, and if the result is **False**, evaluating the **Message** argument, if any, and raising the exception **Assertions.Assertion_Error**, with a message if the **Message** argument is provided. 18/2

Calling the procedure **Assertions.Assert** without a **Message** parameter is equivalent to: 19/2

```
if Check = False then
    raise Ada.Assertions.Assertion_Error;
end if;
```

20/2

Calling the procedure **Assertions.Assert** with a **Message** parameter is equivalent to: 21/2

```
if Check = False then
    raise Ada.Assertions.Assertion_Error with Message;
end if;
```

22/2

The procedures **Assertions.Assert** have these effects independently of the assertion policy in effect. 23/2

Implementation Permissions

Assertion_Error may be declared by renaming an implementation-defined exception from another package. 24/2

Implementations may define their own assertion policies. 25/2

NOTES

2 Normally, the boolean expression in a **pragma Assert** should not call functions that have significant side-effects when the result of the expression is **True**, so that the particular assertion policy in effect will not affect normal operation of the program. 26/2

11.4.3 Example of Exception Handling

Examples

Exception handling may be used to separate the detection of an error from the response to that error: 1

```
package File_System is
    type File_Handle is limited private;
    File_Not_Found : exception;
    procedure Open(F : in out File_Handle; Name : String);
    -- raises File_Not_Found if named file does not exist
    End_Of_File : exception;
    procedure Read(F : in out File_Handle; Data : out Data_Type);
    -- raises End_Of_File if the file is not open
    ...
end File_System;
```

2/2

3

4

5

```

6/2  package body File_System is
      procedure Open(F : in out File_Handle; Name : String) is
      begin
          if File_Exists(Name) then
              ...
          else
              raise File_Not_Found with "File not found: " & Name & ".";
          end if;
      end Open;

7      procedure Read(F : in out File_Handle; Data : out Data_Type) is
      begin
          if F.Current_Position <= F.Last_Position then
              ...
          else
              raise End_Of_File;
          end if;
      end Read;

8      ...
9  end File_System;
10 with Ada.Text_IO;
    with Ada.Exceptions;
    with File_System; use File_System;
    use Ada;
    procedure Main is
    begin
        ... -- call operations in File_System
    exception
        when End_Of_File =>
            Close(Some_File);
        when Not_Found_Error : File_Not_Found =>
            Text_IO.Put_Line(Exceptions.Exception_Message(Not_Found_Error));
        when The_Error : others =>
            Text_IO.Put_Line("Unknown error:");
            if Verbosity_Desired then
                Text_IO.Put_Line(Exceptions.Exception_Information(The_Error));
            else
                Text_IO.Put_Line(Exceptions.Exception_Name(The_Error));
                Text_IO.Put_Line(Exceptions.Exception_Message(The_Error));
            end if;
            raise;
    end Main;

```

- 11 In the above example, the `File_System` package contains information about detecting certain exceptional situations, but it does not specify how to handle those situations. Procedure `Main` specifies how to handle them; other clients of `File_System` might have different handlers, even though the exceptional situations arise from the same basic causes.

11.5 Suppressing Checks

- 1/2 *Checking pragmas* give instructions to an implementation on handling language-defined checks. A `pragma Suppress` gives permission to an implementation to omit certain language-defined checks, while a `pragma Unsuppress` revokes the permission to omit checks..
- 2 A *language-defined check* (or simply, a “check”) is one of the situations defined by this International Standard that requires a check to be made at run time to determine whether some condition is true. A check *fails* when the condition being checked is false, causing an exception to be raised.

Syntax

The forms of checking pragmas are as follows:

pragma Suppress(identifier);

pragma Unsuppress(identifier);

A checking pragma is allowed only immediately within a **declarative_part**, immediately within a **package_specification**, or as a configuration pragma.

Legality Rules

The identifier shall be the name of a check.

This paragraph was deleted.

Static Semantics

A checking pragma applies to the named check in a specific region, and applies to all entities in that region. A checking pragma given in a **declarative_part** or immediately within a **package_specification** applies from the place of the **pragma** to the end of the innermost enclosing declarative region. The region for a checking pragma given as a configuration pragma is the declarative region for the entire compilation unit (or units) to which it applies.

If a checking pragma applies to a generic instantiation, then the checking pragma also applies to the instance. If a checking pragma applies to a call to a subprogram that has a **pragma Inline** applied to it, then the checking pragma also applies to the inlined subprogram body.

A **pragma Suppress** gives permission to an implementation to omit the named check (or every check in the case of **All_Checks**) for any entities to which it applies. If permission has been given to suppress a given check, the check is said to be *suppressed*.

A **pragma Unsuppress** revokes the permission to omit the named check (or every check in the case of **All_Checks**) given by any **pragma Suppress** that applies at the point of the **pragma Unsuppress**. The permission is revoked for the region to which the **pragma Unsuppress** applies. If there is no such permission at the point of a **pragma Unsuppress**, then the **pragma** has no effect. A later **pragma Suppress** can renew the permission.

The following are the language-defined checks:

- The following checks correspond to situations in which the exception **Constraint_Error** is raised upon failure.

Access_Check

When evaluating a dereference (explicit or implicit), check that the value of the **name** is not **null**. When converting to a subtype that excludes null, check that the converted value is not **null**.

Discriminant_Check

Check that the discriminants of a composite value have the values imposed by a discriminant constraint. Also, when accessing a record component, check that it exists for the current discriminant values.

Division_Check

Check that the second operand is not zero for the operations **/**, **rem** and **mod**.

Index_Check

Check that the bounds of an array value are equal to the corresponding bounds of an index constraint. Also, when accessing a component of an array object, check for each dimension that the given index value belongs to the range defined by the bounds of the

array object. Also, when accessing a slice of an array object, check that the given discrete range is compatible with the range defined by the bounds of the array object.

15 Length_Check

Check that two arrays have matching components, in the case of array subtype conversions, and logical operators for arrays of boolean components.

16 Overflow_Check

Check that a scalar value is within the base range of its type, in cases where the implementation chooses to raise an exception instead of returning the correct mathematical result.

17 Range_Check

Check that a scalar value satisfies a range constraint. Also, for the elaboration of a subtype_indication, check that the constraint (if present) is compatible with the subtype denoted by the subtype_mark. Also, for an aggregate, check that an index or discriminant value belongs to the corresponding subtype. Also, check that when the result of an operation yields an array, the value of each component belongs to the component subtype.

18 Tag_Check

Check that operand tags in a dispatching call are all equal. Check for the correct tag on tagged type conversions, for an assignment_statement, and when returning a tagged limited object from a function.

- 19 • The following checks correspond to situations in which the exception Program_Error is raised upon failure.

19.1/2 Accessibility_Check

Check the accessibility level of an entity or view.

19.2/2 Allocation_Check

For an allocator, check that the master of any tasks to be created by the allocator is not yet completed or some dependents have not yet terminated, and that the finalization of the collection has not started.

20 Elaboration_Check

When a subprogram or protected entry is called, a task activation is accomplished, or a generic instantiation is elaborated, check that the body of the corresponding unit has already been elaborated.

21/2 *This paragraph was deleted.*

- 22 • The following check corresponds to situations in which the exception Storage_Error is raised upon failure.

23 Storage_Check

Check that evaluation of an allocator does not require more space than is available for a storage pool. Check that the space available for a task or subprogram has not been exceeded.

- 24 • The following check corresponds to all situations in which any predefined exception is raised.

25 All_Checks

Represents the union of all checks; suppressing All_Checks suppresses all checks.

Erroneous Execution

- 26 If a given check has been suppressed, and the corresponding error situation occurs, the execution of the program is erroneous.

Implementation Permissions

An implementation is allowed to place restrictions on checking pragmas, subject only to the requirement that `pragma Unsuppress` shall allow any check names supported by `pragma Suppress`. An implementation is allowed to add additional check names, with implementation-defined semantics. When `Overflow_Check` has been suppressed, an implementation may also suppress an unspecified subset of the `Range_Checks`. 27/2

An implementation may support an additional parameter on `pragma Unsuppress` similar to the one allowed for `pragma Suppress` (see J.10). The meaning of such a parameter is implementation-defined. 27.1/2

Implementation Advice

The implementation should minimize the code executed for checks that have been suppressed. 28

NOTES

3 There is no guarantee that a suppressed check is actually removed; hence a `pragma Suppress` should be used only for efficiency reasons. 29

4 It is possible to give both a `pragma Suppress` and `Unsuppress` for the same check immediately within the same `declarative_part`. In that case, the last `pragma` given determines whether or not the check is suppressed. Similarly, it is possible to resuppress a check which has been unsuppressed by giving a `pragma Suppress` in an inner declarative region. 29.1/2

Examples

Examples of suppressing and unsuppressing checks: 30/2

```
pragma Suppress (Index_Check);
pragma Unsuppress (Overflow_Check);
```

 31/2

11.6 Exceptions and Optimization

This clause gives permission to the implementation to perform certain “optimizations” that do not necessarily preserve the canonical semantics. 1

Dynamic Semantics

The rest of this International Standard (outside this clause) defines the *canonical semantics* of the language. The canonical semantics of a given (legal) program determines a set of possible external effects that can result from the execution of the program with given inputs. 2

As explained in 1.1.3, “Conformity of an Implementation with the Standard”, the external effect of a program is defined in terms of its interactions with its external environment. Hence, the implementation can perform any internal actions whatsoever, in any order or in parallel, so long as the external effect of the execution of the program is one that is allowed by the canonical semantics, or by the rules of this clause. 3

Implementation Permissions

The following additional permissions are granted to the implementation: 4

- An implementation need not always raise an exception when a language-defined check fails. Instead, the operation that failed the check can simply yield an *undefined result*. The exception need be raised by the implementation only if, in the absence of raising it, the value of this undefined result would have some effect on the external interactions of the program. In determining this, the implementation shall not presume that an undefined result has a value that belongs to its subtype, nor even to the base range of its type, if scalar. Having removed the raise of the exception, the canonical semantics will in general allow the implementation to omit the code for the check, and some or all of the operation itself. 5

- 6
- If an exception is raised due to the failure of a language-defined check, then upon reaching the corresponding `exception_handler` (or the termination of the task, if none), the external interactions that have occurred need reflect only that the exception was raised somewhere within the execution of the `sequence_of_statements` with the handler (or the `task_body`), possibly earlier (or later if the interactions are independent of the result of the checked operation) than that defined by the canonical semantics, but not within the execution of some abort-deferred operation or *independent* subprogram that does not dynamically enclose the execution of the construct whose check failed. An independent subprogram is one that is defined outside the library unit containing the construct whose check failed, and has no `Inline pragma` applied to it. Any assignment that occurred outside of such abort-deferred operations or independent subprograms can be disrupted by the raising of the exception, causing the object or its parts to become abnormal, and certain subsequent uses of the object to be erroneous, as explained in 13.9.1.

NOTES

- 7
- 5 The permissions granted by this clause can have an effect on the semantics of a program only if the program fails a language-defined check.

Section 12: Generic Units

A *generic unit* is a program unit that is either a generic subprogram or a generic package. A generic unit is a *template*, which can be parameterized, and from which corresponding (nongeneric) subprograms or packages can be obtained. The resulting program units are said to be *instances* of the original generic unit.

A generic unit is declared by a `generic_declaration`. This form of declaration has a `generic_formal_part` declaring any generic formal parameters. An instance of a generic unit is obtained as the result of a `generic_instantiation` with appropriate generic actual parameters for the generic formal parameters. An instance of a generic subprogram is a subprogram. An instance of a generic package is a package.

Generic units are templates. As templates they do not have the properties that are specific to their nongeneric counterparts. For example, a generic subprogram can be instantiated but it cannot be called. In contrast, an instance of a generic subprogram is a (nongeneric) subprogram; hence, this instance can be called but it cannot be used to produce further instances.

12.1 Generic Declarations

A `generic_declaration` declares a generic unit, which is either a generic subprogram or a generic package. A `generic_declaration` includes a `generic_formal_part` declaring any generic formal parameters. A generic formal parameter can be an object; alternatively (unlike a parameter of a subprogram), it can be a type, a subprogram, or a package.

Syntax

```
generic_declaration ::= generic_subprogram_declaration | generic_package_declaration
generic_subprogram_declaration ::=
    generic_formal_part subprogram_specification;
generic_package_declaration ::=
    generic_formal_part package_specification;
generic_formal_part ::= generic {generic_formal_parameter_declaration | use_clause}
generic_formal_parameter_declaration ::=
    formal_object_declaration
    | formal_type_declaration
    | formal_subprogram_declaration
    | formal_package_declaration
```

The only form of `subtype_indication` allowed within a `generic_formal_part` is a `subtype_mark` (that is, the `subtype_indication` shall not include an explicit constraint). The defining name of a generic subprogram shall be an identifier (not an operator_symbol).

Static Semantics

A `generic_declaration` declares a generic unit — a generic package, generic procedure, or generic function, as appropriate.

An entity is a *generic formal* entity if it is declared by a `generic_formal_parameter_declaration`. “Generic formal,” or simply “formal,” is used as a prefix in referring to objects, subtypes (and types), functions, procedures and packages, that are generic formal entities, as well as to their respective declarations. Examples: “generic formal procedure” or a “formal integer type declaration.”

10 The elaboration of a `generic_declaration` has no effect.

NOTES

- 11 1 Outside a generic unit a name that denotes the `generic_declaration` denotes the generic unit. In contrast, within the declarative region of the generic unit, a name that denotes the `generic_declaration` denotes the current instance.
- 12 2 Within a generic subprogram `body`, the name of this program unit acts as the name of a subprogram. Hence this name can be overloaded, and it can appear in a recursive call of the current instance. For the same reason, this name cannot appear after the reserved word **new** in a (recursive) `generic_instantiation`.
- 13 3 A `default_expression` or `default_name` appearing in a `generic_formal_part` is not evaluated during elaboration of the `generic_formal_part`; instead, it is evaluated when used. (The usual visibility rules apply to any name used in a default: the denoted declaration therefore has to be visible at the place of the expression.)

Examples

14 *Examples of generic formal parts:*

```

15  generic      -- parameterless
16  generic
    Size : Natural;  -- formal object
17  generic
    Length : Integer := 200;          -- formal object with a default expression
18  Area   : Integer := Length*Length; -- formal object with a default expression
19  generic
    type Item is private;          -- formal type
    type Index is (<>);             -- formal type
    type Row is array (Index range <>) of Item; -- formal type
    with function "<" (X, Y : Item) return Boolean; -- formal subprogram

```

20 *Examples of generic declarations declaring generic subprograms Exchange and Squaring:*

```

21  generic
    type Elem is private;
    procedure Exchange(U, V : in out Elem);
22  generic
    type Item is private;
    with function "*" (U, V : Item) return Item is <>;
    function Squaring(X : Item) return Item;

```

23 *Example of a generic declaration declaring a generic package:*

```

24  generic
    type Item is private;
    type Vector is array (Positive range <>) of Item;
    with function Sum(X, Y : Item) return Item;
    package On_Vectors is
        function Sum (A, B : Vector) return Vector;
        function Sigma(A : Vector) return Item;
        Length_Error : exception;
    end On_Vectors;

```

12.2 Generic Bodies

The body of a generic unit (a *generic body*) is a template for the instance bodies. The syntax of a generic body is identical to that of a nongeneric body.

Dynamic Semantics

The elaboration of a generic body has no other effect than to establish that the generic unit can from then on be instantiated without failing the Elaboration_Check. If the generic body is a child of a generic package, then its elaboration establishes that each corresponding declaration nested in an instance of the parent (see 10.1.1) can from then on be instantiated without failing the Elaboration_Check.

NOTES

4 The syntax of generic subprograms implies that a generic subprogram body is always the completion of a declaration.

Examples

Example of a generic procedure body:

```

procedure Exchange(U, V : in out Elem) is  -- see 12.1
  T : Elem;  -- the generic formal type
begin
  T := U;
  U := V;
  V := T;
end Exchange;
```

Example of a generic function body:

```

function Squaring(X : Item) return Item is  -- see 12.1
begin
  return X*X;  -- the formal operator "*"
end Squaring;
```

Example of a generic package body:

```

package body On_Vectors is  -- see 12.1
  function Sum(A, B : Vector) return Vector is
    Result : Vector(A'Range);  -- the formal type Vector
    Bias   : constant Integer := B'First - A'First;
  begin
    if A'Length /= B'Length then
      raise Length_Error;
    end if;
    for N in A'Range loop
      Result(N) := Sum(A(N), B(N + Bias));  -- the formal function Sum
    end loop;
    return Result;
  end Sum;
  function Sigma(A : Vector) return Item is
    Total : Item := A(A'First);  -- the formal type Item
  begin
    for N in A'First + 1 .. A'Last loop
      Total := Sum(Total, A(N));  -- the formal function Sum
    end loop;
    return Total;
  end Sigma;
end On_Vectors;
```

12.3 Generic Instantiation

- 1 An instance of a generic unit is declared by a `generic_instantiation`.

Syntax

```

2/2  generic_instantiation ::=
      package defining_program_unit_name is
        new generic_package_name [generic_actual_part];
      | [overriding_indicator]
        procedure defining_program_unit_name is
          new generic_procedure_name [generic_actual_part];
        | [overriding_indicator]
          function defining_designator is
            new generic_function_name [generic_actual_part];
3  generic_actual_part ::=
      (generic_association {, generic_association})
4  generic_association ::=
      [generic_formal_parameter_selector_name =>] explicit_generic_actual_parameter
5  explicit_generic_actual_parameter ::= expression | variable_name
      | subprogram_name | entry_name | subtype_mark
      | package_instance_name
6  A generic_association is named or positional according to whether or not the generic_formal_-
parameter_selector_name is specified. Any positional associations shall precede any named
associations.
```

- 7/3 The *generic actual parameter* is either the `explicit_generic_actual_parameter` given in a `generic_`-`association` for each formal, or the corresponding `default_expression` or `default_name` if no `generic_`-`association` is given for the formal. When the meaning is clear from context, the term “generic actual,” or simply “actual,” is used as a synonym for “generic actual parameter” and also for the view denoted by one, or the value of one.

Legality Rules

- 8 In a `generic_instantiation` for a particular kind of program unit (package, procedure, or function), the name shall denote a generic unit of the corresponding kind (generic package, generic procedure, or generic function, respectively).
- 9/3 The *generic_formal_parameter_selector_name* of a named `generic_association` shall denote a `generic_formal_parameter_declaration` of the generic unit being instantiated. If two or more formal subprograms have the same defining name, then named associations are not allowed for the corresponding actuals.
- 9.1/3 The `generic_formal_parameter_declaration` for a positional `generic_association` is the parameter with the corresponding position in the `generic_formal_part` of the generic unit being instantiated.
- 10 A `generic_instantiation` shall contain at most one `generic_association` for each formal. Each formal without an association shall have a `default_expression` or `subprogram_default`.
- 11 In a generic unit Legality Rules are enforced at compile time of the `generic_declaration` and generic body, given the properties of the formals. In the visible part and formal part of an instance, Legality Rules are enforced at compile time of the `generic_instantiation`, given the properties of the actuals. In other parts of

an instance, Legality Rules are not enforced; this rule does not apply when a given rule explicitly specifies otherwise.

Static Semantics

A `generic_instantiation` declares an instance; it is equivalent to the instance declaration (a `package_declaration` or `subprogram_declaration`) immediately followed by the instance body, both at the place of the instantiation. 12

The instance is a copy of the text of the template. Each use of a formal parameter becomes (in the copy) a use of the actual, as explained below. An instance of a generic package is a package, that of a generic procedure is a procedure, and that of a generic function is a function. 13

The interpretation of each construct within a generic declaration or body is determined using the overloading rules when that generic declaration or body is compiled. In an instance, the interpretation of each (copied) construct is the same, except in the case of a name that denotes the `generic_declaration` or some declaration within the generic unit; the corresponding name in the instance then denotes the corresponding copy of the denoted declaration. The overloading rules do not apply in the instance. 14

In an instance, a `generic_formal_parameter_declaration` declares a view whose properties are identical to those of the actual, except as specified in 12.4, “Formal Objects” and 12.6, “Formal Subprograms”. Similarly, for a declaration within a `generic_formal_parameter_declaration`, the corresponding declaration in an instance declares a view whose properties are identical to the corresponding declaration within the declaration of the actual. 15

Implicit declarations are also copied, and a name that denotes an implicit declaration in the generic denotes the corresponding copy in the instance. However, for a type declared within the visible part of the generic, a whole new set of primitive subprograms is implicitly declared for use outside the instance, and may differ from the copied set if the properties of the type in some way depend on the properties of some actual type specified in the instantiation. For example, if the type in the generic is derived from a formal private type, then in the instance the type will inherit subprograms from the corresponding actual type. 16

These new implicit declarations occur immediately after the type declaration in the instance, and override the copied ones. The copied ones can be called only from within the instance; the new ones can be called only from outside the instance, although for tagged types, the body of a new one can be executed by a call to an old one. 17

In the visible part of an instance, an explicit declaration overrides an implicit declaration if they are homographs, as described in 8.3. On the other hand, an explicit declaration in the private part of an instance overrides an implicit declaration in the instance, only if the corresponding explicit declaration in the generic overrides a corresponding implicit declaration in the generic. Corresponding rules apply to the other kinds of overriding described in 8.3. 18

Post-Compilation Rules

Recursive generic instantiation is not allowed in the following sense: if a given generic unit includes an instantiation of a second generic unit, then the instance generated by this instantiation shall not include an instance of the first generic unit (whether this instance is generated directly, or indirectly by intermediate instantiations). 19

Dynamic Semantics

For the elaboration of a `generic_instantiation`, each `generic_association` is first evaluated. If a default is used, an implicit `generic_association` is assumed for this rule. These evaluations are done in an arbitrary 20

order, except that the evaluation for a default actual takes place after the evaluation for another actual if the default includes a **name** that denotes the other one. Finally, the instance declaration and body are elaborated.

- 21 For the evaluation of a **generic_association** the generic actual parameter is evaluated. Additional actions are performed in the case of a formal object of mode **in** (see 12.4).

NOTES

- 22 5 If a formal type is not tagged, then the type is treated as an untagged type within the generic body. Deriving from such a type in a generic body is permitted; the new type does not get a new tag value, even if the actual is tagged. Overriding operations for such a derived type cannot be dispatched to from outside the instance.

Examples

- 23 *Examples of generic instantiations (see 12.1):*

```
24  procedure Swap is new Exchange (Elem => Integer);
    procedure Swap is new Exchange (Character);      -- Swap is overloaded
    function Square is new Squaring (Integer);       -- "*" of Integer used by default
    function Square is new Squaring (Item => Matrix, "*" => Matrix_Product);
    function Square is new Squaring (Matrix, Matrix_Product); -- same as previous
25  package Int_Vectors is new On_Vectors (Integer, Table, "+");
```

- 26 *Examples of uses of instantiated units:*

```
27  Swap (A, B);
    A := Square (A);
28  T : Table (1 .. 5) := (10, 20, 30, 40, 50);
    N : Integer := Int_Vectors.Sigma (T); -- 150 (see 12.2, "Generic Bodies" for the body of
    Sigma)
29  use Int_Vectors;
    M : Integer := Sigma (T); -- 150
```

12.4 Formal Objects

- 1 A generic formal object can be used to pass a value or variable to a generic unit.

Syntax

```
2/3  formal_object_declaration ::=
    defining_identifier_list : mode [null_exclusion] subtype_mark [:= default_expression];
    | defining_identifier_list : mode access_definition [:= default_expression];
```

Name Resolution Rules

- 3 The expected type for the **default_expression**, if any, of a formal object is the type of the formal object.
- 4 For a generic formal object of mode **in**, the expected type for the actual is the type of the formal.
- 5/2 For a generic formal object of mode **in out**, the type of the actual shall resolve to the type determined by the **subtype_mark**, or for a **formal_object_declaration** with an **access_definition**, to a specific anonymous access type. If the anonymous access type is an access-to-object type, the type of the actual shall have the same designated type as that of the **access_definition**. If the anonymous access type is an access-to-subprogram type, the type of the actual shall have a designated profile which is type conformant with that of the **access_definition**.

Legality Rules

If a generic formal object has a **default_expression**, then the mode shall be **in** (either explicitly or by default); otherwise, its mode shall be either **in** or **in out**. 6

For a generic formal object of mode **in**, the actual shall be an **expression**. For a generic formal object of mode **in out**, the actual shall be a **name** that denotes a variable for which renaming is allowed (see 8.5.1). 7

In the case where the type of the formal is defined by an **access_definition**, the type of the actual and the type of the formal: 8/2

- shall both be access-to-object types with statically matching designated subtypes and with both or neither being access-to-constant types; or 8.1/2
- shall both be access-to-subprogram types with subtype conformant designated profiles. 8.2/2

For a **formal_object_declaration** with a **null_exclusion** or an **access_definition** that has a **null_exclusion**: 8.3/2

- if the actual matching the **formal_object_declaration** denotes the generic formal object of another generic unit *G*, and the instantiation containing the actual occurs within the body of *G* or within the body of a generic unit declared within the declarative region of *G*, then the declaration of the formal object of *G* shall have a **null_exclusion**; 8.4/2
- otherwise, the subtype of the actual matching the **formal_object_declaration** shall exclude null. In addition to the places where Legality Rules normally apply (see 12.3), this rule applies also in the private part of an instance of a generic unit. 8.5/2

Static Semantics

A **formal_object_declaration** declares a generic formal object. The default mode is **in**. For a formal object of mode **in**, the nominal subtype is the one denoted by the **subtype_mark** or **access_definition** in the declaration of the formal. For a formal object of mode **in out**, its type is determined by the **subtype_mark** or **access_definition** in the declaration; its nominal subtype is nonstatic, even if the **subtype_mark** denotes a static subtype; for a composite type, its nominal subtype is unconstrained if the first subtype of the type is unconstrained, even if the **subtype_mark** denotes a constrained subtype. 9/2

In an instance, a **formal_object_declaration** of mode **in** is a *full constant declaration* and declares a new stand-alone constant object whose initialization expression is the actual, whereas a **formal_object_declaration** of mode **in out** declares a view whose properties are identical to those of the actual. 10/2

Dynamic Semantics

For the evaluation of a **generic_association** for a formal object of mode **in**, a constant object is created, the value of the actual parameter is converted to the nominal subtype of the formal object, and assigned to the object, including any value adjustment — see 7.6. 11

NOTES

6 The constraints that apply to a generic formal object of mode **in out** are those of the corresponding generic actual parameter (not those implied by the **subtype_mark** that appears in the **formal_object_declaration**). Therefore, to avoid confusion, it is recommended that the name of a first subtype be used for the declaration of such a formal object. 12

12.5 Formal Types

- 1/2 A generic formal subtype can be used to pass to a generic unit a subtype whose type is in a certain category of types.

Syntax

- 2 `formal_type_declaration ::=`
 `type` `defining_identifier`[`discriminant_part`] `is` `formal_type_definition`;
- 3/2 `formal_type_definition ::=`
 `formal_private_type_definition`
 | `formal_derived_type_definition`
 | `formal_discrete_type_definition`
 | `formal_signed_integer_type_definition`
 | `formal_modular_type_definition`
 | `formal_floating_point_definition`
 | `formal_ordinary_fixed_point_definition`
 | `formal_decimal_fixed_point_definition`
 | `formal_array_type_definition`
 | `formal_access_type_definition`
 | `formal_interface_type_definition`

Legality Rules

- 4 For a generic formal subtype, the actual shall be a `subtype_mark`; it denotes the (*generic*) *actual subtype*.

Static Semantics

- 5 A `formal_type_declaration` declares a (*generic*) *formal type*, and its first subtype, the (*generic*) *formal subtype*.
- 6/2 The form of a `formal_type_definition` *determines a category (of types)* to which the formal type belongs. For a `formal_private_type_definition` the reserved words **tagged** and **limited** indicate the category of types (see 12.5.1). For a `formal_derived_type_definition` the category of types is the derivation class rooted at the ancestor type. For other formal types, the name of the syntactic category indicates the category of types; a `formal_discrete_type_definition` defines a discrete type, and so on.

Legality Rules

- 7/2 The actual type shall be in the category determined for the formal.

Static Semantics

- 8/3 The formal type also belongs to each category that contains the determined category. The primitive subprograms of the type are as for any type in the determined category. For a formal type other than a formal derived type, these are the predefined operators of the type. For an elementary formal type, the predefined operators are implicitly declared immediately after the declaration of the formal type. For a composite formal type, the predefined operators are implicitly declared either immediately after the declaration of the formal type, or later immediately within the declarative region in which the type is declared according to the rules of 7.3.1. In an instance, the copy of such an implicit declaration declares a view of the predefined operator of the actual type, even if this operator has been overridden for the actual type and even if it is never declared for the actual type. The rules specific to formal derived types are given in 12.5.1.

NOTES

- 7 Generic formal types, like all types, are not named. Instead, a name can denote a generic formal subtype. Within a generic unit, a generic formal type is considered as being distinct from all other (formal or nonformal) types. 9
- 8 A `discriminant_part` is allowed only for certain kinds of types, and therefore only for certain kinds of generic formal types. See 3.7. 10

Examples

Examples of generic formal types:

```

type Item is private;
type Buffer (Length : Natural) is limited private;
type Enum is (<>);
type Int is range <>;
type Angle is delta <>;
type Mass is digits <>;
type Table is array (Enum) of Item;

```

Example of a generic formal part declaring a formal integer type:

```

generic
  type Rank is range <>;
  First : Rank := Rank'First;
  Second : Rank := First + 1; -- the operator "+" of the type Rank

```

12.5.1 Formal Private and Derived Types

In its most general form, the category determined for a formal private type is all types, but it can be restricted to only nonlimited types or to only tagged types. The category determined for a formal derived type is the derivation class rooted at the ancestor type. 1/2

Syntax

```

formal_private_type_definition ::= [[abstract] tagged] [limited] private
formal_derived_type_definition ::=
  [abstract] [limited | synchronized] new subtype_mark [[and interface_list] with private]

```

Legality Rules

If a generic formal type declaration has a `known_discriminant_part`, then it shall not include a `default_expression` for a discriminant. 4

The *ancestor subtype* of a formal derived type is the subtype denoted by the `subtype_mark` of the `formal_derived_type_definition`. For a formal derived type declaration, the reserved words **with private** shall appear if and only if the ancestor type is a tagged type; in this case the formal derived type is a private extension of the ancestor type and the ancestor shall not be a class-wide type. Similarly, an `interface_list` or the optional reserved words **abstract** or **synchronized** shall appear only if the ancestor type is a tagged type. The reserved word **limited** or **synchronized** shall appear only if the ancestor type and any progenitor types are limited types. The reserved word **synchronized** shall appear (rather than **limited**) if the ancestor type or any of the progenitor types are synchronized interfaces. 5/2

The actual type for a formal derived type shall be a descendant of the ancestor type and every progenitor of the formal type. If the formal type is nonlimited, the actual type shall be nonlimited. If the reserved word **synchronized** appears in the declaration of the formal derived type, the actual type shall be a synchronized tagged type. 5.1/3

If the formal subtype is definite, then the actual subtype shall also be definite. 6

- 7 For a generic formal derived type with no `discriminant_part`:
- 8 • If the ancestor subtype is constrained, the actual subtype shall be constrained, and shall be statically compatible with the ancestor;
- 9 • If the ancestor subtype is an unconstrained access or composite subtype, the actual subtype shall be unconstrained.
- 10 • If the ancestor subtype is an unconstrained discriminated subtype, then the actual shall have the same number of discriminants, and each discriminant of the actual shall correspond to a discriminant of the ancestor, in the sense of 3.7.
- 10.1/2 • If the ancestor subtype is an access subtype, the actual subtype shall exclude null if and only if the ancestor subtype excludes null.
- 11 The declaration of a formal derived type shall not have a `known_discriminant_part`. For a generic formal private type with a `known_discriminant_part`:
- 12 • The actual type shall be a type with the same number of discriminants.
- 13 • The actual subtype shall be unconstrained.
- 14 • The subtype of each discriminant of the actual type shall statically match the subtype of the corresponding discriminant of the formal type.
- 15 For a generic formal type with an `unknown_discriminant_part`, the actual may, but need not, have discriminants, and may be definite or indefinite.

Static Semantics

- 16/2 The category determined for a formal private type is as follows:

<i>Type Definition</i>	<i>Determined Category</i>
limited private	the category of all types
private	the category of all nonlimited types
tagged limited private	the category of all tagged types
tagged private	the category of all nonlimited tagged types

- 18 The presence of the reserved word **abstract** determines whether the actual type may be abstract.
- 19 A formal private or derived type is a private or derived type, respectively. A formal derived tagged type is a private extension. A formal private or derived type is abstract if the reserved word **abstract** appears in its declaration.
- 20/2 If the ancestor type is a composite type that is not an array type, the formal type inherits components from the ancestor type (including discriminants if a new `discriminant_part` is not specified), as for a derived type defined by a `derived_type_definition` (see 3.4 and 7.3.1).
- 21/3 For a formal derived type, the predefined operators and inherited user-defined subprograms are determined by the ancestor type and any progenitor types, and are implicitly declared at the earliest place, if any, immediately within the declarative region in which the formal type is declared, where the corresponding primitive subprogram of the ancestor or progenitor is visible (see 7.3.1). In an instance, the copy of such an implicit declaration declares a view of the corresponding primitive subprogram of the ancestor or progenitor of the formal derived type, even if this primitive has been overridden for the actual type and even if it is never declared for the actual type. When the ancestor or progenitor of the formal derived type is itself a formal type, the copy of the implicit declaration declares a view of the corresponding copied operation of the ancestor or progenitor. In the case of a formal private extension, however, the tag of the

formal type is that of the actual type, so if the tag in a call is statically determined to be that of the formal type, the body executed will be that corresponding to the actual type.

For a prefix *S* that denotes a formal indefinite subtype, the following attribute is defined: 22/1

S'Definite *S'Definite* yields True if the actual subtype corresponding to *S* is definite; otherwise it yields False. The value of this attribute is of the predefined type Boolean. 23

Dynamic Semantics

In the case where a formal type has unknown discriminants, and the actual type is a class-wide type *T*Class: 23.1/3

- For the purposes of defining the primitive operations of the formal type, each of the primitive operations of the actual type is considered to be a subprogram (with an intrinsic calling convention — see 6.3.1) whose body consists of a dispatching call upon the corresponding operation of *T*, with its formal parameters as the actual parameters. If it is a function, the result of the dispatching call is returned. 23.2/2
- If the corresponding operation of *T* has no controlling formal parameters, then the controlling tag value is determined by the context of the call, according to the rules for tag-indeterminate calls (see 3.9.2 and 5.2). In the case where the tag would be statically determined to be that of the formal type, the call raises Program_Error. If such a function is renamed, any call on the renaming raises Program_Error. 23.3/2

NOTES

9 In accordance with the general rule that the actual type shall belong to the category determined for the formal (see 12.5, “Formal Types”): 24/2

- If the formal type is nonlimited, then so shall be the actual; 25
- For a formal derived type, the actual shall be in the class rooted at the ancestor subtype. 26

10 The actual type can be abstract only if the formal type is abstract (see 3.9.3). 27

11 If the formal has a discriminant_part, the actual can be either definite or indefinite. Otherwise, the actual has to be definite. 28

12.5.2 Formal Scalar Types

A *formal scalar type* is one defined by any of the *formal_type_definitions* in this subclause. The category determined for a formal scalar type is the category of all discrete, signed integer, modular, floating point, ordinary fixed point, or decimal types. 1/2

Syntax

```

formal_discrete_type_definition ::= (<>) 2
formal_signed_integer_type_definition ::= range <> 3
formal_modular_type_definition ::= mod <> 4
formal_floating_point_definition ::= digits <> 5
formal_ordinary_fixed_point_definition ::= delta <> 6
formal_decimal_fixed_point_definition ::= delta <> digits <> 7

```

Legality Rules

The actual type for a formal scalar type shall not be a nonstandard numeric type. 8

NOTES

12 The actual type shall be in the class of types implied by the syntactic category of the formal type definition (see 12.5, “Formal Types”). For example, the actual for a `formal_modular_type_definition` shall be a modular type.

12.5.3 Formal Array Types

The category determined for a formal array type is the category of all array types.

Syntax

`formal_array_type_definition ::= array_type_definition`

Legality Rules

The only form of `discrete_subtype_definition` that is allowed within the declaration of a generic formal (constrained) array subtype is a `subtype_mark`.

For a formal array subtype, the actual subtype shall satisfy the following conditions:

- The formal array type and the actual array type shall have the same dimensionality; the formal subtype and the actual subtype shall be either both constrained or both unconstrained.
- For each index position, the index types shall be the same, and the index subtypes (if unconstrained), or the index ranges (if constrained), shall statically match (see 4.9.1).
- The component subtypes of the formal and actual array types shall statically match.
- If the formal type has aliased components, then so shall the actual.

Examples

Example of formal array types:

```
-- given the generic package
generic
  type Item    is private;
  type Index   is (<>);
  type Vector  is array (Index range <>) of Item;
  type Table   is array (Index) of Item;
package P is
  ...
end P;

-- and the types
type Mix      is array (Color range <>) of Boolean;
type Option   is array (Color) of Boolean;

-- then Mix can match Vector and Option can match Table

package R is new P(Item => Boolean, Index => Color,
                  Vector => Mix, Table => Option);

-- Note that Mix cannot match Table and Option cannot match Vector
```

12.5.4 Formal Access Types

The category determined for a formal access type is the category of all access types.

1/2

Syntax

`formal_access_type_definition ::= access_type_definition`

2

Legality Rules

For a formal access-to-object type, the designated subtypes of the formal and actual types shall statically match.

3

If and only if the `general_access_modifier` **constant** applies to the formal, the actual shall be an access-to-constant type. If the `general_access_modifier` **all** applies to the formal, then the actual shall be a general access-to-variable type (see 3.10). If and only if the formal subtype excludes null, the actual subtype shall exclude null.

4/2

For a formal access-to-subprogram subtype, the designated profiles of the formal and the actual shall be mode-conformant, and the calling convention of the actual shall be *protected* if and only if that of the formal is *protected*.

5

Examples

Example of formal access types:

6

-- the formal types of the generic package

7

```
generic
  type Node is private;
  type Link is access Node;
  package P is
    ...
  end P;
```

8

-- can be matched by the actual types

9

```
type Car;
type Car_Name is access Car;
type Car is
  record
    Pred, Succ : Car_Name;
    Number     : License_Number;
    Owner      : Person;
  end record;
```

10

-- in the following generic instantiation

12

```
package R is new P(Node => Car, Link => Car_Name);
```

13

12.5.5 Formal Interface Types

The category determined for a formal interface type is the category of all interface types.

1/2

Syntax

`formal_interface_type_definition ::= interface_type_definition`

2/2

Legality Rules

The actual type shall be a descendant of every progenitor of the formal type.

3/2

- 4/2 The actual type shall be a limited, task, protected, or synchronized interface if and only if the formal type is also, respectively, a limited, task, protected, or synchronized interface.

Examples

```

5/2   type Root_Work_Item is tagged private;
6/2   generic
      type Managed_Task is task interface;
      type Work_Item(<>) is new Root_Work_Item with private;
package Server_Manager is
  task type Server is new Managed_Task with
    entry Start(Data : in out Work_Item);
  end Server;
end Server_Manager;

```

- 7/2 This generic allows an application to establish a standard interface that all tasks need to implement so they can be managed appropriately by an application-specific scheduler.

12.6 Formal Subprograms

- 1 Formal subprograms can be used to pass callable entities to a generic unit.

Syntax

```

2/2   formal_subprogram_declaration ::= formal_concrete_subprogram_declaration
      | formal_abstract_subprogram_declaration
2.1/2 formal_concrete_subprogram_declaration ::=
      with subprogram_specification [is subprogram_default];
2.2/2 formal_abstract_subprogram_declaration ::=
      with subprogram_specification is abstract [subprogram_default];
3/2   subprogram_default ::= default_name | <> | null
4      default_name ::= name
4.1/2   A subprogram_default of null shall not be specified for a formal function or for a
      formal_abstract_subprogram_declaration.

```

Name Resolution Rules

- 5 The expected profile for the `default_name`, if any, is that of the formal subprogram.
- 6 For a generic formal subprogram, the expected profile for the actual is that of the formal subprogram.

Legality Rules

- 7 The profiles of the formal and any named default shall be mode-conformant.
- 8 The profiles of the formal and actual shall be mode-conformant.
- 8.1/2 For a parameter or result subtype of a `formal_subprogram_declaration` that has an explicit `null_exclusion`:
- 8.2/2
- if the actual matching the `formal_subprogram_declaration` denotes a generic formal object of another generic unit *G*, and the instantiation containing the actual that occurs within the body of a generic unit *G* or within the body of a generic unit declared within the declarative region of the generic unit *G*, then the corresponding parameter or result type of the formal subprogram of *G* shall have a `null_exclusion`;
- 8.3/2
- otherwise, the subtype of the corresponding parameter or result type of the actual matching the `formal_subprogram_declaration` shall exclude `null`. In addition to the places where Legality

Rules normally apply (see 12.3), this rule applies also in the private part of an instance of a generic unit.

If a formal parameter of a `formal_abstract_subprogram_declaration` is of a specific tagged type *T* or of an anonymous access type designating a specific tagged type *T*, *T* is called a *controlling type* of the `formal_abstract_subprogram_declaration`. Similarly, if the result of a `formal_abstract_subprogram_declaration` for a function is of a specific tagged type *T* or of an anonymous access type designating a specific tagged type *T*, *T* is called a controlling type of the `formal_abstract_subprogram_declaration`. A `formal_abstract_subprogram_declaration` shall have exactly one controlling type. 8.4/2

The actual subprogram for a `formal_abstract_subprogram_declaration` shall be a dispatching operation of the controlling type or of the actual type corresponding to the controlling type. 8.5/2

Static Semantics

A `formal_subprogram_declaration` declares a generic formal subprogram. The types of the formal parameters and result, if any, of the formal subprogram are those determined by the `subtype_marks` given in the `formal_subprogram_declaration`; however, independent of the particular subtypes that are denoted by the `subtype_marks`, the nominal subtypes of the formal parameters and result, if any, are defined to be nonstatic, and unconstrained if of an array type (no applicable index constraint is provided in a call on a formal subprogram). In an instance, a `formal_subprogram_declaration` declares a view of the actual. The profile of this view takes its subtypes and calling convention from the original profile of the actual entity, while taking the formal parameter names and `default_expressions` from the profile given in the `formal_subprogram_declaration`. The view is a function or procedure, never an entry. 9

If a generic unit has a `subprogram_default` specified by a box, and the corresponding actual parameter is omitted, then it is equivalent to an explicit actual parameter that is a usage name identical to the defining name of the formal. If a `subtype_mark` in the profile of the `formal_subprogram_declaration` denotes a formal private or formal derived type and the actual type for this formal type is a class-wide type *T*Class, then for the purposes of resolving this `default_name` at the point of the instantiation, for each primitive subprogram of *T* that has a matching defining name, that is directly visible at the point of the instantiation, and that has at least one controlling formal parameter, a corresponding subprogram with the same defining name is directly visible, but with *T* systematically replaced by *T*Class in the types of its profile. The body of such a subprogram is as defined in 12.5.1 for primitive subprograms of a formal type when the actual type is class-wide. 10/3

If a generic unit has a `subprogram_default` specified by the reserved word **null**, and the corresponding actual parameter is omitted, then it is equivalent to an explicit actual parameter that is a null procedure having the profile given in the `formal_subprogram_declaration`. 10.1/2

The subprogram declared by a `formal_abstract_subprogram_declaration` with a controlling type *T* is a dispatching operation of type *T*. 10.2/2

NOTES

13 The matching rules for formal subprograms state requirements that are similar to those applying to `subprogram_renaming_declarations` (see 8.5.4). In particular, the name of a parameter of the formal subprogram need not be the same as that of the corresponding parameter of the actual subprogram; similarly, for these parameters, `default_expressions` need not correspond. 11

14 The constraints that apply to a parameter of a formal subprogram are those of the corresponding formal parameter of the matching actual subprogram (not those implied by the corresponding `subtype_mark` in the `_specification` of the formal subprogram). A similar remark applies to the result of a function. Therefore, to avoid confusion, it is recommended that the name of a first subtype be used in any declaration of a formal subprogram. 12

15 The subtype specified for a formal parameter of a generic formal subprogram can be any visible subtype, including a generic formal subtype of the same `generic_formal_part`. 13

- 14 16 A formal subprogram is matched by an attribute of a type if the attribute is a function with a matching specification. An enumeration literal of a given type matches a parameterless formal function whose result type is the given type.
- 15 17 A `default_name` denotes an entity that is visible or directly visible at the place of the `generic_declaration`; a box used as a default is equivalent to a name that denotes an entity that is directly visible at the place of the `_instantiation`.
- 16/2 18 The actual subprogram cannot be abstract unless the formal subprogram is a `formal_abstract_subprogram_declaration` (see 3.9.3).
- 16.1/2 19 The subprogram declared by a `formal_abstract_subprogram_declaration` is an abstract subprogram. All calls on a subprogram declared by a `formal_abstract_subprogram_declaration` must be dispatching calls. See 3.9.3.
- 16.2/2 20 A null procedure as a subprogram default has convention `Intrinsic` (see 6.3.1).

Examples

- 17 *Examples of generic formal subprograms:*
- 18/2 `with function "+"(X, Y : Item) return Item is <>;`
`with function Image(X : Enum) return String is Enum'Image;`
`with procedure Update is Default_Update;`
`with procedure Pre_Action(X : in Item) is null; -- defaults to no action`
`with procedure Write(S : not null access Root_Stream_Type'Class;`
`Desc : Descriptor)`
`is abstract Descriptor'Write; -- see 13.13.2`
-- Dispatching operation on Descriptor with default
-- given the generic procedure declaration
- 19
- 20 `generic`
`with procedure Action(X : in Item);`
`procedure Iterate(Seq : in Item_Sequence);`
- 21 *-- and the procedure*
- 22 `procedure Put_Item(X : in Item);`
- 23 *-- the following instantiation is possible*
- 24 `procedure Put_List is new Iterate(Action => Put_Item);`

12.7 Formal Packages

- 1 Formal packages can be used to pass packages to a generic unit. The `formal_package_declaration` declares that the formal package is an instance of a given generic package. Upon instantiation, the actual package has to be an instance of that generic package.

Syntax

- 2 `formal_package_declaration ::=`
`with package defining_identifier is new generic_package_name formal_package_actual_part;`
- 3/2 `formal_package_actual_part ::=`
`([others =>] <>)`
`| [generic_actual_part]`
`| (formal_package_association {, formal_package_association} [, others => <>])`
- 3.1/2 `formal_package_association ::=`
`generic_association`
`| generic_formal_parameter_selector_name => <>`
- 3.2/2 Any positional `formal_package_associations` shall precede any named `formal_package_associations`.

Legality Rules

The *generic_package_name* shall denote a generic package (the *template* for the formal package); the formal package is an instance of the template. 4

The *generic_formal_parameter_selector_name* of a *formal_package_association* shall denote a *generic_formal_parameter_declaration* of the template. If two or more formal subprograms of the template have the same defining name, then named associations are not allowed for the corresponding actuals. 4.1/3

A *formal_package_actual_part* shall contain at most one *formal_package_association* for each formal parameter. If the *formal_package_actual_part* does not include “**others** => <>”, each formal parameter without an association shall have a *default_expression* or *subprogram_default*. 4.2/2

The actual shall be an instance of the template. If the *formal_package_actual_part* is (<>) or (**others** => <>), then the actual may be any instance of the template; otherwise, certain of the actual parameters of the actual instance shall match the corresponding actual parameters of the formal package, determined as follows: 5/2

- If the *formal_package_actual_part* includes *generic_associations* as well as associations with <>, then only the actual parameters specified explicitly with *generic_associations* are required to match; 5.1/2
- Otherwise, all actual parameters shall match, whether any actual parameter is given explicitly or by default. 5.2/2

The rules for matching of actual parameters between the actual instance and the formal package are as follows: 5.3/2

- For a formal object of mode **in**, the actuals match if they are static expressions with the same value, or if they statically denote the same constant, or if they are both the literal **null**. 6/2
- For a formal subtype, the actuals match if they denote statically matching subtypes. 7
- For other kinds of formals, the actuals match if they statically denote the same entity. 8

For the purposes of matching, any actual parameter that is the name of a formal object of mode **in** is replaced by the formal object's actual expression (recursively). 8.1/1

Static Semantics

A *formal_package_declaration* declares a generic formal package. 9

The visible part of a formal package includes the first list of *basic_declarative_items* of the *package_specification*. In addition, for each actual parameter that is not required to match, a copy of the declaration of the corresponding formal parameter of the template is included in the visible part of the formal package. If the copied declaration is for a formal type, copies of the implicit declarations of the primitive subprograms of the formal type are also included in the visible part of the formal package. 10/2

For the purposes of matching, if the actual instance *A* is itself a formal package, then the actual parameters of *A* are those specified explicitly or implicitly in the *formal_package_actual_part* for *A*, plus, for those not specified, the copies of the formal parameters of the template included in the visible part of *A*. 11/2

Examples

12/2 *Example of a generic package with formal package parameters:*

```

13/2   with Ada.Containers.Ordered_Maps; -- see A.18.6
      generic
        with package Mapping_1 is new Ada.Containers.Ordered_Maps(<>);
        with package Mapping_2 is new Ada.Containers.Ordered_Maps
                                (Key_Type => Mapping_1.Element_Type,
                                 others => <>);

      package Ordered_Join is
        -- Provide a "join" between two mappings
14/2       subtype Key_Type is Mapping_1.Key_Type;
15/2       subtype Element_Type is Mapping_2.Element_Type;
16/2       function Lookup(Key : Key_Type) return Element_Type;
        ...
      end Ordered_Join;
```

Example of an instantiation of a package with formal packages:

```

with Ada.Containers.Ordered_Maps;
package Symbol_Package is
    type String_Id is ...
    type Symbol_Info is ...
    package String_Table is new Ada.Containers.Ordered_Maps
        (Key_Type => String,
         Element_Type => String_Id);
    package Symbol_Table is new Ada.Containers.Ordered_Maps
        (Key_Type => String_Id,
         Element_Type => Symbol_Info);
    package String_Info is new Ordered_Join(Mapping_1 => String_Table,
                                             Mapping_2 => Symbol_Table);
    Apple_Info : constant Symbol_Info := String_Info.Lookup("Apple");
end Symbol_Package;
```

12.8 Example of a Generic Package

The following example provides a possible formulation of stacks by means of a generic package. The size of each stack and the type of the stack elements are provided as generic formal parameters.

Examples

This paragraph was deleted.

```

generic
    Size : Positive;
    type Item is private;
package Stack is
    procedure Push(E : in Item);
    procedure Pop (E : out Item);
    Overflow, Underflow : exception;
end Stack;

package body Stack is
    type Table is array (Positive range <>) of Item;
    Space : Table(1 .. Size);
    Index : Natural := 0;

    procedure Push(E : in Item) is
    begin
        if Index >= Size then
            raise Overflow;
        end if;
        Index := Index + 1;
        Space(Index) := E;
    end Push;

    procedure Pop(E : out Item) is
    begin
        if Index = 0 then
            raise Underflow;
        end if;
        E := Space(Index);
        Index := Index - 1;
    end Pop;
end Stack;
```

Instances of this generic package can be obtained as follows:

```

package Stack_Int  is new Stack(Size => 200, Item => Integer);
package Stack_Bool is new Stack(100, Boolean);

```

Thereafter, the procedures of the instantiated packages can be called as follows:

```

Stack_Int.Push(N);
Stack_Bool.Push(True);

```

Alternatively, a generic formulation of the type Stack can be given as follows (package body omitted):

```

generic
  type Item is private;
package On_Stacks is
  type Stack(Size : Positive) is limited private;
  procedure Push(S : in out Stack; E : in Item);
  procedure Pop (S : in out Stack; E : out Item);
  Overflow, Underflow : exception;
private
  type Table is array (Positive range <>) of Item;
  type Stack(Size : Positive) is
    record
      Space : Table(1 .. Size);
      Index : Natural := 0;
    end record;
end On_Stacks;

```

In order to use such a package, an instance has to be created and thereafter stacks of the corresponding type can be declared:

```

declare
  package Stack_Real is new On_Stacks(Real); use Stack_Real;
  S : Stack(100);
begin
  ...
  Push(S, 2.54);
  ...
end;

```

Section 13: Representation Issues

This section describes features for querying and controlling certain aspects of entities and for interfacing to hardware. 1/1

13.1 Operational and Representation Items

Representation and operational items can be used to specify aspects of entities. Two kinds of aspects of entities can be specified: aspects of representation and operational aspects. Representation items specify how the types and other entities of the language are to be mapped onto the underlying machine. Operational items specify other properties of entities. 0.1/1

There are six kinds of *representation items*: `attribute_definition_clauses` for representation attributes, `enumeration_representation_clauses`, `record_representation_clauses`, `at_clauses`, `component_clauses`, and *representation pragmas*. They can be provided to give more efficient representation or to interface with features that are outside the domain of the language (for example, peripheral hardware). 1/1

An *operational item* is an `attribute_definition_clause` for an operational attribute. 1.1/1

An operational item or a representation item applies to an entity identified by a `local_name`, which denotes an entity declared local to the current declarative region, or a library unit declared immediately preceding a representation pragma in a compilation. 1.2/1

Syntax

```
aspect_clause ::= attribute_definition_clause
                | enumeration_representation_clause
                | record_representation_clause
                | at_clause
local_name ::= direct_name
            | direct_name'attribute_designator
            | library_unit_name
```

2/1

3

A representation pragma is allowed only at places where an `aspect_clause` or `compilation_unit` is allowed. 4/1

Name Resolution Rules

In an operational item or representation item, if the `local_name` is a `direct_name`, then it shall resolve to denote a declaration (or, in the case of a `pragma`, one or more declarations) that occurs immediately within the same declarative region as the item. If the `local_name` has an `attribute_designator`, then it shall resolve to denote an implementation-defined component (see 13.5.1) or a class-wide type implicitly declared immediately within the same declarative region as the item. A `local_name` that is a *library_unit_name* (only permitted in a representation pragma) shall resolve to denote the `library_item` that immediately precedes (except for other pragmas) the representation pragma. 5/1

Legality Rules

The `local_name` of an `aspect_clause` or representation pragma shall statically denote an entity (or, in the case of a `pragma`, one or more entities) declared immediately preceding it in a compilation, or within the same `declarative_part`, `package_specification`, `task_definition`, `protected_definition`, or `record_definition` as the representation or operational item. If a `local_name` denotes a local callable entity, it may do so 6/1

through a local `subprogram_renaming_declaration` (as a way to resolve ambiguity in the presence of overloading); otherwise, the `local_name` shall not denote a `renaming_declaration`.

- 7/2 The *representation* of an object consists of a certain number of bits (the *size* of the object). For an object of an elementary type, these are the bits that are normally read or updated by the machine code when loading, storing, or operating-on the value of the object. For an object of a composite type, these are the bits reserved for this object, and include bits occupied by subcomponents of the object. If the size of an object is greater than that of its subtype, the additional bits are padding bits. For an elementary object, these padding bits are normally read and updated along with the others. For a composite object, padding bits might not be read or updated in any given composite operation, depending on the implementation.
- 8/3 A representation item *directly specifies* an *aspect of representation* of the entity denoted by the `local_name`, except in the case of a type-related representation item, whose `local_name` shall denote a first subtype, and which directly specifies an aspect of the subtype's type. A representation item that names a subtype is either *subtype-specific* (Size and Alignment clauses) or *type-related* (all others). Subtype-specific aspects may differ for different subtypes of the same type. Unless otherwise specified, the name of the aspect of representation specified by a representation pragma is the name of the pragma.
- 8.1/1 An operational item *directly specifies* an *operational aspect* of the type of the subtype denoted by the `local_name`. The `local_name` of an operational item shall denote a first subtype. An operational item that names a subtype is type-related.
- 9 A representation item that directly specifies an aspect of a subtype or type shall appear after the type is completely defined (see 3.11.1), and before the subtype or type is frozen (see 13.14). If a representation item is given that directly specifies an aspect of an entity, then it is illegal to give another representation item that directly specifies the same aspect of the entity.
- 9.1/1 An operational item that directly specifies an aspect of a type shall appear before the type is frozen (see 13.14). If an operational item is given that directly specifies an aspect of a type, then it is illegal to give another operational item that directly specifies the same aspect of the type.
- 9.2/3 Unless otherwise specified, an operational or representation item shall not specify an aspect of a generic formal parameter.
- 10 For an untagged derived type, no type-related representation items are allowed if the parent type is a by-reference type, or has any user-defined primitive subprograms.
- 11/2 Operational and representation aspects of a generic formal parameter are the same as those of the actual. Operational and representation aspects are the same for all views of a type. A type-related representation item is not allowed for a descendant of a generic formal untagged type.
- 12 A representation item that specifies the Size for a given subtype, or the size or storage place for an object (including a component) of a given subtype, shall allow for enough storage space to accommodate any value of the subtype.
- 13/1 A representation or operational item that is not supported by the implementation is illegal, or raises an exception at run time.
- 13.1/2 A `type_declaration` is illegal if it has one or more progenitors, and a representation item applies to an ancestor, and this representation item conflicts with the representation of some other ancestor. The cases that cause conflicts are implementation defined.

Static Semantics

- 14 If two subtypes statically match, then their subtype-specific aspects (Size and Alignment) are the same.

A derived type inherits each type-related aspect of representation of its parent type that was directly specified before the declaration of the derived type, or (in the case where the parent is derived) that was inherited by the parent type from the grandparent type. A derived subtype inherits each subtype-specific aspect of representation of its parent subtype that was directly specified before the declaration of the derived type, or (in the case where the parent is derived) that was inherited by the parent subtype from the grandparent subtype, but only if the parent subtype statically matches the first subtype of the parent type. An inherited aspect of representation is overridden by a subsequent representation item that specifies a different value for the same aspect of the type or subtype. 15/3

In contrast, whether operational aspects are inherited by an untagged derived type depends on each specific aspect. Operational aspects are never inherited for a tagged type. When operational aspects are inherited by an untagged derived type, aspects that were directly specified by operational items that are visible at the point of the derived type declaration, or (in the case where the parent is derived) that were inherited by the parent type from the grandparent type are inherited. An inherited operational aspect is overridden by a subsequent operational item that specifies the same aspect of the type. 15.1/2

When an aspect that is a subprogram is inherited, the derived type inherits the aspect in the same way that a derived type inherits a user-defined primitive subprogram from its parent (see 3.4). 15.2/2

Each aspect of representation of an entity is as follows: 16

- If the aspect is *specified* for the entity, meaning that it is either directly specified or inherited, then that aspect of the entity is as specified, except in the case of `Storage_Size`, which specifies a minimum. 17
- If an aspect of representation of an entity is not specified, it is chosen by default in an unspecified manner. 18

If an operational aspect is *specified* for an entity (meaning that it is either directly specified or inherited), then that aspect of the entity is as specified. Otherwise, the aspect of the entity has the default value for that aspect. 18.1/1

A representation item that specifies an aspect of representation that would have been chosen in the absence of the representation item is said to be *confirming*. 18.2/2

Dynamic Semantics

For the elaboration of an `aspect_clause`, any evaluable constructs within it are evaluated. 19/1

Implementation Permissions

An implementation may interpret aspects of representation in an implementation-defined manner. An implementation may place implementation-defined restrictions on representation items. A *recommended level of support* is specified for representation items and related features in each subclause. These recommendations are changed to requirements for implementations that support the Systems Programming Annex (see C.2, “Required Representation Support”). 20

Implementation Advice

The recommended level of support for all representation items is qualified as follows: 21

- A confirming representation item should be supported. 21.1/2
- An implementation need not support representation items containing nonstatic expressions, except that an implementation should support a representation item for a given entity if each nonstatic expression in the representation item is a name that statically denotes a constant declared before the entity. 22

- 23 • An implementation need not support a specification for the Size for a given composite subtype, nor the size or storage place for an object (including a component) of a given composite subtype, unless the constraints on the subtype and its composite subcomponents (if any) are all static constraints.
- 24/2 • An implementation need not support a nonconfirming representation item if it could cause an aliased object or an object of a by-reference type to be allocated at a nonaddressable location or, when the alignment attribute of the subtype of such an object is nonzero, at an address that is not an integral multiple of that alignment.
- 25/2 • An implementation need not support a nonconfirming representation item if it could cause an aliased object of an elementary type to have a size other than that which would have been chosen by default.
- 26/2 • An implementation need not support a nonconfirming representation item if it could cause an aliased object of a composite type, or an object whose type is by-reference, to have a size smaller than that which would have been chosen by default.
- 27/2 • An implementation need not support a nonconfirming subtype-specific representation item specifying an aspect of representation of an indefinite or abstract subtype.
- 28/2 For purposes of these rules, the determination of whether a representation item applied to a type *could cause* an object to have some property is based solely on the properties of the type itself, not on any available information about how the type is used. In particular, it presumes that minimally aligned objects of this type might be declared at some point.

13.2 Pragma Pack

- 1 A **pragma Pack** specifies that storage minimization should be the main criterion when selecting the representation of a composite type.

Syntax

- 2 The form of a **pragma Pack** is as follows:
- 3 **pragma Pack**(*first_subtype_local_name*);

Legality Rules

- 4 The *first_subtype_local_name* of a **pragma Pack** shall denote a composite subtype.

Static Semantics

- 5 A **pragma Pack** specifies the *packing* aspect of representation; the type (or the extension part) is said to be *packed*. For a type extension, the parent part is packed as for the parent type, and a **pragma Pack** causes packing only of the extension part.

Implementation Advice

- 6 If a type is packed, then the implementation should try to minimize storage allocated to objects of the type, possibly at the expense of speed of accessing components, subject to reasonable complexity in addressing calculations.
- 6.1/2 If a packed type has a component that is not of a by-reference type and has no aliased part, then such a component need not be aligned according to the Alignment of its subtype; in particular it need not be allocated on a storage element boundary.
- 7 The recommended level of support for **pragma Pack** is:

- For a packed record type, the components should be packed as tightly as possible subject to the Sizes of the component subtypes, and subject to any `record_representation_clause` that applies to the type; the implementation may, but need not, reorder components or cross aligned word boundaries to improve the packing. A component whose Size is greater than the word size may be allocated an integral number of words. 8
- For a packed array type, if the Size of the component subtype is less than or equal to the word size, `Component_Size` should be less than or equal to the Size of the component subtype, rounded up to the nearest factor of the word size. 9/3

13.3 Operational and Representation Attributes

The values of certain implementation-dependent characteristics can be obtained by interrogating appropriate operational or representation attributes. Some of these attributes are specifiable via an `attribute_definition_clause`. 1/1

Syntax

```
attribute_definition_clause ::=
    for local_name'attribute_designator use expression;
    | for local_name'attribute_designator use name; 2
```

Name Resolution Rules

For an `attribute_definition_clause` that specifies an attribute that denotes a value, the form with an expression shall be used. Otherwise, the form with a name shall be used. 3

For an `attribute_definition_clause` that specifies an attribute that denotes a value or an object, the expected type for the expression or name is that of the attribute. For an `attribute_definition_clause` that specifies an attribute that denotes a subprogram, the expected profile for the name is the profile required for the attribute. For an `attribute_definition_clause` that specifies an attribute that denotes some other kind of entity, the name shall resolve to denote an entity of the appropriate kind. 4

Legality Rules

An `attribute_designator` is allowed in an `attribute_definition_clause` only if this International Standard explicitly allows it, or for an implementation-defined attribute if the implementation allows it. Each specifiable attribute constitutes an operational aspect or aspect of representation. 5/1

For an `attribute_definition_clause` that specifies an attribute that denotes a subprogram, the profile shall be mode conformant with the one required for the attribute, and the convention shall be Ada. Additional requirements are defined for particular attributes. 6

Static Semantics

A *Size clause* is an `attribute_definition_clause` whose `attribute_designator` is `Size`. Similar definitions apply to the other specifiable attributes. 7/2

A *storage element* is an addressable element of storage in the machine. A *word* is the largest amount of storage that can be conveniently and efficiently manipulated by the hardware, given the implementation's run-time model. A word consists of an integral number of storage elements. 8

A *machine scalar* is an amount of storage that can be conveniently and efficiently loaded, stored, or operated upon by the hardware. Machine scalars consist of an integral number of storage elements. The set of machine scalars is implementation defined, but includes at least the storage element and the word. Machine scalars are used to interpret `component_clauses` when the nondefault bit ordering applies. 8.1/3

9/1 The following representation attributes are defined: Address, Alignment, Size, Storage_Size, and Component_Size.

10/1 For a prefix X that denotes an object, program unit, or label:

11 X'Address Denotes the address of the first of the storage elements allocated to X. For a program unit or label, this value refers to the machine code associated with the corresponding body or statement. The value of this attribute is of type System.Address.

11.1/3 The prefix of X'Address shall not statically denote a subprogram that has convention Intrinsic. X'Address raises Program_Error if X denotes a subprogram that has convention Intrinsic.

12 Address may be specified for stand-alone objects and for program units via an attribute_definition_clause.

Erroneous Execution

13/3 If an Address is specified, it is the programmer's responsibility to ensure that the address is valid and appropriate for the entity and its use; otherwise, program execution is erroneous.

Implementation Advice

14 For an array X, X'Address should point at the first component of the array, and not at the array bounds.

15 The recommended level of support for the Address attribute is:

- 16 • X'Address should produce a useful result if X is an object that is aliased or of a by-reference type, or is an entity whose Address has been specified.
- 17 • An implementation should support Address clauses for imported subprograms.
- 18/2 • *This paragraph was deleted.*
- 19 • If the Address of an object is specified, or it is imported or exported, then the implementation should not perform optimizations based on assumptions of no aliases.

NOTES

20 1 The specification of a link name in a pragma Export (see B.1) for a subprogram or object is an alternative to explicit specification of its link-time address, allowing a link-time directive to place the subprogram or object within memory.

21 2 The rules for the Size attribute imply, for an aliased object X, that if X'Size = Storage_Unit, then X'Address points at a storage element containing all of the bits of X, and only the bits of X.

Static Semantics

22/2 For a prefix X that denotes an object:

23/2 X'Alignment The value of this attribute is of type *universal_integer*, and nonnegative; zero means that the object is not necessarily aligned on a storage element boundary. If X'Alignment is not zero, then X is aligned on a storage unit boundary and X'Address is an integral multiple of X'Alignment (that is, the Address modulo the Alignment is zero).

24/2 *This paragraph was deleted.*

25/2 Alignment may be specified for stand-alone objects via an attribute_definition_clause; the expression of such a clause shall be static, and its value nonnegative.

26/2 *This paragraph was deleted.*

26.1/2 For every subtype S:

26.2/2 S'Alignment The value of this attribute is of type *universal_integer*, and nonnegative.

For an object *X* of subtype *S*, if *S*'Alignment is not zero, then *X*'Alignment is a nonzero integral multiple of *S*'Alignment unless specified otherwise by a representation item. 26.3/2

Alignment may be specified for first subtypes via an `attribute_definition_clause`; the expression of such a clause shall be static, and its value nonnegative. 26.4/2

Erroneous Execution

Program execution is erroneous if an Address clause is given that conflicts with the Alignment. 27

For an object that is not allocated under control of the implementation, execution is erroneous if the object is not aligned according to its Alignment. 28/2

Implementation Advice

For any tagged specific subtype *S*, *S*'Class'Alignment should equal *S*'Alignment. 28.1/3

The recommended level of support for the Alignment attribute for subtypes is: 29

- An implementation should support an Alignment clause for a discrete type, fixed point type, record type, or array type, specifying an Alignment value that is zero or a power of two, subject to the following: 30/2
- An implementation need not support an Alignment clause for a signed integer type specifying an Alignment greater than the largest Alignment value that is ever chosen by default by the implementation for any signed integer type. A corresponding limitation may be imposed for modular integer types, fixed point types, enumeration types, record types, and array types. 31/2
- An implementation need not support a nonconfirming Alignment clause which could enable the creation of an object of an elementary type which cannot be easily loaded and stored by available machine instructions. 32/2
- An implementation need not support an Alignment specified for a derived tagged type which is not a multiple of the Alignment of the parent type. An implementation need not support a nonconfirming Alignment specified for a derived untagged by-reference type. 32.1/2

The recommended level of support for the Alignment attribute for objects is: 33

- *This paragraph was deleted.* 34/2
- For stand-alone library-level objects of statically constrained subtypes, the implementation should support all Alignments supported by the target linker. For example, page alignment is likely to be supported for such objects, but not for subtypes. 35
- For other objects, an implementation should at least support the alignments supported for their subtype, subject to the following: 35.1/2
- An implementation need not support Alignments specified for objects of a by-reference type or for objects of types containing aliased subcomponents if the specified Alignment is not a multiple of the Alignment of the subtype of the object. 35.2/2

NOTES

3 Alignment is a subtype-specific attribute. 36

This paragraph was deleted. 37/2

4 A `component_clause`, `Component_Size` clause, or a `pragma Pack` can override a specified Alignment. 38

Static Semantics

For a prefix *X* that denotes an object: 39/1

X'Size Denotes the size in bits of the representation of the object. The value of this attribute is of the type *universal_integer*. 40

41 Size may be specified for stand-alone objects via an `attribute_definition_clause`; the expression of such a clause shall be static and its value nonnegative.

Implementation Advice

41.1/2 The size of an array object should not include its bounds.

42/2 The recommended level of support for the Size attribute of objects is the same as for subtypes (see below), except that only a confirming Size clause need be supported for an aliased elementary object.

43/2

- *This paragraph was deleted.*

Static Semantics

44 For every subtype S:

45 S'Size If S is definite, denotes the size (in bits) that the implementation would choose for the following objects of subtype S:

- 46
 - A record component of subtype S when the record type is packed.
- 47
 - The formal parameter of an instance of `Unchecked_Conversion` that converts from subtype S to some other subtype.

48 If S is indefinite, the meaning is implementation defined. The value of this attribute is of the type *universal integer*. The Size of an object is at least as large as that of its subtype, unless the object's Size is determined by a Size clause, a `component_clause`, or a `Component_Size` clause. Size may be specified for first subtypes via an `attribute_definition_clause`; the expression of such a clause shall be static and its value nonnegative.

Implementation Requirements

49 In an implementation, `Boolean'Size` shall be 1.

Implementation Advice

50/2 If the Size of a subtype allows for efficient independent addressability (see 9.10) on the target architecture, then the Size of the following objects of the subtype should equal the Size of the subtype:

- 51
 - Aliased objects (including components).
- 52
 - Unaliased components, unless the Size of the component is determined by a `component_clause` or `Component_Size` clause.

53 A Size clause on a composite subtype should not affect the internal layout of components.

54 The recommended level of support for the Size attribute of subtypes is:

- 55
 - The Size (if not specified) of a static discrete or fixed point subtype should be the number of bits needed to represent each value belonging to the subtype using an unbiased representation, leaving space for a sign bit only if the subtype contains negative values. If such a subtype is a first subtype, then an implementation should support a specified Size for it that reflects this representation.
- 56
 - For a subtype implemented with levels of indirection, the Size should include the size of the pointers, but not the size of what they point at.
- 56.1/2
 - An implementation should support a Size clause for a discrete type, fixed point type, record type, or array type, subject to the following:
- 56.2/2
 - An implementation need not support a Size clause for a signed integer type specifying a Size greater than that of the largest signed integer type supported by the implementation in the absence of a size clause (that is, when the size is chosen by default). A corresponding

limitation may be imposed for modular integer types, fixed point types, enumeration types, record types, and array types.

- A nonconfirming size clause for the first subtype of a derived untagged by-reference type need not be supported. 56.3/2

NOTES

5 Size is a subtype-specific attribute. 57

6 A `component_clause` or `Component_Size` clause can override a specified `Size`. A `pragma Pack` cannot. 58

Static Semantics

For a prefix `T` that denotes a task object (after any implicit dereference): 59/1

`T'Storage_Size` 60

Denotes the number of storage elements reserved for the task. The value of this attribute is of the type *universal_integer*. The `Storage_Size` includes the size of the task's stack, if any. The language does not specify whether or not it includes other storage associated with the task (such as the “task control block” used by some implementations.) If a `pragma Storage_Size` is given, the value of the `Storage_Size` attribute is at least the value specified in the `pragma`.

A `pragma Storage_Size` specifies the amount of storage to be reserved for the execution of a task. 61

Syntax

The form of a `pragma Storage_Size` is as follows: 62

pragma `Storage_Size`(`expression`); 63

A `pragma Storage_Size` is allowed only immediately within a `task_definition`. 64

Name Resolution Rules

The expression of a `pragma Storage_Size` is expected to be of any integer type. 65

Dynamic Semantics

A `pragma Storage_Size` is elaborated when an object of the type defined by the immediately enclosing `task_definition` is created. For the elaboration of a `pragma Storage_Size`, the `expression` is evaluated; the `Storage_Size` attribute of the newly created task object is at least the value of the `expression`. 66

At the point of task object creation, or upon task activation, `Storage_Error` is raised if there is insufficient free storage to accommodate the requested `Storage_Size`. 67

Static Semantics

For a prefix `X` that denotes an array subtype or array object (after any implicit dereference): 68/1

`X'Component_Size` 69

Denotes the size in bits of components of the type of `X`. The value of this attribute is of type *universal_integer*.

`Component_Size` may be specified for array types via an `attribute_definition_clause`; the expression of such a clause shall be static, and its value nonnegative. 70

Implementation Advice

The recommended level of support for the `Component_Size` attribute is: 71

- An implementation need not support specified `Component_Sizes` that are less than the `Size` of the component subtype. 72

- 73 • An implementation should support specified `Component_Sizes` that are factors and multiples of the word size. For such `Component_Sizes`, the array should contain no gaps between components. For other `Component_Sizes` (if supported), the array should contain no gaps between components when packing is also specified; the implementation should forbid this combination in cases where it cannot support a no-gaps representation.

Static Semantics

73.1/1 The following operational attribute is defined: `External_Tag`.

74/1 For every subtype *S* of a tagged type *T* (specific or class-wide):

75/3 `S'External_Tag`

`S'External_Tag` denotes an external string representation for `S'Tag`; it is of the predefined type `String`. `External_Tag` may be specified for a specific tagged type via an `attribute_definition_clause`; the expression of such a clause shall be static. The default external tag representation is implementation defined. See 13.13.2. The value of `External_Tag` is never inherited; the default value is always used unless a new value is directly specified for a type.

Dynamic Semantics

75.1/3 If a user-specified external tag `S'External_Tag` is the same as `T'External_Tag` for some other tagged type declared by a different declaration in the partition, `Program_Error` is raised by the elaboration of the `attribute_definition_clause`.

Implementation Requirements

76 In an implementation, the default external tag for each specific tagged type declared in a partition shall be distinct, so long as the type is declared outside an instance of a generic body. If the compilation unit in which a given tagged type is declared, and all compilation units on which it semantically depends, are the same in two different partitions, then the external tag for the type shall be the same in the two partitions. What it means for a compilation unit to be the same in two different partitions is implementation defined. At a minimum, if the compilation unit is not recompiled between building the two different partitions that include it, the compilation unit is considered the same in the two partitions.

Implementation Permissions

76.1/3 If a user-specified external tag `S'External_Tag` is the same as `T'External_Tag` for some other tagged type declared by a different declaration in the partition, the partition may be rejected.

NOTES

- 77/2 7 The following language-defined attributes are specifiable, at least for some of the kinds of entities to which they apply: `Address`, `Alignment`, `Bit_Order`, `Component_Size`, `External_Tag`, `Input`, `Machine_Radix`, `Output`, `Read`, `Size`, `Small`, `Storage_Pool`, `Storage_Size`, `Stream_Size`, and `Write`.
- 78 8 It follows from the general rules in 13.1 that if one writes “**for** `X'Size` **use** `Y`,” then the `X'Size` attribute_reference will return `Y` (assuming the implementation allows the `Size` clause). The same is true for all of the specifiable attributes except `Storage_Size`.

Examples

79 *Examples of attribute definition clauses:*

80 `Byte : constant := 8;`
 `Page : constant := 2**12;`


```

type Medium is range 0 .. 65_000;
for Medium'Size use 2*Byte;
for Medium'Alignment use 2;
Device_Register : Medium;
for Device_Register'Size use Medium'Size;
for Device_Register'Address use
System.Storage_Elements.To_Address(16#FFFF_0020#);

type Short is delta 0.01 range -100.0 .. 100.0;
for Short'Size use 15;

for Car_Name'Storage_Size use -- specify access type's storage pool size
    2000*((Car'Size/System.Storage_Unit) +1); -- approximately 2000 cars

function My_Input(Stream : not null access
Ada.Streams.Root_Stream_Type'Class)
return T;
for T'Input use My_Input; -- see 13.13.2

```

NOTES

9 *Notes on the examples:* In the Size clause for Short, fifteen bits is the minimum necessary, since the type definition requires Short'Small $\leq 2^{**}(-7)$.

13.4 Enumeration Representation Clauses

An `enumeration_representation_clause` specifies the internal codes for enumeration literals.

Syntax

```

enumeration_representation_clause ::=
    for first_subtype_local_name use enumeration_aggregate;

enumeration_aggregate ::= array_aggregate

```

Name Resolution Rules

The `enumeration_aggregate` shall be written as a one-dimensional `array_aggregate`, for which the index subtype is the unconstrained subtype of the enumeration type, and each component expression is expected to be of any integer type.

Legality Rules

The *first_subtype_local_name* of an `enumeration_representation_clause` shall denote an enumeration subtype.

Each component of the `array_aggregate` shall be given by an `expression` rather than a $\langle \rangle$. The expressions given in the `array_aggregate` shall be static, and shall specify distinct integer codes for each value of the enumeration type; the associated integer codes shall satisfy the predefined ordering relation of the type.

Static Semantics

An `enumeration_representation_clause` specifies the *coding* aspect of representation. The coding consists of the *internal code* for each enumeration literal, that is, the integral value used internally to represent each literal.

Implementation Requirements

For nonboolean enumeration types, if the coding is not specified for the type, then for each value of the type, the internal code shall be equal to its position number.

Implementation Advice

- 9 The recommended level of support for `enumeration_representation_clauses` is:
- 10 • An implementation should support at least the internal codes in the range `System.Min_Int..System.Max_Int`. An implementation need not support `enumeration_representation_clauses` for boolean types.

NOTES

- 11/1 10 `Unchecked_Conversion` may be used to query the internal codes used for an enumeration type. The attributes of the type, such as `Succ`, `Pred`, and `Pos`, are unaffected by the `enumeration_representation_clause`. For example, `Pos` always returns the position number, *not* the internal integer code that might have been specified in an `enumeration_representation_clause`.

Examples

- 12 *Example of an enumeration representation clause:*

```
13     type Mix_Code is (ADD, SUB, MUL, LDA, STA, STZ);
14     for Mix_Code use
        (ADD => 1, SUB => 2, MUL => 3, LDA => 8, STA => 24, STZ => 33);
```

13.5 Record Layout

- 1 The (*record*) *layout* aspect of representation consists of the *storage places* for some or all components, that is, storage place attributes of the components. The layout can be specified with a `record_representation_clause`.

13.5.1 Record Representation Clauses

- 1 A `record_representation_clause` specifies the storage representation of records and record extensions, that is, the order, position, and size of components (including discriminants, if any).

Syntax

```
2     record_representation_clause ::=
        for first_subtype_local_name use
            record [mod_clause]
                {component_clause}
            end record;
3     component_clause ::=
        component_local_name at position range first_bit .. last_bit;
4     position ::= static_expression
5     first_bit ::= static_simple_expression
6     last_bit ::= static_simple_expression
```

Name Resolution Rules

- 7 Each position, `first_bit`, and `last_bit` is expected to be of any integer type.

Legality Rules

- 8/2 The *first_subtype_local_name* of a `record_representation_clause` shall denote a specific record or record extension subtype.
- 9 If the *component_local_name* is a `direct_name`, the *local_name* shall denote a component of the type. For a record extension, the component shall not be inherited, and shall not be a discriminant that corresponds

to a discriminant of the parent type. If the *component_local_name* has an *attribute_designator*, the *direct_name* of the *local_name* shall denote either the declaration of the type or a component of the type, and the *attribute_designator* shall denote an implementation-defined implicit component of the type.

The *position*, *first_bit*, and *last_bit* shall be static expressions. The value of *position* and *first_bit* shall be nonnegative. The value of *last_bit* shall be no less than *first_bit* − 1. 10

If the nondefault bit ordering applies to the type, then either: 10.1/2

- the value of *last_bit* shall be less than the size of the largest machine scalar; or 10.2/2
- the value of *first_bit* shall be zero and the value of *last_bit* + 1 shall be a multiple of *System.Storage_Unit*. 10.3/2

At most one *component_clause* is allowed for each component of the type, including for each discriminant (*component_clauses* may be given for some, all, or none of the components). Storage places within a *component_list* shall not overlap, unless they are for components in distinct variants of the same *variant_part*. 11

A name that denotes a component of a type is not allowed within a *record_representation_clause* for the type, except as the *component_local_name* of a *component_clause*. 12

Static Semantics

A *record_representation_clause* (without the *mod_clause*) specifies the layout. 13/2

If the default bit ordering applies to the type, the *position*, *first_bit*, and *last_bit* of each *component_clause* directly specify the position and size of the corresponding component. 13.1/2

If the nondefault bit ordering applies to the type then the layout is determined as follows: 13.2/2

- the *component_clauses* for which the value of *last_bit* is greater than or equal to the size of the largest machine scalar directly specify the position and size of the corresponding component; 13.3/2
- for other *component_clauses*, all of the components having the same value of *position* are considered to be part of a single machine scalar, located at that *position*; this machine scalar has a size which is the smallest machine scalar size larger than the largest *last_bit* for all *component_clauses* at that *position*; the *first_bit* and *last_bit* of each *component_clause* are then interpreted as bit offsets in this machine scalar. 13.4/2

A *record_representation_clause* for a record extension does not override the layout of the parent part; if the layout was specified for the parent type, it is inherited by the record extension. 14

Implementation Permissions

An implementation may generate implementation-defined components (for example, one containing the offset of another component). An implementation may generate names that denote such implementation-defined components; such names shall be implementation-defined *attribute_references*. An implementation may allow such implementation-defined names to be used in *record_representation_clauses*. An implementation can restrict such *component_clauses* in any manner it sees fit. 15

If a *record_representation_clause* is given for an untagged derived type, the storage place attributes for all of the components of the derived type may differ from those of the corresponding components of the parent type, even for components whose storage place is not specified explicitly in the *record_representation_clause*. 16

Implementation Advice

The recommended level of support for *record_representation_clauses* is: 17

- 17.1/2 • An implementation should support machine scalars that correspond to all of the integer, floating point, and address formats supported by the machine.
- 18 • An implementation should support storage places that can be extracted with a load, mask, shift sequence of machine code, and set with a load, shift, mask, store sequence, given the available machine instructions and run-time model.
- 19 • A storage place should be supported if its size is equal to the Size of the component subtype, and it starts and ends on a boundary that obeys the Alignment of the component subtype.
- 20/2 • For a component with a subtype whose Size is less than the word size, any storage place that does not cross an aligned word boundary should be supported.
- 21 • An implementation may reserve a storage place for the tag field of a tagged type, and disallow other components from overlapping that place.
- 22 • An implementation need not support a `component_clause` for a component of an extension part if the storage place is not after the storage places of all components of the parent type, whether or not those storage places had been specified.

NOTES

- 23 11 If no `component_clause` is given for a component, then the choice of the storage place for the component is left to the implementation. If `component_clauses` are given for all components, the `record_representation_clause` completely specifies the representation of the type and will be obeyed exactly by the implementation.

*Examples*24 *Example of specifying the layout of a record type:*

```

25   Word : constant := 4;  -- storage element is byte, 4 bytes per word
26   type State      is (A,M,W,P);
27   type Mode       is (Fix, Dec, Exp, Signif);
28   type Byte_Mask  is array (0..7) of Boolean;
29   type State_Mask is array (State) of Boolean;
30   type Mode_Mask  is array (Mode) of Boolean;
31
32   type Program_Status_Word is
33     record
34       System_Mask      : Byte_Mask;
35       Protection_Key   : Integer range 0 .. 3;
36       Machine_State    : State_Mask;
37       Interrupt_Cause  : Interruption_Code;
38       Ilc              : Integer range 0 .. 3;
39       Cc               : Integer range 0 .. 3;
40       Program_Mask     : Mode_Mask;
41       Inst_Address     : Address;
42     end record;
43
44   for Program_Status_Word use
45     record
46       System_Mask      at 0*Word range 0 .. 7;
47       Protection_Key   at 0*Word range 10 .. 11;  -- bits 8,9 unused
48       Machine_State    at 0*Word range 12 .. 15;
49       Interrupt_Cause  at 0*Word range 16 .. 31;
50       Ilc              at 1*Word range 0 .. 1;  -- second word
51       Cc               at 1*Word range 2 .. 3;
52       Program_Mask     at 1*Word range 4 .. 7;
53       Inst_Address     at 1*Word range 8 .. 31;
54     end record;
55
56   for Program_Status_Word'Size use 8*System.Storage_Unit;
57   for Program_Status_Word'Alignment use 8;

```

NOTES

12 *Note on the example:* The `record_representation_clause` defines the record layout. The Size clause guarantees that (at least) eight storage elements are used for objects of the type. The Alignment clause guarantees that aliased, imported, or exported objects of the type will have addresses divisible by eight. 31

13.5.2 Storage Place Attributes

Static Semantics

For a component C of a composite, non-array object R, the *storage place attributes* are defined: 1

R.C'Position If the nondefault bit ordering applies to the composite type, and if a `component_clause` specifies the placement of C, denotes the value given for the `position` of the `component_clause`; otherwise, denotes the same value as R.C'Address – R'Address. The value of this attribute is of the type *universal_integer*. 2/2

R.C'First_Bit If the nondefault bit ordering applies to the composite type, and if a `component_clause` specifies the placement of C, denotes the value given for the `first_bit` of the `component_clause`; otherwise, denotes the offset, from the start of the first of the storage elements occupied by C, of the first bit occupied by C. This offset is measured in bits. The first bit of a storage element is numbered zero. The value of this attribute is of the type *universal_integer*. 3/2

R.C'Last_Bit If the nondefault bit ordering applies to the composite type, and if a `component_clause` specifies the placement of C, denotes the value given for the `last_bit` of the `component_clause`; otherwise, denotes the offset, from the start of the first of the storage elements occupied by C, of the last bit occupied by C. This offset is measured in bits. The value of this attribute is of the type *universal_integer*. 4/2

Implementation Advice

If a component is represented using some form of pointer (such as an offset) to the actual data of the component, and this data is contiguous with the rest of the object, then the storage place attributes should reflect the place of the actual data, not the pointer. If a component is allocated discontinuously from the rest of the object, then a warning should be generated upon reference to one of its storage place attributes. 5

13.5.3 Bit Ordering

The `Bit_Order` attribute specifies the interpretation of the storage place attributes. 1

Static Semantics

A bit ordering is a method of interpreting the meaning of the storage place attributes. `High_Order_First` (known in the vernacular as “big endian”) means that the first bit of a storage element (bit 0) is the most significant bit (interpreting the sequence of bits that represent a component as an unsigned integer value). `Low_Order_First` (known in the vernacular as “little endian”) means the opposite: the first bit is the least significant. 2

For every specific record subtype S, the following attribute is defined: 3

S'Bit_Order Denotes the bit ordering for the type of S. The value of this attribute is of type `System.Bit_Order`. `Bit_Order` may be specified for specific record types via an `attribute_definition_clause`; the expression of such a clause shall be static. 4

- 5 If `Word_Size = Storage_Unit`, the default bit ordering is implementation defined. If `Word_Size > Storage_Unit`, the default bit ordering is the same as the ordering of storage elements in a word, when interpreted as an integer.
- 6 The storage place attributes of a component of a type are interpreted according to the bit ordering of the type.

Implementation Advice

- 7 The recommended level of support for the nondefault bit ordering is:
- 8/2
- The implementation should support the nondefault bit ordering in addition to the default bit ordering.

NOTES

- 9/2 13 `Bit_Order` clauses make it possible to write `record_representation_clauses` that can be ported between machines having different bit ordering. They do not guarantee transparent exchange of data between such machines.

13.6 Change of Representation

A `type_conversion` (see 4.6) can be used to convert between two different representations of the same array or record. To convert an array from one representation to another, two array types need to be declared with matching component subtypes, and convertible index types. If one type has packing specified and the other does not, then explicit conversion can be used to pack or unpack an array.

To convert a record from one representation to another, two record types with a common ancestor type need to be declared, with no inherited subprograms. Distinct representations can then be specified for the record types, and explicit conversion between the types can be used to effect a change in representation.

Examples

Example of change of representation:

-- Packed_Descriptor and Descriptor are two different types
 -- with identical characteristics, apart from their
 -- representation

```
type Descriptor is
  record
    -- components of a descriptor
  end record;
```

```
type Packed_Descriptor is new Descriptor;
```

```
for Packed_Descriptor use
  record
    -- component clauses for some or for all components
  end record;
```

-- Change of representation can now be accomplished by explicit type conversions:

```
D : Descriptor;
```

```
P : Packed_Descriptor;
```

```
P := Packed_Descriptor(D); -- pack D
```

```
D := Descriptor(P); -- unpack P
```

13.7 The Package System

- 1 For each implementation there is a library package called System which includes the definitions of certain configuration-dependent characteristics.

Static Semantics

- 2 The following language-defined library package exists:

```

3/2  package System is
      pragma Pure(System);
4      type Name is implementation-defined-enumeration-type;
      System_Name : constant Name := implementation-defined;
5      -- System-Dependent Named Numbers:
6      Min_Int      : constant := root_integer'First;
      Max_Int      : constant := root_integer'Last;
7      Max_Binary_Modulus : constant := implementation-defined;
      Max_Nonbinary_Modulus : constant := implementation-defined;
8      Max_Base_Digits : constant := root_real'Digits;
      Max_Digits : constant := implementation-defined;
9      Max_Mantissa : constant := implementation-defined;
      Fine_Delta : constant := implementation-defined;
10     Tick : constant := implementation-defined;
11     -- Storage-related Declarations:
12     type Address is implementation-defined;
      Null_Address : constant Address;
13     Storage_Unit : constant := implementation-defined;
      Word_Size : constant := implementation-defined * Storage_Unit;
      Memory_Size : constant := implementation-defined;
14     -- Address Comparison:
      function "<" (Left, Right : Address) return Boolean;
      function "<=" (Left, Right : Address) return Boolean;
      function ">" (Left, Right : Address) return Boolean;
      function ">=" (Left, Right : Address) return Boolean;
      function "=" (Left, Right : Address) return Boolean;
      -- function "/=" (Left, Right : Address) return Boolean;
      -- "/=" is implicitly defined
      pragma Convention(Intrinsic, "<");
      ... -- and so on for all language-defined subprograms in this package
15/2  -- Other System-Dependent Declarations:
      type Bit_Order is (High_Order_First, Low_Order_First);
      Default_Bit_Order : constant Bit_Order := implementation-defined;
16     -- Priority-related declarations (see D.1):
      subtype Any_Priority is Integer range implementation-defined;
      subtype Priority is Any_Priority range Any_Priority'First ..
          implementation-defined;
      subtype Interrupt_Priority is Any_Priority range Priority'Last+1 ..
          Any_Priority'Last;
17     Default_Priority : constant Priority :=
          (Priority'First + Priority'Last) / 2;
18     private
      ... -- not specified by the language
      end System;

```

- 19 Name is an enumeration subtype. Values of type Name are the names of alternative machine configurations handled by the implementation. System_Name represents the current machine configuration.

The named numbers <code>Fine_Delta</code> and <code>Tick</code> are of the type <i>universal_real</i> ; the others are of the type <i>universal_integer</i> .	20
The meanings of the named numbers are:	21
<code>Min_Int</code> The smallest (most negative) value allowed for the expressions of a <i>signed_integer_type_definition</i> .	22
<code>Max_Int</code> The largest (most positive) value allowed for the expressions of a <i>signed_integer_type_definition</i> .	23
<code>Max_Binary_Modulus</code> A power of two such that it, and all lesser positive powers of two, are allowed as the modulus of a <i>modular_type_definition</i> .	24
<code>Max_Nonbinary_Modulus</code> A value such that it, and all lesser positive integers, are allowed as the modulus of a <i>modular_type_definition</i> .	25
<code>Max_Base_Digits</code> The largest value allowed for the requested decimal precision in a <i>floating_point_definition</i> .	26
<code>Max_Digits</code> The largest value allowed for the requested decimal precision in a <i>floating_point_definition</i> that has no <i>real_range_specification</i> . <code>Max_Digits</code> is less than or equal to <code>Max_Base_Digits</code> .	27
<code>Max_Mantissa</code> The largest possible number of binary digits in the mantissa of machine numbers of a user-defined ordinary fixed point type. (The mantissa is defined in Annex G.)	28
<code>Fine_Delta</code> The smallest delta allowed in an <i>ordinary_fixed_point_definition</i> that has the <i>real_range_specification</i> range <code>-1.0 .. 1.0</code> .	29
<code>Tick</code> A period in seconds approximating the real time interval during which the value of <code>Calendar.Clock</code> remains constant.	30
<code>Storage_Unit</code> The number of bits per storage element.	31
<code>Word_Size</code> The number of bits per word.	32
<code>Memory_Size</code> An implementation-defined value that is intended to reflect the memory size of the configuration in storage elements.	33
<code>Address</code> is a definite, nonlimited type with preelaborable initialization (see 10.2.1). <code>Address</code> represents machine addresses capable of addressing individual storage elements. <code>Null_Address</code> is an address that is distinct from the address of any object or program unit.	34/2
<code>Default_Bit_Order</code> shall be a static constant. See 13.5.3 for an explanation of <code>Bit_Order</code> and <code>Default_Bit_Order</code> .	35/2

Implementation Permissions

An implementation may add additional implementation-defined declarations to package <code>System</code> and its children. However, it is usually better for the implementation to provide additional functionality via implementation-defined children of <code>System</code> .	36/2
---	------

Implementation Advice

<code>Address</code> should be a private type.	37
--	----

NOTES

38 14 There are also some language-defined child packages of System defined elsewhere.

13.7.1 The Package System.Storage_Elements

Static Semantics

1 The following language-defined library package exists:

```

2/2  package System.Storage_Elements is
      pragma Pure (Storage_Elements);
3      type Storage_Offset is range implementation-defined;
4      subtype Storage_Count is Storage_Offset range 0..Storage_Offset'Last;
5      type Storage_Element is mod implementation-defined;
      for Storage_Element'Size use Storage_Unit;
      type Storage_Array is array
        (Storage_Offset range <>) of aliased Storage_Element;
      for Storage_Array'Component_Size use Storage_Unit;
6      -- Address Arithmetic:
7      function "+" (Left : Address; Right : Storage_Offset)
        return Address;
      function "+" (Left : Storage_Offset; Right : Address)
        return Address;
      function "-" (Left : Address; Right : Storage_Offset)
        return Address;
      function "-" (Left, Right : Address)
        return Storage_Offset;
8      function "mod" (Left : Address; Right : Storage_Offset)
        return Storage_Offset;
9      -- Conversion to/from integers:
10     type Integer_Address is implementation-defined;
      function To_Address (Value : Integer_Address) return Address;
      function To_Integer (Value : Address) return Integer_Address;
11     pragma Convention (Intrinsic, "+");
      -- ...and so on for all language-defined subprograms declared in this package.
end System.Storage_Elements;
```

12 Storage_Element represents a storage element. Storage_Offset represents an offset in storage elements. Storage_Count represents a number of storage elements. Storage_Array represents a contiguous sequence of storage elements.

13 Integer_Address is a (signed or modular) integer subtype. To_Address and To_Integer convert back and forth between this type and Address.

Implementation Requirements

14 Storage_Offset'Last shall be greater than or equal to Integer'Last or the largest possible storage offset, whichever is smaller. Storage_Offset'First shall be $\leq (-\text{Storage_Offset'Last})$.

Implementation Permissions

15/2 This paragraph was deleted.

Implementation Advice

16 Operations in System and its children should reflect the target environment semantics as closely as is reasonable. For example, on most machines, it makes sense for address arithmetic to “wrap around.” Operations that do not make sense should raise Program_Error.

13.7.2 The Package System.Address_To_Access_Conversions

Static Semantics

The following language-defined generic library package exists:

```

generic
  type Object(<>) is limited private;
package System.Address_To_Access_Conversions is
  pragma Preelaborate(Address_To_Access_Conversions);
  type Object_Pointer is access all Object;
  function To_Pointer(Value : Address) return Object_Pointer;
  function To_Address(Value : Object_Pointer) return Address;

  pragma Convention(Intrinsic, To_Pointer);
  pragma Convention(Intrinsic, To_Address);
end System.Address_To_Access_Conversions;

```

The To_Pointer and To_Address subprograms convert back and forth between values of types Object_Pointer and Address. To_Pointer(X'Address) is equal to X'Unchecked_Access for any X that allows Unchecked_Access. To_Pointer(Null_Address) returns **null**. For other addresses, the behavior is unspecified. To_Address(**null**) returns Null_Address. To_Address(Y), where Y /= **null**, returns Y.all'Address.

Implementation Permissions

An implementation may place restrictions on instantiations of Address_To_Access_Conversions.

13.8 Machine Code Insertions

A machine code insertion can be achieved by a call to a subprogram whose sequence_of_statements contains code_statements.

Syntax

```
code_statement ::= qualified_expression;
```

A code_statement is only allowed in the handled_sequence_of_statements of a subprogram_body. If a subprogram_body contains any code_statements, then within this subprogram_body the only allowed form of statement is a code_statement (labeled or not), the only allowed declarative_items are use_clauses, and no exception_handler is allowed (comments and pragmas are allowed as usual).

Name Resolution Rules

The qualified_expression is expected to be of any type.

Legality Rules

The qualified_expression shall be of a type declared in package System.Machine_Code.

A code_statement shall appear only within the scope of a with_clause that mentions package System.Machine_Code.

Static Semantics

The contents of the library package System.Machine_Code (if provided) are implementation defined. The meaning of code_statements is implementation defined. Typically, each qualified_expression represents a machine instruction or assembly directive.

Implementation Permissions

- 8 An implementation may place restrictions on `code_statements`. An implementation is not required to provide package `System.Machine_Code`.

NOTES

- 9 15 An implementation may provide implementation-defined pragmas specifying register conventions and calling conventions.
- 10/2 16 Machine code functions are exempt from the rule that a return statement is required. In fact, return statements are forbidden, since only `code_statements` are allowed.
- 11 17 Intrinsic subprograms (see 6.3.1, “Conformance Rules”) can also be used to achieve machine code insertions. Interface to assembly language can be achieved using the features in Annex B, “Interface to Other Languages”.

Examples

- 12 *Example of a code statement:*

```

13 M : Mask;
14 procedure Set_Mask; pragma Inline(Set_Mask);
15 procedure Set_Mask is
16   use System.Machine_Code; -- assume “with System.Machine_Code;” appears somewhere above
17 begin
18   SI_Format'(Code => SSM, B => M'Base_Reg, D => M'Disp);
19   -- Base_Reg and Disp are implementation-defined attributes
20 end Set_Mask;
```

13.9 Unchecked Type Conversions

- 1 An unchecked type conversion can be achieved by a call to an instance of the generic function `Unchecked_Conversion`.

Static Semantics

- 2 The following language-defined generic library function exists:

```

3 generic
4   type Source(<>) is limited private;
5   type Target(<>) is limited private;
6   function Ada.Unchecked_Conversion(S : Source) return Target;
7 pragma Convention(Intrinsic, Ada.Unchecked_Conversion);
8 pragma Pure(Ada.Unchecked_Conversion);
```

Dynamic Semantics

- 4 The size of the formal parameter S in an instance of `Unchecked_Conversion` is that of its subtype. This is the actual subtype passed to Source, except when the actual is an unconstrained composite subtype, in which case the subtype is constrained by the bounds or discriminants of the value of the actual expression passed to S.
- 5 If all of the following are true, the effect of an unchecked conversion is to return the value of an object of the target subtype whose representation is the same as that of the source object S:
- 6 • S'Size = Target'Size.
 - 7/3 • S'Alignment is a multiple of Target'Alignment or Target'Alignment is zero.
 - 8 • The target subtype is not an unconstrained composite subtype.
 - 9 • S and the target subtype both have a contiguous representation.
 - 10 • The representation of S is a representation of an object of the target subtype.

Otherwise, if the result type is scalar, the result of the function is implementation defined, and can have an invalid representation (see 13.9.1). If the result type is nonscalar, the effect is implementation defined; in particular, the result can be abnormal (see 13.9.1). 11/2

Implementation Permissions

An implementation may return the result of an unchecked conversion by reference, if the Source type is not a by-copy type. In this case, the result of the unchecked conversion represents simply a different (read-only) view of the operand of the conversion. 12

An implementation may place restrictions on Unchecked_Conversion. 13

Implementation Advice

Since the Size of an array object generally does not include its bounds, the bounds should not be part of the converted data. 14/2

The implementation should not generate unnecessary run-time checks to ensure that the representation of S is a representation of the target type. It should take advantage of the permission to return by reference when possible. Restrictions on unchecked conversions should be avoided unless required by the target environment. 15

The recommended level of support for unchecked conversions is: 16

- Unchecked conversions should be supported and should be reversible in the cases where this clause defines the result. To enable meaningful use of unchecked conversion, a contiguous representation should be used for elementary subtypes, for statically constrained array subtypes whose component subtype is one of the subtypes described in this paragraph, and for record subtypes without discriminants whose component subtypes are described in this paragraph. 17

13.9.1 Data Validity

Certain actions that can potentially lead to erroneous execution are not directly erroneous, but instead can cause objects to become *abnormal*. Subsequent uses of abnormal objects can be erroneous. 1

A scalar object can have an *invalid representation*, which means that the object's representation does not represent any value of the object's subtype. The primary cause of invalid representations is uninitialized variables. 2

Abnormal objects and invalid representations are explained in this subclause. 3

Dynamic Semantics

When an object is first created, and any explicit or default initializations have been performed, the object and all of its parts are in the *normal* state. Subsequent operations generally leave them normal. However, an object or part of an object can become *abnormal* in the following ways: 4

- An assignment to the object is disrupted due to an abort (see 9.8) or due to the failure of a language-defined check (see 11.6). 5
- The object is not scalar, and is passed to an **in out** or **out** parameter of an imported procedure, the Read procedure of an instance of Sequential_IO, Direct_IO, or Storage_IO, or the stream attribute T'Read, if after return from the procedure the representation of the parameter does not represent a value of the parameter's subtype. 6/2
- The object is the return object of a function call of a nonscalar type, and the function is an imported function, an instance of Unchecked_Conversion, or the stream attribute T'Input, if after 6.1/2

return from the function the representation of the return object does not represent a value of the function's subtype.

6.2/2 For an imported object, it is the programmer's responsibility to ensure that the object remains in a normal state.

7 Whether or not an object actually becomes abnormal in these cases is not specified. An abnormal object becomes normal again upon successful completion of an assignment to the object as a whole.

Erroneous Execution

8 It is erroneous to evaluate a **primary** that is a **name** denoting an abnormal object, or to evaluate a **prefix** that denotes an abnormal object.

Bounded (Run-Time) Errors

9 If the representation of a scalar object does not represent a value of the object's subtype (perhaps because the object was not initialized), the object is said to have an *invalid representation*. It is a bounded error to evaluate the value of such an object. If the error is detected, either `Constraint_Error` or `Program_Error` is raised. Otherwise, execution continues using the invalid representation. The rules of the language outside this subclause assume that all objects have valid representations. The semantics of operations on invalid representations are as follows:

- 10 • If the representation of the object represents a value of the object's type, the value of the type is used.
- 11 • If the representation of the object does not represent a value of the object's type, the semantics of operations on such representations is implementation-defined, but does not by itself lead to erroneous or unpredictable execution, or to other objects becoming abnormal.

Erroneous Execution

12/2 A call to an imported function or an instance of `Unchecked_Conversion` is erroneous if the result is scalar, the result object has an invalid representation, and the result is used other than as the **expression** of an **assignment_statement** or an **object_declaration**, or as the **prefix** of a **Valid** attribute. If such a result object is used as the source of an assignment, and the assigned value is an invalid representation for the target of the assignment, then any use of the target object prior to a further assignment to the target object, other than as the **prefix** of a **Valid** attribute reference, is erroneous.

13/3 The dereference of an access value is erroneous if it does not designate an object of an appropriate type or a subprogram with an appropriate profile, if it designates a nonexistent object, or if it is an access-to-variable value that designates a constant object and it did not originate from an **attribute_reference** applied to an aliased variable view of a controlled or immutably limited object. An access value whose dereference is erroneous can exist, for example, because of `Unchecked_Deallocation`, `Unchecked_Access`, or `Unchecked_Conversion`.

NOTES

14 18 Objects can become abnormal due to other kinds of actions that directly update the object's representation; such actions are generally considered directly erroneous, however.

13.9.2 The Valid Attribute

1 The **Valid** attribute can be used to check the validity of data produced by unchecked conversion, input, interface to foreign languages, and the like.

Static Semantics

For a prefix `X` that denotes a scalar object (after any implicit dereference), the following attribute is defined: 2

`X'Valid` Yields True if and only if the object denoted by `X` is normal and has a valid representation. 3
The value of this attribute is of the predefined type Boolean.

NOTES

19 Invalid data can be created in the following cases (not counting erroneous or unpredictable execution): 4

- an uninitialized scalar object, 5
- the result of an unchecked conversion, 6
- input, 7
- interface to another language (including machine code), 8
- aborting an assignment, 9
- disrupting an assignment due to the failure of a language-defined check (see 11.6), and 10
- use of an object whose Address has been specified. 11

20 `X'Valid` is not considered to be a read of `X`; hence, it is not an error to check the validity of invalid data. 12

21 The `Valid` attribute may be used to check the result of calling an instance of `Unchecked_Conversion` (or any other operation that can return invalid values). However, an exception handler should also be provided because implementations are permitted to raise `Constraint_Error` or `Program_Error` if they detect the use of an invalid representation (see 13.9.1). 13/2

13.10 Unchecked Access Value Creation

The attribute `Unchecked_Access` is used to create access values in an unsafe manner — the programmer is responsible for preventing “dangling references.” 1

Static Semantics

The following attribute is defined for a prefix `X` that denotes an aliased view of an object: 2

`X'Unchecked_Access` 3
All rules and semantics that apply to `X'Access` (see 3.10.2) apply also to `X'Unchecked_Access`, except that, for the purposes of accessibility rules and checks, it is as if `X` were declared immediately within a library package.

NOTES

22 This attribute is provided to support the situation where a local object is to be inserted into a global linked data structure, when the programmer knows that it will always be removed from the data structure prior to exiting the object's scope. The `Access` attribute would be illegal in this case (see 3.10.2, “Operations of Access Types”). 4

23 There is no `Unchecked_Access` attribute for subprograms. 5

13.11 Storage Management

Each access-to-object type has an associated storage pool. The storage allocated by an `allocator` comes from the pool; instances of `Unchecked_Deallocation` return storage to the pool. Several access types can share the same pool. 1

A storage pool is a variable of a type in the class rooted at `Root_Storage_Pool`, which is an abstract limited controlled type. By default, the implementation chooses a *standard storage pool* for each access-to-object type. The user may define new pool types, and may override the choice of pool for an access-to-object type by specifying `Storage_Pool` for the type. 2/2

Legality Rules

- 3 If `Storage_Pool` is specified for a given access type, `Storage_Size` shall not be specified for it.

Static Semantics

- 4 The following language-defined library package exists:

```

5   with Ada.Finalization;
   with System.Storage_Elements;
   package System.Storage_Pools is
       pragma Preelaborate(System.Storage_Pools);
6/2   type Root_Storage_Pool is
       abstract new Ada.Finalization.Limited_Controlled with private;
       pragma Preelaborable_Initialization(Root_Storage_Pool);
7   procedure Allocate(
       Pool : in out Root_Storage_Pool;
       Storage_Address : out Address;
       Size_In_Storage_Elements : in Storage_Elements.Storage_Count;
       Alignment : in Storage_Elements.Storage_Count) is abstract;
8   procedure Deallocate(
       Pool : in out Root_Storage_Pool;
       Storage_Address : in Address;
       Size_In_Storage_Elements : in Storage_Elements.Storage_Count;
       Alignment : in Storage_Elements.Storage_Count) is abstract;
9   function Storage_Size(Pool : Root_Storage_Pool)
       return Storage_Elements.Storage_Count is abstract;
10  private
       ... -- not specified by the language
       end System.Storage_Pools;
```

- 11 A *storage pool type* (or *pool type*) is a descendant of `Root_Storage_Pool`. The *elements* of a storage pool are the objects allocated in the pool by allocators.

- 12/2 For every access-to-object subtype *S*, the following representation attributes are defined:

13 `S'Storage_Pool`

Denotes the storage pool of the type of *S*. The type of this attribute is `Root_Storage_Pool'Class`.

14 `S'Storage_Size`

Yields the result of calling `Storage_Size(S'Storage_Pool)`, which is intended to be a measure of the number of storage elements reserved for the pool. The type of this attribute is *universal_integer*.

- 15 `Storage_Size` or `Storage_Pool` may be specified for a non-derived access-to-object type via an `attribute_definition_clause`; the name in a `Storage_Pool` clause shall denote a variable.

- 16/3 An allocator of type *T* allocates storage from *T*'s storage pool. If the storage pool is a user-defined object, then the storage is allocated by calling `Allocate` as described below.

- 17 If `Storage_Pool` is not specified for a type defined by an `access_to_object_definition`, then the implementation chooses a standard storage pool for it in an implementation-defined manner. In this case, the exception `Storage_Error` is raised by an allocator if there is not enough storage. It is implementation defined whether or not the implementation provides user-accessible names for the standard pool type(s).

- 18 If `Storage_Size` is specified for an access type, then the `Storage_Size` of this pool is at least that requested, and the storage for the pool is reclaimed when the master containing the declaration of the access type is left. If the implementation cannot satisfy the request, `Storage_Error` is raised at the point of the `attribute_-`

definition_clause. If neither `Storage_Pool` nor `Storage_Size` are specified, then the meaning of `Storage_Size` is implementation defined.

If `Storage_Pool` is specified for an access type, then the specified pool is used. 19

The effect of calling `Allocate` and `Deallocate` for a standard storage pool directly (rather than implicitly via an `allocator` or an instance of `Unchecked_Deallocation`) is unspecified. 20

Erroneous Execution

If `Storage_Pool` is specified for an access type, then if `Allocate` can satisfy the request, it should allocate a contiguous block of memory, and return the address of the first storage element in `Storage_Address`. The block should contain `Size_In_Storage_Elements` storage elements, and should be aligned according to `Alignment`. The allocated storage should not be used for any other purpose while the pool element remains in existence. If the request cannot be satisfied, then `Allocate` should propagate an exception (such as `Storage_Error`). If `Allocate` behaves in any other manner, then the program execution is erroneous. 21

Implementation Requirements

The `Allocate` procedure of a user-defined storage pool object *P* may be called by the implementation only to allocate storage for a type *T* whose pool is *P* and: 21.1/3

- During the execution of an `allocator` of type *T*; 21.2/3
- During the execution of a return statement for a function whose result is built-in-place in the result of an `allocator` of type *T*; 21.3/3
- During the execution of an assignment operation with a target of an allocated object of type *T* with a part that has an unconstrained discriminated subtype with defaults. 21.4/3

For one of the calls of `Allocate` described above, *P* (equivalent to *T*`Storage_Pool`) is passed as the `Pool` parameter. The `Size_In_Storage_Elements` parameter indicates the number of storage elements to be allocated, and is no more than *D*`Max_Size_In_Storage_Elements`, where *D* is the designated subtype of *T*. The `Alignment` parameter is at least *D*`Alignment` if *D* is a specific type, and otherwise is at least the alignment of the specific type identified by the tag of the object being created. The `Alignment` parameter is no more than *D*`Max_Alignment_For_Allocation`. The result returned in the `Storage_Address` parameter is used as the address of the allocated storage, which is a contiguous block of memory of `Size_In_Storage_Elements` storage elements. Any exception propagated by `Allocate` is propagated by the construct that contained the call. 21.5/3

The number of calls to `Allocate` needed to implement an `allocator` for any particular type is unspecified. 21.6/3
The number of calls to `Deallocate` needed to implement an instance of `Unchecked_Deallocation` (see 13.11.2) for any particular object is the same as the number of `Allocate` calls for that object.

The `Deallocate` procedure of a user-defined storage pool object *P* may be called by the implementation to deallocate storage for a type *T* whose pool is *P* only at the places when an `Allocate` call is allowed for *P*, during the execution of an instance of `Unchecked_Deallocation` for *T*, or as part of the finalization of the collection of *T*. For such a call of `Deallocate`, *P* (equivalent to *T*`Storage_Pool`) is passed as the `Pool` parameter. The value of the `Storage_Address` parameter for a call to `Deallocate` is the value returned in the `Storage_Address` parameter of the corresponding successful call to `Allocate`. The values of the `Size_In_Storage_Elements` and `Alignment` parameters are the same values passed to the corresponding `Allocate` call. Any exception propagated by `Deallocate` is propagated by the construct that contained the call. 21.7/3

Documentation Requirements

- 22 An implementation shall document the set of values that a user-defined Allocate procedure needs to accept for the Alignment parameter. An implementation shall document how the standard storage pool is chosen, and how storage is allocated by standard storage pools.

Implementation Advice

- 23 An implementation should document any cases in which it dynamically allocates heap storage for a purpose other than the evaluation of an allocator.
- 24 A default (implementation-provided) storage pool for an access-to-constant type should not have overhead to support deallocation of individual objects.
- 25/2 The storage pool used for an allocator of an anonymous access type should be determined as follows:
- 25.1/2 • If the allocator is defining a coextension (see 3.10.2) of an object being created by an outer allocator, then the storage pool used for the outer allocator should also be used for the coextension;
- 25.2/2 • For other access discriminants and access parameters, the storage pool should be created at the point of the allocator, and be reclaimed when the allocated object becomes inaccessible;
- 25.3/2 • Otherwise, a default storage pool should be created at the point where the anonymous access type is elaborated; such a storage pool need not support deallocation of individual objects.

NOTES

- 26 24 A user-defined storage pool type can be obtained by extending the Root_Storage_Pool type, and overriding the primitive subprograms Allocate, Deallocate, and Storage_Size. A user-defined storage pool can then be obtained by declaring an object of the type extension. The user can override Initialize and Finalize if there is any need for non-trivial initialization and finalization for a user-defined pool type. For example, Finalize might reclaim blocks of storage that are allocated separately from the pool object itself.
- 27 25 The writer of the user-defined allocation and deallocation procedures, and users of allocators for the associated access type, are responsible for dealing with any interactions with tasking. In particular:
- 28 • If the allocators are used in different tasks, they require mutual exclusion.
- 29 • If they are used inside protected objects, they cannot block.
- 30 • If they are used by interrupt handlers (see C.3, “Interrupt Support”), the mutual exclusion mechanism has to work properly in that context.
- 31 26 The primitives Allocate, Deallocate, and Storage_Size are declared as abstract (see 3.9.3), and therefore they have to be overridden when a new (non-abstract) storage pool type is declared.

Examples

- 32 To associate an access type with a storage pool object, the user first declares a pool object of some type derived from Root_Storage_Pool. Then, the user defines its Storage_Pool attribute, as follows:

```
33   Pool_Object : Some_Storage_Pool_Type;
34   type T is access Designated;
   for T'Storage_Pool use Pool_Object;
```

- 35 Another access type may be added to an existing storage pool, via:

```
36   for T2'Storage_Pool use T'Storage_Pool;
```

- 37 The semantics of this is implementation defined for a standard storage pool.

- 38 As usual, a derivative of Root_Storage_Pool may define additional operations. For example, presuming that Mark_Release_Pool_Type has two additional operations, Mark and Release, the following is a possible use:

```

type Mark_Release_Pool_Type                                     39/1
  (Pool_Size : Storage_Elements.Storage_Count;
   Block_Size : Storage_Elements.Storage_Count)
  is new Root_Storage_Pool with private;

...                                                            40
MR_Pool : Mark_Release_Pool_Type (Pool_Size => 2000,           41
                                   Block_Size => 100);

type Acc is access ...;                                       42
for Acc'Storage_Pool use MR_Pool;
...
Mark(MR_Pool);                                                43
... -- Allocate objects using "new Designated(...)".
Release(MR_Pool); -- Reclaim the storage.

```

13.11.1 Storage Allocation Attributes

The Max_Size_In_Storage_Elements and Max_Alignment_For_Allocation attributes may be useful in writing user-defined pool types. 1/3

Static Semantics

For every subtype S, the following attributes are defined: 2/3

S'Max_Size_In_Storage_Elements 3/2

Denotes the maximum value for Size_In_Storage_Elements that could be requested by the implementation via Allocate for an access type whose designated subtype is S. The value of this attribute is of type *universal_integer*.

S'Max_Alignment_For_Allocation 3.1/3

Denotes the maximum value for Alignment that could be requested by the implementation via Allocate for an access type whose designated subtype is S. The value of this attribute is of type *universal_integer*.

For a type with access discriminants, if the implementation allocates space for a coextension in the same pool as that of the object having the access discriminant, then these attributes account for any calls on Allocate that could be performed to provide space for such coextensions. 3.2/3

13.11.2 Unchecked Storage Deallocation

- 1 Unchecked storage deallocation of an object designated by a value of an access type is achieved by a call to an instance of the generic procedure `Unchecked_Deallocation`.

Static Semantics

- 2 The following language-defined generic library procedure exists:

```
3  generic
    type Object(<>) is limited private;
    type Name is access Object;
    procedure Ada.Unchecked_Deallocation(X : in out Name);
    pragma Convention(Intrinsic, Ada.Unchecked_Deallocation);
    pragma Preelaborate(Ada.Unchecked_Deallocation);
```

Legality Rules

- 3.1/3 A call on an instance of `Unchecked_Deallocation` is illegal if the actual access type of the instance is a type for which the `Storage_Size` has been specified by a static expression with value zero or is defined by the language to be zero. In addition to the places where Legality Rules normally apply (see 12.3), this rule applies also in the private part of an instance of a generic unit.

Dynamic Semantics

- 4 Given an instance of `Unchecked_Deallocation` declared as follows:

```
5  procedure Free is
    new Ada.Unchecked_Deallocation(
        object_subtype_name, access_to_variable_subtype_name);
```

- 6 Procedure `Free` has the following effect:

- 7 1. After executing `Free(X)`, the value of `X` is **null**.
- 8 2. `Free(X)`, when `X` is already equal to **null**, has no effect.
- 9/3 3. `Free(X)`, when `X` is not equal to **null** first performs finalization of the object designated by `X` (and any coextensions of the object — see 3.10.2), as described in 7.6.1. It then deallocates the storage occupied by the object designated by `X` (and any coextensions) If the storage pool is a user-defined object, then the storage is deallocated by calling `Deallocate` as described in 13.11. There is one exception: if the object being freed contains tasks, the object might not be deallocated.
- 10/2 After `Free(X)`, the object designated by `X`, and any subcomponents (and coextensions) thereof, no longer exist; their storage can be reused for other purposes.

Bounded (Run-Time) Errors

- 11 It is a bounded error to free a discriminated, unterminated task object. The possible consequences are:
- 12 • No exception is raised.
 - 13 • `Program_Error` or `Tasking_Error` is raised at the point of the deallocation.
 - 14 • `Program_Error` or `Tasking_Error` is raised in the task the next time it references any of the discriminants.
- 15 In the first two cases, the storage for the discriminants (and for any enclosing object if it is designated by an access discriminant of the task) is not reclaimed prior to task termination.

Erroneous Execution

Evaluating a name that denotes a nonexistent object or a protected subprogram whose associated object is nonexistent is erroneous. The execution of a call to an instance of `Unchecked_Deallocation` is erroneous if the object was created other than by an `allocator` for an access type whose pool is `Name'Storage_Pool`. 16/3

Implementation Advice

For a standard storage pool, `Free` should actually reclaim the storage. 17

A call on an instance of `Unchecked_Deallocation` with a non-null access value should raise `Program_Error` if the actual access type of the instance is a type for which the `Storage_Size` has been specified to be zero or is defined by the language to be zero. 17.1/3

NOTES

27 The rules here that refer to `Free` apply to any instance of `Unchecked_Deallocation`. 18

28 `Unchecked_Deallocation` cannot be instantiated for an access-to-constant type. This is implied by the rules of 12.5.4. 19

13.11.3 Pragma Controlled

`Pragma Controlled` is used to prevent any automatic reclamation of storage (garbage collection) for the objects created by `allocators` of a given access type. 1

Syntax

The form of a `pragma Controlled` is as follows: 2

pragma `Controlled`(*first_subtype_local_name*); 3

Legality Rules

The *first_subtype_local_name* of a `pragma Controlled` shall denote a non-derived access subtype. 4

Static Semantics

A `pragma Controlled` is a representation pragma that specifies the *controlled* aspect of representation. 5

Garbage collection is a process that automatically reclaims storage, or moves objects to a different address, while the objects still exist. 6

If a `pragma Controlled` is specified for an access type with a standard storage pool, then garbage collection is not performed for objects in that pool. 7

Implementation Permissions

An implementation need not support garbage collection, in which case, a `pragma Controlled` has no effect. 8

13.12 Pragma Restrictions

A `pragma Restrictions` expresses the user's intent to abide by certain restrictions. This may facilitate the construction of simpler run-time environments. 1

Syntax

The form of a `pragma Restrictions` is as follows: 2

pragma `Restrictions`(*restriction*{, *restriction*}); 3

restriction ::= *restriction_identifier*
| *restriction_parameter_identifier* => *restriction_parameter_argument* 4/2

4.1/2 `restriction_parameter_argument ::= name | expression`

Name Resolution Rules

5 Unless otherwise specified for a particular restriction, the **expression** is expected to be of any integer type.

Legality Rules

6 Unless otherwise specified for a particular restriction, the **expression** shall be static, and its value shall be nonnegative.

Static Semantics

7/2 The set of restrictions is implementation defined.

Post-Compilation Rules

8/3 A **pragma Restrictions** is a configuration pragma. If a **pragma Restrictions** applies to any compilation unit included in the partition, this may impose either (or both) of two kinds of requirements, as specified for the particular restriction:

- 8.1/3 • A restriction may impose requirements on some or all of the units comprising the partition. Unless otherwise specified for a particular restriction, such a requirement applies to all of the units comprising the partition and is enforced via a post-compilation check.
- 8.2/3 • A restriction may impose requirements on the runtime behavior of the program, as indicated by the specification of runtime behavior associated with a violation of the requirement.
- 8.3/1 For the purpose of checking whether a partition contains constructs that violate any restriction (unless specified otherwise for a particular restriction):
- 8.4/1 • Generic instances are logically expanded at the point of instantiation;
- 8.5/1 • If an object of a type is declared or allocated and not explicitly initialized, then all expressions appearing in the definition for the type and any of its ancestors are presumed to be used;
- 8.6/1 • A **default_expression** for a formal parameter or a generic formal object is considered to be used if and only if the corresponding actual parameter is not provided in a given call or instantiation.

Implementation Permissions

- 9 An implementation may place limitations on the values of the **expression** that are supported, and limitations on the supported combinations of restrictions. The consequences of violating such limitations are implementation defined.
- 9.1/1 An implementation is permitted to omit restriction checks for code that is recognized at compile time to be unreachable and for which no code is generated.
- 9.2/1 Whenever enforcement of a restriction is not required prior to execution, an implementation may nevertheless enforce the restriction prior to execution of a partition to which the restriction applies, provided that every execution of the partition would violate the restriction.

NOTES

- 10/2 29 Restrictions intended to facilitate the construction of efficient tasking run-time systems are defined in D.7. Restrictions intended for use when constructing high integrity systems are defined in H.4.
- 11 30 An implementation has to enforce the restrictions in cases where enforcement is required, even if it chooses not to take advantage of the restrictions in terms of efficiency.

13.12.1 Language-Defined Restrictions

Static Semantics

The following *restriction_identifiers* are language-defined (additional restrictions are defined in the Specialized Needs Annexes): 1/2

No_Implementation_Attributes 2/2
There are no implementation-defined attributes. This restriction applies only to the current compilation or environment, not the entire partition.

No_Implementation_Pragmas 3/2
There are no implementation-defined pragmas or pragma arguments. This restriction applies only to the current compilation or environment, not the entire partition.

No_Obsolescent_Features 4/2
There is no use of language features defined in Annex J. It is implementation-defined if uses of the renamings of J.1 are detected by this restriction. This restriction applies only to the current compilation or environment, not the entire partition.

The following *restriction_parameter_identifier* is language defined: 5/2

No_Dependence 6/2
Specifies a library unit on which there are no semantic dependences.

Legality Rules

The *restriction_parameter_argument* of a **No_Dependence** restriction shall be a *name*; the *name* shall have the form of a full expanded name of a library unit, but need not denote a unit present in the environment. 7/2

Post-Compilation Rules

No compilation unit included in the partition shall depend semantically on the library unit identified by the *name*. 8/2

13.13 Streams

A *stream* is a sequence of elements comprising values from possibly different types and allowing sequential access to these values. A *stream type* is a type in the class whose root type is `Streams.Root_Stream_Type`. A stream type may be implemented in various ways, such as an external sequential file, an internal buffer, or a network channel. 1

13.13.1 The Package Streams

Static Semantics

The abstract type `Root_Stream_Type` is the root type of the class of stream types. The types in this class represent different kinds of streams. A new stream type is defined by extending the root type (or some other stream type), overriding the Read and Write operations, and optionally defining additional primitive subprograms, according to the requirements of the particular kind of stream. The predefined stream-oriented attributes like `T'Read` and `T'Write` make dispatching calls on the Read and Write procedures of the `Root_Stream_Type`. (User-defined `T'Read` and `T'Write` attributes can also make such calls, or can call the Read and Write attributes of other types.) 1

```

2      package Ada.Streams is
          pragma Pure(Streams);
3/2      type Root_Stream_Type is abstract tagged limited private;
          pragma Preelaborable_Initialization(Root_Stream_Type);
4/1      type Stream_Element is mod implementation-defined;
          type Stream_Element_Offset is range implementation-defined;
          subtype Stream_Element_Count is
              Stream_Element_Offset range 0..Stream_Element_Offset'Last;
          type Stream_Element_Array is
              array(Stream_Element_Offset range <>) of aliased Stream_Element;
5      procedure Read(
          Stream : in out Root_Stream_Type;
          Item   : out Stream_Element_Array;
          Last    : out Stream_Element_Offset) is abstract;
6      procedure Write(
          Stream : in out Root_Stream_Type;
          Item   : in Stream_Element_Array) is abstract;
7      private
          ... -- not specified by the language
      end Ada.Streams;

```

8/2 The Read operation transfers stream elements from the specified stream to fill the array Item. Elements are transferred until Item'Length elements have been transferred, or until the end of the stream is reached. If any elements are transferred, the index of the last stream element transferred is returned in Last. Otherwise, Item'First - 1 is returned in Last. Last is less than Item'Last only if the end of the stream is reached.

9 The Write operation appends Item to the specified stream.

Implementation Permissions

9/1/1 If Stream_Element'Size is not a multiple of System.Storage_Unit, then the components of Stream_Element_Array need not be aliased.

NOTES

10 31 See A.12.1, “The Package Streams.Stream_IO” for an example of extending type Root_Stream_Type.

11/2 32 If the end of stream has been reached, and Item'First is Stream_Element_Offset'First, Read will raise Constraint_Error.

13.13.2 Stream-Oriented Attributes

1/1 The operational attributes Write, Read, Output, and Input convert values to a stream of elements and reconstruct values from a stream.

Static Semantics

1.1/2 For every subtype S of an elementary type T, the following representation attribute is defined:

1.2/2 S'Stream_Size

Denotes the number of bits occupied in a stream by items of subtype S. Hence, the number of stream elements required per item of elementary type T is:

1.3/2
$$T'Stream_Size / Ada.Streams.Stream_Element'Size$$

1.4/2 The value of this attribute is of type *universal_integer* and is a multiple of Stream_Element'Size.

1.5/2 Stream_Size may be specified for first subtypes via an *attribute_definition_clause*; the expression of such a clause shall be static, nonnegative, and a multiple of Stream_Element'Size.

Implementation Advice

If not specified, the value of `Stream_Size` for an elementary type should be the number of bits that corresponds to the minimum number of stream elements required by the first subtype of the type, rounded up to the nearest factor or multiple of the word size that is also a multiple of the stream element size. 1.6/2

The recommended level of support for the `Stream_Size` attribute is: 1.7/2

- A `Stream_Size` clause should be supported for a discrete or fixed point type T if the specified `Stream_Size` is a multiple of `Stream_Element'Size` and is no less than the size of the first subtype of T , and no greater than the size of the largest type of the same elementary class (signed integer, modular integer, enumeration, ordinary fixed point, or decimal fixed point). 1.8/2

Static Semantics

2 For every subtype *S* of a specific type *T*, the following attributes are defined.

3 S'Write S'Write denotes a procedure with the following specification:

```
4/2      procedure S'Write(  
          Stream : not null access Ada.Streams.Root_Stream_Type'Class;  
          Item : in T)
```

5 S'Write writes the value of *Item* to *Stream*.

6 S'Read S'Read denotes a procedure with the following specification:

```
7/2      procedure S'Read(  
          Stream : not null access Ada.Streams.Root_Stream_Type'Class;  
          Item : out T)
```

8 S'Read reads the value of *Item* from *Stream*.

8.1/2 For an untagged derived type, the Write (resp. Read) attribute is inherited according to the rules given in 13.1 if the attribute is available for the parent type at the point where *T* is declared. For a tagged derived type, these attributes are not inherited, but rather the default implementations are used.

8.2/2 The default implementations of the Write and Read attributes, where available, execute as follows:

9/2 For elementary types, Read reads (and Write writes) the number of stream elements implied by the Stream_Size for the type *T*; the representation of those stream elements is implementation defined. For composite types, the Write or Read attribute for each component is called in canonical order, which is last dimension varying fastest for an array, and positional aggregate order for a record. Bounds are not included in the stream if *T* is an array type. If *T* is a discriminated type, discriminants are included only if they have defaults. If *T* is a tagged type, the tag is not included. For type extensions, the Write or Read attribute for the parent type is called, followed by the Write or Read attribute of each component of the extension part, in canonical order. For a limited type extension, if the attribute of the parent type or any progenitor type of *T* is available anywhere within the immediate scope of *T*, and the attribute of the parent type or the type of any of the extension components is not available at the freezing point of *T*, then the attribute of *T* shall be directly specified.

9.1/3 If *T* is a discriminated type and its discriminants have defaults then S'Read first reads the discriminants from the stream without modifying *Item*. S'Read then creates an object of type *T* constrained by these discriminants. The value of this object is then converted to the subtype of *Item* and is assigned to *Item*. Finally, the Read attribute for each non-discriminant component of *Item* is called in canonical order as described above. Normal default initialization and finalization take place for the created object.

9.2/2 Constraint_Error is raised by the predefined Write attribute if the value of the elementary item is outside the range of values representable using Stream_Size bits. For a signed integer type, an enumeration type, or a fixed point type, the range is unsigned only if the integer code for the lower bound of the first subtype is nonnegative, and a (symmetric) signed range that covers all values of the first subtype would require more than Stream_Size bits; otherwise the range is signed.

10 For every subtype S'Class of a class-wide type *T*'Class:

11 S'Class'Write

S'Class'Write denotes a procedure with the following specification:

```
12/2      procedure S'Class'Write(  
          Stream : not null access Ada.Streams.Root_Stream_Type'Class;  
          Item : in T'Class)
```

13 Dispatches to the subprogram denoted by the Write attribute of the specific type identified by the tag of *Item*.

S'Class'Read	S'Class'Read denotes a procedure with the following specification:	14
	<pre> procedure S'Class'Read (<i>Stream</i> : not null access Ada.Streams.Root_Stream_Type'Class; <i>Item</i> : out T'Class) </pre>	15/2
	Dispatches to the subprogram denoted by the Read attribute of the specific type identified by the tag of <i>Item</i> .	16
<i>Implementation Advice</i>		
<i>This paragraph was deleted.</i>		
<i>Static Semantics</i>		
	For every subtype <i>S</i> of a specific type <i>T</i> , the following attributes are defined.	18
S'Output	S'Output denotes a procedure with the following specification:	19
	<pre> procedure S'Output (<i>Stream</i> : not null access Ada.Streams.Root_Stream_Type'Class; <i>Item</i> : in T) </pre>	20/2
	S'Output writes the value of <i>Item</i> to <i>Stream</i> , including any bounds or discriminants.	21
S'Input	S'Input denotes a function with the following specification:	22
	<pre> function S'Input (<i>Stream</i> : not null access Ada.Streams.Root_Stream_Type'Class) return T </pre>	23/2
	S'Input reads and returns one value from <i>Stream</i> , using any bounds or discriminants written by a corresponding S'Output to determine how much to read.	24
	For an untagged derived type, the Output (resp. Input) attribute is inherited according to the rules given in 13.1 if the attribute is available for the parent type at the point where <i>T</i> is declared. For a tagged derived type, these attributes are not inherited, but rather the default implementations are used.	25/2
	The default implementations of the Output and Input attributes, where available, execute as follows:	25.1/2
	• If <i>T</i> is an array type, S'Output first writes the bounds, and S'Input first reads the bounds. If <i>T</i> has discriminants without defaults, S'Output first writes the discriminants (using S'Write for each), and S'Input first reads the discriminants (using S'Read for each). • S'Output then calls S'Write to write the value of <i>Item</i> to the stream. S'Input then creates an object of type <i>T</i>, with the bounds or (when without defaults) the discriminants, if any, taken from the stream, passes it to S'Read, and returns the value of the object. If <i>T</i> has discriminants, then this object is unconstrained if and only the discriminants have defaults. Normal default initialization and finalization take place for this object (see 3.3.1, 7.6, and 7.6.1).	26 27/3
	If <i>T</i> is an abstract type, then S'Input is an abstract function.	27.1/2
	For every subtype S'Class of a class-wide type T'Class:	28
S'Class'Output	S'Class'Output denotes a procedure with the following specification:	29
	<pre> procedure S'Class'Output (<i>Stream</i> : not null access Ada.Streams.Root_Stream_Type'Class; <i>Item</i> : in T'Class) </pre>	30/2
	First writes the external tag of <i>Item</i> to <i>Stream</i> (by calling String'Output(<i>Stream</i> , Tags.-External_Tag(<i>Item</i> .Tag)) — see 3.9) and then dispatches to the subprogram denoted by the Output attribute of the specific type identified by the tag. Tag_Error is raised if the tag of <i>Item</i> identifies a type declared at an accessibility level deeper than that of <i>S</i> .	31/2

32 S'Class'Input

S'Class'Input denotes a function with the following specification:

```
33/2      function S'Class'Input (  
          Stream : not null access Ada.Streams.Root_Stream_Type'Class)  
          return T'Class
```

34/3 First reads the external tag from *Stream* and determines the corresponding internal tag (by calling Tags.Descendant_Tag(String'Input(*Stream*), S'Tag) which might raise Tag_Error — see 3.9) and then dispatches to the subprogram denoted by the Input attribute of the specific type identified by the internal tag; returns that result. If the specific type identified by the internal tag is abstract, Constraint_Error is raised.

35/2 In the default implementation of Read and Input for a composite type, for each scalar component that is a discriminant or whose component_declaration includes a default_expression, a check is made that the value returned by Read for the component belongs to its subtype. Constraint_Error is raised if this check fails. For other scalar components, no check is made. For each component that is of an access type, if the implementation can detect that the value returned by Read for the component is not a value of its subtype, Constraint_Error is raised. If the value is not a value of its subtype and this error is not detected, the component has an abnormal value, and erroneous execution can result (see 13.9.1). In the default implementation of Read for a composite type with defaulted discriminants, if the actual parameter of Read is constrained, a check is made that the discriminants read from the stream are equal to those of the actual parameter. Constraint_Error is raised if this check fails.

36/2 It is unspecified at which point and in which order these checks are performed. In particular, if Constraint_Error is raised due to the failure of one of these checks, it is unspecified how many stream elements have been read from the stream.

37/1 In the default implementation of Read and Input for a type, End_Error is raised if the end of the stream is reached before the reading of a value of the type is completed.

38/3 The stream-oriented attributes may be specified for any type via an attribute_definition_clause. The subprogram name given in such a clause shall statically denote a subprogram that is not an abstract subprogram. Furthermore, if a stream-oriented attribute is specified for an interface type by an attribute_definition_clause, the subprogram name given in the clause shall statically denote a null procedure.

39/2 A stream-oriented attribute for a subtype of a specific type *T* is *available* at places where one of the following conditions is true:

- 40/2 • *T* is nonlimited.
- 41/2 • The attribute_designator is Read (resp. Write) and *T* is a limited record extension, and the attribute Read (resp. Write) is available for the parent type of *T* and for the types of all of the extension components.
- 42/2 • *T* is a limited untagged derived type, and the attribute was inherited for the type.
- 43/2 • The attribute_designator is Input (resp. Output), and *T* is a limited type, and the attribute Read (resp. Write) is available for *T*.
- 44/2 • The attribute has been specified via an attribute_definition_clause, and the attribute_definition_clause is visible.

45/2 A stream-oriented attribute for a subtype of a class-wide type *T*Class is available at places where one of the following conditions is true:

- 46/2 • *T* is nonlimited;

- the attribute has been specified via an `attribute_definition_clause`, and the `attribute_definition_clause` is visible; or 47/2
- the corresponding attribute of *T* is available, provided that if *T* has a partial view, the corresponding attribute is available at the end of the visible part where *T* is declared. 48/2

An `attribute_reference` for one of the stream-oriented attributes is illegal unless the attribute is available at the place of the `attribute_reference`. Furthermore, an `attribute_reference` for *T*Input is illegal if *T* is an abstract type. 49/2

In the `parameter_and_result_profiles` for the stream-oriented attributes, the subtype of the Item parameter is the base subtype of *T* if *T* is a scalar type, and the first subtype otherwise. The same rule applies to the result of the Input attribute. 50/2

For an `attribute_definition_clause` specifying one of these attributes, the subtype of the Item parameter shall be the first subtype or the base subtype if scalar, and the first subtype if not scalar. The same rule applies to the result of the Input function. 51/3

A type is said to *support external streaming* if Read and Write attributes are provided for sending values of such a type between active partitions, with Write marshalling the representation, and Read unmarshalling the representation. A limited type supports external streaming only if it has available Read and Write attributes. A type with a part that is of a non-remote access type supports external streaming only if that access type or the type of some part that includes the access type component, has Read and Write attributes that have been specified via an `attribute_definition_clause`, and that `attribute_definition_clause` is visible. An anonymous access type does not support external streaming. All other types (including remote access types, see E.2.2) support external streaming. 52/3

Erroneous Execution

If the internal tag returned by `Descendant_Tag` to *T*'ClassInput identifies a type that is not library-level and whose tag has not been created, or does not exist in the partition at the time of the call, execution is erroneous. 53/2

Implementation Requirements

For every subtype *S* of a language-defined nonlimited specific type *T*, the output generated by *S*'Output or *S*'Write shall be readable by *S*'Input or *S*'Read, respectively. This rule applies across partitions if the implementation conforms to the Distributed Systems Annex. 54/1

If `Constraint_Error` is raised during a call to Read because of failure of one the above checks, the implementation shall ensure that the discriminants of the actual parameter of Read are not modified. 55/3

Implementation Permissions

The number of calls performed by the predefined implementation of the stream-oriented attributes on the Read and Write operations of the stream type is unspecified. An implementation may take advantage of this permission to perform internal buffering. However, all the calls on the Read and Write operations of the stream type needed to implement an explicit invocation of a stream-oriented attribute shall take place before this invocation returns. An explicit invocation is one appearing explicitly in the program text, possibly through a generic instantiation (see 12.3). 56/3

If *T* is a discriminated type and its discriminants have defaults then in two cases an execution of the default implementation of *S*'Read is not required to create an anonymous object of type *T*: If the discriminants values that are read in are equal to the corresponding discriminant values of *Item*, then no object of type *T* need be created and *Item* may be used instead. If they are not equal and *Item* is a 56.1/3

constrained variable, then `Constraint_Error` may be raised at that point, before any further values are read from the stream and before the object of type *T* is created.

- 56.2/3 A default implementation of `S'Input` that calls the default implementation of `S'Read` may create a constrained anonymous object with discriminants that match those in the stream.

NOTES

- 57 33 For a definite subtype *S* of a type *T*, only *T'Write* and *T'Read* are needed to pass an arbitrary value of the subtype through a stream. For an indefinite subtype *S* of a type *T*, *T'Output* and *T'Input* will normally be needed, since *T'Write* and *T'Read* do not pass bounds, discriminants, or tags.
- 58 34 User-specified attributes of `S'Class` are not inherited by other class-wide types descended from *S*.

Examples

- 59 *Example of user-defined Write attribute:*

```
60/2  procedure My_Write(
      Stream : not null access Ada.Streams.Root_Stream_Type'Class;
      Item   : My_Integer'Base);
      for My_Integer'Write use My_Write;
```

13.14 Freezing Rules

- 1 This clause defines a place in the program text where each declared entity becomes “frozen.” A use of an entity, such as a reference to it by name, or (for a type) an expression of the type, causes freezing of the entity in some contexts, as described below. The Legality Rules forbid certain kinds of uses of an entity in the region of text where it is frozen.
- 2 The *freezing* of an entity occurs at one or more places (*freezing points*) in the program text where the representation for the entity has to be fully determined. Each entity is frozen from its first freezing point to the end of the program text (given the ordering of compilation units defined in 10.1.4).
- 2.1/3 This clause also defines a place in the program text where the profile of each declared callable entity becomes *frozen*. A use of a callable entity causes freezing of its profile in some contexts, as described below. At the place where the profile of a callable entity becomes frozen, the entity itself becomes frozen.
- 3/3 The end of a `declarative_part`, `protected_body`, or a declaration of a library package or generic library package, causes *freezing* of each entity and profile declared within it, except for incomplete types. A noninstance body other than a `renames-as-body` causes freezing of each entity and profile declared before it within the same `declarative_part` that is not an incomplete type; it only causes freezing of an incomplete type if the body is within the immediate scope of the incomplete type.
- 4/1 A construct that (explicitly or implicitly) references an entity can cause the *freezing* of the entity, as defined by subsequent paragraphs. At the place where a construct causes freezing, each `name`, `expression`, `implicit_dereference`, or `range` within the construct causes freezing:
- 5 • The occurrence of a `generic_instantiation` causes freezing; also, if a parameter of the instantiation is defaulted, the `default_expression` or `default_name` for that parameter causes freezing.
 - 6 • The occurrence of an `object_declaration` that has no corresponding completion causes freezing.
 - 7 • The declaration of a record extension causes freezing of the parent subtype.
 - 7.1/2 • The declaration of a record extension, interface type, task unit, or protected unit causes freezing of any progenitor types specified in the declaration.

A static expression causes freezing where it occurs. An object name or nonstatic expression causes freezing where it occurs, unless the name or expression is part of a <code>default_expression</code> , a <code>default_name</code> , or a per-object expression of a component's constraint, in which case, the freezing occurs later as part of another construct.	8/1
An implicit call freezes the same entities and profiles that would be frozen by an explicit call. This is true even if the implicit call is removed via implementation permissions.	8.1/3
If an expression is implicitly converted to a type or subtype <i>T</i> , then at the place where the expression causes freezing, <i>T</i> is frozen.	8.2/1
The following rules define which entities are frozen at the place where a construct causes freezing:	9
At the place where an expression causes freezing, the type of the expression is frozen, unless the expression is an enumeration literal used as a <code>discrete_choice</code> of the <code>array_aggregate</code> of an <code>enumeration_representation_clause</code>.	10
At the place where a function call causes freezing, the profile of the function is frozen. Furthermore, if a parameter of the call is defaulted, the <code>default_expression</code> for that parameter causes freezing.	10.1/3
At the place where a <code>generic_instantiation</code> causes freezing of a callable entity, the profile of that entity is frozen.	10.2/3
At the place where a <code>name</code> causes freezing, the entity denoted by the <code>name</code> is frozen, unless the <code>name</code> is a prefix of an expanded name; at the place where an object <code>name</code> causes freezing, the nominal subtype associated with the <code>name</code> is frozen.	11
At the place where an <code>implicit_dereference</code> causes freezing, the nominal subtype associated with the <code>implicit_dereference</code> is frozen.	11.1/1
At the place where a <code>range</code> causes freezing, the type of the <code>range</code> is frozen.	12
At the place where an <code>allocator</code> causes freezing, the designated subtype of its type is frozen. If the type of the <code>allocator</code> is a derived type, then all ancestor types are also frozen.	13
At the place where a profile is frozen, each subtype of the profile is frozen. If the corresponding callable entity is a member of an entry family, the index subtype of the family is frozen.	14/3
At the place where a subtype is frozen, its type is frozen. At the place where a type is frozen, any expressions or names within the full type definition cause freezing; the first subtype, and any component subtypes, index subtypes, and parent subtype of the type are frozen as well. For a specific tagged type, the corresponding class-wide type is frozen as well. For a class-wide type, the corresponding specific type is frozen as well.	15
At the place where a specific tagged type is frozen, the primitive subprograms of the type are frozen. At the place where a type is frozen, any subprogram named in an <code>attribute_definition_clause</code> for the type is frozen.	15.1/3

Legality Rules

The explicit declaration of a primitive subprogram of a tagged type shall occur before the type is frozen (see 3.9.2).	16
A type shall be completely defined before it is frozen (see 3.11.1 and 7.3).	17
The completion of a deferred constant declaration shall occur before the constant is frozen (see 7.4).	18
An operational or representation item that directly specifies an aspect of an entity shall appear before the entity is frozen (see 13.1).	19/1

20/2 The tag (see 3.9) of a tagged type T is created at the point where T is frozen.

The Standard Libraries

Annex A

(normative)

Predefined Language Environment

This Annex contains the specifications of library units that shall be provided by every implementation. 1
There are three root library units: Ada, Interfaces, and System; other library units are children of these:

2/3

Standard — A.1

Ada — A.2

- Assertions — 11.4.2
- Asynchronous_Task_Control — D.11
- Calendar — 9.6
 - Arithmetic — 9.6.1
 - Formatting — 9.6.1
 - Time_Zones — 9.6.1
- Characters — A.3.1
 - Conversions — A.3.4
 - Handling — A.3.2
 - Latin_1 — A.3.3
- Command_Line — A.15
- Complex_Text_IO — G.1.3
- Containers — A.18.1
 - Bounded_Doubly_Linked_Lists — A.18.20
 - Bounded_Hashed_Maps — A.18.21
 - Bounded_Hashed_Sets — A.18.23
 - Bounded_Multiway_Trees — A.18.25
 - Bounded_Ordered_Maps — A.18.22
 - Bounded_Ordered_Sets — A.18.24
 - Bounded_Vectors — A.18.19
 - Doubly_Linked_Lists — A.18.3
 - Generic_Array_Sort — A.18.26
 - Generic_Constrained_Array_Sort — A.18.26
 - Generic_Sort — A.18.26
 - Hashed_Maps — A.18.5
 - Hashed_Sets — A.18.8
 - Indefinite_Doubly_Linked_Lists — A.18.12
 - Indefinite_Hashed_Maps — A.18.13
 - Indefinite_Hashed_Sets — A.18.15
 - Indefinite_Holders — A.18.18
 - Indefinite_Multiway_Trees — A.18.17
 - Indefinite_Ordered_Maps — A.18.14
 - Indefinite_Ordered_Sets — A.18.16
 - Indefinite_Vectors — A.18.11
 - Multiway_Trees — A.18.10
 - Ordered_Maps — A.18.6
 - Ordered_Sets — A.18.9
 - Vectors — A.18.2
- Decimal — F.2
- Direct_IO — A.8.4
- Directories — A.16
 - Hierarchical_File_Names — A.16.1
 - Information — A.16
- Dispatching — D.2.1
 - EDF — D.2.6
 - Non_Preemptive — D.2.4
 - Round_Robin — D.2.5
- Dynamic_Priorities — D.5

Standard (...continued)

Ada (...continued)

- Environment_Variables — A.17
- Exceptions — 11.4.1
- Execution_Time — D.14
 - Group_Budgets — D.14.2
 - Timers — D.14.1
- Finalization — 7.6
- Float_Text_IO — A.10.9
- Float_Wide_Text_IO — A.11
- Float_Wide_Wide_Text_IO — A.11
- Integer_Text_IO — A.10.8
- Integer_Wide_Text_IO — A.11
- Integer_Wide_Wide_Text_IO — A.11
- Interrupts — C.3.2
 - Names — C.3.2
- IO_Exceptions — A.13
- Numerics — A.5
 - Complex_Arrays — G.3.2
 - Complex_Elementary_Functions — G.1.2
 - Complex_Types — G.1.1
 - Discrete_Random — A.5.2
 - Elementary_Functions — A.5.1
 - Float_Random — A.5.2
 - Generic_Complex_Arrays — G.3.2
 - Generic_Complex_Elementary_Functions — G.1.2
 - Generic_Complex_Types — G.1.1
 - Generic_Elementary_Functions — A.5.1
 - Generic_Real_Arrays — G.3.1
 - Real_Arrays — G.3.1
- Real_Time — D.8
 - Timing_Events — D.15
- Sequential_IO — A.8.1
- Storage_IO — A.9
- Streams — 13.13.1
 - Stream_IO — A.12.1

Standard (...continued)

Ada (...continued)

Strings — A.4.1

Bounded — A.4.4

Equal_Case_Insensitive — A.4.10

Hash — A.4.9

Hash_Case_Insensitive — A.4.9

Less_Case_Insensitive — A.4.10

Fixed — A.4.3

Equal_Case_Insensitive — A.4.10

Hash — A.4.9

Hash_Case_Insensitive — A.4.9

Less_Case_Insensitive — A.4.10

Equal_Case_Insensitive — A.4.10

Hash — A.4.9

Hash_Case_Insensitive — A.4.9

Less_Case_Insensitive — A.4.10

Maps — A.4.2

Constants — A.4.6

Unbounded — A.4.5

Equal_Case_Insensitive — A.4.10

Hash — A.4.9

Hash_Case_Insensitive — A.4.9

Less_Case_Insensitive — A.4.10

UTF_Encoding — A.4.11

Conversions — A.4.11

Wide_Encoding — A.4.11

Wide_Wide_Encoding — A.4.11

), Old=() { Wide_Bounded — A.4.7

Wide_Hash — A.4.7

Wide_Fixed — A.4.7

Wide_Hash — A.4.7

Wide_Hash — A.4.7

Wide_Maps — A.4.7

Wide_Constants — A.4.7

Wide_Unbounded — A.4.7

Wide_Hash — A.4.7

Wide_Wide_Bounded — A.4.8

Wide_Wide_Hash — A.4.8

Wide_Wide_Fixed — A.4.8

Wide_Wide_Hash — A.4.8

Wide_Wide_Hash — A.4.8

Wide_Wide_Maps — A.4.8

Wide_Wide_Constants — A.4.8

Wide_Wide_Unbounded — A.4.8

Wide_Wide_Hash — A.4.8

Synchronous_Task_Control — D.10

EDF — D.10

Tags — 3.9

Generic_Dispatching_Constructor — 3.9

Task_Attributes — C.7.2

Task_Identification — C.7.1

Task_Termination — C.7.3

Standard (...continued)

Ada (...continued)

Text_IO — A.10.1

Bounded_IO — A.10.11

Complex_IO — G.1.3

Editing — F.3.3

Text_Streams — A.12.2

Unbounded_IO — A.10.12

Unchecked_Conversion — 13.9

Unchecked_Deallocation — 13.11.2

Wide_Characters — A.3.1

Wide_Text_IO — A.11

Complex_IO — G.1.4

Editing — F.3.4

Text_Streams — A.12.3

Wide_Bounded_IO — A.11

Wide_Unbounded_IO — A.11

Wide_Wide_Characters — A.3.1

Wide_Wide_Text_IO — A.11

Complex_IO — G.1.5

Editing — F.3.5

Text_Streams — A.12.4

Wide_Wide_Bounded_IO — A.11

Wide_Wide_Unbounded_IO — A.11

Interfaces — B.2

C — B.3

Pointers — B.3.2

Strings — B.3.1

COBOL — B.4

Fortran — B.5

System — 13.7

Address_To_Access_Conversions — 13.7.2

Machine_Code — 13.8

RPC — E.5

Storage_Elements — 13.7.1

Storage_Pools — 13.11

Implementation Requirements

- 3/2 The implementation shall ensure that each language-defined subprogram is reentrant in the sense that concurrent calls on the same subprogram perform as specified, so long as all parameters that could be passed by reference denote nonoverlapping objects.
- 3.1/3 For a descendant of a language-defined tagged type, the implementation shall ensure that each inherited language-defined subprogram behaves as described in this International Standard. In particular, overriding a language-defined subprogram shall not alter the effect of any inherited language-defined subprogram.

Implementation Permissions

- 4 The implementation may restrict the replacement of language-defined compilation units. The implementation may restrict children of language-defined library units (other than Standard).
- 5/3 An implementation may omit pragma Remote_Types (see E.2.2) in language-defined library units if the implementation does not conform to Annex E.

A.1 The Package Standard

This clause outlines the specification of the package Standard containing all predefined identifiers in the language. The corresponding package body is not specified by the language.

The operators that are predefined for the types declared in the package Standard are given in comments since they are implicitly declared. *Italics* are used for pseudo-names of anonymous types (such as *root_real*) and for undefined information (such as *implementation-defined*).

Static Semantics

The library package Standard has the following declaration:

```

package Standard is
  pragma Pure(Standard);
  type Boolean is (False, True);
  -- The predefined relational operators for this type are as follows:
  -- function "=" (Left, Right : Boolean'Base) return Boolean;
  -- function "/=" (Left, Right : Boolean'Base) return Boolean;
  -- function "<" (Left, Right : Boolean'Base) return Boolean;
  -- function "<=" (Left, Right : Boolean'Base) return Boolean;
  -- function ">" (Left, Right : Boolean'Base) return Boolean;
  -- function ">=" (Left, Right : Boolean'Base) return Boolean;
  -- The predefined logical operators and the predefined logical
  -- negation operator are as follows:
  -- function "and" (Left, Right : Boolean'Base) return Boolean'Base;
  -- function "or" (Left, Right : Boolean'Base) return Boolean'Base;
  -- function "xor" (Left, Right : Boolean'Base) return Boolean'Base;
  -- function "not" (Right : Boolean'Base) return Boolean'Base;
  -- The integer type root_integer and the
  -- corresponding universal type universal_integer are predefined.
  type Integer is range implementation-defined;
  subtype Natural is Integer range 0 .. Integer'Last;
  subtype Positive is Integer range 1 .. Integer'Last;
  -- The predefined operators for type Integer are as follows:
  -- function "=" (Left, Right : Integer'Base) return Boolean;
  -- function "/=" (Left, Right : Integer'Base) return Boolean;
  -- function "<" (Left, Right : Integer'Base) return Boolean;
  -- function "<=" (Left, Right : Integer'Base) return Boolean;
  -- function ">" (Left, Right : Integer'Base) return Boolean;
  -- function ">=" (Left, Right : Integer'Base) return Boolean;
  -- function "+" (Right : Integer'Base) return Integer'Base;
  -- function "-" (Right : Integer'Base) return Integer'Base;
  -- function "abs" (Right : Integer'Base) return Integer'Base;
  -- function "+" (Left, Right : Integer'Base) return Integer'Base;
  -- function "-" (Left, Right : Integer'Base) return Integer'Base;
  -- function "*" (Left, Right : Integer'Base) return Integer'Base;
  -- function "/" (Left, Right : Integer'Base) return Integer'Base;
  -- function "rem" (Left, Right : Integer'Base) return Integer'Base;
  -- function "mod" (Left, Right : Integer'Base) return Integer'Base;
  -- function "***" (Left : Integer'Base; Right : Natural)
  -- return Integer'Base;

```

```

19      -- The specification of each operator for the type
      -- root_integer, or for any additional predefined integer
      -- type, is obtained by replacing Integer by the name of the type
      -- in the specification of the corresponding operator of the type
      -- Integer. The right operand of the exponentiation operator
      -- remains as subtype Natural.

20/2    -- The floating point type root_real and the
      -- corresponding universal type universal_real are predefined.

21    type Float is digits implementation-defined;
22    -- The predefined operators for this type are as follows:
23    -- function "=" (Left, Right : Float) return Boolean;
      -- function "/"= (Left, Right : Float) return Boolean;
      -- function "<" (Left, Right : Float) return Boolean;
      -- function "<=" (Left, Right : Float) return Boolean;
      -- function ">" (Left, Right : Float) return Boolean;
      -- function ">=" (Left, Right : Float) return Boolean;
24    -- function "+" (Right : Float) return Float;
      -- function "-" (Right : Float) return Float;
      -- function "abs" (Right : Float) return Float;
25    -- function "+" (Left, Right : Float) return Float;
      -- function "-" (Left, Right : Float) return Float;
      -- function "*" (Left, Right : Float) return Float;
      -- function "/" (Left, Right : Float) return Float;
26    -- function "***" (Left : Float; Right : Integer'Base) return Float;
27    -- The specification of each operator for the type root_real, or for
      -- any additional predefined floating point type, is obtained by
      -- replacing Float by the name of the type in the specification of the
      -- corresponding operator of the type Float.
28    -- In addition, the following operators are predefined for the root
      -- numeric types:
29    function "*" (Left : root_integer; Right : root_real)
      return root_real;
30    function "*" (Left : root_real; Right : root_integer)
      return root_real;
31    function "/" (Left : root_real; Right : root_integer)
      return root_real;
32    -- The type universal_fixed is predefined.
      -- The only multiplying operators defined between
      -- fixed point types are
33    function "*" (Left : universal_fixed; Right : universal_fixed)
      return universal_fixed;
34    function "/" (Left : universal_fixed; Right : universal_fixed)
      return universal_fixed;
34.1/2  -- The type universal_access is predefined.
      -- The following equality operators are predefined:
34.2/2  function "=" (Left, Right : universal_access) return Boolean;
      function "/"= (Left, Right : universal_access) return Boolean;

```


-- The declaration of type *Character* is based on the standard ISO 8859-1 character set.

-- There are no character literals corresponding to the positions for control characters.

-- They are indicated in *italics* in this definition. See 3.5.2.

```

type Character is
  (nul,  soh,  stx,  etx,  eot,  enq,  ack,  bel,  -- 0 (16#00#) .. 7 (16#07#)
   bs,   ht,   lf,   vt,   ff,   cr,   so,   si,   -- 8 (16#08#) .. 15 (16#0F#)

   dle,  dc1,  dc2,  dc3,  dc4,  nak,  syn,  etb,  -- 16 (16#10#) .. 23 (16#17#)
   can,  em,   sub,  esc,  fs,   gs,   rs,   us,   -- 24 (16#18#) .. 31 (16#1F#)

   ' ', '!', ' "', ' # ', ' $ ', ' % ', ' & ', ' ', -- 32 (16#20#) .. 39 (16#27#)
   ' ( ', ' ) ', ' * ', ' + ', ' , ', ' - ', ' . ', ' / ', -- 40 (16#28#) .. 47 (16#2F#)

   ' 0 ', ' 1 ', ' 2 ', ' 3 ', ' 4 ', ' 5 ', ' 6 ', ' 7 ', -- 48 (16#30#) .. 55 (16#37#)
   ' 8 ', ' 9 ', ' : ', ' ; ', ' < ', ' = ', ' > ', ' ? ', -- 56 (16#38#) .. 63 (16#3F#)

   ' @ ', ' A ', ' B ', ' C ', ' D ', ' E ', ' F ', ' G ', -- 64 (16#40#) .. 71 (16#47#)
   ' H ', ' I ', ' J ', ' K ', ' L ', ' M ', ' N ', ' O ', -- 72 (16#48#) .. 79 (16#4F#)

   ' P ', ' Q ', ' R ', ' S ', ' T ', ' U ', ' V ', ' W ', -- 80 (16#50#) .. 87 (16#57#)
   ' X ', ' Y ', ' Z ', ' [ ', ' \ ', ' ] ', ' ^ ', ' _ ', -- 88 (16#58#) .. 95 (16#5F#)

   ' ` ', ' a ', ' b ', ' c ', ' d ', ' e ', ' f ', ' g ', -- 96 (16#60#) .. 103 (16#67#)
   ' h ', ' i ', ' j ', ' k ', ' l ', ' m ', ' n ', ' o ', -- 104 (16#68#) .. 111 (16#6F#)

   ' p ', ' q ', ' r ', ' s ', ' t ', ' u ', ' v ', ' w ', -- 112 (16#70#) .. 119 (16#77#)
   ' x ', ' y ', ' z ', ' { ', ' | ', ' } ', ' ~ ', del, -- 120 (16#78#) .. 127 (16#7F#)

   reserved_128, reserved_129, bph,  nbh,          -- 128 (16#80#) .. 131 (16#83#)
   reserved_132, nel,   ssa,    esa,          -- 132 (16#84#) .. 135 (16#87#)
   hts,   htj,   vis,   pld,    plu,   ri,   ss2,  ss3,  -- 136 (16#88#) .. 143 (16#8F#)

   dcs,   pul,   pu2,   sts,    cch,   mw,   spa,   epa,  -- 144 (16#90#) .. 151 (16#97#)
   sos,   reserved_153, sci,    cst,          -- 152 (16#98#) .. 155 (16#9B#)
   st,    osc,   pm,    apc,          -- 156 (16#9C#) .. 159 (16#9F#)

   ' ', '!', ' ¢ ', ' £ ', ' ¤ ', ' ¥ ', ' ¦ ', ' § ', -- 160 (16#A0#) .. 167 (16#A7#)
   ' ¨ ', ' © ', ' ª ', ' « ', ' ¬ ', soft_hyphen, ' ® ', ' ¯ ', -- 168 (16#A8#) ..
175 (16#AF#)

   ' ° ', ' ± ', ' ² ', ' ³ ', ' ´ ', ' µ ', ' ¶ ', ' · ', -- 176 (16#B0#) .. 183 (16#B7#)
   ' , ', ' ¹ ', ' º ', ' » ', ' ¼ ', ' ½ ', ' ¾ ', ' ¿ ', -- 184 (16#B8#) .. 191 (16#BF#)

   ' À ', ' Á ', ' Â ', ' Ã ', ' Ä ', ' Å ', ' Æ ', ' Ç ', -- 192 (16#C0#) .. 199 (16#C7#)
   ' È ', ' É ', ' Ê ', ' Ë ', ' Ì ', ' Í ', ' Î ', ' Ï ', -- 200 (16#C8#) .. 207 (16#CF#)

   ' Ð ', ' Ñ ', ' Ò ', ' Ó ', ' Ô ', ' Õ ', ' Ö ', ' × ', -- 208 (16#D0#) .. 215 (16#D7#)
   ' Ø ', ' Ù ', ' Ú ', ' Û ', ' Ü ', ' Ý ', ' Þ ', ' ß ', -- 216 (16#D8#) .. 223 (16#DF#)

   ' à ', ' á ', ' â ', ' ä ', ' ä ', ' å ', ' æ ', ' ç ', -- 224 (16#E0#) .. 231 (16#E7#)
   ' è ', ' é ', ' ê ', ' ë ', ' ì ', ' í ', ' î ', ' ï ', -- 232 (16#E8#) .. 239 (16#EF#)

   ' ð ', ' ñ ', ' ò ', ' ó ', ' ô ', ' õ ', ' ö ', ' ÷ ', -- 240 (16#F0#) .. 247 (16#F7#)
   ' ø ', ' ù ', ' ú ', ' û ', ' ü ', ' ý ', ' þ ', ' ÿ ', -- 248 (16#F8#) .. 255 (16#FF#)

```

-- The predefined operators for the type *Character* are the same as for

-- any enumeration type.

-- The declaration of type *Wide_Character* is based on the standard ISO/IEC 10646:2003 BMP character set. The first 256 positions have the same contents as type *Character*. See 3.5.2.

```

type Wide_Character is (nul, soh ... Hex_0000FFFE, Hex_0000FFFF) ;

```

```

36.2/2      -- The declaration of type Wide_Wide_Character is based on the full
            -- ISO/IEC 10646:2003 character set. The first 65536 positions have the
            -- same contents as type Wide_Character. See 3.5.2.

            type Wide_Wide_Character is (nul, soh ... Hex_7FFFFFFE, Hex_7FFFFFFF);
            for Wide_Wide_Character'Size use 32;

36.3/2      package ASCII is ... end ASCII;  -- Obsolescent; see J.5

37          -- Predefined string types:

            type String is array(Positive range <>) of Character;
            pragma Pack(String);

38          -- The predefined operators for this type are as follows:

39          --      function "=" (Left: String; Right: String) return Boolean;
            --      function "/=" (Left: String; Right: String) return Boolean;
            --      function "<" (Left: String; Right: String) return Boolean;
            --      function "<=" (Left: String; Right: String) return Boolean;
            --      function ">" (Left: String; Right: String) return Boolean;
            --      function ">=" (Left: String; Right: String) return Boolean;

40          --      function "&" (Left: String; Right: String) return String;
            --      function "&" (Left: Character; Right: String) return String;
            --      function "&" (Left: String; Right: Character) return String;
            --      function "&" (Left: Character; Right: Character) return String;

41          type Wide_String is array(Positive range <>) of Wide_Character;
            pragma Pack(Wide_String);

42          -- The predefined operators for this type correspond to those for String.

42.1/2      type Wide_Wide_String is array (Positive range <>)
            of Wide_Wide_Character;
            pragma Pack (Wide_Wide_String);

42.2/2      -- The predefined operators for this type correspond to those for String.

43          type Duration is delta implementation-defined range implementation-defined;
44          -- The predefined operators for the type Duration are the same as for
            -- any fixed point type.

45          -- The predefined exceptions:

46          Constraint_Error: exception;
            Program_Error   : exception;
            Storage_Error   : exception;
            Tasking_Error   : exception;

47          end Standard;

48          Standard has no private part.

49/2      In each of the types Character, Wide_Character, and Wide_Wide_Character, the character literals for the
            space character (position 32) and the non-breaking space character (position 160) correspond to different
            values. Unless indicated otherwise, each occurrence of the character literal ' ' in this International Standard
            refers to the space character. Similarly, the character literals for hyphen (position 45) and soft hyphen
            (position 173) correspond to different values. Unless indicated otherwise, each occurrence of the character
            literal '-' in this International Standard refers to the hyphen character.

```

Dynamic Semantics

50 Elaboration of the body of Standard has no effect.

Implementation Permissions

51 An implementation may provide additional predefined integer types and additional predefined floating point types. Not all of these types need have names.

Implementation Advice

If an implementation provides additional named predefined integer types, then the names should end with “Integer” as in “Long_Integer”. If an implementation provides additional named predefined floating point types, then the names should end with “Float” as in “Long_Float”.

NOTES

- 1 Certain aspects of the predefined entities cannot be completely described in the language itself. For example, although the enumeration type Boolean can be written showing the two enumeration literals False and True, the short-circuit control forms cannot be expressed in the language. 53
- 2 As explained in 8.1, “Declarative Region” and 10.1.4, “The Compilation Process”, the declarative region of the package Standard encloses every library unit and consequently the main subprogram; the declaration of every library unit is assumed to occur within this declarative region. Library_items are assumed to be ordered in such a way that there are no forward semantic dependences. However, as explained in 8.3, “Visibility”, the only library units that are visible within a given compilation unit are the library units named by all with_clauses that apply to the given unit, and moreover, within the declarative region of a given library unit, that library unit itself. 54
- 3 If all block_statements of a program are named, then the name of each program unit can always be written as an expanded name starting with Standard (unless Standard is itself hidden). The name of a library unit cannot be a homograph of a name (such as Integer) that is already declared in Standard. 55
- 4 The exception Standard.Numeric_Error is defined in J.6. 56

A.2 The Package Ada

Static Semantics

The following language-defined library package exists: 1

```
package Ada is
  pragma Pure (Ada);
end Ada;
```

Ada serves as the parent of most of the other language-defined library units; its declaration is empty (except for the pragma Pure). 2 3

Legality Rules

In the standard mode, it is illegal to compile a child of package Ada. 4

A.3 Character Handling

This clause presents the packages related to character processing: an empty pure package Characters and child packages Characters.Handling and Characters.Latin_1. The package Characters.Handling provides classification and conversion functions for Character data, and some simple functions for dealing with Wide_Character and Wide_Wide_Character data. The child package Characters.Latin_1 declares a set of constants initialized to values of type Character. 1/2

A.3.1 The Packages Characters, Wide_Characters, and Wide_Wide_Characters

Static Semantics

1 The library package Characters has the following declaration:

```
2 package Ada.Characters is
  pragma Pure(Characters);
end Ada.Characters;
```

3/2 The library package Wide_Characters has the following declaration:

```
4/2 package Ada.Wide_Characters is
  pragma Pure(Wide_Characters);
end Ada.Wide_Characters;
```

5/2 The library package Wide_Wide_Characters has the following declaration:

```
6/2 package Ada.Wide_Wide_Characters is
  pragma Pure(Wide_Wide_Characters);
end Ada.Wide_Wide_Characters;
```

Implementation Advice

7/2 If an implementation chooses to provide implementation-defined operations on Wide_Character or Wide_String (such as case mapping, classification, collating and sorting, etc.) it should do so by providing child units of Wide_Characters. Similarly if it chooses to provide implementation-defined operations on Wide_Wide_Character or Wide_Wide_String it should do so by providing child units of Wide_Wide_Characters.

A.3.2 The Package Characters.Handling

Static Semantics

1 The library package Characters.Handling has the following declaration:

```
2/2 with Ada.Characters.Conversions;
package Ada.Characters.Handling is
  pragma Pure(Handling);
3 -- Character classification functions
4 function Is_Control (Item : in Character) return Boolean;
  function Is_Graphic (Item : in Character) return Boolean;
  function Is_Letter (Item : in Character) return Boolean;
  function Is_Lower (Item : in Character) return Boolean;
  function Is_Upper (Item : in Character) return Boolean;
  function Is_Basic (Item : in Character) return Boolean;
  function Is_Digit (Item : in Character) return Boolean;
  function Is_Decimal_Digit (Item : in Character) return Boolean;
  renames Is_Digit;
  function Is_Hexadecimal_Digit (Item : in Character) return Boolean;
  function Is_Alphanumeric (Item : in Character) return Boolean;
  function Is_Special (Item : in Character) return Boolean;
5 -- Conversion functions for Character and String
6 function To_Lower (Item : in Character) return Character;
  function To_Upper (Item : in Character) return Character;
  function To_Basic (Item : in Character) return Character;
7 function To_Lower (Item : in String) return String;
  function To_Upper (Item : in String) return String;
  function To_Basic (Item : in String) return String;
```

```

--Classifications of and conversions between Character and ISO 646
8
subtype ISO_646 is
9
  Character range Character'Val(0) .. Character'Val(127);
function Is_ISO_646 (Item : in Character) return Boolean;
10
function Is_ISO_646 (Item : in String) return Boolean;
function To_ISO_646 (Item      : in Character;
11
                   Substitute : in ISO_646 := ' ')
  return ISO_646;
function To_ISO_646 (Item      : in String;
12
                   Substitute : in ISO_646 := ' ')
  return String;
-- The functions Is_Character, Is_String, To_Character, To_String, To_Wide_Character,
13/2
-- and To_Wide_String are obsolescent; see J.14.
Paragraphs 14 through 18 were deleted.
end Ada.Characters.Handling;
19

```

In the description below for each function that returns a Boolean result, the effect is described in terms of the conditions under which the value True is returned. If these conditions are not met, then the function returns False.

Each of the following classification functions has a formal Character parameter, Item, and returns a Boolean result.

Is_Control	True if Item is a control character. A <i>control character</i> is a character whose position is in one of the ranges 0..31 or 127..159.	22
Is_Graphic	True if Item is a graphic character. A <i>graphic character</i> is a character whose position is in one of the ranges 32..126 or 160..255.	23
Is_Letter	True if Item is a letter. A <i>letter</i> is a character that is in one of the ranges 'A'..'Z' or 'a'..'z', or whose position is in one of the ranges 192..214, 216..246, or 248..255.	24
Is_Lower	True if Item is a lower-case letter. A <i>lower-case letter</i> is a character that is in the range 'a'..'z', or whose position is in one of the ranges 223..246 or 248..255.	25
Is_Upper	True if Item is an upper-case letter. An <i>upper-case letter</i> is a character that is in the range 'A'..'Z' or whose position is in one of the ranges 192..214 or 216.. 222.	26
Is_Basic	True if Item is a basic letter. A <i>basic letter</i> is a character that is in one of the ranges 'A'..'Z' and 'a'..'z', or that is one of the following: 'Æ', 'æ', 'Ð', 'ð', 'Þ', 'þ', or 'ß'.	27
Is_Digit	True if Item is a decimal digit. A <i>decimal digit</i> is a character in the range '0'..'9'.	28
Is_Decimal_Digit	A renaming of Is_Digit.	29
Is_Hexadecimal_Digit	True if Item is a hexadecimal digit. A <i>hexadecimal digit</i> is a character that is either a decimal digit or that is in one of the ranges 'A' .. 'F' or 'a' .. 'f'.	30
Is_Alphanumeric	True if Item is an alphanumeric character. An <i>alphanumeric character</i> is a character that is either a letter or a decimal digit.	31
Is_Special	True if Item is a special graphic character. A <i>special graphic character</i> is a graphic character that is not alphanumeric.	32

Each of the names To_Lower, To_Upper, and To_Basic refers to two functions: one that converts from Character to Character, and the other that converts from String to String. The result of each Character-to-Character function is described below, in terms of the conversion applied to Item, its formal Character

parameter. The result of each String-to-String conversion is obtained by applying to each element of the function's String parameter the corresponding Character-to-Character conversion; the result is the null String if the value of the formal parameter is the null String. The lower bound of the result String is 1.

- 34 To_Lower Returns the corresponding lower-case value for Item if Is_Upper(Item), and returns Item otherwise.
- 35 To_Upper Returns the corresponding upper-case value for Item if Is_Lower(Item) and Item has an upper-case form, and returns Item otherwise. The lower case letters 'b' and 'y' do not have upper case forms.
- 36 To_Basic Returns the letter corresponding to Item but with no diacritical mark, if Item is a letter but not a basic letter; returns Item otherwise.
- 37 The following set of functions test for membership in the ISO 646 character range, or convert between ISO 646 and Character.
- 38 Is_ISO_646 The function whose formal parameter, Item, is of type Character returns True if Item is in the subtype ISO_646.
- 39 Is_ISO_646 The function whose formal parameter, Item, is of type String returns True if Is_ISO_646(Item(I)) is True for each I in Item'Range.
- 40 To_ISO_646 The function whose first formal parameter, Item, is of type Character returns Item if Is_ISO_646(Item), and returns the Substitute ISO_646 character otherwise.
- 41 To_ISO_646 The function whose first formal parameter, Item, is of type String returns the String whose Range is 1..Item'Length and each of whose elements is given by To_ISO_646 of the corresponding element in Item.

Paragraphs 42 through 48 were deleted.

Implementation Advice

49/2 *This paragraph was deleted.*

NOTES

- 50 5 A basic letter is a letter without a diacritical mark.
- 51 6 Except for the hexadecimal digits, basic letters, and ISO_646 characters, the categories identified in the classification functions form a strict hierarchy:
- 52 — Control characters
- 53 — Graphic characters
- 54 — Alphanumeric characters
- 55 — Letters
- 56 — Upper-case letters
- 57 — Lower-case letters
- 58 — Decimal digits
- 59 — Special graphic characters

A.3.3 The Package Characters.Latin_1

The package Characters.Latin_1 declares constants for characters in ISO 8859-1.

Static Semantics

The library package Characters.Latin_1 has the following declaration:

```

package Ada.Characters.Latin_1 is
  pragma Pure(Latin_1);
-- Control characters:
  NUL      : constant Character := Character'Val(0);
  SOH      : constant Character := Character'Val(1);
  STX      : constant Character := Character'Val(2);
  ETX      : constant Character := Character'Val(3);
  EOT      : constant Character := Character'Val(4);
  ENQ      : constant Character := Character'Val(5);
  ACK      : constant Character := Character'Val(6);
  BEL      : constant Character := Character'Val(7);
  BS       : constant Character := Character'Val(8);
  HT       : constant Character := Character'Val(9);
  LF       : constant Character := Character'Val(10);
  VT       : constant Character := Character'Val(11);
  FF       : constant Character := Character'Val(12);
  CR       : constant Character := Character'Val(13);
  SO       : constant Character := Character'Val(14);
  SI       : constant Character := Character'Val(15);
  DLE      : constant Character := Character'Val(16);
  DC1      : constant Character := Character'Val(17);
  DC2      : constant Character := Character'Val(18);
  DC3      : constant Character := Character'Val(19);
  DC4      : constant Character := Character'Val(20);
  NAK      : constant Character := Character'Val(21);
  SYN      : constant Character := Character'Val(22);
  ETB      : constant Character := Character'Val(23);
  CAN      : constant Character := Character'Val(24);
  EM       : constant Character := Character'Val(25);
  SUB      : constant Character := Character'Val(26);
  ESC      : constant Character := Character'Val(27);
  FS       : constant Character := Character'Val(28);
  GS       : constant Character := Character'Val(29);
  RS       : constant Character := Character'Val(30);
  US       : constant Character := Character'Val(31);
-- ISO 646 graphic characters:
  Space     : constant Character := ' '; -- Character'Val(32)
  Exclamation : constant Character := '!'; -- Character'Val(33)
  Quotation  : constant Character := '"'; -- Character'Val(34)
  Number_Sign : constant Character := '#'; -- Character'Val(35)
  Dollar_Sign : constant Character := '$'; -- Character'Val(36)
  Percent_Sign : constant Character := '%'; -- Character'Val(37)
  Ampersand  : constant Character := '&'; -- Character'Val(38)
  Apostrophe : constant Character := '\''; -- Character'Val(39)
  Left_Parenthesis : constant Character := '('; -- Character'Val(40)
  Right_Parenthesis : constant Character := ')'; -- Character'Val(41)
  Asterisk    : constant Character := '*'; -- Character'Val(42)
  Plus_Sign   : constant Character := '+'; -- Character'Val(43)
  Comma      : constant Character := ','; -- Character'Val(44)
  Hyphen     : constant Character := '-'; -- Character'Val(45)
  Minus_Sign : constant Character renames Hyphen;
  Full_Stop  : constant Character := '.'; -- Character'Val(46)
  Solidus    : constant Character := '/'; -- Character'Val(47)

```

```

9      -- Decimal digits '0' through '9' are at positions 48 through 57
10     Colon                : constant Character := ':'; -- Character'Val(58)
      Semicolon            : constant Character := ';'; -- Character'Val(59)
      Less_Than_Sign       : constant Character := '<'; -- Character'Val(60)
      Equals_Sign          : constant Character := '='; -- Character'Val(61)
      Greater_Than_Sign    : constant Character := '>'; -- Character'Val(62)
      Question             : constant Character := '?'; -- Character'Val(63)
      Commercial_At        : constant Character := '@'; -- Character'Val(64)
11     -- Letters 'A' through 'Z' are at positions 65 through 90
12     Left_Square_Bracket  : constant Character := '['; -- Character'Val(91)
      Reverse_Solidus      : constant Character := '\'; -- Character'Val(92)
      Right_Square_Bracket : constant Character := ']'; -- Character'Val(93)
      Circumflex           : constant Character := '^'; -- Character'Val(94)
      Low_Line             : constant Character := '_'; -- Character'Val(95)
13     Grave               : constant Character := '`'; -- Character'Val(96)
      LC_A                 : constant Character := 'a'; -- Character'Val(97)
      LC_B                 : constant Character := 'b'; -- Character'Val(98)
      LC_C                 : constant Character := 'c'; -- Character'Val(99)
      LC_D                 : constant Character := 'd'; -- Character'Val(100)
      LC_E                 : constant Character := 'e'; -- Character'Val(101)
      LC_F                 : constant Character := 'f'; -- Character'Val(102)
      LC_G                 : constant Character := 'g'; -- Character'Val(103)
      LC_H                 : constant Character := 'h'; -- Character'Val(104)
      LC_I                 : constant Character := 'i'; -- Character'Val(105)
      LC_J                 : constant Character := 'j'; -- Character'Val(106)
      LC_K                 : constant Character := 'k'; -- Character'Val(107)
      LC_L                 : constant Character := 'l'; -- Character'Val(108)
      LC_M                 : constant Character := 'm'; -- Character'Val(109)
      LC_N                 : constant Character := 'n'; -- Character'Val(110)
      LC_O                 : constant Character := 'o'; -- Character'Val(111)
14     LC_P                 : constant Character := 'p'; -- Character'Val(112)
      LC_Q                 : constant Character := 'q'; -- Character'Val(113)
      LC_R                 : constant Character := 'r'; -- Character'Val(114)
      LC_S                 : constant Character := 's'; -- Character'Val(115)
      LC_T                 : constant Character := 't'; -- Character'Val(116)
      LC_U                 : constant Character := 'u'; -- Character'Val(117)
      LC_V                 : constant Character := 'v'; -- Character'Val(118)
      LC_W                 : constant Character := 'w'; -- Character'Val(119)
      LC_X                 : constant Character := 'x'; -- Character'Val(120)
      LC_Y                 : constant Character := 'y'; -- Character'Val(121)
      LC_Z                 : constant Character := 'z'; -- Character'Val(122)
      Left_Curly_Bracket   : constant Character := '{'; -- Character'Val(123)
      Vertical_Line        : constant Character := '|'; -- Character'Val(124)
      Right_Curly_Bracket  : constant Character := '}'; -- Character'Val(125)
      Tilde                : constant Character := '~'; -- Character'Val(126)
      DEL                  : constant Character := Character'Val(127);
15     -- ISO 6429 control characters:
16     IS4                  : Character renames FS;
      IS3                  : Character renames GS;
      IS2                  : Character renames RS;
      IS1                  : Character renames US;

```



```

Reserved_128      : constant Character := Character'Val(128);
Reserved_129      : constant Character := Character'Val(129);
BPH               : constant Character := Character'Val(130);
NBH               : constant Character := Character'Val(131);
Reserved_132      : constant Character := Character'Val(132);
NEL               : constant Character := Character'Val(133);
SSA               : constant Character := Character'Val(134);
ESA               : constant Character := Character'Val(135);
HTS               : constant Character := Character'Val(136);
HTJ               : constant Character := Character'Val(137);
VTS               : constant Character := Character'Val(138);
PLD               : constant Character := Character'Val(139);
PLU               : constant Character := Character'Val(140);
RI                : constant Character := Character'Val(141);
SS2               : constant Character := Character'Val(142);
SS3               : constant Character := Character'Val(143);

DCS               : constant Character := Character'Val(144);
PU1               : constant Character := Character'Val(145);
PU2               : constant Character := Character'Val(146);
STS               : constant Character := Character'Val(147);
CCH               : constant Character := Character'Val(148);
MW                : constant Character := Character'Val(149);
SPA               : constant Character := Character'Val(150);
EPA               : constant Character := Character'Val(151);

SOS               : constant Character := Character'Val(152);
Reserved_153      : constant Character := Character'Val(153);
SCI               : constant Character := Character'Val(154);
CSI               : constant Character := Character'Val(155);
ST                : constant Character := Character'Val(156);
OSC               : constant Character := Character'Val(157);
PM                : constant Character := Character'Val(158);
APC               : constant Character := Character'Val(159);

-- Other graphic characters:
-- Character positions 160 (16#A0#) .. 175 (16#AF#):
No_Break_Space    : constant Character := ' '; --Character'Val(160)
NBSP              : Character renames No_Break_Space;
Inverted_Exclamation : constant Character := '¡'; --Character'Val(161)
Cent_Sign         : constant Character := '¢'; --Character'Val(162)
Pound_Sign        : constant Character := '£'; --Character'Val(163)
Currency_Sign     : constant Character := '¤'; --Character'Val(164)
Yen_Sign          : constant Character := '¥'; --Character'Val(165)
Broken_Bar        : constant Character := '¦'; --Character'Val(166)
Section_Sign      : constant Character := '§'; --Character'Val(167)
Diaeresis        : constant Character := '¨'; --Character'Val(168)
Copyright_Sign    : constant Character := '©'; --Character'Val(169)
Feminine_Ordinal_Indicator : constant Character := 'ª'; --Character'Val(170)
Left_Angle_Quotation : constant Character := '«'; --Character'Val(171)
Not_Sign          : constant Character := '¬'; --Character'Val(172)
Soft_Hyphen       : constant Character := Character'Val(173);
Registered_Trade_Mark_Sign : constant Character := '®'; --Character'Val(174)
Macron            : constant Character := '¯'; --Character'Val(175)

```

```

22  -- Character positions 176 (16#B0#) .. 191 (16#BF#):
    Degree_Sign           : constant Character := '°'; --Character'Val(176)
    Ring_Above            : Character renames Degree_Sign;
    Plus_Minus_Sign       : constant Character := '±'; --Character'Val(177)
    Superscript_Two       : constant Character := '²'; --Character'Val(178)
    Superscript_Three     : constant Character := '³'; --Character'Val(179)
    Acute                 : constant Character := '´'; --Character'Val(180)
    Micro_Sign            : constant Character := 'µ'; --Character'Val(181)
    Pilcrow_Sign          : constant Character := '¶'; --Character'Val(182)
    Paragraph_Sign        : Character renames Pilcrow_Sign;
    Middle_Dot            : constant Character := '·'; --Character'Val(183)
    Cedilla               : constant Character := '¸'; --Character'Val(184)
    Superscript_One       : constant Character := '¹'; --Character'Val(185)
    Masculine_Ordinal_Indicator : constant Character := 'º'; --Character'Val(186)
    Right_Angle_Quotation : constant Character := '»'; --Character'Val(187)
    Fraction_One_Quarter  : constant Character := '¼'; --Character'Val(188)
    Fraction_One_Half     : constant Character := '½'; --Character'Val(189)
    Fraction_Three_Quarters : constant Character := '¾'; --Character'Val(190)
    Inverted_Question     : constant Character := '¿'; --Character'Val(191)

23  -- Character positions 192 (16#C0#) .. 207 (16#CF#):
    UC_A_Grave            : constant Character := 'À'; --Character'Val(192)
    UC_A_Acute            : constant Character := 'Á'; --Character'Val(193)
    UC_A_Circumflex       : constant Character := 'Â'; --Character'Val(194)
    UC_A_Tilde            : constant Character := 'Ã'; --Character'Val(195)
    UC_A_Diaeresis        : constant Character := 'Ä'; --Character'Val(196)
    UC_A_Ring             : constant Character := 'Å'; --Character'Val(197)
    UC_AE_Diphthong       : constant Character := 'Æ'; --Character'Val(198)
    UC_C_Cedilla          : constant Character := 'Ç'; --Character'Val(199)
    UC_E_Grave            : constant Character := 'È'; --Character'Val(200)
    UC_E_Acute            : constant Character := 'É'; --Character'Val(201)
    UC_E_Circumflex       : constant Character := 'Ê'; --Character'Val(202)
    UC_E_Diaeresis        : constant Character := 'Ë'; --Character'Val(203)
    UC_I_Grave            : constant Character := 'Ì'; --Character'Val(204)
    UC_I_Acute            : constant Character := 'Í'; --Character'Val(205)
    UC_I_Circumflex       : constant Character := 'Î'; --Character'Val(206)
    UC_I_Diaeresis        : constant Character := 'Ï'; --Character'Val(207)

24  -- Character positions 208 (16#D0#) .. 223 (16#DF#):
    UC_Icelandic_Eth      : constant Character := 'Ð'; --Character'Val(208)
    UC_N_Tilde            : constant Character := 'Ñ'; --Character'Val(209)
    UC_O_Grave            : constant Character := 'Ò'; --Character'Val(210)
    UC_O_Acute            : constant Character := 'Ó'; --Character'Val(211)
    UC_O_Circumflex       : constant Character := 'Ô'; --Character'Val(212)
    UC_O_Tilde            : constant Character := 'Õ'; --Character'Val(213)
    UC_O_Diaeresis        : constant Character := 'Ö'; --Character'Val(214)
    Multiplication_Sign    : constant Character := '×'; --Character'Val(215)
    UC_O_Oblique_Stroke   : constant Character := 'Ø'; --Character'Val(216)
    UC_U_Grave            : constant Character := 'Ù'; --Character'Val(217)
    UC_U_Acute            : constant Character := 'Ú'; --Character'Val(218)
    UC_U_Circumflex       : constant Character := 'Û'; --Character'Val(219)
    UC_U_Diaeresis        : constant Character := 'Ü'; --Character'Val(220)
    UC_Y_Acute            : constant Character := 'Ý'; --Character'Val(221)
    UC_Icelandic_Thorn    : constant Character := 'Þ'; --Character'Val(222)
    LC_German_Sharp_S     : constant Character := 'ß'; --Character'Val(223)

```

-- Character positions 224 (16#E0#) .. 239 (16#EF#):

25

```

LC_A_Grave           : constant Character := 'à' ; --Character'Val(224)
LC_A_Acute           : constant Character := 'á' ; --Character'Val(225)
LC_A_Circumflex      : constant Character := 'â' ; --Character'Val(226)
LC_A_Tilde           : constant Character := 'ã' ; --Character'Val(227)
LC_A_Diaeresis       : constant Character := 'ä' ; --Character'Val(228)
LC_A_Ring            : constant Character := 'å' ; --Character'Val(229)
LC_AE_Diphthong      : constant Character := 'æ' ; --Character'Val(230)
LC_C_Cedilla         : constant Character := 'ç' ; --Character'Val(231)
LC_E_Grave           : constant Character := 'è' ; --Character'Val(232)
LC_E_Acute           : constant Character := 'é' ; --Character'Val(233)
LC_E_Circumflex      : constant Character := 'ê' ; --Character'Val(234)
LC_E_Diaeresis       : constant Character := 'ë' ; --Character'Val(235)
LC_I_Grave           : constant Character := 'ì' ; --Character'Val(236)
LC_I_Acute           : constant Character := 'í' ; --Character'Val(237)
LC_I_Circumflex      : constant Character := 'î' ; --Character'Val(238)
LC_I_Diaeresis       : constant Character := 'ï' ; --Character'Val(239)

```

-- Character positions 240 (16#F0#) .. 255 (16#FF#):

26

```

LC_Icelandic_Eth     : constant Character := 'ð' ; --Character'Val(240)
LC_N_Tilde           : constant Character := 'ñ' ; --Character'Val(241)
LC_O_Grave           : constant Character := 'ò' ; --Character'Val(242)
LC_O_Acute           : constant Character := 'ó' ; --Character'Val(243)
LC_O_Circumflex      : constant Character := 'ô' ; --Character'Val(244)
LC_O_Tilde           : constant Character := 'õ' ; --Character'Val(245)
LC_O_Diaeresis       : constant Character := 'ö' ; --Character'Val(246)
Division_Sign         : constant Character := '÷' ; --Character'Val(247)
LC_O_Oblique_Stroke  : constant Character := 'ø' ; --Character'Val(248)
LC_U_Grave           : constant Character := 'ù' ; --Character'Val(249)
LC_U_Acute           : constant Character := 'ú' ; --Character'Val(250)
LC_U_Circumflex      : constant Character := 'û' ; --Character'Val(251)
LC_U_Diaeresis       : constant Character := 'ü' ; --Character'Val(252)
LC_Y_Acute           : constant Character := 'ý' ; --Character'Val(253)
LC_Icelandic_Thorn   : constant Character := 'þ' ; --Character'Val(254)
LC_Y_Diaeresis       : constant Character := 'ÿ' ; --Character'Val(255)

```

end Ada.Characters.Latin_1;

Implementation Permissions

An implementation may provide additional packages as children of Ada.Characters, to declare names for the symbols of the local character set or other character sets. 27

A.3.4 The Package Characters.Conversions

Static Semantics

1/2 The library package Characters.Conversions has the following declaration:

```

2/2  package Ada.Characters.Conversions is
      pragma Pure(Conversions);
3/2  function Is_Character (Item : in Wide_Character)      return Boolean;
      function Is_String   (Item : in Wide_String)        return Boolean;
      function Is_Character (Item : in Wide_Wide_Character) return Boolean;
      function Is_String   (Item : in Wide_Wide_String)   return Boolean;
      function Is_Wide_Character (Item : in Wide_Wide_Character)
          return Boolean;
      function Is_Wide_String   (Item : in Wide_Wide_String)
          return Boolean;
4/2  function To_Wide_Character (Item : in Character)      return Wide_Character;
      function To_Wide_String   (Item : in String)        return Wide_String;
      function To_Wide_Wide_Character (Item : in Character)
          return Wide_Wide_Character;
      function To_Wide_Wide_String (Item : in String)
          return Wide_Wide_String;
      function To_Wide_Wide_Character (Item : in Wide_Character)
          return Wide_Wide_Character;
      function To_Wide_Wide_String (Item : in Wide_String)
          return Wide_Wide_String;
5/2  function To_Character (Item      : in Wide_Character;
                          Substitute : in Character := ' ')
          return Character;
      function To_String   (Item      : in Wide_String;
                          Substitute  : in Character := ' ')
          return String;
      function To_Character (Item : in Wide_Wide_Character;
                          Substitute : in Character := ' ')
          return Character;
      function To_String   (Item : in Wide_Wide_String;
                          Substitute : in Character := ' ')
          return String;
      function To_Wide_Character (Item : in Wide_Wide_Character;
                          Substitute  : in Wide_Character := ' ')
          return Wide_Character;
      function To_Wide_String   (Item : in Wide_Wide_String;
                          Substitute  : in Wide_Character := ' ')
          return Wide_String;
6/2  end Ada.Characters.Conversions;
7/2  The functions in package Characters.Conversions test Wide_Wide_Character or Wide_Character values
      for membership in Wide_Character or Character, or convert between corresponding characters of
      Wide_Wide_Character, Wide_Character, and Character.
8/2  function Is_Character (Item : in Wide_Character) return Boolean;
9/2  Returns True if Wide_Character'Pos(Item) <= Character'Pos(Character'Last).
10/2 function Is_Character (Item : in Wide_Wide_Character) return Boolean;
11/2 Returns True if Wide_Wide_Character'Pos(Item) <= Character'Pos(Character'Last).
12/2 function Is_Wide_Character (Item : in Wide_Wide_Character) return Boolean;
13/2 Returns True if Wide_Wide_Character'Pos(Item) <= Wide_Character'Pos(Wide_Character'Last).
```

function Is_String (Item : in Wide_String)	return Boolean;	14/2
function Is_String (Item : in Wide_Wide_String)	return Boolean;	
Returns True if Is_Character(Item(I)) is True for each I in Item'Range.		15/2
function Is_Wide_String (Item : in Wide_Wide_String)	return Boolean;	16/2
Returns True if Is_Wide_Character(Item(I)) is True for each I in Item'Range.		17/2
function To_Character (Item : in Wide_Character;		18/2
Substitute : in Character := ' ')	return Character;	
function To_Character (Item : in Wide_Wide_Character;		
Substitute : in Character := ' ')	return Character;	
Returns the Character corresponding to Item if Is_Character(Item), and returns the Substitute Character otherwise.		19/2
function To_Wide_Character (Item : in Character)	return Wide_Character;	20/2
Returns the Wide_Character X such that Character'Pos(Item) = Wide_Character'Pos (X).		21/2
function To_Wide_Character (Item : in Wide_Wide_Character;		22/2
Substitute : in Wide_Character := ' ')	return Wide_Character;	
Returns the Wide_Character corresponding to Item if Is_Wide_Character(Item), and returns the Substitute Wide_Character otherwise.		23/2
function To_Wide_Wide_Character (Item : in Character)		24/2
return Wide_Wide_Character;		
Returns the Wide_Wide_Character X such that Character'Pos(Item) =		25/2
Wide_Wide_Character'Pos (X).		
function To_Wide_Wide_Character (Item : in Wide_Character)		26/2
return Wide_Wide_Character;		
Returns the Wide_Wide_Character X such that Wide_Character'Pos(Item) =		27/2
Wide_Wide_Character'Pos (X).		
function To_String (Item : in Wide_String;		28/2
Substitute : in Character := ' ')	return String;	
function To_String (Item : in Wide_Wide_String;		
Substitute : in Character := ' ')	return String;	
Returns the String whose range is 1..Item'Length and each of whose elements is given by		29/2
To_Character of the corresponding element in Item.		
function To_Wide_String (Item : in String)	return Wide_String;	30/2
Returns the Wide_String whose range is 1..Item'Length and each of whose elements is given by		31/2
To_Wide_Character of the corresponding element in Item.		
function To_Wide_String (Item : in Wide_Wide_String;		32/2
Substitute : in Wide_Character := ' ')	return Wide_String;	
Returns the Wide_String whose range is 1..Item'Length and each of whose elements is given by		33/2
To_Wide_Character of the corresponding element in Item with the given Substitute Wide_Character.		

- 34/2 **function** To_Wide_Wide_String (Item : **in** String) **return** Wide_Wide_String;
 function To_Wide_Wide_String (Item : **in** Wide_String)
 return Wide_Wide_String;
- 35/2 Returns the Wide_Wide_String whose range is 1..Item'Length and each of whose elements is given by To_Wide_Wide_Character of the corresponding element in Item.

A.4 String Handling

- 1/2 This clause presents the specifications of the package Strings and several child packages, which provide facilities for dealing with string data. Fixed-length, bounded-length, and unbounded-length strings are supported, for String, Wide_String, and Wide_Wide_String. The string-handling subprograms include searches for pattern strings and for characters in program-specified sets, translation (via a character-to-character mapping), and transformation (replacing, inserting, overwriting, and deleting of substrings).

A.4.1 The Package Strings

- 1 The package Strings provides declarations common to the string handling packages.

Static Semantics

- 2 The library package Strings has the following declaration:

```

3      package Ada.Strings is
4          pragma Pure(Strings);
5
6          Space      : constant Character      := ' ';
7          Wide_Space : constant Wide_Character := ' ';
8          Wide_Wide_Space : constant Wide_Wide_Character := ' ';
9
10         Length_Error, Pattern_Error, Index_Error, Translation_Error : exception;
11
12         type Alignment is (Left, Right, Center);
13         type Truncation is (Left, Right, Error);
14         type Membership is (Inside, Outside);
15         type Direction is (Forward, Backward);
16         type Trim_End is (Left, Right, Both);
17     end Ada.Strings;
```

A.4.2 The Package Strings.Maps

- 1 The package Strings.Maps defines the types, operations, and other entities needed for character sets and character-to-character mappings.

Static Semantics

- 2 The library package Strings.Maps has the following declaration:

```

3/2     package Ada.Strings.Maps is
4         pragma Pure(Maps);
5
6         -- Representation for a set of character values:
7         type Character_Set is private;
8         pragma Preelaborable_Initialization(Character_Set);
9
10        Null_Set : constant Character_Set;
11
12        type Character_Range is
13            record
14                Low : Character;
15                High : Character;
16            end record;
17        -- Represents Character range Low..High
```

```

type Character_Ranges is array (Positive range <>) of Character_Range;      7
function To_Set      (Ranges : in Character_Ranges) return Character_Set;      8
function To_Set      (Span   : in Character_Range) return Character_Set;      9
function To_Ranges   (Set     : in Character_Set)  return Character_Ranges;    10
function "="        (Left, Right : in Character_Set) return Boolean;          11
function "not"       (Right : in Character_Set)      return Character_Set;    12
function "and"       (Left, Right : in Character_Set) return Character_Set;
function "or"        (Left, Right : in Character_Set) return Character_Set;
function "xor"       (Left, Right : in Character_Set) return Character_Set;
function "-"        (Left, Right : in Character_Set) return Character_Set;
function Is_In (Element : in Character;
                  Set     : in Character_Set)
return Boolean;
function Is_Subset (Elements : in Character_Set;
                   Set       : in Character_Set)
return Boolean;
function "<=" (Left : in Character_Set;
              Right : in Character_Set)
return Boolean renames Is_Subset;
-- Alternative representation for a set of character values:
subtype Character_Sequence is String;
function To_Set (Sequence : in Character_Sequence) return Character_Set;
function To_Set (Singleton : in Character)      return Character_Set;
function To_Sequence (Set : in Character_Set) return Character_Sequence;
-- Representation for a character to character mapping:
type Character_Mapping is private;
pragma Preelaborable_Initialization(Character_Mapping);
function Value (Map      : in Character_Mapping;
                Element : in Character)
return Character;
Identity : constant Character_Mapping;
function To_Mapping (From, To : in Character_Sequence)
return Character_Mapping;
function To_Domain (Map : in Character_Mapping)
return Character_Sequence;
function To_Range (Map : in Character_Mapping)
return Character_Sequence;
type Character_Mapping_Function is
  access function (From : in Character) return Character;
private
  ... -- not specified by the language
end Ada.Strings.Maps;

```

An object of type `Character_Set` represents a set of characters.

`Null_Set` represents the set containing no characters.

An object `Obj` of type `Character_Range` represents the set of characters in the range `Obj.Low .. Obj.High`.

An object `Obj` of type `Character_Ranges` represents the union of the sets corresponding to `Obj(I)` for `I` in `Obj.Range`.

```

function To_Set (Ranges : in Character_Ranges) return Character_Set;
  If Ranges'Length=0 then Null_Set is returned; otherwise the returned value represents the set
  corresponding to Ranges.

```

```

33     function To_Set (Span : in Character_Range) return Character_Set;
34     The returned value represents the set containing each character in Span.
35     function To_Ranges (Set : in Character_Set) return Character_Ranges;
36     If Set = Null_Set then an empty Character_Ranges array is returned; otherwise the shortest array
        of contiguous ranges of Character values in Set, in increasing order of Low, is returned.
37     function "=" (Left, Right : in Character_Set) return Boolean;
38     The function "=" returns True if Left and Right represent identical sets, and False otherwise.
39     Each of the logical operators "not", "and", "or", and "xor" returns a Character_Set value that represents
        the set obtained by applying the corresponding operation to the set(s) represented by the parameter(s) of
        the operator. "-"(Left, Right) is equivalent to "and"(Left, "not"(Right)).
40     function Is_In (Element : in Character;
        Set : in Character_Set);
        return Boolean;
41     Is_In returns True if Element is in Set, and False otherwise.
42     function Is_Subset (Elements : in Character_Set;
        Set : in Character_Set)
        return Boolean;
43     Is_Subset returns True if Elements is a subset of Set, and False otherwise.
44     subtype Character_Sequence is String;
45     The Character_Sequence subtype is used to portray a set of character values and also to identify
        the domain and range of a character mapping.
46     function To_Set (Sequence : in Character_Sequence) return Character_Set;
        function To_Set (Singleton : in Character) return Character_Set;
47     Sequence portrays the set of character values that it explicitly contains (ignoring duplicates).
        Singleton portrays the set comprising a single Character. Each of the To_Set functions returns a
        Character_Set value that represents the set portrayed by Sequence or Singleton.
48     function To_Sequence (Set : in Character_Set) return Character_Sequence;
49     The function To_Sequence returns a Character_Sequence value containing each of the characters
        in the set represented by Set, in ascending order with no duplicates.
50     type Character_Mapping is private;
51     An object of type Character_Mapping represents a Character-to-Character mapping.
52     function Value (Map : in Character_Mapping;
        Element : in Character)
        return Character;
53     The function Value returns the Character value to which Element maps with respect to the
        mapping represented by Map.
54     A character C matches a pattern character P with respect to a given Character_Mapping value Map if
        Value(Map, C) = P. A string S matches a pattern string P with respect to a given Character_Mapping if
        their lengths are the same and if each character in S matches its corresponding character in the pattern
        string P.

```


String handling subprograms that deal with character mappings have parameters whose type is `Character_Mapping`. 55

```
Identity : constant Character_Mapping; 56
```

Identity maps each Character to itself. 57

```
function To_Mapping (From, To : in Character_Sequence) 58
return Character_Mapping;
```

To_Mapping produces a Character_Mapping such that each element of From maps to the corresponding element of To, and each other character maps to itself. If From'Length /= To'Length, or if some character is repeated in From, then Translation_Error is propagated. 59

```
function To_Domain (Map : in Character_Mapping) return Character_Sequence; 60
```

To_Domain returns the shortest Character_Sequence value D such that each character not in D maps to itself, and such that the characters in D are in ascending order. The lower bound of D is 1. 61

```
function To_Range (Map : in Character_Mapping) return Character_Sequence; 62
```

To_Range returns the Character_Sequence value R, such that if D = To_Domain(Map), then R has the same bounds as D, and D(I) maps to R(I) for each I in D'Range. 63/1

An object F of type `Character_Mapping_Function` maps a Character value C to the Character value `F.all(C)`, which is said to *match* C with respect to mapping function F. 64

NOTES

7 Character_Mapping and Character_Mapping_Function are used both for character equivalence mappings in the search subprograms (such as for case insensitivity) and as transformational mappings in the Translate subprograms. 65

8 To_Domain(Identity) and To_Range(Identity) each returns the null string. 66

Examples

To_Mapping("ABCD", "ZZAB") returns a Character_Mapping that maps 'A' and 'B' to 'Z', 'C' to 'A', 'D' to 'B', and each other Character to itself. 67

A.4.3 Fixed-Length String Handling

The language-defined package `Strings.Fixed` provides string-handling subprograms for fixed-length strings; that is, for values of type `Standard.String`. Several of these subprograms are procedures that modify the contents of a String that is passed as an **out** or an **in out** parameter; each has additional parameters to control the effect when the logical length of the result differs from the parameter's length. 1

For each function that returns a String, the lower bound of the returned value is 1. 2

The basic model embodied in the package is that a fixed-length string comprises significant characters and possibly padding (with space characters) on either or both ends. When a shorter string is copied to a longer string, padding is inserted, and when a longer string is copied to a shorter one, padding is stripped. The Move procedure in `Strings.Fixed`, which takes a String as an **out** parameter, allows the programmer to control these effects. Similar control is provided by the string transformation procedures. 3

Static Semantics

4 The library package Strings.Fixed has the following declaration:

```

5  with Ada.Strings.Maps;
6  package Ada.Strings.Fixed is
7      pragma Preelaborate(Fixed);
8  -- "Copy" procedure for strings of possibly different lengths
9      procedure Move (Source : in String;
10                     Target : out String;
11                     Drop   : in Truncation := Error;
12                     Justify : in Alignment := Left;
13                     Pad    : in Character := Space);
14
15  -- Search subprograms
16
17  8.1/2 function Index (Source : in String;
18                     Pattern : in String;
19                     From     : in Positive;
20                     Going    : in Direction := Forward;
21                     Mapping  : in Maps.Character_Mapping := Maps.Identity)
22      return Natural;
23
24  8.2/2 function Index (Source : in String;
25                     Pattern : in String;
26                     From     : in Positive;
27                     Going    : in Direction := Forward;
28                     Mapping  : in Maps.Character_Mapping_Function)
29      return Natural;
30
31  9 function Index (Source : in String;
32                  Pattern : in String;
33                  Going    : in Direction := Forward;
34                  Mapping  : in Maps.Character_Mapping
35                          := Maps.Identity)
36      return Natural;
37
38  10 function Index (Source : in String;
39                  Pattern : in String;
40                  Going    : in Direction := Forward;
41                  Mapping  : in Maps.Character_Mapping_Function)
42      return Natural;
43
44  10.1/2 function Index (Source : in String;
45                      Set      : in Maps.Character_Set;
46                      From     : in Positive;
47                      Test     : in Membership := Inside;
48                      Going    : in Direction := Forward)
49      return Natural;
50
51  11 function Index (Source : in String;
52                  Set      : in Maps.Character_Set;
53                  Test     : in Membership := Inside;
54                  Going    : in Direction := Forward)
55      return Natural;
56
57  11.1/2 function Index_Non_Blank (Source : in String;
58                                From     : in Positive;
59                                Going    : in Direction := Forward)
60      return Natural;
61
62  12 function Index_Non_Blank (Source : in String;
63                              Going    : in Direction := Forward)
64      return Natural;
65
66  13 function Count (Source : in String;
67                  Pattern : in String;
68                  Mapping  : in Maps.Character_Mapping
69                          := Maps.Identity)
70      return Natural;
```

```

function Count (Source   : in String;                               14
                Pattern  : in String;
                Mapping   : in Maps.Character_Mapping_Function)
    return Natural;
function Count (Source   : in String;                               15
                Set       : in Maps.Character_Set)
    return Natural;
procedure Find-Token (Source : in String;                            15.1/3
                    Set      : in Maps.Character_Set;
                    From     : in Positive;
                    Test     : in Membership;
                    First    : out Positive;
                    Last     : out Natural);

procedure Find-Token (Source : in String;                            16
                    Set      : in Maps.Character_Set;
                    Test     : in Membership;
                    First    : out Positive;
                    Last     : out Natural);

-- String translation subprograms                                     17
function Translate (Source : in String;                               18
                  Mapping  : in Maps.Character_Mapping)
    return String;
procedure Translate (Source : in out String;                          19
                  Mapping  : in Maps.Character_Mapping);
function Translate (Source : in String;                               20
                  Mapping  : in Maps.Character_Mapping_Function)
    return String;
procedure Translate (Source : in out String;                          21
                  Mapping  : in Maps.Character_Mapping_Function);

-- String transformation subprograms                                 22
function Replace_Slice (Source : in String;                            23
                    Low       : in Positive;
                    High      : in Natural;
                    By        : in String)
    return String;
procedure Replace_Slice (Source : in out String;                      24
                    Low       : in Positive;
                    High      : in Natural;
                    By        : in String;
                    Drop      : in Truncation := Error;
                    Justify   : in Alignment  := Left;
                    Pad       : in Character  := Space);
function Insert (Source : in String;                                   25
                Before  : in Positive;
                New_Item : in String)
    return String;
procedure Insert (Source : in out String;                              26
                Before  : in Positive;
                New_Item : in String;
                Drop     : in Truncation := Error);
function Overwrite (Source : in String;                               27
                  Position : in Positive;
                  New_Item : in String)
    return String;
procedure Overwrite (Source : in out String;                          28
                  Position : in Positive;
                  New_Item : in String;
                  Drop     : in Truncation := Right);

```

```

29     function Delete (Source : in String;
                      From   : in Positive;
                      Through : in Natural)
        return String;
30     procedure Delete (Source : in out String;
                      From   : in Positive;
                      Through : in Natural;
                      Justify : in Alignment := Left;
                      Pad    : in Character := Space);
31     --String selector subprograms
        function Trim (Source : in String;
                      Side   : in Trim_End)
            return String;
32     procedure Trim (Source : in out String;
                      Side   : in Trim_End;
                      Justify : in Alignment := Left;
                      Pad    : in Character := Space);
33     function Trim (Source : in String;
                      Left   : in Maps.Character_Set;
                      Right  : in Maps.Character_Set)
        return String;
34     procedure Trim (Source : in out String;
                      Left   : in Maps.Character_Set;
                      Right  : in Maps.Character_Set;
                      Justify : in Alignment := Strings.Left;
                      Pad    : in Character := Space);
35     function Head (Source : in String;
                    Count  : in Natural;
                    Pad    : in Character := Space)
        return String;
36     procedure Head (Source : in out String;
                    Count  : in Natural;
                    Justify : in Alignment := Left;
                    Pad    : in Character := Space);
37     function Tail (Source : in String;
                    Count  : in Natural;
                    Pad    : in Character := Space)
        return String;
38     procedure Tail (Source : in out String;
                    Count  : in Natural;
                    Justify : in Alignment := Left;
                    Pad    : in Character := Space);
39     --String constructor functions
        function "*" (Left : in Natural;
                     Right : in Character) return String;
41     function "*" (Left : in Natural;
                     Right : in String) return String;
42     end Ada.Strings.Fixed;

```

43 The effects of the above subprograms are as follows.

```

44     procedure Move (Source : in String;
                    Target  : out String;
                    Drop    : in Truncation := Error;
                    Justify : in Alignment := Left;
                    Pad     : in Character := Space);

```

45 The Move procedure copies characters from Source to Target. If Source has the same length as Target, then the effect is to assign Source to Target. If Source is shorter than Target then:

- 46 • If Justify=Left, then Source is copied into the first Source'Length characters of Target.

- If Justify=Right, then Source is copied into the last Source'Length characters of Target. 47
- If Justify=Center, then Source is copied into the middle Source'Length characters of Target. In this case, if the difference in length between Target and Source is odd, then the extra Pad character is on the right. 48
- Pad is copied to each Target character not otherwise assigned. 49

If Source is longer than Target, then the effect is based on Drop. 50

- If Drop=Left, then the rightmost Target'Length characters of Source are copied into Target. 51
- If Drop=Right, then the leftmost Target'Length characters of Source are copied into Target. 52
- If Drop=Error, then the effect depends on the value of the Justify parameter and also on whether any characters in Source other than Pad would fail to be copied: 53
 - If Justify=Left, and if each of the rightmost Source'Length-Target'Length characters in Source is Pad, then the leftmost Target'Length characters of Source are copied to Target. 54
 - If Justify=Right, and if each of the leftmost Source'Length-Target'Length characters in Source is Pad, then the rightmost Target'Length characters of Source are copied to Target. 55
 - Otherwise, Length_Error is propagated. 56

```
function Index (Source : in String;
               Pattern : in String;
               From    : in Positive;
               Going    : in Direction := Forward;
               Mapping  : in Maps.Character_Mapping := Maps.Identity)
  return Natural;
```

```
function Index (Source : in String;
               Pattern : in String;
               From    : in Positive;
               Going    : in Direction := Forward;
               Mapping  : in Maps.Character_Mapping_Function)
  return Natural;
```

Each Index function searches, starting from From, for a slice of Source, with length Pattern'Length, that matches Pattern with respect to Mapping; the parameter Going indicates the direction of the lookup. If Source is the null string, Index returns 0; otherwise, if From is not in Source'Range, then Index_Error is propagated. If Going = Forward, then Index returns the smallest index I which is greater than or equal to From such that the slice of Source starting at I matches Pattern. If Going = Backward, then Index returns the largest index I such that the slice of Source starting at I matches Pattern and has an upper bound less than or equal to From. If there is no such slice, then 0 is returned. If Pattern is the null string, then Pattern_Error is propagated. 56.2/3

```

57      function Index (Source   : in String;
                      Pattern   : in String;
                      Going     : in Direction := Forward;
                      Mapping    : in Maps.Character_Mapping
                                := Maps.Identity)
        return Natural;

      function Index (Source   : in String;
                      Pattern   : in String;
                      Going     : in Direction := Forward;
                      Mapping    : in Maps.Character_Mapping_Function)
        return Natural;

58/2      If Going = Forward, returns

58.1/2      Index (Source, Pattern, Source'First, Forward, Mapping);

58.2/2      otherwise returns

58.3/2      Index (Source, Pattern, Source'Last, Backward, Mapping);

58.4/2      function Index (Source   : in String;
                          Set       : in Maps.Character_Set;
                          From      : in Positive;
                          Test      : in Membership := Inside;
                          Going     : in Direction := Forward)
        return Natural;

58.5/3      Index searches for the first or last occurrence of any of a set of characters (when Test=Inside), or
any of the complement of a set of characters (when Test=Outside). If Source is the null string,
Index returns 0; otherwise, if From is not in Source'Range, then Index_Error is propagated.
Otherwise, it returns the smallest index I >= From (if Going=Forward) or the largest index I <=
From (if Going=Backward) such that Source(I) satisfies the Test condition with respect to Set; it
returns 0 if there is no such Character in Source.

59      function Index (Source : in String;
                      Set      : in Maps.Character_Set;
                      Test     : in Membership := Inside;
                      Going    : in Direction := Forward)
        return Natural;

60/2      If Going = Forward, returns

60.1/2      Index (Source, Set, Source'First, Test, Forward);

60.2/2      otherwise returns

60.3/2      Index (Source, Set, Source'Last, Test, Backward);

60.4/2      function Index_Non_Blank (Source : in String;
                                    From    : in Positive;
                                    Going    : in Direction := Forward)
        return Natural;

60.5/2      Returns Index (Source, Maps.To_Set(Space), From, Outside, Going);

61      function Index_Non_Blank (Source : in String;
                                    Going  : in Direction := Forward)
        return Natural;

62      Returns Index(Source, Maps.To_Set(Space), Outside, Going)

```

function Count (Source : in String; Pattern : in String; Mapping : in Maps.Character_Mapping := Maps.Identity) return Natural;	63
function Count (Source : in String; Pattern : in String; Mapping : in Maps.Character_Mapping_Function) return Natural;	
Returns the maximum number of nonoverlapping slices of Source that match Pattern with respect to Mapping. If Pattern is the null string then Pattern_Error is propagated.	64
function Count (Source : in String; Set : in Maps.Character_Set) return Natural;	65
Returns the number of occurrences in Source of characters that are in Set.	66
procedure Find-Token (Source : in String; Set : in Maps.Character_Set; From : in Positive; Test : in Membership; First : out Positive; Last : out Natural);	66.1/3
If Source is not the null string and From is not in Source'Range, then Index_Error is raised. Otherwise, First is set to the index of the first character in Source(From..Source'Last) that satisfies the Test condition. Last is set to the largest index such that all characters in Source(First..Last) satisfy the Test condition. If no characters in Source(From..Source'Last) satisfy the Test condition, First is set to From, and Last is set to 0.	66.2/3
procedure Find-Token (Source : in String; Set : in Maps.Character_Set; Test : in Membership; First : out Positive; Last : out Natural);	67
Equivalent to Find-Token (Source, Set, Source'First, Test, First, Last).	68/3
function Translate (Source : in String; Mapping : in Maps.Character_Mapping) return String;	69
function Translate (Source : in String; Mapping : in Maps.Character_Mapping_Function) return String;	
Returns the string S whose length is Source'Length and such that S(I) is the character to which Mapping maps the corresponding element of Source, for I in 1..Source'Length.	70
procedure Translate (Source : in out String; Mapping : in Maps.Character_Mapping);	71
procedure Translate (Source : in out String; Mapping : in Maps.Character_Mapping_Function);	
Equivalent to Source := Translate(Source, Mapping).	72

```

73      function Replace_Slice (Source   : in String;
                               Low      : in Positive;
                               High     : in Natural;
                               By       : in String)
      return String;

```

74/1 If Low > Source'Last+1, or High < Source'First-1, then Index_Error is propagated. Otherwise:

- 74.1/1 • If High >= Low, then the returned string comprises Source(Source'First..Low-1) & By & Source(High+1..Source'Last), but with lower bound 1.
- 74.2/1 • If High < Low, then the returned string is Insert(Source, Before=>Low, New_Item=>By).

```

75      procedure Replace_Slice (Source   : in out String;
                               Low      : in Positive;
                               High     : in Natural;
                               By       : in String;
                               Drop     : in Truncation := Error;
                               Justify  : in Alignment  := Left;
                               Pad     : in Character  := Space);

```

76 Equivalent to Move(Replace_Slice(Source, Low, High, By), Source, Drop, Justify, Pad).

```

77      function Insert (Source   : in String;
                       Before   : in Positive;
                       New_Item : in String)
      return String;

```

78 Propagates Index_Error if Before is not in Source'First .. Source'Last+1; otherwise returns Source(Source'First..Before-1) & New_Item & Source(Before..Source'Last), but with lower bound 1.

```

79      procedure Insert (Source   : in out String;
                       Before   : in Positive;
                       New_Item : in String;
                       Drop     : in Truncation := Error);

```

80 Equivalent to Move(Insert(Source, Before, New_Item), Source, Drop).

```

81      function Overwrite (Source   : in String;
                          Position : in Positive;
                          New_Item : in String)
      return String;

```

82 Propagates Index_Error if Position is not in Source'First .. Source'Last+1; otherwise returns the string obtained from Source by consecutively replacing characters starting at Position with corresponding characters from New_Item. If the end of Source is reached before the characters in New_Item are exhausted, the remaining characters from New_Item are appended to the string.

```

83      procedure Overwrite (Source   : in out String;
                          Position : in Positive;
                          New_Item : in String;
                          Drop     : in Truncation := Right);

```

84 Equivalent to Move(Overwrite(Source, Position, New_Item), Source, Drop).

```

85      function Delete (Source   : in String;
                      From      : in Positive;
                      Through   : in Natural)
      return String;

```

86/1 If From <= Through, the returned string is Replace_Slice(Source, From, Through, ""), otherwise it is Source with lower bound 1.

procedure Delete (Source : in out String; From : in Positive; Through : in Natural; Justify : in Alignment := Left; Pad : in Character := Space);	87
Equivalent to Move>Delete(Source, From, Through), Source, Justify => Justify, Pad => Pad).	88
function Trim (Source : in String; Side : in Trim_End) return String;	89
Returns the string obtained by removing from Source all leading Space characters (if Side = Left), all trailing Space characters (if Side = Right), or all leading and trailing Space characters (if Side = Both).	90
procedure Trim (Source : in out String; Side : in Trim_End; Justify : in Alignment := Left; Pad : in Character := Space);	91
Equivalent to Move>Trim(Source, Side), Source, Justify=>Justify, Pad=>Pad).	92
function Trim (Source : in String; Left : in Maps.Character_Set; Right : in Maps.Character_Set) return String;	93
Returns the string obtained by removing from Source all leading characters in Left and all trailing characters in Right.	94
procedure Trim (Source : in out String; Left : in Maps.Character_Set; Right : in Maps.Character_Set; Justify : in Alignment := Strings.Left; Pad : in Character := Space);	95
Equivalent to Move>Trim(Source, Left, Right), Source, Justify => Justify, Pad=>Pad).	96
function Head (Source : in String; Count : in Natural; Pad : in Character := Space) return String;	97
Returns a string of length Count. If Count <= Source'Length, the string comprises the first Count characters of Source. Otherwise its contents are Source concatenated with Count–Source'Length Pad characters.	98
procedure Head (Source : in out String; Count : in Natural; Justify : in Alignment := Left; Pad : in Character := Space);	99
Equivalent to Move>Head(Source, Count, Pad), Source, Drop=>Error, Justify=>Justify, Pad=>Pad).	100
function Tail (Source : in String; Count : in Natural; Pad : in Character := Space) return String;	101
Returns a string of length Count. If Count <= Source'Length, the string comprises the last Count characters of Source. Otherwise its contents are Count–Source'Length Pad characters concatenated with Source.	102

```

103  procedure Tail (Source  : in out String;
                  Count   : in Natural;
                  Justify  : in Alignment := Left;
                  Pad     : in Character := Space);

104      Equivalent to Move(Tail(Source, Count, Pad), Source, Drop=>Error, Justify=>Justify,
                          Pad=>Pad).

105  function "*" (Left  : in Natural;
                Right : in Character) return String;

function "*" (Left  : in Natural;
                Right : in String) return String;

106/1      These functions replicate a character or string a specified number of times. The first function
            returns a string whose length is Left and each of whose elements is Right. The second function
            returns a string whose length is Left*Right'Length and whose value is the null string if Left = 0
            and otherwise is (Left-1)*Right & Right with lower bound 1.

NOTES
107  9 In the Index and Count functions taking Pattern and Mapping parameters, the actual String parameter passed to Pattern
    should comprise characters occurring as target characters of the mapping. Otherwise the pattern will not match.

108  10 In the Insert subprograms, inserting at the end of a string is obtained by passing Source'Last+1 as the Before
    parameter.

109  11 If a null Character_Mapping_Function is passed to any of the string handling subprograms, Constraint_Error is
    propagated.

```

A.4.4 Bounded-Length String Handling

1 The language-defined package Strings.Bounded provides a generic package each of whose instances yields a private type Bounded_String and a set of operations. An object of a particular Bounded_String type represents a String whose low bound is 1 and whose length can vary conceptually between 0 and a maximum size established at the generic instantiation. The subprograms for fixed-length string handling are either overloaded directly for Bounded_String, or are modified as needed to reflect the variability in length. Additionally, since the Bounded_String type is private, appropriate constructor and selector operations are provided.

Static Semantics

2 The library package Strings.Bounded has the following declaration:

```

3  with Ada.Strings.Maps;
package Ada.Strings.Bounded is
    pragma Preelaborate(Bounded);

4  generic
    Max : Positive;    -- Maximum length of a Bounded_String
package Generic_Bounded_Length is

5      Max_Length : constant Positive := Max;

6      type Bounded_String is private;

7      Null_Bounded_String : constant Bounded_String;

8      subtype Length_Range is Natural range 0 .. Max_Length;

9      function Length (Source : in Bounded_String) return Length_Range;

10     -- Conversion, Concatenation, and Selection functions

11     function To_Bounded_String (Source : in String;
                                Drop   : in Truncation := Error)
        return Bounded_String;

```

```

function To_String (Source : in Bounded_String) return String;           12
procedure Set_Bounded_String                                           12.1/2
  (Target : out Bounded_String;
   Source : in String;
   Drop   : in Truncation := Error);

function Append (Left, Right : in Bounded_String;                       13
                 Drop        : in Truncation := Error)
  return Bounded_String;

function Append (Left : in Bounded_String;                               14
                 Right : in String;
                 Drop  : in Truncation := Error)
  return Bounded_String;

function Append (Left : in String;                                       15
                 Right : in Bounded_String;
                 Drop  : in Truncation := Error)
  return Bounded_String;

function Append (Left : in Bounded_String;                               16
                 Right : in Character;
                 Drop  : in Truncation := Error)
  return Bounded_String;

function Append (Left : in Character;                                    17
                 Right : in Bounded_String;
                 Drop  : in Truncation := Error)
  return Bounded_String;

procedure Append (Source : in out Bounded_String;                       18
                  New_Item : in Bounded_String;
                  Drop     : in Truncation := Error);

procedure Append (Source : in out Bounded_String;                       19
                  New_Item : in String;
                  Drop     : in Truncation := Error);

procedure Append (Source : in out Bounded_String;                       20
                  New_Item : in Character;
                  Drop     : in Truncation := Error);

function "&" (Left, Right : in Bounded_String)                          21
  return Bounded_String;

function "&" (Left : in Bounded_String; Right : in String)              22
  return Bounded_String;

function "&" (Left : in String; Right : in Bounded_String)              23
  return Bounded_String;

function "&" (Left : in Bounded_String; Right : in Character)           24
  return Bounded_String;

function "&" (Left : in Character; Right : in Bounded_String)           25
  return Bounded_String;

function Element (Source : in Bounded_String;                          26
                  Index  : in Positive)
  return Character;

procedure Replace_Element (Source : in out Bounded_String;              27
                           Index  : in Positive;
                           By     : in Character);

function Slice (Source : in Bounded_String;                             28
                Low     : in Positive;
                High    : in Natural)
  return String;

function Bounded_Slice                                           28.1/2
  (Source : in Bounded_String;
   Low    : in Positive;
   High   : in Natural)
  return Bounded_String;

```

```

28.2/2      procedure Bounded_Slice
              (Source : in   Bounded_String;
               Target  : out Bounded_String;
               Low     : in   Positive;
               High    : in   Natural);

29      function "=" (Left, Right : in Bounded_String) return Boolean;
      function "=" (Left : in Bounded_String; Right : in String)
        return Boolean;

30      function "=" (Left : in String; Right : in Bounded_String)
        return Boolean;

31      function "<" (Left, Right : in Bounded_String) return Boolean;
32      function "<" (Left : in Bounded_String; Right : in String)
        return Boolean;

33      function "<" (Left : in String; Right : in Bounded_String)
        return Boolean;

34      function "<=" (Left, Right : in Bounded_String) return Boolean;
35      function "<=" (Left : in Bounded_String; Right : in String)
        return Boolean;

36      function "<=" (Left : in String; Right : in Bounded_String)
        return Boolean;

37      function ">" (Left, Right : in Bounded_String) return Boolean;
38      function ">" (Left : in Bounded_String; Right : in String)
        return Boolean;

39      function ">" (Left : in String; Right : in Bounded_String)
        return Boolean;

40      function ">=" (Left, Right : in Bounded_String) return Boolean;
41      function ">=" (Left : in Bounded_String; Right : in String)
        return Boolean;

42      function ">=" (Left : in String; Right : in Bounded_String)
        return Boolean;

43/2      -- Search subprograms
43.1/2      function Index (Source : in Bounded_String;
                           Pattern : in String;
                           From    : in Positive;
                           Going   : in Direction := Forward;
                           Mapping  : in Maps.Character_Mapping := Maps.Identity)
        return Natural;

43.2/2      function Index (Source : in Bounded_String;
                           Pattern : in String;
                           From    : in Positive;
                           Going   : in Direction := Forward;
                           Mapping  : in Maps.Character_Mapping_Function)
        return Natural;

44      function Index (Source : in Bounded_String;
                           Pattern : in String;
                           Going   : in Direction := Forward;
                           Mapping  : in Maps.Character_Mapping
                                     := Maps.Identity)
        return Natural;

45      function Index (Source : in Bounded_String;
                           Pattern : in String;
                           Going   : in Direction := Forward;
                           Mapping  : in Maps.Character_Mapping_Function)
        return Natural;

```

```

function Index (Source : in Bounded_String;                                45.1/2
                Set    : in Maps.Character_Set;
                From    : in Positive;
                Test     : in Membership := Inside;
                Going    : in Direction := Forward)
    return Natural;

function Index (Source : in Bounded_String;                                46
                Set    : in Maps.Character_Set;
                Test     : in Membership := Inside;
                Going    : in Direction := Forward)
    return Natural;

function Index_Non_Blank (Source : in Bounded_String;                      46.1/2
                        From    : in Positive;
                        Going    : in Direction := Forward)
    return Natural;

function Index_Non_Blank (Source : in Bounded_String;                      47
                        Going    : in Direction := Forward)
    return Natural;

function Count (Source : in Bounded_String;                                48
                Pattern : in String;
                Mapping  : in Maps.Character_Mapping
                        := Maps.Identity)
    return Natural;

function Count (Source : in Bounded_String;                                49
                Pattern : in String;
                Mapping  : in Maps.Character_Mapping_Function)
    return Natural;

function Count (Source : in Bounded_String;                                50
                Set     : in Maps.Character_Set)
    return Natural;

procedure Find-Token (Source : in Bounded_String;                          50.1/3
                    Set    : in Maps.Character_Set;
                    From    : in Positive;
                    Test     : in Membership;
                    First    : out Positive;
                    Last     : out Natural);

procedure Find-Token (Source : in Bounded_String;                          51
                    Set    : in Maps.Character_Set;
                    Test     : in Membership;
                    First    : out Positive;
                    Last     : out Natural);

-- String translation subprograms                                          52

function Translate (Source : in Bounded_String;                            53
                  Mapping  : in Maps.Character_Mapping)
    return Bounded_String;

procedure Translate (Source : in out Bounded_String;                      54
                  Mapping  : in Maps.Character_Mapping);

function Translate (Source : in Bounded_String;                            55
                  Mapping  : in Maps.Character_Mapping_Function)
    return Bounded_String;

procedure Translate (Source : in out Bounded_String;                      56
                  Mapping  : in Maps.Character_Mapping_Function);

-- String transformation subprograms                                       57

function Replace_Slice (Source : in Bounded_String;                        58
                      Low      : in Positive;
                      High     : in Natural;
                      By       : in String;
                      Drop     : in Truncation := Error)
    return Bounded_String;

```

```

59     procedure Replace_Slice (Source   : in out Bounded_String;
                               Low      : in Positive;
                               High     : in Natural;
                               By       : in String;
                               Drop     : in Truncation := Error);

60     function Insert (Source   : in Bounded_String;
                      Before   : in Positive;
                      New_Item  : in String;
                      Drop     : in Truncation := Error)
        return Bounded_String;

61     procedure Insert (Source   : in out Bounded_String;
                      Before   : in Positive;
                      New_Item  : in String;
                      Drop     : in Truncation := Error);

62     function Overwrite (Source   : in Bounded_String;
                         Position  : in Positive;
                         New_Item  : in String;
                         Drop     : in Truncation := Error)
        return Bounded_String;

63     procedure Overwrite (Source   : in out Bounded_String;
                         Position  : in Positive;
                         New_Item  : in String;
                         Drop     : in Truncation := Error);

64     function Delete (Source   : in Bounded_String;
                     From      : in Positive;
                     Through   : in Natural)
        return Bounded_String;

65     procedure Delete (Source   : in out Bounded_String;
                     From      : in Positive;
                     Through   : in Natural);

66     --String selector subprograms
67     function Trim (Source : in Bounded_String;
                   Side   : in Trim_End)
        return Bounded_String;
        procedure Trim (Source : in out Bounded_String;
                   Side   : in Trim_End);

68     function Trim (Source : in Bounded_String;
                   Left    : in Maps.Character_Set;
                   Right   : in Maps.Character_Set)
        return Bounded_String;

69     procedure Trim (Source : in out Bounded_String;
                   Left    : in Maps.Character_Set;
                   Right   : in Maps.Character_Set);

70     function Head (Source : in Bounded_String;
                   Count   : in Natural;
                   Pad     : in Character := Space;
                   Drop    : in Truncation := Error)
        return Bounded_String;

71     procedure Head (Source : in out Bounded_String;
                   Count   : in Natural;
                   Pad     : in Character := Space;
                   Drop    : in Truncation := Error);

72     function Tail (Source : in Bounded_String;
                   Count   : in Natural;
                   Pad     : in Character := Space;
                   Drop    : in Truncation := Error)
        return Bounded_String;

```

```

procedure Tail (Source : in out Bounded_String;
                Count  : in Natural;
                Pad    : in Character := Space;
                Drop   : in Truncation := Error);
73

--String constructor subprograms
74

function "*" (Left  : in Natural;
              Right : in Character)
75
    return Bounded_String;

function "*" (Left  : in Natural;
              Right : in String)
76
    return Bounded_String;

function "*" (Left  : in Natural;
              Right : in Bounded_String)
77
    return Bounded_String;

function Replicate (Count : in Natural;
                    Item   : in Character;
                    Drop   : in Truncation := Error)
78
    return Bounded_String;

function Replicate (Count : in Natural;
                    Item   : in String;
                    Drop   : in Truncation := Error)
79
    return Bounded_String;

function Replicate (Count : in Natural;
                    Item   : in Bounded_String;
                    Drop   : in Truncation := Error)
80
    return Bounded_String;

private
81
    ... -- not specified by the language

end Generic_Bounded_Length;

end Ada.Strings.Bounded;
82

```

Null_Bounded_String represents the null string. If an object of type Bounded_String is not otherwise initialized, it will be initialized to the same value as Null_Bounded_String.
83

```

function Length (Source : in Bounded_String) return Length_Range;
84
    The Length function returns the length of the string represented by Source.
85

```

```

function To_Bounded_String (Source : in String;
                           Drop    : in Truncation := Error)
86
    return Bounded_String;

```

If Source/Length <= Max_Length then this function returns a Bounded_String that represents Source. Otherwise the effect depends on the value of Drop:
87

- If Drop=Left, then the result is a Bounded_String that represents the string comprising the rightmost Max_Length characters of Source.
88
- If Drop=Right, then the result is a Bounded_String that represents the string comprising the leftmost Max_Length characters of Source.
89
- If Drop=Error, then Strings.Length_Error is propagated.
90

```

function To_String (Source : in Bounded_String) return String;
91
    To_String returns the String value with lower bound 1 represented by Source. If B is a
92
    Bounded_String, then B = To_Bounded_String(To_String(B)).

```

```

92.1/2      procedure Set_Bounded_String
              (Target : out Bounded_String;
               Source : in String;
               Drop   : in Truncation := Error);

```

92.2/2 Equivalent to Target := To_Bounded_String (Source, Drop);

93 Each of the Append functions returns a Bounded_String obtained by concatenating the string or character given or represented by one of the parameters, with the string or character given or represented by the other parameter, and applying To_Bounded_String to the concatenation result string, with Drop as provided to the Append function.

94 Each of the procedures `Append(Source, New_Item, Drop)` has the same effect as the corresponding assignment `Source := Append(Source, New_Item, Drop)`.

95 Each of the "&" functions has the same effect as the corresponding Append function, with Error as the Drop parameter.

```

96     function Element (Source : in Bounded_String;
                        Index  : in Positive)
        return Character;

```

97 Returns the character at position Index in the string represented by Source; propagates Index_Error if Index > Length(Source).

```
98      procedure Replace_Element (Source : in out Bounded_String;  
                                Index   : in Positive;  
                                By       : in Character);
```

99 Updates Source such that the character at position Index in the string represented by Source is By; propagates Index Error if Index > Length(Source).

[illegible]

101/1 Returns the slice at positions Low through High in the string represented by Source; propagates Index_Error if Low > Length(Source)+1 or High > Length(Source). The bounds of the returned string are Low and High..

```

101.1/2      function Bounded_Slice
              (Source : in Bounded_String;
               Low    : in Positive;
               High   : in Natural)
              return Bounded_String;

```

101.2/2	Returns the slice at positions Low through High in the string represented by Source as a bounded string; propagates Index Error if Low > Length(Source)+1 or High > Length(Source).
---------	---

```

101.3/2  procedure Bounded_Slice
          (Source : in    Bounded_String;
           Target  : out  Bounded_String;
           Low     : in    Positive;
           High    : in    Natural);

```

101.4/2 Equivalent to Target := Bounded Slice (Source, Low, High);

102 Each of the functions "=", "<", ">", "<=", and ">=" returns the same result as the corresponding String operation applied to the String values given or represented by the two parameters.

Each of the search subprograms (Index, Index_Non_Blank, Count, Find-Token) has the same effect as the corresponding subprogram in Strings.Fixed applied to the string represented by the Bounded_String parameter. 103

Each of the Translate subprograms, when applied to a Bounded_String, has an analogous effect to the corresponding subprogram in Strings.Fixed. For the Translate function, the translation is applied to the string represented by the Bounded_String parameter, and the result is converted (via To_Bounded_String) to a Bounded_String. For the Translate procedure, the string represented by the Bounded_String parameter after the translation is given by the Translate function for fixed-length strings applied to the string represented by the original value of the parameter. 104

Each of the transformation subprograms (Replace_Slice, Insert, Overwrite, Delete), selector subprograms (Trim, Head, Tail), and constructor functions ("*") has an effect based on its corresponding subprogram in Strings.Fixed, and Replicate is based on Fixed.*". In the case of a function, the corresponding fixed-length string subprogram is applied to the string represented by the Bounded_String parameter. To_Bounded_String is applied the result string, with Drop (or Error in the case of Generic_Bounded_Length.*) determining the effect when the string length exceeds Max_Length. In the case of a procedure, the corresponding function in Strings.Bounded.Generic_Bounded_Length is applied, with the result assigned into the Source parameter. 105/1

Implementation Advice

Bounded string objects should not be implemented by implicit pointers and dynamic allocation. 106

A.4.5 Unbounded-Length String Handling

The language-defined package Strings.Unbounded provides a private type Unbounded_String and a set of operations. An object of type Unbounded_String represents a String whose low bound is 1 and whose length can vary conceptually between 0 and Natural'Last. The subprograms for fixed-length string handling are either overloaded directly for Unbounded_String, or are modified as needed to reflect the flexibility in length. Since the Unbounded_String type is private, relevant constructor and selector operations are provided. 1

Static Semantics

The library package Strings.Unbounded has the following declaration: 2

```

with Ada.Strings.Maps;
package Ada.Strings.Unbounded is
  pragma Preelaborate(Unbounded);
  type Unbounded_String is private;
  pragma Preelaborable_Initialization(Unbounded_String);
  Null_Unbounded_String : constant Unbounded_String;
  function Length (Source : in Unbounded_String) return Natural;
  type String_Access is access all String;
  procedure Free (X : in out String_Access);
  -- Conversion, Concatenation, and Selection functions
  function To_Unbounded_String (Source : in String)
    return Unbounded_String;
  function To_Unbounded_String (Length : in Natural)
    return Unbounded_String;
  function To_String (Source : in Unbounded_String) return String;

```

3
4/2
5
6
7
8
9
10
11

```

11.1/2  procedure Set_Unbounded_String
        (Target : out Unbounded_String;
         Source  : in String);

12      procedure Append (Source  : in out Unbounded_String;
                       New_Item : in Unbounded_String);

13      procedure Append (Source  : in out Unbounded_String;
                       New_Item : in String);

14      procedure Append (Source  : in out Unbounded_String;
                       New_Item : in Character);

15      function "&" (Left, Right : in Unbounded_String)
        return Unbounded_String;

16      function "&" (Left : in Unbounded_String; Right : in String)
        return Unbounded_String;

17      function "&" (Left : in String; Right : in Unbounded_String)
        return Unbounded_String;

18      function "&" (Left : in Unbounded_String; Right : in Character)
        return Unbounded_String;

19      function "&" (Left : in Character; Right : in Unbounded_String)
        return Unbounded_String;

20      function Element (Source : in Unbounded_String;
                       Index  : in Positive)
        return Character;

21      procedure Replace_Element (Source : in out Unbounded_String;
                                Index  : in Positive;
                                By      : in Character);

22      function Slice (Source : in Unbounded_String;
                     Low     : in Positive;
                     High    : in Natural)
        return String;

22.1/2  function Unbounded_Slice
        (Source : in Unbounded_String;
         Low    : in Positive;
         High   : in Natural)
        return Unbounded_String;

22.2/2  procedure Unbounded_Slice
        (Source : in Unbounded_String;
         Target  : out Unbounded_String;
         Low     : in Positive;
         High    : in Natural);

23      function "=" (Left, Right : in Unbounded_String) return Boolean;

24      function "=" (Left : in Unbounded_String; Right : in String)
        return Boolean;

25      function "=" (Left : in String; Right : in Unbounded_String)
        return Boolean;

26      function "<" (Left, Right : in Unbounded_String) return Boolean;

27      function "<" (Left : in Unbounded_String; Right : in String)
        return Boolean;

28      function "<" (Left : in String; Right : in Unbounded_String)
        return Boolean;

29      function "<=" (Left, Right : in Unbounded_String) return Boolean;

30      function "<=" (Left : in Unbounded_String; Right : in String)
        return Boolean;

31      function "<=" (Left : in String; Right : in Unbounded_String)
        return Boolean;

32      function ">" (Left, Right : in Unbounded_String) return Boolean;

```

```

function ">" (Left : in Unbounded_String; Right : in String)           33
    return Boolean;

function ">" (Left : in String; Right : in Unbounded_String)          34
    return Boolean;

function ">=" (Left, Right : in Unbounded_String) return Boolean;      35

function ">=" (Left : in Unbounded_String; Right : in String)          36
    return Boolean;

function ">=" (Left : in String; Right : in Unbounded_String)          37
    return Boolean;

-- Search subprograms                                                    38

function Index (Source : in Unbounded_String;                          38.1/2
    Pattern : in String;
    From : in Positive;
    Going : in Direction := Forward;
    Mapping : in Maps.Character_Mapping := Maps.Identity)

    return Natural;

function Index (Source : in Unbounded_String;                          38.2/2
    Pattern : in String;
    From : in Positive;
    Going : in Direction := Forward;
    Mapping : in Maps.Character_Mapping_Function)

    return Natural;

function Index (Source : in Unbounded_String;                          39
    Pattern : in String;
    Going : in Direction := Forward;
    Mapping : in Maps.Character_Mapping
        := Maps.Identity)

    return Natural;

function Index (Source : in Unbounded_String;                          40
    Pattern : in String;
    Going : in Direction := Forward;
    Mapping : in Maps.Character_Mapping_Function)

    return Natural;

function Index (Source : in Unbounded_String;                          40.1/2
    Set : in Maps.Character_Set;
    From : in Positive;
    Test : in Membership := Inside;
    Going : in Direction := Forward)

    return Natural;

function Index (Source : in Unbounded_String;                          41
    Set : in Maps.Character_Set;
    Test : in Membership := Inside;
    Going : in Direction := Forward) return Natural;

function Index_Non_Blank (Source : in Unbounded_String;                41.1/2
    From : in Positive;
    Going : in Direction := Forward)

    return Natural;

function Index_Non_Blank (Source : in Unbounded_String;                42
    Going : in Direction := Forward)

    return Natural;

function Count (Source : in Unbounded_String;                          43
    Pattern : in String;
    Mapping : in Maps.Character_Mapping
        := Maps.Identity)

    return Natural;

function Count (Source : in Unbounded_String;                          44
    Pattern : in String;
    Mapping : in Maps.Character_Mapping_Function)

    return Natural;

```

```

45     function Count (Source : in Unbounded_String;
                     Set      : in Maps.Character_Set)
                     return Natural;
45.1/3 procedure Find-Token (Source : in Unbounded_String;
                           Set      : in Maps.Character_Set;
                           From      : in Positive;
                           Test      : in Membership;
                           First     : out Positive;
                           Last      : out Natural);
46     procedure Find-Token (Source : in Unbounded_String;
                           Set      : in Maps.Character_Set;
                           Test      : in Membership;
                           First     : out Positive;
                           Last      : out Natural);
47 -- String translation subprograms
48     function Translate (Source : in Unbounded_String;
                        Mapping : in Maps.Character_Mapping)
                        return Unbounded_String;
49     procedure Translate (Source : in out Unbounded_String;
                        Mapping : in Maps.Character_Mapping);
50     function Translate (Source : in Unbounded_String;
                        Mapping : in Maps.Character_Mapping_Function)
                        return Unbounded_String;
51     procedure Translate (Source : in out Unbounded_String;
                        Mapping : in Maps.Character_Mapping_Function);
52 -- String transformation subprograms
53     function Replace_Slice (Source : in Unbounded_String;
                           Low      : in Positive;
                           High     : in Natural;
                           By       : in String)
                           return Unbounded_String;
54     procedure Replace_Slice (Source : in out Unbounded_String;
                           Low      : in Positive;
                           High     : in Natural;
                           By       : in String);
55     function Insert (Source : in Unbounded_String;
                    Before  : in Positive;
                    New_Item : in String)
                    return Unbounded_String;
56     procedure Insert (Source : in out Unbounded_String;
                    Before  : in Positive;
                    New_Item : in String);
57     function Overwrite (Source : in Unbounded_String;
                       Position : in Positive;
                       New_Item : in String)
                       return Unbounded_String;
58     procedure Overwrite (Source : in out Unbounded_String;
                       Position : in Positive;
                       New_Item : in String);
59     function Delete (Source : in Unbounded_String;
                    From      : in Positive;
                    Through   : in Natural)
                    return Unbounded_String;
60     procedure Delete (Source : in out Unbounded_String;
                    From      : in Positive;
                    Through   : in Natural);
61     function Trim (Source : in Unbounded_String;
                   Side     : in Trim_End)
                   return Unbounded_String;

```

```

procedure Trim (Source : in out Unbounded_String;           62
                Side   : in Trim_End);

function Trim (Source : in Unbounded_String;                63
                Left    : in Maps.Character_Set;
                Right   : in Maps.Character_Set)
    return Unbounded_String;

procedure Trim (Source : in out Unbounded_String;           64
                Left    : in Maps.Character_Set;
                Right   : in Maps.Character_Set);

function Head (Source : in Unbounded_String;                65
                Count   : in Natural;
                Pad      : in Character := Space)
    return Unbounded_String;

procedure Head (Source : in out Unbounded_String;           66
                Count   : in Natural;
                Pad      : in Character := Space);

function Tail (Source : in Unbounded_String;                67
                Count   : in Natural;
                Pad      : in Character := Space)
    return Unbounded_String;

procedure Tail (Source : in out Unbounded_String;           68
                Count   : in Natural;
                Pad      : in Character := Space);

function "*" (Left  : in Natural;                             69
              Right : in Character)
    return Unbounded_String;

function "*" (Left  : in Natural;                             70
              Right : in String)
    return Unbounded_String;

function "*" (Left  : in Natural;                             71
              Right : in Unbounded_String)
    return Unbounded_String;

private                                                72
    ... -- not specified by the language
end Ada.Strings.Unbounded;

```

The type `Unbounded_String` needs finalization (see 7.6). 72.1/2

`Null_Unbounded_String` represents the null String. If an object of type `Unbounded_String` is not otherwise initialized, it will be initialized to the same value as `Null_Unbounded_String`. 73

The function `Length` returns the length of the String represented by `Source`. 74

The type `String_Access` provides a (non-private) access type for explicit processing of unbounded-length strings. The procedure `Free` performs an unchecked deallocation of an object of type `String_Access`. 75

The function `To_Unbounded_String(Source : in String)` returns an `Unbounded_String` that represents `Source`. The function `To_Unbounded_String(Length : in Natural)` returns an `Unbounded_String` that represents an uninitialized String whose length is `Length`. 76

The function `To_String` returns the String with lower bound 1 represented by `Source`. `To_String` and `To_Unbounded_String` are related as follows: 77

- If `S` is a String, then `To_String(To_Unbounded_String(S)) = S`. 78
- If `U` is an `Unbounded_String`, then `To_Unbounded_String(To_String(U)) = U`. 79

The procedure `Set_Unbounded_String` sets `Target` to an `Unbounded_String` that represents `Source`. 79.1/2

- 80 For each of the Append procedures, the resulting string represented by the Source parameter is given by the concatenation of the original value of Source and the value of New_Item.
- 81 Each of the "&" functions returns an Unbounded_String obtained by concatenating the string or character given or represented by one of the parameters, with the string or character given or represented by the other parameter, and applying To_Unbounded_String to the concatenation result string.
- 82 The Element, Replace_Element, and Slice subprograms have the same effect as the corresponding bounded-length string subprograms.
- 82.1/2 The function Unbounded_Slice returns the slice at positions Low through High in the string represented by Source as an Unbounded_String. The procedure Unbounded_Slice sets Target to the Unbounded_String representing the slice at positions Low through High in the string represented by Source. Both routines propagate Index_Error if Low > Length(Source)+1 or High > Length(Source).
- 83 Each of the functions "=", "<", ">", "<=", and ">=" returns the same result as the corresponding String operation applied to the String values given or represented by Left and Right.
- 84 Each of the search subprograms (Index, Index_Non_Blank, Count, Find-Token) has the same effect as the corresponding subprogram in Strings.Fixed applied to the string represented by the Unbounded_String parameter.
- 85 The Translate function has an analogous effect to the corresponding subprogram in Strings.Fixed. The translation is applied to the string represented by the Unbounded_String parameter, and the result is converted (via To_Unbounded_String) to an Unbounded_String.
- 86 Each of the transformation functions (Replace_Slice, Insert, Overwrite, Delete), selector functions (Trim, Head, Tail), and constructor functions ("*") is likewise analogous to its corresponding subprogram in Strings.Fixed. For each of the subprograms, the corresponding fixed-length string subprogram is applied to the string represented by the Unbounded_String parameter, and To_Unbounded_String is applied the result string.
- 87 For each of the procedures Translate, Replace_Slice, Insert, Overwrite, Delete, Trim, Head, and Tail, the resulting string represented by the Source parameter is given by the corresponding function for fixed-length strings applied to the string represented by Source's original value.

Implementation Requirements

- 88 No storage associated with an Unbounded_String object shall be lost upon assignment or scope exit.

A.4.6 String-Handling Sets and Mappings

- 1 The language-defined package Strings.Maps.Constants declares Character_Set and Character_Mapping constants corresponding to classification and conversion functions in package Characters.Handling.

Static Semantics

- 2 The library package Strings.Maps.Constants has the following declaration:

```
3/2 package Ada.Strings.Maps.Constants is
    pragma Pure(Constants);
```

```

Control_Set           : constant Character_Set;           4
Graphic_Set           : constant Character_Set;
Letter_Set            : constant Character_Set;
Lower_Set             : constant Character_Set;
Upper_Set             : constant Character_Set;
Basic_Set             : constant Character_Set;
Decimal_Digit_Set     : constant Character_Set;
Hexadecimal_Digit_Set : constant Character_Set;
Alphanumeric_Set      : constant Character_Set;
Special_Set           : constant Character_Set;
ISO_646_Set           : constant Character_Set;

Lower_Case_Map         : constant Character_Mapping;      5
  --Maps to lower case for letters, else identity
Upper_Case_Map         : constant Character_Mapping;
  --Maps to upper case for letters, else identity
Basic_Map              : constant Character_Mapping;
  --Maps to basic letter for letters, else identity

private                                                     6
  ... -- not specified by the language
end Ada.Strings.Maps.Constants;

```

Each of these constants represents a correspondingly named set of characters or character mapping in Characters.Handling (see A.3.2). 7

A.4.7 Wide_String Handling

Facilities for handling strings of Wide_Character elements are found in the packages Strings.Wide_Maps, Strings.Wide_Fixed, Strings.Wide_Bounded, Strings.Wide_Unbounded, and Strings.Wide_Maps.Wide_Constants, and in the functions Strings.Wide_Hash, Strings.Wide_Fixed.Wide_Hash, Strings.Wide_Bounded.Wide_Hash, and Strings.Wide_Unbounded.Wide_Hash. They provide the same string-handling operations as the corresponding packages and functions for strings of Character elements. 1/2

Static Semantics

The package Strings.Wide_Maps has the following declaration. 2

```

package Ada.Strings.Wide_Maps is                             3
  pragma Preelaborate(Wide_Maps);
  -- Representation for a set of Wide_Character values:      4/2
  type Wide_Character_Set is private;
  pragma Preelaborable_Initialization(Wide_Character_Set);
  Null_Set : constant Wide_Character_Set;                    5
  type Wide_Character_Range is                                6
    record
      Low   : Wide_Character;
      High  : Wide_Character;
    end record;
  -- Represents Wide_Character range Low..High
  type Wide_Character_Ranges is array (Positive range <>)   7
    of Wide_Character_Range;
  function To_Set (Ranges : in Wide_Character_Ranges)        8
    return Wide_Character_Set;
  function To_Set (Span   : in Wide_Character_Range)         9
    return Wide_Character_Set;
  function To_Ranges (Set   : in Wide_Character_Set)         10
    return Wide_Character_Ranges;
  function "=" (Left, Right : in Wide_Character_Set) return Boolean; 11

```

```

12     function "not" (Right : in Wide_Character_Set)
        return Wide_Character_Set;
    function "and" (Left, Right : in Wide_Character_Set)
        return Wide_Character_Set;
    function "or" (Left, Right : in Wide_Character_Set)
        return Wide_Character_Set;
    function "xor" (Left, Right : in Wide_Character_Set)
        return Wide_Character_Set;
    function "-" (Left, Right : in Wide_Character_Set)
        return Wide_Character_Set;
13     function Is_In (Element : in Wide_Character;
        Set : in Wide_Character_Set)
        return Boolean;
14     function Is_Subset (Elements : in Wide_Character_Set;
        Set : in Wide_Character_Set)
        return Boolean;
15     function "<=" (Left : in Wide_Character_Set;
        Right : in Wide_Character_Set)
        return Boolean renames Is_Subset;
16     -- Alternative representation for a set of Wide_Character values:
    subtype Wide_Character_Sequence is Wide_String;
17     function To_Set (Sequence : in Wide_Character_Sequence)
        return Wide_Character_Set;
18     function To_Set (Singleton : in Wide_Character)
        return Wide_Character_Set;
19     function To_Sequence (Set : in Wide_Character_Set)
        return Wide_Character_Sequence;
20/2    -- Representation for a Wide_Character to Wide_Character mapping:
    type Wide_Character_Mapping is private;
    pragma Preelaborable_Initialization(Wide_Character_Mapping);
21     function Value (Map : in Wide_Character_Mapping;
        Element : in Wide_Character)
        return Wide_Character;
22     Identity : constant Wide_Character_Mapping;
23     function To_Mapping (From, To : in Wide_Character_Sequence)
        return Wide_Character_Mapping;
24     function To_Domain (Map : in Wide_Character_Mapping)
        return Wide_Character_Sequence;
25     function To_Range (Map : in Wide_Character_Mapping)
        return Wide_Character_Sequence;
26     type Wide_Character_Mapping_Function is
        access function (From : in Wide_Character) return Wide_Character;
27     private
        ... -- not specified by the language
    end Ada.Strings.Wide_Maps;

```

28 The context clause for each of the packages Strings.Wide_Fixed, Strings.Wide_Bounded, and Strings.Wide_Unbounded identifies Strings.Wide_Maps instead of Strings.Maps.

29/2 For each of the packages Strings.Fixed, Strings.Bounded, Strings.Unbounded, and Strings.Maps.Constants, and for functions Strings.Hash, Strings.Fixed.Hash, Strings.Bounded.Hash, and Strings.Unbounded.Hash, the corresponding wide string package has the same contents except that

- 30 • Wide_Space replaces Space
- 31 • Wide_Character replaces Character
- 32 • Wide_String replaces String

- Wide_Character_Set replaces Character_Set 33
- Wide_Character_Mapping replaces Character_Mapping 34
- Wide_Character_Mapping_Function replaces Character_Mapping_Function 35
- Wide_Maps replaces Maps 36
- Bounded_Wide_String replaces Bounded_String 37
- Null_Bounded_Wide_String replaces Null_Bounded_String 38
- To_Bounded_Wide_String replaces To_Bounded_String 39
- To_Wide_String replaces To_String 40
- Set_Bounded_Wide_String replaces Set_Bounded_String 40.1/2
- Unbounded_Wide_String replaces Unbounded_String 41
- Null_Unbounded_Wide_String replaces Null_Unbounded_String 42
- Wide_String_Access replaces String_Access 43
- To_Unbounded_Wide_String replaces To_Unbounded_String 44
- Set_Unbounded_Wide_String replaces Set_Unbounded_String 44.1/2

The following additional declaration is present in Strings.Wide_Maps.Wide_Constants: 45

```
Character_Set : constant Wide_Maps.Wide_Character_Set;
--Contains each Wide_Character value WC such that
--Characters.Conversions.Is_Character(WC) is True 46/2
```

Each Wide_Character_Set constant in the package Strings.Wide_Maps.Wide_Constants contains no values outside the Character portion of Wide_Character. Similarly, each Wide_Character_Mapping constant in this package is the identity mapping when applied to any element outside the Character portion of Wide_Character. 46.1/2

Pragma Pure is replaced by pragma Preelaborate in Strings.Wide_Maps.Wide_Constants. 46.2/2

NOTES

12 If a null Wide_Character_Mapping_Function is passed to any of the Wide_String handling subprograms, Constraint_Error is propagated. 47

This paragraph was deleted. 48/2

A.4.8 Wide_Wide_String Handling

Facilities for handling strings of Wide_Wide_Character elements are found in the packages Strings.-Wide_Wide_Maps, Strings.Wide_Wide_Fixed, Strings.Wide_Wide_Bounded, Strings.Wide_Wide_Unbounded, and Strings.Wide_Wide_Maps.Wide_Wide_Constants, and in the functions Strings.-Wide_Wide_Hash, Strings.Wide_Wide_Fixed.Wide_Wide_Hash, Strings.Wide_Wide_Bounded.Wide_Wide_Hash, and Strings.Wide_Wide_Unbounded.Wide_Wide_Hash. They provide the same string-handling operations as the corresponding packages and functions for strings of Character elements. 1/2

Static Semantics

The library package Strings.Wide_Wide_Maps has the following declaration. 2/2

```
package Ada.Strings.Wide_Wide_Maps is 3/2
  pragma Preelaborate(Wide_Wide_Maps);
```

```

4/2      -- Representation for a set of Wide_Wide_Character values:
         type Wide_Wide_Character_Set is private;
         pragma Preelaborable_Initialization(Wide_Wide_Character_Set);

5/2      Null_Set : constant Wide_Wide_Character_Set;

6/2      type Wide_Wide_Character_Range is
         record
             Low   : Wide_Wide_Character;
             High  : Wide_Wide_Character;
         end record;
         -- Represents Wide_Wide_Character range Low..High

7/2      type Wide_Wide_Character_Ranges is array (Positive range <>)
         of Wide_Wide_Character_Range;

8/2      function To_Set (Ranges : in Wide_Wide_Character_Ranges)
         return Wide_Wide_Character_Set;

9/2      function To_Set (Span : in Wide_Wide_Character_Range)
         return Wide_Wide_Character_Set;

10/2     function To_Ranges (Set : in Wide_Wide_Character_Set)
         return Wide_Wide_Character_Ranges;

11/2     function "=" (Left, Right : in Wide_Wide_Character_Set) return Boolean;

12/2     function "not" (Right : in Wide_Wide_Character_Set)
         return Wide_Wide_Character_Set;
         function "and" (Left, Right : in Wide_Wide_Character_Set)
         return Wide_Wide_Character_Set;
         function "or" (Left, Right : in Wide_Wide_Character_Set)
         return Wide_Wide_Character_Set;
         function "xor" (Left, Right : in Wide_Wide_Character_Set)
         return Wide_Wide_Character_Set;
         function "-" (Left, Right : in Wide_Wide_Character_Set)
         return Wide_Wide_Character_Set;

13/2     function Is_In (Element : in Wide_Wide_Character;
         Set : in Wide_Wide_Character_Set)
         return Boolean;

14/2     function Is_Subset (Elements : in Wide_Wide_Character_Set;
         Set : in Wide_Wide_Character_Set)
         return Boolean;

15/2     function "<=" (Left : in Wide_Wide_Character_Set;
         Right : in Wide_Wide_Character_Set)
         return Boolean renames Is_Subset;

16/2     -- Alternative representation for a set of Wide_Wide_Character values:
         subtype Wide_Wide_Character_Sequence is Wide_Wide_String;

17/2     function To_Set (Sequence : in Wide_Wide_Character_Sequence)
         return Wide_Wide_Character_Set;

18/2     function To_Set (Singleton : in Wide_Wide_Character)
         return Wide_Wide_Character_Set;

19/2     function To_Sequence (Set : in Wide_Wide_Character_Set)
         return Wide_Wide_Character_Sequence;

20/2     -- Representation for a Wide_Wide_Character to Wide_Wide_Character
         -- mapping:
         type Wide_Wide_Character_Mapping is private;
         pragma Preelaborable_Initialization(Wide_Wide_Character_Mapping);

21/2     function Value (Map : in Wide_Wide_Character_Mapping;
         Element : in Wide_Wide_Character)
         return Wide_Wide_Character;

22/2     Identity : constant Wide_Wide_Character_Mapping;

23/2     function To_Mapping (From, To : in Wide_Wide_Character_Sequence)
         return Wide_Wide_Character_Mapping;

```

```

function To_Domain (Map : in Wide_Wide_Character_Mapping)                24/2
    return Wide_Wide_Character_Sequence;

function To_Range (Map : in Wide_Wide_Character_Mapping)                25/2
    return Wide_Wide_Character_Sequence;

type Wide_Wide_Character_Mapping_Function is                            26/2
    access function (From : in Wide_Wide_Character)
        return Wide_Wide_Character;

private                                                                    27/2
    ... -- not specified by the language
end Ada.Strings.Wide_Wide_Maps;

```

The context clause for each of the packages Strings.Wide_Wide_Fixed, Strings.Wide_Wide_Bounded, and Strings.Wide_Wide_Unbounded identifies Strings.Wide_Wide_Maps instead of Strings.Maps. 28/2

For each of the packages Strings.Fixed, Strings.Bounded, Strings.Unbounded, and Strings.- 29/2
Maps.Constants, and for functions Strings.Hash, Strings.Fixed.Hash, Strings.Bounded.Hash, and Strings.-
Unbounded.Hash, the corresponding wide wide string package or function has the same contents except
that

- Wide_Wide_Space replaces Space 30/2
- Wide_Wide_Character replaces Character 31/2
- Wide_Wide_String replaces String 32/2
- Wide_Wide_Character_Set replaces Character_Set 33/2
- Wide_Wide_Character_Mapping replaces Character_Mapping 34/2
- Wide_Wide_Character_Mapping_Function replaces Character_Mapping_Function 35/2
- Wide_Wide_Maps replaces Maps 36/2
- Bounded_Wide_Wide_String replaces Bounded_String 37/2
- Null_Bounded_Wide_Wide_String replaces Null_Bounded_String 38/2
- To_Bounded_Wide_Wide_String replaces To_Bounded_String 39/2
- To_Wide_Wide_String replaces To_String 40/2
- Set_Bounded_Wide_Wide_String replaces Set_Bounded_String 41/2
- Unbounded_Wide_Wide_String replaces Unbounded_String 42/2
- Null_Unbounded_Wide_Wide_String replaces Null_Unbounded_String 43/2
- Wide_Wide_String_Access replaces String_Access 44/2
- To_Unbounded_Wide_Wide_String replaces To_Unbounded_String 45/2
- Set_Unbounded_Wide_Wide_String replaces Set_Unbounded_String 46/2

The following additional declarations are present in Strings.Wide_Wide_Maps.Wide_Wide_Constants: 47/2

```

Character_Set : constant Wide_Wide_Maps.Wide_Wide_Character_Set;        48/2
-- Contains each Wide_Wide_Character value WWC such that
-- Characters.Conversions.Is_Character(WWC) is True
Wide_Character_Set : constant Wide_Wide_Maps.Wide_Wide_Character_Set;
-- Contains each Wide_Wide_Character value WWC such that
-- Characters.Conversions.Is_Wide_Character(WWC) is True

```

Each Wide_Wide_Character_Set constant in the package Strings.Wide_Wide_Maps.Wide_Wide_ 49/2
Constants contains no values outside the Character portion of Wide_Wide_Character. Similarly, each
Wide_Wide_Character_Mapping constant in this package is the identity mapping when applied to any
element outside the Character portion of Wide_Wide_Character.

50/2 **Pragma Pure** is replaced by **pragma Preelaborate** in `Strings.Wide_Wide_Maps.Wide_Wide_Constants`.

NOTES

51/2 13 If a null `Wide_Wide_Character_Mapping_Function` is passed to any of the `Wide_Wide_String` handling subprograms, `Constraint_Error` is propagated.

A.4.9 String Hashing

Static Semantics

1/2 The library function `Strings.Hash` has the following declaration:

```
2/2  with Ada.Containers;
    function Ada.Strings.Hash (Key : String) return Containers.Hash_Type;
    pragma Pure (Hash);
```

3/2 Returns an implementation-defined value which is a function of the value of `Key`. If *A* and *B* are strings such that *A* equals *B*, `Hash(A)` equals `Hash(B)`.

4/2 The library function `Strings.Fixed.Hash` has the following declaration:

```
5/2  with Ada.Containers, Ada.Strings.Hash;
    function Ada.Strings.Fixed.Hash (Key : String) return Containers.Hash_Type
        renames Ada.Strings.Hash;
    pragma Pure (Hash);
```

6/2 The generic library function `Strings.Bounded.Hash` has the following declaration:

```
7/2  with Ada.Containers;
    generic
        with package Bounded is
            new Ada.Strings.Bounded.Generic_Bounded_Length (<>);
        function Ada.Strings.Bounded.Hash (Key : Bounded.Bounded_String)
            return Containers.Hash_Type;
        pragma Preelaborate (Hash);
```

8/3 Equivalent to `Strings.Hash (Bounded.To_String (Key))`;

9/2 The library function `Strings.Unbounded.Hash` has the following declaration:

```
10/2 with Ada.Containers;
    function Ada.Strings.Unbounded.Hash (Key : Unbounded_String)
        return Containers.Hash_Type;
    pragma Preelaborate (Hash);
```

11/3 Equivalent to `Strings.Hash (To_String (Key))`;

12/3 The library function `Strings.Hash_Case_Insensitive` has the following declaration:

```
13/3 with Ada.Containers;
    function Ada.Strings.Hash_Case_Insensitive (Key : String)
        return Containers.Hash_Type;
    pragma Pure (Hash_Case_Insensitive);
```

14/3 Returns an implementation-defined value which is a function of the value of `Key`, converted to lower case. If *A* and *B* are strings such that `Strings.Equal_Case_Insensitive (A, B)` (see A.4.10) is `True`, then `Hash_Case_Insensitive(A)` equals `Hash_Case_Insensitive(B)`.

14.1/3 The library function `Strings.Fixed.Hash_Case_Insensitive` has the following declaration:

```
14.2/3 with Ada.Containers, Ada.Strings.Hash_Case_Insensitive;
    function Ada.Strings.Fixed.Hash_Case_Insensitive (Key : String)
        return Containers.Hash_Type renames Ada.Strings.Hash_Case_Insensitive;
    pragma Pure (Hash_Case_Insensitive);
```

The generic library function `Strings.Bounded.Hash_Case_Insensitive` has the following declaration: 14.3/2

```
with Ada.Containers;
generic
  with package Bounded is
    new Ada.Strings.Bounded.Generic_Bounded_Length (<>);
  function Ada.Strings.Bounded.Hash_Case_Insensitive
    (Key : Bounded.Bounded_String) return Containers.Hash_Type;
  pragma Preelaborate(Hash_Case_Insensitive);
```

Equivalent to `Strings.Hash_Case_Insensitive (Bounded.To_String (Key))`; 14.5/3

The library function `Strings.Unbounded.Hash_Case_Insensitive` has the following declaration: 14.6/3

```
with Ada.Containers;
function Ada.Strings.Unbounded.Hash_Case_Insensitive
  (Key : Unbounded_String) return Containers.Hash_Type;
pragma Preelaborate(Hash_Case_Insensitive);
```

Equivalent to `Strings.Hash_Case_Insensitive (To_String (Key))`; 14.8/3

Implementation Advice

The Hash functions should be good hash functions, returning a wide spread of values for different string values. It should be unlikely for similar strings to return the same value. 15/2

A.4.10 String Comparison

Static Semantics

The library function `Strings.Equal_Case_Insensitive` has the following declaration: 1/3

```
function Ada.Strings.Equal_Case_Insensitive (Left, Right : String)
  return Boolean;
pragma Pure(Equal_Case_Insensitive);
```

Compares strings Left and Right, converted to lower case, for equality. 3/3

The library function `Strings.Fixed.Equal_Case_Insensitive` has the following declaration: 4/3

```
with Ada.Containers, Ada.Strings.Equal_Case_Insensitive;
function Ada.Strings.Fixed.Equal_Case_Insensitive
  (Left, Right : String) return Boolean
  renames Ada.Strings.Equal_Case_Insensitive;
pragma Pure(Equal_Case_Insensitive);
```

The generic library function `Strings.Bounded.Equal_Case_Insensitive` has the following declaration: 6/3

```
with Ada.Containers;
generic
  with package Bounded is
    new Ada.Strings.Bounded.Generic_Bounded_Length (<>);
  function Ada.Strings.Bounded.Equal_Case_Insensitive
    (Left, Right : Bounded.Bounded_String) return Boolean;
  pragma Preelaborate(Equal_Case_Insensitive);
```

Equivalent to `Strings.Equal_Case_Insensitive (Bounded.To_String (Key))`; 8/3

The library function `Strings.Unbounded.Equal_Case_Insensitive` has the following declaration: 9/3

```
with Ada.Containers;
function Ada.Strings.Unbounded.Equal_Case_Insensitive
  (Left, Right : Unbounded_String) return Boolean;
pragma Preelaborate(Equal_Case_Insensitive);
```

Equivalent to `Strings.Equal_Case_Insensitive (To_String (Key))`; 11/3

12/3 The library function `Strings.Less_Case_Insensitive` has the following declaration:

```
13/3   function Ada.Strings.Less_Case_Insensitive (Left, Right : String)
      return Boolean;
14/3   pragma Pure (Less_Case_Insensitive);
```

14/3 Performs a lexicographic comparison of strings `Left` and `Right`, converted to lower case.

15/3 The library function `Strings.Fixed.Less_Case_Insensitive` has the following declaration:

```
16/3   with Ada.Containers, Ada.Strings.Less_Case_Insensitive;
      function Ada.Strings.Fixed.Less_Case_Insensitive
        (Left, Right : String) return Boolean
      renames Ada.Strings.Less_Case_Insensitive;
      pragma Pure (Less_Case_Insensitive);
```

17/3 The generic library function `Strings.Bounded.Less_Case_Insensitive` has the following declaration:

```
18/3   with Ada.Containers;
      generic
        with package Bounded is
          new Ada.Strings.Bounded.Generic_Bounded_Length (<>);
      function Ada.Strings.Bounded.Less_Case_Insensitive
        (Left, Right : Bounded.Bounded_String) return Boolean;
      pragma Preelaborate (Less_Case_Insensitive);
```

19/3 Equivalent to `Strings.Less_Case_Insensitive (Bounded.To_String (Key))`;

20/3 The library function `Strings.Unbounded.Less_Case_Insensitive` has the following declaration:

```
21/3   with Ada.Containers;
      function Ada.Strings.Unbounded.Less_Case_Insensitive
        (Left, Right : Unbounded_String) return Boolean;
      pragma Preelaborate (Less_Case_Insensitive);
```

22/3 Equivalent to `Strings.Less_Case_Insensitive (To_String (Key))`;

A.4.11 String Encoding

1/3 Facilities for encoding, decoding, and converting strings in various character encoding schemes are provided by packages `Strings.UTF_Encoding`, `Strings.UTF_Encoding.Conversions`, `Strings.UTF_Encoding.Wide_Encoding`, and `Strings.UTF_Encoding.Wide_Wide_Encoding`.

Static Semantics

2/3 The encoding library packages have the following declaration:

```
3/3   package Ada.Strings.UTF_Encoding is
      pragma Pure (UTF_Encoding);
4/3   -- Declarations common to the string encoding packages
      type Encoding_Scheme is (UTF_8, UTF_16BE, UTF_16LE);
5/3   subtype UTF_String is String;
6/3   subtype UTF_8_String is String;
7/3   subtype UTF_16_Wide_String is Wide_String;
8/3   Encoding_Error : exception;
9/3   BOM_8 : constant String :=
          Character'Val (16#EF#) &
          Character'Val (16#BB#) &
          Character'Val (16#BF#);
10/3   BOM_16BE : constant String :=
          Character'Val (16#FE#) &
          Character'Val (16#FF#);
```

```

BOM_16LE : constant String :=
    Character'Val(16#FF#) &
    Character'Val(16#FE#);
11/3

BOM_16 : constant Wide String :=
    (1 => Wide_Character'Val(16#FEFF#));
12/3

function Encoding (Item : UTF_String;
    Default : Encoding_Scheme := UTF_8) return
13/3
Encoding_Scheme;
end Ada.Strings.UTF_Encoding;
14/3

package Ada.Strings.UTF_Encoding.Conversions is
15/3
    pragma Pure (Conversions);
    -- Conversions between various encoding schemes
    function Convert (Item : UTF_String;
        Input_Scheme : Encoding_Scheme;
        Output_Scheme : Encoding_Scheme;
        Output_BOM : Boolean := False) return UTF_String;
16/3

    function Convert (Item : UTF_String;
        Input_Scheme : Encoding_Scheme;
        Output_BOM : Boolean := False) return
17/3
UTF_16_Wide_String;
    function Convert (Item : UTF_8_String;
        Output_BOM : Boolean := False) return
18/3
UTF_16_Wide_String;
    function Convert (Item : UTF_16_Wide_String;
        Output_Scheme : Encoding_Scheme;
        Output_BOM : Boolean := False) return UTF_String;
19/3

    function Convert (Item : UTF_16_Wide_String;
        Output_BOM : Boolean := False) return UTF_8_String;
20/3
end Ada.Strings.UTF_Encoding.Conversions;
21/3

package Ada.Strings.UTF_Encoding.Wide_Encoding is
22/3
    pragma Pure (Wide_Encoding);
    -- Encoding / decoding between Wide_String and various encoding schemes
    function Encode (Item : Wide_String;
        Output_Scheme : Encoding_Scheme;
        Output_BOM : Boolean := False) return UTF_String;
23/3

    function Encode (Item : Wide_String;
        Output_BOM : Boolean := False) return UTF_8_String;
24/3

    function Encode (Item : Wide_String;
        Output_BOM : Boolean := False) return
25/3
UTF_16_Wide_String;
    function Decode (Item : UTF_String;
        Input_Scheme : Encoding_Scheme) return Wide_String;
26/3

    function Decode (Item : UTF_8_String) return Wide_String;
27/3

    function Decode (Item : UTF_16_Wide_String) return Wide_String;
28/3
end Ada.Strings.UTF_Encoding.Wide_Encoding;
29/3

package Ada.Strings.UTF_Encoding.Wide_Wide_Encoding is
30/3
    pragma Pure (Wide_Wide_Encoding);
    -- Encoding / decoding between Wide_Wide_String and various encoding schemes
    function Encode (Item : Wide_Wide_String;
        Output_Scheme : Encoding_Scheme;
        Output_BOM : Boolean := False) return UTF_String;
31/3

    function Encode (Item : Wide_Wide_String;
        Output_BOM : Boolean := False) return UTF_8_String;
32/3

    function Encode (Item : Wide_Wide_String;
        Output_BOM : Boolean := False) return
33/3
UTF_16_Wide_String;

```

```

34/3      function Decode (Item           : UTF_String;
                        Input_Scheme : Encoding_Scheme) return Wide_Wide_String;
35/3      function Decode (Item : UTF_8_String) return Wide_Wide_String;
36/3      function Decode (Item : UTF_16_Wide_String) return Wide_Wide_String;
37/3      end Ada.Strings.UTF_Encoding.Wide_Wide_Encoding;
38/3      The type Encoding_Scheme defines encoding schemes. UTF_8 corresponds to the UTF-8 encoding
scheme defined by Annex D of ISO/IEC 106046. UTF_16BE corresponds to the UTF-16 encoding scheme
defined by Annex C of ISO/IEC 106046 stored in 8 bits, big endian; and UTF_16LE corresponds to the
UTF-16 encoding scheme on 8 bits, little endian.

39/3      The subtype UTF_String is used to represent a String of 8-bit values containing a sequence of values
encoded in one of three ways (UTF-8, UTF-16BE, or UTF-16LE). The subtype UTF_8_String is used to
represent a String of 8-bit values containing a sequence of values encoded in UTF-8. The subtype
UTF_16_Wide_String is is used to represent a Wide_String of 16-bit containing a sequence of values
encoded in UTF-16.

40/3      The BOM_8, BOM_16BE, BOM_16LE and BOM_16 constants correspond to values used at the start of a
string to indicate the encoding.

41/3      For all Convert and Decode functions, an initial BOM in the input that matches the expected encoding
scheme is ignored, and a different initial BOM causes Encoding_Error to be propagated.

42/3      For all Convert and Encode functions, a BOM is included at the start of the output string if the
Output_BOM parameter is set to True.

43/3      The exception Encoding_Error is also propagated in the following situations:
44/3      • By a Decode function when an UTF encoded string contains an invalid encoding sequence.
45/3      • By a Decode function when the expected encoding is UTF-16BE or UTF-16LE and the input
string has an odd length.
46/3      • By a Decode function yielding a Wide_String when the decoding of a sequence results in a
code-point whose value exceeds 16#FFFF#
47/3      • By an Encode function taking a Wide_String as input when an invalid character appears in the
input. In particular the characters whose position is in the range 16#D800# .. 16#DFFF# are
invalid because they conflict with UTF-16 surrogate encodings, and the characters whose
position is 16#FFFE# or 16#FFFF# are also invalid because they conflict with BOM codes.

48/3      Each of the Convert and Encode functions returns a String (respectively Wide_String) value whose
characters have position values that correspond to the encoding of the Item parameter according to the
encoding scheme required by the function or specified by its Output_Scheme parameter. For UTF_8, no
overlong encoding is returned. The lower bound of the returned string is 1.

49/3      Each of the Decode functions takes a String (respectively Wide_String) Item parameter which is assumed
to contain characters whose position values correspond to a valid encoding sequence according to the
encoding scheme required by the function or specified by its Input_Scheme parameter, and returns the
corresponding Wide_String (respectively Wide_Wide_String) value.

50/3      function Encoding (Item           : UTF_String;
                        Default : Encoding_Scheme := UTF_8) return
Encoding_Scheme;
51/3      Inspects a UTF_String value to determine whether it starts with a BOM for UTF-8, UTF-16BE,
or UTF_16LE. If so, returns the scheme corresponding to the BOM; returns the value of Default
otherwise.

```


function Convert (Item : UTF_String;	52/3
Input_Scheme : EncOding_Scheme;	
Output_Scheme : EncOding_Scheme;	
Output_BOM : Boolean := False) return UTF_String;	
Converts from input encoded in UTF-8, UTF-16LE, or UTF-16BE as specified by Input_Scheme, and generates an output encoded in one of these three schemes as specified by Output_Scheme.	53/3
function Convert (Item : UTF_String;	54/3
Input_Scheme : EncOding_Scheme;	
Output_BOM : Boolean := False) return	
UTF_16_Wide_String;	
Converts from input encoded in UTF-8, UTF-16LE, or UTF-16BE as specified by Input_Scheme, and generates an output encoded in UTF-16.	55/3
function Convert (Item : UTF_8_String;	56/3
Output_BOM : Boolean := False) return	
UTF_16_Wide_String;	
Converts from input encoded in UTF-8 and generates an output encoded in UTF-16.	57/3
function Convert (Item : UTF_16_Wide_String;	58/3
Output_Scheme : EncOding_Scheme;	
Output_BOM : Boolean := False) return UTF_String;	
Converts from input encoded in UTF-16 and generates an output encoded in UTF-8, UTF-16LE, or UTF-16BE as specified by Output_Scheme.	59/3
function Convert (Item : UTF_16_Wide_String;	60/3
Output_BOM : Boolean := False) return UTF_8_String;	
Converts from input encoded in UTF-16 and generates an output encoded in UTF-8.	61/3
function Encode (Item : Wide_String;	62/3
Output_Scheme : EncOding_Scheme;	
Output_BOM : Boolean := False) return UTF_String;	
Encodes from Wide_String input, and generates an output encoded in UTF-8, UTF-16LE or UTF-16BE encoding as specified by Output_Scheme.	63/3
function Encode (Item : Wide_String;	64/3
Output_BOM : Boolean := False) return UTF_8_String;	
Encodes from Wide_String input, and generates an output encoded in UTF-8 encoding.	65/3
function Encode (Item : Wide_String;	66/3
Output_BOM : Boolean := False) return UTF_16_Wide_String;	
Encodes from Wide_String input, and generates an output encoded in UTF_16 encoding.	67/3
function Decode (Item : UTF_String;	68/3
Input_Scheme : EncOding_Scheme) return Wide_String;	
Decodes from input encoded in UTF-8, UTF-16LE, or UTF-16BE as specified by Input_Scheme, and returns the corresponding Wide_String value.	69/3
function Decode (Item : UTF_8_String) return Wide_String;	70/3
Decodes from input encoded in UTF-8, and returns the corresponding Wide_String value.	71/3
function Decode (Item : UTF_16_Wide_String) return Wide_String;	72/3
Decodes from input encoded in UTF-16, and returns the corresponding Wide_String value.	73/3

```

74/3      function Encode (Item           : Wide_Wide_String;
                          Output_Scheme : Encoding_Scheme;
                          Output_BOM    : Boolean := False) return UTF_String;

75/3      Encodes from Wide_Wide_String input, and generates an output encoded in UTF-8, UTF-16LE
          or UTF-16BE encoding as specified by Output_Scheme.

76/3      function Encode (Item           : Wide_Wide_String;
                          Output_BOM    : Boolean := False) return UTF_8_String;

77/3      Encodes from Wide_Wide_String input, and generates an output encoded in UTF-8 encoding.

78/3      function Encode (Item           : Wide_Wide_String;
                          Output_BOM    : Boolean := False) return UTF_16_Wide_String;

79/3      Encodes from Wide_Wide_String input, and generates an output encoded in UTF_16 encoding.

80/3      function Decode (Item           : UTF_String;
                          Input_Scheme  : Encoding_Scheme) return Wide_Wide_String;

81/3      Decodes from input encoded in UTF-8, UTF-16LE, or UTF-16BE as specified by
          Input_Scheme, and returns the corresponding Wide_Wide_String value.

82/3      function Decode (Item : UTF_8_String) return Wide_Wide_String;

83/3      Decodes from input encoded in UTF-8, and returns the corresponding Wide_Wide_String value.

84/3      function Decode (Item : UTF_16_Wide_String) return Wide_Wide_String;

85/3      Decodes from input encoded in UTF-16, and returns the corresponding Wide_Wide_String
          value.

```

Implementation Advice

86/3 If an implementation supports other encoding schemes, another similar child of Ada.Strings should be defined.

NOTES

87/3 14 A BOM (Byte-Order Mark, code position 16#FEFF#) can be included in a file or other entity to indicate the encoding; it is skipped when decoding. Typically, only the first line of a file or other entity contains a BOM. When decoding, the Encoding function can be called on the first line to determine the encoding; this encoding will then be used in subsequent calls to Decode to convert all of the lines to an internal format.

A.5 The Numerics Packages

1 The library package Numerics is the parent of several child units that provide facilities for mathematical computation. One child, the generic package Generic_Elementary_Functions, is defined in A.5.1, together with nongeneric equivalents; two others, the package Float_Random and the generic package Discrete_Random, are defined in A.5.2. Additional (optional) children are defined in Annex G, “Numerics”.

Static Semantics

2/1 *This paragraph was deleted.*

```

package Ada.Numerics is
  pragma Pure(Numerics);
  Argument_Error : exception;
  Pi : constant :=
    3.14159_26535_89793_23846_26433_83279_50288_41971_69399_37511;
   $\pi$  : constant := Pi;
  e : constant :=
    2.71828_18284_59045_23536_02874_71352_66249_77572_47093_69996;
end Ada.Numerics;

```

3/2

The Argument_Error exception is raised by a subprogram in a child unit of Numerics to signal that one or more of the actual subprogram parameters are outside the domain of the corresponding mathematical function.

4

Implementation Permissions

The implementation may specify the values of Pi and e to a larger number of significant digits.

5

A.5.1 Elementary Functions

Implementation-defined approximations to the mathematical functions known as the “elementary functions” are provided by the subprograms in Numerics.Generic_Elementary_Functions. Nongeneric equivalents of this generic package for each of the predefined floating point types are also provided as children of Numerics.

1

Static Semantics

The generic library package Numerics.Generic_Elementary_Functions has the following declaration:

2

```

generic
  type Float_Type is digits <>;

```

3

```

package Ada.Numerics.Generic_Elementary_Functions is
  pragma Pure(Generic_Elementary_Functions);

  function Sqrt      (X           : Float_Type'Base) return Float_Type'Base;
  function Log       (X           : Float_Type'Base) return Float_Type'Base;
  function Log       (X, Base    : Float_Type'Base) return Float_Type'Base;
  function Exp       (X           : Float_Type'Base) return Float_Type'Base;
  function "***"     (Left, Right : Float_Type'Base) return Float_Type'Base;

  function Sin       (X           : Float_Type'Base) return Float_Type'Base;
  function Sin       (X, Cycle   : Float_Type'Base) return Float_Type'Base;
  function Cos       (X           : Float_Type'Base) return Float_Type'Base;
  function Cos       (X, Cycle   : Float_Type'Base) return Float_Type'Base;
  function Tan       (X           : Float_Type'Base) return Float_Type'Base;
  function Tan       (X, Cycle   : Float_Type'Base) return Float_Type'Base;
  function Cot       (X           : Float_Type'Base) return Float_Type'Base;
  function Cot       (X, Cycle   : Float_Type'Base) return Float_Type'Base;

```

4

5

```

6      function Arcsin (X           : Float_Type'Base) return Float_Type'Base;
      function Arcsin (X, Cycle    : Float_Type'Base) return Float_Type'Base;
      function Arccos (X           : Float_Type'Base) return Float_Type'Base;
      function Arccos (X, Cycle    : Float_Type'Base) return Float_Type'Base;
      function Arctan (Y           : Float_Type'Base;
                      X           : Float_Type'Base := 1.0)
                                return Float_Type'Base;

      function Arctan (Y           : Float_Type'Base;
                      X           : Float_Type'Base := 1.0;
                      Cycle       : Float_Type'Base) return Float_Type'Base;
      function Arccot (X           : Float_Type'Base;
                      Y           : Float_Type'Base := 1.0)
                                return Float_Type'Base;
      function Arccot (X           : Float_Type'Base;
                      Y           : Float_Type'Base := 1.0;
                      Cycle       : Float_Type'Base) return Float_Type'Base;

7      function Sinh (X           : Float_Type'Base) return Float_Type'Base;
      function Cosh (X           : Float_Type'Base) return Float_Type'Base;
      function Tanh (X           : Float_Type'Base) return Float_Type'Base;
      function Coth (X           : Float_Type'Base) return Float_Type'Base;
      function Arcsinh (X         : Float_Type'Base) return Float_Type'Base;
      function Arccosh (X         : Float_Type'Base) return Float_Type'Base;
      function Arctanh (X         : Float_Type'Base) return Float_Type'Base;
      function Arccoth (X         : Float_Type'Base) return Float_Type'Base;

8      end Ada.Numerics.Generic_Elementary_Functions;

```

9/1 The library package `Numerics.Elementary_Functions` is declared pure and defines the same subprograms as `Numerics.Generic_Elementary_Functions`, except that the predefined type `Float` is systematically substituted for `Float_Type'Base` throughout. Nongeneric equivalents of `Numerics.Generic_Elementary_Functions` for each of the other predefined floating point types are defined similarly, with the names `Numerics.Short_Elementary_Functions`, `Numerics.Long_Elementary_Functions`, etc.

10 The functions have their usual mathematical meanings. When the `Base` parameter is specified, the `Log` function computes the logarithm to the given base; otherwise, it computes the natural logarithm. When the `Cycle` parameter is specified, the parameter `X` of the forward trigonometric functions (`Sin`, `Cos`, `Tan`, and `Cot`) and the results of the inverse trigonometric functions (`Arcsin`, `Arccos`, `Arctan`, and `Arccot`) are measured in units such that a full cycle of revolution has the given value; otherwise, they are measured in radians.

11 The computed results of the mathematically multivalued functions are rendered single-valued by the following conventions, which are meant to imply the principal branch:

- 12 • The results of the `Sqrt` and `Arccosh` functions and that of the exponentiation operator are nonnegative.
- 13 • The result of the `Arcsin` function is in the quadrant containing the point $(1.0, x)$, where x is the value of the parameter `X`. This quadrant is I or IV; thus, the range of the `Arcsin` function is approximately $-\pi/2.0$ to $\pi/2.0$ ($-Cycle/4.0$ to $Cycle/4.0$, if the parameter `Cycle` is specified).
- 14 • The result of the `Arccos` function is in the quadrant containing the point $(x, 1.0)$, where x is the value of the parameter `X`. This quadrant is I or II; thus, the `Arccos` function ranges from 0.0 to approximately π ($Cycle/2.0$, if the parameter `Cycle` is specified).
- 15 • The results of the `Arctan` and `Arccot` functions are in the quadrant containing the point (x, y) , where x and y are the values of the parameters `X` and `Y`, respectively. This may be any quadrant (I through IV) when the parameter `X` (resp., `Y`) of `Arctan` (resp., `Arccot`) is specified, but it is restricted to quadrants I and IV (resp., I and II) when that parameter is omitted. Thus, the range when that parameter is specified is approximately $-\pi$ to π ($-Cycle/2.0$ to $Cycle/2.0$, if the parameter `Cycle` is specified); when omitted, the range of `Arctan` (resp., `Arccot`) is that of `Arcsin`

(resp., Arccos), as given above. When the point (x, y) lies on the negative x -axis, the result approximates

- π (resp., $-\pi$) when the sign of the parameter Y is positive (resp., negative), if `Float_Type'Signed_Zeros` is `True`; 16
- π , if `Float_Type'Signed_Zeros` is `False`. 17

(In the case of the inverse trigonometric functions, in which a result lying on or near one of the axes may not be exactly representable, the approximation inherent in computing the result may place it in an adjacent quadrant, close to but on the wrong side of the axis.) 18

Dynamic Semantics

The exception `Numerics.Argument_Error` is raised, signaling a parameter value outside the domain of the corresponding mathematical function, in the following cases: 19

- by any forward or inverse trigonometric function with specified cycle, when the value of the parameter `Cycle` is zero or negative; 20
- by the `Log` function with specified base, when the value of the parameter `Base` is zero, one, or negative; 21
- by the `Sqrt` and `Log` functions, when the value of the parameter `X` is negative; 22
- by the exponentiation operator, when the value of the left operand is negative or when both operands have the value zero; 23
- by the `Arcsin`, `Arccos`, and `Arctanh` functions, when the absolute value of the parameter `X` exceeds one; 24
- by the `Arctan` and `Arccot` functions, when the parameters `X` and `Y` both have the value zero; 25
- by the `Arccosh` function, when the value of the parameter `X` is less than one; and 26
- by the `Arccoth` function, when the absolute value of the parameter `X` is less than one. 27

The exception `Constraint_Error` is raised, signaling a pole of the mathematical function (analogous to dividing by zero), in the following cases, provided that `Float_Type'Machine_Overflows` is `True`: 28

- by the `Log`, `Cot`, and `Coth` functions, when the value of the parameter `X` is zero; 29
- by the exponentiation operator, when the value of the left operand is zero and the value of the exponent is negative; 30
- by the `Tan` function with specified cycle, when the value of the parameter `X` is an odd multiple of the quarter cycle; 31
- by the `Cot` function with specified cycle, when the value of the parameter `X` is zero or a multiple of the half cycle; and 32
- by the `Arctanh` and `Arccoth` functions, when the absolute value of the parameter `X` is one. 33

`Constraint_Error` can also be raised when a finite result overflows (see G.2.4); this may occur for parameter values sufficiently *near* poles, and, in the case of some of the functions, for parameter values with sufficiently large magnitudes. When `Float_Type'Machine_Overflows` is `False`, the result at poles is unspecified. 34

When one parameter of a function with multiple parameters represents a pole and another is outside the function's domain, the latter takes precedence (i.e., `Numerics.Argument_Error` is raised). 35

Implementation Requirements

- 36 In the implementation of `Numerics.Generic_Elementary_Functions`, the range of intermediate values allowed during the calculation of a final result shall not be affected by any range constraint of the subtype `Float_Type`.
- 37 In the following cases, evaluation of an elementary function shall yield the *prescribed result*, provided that the preceding rules do not call for an exception to be raised:
- 38 • When the parameter `X` has the value zero, the `Sqrt`, `Sin`, `Arcsin`, `Tan`, `Sinh`, `Arcsinh`, `Tanh`, and `Arctanh` functions yield a result of zero, and the `Exp`, `Cos`, and `Cosh` functions yield a result of one.
 - 39 • When the parameter `X` has the value one, the `Sqrt` function yields a result of one, and the `Log`, `Arccos`, and `Arccosh` functions yield a result of zero.
 - 40 • When the parameter `Y` has the value zero and the parameter `X` has a positive value, the `Arctan` and `Arccot` functions yield a result of zero.
 - 41 • The results of the `Sin`, `Cos`, `Tan`, and `Cot` functions with specified cycle are exact when the mathematical result is zero; those of the first two are also exact when the mathematical result is ± 1.0 .
 - 42 • Exponentiation by a zero exponent yields the value one. Exponentiation by a unit exponent yields the value of the left operand. Exponentiation of the value one yields the value one. Exponentiation of the value zero yields the value zero.
- 43 Other accuracy requirements for the elementary functions, which apply only in implementations conforming to the Numerics Annex, and then only in the “strict” mode defined there (see G.2), are given in G.2.4.
- 44 When `Float_Type.Signed_Zeros` is `True`, the sign of a zero result shall be as follows:
- 45 • A prescribed zero result delivered *at the origin* by one of the odd functions (`Sin`, `Arcsin`, `Sinh`, `Arcsinh`, `Tan`, `Arctan` or `Arccot` as a function of `Y` when `X` is fixed and positive, `Tanh`, and `Arctanh`) has the sign of the parameter `X` (`Y`, in the case of `Arctan` or `Arccot`).
 - 46 • A prescribed zero result delivered by one of the odd functions *away from the origin*, or by some other elementary function, has an implementation-defined sign.
 - 47 • A zero result that is not a prescribed result (i.e., one that results from rounding or underflow) has the correct mathematical sign.

Implementation Permissions

- 48 The nongeneric equivalent packages may, but need not, be actual instantiations of the generic package for the appropriate predefined type.

A.5.2 Random Number Generation

- 1 Facilities for the generation of pseudo-random floating point numbers are provided in the package `Numerics.Float_Random`; the generic package `Numerics.Discrete_Random` provides similar facilities for the generation of pseudo-random integers and pseudo-random values of enumeration types. For brevity, pseudo-random values of any of these types are called *random numbers*.
- 2 Some of the facilities provided are basic to all applications of random numbers. These include a limited private type each of whose objects serves as the generator of a (possibly distinct) sequence of random numbers; a function to obtain the “next” random number from a given sequence of random numbers (that

is, from its generator); and subprograms to initialize or reinitialize a given generator to a time-dependent state or a state denoted by a single integer.

Other facilities are provided specifically for advanced applications. These include subprograms to save and restore the state of a given generator; a private type whose objects can be used to hold the saved state of a generator; and subprograms to obtain a string representation of a given generator state, or, given such a string representation, the corresponding state.

Static Semantics

The library package Numerics.Float_Random has the following declaration:

```

package Ada.Numerics.Float_Random is
  -- Basic facilities
  type Generator is limited private;
  subtype Uniformly_Distributed is Float range 0.0 .. 1.0;
  function Random (Gen : Generator) return Uniformly_Distributed;
  procedure Reset (Gen      : in Generator;
                  Initiator : in Integer);
  procedure Reset (Gen      : in Generator);
  -- Advanced facilities
  type State is private;
  procedure Save  (Gen      : in Generator;
                  To_State : out State);
  procedure Reset (Gen      : in Generator;
                  From_State : in State);
  Max_Image_Width : constant := implementation-defined integer value;
  function Image (Of_State : State) return String;
  function Value (Coded_State : String) return State;
private
  ... -- not specified by the language
end Ada.Numerics.Float_Random;
```

The type Generator needs finalization (see 7.6).

The generic library package Numerics.Discrete_Random has the following declaration:

```

generic
  type Result_Subtype is (<>);
package Ada.Numerics.Discrete_Random is
  -- Basic facilities
  type Generator is limited private;
  function Random (Gen : Generator) return Result_Subtype;
  procedure Reset (Gen      : in Generator;
                  Initiator : in Integer);
  procedure Reset (Gen      : in Generator);
  -- Advanced facilities
  type State is private;
  procedure Save  (Gen      : in Generator;
                  To_State : out State);
  procedure Reset (Gen      : in Generator;
                  From_State : in State);
  Max_Image_Width : constant := implementation-defined integer value;
  function Image (Of_State : State) return String;
  function Value (Coded_State : String) return State;
```

```

27     private
        ... -- not specified by the language
    end Ada.Numerics.Discrete_Random;

```

27.1/2 The type Generator needs finalization (see 7.6) in every instantiation of Numerics.Discrete_Random.

28 An object of the limited private type Generator is associated with a sequence of random numbers. Each generator has a hidden (internal) state, which the operations on generators use to determine the position in the associated sequence. All generators are implicitly initialized to an unspecified state that does not vary from one program execution to another; they may also be explicitly initialized, or reinitialized, to a time-dependent state, to a previously saved state, or to a state uniquely denoted by an integer value.

29 An object of the private type State can be used to hold the internal state of a generator. Such objects are only needed if the application is designed to save and restore generator states or to examine or manufacture them.

30 The operations on generators affect the state and therefore the future values of the associated sequence. The semantics of the operations on generators and states are defined below.

```

31     function Random (Gen : Generator) return Uniformly_Distributed;
    function Random (Gen : Generator) return Result_Subtype;

```

32 Obtains the “next” random number from the given generator, relative to its current state, according to an implementation-defined algorithm. The result of the function in Numerics.Float_Random is delivered as a value of the subtype Uniformly_Distributed, which is a subtype of the predefined type Float having a range of 0.0 .. 1.0. The result of the function in an instantiation of Numerics.Discrete_Random is delivered as a value of the generic formal subtype Result_Subtype.

```

33     procedure Reset (Gen      : in Generator;
                     Initiator : in Integer);
    procedure Reset (Gen      : in Generator);

```

34 Sets the state of the specified generator to one that is an unspecified function of the value of the parameter Initiator (or to a time-dependent state, if only a generator parameter is specified). The latter form of the procedure is known as the *time-dependent Reset procedure*.

```

35     procedure Save  (Gen      : in Generator;
                     To_State : out State);
    procedure Reset  (Gen      : in Generator;
                     From_State : in State);

```

36 Save obtains the current state of a generator. Reset gives a generator the specified state. A generator that is reset to a state previously obtained by invoking Save is restored to the state it had when Save was invoked.

```

37     function Image (Of_State : State) return String;
    function Value  (Coded_State : String) return State;

```

38 Image provides a representation of a state coded (in an implementation-defined way) as a string whose length is bounded by the value of Max_Image_Width. Value is the inverse of Image: Value(Image(S)) = S for each state S that can be obtained from a generator by invoking Save.

Dynamic Semantics

39 Instantiation of Numerics.Discrete_Random with a subtype having a null range raises Constraint_Error.

40/1 *This paragraph was deleted.*

Bounded (Run-Time) Errors

It is a bounded error to invoke Value with a string that is not the image of any generator state. If the error is detected, Constraint_Error or Program_Error is raised. Otherwise, a call to Reset with the resulting state will produce a generator such that calls to Random with this generator will produce a sequence of values of the appropriate subtype, but which might not be random in character. That is, the sequence of values might not fulfill the implementation requirements of this subclass. 40.1/1

Implementation Requirements

A sufficiently long sequence of random numbers obtained by successive calls to Random is approximately uniformly distributed over the range of the result subtype. 41

The Random function in an instantiation of Numerics.Discrete_Random is guaranteed to yield each value in its result subtype in a finite number of calls, provided that the number of such values does not exceed 2¹⁵. 42

Other performance requirements for the random number generator, which apply only in implementations conforming to the Numerics Annex, and then only in the “strict” mode defined there (see G.2), are given in G.2.5. 43

Documentation Requirements

No one algorithm for random number generation is best for all applications. To enable the user to determine the suitability of the random number generators for the intended application, the implementation shall describe the algorithm used and shall give its period, if known exactly, or a lower bound on the period, if the exact period is unknown. Periods that are so long that the periodicity is unobservable in practice can be described in such terms, without giving a numerical bound. 44

The implementation also shall document the minimum time interval between calls to the time-dependent Reset procedure that are guaranteed to initiate different sequences, and it shall document the nature of the strings that Value will accept without raising Constraint_Error. 45

Implementation Advice

Any storage associated with an object of type Generator should be reclaimed on exit from the scope of the object. 46

If the generator period is sufficiently long in relation to the number of distinct initiator values, then each possible value of Initiator passed to Reset should initiate a sequence of random numbers that does not, in a practical sense, overlap the sequence initiated by any other value. If this is not possible, then the mapping between initiator values and generator states should be a rapidly varying function of the initiator value. 47

NOTES

15 If two or more tasks are to share the same generator, then the tasks have to synchronize their access to the generator as for any shared variable (see 9.10). 48

16 Within a given implementation, a repeatable random number sequence can be obtained by relying on the implicit initialization of generators or by explicitly initializing a generator with a repeatable initiator value. Different sequences of random numbers can be obtained from a given generator in different program executions by explicitly initializing the generator to a time-dependent state. 49

17 A given implementation of the Random function in Numerics.Float_Random may or may not be capable of delivering the values 0.0 or 1.0. Portable applications should assume that these values, or values sufficiently close to them to behave indistinguishably from them, can occur. If a sequence of random integers from some fixed range is needed, the application should use the Random function in an appropriate instantiation of Numerics.Discrete_Random, rather than transforming the result of the Random function in Numerics.Float_Random. However, some applications with unusual requirements, such as for a sequence of random integers each drawn from a different range, will find it more convenient to transform the result of the floating point Random function. For $M \geq 1$, the expression 50

51 Integer(Float(M) * Random(G)) mod M
 52 transforms the result of Random(G) to an integer uniformly distributed over the range 0 .. M-1; it is valid even if Random delivers 0.0 or 1.0. Each value of the result range is possible, provided that M is not too large. Exponentially distributed (floating point) random numbers with mean and standard deviation 1.0 can be obtained by the transformation

53/2 -Log(Random(G) + Float'Model_Small)

54 where Log comes from Numerics.Elementary_Functions (see A.5.1); in this expression, the addition of Float'Model_Small avoids the exception that would be raised were Log to be given the value zero, without affecting the result (in most implementations) when Random returns a nonzero value.

Examples

55 *Example of a program that plays a simulated dice game:*

```
56   with Ada.Numerics.Discrete_Random;
    procedure Dice_Game is
      subtype Die is Integer range 1 .. 6;
      subtype Dice is Integer range 2*Die'First .. 2*Die'Last;
      package Random_Die is new Ada.Numerics.Discrete_Random (Die);
      use Random_Die;
      G : Generator;
      D : Dice;
    begin
      Reset (G); -- Start the generator in a unique state in each run
      loop
        -- Roll a pair of dice; sum and process the results
        D := Random(G) + Random(G);
        ...
      end loop;
    end Dice_Game;
```

57 *Example of a program that simulates coin tosses:*

```
58   with Ada.Numerics.Discrete_Random;
    procedure Flip_A_Coin is
      type Coin is (Heads, Tails);
      package Random_Coin is new Ada.Numerics.Discrete_Random (Coin);
      use Random_Coin;
      G : Generator;
    begin
      Reset (G); -- Start the generator in a unique state in each run
      loop
        -- Toss a coin and process the result
        case Random(G) is
          when Heads =>
            ...
          when Tails =>
            ...
        end case;
        ...
      end loop;
    end Flip_A_Coin;
```

Example of a parallel simulation of a physical system, with a separate generator of event probabilities in each task: 59

```

with Ada.Numerics.Float_Random;
procedure Parallel_Simulation is
  use Ada.Numerics.Float_Random;
  task type Worker is
    entry Initialize_Generator (Initiator : in Integer);
    ...
  end Worker;
  W : array (1 .. 10) of Worker;
  task body Worker is
    G : Generator;
    Probability_Of_Event : Uniformly_Distributed;
  begin
    accept Initialize_Generator (Initiator : in Integer) do
      Reset (G, Initiator);
    end Initialize_Generator;
    loop
      ...
      Probability_Of_Event := Random(G);
      ...
    end loop;
  end Worker;
begin
  -- Initialize the generators in the Worker tasks to different states
  for I in W'Range loop
    W(I).Initialize_Generator (I);
  end loop;
  ... -- Wait for the Worker tasks to terminate
end Parallel_Simulation;

```

60

NOTES

18 *Notes on the last example:* Although each Worker task initializes its generator to a different state, those states will be the same in every execution of the program. The generator states can be initialized uniquely in each program execution by instantiating Ada.Numerics.Discrete_Random for the type Integer in the main procedure, resetting the generator obtained from that instance to a time-dependent state, and then using random integers obtained from that generator to initialize the generators in each Worker task. 61

A.5.3 Attributes of Floating Point Types

Static Semantics

The following *representation-oriented attributes* are defined for every subtype S of a floating point type T. 1

S'Machine_Radix 2

Yields the radix of the hardware representation of the type T. The value of this attribute is of the type *universal_integer*.

The values of other representation-oriented attributes of a floating point subtype, and of the “primitive function” attributes of a floating point subtype described later, are defined in terms of a particular representation of nonzero values called the *canonical form*. The canonical form (for the type T) is the form 3

$$\pm \text{mantissa} \cdot T\text{Machine_Radix}^{\text{exponent}}$$

where

- *mantissa* is a fraction in the number base T'Machine_Radix, the first digit of which is nonzero, 4
- *exponent* is an integer. 5

S'Machine_Mantissa 6

Yields the largest value of p such that every value expressible in the canonical form (for the type T), having a p-digit *mantissa* and an *exponent* between T'Machine_Emin and

T Machine_Emax, is a machine number (see 3.5.7) of the type T . This attribute yields a value of the type *universal_integer*.

- 7 S'Machine_Emin
Yields the smallest (most negative) value of *exponent* such that every value expressible in the canonical form (for the type T), having a *mantissa* of T Machine_Mantissa digits, is a machine number (see 3.5.7) of the type T . This attribute yields a value of the type *universal_integer*.
- 8 S'Machine_Emax
Yields the largest (most positive) value of *exponent* such that every value expressible in the canonical form (for the type T), having a *mantissa* of T Machine_Mantissa digits, is a machine number (see 3.5.7) of the type T . This attribute yields a value of the type *universal_integer*.
- 9 S'Denorm
Yields the value True if every value expressible in the form

$$\pm \text{mantissa} \cdot T\text{Machine_Radix}^{T\text{Machine_Emin}}$$
where *mantissa* is a nonzero T Machine_Mantissa-digit fraction in the number base T Machine_Radix, the first digit of which is zero, is a machine number (see 3.5.7) of the type T ; yields the value False otherwise. The value of this attribute is of the predefined type Boolean.
- 10 The values described by the formula in the definition of S'Denorm are called *denormalized numbers*. A nonzero machine number that is not a denormalized number is a *normalized number*. A normalized number x of a given type T is said to be *represented in canonical form* when it is expressed in the canonical form (for the type T) with a *mantissa* having T Machine_Mantissa digits; the resulting form is the *canonical-form representation* of x .
- 11 S'Machine_Rounds
Yields the value True if rounding is performed on inexact results of every predefined operation that yields a result of the type T ; yields the value False otherwise. The value of this attribute is of the predefined type Boolean.
- 12 S'Machine_Overflows
Yields the value True if overflow and divide-by-zero are detected and reported by raising Constraint_Error for every predefined operation that yields a result of the type T ; yields the value False otherwise. The value of this attribute is of the predefined type Boolean.
- 13 S'Signed_Zeros
Yields the value True if the hardware representation for the type T has the capability of representing both positively and negatively signed zeros, these being generated and used by the predefined operations of the type T as specified in IEC 559:1989; yields the value False otherwise. The value of this attribute is of the predefined type Boolean.
- 14 For every value x of a floating point type T , the *normalized exponent* of x is defined as follows:
- 15 • the normalized exponent of zero is (by convention) zero;
- 16 • for nonzero x , the normalized exponent of x is the unique integer k such that $T\text{Machine_Radix}^{k-1} \leq |x| < T\text{Machine_Radix}^k$.
- 17 The following *primitive function attributes* are defined for any subtype S of a floating point type T .
- 18 S'Exponent S'Exponent denotes a function with the following specification:
- 19 **function** S'Exponent ($X : T$)
 return *universal_integer*
- 20 The function yields the normalized exponent of X .

S'Fraction	S'Fraction denotes a function with the following specification:	21
	function S'Fraction (<i>X</i> : <i>T</i>)	22
	return <i>T</i>	
	The function yields the value $X \cdot TMachine_Radix^{-k}$, where k is the normalized exponent of X . A zero result, which can only occur when X is zero, has the sign of X .	23
S'Compose	S'Compose denotes a function with the following specification:	24
	function S'Compose (<i>Fraction</i> : <i>T</i> ; <i>Exponent</i> : <i>universal_integer</i>)	25
	return <i>T</i>	
	Let v be the value $Fraction \cdot TMachine_Radix^{Exponent-k}$, where k is the normalized exponent of <i>Fraction</i> . If v is a machine number of the type <i>T</i> , or if $ v \geq TModel_Small$, the function yields v ; otherwise, it yields either one of the machine numbers of the type <i>T</i> adjacent to v . Constraint_Error is optionally raised if v is outside the base range of <i>S</i> . A zero result has the sign of <i>Fraction</i> when S'Signed_Zeros is True.	26
S'Scaling	S'Scaling denotes a function with the following specification:	27
	function S'Scaling (<i>X</i> : <i>T</i> ; <i>Adjustment</i> : <i>universal_integer</i>)	28
	return <i>T</i>	
	Let v be the value $X \cdot TMachine_Radix^{Adjustment}$. If v is a machine number of the type <i>T</i> , or if $ v \geq TModel_Small$, the function yields v ; otherwise, it yields either one of the machine numbers of the type <i>T</i> adjacent to v . Constraint_Error is optionally raised if v is outside the base range of <i>S</i> . A zero result has the sign of X when S'Signed_Zeros is True.	29
S'Floor	S'Floor denotes a function with the following specification:	30
	function S'Floor (<i>X</i> : <i>T</i>)	31
	return <i>T</i>	
	The function yields the value $\lfloor X \rfloor$, i.e., the largest (most positive) integral value less than or equal to X . When X is zero, the result has the sign of X ; a zero result otherwise has a positive sign.	32
S'Ceiling	S'Ceiling denotes a function with the following specification:	33
	function S'Ceiling (<i>X</i> : <i>T</i>)	34
	return <i>T</i>	
	The function yields the value $\lceil X \rceil$, i.e., the smallest (most negative) integral value greater than or equal to X . When X is zero, the result has the sign of X ; a zero result otherwise has a negative sign when S'Signed_Zeros is True.	35
S'Rounding	S'Rounding denotes a function with the following specification:	36
	function S'Rounding (<i>X</i> : <i>T</i>)	37
	return <i>T</i>	
	The function yields the integral value nearest to X , rounding away from zero if X lies exactly halfway between two integers. A zero result has the sign of X when S'Signed_Zeros is True.	38
S'Unbiased_Rounding	S'Unbiased_Rounding denotes a function with the following specification:	39
	function S'Unbiased_Rounding (<i>X</i> : <i>T</i>)	40
	return <i>T</i>	
	The function yields the integral value nearest to X , rounding toward the even integer if X lies exactly halfway between two integers. A zero result has the sign of X when S'Signed_Zeros is True.	41

- 41.1/2 S'Machine_Rounding
S'Machine_Rounding denotes a function with the following specification:
- 41.2/2 **function** S'Machine_Rounding ($X : T$)
 return T
- 41.3/2 The function yields the integral value nearest to X . If X lies exactly halfway between two integers, one of those integers is returned, but which of them is returned is unspecified. A zero result has the sign of X when S'Signed_Zeros is True. This function provides access to the rounding behavior which is most efficient on the target processor.
- 42 S'Truncation S'Truncation denotes a function with the following specification:
- 43 **function** S'Truncation ($X : T$)
 return T
- 44 The function yields the value $\lceil X \rceil$ when X is negative, and $\lfloor X \rfloor$ otherwise. A zero result has the sign of X when S'Signed_Zeros is True.
- 45 S'Remainder S'Remainder denotes a function with the following specification:
- 46 **function** S'Remainder ($X, Y : T$)
 return T
- 47 For nonzero Y , let v be the value $X - n \cdot Y$, where n is the integer nearest to the exact value of X/Y ; if $|n - X/Y| = 1/2$, then n is chosen to be even. If v is a machine number of the type T , the function yields v ; otherwise, it yields zero. Constraint_Error is raised if Y is zero. A zero result has the sign of X when S'Signed_Zeros is True.
- 48 S'Adjacent S'Adjacent denotes a function with the following specification:
- 49 **function** S'Adjacent ($X, Towards : T$)
 return T
- 50 If $Towards = X$, the function yields X ; otherwise, it yields the machine number of the type T adjacent to X in the direction of $Towards$, if that machine number exists. If the result would be outside the base range of S , Constraint_Error is raised. When $T'Signed_Zeros$ is True, a zero result has the sign of X . When $Towards$ is zero, its sign has no bearing on the result.
- 51 S'Copy_Sign S'Copy_Sign denotes a function with the following specification:
- 52 **function** S'Copy_Sign ($Value, Sign : T$)
 return T
- 53 If the value of $Value$ is nonzero, the function yields a result whose magnitude is that of $Value$ and whose sign is that of $Sign$; otherwise, it yields the value zero. Constraint_Error is optionally raised if the result is outside the base range of S . A zero result has the sign of $Sign$ when S'Signed_Zeros is True.
- 54 S'Leading_Part S'Leading_Part denotes a function with the following specification:
- 55 **function** S'Leading_Part ($X : T$;
 $Radix_Digits : universal_integer$)
 return T
- 56 Let v be the value $T'Machine_Radix^{k-Radix_Digits}$, where k is the normalized exponent of X . The function yields the value
- 57 • $\lfloor X/v \rfloor \cdot v$, when X is nonnegative and $Radix_Digits$ is positive;
- 58 • $\lceil X/v \rceil \cdot v$, when X is negative and $Radix_Digits$ is positive.
- 59 Constraint_Error is raised when $Radix_Digits$ is zero or negative. A zero result, which can only occur when X is zero, has the sign of X .
- 60 S'Machine S'Machine denotes a function with the following specification:

```

function S'Machine ( $X : T$ )
  return  $T$ 

```

61

If X is a machine number of the type T , the function yields X ; otherwise, it yields the value obtained by rounding or truncating X to either one of the adjacent machine numbers of the type T . `Constraint_Error` is raised if rounding or truncating X to the precision of the machine numbers results in a value outside the base range of S . A zero result has the sign of X when `S'Signed_Zeros` is `True`.

62

The following *model-oriented attributes* are defined for any subtype S of a floating point type T .

63

`S'Model_Mantissa`

64

If the Numerics Annex is not supported, this attribute yields an implementation defined value that is greater than or equal to $\lceil d \cdot \log(10) / \log(T'Machine_Radix) \rceil + 1$, where d is the requested decimal precision of T , and less than or equal to the value of $T'Machine_Mantissa$. See G.2.2 for further requirements that apply to implementations supporting the Numerics Annex. The value of this attribute is of the type *universal_integer*.

`S'Model_Emin`

65

If the Numerics Annex is not supported, this attribute yields an implementation defined value that is greater than or equal to the value of $T'Machine_Emin$. See G.2.2 for further requirements that apply to implementations supporting the Numerics Annex. The value of this attribute is of the type *universal_integer*.

`S'Model_Epsilon`

66

Yields the value $T'Machine_Radix^{1 - T'Model_Mantissa}$. The value of this attribute is of the type *universal_real*.

`S'Model_Small`

67

Yields the value $T'Machine_Radix^{T'Model_Emin - 1}$. The value of this attribute is of the type *universal_real*.

`S'Model`

`S'Model` denotes a function with the following specification:

68

```

function S'Model ( $X : T$ )
  return  $T$ 

```

69

If the Numerics Annex is not supported, the meaning of this attribute is implementation defined; see G.2.2 for the definition that applies to implementations supporting the Numerics Annex.

70

`S'Safe_First`

71

Yields the lower bound of the safe range (see 3.5.7) of the type T . If the Numerics Annex is not supported, the value of this attribute is implementation defined; see G.2.2 for the definition that applies to implementations supporting the Numerics Annex. The value of this attribute is of the type *universal_real*.

`S'Safe_Last`

72

Yields the upper bound of the safe range (see 3.5.7) of the type T . If the Numerics Annex is not supported, the value of this attribute is implementation defined; see G.2.2 for the definition that applies to implementations supporting the Numerics Annex. The value of this attribute is of the type *universal_real*.

A.5.4 Attributes of Fixed Point Types

Static Semantics

The following *representation-oriented attributes* are defined for every subtype S of a fixed point type T .

1

- 2 S'Machine_Radix
Yields the radix of the hardware representation of the type *T*. The value of this attribute is of the type *universal_integer*.
- 3 S'Machine_Rounds
Yields the value True if rounding is performed on inexact results of every predefined operation that yields a result of the type *T*; yields the value False otherwise. The value of this attribute is of the predefined type Boolean.
- 4 S'Machine_Overflows
Yields the value True if overflow and divide-by-zero are detected and reported by raising Constraint_Error for every predefined operation that yields a result of the type *T*; yields the value False otherwise. The value of this attribute is of the predefined type Boolean.

A.6 Input-Output

- 1/2 Input-output is provided through language-defined packages, each of which is a child of the root package Ada. The generic packages Sequential_IO and Direct_IO define input-output operations applicable to files containing elements of a given type. The generic package Storage_IO supports reading from and writing to an in-memory buffer. Additional operations for text input-output are supplied in the packages Text_IO, Wide_Text_IO, and Wide_Wide_Text_IO. Heterogeneous input-output is provided through the child packages Streams.Stream_IO and Text_IO.Text_Streams (see also 13.13). The package IO_Exceptions defines the exceptions needed by the predefined input-output packages.

A.7 External Files and File Objects

Static Semantics

- 1 Values input from the external environment of the program, or output to the external environment, are considered to occupy *external files*. An external file can be anything external to the program that can produce a value to be read or receive a value to be written. An external file is identified by a string (the *name*). A second string (the *form*) gives further system-dependent characteristics that may be associated with the file, such as the physical organization or access rights. The conventions governing the interpretation of such strings shall be documented.
- 2 Input and output operations are expressed as operations on objects of some *file type*, rather than directly in terms of the external files. In the remainder of this section, the term *file* is always used to refer to a file object; the term *external file* is used otherwise.
- 3 Input-output for sequential files of values of a single element type is defined by means of the generic package Sequential_IO. In order to define sequential input-output for a given element type, an instantiation of this generic unit, with the given type as actual parameter, has to be declared. The resulting package contains the declaration of a file type (called File_Type) for files of such elements, as well as the operations applicable to these files, such as the Open, Read, and Write procedures.
- 4/2 Input-output for direct access files is likewise defined by a generic package called Direct_IO. Input-output in human-readable form is defined by the (nongeneric) packages Text_IO for Character and String data, Wide_Text_IO for Wide_Character and Wide_String data, and Wide_Wide_Text_IO for Wide_Wide_Character and Wide_Wide_String data. Input-output for files containing streams of elements representing values of possibly different types is defined by means of the (nongeneric) package Streams.Stream_IO.

Before input or output operations can be performed on a file, the file first has to be associated with an external file. While such an association is in effect, the file is said to be *open*, and otherwise the file is said to be *closed*. 5

The language does not define what happens to external files after the completion of the main program and all the library tasks (in particular, if corresponding files have not been closed). The effect of input-output for access types is unspecified. 6

An open file has a *current mode*, which is a value of one of the following enumeration types: 7

```
type File_Mode is (In_File, Inout_File, Out_File); -- for Direct_IO 8
```

These values correspond respectively to the cases where only reading, both reading and writing, or only writing are to be performed. 9

```
type File_Mode is (In_File, Out_File, Append_File); 10/2
-- for Sequential_IO, Text_IO, Wide_Text_IO, Wide_Wide_Text_IO, and Stream_IO
```

These values correspond respectively to the cases where only reading, only writing, or only appending are to be performed. 11

The mode of a file can be changed. 12

Several file management operations are common to Sequential_IO, Direct_IO, Text_IO, Wide_Text_IO, and Wide_Wide_Text_IO. These operations are described in subclause A.8.2 for sequential and direct files. Any additional effects concerning text input-output are described in subclause A.10.2. 13/2

The exceptions that can be propagated by the execution of an input-output subprogram are defined in the package IO_Exceptions; the situations in which they can be propagated are described following the description of the subprogram (and in clause A.13). The exceptions Storage_Error and Program_Error may be propagated. (Program_Error can only be propagated due to errors made by the caller of the subprogram.) Finally, exceptions can be propagated in certain implementation-defined situations. 14

NOTES

19 Each instantiation of the generic packages Sequential_IO and Direct_IO declares a different type File_Type. In the case of Text_IO, Wide_Text_IO, Wide_Wide_Text_IO, and Streams.Stream_IO, the corresponding type File_Type is unique. 15/2

20 A bidirectional device can often be modeled as two sequential files associated with the device, one of mode In_File, and one of mode Out_File. An implementation may restrict the number of files that may be associated with a given external file. 16

A.8 Sequential and Direct Files

Static Semantics

Two kinds of access to external files are defined in this subclause: *sequential access* and *direct access*. The corresponding file types and the associated operations are provided by the generic packages Sequential_IO and Direct_IO. A file object to be used for sequential access is called a *sequential file*, and one to be used for direct access is called a *direct file*. Access to *stream files* is described in A.12.1. 1/2

For sequential access, the file is viewed as a sequence of values that are transferred in the order of their appearance (as produced by the program or by the external environment). When the file is opened with mode In_File or Out_File, transfer starts respectively from or to the beginning of the file. When the file is opened with mode Append_File, transfer to the file starts after the last element of the file. 2

- 3 For direct access, the file is viewed as a set of elements occupying consecutive positions in linear order; a value can be transferred to or from an element of the file at any selected position. The position of an element is specified by its *index*, which is a number, greater than zero, of the implementation-defined integer type Count. The first element, if any, has index one; the index of the last element, if any, is called the *current size*; the current size is zero if there are no elements. The current size is a property of the external file.
- 4 An open direct file has a *current index*, which is the index that will be used by the next read or write operation. When a direct file is opened, the current index is set to one. The current index of a direct file is a property of a file object, not of an external file.

A.8.1 The Generic Package Sequential_IO

Static Semantics

1 The generic library package Sequential_IO has the following declaration:

```

2   with Ada.IO_Exceptions;
3   generic
4     type Element_Type(<>) is private;
5   package Ada.Sequential_IO is
6
7     type File_Type is limited private;
8     type File_Mode is (In_File, Out_File, Append_File);
9     -- File management
10
11    procedure Create(File : in out File_Type;
12                    Mode : in File_Mode := Out_File;
13                    Name : in String := "";
14                    Form : in String := "");
15
16    procedure Open  (File : in out File_Type;
17                    Mode : in File_Mode;
18                    Name : in String;
19                    Form : in String := "");
20
21    procedure Close (File : in out File_Type);
22    procedure Delete(File : in out File_Type);
23    procedure Reset (File : in out File_Type; Mode : in File_Mode);
24    procedure Reset (File : in out File_Type);
25
26    function Mode  (File : in File_Type) return File_Mode;
27    function Name  (File : in File_Type) return String;
28    function Form  (File : in File_Type) return String;
29
30    function Is_Open(File : in File_Type) return Boolean;
31
32    -- Input and output operations
33
34    procedure Read  (File : in File_Type; Item : out Element_Type);
35    procedure Write (File : in File_Type; Item : in Element_Type);
36
37    function End_Of_File(File : in File_Type) return Boolean;
38
39    -- Exceptions
40
41    Status_Error : exception renames IO_Exceptions.Status_Error;
42    Mode_Error   : exception renames IO_Exceptions.Mode_Error;
43    Name_Error   : exception renames IO_Exceptions.Name_Error;
44    Use_Error    : exception renames IO_Exceptions.Use_Error;
45    Device_Error : exception renames IO_Exceptions.Device_Error;
46    End_Error    : exception renames IO_Exceptions.End_Error;
47    Data_Error   : exception renames IO_Exceptions.Data_Error;
48
49    private
50      ... -- not specified by the language
51    end Ada.Sequential_IO;
```

The type `File_Type` needs finalization (see 7.6) in every instantiation of `Sequential_IO`.

17/2

A.8.2 File Management

Static Semantics

The procedures and functions described in this subclause provide for the control of external files; their declarations are repeated in each of the packages for sequential, direct, text, and stream input-output. For text input-output, the procedures `Create`, `Open`, and `Reset` have additional effects described in subclause A.10.2.

```
procedure Create(File : in out File_Type;  
                Mode : in File_Mode := default_mode;  
                Name : in String := "";  
                Form : in String := "");
```

Establishes a new external file, with the given name and form, and associates this external file with the given file. The given file is left open. The current mode of the given file is set to the given access mode. The default access mode is the mode `Out_File` for sequential, stream, and text input-output; it is the mode `Inout_File` for direct input-output. For direct access, the size of the created file is implementation defined.

A null string for `Name` specifies an external file that is not accessible after the completion of the main program (a temporary file). A null string for `Form` specifies the use of the default options of the implementation for the external file.

The exception `Status_Error` is propagated if the given file is already open. The exception `Name_Error` is propagated if the string given as `Name` does not allow the identification of an external file. The exception `Use_Error` is propagated if, for the specified mode, the external environment does not support creation of an external file with the given name (in the absence of `Name_Error`) and form.

```
procedure Open(File : in out File_Type;  
              Mode : in File_Mode;  
              Name : in String;  
              Form : in String := "");
```

Associates the given file with an existing external file having the given name and form, and sets the current mode of the given file to the given mode. The given file is left open.

The exception `Status_Error` is propagated if the given file is already open. The exception `Name_Error` is propagated if the string given as `Name` does not allow the identification of an external file; in particular, this exception is propagated if no external file with the given name exists. The exception `Use_Error` is propagated if, for the specified mode, the external environment does not support opening for an external file with the given name (in the absence of `Name_Error`) and form.

```
procedure Close(File : in out File_Type);
```

Severs the association between the given file and its associated external file. The given file is left closed. In addition, for sequential files, if the file being closed has mode `Out_File` or `Append_File`, then the last element written since the most recent open or reset is the last element that can be read from the file. If no elements have been written and the file mode is `Out_File`, then the closed file is empty. If no elements have been written and the file mode is `Append_File`, then the closed file is unchanged.

11 The exception `Status_Error` is propagated if the given file is not open.

12 **procedure** `Delete`(`File` : **in out** `File_Type`);

13 Deletes the external file associated with the given file. The given file is closed, and the external file ceases to exist.

14 The exception `Status_Error` is propagated if the given file is not open. The exception `Use_Error` is propagated if deletion of the external file is not supported by the external environment.

15 **procedure** `Reset`(`File` : **in out** `File_Type`; `Mode` : **in** `File_Mode`);

16/2 **procedure** `Reset`(`File` : **in out** `File_Type`);

 Resets the given file so that reading from its elements can be restarted from the beginning of the external file (for modes `In_File` and `Inout_File`), and so that writing to its elements can be restarted at the beginning of the external file (for modes `Out_File` and `Inout_File`) or after the last element of the external file (for mode `Append_File`). In particular, for direct access this means that the current index is set to one. If a `Mode` parameter is supplied, the current mode of the given file is set to the given mode. In addition, for sequential files, if the given file has mode `Out_File` or `Append_File` when `Reset` is called, the last element written since the most recent open or reset is the last element that can be read from the external file. If no elements have been written and the file mode is `Out_File`, the reset file is empty. If no elements have been written and the file mode is `Append_File`, then the reset file is unchanged.

17 The exception `Status_Error` is propagated if the file is not open. The exception `Use_Error` is propagated if the external environment does not support resetting for the external file and, also, if the external environment does not support resetting to the specified mode for the external file.

18 **function** `Mode`(`File` : **in** `File_Type`) **return** `File_Mode`;

19 Returns the current mode of the given file.

20 The exception `Status_Error` is propagated if the file is not open.

21 **function** `Name`(`File` : **in** `File_Type`) **return** `String`;

22/2 Returns a string which uniquely identifies the external file currently associated with the given file (and may thus be used in an `Open` operation).

23 The exception `Status_Error` is propagated if the given file is not open. The exception `Use_Error` is propagated if the associated external file is a temporary file that cannot be opened by any name.

24 **function** `Form`(`File` : **in** `File_Type`) **return** `String`;

25 Returns the form string for the external file currently associated with the given file. If an external environment allows alternative specifications of the form (for example, abbreviations using default options), the string returned by the function should correspond to a full specification (that is, it should indicate explicitly all options selected, including default options).

26 The exception `Status_Error` is propagated if the given file is not open.

27 **function** `Is_Open`(`File` : **in** `File_Type`) **return** `Boolean`;

28 Returns `True` if the file is open (that is, if it is associated with an external file), otherwise returns `False`.

Implementation Permissions

An implementation may propagate `Name_Error` or `Use_Error` if an attempt is made to use an I/O feature that cannot be supported by the implementation due to limitations in the external environment. Any such restriction should be documented. 29

A.8.3 Sequential Input-Output Operations*Static Semantics*

The operations available for sequential input and output are described in this subclause. The exception `Status_Error` is propagated if any of these operations is attempted for a file that is not open. 1

procedure `Read`(`File` : **in** `File_Type`; `Item` : **out** `Element_Type`); 2

Operates on a file of mode `In_File`. Reads an element from the given file, and returns the value of this element in the `Item` parameter. 3

The exception `Mode_Error` is propagated if the mode is not `In_File`. The exception `End_Error` is propagated if no more elements can be read from the given file. The exception `Data_Error` can be propagated if the element read cannot be interpreted as a value of the subtype `Element_Type` (see A.13, “Exceptions in Input-Output”). 4

procedure `Write`(`File` : **in** `File_Type`; `Item` : **in** `Element_Type`); 5

Operates on a file of mode `Out_File` or `Append_File`. Writes the value of `Item` to the given file. 6

The exception `Mode_Error` is propagated if the mode is not `Out_File` or `Append_File`. The exception `Use_Error` is propagated if the capacity of the external file is exceeded. 7

function `End_Of_File`(`File` : **in** `File_Type`) **return** `Boolean`; 8

Operates on a file of mode `In_File`. Returns `True` if no more elements can be read from the given file; otherwise returns `False`. 9

The exception `Mode_Error` is propagated if the mode is not `In_File`. 10

A.8.4 The Generic Package `Direct_IO`*Static Semantics*

The generic library package `Direct_IO` has the following declaration: 1

with `Ada.IO_Exceptions`; 2

generic

type `Element_Type` **is private**;

package `Ada.Direct_IO` **is**

type `File_Type` **is limited private**; 3

type `File_Mode` **is** (`In_File`, `Inout_File`, `Out_File`); 4

type `Count` **is range** 0 .. *implementation-defined*;

subtype `Positive_Count` **is** `Count` **range** 1 .. `Count'Last`;

 -- *File management*

procedure `Create`(`File` : **in out** `File_Type`; 5

`Mode` : **in** `File_Mode` := `Inout_File`;

`Name` : **in** `String` := "";

`Form` : **in** `String` := ""); 6

```

7      procedure Open (File : in out File_Type;
                      Mode : in File_Mode;
                      Name : in String;
                      Form : in String := "");
8
9      procedure Close (File : in out File_Type);
      procedure Delete (File : in out File_Type);
      procedure Reset (File : in out File_Type; Mode : in File_Mode);
      procedure Reset (File : in out File_Type);
10
11     function Mode (File : in File_Type) return File_Mode;
12     function Name (File : in File_Type) return String;
13     function Form (File : in File_Type) return String;
14
15     function Is_Open (File : in File_Type) return Boolean;
16
17     -- Input and output operations
18
19     procedure Read (File : in File_Type; Item : out Element_Type;
                    From : in Positive_Count);
20     procedure Read (File : in File_Type; Item : out Element_Type);
21
22     procedure Write (File : in File_Type; Item : in Element_Type;
                     To : in Positive_Count);
23     procedure Write (File : in File_Type; Item : in Element_Type);
24
25     procedure Set_Index (File : in File_Type; To : in Positive_Count);
26
27     function Index (File : in File_Type) return Positive_Count;
28     function Size (File : in File_Type) return Count;
29
30     function End_Of_File (File : in File_Type) return Boolean;
31
32     -- Exceptions
33
34     Status_Error : exception renames IO_Exceptions.Status_Error;
35     Mode_Error : exception renames IO_Exceptions.Mode_Error;
36     Name_Error : exception renames IO_Exceptions.Name_Error;
37     Use_Error : exception renames IO_Exceptions.Use_Error;
38     Device_Error : exception renames IO_Exceptions.Device_Error;
39     End_Error : exception renames IO_Exceptions.End_Error;
40     Data_Error : exception renames IO_Exceptions.Data_Error;
41
42     private
43     ... -- not specified by the language
44     end Ada.Direct_IO;

```

20/2 The type `File_Type` needs finalization (see 7.6) in every instantiation of `Direct_IO`.

A.8.5 Direct Input-Output Operations

Static Semantics

1 The operations available for direct input and output are described in this subclause. The exception `Status_Error` is propagated if any of these operations is attempted for a file that is not open.

```

2      procedure Read (File : in File_Type; Item : out Element_Type;
                    From : in Positive_Count);
3      procedure Read (File : in File_Type; Item : out Element_Type);

```

3 Operates on a file of mode `In_File` or `Inout_File`. In the case of the first form, sets the current index of the given file to the index value given by the parameter `From`. Then (for both forms) returns, in the parameter `Item`, the value of the element whose position in the given file is specified by the current index of the file; finally, increases the current index by one.

4 The exception `Mode_Error` is propagated if the mode of the given file is `Out_File`. The exception `End_Error` is propagated if the index to be used exceeds the size of the external file. The exception `Data_Error` can be propagated if the element read cannot be interpreted as a value of the subtype `Element_Type` (see A.13).

procedure Write(File : in File_Type; Item : in Element_Type;	5
To : in Positive_Count);	
procedure Write(File : in File_Type; Item : in Element_Type);	
Operates on a file of mode Inout_File or Out_File. In the case of the first form, sets the index of the given file to the index value given by the parameter To. Then (for both forms) gives the value of the parameter Item to the element whose position in the given file is specified by the current index of the file; finally, increases the current index by one.	6
The exception Mode_Error is propagated if the mode of the given file is In_File. The exception Use_Error is propagated if the capacity of the external file is exceeded.	7
procedure Set_Index(File : in File_Type; To : in Positive_Count);	8
Operates on a file of any mode. Sets the current index of the given file to the given index value (which may exceed the current size of the file).	9
function Index(File : in File_Type) return Positive_Count;	10
Operates on a file of any mode. Returns the current index of the given file.	11
function Size(File : in File_Type) return Count;	12
Operates on a file of any mode. Returns the current size of the external file that is associated with the given file.	13
function End_Of_File(File : in File_Type) return Boolean;	14
Operates on a file of mode In_File or Inout_File. Returns True if the current index exceeds the size of the external file; otherwise returns False.	15
The exception Mode_Error is propagated if the mode of the given file is Out_File.	16
NOTES	
21 Append_File mode is not supported for the generic package Direct_IO.	17

A.9 The Generic Package Storage_IO

The generic package Storage_IO provides for reading from and writing to an in-memory buffer. This generic package supports the construction of user-defined input-output packages.

Static Semantics

The generic library package Storage_IO has the following declaration:

with Ada.IO_Exceptions;	3
with System.Storage_Elements;	
generic	
type Element_Type is private ;	
package Ada.Storage_IO is	
pragma Preelaborate(Storage_IO);	
Buffer_Size : constant System.Storage_Elements.Storage_Count :=	4
<i>implementation-defined</i> ;	
subtype Buffer_Type is	
System.Storage_Elements.Storage_Array(1..Buffer_Size);	
-- Input and output operations	5
procedure Read (Buffer : in Buffer_Type; Item : out Element_Type);	6
procedure Write(Buffer : out Buffer_Type; Item : in Element_Type);	7

-- *Exceptions*

```
Data_Error      : exception renames IO_Exceptions.Data_Error;
end Ada.Storage_IO;
```

In each instance, the constant `Buffer_Size` has a value that is the size (in storage elements) of the buffer required to represent the content of an object of subtype `Element_Type`, including any implicit levels of indirection used by the implementation. The `Read` and `Write` procedures of `Storage_IO` correspond to the `Read` and `Write` procedures of `Direct_IO` (see A.8.4), but with the content of the `Item` parameter being read from or written into the specified `Buffer`, rather than an external file.

NOTES

22 A buffer used for `Storage_IO` holds only one element at a time; an external file used for `Direct_IO` holds a sequence of elements.

A.10 Text Input-Output

Static Semantics

This clause describes the package `Text_IO`, which provides facilities for input and output in human-readable form. Each file is read or written sequentially, as a sequence of characters grouped into lines, and as a sequence of lines grouped into pages. The specification of the package is given below in subclause A.10.1.

The facilities for file management given above, in subclauses A.8.2 and A.8.3, are available for text input-output. In place of `Read` and `Write`, however, there are procedures `Get` and `Put` that input values of suitable types from text files, and output values to them. These values are provided to the `Put` procedures, and returned by the `Get` procedures, in a parameter `Item`. Several overloaded procedures of these names exist, for different types of `Item`. These `Get` procedures analyze the input sequences of characters based on lexical elements (see Section 2) and return the corresponding values; the `Put` procedures output the given values as appropriate lexical elements. Procedures `Get` and `Put` are also available that input and output individual characters treated as character values rather than as lexical elements. Related to character input are procedures to look ahead at the next character without reading it, and to read a character “immediately” without waiting for an end-of-line to signal availability.

In addition to the procedures `Get` and `Put` for numeric and enumeration types of `Item` that operate on text files, analogous procedures are provided that read from and write to a parameter of type `String`. These procedures perform the same analysis and composition of character sequences as their counterparts which have a file parameter.

For all `Get` and `Put` procedures that operate on text files, and for many other subprograms, there are forms with and without a file parameter. Each such `Get` procedure operates on an input file, and each such `Put` procedure operates on an output file. If no file is specified, a default input file or a default output file is used.

At the beginning of program execution the default input and output files are the so-called standard input file and standard output file. These files are open, have respectively the current modes `In_File` and `Out_File`, and are associated with two implementation-defined external files. Procedures are provided to change the current default input file and the current default output file.

At the beginning of program execution a default file for program-dependent error-related text output is the so-called standard error file. This file is open, has the current mode `Out_File`, and is associated with an implementation-defined external file. A procedure is provided to change the current default error file.

From a logical point of view, a text file is a sequence of pages, a page is a sequence of lines, and a line is a sequence of characters; the end of a line is marked by a *line terminator*; the end of a page is marked by the combination of a line terminator immediately followed by a *page terminator*; and the end of a file is marked by the combination of a line terminator immediately followed by a page terminator and then a *file terminator*. Terminators are generated during output; either by calls of procedures provided expressly for that purpose; or implicitly as part of other operations, for example, when a bounded line length, a bounded page length, or both, have been specified for a file.

The actual nature of terminators is not defined by the language and hence depends on the implementation. Although terminators are recognized or generated by certain of the procedures that follow, they are not necessarily implemented as characters or as sequences of characters. Whether they are characters (and if so which ones) in any particular implementation need not concern a user who neither explicitly outputs nor explicitly inputs control characters. The effect of input (Get) or output (Put) of control characters (other than horizontal tabulation) is not specified by the language.

The characters of a line are numbered, starting from one; the number of a character is called its *column number*. For a line terminator, a column number is also defined: it is one more than the number of characters in the line. The lines of a page, and the pages of a file, are similarly numbered. The current column number is the column number of the next character or line terminator to be transferred. The current line number is the number of the current line. The current page number is the number of the current page. These numbers are values of the subtype Positive_Count of the type Count (by convention, the value zero of the type Count is used to indicate special conditions).

```
type Count is range 0 .. implementation-defined;  
subtype Positive_Count is Count range 1 .. Count'Last;
```

For an output file or an append file, a *maximum line length* can be specified and a *maximum page length* can be specified. If a value to be output cannot fit on the current line, for a specified maximum line length, then a new line is automatically started before the value is output; if, further, this new line cannot fit on the current page, for a specified maximum page length, then a new page is automatically started before the value is output. Functions are provided to determine the maximum line length and the maximum page length. When a file is opened with mode Out_File or Append_File, both values are zero: by convention, this means that the line lengths and page lengths are unbounded. (Consequently, output consists of a single line if the subprograms for explicit control of line and page structure are not used.) The constant Unbounded is provided for this purpose.

A.10.1 The Package Text_IO

Static Semantics

The library package Text_IO has the following declaration:

```
with Ada.IO_Exceptions;  
package Ada.Text_IO is  
    type File_Type is limited private;  
    type File_Mode is (In_File, Out_File, Append_File);  
    type Count is range 0 .. implementation-defined;  
    subtype Positive_Count is Count range 1 .. Count'Last;  
    Unbounded : constant Count := 0; -- line and page length  
    subtype Field      is Integer range 0 .. implementation-defined;  
    subtype Number_Base is Integer range 2 .. 16;  
    type Type_Set is (Lower_Case, Upper_Case);  
    -- File Management
```

```

9      procedure Create (File : in out File_Type;
                        Mode : in File_Mode := Out_File;
                        Name : in String    := "";
                        Form : in String    := "");
10
11     procedure Open   (File : in out File_Type;
                        Mode : in File_Mode;
                        Name : in String;
                        Form : in String := "");
12
13     procedure Close  (File : in out File_Type);
14     procedure Delete (File : in out File_Type);
15     procedure Reset  (File : in out File_Type; Mode : in File_Mode);
16     procedure Reset  (File : in out File_Type);
17
18     function Mode     (File : in File_Type) return File_Mode;
19     function Name     (File : in File_Type) return String;
20     function Form     (File : in File_Type) return String;
21
22     function Is_Open (File : in File_Type) return Boolean;
23
24     -- Control of default input and output files
25
26     procedure Set_Input  (File : in File_Type);
27     procedure Set_Output (File : in File_Type);
28     procedure Set_Error  (File : in File_Type);
29
30     function Standard_Input  return File_Type;
31     function Standard_Output return File_Type;
32     function Standard_Error  return File_Type;
33
34     function Current_Input   return File_Type;
35     function Current_Output  return File_Type;
36     function Current_Error   return File_Type;
37
38     type File_Access is access constant File_Type;
39
40     function Standard_Input  return File_Access;
41     function Standard_Output return File_Access;
42     function Standard_Error  return File_Access;
43
44     function Current_Input   return File_Access;
45     function Current_Output  return File_Access;
46     function Current_Error   return File_Access;
47
48 21/1 -- Buffer control
49     procedure Flush (File : in File_Type);
50     procedure Flush;
51
52     -- Specification of line and page lengths
53
54     procedure Set_Line_Length (File : in File_Type; To : in Count);
55     procedure Set_Line_Length (To : in Count);
56
57     procedure Set_Page_Length (File : in File_Type; To : in Count);
58     procedure Set_Page_Length (To : in Count);
59
60     function Line_Length (File : in File_Type) return Count;
61     function Line_Length return Count;
62
63     function Page_Length (File : in File_Type) return Count;
64     function Page_Length return Count;
65
66     -- Column, Line, and Page Control
67
68     procedure New_Line (File : in File_Type;
69                        Spacing : in Positive_Count := 1);
70     procedure New_Line (Spacing : in Positive_Count := 1);
71
72     procedure Skip_Line (File : in File_Type;
73                        Spacing : in Positive_Count := 1);
74     procedure Skip_Line (Spacing : in Positive_Count := 1);
75
76     function End_Of_Line (File : in File_Type) return Boolean;
77     function End_Of_Line return Boolean;
78
79     procedure New_Page (File : in File_Type);
80     procedure New_Page;

```

```

procedure Skip_Page (File : in File_Type);
procedure Skip_Page;
function End_Of_Page(File : in File_Type) return Boolean;
function End_Of_Page return Boolean;
function End_Of_File(File : in File_Type) return Boolean;
function End_Of_File return Boolean;
procedure Set_Col (File : in File_Type; To : in Positive_Count);
procedure Set_Col (To : in Positive_Count);
procedure Set_Line(File : in File_Type; To : in Positive_Count);
procedure Set_Line(To : in Positive_Count);
function Col (File : in File_Type) return Positive_Count;
function Col return Positive_Count;
function Line (File : in File_Type) return Positive_Count;
function Line return Positive_Count;
function Page(File : in File_Type) return Positive_Count;
function Page return Positive_Count;
-- Character Input-Output
procedure Get(File : in File_Type; Item : out Character);
procedure Get(Item : out Character);
procedure Put(File : in File_Type; Item : in Character);
procedure Put(Item : in Character);
procedure Look_Ahead (File : in File_Type;
                      Item : out Character;
                      End_Of_Line : out Boolean);
procedure Look_Ahead (Item : out Character;
                      End_Of_Line : out Boolean);
procedure Get_Immediate(File : in File_Type;
                        Item : out Character);
procedure Get_Immediate(Item : out Character);
procedure Get_Immediate(File : in File_Type;
                        Item : out Character;
                        Available : out Boolean);
procedure Get_Immediate(Item : out Character;
                        Available : out Boolean);
-- String Input-Output
procedure Get(File : in File_Type; Item : out String);
procedure Get(Item : out String);
procedure Put(File : in File_Type; Item : in String);
procedure Put(Item : in String);
procedure Get_Line(File : in File_Type;
                  Item : out String;
                  Last : out Natural);
procedure Get_Line(Item : out String; Last : out Natural);
function Get_Line(File : in File_Type) return String;
function Get_Line return String;
procedure Put_Line(File : in File_Type; Item : in String);
procedure Put_Line(Item : in String);
-- Generic packages for Input-Output of Integer Types
generic
  type Num is range <>;
package Integer_IO is
  Default_Width : Field := Num'Width;
  Default_Base : Number_Base := 10;

```

```

54      procedure Get(File : in File_Type;
                    Item : out Num;
                    Width : in Field := 0);
      procedure Get(Item : out Num;
                    Width : in Field := 0);
55      procedure Put(File : in File_Type;
                    Item : in Num;
                    Width : in Field := Default_Width;
                    Base : in Number_Base := Default_Base);
      procedure Put(Item : in Num;
                    Width : in Field := Default_Width;
                    Base : in Number_Base := Default_Base);
      procedure Get(From : in String;
                    Item : out Num;
                    Last : out Positive);
      procedure Put(To : out String;
                    Item : in Num;
                    Base : in Number_Base := Default_Base);
56  end Integer_IO;
57  generic
      type Num is mod <>;
      package Modular_IO is
58      Default_Width : Field := Num'Width;
      Default_Base : Number_Base := 10;
59      procedure Get(File : in File_Type;
                    Item : out Num;
                    Width : in Field := 0);
      procedure Get(Item : out Num;
                    Width : in Field := 0);
60      procedure Put(File : in File_Type;
                    Item : in Num;
                    Width : in Field := Default_Width;
                    Base : in Number_Base := Default_Base);
      procedure Put(Item : in Num;
                    Width : in Field := Default_Width;
                    Base : in Number_Base := Default_Base);
      procedure Get(From : in String;
                    Item : out Num;
                    Last : out Positive);
      procedure Put(To : out String;
                    Item : in Num;
                    Base : in Number_Base := Default_Base);
61  end Modular_IO;
62  -- Generic packages for Input-Output of Real Types
63  generic
      type Num is digits <>;
      package Float_IO is
64      Default_Fore : Field := 2;
      Default_Aft : Field := Num'Digits-1;
      Default_Exp : Field := 3;
65      procedure Get(File : in File_Type;
                    Item : out Num;
                    Width : in Field := 0);
      procedure Get(Item : out Num;
                    Width : in Field := 0);

```

```

procedure Put (File : in File_Type;
                Item : in Num;
                Fore : in Field := Default_Fore;
                Aft  : in Field := Default_Aft;
                Exp  : in Field := Default_Exp);
procedure Put (Item : in Num;
                Fore : in Field := Default_Fore;
                Aft  : in Field := Default_Aft;
                Exp  : in Field := Default_Exp);

procedure Get (From : in String;
                Item : out Num;
                Last : out Positive);
procedure Put (To   : out String;
                Item : in Num;
                Aft  : in Field := Default_Aft;
                Exp  : in Field := Default_Exp);
end Float_IO;

generic
    type Num is delta <>;
package Fixed_IO is
    Default_Fore : Field := Num'Fore;
    Default_Aft  : Field := Num'Aft;
    Default_Exp  : Field := 0;

    procedure Get (File : in File_Type;
                  Item : out Num;
                  Width : in Field := 0);
    procedure Get (Item : out Num;
                  Width : in Field := 0);

    procedure Put (File : in File_Type;
                  Item : in Num;
                  Fore : in Field := Default_Fore;
                  Aft  : in Field := Default_Aft;
                  Exp  : in Field := Default_Exp);
    procedure Put (Item : in Num;
                  Fore : in Field := Default_Fore;
                  Aft  : in Field := Default_Aft;
                  Exp  : in Field := Default_Exp);

    procedure Get (From : in String;
                  Item : out Num;
                  Last : out Positive);
    procedure Put (To   : out String;
                  Item : in Num;
                  Aft  : in Field := Default_Aft;
                  Exp  : in Field := Default_Exp);
end Fixed_IO;

generic
    type Num is delta <> digits <>;
package Decimal_IO is
    Default_Fore : Field := Num'Fore;
    Default_Aft  : Field := Num'Aft;
    Default_Exp  : Field := 0;

    procedure Get (File : in File_Type;
                  Item : out Num;
                  Width : in Field := 0);
    procedure Get (Item : out Num;
                  Width : in Field := 0);

```

```

76      procedure Put(File : in File_Type;
                    Item : in Num;
                    Fore : in Field := Default_Fore;
                    Aft  : in Field := Default_Aft;
                    Exp  : in Field := Default_Exp);
      procedure Put(Item : in Num;
                    Fore : in Field := Default_Fore;
                    Aft  : in Field := Default_Aft;
                    Exp  : in Field := Default_Exp);
77      procedure Get(From : in String;
                    Item : out Num;
                    Last : out Positive);
      procedure Put(To : out String;
                    Item : in Num;
                    Aft  : in Field := Default_Aft;
                    Exp  : in Field := Default_Exp);
      end Decimal_IO;
78      -- Generic package for Input-Output of Enumeration Types
79      generic
      type Enum is (<>);
      package Enumeration_IO is
80          Default_Width : Field := 0;
          Default_Setting : Type_Set := Upper_Case;
81          procedure Get(File : in File_Type;
                        Item : out Enum);
          procedure Get(Item : out Enum);
82          procedure Put(File : in File_Type;
                        Item : in Enum;
                        Width : in Field := Default_Width;
                        Set   : in Type_Set := Default_Setting);
          procedure Put(Item : in Enum;
                        Width : in Field := Default_Width;
                        Set   : in Type_Set := Default_Setting);
83          procedure Get(From : in String;
                        Item : out Enum;
                        Last : out Positive);
          procedure Put(To : out String;
                        Item : in Enum;
                        Set : in Type_Set := Default_Setting);
      end Enumeration_IO;
84      -- Exceptions
85      Status_Error : exception renames IO_Exceptions.Status_Error;
      Mode_Error   : exception renames IO_Exceptions.Mode_Error;
      Name_Error   : exception renames IO_Exceptions.Name_Error;
      Use_Error    : exception renames IO_Exceptions.Use_Error;
      Device_Error : exception renames IO_Exceptions.Device_Error;
      End_Error    : exception renames IO_Exceptions.End_Error;
      Data_Error   : exception renames IO_Exceptions.Data_Error;
      Layout_Error : exception renames IO_Exceptions.Layout_Error;
      private
      ... -- not specified by the language
      end Ada.Text_IO;

```

86/2 The type File_Type needs finalization (see 7.6).

A.10.2 Text File Management

Static Semantics

- 1 The only allowed file modes for text files are the modes In_File, Out_File, and Append_File. The subprograms given in subclause A.8.2 for the control of external files, and the function End_Of_File given

in subclause A.8.3 for sequential input-output, are also available for text files. There is also a version of `End_Of_File` that refers to the current default input file. For text files, the procedures have the following additional effects:

- For the procedures `Create` and `Open`: After a file with mode `Out_File` or `Append_File` is opened, the page length and line length are unbounded (both have the conventional value zero). After a file (of any mode) is opened, the current column, current line, and current page numbers are set to one. If the mode is `Append_File`, it is implementation defined whether a page terminator will separate preexisting text in the file from the new text to be written. 2
- For the procedure `Close`: If the file has the current mode `Out_File` or `Append_File`, has the effect of calling `New_Page`, unless the current page is already terminated; then outputs a file terminator. 3
- For the procedure `Reset`: If the file has the current mode `Out_File` or `Append_File`, has the effect of calling `New_Page`, unless the current page is already terminated; then outputs a file terminator. The current column, line, and page numbers are set to one, and the line and page lengths to Unbounded. If the new mode is `Append_File`, it is implementation defined whether a page terminator will separate preexisting text in the file from the new text to be written. 4

The exception `Mode_Error` is propagated by the procedure `Reset` upon an attempt to change the mode of a file that is the current default input file, the current default output file, or the current default error file. 5

NOTES

23 An implementation can define the `Form` parameter of `Create` and `Open` to control effects including the following: 6

- the interpretation of line and column numbers for an interactive file, and 7
- the interpretation of text formats in a file created by a foreign program. 8

A.10.3 Default Input, Output, and Error Files

Static Semantics

The following subprograms provide for the control of the particular default files that are used when a file parameter is omitted from a `Get`, `Put`, or other operation of text input-output described below, or when application-dependent error-related text is to be output. 1

```
procedure Set_Input (File : in File_Type); 2
```

Operates on a file of mode `In_File`. Sets the current default input file to `File`. 3

The exception `Status_Error` is propagated if the given file is not open. The exception `Mode_Error` is propagated if the mode of the given file is not `In_File`. 4

```
procedure Set_Output (File : in File_Type); 5
procedure Set_Error (File : in File_Type);
```

Each operates on a file of mode `Out_File` or `Append_File`. `Set_Output` sets the current default output file to `File`. `Set_Error` sets the current default error file to `File`. The exception `Status_Error` is propagated if the given file is not open. The exception `Mode_Error` is propagated if the mode of the given file is not `Out_File` or `Append_File`. 6

```
function Standard_Input return File_Type; 7
function Standard_Input return File_Access;
```

Returns the standard input file (see A.10), or an access value designating the standard input file, respectively. 8

9 **function** Standard_Output **return** File_Type;
 function Standard_Output **return** File_Access;

10 Returns the standard output file (see A.10) or an access value designating the standard output file, respectively.

11 **function** Standard_Error **return** File_Type;
 function Standard_Error **return** File_Access;

12/1 Returns the standard error file (see A.10), or an access value designating the standard error file, respectively.

13 The Form strings implicitly associated with the opening of Standard_Input, Standard_Output, and Standard_Error at the start of program execution are implementation defined.

14 **function** Current_Input **return** File_Type;
 function Current_Input **return** File_Access;

15 Returns the current default input file, or an access value designating the current default input file, respectively.

16 **function** Current_Output **return** File_Type;
 function Current_Output **return** File_Access;

17 Returns the current default output file, or an access value designating the current default output file, respectively.

18 **function** Current_Error **return** File_Type;
 function Current_Error **return** File_Access;

19 Returns the current default error file, or an access value designating the current default error file, respectively.

20/1 **procedure** Flush (File : **in** File_Type);
 procedure Flush;

21 The effect of Flush is the same as the corresponding subprogram in Streams.Stream_IO (see A.12.1). If File is not explicitly specified, Current_Output is used.

Erroneous Execution

22/1 The execution of a program is erroneous if it invokes an operation on a current default input, default output, or default error file, and if the corresponding file object is closed or no longer exists.

23/1 *This paragraph was deleted.*

NOTES

24 24 The standard input, standard output, and standard error files cannot be opened, closed, reset, or deleted, because the parameter File of the corresponding procedures has the mode **in out**.

25 25 The standard input, standard output, and standard error files are different file objects, but not necessarily different external files.

A.10.4 Specification of Line and Page Lengths

Static Semantics

1 The subprograms described in this subclause are concerned with the line and page structure of a file of mode Out_File or Append_File. They operate either on the file given as the first parameter, or, in the absence of such a file parameter, on the current default output file. They provide for output of text with a specified maximum line length or page length. In these cases, line and page terminators are output

implicitly and automatically when needed. When line and page lengths are unbounded (that is, when they have the conventional value zero), as in the case of a newly opened file, new lines and new pages are only started when explicitly called for.

In all cases, the exception `Status_Error` is propagated if the file to be used is not open; the exception `Mode_Error` is propagated if the mode of the file is not `Out_File` or `Append_File`.

```
procedure Set_Line_Length(File : in File_Type; To : in Count);
procedure Set_Line_Length(To : in Count);
```

Sets the maximum line length of the specified output or append file to the number of characters specified by `To`. The value zero for `To` specifies an unbounded line length.

The exception `Use_Error` is propagated if the specified line length is inappropriate for the associated external file.

```
procedure Set_Page_Length(File : in File_Type; To : in Count);
procedure Set_Page_Length(To : in Count);
```

Sets the maximum page length of the specified output or append file to the number of lines specified by `To`. The value zero for `To` specifies an unbounded page length.

The exception `Use_Error` is propagated if the specified page length is inappropriate for the associated external file.

```
function Line_Length(File : in File_Type) return Count;
function Line_Length return Count;
```

Returns the maximum line length currently set for the specified output or append file, or zero if the line length is unbounded.

```
function Page_Length(File : in File_Type) return Count;
function Page_Length return Count;
```

Returns the maximum page length currently set for the specified output or append file, or zero if the page length is unbounded.

A.10.5 Operations on Columns, Lines, and Pages

Static Semantics

The subprograms described in this subclause provide for explicit control of line and page structure; they operate either on the file given as the first parameter, or, in the absence of such a file parameter, on the appropriate (input or output) current default file. The exception `Status_Error` is propagated by any of these subprograms if the file to be used is not open.

```
procedure New_Line(File : in File_Type; Spacing : in Positive_Count := 1);
procedure New_Line(Spacing : in Positive_Count := 1);
```

Operates on a file of mode `Out_File` or `Append_File`.

For a `Spacing` of one: Outputs a line terminator and sets the current column number to one. Then increments the current line number by one, except in the case that the current line number is already greater than or equal to the maximum page length, for a bounded page length; in that case a page terminator is output, the current page number is incremented by one, and the current line number is set to one.

For a `Spacing` greater than one, the above actions are performed `Spacing` times.

The exception `Mode_Error` is propagated if the mode is not `Out_File` or `Append_File`.

```
procedure Skip_Line(File : in File_Type; Spacing : in Positive_Count := 1);
procedure Skip_Line(Spacing : in Positive_Count := 1);
```

Operates on a file of mode `In_File`.

For a `Spacing` of one: Reads and discards all characters until a line terminator has been read, and then sets the current column number to one. If the line terminator is not immediately followed by a page terminator, the current line number is incremented by one. Otherwise, if the line terminator is immediately followed by a page terminator, then the page terminator is skipped, the current page number is incremented by one, and the current line number is set to one.

For a `Spacing` greater than one, the above actions are performed `Spacing` times.

The exception `Mode_Error` is propagated if the mode is not `In_File`. The exception `End_Error` is propagated if an attempt is made to read a file terminator.

```
function End_Of_Line(File : in File_Type) return Boolean;
function End_Of_Line return Boolean;
```

Operates on a file of mode `In_File`. Returns `True` if a line terminator or a file terminator is next; otherwise returns `False`.

The exception `Mode_Error` is propagated if the mode is not `In_File`.

```
procedure New_Page(File : in File_Type);
procedure New_Page;
```

Operates on a file of mode `Out_File` or `Append_File`. Outputs a line terminator if the current line is not terminated, or if the current page is empty (that is, if the current column and line numbers are both equal to one). Then outputs a page terminator, which terminates the current page. Adds one to the current page number and sets the current column and line numbers to one.

The exception `Mode_Error` is propagated if the mode is not `Out_File` or `Append_File`.

```
procedure Skip_Page(File : in File_Type);
procedure Skip_Page;
```

Operates on a file of mode `In_File`. Reads and discards all characters and line terminators until a page terminator has been read. Then adds one to the current page number, and sets the current column and line numbers to one.

The exception `Mode_Error` is propagated if the mode is not `In_File`. The exception `End_Error` is propagated if an attempt is made to read a file terminator.

```
function End_Of_Page(File : in File_Type) return Boolean;
function End_Of_Page return Boolean;
```

Operates on a file of mode `In_File`. Returns `True` if the combination of a line terminator and a page terminator is next, or if a file terminator is next; otherwise returns `False`.

The exception `Mode_Error` is propagated if the mode is not `In_File`.

```
function End_Of_File(File : in File_Type) return Boolean;
function End_Of_File return Boolean;
```

Operates on a file of mode `In_File`. Returns `True` if a file terminator is next, or if the combination of a line, a page, and a file terminator is next; otherwise returns `False`.

The exception `Mode_Error` is propagated if the mode is not `In_File`.

The following subprograms provide for the control of the current position of reading or writing in a file. In all cases, the default file is the current output file.

```
procedure Set_Col(File : in File_Type; To : in Positive_Count);
procedure Set_Col(To   : in Positive_Count);
```

If the file mode is `Out_File` or `Append_File`:

- If the value specified by `To` is greater than the current column number, outputs spaces, adding one to the current column number after each space, until the current column number equals the specified value. If the value specified by `To` is equal to the current column number, there is no effect. If the value specified by `To` is less than the current column number, has the effect of calling `New_Line` (with a spacing of one), then outputs $(To - 1)$ spaces, and sets the current column number to the specified value.
- The exception `Layout_Error` is propagated if the value specified by `To` exceeds `Line_Length` when the line length is bounded (that is, when it does not have the conventional value zero).

If the file mode is `In_File`:

- Reads (and discards) individual characters, line terminators, and page terminators, until the next character to be read has a column number that equals the value specified by `To`; there is no effect if the current column number already equals this value. Each transfer of a character or terminator maintains the current column, line, and page numbers in the same way as a `Get` procedure (see A.10.6). (Short lines will be skipped until a line is reached that has a character at the specified column position.)
- The exception `End_Error` is propagated if an attempt is made to read a file terminator.

```
procedure Set_Line(File : in File_Type; To : in Positive_Count);
procedure Set_Line(To   : in Positive_Count);
```

If the file mode is `Out_File` or `Append_File`:

- If the value specified by `To` is greater than the current line number, has the effect of repeatedly calling `New_Line` (with a spacing of one), until the current line number equals the specified value. If the value specified by `To` is equal to the current line number, there is no effect. If the value specified by `To` is less than the current line number, has the effect of calling `New_Page` followed, if `To` is greater than 1, by a call of `New_Line` with a spacing equal to $(To - 1)$.
- The exception `Layout_Error` is propagated if the value specified by `To` exceeds `Page_Length` when the page length is bounded (that is, when it does not have the conventional value zero).

If the mode is `In_File`:

- Has the effect of repeatedly calling `Skip_Line` (with a spacing of one), until the current line number equals the value specified by `To`; there is no effect if the current line number already equals this value. (Short pages will be skipped until a page is reached that has a line at the specified line position.)
- The exception `End_Error` is propagated if an attempt is made to read a file terminator.

```
function Col(File : in File_Type) return Positive_Count;
function Col return Positive_Count;
```

Returns the current column number.

The exception `Layout_Error` is propagated if this number exceeds `Count'Last`.

```
function Line(File : in File_Type) return Positive_Count;
function Line return Positive_Count;
```

Returns the current line number.

The exception `Layout_Error` is propagated if this number exceeds `Count'Last`.

```
function Page(File : in File_Type) return Positive_Count;
function Page return Positive_Count;
```

Returns the current page number.

The exception `Layout_Error` is propagated if this number exceeds `Count'Last`.

The column number, line number, or page number are allowed to exceed `Count'Last` (as a consequence of the input or output of sufficiently many characters, lines, or pages). These events do not cause any exception to be propagated. However, a call of `Col`, `Line`, or `Page` propagates the exception `Layout_Error` if the corresponding number exceeds `Count'Last`.

NOTES

26 A page terminator is always skipped whenever the preceding line terminator is skipped. An implementation may represent the combination of these terminators by a single character, provided that it is properly recognized on input.

A.10.6 Get and Put Procedures

Static Semantics

The procedures `Get` and `Put` for items of the type `Character`, `String`, numeric types, and enumeration types are described in subsequent subclauses. Features of these procedures that are common to most of these types are described in this subclause. The `Get` and `Put` procedures for items of type `Character` and `String` deal with individual character values; the `Get` and `Put` procedures for numeric and enumeration types treat the items as lexical elements.

All procedures `Get` and `Put` have forms with a file parameter, written first. Where this parameter is omitted, the appropriate (input or output) current default file is understood to be specified. Each procedure `Get` operates on a file of mode `In_File`. Each procedure `Put` operates on a file of mode `Out_File` or `Append_File`.

All procedures `Get` and `Put` maintain the current column, line, and page numbers of the specified file: the effect of each of these procedures upon these numbers is the result of the effects of individual transfers of characters and of individual output or skipping of terminators. Each transfer of a character adds one to the current column number. Each output of a line terminator sets the current column number to one and adds one to the current line number. Each output of a page terminator sets the current column and line numbers to one and adds one to the current page number. For input, each skipping of a line terminator sets the current column number to one and adds one to the current line number; each skipping of a page terminator sets the current column and line numbers to one and adds one to the current page number. Similar considerations apply to the procedures `Get_Line`, `Put_Line`, and `Set_Col`.

Several `Get` and `Put` procedures, for numeric and enumeration types, have *format* parameters which specify field lengths; these parameters are of the nonnegative subtype `Field` of the type `Integer`.

Input-output of enumeration values uses the syntax of the corresponding lexical elements. Any `Get` procedure for an enumeration type begins by skipping any leading blanks, or line or page terminators. A *blank* is defined as a space or a horizontal tabulation character. Next, characters are input only so long as

the sequence input is an initial sequence of an identifier or of a character literal (in particular, input ceases when a line terminator is encountered). The character or line terminator that causes input to cease remains available for subsequent input.

For a numeric type, the Get procedures have a format parameter called Width. If the value given for this parameter is zero, the Get procedure proceeds in the same manner as for enumeration types, but using the syntax of numeric literals instead of that of enumeration literals. If a nonzero value is given, then exactly Width characters are input, or the characters up to a line terminator, whichever comes first; any skipped leading blanks are included in the count. The syntax used for numeric literals is an extended syntax that allows a leading sign (but no intervening blanks, or line or page terminators) and that also allows (for real types) an integer literal as well as forms that have digits only before the point or only after the point.

Any Put procedure, for an item of a numeric or an enumeration type, outputs the value of the item as a numeric literal, identifier, or character literal, as appropriate. This is preceded by leading spaces if required by the format parameters Width or Fore (as described in later subclauses), and then a minus sign for a negative value; for an enumeration type, the spaces follow instead of leading. The format given for a Put procedure is overridden if it is insufficiently wide, by using the minimum needed width.

Two further cases arise for Put procedures for numeric and enumeration types, if the line length of the specified output file is bounded (that is, if it does not have the conventional value zero). If the number of characters to be output does not exceed the maximum line length, but is such that they cannot fit on the current line, starting from the current column, then (in effect) New_Line is called (with a spacing of one) before output of the item. Otherwise, if the number of characters exceeds the maximum line length, then the exception Layout_Error is propagated and nothing is output.

The exception Status_Error is propagated by any of the procedures Get, Get_Line, Put, and Put_Line if the file to be used is not open. The exception Mode_Error is propagated by the procedures Get and Get_Line if the mode of the file to be used is not In_File; and by the procedures Put and Put_Line, if the mode is not Out_File or Append_File.

The exception End_Error is propagated by a Get procedure if an attempt is made to skip a file terminator. The exception Data_Error is propagated by a Get procedure if the sequence finally input is not a lexical element corresponding to the type, in particular if no characters were input; for this test, leading blanks are ignored; for an item of a numeric type, when a sign is input, this rule applies to the succeeding numeric literal. The exception Layout_Error is propagated by a Put procedure that outputs to a parameter of type String, if the length of the actual string is insufficient for the output of the item.

Examples

In the examples, here and in subclauses A.10.8 and A.10.9, the string quotes and the lower case letter b are not transferred: they are shown only to reveal the layout and spaces.

```
N : Integer;
...
Get (N);
-- Characters at input  Sequence input  Value of N
-- bb-12535b          -12535          -12535
-- bb12 535e1b        12 535e1        125350
-- bb12_535e;         12_535e        (none) Data_Error raised
```

Example of overridden width parameter:

```
Put (Item => -23, Width => 2); -- "-23"
```

A.10.7 Input-Output of Characters and Strings

Static Semantics

1 For an item of type Character the following procedures are provided:

2 **procedure** Get (File : **in** File_Type; Item : **out** Character);
 procedure Get (Item : **out** Character);

3 After skipping any line terminators and any page terminators, reads the next character from the specified input file and returns the value of this character in the out parameter Item.

4 The exception End_Error is propagated if an attempt is made to skip a file terminator.

5 **procedure** Put (File : **in** File_Type; Item : **in** Character);
 procedure Put (Item : **in** Character);

6 If the line length of the specified output file is bounded (that is, does not have the conventional value zero), and the current column number exceeds it, has the effect of calling New_Line with a spacing of one. Then, or otherwise, outputs the given character to the file.

7 **procedure** Look_Ahead (File : **in** File_Type;
 Item : **out** Character;
 End_Of_Line : **out** Boolean);
 procedure Look_Ahead (Item : **out** Character;
 End_Of_Line : **out** Boolean);

8/3 Status_Error is propagated if the file is not open. Mode_Error is propagated if the mode of the file is not In_File. Sets End_Of_Line to True if at end of line, including if at end of page or at end of file; in each of these cases the value of Item is not specified. Otherwise End_Of_Line is set to False and Item is set to the next character (without consuming it) from the file.

9 **procedure** Get_Immediate (File : **in** File_Type;
 Item : **out** Character);
 procedure Get_Immediate (Item : **out** Character);

10/3 Reads the next character, either control or graphic, from the specified File or the default input file. Status_Error is propagated if the file is not open. Mode_Error is propagated if the mode of the file is not In_File. End_Error is propagated if at the end of the file. The current column, line and page numbers for the file are not affected.

11 **procedure** Get_Immediate (File : **in** File_Type;
 Item : **out** Character;
 Available : **out** Boolean);
 procedure Get_Immediate (Item : **out** Character;
 Available : **out** Boolean);

12/3 If a character, either control or graphic, is available from the specified File or the default input file, then the character is read; Available is True and Item contains the value of this character. If a character is not available, then Available is False and the value of Item is not specified. Status_Error is propagated if the file is not open. Mode_Error is propagated if the mode of the file is not In_File. End_Error is propagated if at the end of the file. The current column, line and page numbers for the file are not affected.

13/2 For an item of type String the following subprograms are provided:

- 26 29 A string read by Get or written by Put can extend over several lines. An implementation is allowed to assume that certain external files do not contain page terminators, in which case Get_Line and Skip_Line can return as soon as a line terminator is read.

A.10.8 Input-Output for Integer Types

Static Semantics

- 1 The following procedures are defined in the generic packages Integer_IO and Modular_IO, which have to be instantiated for the appropriate signed integer or modular type respectively (indicated by Num in the specifications).

- 2 Values are output as decimal or based literals, without low line characters or exponent, and, for Integer_IO, preceded by a minus sign if negative. The format (which includes any leading spaces and minus sign) can be specified by an optional field width parameter. Values of widths of fields in output formats are of the nonnegative integer subtype Field. Values of bases are of the integer subtype Number_Base.

3 **subtype** Number_Base **is** Integer **range** 2 .. 16;

- 4 The default field width and base to be used by output procedures are defined by the following variables that are declared in the generic packages Integer_IO and Modular_IO:

5 Default_Width : Field := Num'Width;
Default_Base : Number_Base := 10;

- 6 The following procedures are provided:

7 **procedure** Get(File : **in** File_Type; Item : **out** Num; Width : **in** Field := 0);
procedure Get(Item : **out** Num; Width : **in** Field := 0);

- 8 If the value of the parameter Width is zero, skips any leading blanks, line terminators, or page terminators, then reads a plus sign if present or (for a signed type only) a minus sign if present, then reads the longest possible sequence of characters matching the syntax of a numeric literal without a point. If a nonzero value of Width is supplied, then exactly Width characters are input, or the characters (possibly none) up to a line terminator, whichever comes first; any skipped leading blanks are included in the count.

- 9 Returns, in the parameter Item, the value of type Num that corresponds to the sequence input.

- 10/3 The exception Data_Error is propagated if the sequence of characters read does not form a legal integer literal or if the value obtained is not of the subtype Num.

11 **procedure** Put(File : **in** File_Type;
 Item : **in** Num;
 Width : **in** Field := Default_Width;
 Base : **in** Number_Base := Default_Base);

procedure Put(Item : **in** Num;
 Width : **in** Field := Default_Width;
 Base : **in** Number_Base := Default_Base);

- 12 Outputs the value of the parameter Item as an integer literal, with no low lines, no exponent, and no leading zeros (but a single zero for the value zero), and a preceding minus sign for a negative value.

- 13 If the resulting sequence of characters to be output has fewer than Width characters, then leading spaces are first output to make up the difference.

Uses the syntax for decimal literal if the parameter Base has the value ten (either explicitly or through Default_Base); otherwise, uses the syntax for based literal, with any letters in upper case. 14

```
procedure Get (From : in String; Item : out Num; Last : out Positive); 15
```

Reads an integer value from the beginning of the given string, following the same rules as the Get procedure that reads an integer value from a file, but treating the end of the string as a file terminator. Returns, in the parameter Item, the value of type Num that corresponds to the sequence input. Returns in Last the index value such that From(Last) is the last character read. 16

The exception Data_Error is propagated if the sequence input does not have the required syntax or if the value obtained is not of the subtype Num. 17

```
procedure Put (To      : out String; 18
               Item    : in Num;
               Base     : in Number_Base := Default_Base); 19
```

Outputs the value of the parameter Item to the given string, following the same rule as for output to a file, using the length of the given string as the value for Width. 19

Integer_Text_IO is a library package that is a nongeneric equivalent to Text_IO.Integer_IO for the predefined type Integer: 20

```
with Ada.Text_IO; 21
package Ada.Integer_Text_IO is new Ada.Text_IO.Integer_IO(Integer); 21
```

For each predefined signed integer type, a nongeneric equivalent to Text_IO.Integer_IO is provided, with names such as Ada.Long_Integer_Text_IO. 22

Implementation Permissions

The nongeneric equivalent packages may, but need not, be actual instantiations of the generic package for the appropriate predefined type. 23

NOTES

This paragraph was deleted. 24/3

Examples

This paragraph was deleted. 25/1

```
package Int_IO is new Integer_IO(Small_Int); use Int_IO; 26
-- default format used at instantiation,
-- Default_Width = 4, Default_Base = 10

Put (126); -- "b126" 27
Put (-126, 7); -- "bbb-126"
Put (126, Width => 13, Base => 2); -- "bbb2#1111110#"
```

A.10.9 Input-Output for Real Types

Static Semantics

The following procedures are defined in the generic packages Float_IO, Fixed_IO, and Decimal_IO, which have to be instantiated for the appropriate floating point, ordinary fixed point, or decimal fixed point type respectively (indicated by Num in the specifications). 1

Values are output as decimal literals without low line characters. The format of each value output consists of a Fore field, a decimal point, an Aft field, and (if a nonzero Exp parameter is supplied) the letter E and an Exp field. The two possible formats thus correspond to: 2

Fore . Aft

and to:

Fore . Aft E Exp

without any spaces between these fields. The Fore field may include leading spaces, and a minus sign for negative values. The Aft field includes only decimal digits (possibly with trailing zeros). The Exp field includes the sign (plus or minus) and the exponent (possibly with leading zeros).

For floating point types, the default lengths of these fields are defined by the following variables that are declared in the generic package Float_IO:

```
Default_Fore : Field := 2;
Default_Aft  : Field := Num'Digits-1;
Default_Exp  : Field := 3;
```

For ordinary or decimal fixed point types, the default lengths of these fields are defined by the following variables that are declared in the generic packages Fixed_IO and Decimal_IO, respectively:

```
Default_Fore : Field := Num'Fore;
Default_Aft  : Field := Num'Aft;
Default_Exp  : Field := 0;
```

The following procedures are provided:

```
procedure Get(File : in File_Type; Item : out Num; Width : in Field := 0);
procedure Get(Item : out Num; Width : in Field := 0);
```

If the value of the parameter Width is zero, skips any leading blanks, line terminators, or page terminators, then reads the longest possible sequence of characters matching the syntax of any of the following (see 2.4):

- [+|-]numeric_literal
- [+|-]numeral.[exponent]
- [+|-].numeral[exponent]
- [+|-]base#based_numeral#[exponent]
- [+|-]base#.based_numeral#[exponent]

If a nonzero value of Width is supplied, then exactly Width characters are input, or the characters (possibly none) up to a line terminator, whichever comes first; any skipped leading blanks are included in the count.

Returns in the parameter Item the value of type Num that corresponds to the sequence input, preserving the sign (positive if none has been specified) of a zero value if Num is a floating point type and Num'Signed_Zeros is True.

The exception Data_Error is propagated if the sequence input does not have the required syntax or if the value obtained is not of the subtype Num.

```

procedure Put (File : in File_Type;                                22
               Item : in Num;
               Fore : in Field := Default_Fore;
               Aft  : in Field := Default_Aft;
               Exp  : in Field := Default_Exp);

```

```

procedure Put (Item : in Num;
               Fore : in Field := Default_Fore;
               Aft  : in Field := Default_Aft;
               Exp  : in Field := Default_Exp);

```

Outputs the value of the parameter Item as a decimal literal with the format defined by Fore, Aft and Exp. If the value is negative, or if Num is a floating point type where Num'Signed_Zeros is True and the value is a negatively signed zero, then a minus sign is included in the integer part. If Exp has the value zero, then the integer part to be output has as many digits as are needed to represent the integer part of the value of Item, overriding Fore if necessary, or consists of the digit zero if the value of Item has no integer part. 23

If Exp has a value greater than zero, then the integer part to be output has a single digit, which is nonzero except for the value 0.0 of Item. 24

In both cases, however, if the integer part to be output has fewer than Fore characters, including any minus sign, then leading spaces are first output to make up the difference. The number of digits of the fractional part is given by Aft, or is one if Aft equals zero. The value is rounded; a value of exactly one half in the last place is rounded away from zero. 25

If Exp has the value zero, there is no exponent part. If Exp has a value greater than zero, then the exponent part to be output has as many digits as are needed to represent the exponent part of the value of Item (for which a single digit integer part is used), and includes an initial sign (plus or minus). If the exponent part to be output has fewer than Exp characters, including the sign, then leading zeros precede the digits, to make up the difference. For the value 0.0 of Item, the exponent has the value zero. 26

```

procedure Get (From : in String; Item : out Num; Last : out Positive); 27

```

Reads a real value from the beginning of the given string, following the same rule as the Get procedure that reads a real value from a file, but treating the end of the string as a file terminator. Returns, in the parameter Item, the value of type Num that corresponds to the sequence input. Returns in Last the index value such that From(Last) is the last character read. 28

The exception Data_Error is propagated if the sequence input does not have the required syntax, or if the value obtained is not of the subtype Num. 29

```

procedure Put (To      : out String;                                30
               Item    : in Num;
               Aft     : in Field := Default_Aft;
               Exp     : in Field := Default_Exp);

```

Outputs the value of the parameter Item to the given string, following the same rule as for output to a file, using a value for Fore such that the sequence of characters output exactly fills the string, including any leading spaces. 31

Float_Text_IO is a library package that is a nongeneric equivalent to Text_IO.Float_IO for the predefined type Float. 32

```

with Ada.Text_IO;
package Ada.Float_Text_IO is new Ada.Text_IO.Float_IO (Float); 33

```

- 34 For each predefined floating point type, a nongeneric equivalent to Text_IO.Float_IO is provided, with names such as Ada.Long_Float_Text_IO.

Implementation Permissions

- 35 An implementation may extend Get and Put for floating point types to support special values such as infinities and NaNs.
- 36 The implementation of Put need not produce an output value with greater accuracy than is supported for the base subtype. The additional accuracy, if any, of the value produced by Put when the number of requested digits in the integer and fractional parts exceeds the required accuracy is implementation defined.
- 37 The nongeneric equivalent packages may, but need not, be actual instantiations of the generic package for the appropriate predefined type.

NOTES

- 38 30 For an item with a positive value, if output to a string exactly fills the string without leading spaces, then output of the corresponding negative value will propagate Layout_Error.
- 39 31 The rules for the Value attribute (see 3.5) and the rules for Get are based on the same set of formats.

Examples

40/1 *This paragraph was deleted.*

41 **package** Real_IO **is new** Float_IO(Real); **use** Real_IO;
 -- default format used at instantiation, Default_Exp = 3

42 X : Real := -123.4567; -- digits 8 (see 3.5.7)

43 Put(X); -- default format "--1.2345670E+02"
 Put(X, Fore => 5, Aft => 3, Exp => 2); -- "bbb-1.235E+2"
 Put(X, 5, 3, 0); -- "b-123.457"

A.10.10 Input-Output for Enumeration Types

Static Semantics

- 1 The following procedures are defined in the generic package Enumeration_IO, which has to be instantiated for the appropriate enumeration type (indicated by Enum in the specification).
- 2 Values are output using either upper or lower case letters for identifiers. This is specified by the parameter Set, which is of the enumeration type Type_Set.

3 **type** Type_Set **is** (Lower_Case, Upper_Case);

- 4 The format (which includes any trailing spaces) can be specified by an optional field width parameter. The default field width and letter case are defined by the following variables that are declared in the generic package Enumeration_IO:

5 Default_Width : Field := 0;
 Default_Setting : Type_Set := Upper_Case;

- 6 The following procedures are provided:

7 **procedure** Get(File : **in** File_Type; Item : **out** Enum);
procedure Get(Item : **out** Enum);

- 8 After skipping any leading blanks, line terminators, or page terminators, reads an identifier according to the syntax of this lexical element (lower and upper case being considered equivalent), or a character literal according to the syntax of this lexical element (including the

apostrophes). Returns, in the parameter Item, the value of type Enum that corresponds to the sequence input.

The exception Data_Error is propagated if the sequence input does not have the required syntax, or if the identifier or character literal does not correspond to a value of the subtype Enum.

```

procedure Put (File   : in File_Type;
               Item   : in Enum;
               Width  : in Field := Default_Width;
               Set    : in Type_Set := Default_Setting);
procedure Put (Item   : in Enum;
               Width  : in Field := Default_Width;
               Set    : in Type_Set := Default_Setting);

```

Outputs the value of the parameter Item as an enumeration literal (either an identifier or a character literal). The optional parameter Set indicates whether lower case or upper case is used for identifiers; it has no effect for character literals. If the sequence of characters produced has fewer than Width characters, then trailing spaces are finally output to make up the difference. If Enum is a character type, the sequence of characters produced is as for Enum'Image(Item), as modified by the Width and Set parameters.

```

procedure Get (From : in String; Item : out Enum; Last : out Positive);

```

Reads an enumeration value from the beginning of the given string, following the same rule as the Get procedure that reads an enumeration value from a file, but treating the end of the string as a file terminator. Returns, in the parameter Item, the value of type Enum that corresponds to the sequence input. Returns in Last the index value such that From(Last) is the last character read.

The exception Data_Error is propagated if the sequence input does not have the required syntax, or if the identifier or character literal does not correspond to a value of the subtype Enum.

```

procedure Put (To     : out String;
               Item   : in Enum;
               Set    : in Type_Set := Default_Setting);

```

Outputs the value of the parameter Item to the given string, following the same rule as for output to a file, using the length of the given string as the value for Width.

Although the specification of the generic package Enumeration_IO would allow instantiation for an integer type, this is not the intended purpose of this generic package, and the effect of such instantiations is not defined by the language.

NOTES

32 There is a difference between Put defined for characters, and for enumeration values. Thus

```
Ada.Text_IO.Put('A'); -- outputs the character A
```

```

package Char_IO is new Ada.Text_IO.Enumeration_IO(Character);
Char_IO.Put('A'); -- outputs the character 'A', between apostrophes

```

33 The type Boolean is an enumeration type, hence Enumeration_IO can be instantiated for this type.

A.10.11 Input-Output for Bounded Strings

The package Text_IO.Bounded_IO provides input-output in human-readable form for Bounded_Strings.

Static Semantics

2/2 The generic library package Text_IO.Bounded_IO has the following declaration:

```

3/2   with Ada.Strings.Bounded;
      generic
        with package Bounded is
          new Ada.Strings.Bounded.Generic_Bounded_Length (<>);
      package Ada.Text_IO.Bounded_IO is
4/2   procedure Put
        (File : in File_Type;
         Item : in Bounded.Bounded_String);
5/2   procedure Put
        (Item : in Bounded.Bounded_String);
6/2   procedure Put_Line
        (File : in File_Type;
         Item : in Bounded.Bounded_String);
7/2   procedure Put_Line
        (Item : in Bounded.Bounded_String);
8/2   function Get_Line
        (File : in File_Type)
        return Bounded.Bounded_String;
9/2   function Get_Line
        return Bounded.Bounded_String;
10/2  procedure Get_Line
        (File : in File_Type; Item : out Bounded.Bounded_String);
11/2  procedure Get_Line
        (Item : out Bounded.Bounded_String);
12/2  end Ada.Text_IO.Bounded_IO;
```

13/2 For an item of type Bounded_String, the following subprograms are provided:

```

14/2  procedure Put
        (File : in File_Type;
         Item : in Bounded.Bounded_String);
15/2      Equivalent to Text_IO.Put (File, Bounded.To_String(Item));
16/2  procedure Put
        (Item : in Bounded.Bounded_String);
17/2      Equivalent to Text_IO.Put (Bounded.To_String(Item));
18/2  procedure Put_Line
        (File : in File_Type;
         Item : in Bounded.Bounded_String);
19/2      Equivalent to Text_IO.Put_Line (File, Bounded.To_String(Item));
20/2  procedure Put_Line
        (Item : in Bounded.Bounded_String);
21/2      Equivalent to Text_IO.Put_Line (Bounded.To_String(Item));
22/2  function Get_Line
        (File : in File_Type)
        return Bounded.Bounded_String;
23/2      Returns Bounded.To_Bounded_String(Text_IO.Get_Line(File));
24/2  function Get_Line
        return Bounded.Bounded_String;
25/2      Returns Bounded.To_Bounded_String(Text_IO.Get_Line);
```

procedure Get_Line (File : in File_Type; Item : out Bounded.Bounded_String);	26/2
Equivalent to Item := Get_Line (File);	27/2
procedure Get_Line (Item : out Bounded.Bounded_String);	28/2
Equivalent to Item := Get_Line;	29/2

A.10.12 Input-Output for Unbounded Strings

The package Text_IO.Unbounded_IO provides input-output in human-readable form for Unbounded_Strings. 1/2

Static Semantics

The library package Text_IO.Unbounded_IO has the following declaration: 2/2

with Ada.Strings.Unbounded;	3/2
package Ada.Text_IO.Unbounded_IO is	
procedure Put (File : in File_Type; Item : in Strings.Unbounded.Unbounded_String);	4/2
procedure Put (Item : in Strings.Unbounded.Unbounded_String);	5/2
procedure Put_Line (File : in File_Type; Item : in Strings.Unbounded.Unbounded_String);	6/2
procedure Put_Line (Item : in Strings.Unbounded.Unbounded_String);	7/2
function Get_Line (File : in File_Type) return Strings.Unbounded.Unbounded_String;	8/2
function Get_Line return Strings.Unbounded.Unbounded_String;	9/2
procedure Get_Line (File : in File_Type; Item : out Strings.Unbounded.Unbounded_String);	10/2
procedure Get_Line (Item : out Strings.Unbounded.Unbounded_String);	11/2
end Ada.Text_IO.Unbounded_IO;	12/2

For an item of type Unbounded_String, the following subprograms are provided: 13/2

procedure Put (File : in File_Type; Item : in Strings.Unbounded.Unbounded_String);	14/2
Equivalent to Text_IO.Put (File, Strings.Unbounded.To_String(Item));	15/2
procedure Put (Item : in Strings.Unbounded.Unbounded_String);	16/2
Equivalent to Text_IO.Put (Strings.Unbounded.To_String(Item));	17/2
procedure Put_Line (File : in File_Type; Item : in Strings.Unbounded.Unbounded_String);	18/2
Equivalent to Text_IO.Put_Line (File, Strings.Unbounded.To_String(Item));	19/2

```

20/2      procedure Put_Line
          (Item : in Strings.Unbounded.Unbounded_String);
21/2      Equivalent to Text_IO.Put_Line (Strings.Unbounded.To_String(Item));
22/2      function Get_Line
          (File : in File_Type)
          return Strings.Unbounded.Unbounded_String;
23/2      Returns Strings.Unbounded.To_Unbounded_String(Text_IO.Get_Line(File));
24/2      function Get_Line
          return Strings.Unbounded.Unbounded_String;
25/2      Returns Strings.Unbounded.To_Unbounded_String(Text_IO.Get_Line);
26/2      procedure Get_Line
          (File : in File_Type; Item : out Strings.Unbounded.Unbounded_String);
27/2      Equivalent to Item := Get_Line (File);
28/2      procedure Get_Line
          (Item : out Strings.Unbounded.Unbounded_String);
29/2      Equivalent to Item := Get_Line;

```

A.11 Wide Text Input-Output and Wide Wide Text Input-Output

- 1/2 The packages `Wide_Text_IO` and `Wide_Wide_Text_IO` provide facilities for input and output in human-readable form. Each file is read or written sequentially, as a sequence of wide characters (or wide wide characters) grouped into lines, and as a sequence of lines grouped into pages.

Static Semantics

- 2/2 The specification of package `Wide_Text_IO` is the same as that for `Text_IO`, except that in each `Get`, `Look_Ahead`, `Get_Immediate`, `Get_Line`, `Put`, and `Put_Line` subprogram, any occurrence of `Character` is replaced by `Wide_Character`, and any occurrence of `String` is replaced by `Wide_String`. Nongeneric equivalents of `Wide_Text_IO.Integer_IO` and `Wide_Text_IO.Float_IO` are provided (as for `Text_IO`) for each predefined numeric type, with names such as `Ada.Integer_Wide_Text_IO`, `Ada.Long_Integer_Wide_Text_IO`, `Ada.Float_Wide_Text_IO`, `Ada.Long_Float_Wide_Text_IO`.
- 3/2 The specification of package `Wide_Wide_Text_IO` is the same as that for `Text_IO`, except that in each `Get`, `Look_Ahead`, `Get_Immediate`, `Get_Line`, `Put`, and `Put_Line` subprogram, any occurrence of `Character` is replaced by `Wide_Wide_Character`, and any occurrence of `String` is replaced by `Wide_Wide_String`. Nongeneric equivalents of `Wide_Wide_Text_IO.Integer_IO` and `Wide_Wide_Text_IO.Float_IO` are provided (as for `Text_IO`) for each predefined numeric type, with names such as `Ada.Integer_Wide_Wide_Text_IO`, `Ada.Long_Integer_Wide_Wide_Text_IO`, `Ada.Float_Wide_Wide_Text_IO`, `Ada.Long_Float_Wide_Wide_Text_IO`.
- 4/3 The specification of package `Wide_Text_IO.Wide_Bounded_IO` is the same as that for `Text_IO.Bounded_IO`, except that any occurrence of `Bounded_String` is replaced by `Bounded_Wide_String`, and any occurrence of package `Bounded` is replaced by `Wide_Bounded`. The specification of package `Wide_Wide_Text_IO.Wide_Wide_Bounded_IO` is the same as that for `Text_IO.Bounded_IO`, except that any occurrence of `Bounded_String` is replaced by `Bounded_Wide_Wide_String`, and any occurrence of package `Bounded` is replaced by `Wide_Wide_Bounded`.
- 5/3 The specification of package `Wide_Text_IO.Wide_Unbounded_IO` is the same as that for `Text_IO.Unbounded_IO`, except that any occurrence of `Unbounded_String` is replaced by `Unbounded_Wide_String`, and any occurrence of package `Unbounded` is replaced by `Wide_Unbounded`. The specification of

package Wide_Wide_Text_IO.Wide_Wide_Unbounded_IO is the same as that for Text_IO.Unbounded_IO, except that any occurrence of Unbounded_String is replaced by Unbounded_Wide_Wide_String, and any occurrence of package Unbounded is replaced by Wide_Wide_Unbounded.

A.12 Stream Input-Output

The packages Streams.Stream_IO, Text_IO.Text_Streams, Wide_Text_IO.Text_Streams, and Wide_Wide_Text_IO.Text_Streams provide stream-oriented operations on files. 1/2

A.12.1 The Package Streams.Stream_IO

The subprograms in the child package Streams.Stream_IO provide control over stream files. Access to a stream file is either sequential, via a call on Read or Write to transfer an array of stream elements, or positional (if supported by the implementation for the given file), by specifying a relative index for an element. Since a stream file can be converted to a Stream_Access value, calling stream-oriented attribute subprograms of different element types with the same Stream_Access value provides heterogeneous input-output. See 13.13 for a general discussion of streams. 1

Static Semantics

The elements of a stream file are stream elements. If positioning is supported for the specified external file, a current index and current size are maintained for the file as described in A.8. If positioning is not supported, a current index is not maintained, and the current size is implementation defined. 1.1/1

The library package Streams.Stream_IO has the following declaration: 2

```

with Ada.IO_Exceptions;
package Ada.Streams.Stream_IO is
    type Stream_Access is access all Root_Stream_Type'Class;
    type File_Type is limited private;
    type File_Mode is (In_File, Out_File, Append_File);
    type Count is range 0 .. implementation-defined;
    subtype Positive_Count is Count range 1 .. Count'Last;
    -- Index into file, in stream elements.
    procedure Create (File : in out File_Type;
                     Mode : in File_Mode := Out_File;
                     Name : in String := "";
                     Form : in String := "");
    procedure Open (File : in out File_Type;
                   Mode : in File_Mode;
                   Name : in String;
                   Form : in String := "");
    procedure Close (File : in out File_Type);
    procedure Delete (File : in out File_Type);
    procedure Reset (File : in out File_Type; Mode : in File_Mode);
    procedure Reset (File : in out File_Type);
    function Mode (File : in File_Type) return File_Mode;
    function Name (File : in File_Type) return String;
    function Form (File : in File_Type) return String;
    function Is_Open (File : in File_Type) return Boolean;
    function End_Of_File (File : in File_Type) return Boolean;
    function Stream (File : in File_Type) return Stream_Access;
    -- Return stream access for use with T'Input and T'Output

```

This paragraph was deleted.

```

15      -- Read array of stream elements from file
      procedure Read (File : in File_Type;
                     Item : out Stream_Element_Array;
                     Last : out Stream_Element_Offset;
                     From : in Positive_Count);

16      procedure Read (File : in File_Type;
                     Item : out Stream_Element_Array;
                     Last : out Stream_Element_Offset);

17/1  This paragraph was deleted.

18      -- Write array of stream elements into file
      procedure Write (File : in File_Type;
                     Item : in Stream_Element_Array;
                     To   : in Positive_Count);

19      procedure Write (File : in File_Type;
                     Item : in Stream_Element_Array);

20/1  This paragraph was deleted.

21      -- Operations on position within file
22      procedure Set_Index(File : in File_Type; To : in Positive_Count);
23      function Index(File : in File_Type) return Positive_Count;
24      function Size (File : in File_Type) return Count;
25/1  procedure Set_Mode(File : in out File_Type; Mode : in File_Mode);
26      procedure Flush(File : in File_Type);

      -- exceptions
      Status_Error : exception renames IO_Exceptions.Status_Error;
      Mode_Error   : exception renames IO_Exceptions.Mode_Error;
      Name_Error   : exception renames IO_Exceptions.Name_Error;
      Use_Error    : exception renames IO_Exceptions.Use_Error;
      Device_Error : exception renames IO_Exceptions.Device_Error;
      End_Error    : exception renames IO_Exceptions.End_Error;
      Data_Error   : exception renames IO_Exceptions.Data_Error;

27      private
      ... -- not specified by the language
      end Ada.Streams.Stream_IO;

```

27.1/2 The type `File_Type` needs finalization (see 7.6).

28/2 The subprograms given in subclause A.8.2 for the control of external files (`Create`, `Open`, `Close`, `Delete`, `Reset`, `Mode`, `Name`, `Form`, and `Is_Open`) are available for stream files.

28.1/2 The `End_Of_File` function:

- 28.2/2 • Propagates `Mode_Error` if the mode of the file is not `In_File`;
- 28.3/2 • If positioning is supported for the given external file, the function returns `True` if the current index exceeds the size of the external file; otherwise it returns `False`;
- 28.4/2 • If positioning is not supported for the given external file, the function returns `True` if no more elements can be read from the given file; otherwise it returns `False`.

28.5/2 The `Set_Mode` procedure sets the mode of the file. If the new mode is `Append_File`, the file is positioned to its end; otherwise, the position in the file is unchanged.

28.6/1 The `Flush` procedure synchronizes the external file with the internal file (by flushing any internal buffers) without closing the file or changing the position. `Mode_Error` is propagated if the mode of the file is `In_File`.

The Stream function returns a Stream_Access result from a File_Type object, thus allowing the stream-oriented attributes Read, Write, Input, and Output to be used on the same file for multiple types. Stream propagates Status_Error if File is not open. 29/1

The procedures Read and Write are equivalent to the corresponding operations in the package Streams. Read propagates Mode_Error if the mode of File is not In_File. Write propagates Mode_Error if the mode of File is not Out_File or Append_File. The Read procedure with a Positive_Count parameter starts reading at the specified index. The Write procedure with a Positive_Count parameter starts writing at the specified index. For a file that supports positioning, Read without a Positive_Count parameter starts reading at the current index, and Write without a Positive_Count parameter starts writing at the current index. 30/2

The Size function returns the current size of the file. 30.1/1

The Index function returns the current index. 31/1

The Set_Index procedure sets the current index to the specified value. 32

If positioning is supported for the external file, the current index is maintained as follows: 32.1/1

- For Open and Create, if the Mode parameter is Append_File, the current index is set to the current size of the file plus one; otherwise, the current index is set to one. 32.2/1
- For Reset, if the Mode parameter is Append_File, or no Mode parameter is given and the current mode is Append_File, the current index is set to the current size of the file plus one; otherwise, the current index is set to one. 32.3/1
- For Set_Mode, if the new mode is Append_File, the current index is set to current size plus one; otherwise, the current index is unchanged. 32.4/1
- For Read and Write without a Positive_Count parameter, the current index is incremented by the number of stream elements read or written. 32.5/1
- For Read and Write with a Positive_Count parameter, the value of the current index is set to the value of the Positive_Count parameter plus the number of stream elements read or written. 32.6/1

If positioning is not supported for the given file, then a call of Index or Set_Index propagates Use_Error. Similarly, a call of Read or Write with a Positive_Count parameter propagates Use_Error. 33

Paragraphs 34 through 36 were deleted.

Erroneous Execution

If the File_Type object passed to the Stream function is later closed or finalized, and the stream-oriented attributes are subsequently called (explicitly or implicitly) on the Stream_Access value returned by Stream, execution is erroneous. This rule applies even if the File_Type object was opened again after it had been closed. 36.1/1

A.12.2 The Package Text_IO.Text_Streams

The package Text_IO.Text_Streams provides a function for treating a text file as a stream. 1

Static Semantics

The library package Text_IO.Text_Streams has the following declaration: 2

```
with Ada.Streams;  
package Ada.Text_IO.Text_Streams is  
    type Stream_Access is access all Streams.Root_Stream_Type'Class;  
3
```

```

4      function Stream (File : in File_Type) return Stream_Access;
      end Ada.Text_IO.Text_Streams;

```

5 The Stream function has the same effect as the corresponding function in Streams.Stream_IO.

NOTES

6 34 The ability to obtain a stream for a text file allows Current_Input, Current_Output, and Current_Error to be processed with the functionality of streams, including the mixing of text and binary input-output, and the mixing of binary input-output for different types.

7 35 Performing operations on the stream associated with a text file does not affect the column, line, or page counts.

A.12.3 The Package Wide_Text_IO.Text_Streams

1 The package Wide_Text_IO.Text_Streams provides a function for treating a wide text file as a stream.

Static Semantics

2 The library package Wide_Text_IO.Text_Streams has the following declaration:

```

3      with Ada.Streams;
      package Ada.Wide_Text_IO.Text_Streams is
        type Stream_Access is access all Streams.Root_Stream_Type'Class;
4      function Stream (File : in File_Type) return Stream_Access;
      end Ada.Wide_Text_IO.Text_Streams;

```

5 The Stream function has the same effect as the corresponding function in Streams.Stream_IO.

A.12.4 The Package Wide_Wide_Text_IO.Text_Streams

1/2 The package Wide_Wide_Text_IO.Text_Streams provides a function for treating a wide wide text file as a stream.

Static Semantics

2/2 The library package Wide_Wide_Text_IO.Text_Streams has the following declaration:

```

3/2      with Ada.Streams;
      package Ada.Wide_Wide_Text_IO.Text_Streams is
        type Stream_Access is access all Streams.Root_Stream_Type'Class;
4/2      function Stream (File : in File_Type) return Stream_Access;
      end Ada.Wide_Wide_Text_IO.Text_Streams;

```

5/2 The Stream function has the same effect as the corresponding function in Streams.Stream_IO.

A.13 Exceptions in Input-Output

1 The package IO_Exceptions defines the exceptions needed by the predefined input-output packages.

Static Semantics

2 The library package IO_Exceptions has the following declaration:

```

3      package Ada.IO_Exceptions is
        pragma Pure(IO_Exceptions);

```

```

Status_Error : exception;
Mode_Error   : exception;
Name_Error   : exception;
Use_Error    : exception;
Device_Error : exception;
End_Error    : exception;
Data_Error   : exception;
Layout_Error : exception;

```

```

end Ada.IO_Exceptions;

```

If more than one error condition exists, the corresponding exception that appears earliest in the following list is the one that is propagated.

The exception `Status_Error` is propagated by an attempt to operate upon a file that is not open, and by an attempt to open a file that is already open.

The exception `Mode_Error` is propagated by an attempt to read from, or test for the end of, a file whose current mode is `Out_File` or `Append_File`, and also by an attempt to write to a file whose current mode is `In_File`. In the case of `Text_IO`, the exception `Mode_Error` is also propagated by specifying a file whose current mode is `Out_File` or `Append_File` in a call of `Set_Input`, `Skip_Line`, `End_Of_Line`, `Skip_Page`, or `End_Of_Page`; and by specifying a file whose current mode is `In_File` in a call of `Set_Output`, `Set_Line_Length`, `Set_Page_Length`, `Line_Length`, `Page_Length`, `New_Line`, or `New_Page`.

The exception `Name_Error` is propagated by a call of `Create` or `Open` if the string given for the parameter `Name` does not allow the identification of an external file. For example, this exception is propagated if the string is improper, or, alternatively, if either none or more than one external file corresponds to the string.

The exception `Use_Error` is propagated if an operation is attempted that is not possible for reasons that depend on characteristics of the external file. For example, this exception is propagated by the procedure `Create`, among other circumstances, if the given mode is `Out_File` but the form specifies an input only device, if the parameter `Form` specifies invalid access rights, or if an external file with the given name already exists and overwriting is not allowed.

The exception `Device_Error` is propagated if an input-output operation cannot be completed because of a malfunction of the underlying system.

The exception `End_Error` is propagated by an attempt to skip (read past) the end of a file.

The exception `Data_Error` can be propagated by the procedure `Read` (or by the `Read` attribute) if the element read cannot be interpreted as a value of the required subtype. This exception is also propagated by a procedure `Get` (defined in the package `Text_IO`) if the input character sequence fails to satisfy the required syntax, or if the value input does not belong to the range of the required subtype.

The exception `Layout_Error` is propagated (in text input-output) by `Col`, `Line`, or `Page` if the value returned exceeds `Count'Last`. The exception `Layout_Error` is also propagated on output by an attempt to set column or line numbers in excess of specified maximum line or page lengths, respectively (excluding the unbounded cases). It is also propagated by an attempt to `Put` too many characters to a string.

Documentation Requirements

The implementation shall document the conditions under which `Name_Error`, `Use_Error` and `Device_Error` are propagated.

Implementation Permissions

- 16 If the associated check is too complex, an implementation need not propagate `Data_Error` as part of a procedure `Read` (or the `Read` attribute) if the value read cannot be interpreted as a value of the required subtype.

Erroneous Execution

- 17 If the element read by the procedure `Read` (or by the `Read` attribute) cannot be interpreted as a value of the required subtype, but this is not detected and `Data_Error` is not propagated, then the resulting value can be abnormal, and subsequent references to the value can lead to erroneous execution, as explained in 13.9.1.

A.14 File Sharing

Dynamic Semantics

- 1 It is not specified by the language whether the same external file can be associated with more than one file object. If such sharing is supported by the implementation, the following effects are defined:
- 2 • Operations on one text file object do not affect the column, line, and page numbers of any other file object.
 - 3/1 • *This paragraph was deleted.*
 - 4 • For direct and stream files, the current index is a property of each file object; an operation on one file object does not affect the current index of any other file object.
 - 5 • For direct and stream files, the current size of the file is a property of the external file.
 - 6 All other effects are identical.

A.15 The Package `Command_Line`

- 1 The package `Command_Line` allows a program to obtain the values of its arguments and to set the exit status code to be returned on normal termination.

Static Semantics

- 2 The library package `Ada.Command_Line` has the following declaration:

```

3  package Ada.Command_Line is
4      pragma Preelaborate(Command_Line);
5      function Argument_Count return Natural;
6      function Argument (Number : in Positive) return String;
7      function Command_Name return String;
8      type Exit_Status is implementation-defined integer type;
9      Success : constant Exit_Status;
10     Failure : constant Exit_Status;
11     procedure Set_Exit_Status (Code : in Exit_Status);
12 private
13     ... -- not specified by the language
14 end Ada.Command_Line;
```

function Argument_Count return Natural;	11
If the external execution environment supports passing arguments to a program, then Argument_Count returns the number of arguments passed to the program invoking the function. Otherwise it returns 0. The meaning of “number of arguments” is implementation defined.	12
function Argument (Number : in Positive) return String;	13
If the external execution environment supports passing arguments to a program, then Argument returns an implementation-defined value corresponding to the argument at relative position Number. If Number is outside the range 1..Argument_Count, then Constraint_Error is propagated.	14
function Command_Name return String;	15
If the external execution environment supports passing arguments to a program, then Command_Name returns an implementation-defined value corresponding to the name of the command invoking the program; otherwise Command_Name returns the null string.	16
type Exit_Status is <i>implementation-defined integer type</i> ;	16.1/1
The type Exit_Status represents the range of exit status values supported by the external execution environment. The constants Success and Failure correspond to success and failure, respectively.	17
procedure Set_Exit_Status (Code : in Exit_Status);	18
If the external execution environment supports returning an exit status from a program, then Set_Exit_Status sets Code as the status. Normal termination of a program returns as the exit status the value most recently set by Set_Exit_Status, or, if no such value has been set, then the value Success. If a program terminates abnormally, the status set by Set_Exit_Status is ignored, and an implementation-defined exit status value is set.	19
If the external execution environment does not support returning an exit value from a program, then Set_Exit_Status does nothing.	20

Implementation Permissions

An alternative declaration is allowed for package Command_Line if different functionality is appropriate for the external execution environment. 21

NOTES

36 Argument_Count, Argument, and Command_Name correspond to the C language's argc, argv[n] (for n>0) and argv[0], respectively. 22

A.16 The Package Directories

1/2 The package Directories provides operations for manipulating files and directories, and their names.

Static Semantics

2/2 The library package Directories has the following declaration:

```

3/2  with Ada.IO_Exceptions;
    with Ada.Calendar;
    package Ada.Directories is
4/2      -- Directory and file operations:
5/2      function Current_Directory return String;
6/2      procedure Set_Directory (Directory : in String);
7/2      procedure Create_Directory (New_Directory : in String;
                                Form           : in String := "");
8/2      procedure Delete_Directory (Directory : in String);
9/2      procedure Create_Path (New_Directory : in String;
                             Form           : in String := "");
10/2     procedure Delete_Tree (Directory : in String);
11/2     procedure Delete_File (Name : in String);
12/2     procedure Rename (Old_Name, New_Name : in String);
13/2     procedure Copy_File (Source_Name,
                           Target_Name : in String;
                           Form        : in String := "");
14/2     -- File and directory name operations:
15/2     function Full_Name (Name : in String) return String;
16/2     function Simple_Name (Name : in String) return String;
17/2     function Containing_Directory (Name : in String) return String;
18/2     function Extension (Name : in String) return String;
19/2     function Base_Name (Name : in String) return String;
20/2     function Compose (Containing_Directory : in String := "";
                       Name                  : in String;
                       Extension             : in String := "") return String;
20.1/3    type Name_Case_Kind is (Unknown, Case_Sensitive, Case_Insensitive,
                              Case_Preserving);
20.2/3    function Name_Case_Equivalence (Name : in String) return Name_Case_Kind;
21/2     -- File and directory queries:
22/2     type File_Kind is (Directory, Ordinary_File, Special_File);
23/2     type File_Size is range 0 .. implementation-defined;
24/2     function Exists (Name : in String) return Boolean;
25/2     function Kind (Name : in String) return File_Kind;
26/2     function Size (Name : in String) return File_Size;
27/2     function Modification_Time (Name : in String) return Ada.Calendar.Time;
28/2     -- Directory searching:
29/2     type Directory_Entry_Type is limited private;
30/2     type Filter_Type is array (File_Kind) of Boolean;
31/2     type Search_Type is limited private;

```



```

procedure Start_Search (Search      : in out Search_Type;           32/2
                        Directory    : in String;
                        Pattern      : in String;
                        Filter       : in Filter_Type := (others => True));

procedure End_Search (Search : in out Search_Type);                33/2
function More_Entries (Search : in Search_Type) return Boolean;    34/2
procedure Get_Next_Entry (Search : in out Search_Type;             35/2
                        Directory_Entry : out Directory_Entry_Type);

procedure Search (                                              36/2
    Directory : in String;
    Pattern   : in String;
    Filter    : in Filter_Type := (others => True);
    Process   : not null access procedure (
        Directory_Entry : in Directory_Entry_Type));

-- Operations on Directory Entries:                               37/2
function Simple_Name (Directory_Entry : in Directory_Entry_Type) 38/2
    return String;
function Full_Name (Directory_Entry : in Directory_Entry_Type)   39/2
    return String;
function Kind (Directory_Entry : in Directory_Entry_Type)        40/2
    return File_Kind;
function Size (Directory_Entry : in Directory_Entry_Type)        41/2
    return File_Size;
function Modification_Time (Directory_Entry : in Directory_Entry_Type) 42/2
    return Ada.Calendar.Time;

Status_Error : exception renames Ada.IO_Exceptions.Status_Error; 43/2
Name_Error   : exception renames Ada.IO_Exceptions.Name_Error;
Use_Error    : exception renames Ada.IO_Exceptions.Use_Error;
Device_Error : exception renames Ada.IO_Exceptions.Device_Error;

private                                                    44/2
    -- Not specified by the language.
end Ada.Directories;

```

External files may be classified as directories, special files, or ordinary files. A *directory* is an external file that is a container for files on the target system. A *special file* is an external file that cannot be created or read by a predefined Ada input-output package. External files that are not special files or directories are called *ordinary files*. 45/2

A *file name* is a string identifying an external file. Similarly, a *directory name* is a string identifying a directory. The interpretation of file names and directory names is implementation-defined. 46/2

The *full name* of an external file is a full specification of the name of the file. If the external environment allows alternative specifications of the name (for example, abbreviations), the full name should not use such alternatives. A full name typically will include the names of all of the directories that contain the item. The *simple name* of an external file is the name of the item, not including any containing directory names. Unless otherwise specified, a file name or directory name parameter in a call to a predefined Ada input-output subprogram can be a full name, a simple name, or any other form of name supported by the implementation. 47/2

The *default directory* is the directory that is used if a directory or file name is not a full name (that is, when the name does not fully identify all of the containing directories). 48/2

A *directory entry* is a single item in a directory, identifying a single external file (including directories and special files). 49/2

For each function that returns a string, the lower bound of the returned value is 1. 50/2

51/2 The following file and directory operations are provided:

```
52/2      function Current_Directory return String;
```

53/2 Returns the full directory name for the current default directory. The name returned shall be suitable for a future call to Set_Directory. The exception Use_Error is propagated if a default directory is not supported by the external environment.

```
54/2  procedure Set_Directory (Directory : in String);
```

55/2 Sets the current default directory. The exception `Name_Error` is propagated if the string given as `Directory` does not identify an existing directory. The exception `Use_Error` is propagated if the external environment does not support making `Directory` (in the absence of `Name_Error`) a default directory.

[illegible]

Creates a directory with name `New_Directory`. The `Form` parameter can be used to give system-dependent characteristics of the directory; the interpretation of the `Form` parameter is implementation-defined. A null string for `Form` specifies the use of the default options of the implementation of the new directory. The exception `Name_Error` is propagated if the string given as `New_Directory` does not allow the identification of a directory. The exception `Use_Error` is propagated if the external environment does not support the creation of a directory with the given name (in the absence of `Name_Error`) and `form`.

```
58/2      procedure Delete_Directory (Directory : in String);
```

Deletes an existing empty directory with name `Directory`. The exception `Name_Error` is propagated if the string given as `Directory` does not identify an existing directory. The exception `Use_Error` is propagated if the external environment does not support the deletion of the directory (or some portion of its contents) with the given name (in the absence of `Name_Error`).

[illegible]

61/2 Creates zero or more directories with name `New_Directory`. Each non-existent directory named by `New_Directory` is created. For example, on a typical Unix system, `Create_Path("/usr/me/my")`; would create directory "me" in directory "usr", then create directory "my" in directory "me". The `Form` parameter can be used to give system-dependent characteristics of the directory; the interpretation of the `Form` parameter is implementation-defined. A null string for `Form` specifies the use of the default options of the implementation of the new directory. The exception `Name_Error` is propagated if the string given as `New_Directory` does not allow the identification of any directory. The exception `Use_Error` is propagated if the external environment does not support the creation of any directories with the given name (in the absence of `Name_Error`) and form.

```
62/2      procedure Delete Tree (Directory : in String);
```

63/2 Deletes an existing directory with name `Directory`. The directory and all of its contents (possibly including other directories) are deleted. The exception `Name_Error` is propagated if the string given as `Directory` does not identify an existing directory. The exception `Use_Error` is propagated if the external environment does not support the deletion of the directory or some portion of its contents with the given name (in the absence of `Name_Error`). If `Use_Error` is propagated, it is unspecified whether a portion of the contents of the directory is deleted.

procedure Delete_File (Name : in String);	64/2
Deletes an existing ordinary or special file with name Name. The exception Name_Error is propagated if the string given as Name does not identify an existing ordinary or special external file. The exception Use_Error is propagated if the external environment does not support the deletion of the file with the given name (in the absence of Name_Error).	65/2
procedure Rename (Old_Name, New_Name : in String);	66/2
Renames an existing external file (including directories) with name Old_Name to New_Name. The exception Name_Error is propagated if the string given as Old_Name does not identify an existing external file. The exception Use_Error is propagated if the external environment does not support the renaming of the file with the given name (in the absence of Name_Error). In particular, Use_Error is propagated if a file or directory already exists with name New_Name.	67/2
procedure Copy_File (Source_Name, Target_Name : in String; Form : in String := "");	68/3
Copies the contents of the existing external file with name Source_Name to an external file with name Target_Name. The resulting external file is a duplicate of the source external file. The Form parameter can be used to give system-dependent characteristics of the resulting external file; the interpretation of the Form parameter is implementation-defined. Exception Name_Error is propagated if the string given as Source_Name does not identify an existing external ordinary or special file, or if the string given as Target_Name does not allow the identification of an external file. The exception Use_Error is propagated if the external environment does not support creating the file with the name given by Target_Name and form given by Form, or copying of the file with the name given by Source_Name (in the absence of Name_Error).	69/2

The following file and directory name operations are provided: 70/2

function Full_Name (Name : in String) return String;	71/2
Returns the full name corresponding to the file name specified by Name. The exception Name_Error is propagated if the string given as Name does not allow the identification of an external file (including directories and special files).	72/2
function Simple_Name (Name : in String) return String;	73/2
Returns the simple name portion of the file name specified by Name. The exception Name_Error is propagated if the string given as Name does not allow the identification of an external file (including directories and special files).	74/2
function Containing_Directory (Name : in String) return String;	75/2
Returns the name of the containing directory of the external file (including directories) identified by Name. (If more than one directory can contain Name, the directory name returned is implementation-defined.) The exception Name_Error is propagated if the string given as Name does not allow the identification of an external file. The exception Use_Error is propagated if the external file does not have a containing directory.	76/2
function Extension (Name : in String) return String;	77/2
Returns the extension name corresponding to Name. The extension name is a portion of a simple name (not including any separator characters), typically used to identify the file class. If the external environment does not have extension names, then the null string is returned. The	78/2

exception `Name_Error` is propagated if the string given as `Name` does not allow the identification of an external file.

79/2 **function** `Base_Name` (`Name` : **in** `String`) **return** `String`;

80/2 Returns the base name corresponding to `Name`. The base name is the remainder of a simple name after removing any extension and extension separators. The exception `Name_Error` is propagated if the string given as `Name` does not allow the identification of an external file (including directories and special files).

81/2 **function** `Compose` (`Containing_Directory` : **in** `String` := "";
 `Name` : **in** `String`;
 `Extension` : **in** `String` := "") **return** `String`;

82/2 Returns the name of the external file with the specified `Containing_Directory`, `Name`, and `Extension`. If `Extension` is the null string, then `Name` is interpreted as a simple name; otherwise `Name` is interpreted as a base name. The exception `Name_Error` is propagated if the string given as `Containing_Directory` is not null and does not allow the identification of a directory, or if the string given as `Extension` is not null and is not a possible extension, or if the string given as `Name` is not a possible simple name (if `Extension` is null) or base name (if `Extension` is non-null).

82.1/3 **function** `Name_Case_Equivalence` (`Name` : **in** `String`) **return** `Name_Case_Kind`;

82.2/3 Returns the file name equivalence rule for the directory containing `Name`. Raises `Name_Error` if `Name` is not a full name. Returns `Case_Sensitive` if file names that differ only in the case of letters are considered different names. If file names that differ only in the case of letters are considered the same name, then `Case_Preserving` is returned if the name has the case of the file name used when a file is created; and `Case_Insensitive` otherwise. Returns `Unknown` if the file name equivalence is not known.

83/2 The following file and directory queries and types are provided:

84/2 **type** `File_Kind` **is** (`Directory`, `Ordinary_File`, `Special_File`);

85/2 The type `File_Kind` represents the kind of file represented by an external file or directory.

86/2 **type** `File_Size` **is range** 0 .. *implementation-defined*;

87/2 The type `File_Size` represents the size of an external file.

88/2 **function** `Exists` (`Name` : **in** `String`) **return** `Boolean`;

89/2 Returns `True` if an external file represented by `Name` exists, and `False` otherwise. The exception `Name_Error` is propagated if the string given as `Name` does not allow the identification of an external file (including directories and special files).

90/2 **function** `Kind` (`Name` : **in** `String`) **return** `File_Kind`;

91/2 Returns the kind of external file represented by `Name`. The exception `Name_Error` is propagated if the string given as `Name` does not allow the identification of an existing external file.

92/2 **function** `Size` (`Name` : **in** `String`) **return** `File_Size`;

93/2 Returns the size of the external file represented by `Name`. The size of an external file is the number of stream elements contained in the file. If the external file is not an ordinary file, the result is *implementation-defined*. The exception `Name_Error` is propagated if the string given as `Name` does not allow the identification of an existing external file. The exception `Constraint_Error` is propagated if the file size is not a value of type `File_Size`.

function Modification_Time (Name : **in** String) **return** Ada.Calendar.Time; 94/2

Returns the time that the external file represented by Name was most recently modified. If the external file is not an ordinary file, the result is implementation-defined. The exception Name_Error is propagated if the string given as Name does not allow the identification of an existing external file. The exception Use_Error is propagated if the external environment does not support reading the modification time of the file with the name given by Name (in the absence of Name_Error). 95/2

The following directory searching operations and types are provided: 96/2

type Directory_Entry_Type **is limited private**; 97/2

The type Directory_Entry_Type represents a single item in a directory. These items can only be created by the Get_Next_Entry procedure in this package. Information about the item can be obtained from the functions declared in this package. A default-initialized object of this type is invalid; objects returned from Get_Next_Entry are valid. 98/2

type Filter_Type **is array** (File_Kind) **of** Boolean; 99/2

The type Filter_Type specifies which directory entries are provided from a search operation. If the Directory component is True, directory entries representing directories are provided. If the Ordinary_File component is True, directory entries representing ordinary files are provided. If the Special_File component is True, directory entries representing special files are provided. 100/2

type Search_Type **is limited private**; 101/2

The type Search_Type contains the state of a directory search. A default-initialized Search_Type object has no entries available (function More_Entries returns False). Type Search_Type needs finalization (see 7.6). 102/2

procedure Start_Search (Search : **in out** Search_Type;
Directory : **in** String;
Pattern : **in** String;
Filter : **in** Filter_Type := (others => True)); 103/2

Starts a search in the directory named by Directory for entries matching Pattern. Pattern represents a pattern for matching file names. If Pattern is the null string, all items in the directory are matched; otherwise, the interpretation of Pattern is implementation-defined. Only items that match Filter will be returned. After a successful call on Start_Search, the object Search may have entries available, but it may have no entries available if no files or directories match Pattern and Filter. The exception Name_Error is propagated if the string given by Directory does not identify an existing directory, or if Pattern does not allow the identification of any possible external file or directory. The exception Use_Error is propagated if the external environment does not support the searching of the directory with the given name (in the absence of Name_Error). When Start_Search propagates Name_Error or Use_Error, the object Search will have no entries available. 104/3

procedure End_Search (Search : **in out** Search_Type); 105/2

Ends the search represented by Search. After a successful call on End_Search, the object Search will have no entries available. 106/2

function More_Entries (Search : **in** Search_Type) **return** Boolean; 107/2

Returns True if more entries are available to be returned by a call to Get_Next_Entry for the specified search object, and False otherwise. 108/2

```
109/2      procedure Get_Next_Entry (Search : in out Search_Type;  
                                     Directory_Entry : out Directory_Entry_Type);
```

110/2 Returns the next `Directory_Entry` for the search described by `Search` that matches the pattern and filter. If no further matches are available, `Status_Error` is raised. It is implementation-defined as to whether the results returned by this routine are altered if the contents of the directory are altered while the `Search` object is valid (for example, by another program). The exception `Use_Error` is propagated if the external environment does not support continued searching of the directory represented by `Search`.

```

111/2      procedure Search (
            Directory : in String;
            Pattern   : in String;
            Filter    : in Filter_Type := (others => True);
            Process   : not null access procedure (
                        Directory_Entry : in Directory_Entry_Type));

```

112/3 Searches in the directory named by `Directory` for entries matching `Pattern`. The subprogram designated by `Process` is called with each matching entry in turn. `Pattern` represents a pattern for matching file names. If `Pattern` is the null string, all items in the directory are matched; otherwise, the interpretation of `Pattern` is implementation-defined. Only items that match `Filter` will be returned. The exception `Name_Error` is propagated if the string given by `Directory` does not identify an existing directory, or if `Pattern` does not allow the identification of any possible external file or directory. The exception `Use_Error` is propagated if the external environment does not support the searching of the directory with the given name (in the absence of `Name_Error`).

```
113/2      function Simple_Name (Directory_Entry : in Directory_Entry_Type)
           return String;
```

114/2 Returns the simple external name of the external file (including directories) represented by `Directory_Entry`. The format of the name returned is implementation-defined. The exception `Status_Error` is propagated if `Directory_Entry` is invalid.

```
115/2      function Full_Name (Directory_Entry : in Directory_Entry_Type)
          return String;
```

116/2 Returns the full external name of the external file (including directories) represented by `Directory_Entry`. The format of the name returned is implementation-defined. The exception `Status_Error` is propagated if `Directory_Entry` is invalid.

```
117/2      function Kind (Directory_Entry : in Directory_Entry_Type)
           return File Kind;
```

118/2 Returns the kind of external file represented by `Directory_Entry`. The exception `Status_Error` is propagated if `Directory_Entry` is invalid.

```
119/2      function Size (Directory_Entry : in Directory_Entry_Type)
           return File Size;
```

120/2	Returns the size of the external file represented by <code>Directory_Entry</code>. The size of an external file is the number of stream elements contained in the file. If the external file represented by <code>Directory_Entry</code> is not an ordinary file, the result is implementation-defined. The exception <code>Status_Error</code> is propagated if <code>Directory_Entry</code> is invalid. The exception <code>Constraint_Error</code> is propagated if the file size is not a value of type <code>File_Size</code>.
-------	--

function Modification_Time (Directory_Entry : **in** Directory_Entry_Type) 121/2
return Ada.Calendar.Time;

Returns the time that the external file represented by Directory_Entry was most recently 122/2
 modified. If the external file represented by Directory_Entry is not an ordinary file, the result is
 implementation-defined. The exception Status_Error is propagated if Directory_Entry is invalid.
 The exception Use_Error is propagated if the external environment does not support reading the
 modification time of the file represented by Directory_Entry.

Implementation Requirements

For Copy_File, if Source_Name identifies an existing external ordinary file created by a predefined Ada 123/2
 input-output package, and Target_Name and Form can be used in the Create operation of that input-output
 package with mode Out_File without raising an exception, then Copy_File shall not propagate Use_Error.

Implementation Advice

If other information about a file (such as the owner or creation date) is available in a directory entry, the 124/2
 implementation should provide functions in a child package Directories.Information to retrieve it.

Start_Search and Search should raise Use_Error if Pattern is malformed, but not if it could represent a file 125/2
 in the directory but does not actually do so.

Rename should be supported at least when both New_Name and Old_Name are simple names and 126/2
 New_Name does not identify an existing external file.

NOTES

37 The operations Containing_Directory, Full_Name, Simple_Name, Base_Name, Extension, and Compose operate on 127/2
 file names, not external files. The files identified by these operations do not need to exist. Name_Error is raised only if the
 file name is malformed and cannot possibly identify a file. Of these operations, only the result of Full_Name depends on
 the current default directory; the result of the others depends only on their parameters.

38 Using access types, values of Search_Type and Directory_Entry_Type can be saved and queried later. However, 128/2
 another task or application can modify or delete the file represented by a Directory_Entry_Type value or the directory
 represented by a Search_Type value; such a value can only give the information valid at the time it is created. Therefore,
 long-term storage of these values is not recommended.

39 If the target system does not support directories inside of directories, then Kind will never return Directory and 129/2
 Containing_Directory will always raise Use_Error.

40 If the target system does not support creation or deletion of directories, then Create_Directory, Create_Path, 130/2
 Delete_Directory, and Delete_Tree will always propagate Use_Error.

41 To move a file or directory to a different location, use Rename. Most target systems will allow renaming of files from 131/2
 one directory to another. If the target file or directory might already exist, it should be deleted first.

A.16.1 The Package Directories.Hierarchical_File_Names

The library package Directories.Hierarchical_File_Names is an optional package providing operations for 1/3
 file name construction and decomposition for targets with hierarchical file naming.

Static Semantics

If provided, the library package Directories.Hierarchical_File_Names has the following declaration: 2/3

```
package Ada.Directories.Hierarchical_File_Names is 3/3
    function Is_Simple_Name (Name : in String) return Boolean; 4/3
    function Is_Root_Directory_Name (Name : in String) return Boolean; 5/3
    function Is_Parent_Directory_Name (Name : in String) return Boolean; 6/3
```

```

7/3      function Is_Current_Directory_Name (Name : in String) return Boolean;
8/3      function Is_Full_Name (Name : in String) return Boolean;
9/3      function Is_Relative_Name (Name : in String) return Boolean;
10/3     function Simple_Name (Name : in String) return String
        renames Ada.Directories.Simple_Name;
11/3     function Containing_Directory (Name : in String) return String
        renames Ada.Directories.Containing_Directory;
12/3     function Initial_Directory (Name : in String) return String;
13/3     function Relative_Name (Name : in String) return String;
14/3     function Compose (Directory      : in String := "";
                          Relative_Name  : in String;
                          Extension      : in String := "") return String;
15/3     end Ada.Directories.Hierarchical_File_Names;

```

In addition to the operations provided in package `Directories.Hierarchical_File_Names`, operations in package `Directories` can be used. In particular, functions `Full_Name`, `Base_Name`, and `Extension` are useable with hierarchical file names.

```

17/3     function Is_Simple_Name (Name : in String) return Boolean;
18/3     Returns True if Name is a simple name, and returns False otherwise.
19/3     function Is_Root_Directory_Name (Name : in String) return Boolean;
20/3     Returns True if Name is syntactically a root (a directory that cannot be decomposed further), and
        returns False otherwise.
21/3     function Is_Parent_Directory_Name (Name : in String) return Boolean;
22/3     Returns True if Name can be used to indicate symbolically the the parent directory of any
        directory, and returns False otherwise.
23/3     function Is_Current_Directory_Name (Name : in String) return Boolean;
24/3     Returns True if Name can be used to indicate symbolically the directory itself for any directory,
        and returns False otherwise.
25/3     function Is_Full_Name (Name : in String) return Boolean;
26/3     Returns True if the leftmost directory part of Name is a root, and returns False otherwise.
27/3     function Is_Relative_Name (Name : in String) return Boolean;
28/3     Returns True if Name allows the identification of an external file but is not a full name, and
        returns False otherwise.
29/3     function Initial_Directory (Name : in String) return String;
30/3     Returns the leftmost directory part in Name. That is a root directory name (for a full name), or
        one of a parent directory name, a current directory name, or a simple name (for a relative name).
        The exception Name_Error is propagated if the string given as Name does not allow the
        identification of an external file (including directories and special files).
31/3     function Relative_Name (Name : in String) return String;
32/3     Returns the entire file name except the Initial_Directory portion. The exception Name_Error is
        propagated if the string given as Name does not allow the identification of an external file
        (including directories and special files), or if Name has a single part (this includes if any of
        Is_Simple_Name, Is_Root_Directory_Name, Is_Parent_Directory_Name, or
        Is_Current_Directory_Name are True).

```



```

function Compose (Directory      : in String := "";
                  Relative_Name  : in String;
                  Extension      : in String := "") return String;

```

Returns the name of the external file with the specified Directory, Relative_Name, and Extension. The exception Name_Error is propagated if the string given as Directory is not the null string and does not allow the identification of a directory, or if Is_Relative_Name (Relative_Name) is False, or if the string given as Extension is not the null string and is not a possible extension, or if Extension is not the null string and Simple_Name (Relative_Name) is not a base name.

The result of Compose is a full name if Is_Full_Name (Directory) is True; result is a relative name otherwise.

Implementation Advice

Directories.Hierarchical_File_Names should be provided for systems with hierarchical file naming, and should not be provided on other systems.

NOTES

42 These operations operate on file names, not external files. The files identified by these operations do not need to exist. Name_Error is raised only as specified or if the file name is malformed and cannot possibly identify a file. The result of these operations depends only on their parameters.

43 Containing_Directory raises Use_Error if Name does not have a containing directory, including when any of Is_Simple_Name, Is_Root_Directory_Name, Is_Parent_Directory_Name, or Is_Current_Directory_Name are True.

A.17 The Package Environment_Variables

The package Environment_Variables allows a program to read or modify environment variables. Environment variables are name-value pairs, where both the name and value are strings. The definition of what constitutes an *environment variable*, and the meaning of the name and value, are implementation defined.

Static Semantics

The library package Environment_Variables has the following declaration:

```

package Ada.Environment_Variables is
  pragma Preelaborate(Environment_Variables);
  function Value (Name : in String) return String;
  function Exists (Name : in String) return Boolean;
  procedure Set (Name : in String; Value : in String);
  procedure Clear (Name : in String);
  procedure Clear;
  procedure Iterate (
    Process : not null access procedure (Name, Value : in String));
end Ada.Environment_Variables;
function Value (Name : in String) return String;

```

If the external execution environment supports environment variables, then Value returns the value of the environment variable with the given name. If no environment variable with the given name exists, then Constraint_Error is propagated. If the execution environment does not support environment variables, then Program_Error is propagated.

- 12/2 **function** Exists (Name : **in** String) **return** Boolean;
- 13/2 If the external execution environment supports environment variables and an environment variable with the given name currently exists, then Exists returns True; otherwise it returns False.
- 14/2 **procedure** Set (Name : **in** String; Value : **in** String);
- 15/2 If the external execution environment supports environment variables, then Set first clears any existing environment variable with the given name, and then defines a single new environment variable with the given name and value. Otherwise Program_Error is propagated.
- 16/2 If implementation-defined circumstances prohibit the definition of an environment variable with the given name and value, then Constraint_Error is propagated.
- 17/2 It is implementation defined whether there exist values for which the call Set(Name, Value) has the same effect as Clear (Name).
- 18/2 **procedure** Clear (Name : **in** String);
- 19/2 If the external execution environment supports environment variables, then Clear deletes all existing environment variable with the given name. Otherwise Program_Error is propagated.
- 20/2 **procedure** Clear;
- 21/2 If the external execution environment supports environment variables, then Clear deletes all existing environment variables. Otherwise Program_Error is propagated.
- 22/2 **procedure** Iterate (
 Process : **not null access procedure** (Name, Value : **in** String));
- 23/2 If the external execution environment supports environment variables, then Iterate calls the subprogram designated by Process for each existing environment variable, passing the name and value of that environment variable. Otherwise Program_Error is propagated.
- 24/2 If several environment variables exist that have the same name, Process is called once for each such variable.

Bounded (Run-Time) Errors

- 25/2 It is a bounded error to call Value if more than one environment variable exists with the given name; the possible outcomes are that:
- 26/2 • one of the values is returned, and that same value is returned in subsequent calls in the absence of changes to the environment; or
- 27/2 • Program_Error is propagated.

Erroneous Execution

- 28/2 Making calls to the procedures Set or Clear concurrently with calls to any subprogram of package Environment_Variables, or to any instantiation of Iterate, results in erroneous execution.
- 29/2 Making calls to the procedures Set or Clear in the actual subprogram corresponding to the Process parameter of Iterate results in erroneous execution.

Documentation Requirements

- 30/2 An implementation shall document how the operations of this package behave if environment variables are changed by external mechanisms (for instance, calling operating system services).

Implementation Permissions

An implementation running on a system that does not support environment variables is permitted to define the operations of package `Environment_Variables` with the semantics corresponding to the case where the external execution environment does support environment variables. In this case, it shall provide a mechanism to initialize a nonempty set of environment variables prior to the execution of a partition. 31/2

Implementation Advice

If the execution environment supports subprocesses, the currently defined environment variables should be used to initialize the environment variables of a subprocess. 32/2

Changes to the environment variables made outside the control of this package should be reflected immediately in the effect of the operations of this package. Changes to the environment variables made using this package should be reflected immediately in the external execution environment. This package should not perform any buffering of the environment variables. 33/2

A.18 Containers

- 1/2 This clause presents the specifications of the package Containers and several child packages, which provide facilities for storing collections of elements.
- 2/2 A variety of sequence and associative containers are provided. Each container includes a *cursor* type. A cursor is a reference to an element within a container. Many operations on cursors are common to all of the containers. A cursor referencing an element in a container is considered to be overlapping with the container object itself.
- 3/2 Within this clause we provide Implementation Advice for the desired average or worst case time complexity of certain operations on a container. This advice is expressed using the Landau symbol $O(X)$. Presuming f is some function of a length parameter N and $t(N)$ is the time the operation takes (on average or worst case, as specified) for the length N , a complexity of $O(f(N))$ means that there exists a finite A such that for any N , $t(N)/f(N) < A$.
- 4/2 If the advice suggests that the complexity should be less than $O(f(N))$, then for any arbitrarily small positive real D , there should exist a positive integer M such that for all $N > M$, $t(N)/f(N) < D$.
- 4.1/3 When a formal function is used to provide an ordering for a container, it is generally required to define a strict weak ordering. A function " $<$ " defines a *strict weak ordering* if it is irreflexive, asymmetric, transitive, and in addition, if $x < y$ for any values x and y , then for all other values z , $(x < z)$ or $(z < y)$.

A.18.1 The Package Containers

- 1/2 The package Containers is the root of the containers subsystem.

Static Semantics

- 2/2 The library package Containers has the following declaration:

```

3/2   package Ada.Containers is
        pragma Pure(Containers);
4/2   type Hash_Type is mod implementation-defined;
5/2   type Count_Type is range 0 .. implementation-defined;
5.1/3   Capacity_Error : exception;
6/2   end Ada.Containers;
```

- 7/2 Hash_Type represents the range of the result of a hash function. Count_Type represents the (potential or actual) number of elements of a container.

Implementation Advice

- 8/2 Hash_Type'Modulus should be at least $2^{*}32$. Count_Type'Last should be at least $2^{*}31-1$.

A.18.2 The Package Containers.Vectors

- 1/2 The language-defined generic package Containers.Vectors provides private types Vector and Cursor, and a set of operations for each type. A vector container allows insertion and deletion at any position, but it is specifically optimized for insertion and deletion at the high end (the end with the higher index) of the container. A vector container also provides random access to its elements.

A vector container behaves conceptually as an array that expands as necessary as items are inserted. The *length* of a vector is the number of elements that the vector contains. The *capacity* of a vector is the maximum number of elements that can be inserted into the vector prior to it being automatically expanded. 2/2

Elements in a vector container can be referred to by an index value of a generic formal type. The first element of a vector always has its index value equal to the lower bound of the formal type. 3/2

A vector container may contain *empty elements*. Empty elements do not have a specified value. 4/2

Static Semantics

The generic library package Containers.Vectors has the following declaration: 5/2

```

generic
  type Index_Type is range <>;
  type Element_Type is private;
  with function "=" (Left, Right : Element_Type)
    return Boolean is <>;
package Ada.Containers.Vectors is
  pragma Preelaborate(Vectors);
  pragma Remote_Types(Vectors);

  subtype Extended_Index is
    Index_Type'Base range
      Index_Type'First-1 ..
      Index_Type'Min (Index_Type'Base'Last - 1, Index_Type'Last) + 1;
  No_Index : constant Extended_Index := Extended_Index'First;

  type Vector is tagged private;
  pragma Preelaborable_Initialization(Vector);
  type Cursor is private;
  pragma Preelaborable_Initialization(Cursor);

  Empty_Vector : constant Vector;
  No_Element : constant Cursor;

  function "=" (Left, Right : Vector) return Boolean;
  function To_Vector (Length : Count_Type) return Vector;
  function To_Vector
    (New_Item : Element_Type;
     Length   : Count_Type) return Vector;
  function "&" (Left, Right : Vector) return Vector;
  function "&" (Left   : Vector;
              Right  : Element_Type) return Vector;
  function "&" (Left   : Element_Type;
              Right  : Vector) return Vector;
  function "&" (Left, Right : Element_Type) return Vector;
  function Capacity (Container : Vector) return Count_Type;
  procedure Reserve_Capacity (Container : in out Vector;
                             Capacity  : in Count_Type);
  function Length (Container : Vector) return Count_Type;
  procedure Set_Length (Container : in out Vector;
                       Length    : in Count_Type);
  function Is_Empty (Container : Vector) return Boolean;
  procedure Clear (Container : in out Vector);
  function To_Cursor (Container : Vector;
                     Index     : Extended_Index) return Cursor;
  function To_Index (Position : Cursor) return Extended_Index;

```

```

27/2      function Element (Container : Vector;
                          Index      : Index_Type)
          return Element_Type;

28/2      function Element (Position : Cursor) return Element_Type;

29/2      procedure Replace_Element (Container : in out Vector;
                                   Index      : in   Index_Type;
                                   New_Item   : in   Element_Type);

30/2      procedure Replace_Element (Container : in out Vector;
                                   Position   : in   Cursor;
                                   New_item    : in   Element_Type);

31/2      procedure Query_Element
        (Container : in Vector;
         Index     : in Index_Type;
         Process   : not null access procedure (Element : in Element_Type));

32/2      procedure Query_Element
        (Position : in Cursor;
         Process   : not null access procedure (Element : in Element_Type));

33/2      procedure Update_Element
        (Container : in out Vector;
         Index     : in   Index_Type;
         Process   : not null access procedure
                   (Element : in out Element_Type));

34/2      procedure Update_Element
        (Container : in out Vector;
         Position  : in   Cursor;
         Process   : not null access procedure
                   (Element : in out Element_Type));

34.1/3    procedure Assign (Target : in out Vector; Source : in Vector);

34.2/3    function Copy (Source : Vector; Capacity : Count_Type := 0)
          return Vector;

35/2      procedure Move (Target : in out Vector;
                        Source : in out Vector);

36/2      procedure Insert (Container : in out Vector;
                          Before     : in   Extended_Index;
                          New_Item   : in   Vector);

37/2      procedure Insert (Container : in out Vector;
                          Before     : in   Cursor;
                          New_Item   : in   Vector);

38/2      procedure Insert (Container : in out Vector;
                          Before     : in   Cursor;
                          New_Item   : in   Vector;
                          Position   : out Cursor);

39/2      procedure Insert (Container : in out Vector;
                          Before     : in   Extended_Index;
                          New_Item   : in   Element_Type;
                          Count      : in   Count_Type := 1);

40/2      procedure Insert (Container : in out Vector;
                          Before     : in   Cursor;
                          New_Item   : in   Element_Type;
                          Count      : in   Count_Type := 1);

41/2      procedure Insert (Container : in out Vector;
                          Before     : in   Cursor;
                          New_Item   : in   Element_Type;
                          Position   : out Cursor;
                          Count      : in   Count_Type := 1);

42/2      procedure Insert (Container : in out Vector;
                          Before     : in   Extended_Index;
                          Count      : in   Count_Type := 1);

```

```

procedure Insert (Container : in out Vector;                               43/2
                  Before   : in      Cursor;
                  Position  : out     Cursor;
                  Count     : in      Count_Type := 1);

procedure Prepend (Container : in out Vector;                               44/2
                  New_Item  : in      Vector);

procedure Prepend (Container : in out Vector;                               45/2
                  New_Item  : in      Element_Type;
                  Count     : in      Count_Type := 1);

procedure Append (Container : in out Vector;                               46/2
                  New_Item  : in      Vector);

procedure Append (Container : in out Vector;                               47/2
                  New_Item  : in      Element_Type;
                  Count     : in      Count_Type := 1);

procedure Insert_Space (Container : in out Vector;                          48/2
                       Before   : in      Extended_Index;
                       Count     : in      Count_Type := 1);

procedure Insert_Space (Container : in out Vector;                          49/2
                       Before   : in      Cursor;
                       Position  : out     Cursor;
                       Count     : in      Count_Type := 1);

procedure Delete (Container : in out Vector;                               50/2
                  Index     : in      Extended_Index;
                  Count     : in      Count_Type := 1);

procedure Delete (Container : in out Vector;                               51/2
                  Position  : in out Cursor;
                  Count     : in      Count_Type := 1);

procedure Delete_First (Container : in out Vector;                          52/2
                       Count     : in      Count_Type := 1);

procedure Delete_Last (Container : in out Vector;                          53/2
                       Count     : in      Count_Type := 1);

procedure Reverse_Elements (Container : in out Vector);                      54/2

procedure Swap (Container : in out Vector;                               55/2
               I, J       : in      Index_Type);

procedure Swap (Container : in out Vector;                               56/2
               I, J       : in      Cursor);

function First_Index (Container : Vector) return Index_Type;                57/2

function First (Container : Vector) return Cursor;                        58/2

function First_Element (Container : Vector)                               59/2
return Element_Type;

function Last_Index (Container : Vector) return Extended_Index;            60/2

function Last (Container : Vector) return Cursor;                        61/2

function Last_Element (Container : Vector)                               62/2
return Element_Type;

function Next (Position : Cursor) return Cursor;                          63/2

procedure Next (Position : in out Cursor);                                64/2

function Previous (Position : Cursor) return Cursor;                      65/2

procedure Previous (Position : in out Cursor);                            66/2

function Find_Index (Container : Vector;                               67/2
                   Item      : Element_Type;
                   Index     : Index_Type := Index_Type'First)
return Extended_Index;

```

```

68/2      function Find (Container : Vector;
                        Item       : Element_Type;
                        Position   : Cursor := No_Element)
        return Cursor;
69/2      function Reverse_Find_Index (Container : Vector;
                                      Item       : Element_Type;
                                      Index      : Index_Type := Index_Type'Last)
        return Extended_Index;
70/2      function Reverse_Find (Container : Vector;
                                Item       : Element_Type;
                                Position   : Cursor := No_Element)
        return Cursor;
71/2      function Contains (Container : Vector;
                             Item      : Element_Type) return Boolean;
72/2      function Has_Element (Position : Cursor) return Boolean;
73/2      procedure Iterate
        (Container : in Vector;
         Process   : not null access procedure (Position : in Cursor));
74/2      procedure Reverse_Iterate
        (Container : in Vector;
         Process   : not null access procedure (Position : in Cursor));
75/2      generic
        with function "<" (Left, Right : Element_Type)
          return Boolean is <>;
        package Generic_Sorting is
76/2          function Is_Sorted (Container : Vector) return Boolean;
77/2          procedure Sort (Container : in out Vector);
78/2          procedure Merge (Target  : in out Vector;
                              Source  : in out Vector);
79/2      end Generic_Sorting;
80/2      private
81/2      ... -- not specified by the language
82/2      end Ada.Containers.Vectors;

```

83/2 The actual function for the generic formal function "=" on Element_Type values is expected to define a reflexive and symmetric relationship and return the same result value each time it is called with a particular pair of values. If it behaves in some other manner, the functions defined to use it return an unspecified value. The exact arguments and number of calls of this generic formal function by the functions defined to use it are unspecified.

84/2 The type Vector is used to represent vectors. The type Vector needs finalization (see 7.6).

85/2 Empty_Vector represents the empty vector object. It has a length of 0. If an object of type Vector is not otherwise initialized, it is initialized to the same value as Empty_Vector.

86/2 No_Element represents a cursor that designates no element. If an object of type Cursor is not otherwise initialized, it is initialized to the same value as No_Element.

87/2 The predefined "=" operator for type Cursor returns True if both cursors are No_Element, or designate the same element in the same container.

88/2 Execution of the default implementation of the Input, Output, Read, or Write attribute of type Cursor raises Program_Error.

88.1/3 Vector'Write writes the length of Vector elements to the stream. Vector'Read reads the length Vector elements from the stream.

No_Index represents a position that does not correspond to any element. The subtype Extended_Index includes the indices covered by Index_Type plus the value No_Index and, if it exists, the successor to the Index_Type'Last. 89/2

If an operation attempts to modify the vector such that the position of the last element would be greater than Index_Type'Last, then the operation propagates Constraint_Error. 89.1/3

Some operations of this generic package have access-to-subprogram parameters. To ensure such operations are well-defined, they guard against certain actions by the designated subprogram. In particular, some operations check for “tampering with cursors” of a container because they depend on the set of elements of the container remaining constant, and others check for “tampering with elements” of a container because they depend on elements of the container not being replaced. 90/2

A subprogram is said to *tamper with cursors* of a vector object *V* if: 91/2

- it inserts or deletes elements of *V*, that is, it calls the Insert, Insert_Space, Clear, Delete, or Set_Length procedures with *V* as a parameter; or 92/2
- it finalizes *V*; or 93/2
- it calls the Assign procedure with *V* as the Target parameter; or 93.1/3
- it calls the Move procedure with *V* as a parameter. 94/2

A subprogram is said to *tamper with elements* of a vector object *V* if: 95/2

- it tampers with cursors of *V*; or 96/2
- it replaces one or more elements of *V*, that is, it calls the Replace_Element, Reverse_Elements, or Swap procedures or the Sort or Merge procedures of an instance of Generic_Sorting with *V* as a parameter. 97/2

function "=" (Left, Right : Vector) **return** Boolean; 98/2

If Left and Right denote the same vector object, then the function returns True. If Left and Right have different lengths, then the function returns False. Otherwise, it compares each element in Left to the corresponding element in Right using the generic formal equality operator. If any such comparison returns False, the function returns False; otherwise it returns True. Any exception raised during evaluation of element equality is propagated. 99/2

function To_Vector (Length : Count_Type) **return** Vector; 100/2

Returns a vector with a length of Length, filled with empty elements. 101/2

function To_Vector
(New_Item : Element_Type;
Length : Count_Type) **return** Vector; 102/2

Returns a vector with a length of Length, filled with elements initialized to the value New_Item. 103/2

function "&" (Left, Right : Vector) **return** Vector; 104/2

Returns a vector comprising the elements of Left followed by the elements of Right. 105/2

function "&" (Left : Vector;
Right : Element_Type) **return** Vector; 106/2

Returns a vector comprising the elements of Left followed by the element Right. 107/2

function "&" (Left : Element_Type;
Right : Vector) **return** Vector; 108/2

Returns a vector comprising the element Left followed by the elements of Right. 109/2

```

110/2    function "&" (Left, Right : Element_Type) return Vector;
111/2        Returns a vector comprising the element Left followed by the element Right.

112/2    function Capacity (Container : Vector) return Count_Type;
113/2        Returns the capacity of Container.

114/2    procedure Reserve_Capacity (Container : in out Vector;
                                Capacity  : in      Count_Type);
115/3        If the capacity of Container is already greater than or equal to Capacity then Reserve_Capacity
        has no effect. Otherwise, Reserve_Capacity allocates additional storage to ensure that the length
        of the resulting vector can become at least the value Capacity without requiring an additional
        call to Reserve_Capacity, and is large enough to hold the current length of Container.
        Reserve_Capacity then, as necessary, moves elements into the new data structures and
        deallocates any storage no longer needed. Any exception raised during allocation is propagated
        and Container is not modified.

116/2    function Length (Container : Vector) return Count_Type;
117/2        Returns the number of elements in Container.

118/2    procedure Set_Length (Container : in out Vector;
                           Length    : in      Count_Type);
119/2        If Length is larger than the capacity of Container, Set_Length calls Reserve_Capacity
        (Container, Length), then sets the length of the Container to Length. If Length is greater than the
        original length of Container, empty elements are added to Container; otherwise elements are
        removed from Container.

120/2    function Is_Empty (Container : Vector) return Boolean;
121/2        Equivalent to Length (Container) = 0.

122/2    procedure Clear (Container : in out Vector);
123/2        Removes all the elements from Container. The capacity of Container does not change.

124/2    function To_Cursor (Container : Vector;
                        Index      : Extended_Index) return Cursor;
125/2        If Index is not in the range First_Index (Container) .. Last_Index (Container), then No_Element
        is returned. Otherwise, a cursor designating the element at position Index in Container is
        returned.

126/2    function To_Index (Position : Cursor) return Extended_Index;
127/2        If Position is No_Element, No_Index is returned. Otherwise, the index (within its containing
        vector) of the element designated by Position is returned.

128/2    function Element (Container : Vector;
                      Index      : Index_Type)
        return Element_Type;
129/2        If Index is not in the range First_Index (Container) .. Last_Index (Container), then
        Constraint_Error is propagated. Otherwise, Element returns the element at position Index.

130/2    function Element (Position : Cursor) return Element_Type;
131/2        If Position equals No_Element, then Constraint_Error is propagated. Otherwise, Element returns
        the element designated by Position.

```

procedure Replace_Element	(Container : in out Vector; Index : in Index_Type; New_Item : in Element_Type);	132/2
If Index is not in the range First_Index (Container) .. Last_Index (Container), then Constraint_Error is propagated. Otherwise Replace_Element assigns the value New_Item to the element at position Index. Any exception raised during the assignment is propagated. The element at position Index is not an empty element after successful call to Replace_Element.		133/2
procedure Replace_Element	(Container : in out Vector; Position : in Cursor; New_Item : in Element_Type);	134/2
If Position equals No_Element, then Constraint_Error is propagated; if Position does not designate an element in Container, then Program_Error is propagated. Otherwise Replace_Element assigns New_Item to the element designated by Position. Any exception raised during the assignment is propagated. The element at Position is not an empty element after successful call to Replace_Element.		135/2
procedure Query_Element	(Container : in Vector; Index : in Index_Type; Process : not null access procedure (Element : in Element_Type));	136/2
If Index is not in the range First_Index (Container) .. Last_Index (Container), then Constraint_Error is propagated. Otherwise, Query_Element calls Process.all with the element at position Index as the argument. Program_Error is propagated if Process.all tampers with the elements of Container. Any exception raised by Process.all is propagated.		137/2
procedure Query_Element	(Position : in Cursor; Process : not null access procedure (Element : in Element_Type));	138/2
If Position equals No_Element, then Constraint_Error is propagated. Otherwise, Query_Element calls Process.all with the element designated by Position as the argument. Program_Error is propagated if Process.all tampers with the elements of Container. Any exception raised by Process.all is propagated.		139/2
procedure Update_Element	(Container : in out Vector; Index : in Index_Type; Process : not null access procedure (Element : in out Element_Type));	140/2
If Index is not in the range First_Index (Container) .. Last_Index (Container), then Constraint_Error is propagated. Otherwise, Update_Element calls Process.all with the element at position Index as the argument. Program_Error is propagated if Process.all tampers with the elements of Container. Any exception raised by Process.all is propagated.		141/2
If Element_Type is unconstrained and definite, then the actual Element parameter of Process.all shall be unconstrained.		142/2
The element at position Index is not an empty element after successful completion of this operation.		143/2

- 144/2 **procedure** Update_Element
 (Container : **in out** Vector;
 Position : **in** Cursor;
 Process : **not null access procedure** (Element : **in out** Element_Type));
- 145/2 If Position equals No_Element, then Constraint_Error is propagated; if Position does not designate an element in Container, then Program_Error is propagated. Otherwise Update_Element calls Process.all with the element designated by Position as the argument. Program_Error is propagated if Process.all tampers with the elements of Container. Any exception raised by Process.all is propagated.
- 146/2 If Element_Type is unconstrained and definite, then the actual Element parameter of Process.all shall be unconstrained.
- 147/2 The element designated by Position is not an empty element after successful completion of this operation.
- 147.1/3 **procedure** Assign (Target : **in out** Vector; Source : **in** Vector);
- 147.2/3 If Target denotes the same object as Source, the operation has no effect. If the length of Source is greater than the capacity of Target, Reserve_Capacity is called with the length of Source as the capacity. Each element of Source is assigned to the corresponding element of Target.
- 147.3/3 **function** Copy (Source : Vector; Capacity : Count_Type := 0)
 return Vector;
- 147.4/3 Returns a vector whose elements are initialized from the corresponding elements of Source. If Capacity is 0, then the vector capacity is the length of Source; if Capacity is equal to or greater than the length of Source, the vector capacity is at least the specified value. Otherwise, the operation propagated Capacity_Error.
- 148/2 **procedure** Move (Target : **in out** Vector;
 Source : **in out** Vector);
- 149/3 If Target denotes the same object as Source, then Move has no effect. Otherwise, Move first calls Reserve_Capacity (Target, Length (Source)) and then Clear (Target); then, each element from Source is removed from Source and inserted into Target in the original order. The length of Source is 0 after a successful call to Move.
- 150/2 **procedure** Insert (Container : **in out** Vector;
 Before : **in** Extended_Index;
 New_Item : **in** Vector);
- 151/2 If Before is not in the range First_Index (Container) .. Last_Index (Container) + 1, then Constraint_Error is propagated. If Length(New_Item) is 0, then Insert does nothing. Otherwise, it computes the new length *NL* as the sum of the current length and Length (New_Item); if the value of Last appropriate for length *NL* would be greater than Index_Type'Last then Constraint_Error is propagated.
- 152/2 If the current vector capacity is less than *NL*, Reserve_Capacity (Container, *NL*) is called to increase the vector capacity. Then Insert slides the elements in the range Before .. Last_Index (Container) up by Length(New_Item) positions, and then copies the elements of New_Item to the positions starting at Before. Any exception raised during the copying is propagated.

procedure Insert	(Container : in out Vector; Before : in Cursor; New_Item : in Vector);	153/2
If Before is not No_Element, and does not designate an element in Container, then Program_Error is propagated. Otherwise, if Length(New_Item) is 0, then Insert does nothing. If Before is No_Element, then the call is equivalent to Insert (Container, Last_Index (Container) + 1, New_Item); otherwise the call is equivalent to Insert (Container, To_Index (Before), New_Item);		
procedure Insert	(Container : in out Vector; Before : in Cursor; New_Item : in Vector; Position : out Cursor);	155/2
If Before is not No_Element, and does not designate an element in Container, then Program_Error is propagated. If Before equals No_Element, then let <i>T</i> be Last_Index (Container) + 1; otherwise, let <i>T</i> be To_Index (Before). Insert (Container, <i>T</i> , New_Item) is called, and then Position is set to To_Cursor (Container, <i>T</i>).		
procedure Insert	(Container : in out Vector; Before : in Extended_Index; New_Item : in Element_Type; Count : in Count_Type := 1);	157/2
Equivalent to Insert (Container, Before, To_Vector (New_Item, Count));		
procedure Insert	(Container : in out Vector; Before : in Cursor; New_Item : in Element_Type; Count : in Count_Type := 1);	159/2
Equivalent to Insert (Container, Before, To_Vector (New_Item, Count));		
procedure Insert	(Container : in out Vector; Before : in Cursor; New_Item : in Element_Type; Position : out Cursor; Count : in Count_Type := 1);	161/2
Equivalent to Insert (Container, Before, To_Vector (New_Item, Count), Position);		
procedure Insert	(Container : in out Vector; Before : in Extended_Index; Count : in Count_Type := 1);	163/2
If Before is not in the range First_Index (Container) .. Last_Index (Container) + 1, then Constraint_Error is propagated. If Count is 0, then Insert does nothing. Otherwise, it computes the new length <i>NL</i> as the sum of the current length and Count; if the value of Last appropriate for length <i>NL</i> would be greater than Index_Type'Last then Constraint_Error is propagated.		
If the current vector capacity is less than <i>NL</i> , Reserve_Capacity (Container, <i>NL</i>) is called to increase the vector capacity. Then Insert slides the elements in the range Before .. Last_Index (Container) up by Count positions, and then inserts elements that are initialized by default (see 3.3.1) in the positions starting at Before.		

166/2 **procedure** Insert (Container : **in out** Vector;
 Before : **in** Cursor;
 Position : **out** Cursor;
 Count : **in** Count_Type := 1);

167/2 If Before is not No_Element, and does not designate an element in Container, then Program_Error is propagated. If Before equals No_Element, then let *T* be Last_Index (Container) + 1; otherwise, let *T* be To_Index (Before). Insert (Container, *T*, Count) is called, and then Position is set to To_Cursor (Container, *T*).

168/2 **procedure** Prepend (Container : **in out** Vector;
 New_Item : **in** Vector;
 Count : **in** Count_Type := 1);

169/2 Equivalent to Insert (Container, First_Index (Container), New_Item).

170/2 **procedure** Prepend (Container : **in out** Vector;
 New_Item : **in** Element_Type;
 Count : **in** Count_Type := 1);

171/2 Equivalent to Insert (Container, First_Index (Container), New_Item, Count).

172/2 **procedure** Append (Container : **in out** Vector;
 New_Item : **in** Vector);

173/2 Equivalent to Insert (Container, Last_Index (Container) + 1, New_Item).

174/2 **procedure** Append (Container : **in out** Vector;
 New_Item : **in** Element_Type;
 Count : **in** Count_Type := 1);

175/2 Equivalent to Insert (Container, Last_Index (Container) + 1, New_Item, Count).

176/2 **procedure** Insert_Space (Container : **in out** Vector;
 Before : **in** Extended_Index;
 Count : **in** Count_Type := 1);

177/2 If Before is not in the range First_Index (Container) .. Last_Index (Container) + 1, then Constraint_Error is propagated. If Count is 0, then Insert_Space does nothing. Otherwise, it computes the new length *NL* as the sum of the current length and Count; if the value of Last appropriate for length *NL* would be greater than Index_Type'Last then Constraint_Error is propagated.

178/2 If the current vector capacity is less than *NL*, Reserve_Capacity (Container, *NL*) is called to increase the vector capacity. Then Insert_Space slides the elements in the range Before .. Last_Index (Container) up by Count positions, and then inserts empty elements in the positions starting at Before.

179/2 **procedure** Insert_Space (Container : **in out** Vector;
 Before : **in** Cursor;
 Position : **out** Cursor;
 Count : **in** Count_Type := 1);

180/2 If Before is not No_Element, and does not designate an element in Container, then Program_Error is propagated. If Before equals No_Element, then let *T* be Last_Index (Container) + 1; otherwise, let *T* be To_Index (Before). Insert_Space (Container, *T*, Count) is called, and then Position is set to To_Cursor (Container, *T*).

procedure Delete (Container : in out Vector; Index : in Extended_Index; Count : in Count_Type := 1);	181/2
If Index is not in the range First_Index (Container) .. Last_Index (Container) + 1, then Constraint_Error is propagated. If Count is 0, Delete has no effect. Otherwise Delete slides the elements (if any) starting at position Index + Count down to Index. Any exception raised during element assignment is propagated.	182/2
procedure Delete (Container : in out Vector; Position : in out Cursor; Count : in Count_Type := 1);	183/2
If Position equals No_Element, then Constraint_Error is propagated. If Position does not designate an element in Container, then Program_Error is propagated. Otherwise, Delete (Container, To_Index (Position), Count) is called, and then Position is set to No_Element.	184/2
procedure Delete_First (Container : in out Vector; Count : in Count_Type := 1);	185/2
Equivalent to Delete (Container, First_Index (Container), Count).	186/2
procedure Delete_Last (Container : in out Vector; Count : in Count_Type := 1);	187/2
If Length (Container) <= Count then Delete_Last is equivalent to Clear (Container). Otherwise it is equivalent to Delete (Container, Index_Type'Val(Index_Type'Pos(Last_Index (Container)) – Count + 1), Count).	188/2
procedure Reverse_Elements (Container : in out List);	189/2
Reorders the elements of Container in reverse order.	190/2
procedure Swap (Container : in out Vector; I, J : in Index_Type);	191/2
If either I or J is not in the range First_Index (Container) .. Last_Index (Container), then Constraint_Error is propagated. Otherwise, Swap exchanges the values of the elements at positions I and J.	192/2
procedure Swap (Container : in out Vector; I, J : in Cursor);	193/2
If either I or J is No_Element, then Constraint_Error is propagated. If either I or J do not designate an element in Container, then Program_Error is propagated. Otherwise, Swap exchanges the values of the elements designated by I and J.	194/2
function First_Index (Container : Vector) return Index_Type;	195/2
Returns the value Index_Type'First.	196/2
function First (Container : Vector) return Cursor;	197/2
If Container is empty, First returns No_Element. Otherwise, it returns a cursor that designates the first element in Container.	198/2
function First_Element (Container : Vector) return Element_Type;	199/2
Equivalent to Element (Container, First_Index (Container)).	200/2

201/2 **function** Last_Index (Container : Vector) **return** Extended_Index;

202/2 If Container is empty, Last_Index returns No_Index. Otherwise, it returns the position of the last element in Container.

203/2 **function** Last (Container : Vector) **return** Cursor;

204/2 If Container is empty, Last returns No_Element. Otherwise, it returns a cursor that designates the last element in Container.

205/2 **function** Last_Element (Container : Vector) **return** Element_Type;

206/2 Equivalent to Element (Container, Last_Index (Container)).

207/2 **function** Next (Position : Cursor) **return** Cursor;

208/2 If Position equals No_Element or designates the last element of the container, then Next returns the value No_Element. Otherwise, it returns a cursor that designates the element with index To_Index (Position) + 1 in the same vector as Position.

209/2 **procedure** Next (Position : **in out** Cursor);

210/2 Equivalent to Position := Next (Position).

211/2 **function** Previous (Position : Cursor) **return** Cursor;

212/2 If Position equals No_Element or designates the first element of the container, then Previous returns the value No_Element. Otherwise, it returns a cursor that designates the element with index To_Index (Position) – 1 in the same vector as Position.

213/2 **procedure** Previous (Position : **in out** Cursor);

214/2 Equivalent to Position := Previous (Position).

215/2 **function** Find_Index (Container : Vector;
 Item : Element_Type;
 Index : Index_Type := Index_Type'First)
 return Extended_Index;

216/2 Searches the elements of Container for an element equal to Item (using the generic formal equality operator). The search starts at position Index and proceeds towards Last_Index (Container). If no equal element is found, then Find_Index returns No_Index. Otherwise, it returns the index of the first equal element encountered.

217/2 **function** Find (Container : Vector;
 Item : Element_Type;
 Position : Cursor := No_Element)
 return Cursor;

218/2 If Position is not No_Element, and does not designate an element in Container, then Program_Error is propagated. Otherwise Find searches the elements of Container for an element equal to Item (using the generic formal equality operator). The search starts at the first element if Position equals No_Element, and at the element designated by Position otherwise. It proceeds towards the last element of Container. If no equal element is found, then Find returns No_Element. Otherwise, it returns a cursor designating the first equal element encountered.


```

function Reverse_Find_Index (Container : Vector;                               219/2
                             Item      : Element_Type;
                             Index     : Index_Type := Index_Type'Last)
return Extended_Index;

```

Searches the elements of Container for an element equal to Item (using the generic formal equality operator). The search starts at position Index or, if Index is greater than Last_Index (Container), at position Last_Index (Container). It proceeds towards First_Index (Container). If no equal element is found, then Reverse_Find_Index returns No_Index. Otherwise, it returns the index of the first equal element encountered. 220/2

```

function Reverse_Find (Container : Vector;                               221/2
                       Item      : Element_Type;
                       Position  : Cursor := No_Element)
return Cursor;

```

If Position is not No_Element, and does not designate an element in Container, then Program_Error is propagated. Otherwise Reverse_Find searches the elements of Container for an element equal to Item (using the generic formal equality operator). The search starts at the last element if Position equals No_Element, and at the element designated by Position otherwise. It proceeds towards the first element of Container. If no equal element is found, then Reverse_Find returns No_Element. Otherwise, it returns a cursor designating the first equal element encountered. 222/2

```

function Contains (Container : Vector;                               223/2
                  Item      : Element_Type) return Boolean;

```

Equivalent to Has_Element (Find (Container, Item)). 224/2

```

function Has_Element (Position : Cursor) return Boolean;               225/2

```

Returns True if Position designates an element, and returns False otherwise. 226/2

```

procedure Iterate                               227/2
(Container : in Vector;
 Process   : not null access procedure (Position : in Cursor));

```

Invokes Process.all with a cursor that designates each element in Container, in index order. Program_Error is propagated if Process.all tampers with the cursors of Container. Any exception raised by Process is propagated. 228/2

```

procedure Reverse_Iterate                               229/2
(Container : in Vector;
 Process   : not null access procedure (Position : in Cursor));

```

Iterates over the elements in Container as per Iterate, except that elements are traversed in reverse index order. 230/2

The actual function for the generic formal function "<" of Generic_Sorting is expected to return the same value each time it is called with a particular pair of element values. It should define a strict weak ordering relationship (see A.18); it should not modify Container. If the actual for "<" behaves in some other manner, the behavior of the subprograms of Generic_Sorting are unspecified. How many times the subprograms of Generic_Sorting call "<" is unspecified. 231/3

```

function Is_Sorted (Container : Vector) return Boolean;               232/2

```

Returns True if the elements are sorted smallest first as determined by the generic formal "<" operator; otherwise, Is_Sorted returns False. Any exception raised during evaluation of "<" is propagated. 233/2

- 234/2 **procedure** Sort (Container : **in out** Vector);
- 235/2 Reorders the elements of Container such that the elements are sorted smallest first as determined by the generic formal "<" operator provided. Any exception raised during evaluation of "<" is propagated.
- 236/2 **procedure** Merge (Target : **in out** Vector;
 Source : **in out** Vector);
- 237/2 Merge removes elements from Source and inserts them into Target; afterwards, Target contains the union of the elements that were initially in Source and Target; Source is left empty. If Target and Source are initially sorted smallest first, then Target is ordered smallest first as determined by the generic formal "<" operator; otherwise, the order of elements in Target is unspecified. Any exception raised during evaluation of "<" is propagated.

Bounded (Run-Time) Errors

- 238/2 Reading the value of an empty element by calling Element, Query_Element, Update_Element, Swap, Is_Sorted, Sort, Merge, "=", Find, or Reverse_Find is a bounded error. The implementation may treat the element as having any normal value (see 13.9.1) of the element type, or raise Constraint_Error or Program_Error before modifying the vector.
- 239/2 Calling Merge in an instance of Generic_Sorting with either Source or Target not ordered smallest first using the provided generic formal "<" operator is a bounded error. Either Program_Error is raised after Target is updated as described for Merge, or the operation works as defined.
- 239.1/3 It is a bounded error for the actual function associated with a generic formal subprogram, when called as part of an operation of this package, to tamper with elements of any Vector parameter to the operation. Either Program_Error is raised, or the operation works as defined on the value of the Vector either prior to, or subsequent to, some or all of the modifications to the Vector.
- 239.2/3 It is a bounded error to call any subprogram declared in the visible part of Containers.Vectors when the associated container has been finalized. If the operation takes Container as an **in out** parameter, then it raises Constraint_Error or Program_Error. Otherwise, the operation either proceeds as it would for an empty container, or it raises Constraint_Error or Program_Error.
- 240/2 A Cursor value is *ambiguous* if any of the following have occurred since it was created:
- 241/2 • Insert, Insert_Space, or Delete has been called on the vector that contains the element the cursor designates with an index value (or a cursor designating an element at such an index value) less than or equal to the index value of the element designated by the cursor; or
- 242/2 • The vector that contains the element it designates has been passed to the Sort or Merge procedures of an instance of Generic_Sorting, or to the Reverse_Elements procedure.
- 243/2 It is a bounded error to call any subprogram other than "=" or Has_Element declared in Containers.Vectors with an ambiguous (but not invalid, see below) cursor parameter. Possible results are:
- 244/2 • The cursor may be treated as if it were No_Element;
- 245/2 • The cursor may designate some element in the vector (but not necessarily the element that it originally designated);
- 246/2 • Constraint_Error may be raised; or
- 247/2 • Program_Error may be raised.

Erroneous Execution

- A Cursor value is *invalid* if any of the following have occurred since it was created: 248/2
- The vector that contains the element it designates has been finalized; 249/2
 - The vector that contains the element it designates has been used as the Target of a call to Assign, or as the target of an `assignment_statement`; 249.1/3
 - The vector that contains the element it designates has been used as the Source or Target of a call to Move; or 250/2
 - The element it designates has been deleted or removed from the vector that contains the element. 251/3

The result of "=" or Has_Element is unspecified if it is called with an invalid cursor parameter. Execution is erroneous if any other subprogram declared in Containers.Vectors is called with an invalid cursor parameter. 252/2

Implementation Requirements

- No storage associated with a vector object shall be lost upon assignment or scope exit. 253/2
- The execution of an `assignment_statement` for a vector shall have the effect of copying the elements from the source vector object to the target vector object. 254/2

Implementation Advice

- Containers.Vectors should be implemented similarly to an array. In particular, if the length of a vector is N , then 255/2
- the worst-case time complexity of Element should be $O(\log N)$; 256/2
 - the worst-case time complexity of Append with Count=1 when N is less than the capacity of the vector should be $O(\log N)$; and 257
 - the worst-case time complexity of Prepend with Count=1 and Delete_First with Count=1 should be $O(N \log N)$. 258/2

The worst-case time complexity of a call on procedure Sort of an instance of Containers.Vectors.Generic_Sorting should be $O(N^2)$, and the average time complexity should be better than $O(N^2)$. 259/2

Containers.Vectors.Generic_Sorting.Sort and Containers.Vectors.Generic_Sorting.Merge should minimize copying of elements. 260/2

Move should not copy elements, and should minimize copying of internal data structures. 261/2

If an exception is propagated from a vector operation, no storage should be lost, nor any elements removed from a vector unless specified by the operation. 262/2

NOTES

44 All elements of a vector occupy locations in the internal array. If a sparse container is required, a Hashed_Map should be used rather than a vector. 263/2

45 If Index_Type'Base'First = Index_Type'First an instance of Ada.Containers.Vectors will raise Constraint_Error. A value below Index_Type'First is required so that an empty vector has a meaningful value of Last_Index. 264/2

A.18.3 The Package Containers.Doubly_Linked_Lists

1/2 The language-defined generic package Containers.Doubly_Linked_Lists provides private types List and Cursor, and a set of operations for each type. A list container is optimized for insertion and deletion at any position.

2/2 A doubly-linked list container object manages a linked list of internal *nodes*, each of which contains an element and pointers to the next (successor) and previous (predecessor) internal nodes. A cursor designates a particular node within a list (and by extension the element contained in that node). A cursor keeps designating the same node (and element) as long as the node is part of the container, even if the node is moved in the container.

3/2 The *length* of a list is the number of elements it contains.

Static Semantics

4/2 The generic library package Containers.Doubly_Linked_Lists has the following declaration:

```

5/3  generic
      type Element_Type is private;
      with function "=" (Left, Right : Element_Type)
          return Boolean is <>;
  package Ada.Containers.Doubly_Linked_Lists is
      pragma Preelaborate(Doubly_Linked_Lists);
      pragma Remote_Types(Doubly_Linked_Lists);
6/2  type List is tagged private;
      pragma Preelaborable_Initialization(List);
7/2  type Cursor is private;
      pragma Preelaborable_Initialization(Cursor);
8/2  Empty_List : constant List;
9/2  No_Element : constant Cursor;
10/2 function "=" (Left, Right : List) return Boolean;
11/2 function Length (Container : List) return Count_Type;
12/2 function Is_Empty (Container : List) return Boolean;
13/2 procedure Clear (Container : in out List);
14/2 function Element (Position : Cursor)
      return Element_Type;
15/2 procedure Replace_Element (Container : in out List;
                              Position : in Cursor;
                              New_Item : in Element_Type);
16/2 procedure Query_Element
      (Position : in Cursor;
       Process : not null access procedure (Element : in Element_Type));
17/2 procedure Update_Element
      (Container : in out List;
       Position : in Cursor;
       Process : not null access procedure
         (Element : in out Element_Type));
17.1/3 procedure Assign (Target : in out List; Source : in List);
17.2/3 function Copy (Source : List) return List;
18/2 procedure Move (Target : in out List;
                   Source : in out List);

```

```

procedure Insert (Container : in out List;                               19/2
                  Before   : in   Cursor;
                  New_Item  : in   Element_Type;
                  Count     : in   Count_Type := 1);

procedure Insert (Container : in out List;                               20/2
                  Before   : in   Cursor;
                  New_Item  : in   Element_Type;
                  Position  : out Cursor;
                  Count     : in   Count_Type := 1);

procedure Insert (Container : in out List;                               21/2
                  Before   : in   Cursor;
                  Position  : out Cursor;
                  Count     : in   Count_Type := 1);

procedure Prepend (Container : in out List;                               22/2
                  New_Item  : in   Element_Type;
                  Count     : in   Count_Type := 1);

procedure Append (Container : in out List;                               23/2
                  New_Item  : in   Element_Type;
                  Count     : in   Count_Type := 1);

procedure Delete (Container : in out List;                               24/2
                  Position  : in out Cursor;
                  Count     : in   Count_Type := 1);

procedure Delete_First (Container : in out List;                          25/2
                      Count       : in   Count_Type := 1);

procedure Delete_Last (Container : in out List;                          26/2
                      Count       : in   Count_Type := 1);

procedure Reverse_Elements (Container : in out List);                    27/2

procedure Swap (Container : in out List;                                  28/2
               I, J       : in   Cursor);

procedure Swap_Links (Container : in out List;                          29/2
                   I, J       : in   Cursor);

procedure Splice (Target   : in out List;                               30/2
                 Before   : in   Cursor;
                 Source   : in out List);

procedure Splice (Target   : in out List;                               31/2
                 Before   : in   Cursor;
                 Source   : in out List;
                 Position : in out Cursor);

procedure Splice (Container: in out List;                               32/2
                 Before   : in   Cursor;
                 Position : in   Cursor);

function First (Container : List) return Cursor;                        33/2

function First_Element (Container : List)                               34/2
    return Element_Type;

function Last (Container : List) return Cursor;                          35/2

function Last_Element (Container : List)                               36/2
    return Element_Type;

function Next (Position : Cursor) return Cursor;                        37/2

function Previous (Position : Cursor) return Cursor;                    38/2

procedure Next (Position : in out Cursor);                              39/2

procedure Previous (Position : in out Cursor);                          40/2

function Find (Container : List;                                         41/2
              Item       : Element_Type;
              Position   : Cursor := No_Element)
    return Cursor;

```

```

42/2      function Reverse_Find (Container : List;
                                Item      : Element_Type;
                                Position  : Cursor := No_Element)
                                return Cursor;
43/2      function Contains (Container : List;
                             Item      : Element_Type) return Boolean;
44/2      function Has_Element (Position : Cursor) return Boolean;
45/2      procedure Iterate
        (Container : in List;
         Process   : not null access procedure (Position : in Cursor));
46/2      procedure Reverse_Iterate
        (Container : in List;
         Process   : not null access procedure (Position : in Cursor));
47/2      generic
        with function "<" (Left, Right : Element_Type)
            return Boolean is <>;
        package Generic_Sorting is
48/2          function Is_Sorted (Container : List) return Boolean;
49/2          procedure Sort (Container : in out List);
50/2          procedure Merge (Target  : in out List;
                              Source  : in out List);
51/2      end Generic_Sorting;
52/2      private
53/2      ... -- not specified by the language
54/2      end Ada.Containers.Doubly_Linked_Lists;

```

55/2 The actual function for the generic formal function "=" on Element_Type values is expected to define a reflexive and symmetric relationship and return the same result value each time it is called with a particular pair of values. If it behaves in some other manner, the functions Find, Reverse_Find, and "=" on list values return an unspecified value. The exact arguments and number of calls of this generic formal function by the functions Find, Reverse_Find, and "=" on list values are unspecified.

56/2 The type List is used to represent lists. The type List needs finalization (see 7.6).

57/2 Empty_List represents the empty List object. It has a length of 0. If an object of type List is not otherwise initialized, it is initialized to the same value as Empty_List.

58/2 No_Element represents a cursor that designates no element. If an object of type Cursor is not otherwise initialized, it is initialized to the same value as No_Element.

59/2 The predefined "=" operator for type Cursor returns True if both cursors are No_Element, or designate the same element in the same container.

60/2 Execution of the default implementation of the Input, Output, Read, or Write attribute of type Cursor raises Program_Error.

60.1/3 List'Write writes the length of List elements to the stream. List'Read reads the length of List elements from the stream.

61/2 Some operations of this generic package have access-to-subprogram parameters. To ensure such operations are well-defined, they guard against certain actions by the designated subprogram. In particular, some operations check for "tampering with cursors" of a container because they depend on the set of elements of the container remaining constant, and others check for "tampering with elements" of a container because they depend on elements of the container not being replaced.

A subprogram is said to *tamper with cursors* of a list object *L* if:

- it inserts or deletes elements of *L*, that is, it calls the Insert, Clear, Delete, or Delete_Last procedures with *L* as a parameter; or 62/2 63/2
- it reorders the elements of *L*, that is, it calls the Splice, Swap_Links, or Reverse_Elements procedures or the Sort or Merge procedures of an instance of Generic_Sorting with *L* as a parameter; or 64/2
- it finalizes *L*; or 65/2
- it calls the Assign procedure with *L* as the Target parameter; or 65.1/3
- it calls the Move procedure with *L* as a parameter. 66/2

A subprogram is said to *tamper with elements* of a list object *L* if:

- it tampers with cursors of *L*; or 68/2
- it replaces one or more elements of *L*, that is, it calls the Replace_Element or Swap procedures with *L* as a parameter. 69/2

function "=" (Left, Right : List) **return** Boolean; 70/2

If Left and Right denote the same list object, then the function returns True. If Left and Right have different lengths, then the function returns False. Otherwise, it compares each element in Left to the corresponding element in Right using the generic formal equality operator. If any such comparison returns False, the function returns False; otherwise it returns True. Any exception raised during evaluation of element equality is propagated. 71/2

function Length (Container : List) **return** Count_Type; 72/2

Returns the number of elements in Container. 73/2

function Is_Empty (Container : List) **return** Boolean; 74/2

Equivalent to Length (Container) = 0. 75/2

procedure Clear (Container : **in out** List); 76/2

Removes all the elements from Container. 77/2

function Element (Position : Cursor) **return** Element_Type; 78/2

If Position equals No_Element, then Constraint_Error is propagated. Otherwise, Element returns the element designated by Position. 79/2

procedure Replace_Element (Container : **in out** List;
Position : **in** Cursor;
New_Item : **in** Element_Type); 80/2

If Position equals No_Element, then Constraint_Error is propagated; if Position does not designate an element in Container, then Program_Error is propagated. Otherwise Replace_Element assigns the value New_Item to the element designated by Position. 81/2

procedure Query_Element
(Position : **in** Cursor;
Process : **not null access procedure** (Element : **in** Element_Type)); 82/2

If Position equals No_Element, then Constraint_Error is propagated. Otherwise, Query_Element calls Process.all with the element designated by Position as the argument. Program_Error is propagated if Process.all tampers with the elements of Container. Any exception raised by Process.all is propagated. 83/2

84/2 **procedure** Update_Element
 (Container : **in out** List;
 Position : **in** Cursor;
 Process : **not null access procedure** (Element : **in out** Element_Type));

85/2 If Position equals No_Element, then Constraint_Error is propagated; if Position does not designate an element in Container, then Program_Error is propagated. Otherwise Update_Element calls Process.all with the element designated by Position as the argument. Program_Error is propagated if Process.all tampers with the elements of Container. Any exception raised by Process.all is propagated.

86/2 If Element_Type is unconstrained and definite, then the actual Element parameter of Process.all shall be unconstrained.

86.1/3 **procedure** Assign (Target : **in out** List;
 Source : **in** List);

86.2/3 If Target denotes the same object as Source, the operation has no effect. Otherwise, it clears Target, and each element of Source is assigned to the corresponding element of Target.

86.3/3 **function** Copy (Source : List) **return** List;

86.4/3 Returns a list whose elements match the elements of Source.

87/2 **procedure** Move (Target : **in out** List;
 Source : **in out** List);

88/3 If Target denotes the same object as Source, then Move has no effect. Otherwise, equivalent to Assign (Target, Source) followed by Clear (Source).

89/2 **procedure** Insert (Container : **in out** List;
 Before : **in** Cursor;
 New_Item : **in** Element_Type;
 Count : **in** Count_Type := 1);

90/2 If Before is not No_Element, and does not designate an element in Container, then Program_Error is propagated. Otherwise, Insert inserts Count copies of New_Item prior to the element designated by Before. If Before equals No_Element, the new elements are inserted after the last node (if any). Any exception raised during allocation of internal storage is propagated, and Container is not modified.

91/2 **procedure** Insert (Container : **in out** List;
 Before : **in** Cursor;
 New_Item : **in** Element_Type;
 Position : **out** Cursor;
 Count : **in** Count_Type := 1);

92/2 If Before is not No_Element, and does not designate an element in Container, then Program_Error is propagated. Otherwise, Insert allocates Count copies of New_Item, and inserts them prior to the element designated by Before. If Before equals No_Element, the new elements are inserted after the last element (if any). Position designates the first newly-inserted element. Any exception raised during allocation of internal storage is propagated, and Container is not modified.

93/2 **procedure** Insert (Container : **in out** List;
 Before : **in** Cursor;
 Position : **out** Cursor;
 Count : **in** Count_Type := 1);

94/2 If Before is not No_Element, and does not designate an element in Container, then Program_Error is propagated. Otherwise, Insert inserts Count new elements prior to the element

designated by Before. If Before equals No_Element, the new elements are inserted after the last node (if any). The new elements are initialized by default (see 3.3.1). Any exception raised during allocation of internal storage is propagated, and Container is not modified.

```
procedure Prepend (Container : in out List;
                   New_Item  : in      Element_Type;
                   Count      : in      Count_Type := 1);
```

Equivalent to Insert (Container, First (Container), New_Item, Count). 96/2

```
procedure Append (Container : in out List;
                  New_Item  : in      Element_Type;
                  Count      : in      Count_Type := 1);
```

Equivalent to Insert (Container, No_Element, New_Item, Count). 98/2

```
procedure Delete (Container : in out List;
                  Position  : in out Cursor;
                  Count      : in      Count_Type := 1);
```

If Position equals No_Element, then Constraint_Error is propagated. If Position does not designate an element in Container, then Program_Error is propagated. Otherwise Delete removes (from Container) Count elements starting at the element designated by Position (or all of the elements starting at Position if there are fewer than Count elements starting at Position). Finally, Position is set to No_Element. 100/2

```
procedure Delete_First (Container : in out List;
                       Count       : in      Count_Type := 1);
```

Equivalent to Delete (Container, First (Container), Count). 102/2

```
procedure Delete_Last (Container : in out List;
                      Count       : in      Count_Type := 1);
```

If Length (Container) <= Count then Delete_Last is equivalent to Clear (Container). Otherwise it removes the last Count nodes from Container. 104/2

```
procedure Reverse_Elements (Container : in out List);
```

Reorders the elements of Container in reverse order. 106/2

```
procedure Swap (Container : in out List;
                I, J       : in      Cursor);
```

If either I or J is No_Element, then Constraint_Error is propagated. If either I or J do not designate an element in Container, then Program_Error is propagated. Otherwise, Swap exchanges the values of the elements designated by I and J. 108/2

```
procedure Swap_Links (Container : in out List;
                     I, J       : in      Cursor);
```

If either I or J is No_Element, then Constraint_Error is propagated. If either I or J do not designate an element in Container, then Program_Error is propagated. Otherwise, Swap_Links exchanges the nodes designated by I and J. 110/2

```
procedure Splice (Target   : in out List;
                 Before    : in      Cursor;
                 Source     : in out List);
```

If Before is not No_Element, and does not designate an element in Target, then Program_Error is propagated. Otherwise, if Source denotes the same object as Target, the operation has no effect. Otherwise, Splice reorders elements such that they are removed from Source and moved 112/2

to Target, immediately prior to Before. If Before equals No_Element, the nodes of Source are spliced after the last node of Target. The length of Target is incremented by the number of nodes in Source, and the length of Source is set to 0.

```
113/2  procedure Splice (Target   : in out List;  
                  Before   : in   Cursor;  
                  Source   : in out List;  
                  Position : in out Cursor);
```

114/2 If Position is No_Element then Constraint_Error is propagated. If Before does not equal No_Element, and does not designate an element in Target, then Program_Error is propagated. If Position does not equal No_Element, and does not designate a node in Source, then Program_Error is propagated. If Source denotes the same object as Target, then there is no effect if Position equals Before, else the element designated by Position is moved immediately prior to Before, or, if Before equals No_Element, after the last element. In both cases, Position and the length of Target are unchanged. Otherwise the element designated by Position is removed from Source and moved to Target, immediately prior to Before, or, if Before equals No_Element, after the last element of Target. The length of Target is incremented, the length of Source is decremented, and Position is updated to represent an element in Target.

```
115/2  procedure Splice (Container: in out List;  
                  Before   : in   Cursor;  
                  Position : in   Cursor);
```

116/2 If Position is No_Element then Constraint_Error is propagated. If Before does not equal No_Element, and does not designate an element in Container, then Program_Error is propagated. If Position does not equal No_Element, and does not designate a node in Container, then Program_Error is propagated. If Position equals Before there is no effect. Otherwise, the element designated by Position is moved immediately prior to Before, or, if Before equals No_Element, after the last element. The length of Container is unchanged.

```
117/2  function First (Container : List) return Cursor;
```

118/2 If Container is empty, First returns the value No_Element. Otherwise it returns a cursor that designates the first node in Container.

```
119/2  function First_Element (Container : List) return Element_Type;
```

120/2 Equivalent to Element (First (Container)).

```
121/2  function Last (Container : List) return Cursor;
```

122/2 If Container is empty, Last returns the value No_Element. Otherwise it returns a cursor that designates the last node in Container.

```
123/2  function Last_Element (Container : List) return Element_Type;
```

124/2 Equivalent to Element (Last (Container)).

```
125/2  function Next (Position : Cursor) return Cursor;
```

126/2 If Position equals No_Element or designates the last element of the container, then Next returns the value No_Element. Otherwise, it returns a cursor that designates the successor of the element designated by Position.

function Previous (Position : Cursor) return Cursor;	127/2
If Position equals No_Element or designates the first element of the container, then Previous returns the value No_Element. Otherwise, it returns a cursor that designates the predecessor of the element designated by Position.	128/2
procedure Next (Position : in out Cursor);	129/2
Equivalent to Position := Next (Position).	130/2
procedure Previous (Position : in out Cursor);	131/2
Equivalent to Position := Previous (Position).	132/2
function Find (Container : List; Item : Element_Type; Position : Cursor := No_Element) return Cursor;	133/2
If Position is not No_Element, and does not designate an element in Container, then Program_Error is propagated. Find searches the elements of Container for an element equal to Item (using the generic formal equality operator). The search starts at the element designated by Position, or at the first element if Position equals No_Element. It proceeds towards Last (Container). If no equal element is found, then Find returns No_Element. Otherwise, it returns a cursor designating the first equal element encountered.	134/2
function Reverse_Find (Container : List; Item : Element_Type; Position : Cursor := No_Element) return Cursor;	135/2
If Position is not No_Element, and does not designate an element in Container, then Program_Error is propagated. Find searches the elements of Container for an element equal to Item (using the generic formal equality operator). The search starts at the element designated by Position, or at the last element if Position equals No_Element. It proceeds towards First (Container). If no equal element is found, then Reverse_Find returns No_Element. Otherwise, it returns a cursor designating the first equal element encountered.	136/2
function Contains (Container : List; Item : Element_Type) return Boolean;	137/2
Equivalent to Find (Container, Item) /= No_Element.	138/2
function Has_Element (Position : Cursor) return Boolean;	139/2
Returns True if Position designates an element, and returns False otherwise.	140/2
procedure Iterate (Container : in List; Process : not null access procedure (Position : in Cursor));	141/2
Iterate calls Process.all with a cursor that designates each node in Container, starting with the first node and moving the cursor as per the Next function. Program_Error is propagated if Process.all tampers with the cursors of Container. Any exception raised by Process.all is propagated.	142/2

- 143/2 **procedure** Reverse_Iterate
 (Container : **in** List;
 Process : **not null access procedure** (Position : **in** Cursor));
- 144/2 Iterates over the nodes in Container as per Iterate, except that elements are traversed in reverse order, starting with the last node and moving the cursor as per the Previous function.
- 145/3 The actual function for the generic formal function "<" of Generic_Sorting is expected to return the same value each time it is called with a particular pair of element values. It should define a strict weak ordering relationship (see A.18); it should not modify Container. If the actual for "<" behaves in some other manner, the behavior of the subprograms of Generic_Sorting are unspecified. How many times the subprograms of Generic_Sorting call "<" is unspecified.
- 146/2 **function** Is_Sorted (Container : List) **return** Boolean;
- 147/2 Returns True if the elements are sorted smallest first as determined by the generic formal "<" operator; otherwise, Is_Sorted returns False. Any exception raised during evaluation of "<" is propagated.
- 148/2 **procedure** Sort (Container : **in out** List);
- 149/2 Reorders the nodes of Container such that the elements are sorted smallest first as determined by the generic formal "<" operator provided. The sort is stable. Any exception raised during evaluation of "<" is propagated.
- 150/2 **procedure** Merge (Target : **in out** List;
 Source : **in out** List);
- 151/2 Merge removes elements from Source and inserts them into Target; afterwards, Target contains the union of the elements that were initially in Source and Target; Source is left empty. If Target and Source are initially sorted smallest first, then Target is ordered smallest first as determined by the generic formal "<" operator; otherwise, the order of elements in Target is unspecified. Any exception raised during evaluation of "<" is propagated.

Bounded (Run-Time) Errors

- 152/2 Calling Merge in an instance of Generic_Sorting with either Source or Target not ordered smallest first using the provided generic formal "<" operator is a bounded error. Either Program_Error is raised after Target is updated as described for Merge, or the operation works as defined.
- 152.1/3 It is a bounded error for the actual function associated with a generic formal subprogram, when called as part of an operation of this package, to tamper with elements of any List parameter to the operation. Either Program_Error is raised, or the operation works as defined on the value of the List either prior to, or subsequent to, some or all of the modifications to the List.
- 152.2/3 It is a bounded error to call any subprogram declared in the visible part of Containers.Doubly_Linked_List when the associated container has been finalized. If the operation takes Container as an **in out** parameter, then it raises Constraint_Error or Program_Error. Otherwise, the operation either proceeds as it would for an empty container, or it raises Constraint_Error or Program_Error.

Erroneous Execution

- 153/2 A Cursor value is *invalid* if any of the following have occurred since it was created:
- 154/2 • The list that contains the element it designates has been finalized;
- 154.1/3 • The list that contains the element it designates has been used as the Target of a call to Assign, or as the target of an assignment_statement;

- The list that contains the element it designates has been used as the Source or Target of a call to Move; or 155/2
- The element it designates has been removed from the list that contains the element. 156/3

The result of "=" or Has_Element is unspecified if it is called with an invalid cursor parameter. Execution is erroneous if any other subprogram declared in Containers.Doubly_Linked_Lists is called with an invalid cursor parameter. 157/2

Implementation Requirements

No storage associated with a doubly-linked List object shall be lost upon assignment or scope exit. 158/2

The execution of an assignment_statement for a list shall have the effect of copying the elements from the source list object to the target list object. 159/2

Implementation Advice

Containers.Doubly_Linked_Lists should be implemented similarly to a linked list. In particular, if N is the length of a list, then the worst-case time complexity of Element, Insert with Count=1, and Delete with Count=1 should be $O(\log N)$. 160/2

The worst-case time complexity of a call on procedure Sort of an instance of Containers.Doubly_Linked_Lists.Generic_Sorting should be $O(N^2)$, and the average time complexity should be better than $O(N^2)$. 161/2

Move should not copy elements, and should minimize copying of internal data structures. 162/2

If an exception is propagated from a list operation, no storage should be lost, nor any elements removed from a list unless specified by the operation. 163/2

NOTES

- 46 Sorting a list never copies elements, and is a stable sort (equal elements remain in the original order). This is different than sorting an array or vector, which may need to copy elements, and is probably not a stable sort. 164/2

A.18.4 Maps

The language-defined generic packages Containers.Hashed_Maps and Containers.Ordered_Maps provide private types Map and Cursor, and a set of operations for each type. A map container allows an arbitrary type to be used as a key to find the element associated with that key. A hashed map uses a hash function to organize the keys, while an ordered map orders the keys per a specified relation. 1/2

This section describes the declarations that are common to both kinds of maps. See A.18.5 for a description of the semantics specific to Containers.Hashed_Maps and A.18.6 for a description of the semantics specific to Containers.Ordered_Maps. 2/2

Static Semantics

The actual function for the generic formal function "=" on Element_Type values is expected to define a reflexive and symmetric relationship and return the same result value each time it is called with a particular pair of values. If it behaves in some other manner, the function "=" on map values returns an unspecified value. The exact arguments and number of calls of this generic formal function by the function "=" on map values are unspecified. 3/2

The type Map is used to represent maps. The type Map needs finalization (see 7.6). 4/2

- 5/2 A map contains pairs of keys and elements, called *nodes*. Map cursors designate nodes, but also can be thought of as designating an element (the element contained in the node) for consistency with the other containers. There exists an equivalence relation on keys, whose definition is different for hashed maps and ordered maps. A map never contains two or more nodes with equivalent keys. The *length* of a map is the number of nodes it contains.
- 6/2 Each nonempty map has two particular nodes called the *first node* and the *last node* (which may be the same). Each node except for the last node has a *successor node*. If there are no other intervening operations, starting with the first node and repeatedly going to the successor node will visit each node in the map exactly once until the last node is reached. The exact definition of these terms is different for hashed maps and ordered maps.
- 7/2 Some operations of these generic packages have access-to-subprogram parameters. To ensure such operations are well-defined, they guard against certain actions by the designated subprogram. In particular, some operations check for “tampering with cursors” of a container because they depend on the set of elements of the container remaining constant, and others check for “tampering with elements” of a container because they depend on elements of the container not being replaced.
- 8/2 A subprogram is said to *tamper with cursors* of a map object *M* if:
- 9/2 • it inserts or deletes elements of *M*, that is, it calls the Insert, Include, Clear, Delete, or Exclude procedures with *M* as a parameter; or
- 10/2 • it finalizes *M*; or
- 10.1/3 • it calls the Assign procedure with *M* as the Target parameter; or
- 11/2 • it calls the Move procedure with *M* as a parameter; or
- 12/2 • it calls one of the operations defined to tamper with the cursors of *M*.
- 13/2 A subprogram is said to *tamper with elements* of a map object *M* if:
- 14/2 • it tampers with cursors of *M*; or
- 15/2 • it replaces one or more elements of *M*, that is, it calls the Replace or Replace_Element procedures with *M* as a parameter.
- 16/2 Empty_Map represents the empty Map object. It has a length of 0. If an object of type Map is not otherwise initialized, it is initialized to the same value as Empty_Map.
- 17/2 No_Element represents a cursor that designates no node. If an object of type Cursor is not otherwise initialized, it is initialized to the same value as No_Element.
- 18/2 The predefined “=” operator for type Cursor returns True if both cursors are No_Element, or designate the same element in the same container.
- 19/2 Execution of the default implementation of the Input, Output, Read, or Write attribute of type Cursor raises Program_Error.
- 19.1/3 Map'Write writes the length of Map elements to the stream. Map'Read reads the length of Map elements from the stream.
- 20/2 **function** “=” (Left, Right : Map) **return** Boolean;
- 21/2 If Left and Right denote the same map object, then the function returns True. If Left and Right have different lengths, then the function returns False. Otherwise, for each key *K* in Left, the function returns False if:
- 22/2 • a key equivalent to *K* is not present in Right; or

- the element associated with *K* in Left is not equal to the element associated with *K* in Right (using the generic formal equality operator for elements). 23/2

If the function has not returned a result after checking all of the keys, it returns True. Any exception raised during evaluation of key equivalence or element equality is propagated. 24/2

function Length (Container : Map) **return** Count_Type; 25/2

Returns the number of nodes in Container. 26/2

function Is_Empty (Container : Map) **return** Boolean; 27/2

Equivalent to Length (Container) = 0. 28/2

procedure Clear (Container : **in out** Map); 29/2

Removes all the nodes from Container. 30/2

function Key (Position : Cursor) **return** Key_Type; 31/2

If Position equals No_Element, then Constraint_Error is propagated. Otherwise, Key returns the key component of the node designated by Position. 32/2

function Element (Position : Cursor) **return** Element_Type; 33/2

If Position equals No_Element, then Constraint_Error is propagated. Otherwise, Element returns the element component of the node designated by Position. 34/2

procedure Replace_Element (Container : **in out** Map;
Position : **in** Cursor;
New_Item : **in** Element_Type); 35/2

If Position equals No_Element, then Constraint_Error is propagated; if Position does not designate an element in Container, then Program_Error is propagated. Otherwise Replace_Element assigns New_Item to the element of the node designated by Position. 36/2

procedure Query_Element
(Position : **in** Cursor;
Process : **not null access procedure** (Key : **in** Key_Type;
Element : **in** Element_Type)); 37/2

If Position equals No_Element, then Constraint_Error is propagated. Otherwise, Query_Element calls Process.all with the key and element from the node designated by Position as the arguments. Program_Error is propagated if Process.all tampers with the elements of Container. Any exception raised by Process.all is propagated. 38/2

procedure Update_Element
(Container : **in out** Map;
Position : **in** Cursor;
Process : **not null access procedure** (Key : **in** Key_Type;
Element : **in out** Element_Type)); 39/2

If Position equals No_Element, then Constraint_Error is propagated; if Position does not designate an element in Container, then Program_Error is propagated. Otherwise Update_Element calls Process.all with the key and element from the node designated by Position as the arguments. Program_Error is propagated if Process.all tampers with the elements of Container. Any exception raised by Process.all is propagated. 40/2

If Element_Type is unconstrained and definite, then the actual Element parameter of Process.all shall be unconstrained. 41/2

- 41.1/3 **procedure** Assign (Target : **in out** Map; Source : **in** Map);
- 41.2/3 If Target denotes the same object as Source, the operation has no effect. Otherwise, each key/element pair of Source is assigned to the corresponding key/element pair of Target.
- 42/2 **procedure** Move (Target : **in out** Map;
 Source : **in out** Map);
- 43/3 If Target denotes the same object as Source, then Move has no effect. Otherwise, equivalent to Assign (Target, Source) followed by Clear (Source).
- 44/2 **procedure** Insert (Container : **in out** Map;
 Key : **in** Key_Type;
 New_Item : **in** Element_Type;
 Position : **out** Cursor;
 Inserted : **out** Boolean);
- 45/2 Insert checks if a node with a key equivalent to Key is already present in Container. If a match is found, Inserted is set to False and Position designates the element with the matching key. Otherwise, Insert allocates a new node, initializes it to Key and New_Item, and adds it to Container; Inserted is set to True and Position designates the newly-inserted node. Any exception raised during allocation is propagated and Container is not modified.
- 46/2 **procedure** Insert (Container : **in out** Map;
 Key : **in** Key_Type;
 Position : **out** Cursor;
 Inserted : **out** Boolean);
- 47/2 Insert inserts Key into Container as per the five-parameter Insert, with the difference that an element initialized by default (see 3.3.1) is inserted.
- 48/2 **procedure** Insert (Container : **in out** Map;
 Key : **in** Key_Type;
 New_Item : **in** Element_Type);
- 49/2 Insert inserts Key and New_Item into Container as per the five-parameter Insert, with the difference that if a node with a key equivalent to Key is already in the map, then Constraint_Error is propagated.
- 50/2 **procedure** Include (Container : **in out** Map;
 Key : **in** Key_Type;
 New_Item : **in** Element_Type);
- 51/2 Include inserts Key and New_Item into Container as per the five-parameter Insert, with the difference that if a node with a key equivalent to Key is already in the map, then this operation assigns Key and New_Item to the matching node. Any exception raised during assignment is propagated.
- 52/2 **procedure** Replace (Container : **in out** Map;
 Key : **in** Key_Type;
 New_Item : **in** Element_Type);
- 53/2 Replace checks if a node with a key equivalent to Key is present in Container. If a match is found, Replace assigns Key and New_Item to the matching node; otherwise, Constraint_Error is propagated.
- 54/2 **procedure** Exclude (Container : **in out** Map;
 Key : **in** Key_Type);
- 55/2 Exclude checks if a node with a key equivalent to Key is present in Container. If a match is found, Exclude removes the node from the map.

procedure Delete (Container : in out Map; Key : in Key_Type);	56/2
Delete checks if a node with a key equivalent to Key is present in Container. If a match is found, Delete removes the node from the map; otherwise, Constraint_Error is propagated.	57/2
procedure Delete (Container : in out Map; Position : in out Cursor);	58/2
If Position equals No_Element, then Constraint_Error is propagated. If Position does not designate an element in Container, then Program_Error is propagated. Otherwise, Delete removes the node designated by Position from the map. Position is set to No_Element on return.	59/2
function First (Container : Map) return Cursor;	60/2
If Length (Container) = 0, then First returns No_Element. Otherwise, First returns a cursor that designates the first node in Container.	61/2
function Next (Position : Cursor) return Cursor;	62/2
Returns a cursor that designates the successor of the node designated by Position. If Position designates the last node, then No_Element is returned. If Position equals No_Element, then No_Element is returned.	63/2
procedure Next (Position : in out Cursor);	64/2
Equivalent to Position := Next (Position).	65/2
function Find (Container : Map; Key : Key_Type) return Cursor;	66/2
If Length (Container) equals 0, then Find returns No_Element. Otherwise, Find checks if a node with a key equivalent to Key is present in Container. If a match is found, a cursor designating the matching node is returned; otherwise, No_Element is returned.	67/2
function Element (Container : Map; Key : Key_Type) return Element_Type;	68/2
Equivalent to Element (Find (Container, Key)).	69/2
function Contains (Container : Map; Key : Key_Type) return Boolean;	70/2
Equivalent to Find (Container, Key) /= No_Element.	71/2
function Has_Element (Position : Cursor) return Boolean;	72/2
Returns True if Position designates a node, and returns False otherwise.	73/2
procedure Iterate (Container : in Map; Process : not null access procedure (Position : in Cursor));	74/2
Iterate calls Process.all with a cursor that designates each node in Container, starting with the first node and moving the cursor according to the successor relation. Program_Error is propagated if Process.all tampers with the cursors of Container. Any exception raised by Process.all is propagated.	75/2

Bounded (Run-Time) Errors

It is a bounded error for the actual function associated with a generic formal subprogram, when called as part of an operation of a map package, to tamper with elements of any Map parameter to the operation. 75.1/3

Either Program_Error is raised, or the operation works as defined on the value of the map either prior to, or subsequent to, some or all of the modifications to the map.

- 75.2/3 It is a bounded error to call any subprogram declared in the visible part of a map package when the associated container has been finalized. If the operation takes Container as an **in out** parameter, then it raises Constraint_Error or Program_Error. Otherwise, the operation either proceeds as it would for an empty container, or it raises Constraint_Error or Program_Error.

Erroneous Execution

- 76/2 A Cursor value is *invalid* if any of the following have occurred since it was created:
- 77/2 • The map that contains the node it designates has been finalized;
 - 77.1/3 • The map that contains the node it designates has been used as the Target of a call to Assign, or as the target of an assignment_statement;
 - 78/2 • The map that contains the node it designates has been used as the Source or Target of a call to Move; or
 - 79/3 • The node it designates has been removed from the map that contains the node.
- 80/2 The result of "=" or Has_Element is unspecified if these functions are called with an invalid cursor parameter. Execution is erroneous if any other subprogram declared in Containers.Hashing_Maps or Containers.Ordered_Maps is called with an invalid cursor parameter.

Implementation Requirements

- 81/2 No storage associated with a Map object shall be lost upon assignment or scope exit.
- 82/2 The execution of an assignment_statement for a map shall have the effect of copying the elements from the source map object to the target map object.

Implementation Advice

- 83/2 Move should not copy elements, and should minimize copying of internal data structures.
- 84/2 If an exception is propagated from a map operation, no storage should be lost, nor any elements removed from a map unless specified by the operation.

A.18.5 The Package Containers.Hashing_Maps

Static Semantics

- 1/2 The generic library package Containers.Hashing_Maps has the following declaration:

```

2/3  generic
      type Key_Type is private;
      type Element_Type is private;
      with function Hash (Key : Key_Type) return Hash_Type;
      with function Equivalent_Keys (Left, Right : Key_Type)
        return Boolean;
      with function "=" (Left, Right : Element_Type)
        return Boolean is <>;
  package Ada.Containers.Hashing_Maps is
    pragma Preelaborate (Hashing_Maps);
    pragma Remote_Types (Hashing_Maps);

3/2    type Map is tagged private;
    pragma Preelaborable_Initialization (Map);

4/2    type Cursor is private;
    pragma Preelaborable_Initialization (Cursor);
```

```

Empty_Map : constant Map;
No_Element : constant Cursor;
function "=" (Left, Right : Map) return Boolean;
function Capacity (Container : Map) return Count_Type;
procedure Reserve_Capacity (Container : in out Map;
                           Capacity : in Count_Type);
function Length (Container : Map) return Count_Type;
function Is_Empty (Container : Map) return Boolean;
procedure Clear (Container : in out Map);
function Key (Position : Cursor) return Key_Type;
function Element (Position : Cursor) return Element_Type;
procedure Replace_Element (Container : in out Map;
                          Position : in Cursor;
                          New_Item : in Element_Type);

procedure Query_Element
(Position : in Cursor;
 Process : not null access procedure (Key : in Key_Type;
                                     Element : in Element_Type));

procedure Update_Element
(Container : in out Map;
 Position : in Cursor;
 Process : not null access procedure
 (Key : in Key_Type;
  Element : in out Element_Type));

procedure Assign (Target : in out Map; Source : in Map);
function Copy (Source : Map; Capacity : Count_Type := 0) return Map;
procedure Move (Target : in out Map;
               Source : in out Map);

procedure Insert (Container : in out Map;
                 Key : in Key_Type;
                 New_Item : in Element_Type;
                 Position : out Cursor;
                 Inserted : out Boolean);

procedure Insert (Container : in out Map;
                 Key : in Key_Type;
                 Position : out Cursor;
                 Inserted : out Boolean);

procedure Insert (Container : in out Map;
                 Key : in Key_Type;
                 New_Item : in Element_Type);

procedure Include (Container : in out Map;
                  Key : in Key_Type;
                  New_Item : in Element_Type);

procedure Replace (Container : in out Map;
                  Key : in Key_Type;
                  New_Item : in Element_Type);

procedure Exclude (Container : in out Map;
                  Key : in Key_Type);

procedure Delete (Container : in out Map;
                  Key : in Key_Type);

procedure Delete (Container : in out Map;
                  Position : in out Cursor);

function First (Container : Map)
return Cursor;

```

```

28/2     function Next (Position : Cursor) return Cursor;
29/2     procedure Next (Position : in out Cursor);
30/2     function Find (Container : Map;
                      Key       : Key_Type)
          return Cursor;
31/2     function Element (Container : Map;
                        Key       : Key_Type)
          return Element_Type;
32/2     function Contains (Container : Map;
                        Key       : Key_Type) return Boolean;
33/2     function Has_Element (Position : Cursor) return Boolean;
34/2     function Equivalent_Keys (Left, Right : Cursor)
          return Boolean;
35/2     function Equivalent_Keys (Left  : Cursor;
                                Right : Key_Type)
          return Boolean;
36/2     function Equivalent_Keys (Left  : Key_Type;
                                Right : Cursor)
          return Boolean;
37/2     procedure Iterate
          (Container : in Map;
           Process   : not null access procedure (Position : in Cursor));
38/2     private
39/2         ... -- not specified by the language
40/2     end Ada.Containers.Hashed_Maps;

```

41/2 An object of type Map contains an expandable hash table, which is used to provide direct access to nodes. The *capacity* of an object of type Map is the maximum number of nodes that can be inserted into the hash table prior to it being automatically expanded.

42/2 Two keys *K1* and *K2* are defined to be *equivalent* if Equivalent_Keys (*K1*, *K2*) returns True.

43/2 The actual function for the generic formal function Hash is expected to return the same value each time it is called with a particular key value. For any two equivalent key values, the actual for Hash is expected to return the same value. If the actual for Hash behaves in some other manner, the behavior of this package is unspecified. Which subprograms of this package call Hash, and how many times they call it, is unspecified.

44/2 The actual function for the generic formal function Equivalent_Keys on Key_Type values is expected to return the same value each time it is called with a particular pair of key values. It should define an equivalence relationship, that is, be reflexive, symmetric, and transitive. If the actual for Equivalent_Keys behaves in some other manner, the behavior of this package is unspecified. Which subprograms of this package call Equivalent_Keys, and how many times they call it, is unspecified.

45/2 If the value of a key stored in a node of a map is changed other than by an operation in this package such that at least one of Hash or Equivalent_Keys give different results, the behavior of this package is unspecified.

46/2 Which nodes are the first node and the last node of a map, and which node is the successor of a given node, are unspecified, other than the general semantics described in A.18.4.

```

47/2     function Capacity (Container : Map) return Count_Type;

```

48/2 Returns the capacity of Container.

procedure Reserve_Capacity (Container : in out Map; Capacity : in Count_Type);	49/2
Reserve_Capacity allocates a new hash table such that the length of the resulting map can become at least the value Capacity without requiring an additional call to Reserve_Capacity, and is large enough to hold the current length of Container. Reserve_Capacity then rehashes the nodes in Container onto the new hash table. It replaces the old hash table with the new hash table, and then deallocates the old hash table. Any exception raised during allocation is propagated and Container is not modified.	50/2
Reserve_Capacity tampers with the cursors of Container.	51/2
procedure Clear (Container : in out Map);	52/2
In addition to the semantics described in A.18.4, Clear does not affect the capacity of Container.	53/2
procedure Assign (Target : in out Map; Source : in Map);	53.1/3
In addition to the semantics described in A.18.4, if the length of Source is greater than the capacity of Target, Reserve_Capacity is called with the length of Source as the capacity before assigning any elements.	53.2/3
function Copy (Source : Map; Capacity : Count_Type := 0) return Map;	53.3/3
Returns a map whose keys and elements are initialized from the keys and elements of Source. If Capacity is 0, then the map capacity is the length of Source; if Capacity is equal to or greater than the length of Source, the map capacity is at least the specified value. Otherwise, the operation propagates Capacity_Error.	53.4/3
procedure Insert (Container : in out Map; Key : in Key_Type; New_Item : in Element_Type; Position : out Cursor; Inserted : out Boolean);	54/2
In addition to the semantics described in A.18.4, if Length (Container) equals Capacity (Container), then Insert first calls Reserve_Capacity to increase the capacity of Container to some larger value.	55/2
function Equivalent_Keys (Left, Right : Cursor) return Boolean;	56/2
Equivalent to Equivalent_Keys (Key (Left), Key (Right)).	57/2
function Equivalent_Keys (Left : Cursor; Right : Key_Type) return Boolean;	58/2
Equivalent to Equivalent_Keys (Key (Left), Right).	59/2
function Equivalent_Keys (Left : Key_Type; Right : Cursor) return Boolean;	60/2
Equivalent to Equivalent_Keys (Left, Key (Right)).	61/2

Implementation Advice

If N is the length of a map, the average time complexity of the subprograms Element, Insert, Include, Replace, Delete, Exclude and Find that take a key parameter should be $O(\log N)$. The average time complexity of the subprograms that take a cursor parameter should be $O(1)$. The average time complexity of Reserve_Capacity should be $O(N)$.

A.18.6 The Package Containers.Ordered_Maps

Static Semantics

1/2 The generic library package Containers.Ordered_Maps has the following declaration:

```

2/3  generic
      type Key_Type is private;
      type Element_Type is private;
      with function "<" (Left, Right : Key_Type) return Boolean is <>;
      with function "=" (Left, Right : Element_Type) return Boolean is <>;
package Ada.Containers.Ordered_Maps is
      pragma Preelaborate(Ordered_Maps);
      pragma Remote_Types(Ordered_Maps);

3/2  function Equivalent_Keys (Left, Right : Key_Type) return Boolean;

4/2  type Map is tagged private;
      pragma Preelaborable_Initialization(Map);

5/2  type Cursor is private;
      pragma Preelaborable_Initialization(Cursor);

6/2  Empty_Map : constant Map;

7/2  No_Element : constant Cursor;

8/2  function "=" (Left, Right : Map) return Boolean;

9/2  function Length (Container : Map) return Count_Type;

10/2 function Is_Empty (Container : Map) return Boolean;

11/2 procedure Clear (Container : in out Map);

12/2 function Key (Position : Cursor) return Key_Type;

13/2 function Element (Position : Cursor) return Element_Type;

14/2 procedure Replace_Element (Container : in out Map;
                             Position : in Cursor;
                             New_Item : in Element_Type);

15/2 procedure Query_Element
      (Position : in Cursor;
       Process : not null access procedure (Key : in Key_Type;
                                           Element : in Element_Type));

16/2 procedure Update_Element
      (Container : in out Map;
       Position : in Cursor;
       Process : not null access procedure
           (Key : in Key_Type;
            Element : in out Element_Type));

16.1/3 procedure Assign (Target : in out Map; Source : in Map);

16.2/3 function Copy (Source : Map) return Map;

17/2 procedure Move (Target : in out Map;
                   Source : in out Map);

18/2 procedure Insert (Container : in out Map;
                     Key : in Key_Type;
                     New_Item : in Element_Type;
                     Position : out Cursor;
                     Inserted : out Boolean);

19/2 procedure Insert (Container : in out Map;
                     Key : in Key_Type;
                     Position : out Cursor;
                     Inserted : out Boolean);

20/2 procedure Insert (Container : in out Map;
                     Key : in Key_Type;
                     New_Item : in Element_Type);

```

```

procedure Include (Container : in out Map;                               21/2
                   Key       : in      Key_Type;
                   New_Item  : in      Element_Type);

procedure Replace (Container : in out Map;                               22/2
                   Key       : in      Key_Type;
                   New_Item  : in      Element_Type);

procedure Exclude (Container : in out Map;                               23/2
                   Key       : in      Key_Type);

procedure Delete (Container : in out Map;                               24/2
                   Key       : in      Key_Type);

procedure Delete (Container : in out Map;                               25/2
                   Position  : in out Cursor);

procedure Delete_First (Container : in out Map);                       26/2
procedure Delete_Last (Container : in out Map);                       27/2
function First (Container : Map) return Cursor;                       28/2
function First_Element (Container : Map) return Element_Type;         29/2
function First_Key (Container : Map) return Key_Type;                 30/2
function Last (Container : Map) return Cursor;                       31/2
function Last_Element (Container : Map) return Element_Type;         32/2
function Last_Key (Container : Map) return Key_Type;                 33/2
function Next (Position : Cursor) return Cursor;                   34/2
procedure Next (Position : in out Cursor);                           35/2
function Previous (Position : Cursor) return Cursor;               36/2
procedure Previous (Position : in out Cursor);                       37/2
function Find (Container : Map;                                         38/2
               Key       : Key_Type) return Cursor;

function Element (Container : Map;                                       39/2
                  Key       : Key_Type) return Element_Type;

function Floor (Container : Map;                                         40/2
               Key       : Key_Type) return Cursor;

function Ceiling (Container : Map;                                       41/2
                Key       : Key_Type) return Cursor;

function Contains (Container : Map;                                       42/2
                  Key       : Key_Type) return Boolean;

function Has_Element (Position : Cursor) return Boolean;             43/2
function "<" (Left, Right : Cursor) return Boolean;                   44/2
function ">" (Left, Right : Cursor) return Boolean;                   45/2
function "<" (Left : Cursor; Right : Key_Type) return Boolean;         46/2
function ">" (Left : Cursor; Right : Key_Type) return Boolean;         47/2
function "<" (Left : Key_Type; Right : Cursor) return Boolean;         48/2
function ">" (Left : Key_Type; Right : Cursor) return Boolean;         49/2

procedure Iterate (Container : in Map;                               50/2
                  Process   : not null access procedure (Position : in Cursor));

procedure Reverse_Iterate (Container : in Map;                       51/2
                          Process   : not null access procedure (Position : in Cursor));

private                                                         52/2
    ... -- not specified by the language                             53/2
end Ada.Containers.Ordered_Maps;                                   54/2

```

- 55/2 Two keys $K1$ and $K2$ are *equivalent* if both $K1 < K2$ and $K2 < K1$ return False, using the generic formal "<" operator for keys. Function Equivalent_Keys returns True if Left and Right are equivalent, and False otherwise.
- 56/3 The actual function for the generic formal function "<" on Key_Type values is expected to return the same value each time it is called with a particular pair of key values. It should define a strict weak ordering relationship (see A.18). If the actual for "<" behaves in some other manner, the behavior of this package is unspecified. Which subprograms of this package call "<" and how many times they call it, is unspecified.
- 57/2 If the value of a key stored in a map is changed other than by an operation in this package such that at least one of "<" or "=" give different results, the behavior of this package is unspecified.
- 58/2 The first node of a nonempty map is the one whose key is less than the key of all the other nodes in the map. The last node of a nonempty map is the one whose key is greater than the key of all the other elements in the map. The successor of a node is the node with the smallest key that is larger than the key of the given node. The predecessor of a node is the node with the largest key that is smaller than the key of the given node. All comparisons are done using the generic formal "<" operator for keys.
- 58.1/3 **function** Copy (Source : Map) **return** Map;
- 58.2/3 Returns a map whose keys and elements are initialized from the corresponding keys and elements of Source.
- 59/2 **procedure** Delete_First (Container : **in out** Map);
- 60/2 If Container is empty, Delete_First has no effect. Otherwise the node designated by First (Container) is removed from Container. Delete_First tampers with the cursors of Container.
- 61/2 **procedure** Delete_Last (Container : **in out** Map);
- 62/2 If Container is empty, Delete_Last has no effect. Otherwise the node designated by Last (Container) is removed from Container. Delete_Last tampers with the cursors of Container.
- 63/2 **function** First_Element (Container : Map) **return** Element_Type;
- 64/2 Equivalent to Element (First (Container)).
- 65/2 **function** First_Key (Container : Map) **return** Key_Type;
- 66/2 Equivalent to Key (First (Container)).
- 67/2 **function** Last (Container : Map) **return** Cursor;
- 68/2 Returns a cursor that designates the last node in Container. If Container is empty, returns No_Element.
- 69/2 **function** Last_Element (Container : Map) **return** Element_Type;
- 70/2 Equivalent to Element (Last (Container)).
- 71/2 **function** Last_Key (Container : Map) **return** Key_Type;
- 72/2 Equivalent to Key (Last (Container)).
- 73/2 **function** Previous (Position : Cursor) **return** Cursor;
- 74/2 If Position equals No_Element, then Previous returns No_Element. Otherwise Previous returns a cursor designating the node that precedes the one designated by Position. If Position designates the first element, then Previous returns No_Element.

procedure Previous (Position : in out Cursor);	75/2
Equivalent to Position := Previous (Position).	76/2
function Floor (Container : Map; Key : Key_Type) return Cursor;	77/2
Floor searches for the last node whose key is not greater than Key, using the generic formal "<" operator for keys. If such a node is found, a cursor that designates it is returned. Otherwise No_Element is returned.	78/2
function Ceiling (Container : Map; Key : Key_Type) return Cursor;	79/2
Ceiling searches for the first node whose key is not less than Key, using the generic formal "<" operator for keys. If such a node is found, a cursor that designates it is returned. Otherwise No_Element is returned.	80/2
function "<" (Left, Right : Cursor) return Boolean;	81/2
Equivalent to Key (Left) < Key (Right).	82/2
function ">" (Left, Right : Cursor) return Boolean;	83/2
Equivalent to Key (Right) < Key (Left).	84/2
function "<" (Left : Cursor; Right : Key_Type) return Boolean;	85/2
Equivalent to Key (Left) < Right.	86/2
function ">" (Left : Cursor; Right : Key_Type) return Boolean;	87/2
Equivalent to Right < Key (Left).	88/2
function "<" (Left : Key_Type; Right : Cursor) return Boolean;	89/2
Equivalent to Left < Key (Right).	90/2
function ">" (Left : Key_Type; Right : Cursor) return Boolean;	91/2
Equivalent to Key (Right) < Left.	92/2
procedure Reverse_Iterate (Container : in Map; Process : not null access procedure (Position : in Cursor));	93/2
Iterates over the nodes in Container as per Iterate, with the difference that the nodes are traversed in predecessor order, starting with the last node.	94/2

Implementation Advice

If N is the length of a map, then the worst-case time complexity of the Element, Insert, Include, Replace, Delete, Exclude and Find operations that take a key parameter should be $O((\log N)^2)$ or better. The worst-case time complexity of the subprograms that take a cursor parameter should be $O(1)$. 95/2

A.18.7 Sets

The language-defined generic packages Containers.Hash_Sets and Containers.Ordered_Sets provide private types Set and Cursor, and a set of operations for each type. A set container allows elements of an arbitrary type to be stored without duplication. A hashed set uses a hash function to organize elements, while an ordered set orders its element per a specified relation. 1/2

2/2 This section describes the declarations that are common to both kinds of sets. See A.18.8 for a description of the semantics specific to Containers.Hashing_Sets and A.18.9 for a description of the semantics specific to Containers.Ordered_Sets.

Static Semantics

3/2 The actual function for the generic formal function "=" on Element_Type values is expected to define a reflexive and symmetric relationship and return the same result value each time it is called with a particular pair of values. If it behaves in some other manner, the function "=" on set values returns an unspecified value. The exact arguments and number of calls of this generic formal function by the function "=" on set values are unspecified.

4/2 The type Set is used to represent sets. The type Set needs finalization (see 7.6).

5/2 A set contains elements. Set cursors designate elements. There exists an equivalence relation on elements, whose definition is different for hashed sets and ordered sets. A set never contains two or more equivalent elements. The *length* of a set is the number of elements it contains.

6/2 Each nonempty set has two particular elements called the *first element* and the *last element* (which may be the same). Each element except for the last element has a *successor element*. If there are no other intervening operations, starting with the first element and repeatedly going to the successor element will visit each element in the set exactly once until the last element is reached. The exact definition of these terms is different for hashed sets and ordered sets.

7/2 Some operations of these generic packages have access-to-subprogram parameters. To ensure such operations are well-defined, they guard against certain actions by the designated subprogram. In particular, some operations check for “tampering with cursors” of a container because they depend on the set of elements of the container remaining constant, and others check for “tampering with elements” of a container because they depend on elements of the container not being replaced.

8/2 A subprogram is said to *tamper with cursors* of a set object *S* if:

- 9/2 • it inserts or deletes elements of *S*, that is, it calls the Insert, Include, Clear, Delete, Exclude, or Replace_Element procedures with *S* as a parameter; or
- 10/2 • it finalizes *S*; or
- 10.1/3 • it calls the Assign procedure with *S* as the Target parameter; or
- 11/2 • it calls the Move procedure with *S* as a parameter; or
- 12/2 • it calls one of the operations defined to tamper with cursors of *S*.

13/2 A subprogram is said to *tamper with elements* of a set object *S* if:

- 14/2 • it tampers with cursors of *S*.

15/2 Empty_Set represents the empty Set object. It has a length of 0. If an object of type Set is not otherwise initialized, it is initialized to the same value as Empty_Set.

16/2 No_Element represents a cursor that designates no element. If an object of type Cursor is not otherwise initialized, it is initialized to the same value as No_Element.

17/2 The predefined "=" operator for type Cursor returns True if both cursors are No_Element, or designate the same element in the same container.

18/2 Execution of the default implementation of the Input, Output, Read, or Write attribute of type Cursor raises Program_Error.

Set'Write writes the length of Set elements to the stream. Set'Read reads the length of Set elements from the stream. 18.1/3

function "=" (Left, Right : Set) **return** Boolean; 19/2

If Left and Right denote the same set object, then the function returns True. If Left and Right have different lengths, then the function returns False. Otherwise, for each element *E* in Left, the function returns False if an element equal to *E* (using the generic formal equality operator) is not present in Right. If the function has not returned a result after checking all of the elements, it returns True. Any exception raised during evaluation of element equality is propagated. 20/2

function Equivalent_Sets (Left, Right : Set) **return** Boolean; 21/2

If Left and Right denote the same set object, then the function returns True. If Left and Right have different lengths, then the function returns False. Otherwise, for each element *E* in Left, the function returns False if an element equivalent to *E* is not present in Right. If the function has not returned a result after checking all of the elements, it returns True. Any exception raised during evaluation of element equivalence is propagated. 22/2

function To_Set (New_Item : Element_Type) **return** Set; 23/2

Returns a set containing the single element New_Item. 24/2

function Length (Container : Set) **return** Count_Type; 25/2

Returns the number of elements in Container. 26/2

function Is_Empty (Container : Set) **return** Boolean; 27/2

Equivalent to Length (Container) = 0. 28/2

procedure Clear (Container : **in out** Set); 29/2

Removes all the elements from Container. 30/2

function Element (Position : Cursor) **return** Element_Type; 31/2

If Position equals No_Element, then Constraint_Error is propagated. Otherwise, Element returns the element designated by Position. 32/2

procedure Replace_Element (Container : **in out** Set;
Position : **in** Cursor;
New_Item : **in** Element_Type); 33/2

If Position equals No_Element, then Constraint_Error is propagated; if Position does not designate an element in Container, then Program_Error is propagated. If an element equivalent to New_Item is already present in Container at a position other than Position, Program_Error is propagated. Otherwise, Replace_Element assigns New_Item to the element designated by Position. Any exception raised by the assignment is propagated. 34/2

procedure Query_Element 35/2

(Position : **in** Cursor;
Process : **not null access procedure** (Element : **in** Element_Type));

If Position equals No_Element, then Constraint_Error is propagated. Otherwise, Query_Element calls Process.all with the element designated by Position as the argument. Program_Error is propagated if Process.all tampers with the elements of Container. Any exception raised by Process.all is propagated. 36/2

- 36.1/3 **procedure** Assign (Target : **in out** Set; Source : **in** Set);
- 36.2/3 If Target denotes the same object as Source, the operation has no effect. Otherwise, each element of Source is assigned to the corresponding element of Target.
- 37/2 **procedure** Move (Target : **in out** Set;
 Source : **in out** Set);
- 38/3 If Target denotes the same object as Source, then Move has no effect. Otherwise, equivalent to Assign (Target, Source) followed by Clear (Source).
- 39/2 **procedure** Insert (Container : **in out** Set;
 New_Item : **in** Element_Type;
 Position : **out** Cursor;
 Inserted : **out** Boolean);
- 40/2 Insert checks if an element equivalent to New_Item is already present in Container. If a match is found, Inserted is set to False and Position designates the matching element. Otherwise, Insert adds New_Item to Container; Inserted is set to True and Position designates the newly-inserted element. Any exception raised during allocation is propagated and Container is not modified.
- 41/2 **procedure** Insert (Container : **in out** Set;
 New_Item : **in** Element_Type);
- 42/2 Insert inserts New_Item into Container as per the four-parameter Insert, with the difference that if an element equivalent to New_Item is already in the set, then Constraint_Error is propagated.
- 43/2 **procedure** Include (Container : **in out** Set;
 New_Item : **in** Element_Type);
- 44/2 Include inserts New_Item into Container as per the four-parameter Insert, with the difference that if an element equivalent to New_Item is already in the set, then it is replaced. Any exception raised during assignment is propagated.
- 45/2 **procedure** Replace (Container : **in out** Set;
 New_Item : **in** Element_Type);
- 46/2 Replace checks if an element equivalent to New_Item is already in the set. If a match is found, that element is replaced with New_Item; otherwise, Constraint_Error is propagated.
- 47/2 **procedure** Exclude (Container : **in out** Set;
 Item : **in** Element_Type);
- 48/2 Exclude checks if an element equivalent to Item is present in Container. If a match is found, Exclude removes the element from the set.
- 49/2 **procedure** Delete (Container : **in out** Set;
 Item : **in** Element_Type);
- 50/2 Delete checks if an element equivalent to Item is present in Container. If a match is found, Delete removes the element from the set; otherwise, Constraint_Error is propagated.
- 51/2 **procedure** Delete (Container : **in out** Set;
 Position : **in out** Cursor);
- 52/2 If Position equals No_Element, then Constraint_Error is propagated. If Position does not designate an element in Container, then Program_Error is propagated. Otherwise, Delete removes the element designated by Position from the set. Position is set to No_Element on return.

procedure Union (Target : in out Set; Source : in Set);	53/2
Union inserts into Target the elements of Source that are not equivalent to some element already in Target.	54/2
function Union (Left, Right : Set) return Set;	55/2
Returns a set comprising all of the elements of Left, and the elements of Right that are not equivalent to some element of Left.	56/2
procedure Intersection (Target : in out Set; Source : in Set);	57/2
Intersection deletes from Target the elements of Target that are not equivalent to some element of Source.	58/3
function Intersection (Left, Right : Set) return Set;	59/2
Returns a set comprising all the elements of Left that are equivalent to the some element of Right.	60/2
procedure Difference (Target : in out Set; Source : in Set);	61/2
If Target denotes the same object as Source, then Difference clears Target. Otherwise, it deletes from Target the elements that are equivalent to some element of Source.	62/2
function Difference (Left, Right : Set) return Set;	63/2
Returns a set comprising the elements of Left that are not equivalent to some element of Right.	64/2
procedure Symmetric_Difference (Target : in out Set; Source : in Set);	65/2
If Target denotes the same object as Source, then Symmetric_Difference clears Target. Otherwise, it deletes from Target the elements that are equivalent to some element of Source, and inserts into Target the elements of Source that are not equivalent to some element of Target.	66/2
function Symmetric_Difference (Left, Right : Set) return Set;	67/2
Returns a set comprising the elements of Left that are not equivalent to some element of Right, and the elements of Right that are not equivalent to some element of Left.	68/2
function Overlap (Left, Right : Set) return Boolean;	69/2
If an element of Left is equivalent to some element of Right, then Overlap returns True. Otherwise it returns False.	70/2
function Is_Subset (Subset : Set; Of_Set : Set) return Boolean;	71/2
If an element of Subset is not equivalent to some element of Of_Set, then Is_Subset returns False. Otherwise it returns True.	72/2
function First (Container : Set) return Cursor;	73/2
If Length (Container) = 0, then First returns No_Element. Otherwise, First returns a cursor that designates the first element in Container.	74/2

- 75/2 **function** Next (Position : Cursor) **return** Cursor;
- 76/2 Returns a cursor that designates the successor of the element designated by Position. If Position designates the last element, then No_Element is returned. If Position equals No_Element, then No_Element is returned.
- 77/2 **procedure** Next (Position : **in out** Cursor);
- 78/2 Equivalent to Position := Next (Position).
- 79/3 *This paragraph was deleted.*
- 80/2 **function** Find (Container : Set;
 Item : Element_Type) **return** Cursor;
- 81/2 If Length (Container) equals 0, then Find returns No_Element. Otherwise, Find checks if an element equivalent to Item is present in Container. If a match is found, a cursor designating the matching element is returned; otherwise, No_Element is returned.
- 82/2 **function** Contains (Container : Set;
 Item : Element_Type) **return** Boolean;
- 82.1/3 Equivalent to Find (Container, Item) /= No_Element.
- 83/2 **function** Has_Element (Position : Cursor) **return** Boolean;
- 84/2 Returns True if Position designates an element, and returns False otherwise.
- 85/2 **procedure** Iterate
 (Container : **in** Set;
 Process : **not null access procedure** (Position : **in** Cursor));
- 86/2 Iterate calls Process.all with a cursor that designates each element in Container, starting with the first element and moving the cursor according to the successor relation. Program_Error is propagated if Process.all tampers with the cursors of Container. Any exception raised by Process.all is propagated.
- 87/2 Both Containers.Hashed_Set and Containers.Ordered_Set declare a nested generic package Generic_Keys, which provides operations that allow set manipulation in terms of a key (typically, a portion of an element) instead of a complete element. The formal function Key of Generic_Keys extracts a key value from an element. It is expected to return the same value each time it is called with a particular element. The behavior of Generic_Keys is unspecified if Key behaves in some other manner.
- 88/2 A key is expected to unambiguously determine a single equivalence class for elements. The behavior of Generic_Keys is unspecified if the formal parameters of this package behave in some other manner.
- 89/2 **function** Key (Position : Cursor) **return** Key_Type;
- 90/2 Equivalent to Key (Element (Position)).
- 91/2 The subprograms in package Generic_Keys named Contains, Find, Element, Delete, and Exclude, are equivalent to the corresponding subprograms in the parent package, with the difference that the Key parameter is used to locate an element in the set.
- 92/2 **procedure** Replace (Container : **in out** Set;
 Key : **in** Key_Type;
 New_Item : **in** Element_Type);
- 93/2 Equivalent to Replace_Element (Container, Find (Container, Key), New_Item).

```

procedure Update_Element_Preserving_Key                                     94/2
(Container : in out Set;
 Position  : in      Cursor;
 Process   : not null access procedure
              (Element : in out Element_Type));

```

If Position equals No_Element, then Constraint_Error is propagated; if Position does not designate an element in Container, then Program_Error is propagated. Otherwise, Update_Element_Preserving_Key uses Key to save the key value *K* of the element designated by Position. Update_Element_Preserving_Key then calls Process.all with that element as the argument. Program_Error is propagated if Process.all tampers with the elements of Container. Any exception raised by Process.all is propagated. After Process.all returns, Update_Element_Preserving_Key checks if *K* determines the same equivalence class as that for the new element; if not, the element is removed from the set and Program_Error is propagated. 95/2

If Element_Type is unconstrained and definite, then the actual Element parameter of Process.all shall be unconstrained. 96/2

Bounded (Run-Time) Errors

It is a bounded error for the actual function associated with a generic formal subprogram, when called as part of an operation of a set package, to tamper with elements of any Set parameter to the operation. Either Program_Error is raised, or the operation works as defined on the value of the set either prior to, or subsequent to, some or all of the modifications to the set. 96.1/3

It is a bounded error to call any subprogram declared in the visible part of a set package when the associated container has been finalized. If the operation takes Container as an **in out** parameter, then it raises Constraint_Error or Program_Error. Otherwise, the operation either proceeds as it would for an empty container, or it raises Constraint_Error or Program_Error. 96.2/3

Erroneous Execution

A Cursor value is *invalid* if any of the following have occurred since it was created: 97/2

- The set that contains the element it designates has been finalized; 98/2
- The set that contains the element it designates has been used as the Target of a call to Assign, or as the target of an assignment_statement; 98.1/3
- The set that contains the element it designates has been used as the Source or Target of a call to Move; or 99/2
- The element it designates has been removed from the set that contains the element. 100/3

The result of "=" or Has_Element is unspecified if these functions are called with an invalid cursor parameter. Execution is erroneous if any other subprogram declared in Containers.Hashed_Sets or Containers.Ordered_Sets is called with an invalid cursor parameter. 101/2

Implementation Requirements

No storage associated with a Set object shall be lost upon assignment or scope exit. 102/2

The execution of an assignment_statement for a set shall have the effect of copying the elements from the source set object to the target set object. 103/2

Implementation Advice

Move should not copy elements, and should minimize copying of internal data structures. 104/2

105/2 If an exception is propagated from a set operation, no storage should be lost, nor any elements removed from a set unless specified by the operation.

A.18.8 The Package Containers.Hash_Set

Static Semantics

1/2 The generic library package Containers.Hash_Set has the following declaration:

```

2/3  generic
      type Element_Type is private;
      with function Hash (Element : Element_Type) return Hash_Type;
      with function Equivalent_Elements (Left, Right : Element_Type)
            return Boolean;
      with function "=" (Left, Right : Element_Type) return Boolean is <>;
package Ada.Containers.Hash_Set is
  pragma Preelaborate(Hash_Set);
  pragma Remote_Types(Hash_Set);
3/2  type Set is tagged private;
  pragma Preelaborable_Initialization(Set);
4/2  type Cursor is private;
  pragma Preelaborable_Initialization(Cursor);
5/2  Empty_Set : constant Set;
6/2  No_Element : constant Cursor;
7/2  function "=" (Left, Right : Set) return Boolean;
8/2  function Equivalent_Sets (Left, Right : Set) return Boolean;
9/2  function To_Set (New_Item : Element_Type) return Set;
10/2 function Capacity (Container : Set) return Count_Type;
11/2 procedure Reserve_Capacity (Container : in out Set;
                               Capacity : in Count_Type);
12/2 function Length (Container : Set) return Count_Type;
13/2 function Is_Empty (Container : Set) return Boolean;
14/2 procedure Clear (Container : in out Set);
15/2 function Element (Position : Cursor) return Element_Type;
16/2 procedure Replace_Element (Container : in out Set;
                               Position : in Cursor;
                               New_Item : in Element_Type);
17/2 procedure Query_Element
      (Position : in Cursor;
       Process : not null access procedure (Element : in Element_Type));
17.1/3 procedure Assign (Target : in out Set; Source : in Set);
17.2/3 function Copy (Source : Set; Capacity : Count_Type := 0) return Set;
18/2 procedure Move (Target : in out Set;
                   Source : in out Set);
19/2 procedure Insert (Container : in out Set;
                     New_Item : in Element_Type;
                     Position : out Cursor;
                     Inserted : out Boolean);
20/2 procedure Insert (Container : in out Set;
                     New_Item : in Element_Type);
21/2 procedure Include (Container : in out Set;
                     New_Item : in Element_Type);
22/2 procedure Replace (Container : in out Set;
                     New_Item : in Element_Type);

```



```

procedure Exclude (Container : in out Set;
                   Item      : in      Element_Type);
23/2

procedure Delete (Container : in out Set;
                  Item      : in      Element_Type);
24/2

procedure Delete (Container : in out Set;
                  Position  : in out Cursor);
25/2

procedure Union (Target : in out Set;
                 Source  : in      Set);
26/2

function Union (Left, Right : Set) return Set;
27/2

function "or" (Left, Right : Set) return Set renames Union;
28/2

procedure Intersection (Target : in out Set;
                       Source  : in      Set);
29/2

function Intersection (Left, Right : Set) return Set;
30/2

function "and" (Left, Right : Set) return Set renames Intersection;
31/2

procedure Difference (Target : in out Set;
                     Source  : in      Set);
32/2

function Difference (Left, Right : Set) return Set;
33/2

function "-" (Left, Right : Set) return Set renames Difference;
34/2

procedure Symmetric_Difference (Target : in out Set;
                               Source  : in      Set);
35/2

function Symmetric_Difference (Left, Right : Set) return Set;
36/2

function "xor" (Left, Right : Set) return Set
renames Symmetric_Difference;
37/2

function Overlap (Left, Right : Set) return Boolean;
38/2

function Is_Subset (Subset : Set;
                   Of_Set  : Set) return Boolean;
39/2

function First (Container : Set) return Cursor;
40/2

function Next (Position : Cursor) return Cursor;
41/2

procedure Next (Position : in out Cursor);
42/2

function Find (Container : Set;
               Item      : Element_Type) return Cursor;
43/2

function Contains (Container : Set;
                  Item      : Element_Type) return Boolean;
44/2

function Has_Element (Position : Cursor) return Boolean;
45/2

function Equivalent_Elements (Left, Right : Cursor)
return Boolean;
46/2

function Equivalent_Elements (Left  : Cursor;
                              Right : Element_Type)
return Boolean;
47/2

function Equivalent_Elements (Left  : Element_Type;
                              Right : Cursor)
return Boolean;
48/2

procedure Iterate
(Container : in Set;
 Process  : not null access procedure (Position : in Cursor));
49/2

generic
type Key_Type (<>) is private;
50/2
with function Key (Element : Element_Type) return Key_Type;
with function Hash (Key : Key_Type) return Hash_Type;
with function Equivalent_Keys (Left, Right : Key_Type)
return Boolean;

package Generic_Keys is
function Key (Position : Cursor) return Key_Type;
51/2

```

```

52/2      function Element (Container : Set;
                           Key       : Key_Type)
          return Element_Type;
53/2      procedure Replace (Container : in out Set;
                           Key       : in   Key_Type;
                           New_Item  : in   Element_Type);
54/2      procedure Exclude (Container : in out Set;
                           Key       : in   Key_Type);
55/2      procedure Delete (Container : in out Set;
                           Key       : in   Key_Type);
56/2      function Find (Container : Set;
                        Key       : Key_Type)
          return Cursor;
57/2      function Contains (Container : Set;
                           Key       : Key_Type)
          return Boolean;
58/2      procedure Update_Element_Preserving_Key
        (Container : in out Set;
         Position  : in   Cursor;
         Process   : not null access procedure
                   (Element : in out Element_Type));
59/2      end Generic_Keys;
60/2      private
61/2      ... -- not specified by the language
62/2      end Ada.Containers.Hashed_Sets;

```

63/2 An object of type `Set` contains an expandable hash table, which is used to provide direct access to elements. The *capacity* of an object of type `Set` is the maximum number of elements that can be inserted into the hash table prior to it being automatically expanded.

64/2 Two elements *E1* and *E2* are defined to be *equivalent* if `Equivalent_Elements (E1, E2)` returns `True`.

65/2 The actual function for the generic formal function `Hash` is expected to return the same value each time it is called with a particular element value. For any two equivalent elements, the actual for `Hash` is expected to return the same value. If the actual for `Hash` behaves in some other manner, the behavior of this package is unspecified. Which subprograms of this package call `Hash`, and how many times they call it, is unspecified.

66/2 The actual function for the generic formal function `Equivalent_Elements` is expected to return the same value each time it is called with a particular pair of `Element` values. It should define an equivalence relationship, that is, be reflexive, symmetric, and transitive. If the actual for `Equivalent_Elements` behaves in some other manner, the behavior of this package is unspecified. Which subprograms of this package call `Equivalent_Elements`, and how many times they call it, is unspecified.

66.1/3 If the actual function for the generic formal function `"=`" returns `True` for any pair of non-equivalent elements, then the behavior of the container function `"=`" is unspecified.

67/2 If the value of an element stored in a set is changed other than by an operation in this package such that at least one of `Hash` or `Equivalent_Elements` give different results, the behavior of this package is unspecified.

68/2 Which elements are the first element and the last element of a set, and which element is the successor of a given element, are unspecified, other than the general semantics described in A.18.7.

```

69/2      function Capacity (Container : Set) return Count_Type;
70/2      Returns the capacity of Container.

```

procedure Reserve_Capacity (Container : in out Set; Capacity : in Count_Type);	71/2
Reserve_Capacity allocates a new hash table such that the length of the resulting set can become at least the value Capacity without requiring an additional call to Reserve_Capacity, and is large enough to hold the current length of Container. Reserve_Capacity then rehashes the elements in Container onto the new hash table. It replaces the old hash table with the new hash table, and then deallocates the old hash table. Any exception raised during allocation is propagated and Container is not modified.	72/2
Reserve_Capacity tampers with the cursors of Container.	73/2
procedure Clear (Container : in out Set);	74/2
In addition to the semantics described in A.18.7, Clear does not affect the capacity of Container.	75/2
procedure Assign (Target : in out Set; Source : in Set);	75.1/3
In addition to the semantics described in A.18.7, if the length of Source is greater than the capacity of Target, Reserve_Capacity is called with the length of Source as the capacity before assigning any elements.	75.2/3
function Copy (Source : Set; Capacity : Count_Type := 0) return Set;	75.3/3
Returns a set whose elements are initialized from the elements of Source. If Capacity is 0, then the set capacity is the length of Source; if Capacity is equal to or greater than the length of Source, the set capacity is at least the specified value. Otherwise, the operation propagates Capacity_Error.	75.4/3
procedure Insert (Container : in out Set; New_Item : in Element_Type; Position : out Cursor; Inserted : out Boolean);	76/2
In addition to the semantics described in A.18.7, if Length (Container) equals Capacity (Container), then Insert first calls Reserve_Capacity to increase the capacity of Container to some larger value.	77/2
function First (Container : Set) return Cursor;	78/2
If Length (Container) = 0, then First returns No_Element. Otherwise, First returns a cursor that designates the first hashed element in Container.	79/2
function Equivalent_Elements (Left, Right : Cursor) return Boolean;	80/2
Equivalent to Equivalent_Elements (Element (Left), Element (Right)).	81/2
function Equivalent_Elements (Left : Cursor; Right : Element_Type) return Boolean;	82/2
Equivalent to Equivalent_Elements (Element (Left), Right).	83/2
function Equivalent_Elements (Left : Element_Type; Right : Cursor) return Boolean;	84/2
Equivalent to Equivalent_Elements (Left, Element (Right)).	85/2

For any element E , the actual function for the generic formal function Generic_Keys.Hash is expected to be such that $\text{Hash}(E) = \text{Generic_Keys.Hash}(\text{Key}(E))$. If the actuals for Key or Generic_Keys.Hash

behave in some other manner, the behavior of `Generic_Keys` is unspecified. Which subprograms of `Generic_Keys` call `Generic_Keys.Hash`, and how many times they call it, is unspecified.

- 87/2 For any two elements $E1$ and $E2$, the boolean values `Equivalent_Elements` ($E1$, $E2$) and `Equivalent_Keys` (`Key` ($E1$), `Key` ($E2$)) are expected to be equal. If the actuals for `Key` or `Equivalent_Keys` behave in some other manner, the behavior of `Generic_Keys` is unspecified. Which subprograms of `Generic_Keys` call `Equivalent_Keys`, and how many times they call it, is unspecified.

Implementation Advice

- 88/2 If N is the length of a set, the average time complexity of the subprograms `Insert`, `Include`, `Replace`, `Delete`, `Exclude` and `Find` that take an element parameter should be $O(\log N)$. The average time complexity of the subprograms that take a cursor parameter should be $O(1)$. The average time complexity of `Reserve_Capacity` should be $O(N)$.

A.18.9 The Package `Containers.Ordered_Sets`

Static Semantics

- 1/2 The generic library package `Containers.Ordered_Sets` has the following declaration:

```

2/3  generic
      type Element_Type is private;
      with function "<" (Left, Right : Element_Type) return Boolean is <>;
      with function "=" (Left, Right : Element_Type) return Boolean is <>;
package Ada.Containers.Ordered_Sets is
  pragma Preelaborate(Ordered_Sets);
  pragma Remote_Types(Ordered_Sets);

3/2  function Equivalent_Elements (Left, Right : Element_Type) return Boolean;

4/2  type Set is tagged private;
      pragma Preelaborable_Initialization(Set);

5/2  type Cursor is private;
      pragma Preelaborable_Initialization(Cursor);

6/2  Empty_Set : constant Set;

7/2  No_Element : constant Cursor;

8/2  function "=" (Left, Right : Set) return Boolean;

9/2  function Equivalent_Sets (Left, Right : Set) return Boolean;

10/2 function To_Set (New_Item : Element_Type) return Set;

11/2 function Length (Container : Set) return Count_Type;

12/2 function Is_Empty (Container : Set) return Boolean;

13/2 procedure Clear (Container : in out Set);

14/2 function Element (Position : Cursor) return Element_Type;

15/2 procedure Replace_Element (Container : in out Set;
                             Position : in      Cursor;
                             New_Item  : in      Element_Type);

16/2 procedure Query_Element
      (Position : in Cursor;
       Process  : not null access procedure (Element : in Element_Type));

16.1/3 procedure Assign (Target : in out Set; Source : in Set);

16.2/3 function Copy (Source : Set) return Set;

17/2  procedure Move (Target : in out Set;
                   Source : in out Set);

```

```

procedure Insert (Container : in out Set;                               18/2
                  New_Item  : in    Element_Type;
                  Position  :      out Cursor;
                  Inserted  :      out Boolean);

procedure Insert (Container : in out Set;                               19/2
                  New_Item  : in    Element_Type);

procedure Include (Container : in out Set;                               20/2
                  New_Item  : in    Element_Type);

procedure Replace (Container : in out Set;                               21/2
                  New_Item  : in    Element_Type);

procedure Exclude (Container : in out Set;                               22/2
                  Item      : in    Element_Type);

procedure Delete (Container : in out Set;                               23/2
                  Item      : in    Element_Type);

procedure Delete (Container : in out Set;                               24/2
                  Position  : in out Cursor);

procedure Delete_First (Container : in out Set);                       25/2
procedure Delete_Last (Container : in out Set);                       26/2

procedure Union (Target : in out Set;                                   27/2
                 Source : in    Set);

function Union (Left, Right : Set) return Set;                       28/2
function "or" (Left, Right : Set) return Set renames Union;          29/2

procedure Intersection (Target : in out Set;                          30/2
                       Source : in    Set);

function Intersection (Left, Right : Set) return Set;                31/2
function "and" (Left, Right : Set) return Set renames Intersection; 32/2

procedure Difference (Target : in out Set;                            33/2
                     Source : in    Set);

function Difference (Left, Right : Set) return Set;                 34/2
function "-" (Left, Right : Set) return Set renames Difference;      35/2

procedure Symmetric_Difference (Target : in out Set;                  36/2
                                Source : in    Set);

function Symmetric_Difference (Left, Right : Set) return Set;        37/2
function "xor" (Left, Right : Set) return Set renames              38/2
    Symmetric_Difference;

function Overlap (Left, Right : Set) return Boolean;                 39/2
function Is_Subset (Subset : Set;                                       40/2
                  Of_Set  : Set) return Boolean;

function First (Container : Set) return Cursor;                     41/2
function First_Element (Container : Set) return Element_Type;        42/2
function Last (Container : Set) return Cursor;                      43/2
function Last_Element (Container : Set) return Element_Type;        44/2
function Next (Position : Cursor) return Cursor;                   45/2
procedure Next (Position : in out Cursor);                          46/2
function Previous (Position : Cursor) return Cursor;               47/2
procedure Previous (Position : in out Cursor);                     48/2
function Find (Container : Set;                                       49/2
               Item      : Element_Type)
    return Cursor;

```

```

50/2      function Floor (Container : Set;
                        Item       : Element_Type)
          return Cursor;
51/2      function Ceiling (Container : Set;
                          Item       : Element_Type)
          return Cursor;
52/2      function Contains (Container : Set;
                          Item       : Element_Type) return Boolean;
53/2      function Has_Element (Position : Cursor) return Boolean;
54/2      function "<" (Left, Right : Cursor) return Boolean;
55/2      function ">" (Left, Right : Cursor) return Boolean;
56/2      function "<" (Left : Cursor; Right : Element_Type)
          return Boolean;
57/2      function ">" (Left : Cursor; Right : Element_Type)
          return Boolean;
58/2      function "<" (Left : Element_Type; Right : Cursor)
          return Boolean;
59/2      function ">" (Left : Element_Type; Right : Cursor)
          return Boolean;
60/2      procedure Iterate
        (Container : in Set;
         Process   : not null access procedure (Position : in Cursor));
61/2      procedure Reverse_Iterate
        (Container : in Set;
         Process   : not null access procedure (Position : in Cursor));
62/2      generic
        type Key_Type (<>) is private;
        with function Key (Element : Element_Type) return Key_Type;
        with function "<" (Left, Right : Key_Type)
          return Boolean is <>;
        package Generic_Keys is
63/2          function Equivalent_Keys (Left, Right : Key_Type)
          return Boolean;
64/2          function Key (Position : Cursor) return Key_Type;
65/2          function Element (Container : Set;
                              Key       : Key_Type)
          return Element_Type;
66/2          procedure Replace (Container : in out Set;
                              Key       : in   Key_Type;
                              New_Item  : in   Element_Type);
67/2          procedure Exclude (Container : in out Set;
                              Key       : in   Key_Type);
68/2          procedure Delete (Container : in out Set;
                              Key       : in   Key_Type);
69/2          function Find (Container : Set;
                          Key       : Key_Type)
          return Cursor;
70/2          function Floor (Container : Set;
                          Key       : Key_Type)
          return Cursor;
71/2          function Ceiling (Container : Set;
                              Key       : Key_Type)
          return Cursor;
72/2          function Contains (Container : Set;
                              Key       : Key_Type) return Boolean;

```

```

procedure Update_Element_Preserving_Key                                73/2
  (Container : in out Set;
   Position  : in      Cursor;
   Process   : not null access procedure
               (Element : in out Element_Type));

end Generic_Keys;                                                    74/2

private                                                            75/2
  ... -- not specified by the language                                76/2
end Ada.Containers.Ordered_Sets;                                     77/2

```

Two elements *E1* and *E2* are *equivalent* if both *E1* < *E2* and *E2* < *E1* return False, using the generic formal "<" operator for elements. Function *Equivalent_Elements* returns True if Left and Right are equivalent, and False otherwise. 78/2

The actual function for the generic formal function "<" on *Element_Type* values is expected to return the same value each time it is called with a particular pair of key values. It should define a strict weak ordering relationship (see A.18). If the actual for "<" behaves in some other manner, the behavior of this package is unspecified. Which subprograms of this package call "<" and how many times they call it, is unspecified. 79/3

If the actual function for the generic formal function "=" returns True for any pair of non-equivalent elements, then the behavior of the container function "=" is unspecified. 79.1/3

If the value of an element stored in a set is changed other than by an operation in this package such that at least one of "<" or "=" give different results, the behavior of this package is unspecified. 80/2

The first element of a nonempty set is the one which is less than all the other elements in the set. The last element of a nonempty set is the one which is greater than all the other elements in the set. The successor of an element is the smallest element that is larger than the given element. The predecessor of an element is the largest element that is smaller than the given element. All comparisons are done using the generic formal "<" operator for elements. 81/2

```

function Copy (Source : Set) return Set;                             81.1/3
  Returns a set whose elements are initialized from the corresponding elements of Source. 81.2/3

procedure Delete_First (Container : in out Set);                      82/2
  If Container is empty, Delete_First has no effect. Otherwise the element designated by First
  (Container) is removed from Container. Delete_First tampers with the cursors of Container. 83/2

procedure Delete_Last (Container : in out Set);                       84/2
  If Container is empty, Delete_Last has no effect. Otherwise the element designated by Last
  (Container) is removed from Container. Delete_Last tampers with the cursors of Container. 85/2

function First_Element (Container : Set) return Element_Type;        86/2
  Equivalent to Element (First (Container)).                           87/2

function Last (Container : Set) return Cursor;                        88/2
  Returns a cursor that designates the last element in Container. If Container is empty, returns
  No_Element.                                                          89/2

function Last_Element (Container : Set) return Element_Type;         90/2
  Equivalent to Element (Last (Container)).                             91/2

```

- 92/2 **function** Previous (Position : Cursor) **return** Cursor;
- 93/2 If Position equals No_Element, then Previous returns No_Element. Otherwise Previous returns a cursor designating the element that precedes the one designated by Position. If Position designates the first element, then Previous returns No_Element.
- 94/2 **procedure** Previous (Position : **in out** Cursor);
- 95/2 Equivalent to Position := Previous (Position).
- 96/2 **function** Floor (Container : Set;
 Item : Element_Type) **return** Cursor;
- 97/2 Floor searches for the last element which is not greater than Item. If such an element is found, a cursor that designates it is returned. Otherwise No_Element is returned.
- 98/2 **function** Ceiling (Container : Set;
 Item : Element_Type) **return** Cursor;
- 99/2 Ceiling searches for the first element which is not less than Item. If such an element is found, a cursor that designates it is returned. Otherwise No_Element is returned.
- 100/2 **function** "<" (Left, Right : Cursor) **return** Boolean;
- 101/2 Equivalent to Element (Left) < Element (Right).
- 102/2 **function** ">" (Left, Right : Cursor) **return** Boolean;
- 103/2 Equivalent to Element (Right) < Element (Left).
- 104/2 **function** "<" (Left : Cursor; Right : Element_Type) **return** Boolean;
- 105/2 Equivalent to Element (Left) < Right.
- 106/2 **function** ">" (Left : Cursor; Right : Element_Type) **return** Boolean;
- 107/2 Equivalent to Right < Element (Left).
- 108/2 **function** "<" (Left : Element_Type; Right : Cursor) **return** Boolean;
- 109/2 Equivalent to Left < Element (Right).
- 110/2 **function** ">" (Left : Element_Type; Right : Cursor) **return** Boolean;
- 111/2 Equivalent to Element (Right) < Left.
- 112/2 **procedure** Reverse_Iterate
 (Container : **in** Set;
 Process : **not null access procedure** (Position : **in** Cursor));
- 113/2 Iterates over the elements in Container as per Iterate, with the difference that the elements are traversed in predecessor order, starting with the last element.
- 114/2 For any two elements *E1* and *E2*, the boolean values (*E1* < *E2*) and (Key(*E1*) < Key(*E2*)) are expected to be equal. If the actuals for Key or Generic_Keys."<" behave in some other manner, the behavior of this package is unspecified. Which subprograms of this package call Key and Generic_Keys."<", and how many times the functions are called, is unspecified.
- 115/2 In addition to the semantics described in A.18.7, the subprograms in package Generic_Keys named Floor and Ceiling, are equivalent to the corresponding subprograms in the parent package, with the difference that the Key subprogram parameter is compared to elements in the container using the Key and "<" generic formal functions. The function named Equivalent_Keys in package Generic_Keys returns True if

both `Left < Right` and `Right < Left` return `False` using the generic formal `"<"` operator, and returns `True` otherwise.

Implementation Advice

If N is the length of a set, then the worst-case time complexity of the `Insert`, `Include`, `Replace`, `Delete`, `Exclude` and `Find` operations that take an element parameter should be $O((\log N)**2)$ or better. The worst-case time complexity of the subprograms that take a cursor parameter should be $O(1)$. 116/2

A.18.10 The Package Containers.Multiway_Trees

The language-defined generic package `Containers.Multiway_Trees` provides private types `Tree` and `Cursor`, and a set of operations for each type. A multiway tree container is well-suited to represent nested structures. 1/3

A multiway tree container object manages a tree of internal *nodes*, each of which contains an element and pointers to the parent, first child, last child, next (successor) sibling, and previous (predecessor) sibling internal nodes. A cursor designates a particular node within a tree (and by extension the element contained in that node, if any). A cursor keeps designating the same node (and element) as long as the node is part of the container, even if the node is moved within the container. 2/3

A *subtree* is a particular node (which *roots the subtree*) and all of its child nodes (including all of the children of the child nodes, recursively). There is a special node, the *root*, which is always present and has no associated element value. The root node provides a place to add nodes to an otherwise empty tree and represents the bottom of the tree. 3/3

A node that has no children is called a *leaf node*. The *ancestors* of a node are the parent node, the parent of the parent node, and so on until a node with no parent is reached. Similarly, the *descendants* of a node are the child nodes, the children of each child node, and so on. 4/3

The nodes of a subtree could be visited in several different orders. For a *depth-first order*, the last step of visiting a node is to visit the nodes of its child list in order, recursively. 5/3

Static Semantics

The generic library package `Containers.Multiway_Trees` has the following declaration: 6/3

```

generic
  type Element_Type is private;
  with function "=" (Left, Right : Element_Type) return Boolean is <>;
package Ada.Containers.Multiway_Trees is
package Ada.Containers.Multiway_Trees is
  pragma Preelaborate(Multiway_Trees);
  pragma Remote_Types(Multiway_Trees);

  type Tree is tagged private;
  pragma Preelaborable_Initialization(Tree);
  type Cursor is private;
  pragma Preelaborable_Initialization(Cursor);

  Empty_Tree : constant Tree;
  No_Element : constant Cursor;

  function Equal_Subtree (Left_Position : Cursor;
                        Right_Position: Cursor) return Boolean;

  function "=" Left, Right : Tree) return Boolean;

  function Is_Empty (Container : Tree) return Boolean;

  function Node_Count (Container : Tree) return Count_Type;

```

7/3
8/3
9/3
10/3
11/3
12/3
13/3
14/3
15/3

```

16/3      function Subtree_Node_Count (Position : Cursor) return Count_Type;
17/3      function Depth (Position : Cursor) return Count_Type;
18/3      function Is_Root (Position : Cursor) return Boolean;
19/3      function Is_Leaf (Position : Cursor) return Boolean;
20/3      function Root (Container : Tree) return Cursor;
21/3      procedure Clear (Container : in out Tree);
22/3      function Element (Position : Cursor) return Element_Type;
23/3      procedure Replace_Element (Container : in out Tree;
                                   Position  : in      Cursor;
                                   New_Item   : in      Element_Type);

24/3      procedure Query_Element
        (Position : in Cursor;
         Process   : not null access procedure (Element : in Element_Type));

25/3      procedure Update_Element
        (Container : in out Tree;
         Position  : in      Cursor;
         Process   : not null access procedure
           (Element : in out Element_Type));

26/3      procedure Assign (Target : in out Tree; Source : in Tree);
27/3      function Copy (Source : Tree) return Tree;
28/3      procedure Move (Target : in out Tree;
                         Source  : in out Tree);

29/3      procedure Delete_Leaf (Container : in out Tree;
                                Position  : in out Cursor);

30/3      procedure Delete_Subtree (Container : in out Tree;
                                    Position  : in out Cursor);

31/3      procedure Swap (Container : in out Tree;
                         I, J      : in      Cursor);

32/3      function Find (Container : Tree;
                        Item       : Element_Type)
        return Cursor;

33/3      function Find_In_Subtree (Container : Tree;
                                   Item       : Element_Type;
                                   Position  : Cursor)
        return Cursor;

34/3      function Ancestor_Find (Container : Tree;
                                  Item       : Element_Type;
                                  Position  : Cursor)
        return Cursor;

35/3      function Contains (Container : Tree;
                             Item      : Element_Type) return Boolean;

36/3      function Has_Element (Position : Cursor) return Boolean;

37/3      procedure Iterate
        (Container : in Tree;
         Process   : not null access procedure (Position : in Cursor));

38/3      procedure Iterate_Subtree
        (Position : in Cursor;
         Process   : not null access procedure (Position : in Cursor));

39/3      function Child_Count (Parent : Cursor) return Count_Type;
40/3      function Child_Depth (Parent, Child : Cursor) return Count_Type;
41/3      procedure Insert_Child (Container : in out Tree;
                                Parent     : in      Cursor;
                                Before      : in      Cursor;
                                New_Item    : in      Element_Type;
                                Count       : in      Count_Type := 1);

```

```

procedure Insert_Child (Container : in out Tree;                                42/3
                        Parent   : in      Cursor;
                        Before    : in      Cursor;
                        New_Item  : in      Element_Type;
                        Position  : out      Cursor;
                        Count     : in      Count_Type := 1);

procedure Insert_Child (Container : in out Tree;                                43/3
                        Parent   : in      Cursor;
                        Before    : in      Cursor;
                        Position  : out      Cursor;
                        Count     : in      Count_Type := 1);

procedure Prepend_Child (Container : in out Tree;                                44/3
                        Parent   : in      Cursor;
                        New_Item  : in      Element_Type;
                        Count     : in      Count_Type := 1);

procedure Append_Child (Container : in out Tree;                                45/3
                        Parent   : in      Cursor;
                        New_Item  : in      Element_Type;
                        Count     : in      Count_Type := 1);

procedure Delete_Children (Container : in out Tree;                                46/3
                           Parent   : in      Cursor);

procedure Copy_Subtree (Target   : in out Tree;                                47/3
                        Parent   : in      Cursor;
                        Before    : in      Cursor;
                        Source    : in      Cursor);

procedure Splice_Subtree (Target   : in out Tree;                                48/3
                           Parent   : in      Cursor;
                           Before    : in      Cursor;
                           Source    : in out Tree;
                           Position  : in out Cursor);

procedure Splice_Subtree (Container: in out Tree;                                49/3
                           Parent   : in      Cursor;
                           Before    : in      Cursor;
                           Position  : in      Cursor);

procedure Splice_Children (Target   : in out Tree;                                50/3
                           Target_Parent : in      Cursor;
                           Before    : in      Cursor;
                           Source    : in out Tree;
                           Source_Parent : in      Cursor);

procedure Splice_Children (Container : in out Tree;                                51/3
                           Target_Parent : in      Cursor;
                           Before    : in      Cursor;
                           Source_Parent : in      Cursor);

function Parent (Position : Cursor) return Cursor;                                52/3

function First_Child (Parent : Cursor) return Cursor;                                53/3

function First_Child_Element (Parent : Cursor) return Element_Type;                54/3

function Last_Child (Parent : Cursor) return Cursor;                                55/3

function Last_Child_Element (Parent : Cursor) return Element_Type;                56/3

function Next_Sibling (Position : Cursor) return Cursor;                            57/3

function Previous_Sibling (Position : Cursor) return Cursor;                        58/3

procedure Next_Sibling (Position : in out Cursor);                                59/3

procedure Previous_Sibling (Position : in out Cursor);                            60/3

procedure Iterate_Children
  (Container : in Tree;
   Parent    : in Cursor;
   Process    : not null access procedure (Position : in Cursor));                61/3

```

```

62/3      procedure Reverse_Iterate_Children
          (Container : in Tree;
           Parent    : in Cursor;
           Process   : not null access procedure (Position : in Cursor));
63/3      private
          ... -- not specified by the language
          end Ada.Containers.Multiway_Trees;

```

64/3 The actual function for the generic formal function "=" on Element_Type values is expected to define a reflexive and symmetric relationship and return the same result value each time it is called with a particular pair of values. If it behaves in some other manner, the functions Find, Reverse_Find, Equal_Subtree, and "=" on tree values return an unspecified value. The exact arguments and number of calls of this generic formal function by the functions Find, Reverse_Find, Equal_Subtree, and "=" on tree values are unspecified.

65/3 The type Tree is used to represent trees. The type Tree needs finalization (see 7.6).

66/3 Empty_Tree represents the empty Tree object. It contains only the root node (Count (Empty_Tree) returns 1). If an object of type Tree is not otherwise initialized, it is initialized to the same value as Empty_Tree.

67/3 No_Element represents a cursor that designates no element. If an object of type Cursor is not otherwise initialized, it is initialized to the same value as No_Element.

68/3 The predefined "=" operator for type Cursor returns True if both cursors are No_Element, or designate the same element in the same container.

69/3 Execution of the default implementation of the Input, Output, Read, or Write attribute of type Cursor raises Program_Error.

70/3 Tree'Write writes Tree.Count-1 elements to the stream. Tree'Read reads Tree.Count-1 elements from the stream.

71/3 Some operations of this generic package have access-to-subprogram parameters. To ensure such operations are well-defined, they guard against certain actions by the designated subprogram. In particular, some operations check for "tampering with cursors" of a container because they depend on the set of elements of the container remaining constant, and others check for "tampering with elements" of a container because they depend on elements of the container not being replaced.

72/3 A subprogram is said to *tamper with cursors* of a tree object *T* if:

- 73/3 • it inserts or deletes elements of *T*, that is, it calls the Clear, Delete_Leaf, Insert_Child, or Delete_Children procedures with *T* as a parameter; or
- 74/3 • it reorders the elements of *T*, that is, it calls the Splice_Subtree or Splice_Children procedures; or
- 75/3 • it finalizes *T*; or
- 76/3 • it calls Assign with *T* as the Target parameter; or
- 77/3 • it calls the Move procedure with *T* as a parameter.

78/3 A subprogram is said to *tamper with elements* of a tree object *T* if:

- 79/3 • it tampers with cursors of *T*; or
- 80/3 • it replaces one or more elements of *T*, that is, it calls the Replace_Element or Swap procedures with *T* as a parameter.

function Equal_Subtree (Left_Position : Cursor; Right_Position: Cursor) return Boolean;	81/3
If Left_Position or Right_Position equals No_Element, propagates Constraint_Error. If the number of child nodes of the element designated by Left_Position is different than the number of child nodes of the element designated by Right_Position, the function returns False. If Left_Position designates a root node and Right_Position does not, the function returns False. If Right_Position designates a root node and Left_Position does not, the function returns False. Unless both cursors designate a root node, the elements are compared using the generic formal equality operator. If the result is False, the function returns False. Otherwise, it calls Equal_Subtree on a cursor designating each child element of the element designated by Left_Position and a cursor designated the corresponding child element of the element designated by Right_Position. If any such call returns False, the function returns False; otherwise it returns True. Any exception raised during the evaluation of element equality is propagated.	82/3
function "=" (Left, Right : Tree) return Boolean;	83/3
If Left and Right denote the same tree object, then the function returns True. Otherwise, it calls Equal_Subtree with a cursor designating the root node of Left and Right; the result is returned. Any exception raised during the evaluation of Equal_Subtree is propagated.	84/3
function Node_Count (Container : Tree) return Count_Type;	85/3
Node_Count returns the number of nodes in Container.	86/3
function Subtree_Node_Count (Position : Cursor) return Count_Type;	87/3
If Position is No_Element, Subtree_Count returns 0; otherwise, Subtree_Node_Count returns the number of nodes in the subtree that is rooted by Position.	88/3
function Is_Empty (Container : Tree) return Boolean;	89/3
Equivalent to Count (Container) = 1.	90/3
function Depth (Position : Cursor) return Count_Type;	91/3
If Position equals No_Element, Depth returns 0; otherwise Depth returns the number of ancestor nodes of the node designated by Position (including the node itself).	92/3
function Is_Root (Position : Cursor) return Boolean;	93/3
Is_Root returns True if the Position designates the root node of the tree; and returns False otherwise.	94/3
function Is_Leaf (Position : Cursor) return Boolean;	95/3
Is_Leaf returns True if Position designates a node that does not have any child nodes; and returns False otherwise.	96/3
function Root (Container : Tree) return Cursor;	97/3
Root returns a cursor that designates the root node of Container.	98/3
procedure Clear (Container : in out Tree);	99/3
Removes all the elements from Container.	100/3

- 101/3 **function** Element (Position : Cursor) **return** Element_Type;
- 102/3 If Position equals No_Element, then Constraint_Error is propagated; if Position designates the root node of a tree, then Program_Error is propagated. Otherwise, Element returns the element designated by Position.
- 103/3 **procedure** Replace_Element (Container : **in out** Tree;
 Position : **in** Cursor;
 New_Item : **in** Element_Type);
- 104/3 If Position equals No_Element, then Constraint_Error is propagated; if Position does not designate an element in Container (including if it designates the root node), then Program_Error is propagated. Otherwise Replace_Element assigns the value New_Item to the element designated by Position.
- 105/3 **procedure** Query_Element
 (Position : **in** Cursor;
 Process : **not null access procedure** (Element : **in** Element_Type));
- 106/3 If Position equals No_Element, then Constraint_Error is propagated; if Position designates the root node of a tree, then Program_Error is propagated. Otherwise, Query_Element calls Process.all with the element designated by Position as the argument. Program_Error is propagated if Process.all tampers with the elements of Container. Any exception raised by Process.all is propagated.
- 107/3 **procedure** Update_Element
 (Container : **in out** Tree;
 Position : **in** Cursor;
 Process : **not null access procedure**
 (Element : **in out** Element_Type));
- 108/3 If Position equals No_Element, then Constraint_Error is propagated; if Position does not designate an element in Container (including if it designates the root node), then Program_Error is propagated. Otherwise Update_Element calls Process.all with the element designated by Position as the argument. Program_Error is propagated if Process.all tampers with the elements of Container. Any exception raised by Process.all is propagated.
- 109/3 If Element_Type is unconstrained and definite, then the actual Element parameter of Process.all shall be unconstrained.
- 110/3 **procedure** Assign (Target : **in out** Tree; Source : **in** Tree);
- 111/3 If Target denotes the same object as Source, the operation has no effect. Otherwise, it calls Clear (Target), and then each element of Source is assigned to a corresponding element in Target.
- 112/3 **function** Copy (Source : Tree) **return** Tree;
- 113/3 Returns a tree with the same structure as Source and whose elements are initialized from the corresponding elements of Source.
- 114/3 **procedure** Move (Target : **in out** Tree;
 Source : **in out** Tree);
- 115/3 If Target denotes the same object as Source, then Move has no effect. Otherwise, Move first calls Clear (Target). Then, the nodes other than the root node in Source are moved to Target (in the same positions). After Move completes, Count(Target) is the number of nodes originally in Source, and Count(Source) is 1.

procedure Delete_Leaf (Container : in out Tree; Position : in out Cursor);	116/3
If Position equals No_Element, then Constraint_Error is propagated; if Position does not designate an element in Container (including if it designates the root node), then Program_Error is propagated. If the element designated by position has any child elements, then Constraint_Error is propagated. Otherwise Delete_Leaf removes (from Container) the element designated by Position. Finally, Position is set to No_Element.	117/3
procedure Delete_Subtree (Container : in out Tree; Position : in out Cursor);	118/3
If Position equals No_Element, then Constraint_Error is propagated. If Position does not designate an element in Container (including if it designates the root node), then Program_Error is propagated. Otherwise Delete removes (from Container) the subtree designated by Position (that is, the node designated by Position and all of the descendant nodes of that node), and Position is set to No_Element.	119/3
procedure Swap (Container : in out Tree; I, J : in Cursor);	120/3
If either I or J equals No_Element, then Constraint_Error is propagated. If either I or J do not designate an element in Container (including if either designates the root node), then Program_Error is propagated. Otherwise, Swap exchanges the values of the elements designated by I and J.	121/3
function Find (Container : Tree; Item : Element_Type) return Cursor;	122/3
Find searches the elements of Container for an element equal to Item (using the generic formal equality operator). The search starts at the root node. The search checks the tree in a depth-first order. If no equal element is found, then Find returns No_Element. Otherwise, it returns a cursor designating the first equal element encountered.	123/3
function Find_In_Subtree (Container : Tree; Item : Element_Type; Position : Cursor) return Cursor;	124/3
If Position equals No_Element, then Constraint_Error is propagated; if Position does not designate a node in Container, then Program_Error is propagated. Find_In_Subtree searches a subtree of the elements of Container for an element equal to Item (using the generic formal equality operator). The search starts at the element designated by Position. The search checks the subtree rooted by Position in a depth-first order. If no equal element is found, then Find returns No_Element. Otherwise, it returns a cursor designating the first equal element encountered.	125/3
function Ancestor_Find (Container : Tree; Item : Element_Type; Position : Cursor) return Cursor;	126/3
If Position equals No_Element, then Constraint_Error is propagated; if Position does not designate an element in Container (including if it designates the root node), then Program_Error is propagated. Otherwise, Ancestor_Find searches for an element equal to Item (using the generic formal equality operator). The search starts at the element designated by Position, and checks each ancestor proceeding toward the root of the subtree. If no equal element is found,	127/3

then Ancestor_Find returns No_Element. Otherwise, it returns a cursor designating the first equal element encountered.

128/3 **function** Contains (Container : Tree;
 Item : Element_Type) **return** Boolean;

129/3 Equivalent to Find (Container, Item) /= No_Element.

130/3 **function** Has_Element (Position : Cursor) **return** Boolean;

131/3 Returns True if Position designates an element, and returns False otherwise. In particular, Has_Element returns False if the cursor designates the root node or equals No_Element.

132/3 **procedure** Iterate
 (Container : **in** Tree;
 Process : **not null access procedure** (Position : **in** Cursor));

133/3 Iterate calls Process.all with a cursor that designates each element in Container, starting with the root node and proceeding in a depth-first order. Program_Error is propagated if Process.all tampers with the cursors of Container. Any exception raised by Process.all is propagated.

134/3 **procedure** Iterate_Subtree
 (Position : **in** Cursor;
 Process : **not null access procedure** (Position : **in** Cursor));

135/3 If Position equals No_Element, then Constraint_Error is propagated. Iterate calls Process.all with a cursor that designates each element in the subtree rooted by the node designated by Position, starting with the node designated by Position and proceeding in a depth-first order. Program_Error is propagated if Process.all tampers with the cursors of Container. Any exception raised by Process.all is propagated.

136/3 **function** Child_Count (Parent : Cursor) **return** Count_Type;

137/3 Child_Count returns the number of child nodes of the node designated by Parent.

138/3 **function** Child_Depth (Parent, Child : Cursor) **return** Count_Type;

139/3 If Child or Parent is equal to No_Element, then Constraint_Error is propagated. Otherwise, Child_Depth returns the number of ancestor nodes of Child (including Child itself), up to but not including Parent; in this case Program_Error is propagated if Parent is not an ancestor of Child.

140/3 **procedure** Insert_Child (Container : **in out** Tree;
 Parent : **in** Cursor;
 Before : **in** Cursor;
 New_Item : **in** Element_Type;
 Count : **in** Count_Type := 1);

141/3 If Parent equals No_Element, then Constraint_Error is propagated. If Parent does not designate a node in Container, then Program_Error is propagated. If Before is not equal to No_Element, and does not designate a node in Container, then Program_Error is propagated. If Before is not equal to No_Element, and Container.Parent (Before) is not equal to Parent, then Constraint_Error is propagated. Otherwise, Insert_Child allocates Count nodes containing copies of New_Item and inserts them as children of Parent. If Parent already has child nodes then the new nodes are inserted prior to the node designated by Before, or, if Before equals No_Element, the new nodes are inserted after the last existing child node of Parent. Any exception raised during allocation of internal storage is propagated, and Container is not modified.


```

procedure Insert_Child (Container : in out Tree;
                        Parent   : in   Cursor;
                        Before    : in   Cursor;
                        New_Item  : in   Element_Type;
                        Position  : out   Cursor;
                        Count     : in   Count_Type := 1);

```

142/3

If Parent equals No_Element, then Constraint_Error is propagated. If Parent does not designate a node in Container, then Program_Error is propagated. If Before is not equal to No_Element, and does not designate a node in Container, then Program_Error is propagated. If Before is not equal to No_Element, and Container.Parent (Before) is not equal to Parent, then Constraint_Error is propagated. Otherwise, Insert_Child allocates Count nodes containing copies of New_Item and inserts them as children of Parent. If Parent already has child nodes then the new nodes are inserted prior to the node designated by Before, or, if Before equals No_Element, the new nodes are inserted after the last existing child node of Parent. Position designates the first newly-inserted node. Any exception raised during allocation of internal storage is propagated, and Container is not modified.

143/3

```

procedure Insert_Child (Container : in out Tree;
                        Parent   : in   Cursor;
                        Before    : in   Cursor;
                        Position  : out   Cursor;
                        Count     : in   Count_Type := 1);

```

144/3

If Parent equals No_Element, then Constraint_Error is propagated. If Parent does not designate a node in Container, then Program_Error is propagated. If Before is not equal to No_Element, and does not designate a node in Container, then Program_Error is propagated. If Before is not equal to No_Element, and Container.Parent (Before) is not equal to Parent, then Constraint_Error is propagated. Otherwise, Insert_Child allocates Count nodes, the elements contained in the new nodes are initialized by default (see 3.3.1), and the new nodes are inserted as children of Parent. If Parent already has child nodes then the new nodes are inserted prior to the node designated by Before, or, if Before equals No_Element, the new nodes are inserted after the last existing child node of Parent. Position designates the first newly-inserted node. Any exception raised during allocation of internal storage is propagated, and Container is not modified.

145/3

```

procedure Prepend_Child (Container : in out Tree;
                        Parent   : in   Cursor;
                        New_Item  : in   Element_Type;
                        Count     : in   Count_Type := 1);

```

146/3

Equivalent to Insert_Child (Container, Parent, First_Child (Container, Parent), New_Item, Count).

147/3

```

procedure Append_Child (Container : in out Tree;
                        Parent   : in   Cursor;
                        New_Item  : in   Element_Type;
                        Count     : in   Count_Type := 1);

```

148/3

Equivalent to Insert_Child (Container, Parent, Last_Child (Container, Parent), New_Item, Count).

149/3

```

procedure Delete_Children (Container : in out Tree;
                           Parent     : in   Cursor);

```

150/3

If Parent equals No_Element, then Constraint_Error is propagated. If Parent does not designate a node in Container, Program_Error is propagated. Otherwise, Delete_Children removes (from Container) all of the child elements of Parent along with their dependent elements.

151/3

152/3 **procedure** Copy_Subtree (Target : **in out** Tree;
 Parent : **in** Cursor;
 Before : **in** Cursor;
 Source : **in** Cursor);

153/3 If Parent equals No_Element, then Constraint_Error is propagated. If Parent does not designate a node in Target, then Program_Error is propagated. If Before is not equal to No_Element, and does not designate a node in Target, then Program_Error is propagated. If Before is not equal to No_Element, and Target.Parent (Before) is not equal to Parent, then Constraint_Error is propagated. If Source designates a root node, then Constraint_Error is propagated. If Source is equal to No_Element, then the operation has no effect. Otherwise, the subtree rooted by Source (which can be from any tree; it does not have to be a subtree of Target) is copied (new nodes are allocated to create a new subtree with the same structure as the Source subtree, with each element initialized from the corresponding element of the Source subtree) and inserted into Target as a child of Parent. If Parent already has child nodes then the new nodes are inserted prior to the node designated by Before, or, if Before equals No_Element, the new nodes are inserted after the last existing child node of Parent. The parent of the newly created subtree is set to Parent, and the overall count of Target is incremented by Subtree_Count (Source). Any exception raised during allocation of internal storage is propagated, and Container is not modified.

154/3 **procedure** Splice_Subtree (Target : **in out** Tree;
 Parent : **in** Cursor;
 Before : **in** Cursor;
 Source : **in out** Tree;
 Position : **in out** Cursor);

155/3 If Parent equals No_Element, then Constraint_Error is propagated. If Parent does not designate a node in Target, then Program_Error is propagated. If Before is not equal to No_Element, and does not designate a node in Target, then Program_Error is propagated. If Before is not equal to No_Element, and Target.Parent (Before) is not equal to Parent, then Constraint_Error is propagated. If Position equals No_Element, Constraint_Error is propagated. If Position does not designate a node in Source or designates a root node, then Program_Error is propagated. If Source denotes the same object as Target, then there is no effect; if Position designates an ancestor of Parent or is equal to Parent, Constraint_Error is propagated; else the subtree rooted by the element designated by Position is moved to be a child of Parent. If Parent already has child nodes then the moved nodes are inserted prior to the node designated by Before, or, if Before equals No_Element, the moved nodes are inserted after the last existing child node of Parent. In each of these cases, Position and the count of Target are unchanged, and the parent of the element designated by Position is set to Parent.

156/3 Otherwise (if Source does not denote the same object as Target), the subtree designated by Position is removed from Source and moved to Target. The subtree is inserted as a child of Parent. If Parent already has child nodes then the moved nodes are inserted prior to the node designated by Before, or, if Before equals No_Element, the moved nodes are inserted after the last existing child node of Parent. In each of these cases, the count of Target is incremented by Subtree_Count (Position), and the count of Source is decremented by Subtree_Count (Position), Position is updated to represent an element in Target.

```

procedure Splice_Subtree (Container: in out Tree;           157/3
                          Parent   : in      Cursor;
                          Before   : in      Cursor;
                          Position : in      Cursor);

```

If Parent equals No_Element, then Constraint_Error is propagated. If Parent does not designate a node in Container, then Program_Error is propagated. If Before is not equal to No_Element, and does not designate a node in Container, then Program_Error is propagated. If Before is not equal to No_Element, and Target.Parent (Before) is not equal to Parent, then Constraint_Error is propagated. If Position equals No_Element or designates a root node, Constraint_Error is propagated. If Position does not designate a node in Container, then Program_Error is propagated. If Position equals Before, there is no effect. If Position designates an ancestor of Parent or is equal to Parent, Constraint_Error is propagated. Otherwise, the subtree rooted by the element designated by Position is moved to be a child of Parent. If Parent already has child nodes then the moved nodes are inserted prior to the node designated by Before, or, if Before equals No_Element, the moved nodes are inserted after the last existing child node of Parent. The parent of the element designated by Position is set to Parent. 158/3

```

procedure Splice_Children (Target      : in out Tree;           159/3
                           Target_Parent : in      Cursor;
                           Before        : in      Cursor;
                           Source        : in out Tree;
                           Source_Parent : in      Cursor);

```

If Target_Parent equals No_Element, then Constraint_Error is propagated. If Target_Parent does not designate a node in Target, then Program_Error is propagated. If Before is not equal to No_Element, and does not designate an element in Target, then Program_Error is propagated. If Source_Parent equals No_Element, then Constraint_Error is propagated. If Source_Parent does not designate a node in Source, then Program_Error is propagated. If Before is not equal to No_Element, and Target.Parent (Before) is not equal to Target_Parent, then Constraint_Error is propagated. 160/3

If Source denotes the same object as Target, then: 161/3

- if Target_Parent equals Source_Parent there is no effect; else 162/3
- if Source_Parent is an ancestor of Target_Parent, then Constraint_Error is propagated; else 163/3
- the child elements (and their descendants) of Source_Parent are moved to be child elements of Target_Parent. If Target_Parent already has child elements then the moved elements are inserted prior to the node designated by Before, or, if Before equals No_Element, the moved elements are inserted after the last existing child node of Target_Parent. The parent of each moved child element is set to Target_Parent. 164/3

Otherwise (if Source does not denote the same object as Target), the child elements (and their descendants) of Source_Parent are removed from Source and moved to Target. The child elements are inserted as children of Target_Parent. If Target_Parent already has child elements then the moved elements are inserted prior to the node designated by Before, or, if Before equals No_Element, the moved elements are inserted after the last existing child node of Target_Parent. In each of these cases, the overall count of Target is incremented by Subtree_Count (Source_Parent)-1, and overall count of Source is decremented by Subtree_Count (Source_Parent)-1. 165/3

```

166/3      procedure Splice_Children (Container      : in out Tree;
                                     Target_Parent  : in      Cursor;
                                     Before         : in      Cursor;
                                     Source_Parent  : in      Cursor);

```

167/3 If Target_Parent equals No_Element, then Constraint_Error is propagated. If Target_Parent does not designate a node in Container, then Program_Error is propagated. If Before is not equal to No_Element, and does not designate an element in Container, then Program_Error is propagated. If Source_Parent equals No_Element, then Constraint_Error is propagated. If Source_Parent does not designate a node in Container, then Program_Error is propagated. If Before is not equal to No_Element, and Container.Parent (Before) is not equal to Target_Parent, then Constraint_Error is propagated. If Target_Parent equals Source_Parent there is no effect. If Source_Parent is an ancestor of Target_Parent, then Constraint_Error is propagated. Otherwise the child elements (and their descendants) of Source_Parent are moved to be child elements of Target_Parent. If Target_Parent already has child elements then the moved elements are inserted prior to the node designated by Before, or, if Before equals No_Element, the moved elements are inserted after the last existing child node of Target_Parent. The parent of each moved child element is set to Target_Parent.

```

168/3      function Parent (Position : Cursor) return Cursor;

```

169/3 If Position is equal to No_Element or designates a root node, No_Element is returned. Otherwise, a cursor designating the parent node of the node designated by Position is returned.

```

170/3      function First_Child (Parent : Cursor) return Cursor;

```

171/3 If Parent is equal to No_Element, then Constraint_Error is propagated. Otherwise, First_Child returns a cursor designating the first child node of the node designated by Parent; if there is no such node, No_Element is returned.

```

172/3      function First_Child_Element (Parent : Cursor) return Element_Type;

```

173/3 Equivalent to Element (First_Child (Parent)).

```

174/3      function Last_Child (Parent : Cursor) return Cursor;

```

175/3 If Parent is equal to No_Element, then Constraint_Error is propagated. Otherwise, Last_Child returns a cursor designating the last child node of the node designated by Parent; if there is no such node, No_Element is returned.

```

176/3      function Last_Child_Element (Parent : Cursor) return Element_Type;

```

177/3 Equivalent to Element (Last_Child (Parent)).

```

178/3      function Next_Sibling (Position : Cursor) return Cursor;

```

179/3 If Position equals No_Element or designates the last child node of its parent, then Next_Sibling returns the value No_Element. Otherwise, it returns a cursor that designates the successor (with the same parent) of the node designated by Position.

```

180/3      function Previous_Sibling (Position : Cursor) return Cursor;

```

181/3 If Position equals No_Element or designates the first child node of its parent, then Previous_Sibling returns the value No_Element. Otherwise, it returns a cursor that designates the predecessor (with the same parent) of the node designated by Position.

```

182/3      procedure Next_Sibling (Position : in out Cursor);

```

183/3 Equivalent to Position := Next_Sibling (Position);

procedure Previous_Sibling (Position : in out Cursor);	184/3
Equivalent to Position := Previous_Sibling (Position);	185/3
procedure Iterate_Children	186/3
(Container : in Tree;	
Parent : in Cursor;	
Process : not null access procedure (Position : in Cursor));	
If Parent equals No_Element, then Constraint_Error is propagated. If Parent does not designate a node in Container, then Program_Error is propagated.	187/3
Iterate_Children calls Process. all with a cursor that designates each child node of Parent, starting with the first child node and moving the cursor as per the function Next_Sibling.	188/3
Program_Error is propagated if Process. all tampers with the cursors of Container. Any exception raised by Process. all is propagated.	189/3
procedure Reverse_Iterate_Children	190/3
(Container : in Tree;	
Parent : in Cursor;	
Process : not null access procedure (Position : in Cursor));	
If Parent equals No_Element, then Constraint_Error is propagated. If Parent does not designate a node in Container, then Program_Error is propagated.	191/3
Reverse_Iterate_Children calls Process. all with a cursor that designates each child node of Parent, starting with the last child node and moving the cursor as per the function Previous_Sibling.	192/3
Program_Error is propagated if Process. all tampers with the cursors of Container. Any exception raised by Process. all is propagated.	193/3

Bounded (Run-Time) Errors

It is a bounded error for the actual function associated with a generic formal subprogram, when called as part of an operation of this package, to tamper with elements of any Tree parameter to the operation. Either Program_Error is raised, or the operation works as defined on the value of the Tree either prior to, or subsequent to, some or all of the modifications to the Tree. 194/3

It is a bounded error to call any subprogram declared in the visible part of Containers.Multiway_Trees when the associated container has been finalized. If the operation takes Container as an in out parameter, then it raises Constraint_Error or Program_Error. Otherwise, the operation either proceeds as it would for an empty container, or it raises Constraint_Error or Program_Error. 195/3

Erroneous Execution

A Cursor value is *invalid* if any of the following have occurred since it was created: 196/3

- The tree that contains the element it designates has been finalized; 197/3
- The tree that contains the element it designates has been used as the Source or Target of a call to Move; 198/3
- The tree that contains the element it designates has been used as the Target of a call to Assign or the target of an assignment_statement; 199/3
- The element it designates has been removed from the tree that previously contained the element. 200/3

- 201/3 The result of "=" or Has_Element is unspecified if it is called with an invalid cursor parameter. Execution is erroneous if any other subprogram declared in Containers.Multiway_Trees is called with an invalid cursor parameter.

Implementation Requirements

- 202/3 No storage associated with a multiway tree object shall be lost upon assignment or scope exit.
- 203/3 The execution of an assignment_statement for a tree shall have the effect of copying the elements from the source tree object to the target tree object.

Implementation Advice

- 204/3 Containers.Multiway_Trees should be implemented similarly to a multiway tree. In particular, if N is the overall number of nodes for a particular tree, then the worst-case time complexity of Element, Parent, First_Child, Last_Child, Insert_Child with Count=1, and Delete should be $O(\log N)$.
- 205/3 Move should not copy elements, and should minimize copying of internal data structures.
- 206/3 If an exception is propagated from a tree operation, no storage should be lost, nor any elements removed from a tree unless specified by the operation.

A.18.11 The Package Containers.Indefinite_Vectors

- 1/2 The language-defined generic package Containers.Indefinite_Vectors provides a private type Vector and a set of operations. It provides the same operations as the package Containers.Vectors (see A.18.2), with the difference that the generic formal Element_Type is indefinite.

Static Semantics

- 2/3 The declaration of the generic library package Containers.Indefinite_Vectors has the same contents and semantics as Containers.Vectors except:
- 3/2 • The generic formal Element_Type is indefinite.
- 4/2 • The procedures with the profiles:
- 5/2 **procedure** Insert (Container : **in out** Vector;
 Before : **in** Extended_Index;
 Count : **in** Count_Type := 1);
- 6 **procedure** Insert (Container : **in out** Vector;
 Before : **in** Cursor;
 Position : **out** Cursor;
 Count : **in** Count_Type := 1);
- 7/2 are omitted.
- 8/2 • The actual Element parameter of access subprogram Process of Update_Element may be constrained even if Element_Type is unconstrained.

A.18.12 The Package Containers.Indefinite_Doubly_Linked_Lists

- 1/2 The language-defined generic package Containers.Indefinite_Doubly_Linked_Lists provides private types List and Cursor, and a set of operations for each type. It provides the same operations as the package Containers.Doubly_Linked_Lists (see A.18.3), with the difference that the generic formal Element_Type is indefinite.

Static Semantics

The declaration of the generic library package Containers.Indefinite_Doubly_Linked_Lists has the same contents and semantics as Containers.Doubly_Linked_Lists except: 2/3

- The generic formal Element_Type is indefinite. 3/2
- The procedure with the profile: 4/2


```

procedure Insert (Container : in out List;
                  Before   : in   Cursor;
                  Position  :      out Cursor;
                  Count     : in   Count_Type := 1);
      5/2
      
```

is omitted. 6/2
- The actual Element parameter of access subprogram Process of Update_Element may be constrained even if Element_Type is unconstrained. 7/2

A.18.13 The Package Containers.Indefinite_Hashed_Maps

The language-defined generic package Containers.Indefinite_Hashed_Maps provides a map with the same operations as the package Containers.Hashed_Maps (see A.18.5), with the difference that the generic formal types Key_Type and Element_Type are indefinite. 1/2

Static Semantics

The declaration of the generic library package Containers.Indefinite_Hashed_Maps has the same contents and semantics as Containers.Hashed_Maps except: 2/3

- The generic formal Key_Type is indefinite. 3/2
- The generic formal Element_Type is indefinite. 4/2
- The procedure with the profile: 5/2


```

procedure Insert (Container : in out Map;
                  Key       : in   Key_Type;
                  Position  :      out Cursor;
                  Inserted  :      out Boolean);
      6/2
      
```

is omitted. 7/2
- The actual Element parameter of access subprogram Process of Update_Element may be constrained even if Element_Type is unconstrained. 8/2

A.18.14 The Package Containers.Indefinite_Ordered_Maps

The language-defined generic package Containers.Indefinite_Ordered_Maps provides a map with the same operations as the package Containers.Ordered_Maps (see A.18.6), with the difference that the generic formal types Key_Type and Element_Type are indefinite. 1/2

Static Semantics

The declaration of the generic library package Containers.Indefinite_Ordered_Maps has the same contents and semantics as Containers.Ordered_Maps except: 2/3

- The generic formal Key_Type is indefinite. 3/2
- The generic formal Element_Type is indefinite. 4/2

- 5/2 • The procedure with the profile:
- 6/2 **procedure** Insert (Container : **in out** Map;
 Key : **in** Key_Type;
 Position : **out** Cursor;
 Inserted : **out** Boolean);
- 7/2 is omitted.
- 8/2 • The actual Element parameter of access subprogram Process of Update_Element may be constrained even if Element_Type is unconstrained.

A.18.15 The Package Containers.Indefinite_Hashed_Sets

- 1/2 The language-defined generic package Containers.Indefinite_Hashed_Sets provides a set with the same operations as the package Containers.Hashed_Sets (see A.18.8), with the difference that the generic formal type Element_Type is indefinite.

Static Semantics

- 2/3 The declaration of the generic library package Containers.Indefinite_Hashed_Sets has the same contents and semantics as Containers.Hashed_Sets except:
- 3/2 • The generic formal Element_Type is indefinite.
- 4/2 • The actual Element parameter of access subprogram Process of Update_Element - Preserving_Key may be constrained even if Element_Type is unconstrained.

A.18.16 The Package Containers.Indefinite_Ordered_Sets

- 1/2 The language-defined generic package Containers.Indefinite_Ordered_Sets provides a set with the same operations as the package Containers.Ordered_Sets (see A.18.9), with the difference that the generic formal type Element_Type is indefinite.

Static Semantics

- 2/3 The declaration of the generic library package Containers.Indefinite_Ordered_Sets has the same contents and semantics as Containers.Ordered_Sets except:
- 3/2 • The generic formal Element_Type is indefinite.
- 4/2 • The actual Element parameter of access subprogram Process of Update_Element - Preserving_Key may be constrained even if Element_Type is unconstrained.

A.18.17 The Package Containers.Indefinite_Multiway_Trees

- 1/3 The language-defined generic package Containers.Indefinite_Multiway_Trees provides a multiway tree with the same operations as the package Containers.Multiway_Trees (see A.18.10), with the difference that the generic formal Element_Type is indefinite.

Static Semantics

- 2/3 The declaration of the generic library package Containers.Indefinite_Multiway_Trees has the same contents and semantics as Containers.Multiway_Trees except:
- 3/3 • The generic formal Element_Type is indefinite.

- The procedure with the profile: 4/3

```

procedure Insert_Child (Container : in out Tree;
                        Parent   : in      Cursor;
                        Before    : in      Cursor;
                        Position   : in out Cursor;
                        Count      : in      Count_Type := 1);

```

is omitted. 6/3
- The actual Element parameter of access subprogram Process of Update_Element may be constrained even if Element_Type is unconstrained. 7/3

A.18.18 The Package Containers.Indefinite_Holders

The language-defined generic package Containers.Indefinite_Holders provides a private type Holder and a set of operations for that type. A holder container holds a single element of an indefinite type. 1/3

A holder containers allows the declaration of an object that can be used like an uninitialized variable or component of an indefinite type. 2/3

A holder container may be *empty*. An empty holder does not contain an element. 3/3

Static Semantics

The generic library package Containers.Indefinite_Holders has the following declaration: 4/3

```

generic
  type Element_Type (<>) is private;
  with function "=" (Left, Right : Element_Type) return Boolean is <>;
package Ada.Containers.Indefinite_Holders is
  pragma Preelaborate(Indefinite_Holders);
  pragma Remote_Types(Indefinite_Holders);

  type Holder is tagged private;
  pragma Preelaborable_Initialization (Holder);
  Empty_Holder : constant Holder;
  function "=" (Left, Right : Holder) return Boolean;
  function To_Holder (New_Item : Element_Type) return Holder;
  function Is_Empty (Container : Holder) return Boolean;
  procedure Clear (Container : in out Holder);
  function Element (Container : Holder) return Element_Type;
  procedure Replace_Element (Container : in out Holder;
                           New_Item  : in      Element_Type);

  procedure Query_Element
    (Container : in Holder;
     Process   : not null access procedure (Element : in Element_Type));
  procedure Update_Element
    (Container : in Holder;
     Process   : not null access procedure (Element : in out Element_Type));
  procedure Assign (Target : in out Holder; Source : in Holder);
  function Copy (Source : Holder) return Holder;
  procedure Move (Target : in out Holder; Source : in out Holder);

private
  ... -- not specified by the language
end Ada.Containers.Indefinite_Holders;

```

- 22/3 The actual function for the generic formal function "=" on `Element_Type` values is expected to define a reflexive and symmetric relationship and return the same result value each time it is called with a particular pair of values. If it behaves in some other manner, the function "=" on holder values returns an unspecified value. The exact arguments and number of calls of this generic formal function by the function "=" on holder values are unspecified.
- 23/3 The type `Holder` is used to represent holder containers. The type `Holder` needs finalization (see 7.6).
- 24/3 `Empty_Holder` represents an empty holder object. If an object of type `Holder` is not otherwise initialized, it is initialized to the same value as `Empty_Holder`.
- 25/3 Some operations of this generic package have access-to-subprogram parameters. To ensure such operations are well-defined, they guard against certain actions by the designated subprogram. In particular, some operations check for "tampering with elements" of a container because they depend on elements of the container not being replaced.
- 26/3 A subprogram is said to *tamper with elements* of a holder object *H* if:
- 27/3 • It clears the element contained by *H*, that is, it calls the `Clear` procedure with *H* as a parameter;
 - 28/3 • It replaces the element contained by *H*, that is, it calls the `Replace_Element` procedure with *H* as a parameter;
 - 29/3 • It calls the `Move` procedure with *H* as a parameter;
 - 30/3 • It finalizes *H*.
- 31/3 **function** "=" (Left, Right : `Holder`) **return** `Boolean`;
- 32/3 If Left and Right denote the same holder object, then the function returns `True`. Otherwise, it compares the element contained in Left to the element contained in Right using the generic formal equality operator, returning the result of that operation. Any exception raised during the evaluation of element equality is propagated.
- 33/3 **function** `To_Holder` (New_Item : `Element_Type`) **return** `Holder`;
- 34/3 Returns a non-empty holder containing an element initialized to `New_Item`.
- 35/3 **function** `Is_Empty` (Container : `Holder`) **return** `Boolean`;
- 36/3 Returns `True` if the holder is empty, and `False` if it contains an element.
- 37/3 **procedure** `Clear` (Container : **in out** `Holder`);
- 38/3 Removes the element from Container. Container is empty after a successful `Clear` operation.
- 39/3 **function** `Element` (Container : `Holder`) **return** `Element_Type`;
- 40/3 If Container is empty, `Constraint_Error` is propagated. Otherwise, returns the element stored in Container.
- 41/3 **procedure** `Replace_Element` (Container : **in out** `Holder`;
 New_Item : **in** `Element_Type`);
- 42/3 `Replace_Element` assigns the value `New_Item` into Container, replacing any preexisting content of Container. Container is not empty after a successful call to `Replace_Element`.

procedure Query_Element 43/3
 (Container : **in** Holder;
 Process : **not null access procedure** (Element : **in** Element_Type));
 If Container is empty, Constraint_Error is propagated. Otherwise, Query_Element calls 44/3
 Process.**all** with the contained element as the argument. Program_Error is raised if Process.**all**
 tampers with the elements of Container. Any exception raised by Process.**all** is propagated.

procedure Update_Element 45/3
 (Container : **in** Holder;
 Process : **not null access procedure** (Element : **in out** Element_Type));
 If Container is empty, Constraint_Error is propagated. Otherwise, Update_Element calls 46/3
 Process.**all** with the contained element as the argument. Program_Error is raised if Process.**all**
 tampers with the elements of Container. Any exception raised by Process.**all** is propagated.

procedure Assign (Target : **in out** Holder; Source : **in** Holder); 47/3
 If Target denotes the same object as Source, the operation has no effect. If Source is empty, 48/3
 Clear (Target) is called. Otherwise, Replace_Element (Target, Element (Source)) is called.

function Copy (Source : Holder) **return** Holder; 49/3
 If Source is empty, returns an empty holder; otherwise, returns To_Holder (Element (Source)). 50/3

procedure Move (Target : **in out** Holder; Source : **in out** Holder); 51/3
 If Target denotes the same object as Source, then Move has no effect. Otherwise, the element 52/3
 contained by Source (if any) is removed from Source and inserted into Target, replacing any
 preexisting content. Source is empty after a successful call to Move.

Bounded (Run-Time) Errors

It is a bounded error for the actual function associated with a generic formal subprogram, when called as 53/3
 part of an operation of this package, to tamper with elements of any Holder parameter to the operation.
 Either Program_Error is raised, or the operation works as defined on the value of the Holder either prior
 to, or subsequent to, some or all of the modifications to the Holder.

It is a bounded error to call any subprogram declared in the visible part of Containers.Indefinite_Holders 54/3
 when the associated container has been finalized. If the operation takes Container as an **in out** parameter,
 then it raises Constraint_Error or Program_Error. Otherwise, the operation either proceeds as it would for
 an empty container, or it raises Constraint_Error or Program_Error.

Implementation Requirements

No storage associated with a holder object shall be lost upon assignment or scope exit. 55/3

The execution of an **assignment_statement** for a holder shall have the effect of copying the element (if 56/3
 any) from the source holder object to the target holder object.

Implementation Advice

Move should not copy elements, and should minimize copying of internal data structures. 57/3

If an exception is propagated from a holder operation, no storage should be lost, nor should the element be 58/3
 removed from a holder unless specified by the operation.

A.18.19 The Package Containers.Bounded_Vectors

- 1/3 The language-defined generic package Containers.Bounded_Vectors provides a private type Vector and a set of operations. It provides the same operations as the package Containers.Vectors (see A.18.2), with the difference that the maximum storage is bounded.

Static Semantics

- 2/3 The declaration of the generic library package Containers.Bounded_Vectors has the same contents and semantics as Containers.Vectors except:
- 3/3 • The **pragma** Preelaborate is replaced with **pragma** Pure.
 - 4/3 • The type Vector is declared with a discriminant that specifies the capacity:
5/3 **type** Vector (Capacity : Count_Type) **is tagged private**;
 - 6/3 • The type Vector needs finalization if and only if type Element_Type needs finalization.
 - 7/3 • In function Copy, if the Capacity parameter is equal to or greater than the length of Source, the vector capacity exactly equals the value of the Capacity parameter.
 - 8/3 • The description of Reserve_Capacity is replaced with:
9/3 If the specified Capacity is larger than the capacity of Container, then Reserve_Capacity propagates Capacity_Error. Otherwise, the operation has no effect.

Bounded (Run-Time) Errors

- 10/3 It is a bounded error to use a bounded vector if it was the target of an **assignment_statement** whose source was in the middle of an operation that disallows tampering with elements or cursors. Either Program_Error is raised, or the operation proceeds as defined.

Implementation Advice

- 11/3 Bounded vector objects should be implemented without implicit pointers or dynamic allocation.
- 12/3 The implementation advice for procedure Move to minimize copying does not apply.

A.18.20 The Package Containers.Bounded_Doubly_Linked_Lists

- 1/3 The language-defined generic package Containers.Bounded_Doubly_Linked_Lists provides a private type List and a set of operations. It provides the same operations as the package Containers.Doubly_Linked_Lists (see A.18.3), with the difference that the maximum storage is bounded.

Static Semantics

- 2/3 The declaration of the generic library package Containers.Bounded_Doubly_Linked_Lists has the same contents and semantics as Containers.Doubly_Linked_Lists except:
- 3/3 • The **pragma** Preelaborate is replaced with **pragma** Pure.
 - 4/3 • The type List is declared with a discriminant that specifies the capacity (maximum number of elements) as follows:
5/3 **type** List (Capacity : Count_Type) **is tagged private**;
 - 6/3 • The type List needs finalization if and only if type Element_Type needs finalization.
 - 7/3 • The allocation of internal storage includes a check that the capacity is not exceeded, and Capacity_Error is raised if this check fails.

- In procedure Assign, if Source length is greater than Target capacity, then Capacity_Error is propagated. 8/3
- The function Copy is replaced with: 9/3


```
function Copy (Source : List; Capacity : Count_Type := 0)
  return List; 10/3
```

If Capacity is 0, then the list capacity is the length of Source; if Capacity is equal to or greater than the length of Source, the list capacity equals the value of the Capacity parameter; otherwise, the operation propagates Capacity_Error. 11/3
- In the three-parameter procedure Splice whose Source has type List, if the sum of the length of Target and the length of Source is greater than the capacity of Target, then Splice propagates Capacity_Error. 12/3
- In the four-parameter procedure Splice, if the length of Target equals the capacity of Target, then Splice propagates Capacity_Error. 13/3

Bounded (Run-Time) Errors

It is a bounded error to use a bounded list if it was the target of an `assignment_statement` whose source was in the middle of an operation that disallows tampering with elements or cursors. Either Program_Error is raised, or the operation proceeds as defined. 14/3

Implementation Advice

Bounded list objects should be implemented without implicit pointers or dynamic allocation. 15/3

The implementation advice for procedure Move to minimize copying does not apply. 16/3

A.18.21 The Package Containers.Bounded_Hashed_Maps

The language-defined generic package Containers.Bounded_Hashed_Maps provides a private type Map and a set of operations. It provides the same operations as the package Containers.Hashed_Maps (see A.18.5), with the difference that the maximum storage is bounded. 1/3

Static Semantics

The declaration of the generic library package Containers.Bounded_Hashed_Maps has the same contents and semantics as Containers.Hashed_Maps except: 2/3

- The pragma Preelaborate is replaced with pragma Pure. 3/3
- The type Map is declared with discriminants that specify both the capacity (number of elements) and modulus (number of distinct hash values) of the hash table as follows: 4/3


```
type Map (Capacity : Count_Type;
  Modulus : Hash_Type) is tagged private; 5/3
```
- The type Map needs finalization if and only if type Key_Type or type Element_Type needs finalization. 6/3
- The description of Reserve_Capacity is replaced with: 7/3

If the specified Capacity is larger than the capacity of Container, then Reserve_Capacity propagates Capacity_Error. Otherwise, the operation has no effect. 8/3
- An additional operation is added immediately following Reserve_Capacity: 9/3


```
function Default_Modulus (Capacity : Count_Type) return Hash_Type; 10/3
```

11/3 Default_Modulus returns an implementation-defined value for the number of distinct hash values to be used for the given capacity (maximum number of elements).

- 12/3 • The function Copy is replaced with:

```
13/3 function Copy (Source : Map;  
                Capacity : Count_Type := 0;  
                Modulus : Hash_Type := 0) return Map;
```

14/3 Returns a map with key/element pairs initialized from the values in Source. If Capacity is 0, then the map capacity is the length of Source; if Capacity is equal to or greater than the length of Source, the map capacity is the value of the Capacity parameter; otherwise, the operation propagates Capacity_Error. If the Modulus argument is 0, then the map modulus is the value returned by a call to Default_Modulus with the map capacity as its argument; otherwise the map modulus is the value of the Modulus parameter.

Bounded (Run-Time) Errors

15/3 It is a bounded error to use a bounded map if it was the target of an assignment_statement whose source was in the middle of an operation that disallows tampering with elements or cursors. Either Program_Error is raised, or the operation proceeds as defined.

Implementation Advice

16/3 Bounded map objects should be implemented without implicit pointers or dynamic allocation.

17/3 The implementation advice for procedure Move to minimize copying does not apply.

A.18.22 The Package Containers.Bounded_Ordered_Maps

1/3 The language-defined generic package Containers.Bounded_Ordered_Maps provides a private type Map and a set of operations. It provides the same operations as the package Containers.Ordered_Maps (see A.18.6), with the difference that the maximum storage is bounded.

Static Semantics

2/3 The declaration of the generic library package Containers.Bounded_Ordered_Maps has the same contents and semantics as Containers.Ordered_Maps except:

- 3/3 • The pragma Preelaborate is replaced with pragma Pure.
- 4/3 • The type Map is declared with a discriminant that specifies the capacity (maximum number of elements) as follows:

```
5/3 type Map (Capacity : Count_Type) is tagged private;
```

- 6/3 • The type Map needs finalization if and only if type Element_Type needs finalization.
- 7/3 • The allocation of a new node includes a check that the capacity is not exceeded, and Capacity_Error is raised if this check fails.
- 8/3 • In procedure Assign, if Source length is greater than Target capacity, then Capacity_Error is propagated.

- 9/3 • The function Copy is replaced with:

```
10/3 function Copy (Source : Map;  
                Capacity : Count_Type := 0) return Map;
```

11/3 Returns a map with key/element pairs initialized from the values in Source. If Capacity is 0, then the map capacity is the length of Source; if Capacity is equal to or greater than the length of

Source, the map capacity is the specified value; otherwise, the operation propagates `Capacity_Error`.

Bounded (Run-Time) Errors

It is a bounded error to use a bounded map if it was the target of an `assignment_statement` whose source was in the middle of an operation that disallows tampering with elements or cursors. Either `Program_Error` is raised, or the operation proceeds as defined. 12/3

Implementation Advice

Bounded map objects should be implemented without implicit pointers or dynamic allocation. 13/3

The implementation advice for procedure `Move` to minimize copying does not apply. 14/3

A.18.23 The Package `Containers.Bounded_Hashed_Sets`

The language-defined generic package `Containers.Bounded_Hashed_Sets` provides a private type `Set` and a set of operations. It provides the same operations as the package `Containers.Hashed_Sets` (see A.18.8), with the difference that the maximum storage is bounded. 1/3

Static Semantics

The declaration of the generic library package `Containers.Bounded_Hashed_Sets` has the same contents and semantics as `Containers.Hashed_Sets` except: 2/3

- The pragma `Preelaborate` is replaced with `pragma Pure`. 3/3
- The type `Set` is declared with discriminants that specify both the capacity (number of elements) and modulus (number of distinct hash values) of the hash table as follows: 4/3

```
type Set (Capacity : Count_Type;  
          Modulus   : Hash_Type) is tagged private; 5/3
```

- The type `Set` needs finalization if and only if type `Key_Type` or type `Element_Type` needs finalization. 6/3
- The description of `Reserve_Capacity` is replaced with: 7/3

If the specified `Capacity` is larger than the capacity of `Container`, then `Reserve_Capacity` propagates `Capacity_Error`. Otherwise, the operation has no effect. 8/3

- An additional operation is added immediately following `Reserve_Capacity`: 9/3

```
function Default_Modulus (Capacity : Count_Type) return Hash_Type; 10/3
```

`Default_Modulus` returns an implementation-defined value for the number of distinct hash values to be used for the given capacity (maximum number of elements). 11/3

- The function `Copy` is replaced with: 12/3

```
function Copy (Source : Set;  
              Capacity : Count_Type := 0;  
              Modulus   : Hash_Type := 0) return Set; 13/3
```

Returns a set whose elements are initialized from the values in `Source`. If `Capacity` is 0, then the set capacity is the length of `Source`; if `Capacity` is equal to or greater than the length of `Source`, the set capacity is the value of the `Capacity` parameter; otherwise, the operation propagates `Capacity_Error`. If the `Modulus` argument is 0, then the set modulus is the value returned by a call to `Default_Modulus` with the set capacity as its argument; otherwise the set modulus is the value of the `Modulus` parameter. 14/3

Bounded (Run-Time) Errors

- 15/3 It is a bounded error to use a bounded set if it was the target of an **assignment_statement** whose source was in the middle of an operation that disallows tampering with elements or cursors. Either **Program_Error** is raised, or the operation proceeds as defined.

Implementation Advice

- 16/3 Bounded set objects should be implemented without implicit pointers or dynamic allocation.
- 17/3 The implementation advice for procedure **Move** to minimize copying does not apply.

A.18.24 The Package Containers.Bounded_Ordered_Sets

- 1/3 The language-defined generic package **Containers.Bounded_Ordered_Sets** provides a private type **Set** and a set of operations. It provides the same operations as the package **Containers.Ordered_Sets** (see A.18.9), with the difference that the maximum storage is bounded.

Static Semantics

- 2/3 The declaration of the generic library package **Containers.Bounded_Ordered_Sets** has the same contents and semantics as **Containers.Ordered_Sets** except:
- 3/3 • The **pragma Preelaborate** is replaced with **pragma Pure**.
 - 4/3 • The type **Set** is declared with a discriminant that specifies the capacity (maximum number of elements) as follows:


```
5/3     type Set (Capacity : Count_Type) is tagged private;
```
 - 6/3 • The type **Set** needs finalization if and only if type **Element_Type** needs finalization.
 - 7/3 • If **Insert** (or **Include**) adds an element, a check is made that the capacity is not exceeded, and **Capacity_Error** is raised if this check fails.
 - 8/3 • In procedure **Assign**, if **Source** length is greater than **Target** capacity, then **Capacity_Error** is propagated.
 - 9/3 • The function **Copy** is replaced with:


```
10/3     function Copy (Source : Set;
                      Capacity : Count_Type := 0) return Set;
```
 - 11/3 Returns a set whose elements are initialized from the values in **Source**. If **Capacity** is 0, then the set capacity is the length of **Source**; if **Capacity** is equal to or greater than the length of **Source**, the set capacity is the specified value; otherwise, the operation propagates **Capacity_Error**.

Bounded (Run-Time) Errors

- 12/3 It is a bounded error to use a bounded set if it was the target of an **assignment_statement** whose source was in the middle of an operation that disallows tampering with elements or cursors. Either **Program_Error** is raised, or the operation proceeds as defined.

Implementation Advice

- 13/3 Bounded set objects should be implemented without implicit pointers or dynamic allocation.
- 14/3 The implementation advice for procedure **Move** to minimize copying does not apply.

A.18.25 The Package Containers.Bounded_Multiway_Trees

The language-defined generic package Containers.Bounded_Multiway_Trees provides a private type Tree and a set of operations. It provides the same operations as the package Containers.Multiway_Trees (see A.18.10), with the difference that the maximum storage is bounded. 1/3

Static Semantics

The declaration of the generic library package Containers.Bounded_Multiway_Trees has the same contents and semantics as Containers.Multiway_Trees except: 2/3

- The pragma Preelaborate is replaced with pragma Pure. 3/3
- The type Tree is declared with a discriminant that specifies the capacity (maximum number of elements) as follows: 4/3

```
type Tree (Capacity : Count_Type) is tagged private; 5/3
```

- The type Tree needs finalization if and only if type Element_Type needs finalization. 6/3
- The allocation of internal storage includes a check that the capacity is not exceeded, and Capacity_Error is raised if this check fails. 7/3
- In procedure Assign, if Source length is greater than Target capacity, then Capacity_Error is propagated. 8/3
- Function Copy is declared as follows: 9/3

```
function Copy (Source : Tree; Capacity : Count_Type := 0) 10/3
return List;
```

If Capacity is 0, then the tree capacity is the count of Source; if Capacity is equal to or greater than Source.Count, the tree capacity equals the value of the Capacity parameter; otherwise, the operation propagates Capacity_Error. 11/3

- In the five-parameter procedure Splice_Subtree, if Source is not the same object as Target, and if the sum of Target.Count and Subtree_Count (Position) is greater than Target.Capacity, then Splice_Subtree propagates Capacity_Error. 12/3
- In the five-parameter procedure Splice_Children, if Source is not the same object as Target, and if the sum of Target.Count and Subtree_Count (Source_Parent)-1 is greater than Target.Capacity, then Splice_Children propagates Capacity_Error. 13/3

Bounded (Run-Time) Errors

It is a bounded error to use a bounded tree if it was the target of an assignment_statement whose source was in the middle of an operation that disallows tampering with elements or cursors. Either Program_Error is raised, or the operation proceeds as defined. 14/3

Implementation Advice

Bounded tree objects should be implemented without implicit pointers or dynamic allocation. 15/3

The implementation advice for procedure Move to minimize copying does not apply. 16/3

A.18.26 Array Sorting

The language-defined generic procedures Containers.Generic_Array_Sort, Containers.Generic_Constrained_Array_Sort, and Containers.Generic_Sort provide sorting on arbitrary array types. 1/3

2/2 The generic library procedure Containers.Generic_Array_Sort has the following declaration:

```
3/2  generic
      type Index_Type is (<>);
      type Element_Type is private;
      type Array_Type is array (Index_Type range <>) of Element_Type;
      with function "<" (Left, Right : Element_Type)
          return Boolean is <>;
  procedure Ada.Containers.Generic_Array_Sort (Container : in out Array_Type);
  pragma Pure (Ada.Containers.Generic_Array_Sort);
```

4/2 Reorders the elements of Container such that the elements are sorted smallest first as determined by the generic formal "<" operator provided. Any exception raised during evaluation of "<" is propagated.

5/3 The actual function for the generic formal function "<" of Generic_Array_Sort is expected to return the same value each time it is called with a particular pair of element values. It should define a strict weak ordering relationship (see A.18); it should not modify Container. If the actual for "<" behaves in some other manner, the behavior of the instance of Generic_Array_Sort is unspecified. How many times Generic_Array_Sort calls "<" is unspecified.

6/2 The generic library procedure Containers.Generic_Constrained_Array_Sort has the following declaration:

```
7/2  generic
      type Index_Type is (<>);
      type Element_Type is private;
      type Array_Type is array (Index_Type) of Element_Type;
      with function "<" (Left, Right : Element_Type)
          return Boolean is <>;
  procedure Ada.Containers.Generic_Constrained_Array_Sort
      (Container : in out Array_Type);
  pragma Pure (Ada.Containers.Generic_Constrained_Array_Sort);
```

8/2 Reorders the elements of Container such that the elements are sorted smallest first as determined by the generic formal "<" operator provided. Any exception raised during evaluation of "<" is propagated.

9/3 The actual function for the generic formal function "<" of Generic_Constrained_Array_Sort is expected to return the same value each time it is called with a particular pair of element values. It should define a strict weak ordering relationship (see A.18); it should not modify Container. If the actual for "<" behaves in some other manner, the behavior of the instance of Generic_Constrained_Array_Sort is unspecified. How many times Generic_Constrained_Array_Sort calls "<" is unspecified.

9.1/3 The generic library procedure Containers.Generic_Sort has the following declaration:

```
9.2/3  generic
      type Index_Type is (<>);
      with function Before (Left, Right : Index_Type) return Boolean;
      with procedure Swap (Left, Right : Index_Type);
  procedure Ada.Containers.Generic_Sort
      (First, Last : Index_Type'Base);
  pragma Pure (Ada.Containers.Generic_Sort);
```

9.3/3 Reorders the elements of an indexable structure, over the range First .. Last, such that the elements are sorted in the ordering determined by the generic formal function Before. The generic formal Before compares the elements having the given indices, and the generic formal Swap exchanges the values of the indicated elements. Any exception raised during evaluation of Before or Swap is propagated.

The actual function for the generic formal function `Before` of `Generic_Sort` is expected to return the same value each time it is called with index values that identify a particular pair of element values. It should define a strict weak ordering relationship (see A.18); it should not modify the elements. The actual function for the generic formal `Swap` should exchange the values of the indicated elements. If the actual for either `Before` or `Swap` behaves in some other manner, the behavior of `Generic_Sort` is unspecified. How many times the `Generic_Sort` calls `Before` or `Swap` is unspecified. 9.4/3

Implementation Advice

The worst-case time complexity of a call on an instance of `Containers.Generic_Array_Sort` or `Containers.Generic_Constrained_Array_Sort` should be $O(N^2)$ or better, and the average time complexity should be better than $O(N^2)$, where N is the length of the `Container` parameter. 10/2

`Containers.Generic_Array_Sort` and `Containers.Generic_Constrained_Array_Sort` should minimize copying of elements. 11/2

Annex B

(normative)

Interface to Other Languages

This Annex describes features for writing mixed-language programs. General interface support is presented first; then specific support for C, COBOL, and Fortran is defined, in terms of language interface packages for each of these languages.

B.1 Interfacing Pragmas

A **pragma Import** is used to import an entity defined in a foreign language into an Ada program, thus allowing a foreign-language subprogram to be called from Ada, or a foreign-language variable to be accessed from Ada. In contrast, a **pragma Export** is used to export an Ada entity to a foreign language, thus allowing an Ada subprogram to be called from a foreign language, or an Ada object to be accessed from a foreign language. The **pragmas Import** and **Export** are intended primarily for objects and subprograms, although implementations are allowed to support other entities.

A **pragma Convention** is used to specify that an Ada entity should use the conventions of another language. It is intended primarily for types and “callback” subprograms. For example, “**pragma Convention**(Fortran, Matrix);” implies that Matrix should be represented according to the conventions of the supported Fortran implementation, namely column-major order.

A **pragma Linker_Options** is used to specify the system linker parameters needed when a given compilation unit is included in a partition.

Syntax

An *interfacing pragma* is a representation **pragma** that is one of the **pragmas Import**, **Export**, or **Convention**. Their forms, together with that of the related **pragma Linker_Options**, are as follows:

```
pragma Import(
  [Convention =>] convention_identifier, [Entity =>] local_name
  [, [External_Name =>] string_expression] [, [Link_Name =>] string_expression]);

pragma Export(
  [Convention =>] convention_identifier, [Entity =>] local_name
  [, [External_Name =>] string_expression] [, [Link_Name =>] string_expression]);

pragma Convention([Convention =>] convention_identifier, [Entity =>] local_name);

pragma Linker_Options(string_expression);
```

A **pragma Linker_Options** is allowed only at the place of a **declarative_item**.

For **pragmas Import** and **Export**, the argument for *Link_Name* shall not be given without the *pragma_argument_identifier* unless the argument for *External_Name* is given.

Name Resolution Rules

The expected type for a *string_expression* in an interfacing **pragma** or in **pragma Linker_Options** is String.

Legality Rules

The *convention_identifier* of an interfacing **pragma** shall be the name of a *convention*. The convention names are implementation defined, except for certain language-defined ones, such as Ada and Intrinsic, as

explained in 6.3.1, “Conformance Rules”. Additional convention names generally represent the calling conventions of foreign languages, language implementations, or specific run-time models. The convention of a callable entity is its *calling convention*.

If *L* is a *convention_identifier* for a language, then a type *T* is said to be *compatible with convention L*, (alternatively, is said to be an *L-compatible type*) if any of the following conditions are met:

- *T* is declared in a language interface package corresponding to *L* and is defined to be *L-compatible* (see B.3, B.3.1, B.3.2, B.4, B.5),
- Convention *L* has been specified for *T* in a **pragma Convention**, and *T* is *eligible for convention L*; that is:
 - *T* is an array type with either an unconstrained or statically-constrained first subtype, and its component type is *L-compatible*,
 - *T* is a record type that has no discriminants and that only has components with statically-constrained subtypes, and each component type is *L-compatible*,
 - *T* is an access-to-object type, its designated type is *L-compatible*, and its designated subtype is not an unconstrained array subtype,
 - *T* is an access-to-subprogram type, and its designated profile's parameter and result types are all *L-compatible*.
- *T* is derived from an *L-compatible* type,
- The implementation permits *T* as an *L-compatible* type.

If **pragma Convention** applies to a type, then the type shall either be compatible with or eligible for the convention specified in the pragma.

A **pragma Import** shall be the completion of a declaration. Notwithstanding any rule to the contrary, a **pragma Import** may serve as the completion of any kind of (explicit) declaration if supported by an implementation for that kind of declaration. If a completion is a **pragma Import**, then it shall appear in the same **declarative_part**, **package_specification**, **task_definition** or **protected_definition** as the declaration. For a library unit, it shall appear in the same **compilation**, before any subsequent **compilation_units** other than **pragmas**. If the **local_name** denotes more than one entity, then the **pragma Import** is the completion of all of them.

An entity specified as the Entity argument to a **pragma Import** (or **pragma Export**) is said to be *imported* (respectively, *exported*).

The declaration of an imported object shall not include an explicit initialization expression. Default initializations are not performed.

The type of an imported or exported object shall be compatible with the convention specified in the corresponding **pragma**.

For an imported or exported subprogram, the result and parameter types shall each be compatible with the convention specified in the corresponding **pragma**.

The external name and link name *string_expressions* of a **pragma Import** or **Export**, and the *string_expression* of a **pragma Linker_Options**, shall be static.

Static Semantics

Import, Export, and Convention pragmas are representation pragmas that specify the *convention* aspect of representation. In addition, Import and Export pragmas specify the *imported* and *exported* aspects of representation, respectively. 28

An interfacing pragma is a program unit pragma when applied to a program unit (see 10.1.5). 29

An interfacing pragma defines the convention of the entity denoted by the *local_name*. The convention represents the calling convention or representation convention of the entity. For an access-to-subprogram type, it represents the calling convention of designated subprograms. In addition: 30

- A pragma Import specifies that the entity is defined externally (that is, outside the Ada program). 31
- A pragma Export specifies that the entity is used externally. 32
- A pragma Import or Export optionally specifies an entity's external name, link name, or both. 33

An *external name* is a string value for the name used by a foreign language program either for an entity that an Ada program imports, or for referring to an entity that an Ada program exports. 34

A *link name* is a string value for the name of an exported or imported entity, based on the conventions of the foreign language's compiler in interfacing with the system's linker tool. 35

The meaning of link names is implementation defined. If neither a link name nor the Address attribute of an imported or exported entity is specified, then a link name is chosen in an implementation-defined manner, based on the external name if one is specified. 36

Pragma Linker_Options has the effect of passing its string argument as a parameter to the system linker (if one exists), if the immediately enclosing compilation unit is included in the partition being linked. The interpretation of the string argument, and the way in which the string arguments from multiple Linker_Options pragmas are combined, is implementation defined. 37

Dynamic Semantics

Notwithstanding what this International Standard says elsewhere, the elaboration of a declaration denoted by the *local_name* of a pragma Import does not create the entity. Such an elaboration has no other effect than to allow the defining name to denote the external entity. 38

Erroneous Execution

It is the programmer's responsibility to ensure that the use of interfacing pragmas does not violate Ada semantics; otherwise, program execution is erroneous. 38.1/2

Implementation Advice

If an implementation supports pragma Export to a given language, then it should also allow the main subprogram to be written in that language. It should support some mechanism for invoking the elaboration of the Ada library units included in the system, and for invoking the finalization of the environment task. On typical systems, the recommended mechanism is to provide two subprograms whose link names are "adainit" and "adafinal". Adainit should contain the elaboration code for library units. Adafinal should contain the finalization code. These subprograms should have no effect the second and subsequent time they are called. 39

Automatic elaboration of preelaborated packages should be provided when pragma Export is supported. 40

For each supported convention *L* other than Intrinsic, an implementation should support Import and Export pragmas for objects of *L*-compatible types and for subprograms, and pragma Convention for *L*-eligible types and for subprograms, presuming the other language has corresponding features. Pragma Convention need not be supported for scalar types.

NOTES

1 Implementations may place restrictions on interfacing pragmas; for example, requiring each exported entity to be declared at the library level.

2 A pragma Import specifies the conventions for accessing external entities. It is possible that the actual entity is written in assembly language, but reflects the conventions of a particular language. For example, **pragma** Import(Ada, ...) can be used to interface to an assembly language routine that obeys the Ada compiler's calling conventions.

3 To obtain “call-back” to an Ada subprogram from a foreign language environment, **pragma** Convention should be specified both for the access-to-subprogram type and the specific subprogram(s) to which 'Access is applied.

4 It is illegal to specify more than one of Import, Export, or Convention for a given entity.

5 The *local_name* in an interfacing pragma can denote more than one entity in the case of overloading. Such a pragma applies to all of the denoted entities.

6 See also 13.8, “Machine Code Insertions”.

7 If both *External_Name* and *Link_Name* are specified for an Import or Export pragma, then the *External_Name* is ignored.

This paragraph was deleted.

Examples

Example of interfacing pragmas:

```
package Fortran_Library is
  function Sqrt(X : Float) return Float;
  function Exp (X : Float) return Float;
private
  pragma Import(Fortran, Sqrt);
  pragma Import(Fortran, Exp);
end Fortran_Library;
```

B.2 The Package Interfaces

Package Interfaces is the parent of several library packages that declare types and other entities useful for interfacing to foreign languages. It also contains some implementation-defined types that are useful across more than one language (in particular for interfacing to assembly language).

Static Semantics

The library package Interfaces has the following skeletal declaration:

```
package Interfaces is
  pragma Pure(Interfaces);
  type Integer_n is range -2**(n-1) .. 2**(n-1) - 1; --2's complement
  type Unsigned_n is mod 2**n;
```



```

function Shift_Left (Value : Unsigned_n; Amount : Natural)
    return Unsigned_n;
function Shift_Right (Value : Unsigned_n; Amount : Natural)
    return Unsigned_n;
function Shift_Right_Arithmetic (Value : Unsigned_n; Amount : Natural)
    return Unsigned_n;
function Rotate_Left (Value : Unsigned_n; Amount : Natural)
    return Unsigned_n;
function Rotate_Right (Value : Unsigned_n; Amount : Natural)
    return Unsigned_n;
...
end Interfaces;

```

Implementation Requirements

An implementation shall provide the following declarations in the visible part of package Interfaces: 7

- Signed and modular integer types of n bits, if supported by the target architecture, for each n that is at least the size of a storage element and that is a factor of the word size. The names of these types are of the form Integer_ n for the signed types, and Unsigned_ n for the modular types; 8
- For each such modular type in Interfaces, shifting and rotating subprograms as specified in the declaration of Interfaces above. These subprograms are Intrinsic. They operate on a bit-by-bit basis, using the binary representation of the value of the operands to yield a binary representation for the result. The Amount parameter gives the number of bits by which to shift or rotate. For shifting, zero bits are shifted in, except in the case of Shift_Right_Arithmetic, where one bits are shifted in if Value is at least half the modulus. 9
- Floating point types corresponding to each floating point format fully supported by the hardware. 10

Support for interfacing to any foreign language is optional. However, an implementation shall not provide any attribute, library unit, or pragma having the same name as an attribute, library unit, or pragma (respectively) specified in the following clauses of this Annex unless the provided construct is either as specified in those clauses or is more limited in capability than that required by those clauses. A program that attempts to use an unsupported capability of this Annex shall either be identified by the implementation before run time or shall raise an exception at run time. 10.1/2

Implementation Permissions

An implementation may provide implementation-defined library units that are children of Interfaces, and may add declarations to the visible part of Interfaces in addition to the ones defined above. 11

A child package of package Interfaces with the name of a convention may be provided independently of whether the convention is supported by the pragma Convention and vice versa. Such a child package should contain any declarations that would be useful for interfacing to the language (implementation) represented by the convention. Any declarations useful for interfacing to any language on the given hardware architecture should be provided directly in Interfaces. 11.1/2

Implementation Advice

This paragraph was deleted. 12/2

An implementation supporting an interface to C, COBOL, or Fortran should provide the corresponding package or packages described in the following clauses. 13

B.3 Interfacing with C and C++

- 1/2 The facilities relevant to interfacing with the C language and the corresponding subset of the C++ language are the package `Interfaces.C` and its children; support for the `Import`, `Export`, and `Convention` pragmas with *convention_identifier* `C`; and support for the `Convention` pragma with *convention_identifier* `C_Pass_By_Copy`.
- 2/2 The package `Interfaces.C` contains the basic types, constants and subprograms that allow an Ada program to pass scalars and strings to C and C++ functions. When this clause mentions a C entity, the reference also applies to the corresponding entity in C++.

Static Semantics

- 3 The library package `Interfaces.C` has the following declaration:

```

4  package Interfaces.C is
5      pragma Pure(C);
6      -- Declarations based on C's <limits.h>
7      CHAR_BIT : constant := implementation-defined; -- typically 8
8      SCHAR_MIN : constant := implementation-defined; -- typically -128
9      SCHAR_MAX : constant := implementation-defined; -- typically 127
10     UCHAR_MAX : constant := implementation-defined; -- typically 255
11
12     -- Signed and Unsigned Integers
13     type int is range implementation-defined;
14     type short is range implementation-defined;
15     type long is range implementation-defined;
16
17     type signed_char is range SCHAR_MIN .. SCHAR_MAX;
18     for signed_char'Size use CHAR_BIT;
19
20     type unsigned is mod implementation-defined;
21     type unsigned_short is mod implementation-defined;
22     type unsigned_long is mod implementation-defined;
23
24     type unsigned_char is mod (UCHAR_MAX+1);
25     for unsigned_char'Size use CHAR_BIT;
26
27     subtype plain_char is implementation-defined;
28     type ptrdiff_t is range implementation-defined;
29     type size_t is mod implementation-defined;
30
31     -- Floating Point
32     type C_float is digits implementation-defined;
33     type C_double is digits implementation-defined;
34     type C_long_double is digits implementation-defined;
35
36     -- Characters and Strings
37     type char is <implementation-defined character type>;
38     nul : constant char := implementation-defined;
39
40     function To_C (Item : in Character) return char;
41     function To_Ada (Item : in char) return Character;
42
43     type char_array is array (size_t range <>) of aliased char;
44     pragma Pack(char_array);
45     for char_array'Component_Size use CHAR_BIT;
46
47     function Is_Nul_Terminated (Item : in char_array) return Boolean;
48
49     function To_C (Item : in String;
50                  Append_Nul : in Boolean := True)
51     return char_array;

```

```

function To_Ada (Item      : in char_array;
                  Trim_Nul : in Boolean := True)
    return String;
26

procedure To_C (Item      : in String;
                 Target    : out char_array;
                 Count     : out size_t;
                 Append_Nul : in Boolean := True);
27

procedure To_Ada (Item      : in char_array;
                  Target    : out String;
                  Count     : out Natural;
                  Trim_Nul : in Boolean := True);
28

-- Wide Character and Wide String
29

type wchar_t is <implementation-defined character type>;
30/1

wide_nul : constant wchar_t := implementation-defined;
31/1

function To_C (Item : in Wide_Character) return wchar_t;
32

function To_Ada (Item : in wchar_t) return Wide_Character;

type wchar_array is array (size_t range <>) of aliased wchar_t;
33

pragma Pack(wchar_array);
34

function Is_Nul_Terminated (Item : in wchar_array) return Boolean;
35

function To_C (Item      : in Wide_String;
               Append_Nul : in Boolean := True)
    return wchar_array;
36

function To_Ada (Item      : in wchar_array;
                 Trim_Nul : in Boolean := True)
    return Wide_String;
37

procedure To_C (Item      : in Wide_String;
                 Target    : out wchar_array;
                 Count     : out size_t;
                 Append_Nul : in Boolean := True);
38

procedure To_Ada (Item      : in wchar_array;
                  Target    : out Wide_String;
                  Count     : out Natural;
                  Trim_Nul : in Boolean := True);
39

-- ISO/IEC 10646:2003 compatible types defined by ISO/IEC TR 19769:2004.
39.1/2

type char16_t is <implementation-defined character type>;
39.2/2

char16_nul : constant char16_t := implementation-defined;
39.3/2

function To_C (Item : in Wide_Character) return char16_t;
39.4/2

function To_Ada (Item : in char16_t) return Wide_Character;

type char16_array is array (size_t range <>) of aliased char16_t;
39.5/2

pragma Pack(char16_array);
39.6/2

function Is_Nul_Terminated (Item : in char16_array) return Boolean;
39.7/2

function To_C (Item      : in Wide_String;
               Append_Nul : in Boolean := True)
    return char16_array;

function To_Ada (Item      : in char16_array;
                 Trim_Nul : in Boolean := True)
    return Wide_String;
39.8/2

procedure To_C (Item      : in Wide_String;
                 Target    : out char16_array;
                 Count     : out size_t;
                 Append_Nul : in Boolean := True);
39.9/2

procedure To_Ada (Item      : in char16_array;
                  Target    : out Wide_String;
                  Count     : out Natural;
                  Trim_Nul : in Boolean := True);
39.10/2

```

```

39.11/2    type char32_t is <implementation-defined character type>;
39.12/2    char32_nul : constant char32_t := implementation-defined;
39.13/2    function To_C (Item : in Wide_Wide_Character) return char32_t;
39.13/2    function To_Ada (Item : in char32_t) return Wide_Wide_Character;
39.14/2    type char32_array is array (size_t range <>) of aliased char32_t;
39.15/2    pragma Pack(char32_array);
39.16/2    function Is_Nul_Terminated (Item : in char32_array) return Boolean;
39.16/2    function To_C (Item      : in Wide_Wide_String;
39.16/2                Append_Nul : in Boolean := True)
39.16/2    return char32_array;
39.17/2    function To_Ada (Item      : in char32_array;
39.17/2                Trim_Nul  : in Boolean := True)
39.17/2    return Wide_Wide_String;
39.18/2    procedure To_C (Item      : in Wide_Wide_String;
39.18/2                Target     : out char32_array;
39.18/2                Count      : out size_t;
39.18/2                Append_Nul : in Boolean := True);
39.19/2    procedure To_Ada (Item      : in char32_array;
39.19/2                Target     : out Wide_Wide_String;
39.19/2                Count      : out Natural;
39.19/2                Trim_Nul  : in Boolean := True);
40        Terminator_Error : exception;
41    end Interfaces.C;

```

Each of the types declared in Interfaces.C is C-compatible.

The types int, short, long, unsigned, ptrdiff_t, size_t, double, char, wchar_t, char16_t, and char32_t correspond respectively to the C types having the same names. The types signed_char, unsigned_short, unsigned_long, unsigned_char, C_float, and long_double correspond respectively to the C types signed char, unsigned short, unsigned long, unsigned char, float, and long double.

The type of the subtype plain_char is either signed_char or unsigned_char, depending on the C implementation.

```

45    function To_C (Item : in Character) return char;
45    function To_Ada (Item : in char) return Character;

```

The functions To_C and To_Ada map between the Ada type Character and the C type char.

```

47    function Is_Nul_Terminated (Item : in char_array) return Boolean;

```

The result of Is_Nul_Terminated is True if Item contains nul, and is False otherwise.

```

49    function To_C (Item : in String;      Append_Nul : in Boolean := True)
49    return char_array;
50/2    function To_Ada (Item : in char_array; Trim_Nul : in Boolean := True)
50/2    return String;

```

The result of To_C is a char_array value of length Item'Length (if Append_Nul is False) or Item'Length+1 (if Append_Nul is True). The lower bound is 0. For each component Item(I), the corresponding component in the result is To_C applied to Item(I). The value nul is appended if Append_Nul is True. If Append_Nul is False and Item'Length is 0, then To_C propagates Constraint_Error.

The result of To_Ada is a String whose length is Item'Length (if Trim_Nul is False) or the length of the slice of Item preceding the first nul (if Trim_Nul is True). The lower bound of the result is 1. If Trim_Nul is False, then for each component Item(I) the corresponding component in the

result is To_Ada applied to Item(I). If Trim_Nul is True, then for each component Item(I) before the first nul the corresponding component in the result is To_Ada applied to Item(I). The function propagates Terminator_Error if Trim_Nul is True and Item does not contain nul.

```

procedure To_C (Item      : in String;
                Target    : out char_array;
                Count     : out size_t;
                Append_Nul : in Boolean := True);
52

procedure To_Ada (Item      : in char_array;
                 Target    : out String;
                 Count     : out Natural;
                 Trim_Nul  : in Boolean := True);

```

For procedure To_C, each element of Item is converted (via the To_C function) to a char, which is assigned to the corresponding element of Target. If Append_Nul is True, nul is then assigned to the next element of Target. In either case, Count is set to the number of Target elements assigned. If Target is not long enough, Constraint_Error is propagated. 53

For procedure To_Ada, each element of Item (if Trim_Nul is False) or each element of Item preceding the first nul (if Trim_Nul is True) is converted (via the To_Ada function) to a Character, which is assigned to the corresponding element of Target. Count is set to the number of Target elements assigned. If Target is not long enough, Constraint_Error is propagated. If Trim_Nul is True and Item does not contain nul, then Terminator_Error is propagated. 54

```

function Is_Nul_Terminated (Item : in wchar_array) return Boolean;
55

The result of Is_Nul_Terminated is True if Item contains wide_nul, and is False otherwise.
56

```

```

function To_C (Item : in Wide_Character) return wchar_t;
function To_Ada (Item : in wchar_t) return Wide_Character;
57

```

To_C and To_Ada provide the mappings between the Ada and C wide character types. 58

```

function To_C (Item      : in Wide_String;
               Append_Nul : in Boolean := True)
return wchar_array;
59

function To_Ada (Item      : in wchar_array;
                 Trim_Nul  : in Boolean := True)
return Wide_String;

procedure To_C (Item      : in Wide_String;
                Target    : out wchar_array;
                Count     : out size_t;
                Append_Nul : in Boolean := True);

procedure To_Ada (Item      : in wchar_array;
                  Target    : out Wide_String;
                  Count     : out Natural;
                  Trim_Nul  : in Boolean := True);

```

The To_C and To_Ada subprograms that convert between Wide_String and wchar_array have analogous effects to the To_C and To_Ada subprograms that convert between String and char_array, except that wide_nul is used instead of nul. 60

```

function Is_Nul_Terminated (Item : in char16_array) return Boolean;
60.1/2

The result of Is_Nul_Terminated is True if Item contains char16_nul, and is False otherwise.
60.2/2

```

60.3/2 **function** To_C (Item : **in** Wide_Character) **return** char16_t;
function To_Ada (Item : **in** char16_t) **return** Wide_Character;

60.4/2 To_C and To_Ada provide mappings between the Ada and C 16-bit character types.

60.5/2 **function** To_C (Item : **in** Wide_String;
Append_Nul : **in** Boolean := True)
return char16_array;
function To_Ada (Item : **in** char16_array;
Trim_Nul : **in** Boolean := True)
return Wide_String;
procedure To_C (Item : **in** Wide_String;
Target : **out** char16_array;
Count : **out** size_t;
Append_Nul : **in** Boolean := True);
procedure To_Ada (Item : **in** char16_array;
Target : **out** Wide_String;
Count : **out** Natural;
Trim_Nul : **in** Boolean := True);

60.6/2 The To_C and To_Ada subprograms that convert between Wide_String and char16_array have analogous effects to the To_C and To_Ada subprograms that convert between String and char_array, except that char16_nul is used instead of nul.

60.7/2 **function** Is_Nul_Terminated (Item : **in** char32_array) **return** Boolean;

60.8/2 The result of Is_Nul_Terminated is True if Item contains char16_nul, and is False otherwise.

60.9/2 **function** To_C (Item : **in** Wide_Wide_Character) **return** char32_t;
function To_Ada (Item : **in** char32_t) **return** Wide_Wide_Character;

60.10/2 To_C and To_Ada provide mappings between the Ada and C 32-bit character types.

60.11/2 **function** To_C (Item : **in** Wide_Wide_String;
Append_Nul : **in** Boolean := True)
return char32_array;
function To_Ada (Item : **in** char32_array;
Trim_Nul : **in** Boolean := True)
return Wide_Wide_String;
procedure To_C (Item : **in** Wide_Wide_String;
Target : **out** char32_array;
Count : **out** size_t;
Append_Nul : **in** Boolean := True);
procedure To_Ada (Item : **in** char32_array;
Target : **out** Wide_Wide_String;
Count : **out** Natural;
Trim_Nul : **in** Boolean := True);

60.12/2 The To_C and To_Ada subprograms that convert between Wide_Wide_String and char32_array have analogous effects to the To_C and To_Ada subprograms that convert between String and char_array, except that char32_nul is used instead of nul.

60.13/1 A Convention pragma with *convention_identifier* C_Pass_By_Copy shall only be applied to a type.

60.14/2 The eligibility rules in B.1 do not apply to convention C_Pass_By_Copy. Instead, a type T is eligible for convention C_Pass_By_Copy if T is an unchecked union type or if T is a record type that has no discriminants and that only has components with statically constrained subtypes, and each component is C-compatible.

If a type is C_Pass_By_Copy-compatible then it is also C-compatible.

60.15/1

Implementation Requirements

An implementation shall support pragma Convention with a C *convention_identifier* for a C-eligible type (see B.1). An implementation shall support pragma Convention with a C_Pass_By_Copy *convention_identifier* for a C_Pass_By_Copy-eligible type. 61/1

Implementation Permissions

An implementation may provide additional declarations in the C interface packages. 62

An implementation need not support an interfacing pragma specifying convention C or C_Pass_By_Copy in the following cases: 62.1/3

- an Export or Convention pragma applied to a subprogram which has a parameter of an unconstrained array subtype; 62.2/3
- an interfacing pragma applied to a function with an unconstrained array result subtype; 62.3/3
- an interfacing pragma applied to an object whose nominal subtype is an unconstrained array subtype. 62.4/3

Implementation Advice

The constants nul, wide_nul, char16_nul, and char32_nul should have a representation of zero. 62.5/2

An implementation should support the following interface correspondences between Ada and C. 63

- An Ada procedure corresponds to a void-returning C function. 64
- An Ada function corresponds to a non-void C function. 65
- An Ada **in** scalar parameter is passed as a scalar argument to a C function. 66
- An Ada **in** parameter of an access-to-object type with designated type T is passed as a t* argument to a C function, where t is the C type corresponding to the Ada type T. 67
- An Ada **access** T parameter, or an Ada **out** or **in out** parameter of an elementary type T, is passed as a t* argument to a C function, where t is the C type corresponding to the Ada type T. In the case of an elementary **out** or **in out** parameter, a pointer to a temporary copy is used to preserve by-copy semantics. 68
- An Ada parameter of a (record) type T of convention C_Pass_By_Copy, of mode **in**, is passed as a t argument to a C function, where t is the C struct corresponding to the Ada type T. 68.1/2
- An Ada parameter of a record type T, of any mode, other than an **in** parameter of a type of convention C_Pass_By_Copy, is passed as a t* argument to a C function, where t is the C struct corresponding to the Ada type T. 69/2
- An Ada parameter of an array type with component type T, of any mode, is passed as a t* argument to a C function, where t is the C type corresponding to the Ada type T. 70
- An Ada parameter of an access-to-subprogram type is passed as a pointer to a C function whose prototype corresponds to the designated subprogram's specification. 71
- An Ada parameter of a private type is passed as specified for the full view of the type. 71.1/3
- The rules of correspondence given above for parameters of mode **in** also apply to the return object of a function. 71.2/3

This paragraph was deleted.

71.3/3

NOTES

- 72 8 Values of type `char_array` are not implicitly terminated with `nul`. If a `char_array` is to be passed as a parameter to an imported C function requiring nul termination, it is the programmer's responsibility to obtain this effect.
- 73 9 To obtain the effect of C's `sizeof(item_type)`, where `Item_Type` is the corresponding Ada type, evaluate the expression: `size_t(Item_Type'Size/CHAR_BIT)`.
- 74/2 *This paragraph was deleted.*
- 75 10 A C function that takes a variable number of arguments can correspond to several Ada subprograms, taking various specific numbers and types of parameters.

Examples

76 *Example of using the Interfaces.C package:*

```

77 --Calling the C Library Function strcpy
with Interfaces.C;
procedure Test is
  package C renames Interfaces.C;
  use type C.char_array;
  -- Call <string.h>strcpy:
  -- C definition of strcpy: char *strcpy(char *s1, const char *s2);
  -- This function copies the string pointed to by s2 (including the terminating null character)
  -- into the array pointed to by s1. If copying takes place between objects that overlap,
  -- the behavior is undefined. The strcpy function returns the value of s1.
78 -- Note: since the C function's return value is of no interest, the Ada interface is a procedure
  procedure Strcpy (Target : out C.char_array;
                   Source : in C.char_array);
79
  pragma Import (C, Strcpy, "strcpy");
80
  Chars1 : C.char_array(1..20);
  Chars2 : C.char_array(1..20);
81
begin
  Chars2(1..6) := "qwert" & C.nul;
82
  Strcpy(Chars1, Chars2);
83
  -- Now Chars1(1..6) = "qwert" & C.Nul
84
end Test;
```

B.3.1 The Package Interfaces.C.Strings

- 1 The package `Interfaces.C.Strings` declares types and subprograms allowing an Ada program to allocate, reference, update, and free C-style strings. In particular, the private type `chars_ptr` corresponds to a common use of “char *” in C programs, and an object of this type can be passed to a subprogram to which `pragma Import(C,...)` has been applied, and for which “char *” is the type of the argument of the C function.

Static Semantics

- 2 The library package `Interfaces.C.Strings` has the following declaration:
- ```

3 package Interfaces.C.Strings is
 pragma Preelaborate(Strings);
4
 type char_array_access is access all char_array;
5/2
 type chars_ptr is private;
 pragma Preelaborable_Initialization(chars_ptr);
6/2
 type chars_ptr_array is array (size_t range <>) of aliased chars_ptr;
7
 Null_Ptr : constant chars_ptr;
```



```

function To_Chars_Ptr (Item : in char_array_access; 8
 Nul_Check : in Boolean := False)
 return chars_ptr;
function New_Char_Array (Chars : in char_array) return chars_ptr; 9
function New_String (Str : in String) return chars_ptr; 10
procedure Free (Item : in out chars_ptr); 11
Dereference_Error : exception; 12
function Value (Item : in chars_ptr) return char_array; 13
function Value (Item : in chars_ptr; Length : in size_t) 14
 return char_array;
function Value (Item : in chars_ptr) return String; 15
function Value (Item : in chars_ptr; Length : in size_t) 16
 return String;
function Strlen (Item : in chars_ptr) return size_t; 17
procedure Update (Item : in chars_ptr; 18
 Offset : in size_t;
 Chars : in char_array;
 Check : in Boolean := True);
procedure Update (Item : in chars_ptr; 19
 Offset : in size_t;
 Str : in String;
 Check : in Boolean := True);
Update_Error : exception; 20
private 21
 ... -- not specified by the language
end Interfaces.C.Strings;

```

The type `chars_ptr` is C-compatible and corresponds to the use of C's "char \*" for a pointer to the first char in a char array terminated by nul. When an object of type `chars_ptr` is declared, its value is by default set to `Null_Ptr`, unless the object is imported (see B.1).

```

function To_Chars_Ptr (Item : in char_array_access; 23
 Nul_Check : in Boolean := False)
 return chars_ptr;

```

If `Item` is **null**, then `To_Chars_Ptr` returns `Null_Ptr`. If `Item` is not **null**, `Nul_Check` is **True**, and `Item.all` does not contain nul, then the function propagates `Terminator_Error`; otherwise `To_Chars_Ptr` performs a pointer conversion with no allocation of memory.

```

function New_Char_Array (Chars : in char_array) return chars_ptr; 25

```

This function returns a pointer to an allocated object initialized to `Chars(Chars'First .. Index) & nul`, where

- `Index = Chars'Last` if `Chars` does not contain nul, or 27
- `Index` is the smallest `size_t` value `I` such that `Chars(I+1) = nul`. 28

`Storage_Error` is propagated if the allocation fails. 28.1

```

function New_String (Str : in String) return chars_ptr; 29

```

This function is equivalent to `New_Char_Array(To_C(Str))`. 30

```

procedure Free (Item : in out chars_ptr); 31

```

If `Item` is `Null_Ptr`, then `Free` has no effect. Otherwise, `Free` releases the storage occupied by `Value(Item)`, and resets `Item` to `Null_Ptr`. 32

33     **function** Value (Item : **in** chars\_ptr) **return** char\_array;

34     If Item = Null\_Ptr then Value propagates Dereference\_Error. Otherwise Value returns the prefix of the array of chars pointed to by Item, up to and including the first nul. The lower bound of the result is 0. If Item does not point to a nul-terminated string, then execution of Value is erroneous.

35     **function** Value (Item : **in** chars\_ptr; Length : **in** size\_t)  
       **return** char\_array;

36/1     If Item = Null\_Ptr then Value propagates Dereference\_Error. Otherwise Value returns the shorter of two arrays, either the first Length chars pointed to by Item, or Value(Item). The lower bound of the result is 0. If Length is 0, then Value propagates Constraint\_Error.

37     **function** Value (Item : **in** chars\_ptr) **return** String;

38     Equivalent to To\_Ada(Value(Item), Trim\_Nul=>True).

39     **function** Value (Item : **in** chars\_ptr; Length : **in** size\_t)  
       **return** String;

40/1     Equivalent to To\_Ada(Value(Item, Length) & nul, Trim\_Nul=>True).

41     **function** Strlen (Item : **in** chars\_ptr) **return** size\_t;

42     Returns *Val*/Length-1 where *Val* = Value(Item); propagates Dereference\_Error if Item = Null\_Ptr.

43     **procedure** Update (Item    : **in** chars\_ptr;  
                       Offset : **in** size\_t;  
                       Chars  : **in** char\_array;  
                       Check  : Boolean := True);

44/1     If Item = Null\_Ptr, then Update propagates Dereference\_Error. Otherwise, this procedure updates the value pointed to by Item, starting at position Offset, using Chars as the data to be copied into the array. Overwriting the nul terminator, and skipping with the Offset past the nul terminator, are both prevented if Check is True, as follows:

- 45       • Let N = Strlen(Item). If Check is True, then:
  - 46          • If Offset+Chars'Length>N, propagate Update\_Error.
  - 47          • Otherwise, overwrite the data in the array pointed to by Item, starting at the char at position Offset, with the data in Chars.
- 48       • If Check is False, then processing is as above, but with no check that Offset+Chars'Length>N.

49     **procedure** Update (Item    : **in** chars\_ptr;  
                       Offset : **in** size\_t;  
                       Str    : **in** String;  
                       Check  : **in** Boolean := True);

50/2     Equivalent to Update(Item, Offset, To\_C(Str, Append\_Nul => False), Check).

#### *Erroneous Execution*

51     Execution of any of the following is erroneous if the Item parameter is not null\_ptr and Item does not point to a nul-terminated array of chars.

- 52       • a Value function not taking a Length parameter,
- 53       • the Free procedure,

- the Strlen function.

Execution of Free(X) is also erroneous if the chars\_ptr X was not returned by New\_Char\_Array or New\_String.

Reading or updating a freed char\_array is erroneous.

Execution of Update is erroneous if Check is False and a call with Check equal to True would have propagated Update\_Error.

#### NOTES

11 New\_Char\_Array and New\_String might be implemented either through the allocation function from the C environment ("malloc") or through Ada dynamic memory allocation ("new"). The key points are

- the returned value (a chars\_ptr) is represented as a C "char \*" so that it may be passed to C functions;
- the allocated object should be freed by the programmer via a call of Free, not by a called C function.

## B.3.2 The Generic Package Interfaces.C.Pointers

The generic package Interfaces.C.Pointers allows the Ada programmer to perform C-style operations on pointers. It includes an access type Pointer, Value functions that dereference a Pointer and deliver the designated array, several pointer arithmetic operations, and "copy" procedures that copy the contents of a source pointer into the array designated by a destination pointer. As in C, it treats an object Ptr of type Pointer as a pointer to the first element of an array, so that for example, adding 1 to Ptr yields a pointer to the second element of the array.

The generic allows two styles of usage: one in which the array is terminated by a special terminator element; and another in which the programmer needs to keep track of the length.

#### Static Semantics

The generic library package Interfaces.C.Pointers has the following declaration:

```

generic
 type Index is (<>);
 type Element is private;
 type Element_Array is array (Index range <>) of aliased Element;
 Default_Terminator : Element;
package Interfaces.C.Pointers is
 pragma Preelaborate(Pointers);
 type Pointer is access all Element;
 function Value(Ref : in Pointer;
 Terminator : in Element := Default_Terminator)
 return Element_Array;
 function Value(Ref : in Pointer;
 Length : in ptrdiff_t)
 return Element_Array;
 Pointer_Error : exception;
 -- C-style Pointer arithmetic
 function "+" (Left : in Pointer; Right : in ptrdiff_t) return Pointer;
 function "+" (Left : in ptrdiff_t; Right : in Pointer) return Pointer;
 function "-" (Left : in Pointer; Right : in ptrdiff_t) return Pointer;
 function "-" (Left : in Pointer; Right : in Pointer) return ptrdiff_t;
 procedure Increment (Ref : in out Pointer);
 procedure Decrement (Ref : in out Pointer);

```

```

12 pragma Convention (Intrinsic, "+");
13 pragma Convention (Intrinsic, "-");
14 pragma Convention (Intrinsic, Increment);
15 pragma Convention (Intrinsic, Decrement);
16
17 function Virtual_Length (Ref : in Pointer;
18 Terminator : in Element := Default_Terminator)
19 return ptrdiff_t;
20
21 procedure Copy_Terminated_Array
22 (Source : in Pointer;
23 Target : in Pointer;
24 Limit : in ptrdiff_t := ptrdiff_t'Last;
25 Terminator : in Element := Default_Terminator);
26
27 procedure Copy_Array (Source : in Pointer;
28 Target : in Pointer;
29 Length : in ptrdiff_t);
30
31 end Interfaces.C.Pointers;

```

The type `Pointer` is C-compatible and corresponds to one use of C's "Element \*". An object of type `Pointer` is interpreted as a pointer to the initial `Element` in an `Element_Array`. Two styles are supported:

- Explicit termination of an array value with `Default_Terminator` (a special terminator value);
- Programmer-managed length, with `Default_Terminator` treated simply as a data element.

```

20 function Value(Ref : in Pointer;
21 Terminator : in Element := Default_Terminator)
22 return Element_Array;

```

This function returns an `Element_Array` whose value is the array pointed to by `Ref`, up to and including the first `Terminator`; the lower bound of the array is `Index'First`. `Interfaces.C.Strings.Dereference_Error` is propagated if `Ref` is **null**.

```

22 function Value(Ref : in Pointer;
23 Length : in ptrdiff_t)
24 return Element_Array;

```

This function returns an `Element_Array` comprising the first `Length` elements pointed to by `Ref`. The exception `Interfaces.C.Strings.Dereference_Error` is propagated if `Ref` is **null**.

The "+" and "-" functions perform arithmetic on `Pointer` values, based on the `Size` of the array elements. In each of these functions, `Pointer_Error` is propagated if a `Pointer` parameter is **null**.

```

25 procedure Increment (Ref : in out Pointer);
26 Equivalent to Ref := Ref+1.

```

```

27 procedure Decrement (Ref : in out Pointer);
28 Equivalent to Ref := Ref-1.

```

```

29 function Virtual_Length (Ref : in Pointer;
30 Terminator : in Element := Default_Terminator)
31 return ptrdiff_t;

```

Returns the number of `Elements`, up to the one just before the first `Terminator`, in `Value(Ref, Terminator)`.

```

procedure Copy_Terminated_Array 31
 (Source : in Pointer;
 Target : in Pointer;
 Limit : in ptrdiff_t := ptrdiff_t'Last;
 Terminator : in Element := Default_Terminator);

```

This procedure copies Value(Source, Terminator) into the array pointed to by Target; it stops 32  
either after Terminator has been copied, or the number of elements copied is Limit, whichever  
occurs first. Dereference\_Error is propagated if either Source or Target is **null**.

```

procedure Copy_Array (Source : in Pointer; 33
 Target : in Pointer;
 Length : in ptrdiff_t);

```

This procedure copies the first Length elements from the array pointed to by Source, into the 34  
array pointed to by Target. Dereference\_Error is propagated if either Source or Target is **null**.

#### *Erroneous Execution*

It is erroneous to dereference a Pointer that does not designate an aliased Element. 35

Execution of Value(Ref, Terminator) is erroneous if Ref does not designate an aliased Element in an 36  
Element\_Array terminated by Terminator.

Execution of Value(Ref, Length) is erroneous if Ref does not designate an aliased Element in an 37  
Element\_Array containing at least Length Elements between the designated Element and the end of the  
array, inclusive.

Execution of Virtual\_Length(Ref, Terminator) is erroneous if Ref does not designate an aliased Element in 38  
an Element\_Array terminated by Terminator.

Execution of Copy\_Terminated\_Array(Source, Target, Limit, Terminator) is erroneous in either of the 39  
following situations:

- Execution of both Value(Source, Terminator) and Value(Source, Limit) are erroneous, or 40
- Copying writes past the end of the array containing the Element designated by Target. 41

Execution of Copy\_Array(Source, Target, Length) is erroneous if either Value(Source, Length) is 42  
erroneous, or copying writes past the end of the array containing the Element designated by Target.

#### NOTES

12 To compose a Pointer from an Element\_Array, use 'Access on the first element. For example (assuming appropriate 43  
instantiations):

```

Some_Array : Element_Array(0..5) ;
Some_Pointer : Pointer := Some_Array(0) 'Access;

```

#### *Examples*

*Example of Interfaces.C.Pointers:* 45

```

with Interfaces.C.Pointers;
with Interfaces.C.Strings;
procedure Test_Pointers is
 package C renames Interfaces.C;
 package Char_Ptrs is
 new C.Pointers (Index => C.size_t,
 Element => C.char,
 Element_Array => C.char_array,
 Default_Terminator => C.null);

 use type Char_Ptrs.Pointer;
 subtype Char_Star is Char_Ptrs.Pointer;

```

```

48 procedure Strcpy (Target_Ptr, Source_Ptr : Char_Star) is
 Target_Temp_Ptr : Char_Star := Target_Ptr;
 Source_Temp_Ptr : Char_Star := Source_Ptr;
 Element : C.char;
 begin
 if Target_Temp_Ptr = null or Source_Temp_Ptr = null then
 raise C.Strings.Dereference_Error;
 end if;
49/1 loop
 Element := Source_Temp_Ptr.all;
 Target_Temp_Ptr.all := Element;
 exit when C."="(Element, C.nul);
 Char_Ptrs.Increment(Target_Temp_Ptr);
 Char_Ptrs.Increment(Source_Temp_Ptr);
 end loop;
 end Strcpy;
 begin
 ...
 end Test_Pointers;

```

### B.3.3 Pragma Unchecked\_Union

1/2 A pragma `Unchecked_Union` specifies an interface correspondence between a given discriminated type and some C union. The pragma specifies that the associated type shall be given a representation that leaves no space for its discriminant(s).

#### Syntax

2/2 The form of a pragma `Unchecked_Union` is as follows:

3/2 **pragma** `Unchecked_Union` (*first\_subtype\_local\_name*);

#### Legality Rules

4/2 `Unchecked_Union` is a representation pragma, specifying the unchecked union aspect of representation.

5/2 The *first\_subtype\_local\_name* of a pragma `Unchecked_Union` shall denote an unconstrained discriminated record subtype having a `variant_part`.

6/2 A type to which a pragma `Unchecked_Union` applies is called an *unchecked union type*. A subtype of an unchecked union type is defined to be an *unchecked union subtype*. An object of an unchecked union type is defined to be an *unchecked union object*.

7/2 All component subtypes of an unchecked union type shall be C-compatible.

8/2 If a component subtype of an unchecked union type is subject to a per-object constraint, then the component subtype shall be an unchecked union subtype.

9/2 Any name that denotes a discriminant of an object of an unchecked union type shall occur within the declarative region of the type, and shall not occur within a `record_representation_clause`.

10/3 The type of a component declared in a `variant_part` of an unchecked union type shall not need finalization. In addition to the places where Legality Rules normally apply (see 12.3), this rule also applies in the private part of an instance of a generic unit. For an unchecked union type declared within the body of a generic unit, or within the body of any of its descendant library units, no part of the type of a component declared in a `variant_part` of the unchecked union type shall be of a formal private type or formal private extension declared within the formal part of the generic unit.

11/2 The completion of an incomplete or private type declaration having a `known_discriminant_part` shall not be an unchecked union type.

An unchecked union subtype shall only be passed as a generic actual parameter if the corresponding formal type has no known discriminants or is an unchecked union type. 12/2

*Static Semantics*

An unchecked union type is eligible for convention C. 13/2

All objects of an unchecked union type have the same size. 14/2

Discriminants of objects of an unchecked union type are of size zero. 15/2

Any check which would require reading a discriminant of an unchecked union object is suppressed (see 11.5). These checks include: 16/2

- The check performed when addressing a variant component (i.e., a component that was declared in a variant part) of an unchecked union object that the object has this component (see 4.1.3). 17/2
- Any checks associated with a type or subtype conversion of a value of an unchecked union type (see 4.6). This includes, for example, the check associated with the implicit subtype conversion of an assignment statement. 18/2
- The subtype membership check associated with the evaluation of a qualified expression (see 4.7) or an uninitialized allocator (see 4.8). 19/2

*Dynamic Semantics*

A view of an unchecked union object (including a type conversion or function call) has *inferable discriminants* if it has a constrained nominal subtype, unless the object is a component of an enclosing unchecked union object that is subject to a per-object constraint and the enclosing object lacks inferable discriminants. 20/2

An expression of an unchecked union type has inferable discriminants if it is either a name of an object with inferable discriminants or a qualified expression whose `subtype_mark` denotes a constrained subtype. 21/2

Program\_Error is raised in the following cases: 22/2

- Evaluation of the predefined equality operator for an unchecked union type if either of the operands lacks inferable discriminants. 23/2
- Evaluation of the predefined equality operator for a type which has a subcomponent of an unchecked union type whose nominal subtype is unconstrained. 24/2
- Evaluation of a membership test if the `subtype_mark` denotes a constrained unchecked union subtype and the expression lacks inferable discriminants. 25/2
- Conversion from a derived unchecked union type to an unconstrained non-unchecked-union type if the operand of the conversion lacks inferable discriminants. 26/2
- Execution of the default implementation of the Write or Read attribute of an unchecked union type. 27/2
- Execution of the default implementation of the Output or Input attribute of an unchecked union type if the type lacks default discriminant values. 28/2

*Implementation Permissions*

An implementation may require that `pragma Controlled` be specified for the type of an access subcomponent of an unchecked union type. 29/2

## NOTES

30/2 13 The use of an unchecked union to obtain the effect of an unchecked conversion results in erroneous execution (see 11.5). Execution of the following example is erroneous even if `Float'Size = Integer'Size`:

```
31/2 type T (Flag : Boolean := False) is
 record
 case Flag is
 when False =>
 F1 : Float := 0.0;
 when True =>
 F2 : Integer := 0;
 end case;
 end record;
pragma Unchecked_Union (T);

32/2 X : T;
 Y : Integer := X.F2; -- erroneous
```



## B.4 Interfacing with COBOL

The facilities relevant to interfacing with the COBOL language are the package Interfaces.COBOL and support for the Import, Export and Convention pragmas with *convention\_identifier* COBOL.

The COBOL interface package supplies several sets of facilities:

- A set of types corresponding to the native COBOL types of the supported COBOL implementation (so-called “internal COBOL representations”), allowing Ada data to be passed as parameters to COBOL programs
- A set of types and constants reflecting external data representations such as might be found in files or databases, allowing COBOL-generated data to be read by an Ada program, and Ada-generated data to be read by COBOL programs
- A generic package for converting between an Ada decimal type value and either an internal or external COBOL representation

### Static Semantics

The library package Interfaces.COBOL has the following declaration:

```

package Interfaces.COBOL is
 pragma Preelaborate(COBOL);

 -- Types and operations for internal data representations

 type Floating is digits implementation-defined;
 type Long_Floating is digits implementation-defined;

 type Binary is range implementation-defined;
 type Long_Binary is range implementation-defined;

 Max_Digits_Binary : constant := implementation-defined;
 Max_Digits_Long_Binary : constant := implementation-defined;

 type Decimal_Element is mod implementation-defined;
 type Packed_Decimal is array (Positive range <>) of Decimal_Element;
 pragma Pack(Packed_Decimal);

 type COBOL_Character is implementation-defined character type;

 Ada_To_COBOL : array (Character) of COBOL_Character := implementation-defined;
 COBOL_To_Ada : array (COBOL_Character) of Character := implementation-defined;

 type Alphanumeric is array (Positive range <>) of COBOL_Character;
 pragma Pack(Alphanumeric);

 function To_COBOL (Item : in String) return Alphanumeric;
 function To_Ada (Item : in Alphanumeric) return String;

 procedure To_COBOL (Item : in String;
 Target : out Alphanumeric;
 Last : out Natural);

 procedure To_Ada (Item : in Alphanumeric;
 Target : out String;
 Last : out Natural);

 type Numeric is array (Positive range <>) of COBOL_Character;
 pragma Pack(Numeric);

 -- Formats for COBOL data representations

 type Display_Format is private;

 Unsigned : constant Display_Format;
 Leading_Separate : constant Display_Format;
 Trailing_Separate : constant Display_Format;
 Leading_Nonseparate : constant Display_Format;
 Trailing_Nonseparate : constant Display_Format;

```

```

24 type Binary_Format is private;
25 High_Order_First : constant Binary_Format;
26 Low_Order_First : constant Binary_Format;
27 Native_Binary : constant Binary_Format;
28
29 type Packed_Format is private;
30 Packed_Unsigned : constant Packed_Format;
31 Packed_Signed : constant Packed_Format;
32
33 -- Types for external representation of COBOL binary data
34
35 type Byte is mod 2**COBOL_Character'Size;
36 type Byte_Array is array (Positive range <>) of Byte;
37 pragma Pack (Byte_Array);
38
39 Conversion_Error : exception;
40
41 generic
42 type Num is delta <> digits <>;
43 package Decimal_Conversions is
44
45 -- Display Formats: data values are represented as Numeric
46
47 function Valid (Item : in Numeric;
48 Format : in Display_Format) return Boolean;
49
50 function Length (Format : in Display_Format) return Natural;
51
52 function To_Decimal (Item : in Numeric;
53 Format : in Display_Format) return Num;
54
55 function To_Display (Item : in Num;
56 Format : in Display_Format) return Numeric;
57
58 -- Packed Formats: data values are represented as Packed_Decimal
59
60 function Valid (Item : in Packed_Decimal;
61 Format : in Packed_Format) return Boolean;
62
63 function Length (Format : in Packed_Format) return Natural;
64
65 function To_Decimal (Item : in Packed_Decimal;
66 Format : in Packed_Format) return Num;
67
68 function To_Packed (Item : in Num;
69 Format : in Packed_Format) return Packed_Decimal;
70
71 -- Binary Formats: external data values are represented as Byte_Array
72
73 function Valid (Item : in Byte_Array;
74 Format : in Binary_Format) return Boolean;
75
76 function Length (Format : in Binary_Format) return Natural;
77
78 function To_Decimal (Item : in Byte_Array;
79 Format : in Binary_Format) return Num;
80
81 function To_Binary (Item : in Num;
82 Format : in Binary_Format) return Byte_Array;
83
84 -- Internal Binary formats: data values are of type Binary or Long_Binary
85
86 function To_Decimal (Item : in Binary) return Num;
87 function To_Decimal (Item : in Long_Binary) return Num;
88
89 function To_Binary (Item : in Num) return Binary;
90 function To_Long_Binary (Item : in Num) return Long_Binary;
91
92 end Decimal_Conversions;
93
94 private
95 ... -- not specified by the language
96 end Interfaces.COBOL;

```

Each of the types in Interfaces.COBOL is COBOL-compatible.

The types Floating and Long\_Floating correspond to the native types in COBOL for data items with computational usage implemented by floating point. The types Binary and Long\_Binary correspond to the

native types in COBOL for data items with binary usage, or with computational usage implemented by binary.

Max\_Digits\_Binary is the largest number of decimal digits in a numeric value that is represented as Binary. Max\_Digits\_Long\_Binary is the largest number of decimal digits in a numeric value that is represented as Long\_Binary. 53

The type Packed\_Decimal corresponds to COBOL's packed-decimal usage. 54

The type COBOL\_Character defines the run-time character set used in the COBOL implementation. Ada\_To\_COBOL and COBOL\_To\_Ada are the mappings between the Ada and COBOL run-time character sets. 55

Type Alphanumeric corresponds to COBOL's alphanumeric data category. 56

Each of the functions To\_COBOL and To\_Ada converts its parameter based on the mappings Ada\_To\_COBOL and COBOL\_To\_Ada, respectively. The length of the result for each is the length of the parameter, and the lower bound of the result is 1. Each component of the result is obtained by applying the relevant mapping to the corresponding component of the parameter. 57

Each of the procedures To\_COBOL and To\_Ada copies converted elements from Item to Target, using the appropriate mapping (Ada\_To\_COBOL or COBOL\_To\_Ada, respectively). The index in Target of the last element assigned is returned in Last (0 if Item is a null array). If Item'Length exceeds Target'Length, Constraint\_Error is propagated. 58

Type Numeric corresponds to COBOL's numeric data category with display usage. 59

The types Display\_Format, Binary\_Format, and Packed\_Format are used in conversions between Ada decimal type values and COBOL internal or external data representations. The value of the constant Native\_Binary is either High\_Order\_First or Low\_Order\_First, depending on the implementation. 60

```
function Valid (Item : in Numeric;
 Format : in Display_Format) return Boolean; 61
```

The function Valid checks that the Item parameter has a value consistent with the value of Format. If the value of Format is other than Unsigned, Leading\_Separate, and Trailing\_Separate, the effect is implementation defined. If Format does have one of these values, the following rules apply: 62

- Format=Unsigned: if Item comprises one or more decimal digit characters then Valid returns True, else it returns False. 63/1
- Format=Leading\_Separate: if Item comprises a single occurrence of the plus or minus sign character, and then one or more decimal digit characters, then Valid returns True, else it returns False. 64/1
- Format=Trailing\_Separate: if Item comprises one or more decimal digit characters and finally a plus or minus sign character, then Valid returns True, else it returns False. 65/1

```
function Length (Format : in Display_Format) return Natural; 66
```

The Length function returns the minimal length of a Numeric value sufficient to hold any value of type Num when represented as Format. 67

```

68 function To_Decimal (Item : in Numeric;
 Format : in Display_Format) return Num;

```

69       Produces a value of type Num corresponding to Item as represented by Format. The number of digits after the assumed radix point in Item is Num'Scale. Conversion\_Error is propagated if the value represented by Item is outside the range of Num.

```

70 function To_Display (Item : in Num;
 Format : in Display_Format) return Numeric;

```

71/1       This function returns the Numeric value for Item, represented in accordance with Format. The length of the returned value is Length(Format), and the lower bound is 1. Conversion\_Error is propagated if Num is negative and Format is Unsigned.

```

72 function Valid (Item : in Packed_Decimal;
 Format : in Packed_Format) return Boolean;

```

73       This function returns True if Item has a value consistent with Format, and False otherwise. The rules for the formation of Packed\_Decimal values are implementation defined.

```

74 function Length (Format : in Packed_Format) return Natural;

```

75       This function returns the minimal length of a Packed\_Decimal value sufficient to hold any value of type Num when represented as Format.

```

76 function To_Decimal (Item : in Packed_Decimal;
 Format : in Packed_Format) return Num;

```

77       Produces a value of type Num corresponding to Item as represented by Format. Num'Scale is the number of digits after the assumed radix point in Item. Conversion\_Error is propagated if the value represented by Item is outside the range of Num.

```

78 function To_Packed (Item : in Num;
 Format : in Packed_Format) return Packed_Decimal;

```

79/1       This function returns the Packed\_Decimal value for Item, represented in accordance with Format. The length of the returned value is Length(Format), and the lower bound is 1. Conversion\_Error is propagated if Num is negative and Format is Packed\_Unsigned.

```

80 function Valid (Item : in Byte_Array;
 Format : in Binary_Format) return Boolean;

```

81       This function returns True if Item has a value consistent with Format, and False otherwise.

```

82 function Length (Format : in Binary_Format) return Natural;

```

83       This function returns the minimal length of a Byte\_Array value sufficient to hold any value of type Num when represented as Format.

```

84 function To_Decimal (Item : in Byte_Array;
 Format : in Binary_Format) return Num;

```

85       Produces a value of type Num corresponding to Item as represented by Format. Num'Scale is the number of digits after the assumed radix point in Item. Conversion\_Error is propagated if the value represented by Item is outside the range of Num.

```

86 function To_Binary (Item : in Num;
 Format : in Binary_Format) return Byte_Array;

```

87/1       This function returns the Byte\_Array value for Item, represented in accordance with Format. The length of the returned value is Length(Format), and the lower bound is 1.

**function** To\_Decimal (Item : **in** Binary)            **return** Num; 88

**function** To\_Decimal (Item : **in** Long\_Binary) **return** Num;

These functions convert from COBOL binary format to a corresponding value of the decimal type Num. Conversion\_Error is propagated if Item is too large for Num. 89

**function** To\_Binary            (Item : **in** Num)   **return** Binary; 90

**function** To\_Long\_Binary (Item : **in** Num)   **return** Long\_Binary;

These functions convert from Ada decimal to COBOL binary format. Conversion\_Error is propagated if the value of Item is too large to be represented in the result type. 91

#### *Implementation Requirements*

An implementation shall support pragma Convention with a COBOL *convention\_identifier* for a COBOL-eligible type (see B.1). 92

#### *Implementation Permissions*

An implementation may provide additional constants of the private types Display\_Format, Binary\_Format, or Packed\_Format. 93

An implementation may provide further floating point and integer types in Interfaces.COBOL to match additional native COBOL types, and may also supply corresponding conversion functions in the generic package Decimal\_Conversions. 94

#### *Implementation Advice*

An Ada implementation should support the following interface correspondences between Ada and COBOL. 95

- An Ada **access** T parameter is passed as a “BY REFERENCE” data item of the COBOL type corresponding to T. 96
- An Ada **in** scalar parameter is passed as a “BY CONTENT” data item of the corresponding COBOL type. 97
- Any other Ada parameter is passed as a “BY REFERENCE” data item of the COBOL type corresponding to the Ada parameter type; for scalars, a local copy is used if necessary to ensure by-copy semantics. 98

#### NOTES

14 An implementation is not required to support pragma Convention for access types, nor is it required to support pragma Import, Export or Convention for functions. 99

15 If an Ada subprogram is exported to COBOL, then a call from COBOL call may specify either “BY CONTENT” or “BY REFERENCE”. 100

#### *Examples*

*Examples of Interfaces.COBOL:* 101

```
with Interfaces.COBOL;
procedure Test_Call is 102
```

```

103 -- Calling a foreign COBOL program
104 -- Assume that a COBOL program PROG has the following declaration
105 -- in its LINKAGE section:
106 -- 01 Parameter-Area
107 -- 05 NAME PIC X(20).
108 -- 05 SSN PIC X(9).
109 -- 05 SALARY PIC 99999V99 USAGE COMP.
110 -- The effect of PROG is to update SALARY based on some algorithm

104 package COBOL renames Interfaces.COBOL;
105 type Salary_Type is delta 0.01 digits 7;
106 type COBOL_Record is
107 record
108 Name : COBOL.Numeric(1..20);
109 SSN : COBOL.Numeric(1..9);
110 Salary : COBOL.Binary; -- Assume Binary = 32 bits
111 end record;
112 pragma Convention (COBOL, COBOL_Record);
113 procedure Prog (Item : in out COBOL_Record);
114 pragma Import (COBOL, Prog, "PROG");
115 package Salary_Conversions is
116 new COBOL.Decimal_Conversions(Salary_Type);
117 Some_Salary : Salary_Type := 12_345.67;
118 Some_Record : COBOL_Record :=
119 (Name => "Johnson, John ",
120 SSN => "111223333",
121 Salary => Salary_Conversions.To_Binary(Some_Salary));
122 begin
123 Prog (Some_Record);
124 ...
125 end Test_Call;
126 with Interfaces.COBOL;
127 with COBOL_Sequential_IO; -- Assumed to be supplied by implementation
128 procedure Test_External_Formats is
129 -- Using data created by a COBOL program
130 -- Assume that a COBOL program has created a sequential file with
131 -- the following record structure, and that we need to
132 -- process the records in an Ada program
133 -- 01 EMPLOYEE-RECORD
134 -- 05 NAME PIC X(20).
135 -- 05 SSN PIC X(9).
136 -- 05 SALARY PIC 99999V99 USAGE COMP.
137 -- 05 ADJUST PIC S999V999 SIGN LEADING SEPARATE.
138 -- The COMP data is binary (32 bits), high-order byte first

113 package COBOL renames Interfaces.COBOL;
114 type Salary_Type is delta 0.01 digits 7;
115 type Adjustments_Type is delta 0.001 digits 6;
116 type COBOL_Employee_Record_Type is -- External representation
117 record
118 Name : COBOL.Alphanumeric(1..20);
119 SSN : COBOL.Alphanumeric(1..9);
120 Salary : COBOL.Byte_Array(1..4);
121 Adjust : COBOL.Numeric(1..7); -- Sign and 6 digits
122 end record;
123 pragma Convention (COBOL, COBOL_Employee_Record_Type);
124 package COBOL_Employee_IO is
125 new COBOL_Sequential_IO(COBOL_Employee_Record_Type);
126 use COBOL_Employee_IO;
127 COBOL_File : File_Type;

```

```

type Ada_Employee_Record_Type is -- Internal representation
 record
 Name : String(1..20);
 SSN : String(1..9);
 Salary : Salary_Type;
 Adjust : Adjustments_Type;
 end record;

COBOL_Record : COBOL_Employee_Record_Type;
Ada_Record : Ada_Employee_Record_Type;

package Salary_Conversions is
 new COBOL.Decimal_Conversions(Salary_Type);
use Salary_Conversions;

package Adjustments_Conversions is
 new COBOL.Decimal_Conversions(Adjustments_Type);
use Adjustments_Conversions;

begin
 Open (COBOL_File, Name => "Some_File");
 loop
 Read (COBOL_File, COBOL_Record);
 Ada_Record.Name := To_Ada(COBOL_Record.Name);
 Ada_Record.SSN := To_Ada(COBOL_Record.SSN);
 Ada_Record.Salary :=
 To_Decimal(COBOL_Record.Salary, COBOL.High_Order_First);
 Ada_Record.Adjust :=
 To_Decimal(COBOL_Record.Adjust, COBOL.Leading_Separate);
 ... --Process Ada_Record
 end loop;
exception
 when End_Error => ...
end Test_External_Formats;

```

## B.5 Interfacing with Fortran

The facilities relevant to interfacing with the Fortran language are the package `Interfaces.Fortran` and support for the `Import`, `Export` and `Convention` pragmas with *convention\_identifier* Fortran.

The package `Interfaces.Fortran` defines Ada types whose representations are identical to the default representations of the Fortran intrinsic types `Integer`, `Real`, `Double Precision`, `Complex`, `Logical`, and `Character` in a supported Fortran implementation. These Ada types can therefore be used to pass objects between Ada and Fortran programs.

### Static Semantics

The library package `Interfaces.Fortran` has the following declaration:

```

with Ada.Numerics.Generic_Complex_Types; -- see G.1.1
pragma Elaborate_All(Ada.Numerics.Generic_Complex_Types);
package Interfaces.Fortran is
 pragma Pure(Fortran);

 type Fortran_Integer is range implementation-defined;
 type Real is digits implementation-defined;
 type Double_Precision is digits implementation-defined;
 type Logical is new Boolean;

 package Single_Precision_Complex_Types is
 new Ada.Numerics.Generic_Complex_Types (Real);
 type Complex is new Single_Precision_Complex_Types.Complex;

```

```

10 subtype Imaginary is Single_Precision_Complex_Types.Imaginary;
11 i : Imaginary renames Single_Precision_Complex_Types.i;
12 j : Imaginary renames Single_Precision_Complex_Types.j;
13 type Character_Set is implementation-defined character type;
14 type Fortran_Character is array (Positive range <>) of Character_Set;
15 pragma Pack (Fortran_Character);
16 function To_Fortran (Item : in Character) return Character_Set;
17 function To_Ada (Item : in Character_Set) return Character;
18 function To_Fortran (Item : in String) return Fortran_Character;
19 function To_Ada (Item : in Fortran_Character) return String;
20 procedure To_Fortran (Item : in String;
21 Target : out Fortran_Character;
22 Last : out Natural);
23 procedure To_Ada (Item : in Fortran_Character;
24 Target : out String;
25 Last : out Natural);
26 end Interfaces.Fortran;

```

The types Fortran\_Integer, Real, Double\_Precision, Logical, Complex, and Fortran\_Character are Fortran-compatible.

The To\_Fortran and To\_Ada functions map between the Ada type Character and the Fortran type Character\_Set, and also between the Ada type String and the Fortran type Fortran\_Character. The To\_Fortran and To\_Ada procedures have analogous effects to the string conversion subprograms found in Interfaces.COBOL.

#### Implementation Requirements

An implementation shall support pragma Convention with a Fortran *convention\_identifier* for a Fortran-eligible type (see B.1).

#### Implementation Permissions

An implementation may add additional declarations to the Fortran interface packages. For example, the Fortran interface package for an implementation of Fortran 77 (ANSI X3.9-1978) that defines types like Integer\**n*, Real\**n*, Logical\**n*, and Complex\**n* may contain the declarations of types named Integer\_Star\_*n*, Real\_Star\_*n*, Logical\_Star\_*n*, and Complex\_Star\_*n*. (This convention should not apply to Character\**n*, for which the Ada analog is the constrained array subtype Fortran\_Character (1..*n*)). Similarly, the Fortran interface package for an implementation of Fortran 90 that provides multiple *kinds* of intrinsic types, e.g. Integer (Kind=*n*), Real (Kind=*n*), Logical (Kind=*n*), Complex (Kind=*n*), and Character (Kind=*n*), may contain the declarations of types with the recommended names Integer\_Kind\_*n*, Real\_Kind\_*n*, Logical\_Kind\_*n*, Complex\_Kind\_*n*, and Character\_Kind\_*n*.

#### Implementation Advice

An Ada implementation should support the following interface correspondences between Ada and Fortran:

- An Ada procedure corresponds to a Fortran subroutine.
- An Ada function corresponds to a Fortran function.
- An Ada parameter of an elementary, array, or record type T is passed as a T<sub>F</sub> argument to a Fortran procedure, where T<sub>F</sub> is the Fortran type corresponding to the Ada type T, and where the INTENT attribute of the corresponding dummy argument matches the Ada formal parameter mode; the Fortran implementation's parameter passing conventions are used. For elementary types, a local copy is used if necessary to ensure by-copy semantics.



- An Ada parameter of an access-to-subprogram type is passed as a reference to a Fortran procedure whose interface corresponds to the designated subprogram's specification. 26

## NOTES

- 16 An object of a Fortran-compatible record type, declared in a library package or subprogram, can correspond to a Fortran common block; the type also corresponds to a Fortran “derived type”. 27

*Examples**Example of Interfaces.Fortran:* 28

```

with Interfaces.Fortran;
use Interfaces.Fortran;
procedure Ada_Application is
 type Fortran_Matrix is array (Integer range <>,
 Integer range <>) of Double_Precision;
 pragma Convention (Fortran, Fortran_Matrix); -- stored in Fortran's
 -- column-major order
 procedure Invert (Rank : in Fortran_Integer; X : in out Fortran_Matrix);
 pragma Import (Fortran, Invert); -- a Fortran subroutine
 Rank : constant Fortran_Integer := 100;
 My_Matrix : Fortran_Matrix (1 .. Rank, 1 .. Rank);
begin
 ...
 My_Matrix := ...;
 ...
 Invert (Rank, My_Matrix);
 ...
end Ada_Application;

```



# Annex C

## (normative)

### Systems Programming

The Systems Programming Annex specifies additional capabilities provided for low-level programming. 1  
These capabilities are also required in many real-time, embedded, distributed, and information systems.

## C.1 Access to Machine Operations

This clause specifies rules regarding access to machine instructions from within an Ada program. 1

### *Implementation Requirements*

The implementation shall support machine code insertions (see 13.8) or intrinsic subprograms (see 6.3.1) 2  
(or both). Implementation-defined attributes shall be provided to allow the use of Ada entities as operands.

### *Implementation Advice*

The machine code or intrinsics support should allow access to all operations normally available to 3  
assembly language programmers for the target environment, including privileged instructions, if any.

The interfacing pragmas (see Annex B) should support interface to assembler; the default assembler 4  
should be associated with the convention identifier Assembler.

If an entity is exported to assembly language, then the implementation should allocate it at an addressable 5  
location, and should ensure that it is retained by the linking process, even if not otherwise referenced from the Ada code. The implementation should assume that any call to a machine code or assembler subprogram is allowed to read or update every object that is specified as exported.

### *Documentation Requirements*

The implementation shall document the overhead associated with calling machine-code or intrinsic 6  
subprograms, as compared to a fully-inlined call, and to a regular out-of-line call.

The implementation shall document the types of the package System.Machine\_Code usable for machine 7  
code insertions, and the attributes to be used in machine code insertions for references to Ada entities.

The implementation shall document the subprogram calling conventions associated with the convention 8  
identifiers available for use with the interfacing pragmas (Ada and Assembler, at a minimum), including register saving, exception propagation, parameter passing, and function value returning.

For exported and imported subprograms, the implementation shall document the mapping between the 9  
Link\_Name string, if specified, or the Ada designator, if not, and the external link name used for such a subprogram.

### *Implementation Advice*

The implementation should ensure that little or no overhead is associated with calling intrinsic and 10  
machine-code subprograms.

It is recommended that intrinsic subprograms be provided for convenient access to any machine operations 11  
that provide special capabilities or efficiency and that are not otherwise available through the language constructs. Examples of such instructions include:

- 12 • Atomic read-modify-write operations — e.g., test and set, compare and swap, decrement and test, enqueue/dequeue.
- 13 • Standard numeric functions — e.g., *sin*, *log*.
- 14 • String manipulation operations — e.g., translate and test.
- 15 • Vector operations — e.g., compare vector against thresholds.
- 16 • Direct operations on I/O ports.

## C.2 Required Representation Support

- 1/2 This clause specifies minimal requirements on the support for representation items and related features.

### Implementation Requirements

- 2 The implementation shall support at least the functionality defined by the recommended levels of support in Section 13.

## C.3 Interrupt Support

- 1 This clause specifies the language-defined model for hardware interrupts in addition to mechanisms for handling interrupts.

### Dynamic Semantics

- 2 An *interrupt* represents a class of events that are detected by the hardware or the system software. Interrupts are said to occur. An *occurrence* of an interrupt is separable into generation and delivery. *Generation* of an interrupt is the event in the underlying hardware or system that makes the interrupt available to the program. *Delivery* is the action that invokes part of the program as response to the interrupt occurrence. Between generation and delivery, the interrupt occurrence (or interrupt) is *pending*. Some or all interrupts may be *blocked*. When an interrupt is blocked, all occurrences of that interrupt are prevented from being delivered. Certain interrupts are *reserved*. The set of reserved interrupts is implementation defined. A reserved interrupt is either an interrupt for which user-defined handlers are not supported, or one which already has an attached handler by some other implementation-defined means. Program units can be connected to non-reserved interrupts. While connected, the program unit is said to be *attached* to that interrupt. The execution of that program unit, the *interrupt handler*, is invoked upon delivery of the interrupt occurrence.
- 3 While a handler is attached to an interrupt, it is called once for each delivered occurrence of that interrupt. While the handler executes, the corresponding interrupt is blocked.
- 4 While an interrupt is blocked, all occurrences of that interrupt are prevented from being delivered. Whether such occurrences remain pending or are lost is implementation defined.
- 5 Each interrupt has a *default treatment* which determines the system's response to an occurrence of that interrupt when no user-defined handler is attached. The set of possible default treatments is implementation defined, as is the method (if one exists) for configuring the default treatments for interrupts.
- 6 An interrupt is delivered to the handler (or default treatment) that is in effect for that interrupt at the time of delivery.
- 7 An exception propagated from a handler that is invoked by an interrupt has no effect.

If the Ceiling\_Locking policy (see D.3) is in effect, the interrupt handler executes with the active priority that is the ceiling priority of the corresponding protected object. 8

#### *Implementation Requirements*

The implementation shall provide a mechanism to determine the minimum stack space that is needed for each interrupt handler and to reserve that space for the execution of the handler. This space should accommodate nested invocations of the handler where the system permits this. 9

If the hardware or the underlying system holds pending interrupt occurrences, the implementation shall provide for later delivery of these occurrences to the program. 10

If the Ceiling\_Locking policy is not in effect, the implementation shall provide means for the application to specify whether interrupts are to be blocked during protected actions. 11

#### *Documentation Requirements*

The implementation shall document the following items: 12

1. For each interrupt, which interrupts are blocked from delivery when a handler attached to that interrupt executes (either as a result of an interrupt delivery or of an ordinary call on a procedure of the corresponding protected object). 13
2. Any interrupts that cannot be blocked, and the effect of attaching handlers to such interrupts, if this is permitted. 14
3. Which run-time stack an interrupt handler uses when it executes as a result of an interrupt delivery; if this is configurable, what is the mechanism to do so; how to specify how much space to reserve on that stack. 15
4. Any implementation- or hardware-specific activity that happens before a user-defined interrupt handler gets control (e.g., reading device registers, acknowledging devices). 16
5. Any timing or other limitations imposed on the execution of interrupt handlers. 17
6. The state (blocked/unblocked) of the non-reserved interrupts when the program starts; if some interrupts are unblocked, what is the mechanism a program can use to protect itself before it can attach the corresponding handlers. 18
7. Whether the interrupted task is allowed to resume execution before the interrupt handler returns. 19
8. The treatment of interrupt occurrences that are generated while the interrupt is blocked; i.e., whether one or more occurrences are held for later delivery, or all are lost. 20
9. Whether predefined or implementation-defined exceptions are raised as a result of the occurrence of any interrupt, and the mapping between the machine interrupts (or traps) and the predefined exceptions. 21
10. On a multi-processor, the rules governing the delivery of an interrupt to a particular processor. 22

#### *Implementation Permissions*

If the underlying system or hardware does not allow interrupts to be blocked, then no blocking is required as part of the execution of subprograms of a protected object for which one of its subprograms is an interrupt handler. 23/2

In a multi-processor with more than one interrupt subsystem, it is implementation defined whether (and how) interrupt sources from separate subsystems share the same Interrupt\_ID type (see C.3.2). In particular, the meaning of a blocked or pending interrupt may then be applicable to one processor only. 24

- 25 Implementations are allowed to impose timing or other limitations on the execution of interrupt handlers.
- 26/2 Other forms of handlers are allowed to be supported, in which case the rules of this clause should be adhered to.
- 27 The active priority of the execution of an interrupt handler is allowed to vary from one occurrence of the same interrupt to another.

*Implementation Advice*

- 28/2 If the Ceiling\_Locking policy is not in effect, the implementation should provide means for the application to specify which interrupts are to be blocked during protected actions, if the underlying system allows for finer-grained control of interrupt blocking.

NOTES

- 29 1 The default treatment for an interrupt can be to keep the interrupt pending or to deliver it to an implementation-defined handler. Examples of actions that an implementation-defined handler is allowed to perform include aborting the partition, ignoring (i.e., discarding occurrences of) the interrupt, or queuing one or more occurrences of the interrupt for possible later delivery when a user-defined handler is attached to that interrupt.
- 30 2 It is a bounded error to call Task\_Identification.Current\_Task (see C.7.1) from an interrupt handler.
- 31 3 The rule that an exception propagated from an interrupt handler has no effect is modeled after the rule about exceptions propagated out of task bodies.

## C.3.1 Protected Procedure Handlers

*Syntax*

- 1 The form of a pragma Interrupt\_Handler is as follows:
- 2 **pragma** Interrupt\_Handler(*handler\_name*);
- 3 The form of a pragma Attach\_Handler is as follows:
- 4 **pragma** Attach\_Handler(*handler\_name*, *expression*);

*Name Resolution Rules*

- 5 For the Interrupt\_Handler and Attach\_Handler pragmas, the *handler\_name* shall resolve to denote a protected procedure with a parameterless profile.
- 6 For the Attach\_Handler pragma, the expected type for the *expression* is Interrupts.Interrupt\_ID (see C.3.2).

*Legality Rules*

- 7/3 The Attach\_Handler and Interrupt\_Handler pragmas are only allowed immediately within the protected\_definition where the corresponding subprogram is declared. The corresponding protected\_type\_declaration or single\_protected\_declaration shall be a library-level declaration, and shall not be declared within a generic body. In addition to the places where Legality Rules normally apply (see 12.3), these rules also apply in the private part of an instance of a generic unit.
- 8/3 *This paragraph was deleted.*

*Dynamic Semantics*

- 9 If the pragma Interrupt\_Handler appears in a protected\_definition, then the corresponding procedure can be attached dynamically, as a handler, to interrupts (see C.3.2). Such procedures are allowed to be attached to multiple interrupts.

The expression in the Attach\_Handler pragma as evaluated at object creation time specifies an interrupt. As part of the initialization of that object, if the Attach\_Handler pragma is specified, the *handler* procedure is attached to the specified interrupt. A check is made that the corresponding interrupt is not reserved. Program\_Error is raised if the check fails, and the existing treatment for the interrupt is not affected. 10

If the Ceiling\_Locking policy (see D.3) is in effect, then upon the initialization of a protected object for which either an Attach\_Handler or Interrupt\_Handler pragma applies to one of its procedures, a check is made that the ceiling priority defined in the protected\_definition is in the range of System.-Interrupt\_Priority. If the check fails, Program\_Error is raised. 11/2

When a protected object is finalized, for any of its procedures that are attached to interrupts, the handler is detached. If the handler was attached by a procedure in the Interrupts package or if no user handler was previously attached to the interrupt, the default treatment is restored. If an Attach\_Handler pragma was used and the most recently attached handler for the same interrupt is the same as the one that was attached at the time the protected object was initialized, the previous handler is restored. 12/1

When a handler is attached to an interrupt, the interrupt is blocked (subject to the Implementation Permission in C.3) during the execution of every protected action on the protected object containing the handler. 13

#### *Erroneous Execution*

If the Ceiling\_Locking policy (see D.3) is in effect and an interrupt is delivered to a handler, and the interrupt hardware priority is higher than the ceiling priority of the corresponding protected object, the execution of the program is erroneous. 14

If the handlers for a given interrupt attached via pragma Attach\_Handler are not attached and detached in a stack-like (LIFO) order, program execution is erroneous. In particular, when a protected object is finalized, the execution is erroneous if any of the procedures of the protected object are attached to interrupts via pragma Attach\_Handler and the most recently attached handler for the same interrupt is not the same as the one that was attached at the time the protected object was initialized. 14.1/1

#### *Metrics*

The following metric shall be documented by the implementation: 15

- The worst-case overhead for an interrupt handler that is a parameterless protected procedure, in clock cycles. This is the execution time not directly attributable to the handler procedure or the interrupted execution. It is estimated as  $C - (A+B)$ , where A is how long it takes to complete a given sequence of instructions without any interrupt, B is how long it takes to complete a normal call to a given protected procedure, and C is how long it takes to complete the same sequence of instructions when it is interrupted by one execution of the same procedure called via an interrupt. 16/2

#### *Implementation Permissions*

When the pragmas Attach\_Handler or Interrupt\_Handler apply to a protected procedure, the implementation is allowed to impose implementation-defined restrictions on the corresponding protected\_type\_declaration and protected\_body. 17

An implementation may use a different mechanism for invoking a protected procedure in response to a hardware interrupt than is used for a call to that protected procedure from a task. 18

Notwithstanding what this subclause says elsewhere, the Attach\_Handler and Interrupt\_Handler pragmas are allowed to be used for other, implementation defined, forms of interrupt handlers. 19

*Implementation Advice*

20 Whenever possible, the implementation should allow interrupt handlers to be called directly by the hardware.

21 Whenever practical, the implementation should detect violations of any implementation-defined restrictions before run time.

## NOTES

22 4 The `Attach_Handler` pragma can provide static attachment of handlers to interrupts if the implementation supports preelaboration of protected objects. (See C.4.)

23/2 5 A protected object that has a (protected) procedure attached to an interrupt should have a ceiling priority at least as high as the highest processor priority at which that interrupt will ever be delivered.

24 6 Protected procedures can also be attached dynamically to interrupts via operations declared in the predefined package `Interrupts`.

25 7 An example of a possible implementation-defined restriction is disallowing the use of the standard storage pools within the body of a protected procedure that is an interrupt handler.

**C.3.2 The Package `Interrupts`***Static Semantics*

1 The following language-defined packages exist:

```

2 with System;
 package Ada.Interrupts is
 type Interrupt_ID is implementation-defined;
 type Parameterless_Handler is
 access protected procedure;
3/1 This paragraph was deleted.
4 function Is_Reserved (Interrupt : Interrupt_ID)
 return Boolean;
5 function Is_Attached (Interrupt : Interrupt_ID)
 return Boolean;
6 function Current_Handler (Interrupt : Interrupt_ID)
 return Parameterless_Handler;
7 procedure Attach_Handler
 (New_Handler : in Parameterless_Handler;
 Interrupt : in Interrupt_ID);
8 procedure Exchange_Handler
 (Old_Handler : out Parameterless_Handler;
 New_Handler : in Parameterless_Handler;
 Interrupt : in Interrupt_ID);
9 procedure Detach_Handler
 (Interrupt : in Interrupt_ID);
10 function Reference (Interrupt : Interrupt_ID)
 return System.Address;
11 private
 ... -- not specified by the language
 end Ada.Interrupts;
12 package Ada.Interrupts.Names is
 implementation-defined : constant Interrupt_ID :=
 implementation-defined;
 .
 .
 implementation-defined : constant Interrupt_ID :=
 implementation-defined;
 end Ada.Interrupts.Names;
```



*Dynamic Semantics*

- The Interrupt\_ID type is an implementation-defined discrete type used to identify interrupts. 13
- The Is\_Reserved function returns True if and only if the specified interrupt is reserved. 14
- The Is\_Attached function returns True if and only if a user-specified interrupt handler is attached to the interrupt. 15
- The Current\_Handler function returns a value that represents the attached handler of the interrupt. If no user-defined handler is attached to the interrupt, Current\_Handler returns **null**. 16/1
- The Attach\_Handler procedure attaches the specified handler to the interrupt, overriding any existing treatment (including a user handler) in effect for that interrupt. If New\_Handler is **null**, the default treatment is restored. If New\_Handler designates a protected procedure to which the pragma Interrupt\_Handler does not apply, Program\_Error is raised. In this case, the operation does not modify the existing interrupt treatment. 17
- The Exchange\_Handler procedure operates in the same manner as Attach\_Handler with the addition that the value returned in Old\_Handler designates the previous treatment for the specified interrupt. If the previous treatment is not a user-defined handler, **null** is returned. 18/1
- The Detach\_Handler procedure restores the default treatment for the specified interrupt. 19
- For all operations defined in this package that take a parameter of type Interrupt\_ID, with the exception of Is\_Reserved and Reference, a check is made that the specified interrupt is not reserved. Program\_Error is raised if this check fails. 20
- If, by using the Attach\_Handler, Detach\_Handler, or Exchange\_Handler procedures, an attempt is made to detach a handler that was attached statically (using the pragma Attach\_Handler), the handler is not detached and Program\_Error is raised. 21
- The Reference function returns a value of type System.Address that can be used to attach a task entry via an address clause (see J.7.1) to the interrupt specified by Interrupt. This function raises Program\_Error if attaching task entries to interrupts (or to this particular interrupt) is not supported. 22/2

*Implementation Requirements*

- At no time during attachment or exchange of handlers shall the current handler of the corresponding interrupt be undefined. 23

*Documentation Requirements*

- If the Ceiling\_Locking policy (see D.3) is in effect, the implementation shall document the default ceiling priority assigned to a protected object that contains either the Attach\_Handler or Interrupt\_Handler pragmas, but not the Interrupt\_Priority pragma. This default need not be the same for all interrupts. 24/2

*Implementation Advice*

- If implementation-defined forms of interrupt handler procedures are supported, such as protected procedures with parameters, then for each such form of a handler, a type analogous to Parameterless\_Handler should be specified in a child package of Interrupts, with the same operations as in the predefined package Interrupts. 25

## NOTES

- 8 The package Interrupts.Names contains implementation-defined names (and constant values) for the interrupts that are supported by the implementation. 26

*Examples*

27 *Example of interrupt handlers:*

```

28 Device_Priority : constant
 array (1..5) of System.Interrupt_Priority := (...);
 protected type Device_Interface
 (Int_ID : Ada.Interrupts.Interrupt_ID) is
 procedure Handler;
 pragma Attach_Handler(Handler, Int_ID);
 ...
 pragma Interrupt_Priority(Device_Priority(Int_ID));
 end Device_Interface;
 ...
 Device_1_Driver : Device_Interface(1);
 ...
 Device_5_Driver : Device_Interface(5);
 ...

```

## C.4 Preelaboration Requirements

1 This clause specifies additional implementation and documentation requirements for the Preelaborate pragma (see 10.2.1).

*Implementation Requirements*

2 The implementation shall not incur any run-time overhead for the elaboration checks of subprograms and `protected_bodies` declared in preelaborated library units.

3 The implementation shall not execute any memory write operations after load time for the elaboration of constant objects declared immediately within the declarative region of a preelaborated library package, so long as the subtype and initial expression (or default initial expressions if initialized by default) of the `object_declaration` satisfy the following restrictions. The meaning of *load time* is implementation defined.

- 4 • Any `subtype_mark` denotes a statically constrained subtype, with statically constrained subcomponents, if any;
- 4.1/2 • no `subtype_mark` denotes a controlled type, a private type, a private extension, a generic formal private type, a generic formal derived type, or a descendant of such a type;
- 5 • any constraint is a static constraint;
- 6 • any `allocator` is for an access-to-constant type;
- 7 • any uses of predefined operators appear only within static expressions;
- 8 • any `primaries` that are `names`, other than `attribute_references` for the Access or Address attributes, appear only within static expressions;
- 9 • any `name` that is not part of a static expression is an expanded name or `direct_name` that statically denotes some entity;
- 10 • any `discrete_choice` of an `array_aggregate` is static;
- 11 • no language-defined check associated with the elaboration of the `object_declaration` can fail.

*Documentation Requirements*

12 The implementation shall document any circumstances under which the elaboration of a preelaborated package causes code to be executed at run time.

13 The implementation shall document whether the method used for initialization of preelaborated variables allows a partition to be restarted without reloading.

*Implementation Advice*

It is recommended that preelaborated packages be implemented in such a way that there should be little or no code executed at run time for the elaboration of entities not already covered by the Implementation Requirements. 14

## C.5 Pragma Discard\_Names

A pragma Discard\_Names may be used to request a reduction in storage used for the names of certain entities. 1

*Syntax*

The form of a pragma Discard\_Names is as follows: 2

**pragma** Discard\_Names[([On => ] local\_name)]; 3

A pragma Discard\_Names is allowed only immediately within a declarative\_part, immediately within a package\_specification, or as a configuration pragma. 4

*Legality Rules*

The local\_name (if present) shall denote a non-derived enumeration first subtype, a tagged first subtype, or an exception. The pragma applies to the type or exception. Without a local\_name, the pragma applies to all such entities declared after the pragma, within the same declarative region. Alternatively, the pragma can be used as a configuration pragma. If the pragma applies to a type, then it applies also to all descendants of the type. 5

*Static Semantics*

If a local\_name is given, then a pragma Discard\_Names is a representation pragma. 6

If the pragma applies to an enumeration type, then the semantics of the Wide\_Wide\_Image and Wide\_Wide\_Value attributes are implementation defined for that type; the semantics of Image, Wide\_Image, Value, and Wide\_Value are still defined in terms of Wide\_Wide\_Image and Wide\_Wide\_Value. In addition, the semantics of Text\_IO.Enumeration\_IO are implementation defined. If the pragma applies to a tagged type, then the semantics of the Tags.Wide\_Wide\_Expanded\_Name function are implementation defined for that type; the semantics of Tags.Expanded\_Name and Tags.Wide\_Expanded\_Name are still defined in terms of Tags.Wide\_Wide\_Expanded\_Name. If the pragma applies to an exception, then the semantics of the Exceptions.Wide\_Wide\_Exception\_Name function are implementation defined for that exception; the semantics of Exceptions.Exception\_Name and Exceptions.Wide\_Exception\_Name are still defined in terms of Exceptions.Wide\_Wide\_Exception\_Name. 7/2

*Implementation Advice*

If the pragma applies to an entity, then the implementation should reduce the amount of storage used for storing names associated with that entity. 8

## C.6 Shared Variable Control

This clause specifies representation pragmas that control the use of shared variables. 1

*Syntax*

The form for pragmas Atomic, Volatile, Independent, Atomic\_Components, Volatile\_Components, and Independent\_Components is as follows: 2/3

```

3 pragma Atomic(local_name);
4 pragma Volatile(local_name);
4.1/3 pragma Independent(component_local_name);
5 pragma Atomic_Components(array_local_name);
6 pragma Volatile_Components(array_local_name);
6.1/3 pragma Independent_Components(local_name);

```

7/2 An *atomic* type is one to which a pragma Atomic applies. An *atomic* object (including a component) is one to which a pragma Atomic applies, or a component of an array to which a pragma Atomic\_Components applies, or any object of an atomic type, other than objects obtained by evaluating a slice.

8 A *volatile* type is one to which a pragma Volatile applies. A *volatile* object (including a component) is one to which a pragma Volatile applies, or a component of an array to which a pragma Volatile\_Components applies, or any object of a volatile type. In addition, every atomic type or object is also defined to be volatile. Finally, if an object is volatile, then so are all of its subcomponents (the same does not apply to atomic).

#### *Name Resolution Rules*

9/3 The *local\_name* in an Atomic or Volatile pragma shall resolve to denote either an *object\_declaration*, a non-inherited *component\_declaration*, or a *full\_type\_declaration*. The *component\_local\_name* in an Independent pragma shall resolve to denote a non-inherited *component\_declaration*. The *array\_local\_name* in an Atomic\_Components or Volatile\_Components pragma shall resolve to denote the declaration of an array type or an array object of an anonymous type. The *local\_name* in an Independent\_Components pragma shall resolve to denote the declaration of an array or record type or an array object of an anonymous type.

#### *Legality Rules*

- 10 It is illegal to apply either an Atomic or Atomic\_Components pragma to an object or type if the implementation cannot support the indivisible reads and updates required by the pragma (see below).
- 11 It is illegal to specify the Size attribute of an atomic object, the Component\_Size attribute for an array type with atomic components, or the layout attributes of an atomic component, in a way that prevents the implementation from performing the required indivisible reads and updates.
- 12 If an atomic object is passed as a parameter, then the type of the formal parameter shall either be atomic or allow pass by copy (that is, not be a nonatomic by-reference type). If an atomic object is used as an actual for a generic formal object of mode **in out**, then the type of the generic formal object shall be atomic. If the prefix of an *attribute\_reference* for an Access attribute denotes an atomic object (including a component), then the designated type of the resulting access type shall be atomic. If an atomic type is used as an actual for a generic formal derived type, then the ancestor of the formal type shall be atomic or allow pass by copy. Corresponding rules apply to volatile objects and types.
- 13 If a pragma Volatile, Volatile\_Components, Atomic, or Atomic\_Components applies to a stand-alone constant object, then a pragma Import shall also apply to it.
- 13.1/3 It is illegal to apply either an Independent or Independent\_Components pragma to a component, object, or type if the implementation cannot provide the independent addressability required by the pragma (see 9.10).

It is illegal to specify a representation aspect for a component, object, or type to which pragma Independent or Independent\_Components applies, in a way that prevents the implementation from providing the independent addressability required by the pragma. 13.2/3

#### Static Semantics

These pragmas are representation pragmas (see 13.1). 14

Pragmas Independent and Independent\_Components *specify as independently addressable* the named object or component(s), or in the case of a type, all objects of that type. All atomic objects are considered to be specified as independently addressable. 14.1/3

#### Dynamic Semantics

For an atomic object (including an atomic component) all reads and updates of the object as a whole are indivisible. 15

For a volatile object all reads and updates of the object as a whole are performed directly to memory. 16

Two actions are sequential (see 9.10) if each is the read or update of the same atomic object. 17

If a type is atomic or volatile and it is not a by-copy type, then the type is defined to be a by-reference type. If any subcomponent of a type is atomic or volatile, then the type is defined to be a by-reference type. 18

If an actual parameter is atomic or volatile, and the corresponding formal parameter is not, then the parameter is passed by copy. 19

#### Implementation Requirements

The external effect of a program (see 1.1.3) is defined to include each read and update of a volatile or atomic object. The implementation shall not generate any memory reads or updates of atomic or volatile objects other than those specified by the program. 20

If a pragma Pack applies to a type any of whose subcomponents are atomic, the implementation shall not pack the atomic subcomponents more tightly than that for which it can support indivisible reads and updates. 21

#### Implementation Advice

A load or store of a volatile object whose size is a multiple of System.Storage\_Unit and whose alignment is nonzero, should be implemented by accessing exactly the bits of the object and no others. 22/2

A load or store of an atomic object should, where possible, be implemented by a single load or store instruction. 23/2

#### NOTES

9 An imported volatile or atomic constant behaves as a constant (i.e. read-only) with respect to other parts of the Ada program, but can still be modified by an “external source.” 24

## C.7 Task Information

This clause describes operations and attributes that can be used to obtain the identity of a task. In addition, a package that associates user-defined information with a task is defined. Finally, a package that associates termination procedures with a task or set of tasks is defined. 1/2

## C.7.1 The Package Task\_Identification

### Static Semantics

1 The following language-defined library package exists:

```

2/2 package Ada.Task_Identification is
 pragma Preelaborate(Task_Identification);
 type Task_Id is private;
 pragma Preelaborable_Initialization (Task_Id);
 Null_Task_Id : constant Task_Id;
 function "=" (Left, Right : Task_Id) return Boolean;
3/1 function Image (T : Task_Id) return String;
 function Current_Task return Task_Id;
 procedure Abort_Task (T : in Task_Id);
4 function Is_Terminated(T : Task_Id) return Boolean;
 function Is_Callable (T : Task_Id) return Boolean;
private
 ... -- not specified by the language
end Ada.Task_Identification;
```

### Dynamic Semantics

- 5 A value of the type `Task_Id` identifies an existent task. The constant `Null_Task_Id` does not identify any task. Each object of the type `Task_Id` is default initialized to the value of `Null_Task_Id`.
- 6 The function `"=` returns `True` if and only if `Left` and `Right` identify the same task or both have the value `Null_Task_Id`.
- 7 The function `Image` returns an implementation-defined string that identifies `T`. If `T` equals `Null_Task_Id`, `Image` returns an empty string.
- 8 The function `Current_Task` returns a value that identifies the calling task.
- 9 The effect of `Abort_Task` is the same as the `abort_statement` for the task identified by `T`. In addition, if `T` identifies the environment task, the entire partition is aborted, See E.1.
- 10 The functions `Is_Terminated` and `Is_Callable` return the value of the corresponding attribute of the task identified by `T`.
- 11 For a prefix `T` that is of a task type (after any implicit dereference), the following attribute is defined:
- 12 `T'Identity` Yields a value of the type `Task_Id` that identifies the task denoted by `T`.
- 13 For a prefix `E` that denotes an `entry_declaration`, the following attribute is defined:
- 14 `E'Caller` Yields a value of the type `Task_Id` that identifies the task whose call is now being serviced. Use of this attribute is allowed only inside an `entry_body` or `accept_statement` corresponding to the `entry_declaration` denoted by `E`.
- 15 `Program_Error` is raised if a value of `Null_Task_Id` is passed as a parameter to `Abort_Task`, `Is_Terminated`, and `Is_Callable`.
- 16 `Abort_Task` is a potentially blocking operation (see 9.5.1).

### Bounded (Run-Time) Errors

- 17/3 It is a bounded error to call the `Current_Task` function from an `entry_body`, interrupt handler, or finalization of a task attribute. `Program_Error` is raised, or an implementation-defined value of the type `Task_Id` is returned.

*Erroneous Execution*

If a value of `Task_Id` is passed as a parameter to any of the operations declared in this package (or any language-defined child of this package), and the corresponding task object no longer exists, the execution of the program is erroneous. 18

*Documentation Requirements*

The implementation shall document the effect of calling `Current_Task` from an entry body or interrupt handler. 19

## NOTES

10 This package is intended for use in writing user-defined task scheduling packages and constructing server tasks. `Current_Task` can be used in conjunction with other operations requiring a task as an argument such as `Set_Priority` (see D.5). 20

11 The function `Current_Task` and the attribute `Caller` can return a `Task_Id` value that identifies the environment task. 21

## C.7.2 The Package `Task_Attributes`

*Static Semantics*

The following language-defined generic library package exists: 1

```

with Ada.Task_Identification; use Ada.Task_Identification; 2
generic
 type Attribute is private;
 Initial_Value : in Attribute;
package Ada.Task_Attributes is
 type Attribute_Handle is access all Attribute; 3
 function Value(T : Task_Id := Current_Task) 4
 return Attribute;
 function Reference(T : Task_Id := Current_Task) 5
 return Attribute_Handle;
 procedure Set_Value(Val : in Attribute; 6
 T : in Task_Id := Current_Task);
 procedure Reinitialize(T : in Task_Id := Current_Task);
end Ada.Task_Attributes; 7

```

*Dynamic Semantics*

When an instance of `Task_Attributes` is elaborated in a given active partition, an object of the actual type corresponding to the formal type `Attribute` is implicitly created for each task (of that partition) that exists and is not yet terminated. This object acts as a user-defined attribute of the task. A task created previously in the partition and not yet terminated has this attribute from that point on. Each task subsequently created in the partition will have this attribute when created. In all these cases, the initial value of the given attribute is `Initial_Value`. 8

The `Value` operation returns the value of the corresponding attribute of `T`. 9

The `Reference` operation returns an access value that designates the corresponding attribute of `T`. 10

The `Set_Value` operation performs any finalization on the old value of the attribute of `T` and assigns `Val` to that attribute (see 5.2 and 7.6). 11

The effect of the `Reinitialize` operation is the same as `Set_Value` where the `Val` parameter is replaced with `Initial_Value`. 12

- 13 For all the operations declared in this package, `Tasking_Error` is raised if the task identified by `T` is terminated. `Program_Error` is raised if the value of `T` is `Null_Task_Id`.
- 13.1/2 After a task has terminated, all of its attributes are finalized, unless they have been finalized earlier. When the master of an instantiation of `Ada.Task_Attributes` is finalized, the corresponding attribute of each task is finalized, unless it has been finalized earlier.

*Bounded (Run-Time) Errors*

- 13.2/1 If the package `Ada.Task_Attributes` is instantiated with a controlled type and the controlled type has user-defined `Adjust` or `Finalize` operations that in turn access task attributes by any of the above operations, then a call of `Set_Value` of the instantiated package constitutes a bounded error. The call may perform as expected or may result in forever blocking the calling task and subsequently some or all tasks of the partition.

*Erroneous Execution*

- 14 It is erroneous to dereference the access value returned by a given call on `Reference` after a subsequent call on `Reinitialize` for the same task attribute, or after the associated task terminates.
- 15 If a value of `Task_Id` is passed as a parameter to any of the operations declared in this package and the corresponding task object no longer exists, the execution of the program is erroneous.
- 15.1/2 An access to a task attribute via a value of type `Attribute_Handle` is erroneous if executed concurrently with another such access or a call of any of the operations declared in package `Task_Attributes`. An access to a task attribute is erroneous if executed concurrently with or after the finalization of the task attribute.

*Implementation Requirements*

- 16/1 For a given attribute of a given task, the implementation shall perform the operations declared in this package atomically with respect to any of these operations of the same attribute of the same task. The granularity of any locking mechanism necessary to achieve such atomicity is implementation defined.
- 17/2 After task attributes are finalized, the implementation shall reclaim any storage associated with the attributes.

*Documentation Requirements*

- 18 The implementation shall document the limit on the number of attributes per task, if any, and the limit on the total storage for attribute values per task, if such a limit exists.
- 19 In addition, if these limits can be configured, the implementation shall document how to configure them.

*Metrics*

- 20/2 The implementation shall document the following metrics: A task calling the following subprograms shall execute at a sufficiently high priority as to not be preempted during the measurement period. This period shall start just before issuing the call and end just after the call completes. If the attributes of task `T` are accessed by the measurement tests, no other task shall access attributes of that task during the measurement period. For all measurements described here, the `Attribute` type shall be a scalar type whose size is equal to the size of the predefined type `Integer`. For each measurement, two cases shall be documented: one where the accessed attributes are of the calling task (that is, the default value for the `T` parameter is used), and the other, where `T` identifies another, non-terminated, task.
- 21 The following calls (to subprograms in the `Task_Attributes` package) shall be measured:
- 22 • a call to `Value`, where the return value is `Initial_Value`;



- a call to Value, where the return value is not equal to Initial\_Value; 23
- a call to Reference, where the return value designates a value equal to Initial\_Value; 24
- a call to Reference, where the return value designates a value not equal to Initial\_Value; 25
- a call to Set\_Value where the Val parameter is not equal to Initial\_Value and the old attribute value is equal to Initial\_Value; 26/2
- a call to Set\_Value where the Val parameter is not equal to Initial\_Value and the old attribute value is not equal to Initial\_Value. 27

*Implementation Permissions*

An implementation need not actually create the object corresponding to a task attribute until its value is set to something other than that of Initial\_Value, or until Reference is called for the task attribute. Similarly, when the value of the attribute is to be reinitialized to that of Initial\_Value, the object may instead be finalized and its storage reclaimed, to be recreated when needed later. While the object does not exist, the function Value may simply return Initial\_Value, rather than implicitly creating the object. 28

An implementation is allowed to place restrictions on the maximum number of attributes a task may have, the maximum size of each attribute, and the total storage size allocated for all the attributes of a task. 29

*Implementation Advice*

Some implementations are targeted to domains in which memory use at run time must be completely deterministic. For such implementations, it is recommended that the storage for task attributes will be pre-allocated statically and not from the heap. This can be accomplished by either placing restrictions on the number and the size of the attributes of a task, or by using the pre-allocated storage for the first N attribute objects, and the heap for the others. In the latter case, N should be documented. 30/2

Finalization of task attributes and reclamation of associated storage should be performed as soon as possible after task termination. 30.1/2

## NOTES

12 An attribute always exists (after instantiation), and has the initial value. It need not occupy memory until the first operation that potentially changes the attribute value. The same holds true after Reinitialize. 31

13 The result of the Reference function should be used with care; it is always safe to use that result in the task body whose attribute is being accessed. However, when the result is being used by another task, the programmer must make sure that the task whose attribute is being accessed is not yet terminated. Failing to do so could make the program execution erroneous. 32

*This paragraph was deleted.*

33/2

## C.7.3 The Package Task\_Termination

*Static Semantics*

The following language-defined library package exists: 1/2

```

with Ada.Task_Identification;
with Ada.Exceptions;
package Ada.Task_Termination is
 pragma Preelaborate(Task_Termination);
 type Cause_Of_Termination is (Normal, Abnormal, Unhandled_Exception);
 type Termination_Handler is access protected procedure
 (Cause : in Cause_Of_Termination;
 T : in Ada.Task_Identification.Task_Id;
 X : in Ada.Exceptions.Exception_Occurrence);

```

2/2  
3/2  
4/2

```

5/2 procedure Set_Dependents_Fallback_Handler
 (Handler: in Termination_Handler);
 function Current_Task_Fallback_Handler return Termination_Handler;

6/2 procedure Set_Specific_Handler
 (T : in Ada.Task_Identification.Task_Id;
 Handler : in Termination_Handler);
 function Specific_Handler (T : Ada.Task_Identification.Task_Id)
 return Termination_Handler;

7/2 end Ada.Task_Termination;

```

*Dynamic Semantics*

- 8/2 The type `Termination_Handler` identifies a protected procedure to be executed by the implementation when a task terminates. Such a protected procedure is called a *handler*. In all cases T identifies the task that is terminating. If the task terminates due to completing the last statement of its body, or as a result of waiting on a terminate alternative, then `Cause` is set to `Normal` and `X` is set to `Null_Occurrence`. If the task terminates because it is being aborted, then `Cause` is set to `Abnormal` and `X` is set to `Null_Occurrence`. If the task terminates because of an exception raised by the execution of its `task_body`, then `Cause` is set to `Unhandled_Exception` and `X` is set to the associated exception occurrence.
- 9/2 Each task has two termination handlers, a *fall-back handler* and a *specific handler*. The specific handler applies only to the task itself, while the fall-back handler applies only to the dependent tasks of the task. A handler is said to be *set* if it is associated with a non-null value of type `Termination_Handler`, and *cleared* otherwise. When a task is created, its specific handler and fall-back handler are cleared.
- 10/2 The procedure `Set_Dependents_Fallback_Handler` changes the fall-back handler for the calling task; if `Handler` is **null**, that fall-back handler is cleared, otherwise it is set to be `Handler.all`. If a fall-back handler had previously been set it is replaced.
- 11/2 The function `Current_Task_Fallback_Handler` returns the fall-back handler that is currently set for the calling task, if one is set; otherwise it returns **null**.
- 12/2 The procedure `Set_Specific_Handler` changes the specific handler for the task identified by T; if `Handler` is **null**, that specific handler is cleared, otherwise it is set to be `Handler.all`. If a specific handler had previously been set it is replaced.
- 13/2 The function `Specific_Handler` returns the specific handler that is currently set for the task identified by T, if one is set; otherwise it returns **null**.
- 14/2 As part of the finalization of a `task_body`, after performing the actions specified in 7.6 for finalization of a master, the specific handler for the task, if one is set, is executed. If the specific handler is cleared, a search for a fall-back handler proceeds by recursively following the master relationship for the task. If a task is found whose fall-back handler is set, that handler is executed; otherwise, no handler is executed.
- 15/2 For `Set_Specific_Handler` or `Specific_Handler`, `Tasking_Error` is raised if the task identified by T has already terminated. `Program_Error` is raised if the value of T is `Ada.Task_Identification.Null_Task_Id`.
- 16/2 An exception propagated from a handler that is invoked as part of the termination of a task has no effect.

*Erroneous Execution*

- 17/2 For a call of `Set_Specific_Handler` or `Specific_Handler`, if the task identified by T no longer exists, the execution of the program is erroneous.

## Annex D (normative) Real-Time Systems

This Annex specifies additional characteristics of Ada implementations intended for real-time systems software. To conform to this Annex, an implementation shall also conform to the Systems Programming Annex.

### *Metrics*

The metrics are documentation requirements; an implementation shall document the values of the language-defined metrics for at least one configuration of hardware or an underlying system supported by the implementation, and shall document the details of that configuration.

The metrics do not necessarily yield a simple number. For some, a range is more suitable, for others a formula dependent on some parameter is appropriate, and for others, it may be more suitable to break the metric into several cases. Unless specified otherwise, the metrics in this annex are expressed in processor clock cycles. For metrics that require documentation of an upper bound, if there is no upper bound, the implementation shall report that the metric is unbounded.

#### NOTES

1 The specification of the metrics makes a distinction between upper bounds and simple execution times. Where something is just specified as “the execution time of” a piece of code, this leaves one the freedom to choose a nonpathological case. This kind of metric is of the form “there exists a program such that the value of the metric is V”. Conversely, the meaning of upper bounds is “there is no program such that the value of the metric is greater than V”. This kind of metric can only be partially tested, by finding the value of V for one or more test programs.

2 The metrics do not cover the whole language; they are limited to features that are specified in Annex C, “Systems Programming” and in this Annex. The metrics are intended to provide guidance to potential users as to whether a particular implementation of such a feature is going to be adequate for a particular real-time application. As such, the metrics are aimed at known implementation choices that can result in significant performance differences.

3 The purpose of the metrics is not necessarily to provide fine-grained quantitative results or to serve as a comparison between different implementations on the same or different platforms. Instead, their goal is rather qualitative; to define a standard set of approximate values that can be measured and used to estimate the general suitability of an implementation, or to evaluate the comparative utility of certain features of an implementation for a particular real-time application.

## D.1 Task Priorities

This clause specifies the priority model for real-time systems. In addition, the methods for specifying priorities are defined.

### *Syntax*

The form of a **pragma Priority** is as follows:

**pragma Priority**(expression);

The form of a **pragma Interrupt\_Priority** is as follows:

**pragma Interrupt\_Priority**[(expression)];

### *Name Resolution Rules*

The expected type for the **expression** in a **Priority** or **Interrupt\_Priority** pragma is Integer.

*Legality Rules*

- 7 A Priority pragma is allowed only immediately within a `task_definition`, a `protected_definition`, or the `declarative_part` of a `subprogram_body`. An Interrupt\_Priority pragma is allowed only immediately within a `task_definition` or a `protected_definition`. At most one such pragma shall appear within a given construct.
- 8 For a Priority pragma that appears in the `declarative_part` of a `subprogram_body`, the expression shall be static, and its value shall be in the range of `System.Priority`.

*Static Semantics*

- 9 The following declarations exist in package `System`:
- 10 **subtype** Any\_Priority **is** Integer **range** *implementation-defined*;  
**subtype** Priority **is** Any\_Priority  
**range** Any\_Priority'First .. *implementation-defined*;  
**subtype** Interrupt\_Priority **is** Any\_Priority  
**range** Priority'Last+1 .. Any\_Priority'Last;
- 11 `Default_Priority : constant Priority := (Priority'First + Priority'Last)/2;`
- 12 The full range of priority values supported by an implementation is specified by the subtype `Any_Priority`. The subrange of priority values that are high enough to require the blocking of one or more interrupts is specified by the subtype `Interrupt_Priority`. The subrange of priority values below `System.Interrupt_Priority'First` is specified by the subtype `System.Priority`.
- 13 The priority specified by a Priority or Interrupt\_Priority pragma is the value of the expression in the pragma, if any. If there is no expression in an Interrupt\_Priority pragma, the priority value is `Interrupt_Priority'Last`.

*Dynamic Semantics*

- 14 A Priority pragma has no effect if it occurs in the `declarative_part` of the `subprogram_body` of a subprogram other than the main subprogram.
- 15 A *task priority* is an integer value that indicates a degree of urgency and is the basis for resolving competing demands of tasks for resources. Unless otherwise specified, whenever tasks compete for processors or other implementation-defined resources, the resources are allocated to the task with the highest priority value. The *base priority* of a task is the priority with which it was created, or to which it was later set by `Dynamic_Priorities.Set_Priority` (see D.5). At all times, a task also has an *active priority*, which generally reflects its base priority as well as any priority it inherits from other sources. *Priority inheritance* is the process by which the priority of a task or other entity (e.g. a protected object; see D.3) is used in the evaluation of another task's active priority.
- 16 The effect of specifying such a pragma in a `protected_definition` is discussed in D.3.
- 17 The expression in a Priority or Interrupt\_Priority pragma that appears in a `task_definition` is evaluated for each task object (see 9.1). For a Priority pragma, the value of the expression is converted to the subtype `Priority`; for an Interrupt\_Priority pragma, this value is converted to the subtype `Any_Priority`. The priority value is then associated with the task object whose `task_definition` contains the pragma.
- 18 Likewise, the priority value is associated with the environment task if the pragma appears in the `declarative_part` of the main subprogram.
- 19 The initial value of a task's base priority is specified by default or by means of a Priority or Interrupt\_Priority pragma. After a task is created, its base priority can be changed only by a call to `Dynamic_Priorities.Set_Priority` (see D.5). The initial base priority of a task in the absence of a pragma is

the base priority of the task that creates it at the time of creation (see 9.1). If a pragma Priority does not apply to the main subprogram, the initial base priority of the environment task is System.Default\_Priority. The task's active priority is used when the task competes for processors. Similarly, the task's active priority is used to determine the task's position in any queue when Priority\_Queueing is specified (see D.4).

At any time, the active priority of a task is the maximum of all the priorities the task is inheriting at that instant. For a task that is not held (see D.11), its base priority is a source of priority inheritance unless otherwise specified for a particular task dispatching policy. Other sources of priority inheritance are specified under the following conditions:

- During activation, a task being activated inherits the active priority that its activator (see 9.2) had at the time the activation was initiated. 21/1
- During rendezvous, the task accepting the entry call inherits the priority of the entry call (see 9.5.3 and D.4). 22/1
- During a protected action on a protected object, a task inherits the ceiling priority of the protected object (see 9.5 and D.3). 23

In all of these cases, the priority ceases to be inherited as soon as the condition calling for the inheritance no longer exists. 24

#### *Implementation Requirements*

The range of System.Interrupt\_Priority shall include at least one value. 25

The range of System.Priority shall include at least 30 values. 26

#### NOTES

- 4 The priority expression can include references to discriminants of the enclosing type. 27
- 5 It is a consequence of the active priority rules that at the point when a task stops inheriting a priority from another source, its active priority is re-evaluated. This is in addition to other instances described in this Annex for such re-evaluation. 28
- 6 An implementation may provide a non-standard mode in which tasks inherit priorities under conditions other than those specified above. 29

## D.2 Priority Scheduling

This clause describes the rules that determine which task is selected for execution when more than one task is ready (see 9). 1/2

### D.2.1 The Task Dispatching Model

The task dispatching model specifies task scheduling, based on conceptual priority-ordered ready queues. 1/2

#### *Static Semantics*

The following language-defined library package exists: 1.1/2

```

package Ada.Dispatching is
 pragma Preelaborate (Dispatching);
 procedure Yield;
 Dispatching_Policy_Error : exception;
end Ada.Dispatching;
```

Dispatching serves as the parent of other language-defined library units concerned with task dispatching. 1.5/2

*Dynamic Semantics*

- 2/2 A task can become a *running task* only if it is ready (see 9) and the execution resources required by that task are available. Processors are allocated to tasks based on each task's active priority.
- 3 It is implementation defined whether, on a multiprocessor, a task that is waiting for access to a protected object keeps its processor busy.
- 4/2 *Task dispatching* is the process by which one ready task is selected for execution on a processor. This selection is done at certain points during the execution of a task called *task dispatching points*. A task reaches a task dispatching point whenever it becomes blocked, and when it terminates. Other task dispatching points are defined throughout this Annex for specific policies.
- 5/2 *Task dispatching policies* are specified in terms of conceptual *ready queues* and task states. A ready queue is an ordered list of ready tasks. The first position in a queue is called the *head of the queue*, and the last position is called the *tail of the queue*. A task is *ready* if it is in a ready queue, or if it is running. Each processor has one ready queue for each priority value. At any instant, each ready queue of a processor contains exactly the set of tasks of that priority that are ready for execution on that processor, but are not running on any processor; that is, those tasks that are ready, are not running on any processor, and can be executed using that processor and other available resources. A task can be on the ready queues of more than one processor.
- 6/2 Each processor also has one *running task*, which is the task currently being executed by that processor. Whenever a task running on a processor reaches a task dispatching point it goes back to one or more ready queues; a task (possibly the same task) is then selected to run on that processor. The task selected is the one at the head of the highest priority nonempty ready queue; this task is then removed from all ready queues to which it belongs.
- 7/3 A call of Yield is a task dispatching point. Yield is a potentially blocking operation (see 9.5.1).
- 8/2 *This paragraph was deleted.*

*Implementation Permissions*

- 9/2 An implementation is allowed to define additional resources as execution resources, and to define the corresponding allocation policies for them. Such resources may have an implementation-defined effect on task dispatching.
- 10 An implementation may place implementation-defined restrictions on tasks whose active priority is in the Interrupt\_Priority range.
- 10.1/2 For optimization purposes, an implementation may alter the points at which task dispatching occurs, in an implementation-defined manner. However, a `delay_statement` always corresponds to at least one task dispatching point.

## NOTES

- 11 7 Section 9 specifies under which circumstances a task becomes ready. The ready state is affected by the rules for task activation and termination, delay statements, and entry calls. When a task is not ready, it is said to be blocked.
- 12 8 An example of a possible implementation-defined execution resource is a page of physical memory, which needs to be loaded with a particular page of virtual memory before a task can continue execution.
- 13 9 The ready queues are purely conceptual; there is no requirement that such lists physically exist in an implementation.
- 14 10 While a task is running, it is not on any ready queue. Any time the task that is running on a processor is added to a ready queue, a new running task is selected for that processor.
- 15 11 In a multiprocessor system, a task can be on the ready queues of more than one processor. At the extreme, if several processors share the same set of ready tasks, the contents of their ready queues is identical, and so they can be viewed as

sharing one ready queue, and can be implemented that way. Thus, the dispatching model covers multiprocessors where dispatching is implemented using a single ready queue, as well as those with separate dispatching domains.

12 The priority of a task is determined by rules specified in this subclause, and under D.1, “Task Priorities”, D.3, “Priority Ceiling Locking”, and D.5, “Dynamic Priorities”. 16

13 The setting of a task's base priority as a result of a call to Set\_Priority does not always take effect immediately when Set\_Priority is called. The effect of setting the task's base priority is deferred while the affected task performs a protected action. 17/2

## D.2.2 Task Dispatching Pragmas

This clause allows a single task dispatching policy to be defined for all priorities, or the range of priorities to be split into subranges that are assigned individual dispatching policies. 0.1/2

### *Syntax*

The form of a **pragma** Task\_Dispatching\_Policy is as follows: 1

**pragma** Task\_Dispatching\_Policy(*policy\_identifier*); 2

The form of a **pragma** Priority\_Specific\_Dispatching is as follows: 2.1/2

**pragma** Priority\_Specific\_Dispatching (  
    *policy\_identifier*, *first\_priority\_expression*, *last\_priority\_expression*); 2.2/2

### *Name Resolution Rules*

The expected type for *first\_priority\_expression* and *last\_priority\_expression* is Integer. 2.3/2

### *Legality Rules*

The *policy\_identifier* used in a **pragma** Task\_Dispatching\_Policy shall be the name of a task dispatching policy. 3/2

The *policy\_identifier* used in a **pragma** Priority\_Specific\_Dispatching shall be the name of a task dispatching policy. 3.1/2

Both *first\_priority\_expression* and *last\_priority\_expression* shall be static expressions in the range of System.Any\_Priority; *last\_priority\_expression* shall have a value greater than or equal to *first\_priority\_expression*. 3.2/2

### *Static Semantics*

**Pragma** Task\_Dispatching\_Policy specifies the single task dispatching policy. 3.3/2

**Pragma** Priority\_Specific\_Dispatching specifies the task dispatching policy for the specified range of priorities. Tasks with base priorities within the range of priorities specified in a Priority\_Specific\_Dispatching pragma have their active priorities determined according to the specified dispatching policy. Tasks with active priorities within the range of priorities specified in a Priority\_Specific\_Dispatching pragma are dispatched according to the specified dispatching policy. 3.4/2

If a partition contains one or more Priority\_Specific\_Dispatching pragmas the dispatching policy for priorities not covered by any Priority\_Specific\_Dispatching pragmas is FIFO\_Within\_Priorities. 3.5/2

### *Post-Compilation Rules*

A Task\_Dispatching\_Policy pragma is a configuration pragma. A Priority\_Specific\_Dispatching pragma is a configuration pragma. 4/2

- 4.1/2 The priority ranges specified in more than one `Priority_Specific_Dispatching` pragma within the same partition shall not be overlapping.
- 4.2/2 If a partition contains one or more `Priority_Specific_Dispatching` pragmas it shall not contain a `Task_Dispatching_Policy` pragma.
- 5/2 *This paragraph was deleted.*

*Dynamic Semantics*

- 6/2 A *task dispatching policy* specifies the details of task dispatching that are not covered by the basic task dispatching model. These rules govern when tasks are inserted into and deleted from the ready queues. A single task dispatching policy is specified by a `Task_Dispatching_Policy` pragma. Pragma `Priority_Specific_Dispatching` assigns distinct dispatching policies to subranges of `System.Any_Priority`.
- 6.1/2 If neither pragma applies to any of the program units comprising a partition, the task dispatching policy for that partition is unspecified.
- 6.2/2 If a partition contains one or more `Priority_Specific_Dispatching` pragmas a task dispatching point occurs for the currently running task of a processor whenever there is a non-empty ready queue for that processor with a higher priority than the priority of the running task.
- 6.3/2 A task that has its base priority changed may move from one dispatching policy to another. It is immediately subject to the new dispatching policy.

*Paragraphs 7 through 13 were moved to D.2.3.*

*Implementation Requirements*

- 13.1/2 An implementation shall allow, for a single partition, both the locking policy (see D.3) to be specified as `Ceiling_Locking` and also one or more `Priority_Specific_Dispatching` pragmas to be given.

*Documentation Requirements*

*Paragraphs 14 through 16 were moved to D.2.3.*

*Implementation Permissions*

- 17/2 Implementations are allowed to define other task dispatching policies, but need not support more than one task dispatching policy per partition.
- 18/2 An implementation need not support pragma `Priority_Specific_Dispatching` if it is infeasible to support it in the target environment.

NOTES

*Paragraphs 19 through 21 were deleted.*

## D.2.3 Preemptive Dispatching

- 1/2 This clause defines a preemptive task dispatching policy.

*Static Semantics*

- 2/2 The *policy\_identifier* `FIFO_Within_Priorities` is a task dispatching policy.

*Dynamic Semantics*

- 3/2 When `FIFO_Within_Priorities` is in effect, modifications to the ready queues occur only as follows:



- When a blocked task becomes ready, it is added at the tail of the ready queue for its active priority. 4/2
- When the active priority of a ready task that is not running changes, or the setting of its base priority takes effect, the task is removed from the ready queue for its old active priority and is added at the tail of the ready queue for its new active priority, except in the case where the active priority is lowered due to the loss of inherited priority, in which case the task is added at the head of the ready queue for its new active priority. 5/2
- When the setting of the base priority of a running task takes effect, the task is added to the tail of the ready queue for its active priority. 6/2
- When a task executes a `delay_statement` that does not result in blocking, it is added to the tail of the ready queue for its active priority. 7/2

Each of the events specified above is a task dispatching point (see D.2.1). 8/2

A task dispatching point occurs for the currently running task of a processor whenever there is a nonempty ready queue for that processor with a higher priority than the priority of the running task. The currently running task is said to be *preempted* and it is added at the head of the ready queue for its active priority. 9/2

#### Implementation Requirements

An implementation shall allow, for a single partition, both the task dispatching policy to be specified as FIFO\_Within\_Priorities and also the locking policy (see D.3) to be specified as Ceiling\_Locking. 10/2

#### Documentation Requirements

*Priority inversion* is the duration for which a task remains at the head of the highest priority nonempty ready queue while the processor executes a lower priority task. The implementation shall document: 11/2

- The maximum priority inversion a user task can experience due to activity of the implementation (on behalf of lower priority tasks), and 12/2
- whether execution of a task can be preempted by the implementation processing of delay expirations for lower priority tasks, and if so, for how long. 13/2

#### NOTES

14 If the active priority of a running task is lowered due to loss of inherited priority (as it is on completion of a protected operation) and there is a ready task of the same active priority that is not running, the running task continues to run (provided that there is no higher priority task). 14/2

15 Setting the base priority of a ready task causes the task to move to the tail of the queue for its active priority, regardless of whether the active priority of the task actually changes. 15/2

## D.2.4 Non-Preemptive Dispatching

This clause defines a non-preemptive task dispatching policy. 1/2

#### Static Semantics

The *policy\_identifier* Non\_Preemptive\_FIFO\_Within\_Priorities is a task dispatching policy. 2/2

The following language-defined library package exists: 2.1/3

```
package Ada.Dispatching.Non_Preemptive is
 pragma Preelaborate(Dispatching.Non_Preemptive);
 procedure Yield_To_Higher;
 procedure Yield_To_Same_Or_Higher renames Yield;
end Ada.Dispatching.Non_Preemptive; 2.2/3
```

- 2.3/3 A call of `Yield_To_Higher` is a task dispatching point for this policy. If the task at the head of the highest priority ready queue has a higher active priority than the calling task then the calling task is preempted.

*Legality Rules*

- 3/2 `Non_Preemptive_FIFO_Within_Priorities` shall not be specified as the *policy\_identifier* of `pragma Priority_Specific_Dispatching` (see D.2.2).

*Dynamic Semantics*

- 4/2 When `Non_Preemptive_FIFO_Within_Priorities` is in effect, modifications to the ready queues occur only as follows:
- 5/2 • When a blocked task becomes ready, it is added at the tail of the ready queue for its active priority.
  - 6/2 • When the active priority of a ready task that is not running changes, or the setting of its base priority takes effect, the task is removed from the ready queue for its old active priority and is added at the tail of the ready queue for its new active priority.
  - 7/2 • When the setting of the base priority of a running task takes effect, the task is added to the tail of the ready queue for its active priority.
  - 8/2 • When a task executes a `delay_statement` that does not result in blocking, it is added to the tail of the ready queue for its active priority.
- 9/3 For this policy, blocking or termination of a task, a `delay_statement`, a call to `Yield_To_Higher`, and a call to `Yield_To_Same_Or_Higher` or `Yield` are the only task dispatching points (see D.2.1).

*Implementation Requirements*

- 10/2 An implementation shall allow, for a single partition, both the task dispatching policy to be specified as `Non_Preemptive_FIFO_Within_Priorities` and also the locking policy (see D.3) to be specified as `Ceiling_Locking`.

*Implementation Permissions*

- 11/2 Since implementations are allowed to round all ceiling priorities in subrange `System.Priority` to `System.Priority'Last` (see D.3), an implementation may allow a task to execute within a protected object without raising its active priority provided the associated protected unit does not contain `pragma Interrupt_Priority`, `Interrupt_Handler`, or `Attach_Handler`.

## D.2.5 Round Robin Dispatching

- 1/2 This clause defines the task dispatching policy `Round_Robin_Within_Priorities` and the package `Round_Robin`.

*Static Semantics*

- 2/2 The *policy\_identifier* `Round_Robin_Within_Priorities` is a task dispatching policy.

The following language-defined library package exists:

```

with System;
with Ada.Real_Time;
package Ada.Dispatching.Round_Robin is
 Default_Quantum : constant Ada.Real_Time.Time_Span :=
 implementation-defined;
 procedure Set_Quantum (Pri : in System.Priority;
 Quantum : in Ada.Real_Time.Time_Span);
 procedure Set_Quantum (Low, High : in System.Priority;
 Quantum : in Ada.Real_Time.Time_Span);
 function Actual_Quantum (Pri : System.Priority)
 return Ada.Real_Time.Time_Span;
 function Is_Round_Robin (Pri : System.Priority) return Boolean;
end Ada.Dispatching.Round_Robin;

```

When task dispatching policy Round\_Robin\_Within\_Priorities is the single policy in effect for a partition, each task with priority in the range of System.Interrupt\_Priority is dispatched according to policy FIFO\_Within\_Priorities.

#### *Dynamic Semantics*

The procedures Set\_Quantum set the required Quantum value for a single priority level Pri or a range of priority levels Low .. High. If no quantum is set for a Round Robin priority level, Default\_Quantum is used.

The function Actual\_Quantum returns the actual quantum used by the implementation for the priority level Pri.

The function Is\_Round\_Robin returns True if priority Pri is covered by task dispatching policy Round\_Robin\_Within\_Priorities; otherwise it returns False.

A call of Actual\_Quantum or Set\_Quantum raises exception Dispatching.Dispatching\_Policy\_Error if a predefined policy other than Round\_Robin\_Within\_Priorities applies to the specified priority or any of the priorities in the specified range.

For Round\_Robin\_Within\_Priorities, the dispatching rules for FIFO\_Within\_Priorities apply with the following additional rules:

- When a task is added or moved to the tail of the ready queue for its base priority, it has an execution time budget equal to the quantum for that priority level. This will also occur when a blocked task becomes executable again. 11/2
- When a task is preempted (by a higher priority task) and is added to the head of the ready queue for its priority level, it retains its remaining budget. 12/2
- While a task is executing, its budget is decreased by the amount of execution time it uses. The accuracy of this accounting is the same as that for execution time clocks (see D.14). 13/2
- When a task has exhausted its budget and is without an inherited priority (and is not executing within a protected operation), it is moved to the tail of the ready queue for its priority level. This is a task dispatching point. 14/2

#### *Implementation Requirements*

An implementation shall allow, for a single partition, both the task dispatching policy to be specified as Round\_Robin\_Within\_Priorities and also the locking policy (see D.3) to be specified as Ceiling\_Locking.

#### *Documentation Requirements*

An implementation shall document the quantum values supported.

17/2 An implementation shall document the accuracy with which it detects the exhaustion of the budget of a task.

NOTES

18/2 16 Due to implementation constraints, the quantum value returned by `Actual_Quantum` might not be identical to that set with `Set_Quantum`.

19/2 17 A task that executes continuously with an inherited priority will not be subject to round robin dispatching.

## D.2.6 Earliest Deadline First Dispatching

The deadline of a task is an indication of the urgency of the task; it represents a point on an ideal physical time line. The deadline might affect how resources are allocated to the task. 1/2

This clause defines a package for representing the deadline of a task and a dispatching policy that defines Earliest Deadline First (EDF) dispatching. A pragma is defined to assign an initial deadline to a task. 2/2

### Syntax

The form of a pragma `Relative_Deadline` is as follows: 3/2

**pragma** `Relative_Deadline` (*relative\_deadline\_expression*); 4/2

### Name Resolution Rules

The expected type for *relative\_deadline\_expression* is `Real_Time.Time_Span`. 5/2

### Legality Rules

A `Relative_Deadline` pragma is allowed only immediately within a `task_definition` or the `declarative_part` of a `subprogram_body`. At most one such pragma shall appear within a given construct. 6/2

### Static Semantics

The *policy\_identifier* `EDF_Across_Priorities` is a task dispatching policy. 7/2

The following language-defined library package exists: 8/2

```
with Ada.Real_Time;
with Ada.Task_Identification;
package Ada.Dispatching.EDF is
 subtype Deadline is Ada.Real_Time.Time;
 Default_Deadline : constant Deadline :=
 Ada.Real_Time.Time_Last;
 procedure Set_Deadline (D : in Deadline;
 T : in Ada.Task_Identification.Task_Id :=
 Ada.Task_Identification.Current_Task);
 procedure Delay_Until_And_Set_Deadline (
 Delay_Until_Time : in Ada.Real_Time.Time;
 Deadline_Offset : in Ada.Real_Time.Time_Span);
 function Get_Deadline (T : Ada.Task_Identification.Task_Id :=
 Ada.Task_Identification.Current_Task) return Deadline;
end Ada.Dispatching.EDF; 9/2
```

### Post-Compilation Rules

If the `EDF_Across_Priorities` policy is specified for a partition, then the `Ceiling_Locking` policy (see D.3) shall also be specified for the partition. 10/2

If the `EDF_Across_Priorities` policy appears in a `Priority_Specific_Dispatching` pragma (see D.2.2) in a partition, then the `Ceiling_Locking` policy (see D.3) shall also be specified for the partition. 11/2

### Dynamic Semantics

A `Relative_Deadline` pragma has no effect if it occurs in the `declarative_part` of the `subprogram_body` of a subprogram other than the main subprogram. 12/2

The initial absolute deadline of a task containing pragma `Relative_Deadline` is the value of `Real_Time.Clock + relative_deadline_expression`, where the call of `Real_Time.Clock` is made between task creation and the start of its activation. If there is no `Relative_Deadline` pragma then the initial absolute 13/2

deadline of a task is the value of `Default_Deadline`. The environment task is also given an initial deadline by this rule.

- 14/2 The procedure `Set_Deadline` changes the absolute deadline of the task to `D`. The function `Get_Deadline` returns the absolute deadline of the task.
- 15/2 The procedure `Delay_Until_And_Set_Deadline` delays the calling task until time `Delay_Until_Time`. When the task becomes runnable again it will have deadline `Delay_Until_Time + Deadline_Offset`.
- 16/2 On a system with a single processor, the setting of the deadline of a task to the new value occurs immediately at the first point that is outside the execution of a protected action. If the task is currently on a ready queue it is removed and re-entered on to the ready queue determined by the rules defined below.
- 17/2 When `EDF_Across_Priorities` is specified for priority range *Low..High* all ready queues in this range are ordered by deadline. The task at the head of a queue is the one with the earliest deadline.
- 18/2 A task dispatching point occurs for the currently running task *T* to which policy `EDF_Across_Priorities` applies:
- 19/2 • when a change to the deadline of *T* occurs;
  - 20/2 • there is a task on the ready queue for the active priority of *T* with a deadline earlier than the deadline of *T*; or
  - 21/2 • there is a non-empty ready queue for that processor with a higher priority than the active priority of the running task.
- 22/2 In these cases, the currently running task is said to be preempted and is returned to the ready queue for its active priority.
- 23/2 For a task *T* to which policy `EDF_Across_Priorities` applies, the base priority is not a source of priority inheritance; the active priority when first activated or while it is blocked is defined as the maximum of the following:
- 24/2 • the lowest priority in the range specified as `EDF_Across_Priorities` that includes the base priority of *T*;
  - 25/2 • the priorities, if any, currently inherited by *T*;
  - 26/3 • the highest priority *P*, if any, less than the base priority of *T* such that one or more tasks are executing within a protected object with ceiling priority *P* and task *T* has an earlier deadline than all such tasks; and furthermore *T* has an earlier deadline than all other tasks on ready queues with priorities in the given `EDF_Across_Priorities` range that are strictly less than *P*.
- 27/2 When a task *T* is first activated or becomes unblocked, it is added to the ready queue corresponding to this active priority. Until it becomes blocked again, the active priority of *T* remains no less than this value; it will exceed this value only while it is inheriting a higher priority.
- 28/2 When the setting of the base priority of a ready task takes effect and the new priority is in a range specified as `EDF_Across_Priorities`, the task is added to the ready queue corresponding to its new active priority, as determined above.
- 29/2 For all the operations defined in `Dispatching.EDF`, `Tasking_Error` is raised if the task identified by *T* has terminated. `Program_Error` is raised if the value of *T* is `Null_Task_Id`.

*Bounded (Run-Time) Errors*

If EDF\_Across\_Priorities is specified for priority range *Low..High*, it is a bounded error to declare a protected object with ceiling priority *Low* or to assign the value *Low* to attribute 'Priority. In either case either Program\_Error is raised or the ceiling of the protected object is assigned the value *Low*+1. 30/2

*Erroneous Execution*

If a value of Task\_Id is passed as a parameter to any of the subprograms of this package and the corresponding task object no longer exists, the execution of the program is erroneous. 31/2

*Documentation Requirements*

On a multiprocessor, the implementation shall document any conditions that cause the completion of the setting of the deadline of a task to be delayed later than what is specified for a single processor. 32/2

## NOTES

18 If two adjacent priority ranges, *A..B* and *B+1..C* are specified to have policy EDF\_Across\_Priorities then this is not equivalent to this policy being specified for the single range, *A..C*. 33/2

19 The above rules implement the preemption-level protocol (also called Stack Resource Policy protocol) for resource sharing under EDF dispatching. The preemption-level for a task is denoted by its base priority. The definition of a ceiling preemption-level for a protected object follows the existing rules for ceiling locking. 34/2

## D.3 Priority Ceiling Locking

This clause specifies the interactions between priority task scheduling and protected object ceilings. This interaction is based on the concept of the *ceiling priority* of a protected object. 1

*Syntax*

The form of a pragma Locking\_Policy is as follows: 2

**pragma** Locking\_Policy(*policy\_identifier*); 3

*Legality Rules*

The *policy\_identifier* shall either be Ceiling\_Locking or an implementation-defined identifier. 4

*Post-Compilation Rules*

A Locking\_Policy pragma is a configuration pragma. 5

*Dynamic Semantics*

A locking policy specifies the details of protected object locking. All protected objects have a priority. The locking policy specifies the meaning of the priority of a protected object, and the relationships between these priorities and task priorities. In addition, the policy specifies the state of a task when it executes a protected action, and how its active priority is affected by the locking. The *locking policy* is specified by a Locking\_Policy pragma. For implementation-defined locking policies, the meaning of the priority of a protected object is implementation defined. If no Locking\_Policy pragma applies to any of the program units comprising a partition, the locking policy for that partition, as well as the meaning of the priority of a protected object, are implementation defined. 6/2

The expression of a Priority or Interrupt\_Priority pragma (see D.1) is evaluated as part of the creation of the corresponding protected object and converted to the subtype System.Any\_Priority or System.Interrupt\_Priority, respectively. The value of the expression is the initial priority of the corresponding protected object. If no Priority or Interrupt\_Priority pragma applies to a protected object, the initial priority is specified by the locking policy. 6.1/2

- 7 There is one predefined locking policy, `Ceiling_Locking`; this policy is defined as follows:
- 8/2
- Every protected object has a *ceiling priority*, which is determined by either a `Priority` or `Interrupt_Priority` pragma as defined in D.1, or by assignment to the `Priority` attribute as described in D.5.2. The ceiling priority of a protected object (or ceiling, for short) is an upper bound on the active priority a task can have when it calls protected operations of that protected object.
- 9/2
- The initial ceiling priority of a protected object is equal to the initial priority for that object.
- 10/2
- If an `Interrupt_Handler` or `Attach_Handler` pragma (see C.3.1) appears in a `protected_definition` without an `Interrupt_Priority` pragma, the initial priority of protected objects of that type is implementation defined, but in the range of the subtype `System.Interrupt_Priority`.
- 11/2
- If no pragma `Priority`, `Interrupt_Priority`, `Interrupt_Handler`, or `Attach_Handler` is specified in the `protected_definition`, then the initial priority of the corresponding protected object is `System.Priority'Last`.
- 12
- While a task executes a protected action, it inherits the ceiling priority of the corresponding protected object.
- 13
- When a task calls a protected operation, a check is made that its active priority is not higher than the ceiling of the corresponding protected object; `Program_Error` is raised if this check fails.

*Bounded (Run-Time) Errors*

- 13.1/2 Following any change of priority, it is a bounded error for the active priority of any task with a call queued on an entry of a protected object to be higher than the ceiling priority of the protected object. In this case one of the following applies:
- 13.2/2
- at any time prior to executing the entry body `Program_Error` is raised in the calling task;
- 13.3/2
- when the entry is open the entry body is executed at the ceiling priority of the protected object;
- 13.4/2
- when the entry is open the entry body is executed at the ceiling priority of the protected object and then `Program_Error` is raised in the calling task; or
- 13.5/2
- when the entry is open the entry body is executed at the ceiling priority of the protected object that was in effect when the entry call was queued.

*Implementation Permissions*

- 14 The implementation is allowed to round all ceilings in a certain subrange of `System.Priority` or `System.Interrupt_Priority` up to the top of that subrange, uniformly.
- 15/2 Implementations are allowed to define other locking policies, but need not support more than one locking policy per partition.
- 16 Since implementations are allowed to place restrictions on code that runs at an interrupt-level active priority (see C.3.1 and D.2.1), the implementation may implement a language feature in terms of a protected object with an implementation-defined ceiling, but the ceiling shall be no less than `Priority'Last`.

*Implementation Advice*

- 17 The implementation should use names that end with “`_Locking`” for implementation-defined locking policies.

## NOTES

- 18 20 While a task executes in a protected action, it can be preempted only by tasks whose active priorities are higher than the ceiling priority of the protected object.



- 21 If a protected object has a ceiling priority in the range of `Interrupt_Priority`, certain interrupts are blocked while protected actions of that object execute. In the extreme, if the ceiling is `Interrupt_Priority'Last`, all blockable interrupts are blocked during that time. 19
- 22 The ceiling priority of a protected object has to be in the `Interrupt_Priority` range if one of its procedures is to be used as an interrupt handler (see C.3). 20
- 23 When specifying the ceiling of a protected object, one should choose a value that is at least as high as the highest active priority at which tasks can be executing when they call protected operations of that object. In determining this value the following factors, which can affect active priority, should be considered: the effect of `Set_Priority`, nested protected operations, entry calls, task activation, and other implementation-defined factors. 21
- 24 Attaching a protected procedure whose ceiling is below the interrupt hardware priority to an interrupt causes the execution of the program to be erroneous (see C.3.1). 22
- 25 On a single processor implementation, the ceiling priority rules guarantee that there is no possibility of deadlock involving only protected subprograms (excluding the case where a protected operation calls another protected operation on the same protected object). 23

## D.4 Entry Queuing Policies

This clause specifies a mechanism for a user to choose an entry *queuing policy*. It also defines two such policies. Other policies are implementation defined. 1/1

### Syntax

The form of a `pragma Queuing_Policy` is as follows: 2

**pragma** `Queuing_Policy`(*policy\_identifier*); 3

### Legality Rules

The *policy\_identifier* shall be either `FIFO_Queueing`, `Priority_Queueing` or an implementation-defined identifier. 4

### Post-Compilation Rules

A `Queuing_Policy` pragma is a configuration pragma. 5

### Dynamic Semantics

A *queuing policy* governs the order in which tasks are queued for entry service, and the order in which different entry queues are considered for service. The queuing policy is specified by a `Queuing_Policy` pragma. 6

Two queuing policies, `FIFO_Queueing` and `Priority_Queueing`, are language defined. If no `Queuing_Policy` pragma applies to any of the program units comprising the partition, the queuing policy for that partition is `FIFO_Queueing`. The rules for this policy are specified in 9.5.3 and 9.7.1. 7/2

The `Priority_Queueing` policy is defined as follows: 8

- The calls to an entry (including a member of an entry family) are queued in an order consistent with the priorities of the calls. The *priority of an entry call* is initialized from the active priority of the calling task at the time the call is made, but can change later. Within the same priority, the order is consistent with the calling (or requeuing, or priority setting) time (that is, a FIFO order). 9
- After a call is first queued, changes to the active priority of a task do not affect the priority of the call, unless the base priority of the task is set while the task is blocked on an entry call. 10/1
- When the base priority of a task is set (see D.5), if the task is blocked on an entry call, and the call is queued, the priority of the call is updated to the new active priority of the calling task. 11

This causes the call to be removed from and then reinserted in the queue at the new active priority.

- 12 • When more than one condition of an `entry_barrier` of a protected object becomes True, and more than one of the respective queues is nonempty, the call with the highest priority is selected. If more than one such call has the same priority, the call that is queued on the entry whose declaration is first in textual order in the `protected_definition` is selected. For members of the same entry family, the one with the lower family index is selected.
- 13 • If the expiration time of two or more open `delay_alternatives` is the same and no other `accept_alternatives` are open, the `sequence_of_statements` of the `delay_alternative` that is first in textual order in the `selective_accept` is executed.
- 14 • When more than one alternative of a `selective_accept` is open and has queued calls, an alternative whose queue has the highest-priority call at its head is selected. If two or more open alternatives have equal-priority queued calls, then a call on the entry in the `accept_alternative` that is first in textual order in the `selective_accept` is selected.

#### *Implementation Permissions*

- 15/2 Implementations are allowed to define other queuing policies, but need not support more than one queuing policy per partition.
- 15.1/2 Implementations are allowed to defer the reordering of entry queues following a change of base priority of a task blocked on the entry call if it is not practical to reorder the queue immediately.

#### *Implementation Advice*

- 16 The implementation should use names that end with “\_Queuing” for implementation-defined queuing policies.

## D.5 Dynamic Priorities

- 1/2 This clause describes how the priority of an entity can be modified or queried at run time.

### D.5.1 Dynamic Priorities for Tasks

- 1 This clause describes how the base priority of a task can be modified or queried at run time.

#### *Static Semantics*

- 2 The following language-defined library package exists:

```

3/2 with System;
 with Ada.Task_Identification; -- See C.7.1
 package Ada.Dynamic_Priorities is
 pragma Preelaborate(Dynamic_Priorities);
4 procedure Set_Priority(Priority : in System.Any_Priority;
 T : in Ada.Task_Identification.Task_Id :=
 Ada.Task_Identification.Current_Task);
5 function Get_Priority (T : Ada.Task_Identification.Task_Id :=
 Ada.Task_Identification.Current_Task)
 return System.Any_Priority;
6 end Ada.Dynamic_Priorities;
```

*Dynamic Semantics*

The procedure `Set_Priority` sets the base priority of the specified task to the specified `Priority` value. `Set_Priority` has no effect if the task is terminated. 7

The function `Get_Priority` returns `T`'s current base priority. `Tasking_Error` is raised if the task is terminated. 8

`Program_Error` is raised by `Set_Priority` and `Get_Priority` if `T` is equal to `Null_Task_Id`. 9

On a system with a single processor, the setting of the base priority of a task `T` to the new value occurs immediately at the first point when `T` is outside the execution of a protected action. 10/2

*Bounded (Run-Time) Errors*

*This paragraph was deleted.* 11/2

*Erroneous Execution*

If any subprogram in this package is called with a parameter `T` that specifies a task object that no longer exists, the execution of the program is erroneous. 12

*Documentation Requirements*

On a multiprocessor, the implementation shall document any conditions that cause the completion of the setting of the priority of a task to be delayed later than what is specified for a single processor. 12.1/2

*Metrics*

The implementation shall document the following metric: 13

- The execution time of a call to `Set_Priority`, for the nonpreempting case, in processor clock cycles. This is measured for a call that modifies the priority of a ready task that is not running (which cannot be the calling one), where the new base priority of the affected task is lower than the active priority of the calling task, and the affected task is not on any entry queue and is not executing a protected operation. 14

## NOTES

26 Setting a task's base priority affects task dispatching. First, it can change the task's active priority. Second, under the `FIFO_Within_Priorities` policy it always causes the task to move to the tail of the ready queue corresponding to its active priority, even if the new base priority is unchanged. 15/2

27 Under the priority queuing policy, setting a task's base priority has an effect on a queued entry call if the task is blocked waiting for the call. That is, setting the base priority of a task causes the priority of a queued entry call from that task to be updated and the call to be removed and then reinserted in the entry queue at the new priority (see D.4), unless the call originated from the `triggering_statement` of an `asynchronous_select`. 16

28 The effect of two or more `Set_Priority` calls executed in parallel on the same task is defined as executing these calls in some serial order. 17

29 The rule for when `Tasking_Error` is raised for `Set_Priority` or `Get_Priority` is different from the rule for when `Tasking_Error` is raised on an entry call (see 9.5.3). In particular, querying the priority of a completed or an abnormal task is allowed, so long as the task is not yet terminated, and setting the priority of a task is allowed for any task state (including for terminated tasks). 18/3

30 Changing the priorities of a set of tasks can be performed by a series of calls to `Set_Priority` for each task separately. For this to work reliably, it should be done within a protected operation that has high enough ceiling priority to guarantee that the operation completes without being preempted by any of the affected tasks. 19

## D.5.2 Dynamic Priorities for Protected Objects

1/2 This clause specifies how the priority of a protected object can be modified or queried at run time.

### *Static Semantics*

2/2 The following attribute is defined for a prefix *P* that denotes a protected object:

3/2 **P'Priority** Denotes a non-aliased component of the protected object *P*. This component is of type `System.Any_Priority` and its value is the priority of *P*. *P'Priority* denotes a variable if and only if *P* denotes a variable. A reference to this attribute shall appear only within the body of *P*.

4/2 The initial value of this attribute is the initial value of the priority of the protected object, and can be changed by an assignment.

### *Dynamic Semantics*

5/2 If the locking policy `Ceiling_Locking` (see D.3) is in effect then the ceiling priority of a protected object *P* is set to the value of *P'Priority* at the end of each protected action of *P*.

6/2 If the locking policy `Ceiling_Locking` is in effect, then for a protected object *P* with either an `Attach_Handler` or `Interrupt_Handler` pragma applying to one of its procedures, a check is made that the value to be assigned to *P'Priority* is in the range `System.Interrupt_Priority`. If the check fails, `Program_Error` is raised.

### *Metrics*

7/2 The implementation shall document the following metric:

- 8/2 • The difference in execution time of calls to the following procedures in protected object *P*:

```
9/2 protected P is
 procedure Do_Not_Set_Ceiling (Pr : System.Any_Priority);
 procedure Set_Ceiling (Pr : System.Any_Priority);
end P;

10/2 protected body P is
 procedure Do_Not_Set_Ceiling (Pr : System.Any_Priority) is
 begin
 null;
 end;
 procedure Set_Ceiling (Pr : System.Any_Priority) is
 begin
 P'Priority := Pr;
 end;
end P;
```

#### NOTES

11/2 31 Since *P'Priority* is a normal variable, the value following an assignment to the attribute immediately reflects the new value even though its impact on the ceiling priority of *P* is postponed until completion of the protected action in which it is executed.

## D.6 Preemptive Abort

This clause specifies requirements on the immediacy with which an aborted construct is completed. 1

### *Dynamic Semantics*

On a system with a single processor, an aborted construct is completed immediately at the first point that is outside the execution of an abort-deferred operation. 2

### *Documentation Requirements*

On a multiprocessor, the implementation shall document any conditions that cause the completion of an aborted construct to be delayed later than what is specified for a single processor. 3

### *Metrics*

The implementation shall document the following metrics: 4

- The execution time, in processor clock cycles, that it takes for an `abort_statement` to cause the completion of the aborted task. This is measured in a situation where a task T2 preempts task T1 and aborts T1. T1 does not have any finalization code. T2 shall verify that T1 has terminated, by means of the Terminated attribute. 5
- On a multiprocessor, an upper bound in seconds, on the time that the completion of an aborted task can be delayed beyond the point that it is required for a single processor. 6
- An upper bound on the execution time of an `asynchronous_select`, in processor clock cycles. This is measured between a point immediately before a task T1 executes a protected operation Pr.Set that makes the condition of an `entry_barrier` Pr.Wait True, and the point where task T2 resumes execution immediately after an entry call to Pr.Wait in an `asynchronous_select`. T1 preempts T2 while T2 is executing the abortable part, and then blocks itself so that T2 can execute. The execution time of T1 is measured separately, and subtracted. 7/2
- An upper bound on the execution time of an `asynchronous_select`, in the case that no asynchronous transfer of control takes place. This is measured between a point immediately before a task executes the `asynchronous_select` with a nonnull abortable part, and the point where the task continues execution immediately after it. The execution time of the abortable part is subtracted. 8

### *Implementation Advice*

Even though the `abort_statement` is included in the list of potentially blocking operations (see 9.5.1), it is recommended that this statement be implemented in a way that never requires the task executing the `abort_statement` to block. 9

On a multi-processor, the delay associated with aborting a task on another processor should be bounded; the implementation should use periodic polling, if necessary, to achieve this. 10

### NOTES

32 Abortion does not change the active or base priority of the aborted task. 11

33 Abortion cannot be more immediate than is allowed by the rules for deferral of abortion during finalization and in protected actions. 12

## D.7 Tasking Restrictions

- 1 This clause defines restrictions that can be used with a pragma Restrictions (see 13.12) to facilitate the construction of highly efficient tasking run-time systems.

### *Static Semantics*

- 2 The following *restriction\_identifiers* are language defined:

3/3 **No\_Task\_Hierarchy**

No task depends on a task other than the environment task of the partition.

4/3 **No\_Nested\_Finalization**

Objects of a type that needs finalization (see 7.6) shall be declared only at library level. If an access type does not have library-level accessibility, then there are no *allocators* of the type where the type determined by the *subtype\_mark* of the *subtype\_indication* or *qualified\_expression* needs finalization.

5 **No\_Abort\_Statements**

There are no *abort\_statements*, and there are no calls on Task\_Identification.Abort\_Task.

6 **No\_Terminate\_Alternatives**

There are no *selective\_accepts* with *terminate\_alternatives*.

7 **No\_Task\_Allocators**

There are no *allocators* for task types or types containing task subcomponents.

8 **No\_Implicit\_Heap\_Allocations**

There are no operations that implicitly require heap storage allocation to be performed by the implementation. The operations that implicitly require heap storage allocation are implementation defined.

9/2 **No\_Dynamic\_Priorities**

There are no semantic dependences on the package Dynamic\_Priorities, and no occurrences of the attribute Priority.

10/3 **No\_Dynamic\_Attachment**

There is no use of any of the operations defined in package Interrupts (*Is\_Reserved*, *Is\_Attached*, *Current\_Handler*, *Attach\_Handler*, *Exchange\_Handler*, *Detach\_Handler*, and *Reference*).

10.1/3 **No\_Local\_Protected\_Objects**

Protected objects are declared only at library level.

10.2/3 **No\_Local\_Timing\_Events**

Timing\_Events are declared only at library level.

10.3/2 **No\_Protected\_Type\_Allocators**

There are no *allocators* for protected types or types containing protected type subcomponents.

10.4/2 **No\_Relative\_Delay**

There are no *delay\_relative\_statements*.

10.5/2 **No\_Requeue\_Statements**

There are no *requeue\_statements*.

10.6/2 **No\_Select\_Statements**

There are no *select\_statements*.

|                                                                                                                                                                                                                                                                                                                                                                                                                                  |        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| No_Specific_Termination_Handlers                                                                                                                                                                                                                                                                                                                                                                                                 | 10.7/2 |
| There are no calls to the Set_Specific_Handler and Specific_Handler subprograms in Task_Termination.                                                                                                                                                                                                                                                                                                                             |        |
| Simple_Barriers                                                                                                                                                                                                                                                                                                                                                                                                                  | 10.8/3 |
| The Boolean expression in each entry barrier is either a static expression or a name that statically denotes a component of the enclosing protected object.                                                                                                                                                                                                                                                                      |        |
| The following <i>restriction_parameter_identifiers</i> are language defined:                                                                                                                                                                                                                                                                                                                                                     |        |
| Max_Select_Alternatives                                                                                                                                                                                                                                                                                                                                                                                                          | 12     |
| Specifies the maximum number of alternatives in a <i>selective_accept</i> .                                                                                                                                                                                                                                                                                                                                                      |        |
| Max_Task_Entries                                                                                                                                                                                                                                                                                                                                                                                                                 | 13     |
| Specifies the maximum number of entries per task. The bounds of every entry family of a task unit shall be static, or shall be defined by a discriminant of a subtype whose corresponding bound is static. A value of zero indicates that no rendezvous are possible.                                                                                                                                                            |        |
| Max_Protected_Entries                                                                                                                                                                                                                                                                                                                                                                                                            | 14     |
| Specifies the maximum number of entries per protected type. The bounds of every entry family of a protected unit shall be static, or shall be defined by a discriminant of a subtype whose corresponding bound is static.                                                                                                                                                                                                        |        |
| <i>Dynamic Semantics</i>                                                                                                                                                                                                                                                                                                                                                                                                         |        |
| The following <i>restriction_identifier</i> is language defined:                                                                                                                                                                                                                                                                                                                                                                 |        |
| No_Task_Termination                                                                                                                                                                                                                                                                                                                                                                                                              | 15.1/2 |
| All tasks are non-terminating. It is implementation-defined what happens if a task attempts to terminate. If there is a fall-back handler (see C.7.3) set for the partition it should be called when the first task attempts to terminate.                                                                                                                                                                                       |        |
| The following <i>restriction_parameter_identifiers</i> are language defined:                                                                                                                                                                                                                                                                                                                                                     |        |
| Max_Storage_At_Blocking                                                                                                                                                                                                                                                                                                                                                                                                          | 17/1   |
| Specifies the maximum portion (in storage elements) of a task's <i>Storage_Size</i> that can be retained by a blocked task. If an implementation chooses to detect a violation of this restriction, <i>Storage_Error</i> should be raised; otherwise, the behavior is implementation defined.                                                                                                                                    |        |
| Max_Asynchronous_Select_Nesting                                                                                                                                                                                                                                                                                                                                                                                                  | 18/1   |
| Specifies the maximum dynamic nesting level of <i>asynchronous_selects</i> . A value of zero prevents the use of any <i>asynchronous_select</i> and, if a program contains an <i>asynchronous_select</i> , it is illegal. If an implementation chooses to detect a violation of this restriction for values other than zero, <i>Storage_Error</i> should be raised; otherwise, the behavior is implementation defined.           |        |
| Max_Tasks                                                                                                                                                                                                                                                                                                                                                                                                                        | 19/1   |
| Specifies the maximum number of task creations that may be executed over the lifetime of a partition, not counting the creation of the environment task. A value of zero prevents any task creation and, if a program contains a task creation, it is illegal. If an implementation chooses to detect a violation of this restriction, <i>Storage_Error</i> should be raised; otherwise, the behavior is implementation defined. |        |
| Max_Entry_Queue_Length                                                                                                                                                                                                                                                                                                                                                                                                           | 19.1/2 |
| Max_Entry_Queue_Length defines the maximum number of calls that are queued on an entry. Violation of this restriction results in the raising of <i>Program_Error</i> at the point of the call or requeue.                                                                                                                                                                                                                        |        |

- 20 It is implementation defined whether the use of pragma Restrictions results in a reduction in executable program size, storage requirements, or execution time. If possible, the implementation should provide quantitative descriptions of such effects for each restriction.

*Implementation Advice*

- 21 When feasible, the implementation should take advantage of the specified restrictions to produce a more efficient implementation.

NOTES

- 22 34 The above Storage\_Checks can be suppressed with pragma Suppress.

## D.8 Monotonic Time

- 1 This clause specifies a high-resolution, monotonic clock package.

*Static Semantics*

- 2 The following language-defined library package exists:

```

3 package Ada.Real_Time is
4 type Time is private;
 Time_First : constant Time;
 Time_Last : constant Time;
 Time_Unit : constant := implementation-defined-real-number;
5
 type Time_Span is private;
 Time_Span_First : constant Time_Span;
 Time_Span_Last : constant Time_Span;
 Time_Span_Zero : constant Time_Span;
 Time_Span_Unit : constant Time_Span;
6
 Tick : constant Time_Span;
 function Clock return Time;
7
 function "+" (Left : Time; Right : Time_Span) return Time;
 function "+" (Left : Time_Span; Right : Time) return Time;
 function "-" (Left : Time; Right : Time_Span) return Time;
 function "-" (Left : Time; Right : Time) return Time_Span;
8
 function "<" (Left, Right : Time) return Boolean;
 function "<=" (Left, Right : Time) return Boolean;
 function ">" (Left, Right : Time) return Boolean;
 function ">=" (Left, Right : Time) return Boolean;
9
 function "+" (Left, Right : Time_Span) return Time_Span;
 function "-" (Left, Right : Time_Span) return Time_Span;
 function "-" (Right : Time_Span) return Time_Span;
 function "*" (Left : Time_Span; Right : Integer) return Time_Span;
 function "*" (Left : Integer; Right : Time_Span) return Time_Span;
 function "/" (Left, Right : Time_Span) return Integer;
 function "/" (Left : Time_Span; Right : Integer) return Time_Span;
10
 function "abs" (Right : Time_Span) return Time_Span;
11/1 This paragraph was deleted.
12
 function "<" (Left, Right : Time_Span) return Boolean;
 function "<=" (Left, Right : Time_Span) return Boolean;
 function ">" (Left, Right : Time_Span) return Boolean;
 function ">=" (Left, Right : Time_Span) return Boolean;
13
 function To_Duration (TS : Time_Span) return Duration;
 function To_Time_Span (D : Duration) return Time_Span;
```



```

function Nanoseconds (NS : Integer) return Time_Span;
function Microseconds (US : Integer) return Time_Span;
function Milliseconds (MS : Integer) return Time_Span;
function Seconds (S : Integer) return Time_Span;
function Minutes (M : Integer) return Time_Span;

type Seconds_Count is range implementation-defined;

procedure Split(T : in Time; SC : out Seconds_Count; TS : out Time_Span);
function Time_Of(SC : Seconds_Count; TS : Time_Span) return Time;

private
 ... -- not specified by the language
end Ada.Real_Time;

```

In this Annex, *real time* is defined to be the physical time as observed in the external environment. The type Time is a *time type* as defined by 9.6; values of this type may be used in a *delay\_until\_statement*. Values of this type represent segments of an ideal time line. The set of values of the type Time corresponds one-to-one with an implementation-defined range of mathematical integers.

The Time value I represents the half-open real time interval that starts with E+I\*Time\_Unit and is limited by E+(I+1)\*Time\_Unit, where Time\_Unit is an implementation-defined real number and E is an unspecified origin point, the *epoch*, that is the same for all values of the type Time. It is not specified by the language whether the time values are synchronized with any standard time reference. For example, E can correspond to the time of system initialization or it can correspond to the epoch of some time standard.

Values of the type Time\_Span represent length of real time duration. The set of values of this type corresponds one-to-one with an implementation-defined range of mathematical integers. The Time\_Span value corresponding to the integer I represents the real-time duration I\*Time\_Unit.

Time\_First and Time\_Last are the smallest and largest values of the Time type, respectively. Similarly, Time\_Span\_First and Time\_Span\_Last are the smallest and largest values of the Time\_Span type, respectively.

A value of type Seconds\_Count represents an elapsed time, measured in seconds, since the epoch.

#### Dynamic Semantics

Time\_Unit is the smallest amount of real time representable by the Time type; it is expressed in seconds. Time\_Span\_Unit is the difference between two successive values of the Time type. It is also the smallest positive value of type Time\_Span. Time\_Unit and Time\_Span\_Unit represent the same real time duration. A *clock tick* is a real time interval during which the clock value (as observed by calling the Clock function) remains constant. Tick is the average length of such intervals.

The function To\_Duration converts the value TS to a value of type Duration. Similarly, the function To\_Time\_Span converts the value D to a value of type Time\_Span. For To\_Duration, the result is rounded to the nearest value of type Duration (away from zero if exactly halfway between two values). If the result is outside the range of Duration, Constraint\_Error is raised. For To\_Time\_Span, the value of D is first rounded to the nearest integral multiple of Time\_Unit, away from zero if exactly halfway between two multiples. If the rounded value is outside the range of Time\_Span, Constraint\_Error is raised. Otherwise, the value is converted to the type Time\_Span.

To\_Duration(Time\_Span\_Zero) returns 0.0, and To\_Time\_Span(0.0) returns Time\_Span\_Zero.

The functions Nanoseconds, Microseconds, Milliseconds, Seconds, and Minutes convert the input parameter to a value of the type Time\_Span. NS, US, MS, S, and M are interpreted as a number of nanoseconds, microseconds, milliseconds, seconds, and minutes respectively. The input parameter is first converted to seconds and rounded to the nearest integral multiple of Time\_Unit, away from zero if exactly

halfway between two multiples. If the rounded value is outside the range of Time\_Span, Constraint\_Error is raised. Otherwise, the rounded value is converted to the type Time\_Span.

- 27 The effects of the operators on Time and Time\_Span are as for the operators defined for integer types.
- 28 The function Clock returns the amount of time since the epoch.
- 29 The effects of the Split and Time\_Of operations are defined as follows, treating values of type Time, Time\_Span, and Seconds\_Count as mathematical integers. The effect of Split(T,SC,TS) is to set SC and TS to values such that  $T * \text{Time\_Unit} = \text{SC} * 1.0 + \text{TS} * \text{Time\_Unit}$ , and  $0.0 \leq \text{TS} * \text{Time\_Unit} < 1.0$ . The value returned by Time\_Of(SC,TS) is the value T such that  $T * \text{Time\_Unit} = \text{SC} * 1.0 + \text{TS} * \text{Time\_Unit}$ .

#### Implementation Requirements

- 30 The range of Time values shall be sufficient to uniquely represent the range of real times from program start-up to 50 years later. Tick shall be no greater than 1 millisecond. Time\_Unit shall be less than or equal to 20 microseconds.
- 31 Time\_Span\_First shall be no greater than -3600 seconds, and Time\_Span\_Last shall be no less than 3600 seconds.
- 32 A *clock jump* is the difference between two successive distinct values of the clock (as observed by calling the Clock function). There shall be no backward clock jumps.

#### Documentation Requirements

- 33 The implementation shall document the values of Time\_First, Time\_Last, Time\_Span\_First, Time\_Span\_Last, Time\_Span\_Unit, and Tick.
- 34 The implementation shall document the properties of the underlying time base used for the clock and for type Time, such as the range of values supported and any relevant aspects of the underlying hardware or operating system facilities used.
- 35 The implementation shall document whether or not there is any synchronization with external time references, and if such synchronization exists, the sources of synchronization information, the frequency of synchronization, and the synchronization method applied.
- 36/1 The implementation shall document any aspects of the external environment that could interfere with the clock behavior as defined in this clause.

#### Metrics

- 37 For the purpose of the metrics defined in this clause, real time is defined to be the International Atomic Time (TAI).
- 38 The implementation shall document the following metrics:
- 39 • An upper bound on the real-time duration of a clock tick. This is a value D such that if  $t_1$  and  $t_2$  are any real times such that  $t_1 < t_2$  and  $\text{Clock}_{t_1} = \text{Clock}_{t_2}$  then  $t_2 - t_1 \leq D$ .
- 40 • An upper bound on the size of a clock jump.
- 41 • An upper bound on the *drift rate* of Clock with respect to real time. This is a real number D such that
- 42 
$$E * (1 - D) \leq (\text{Clock}_{t+E} - \text{Clock}_t) \leq E * (1 + D)$$
provided that:  $\text{Clock}_t + E * (1 + D) \leq \text{Time\_Last}$ .

- where  $\text{Clock}_t$  is the value of Clock at time  $t$ , and  $E$  is a real time duration not less than 24 hours. The value of  $E$  used for this metric shall be reported. 43
- An upper bound on the execution time of a call to the Clock function, in processor clock cycles. 44
- Upper bounds on the execution times of the operators of the types Time and Time\_Span, in processor clock cycles. 45

#### Implementation Permissions

Implementations targeted to machines with word size smaller than 32 bits need not support the full range and granularity of the Time and Time\_Span types. 46

#### Implementation Advice

When appropriate, implementations should provide configuration mechanisms to change the value of Tick. 47

It is recommended that Calendar.Clock and Real\_Time.Clock be implemented as transformations of the same time base. 48

It is recommended that the “best” time base which exists in the underlying system be available to the application through Clock. “Best” may mean highest accuracy or largest range. 49

#### NOTES

35 The rules in this clause do not imply that the implementation can protect the user from operator or installation errors which could result in the clock being set incorrectly. 50

36 Time\_Unit is the granularity of the Time type. In contrast, Tick represents the granularity of Real\_Time.Clock. There is no requirement that these be the same. 51

## D.9 Delay Accuracy

This clause specifies performance requirements for the `delay_statement`. The rules apply both to `delay_relative_statement` and to `delay_until_statement`. Similarly, they apply equally to a simple `delay_statement` and to one which appears in a `delay_alternative`. 1

#### Dynamic Semantics

The effect of the `delay_statement` for Real\_Time.Time is defined in terms of Real\_Time.Clock: 2

- If  $C_1$  is a value of Clock read before a task executes a `delay_relative_statement` with duration  $D$ , and  $C_2$  is a value of Clock read after the task resumes execution following that `delay_statement`, then  $C_2 - C_1 \geq D$ . 3
- If  $C$  is a value of Clock read after a task resumes execution following a `delay_until_statement` with Real\_Time.Time value  $T$ , then  $C \geq T$ . 4

A simple `delay_statement` with a negative or zero value for the expiration time does not cause the calling task to be blocked; it is nevertheless a potentially blocking operation (see 9.5.1). 5

When a `delay_statement` appears in a `delay_alternative` of a `timed_entry_call` the selection of the entry call is attempted, regardless of the specified expiration time. When a `delay_statement` appears in a `select_alternative`, and a call is queued on one of the open entries, the selection of that entry call proceeds, regardless of the value of the delay expression. 6/3

#### Documentation Requirements

The implementation shall document the minimum value of the delay expression of a `delay_relative_statement` that causes the task to actually be blocked. 7

The implementation shall document the minimum difference between the value of the delay expression of a `delay_until_statement` and the value of `Real_Time.Clock`, that causes the task to actually be blocked.

#### Metrics

The implementation shall document the following metrics:

- An upper bound on the execution time, in processor clock cycles, of a `delay_relative_statement` whose requested value of the delay expression is less than or equal to zero.
- An upper bound on the execution time, in processor clock cycles, of a `delay_until_statement` whose requested value of the delay expression is less than or equal to the value of `Real_Time.Clock` at the time of executing the statement. Similarly, for `Calendar.Clock`.
- An upper bound on the *lateness* of a `delay_relative_statement`, for a positive value of the delay expression, in a situation where the task has sufficient priority to preempt the processor as soon as it becomes ready, and does not need to wait for any other execution resources. The upper bound is expressed as a function of the value of the delay expression. The lateness is obtained by subtracting the value of the delay expression from the *actual duration*. The actual duration is measured from a point immediately before a task executes the `delay_statement` to a point immediately after the task resumes execution following this statement.
- An upper bound on the lateness of a `delay_until_statement`, in a situation where the value of the requested expiration time is after the time the task begins executing the statement, the task has sufficient priority to preempt the processor as soon as it becomes ready, and it does not need to wait for any other execution resources. The upper bound is expressed as a function of the difference between the requested expiration time and the clock value at the time the statement begins execution. The lateness of a `delay_until_statement` is obtained by subtracting the requested expiration time from the real time that the task resumes execution following this statement.

#### NOTES

*This paragraph was deleted.*

## D.10 Synchronous Task Control

This clause describes a language-defined private semaphore (suspension object), which can be used for *two-stage suspend* operations and as a simple building block for implementing higher-level queues.

#### Static Semantics

The following language-defined package exists:

```
package Ada.Synchronous_Task_Control is
 pragma Preelaborate(Synchronous_Task_Control);

 type Suspension_Object is limited private;
 procedure Set_True(S : in out Suspension_Object);
 procedure Set_False(S : in out Suspension_Object);
 function Current_State(S : Suspension_Object) return Boolean;
 procedure Suspend_Until_True(S : in out Suspension_Object);
private
 ... -- not specified by the language
end Ada.Synchronous_Task_Control;
```

The type `Suspension_Object` is a by-reference type.

The following language-defined package exists:

```

package Ada.Synchronous_Task_Control.EDF is
 procedure Suspend_Until_True_And_Set_Deadline
 (S : in out Suspension_Object;
 TS : in Ada.Real_Time.Time_Span);
end Ada.Synchronous_Task_Control.EDF;

```

5.2/3
*Dynamic Semantics*

An object of the type `Suspension_Object` has two visible states: `True` and `False`. Upon initialization, its value is set to `False`. 6/2

The operations `Set_True` and `Set_False` are atomic with respect to each other and with respect to `Suspend_Until_True`; they set the state to `True` and `False` respectively. 7/2

`Current_State` returns the current state of the object. 8

The procedure `Suspend_Until_True` blocks the calling task until the state of the object `S` is `True`; at that point the task becomes ready and the state of the object becomes `False`. 9/2

`Program_Error` is raised upon calling `Suspend_Until_True` if another task is already waiting on that suspension object. `Suspend_Until_True` is a potentially blocking operation (see 9.5.1). 10

The procedure `Suspend_Until_True_And_Set_Deadline` blocks the calling task until the state of the object `S` is `True`; at that point the task becomes ready with a deadline of `Ada.Real_Time.Clock + TS`, and the state of the object becomes `False`. `Suspend_Until_True_And_Set_Deadline` is a potentially blocking operation. 10.1/3

*Implementation Requirements*

The implementation is required to allow the calling of `Set_False` and `Set_True` during any protected action, even one that has its ceiling priority in the `Interrupt_Priority` range. 11

## NOTES

37 More complex schemes, such as setting the deadline relative to when `Set_True` is called, can be programmed using a protected object. 12/3

## D.11 Asynchronous Task Control

This clause introduces a language-defined package to do asynchronous suspend/resume on tasks. It uses a conceptual *held priority* value to represent the task's *held* state. 1

*Static Semantics*

The following language-defined library package exists: 2

```

with Ada.Task_Identification;
package Ada.Asynchronous_Task_Control is
 pragma Preelaborate(Asynchronous_Task_Control);
 procedure Hold(T : in Ada.Task_Identification.Task_Id);
 procedure Continue(T : in Ada.Task_Identification.Task_Id);
 function Is_Held(T : Ada.Task_Identification.Task_Id)
 return Boolean;
end Ada.Asynchronous_Task_Control;

```

3/2
*Dynamic Semantics*

After the `Hold` operation has been applied to a task, the task becomes *held*. For each processor there is a conceptual *idle task*, which is always ready. The base priority of the idle task is below `System.Any_Priority_First`. The *held priority* is a constant of the type `Integer` whose value is below the base priority of the idle task. 4/2

- 4.1/2 For any priority below System.Any\_Priority'First, the task dispatching policy is FIFO\_Within\_Priorities.
- 5/2 The Hold operation sets the state of T to held. For a held task, the active priority is reevaluated as if the base priority of the task were the held priority.
- 6/2 The Continue operation resets the state of T to not-held; its active priority is then reevaluated as determined by the task dispatching policy associated with its base priority.
- 7 The Is\_Held function returns True if and only if T is in the held state.
- 8 As part of these operations, a check is made that the task identified by T is not terminated. Tasking\_Error is raised if the check fails. Program\_Error is raised if the value of T is Null\_Task\_Id.

*Erroneous Execution*

- 9 If any operation in this package is called with a parameter T that specifies a task object that no longer exists, the execution of the program is erroneous.

*Implementation Permissions*

- 10 An implementation need not support Asynchronous\_Task\_Control if it is infeasible to support it in the target environment.

NOTES

- 11 38 It is a consequence of the priority rules that held tasks cannot be dispatched on any processor in a partition (unless they are inheriting priorities) since their priorities are defined to be below the priority of any idle task.
- 12 39 The effect of calling Get\_Priority and Set\_Priority on a Held task is the same as on any other task.
- 13 40 Calling Hold on a held task or Continue on a non-held task has no effect.
- 14 41 The rules affecting queuing are derived from the above rules, in addition to the normal priority rules:
- 15 • When a held task is on the ready queue, its priority is so low as to never reach the top of the queue as long as there are other tasks on that queue.
- 16 • If a task is executing in a protected action, inside a rendezvous, or is inheriting priorities from other sources (e.g. when activated), it continues to execute until it is no longer executing the corresponding construct.
- 17 • If a task becomes held while waiting (as a caller) for a rendezvous to complete, the active priority of the accepting task is not affected.
- 18/1 • If a task becomes held while waiting in a `selective_accept`, and an entry call is issued to one of the open entries, the corresponding `accept_alternative` executes. When the rendezvous completes, the active priority of the accepting task is lowered to the held priority (unless it is still inheriting from other sources), and the task does not execute until another Continue.
- 19 • The same holds if the held task is the only task on a protected entry queue whose barrier becomes open. The corresponding entry body executes.

## D.12 Other Optimizations and Determinism Rules

- 1 This clause describes various requirements for improving the response and determinism in a real-time system.

*Implementation Requirements*

- 2 If the implementation blocks interrupts (see C.3) not as a result of direct user action (e.g. an execution of a protected action) there shall be an upper bound on the duration of this blocking.

The implementation shall recognize entry-less protected types. The overhead of acquiring the execution resource of an object of such a type (see 9.5.1) shall be minimized. In particular, there should not be any overhead due to evaluating `entry_barrier` conditions.

Unchecked\_Deallocation shall be supported for terminated tasks that are designated by access types, and shall have the effect of releasing all the storage associated with the task. This includes any run-time system or heap storage that has been implicitly allocated for the task by the implementation.

#### Documentation Requirements

The implementation shall document the upper bound on the duration of interrupt blocking caused by the implementation. If this is different for different interrupts or interrupt priority levels, it should be documented for each case.

#### Metrics

The implementation shall document the following metric:

- The overhead associated with obtaining a mutual-exclusive access to an entry-less protected object. This shall be measured in the following way:

For a protected object of the form:

```
protected Lock is
 procedure Set;
 function Read return Boolean;
private
 Flag : Boolean := False;
end Lock;

protected body Lock is
 procedure Set is
 begin
 Flag := True;
 end Set;
 function Read return Boolean
 begin
 return Flag;
 end Read;
end Lock;
```

The execution time, in processor clock cycles, of a call to `Set`. This shall be measured between the point just before issuing the call, and the point just after the call completes. The function `Read` shall be called later to verify that `Set` was indeed called (and not optimized away). The calling task shall have sufficiently high priority as to not be preempted during the measurement period. The protected object shall have sufficiently high ceiling priority to allow the task to call `Set`.

For a multiprocessor, if supported, the metric shall be reported for the case where no contention (on the execution resource) exists from tasks executing on other processors.

## D.13 Run-time Profiles

This clause specifies a mechanism for defining run-time profiles.

#### Syntax

The form of a `pragma Profile` is as follows:

```
pragma Profile (profile_identifier {, profile_pragma_argument_association});
```

*Legality Rules*

- 4/2 The *profile\_identifier* shall be the name of a run-time profile. The semantics of any *profile\_pragma\_argument\_associations* are defined by the run-time profile specified by the *profile\_identifier*.

*Static Semantics*

- 5/2 A profile is equivalent to the set of configuration pragmas that is defined for each run-time profile.

*Post-Compilation Rules*

- 6/2 A pragma Profile is a configuration pragma. There may be more than one pragma Profile for a partition.



## D.13.1 The Ravenscar Profile

This clause defines the Ravenscar profile.

1/2

### *Legality Rules*

The *profile identifier* Ravenscar is a run-time profile. For run-time profile Ravenscar, there shall be no *profile pragma\_argument\_associations*.

2/2

### *Static Semantics*

The run-time profile Ravenscar is equivalent to the following set of pragmas:

3/2

```

pragma Task_Dispatching_Policy (FIFO_Within_Priorities);
pragma Locking_Policy (Ceiling_Locking);
pragma Detect_Blocking;
pragma Restrictions (
 No_Abort_Statements,
 No_Dynamic_Attachment,
 No_Dynamic_Priorities,
 No_Implicit_Heap_Allocations,
 No_Local_Protected_Objects,
 No_Local_Timing_Events,
 No_Protected_Type_Allocators,
 No_Relative_Delay,
 No_Requeue_Statements,
 No_Select_Statements,
 No_Specific_Termination_Handlers,
 No_Task_Allocators,
 No_Task_Hierarchy,
 No_Task_Termination,
 Simple_Barriers,
 Max_Entry_Queue_Length => 1,
 Max_Protected_Entries => 1,
 Max_Task_Entries => 0,
 No_Dependence => Ada.Asynchronous_Task_Control,
 No_Dependence => Ada.Calendar,
 No_Dependence => Ada.Execution_Time.Group_Budget,
 No_Dependence => Ada.Execution_Time.Timers,
 No_Dependence => Ada.Task_Attributes);

```

4/2

### NOTES

42 The effect of the Max\_Entry\_Queue\_Length => 1 restriction applies only to protected entry queues due to the accompanying restriction of Max\_Task\_Entries => 0.

5/2

## D.14 Execution Time

1/2 This clause describes a language-defined package to measure execution time.

### Static Semantics

2/2 The following language-defined library package exists:

```

3/2 with Ada.Task_Identification;
 with Ada.Real_Time; use Ada.Real_Time;
 package Ada.Execution_Time is
4/2 type CPU_Time is private;
 CPU_Time_First : constant CPU_Time;
 CPU_Time_Last : constant CPU_Time;
 CPU_Time_Unit : constant := implementation-defined-real-number;
 CPU_Tick : constant Time_Span;
5/2 function Clock
 (T : Ada.Task_Identification.Task_Id
 := Ada.Task_Identification.Current_Task)
 return CPU_Time;
6/2 function "+" (Left : CPU_Time; Right : Time_Span) return CPU_Time;
 function "+" (Left : Time_Span; Right : CPU_Time) return CPU_Time;
 function "-" (Left : CPU_Time; Right : Time_Span) return CPU_Time;
 function "-" (Left : CPU_Time; Right : CPU_Time) return Time_Span;
7/2 function "<" (Left, Right : CPU_Time) return Boolean;
 function "<=" (Left, Right : CPU_Time) return Boolean;
 function ">" (Left, Right : CPU_Time) return Boolean;
 function ">=" (Left, Right : CPU_Time) return Boolean;
8/2 procedure Split
 (T : in CPU_Time; SC : out Seconds_Count; TS : out Time_Span);
9/2 function Time_Of (SC : Seconds_Count;
 TS : Time_Span := Time_Span_Zero) return CPU_Time;
10/2 private
 ... -- not specified by the language
 end Ada.Execution_Time;
```

11/2 The *execution time* or CPU time of a given task is defined as the time spent by the system executing that task, including the time spent executing run-time or system services on its behalf. The mechanism used to measure execution time is implementation defined. It is implementation defined which task, if any, is charged the execution time that is consumed by interrupt handlers and run-time services on behalf of the system.

12/2 The type CPU\_Time represents the execution time of a task. The set of values of this type corresponds one-to-one with an implementation-defined range of mathematical integers.

13/2 The CPU\_Time value I represents the half-open execution-time interval that starts with I\*CPU\_Time\_Unit and is limited by (I+1)\*CPU\_Time\_Unit, where CPU\_Time\_Unit is an implementation-defined real number. For each task, the execution time value is set to zero at the creation of the task.

14/2 CPU\_Time\_First and CPU\_Time\_Last are the smallest and largest values of the CPU\_Time type, respectively.

### Dynamic Semantics

15/2 CPU\_Time\_Unit is the smallest amount of execution time representable by the CPU\_Time type; it is expressed in seconds. A *CPU clock tick* is an execution time interval during which the clock value (as

observed by calling the Clock function) remains constant. CPU\_Tick is the average length of such intervals.

The effects of the operators on CPU\_Time and Time\_Span are as for the operators defined for integer types. 16/2

The function Clock returns the current execution time of the task identified by T; Tasking\_Error is raised if that task has terminated; Program\_Error is raised if the value of T is Task\_Identification.Null\_Task\_Id. 17/2

The effects of the Split and Time\_Of operations are defined as follows, treating values of type CPU\_Time, Time\_Span, and Seconds\_Count as mathematical integers. The effect of Split (T, SC, TS) is to set SC and TS to values such that  $T * \text{CPU\_Time\_Unit} = \text{SC} * 1.0 + \text{TS} * \text{CPU\_Time\_Unit}$ , and  $0.0 \leq \text{TS} * \text{CPU\_Time\_Unit} < 1.0$ . The value returned by Time\_Of(SC,TS) is the execution-time value T such that  $T * \text{CPU\_Time\_Unit} = \text{SC} * 1.0 + \text{TS} * \text{CPU\_Time\_Unit}$ . 18/2

#### *Erroneous Execution*

For a call of Clock, if the task identified by T no longer exists, the execution of the program is erroneous. 19/2

#### *Implementation Requirements*

The range of CPU\_Time values shall be sufficient to uniquely represent the range of execution times from the task start-up to 50 years of execution time later. CPU\_Tick shall be no greater than 1 millisecond. 20/2

#### *Documentation Requirements*

The implementation shall document the values of CPU\_Time\_First, CPU\_Time\_Last, CPU\_Time\_Unit, and CPU\_Tick. 21/2

The implementation shall document the properties of the underlying mechanism used to measure execution times, such as the range of values supported and any relevant aspects of the underlying hardware or operating system facilities used. 22/2

#### *Metrics*

The implementation shall document the following metrics: 23/2

- An upper bound on the execution-time duration of a clock tick. This is a value D such that if t1 and t2 are any execution times of a given task such that  $t1 < t2$  and  $\text{Clock}_{t1} = \text{Clock}_{t2}$  then  $t2 - t1 \leq D$ . 24/2
- An upper bound on the size of a clock jump. A clock jump is the difference between two successive distinct values of an execution-time clock (as observed by calling the Clock function with the same Task\_Id). 25/2
- An upper bound on the execution time of a call to the Clock function, in processor clock cycles. 26/2
- Upper bounds on the execution times of the operators of the type CPU\_Time, in processor clock cycles. 27/2

#### *Implementation Permissions*

Implementations targeted to machines with word size smaller than 32 bits need not support the full range and granularity of the CPU\_Time type. 28/2

#### *Implementation Advice*

When appropriate, implementations should provide configuration mechanisms to change the value of CPU\_Tick. 29/2

## D.14.1 Execution Time Timers

- 1/2 This clause describes a language-defined package that provides a facility for calling a handler when a task has used a defined amount of CPU time.

### Static Semantics

- 2/2 The following language-defined library package exists:

```

3/2 with System;
 package Ada.Execution_Time.Timers is
4/2 type Timer (T : not null access constant
 Ada.Task_Identification.Task_Id) is
 tagged limited private;
5/2 type Timer_Handler is
 access protected procedure (TM : in out Timer);
6/2 Min_Handler_Ceiling : constant System.Any_Priority :=
 implementation-defined;
7/2 procedure Set_Handler (TM : in out Timer;
 In_Time : in Time_Span;
 Handler : in Timer_Handler);
 procedure Set_Handler (TM : in out Timer;
 At_Time : in CPU_Time;
 Handler : in Timer_Handler);
 function Current_Handler (TM : Timer) return Timer_Handler;
 procedure Cancel_Handler (TM : in out Timer;
 Cancelled : out Boolean);
8/2 function Time_Remaining (TM : Timer) return Time_Span;
9/2 Timer_Resource_Error : exception;
10/2 private
 ... -- not specified by the language
 end Ada.Execution_Time.Timers;
```

- 11/2 The type `Timer` represents an execution-time event for a single task and is capable of detecting execution-time overruns. The access discriminant `T` identifies the task concerned. The type `Timer` needs finalization (see 7.6).

- 12/2 An object of type `Timer` is said to be *set* if it is associated with a non-null value of type `Timer_Handler` and *cleared* otherwise. All `Timer` objects are initially cleared.

- 13/2 The type `Timer_Handler` identifies a protected procedure to be executed by the implementation when the timer expires. Such a protected procedure is called a *handler*.

### Dynamic Semantics

- 14/2 When a `Timer` object is created, or upon the first call of a `Set_Handler` procedure with the timer as parameter, the resources required to operate an execution-time timer based on the associated execution-time clock are allocated and initialized. If this operation would exceed the available resources, `Timer_Resource_Error` is raised.

- 15/2 The procedures `Set_Handler` associate the handler `Handler` with the timer `TM`; if `Handler` is **null**, the timer is cleared, otherwise it is set. The first procedure `Set_Handler` loads the timer `TM` with an interval specified by the `Time_Span` parameter. In this mode, the timer `TM` *expires* when the execution time of the task identified by `TM.T.all` has increased by `In_Time`; if `In_Time` is less than or equal to zero, the timer expires immediately. The second procedure `Set_Handler` loads the timer `TM` with the absolute value specified by `At_Time`. In this mode, the timer `TM` expires when the execution time of the task identified

|                                                                                                                                                                                                                       |      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| by TM.T.all reaches At_Time; if the value of At_Time has already been reached when Set_Handler is called, the timer expires immediately.                                                                              |      |
| A call of a procedure Set_Handler for a timer that is already set replaces the handler and the (absolute or relative) execution time; if Handler is not <b>null</b> , the timer remains set.                          | 16/2 |
| When a timer expires, the associated handler is executed, passing the timer as parameter. The initial action of the execution of the handler is to clear the event.                                                   | 17/2 |
| The function Current_Handler returns the handler associated with the timer TM if that timer is set; otherwise it returns <b>null</b> .                                                                                | 18/2 |
| The procedure Cancel_Handler clears the timer if it is set. Cancelled is assigned True if the timer was set prior to it being cleared; otherwise it is assigned False.                                                | 19/2 |
| The function Time_Remaining returns the execution time interval that remains until the timer TM would expire, if that timer is set; otherwise it returns Time_Span_Zero.                                              | 20/2 |
| The constant Min_Handler_Ceiling is the minimum ceiling priority required for a protected object with a handler to ensure that no ceiling violation will occur when that handler is invoked.                          | 21/2 |
| As part of the finalization of an object of type Timer, the timer is cleared.                                                                                                                                         | 22/2 |
| For all the subprograms defined in this package, Tasking_Error is raised if the task identified by TM.T.all has terminated, and Program_Error is raised if the value of TM.T.all is Task_Identification.Null_Task_Id. | 23/2 |
| An exception propagated from a handler invoked as part of the expiration of a timer has no effect.                                                                                                                    | 24/2 |

*Erroneous Execution*

|                                                                                                                                                               |      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| For a call of any of the subprograms defined in this package, if the task identified by TM.T.all no longer exists, the execution of the program is erroneous. | 25/2 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|------|

*Implementation Requirements*

|                                                                                                                                                                                                                                                                                                                   |      |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| For a given Timer object, the implementation shall perform the operations declared in this package atomically with respect to any of these operations on the same Timer object. The replacement of a handler by a call of Set_Handler shall be performed atomically with respect to the execution of the handler. | 26/2 |
| When an object of type Timer is finalized, the system resources used by the timer shall be deallocated.                                                                                                                                                                                                           | 27/2 |

*Implementation Permissions*

|                                                                                                                                                  |      |
|--------------------------------------------------------------------------------------------------------------------------------------------------|------|
| Implementations may limit the number of timers that can be defined for each task. If this limit is exceeded then Timer_Resource_Error is raised. | 28/2 |
|--------------------------------------------------------------------------------------------------------------------------------------------------|------|

## NOTES

|                                                                  |      |
|------------------------------------------------------------------|------|
| 43 A Timer_Handler can be associated with several Timer objects. | 29/2 |
|------------------------------------------------------------------|------|

## D.14.2 Group Execution Time Budgets

1/2 This clause describes a language-defined package to assign execution time budgets to groups of tasks.

### Static Semantics

2/2 The following language-defined library package exists:

```

3/2 with System;
 package Ada.Execution_Time.Group_Budgets is
4/2 type Group_Budget is tagged limited private;
5/2 type Group_Budget_Handler is access
 protected procedure (GB : in out Group_Budget);
6/2 type Task_Array is array (Positive range <>) of
 Ada.Task_Identification.Task_Id;
7/2 Min_Handler_Ceiling : constant System.Any_Priority :=
 implementation-defined;
8/2 procedure Add_Task (GB : in out Group_Budget;
 T : in Ada.Task_Identification.Task_Id);
 procedure Remove_Task (GB: in out Group_Budget;
 T : in Ada.Task_Identification.Task_Id);
 function Is_Member (GB : Group_Budget;
 T : Ada.Task_Identification.Task_Id) return Boolean;
 function Is_A_Group_Member
 (T : Ada.Task_Identification.Task_Id) return Boolean;
 function Members (GB : Group_Budget) return Task_Array;
9/2 procedure Replenish (GB : in out Group_Budget; To : in Time_Span);
 procedure Add (GB : in out Group_Budget; Interval : in Time_Span);
 function Budget_Has_Expired (GB : Group_Budget) return Boolean;
 function Budget_Remaining (GB : Group_Budget) return Time_Span;
10/2 procedure Set_Handler (GB : in out Group_Budget;
 Handler : in Group_Budget_Handler);
 function Current_Handler (GB : Group_Budget)
 return Group_Budget_Handler;
 procedure Cancel_Handler (GB : in out Group_Budget;
 Cancelled : out Boolean);
11/2 Group_Budget_Error : exception;
12/2 private
 -- not specified by the language
 end Ada.Execution_Time.Group_Budgets;

```

13/2 The type `Group_Budget` represents an execution time budget to be used by a group of tasks. The type `Group_Budget` needs finalization (see 7.6). A task can belong to at most one group. Tasks of any priority can be added to a group.

14/2 An object of type `Group_Budget` has an associated nonnegative value of type `Time_Span` known as its *budget*, which is initially `Time_Span_Zero`. The type `Group_Budget_Handler` identifies a protected procedure to be executed by the implementation when the budget is *exhausted*, that is, reaches zero. Such a protected procedure is called a *handler*.

15/2 An object of type `Group_Budget` also includes a handler, which is a value of type `Group_Budget_Handler`. The handler of the object is said to be *set* if it is not null and *cleared* otherwise. The handler of all `Group_Budget` objects is initially cleared.

### Dynamic Semantics

16/2 The procedure `Add_Task` adds the task identified by `T` to the group `GB`; if that task is already a member of some other group, `Group_Budget_Error` is raised.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| The procedure <code>Remove_Task</code> removes the task identified by <code>T</code> from the group <code>GB</code> ; if that task is not a member of the group <code>GB</code> , <code>Group_Budget_Error</code> is raised. After successful execution of this procedure, the task is no longer a member of any group.                                                                                                                                                                                                                                                                      | 17/2 |
| The function <code>Is_Member</code> returns <code>True</code> if the task identified by <code>T</code> is a member of the group <code>GB</code> ; otherwise it return <code>False</code> .                                                                                                                                                                                                                                                                                                                                                                                                   | 18/2 |
| The function <code>Is_A_Group_Member</code> returns <code>True</code> if the task identified by <code>T</code> is a member of some group; otherwise it returns <code>False</code> .                                                                                                                                                                                                                                                                                                                                                                                                          | 19/2 |
| The function <code>Members</code> returns an array of values of type <code>Task_Identification.Task_Id</code> identifying the members of the group <code>GB</code> . The order of the components of the array is unspecified.                                                                                                                                                                                                                                                                                                                                                                | 20/2 |
| The procedure <code>Replenish</code> loads the group budget <code>GB</code> with <code>To</code> as the <code>Time_Span</code> value. The exception <code>Group_Budget_Error</code> is raised if the <code>Time_Span</code> value <code>To</code> is non-positive. Any execution of any member of the group of tasks results in the budget counting down, unless exhausted. When the budget becomes exhausted (reaches <code>Time_Span_Zero</code> ), the associated handler is executed if the handler of group budget <code>GB</code> is set. Nevertheless, the tasks continue to execute. | 21/2 |
| The procedure <code>Add</code> modifies the budget of the group <code>GB</code> . A positive value for <code>Interval</code> increases the budget. A negative value for <code>Interval</code> reduces the budget, but never below <code>Time_Span_Zero</code> . A zero value for <code>Interval</code> has no effect. A call of procedure <code>Add</code> that results in the value of the budget going to <code>Time_Span_Zero</code> causes the associated handler to be executed if the handler of the group budget <code>GB</code> is set.                                              | 22/2 |
| The function <code>Budget_Has_Expired</code> returns <code>True</code> if the budget of group <code>GB</code> is exhausted (equal to <code>Time_Span_Zero</code> ); otherwise it returns <code>False</code> .                                                                                                                                                                                                                                                                                                                                                                                | 23/2 |
| The function <code>Budget_Remaining</code> returns the remaining budget for the group <code>GB</code> . If the budget is exhausted it returns <code>Time_Span_Zero</code> . This is the minimum value for a budget.                                                                                                                                                                                                                                                                                                                                                                          | 24/2 |
| The procedure <code>Set_Handler</code> associates the handler <code>Handler</code> with the <code>Group_Budget</code> <code>GB</code> ; if <code>Handler</code> is <b>null</b> , the handler of <code>Group_Budget</code> is cleared, otherwise it is set.                                                                                                                                                                                                                                                                                                                                   | 25/2 |
| A call of <code>Set_Handler</code> for a <code>Group_Budget</code> that already has a handler set replaces the handler; if <code>Handler</code> is not <b>null</b> , the handler for <code>Group_Budget</code> remains set.                                                                                                                                                                                                                                                                                                                                                                  | 26/2 |
| The function <code>Current_Handler</code> returns the handler associated with the group budget <code>GB</code> if the handler for that group budget is set; otherwise it returns <b>null</b> .                                                                                                                                                                                                                                                                                                                                                                                               | 27/2 |
| The procedure <code>Cancel_Handler</code> clears the handler for the group budget if it is set. <code>Cancelled</code> is assigned <code>True</code> if the handler for the group budget was set prior to it being cleared; otherwise it is assigned <code>False</code> .                                                                                                                                                                                                                                                                                                                    | 28/2 |
| The constant <code>Min_Handler_Ceiling</code> is the minimum ceiling priority required for a protected object with a handler to ensure that no ceiling violation will occur when that handler is invoked.                                                                                                                                                                                                                                                                                                                                                                                    | 29/2 |
| The precision of the accounting of task execution time to a <code>Group_Budget</code> is the same as that defined for execution-time clocks from the parent package.                                                                                                                                                                                                                                                                                                                                                                                                                         | 30/2 |
| As part of the finalization of an object of type <code>Group_Budget</code> all member tasks are removed from the group identified by that object.                                                                                                                                                                                                                                                                                                                                                                                                                                            | 31/2 |
| If a task is a member of a <code>Group_Budget</code> when it terminates then as part of the finalization of the task it is removed from the group.                                                                                                                                                                                                                                                                                                                                                                                                                                           | 32/2 |
| For all the operations defined in this package, <code>Tasking_Error</code> is raised if the task identified by <code>T</code> has terminated, and <code>Program_Error</code> is raised if the value of <code>T</code> is <code>Task_Identification.Null_Task_Id</code> .                                                                                                                                                                                                                                                                                                                     | 33/2 |

- 34/2 An exception propagated from a handler invoked when the budget of a group of tasks becomes exhausted has no effect.

*Erroneous Execution*

- 35/2 For a call of any of the subprograms defined in this package, if the task identified by T no longer exists, the execution of the program is erroneous.

*Implementation Requirements*

- 36/2 For a given Group\_Budget object, the implementation shall perform the operations declared in this package atomically with respect to any of these operations on the same Group\_Budget object. The replacement of a handler, by a call of Set\_Handler, shall be performed atomically with respect to the execution of the handler.

NOTES

- 37/2 44 Clearing or setting of the handler of a group budget does not change the current value of the budget. Exhaustion or loading of a budget does not change whether the handler of the group budget is set or cleared.
- 38/2 45 A Group\_Budget\_Handler can be associated with several Group\_Budget objects.

## D.15 Timing Events

- 1/2 This clause describes a language-defined package to allow user-defined protected procedures to be executed at a specified time without the need for a task or a delay statement.

*Static Semantics*

- 2/2 The following language-defined library package exists:

```

3/2 package Ada.Real_Time.Timing_Events is
4/2 type Timing_Event is tagged limited private;
 type Timing_Event_Handler
 is access protected procedure (Event : in out Timing_Event);
5/2 procedure Set_Handler (Event : in out Timing_Event;
 At_Time : in Time;
 Handler : in Timing_Event_Handler);
 procedure Set_Handler (Event : in out Timing_Event;
 In_Time : in Time_Span;
 Handler : in Timing_Event_Handler);
 function Current_Handler (Event : Timing_Event)
 return Timing_Event_Handler;
 procedure Cancel_Handler (Event : in out Timing_Event;
 Cancelled : out Boolean);
6/2 function Time_Of_Event (Event : Timing_Event) return Time;
7/2 private
 ... -- not specified by the language
 end Ada.Real_Time.Timing_Events;
```

- 8/2 The type Timing\_Event represents a time in the future when an event is to occur. The type Timing\_Event needs finalization 7.6).
- 9/2 An object of type Timing\_Event is said to be *set* if it is associated with a non-null value of type Timing\_Event\_Handler and *cleared* otherwise. All Timing\_Event objects are initially cleared.
- 10/2 The type Timing\_Event\_Handler identifies a protected procedure to be executed by the implementation when the timing event occurs. Such a protected procedure is called a *handler*.



*Dynamic Semantics*

The procedures `Set_Handler` associate the handler `Handler` with the event `Event`; if `Handler` is **null**, the event is cleared, otherwise it is set. The first procedure `Set_Handler` sets the execution time for the event to be `At_Time`. The second procedure `Set_Handler` sets the execution time for the event to be `Real_Time.Clock + In_Time`. 11/2

A call of a procedure `Set_Handler` for an event that is already set replaces the handler and the time of execution; if `Handler` is not **null**, the event remains set. 12/2

As soon as possible after the time set for the event, the handler is executed, passing the event as parameter. The handler is only executed if the timing event is in the set state at the time of execution. The initial action of the execution of the handler is to clear the event. 13/2

If the `Ceiling_Locking` policy (see D.3) is in effect when a procedure `Set_Handler` is called, a check is made that the ceiling priority of `Handler.all` is `Interrupt_Priority'Last`. If the check fails, `Program_Error` is raised. 14/2

If a procedure `Set_Handler` is called with zero or negative `In_Time` or with `At_Time` indicating a time in the past then the handler is executed as soon as possible after the completion of the call of `Set_Handler`. 15/3

The function `Current_Handler` returns the handler associated with the event `Event` if that event is set; otherwise it returns **null**. 16/2

The procedure `Cancel_Handler` clears the event if it is set. `Cancelled` is assigned `True` if the event was set prior to it being cleared; otherwise it is assigned `False`. 17/2

The function `Time_Of_Event` returns the time of the event if the event is set; otherwise it returns `Real_Time.Time_First`. 18/2

As part of the finalization of an object of type `Timing_Event`, the `Timing_Event` is cleared. 19/2

If several timing events are set for the same time, they are executed in FIFO order of being set. 20/2

An exception propagated from a handler invoked by a timing event has no effect. 21/2

*Implementation Requirements*

For a given `Timing_Event` object, the implementation shall perform the operations declared in this package atomically with respect to any of these operations on the same `Timing_Event` object. The replacement of a handler by a call of `Set_Handler` shall be performed atomically with respect to the execution of the handler. 22/2

*Metrics*

The implementation shall document the following metric: 23/2

- An upper bound on the lateness of the execution of a handler. That is, the maximum time between when a handler is actually executed and the time specified when the event was set. 24/2

*Implementation Advice*

The protected handler procedure should be executed directly by the real-time clock interrupt mechanism. 25/2

## NOTES

46 Since a call of `Set_Handler` is not a potentially blocking operation, it can be called from within a handler. 26/2

47 A `Timing_Event_Handler` can be associated with several `Timing_Event` objects. 27/2



## Annex E

### (normative)

## Distributed Systems

This Annex defines facilities for supporting the implementation of distributed systems using multiple partitions working cooperatively as part of a single Ada program. 1

#### *Post-Compilation Rules*

A *distributed system* is an interconnection of one or more *processing nodes* (a system resource that has both computational and storage capabilities), and zero or more *storage nodes* (a system resource that has only storage capabilities, with the storage addressable by one or more processing nodes). 2

A *distributed program* comprises one or more partitions that execute independently (except when they communicate) in a distributed system. 3

The process of mapping the partitions of a program to the nodes in a distributed system is called *configuring the partitions of the program*. 4

#### *Implementation Requirements*

The implementation shall provide means for explicitly assigning library units to a partition and for the configuring and execution of a program consisting of multiple partitions on a distributed system; the means are implementation defined. 5

#### *Implementation Permissions*

An implementation may require that the set of processing nodes of a distributed system be homogeneous. 6

#### NOTES

1 The partitions comprising a program may be executed on differently configured distributed systems or on a non-distributed system without requiring recompilation. A distributed program may be partitioned differently from the same set of library units without recompilation. The resulting execution is semantically equivalent. 7

2 A distributed program retains the same type safety as the equivalent single partition program. 8

## E.1 Partitions

The partitions of a distributed program are classified as either active or passive. 1

#### *Post-Compilation Rules*

An *active partition* is a partition as defined in 10.2. A *passive partition* is a partition that has no thread of control of its own, whose library units are all preelaborated, and whose data and subprograms are accessible to one or more active partitions. 2

A passive partition shall include only `library_items` that either are declared pure or are shared passive (see 10.2.1 and E.2.1). 3

An active partition shall be configured on a processing node. A passive partition shall be configured either on a storage node or on a processing node. 4

The configuration of the partitions of a program onto a distributed system shall be consistent with the possibility for data references or calls between the partitions implied by their semantic dependences. Any reference to data or call of a subprogram across partitions is called a *remote access*. 5

*Dynamic Semantics*

- 6 A `library_item` is elaborated as part of the elaboration of each partition that includes it. If a normal library unit (see E.2) has state, then a separate copy of the state exists in each active partition that elaborates it. The state evolves independently in each such partition.
- 7 An active partition *terminates* when its environment task terminates. A partition becomes *inaccessible* if it terminates or if it is *aborted*. An active partition is aborted when its environment task is aborted. In addition, if a partition fails during its elaboration, it becomes inaccessible to other partitions. Other implementation-defined events can also result in a partition becoming inaccessible.
- 8/1 For a prefix `D` that denotes a library-level declaration, excepting a declaration of or within a declared-pure library unit, the following attribute is defined:
- 9 `D'Partition_Id`  
 Denotes a value of the type *universal\_integer* that identifies the partition in which `D` was elaborated. If `D` denotes the declaration of a remote call interface library unit (see E.2.3) the given partition is the one where the body of `D` was elaborated.

*Bounded (Run-Time) Errors*

- 10/2 It is a bounded error for there to be cyclic elaboration dependences between the active partitions of a single distributed program. The possible effects, in each of the partitions involved, are deadlock during elaboration, or the raising of `Communication_Error` or `Program_Error`.

*Implementation Permissions*

- 11 An implementation may allow multiple active or passive partitions to be configured on a single processing node, and multiple passive partitions to be configured on a single storage node. In these cases, the scheduling policies, treatment of priorities, and management of shared resources between these partitions are implementation defined.
- 12 An implementation may allow separate copies of an active partition to be configured on different processing nodes, and to provide appropriate interactions between the copies to present a consistent state of the partition to other active partitions.
- 13 In an implementation, the partitions of a distributed program need not be loaded and elaborated all at the same time; they may be loaded and elaborated one at a time over an extended period of time. An implementation may provide facilities to abort and reload a partition during the execution of a distributed program.
- 14 An implementation may allow the state of some of the partitions of a distributed program to persist while other partitions of the program terminate and are later reinvoked.

## NOTES

- 15 3 Library units are grouped into partitions after compile time, but before run time. At compile time, only the relevant library unit properties are identified using categorization pragmas.
- 16 4 The value returned by the `Partition_Id` attribute can be used as a parameter to implementation-provided subprograms in order to query information about the partition.

## E.2 Categorization of Library Units

- 1 Library units can be categorized according to the role they play in a distributed program. Certain restrictions are associated with each category to ensure that the semantics of a distributed program remain close to the semantics for a nondistributed program.

A *categorization pragma* is a library unit pragma (see 10.1.5) that restricts the declarations, child units, or semantic dependences of the library unit to which it applies. A *categorized library unit* is a library unit to which a categorization pragma applies. 2

The pragmas `Shared_Passive`, `Remote_Types`, and `Remote_Call_Interface` are categorization pragmas. In addition, for the purposes of this Annex, the pragma `Pure` (see 10.2.1) is considered a categorization pragma. 3

A library package or generic library package is called a *shared passive* library unit if a `Shared_Passive` pragma applies to it. A library package or generic library package is called a *remote types* library unit if a `Remote_Types` pragma applies to it. A library unit is called a *remote call interface* if a `Remote_Call_Interface` pragma applies to it. A *normal library unit* is one to which no categorization pragma applies. 4/1

The various categories of library units and the associated restrictions are described in this clause and its subclauses. The categories are related hierarchically in that the library units of one category can depend semantically only on library units of that category or an earlier one, except that the body of a remote types or remote call interface library unit is unrestricted. 5

The overall hierarchy (including declared pure) is as follows: 6

Declared Pure 7

Can depend only on other declared pure library units;

Shared Passive 8

Can depend only on other shared passive or declared pure library units;

Remote Types 9

The declaration of the library unit can depend only on other remote types library units, or one of the above; the body of the library unit is unrestricted;

Remote Call Interface 10

The declaration of the library unit can depend only on other remote call interfaces, or one of the above; the body of the library unit is unrestricted;

Normal Unrestricted. 11

Declared pure and shared passive library units are preelaborated. The declaration of a remote types or remote call interface library unit is required to be preelaborable. 12

#### *Implementation Requirements*

*This paragraph was deleted.* 13/1

#### *Implementation Permissions*

Implementations are allowed to define other categorization pragmas. 14

## E.2.1 Shared Passive Library Units

A shared passive library unit is used for managing global data shared between active partitions. The restrictions on shared passive library units prevent the data or tasks of one active partition from being accessible to another active partition through references implicit in objects declared in the shared passive library unit. 1

#### *Syntax*

The form of a pragma `Shared_Passive` is as follows: 2

**pragma** Shared\_Passive[(*library\_unit\_name*)];

*Legality Rules*

A *shared passive library unit* is a library unit to which a Shared\_Passive pragma applies. The following restrictions apply to such a library unit:

- it shall be preelaborable (see 10.2.1);
- it shall depend semantically only upon declared pure or shared passive library units;
- it shall not contain a library-level declaration of an access type that designates a class-wide type, task type, or protected type with *entry\_declarations*.

Notwithstanding the definition of accessibility given in 3.10.2, the declaration of a library unit P1 is not accessible from within the declarative region of a shared passive library unit P2, unless the shared passive library unit P2 depends semantically on P1.

*Static Semantics*

A shared passive library unit is preelaborated.

*Post-Compilation Rules*

A shared passive library unit shall be assigned to at most one partition within a given program.

Notwithstanding the rule given in 10.2, a compilation unit in a given partition does not *need* (in the sense of 10.2) the shared passive library units on which it depends semantically to be included in that same partition; they will typically reside in separate passive partitions.

## E.2.2 Remote Types Library Units

A remote types library unit supports the definition of types intended for use in communication between active partitions.

*Syntax*

The form of a **pragma** Remote\_Types is as follows:

**pragma** Remote\_Types[(*library\_unit\_name*)];

*Legality Rules*

A *remote types library unit* is a library unit to which the pragma Remote\_Types applies. The following restrictions apply to the declaration of such a library unit:

- it shall be preelaborable;
- it shall depend semantically only on declared pure, shared passive, or other remote types library units;
- it shall not contain the declaration of any variable within the visible part of the library unit;
- the full view of each type declared in the visible part of the library unit that has any available stream attributes shall support external streaming (see 13.13.2).

A named access type declared in the visible part of a remote types or remote call interface library unit is called a *remote access type*. Such a type shall be:

- an access-to-subprogram type, or

- a general access type that designates a class-wide limited private type, a class-wide limited interface type, or a class-wide private extension all of whose ancestors are either private extensions, limited interface types, or limited private types. 9.2/3

A type that is derived from a remote access type is also a remote access type. 9.3/1

The following restrictions apply to the use of a remote access-to-subprogram type: 10

- A value of a remote access-to-subprogram type shall be converted only to or from another (subtype-conformant) remote access-to-subprogram type; 11/2
- The **prefix** of an Access **attribute\_reference** that yields a value of a remote access-to-subprogram type shall statically denote a (subtype-conformant) remote subprogram. 12

The following restrictions apply to the use of a remote access-to-class-wide type: 13

- The primitive subprograms of the corresponding specific type shall only have access parameters if they are controlling formal parameters. The primitive functions of the corresponding specific type shall only have an access result if it is a controlling access result. Each non-controlling formal parameter and non-controlling result type shall support external streaming (see 13.13.2); 14/3
- The corresponding specific type shall not have a primitive procedure to which a **pragma Implemented** applies unless the **implementation\_kind** of the pragma is **By\_Any**; 14.1/3
- A value of a remote access-to-class-wide type shall be explicitly converted only to another remote access-to-class-wide type; 15
- A value of a remote access-to-class-wide type shall be dereferenced (or implicitly converted to an anonymous access type) only as part of a dispatching call where the value designates a controlling operand of the call (see E.4, “Remote Subprogram Calls”). 16/1
- A controlling access result value for a primitive function with any controlling operands of the corresponding specific type shall either be explicitly converted to a remote access-to-class-wide type or be part of a dispatching call where the value designates a controlling operand of the call. 16.1/3
- The **Storage\_Pool** attribute is not defined for a remote access-to-class-wide type; the expected type for an **allocator** shall not be a remote access-to-class-wide type. A remote access-to-class-wide type shall not be an actual parameter for a generic formal access type. The **Storage\_Size** attribute of a remote access-to-class-wide type yields 0; it is not allowed in an **attribute\_definition\_clause**. 17/2

#### NOTES

5 A remote types library unit need not be pure, and the types it defines may include levels of indirection implemented by using access types. User-specified Read and Write attributes (see 13.13.2) provide for sending values of such a type between active partitions, with Write marshalling the representation, and Read unmarshalling any levels of indirection. 18

6 The value of a remote access-to-class-wide limited interface can designate an object of a nonlimited type derived from the interface. 19/3

7 A remote access type may designate a class-wide synchronized, protected, or task interface type. 20/3

## E.2.3 Remote Call Interface Library Units

A remote call interface library unit can be used as an interface for remote procedure calls (RPCs) (or remote function calls) between active partitions. 1

#### *Syntax*

The form of a **pragma Remote\_Call\_Interface** is as follows: 2

**pragma Remote\_Call\_Interface**[(*library\_unit\_name*)]; 3

The form of a **pragma All\_Calls\_Remote** is as follows: 4

- 5       **pragma** All\_Calls\_Remote[(*library\_unit\_name*)];  
 6       A **pragma** All\_Calls\_Remote is a library unit pragma.

*Legality Rules*

- 7/1   A *remote call interface (RCI)* is a library unit to which the **pragma** Remote\_Call\_Interface applies. A subprogram declared in the visible part of such a library unit, or declared by such a library unit, is called a *remote subprogram*.
- 8     The declaration of an RCI library unit shall be preelaborable (see 10.2.1), and shall depend semantically only upon declared pure, shared passive, remote types, or other remote call interface library units.
- 9/1   In addition, the following restrictions apply to an RCI library unit:
- 10/1   • its visible part shall not contain the declaration of a variable;
- 11/1   • its visible part shall not contain the declaration of a limited type;
- 12/1   • its visible part shall not contain a nested **generic\_declaration**;
- 13/1   • it shall not be, nor shall its visible part contain, the declaration of a subprogram to which a **pragma** Inline applies;
- 14/3   • it shall not be, nor shall its visible part contain, a subprogram (or access-to-subprogram) declaration whose profile has a parameter or result of a type that does not support external streaming (see 13.13.2);
- 15     • any public child of the library unit shall be a remote call interface library unit.
- 16   If a **pragma** All\_Calls\_Remote applies to a library unit, the library unit shall be a remote call interface.

*Post-Compilation Rules*

- 17   A remote call interface library unit shall be assigned to at most one partition of a given program. A remote call interface library unit whose parent is also an RCI library unit shall be assigned only to the same partition as its parent.
- 18   Notwithstanding the rule given in 10.2, a compilation unit in a given partition that semantically depends on the declaration of an RCI library unit, *needs* (in the sense of 10.2) only the declaration of the RCI library unit, not the body, to be included in that same partition. Therefore, the body of an RCI library unit is included only in the partition to which the RCI library unit is explicitly assigned.

*Implementation Requirements*

- 19/1   If a **pragma** All\_Calls\_Remote applies to a given RCI library unit, then the implementation shall route any call to a subprogram of the RCI unit from outside the declarative region of the unit through the Partition Communication Subsystem (PCS); see E.5. Calls to such subprograms from within the declarative region of the unit are defined to be local and shall not go through the PCS.

*Implementation Permissions*

- 20   An implementation need not support the Remote\_Call\_Interface pragma nor the All\_Calls\_Remote pragma. Explicit message-based communication between active partitions can be supported as an alternative to RPC.

## E.3 Consistency of a Distributed System

- 1   This clause defines attributes and rules associated with verifying the consistency of a distributed program.



*Static Semantics*

|                                                                                                                                                                                                                                                                                                                                                            |       |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| For a prefix P that statically denotes a program unit, the following attributes are defined:                                                                                                                                                                                                                                                               | 2/1   |
| P'Version      Yields a value of the predefined type String that identifies the version of the compilation unit that contains the declaration of the program unit.                                                                                                                                                                                         | 3     |
| P'Body_Version      Yields a value of the predefined type String that identifies the version of the compilation unit that contains the body (but not any subunits) of the program unit.                                                                                                                                                                    | 4     |
| The <i>version</i> of a compilation unit changes whenever the compilation unit changes in a semantically significant way. This International Standard does not define the exact meaning of "semantically significant". It is unspecified whether there are other events (such as recompilation) that result in the version of a compilation unit changing. | 5/1   |
| If P is not a library unit, and P has no completion, then P'Body_Version returns the Body_Version of the innermost program unit enclosing the declaration of P. If P is a library unit, and P has no completion, then P'Body_Version returns a value that is different from Body_Version of any version of P that has a completion.                        | 5.1/1 |

*Bounded (Run-Time) Errors*

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| In a distributed program, a library unit is <i>consistent</i> if the same version of its declaration is used throughout. It is a bounded error to elaborate a partition of a distributed program that contains a compilation unit that depends on a different version of the declaration of a shared passive or RCI library unit than that included in the partition to which the shared passive or RCI library unit was assigned. As a result of this error, Program_Error can be raised in one or both partitions during elaboration; in any case, the partitions become inaccessible to one another. | 6 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|

**E.4 Remote Subprogram Calls**

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| A <i>remote subprogram call</i> is a subprogram call that invokes the execution of a subprogram in another partition. The partition that originates the remote subprogram call is the <i>calling partition</i> , and the partition that executes the corresponding subprogram body is the <i>called partition</i> . Some remote procedure calls are allowed to return prior to the completion of subprogram execution. These are called <i>asynchronous remote procedure calls</i> . | 1 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|

|                                                                                                                 |   |
|-----------------------------------------------------------------------------------------------------------------|---|
| There are three different ways of performing a remote subprogram call:                                          | 2 |
| • As a direct call on a (remote) subprogram explicitly declared in a remote call interface;                     | 3 |
| • As an indirect call through a value of a remote access-to-subprogram type;                                    | 4 |
| • As a dispatching call with a controlling operand designated by a value of a remote access-to-class-wide type. | 5 |

|                                                                                                                                                                                                                        |   |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| The first way of calling corresponds to a <i>static</i> binding between the calling and the called partition. The latter two ways correspond to a <i>dynamic</i> binding between the calling and the called partition. | 6 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|

|                                                                                                                                                                                   |     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| Remote types library units (see E.2.2) and remote call interface library units (see E.2.3) define the remote subprograms or remote access types used for remote subprogram calls. | 7/3 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|

*Legality Rules*

|                                                                                                                                                                            |   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| In a dispatching call with two or more controlling operands, if one controlling operand is designated by a value of a remote access-to-class-wide type, then all shall be. | 8 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|

*Dynamic Semantics*

- 9 For the execution of a remote subprogram call, subprogram parameters (and later the results, if any) are passed using a stream-oriented representation (see 13.13.1) which is suitable for transmission between partitions. This action is called *marshalling*. *Unmarshalling* is the reverse action of reconstructing the parameters or results from the stream-oriented representation. Marshalling is performed initially as part of the remote subprogram call in the calling partition; unmarshalling is done in the called partition. After the remote subprogram completes, marshalling is performed in the called partition, and finally unmarshalling is done in the calling partition.
- 10 A *calling stub* is the sequence of code that replaces the subprogram body of a remotely called subprogram in the calling partition. A *receiving stub* is the sequence of code (the “wrapper”) that receives a remote subprogram call on the called partition and invokes the appropriate subprogram body.
- 11 Remote subprogram calls are executed at most once, that is, if the subprogram call returns normally, then the called subprogram's body was executed exactly once.
- 12 The task executing a remote subprogram call blocks until the subprogram in the called partition returns, unless the call is asynchronous. For an asynchronous remote procedure call, the calling task can become ready before the procedure in the called partition returns.
- 13 If a construct containing a remote call is aborted, the remote subprogram call is *cancelled*. Whether the execution of the remote subprogram is immediately aborted as a result of the cancellation is implementation defined.
- 14 If a remote subprogram call is received by a called partition before the partition has completed its elaboration, the call is kept pending until the called partition completes its elaboration (unless the call is cancelled by the calling partition prior to that).
- 15 If an exception is propagated by a remotely called subprogram, and the call is not an asynchronous call, the corresponding exception is reraised at the point of the remote subprogram call. For an asynchronous call, if the remote procedure call returns prior to the completion of the remotely called subprogram, any exception is lost.
- 16 The exception `Communication_Error` (see E.5) is raised if a remote call cannot be completed due to difficulties in communicating with the called partition.
- 17 All forms of remote subprogram calls are potentially blocking operations (see 9.5.1).
- 18/1 In a remote subprogram call with a formal parameter of a class-wide type, a check is made that the tag of the actual parameter identifies a tagged type declared in a declared-pure or shared passive library unit, or in the visible part of a remote types or remote call interface library unit. `Program_Error` is raised if this check fails. In a remote function call which returns a class-wide type, the same check is made on the function result.
- 19 In a dispatching call with two or more controlling operands that are designated by values of a remote access-to-class-wide type, a check is made (in addition to the normal `Tag_Check` — see 11.5) that all the remote access-to-class-wide values originated from `Access attribute_references` that were evaluated by tasks of the same active partition. `Constraint_Error` is raised if this check fails.

*Implementation Requirements*

- 20 The implementation of remote subprogram calls shall conform to the PCS interface as defined by the specification of the language-defined package `System.RPC` (see E.5). The calling stub shall use the

Do\_RPC procedure unless the remote procedure call is asynchronous in which case Do\_APC shall be used. On the receiving side, the corresponding receiving stub shall be invoked by the RPC-receiver.

With respect to shared variables in shared passive library units, the execution of the corresponding subprogram body of a synchronous remote procedure call is considered to be part of the execution of the calling task. The execution of the corresponding subprogram body of an asynchronous remote procedure call proceeds in parallel with the calling task and does not signal the next action of the calling task (see 9.10).

#### NOTES

8 A given active partition can both make and receive remote subprogram calls. Thus, an active partition can act as both a client and a server.

9 If a given exception is propagated by a remote subprogram call, but the exception does not exist in the calling partition, the exception can be handled by an **others** choice or be propagated to and handled by a third partition.

## E.4.1 Pragma Asynchronous

This subclause introduces the pragma Asynchronous which allows a remote subprogram call to return prior to completion of the execution of the corresponding remote subprogram body.

#### *Syntax*

The form of a pragma Asynchronous is as follows:

**pragma Asynchronous**(local\_name);

#### *Legality Rules*

The local\_name of a pragma Asynchronous shall denote either:

- One or more remote procedures; the formal parameters of the procedure(s) shall all be of mode **in**;
- The first subtype of a remote access-to-procedure type; the formal parameters of the designated profile of the type shall all be of mode **in**;
- The first subtype of a remote access-to-class-wide type.

#### *Static Semantics*

A pragma Asynchronous is a representation pragma. When applied to a type, it specifies the type-related *asynchronous* aspect of the type.

#### *Dynamic Semantics*

A remote call is *asynchronous* if it is a call to a procedure, or a call through a value of an access-to-procedure type, to which a pragma Asynchronous applies. In addition, if a pragma Asynchronous applies to a remote access-to-class-wide type, then a dispatching call on a procedure with a controlling operand designated by a value of the type is asynchronous if the formal parameters of the procedure are all of mode **in**.

#### *Implementation Requirements*

Asynchronous remote procedure calls shall be implemented such that the corresponding body executes at most once as a result of the call.

## E.4.2 Example of Use of a Remote Access-to-Class-Wide Type

### Examples

1 *Example of using a remote access-to-class-wide type to achieve dynamic binding across active partitions:*

```

2 package Tapes is
 pragma Pure(Tapes);
 type Tape is abstract tagged limited private;
 -- Primitive dispatching operations where
 -- Tape is controlling operand
 procedure Copy (From, To : access Tape; Num_Recs : in Natural) is
abstract;
 procedure Rewind (T : access Tape) is abstract;
 -- More operations
private
 type Tape is ...
end Tapes;

3 with Tapes;
 package Name_Server is
 pragma Remote_Call_Interface;
 -- Dynamic binding to remote operations is achieved
 -- using the access-to-limited-class-wide type Tape_Ptr
 type Tape_Ptr is access all Tapes.Tape'Class;
 -- The following statically bound remote operations
 -- allow for a name-server capability in this example
 function Find (Name : String) return Tape_Ptr;
 procedure Register (Name : in String; T : in Tape_Ptr);
 procedure Remove (T : in Tape_Ptr);
 -- More operations
end Name_Server;

4 package Tape_Driver is
 -- Declarations are not shown, they are irrelevant here
end Tape_Driver;

5 with Tapes, Name_Server;
 package body Tape_Driver is
 type New_Tape is new Tapes.Tape with ...
 procedure Copy
 (From, To : access New_Tape; Num_Recs: in Natural) is
 begin
 . . .
 end Copy;
 procedure Rewind (T : access New_Tape) is
 begin
 . . .
 end Rewind;
 -- Objects remotely accessible through use
 -- of Name_Server operations
 Tape1, Tape2 : aliased New_Tape;
 begin
 Name_Server.Register ("NINE-TRACK", Tape1'Access);
 Name_Server.Register ("SEVEN-TRACK", Tape2'Access);
 end Tape_Driver;

6 with Tapes, Name_Server;
 -- Tape_Driver is not needed and thus not mentioned in the with_clause
 procedure Tape_Client is
 T1, T2 : Name_Server.Tape_Ptr;
 begin
 T1 := Name_Server.Find ("NINE-TRACK");
 T2 := Name_Server.Find ("SEVEN-TRACK");
 Tapes.Rewind (T1);
 Tapes.Rewind (T2);
 Tapes.Copy (T1, T2, 3);
 end Tape_Client;

```

*Notes on the example:*

*This paragraph was deleted.*

- The package Tapes provides the necessary declarations of the type and its primitive operations.
- Name\_Server is a remote call interface package and is elaborated in a separate active partition to provide the necessary naming services (such as Register and Find) to the entire distributed program through remote subprogram calls.
- Tape\_Driver is a normal package that is elaborated in a partition configured on the processing node that is connected to the tape device(s). The abstract operations are overridden to support the locally declared tape devices (Tape1, Tape2). The package is not visible to its clients, but it exports the tape devices (as remote objects) through the services of the Name\_Server. This allows for tape devices to be dynamically added, removed or replaced without requiring the modification of the clients' code.
- The Tape\_Client procedure references only declarations in the Tapes and Name\_Server packages. Before using a tape for the first time, it needs to query the Name\_Server for a system-wide identity for that tape. From then on, it can use that identity to access the tape device.
- Values of remote access type Tape\_Ptr include the necessary information to complete the remote dispatching operations that result from dereferencing the controlling operands T1 and T2.

## E.5 Partition Communication Subsystem

The *Partition Communication Subsystem* (PCS) provides facilities for supporting communication between the active partitions of a distributed program. The package System.RPC is a language-defined interface to the PCS.

### *Static Semantics*

The following language-defined library package exists:

```

with Ada.Streams; -- see 13.13.1
package System.RPC is
 type Partition_Id is range 0 .. implementation-defined;
 Communication_Error : exception;
 type Params_Stream_Type (
 Initial_Size : Ada.Streams.Stream_Element_Count) is new
 Ada.Streams.Root_Stream_Type with private;
 procedure Read(
 Stream : in out Params_Stream_Type;
 Item : out Ada.Streams.Stream_Element_Array;
 Last : out Ada.Streams.Stream_Element_Offset);
 procedure Write(
 Stream : in out Params_Stream_Type;
 Item : in Ada.Streams.Stream_Element_Array);
 -- Synchronous call
 procedure Do_RPC(
 Partition : in Partition_Id;
 Params : access Params_Stream_Type;
 Result : access Params_Stream_Type);
 -- Asynchronous call
 procedure Do_APC(
 Partition : in Partition_Id;
 Params : access Params_Stream_Type);

```

```

11 -- The handler for incoming RPCs
 type RPC_Receiver is access procedure (
 Params : access Params_Stream_Type;
 Result : access Params_Stream_Type);
12 procedure Establish_RPC_Receiver(
 Partition : in Partition_Id;
 Receiver : in RPC_Receiver);
13 private
 ... -- not specified by the language
 end System.RPC;

```

14 A value of the type Partition\_Id is used to identify a partition.

15 An object of the type Params\_Stream\_Type is used for identifying the particular remote subprogram that is being called, as well as marshalling and unmarshalling the parameters or result of a remote subprogram call, as part of sending them between partitions.

16 The Read and Write procedures override the corresponding abstract operations for the type Params\_Stream\_Type.

#### Dynamic Semantics

17 The Do\_RPC and Do\_APC procedures send a message to the active partition identified by the Partition parameter.

18 After sending the message, Do\_RPC blocks the calling task until a reply message comes back from the called partition or some error is detected by the underlying communication system in which case Communication\_Error is raised at the point of the call to Do\_RPC.

19 Do\_APC operates in the same way as Do\_RPC except that it is allowed to return immediately after sending the message.

20 Upon normal return, the stream designated by the Result parameter of Do\_RPC contains the reply message.

21 The procedure System.RPC.Establish\_RPC\_Receiver is called once, immediately after elaborating the library units of an active partition (that is, right after the *elaboration of the partition*) if the partition includes an RCI library unit, but prior to invoking the main subprogram, if any. The Partition parameter is the Partition\_Id of the active partition being elaborated. The Receiver parameter designates an implementation-provided procedure called the *RPC-receiver* which will handle all RPCs received by the partition from the PCS. Establish\_RPC\_Receiver saves a reference to the RPC-receiver; when a message is received at the called partition, the RPC-receiver is called with the Params stream containing the message. When the RPC-receiver returns, the contents of the stream designated by Result is placed in a message and sent back to the calling partition.

22 If a call on Do\_RPC is aborted, a cancellation message is sent to the called partition, to request that the execution of the remotely called subprogram be aborted.

23 The subprograms declared in System.RPC are potentially blocking operations.

#### Implementation Requirements

24 The implementation of the RPC-receiver shall be reentrant, thereby allowing concurrent calls on it from the PCS to service concurrent remote subprogram calls into the partition.

An implementation shall not restrict the replacement of the body of System.RPC. An implementation shall not restrict children of System.RPC. The related implementation permissions in the introduction to Annex A do not apply. 24.1/1

If the implementation of System.RPC is provided by the user, an implementation shall support remote subprogram calls as specified. 24.2/1

#### *Documentation Requirements*

The implementation of the PCS shall document whether the RPC-receiver is invoked from concurrent tasks. If there is an upper limit on the number of such tasks, this limit shall be documented as well, together with the mechanisms to configure it (if this is supported). 25

#### *Implementation Permissions*

The PCS is allowed to contain implementation-defined interfaces for explicit message passing, broadcasting, etc. Similarly, it is allowed to provide additional interfaces to query the state of some remote partition (given its partition ID) or of the PCS itself, to set timeouts and retry parameters, to get more detailed error status, etc. These additional interfaces should be provided in child packages of System.RPC. 26

A body for the package System.RPC need not be supplied by the implementation. 27

An alternative declaration is allowed for package System.RPC as long as it provides a set of operations that is substantially equivalent to the specification defined in this clause. 27.1/2

#### *Implementation Advice*

Whenever possible, the PCS on the called partition should allow for multiple tasks to call the RPC-receiver with different messages and should allow them to block until the corresponding subprogram body returns. 28

The Write operation on a stream of type Params\_Stream\_Type should raise Storage\_Error if it runs out of space trying to write the Item into the stream. 29

#### NOTES

10 The package System.RPC is not designed for direct calls by user programs. It is instead designed for use in the implementation of remote subprograms calls, being called by the calling stubs generated for a remote call interface library unit to initiate a remote call, and in turn calling back to an RPC-receiver that dispatches to the receiving stubs generated for the body of a remote call interface, to handle a remote call received from elsewhere. 30





## Annex F (normative) Information Systems

This Annex provides a set of facilities relevant to Information Systems programming. These fall into several categories:

- an attribute definition clause specifying `Machine_Radix` for a decimal subtype; 2
- the package `Decimal`, which declares a set of constants defining the implementation's capacity for decimal types, and a generic procedure for decimal division; and 3
- the child packages `Text_IO Editing`, `Wide_Text_IO Editing`, and `Wide_Wide_Text_IO Editing`, which support formatted and localized output of decimal data, based on “picture String” values. 4/2

See also: 3.5.9, “Fixed Point Types”; 3.5.10, “Operations of Fixed Point Types”; 4.6, “Type Conversions”; 13.3, “Operational and Representation Attributes”; A.10.9, “Input-Output for Real Types”; B.3, “Interfacing with C and C++”; B.4, “Interfacing with COBOL”; Annex G, “Numerics”. 5/2

The character and string handling packages in Annex A, “Predefined Language Environment” are also relevant for Information Systems. 6

### *Implementation Advice*

If COBOL (respectively, C) is widely supported in the target environment, implementations supporting the Information Systems Annex should provide the child package `Interfaces.COBOL` (respectively, `Interfaces.C`) specified in Annex B and should support a *convention\_identifier* of COBOL (respectively, C) in the interfacing pragmas (see Annex B), thus allowing Ada programs to interface with programs written in that language. 7

## F.1 `Machine_Radix` Attribute Definition Clause

### *Static Semantics*

`Machine_Radix` may be specified for a decimal first subtype (see 3.5.9) via an `attribute_definition_clause`; the expression of such a clause shall be static, and its value shall be 2 or 10. A value of 2 implies a binary base range; a value of 10 implies a decimal base range. 1

### *Implementation Advice*

Packed decimal should be used as the internal representation for objects of subtype S when `S'Machine_Radix = 10`. 2

### *Examples*

*Example of `Machine_Radix` attribute definition clause:* 3

```
type Money is delta 0.01 digits 15;
for Money'Machine_Radix use 10; 4
```

## F.2 The Package Decimal

### Static Semantics

The library package Decimal has the following declaration:

```

package Ada.Decimal is
 pragma Pure(Decimal);
 Max_Scale : constant := implementation-defined;
 Min_Scale : constant := implementation-defined;
 Min_Delta : constant := 10.0**(-Max_Scale);
 Max_Delta : constant := 10.0**(-Min_Scale);
 Max_Decimal_Digits : constant := implementation-defined;
 generic
 type Dividend_Type is delta <> digits <>;
 type Divisor_Type is delta <> digits <>;
 type Quotient_Type is delta <> digits <>;
 type Remainder_Type is delta <> digits <>;
 procedure Divide (Dividend : in Dividend_Type;
 Divisor : in Divisor_Type;
 Quotient : out Quotient_Type;
 Remainder : out Remainder_Type);
 pragma Convention(Intrinsic, Divide);
 end Ada.Decimal;
```

Max\_Scale is the largest N such that  $10.0^{*(-N)}$  is allowed as a decimal type's delta. Its type is *universal\_integer*.

Min\_Scale is the smallest N such that  $10.0^{*(-N)}$  is allowed as a decimal type's delta. Its type is *universal\_integer*.

Min\_Delta is the smallest value allowed for *delta* in a *decimal\_fixed\_point\_definition*. Its type is *universal\_real*.

Max\_Delta is the largest value allowed for *delta* in a *decimal\_fixed\_point\_definition*. Its type is *universal\_real*.

Max\_Decimal\_Digits is the largest value allowed for *digits* in a *decimal\_fixed\_point\_definition*. Its type is *universal\_integer*.

### Static Semantics

The effect of Divide is as follows. The value of Quotient is Quotient\_Type(Dividend/Divisor). The value of Remainder is Remainder\_Type(Intermediate), where Intermediate is the difference between Dividend and the product of Divisor and Quotient; this result is computed exactly.

### Implementation Requirements

Decimal.Max\_Decimal\_Digits shall be at least 18.

Decimal.Max\_Scale shall be at least 18.

Decimal.Min\_Scale shall be at most 0.

#### NOTES

1 The effect of division yielding a quotient with control over rounding versus truncation is obtained by applying either the function attribute Quotient\_Type'Round or the conversion Quotient\_Type to the expression Dividend/Divisor.

### F.3 Edited Output for Decimal Types

The child packages `Text_IO.Editing`, `Wide_Text_IO.Editing`, and `Wide_Wide_Text_IO.Editing` provide localizable formatted text output, known as *edited output*, for decimal types. An edited output string is a function of a numeric value, program-specifiable locale elements, and a format control value. The numeric value is of some decimal type. The locale elements are:

- the currency string; 2
- the digits group separator character; 3
- the radix mark character; and 4
- the fill character that replaces leading zeros of the numeric value. 5

For `Text_IO`.Editing the edited output and currency strings are of type `String`, and the locale characters are of type `Character`. For `Wide_Text_IO`.Editing their types are `Wide_String` and `Wide_Character`, respectively. For `Wide_Wide_Text_IO`.Editing their types are `Wide_Wide_String` and `Wide_Wide_Character`, respectively.

Each of the locale elements has a default value that can be replaced or explicitly overridden. 7

A format-control value is of the private type `Picture`; it determines the composition of the edited output string and controls the form and placement of the sign, the position of the locale elements and the decimal digits, the presence or absence of a radix mark, suppression of leading zeros, and insertion of particular character values.

A `Picture` object is composed from a `String` value, known as a *picture String*, that serves as a template for the edited output string, and a `Boolean` value that controls whether a string of all space characters is produced when the number's value is zero. A picture String comprises a sequence of one- or two-Character symbols, each serving as a placeholder for a character or string at a corresponding position in the edited output string. The picture String symbols fall into several categories based on their effect on the edited output string:

|                   |      |     |     |     |      |      |    |
|-------------------|------|-----|-----|-----|------|------|----|
| Decimal Digit:    | '9'  |     |     |     |      |      | 10 |
| Radix Control:    | ':'  | 'V' |     |     |      |      |    |
| Sign Control:     | '+'  | '_' | '<' | '>' | "CR" | "DB" |    |
| Currency Control: | '\$' | '#' |     |     |      |      |    |
| Zero Suppression: | 'Z'  | '*' |     |     |      |      |    |
| Simple Insertion: | ' '  | 'B' | '0' | '/' |      |      |    |

The entries are not case-sensitive. Mixed- or lower-case forms for "CR" and "DB", and lower-case forms for 'V', 'Z', and 'B', have the same effect as the upper-case symbols shown.

An occurrence of a '9' Character in the picture String represents a decimal digit position in the edited output string. 12

A radix control Character in the picture String indicates the position of the radix mark in the edited output string; an actual character position for '.', or an assumed position for 'V'. 13

A sign control Character in the picture String affects the form of the sign in the edited output string. The '←' and '→' Character values indicate parentheses for negative values. A Character '+', '-', or '←' appears either singly, signifying a fixed-position sign in the edited output, or repeated, signifying a floating-position sign that is preceded by zero or more space characters and that replaces a leading 0.

- 15 A currency control Character in the picture String indicates an occurrence of the currency string in the edited output string. The '\$' Character represents the complete currency string; the '#' Character represents one character of the currency string. A '\$' Character appears either singly, indicating a fixed-position currency string in the edited output, or repeated, indicating a floating-position currency string that occurs in place of a leading 0. A sequence of '#' Character values indicates either a fixed- or floating-position currency string, depending on context.
- 16 A zero suppression Character in the picture String allows a leading zero to be replaced by either the space character (for 'Z') or the fill character (for '\*').
- 17 A simple insertion Character in the picture String represents, in general, either itself (if '/' or '0'), the space character (if 'B'), or the digits group separator character (if '\_'). In some contexts it is treated as part of a floating sign, floating currency, or zero suppression string.
- 18/2 An example of a picture String is "<###Z\_ZZ9.99>". If the currency string is "kr", the separator character is ',', and the radix mark is '.' then the edited output string values for the decimal values 32.10 and – 5432.10 are "bbkrbbb32.10b" and "(bkr5,432.10)", respectively, where 'b' indicates the space character.
- 19/2 The generic packages Text\_IO.Decimal\_IO, Wide\_Text\_IO.Decimal\_IO, and Wide\_Wide\_Text\_IO.Decimal\_IO (see A.10.9, “Input-Output for Real Types”) provide text input and non-edited text output for decimal types.

## NOTES

- 20/2 2 A picture String is of type Standard.String, for all of Text\_IO Editing, Wide\_Text\_IO Editing, and Wide\_Wide\_Text\_IO Editing.

### F.3.1 Picture String Formation

- 1 A *well-formed picture String*, or simply *picture String*, is a String value that conforms to the syntactic rules, composition constraints, and character replication conventions specified in this clause.

#### Dynamic Semantics

- 2/1 *This paragraph was deleted.*

- 3
- ```

picture_string ::=
    fixed_$_picture_string
  | fixed_#_picture_string
  | floating_currency_picture_string
  | non_currency_picture_string

```

fixed_\$_picture_string ::=

- [fixed_LHS_sign] fixed_\$_char {direct_insertion} [zero_suppression]
number [RHS_sign]
- | [fixed_LHS_sign {direct_insertion}] [zero_suppression]
number fixed_\$_char {direct_insertion} [RHS_sign]
- | floating_LHS_sign number fixed_\$_char {direct_insertion} [RHS_sign]
- | [fixed_LHS_sign] fixed_\$_char {direct_insertion}
all_zero_suppression_number {direct_insertion} [RHS_sign]
- | [fixed_LHS_sign {direct_insertion}] all_zero_suppression_number {direct_insertion}
fixed_\$_char {direct_insertion} [RHS_sign]
- | all_sign_number {direct_insertion} fixed_\$_char {direct_insertion} [RHS_sign]

4

fixed_#_picture_string ::=

- [fixed_LHS_sign] single_#_currency {direct_insertion}
[zero_suppression] number [RHS_sign]
- | [fixed_LHS_sign] multiple_#_currency {direct_insertion}
zero_suppression number [RHS_sign]
- | [fixed_LHS_sign {direct_insertion}] [zero_suppression]
number fixed_#_currency {direct_insertion} [RHS_sign]
- | floating_LHS_sign number fixed_#_currency {direct_insertion} [RHS_sign]
- | [fixed_LHS_sign] single_#_currency {direct_insertion}
all_zero_suppression_number {direct_insertion} [RHS_sign]
- | [fixed_LHS_sign] multiple_#_currency {direct_insertion}
all_zero_suppression_number {direct_insertion} [RHS_sign]
- | [fixed_LHS_sign {direct_insertion}] all_zero_suppression_number {direct_insertion}
fixed_#_currency {direct_insertion} [RHS_sign]
- | all_sign_number {direct_insertion} fixed_#_currency {direct_insertion} [RHS_sign]

5

floating_currency_picture_string ::=

- [fixed_LHS_sign] {direct_insertion} floating_\$_currency number [RHS_sign]
- | [fixed_LHS_sign] {direct_insertion} floating_#_currency number [RHS_sign]
- | [fixed_LHS_sign] {direct_insertion} all_currency_number {direct_insertion} [RHS_sign]

6

```

7      non_currency_picture_string ::=
      [fixed_LHS_sign {direct_insertion}] zero_suppression number [RHS_sign]
      | [floating_LHS_sign] number [RHS_sign]
      | [fixed_LHS_sign {direct_insertion}] all_zero_suppression_number {direct_insertion}
      [RHS_sign]
      | all_sign_number {direct_insertion}
      | fixed_LHS_sign direct_insertion {direct_insertion} number [RHS_sign]

8      fixed_LHS_sign ::= LHS_Sign
9      LHS_Sign ::= + | - | <

10     fixed_$_char ::= $

11     direct_insertion ::= simple_insertion
12     simple_insertion ::= _ | B | 0 | /

13     zero_suppression ::= Z {Z | context_sensitive_insertion} | fill_string
14     context_sensitive_insertion ::= simple_insertion

15     fill_string ::= * { * | context_sensitive_insertion }

16     number ::=
      fore_digits [radix [aft_digits] {direct_insertion}]
      | radix aft_digits {direct_insertion}
17     fore_digits ::= 9 {9 | direct_insertion}
18     aft_digits ::= {9 | direct_insertion} 9
19     radix ::= . | V

20     RHS_sign ::= + | - | > | CR | DB

21     floating_LHS_sign ::=
      LHS_Sign {context_sensitive_insertion} LHS_Sign {LHS_Sign | context_sensitive_insertion}

22     single_#_currency ::= #
23     multiple_#_currency ::= ## {#}

24     fixed_#_currency ::= single_#_currency | multiple_#_currency

25     floating_$_currency ::=
      $ {context_sensitive_insertion} $ {$ | context_sensitive_insertion}

26     floating_#_currency ::=
      # {context_sensitive_insertion} # {# | context_sensitive_insertion}

27     all_sign_number ::= all_sign_fore [radix [all_sign_aft]] [>]

```

all_sign_fore ::=	28
sign_char {context_sensitive_insertion} sign_char {sign_char context_sensitive_insertion}	
all_sign_aft ::= {all_sign_aft_char} sign_char	29
all_sign_aft_char ::= sign_char context_sensitive_insertion	
sign_char ::= + - <	30
all_currency_number ::= all_currency_fore [radix [all_currency_aft]]	31
all_currency_fore ::=	32
currency_char {context_sensitive_insertion}	
currency_char {currency_char context_sensitive_insertion}	
all_currency_aft ::= {all_currency_aft_char} currency_char	33
all_currency_aft_char ::= currency_char context_sensitive_insertion	
currency_char ::= \$ #	34
all_zero_suppression_number ::= all_zero_suppression_fore [radix [all_zero_suppression_aft]]	35
all_zero_suppression_fore ::=	36
zero_suppression_char {zero_suppression_char context_sensitive_insertion}	
all_zero_suppression_aft ::= {all_zero_suppression_aft_char} zero_suppression_char	37
all_zero_suppression_aft_char ::= zero_suppression_char context_sensitive_insertion	
zero_suppression_char ::= Z *	38

The following composition constraints apply to a picture String:

- A floating_LHS_sign does not have occurrences of different LHS_Sign Character values. 40
- If a picture String has '<' as fixed_LHS_sign, then it has '>' as RHS_sign. 41
- If a picture String has '<' in a floating_LHS_sign or in an all_sign_number, then it has an occurrence of '>'. 42
- If a picture String has '+' or '-' as fixed_LHS_sign, in a floating_LHS_sign, or in an all_sign_number, then it has no RHS_sign or '>' character. 43/1
- An instance of all_sign_number does not have occurrences of different sign_char Character values. 44
- An instance of all_currency_number does not have occurrences of different currency_char Character values. 45
- An instance of all_zero_suppression_number does not have occurrences of different zero_suppression_char Character values, except for possible case differences between 'Z' and 'z'. 46

A *replicable Character* is a Character that, by the above rules, can occur in two consecutive positions in a picture String. 47

A *Character replication* is a String 48

char & '(' & *spaces* & *count_string* & ')' 49

where *char* is a replicable Character, *spaces* is a String (possibly empty) comprising only space Character values, and *count_string* is a String of one or more decimal digit Character values. A Character replication 50

in a picture String has the same effect as (and is said to be *equivalent to*) a String comprising n consecutive occurrences of *char*, where $n = \text{Integer'Value}(\text{count_string})$.

51 An *expanded picture String* is a picture String containing no Character replications.

NOTES

52 3 Although a sign to the left of the number can float, a sign to the right of the number is in a fixed position.

F.3.2 Edited Output Generation

Dynamic Semantics

1 The contents of an edited output string are based on:

- 2 • A value, Item, of some decimal type Num,
- 3 • An expanded picture String Pic_String,
- 4 • A Boolean value, Blank_When_Zero,
- 5 • A Currency string,
- 6 • A Fill character,
- 7 • A Separator character, and
- 8 • A Radix_Mark character.

9 The combination of a True value for Blank_When_Zero and a '*' character in Pic_String is inconsistent; no edited output string is defined.

10 A layout error is identified in the rules below if leading non-zero digits of Item, character values of the Currency string, or a negative sign would be truncated; in such cases no edited output string is defined.

11 The edited output string has lower bound 1 and upper bound N where $N = \text{Pic_String'Length} + \text{Currency_Length_Adjustment} - \text{Radix_Adjustment}$, and

- 12 • $\text{Currency_Length_Adjustment} = \text{Currency'Length} - 1$ if there is some occurrence of '\$' in Pic_String, and 0 otherwise.
- 13 • $\text{Radix_Adjustment} = 1$ if there is an occurrence of 'V' or 'v' in Pic_Str, and 0 otherwise.

14 Let the magnitude of Item be expressed as a base-10 number $I_p \cdots I_1.F_1 \cdots F_q$, called the *displayed magnitude* of Item, where:

- 15 • $q = \text{Min}(\text{Max}(\text{Num'Scale}, 0), n)$ where n is 0 if Pic_String has no radix and is otherwise the number of digit positions following radix in Pic_String, where a digit position corresponds to an occurrence of '9', a zero_suppression_char (for an all_zero_suppression_number), a currency_char (for an all_currency_number), or a sign_char (for an all_sign_number).
- 16 • $I_p \neq 0$ if $p > 0$.

17 If $n < \text{Num'Scale}$, then the above number is the result of rounding (away from 0 if exactly midway between values).

18 If Blank_When_Zero = True and the displayed magnitude of Item is zero, then the edited output string comprises all space character values. Otherwise, the picture String is treated as a sequence of instances of syntactic categories based on the rules in F.3.1, and the edited output string is the concatenation of string values derived from these categories according to the following mapping rules.

Table F-1 shows the mapping from a sign control symbol to a corresponding character or string in the edited output. In the columns showing the edited output, a lower-case 'b' represents the space character. If there is no sign control symbol but the value of Item is negative, a layout error occurs and no edited output string is produced.

Table F-1: Edited Output for Sign Control Symbols		
Sign Control Symbol	Edited Output for Non-Negative Number	Edited Output for Negative Number
'+'	'+'	'_'
'_'	'b'	'_'
'<'	'b'	'('
'>'	'b'	')'
"CR"	"bb"	"CR"
"DB"	"bb"	"DB"

An instance of `fixed_LHS_sign` maps to a character as shown in Table F-1.

An instance of `fixed_$_char` maps to Currency.

An instance of `direct_insertion` maps to Separator if `direct_insertion = '_'`, and to the `direct_insertion` Character otherwise.

An instance of `number` maps to a string *integer_part* & *radix_part* & *fraction_part* where:

- The string for *integer_part* is obtained as follows:
 1. Occurrences of '9' in `fore_digits` of `number` are replaced from right to left with the decimal digit character values for $I_1, \dots, I_p$, respectively.
 2. Each occurrence of '9' in `fore_digits` to the left of the leftmost '9' replaced according to rule 1 is replaced with '0'.
 3. If p exceeds the number of occurrences of '9' in `fore_digits` of `number`, then the excess leftmost digits are eligible for use in the mapping of an instance of `zero_suppression`, `floating_LHS_sign`, `floating_$_currency`, or `floating_#_currency` to the left of `number`; if there is no such instance, then a layout error occurs and no edited output string is produced.
- The *radix_part* is:
 - "" if `number` does not include a `radix`, if `radix = 'V'`, or if `radix = 'v'`
 - `Radix_Mark` if `number` includes '.' as `radix`
- The string for *fraction_part* is obtained as follows:
 1. Occurrences of '9' in `aft_digits` of `number` are replaced from left to right with the decimal digit character values for $F_1, \dots, F_q$.
 2. Each occurrence of '9' in `aft_digits` to the right of the rightmost '9' replaced according to rule 1 is replaced by '0'.

An instance of `zero_suppression` maps to the string obtained as follows:

- 35 1. The rightmost 'Z', 'z', or '*' Character values are replaced with the excess digits (if any) from the
integer_part of the mapping of the number to the right of the *zero_suppression* instance,
- 36 2. A *context_sensitive_insertion* Character is replaced as though it were a *direct_insertion*
Character, if it occurs to the right of some 'Z', 'z', or '*' in *zero_suppression* that has been
mapped to an excess digit,
- 37 3. Each Character to the left of the leftmost Character replaced according to rule 1 above is
replaced by:
 - 38 • the space character if the zero suppression Character is 'Z' or 'z', or
 - 39 • the Fill character if the zero suppression Character is '*'.
- 40 4. A layout error occurs if some excess digits remain after all 'Z', 'z', and '*' Character values in
zero_suppression have been replaced via rule 1; no edited output string is produced.

41 An instance of *RHS_sign* maps to a character or string as shown in Table F-1.

42 An instance of *floating_LHS_sign* maps to the string obtained as follows.

- 43 1. Up to all but one of the rightmost *LHS_Sign* Character values are replaced by the excess digits
(if any) from the *integer_part* of the mapping of the number to the right of the
floating_LHS_sign instance.
- 44 2. The next Character to the left is replaced with the character given by the entry in Table F-1
corresponding to the *LHS_Sign* Character.
- 45 3. A *context_sensitive_insertion* Character is replaced as though it were a *direct_insertion*
Character, if it occurs to the right of the leftmost *LHS_Sign* character replaced according to rule
1.
- 46 4. Any other Character is replaced by the space character..
- 47 5. A layout error occurs if some excess digits remain after replacement via rule 1; no edited output
string is produced.

48 An instance of *fixed_#_currency* maps to the Currency string with n space character values concatenated
on the left (if the instance does not follow a *radix*) or on the right (if the instance does follow a *radix*),
where n is the difference between the length of the *fixed_#_currency* instance and *Currency*'Length. A
layout error occurs if *Currency*'Length exceeds the length of the *fixed_#_currency* instance; no edited
output string is produced.

49 An instance of *floating_\$_currency* maps to the string obtained as follows:

- 50 1. Up to all but one of the rightmost '\$' Character values are replaced with the excess digits (if any)
from the *integer_part* of the mapping of the number to the right of the *floating_\$_currency*
instance.
- 51 2. The next Character to the left is replaced by the Currency string.
- 52 3. A *context_sensitive_insertion* Character is replaced as though it were a *direct_insertion*
Character, if it occurs to the right of the leftmost '\$' Character replaced via rule 1.
- 53 4. Each other Character is replaced by the space character.
- 54 5. A layout error occurs if some excess digits remain after replacement by rule 1; no edited output
string is produced.

55 An instance of *floating_#_currency* maps to the string obtained as follows:

1. Up to all but one of the rightmost '#' Character values are replaced with the excess digits (if any) from the *integer_part* of the mapping of the number to the right of the *floating_#_currency* instance. 56
2. The substring whose last Character occurs at the position immediately preceding the leftmost Character replaced via rule 1, and whose length is *Currency'Length*, is replaced by the *Currency* string. 57
3. A *context_sensitive_insertion* Character is replaced as though it were a *direct_insertion* Character, if it occurs to the right of the leftmost '#' replaced via rule 1. 58
4. Any other Character is replaced by the space character. 59
5. A layout error occurs if some excess digits remain after replacement rule 1, or if there is no substring with the required length for replacement rule 2; no edited output string is produced. 60

An instance of *all_zero_suppression_number* maps to: 61

- a string of all spaces if the displayed magnitude of *Item* is zero, the *zero_suppression_char* is 'Z' or 'z', and the instance of *all_zero_suppression_number* does not have a radix at its last character position; 62
- a string containing the *Fill* character in each position except for the character (if any) corresponding to *radix*, if *zero_suppression_char* = '*' and the displayed magnitude of *Item* is zero; 63
- otherwise, the same result as if each *zero_suppression_char* in *all_zero_suppression_aft* were '9', interpreting the instance of *all_zero_suppression_number* as either *zero_suppression_number* (if a *radix* and *all_zero_suppression_aft* are present), or as *zero_suppression* otherwise. 64

An instance of *all_sign_number* maps to: 65

- a string of all spaces if the displayed magnitude of *Item* is zero and the instance of *all_sign_number* does not have a *radix* at its last character position; 66
- otherwise, the same result as if each *sign_char* in *all_sign_number_aft* were '9', interpreting the instance of *all_sign_number* as either *floating_LHS_sign_number* (if a *radix* and *all_sign_number_aft* are present), or as *floating_LHS_sign* otherwise. 67

An instance of *all_currency_number* maps to: 68

- a string of all spaces if the displayed magnitude of *Item* is zero and the instance of *all_currency_number* does not have a *radix* at its last character position; 69
- otherwise, the same result as if each *currency_char* in *all_currency_number_aft* were '9', interpreting the instance of *all_currency_number* as *floating_\$_currency_number* or *floating_#_currency_number* (if a *radix* and *all_currency_number_aft* are present), or as *floating_\$_currency* or *floating_#_currency* otherwise. 70

Examples

In the result string values shown below, 'b' represents the space character. 71

Item:	Picture and Result Strings:	
123456.78	Picture:	"-###** *9.99"
		"bbb\$***123,456.78"
		"bbFF***123.456,78" (currency = "FF", separator = '.', radix mark = ',')

74/1	123456.78	Picture: "-\$** **9.99"	
		Result: "b\$**123,456.78"	
		"bFF**123.456,78"	(currency = "FF", separator = '.', radix mark = ',')
75	0.0	Picture: "-\$\$\$\$\$. \$"	
		Result: "bbbbbbbbb"	
76	0.20	Picture: "-\$\$\$\$\$. \$"	
		Result: "bbbbbb\$.20"	
77	-1234.565	Picture: "<<<<<<<. <####>"	
		Result: "bb(1,234.57DMb)"	(currency = "DM")
78	12345.67	Picture: "###_###_##9.99"	
		Result: "bbCHF12,345.67"	(currency = "CHF")

F.3.3 The Package Text_IO.Editing

- 1 The package Text_IO.Editing provides a private type Picture with associated operations, and a generic package Decimal_Output. An object of type Picture is composed from a well-formed picture String (see F.3.1) and a Boolean item indicating whether a zero numeric value will result in an edited output string of all space characters. The package Decimal_Output contains edited output subprograms implementing the effects defined in F.3.2.

Static Semantics

- 2 The library package Text_IO.Editing has the following declaration:

```

3  package Ada.Text_IO.Editing is
4      type Picture is private;
5      function Valid (Pic_String      : in String;
6                      Blank_When_Zero : in Boolean := False) return Boolean;
7      function To_Picture (Pic_String      : in String;
8                          Blank_When_Zero : in Boolean := False)
9          return Picture;
10     function Pic_String (Pic : in Picture) return String;
11     function Blank_When_Zero (Pic : in Picture) return Boolean;
12     Max_Picture_Length : constant := implementation_defined;
13     Picture_Error      : exception;
14     Default_Currency   : constant String := "$";
15     Default_Fill       : constant Character := '*';
16     Default_Separator  : constant Character := ',';
17     Default_Radix_Mark : constant Character := '.';
18
19     generic
20         type Num is delta <> digits <>;
21         Default_Currency : in String := Text_IO.Editing.Default_Currency;
22         Default_Fill     : in Character := Text_IO.Editing.Default_Fill;
23         Default_Separator : in Character :=
24             Text_IO.Editing.Default_Separator;
25         Default_Radix_Mark : in Character :=
26             Text_IO.Editing.Default_Radix_Mark;
27     package Decimal_Output is
28         function Length (Pic : in Picture;
29                         Currency : in String := Default_Currency)
30             return Natural;
31         function Valid (Item : in Num;
32                       Pic : in Picture;
33                       Currency : in String := Default_Currency)
34             return Boolean;

```

```

function Image (Item      : in Num;                               13
                Pic       : in Picture;
                Currency   : in String := Default_Currency;
                Fill       : in Character := Default_Fill;
                Separator   : in Character := Default_Separator;
                Radix_Mark  : in Character := Default_Radix_Mark)
    return String;

procedure Put (File      : in File_Type;                          14
               Item      : in Num;
               Pic       : in Picture;
               Currency   : in String := Default_Currency;
               Fill       : in Character := Default_Fill;
               Separator   : in Character := Default_Separator;
               Radix_Mark  : in Character := Default_Radix_Mark);

procedure Put (Item      : in Num;                               15
               Pic       : in Picture;
               Currency   : in String := Default_Currency;
               Fill       : in Character := Default_Fill;
               Separator   : in Character := Default_Separator;
               Radix_Mark  : in Character := Default_Radix_Mark);

procedure Put (To       : out String;                             16
               Item      : in Num;
               Pic       : in Picture;
               Currency   : in String := Default_Currency;
               Fill       : in Character := Default_Fill;
               Separator   : in Character := Default_Separator;
               Radix_Mark  : in Character := Default_Radix_Mark);
end Decimal_Output;
private
    ... -- not specified by the language
end Ada.Text_IO.Editing;

```

The exception `Constraint_Error` is raised if the `Image` function or any of the `Put` procedures is invoked with a null string for `Currency`. 17

```

function Valid (Pic_String : in String;                          18
                Blank_When_Zero : in Boolean := False) return Boolean;

Valid returns True if Pic_String is a well-formed picture String (see F.3.1) the length of whose
expansion does not exceed Max_Picture_Length, and if either Blank_When_Zero is False or
Pic_String contains no '*'. 19

```

```

function To_Picture (Pic_String : in String;                     20
                    Blank_When_Zero : in Boolean := False)
    return Picture;

```

`To_Picture` returns a result `Picture` such that the application of the function `Pic_String` to this result yields an expanded picture String equivalent to `Pic_String`, and such that `Blank_When_Zero` applied to the result `Picture` is the same value as the parameter `Blank_When_Zero`. `Picture_Error` is raised if not `Valid(Pic_String, Blank_When_Zero)`. 21

```

function Pic_String (Pic : in Picture) return String;           22

```

```

function Blank_When_Zero (Pic : in Picture) return Boolean;

```

If `Pic` is `To_Picture(String_Item, Boolean_Item)` for some `String_Item` and `Boolean_Item`, then: 23

- `Pic_String(Pic)` returns an expanded picture String equivalent to `String_Item` and with 24
any lower-case letter replaced with its corresponding upper-case form, and
- `Blank_When_Zero(Pic)` returns `Boolean_Item`. 25

If `Pic_1` and `Pic_2` are objects of type `Picture`, then `"="(Pic_1, Pic_2)` is True when 26

```

27         • Pic_String(Pic_1) = Pic_String(Pic_2), and
28         • Blank_When_Zero(Pic_1) = Blank_When_Zero(Pic_2).
29     function Length (Pic      : in Picture;
                      Currency : in String := Default_Currency)
        return Natural;
30     Length returns Pic_String(Pic)'Length + Currency_Length_Adjustment - Radix_Adjustment
    where
31         • Currency_Length_Adjustment =
32             • Currency'Length - 1 if there is some occurrence of '$' in Pic_String(Pic), and
33             • 0 otherwise.
34         • Radix_Adjustment =
35             • 1 if there is an occurrence of 'V' or 'v' in Pic_Str(Pic), and
36             • 0 otherwise.
37     function Valid (Item      : in Num;
                    Pic        : in Picture;
                    Currency   : in String := Default_Currency)
        return Boolean;
38     Valid returns True if Image(Item, Pic, Currency) does not raise Layout_Error, and returns False
        otherwise.
39     function Image (Item      : in Num;
                    Pic        : in Picture;
                    Currency   : in String := Default_Currency;
                    Fill       : in Character := Default_Fill;
                    Separator  : in Character := Default_Separator;
                    Radix_Mark : in Character := Default_Radix_Mark)
        return String;
40     Image returns the edited output String as defined in F.3.2 for Item, Pic_String(Pic),
        Blank_When_Zero(Pic), Currency, Fill, Separator, and Radix_Mark. If these rules identify a
        layout error, then Image raises the exception Layout_Error.
41     procedure Put (File      : in File_Type;
                    Item       : in Num;
                    Pic        : in Picture;
                    Currency   : in String := Default_Currency;
                    Fill       : in Character := Default_Fill;
                    Separator  : in Character := Default_Separator;
                    Radix_Mark : in Character := Default_Radix_Mark);

    procedure Put (Item      : in Num;
                    Pic        : in Picture;
                    Currency   : in String := Default_Currency;
                    Fill       : in Character := Default_Fill;
                    Separator  : in Character := Default_Separator;
                    Radix_Mark : in Character := Default_Radix_Mark);
42     Each of these Put procedures outputs Image(Item, Pic, Currency, Fill, Separator, Radix_Mark)
        consistent with the conventions for Put for other real types in case of bounded line length (see
        A.10.6, “Get and Put Procedures”).

```

```

procedure Put (To           : out String;
               Item         : in Num;
               Pic          : in Picture;
               Currency     : in String   := Default_Currency;
               Fill         : in Character := Default_Fill;
               Separator    : in Character := Default_Separator;
               Radix_Mark   : in Character := Default_Radix_Mark);

```

Put copies Image(Item, Pic, Currency, Fill, Separator, Radix_Mark) to the given string, right justified. Otherwise unassigned Character values in To are assigned the space character. If To'Length is less than the length of the string resulting from Image, then Layout_Error is raised.

Implementation Requirements

Max_Picture_Length shall be at least 30. The implementation shall support currency strings of length up to at least 10, both for Default_Currency in an instantiation of Decimal_Output, and for Currency in an invocation of Image or any of the Put procedures.

NOTES

4 The rules for edited output are based on COBOL (ANSI X3.23:1985, endorsed by ISO as ISO 1989-1985), with the following differences:

- The COBOL provisions for picture string localization and for 'P' format are absent from Ada.
- The following Ada facilities are not in COBOL:
 - currency symbol placement after the number,
 - localization of edited output string for multi-character currency string values, including support for both length-preserving and length-expanding currency symbols in picture strings
 - localization of the radix mark, digits separator, and fill character, and
 - parenthesization of negative values.

The value of 30 for Max_Picture_Length is the same limit as in COBOL.

F.3.4 The Package Wide_Text_IO.Editing

Static Semantics

The child package Wide_Text_IO.Editing has the same contents as Text_IO.Editing, except that:

- each occurrence of Character is replaced by Wide_Character,
- each occurrence of Text_IO is replaced by Wide_Text_IO,
- the subtype of Default_Currency is Wide_String rather than String, and
- each occurrence of String in the generic package Decimal_Output is replaced by Wide_String.

NOTES

5 Each of the functions Wide_Text_IO.Editing.Valid, To_Picture, and Pic_String has String (versus Wide_String) as its parameter or result subtype, since a picture String is not localizable.

F.3.5 The Package Wide_Wide_Text_IO.Editing

Static Semantics

The child package Wide_Wide_Text_IO.Editing has the same contents as Text_IO.Editing, except that:

- each occurrence of Character is replaced by Wide_Wide_Character,

- 3/2 • each occurrence of `Text_IO` is replaced by `Wide_Wide_Text_IO`,
- 4/2 • the subtype of `Default_Currency` is `Wide_Wide_String` rather than `String`, and
- 5/2 • each occurrence of `String` in the generic package `Decimal_Output` is replaced by `Wide_Wide_String`.

NOTES

- 6/2 6 Each of the functions `Wide_Wide_Text_IO.Editing.Valid`, `To_Picture`, and `Pic_String` has `String` (versus `Wide_Wide_String`) as its parameter or result subtype, since a picture `String` is not localizable.

Annex G

(normative)

Numerics

The Numerics Annex specifies	1
• features for complex arithmetic, including complex I/O;	2
• a mode (“strict mode”), in which the predefined arithmetic operations of floating point and fixed point types and the functions and operations of various predefined packages have to provide guaranteed accuracy or conform to other numeric performance requirements, which the Numerics Annex also specifies;	3
• a mode (“relaxed mode”), in which no accuracy or other numeric performance requirements need be satisfied, as for implementations not conforming to the Numerics Annex;	4
• models of floating point and fixed point arithmetic on which the accuracy requirements of strict mode are based;	5/2
• the definitions of the model-oriented attributes of floating point types that apply in the strict mode; and	6/2
• features for the manipulation of real and complex vectors and matrices.	6.1/2

Implementation Advice

If Fortran (respectively, C) is widely supported in the target environment, implementations supporting the Numerics Annex should provide the child package Interfaces.Fortran (respectively, Interfaces.C) specified in Annex B and should support a <i>convention identifier</i> of Fortran (respectively, C) in the interfacing pragmas (see Annex B), thus allowing Ada programs to interface with programs written in that language.	7
--	---

G.1 Complex Arithmetic

Types and arithmetic operations for complex arithmetic are provided in <code>Generic_Complex_Types</code> , which is defined in G.1.1. Implementation-defined approximations to the complex analogs of the mathematical functions known as the “elementary functions” are provided by the subprograms in <code>Generic_Complex_Elementary_Functions</code> , which is defined in G.1.2. Both of these library units are generic children of the predefined package Numerics (see A.5). Nongeneric equivalents of these generic packages for each of the predefined floating point types are also provided as children of Numerics.	1
--	---

G.1.1 Complex Types

Static Semantics

The generic library package <code>Numerics.Generic_Complex_Types</code> has the following declaration:	1
<pre> generic type Real is digits <>; package Ada.Numerics.Generic_Complex_Types is pragma Pure(Generic_Complex_Types); type Complex is record Re, Im : Real'Base; end record; </pre>	2/1
	3

```

4/2  type Imaginary is private;
      pragma Preelaborable_Initialization(Imaginary);

5    i : constant Imaginary;
      j : constant Imaginary;

6    function Re (X : Complex) return Real'Base;
      function Im (X : Complex) return Real'Base;
      function Im (X : Imaginary) return Real'Base;

7    procedure Set_Re (X : in out Complex;
                      Re : in Real'Base);
      procedure Set_Im (X : in out Complex;
                       Im : in Real'Base);
      procedure Set_Im (X : out Imaginary;
                       Im : in Real'Base);

8    function Compose_From_Cartesian (Re, Im : Real'Base) return Complex;
      function Compose_From_Cartesian (Re : Real'Base) return Complex;
      function Compose_From_Cartesian (Im : Imaginary) return Complex;

9    function Modulus (X : Complex) return Real'Base;
      function "abs" (Right : Complex) return Real'Base renames Modulus;

10   function Argument (X : Complex) return Real'Base;
      function Argument (X : Complex;
                        Cycle : Real'Base) return Real'Base;

11   function Compose_From_Polar (Modulus, Argument : Real'Base)
      return Complex;
      function Compose_From_Polar (Modulus, Argument, Cycle : Real'Base)
      return Complex;

12   function "+" (Right : Complex) return Complex;
      function "-" (Right : Complex) return Complex;
      function Conjugate (X : Complex) return Complex;

13   function "+" (Left, Right : Complex) return Complex;
      function "-" (Left, Right : Complex) return Complex;
      function "*" (Left, Right : Complex) return Complex;
      function "/" (Left, Right : Complex) return Complex;

14   function "***" (Left : Complex; Right : Integer) return Complex;

15   function "+" (Right : Imaginary) return Imaginary;
      function "-" (Right : Imaginary) return Imaginary;
      function Conjugate (X : Imaginary) return Imaginary renames "-";
      function "abs" (Right : Imaginary) return Real'Base;

16   function "+" (Left, Right : Imaginary) return Imaginary;
      function "-" (Left, Right : Imaginary) return Imaginary;
      function "*" (Left, Right : Imaginary) return Real'Base;
      function "/" (Left, Right : Imaginary) return Real'Base;

17   function "***" (Left : Imaginary; Right : Integer) return Complex;

18   function "<" (Left, Right : Imaginary) return Boolean;
      function "<=" (Left, Right : Imaginary) return Boolean;
      function ">" (Left, Right : Imaginary) return Boolean;
      function ">=" (Left, Right : Imaginary) return Boolean;

19   function "+" (Left : Complex; Right : Real'Base) return Complex;
      function "+" (Left : Real'Base; Right : Complex) return Complex;
      function "-" (Left : Complex; Right : Real'Base) return Complex;
      function "-" (Left : Real'Base; Right : Complex) return Complex;
      function "*" (Left : Complex; Right : Real'Base) return Complex;
      function "*" (Left : Real'Base; Right : Complex) return Complex;
      function "/" (Left : Complex; Right : Real'Base) return Complex;
      function "/" (Left : Real'Base; Right : Complex) return Complex;

```

```

function "+" (Left : Complex; Right : Imaginary) return Complex;
function "+" (Left : Imaginary; Right : Complex) return Complex;
function "-" (Left : Complex; Right : Imaginary) return Complex;
function "-" (Left : Imaginary; Right : Complex) return Complex;
function "*" (Left : Complex; Right : Imaginary) return Complex;
function "*" (Left : Imaginary; Right : Complex) return Complex;
function "/" (Left : Complex; Right : Imaginary) return Complex;
function "/" (Left : Imaginary; Right : Complex) return Complex;

function "+" (Left : Imaginary; Right : Real'Base) return Complex;
function "+" (Left : Real'Base; Right : Imaginary) return Complex;
function "-" (Left : Imaginary; Right : Real'Base) return Complex;
function "-" (Left : Real'Base; Right : Imaginary) return Complex;
function "*" (Left : Imaginary; Right : Real'Base) return Imaginary;
function "*" (Left : Real'Base; Right : Imaginary) return Imaginary;
function "/" (Left : Imaginary; Right : Real'Base) return Imaginary;
function "/" (Left : Real'Base; Right : Imaginary) return Imaginary;

private
    type Imaginary is new Real'Base;
    i : constant Imaginary := 1.0;
    j : constant Imaginary := 1.0;
end Ada.Numerics.Generic_Complex_Types;

```

The library package `Numerics.Complex_Types` is declared pure and defines the same types, constants, and subprograms as `Numerics.Generic_Complex_Types`, except that the predefined type `Float` is systematically substituted for `Real'Base` throughout. Nongeneric equivalents of `Numerics.Generic_Complex_Types` for each of the other predefined floating point types are defined similarly, with the names `Numerics.Short_Complex_Types`, `Numerics.Long_Complex_Types`, etc.

`Complex` is a visible type with Cartesian components.

`Imaginary` is a private type; its full type is derived from `Real'Base`.

The arithmetic operations and the `Re`, `Im`, `Modulus`, `Argument`, and `Conjugate` functions have their usual mathematical meanings. When applied to a parameter of pure-imaginary type, the “imaginary-part” function `Im` yields the value of its parameter, as the corresponding real value. The remaining subprograms have the following meanings:

- The `Set_Re` and `Set_Im` procedures replace the designated component of a complex parameter with the given real value; applied to a parameter of pure-imaginary type, the `Set_Im` procedure replaces the value of that parameter with the imaginary value corresponding to the given real value.
- The `Compose_From_Cartesian` function constructs a complex value from the given real and imaginary components. If only one component is given, the other component is implicitly zero.
- The `Compose_From_Polar` function constructs a complex value from the given modulus (radius) and argument (angle). When the value of the parameter `Modulus` is positive (resp., negative), the result is the complex value represented by the point in the complex plane lying at a distance from the origin given by the absolute value of `Modulus` and forming an angle measured counterclockwise from the positive (resp., negative) real axis given by the value of the parameter `Argument`.

When the `Cycle` parameter is specified, the result of the `Argument` function and the parameter `Argument` of the `Compose_From_Polar` function are measured in units such that a full cycle of revolution has the given value; otherwise, they are measured in radians.

The computed results of the mathematically multivalued functions are rendered single-valued by the following conventions, which are meant to imply the principal branch:

- 34 • The result of the Modulus function is nonnegative.
- 35 • The result of the Argument function is in the quadrant containing the point in the complex plane represented by the parameter X. This may be any quadrant (I through IV); thus, the range of the Argument function is approximately $-\pi$ to π ($-\text{Cycle}/2.0$ to $\text{Cycle}/2.0$, if the parameter Cycle is specified). When the point represented by the parameter X lies on the negative real axis, the result approximates
 - 36 • π (resp., $-\pi$) when the sign of the imaginary component of X is positive (resp., negative), if Real'Signed_Zeros is True;
 - 37 • π , if Real'Signed_Zeros is False.
- 38 • Because a result lying on or near one of the axes may not be exactly representable, the approximation inherent in computing the result may place it in an adjacent quadrant, close to but on the wrong side of the axis.

Dynamic Semantics

- 39 The exception Numerics.Argument_Error is raised by the Argument and Compose_From_Polar functions with specified cycle, signaling a parameter value outside the domain of the corresponding mathematical function, when the value of the parameter Cycle is zero or negative.
- 40 The exception Constraint_Error is raised by the division operator when the value of the right operand is zero, and by the exponentiation operator when the value of the left operand is zero and the value of the exponent is negative, provided that Real'Machine_Overflows is True; when Real'Machine_Overflows is False, the result is unspecified. Constraint_Error can also be raised when a finite result overflows (see G.2.6).

Implementation Requirements

- 41 In the implementation of Numerics.Generic_Complex_Types, the range of intermediate values allowed during the calculation of a final result shall not be affected by any range constraint of the subtype Real.
- 42 In the following cases, evaluation of a complex arithmetic operation shall yield the *prescribed result*, provided that the preceding rules do not call for an exception to be raised:
- 43 • The results of the Re, Im, and Compose_From_Cartesian functions are exact.
 - 44 • The real (resp., imaginary) component of the result of a binary addition operator that yields a result of complex type is exact when either of its operands is of pure-imaginary (resp., real) type.
 - 45 • The real (resp., imaginary) component of the result of a binary subtraction operator that yields a result of complex type is exact when its right operand is of pure-imaginary (resp., real) type.
 - 46 • The real component of the result of the Conjugate function for the complex type is exact.
 - 47 • When the point in the complex plane represented by the parameter X lies on the nonnegative real axis, the Argument function yields a result of zero.
 - 48 • When the value of the parameter Modulus is zero, the Compose_From_Polar function yields a result of zero.
 - 49 • When the value of the parameter Argument is equal to a multiple of the quarter cycle, the result of the Compose_From_Polar function with specified cycle lies on one of the axes. In this case, one of its components is zero, and the other has the magnitude of the parameter Modulus.
 - 50 • Exponentiation by a zero exponent yields the value one. Exponentiation by a unit exponent yields the value of the left operand. Exponentiation of the value one yields the value one. Exponentiation of the value zero yields the value zero, provided that the exponent is nonzero.

When the left operand is of pure-imaginary type, one component of the result of the exponentiation operator is zero.

When the result, or a result component, of any operator of `Numerics.Generic_Complex_Types` has a mathematical definition in terms of a single arithmetic or relational operation, that result or result component exhibits the accuracy of the corresponding operation of the type `Real`. 51

Other accuracy requirements for the `Modulus`, `Argument`, and `Compose_From_Polar` functions, and accuracy requirements for the multiplication of a pair of complex operands or for division by a complex operand, all of which apply only in the strict mode, are given in G.2.6. 52

The sign of a zero result or zero result component yielded by a complex arithmetic operation or function is implementation defined when `Real'Signed_Zeros` is `True`. 53

Implementation Permissions

The nongeneric equivalent packages may, but need not, be actual instantiations of the generic package for the appropriate predefined type. 54

Implementations may obtain the result of exponentiation of a complex or pure-imaginary operand by repeated complex multiplication, with arbitrary association of the factors and with a possible final complex reciprocation (when the exponent is negative). Implementations are also permitted to obtain the result of exponentiation of a complex operand, but not of a pure-imaginary operand, by converting the left operand to a polar representation; exponentiating the modulus by the given exponent; multiplying the argument by the given exponent; and reconverting to a Cartesian representation. Because of this implementation freedom, no accuracy requirement is imposed on complex exponentiation (except for the prescribed results given above, which apply regardless of the implementation method chosen). 55/2

Implementation Advice

Because the usual mathematical meaning of multiplication of a complex operand and a real operand is that of the scaling of both components of the former by the latter, an implementation should not perform this operation by first promoting the real operand to complex type and then performing a full complex multiplication. In systems that, in the future, support an Ada binding to IEC 559:1989, the latter technique will not generate the required result when one of the components of the complex operand is infinite. (Explicit multiplication of the infinite component by the zero component obtained during promotion yields a NaN that propagates into the final result.) Analogous advice applies in the case of multiplication of a complex operand and a pure-imaginary operand, and in the case of division of a complex operand by a real or pure-imaginary operand. 56

Likewise, because the usual mathematical meaning of addition of a complex operand and a real operand is that the imaginary operand remains unchanged, an implementation should not perform this operation by first promoting the real operand to complex type and then performing a full complex addition. In implementations in which the `Signed_Zeros` attribute of the component type is `True` (and which therefore conform to IEC 559:1989 in regard to the handling of the sign of zero in predefined arithmetic operations), the latter technique will not generate the required result when the imaginary component of the complex operand is a negatively signed zero. (Explicit addition of the negative zero to the zero obtained during promotion yields a positive zero.) Analogous advice applies in the case of addition of a complex operand and a pure-imaginary operand, and in the case of subtraction of a complex operand and a real or pure-imaginary operand. 57

Implementations in which `Real'Signed_Zeros` is `True` should attempt to provide a rational treatment of the signs of zero results and result components. As one example, the result of the `Argument` function should have the sign of the imaginary component of the parameter `X` when the point represented by that 58

parameter lies on the positive real axis; as another, the sign of the imaginary component of the Compose_From_Polar function should be the same as (resp., the opposite of) that of the Argument parameter when that parameter has a value of zero and the Modulus parameter has a nonnegative (resp., negative) value.

G.1.2 Complex Elementary Functions

Static Semantics

1 The generic library package Numerics.Generic_Complex_Elementary_Functions has the following declaration:

```
2/2  with Ada.Numerics.Generic_Complex_Types;
      generic
        with package Complex_Types is
          new Ada.Numerics.Generic_Complex_Types (<>);
          use Complex_Types;
        package Ada.Numerics.Generic_Complex_Elementary_Functions is
          pragma Pure(Generic_Complex_Elementary_Functions);
3      function Sqrt (X : Complex) return Complex;
      function Log   (X : Complex) return Complex;
      function Exp   (X : Complex) return Complex;
      function Exp   (X : Imaginary) return Complex;
      function "***" (Left : Complex; Right : Complex) return Complex;
      function "***" (Left : Complex; Right : Real'Base) return Complex;
      function "***" (Left : Real'Base; Right : Complex) return Complex;
4      function Sin (X : Complex) return Complex;
      function Cos (X : Complex) return Complex;
      function Tan (X : Complex) return Complex;
      function Cot (X : Complex) return Complex;
5      function Arcsin (X : Complex) return Complex;
      function Arccos (X : Complex) return Complex;
      function Arctan (X : Complex) return Complex;
      function Arccot (X : Complex) return Complex;
6      function Sinh (X : Complex) return Complex;
      function Cosh (X : Complex) return Complex;
      function Tanh (X : Complex) return Complex;
      function Coth (X : Complex) return Complex;
7      function Arcsinh (X : Complex) return Complex;
      function Arccosh (X : Complex) return Complex;
      function Arctanh (X : Complex) return Complex;
      function Arccoth (X : Complex) return Complex;
8      end Ada.Numerics.Generic_Complex_Elementary_Functions;
```

9/1 The library package Numerics.Complex_Elementary_Functions is declared pure and defines the same subprograms as Numerics.Generic_Complex_Elementary_Functions, except that the predefined type Float is systematically substituted for Real'Base, and the Complex and Imaginary types exported by Numerics.Complex_Types are systematically substituted for Complex and Imaginary, throughout. Nongeneric equivalents of Numerics.Generic_Complex_Elementary_Functions corresponding to each of the other predefined floating point types are defined similarly, with the names Numerics.Short_Complex_Elementary_Functions, Numerics.Long_Complex_Elementary_Functions, etc.

10 The overloading of the Exp function for the pure-imaginary type is provided to give the user an alternate way to compose a complex value from a given modulus and argument. In addition to Compose_From_Polar(Rho, Theta) (see G.1.1), the programmer may write Rho * Exp(i * Theta).

11 The imaginary (resp., real) component of the parameter X of the forward hyperbolic (resp., trigonometric) functions and of the Exp function (and the parameter X, itself, in the case of the overloading of the Exp

function for the pure-imaginary type) represents an angle measured in radians, as does the imaginary (resp., real) component of the result of the Log and inverse hyperbolic (resp., trigonometric) functions.

The functions have their usual mathematical meanings. However, the arbitrariness inherent in the placement of branch cuts, across which some of the complex elementary functions exhibit discontinuities, is eliminated by the following conventions:

- The imaginary component of the result of the Sqrt and Log functions is discontinuous as the parameter X crosses the negative real axis. 13
- The result of the exponentiation operator when the left operand is of complex type is discontinuous as that operand crosses the negative real axis. 14
- The imaginary component of the result of the Arcsin, Arccos, and Arctanh functions is discontinuous as the parameter X crosses the real axis to the left of -1.0 or the right of 1.0 . 15/2
- The real component of the result of the Arctan and Arcsinh functions is discontinuous as the parameter X crosses the imaginary axis below $-i$ or above i . 16/2
- The real component of the result of the Arccot function is discontinuous as the parameter X crosses the imaginary axis below $-i$ or above i . 17/2
- The imaginary component of the Arccosh function is discontinuous as the parameter X crosses the real axis to the left of 1.0 . 18
- The imaginary component of the result of the Arccoth function is discontinuous as the parameter X crosses the real axis between -1.0 and 1.0 . 19

The computed results of the mathematically multivalued functions are rendered single-valued by the following conventions, which are meant to imply that the principal branch is an analytic continuation of the corresponding real-valued function in Numerics.Generic_Elementary_Functions. (For Arctan and Arccot, the single-argument function in question is that obtained from the two-argument version by fixing the second argument to be its default value.) 20/2

- The real component of the result of the Sqrt and Arccosh functions is nonnegative. 21
- The same convention applies to the imaginary component of the result of the Log function as applies to the result of the natural-cycle version of the Argument function of Numerics.Generic_Complex_Types (see G.1.1). 22
- The range of the real (resp., imaginary) component of the result of the Arcsin and Arctan (resp., Arcsinh and Arctanh) functions is approximately $-\pi/2.0$ to $\pi/2.0$. 23
- The real (resp., imaginary) component of the result of the Arccos and Arccot (resp., Arccoth) functions ranges from 0.0 to approximately π . 24
- The range of the imaginary component of the result of the Arccosh function is approximately $-\pi$ to π . 25

In addition, the exponentiation operator inherits the single-valuedness of the Log function. 26

Dynamic Semantics

The exception Numerics.Argument_Error is raised by the exponentiation operator, signaling a parameter value outside the domain of the corresponding mathematical function, when the value of the left operand is zero and the real component of the exponent (or the exponent itself, when it is of real type) is zero. 27

The exception Constraint_Error is raised, signaling a pole of the mathematical function (analogous to dividing by zero), in the following cases, provided that Complex_Types.Real_Machine_Overflows is True: 28

- by the Log, Cot, and Coth functions, when the value of the parameter X is zero; 29

- 30 • by the exponentiation operator, when the value of the left operand is zero and the real component of the exponent (or the exponent itself, when it is of real type) is negative;
 - 31 • by the Arctan and Arccot functions, when the value of the parameter X is $\pm i$;
 - 32 • by the Arctanh and Arccoth functions, when the value of the parameter X is ± 1.0 .
- 33 Constraint_Error can also be raised when a finite result overflows (see G.2.6); this may occur for parameter values sufficiently *near* poles, and, in the case of some of the functions, for parameter values having components of sufficiently large magnitude. When Complex_Types.Real'Machine_Overflows is False, the result at poles is unspecified.

Implementation Requirements

- 34 In the implementation of Numerics.Generic_Complex_Elementary_Functions, the range of intermediate values allowed during the calculation of a final result shall not be affected by any range constraint of the subtype Complex_Types.Real.
- 35 In the following cases, evaluation of a complex elementary function shall yield the *prescribed result* (or a result having the prescribed component), provided that the preceding rules do not call for an exception to be raised:
- 36 • When the parameter X has the value zero, the Sqrt, Sin, Arcsin, Tan, Arctan, Sinh, Arcsinh, Tanh, and Arctanh functions yield a result of zero; the Exp, Cos, and Cosh functions yield a result of one; the Arccos and Arccot functions yield a real result; and the Arccoth function yields an imaginary result.
 - 37 • When the parameter X has the value one, the Sqrt function yields a result of one; the Log, Arccos, and Arccosh functions yield a result of zero; and the Arcsin function yields a real result.
 - 38 • When the parameter X has the value -1.0 , the Sqrt function yields the result
 - 39 • i (resp., $-i$), when the sign of the imaginary component of X is positive (resp., negative), if Complex_Types.Real'Signed_Zeros is True;
 - 40 • i , if Complex_Types.Real'Signed_Zeros is False;
 - 41/2 • When the parameter X has the value -1.0 , the Log function yields an imaginary result; and the Arcsin and Arccos functions yield a real result.
 - 42 • When the parameter X has the value $\pm i$, the Log function yields an imaginary result.
 - 43 • Exponentiation by a zero exponent yields the value one. Exponentiation by a unit exponent yields the value of the left operand (as a complex value). Exponentiation of the value one yields the value one. Exponentiation of the value zero yields the value zero.
- 44 Other accuracy requirements for the complex elementary functions, which apply only in the strict mode, are given in G.2.6.
- 45 The sign of a zero result or zero result component yielded by a complex elementary function is implementation defined when Complex_Types.Real'Signed_Zeros is True.

Implementation Permissions

- 46 The nongeneric equivalent packages may, but need not, be actual instantiations of the generic package with the appropriate predefined nongeneric equivalent of Numerics.Generic_Complex_Types; if they are, then the latter shall have been obtained by actual instantiation of Numerics.Generic_Complex_Types.

The exponentiation operator may be implemented in terms of the Exp and Log functions. Because this implementation yields poor accuracy in some parts of the domain, no accuracy requirement is imposed on complex exponentiation. 47

The implementation of the Exp function of a complex parameter X is allowed to raise the exception Constraint_Error, signaling overflow, when the real component of X exceeds an unspecified threshold that is approximately $\log(\text{Complex_Types.Real'Safe_Last})$. This permission recognizes the impracticality of avoiding overflow in the marginal case that the exponential of the real component of X exceeds the safe range of Complex_Types.Real but both components of the final result do not. Similarly, the Sin and Cos (resp., Sinh and Cosh) functions are allowed to raise the exception Constraint_Error, signaling overflow, when the absolute value of the imaginary (resp., real) component of the parameter X exceeds an unspecified threshold that is approximately $\log(\text{Complex_Types.Real'Safe_Last}) + \log(2.0)$. This permission recognizes the impracticality of avoiding overflow in the marginal case that the hyperbolic sine or cosine of the imaginary (resp., real) component of X exceeds the safe range of Complex_Types.Real but both components of the final result do not. 48

Implementation Advice

Implementations in which Complex_Types.Real'Signed_Zeros is True should attempt to provide a rational treatment of the signs of zero results and result components. For example, many of the complex elementary functions have components that are odd functions of one of the parameter components; in these cases, the result component should have the sign of the parameter component at the origin. Other complex elementary functions have zero components whose sign is opposite that of a parameter component at the origin, or is always positive or always negative. 49

G.1.3 Complex Input-Output

The generic package Text_IO.Complex_IO defines procedures for the formatted input and output of complex values. The generic actual parameter in an instantiation of Text_IO.Complex_IO is an instance of Numerics.Generic_Complex_Types for some floating point subtype. Exceptional conditions are reported by raising the appropriate exception defined in Text_IO. 1

Static Semantics

The generic library package Text_IO.Complex_IO has the following declaration: 2

```

with Ada.Numerics.Generic_Complex_Types;
generic
  with package Complex_Types is
    new Ada.Numerics.Generic_Complex_Types (<>);
package Ada.Text_IO.Complex_IO is
  use Complex_Types;
  Default_Fore : Field := 2;
  Default_Aft  : Field := Real'Digits - 1;
  Default_Exp  : Field := 3;
  procedure Get (File : in File_Type;
                Item : out Complex;
                Width : in Field := 0);
  procedure Get (Item : out Complex;
                Width : in Field := 0);

```

3 4 5 6

```

7      procedure Put (File : in File_Type;
                   Item : in Complex;
                   Fore : in Field := Default_Fore;
                   Aft  : in Field := Default_Aft;
                   Exp  : in Field := Default_Exp);
      procedure Put (Item : in Complex;
                   Fore : in Field := Default_Fore;
                   Aft  : in Field := Default_Aft;
                   Exp  : in Field := Default_Exp);
8
      procedure Get (From : in String;
                   Item : out Complex;
                   Last : out Positive);
      procedure Put (To   : out String;
                   Item : in Complex;
                   Aft  : in Field := Default_Aft;
                   Exp  : in Field := Default_Exp);
9
      end Ada.Text_IO.Complex_IO;

```

9.1/2 The library package `Complex_Text_IO` defines the same subprograms as `Text_IO.Complex_IO`, except that the predefined type `Float` is systematically substituted for `Real`, and the type `Numerics.Complex_Types.Complex` is systematically substituted for `Complex` throughout. Non-generic equivalents of `Text_IO.Complex_IO` corresponding to each of the other predefined floating point types are defined similarly, with the names `Short_Complex_Text_IO`, `Long_Complex_Text_IO`, etc.

10 The semantics of the `Get` and `Put` procedures are as follows:

```

11      procedure Get (File  : in File_Type;
                   Item  : out Complex;
                   Width : in Field := 0);
      procedure Get (Item  : out Complex;
                   Width : in Field := 0);

```

12/1 The input sequence is a pair of optionally signed real literals representing the real and imaginary components of a complex value. These components have the format defined for the corresponding `Get` procedure of an instance of `Text_IO.Float_IO` (see A.10.9) for the base subtype of `Complex_Types.Real`. The pair of components may be separated by a comma or surrounded by a pair of parentheses or both. Blanks are freely allowed before each of the components and before the parentheses and comma, if either is used. If the value of the parameter `Width` is zero, then

- 13 • line and page terminators are also allowed in these places;
- 14 • the components shall be separated by at least one blank or line terminator if the comma is omitted; and
- 15 • reading stops when the right parenthesis has been read, if the input sequence includes a left parenthesis, or when the imaginary component has been read, otherwise.

15.1 If a nonzero value of `Width` is supplied, then

- 16 • the components shall be separated by at least one blank if the comma is omitted; and
- 17 • exactly `Width` characters are read, or the characters (possibly none) up to a line terminator, whichever comes first (blanks are included in the count).

18 Returns, in the parameter `Item`, the value of type `Complex` that corresponds to the input sequence.

19 The exception `Text_IO.Data_Error` is raised if the input sequence does not have the required syntax or if the components of the complex value obtained are not of the base subtype of `Complex_Types.Real`.

```

procedure Put (File : in File_Type;                                20
               Item : in Complex;
               Fore : in Field := Default_Fore;
               Aft  : in Field := Default_Aft;
               Exp  : in Field := Default_Exp);
procedure Put (Item : in Complex;
               Fore : in Field := Default_Fore;
               Aft  : in Field := Default_Aft;
               Exp  : in Field := Default_Exp);

```

Outputs the value of the parameter Item as a pair of decimal literals representing the real and imaginary components of the complex value, using the syntax of an aggregate. More specifically,

- outputs a left parenthesis; 22
- outputs the value of the real component of the parameter Item with the format defined by the corresponding Put procedure of an instance of Text_IO.Float_IO for the base subtype of Complex_Types.Real, using the given values of Fore, Aft, and Exp; 23
- outputs a comma; 24
- outputs the value of the imaginary component of the parameter Item with the format defined by the corresponding Put procedure of an instance of Text_IO.Float_IO for the base subtype of Complex_Types.Real, using the given values of Fore, Aft, and Exp; 25
- outputs a right parenthesis. 26

```

procedure Get (From : in String;                                27
               Item : out Complex;
               Last : out Positive);

```

Reads a complex value from the beginning of the given string, following the same rule as the Get procedure that reads a complex value from a file, but treating the end of the string as a file terminator. Returns, in the parameter Item, the value of type Complex that corresponds to the input sequence. Returns in Last the index value such that From>Last is the last character read. 28/2

The exception Text_IO.Data_Error is raised if the input sequence does not have the required syntax or if the components of the complex value obtained are not of the base subtype of Complex_Types.Real. 29

```

procedure Put (To   : out String;                                30
               Item : in Complex;
               Aft  : in Field := Default_Aft;
               Exp  : in Field := Default_Exp);

```

Outputs the value of the parameter Item to the given string as a pair of decimal literals representing the real and imaginary components of the complex value, using the syntax of an aggregate. More specifically,

- a left parenthesis, the real component, and a comma are left justified in the given string, with the real component having the format defined by the Put procedure (for output to a file) of an instance of Text_IO.Float_IO for the base subtype of Complex_Types.Real, using a value of zero for Fore and the given values of Aft and Exp; 32
- the imaginary component and a right parenthesis are right justified in the given string, with the imaginary component having the format defined by the Put procedure (for output to a file) of an instance of Text_IO.Float_IO for the base subtype of Complex_Types.Real, using a value for Fore that completely fills the remainder of the string, together with the given values of Aft and Exp. 33

- 34 The exception `Text_IO.Layout_Error` is raised if the given string is too short to hold the formatted output.

Implementation Permissions

- 35 Other exceptions declared (by renaming) in `Text_IO` may be raised by the preceding procedures in the appropriate circumstances, as for the corresponding procedures of `Text_IO.Float_IO`.

G.1.4 The Package `Wide_Text_IO.Complex_IO`

Static Semantics

- 1 Implementations shall also provide the generic library package `Wide_Text_IO.Complex_IO`. Its declaration is obtained from that of `Text_IO.Complex_IO` by systematically replacing `Text_IO` by `Wide_Text_IO` and `String` by `Wide_String`; the description of its behavior is obtained by additionally replacing references to particular characters (commas, parentheses, etc.) by those for the corresponding wide characters.

G.1.5 The Package `Wide_Wide_Text_IO.Complex_IO`

Static Semantics

- 1/2 Implementations shall also provide the generic library package `Wide_Wide_Text_IO.Complex_IO`. Its declaration is obtained from that of `Text_IO.Complex_IO` by systematically replacing `Text_IO` by `Wide_Wide_Text_IO` and `String` by `Wide_Wide_String`; the description of its behavior is obtained by additionally replacing references to particular characters (commas, parentheses, etc.) by those for the corresponding wide wide characters.

G.2 Numeric Performance Requirements

Implementation Requirements

- 1 Implementations shall provide a user-selectable mode in which the accuracy and other numeric performance requirements detailed in the following subclauses are observed. This mode, referred to as the *strict mode*, may or may not be the default mode; it directly affects the results of the predefined arithmetic operations of real types and the results of the subprograms in children of the Numerics package, and indirectly affects the operations in other language defined packages. Implementations shall also provide the opposing mode, which is known as the *relaxed mode*.

Implementation Permissions

- 2 Either mode may be the default mode.
- 3 The two modes need not actually be different.

G.2.1 Model of Floating Point Arithmetic

In the strict mode, the predefined operations of a floating point type shall satisfy the accuracy requirements specified here and shall avoid or signal overflow in the situations described. This behavior is presented in terms of a model of floating point arithmetic that builds on the concept of the canonical form (see A.5.3).

Static Semantics

Associated with each floating point type is an infinite set of model numbers. The model numbers of a type are used to define the accuracy requirements that have to be satisfied by certain predefined operations of the type; through certain attributes of the model numbers, they are also used to explain the meaning of a user-declared floating point type declaration. The model numbers of a derived type are those of the parent type; the model numbers of a subtype are those of its type.

The *model numbers* of a floating point type T are zero and all the values expressible in the canonical form (for the type T), in which *mantissa* has T'Model_Mantissa digits and *exponent* has a value greater than or equal to T'Model_Emin. (These attributes are defined in G.2.2.)

A *model interval* of a floating point type is any interval whose bounds are model numbers of the type. The *model interval* of a type T *associated with a value v* is the smallest model interval of T that includes v. (The model interval associated with a model number of a type consists of that number only.)

Implementation Requirements

The accuracy requirements for the evaluation of certain predefined operations of floating point types are as follows.

An *operand interval* is the model interval, of the type specified for the operand of an operation, associated with the value of the operand.

For any predefined arithmetic operation that yields a result of a floating point type T, the required bounds on the result are given by a model interval of T (called the *result interval*) defined in terms of the operand values as follows:

- The result interval is the smallest model interval of T that includes the minimum and the maximum of all the values obtained by applying the (exact) mathematical operation to values arbitrarily selected from the respective operand intervals.

The result interval of an exponentiation is obtained by applying the above rule to the sequence of multiplications defined by the exponent, assuming arbitrary association of the factors, and to the final division in the case of a negative exponent.

The result interval of a conversion of a numeric value to a floating point type T is the model interval of T associated with the operand value, except when the source expression is of a fixed point type with a *small* that is not a power of T'Machine_Radix or is a fixed point multiplication or division either of whose operands has a *small* that is not a power of T'Machine_Radix; in these cases, the result interval is implementation defined.

For any of the foregoing operations, the implementation shall deliver a value that belongs to the result interval when both bounds of the result interval are in the safe range of the result type T, as determined by the values of T'Safe_First and T'Safe_Last; otherwise,

- if T'Machine_Overflows is True, the implementation shall either deliver a value that belongs to the result interval or raise Constraint_Error;

- 13 • if `TMachine_Overflows` is `False`, the result is implementation defined.
- 14 For any predefined relation on operands of a floating point type `T`, the implementation may deliver any value (i.e., either `True` or `False`) obtained by applying the (exact) mathematical comparison to values arbitrarily chosen from the respective operand intervals.
- 15 The result of a membership test is defined in terms of comparisons of the operand value with the lower and upper bounds of the given range or type mark (the usual rules apply to these comparisons).

Implementation Permissions

- 16 If the underlying floating point hardware implements division as multiplication by a reciprocal, the result interval for division (and exponentiation by a negative exponent) is implementation defined.

G.2.2 Model-Oriented Attributes of Floating Point Types

- 1 In implementations that support the Numerics Annex, the model-oriented attributes of floating point types shall yield the values defined here, in both the strict and the relaxed modes. These definitions add conditions to those in A.5.3.

Static Semantics

- 2 For every subtype `S` of a floating point type `T`:

3/2 `S'Model_Mantissa`

Yields the number of digits in the mantissa of the canonical form of the model numbers of `T` (see A.5.3). The value of this attribute shall be greater than or equal to

$$3.1/2 \quad \lceil d \cdot \log(10) / \log(TMachine_Radix) \rceil + g$$

3.2/2 where d is the requested decimal precision of `T`, and g is 0 if `TMachine_Radix` is a positive power of 10 and 1 otherwise. In addition, `TModel_Mantissa` shall be less than or equal to the value of `TMachine_Mantissa`. This attribute yields a value of the type *universal_integer*.

4 `S'Model_Emin`

Yields the minimum exponent of the canonical form of the model numbers of `T` (see A.5.3). The value of this attribute shall be greater than or equal to the value of `TMachine_Emin`. This attribute yields a value of the type *universal_integer*.

5 `S'Safe_First`

Yields the lower bound of the safe range of `T`. The value of this attribute shall be a model number of `T` and greater than or equal to the lower bound of the base range of `T`. In addition, if `T` is declared by a `floating_point_definition` or is derived from such a type, and the `floating_point_definition` includes a `real_range_specification` specifying a lower bound of lb , then the value of this attribute shall be less than or equal to lb ; otherwise, it shall be less than or equal to $-10.0^{4 \cdot d}$, where d is the requested decimal precision of `T`. This attribute yields a value of the type *universal_real*.

6 `S'Safe_Last`

Yields the upper bound of the safe range of `T`. The value of this attribute shall be a model number of `T` and less than or equal to the upper bound of the base range of `T`. In addition, if `T` is declared by a `floating_point_definition` or is derived from such a type, and the `floating_point_definition` includes a `real_range_specification` specifying an upper bound of ub , then the value of this attribute shall be greater than or equal to ub ; otherwise, it shall be greater than or equal to $10.0^{4 \cdot d}$, where d is the requested decimal precision of `T`. This attribute yields a value of the type *universal_real*.

S'Model Denotes a function (of a parameter X) whose specification is given in A.5.3. If X is a model number of T , the function yields X ; otherwise, it yields the value obtained by rounding or truncating X to either one of the adjacent model numbers of T . Constraint_Error is raised if the resulting model number is outside the safe range of S . A zero result has the sign of X when S'Signed_Zeros is True. 7

Subject to the constraints given above, the values of S'Model_Mantissa and S'Safe_Last are to be maximized, and the values of S'Model_Emin and S'Safe_First minimized, by the implementation as follows: 8

- First, S'Model_Mantissa is set to the largest value for which values of S'Model_Emin, S'Safe_First, and S'Safe_Last can be chosen so that the implementation satisfies the strict-mode requirements of G.2.1 in terms of the model numbers and safe range induced by these attributes. 9
- Next, S'Model_Emin is set to the smallest value for which values of S'Safe_First and S'Safe_Last can be chosen so that the implementation satisfies the strict-mode requirements of G.2.1 in terms of the model numbers and safe range induced by these attributes and the previously determined value of S'Model_Mantissa. 10
- Finally, S'Safe_First and S'Safe_Last are set (in either order) to the smallest and largest values, respectively, for which the implementation satisfies the strict-mode requirements of G.2.1 in terms of the model numbers and safe range induced by these attributes and the previously determined values of S'Model_Mantissa and S'Model_Emin. 11/3

G.2.3 Model of Fixed Point Arithmetic

In the strict mode, the predefined arithmetic operations of a fixed point type shall satisfy the accuracy requirements specified here and shall avoid or signal overflow in the situations described. 1

Implementation Requirements

The accuracy requirements for the predefined fixed point arithmetic operations and conversions, and the results of relations on fixed point operands, are given below. 2

The operands of the fixed point adding operators, absolute value, and comparisons have the same type. These operations are required to yield exact results, unless they overflow. 3

Multiplications and divisions are allowed between operands of any two fixed point types; the result has to be (implicitly or explicitly) converted to some other numeric type. For purposes of defining the accuracy rules, the multiplication or division and the conversion are treated as a single operation whose accuracy depends on three types (those of the operands and the result). For decimal fixed point types, the attribute T'Round may be used to imply explicit conversion with rounding (see 3.5.10). 4

When the result type is a floating point type, the accuracy is as given in G.2.1. For some combinations of the operand and result types in the remaining cases, the result is required to belong to a small set of values called the *perfect result set*; for other combinations, it is required merely to belong to a generally larger and implementation-defined set of values called the *close result set*. When the result type is a decimal fixed point type, the perfect result set contains a single value; thus, operations on decimal types are always fully specified. 5

When one operand of a fixed-fixed multiplication or division is of type *universal_real*, that operand is not implicitly converted in the usual sense, since the context does not determine a unique target type, but the accuracy of the result of the multiplication or division (i.e., whether the result has to belong to the perfect result set or merely the close result set) depends on the value of the operand of type *universal_real* and on the types of the other operand and of the result. 6

For a fixed point multiplication or division whose (exact) mathematical result is v , and for the conversion of a value v to a fixed point type, the perfect result set and close result set are defined as follows:

- If the result type is an ordinary fixed point type with a *small* of s ,
 - if v is an integer multiple of s , then the perfect result set contains only the value v ;
 - otherwise, it contains the integer multiple of s just below v and the integer multiple of s just above v .

The close result set is an implementation-defined set of consecutive integer multiples of s containing the perfect result set as a subset.

- If the result type is a decimal type with a *small* of s ,
 - if v is an integer multiple of s , then the perfect result set contains only the value v ;
 - otherwise, if truncation applies then it contains only the integer multiple of s in the direction toward zero, whereas if rounding applies then it contains only the nearest integer multiple of s (with ties broken by rounding away from zero).

The close result set is an implementation-defined set of consecutive integer multiples of s containing the perfect result set as a subset.

- If the result type is an integer type,
 - if v is an integer, then the perfect result set contains only the value v ;
 - otherwise, it contains the integer nearest to the value v (if v lies equally distant from two consecutive integers, the perfect result set contains the one that is further from zero).

The close result set is an implementation-defined set of consecutive integers containing the perfect result set as a subset.

The result of a fixed point multiplication or division shall belong either to the perfect result set or to the close result set, as described below, if overflow does not occur. In the following cases, if the result type is a fixed point type, let s be its *small*; otherwise, i.e. when the result type is an integer type, let s be 1.0.

- For a multiplication or division neither of whose operands is of type *universal_real*, let l and r be the *smalls* of the left and right operands. For a multiplication, if $(l \cdot r) / s$ is an integer or the reciprocal of an integer (the *smalls* are said to be “compatible” in this case), the result shall belong to the perfect result set; otherwise, it belongs to the close result set. For a division, if $l / (r \cdot s)$ is an integer or the reciprocal of an integer (i.e., the *smalls* are compatible), the result shall belong to the perfect result set; otherwise, it belongs to the close result set.
- For a multiplication or division having one *universal_real* operand with a value of v , note that it is always possible to factor v as an integer multiple of a “compatible” *small*, but the integer multiple may be “too big.” If there exists a factorization in which that multiple is less than some implementation-defined limit, the result shall belong to the perfect result set; otherwise, it belongs to the close result set.

A multiplication $P * Q$ of an operand of a fixed point type F by an operand of an integer type I , or vice-versa, and a division P / Q of an operand of a fixed point type F by an operand of an integer type I , are also allowed. In these cases, the result has a type of F ; explicit conversion of the result is never required. The accuracy required in these cases is the same as that required for a multiplication $F(P * Q)$ or a division $F(P / Q)$ obtained by interpreting the operand of the integer type to have a fixed point type with a *small* of 1.0.

The accuracy of the result of a conversion from an integer or fixed point type to a fixed point type, or from a fixed point type to an integer type, is the same as that of a fixed point multiplication of the source value by a fixed point operand having a *small* of 1.0 and a value of 1.0, as given by the foregoing rules. The result of a conversion from a floating point type to a fixed point type shall belong to the close result set.

The result of a conversion of a *universal_real* operand to a fixed point type shall belong to the perfect result set.

The possibility of overflow in the result of a predefined arithmetic operation or conversion yielding a result of a fixed point type T is analogous to that for floating point types, except for being related to the base range instead of the safe range. If all of the permitted results belong to the base range of T, then the implementation shall deliver one of the permitted results; otherwise,

- if T'Machine_Overflows is True, the implementation shall either deliver one of the permitted results or raise Constraint_Error;
- if T'Machine_Overflows is False, the result is implementation defined.

G.2.4 Accuracy Requirements for the Elementary Functions

In the strict mode, the performance of Numerics.Generic_Elementary_Functions shall be as specified here.

Implementation Requirements

When an exception is not raised, the result of evaluating a function in an instance *EF* of Numerics.Generic_Elementary_Functions belongs to a *result interval*, defined as the smallest model interval of *EF.Float_Type* that contains all the values of the form $f \cdot (1.0 + d)$, where f is the exact value of the corresponding mathematical function at the given parameter values, d is a real number, and $|d|$ is less than or equal to the function's *maximum relative error*. The function delivers a value that belongs to the result interval when both of its bounds belong to the safe range of *EF.Float_Type*; otherwise,

- if *EF.Float_Type*'Machine_Overflows is True, the function either delivers a value that belongs to the result interval or raises Constraint_Error, signaling overflow;
- if *EF.Float_Type*'Machine_Overflows is False, the result is implementation defined.

The maximum relative error exhibited by each function is as follows:

- $2.0 \cdot EF.Float_Type'Model_Epsilon$, in the case of the Sqrt, Sin, and Cos functions;
- $4.0 \cdot EF.Float_Type'Model_Epsilon$, in the case of the Log, Exp, Tan, Cot, and inverse trigonometric functions; and
- $8.0 \cdot EF.Float_Type'Model_Epsilon$, in the case of the forward and inverse hyperbolic functions.

The maximum relative error exhibited by the exponentiation operator, which depends on the values of the operands, is $(4.0 + |Right \cdot \log(Left)| / 32.0) \cdot EF.Float_Type'Model_Epsilon$.

The maximum relative error given above applies throughout the domain of the forward trigonometric functions when the Cycle parameter is specified. When the Cycle parameter is omitted, the maximum relative error given above applies only when the absolute value of the angle parameter X is less than or equal to some implementation-defined *angle threshold*, which shall be at least $EF.Float_Type'Machine_Radix^{\lfloor EF.Float_Type'Machine_Mantissa/2 \rfloor}$. Beyond the angle threshold, the accuracy of the forward trigonometric functions is implementation defined.

The prescribed results specified in A.5.1 for certain functions at particular parameter values take precedence over the maximum relative error bounds; effectively, they narrow to a single value the result interval allowed by the maximum relative error bounds. Additional rules with a similar effect are given by table G-1 for the inverse trigonometric functions, at particular parameter values for which the mathematical result is possibly not a model number of *EF.Float_Type* (or is, indeed, even transcendental). In each table entry, the values of the parameters are such that the result lies on the axis between two

quadrants; the corresponding accuracy rule, which takes precedence over the maximum relative error bounds, is that the result interval is the model interval of *EF.Float_Type* associated with the exact mathematical result given in the table.

12/1 *This paragraph was deleted.*

- 13 The last line of the table is meant to apply when *EF.Float_Type*'Signed_Zeros is False; the two lines just above it, when *EF.Float_Type*'Signed_Zeros is True and the parameter Y has a zero value with the indicated sign.

Table G-1: Tightly Approximated Elementary Function Results				
Function	Value of X	Value of Y	Exact Result when Cycle Specified	Exact Result when Cycle Omitted
Arcsin	1.0	n.a.	Cycle/4.0	$\pi/2.0$
Arcsin	-1.0	n.a.	-Cycle/4.0	$-\pi/2.0$
Arccos	0.0	n.a.	Cycle/4.0	$\pi/2.0$
Arccos	-1.0	n.a.	Cycle/2.0	π
Arctan and Arccot	0.0	positive	Cycle/4.0	$\pi/2.0$
Arctan and Arccot	0.0	negative	-Cycle/4.0	$-\pi/2.0$
Arctan and Arccot	negative	+0.0	Cycle/2.0	π
Arctan and Arccot	negative	-0.0	-Cycle/2.0	$-\pi$
Arctan and Arccot	negative	0.0	Cycle/2.0	π

- 14 The amount by which the result of an inverse trigonometric function is allowed to spill over into a quadrant adjacent to the one corresponding to the principal branch, as given in A.5.1, is limited. The rule is that the result belongs to the smallest model interval of *EF.Float_Type* that contains both boundaries of the quadrant corresponding to the principal branch. This rule also takes precedence over the maximum relative error bounds, effectively narrowing the result interval allowed by them.
- 15 Finally, the following specifications also take precedence over the maximum relative error bounds:
- 16 • The absolute value of the result of the Sin, Cos, and Tanh functions never exceeds one.
 - 17 • The absolute value of the result of the Coth function is never less than one.
 - 18 • The result of the Cosh function is never less than one.

Implementation Advice

- 19 The versions of the forward trigonometric functions without a Cycle parameter should not be implemented by calling the corresponding version with a Cycle parameter of $2.0 * \text{Numerics.Pi}$, since this will not provide the required accuracy in some portions of the domain. For the same reason, the version of Log without a Base parameter should not be implemented by calling the corresponding version with a Base parameter of Numerics.e.

G.2.5 Performance Requirements for Random Number Generation

In the strict mode, the performance of `Numerics.Float_Random` and `Numerics.Discrete_Random` shall be as specified here.

Implementation Requirements

Two different calls to the time-dependent `Reset` procedure shall reset the generator to different states, provided that the calls are separated in time by at least one second and not more than fifty years.

The implementation's representations of generator states and its algorithms for generating random numbers shall yield a period of at least $2^{31}-2$; much longer periods are desirable but not required.

The implementations of `Numerics.Float_Random.Random` and `Numerics.Discrete_Random.Random` shall pass at least 85% of the individual trials in a suite of statistical tests. For `Numerics.Float_Random`, the tests are applied directly to the floating point values generated (i.e., they are not converted to integers first), while for `Numerics.Discrete_Random` they are applied to the generated values of various discrete types. Each test suite performs 6 different tests, with each test repeated 10 times, yielding a total of 60 individual trials. An individual trial is deemed to pass if the chi-square value (or other statistic) calculated for the observed counts or distribution falls within the range of values corresponding to the 2.5 and 97.5 percentage points for the relevant degrees of freedom (i.e., it shall be neither too high nor too low). For the purpose of determining the degrees of freedom, measurement categories are combined whenever the expected counts are fewer than 5.

G.2.6 Accuracy Requirements for Complex Arithmetic

In the strict mode, the performance of `Numerics.Generic_Complex_Types` and `Numerics.Generic_Complex_Elementary_Functions` shall be as specified here.

Implementation Requirements

When an exception is not raised, the result of evaluating a real function of an instance *CT* of `Numerics.Generic_Complex_Types` (i.e., a function that yields a value of subtype *CT.Real*'Base or *CT.Imaginary*) belongs to a result interval defined as for a real elementary function (see G.2.4).

When an exception is not raised, each component of the result of evaluating a complex function of such an instance, or of an instance of `Numerics.Generic_Complex_Elementary_Functions` obtained by instantiating the latter with *CT* (i.e., a function that yields a value of subtype *CT.Complex*), also belongs to a *result interval*. The result intervals for the components of the result are either defined by a *maximum relative error* bound or by a *maximum box error* bound. When the result interval for the real (resp., imaginary) component is defined by maximum relative error, it is defined as for that of a real function, relative to the exact value of the real (resp., imaginary) part of the result of the corresponding mathematical function. When defined by maximum box error, the result interval for a component of the result is the smallest model interval of *CT.Real* that contains all the values of the corresponding part of $f \cdot (1.0 + d)$, where f is the exact complex value of the corresponding mathematical function at the given parameter values, d is complex, and $|d|$ is less than or equal to the given maximum box error. The function delivers a value that belongs to the result interval (or a value both of whose components belong to their respective result intervals) when both bounds of the result interval(s) belong to the safe range of *CT.Real*; otherwise,

- 4 • if *CT.Real'Machine_Overflows* is True, the function either delivers a value that belongs to the result interval (or a value both of whose components belong to their respective result intervals) or raises *Constraint_Error*, signaling overflow;
- 5 • if *CT.Real'Machine_Overflows* is False, the result is implementation defined.

6/2 The error bounds for particular complex functions are tabulated in table G-2. In the table, the error bound is given as the coefficient of *CT.Real'Model_Epsilon*.

7/1 *This paragraph was deleted.*

Table G-2: Error Bounds for Particular Complex Functions			
Function or Operator	Nature of Result	Nature of Bound	Error Bound
Modulus	real	max. rel. error	3.0
Argument	real	max. rel. error	4.0
Compose_From_Polar	complex	max. rel. error	3.0
"*" (both operands complex)	complex	max. box error	5.0
"/" (right operand complex)	complex	max. box error	13.0
Sqrt	complex	max. rel. error	6.0
Log	complex	max. box error	13.0
Exp (complex parameter)	complex	max. rel. error	7.0
Exp (imaginary parameter)	complex	max. rel. error	2.0
Sin, Cos, Sinh, and Cosh	complex	max. rel. error	11.0
Tan, Cot, Tanh, and Coth	complex	max. rel. error	35.0
inverse trigonometric	complex	max. rel. error	14.0
inverse hyperbolic	complex	max. rel. error	14.0

- 8 The maximum relative error given above applies throughout the domain of the *Compose_From_Polar* function when the *Cycle* parameter is specified. When the *Cycle* parameter is omitted, the maximum relative error applies only when the absolute value of the parameter *Argument* is less than or equal to the angle threshold (see G.2.4). For the *Exp* function, and for the forward hyperbolic (resp., trigonometric) functions, the maximum relative error given above likewise applies only when the absolute value of the imaginary (resp., real) component of the parameter *X* (or the absolute value of the parameter itself, in the case of the *Exp* function with a parameter of pure-imaginary type) is less than or equal to the angle threshold. For larger angles, the accuracy is implementation defined.
- 9 The prescribed results specified in G.1.2 for certain functions at particular parameter values take precedence over the error bounds; effectively, they narrow to a single value the result interval allowed by the error bounds for a component of the result. Additional rules with a similar effect are given below for certain inverse trigonometric and inverse hyperbolic functions, at particular parameter values for which a component of the mathematical result is transcendental. In each case, the accuracy rule, which takes precedence over the error bounds, is that the result interval for the stated result component is the model interval of *CT.Real* associated with the component's exact mathematical value. The cases in question are as follows:

- When the parameter *X* has the value zero, the real (resp., imaginary) component of the result of the *Arccot* (resp., *Arccoth*) function is in the model interval of *CT.Real* associated with the value $\pi/2.0$. 10
- When the parameter *X* has the value one, the real component of the result of the *Arcsin* function is in the model interval of *CT.Real* associated with the value $\pi/2.0$. 11
- When the parameter *X* has the value -1.0 , the real component of the result of the *Arcsin* (resp., *Arccos*) function is in the model interval of *CT.Real* associated with the value $-\pi/2.0$ (resp., π). 12

The amount by which a component of the result of an inverse trigonometric or inverse hyperbolic function is allowed to spill over into a quadrant adjacent to the one corresponding to the principal branch, as given in G.1.2, is limited. The rule is that the result belongs to the smallest model interval of *CT.Real* that contains both boundaries of the quadrant corresponding to the principal branch. This rule also takes precedence over the maximum error bounds, effectively narrowing the result interval allowed by them. 13/2

Finally, the results allowed by the error bounds are narrowed by one further rule: The absolute value of each component of the result of the *Exp* function, for a pure-imaginary parameter, never exceeds one. 14

Implementation Advice

The version of the *Compose_From_Polar* function without a *Cycle* parameter should not be implemented by calling the corresponding version with a *Cycle* parameter of $2.0 * \text{Numerics.Pi}$, since this will not provide the required accuracy in some portions of the domain. 15

G.3 Vector and Matrix Manipulation

Types and operations for the manipulation of real vectors and matrices are provided in *Generic_Real_Arrays*, which is defined in G.3.1. Types and operations for the manipulation of complex vectors and matrices are provided in *Generic_Complex_Arrays*, which is defined in G.3.2. Both of these library units are generic children of the predefined package *Numerics* (see A.5). Nongeneric equivalents of these packages for each of the predefined floating point types are also provided as children of *Numerics*. 1/2

G.3.1 Real Vectors and Matrices

Static Semantics

The generic library package *Numerics.Generic_Real_Arrays* has the following declaration: 1/2

```

generic
  type Real is digits <>;
package Ada.Numerics.Generic_Real_Arrays is
  pragma Pure(Generic_Real_Arrays);
  -- Types
  type Real_Vector is array (Integer range <>) of Real'Base;
  type Real_Matrix is array (Integer range <>, Integer range <>)
                                of Real'Base;
  -- Subprograms for Real_Vector types
  -- Real_Vector arithmetic operations
  function "+" (Right : Real_Vector) return Real_Vector;
  function "-" (Right : Real_Vector) return Real_Vector;
  function "abs" (Right : Real_Vector) return Real_Vector;
  function "+" (Left, Right : Real_Vector) return Real_Vector;
  function "-" (Left, Right : Real_Vector) return Real_Vector;
  function "*" (Left, Right : Real_Vector) return Real'Base;
```

```

10/2      function "abs" (Right : Real_Vector)          return Real'Base;
11/2      -- Real_Vector scaling operations
12/2      function "*" (Left : Real'Base;   Right : Real_Vector)
           return Real_Vector;
           function "*" (Left : Real_Vector; Right : Real'Base)
           return Real_Vector;
           function "/" (Left : Real_Vector; Right : Real'Base)
           return Real_Vector;
13/2      -- Other Real_Vector operations
14/2      function Unit_Vector (Index : Integer;
                               Order : Positive;
                               First : Integer := 1) return Real_Vector;
15/2      -- Subprograms for Real_Matrix types
16/2      -- Real_Matrix arithmetic operations
17/2      function "+" (Right : Real_Matrix) return Real_Matrix;
           function "-" (Right : Real_Matrix) return Real_Matrix;
           function "abs" (Right : Real_Matrix) return Real_Matrix;
           function Transpose (X : Real_Matrix) return Real_Matrix;
18/2      function "+" (Left, Right : Real_Matrix) return Real_Matrix;
           function "-" (Left, Right : Real_Matrix) return Real_Matrix;
           function "*" (Left, Right : Real_Matrix) return Real_Matrix;
19/2      function "*" (Left, Right : Real_Vector) return Real_Matrix;
20/2      function "*" (Left : Real_Vector; Right : Real_Matrix)
           return Real_Vector;
           function "*" (Left : Real_Matrix; Right : Real_Vector)
           return Real_Vector;
21/2      -- Real_Matrix scaling operations
22/2      function "*" (Left : Real'Base;   Right : Real_Matrix)
           return Real_Matrix;
           function "*" (Left : Real_Matrix; Right : Real'Base)
           return Real_Matrix;
           function "/" (Left : Real_Matrix; Right : Real'Base)
           return Real_Matrix;
23/2      -- Real_Matrix inversion and related operations
24/2      function Solve (A : Real_Matrix; X : Real_Vector) return Real_Vector;
           function Solve (A, X : Real_Matrix) return Real_Matrix;
           function Inverse (A : Real_Matrix) return Real_Matrix;
           function Determinant (A : Real_Matrix) return Real'Base;
25/2      -- Eigenvalues and vectors of a real symmetric matrix
26/2      function Eigenvalues (A : Real_Matrix) return Real_Vector;
27/2      procedure Eigensystem (A : in Real_Matrix;
                               Values : out Real_Vector;
                               Vectors : out Real_Matrix);
28/2      -- Other Real_Matrix operations
29/2      function Unit_Matrix (Order : Positive;
                               First_1, First_2 : Integer := 1)
                               return Real_Matrix;
30/2      end Ada.Numerics.Generic_Real_Arrays;

```

31/2 The library package `Numerics.Real_Arrays` is declared pure and defines the same types and subprograms as `Numerics.Generic_Real_Arrays`, except that the predefined type `Float` is systematically substituted for `Real'Base` throughout. Nongeneric equivalents for each of the other predefined floating point types are defined similarly, with the names `Numerics.Short_Real_Arrays`, `Numerics.Long_Real_Arrays`, etc.

32/2 Two types are defined and exported by `Numerics.Generic_Real_Arrays`. The composite type `Real_Vector` is provided to represent a vector with components of type `Real`; it is defined as an unconstrained, one-

dimensional array with an index of type Integer. The composite type Real_Matrix is provided to represent a matrix with components of type Real; it is defined as an unconstrained, two-dimensional array with indices of type Integer.

The effect of the various subprograms is as described below. In most cases the subprograms are described in terms of corresponding scalar operations of the type Real; any exception raised by those operations is propagated by the array operation. Moreover, the accuracy of the result for each individual component is as defined for the scalar operation unless stated otherwise. 33/2

In the case of those operations which are defined to *involve an inner product*, Constraint_Error may be raised if an intermediate result is outside the range of Real'Base even though the mathematical final result would not be. 34/2

```
function "+" (Right : Real_Vector) return Real_Vector; 35/2
function "-" (Right : Real_Vector) return Real_Vector;
function "abs" (Right : Real_Vector) return Real_Vector;
```

Each operation returns the result of applying the corresponding operation of the type Real to each component of Right. The index range of the result is Right'Range. 36/2

```
function "+" (Left, Right : Real_Vector) return Real_Vector; 37/2
function "-" (Left, Right : Real_Vector) return Real_Vector;
```

Each operation returns the result of applying the corresponding operation of the type Real to each component of Left and the matching component of Right. The index range of the result is Left'Range. Constraint_Error is raised if Left'Length is not equal to Right'Length. 38/2

```
function "*" (Left, Right : Real_Vector) return Real'Base; 39/2
```

This operation returns the inner product of Left and Right. Constraint_Error is raised if Left'Length is not equal to Right'Length. This operation involves an inner product. 40/2

```
function "abs" (Right : Real_Vector) return Real'Base; 41/2
```

This operation returns the L2-norm of Right (the square root of the inner product of the vector with itself). 42/2

```
function "*" (Left : Real'Base; Right : Real_Vector) return Real_Vector; 43/2
```

This operation returns the result of multiplying each component of Right by the scalar Left using the "*" operation of the type Real. The index range of the result is Right'Range. 44/2

```
function "*" (Left : Real_Vector; Right : Real'Base) return Real_Vector; 45/2
function "/" (Left : Real_Vector; Right : Real'Base) return Real_Vector;
```

Each operation returns the result of applying the corresponding operation of the type Real to each component of Left and to the scalar Right. The index range of the result is Left'Range. 46/2

```
function Unit_Vector (Index : Integer; 47/2
                    Order : Positive;
                    First : Integer := 1) return Real_Vector;
```

This function returns a *unit vector* with Order components and a lower bound of First. All components are set to 0.0 except for the Index component which is set to 1.0. Constraint_Error is raised if Index < First, Index > First + Order – 1 or if First + Order – 1 > Integer'Last. 48/2

```
function "+" (Right : Real_Matrix) return Real_Matrix; 49/2
function "-" (Right : Real_Matrix) return Real_Matrix;
function "abs" (Right : Real_Matrix) return Real_Matrix;
```

Each operation returns the result of applying the corresponding operation of the type Real to each component of Right. The index ranges of the result are those of Right. 50/2

51/2 **function** Transpose (X : Real_Matrix) **return** Real_Matrix;

52/2 This function returns the transpose of a matrix X. The first and second index ranges of the result are X'Range(2) and X'Range(1) respectively.

53/2 **function** "+" (Left, Right : Real_Matrix) **return** Real_Matrix;
function "-" (Left, Right : Real_Matrix) **return** Real_Matrix;

54/2 Each operation returns the result of applying the corresponding operation of the type Real to each component of Left and the matching component of Right. The index ranges of the result are those of Left. Constraint_Error is raised if Left'Length(1) is not equal to Right'Length(1) or Left'Length(2) is not equal to Right'Length(2).

55/2 **function** "*" (Left, Right : Real_Matrix) **return** Real_Matrix;

56/2 This operation provides the standard mathematical operation for matrix multiplication. The first and second index ranges of the result are Left'Range(1) and Right'Range(2) respectively. Constraint_Error is raised if Left'Length(2) is not equal to Right'Length(1). This operation involves inner products.

57/2 **function** "*" (Left, Right : Real_Vector) **return** Real_Matrix;

58/2 This operation returns the outer product of a (column) vector Left by a (row) vector Right using the operation "*" of the type Real for computing the individual components. The first and second index ranges of the result are Left'Range and Right'Range respectively.

59/2 **function** "*" (Left : Real_Vector; Right : Real_Matrix) **return** Real_Vector;

60/2 This operation provides the standard mathematical operation for multiplication of a (row) vector Left by a matrix Right. The index range of the (row) vector result is Right'Range(2). Constraint_Error is raised if Left'Length is not equal to Right'Length(1). This operation involves inner products.

61/2 **function** "*" (Left : Real_Matrix; Right : Real_Vector) **return** Real_Vector;

62/2 This operation provides the standard mathematical operation for multiplication of a matrix Left by a (column) vector Right. The index range of the (column) vector result is Left'Range(1). Constraint_Error is raised if Left'Length(2) is not equal to Right'Length. This operation involves inner products.

63/2 **function** "*" (Left : Real'Base; Right : Real_Matrix) **return** Real_Matrix;

64/2 This operation returns the result of multiplying each component of Right by the scalar Left using the "*" operation of the type Real. The index ranges of the result are those of Right.

65/2 **function** "*" (Left : Real_Matrix; Right : Real'Base) **return** Real_Matrix;
function "/" (Left : Real_Matrix; Right : Real'Base) **return** Real_Matrix;

66/2 Each operation returns the result of applying the corresponding operation of the type Real to each component of Left and to the scalar Right. The index ranges of the result are those of Left.

67/2 **function** Solve (A : Real_Matrix; X : Real_Vector) **return** Real_Vector;

68/2 This function returns a vector Y such that X is (nearly) equal to A * Y. This is the standard mathematical operation for solving a single set of linear equations. The index range of the result is A'Range(2). Constraint_Error is raised if A'Length(1), A'Length(2), and X'Length are not equal. Constraint_Error is raised if the matrix A is ill-conditioned.

69/2 **function** Solve (A, X : Real_Matrix) **return** Real_Matrix;

70/2 This function returns a matrix Y such that X is (nearly) equal to A * Y. This is the standard mathematical operation for solving several sets of linear equations. The index ranges of the

result are A'Range(2) and X'Range(2). Constraint_Error is raised if A'Length(1), A'Length(2), and X'Length(1) are not equal. Constraint_Error is raised if the matrix A is ill-conditioned.

function Inverse (A : Real_Matrix) **return** Real_Matrix; 71/2

This function returns a matrix B such that A * B is (nearly) equal to the unit matrix. The index ranges of the result are A'Range(2) and A'Range(1). Constraint_Error is raised if A'Length(1) is not equal to A'Length(2). Constraint_Error is raised if the matrix A is ill-conditioned. 72/2

function Determinant (A : Real_Matrix) **return** Real_Base; 73/2

This function returns the determinant of the matrix A. Constraint_Error is raised if A'Length(1) is not equal to A'Length(2). 74/2

function Eigenvalues(A : Real_Matrix) **return** Real_Vector; 75/2

This function returns the eigenvalues of the symmetric matrix A as a vector sorted into order with the largest first. Constraint_Error is raised if A'Length(1) is not equal to A'Length(2). The index range of the result is A'Range(1). Argument_Error is raised if the matrix A is not symmetric. 76/2

procedure Eigensystem(A : **in** Real_Matrix;
Values : **out** Real_Vector;
Vectors : **out** Real_Matrix); 77/2

This procedure computes both the eigenvalues and eigenvectors of the symmetric matrix A. The out parameter Values is the same as that obtained by calling the function Eigenvalues. The out parameter Vectors is a matrix whose columns are the eigenvectors of the matrix A. The order of the columns corresponds to the order of the eigenvalues. The eigenvectors are normalized and mutually orthogonal (they are orthonormal), including when there are repeated eigenvalues. Constraint_Error is raised if A'Length(1) is not equal to A'Length(2), or if Values'Range is not equal to A'Range(1), or if the index ranges of the parameter Vectors are not equal to those of A. Argument_Error is raised if the matrix A is not symmetric. Constraint_Error is also raised in implementation-defined circumstances if the algorithm used does not converge quickly enough. 78/3

function Unit_Matrix (Order : Positive;
First_1, First_2 : Integer := 1) **return** Real_Matrix; 79/2

This function returns a square *unit matrix* with Order**2 components and lower bounds of First_1 and First_2 (for the first and second index ranges respectively). All components are set to 0.0 except for the main diagonal, whose components are set to 1.0. Constraint_Error is raised if First_1 + Order - 1 > Integer'Last or First_2 + Order - 1 > Integer'Last. 80/2

Implementation Requirements

Accuracy requirements for the subprograms Solve, Inverse, Determinant, Eigenvalues and Eigensystem are implementation defined. 81/2

For operations not involving an inner product, the accuracy requirements are those of the corresponding operations of the type Real in both the strict mode and the relaxed mode (see G.2). 82/2

For operations involving an inner product, no requirements are specified in the relaxed mode. In the strict mode the modulus of the absolute error of the inner product $X*Y$ shall not exceed $g*\text{abs}(X)*\text{abs}(Y)$ where g is defined as 83/2

$g = X'Length * \text{Real'Machine_Radix}^{**}(1 - \text{Real'Model_Mantissa})$ 84/2

For the L2-norm, no accuracy requirements are specified in the relaxed mode. In the strict mode the relative error on the norm shall not exceed $g / 2.0 + 3.0 * \text{Real'Model_Epsilon}$ where g is defined as above. 85/2

Documentation Requirements

- 86/2 Implementations shall document any techniques used to reduce cancellation errors such as extended precision arithmetic.

Implementation Permissions

- 87/2 The nongeneric equivalent packages may, but need not, be actual instantiations of the generic package for the appropriate predefined type.

Implementation Advice

- 88/2 Implementations should implement the Solve and Inverse functions using established techniques such as LU decomposition with row interchanges followed by back and forward substitution. Implementations are recommended to refine the result by performing an iteration on the residuals; if this is done then it should be documented.
- 89/2 It is not the intention that any special provision should be made to determine whether a matrix is ill-conditioned or not. The naturally occurring overflow (including division by zero) which will result from executing these functions with an ill-conditioned matrix and thus raise `Constraint_Error` is sufficient.
- 90/2 The test that a matrix is symmetric should be performed by using the equality operator to compare the relevant components.
- 91/3 An implementation should minimize the circumstances under which the algorithm used for Eigenvalues and Eigensystem fails to converge.

G.3.2 Complex Vectors and Matrices

Static Semantics

- 1/2 The generic library package `Numerics.Generic_Complex_Arrays` has the following declaration:
- ```

2/2 with Ada.Numerics.Generic_Real_Arrays, Ada.Numerics.Generic_Complex_Types;
 generic
 with package Real_Arrays is new
 Ada.Numerics.Generic_Real_Arrays (<>);
 use Real_Arrays;
 with package Complex_Types is new
 Ada.Numerics.Generic_Complex_Types (Real);
 use Complex_Types;
 package Ada.Numerics.Generic_Complex_Arrays is
 pragma Pure (Generic_Complex_Arrays);
3/2 -- Types
4/2 type Complex_Vector is array (Integer range <>) of Complex;
 type Complex_Matrix is array (Integer range <>,
 Integer range <>) of Complex;
5/2 -- Subprograms for Complex_Vector types
6/2 -- Complex_Vector selection, conversion and composition operations
7/2 function Re (X : Complex_Vector) return Real_Vector;
 function Im (X : Complex_Vector) return Real_Vector;
8/2 procedure Set_Re (X : in out Complex_Vector;
 Re : in Real_Vector);
 procedure Set_Im (X : in out Complex_Vector;
 Im : in Real_Vector);
9/2 function Compose_From_Cartesian (Re : Real_Vector)
 return Complex_Vector;
 function Compose_From_Cartesian (Re, Im : Real_Vector)
 return Complex_Vector;
```

```

function Modulus (X : Complex_Vector) return Real_Vector;
function "abs" (Right : Complex_Vector) return Real_Vector
 renames Modulus;
function Argument (X : Complex_Vector) return Real_Vector;
function Argument (X : Complex_Vector;
 Cycle : Real'Base) return Real_Vector;

function Compose_From_Polar (Modulus, Argument : Real_Vector)
 return Complex_Vector;
function Compose_From_Polar (Modulus, Argument : Real_Vector;
 Cycle : Real'Base)
 return Complex_Vector;
-- Complex_Vector arithmetic operations
function "+" (Right : Complex_Vector) return Complex_Vector;
function "-" (Right : Complex_Vector) return Complex_Vector;
function Conjugate (X : Complex_Vector) return Complex_Vector;
function "+" (Left, Right : Complex_Vector) return Complex_Vector;
function "-" (Left, Right : Complex_Vector) return Complex_Vector;
function "*" (Left, Right : Complex_Vector) return Complex;
function "abs" (Right : Complex_Vector) return Real'Base;
-- Mixed Real_Vector and Complex_Vector arithmetic operations
function "+" (Left : Real_Vector;
 Right : Complex_Vector) return Complex_Vector;
function "+" (Left : Complex_Vector;
 Right : Real_Vector) return Complex_Vector;
function "-" (Left : Real_Vector;
 Right : Complex_Vector) return Complex_Vector;
function "-" (Left : Complex_Vector;
 Right : Real_Vector) return Complex_Vector;
function "*" (Left : Real_Vector; Right : Complex_Vector)
 return Complex;
function "*" (Left : Complex_Vector; Right : Real_Vector)
 return Complex;
-- Complex_Vector scaling operations
function "*" (Left : Complex;
 Right : Complex_Vector) return Complex_Vector;
function "*" (Left : Complex_Vector;
 Right : Complex) return Complex_Vector;
function "/" (Left : Complex_Vector;
 Right : Complex) return Complex_Vector;
function "*" (Left : Real'Base;
 Right : Complex_Vector) return Complex_Vector;
function "*" (Left : Complex_Vector;
 Right : Real'Base) return Complex_Vector;
function "/" (Left : Complex_Vector;
 Right : Real'Base) return Complex_Vector;
-- Other Complex_Vector operations
function Unit_Vector (Index : Integer;
 Order : Positive;
 First : Integer := 1) return Complex_Vector;
-- Subprograms for Complex_Matrix types
-- Complex_Matrix selection, conversion and composition operations
function Re (X : Complex_Matrix) return Real_Matrix;
function Im (X : Complex_Matrix) return Real_Matrix;
procedure Set_Re (X : in out Complex_Matrix;
 Re : in Real_Matrix);
procedure Set_Im (X : in out Complex_Matrix;
 Im : in Real_Matrix);

```

```

29/2 function Compose_From_Cartesian (Re : Real_Matrix)
 return Complex_Matrix;
 function Compose_From_Cartesian (Re, Im : Real_Matrix)
 return Complex_Matrix;

30/2 function Modulus (X : Complex_Matrix) return Real_Matrix;
 function "abs" (Right : Complex_Matrix) return Real_Matrix;
 renames Modulus;

31/2 function Argument (X : Complex_Matrix) return Real_Matrix;
 function Argument (X : Complex_Matrix;
 Cycle : Real'Base) return Real_Matrix;

32/2 function Compose_From_Polar (Modulus, Argument : Real_Matrix)
 return Complex_Matrix;
 function Compose_From_Polar (Modulus, Argument : Real_Matrix;
 Cycle : Real'Base)
 return Complex_Matrix;

33/2 -- Complex_Matrix arithmetic operations

34/2 function "+" (Right : Complex_Matrix) return Complex_Matrix;
 function "-" (Right : Complex_Matrix) return Complex_Matrix;
 function Conjugate (X : Complex_Matrix) return Complex_Matrix;
 function Transpose (X : Complex_Matrix) return Complex_Matrix;

35/2 function "+" (Left, Right : Complex_Matrix) return Complex_Matrix;
 function "-" (Left, Right : Complex_Matrix) return Complex_Matrix;
 function "*" (Left, Right : Complex_Matrix) return Complex_Matrix;

36/2 function "*" (Left, Right : Complex_Vector) return Complex_Matrix;

37/2 function "*" (Left : Complex_Vector;
 Right : Complex_Matrix) return Complex_Vector;
 function "*" (Left : Complex_Matrix;
 Right : Complex_Vector) return Complex_Vector;

38/2 -- Mixed Real_Matrix and Complex_Matrix arithmetic operations

39/2 function "+" (Left : Real_Matrix;
 Right : Complex_Matrix) return Complex_Matrix;
 function "-" (Left : Real_Matrix;
 Right : Complex_Matrix) return Complex_Matrix;
 function "+" (Left : Real_Matrix;
 Right : Real_Matrix) return Complex_Matrix;
 function "-" (Left : Real_Matrix;
 Right : Real_Matrix) return Complex_Matrix;
 function "*" (Left : Real_Matrix;
 Right : Complex_Matrix) return Complex_Matrix;
 function "*" (Left : Complex_Matrix;
 Right : Real_Matrix) return Complex_Matrix;

40/2 function "*" (Left : Real_Vector;
 Right : Complex_Vector) return Complex_Matrix;
 function "*" (Left : Complex_Vector;
 Right : Real_Vector) return Complex_Matrix;

41/2 function "*" (Left : Real_Vector;
 Right : Complex_Matrix) return Complex_Vector;
 function "*" (Left : Complex_Vector;
 Right : Real_Matrix) return Complex_Vector;
 function "*" (Left : Real_Matrix;
 Right : Complex_Vector) return Complex_Vector;
 function "*" (Left : Complex_Matrix;
 Right : Real_Vector) return Complex_Vector;

42/2 -- Complex_Matrix scaling operations

43/2 function "*" (Left : Complex;
 Right : Complex_Matrix) return Complex_Matrix;
 function "*" (Left : Complex_Matrix;
 Right : Complex) return Complex_Matrix;
 function "/" (Left : Complex_Matrix;
 Right : Complex) return Complex_Matrix;

```

```

function "*" (Left : Real'Base; 44/2
 Right : Complex_Matrix) return Complex_Matrix;
function "*" (Left : Complex_Matrix;
 Right : Real'Base) return Complex_Matrix;
function "/" (Left : Complex_Matrix;
 Right : Real'Base) return Complex_Matrix;

-- Complex_Matrix inversion and related operations 45/2
function Solve (A : Complex_Matrix; X : Complex_Vector) 46/2
return Complex_Vector;
function Solve (A, X : Complex_Matrix) return Complex_Matrix;
function Inverse (A : Complex_Matrix) return Complex_Matrix;
function Determinant (A : Complex_Matrix) return Complex;

-- Eigenvalues and vectors of a Hermitian matrix 47/2
function Eigenvalues(A : Complex_Matrix) return Real_Vector; 48/2
procedure Eigensystem(A : in Complex_Matrix;
 Values : out Real_Vector;
 Vectors : out Complex_Matrix); 49/2

-- Other Complex_Matrix operations 50/2
function Unit_Matrix (Order : Positive;
 First_1, First_2 : Integer := 1) 51/2
return Complex_Matrix;

end Ada.Numerics.Generic_Complex_Arrays; 52/2

```

The library package Numerics.Complex\_Arrays is declared pure and defines the same types and subprograms as Numerics.Generic\_Complex\_Arrays, except that the predefined type Float is systematically substituted for Real'Base, and the Real\_Vector and Real\_Matrix types exported by Numerics.Real\_Arrays are systematically substituted for Real\_Vector and Real\_Matrix, and the Complex type exported by Numerics.Complex\_Types is systematically substituted for Complex, throughout. Nongeneric equivalents for each of the other predefined floating point types are defined similarly, with the names Numerics.Short\_Complex\_Arrays, Numerics.Long\_Complex\_Arrays, etc. 53/2

Two types are defined and exported by Numerics.Generic\_Complex\_Arrays. The composite type Complex\_Vector is provided to represent a vector with components of type Complex; it is defined as an unconstrained one-dimensional array with an index of type Integer. The composite type Complex\_Matrix is provided to represent a matrix with components of type Complex; it is defined as an unconstrained, two-dimensional array with indices of type Integer. 54/2

The effect of the various subprograms is as described below. In many cases they are described in terms of corresponding scalar operations in Numerics.Generic\_Complex\_Types. Any exception raised by those operations is propagated by the array subprogram. Moreover, any constraints on the parameters and the accuracy of the result for each individual component are as defined for the scalar operation. 55/2

In the case of those operations which are defined to *involve an inner product*, Constraint\_Error may be raised if an intermediate result has a component outside the range of Real'Base even though the final mathematical result would not. 56/2

```

function Re (X : Complex_Vector) return Real_Vector; 57/2
function Im (X : Complex_Vector) return Real_Vector;

```

Each function returns a vector of the specified Cartesian components of X. The index range of the result is X'Range. 58/2

```

procedure Set_Re (X : in out Complex_Vector; Re : in Real_Vector); 59/2
procedure Set_Im (X : in out Complex_Vector; Im : in Real_Vector);

```

Each procedure replaces the specified (Cartesian) component of each of the components of X by the value of the matching component of Re or Im; the other (Cartesian) component of each of 60/2

the components is unchanged. `Constraint_Error` is raised if `X'Length` is not equal to `Re'Length` or `Im'Length`.

```
61/2 function Compose_From_Cartesian (Re : Real_Vector)
 return Complex_Vector;
function Compose_From_Cartesian (Re, Im : Real_Vector)
 return Complex_Vector;
```

62/2 Each function constructs a vector of Complex results (in Cartesian representation) formed from given vectors of Cartesian components; when only the real components are given, imaginary components of zero are assumed. The index range of the result is `Re'Range`. `Constraint_Error` is raised if `Re'Length` is not equal to `Im'Length`.

```
63/2 function Modulus (X : Complex_Vector) return Real_Vector;
function "abs" (Right : Complex_Vector) return Real_Vector
 renames Modulus;
function Argument (X : Complex_Vector) return Real_Vector;
function Argument (X : Complex_Vector;
 Cycle : Real'Base) return Real_Vector;
```

64/2 Each function calculates and returns a vector of the specified polar components of `X` or `Right` using the corresponding function in `numerics.generic_complex_types`. The index range of the result is `X'Range` or `Right'Range`.

```
65/2 function Compose_From_Polar (Modulus, Argument : Real_Vector)
 return Complex_Vector;
function Compose_From_Polar (Modulus, Argument : Real_Vector;
 Cycle : Real'Base)
 return Complex_Vector;
```

66/2 Each function constructs a vector of Complex results (in Cartesian representation) formed from given vectors of polar components using the corresponding function in `numerics-generic_complex_types` on matching components of `Modulus` and `Argument`. The index range of the result is `Modulus'Range`. `Constraint_Error` is raised if `Modulus'Length` is not equal to `Argument'Length`.

```
67/2 function "+" (Right : Complex_Vector) return Complex_Vector;
function "-" (Right : Complex_Vector) return Complex_Vector;
```

68/2 Each operation returns the result of applying the corresponding operation in `numerics-generic_complex_types` to each component of `Right`. The index range of the result is `Right'Range`.

```
69/2 function Conjugate (X : Complex_Vector) return Complex_Vector;
```

70/2 This function returns the result of applying the appropriate function `Conjugate` in `numerics-generic_complex_types` to each component of `X`. The index range of the result is `X'Range`.

```
71/2 function "+" (Left, Right : Complex_Vector) return Complex_Vector;
function "-" (Left, Right : Complex_Vector) return Complex_Vector;
```

72/2 Each operation returns the result of applying the corresponding operation in `numerics-generic_complex_types` to each component of `Left` and the matching component of `Right`. The index range of the result is `Left'Range`. `Constraint_Error` is raised if `Left'Length` is not equal to `Right'Length`.

```
73/2 function "*" (Left, Right : Complex_Vector) return Complex;
```

74/2 This operation returns the inner product of `Left` and `Right`. `Constraint_Error` is raised if `Left'Length` is not equal to `Right'Length`. This operation involves an inner product.

**function "abs"** (Right : Complex\_Vector) **return** Real'Base; 75/3

This operation returns the Hermitian L2-norm of Right (the square root of the inner product of the vector with its conjugate). 76/2

**function "+"** (Left : Real\_Vector; Right : Complex\_Vector) **return** Complex\_Vector; 77/2

**function "+"** (Left : Complex\_Vector; Right : Real\_Vector) **return** Complex\_Vector;

**function "-"** (Left : Real\_Vector; Right : Complex\_Vector) **return** Complex\_Vector;

**function "-"** (Left : Complex\_Vector; Right : Real\_Vector) **return** Complex\_Vector;

Each operation returns the result of applying the corresponding operation in numerics.-generic\_complex\_types to each component of Left and the matching component of Right. The index range of the result is Left'Range. Constraint\_Error is raised if Left'Length is not equal to Right'Length. 78/2

**function "\*" (Left : Real\_Vector; Right : Complex\_Vector) return** Complex; 79/2

**function "\*" (Left : Complex\_Vector; Right : Real\_Vector) return** Complex;

Each operation returns the inner product of Left and Right. Constraint\_Error is raised if Left'Length is not equal to Right'Length. These operations involve an inner product. 80/2

**function "\*" (Left : Complex; Right : Complex\_Vector) return** Complex\_Vector; 81/2

This operation returns the result of multiplying each component of Right by the complex number Left using the appropriate operation "\*" in numerics.generic\_complex\_types. The index range of the result is Right'Range. 82/2

**function "\*" (Left : Complex\_Vector; Right : Complex) return** Complex\_Vector; 83/2

**function "/" (Left : Complex\_Vector; Right : Complex) return** Complex\_Vector;

Each operation returns the result of applying the corresponding operation in numerics.-generic\_complex\_types to each component of the vector Left and the complex number Right. The index range of the result is Left'Range. 84/2

**function "\*" (Left : Real'Base; Right : Complex\_Vector) return** Complex\_Vector; 85/2

This operation returns the result of multiplying each component of Right by the real number Left using the appropriate operation "\*" in numerics.generic\_complex\_types. The index range of the result is Right'Range. 86/2

**function "\*" (Left : Complex\_Vector; Right : Real'Base) return** Complex\_Vector; 87/2

**function "/" (Left : Complex\_Vector; Right : Real'Base) return** Complex\_Vector;

Each operation returns the result of applying the corresponding operation in numerics.-generic\_complex\_types to each component of the vector Left and the real number Right. The index range of the result is Left'Range. 88/2

**function Unit\_Vector** (Index : Integer; Order : Positive; First : Integer := 1) **return** Complex\_Vector; 89/2

This function returns a *unit vector* with Order components and a lower bound of First. All components are set to (0.0, 0.0) except for the Index component which is set to (1.0, 0.0). Constraint\_Error is raised if Index < First, Index > First + Order - 1, or if First + Order - 1 > Integer'Last. 90/2

- 91/2      **function** Re (X : Complex\_Matrix) **return** Real\_Matrix;  
**function** Im (X : Complex\_Matrix) **return** Real\_Matrix;
- 92/2      Each function returns a matrix of the specified Cartesian components of X. The index ranges of the result are those of X.
- 93/2      **procedure** Set\_Re (X : **in out** Complex\_Matrix; Re : **in** Real\_Matrix);  
**procedure** Set\_Im (X : **in out** Complex\_Matrix; Im : **in** Real\_Matrix);
- 94/2      Each procedure replaces the specified (Cartesian) component of each of the components of X by the value of the matching component of Re or Im; the other (Cartesian) component of each of the components is unchanged. Constraint\_Error is raised if X'Length(1) is not equal to Re'Length(1) or Im'Length(1) or if X'Length(2) is not equal to Re'Length(2) or Im'Length(2).
- 95/2      **function** Compose\_From\_Cartesian (Re : Real\_Matrix)  
**return** Complex\_Matrix;  
**function** Compose\_From\_Cartesian (Re, Im : Real\_Matrix)  
**return** Complex\_Matrix;
- 96/2      Each function constructs a matrix of Complex results (in Cartesian representation) formed from given matrices of Cartesian components; when only the real components are given, imaginary components of zero are assumed. The index ranges of the result are those of Re. Constraint\_Error is raised if Re'Length(1) is not equal to Im'Length(1) or Re'Length(2) is not equal to Im'Length(2).
- 97/2      **function** Modulus (X : Complex\_Matrix) **return** Real\_Matrix;  
**function** "abs" (Right : Complex\_Matrix) **return** Real\_Matrix  
**renames** Modulus;  
**function** Argument (X : Complex\_Matrix) **return** Real\_Matrix;  
**function** Argument (X : Complex\_Matrix;  
Cycle : Real'Base) **return** Real\_Matrix;
- 98/2      Each function calculates and returns a matrix of the specified polar components of X or Right using the corresponding function in numerics.generic\_complex\_types. The index ranges of the result are those of X or Right.
- 99/2      **function** Compose\_From\_Polar (Modulus, Argument : Real\_Matrix)  
**return** Complex\_Matrix;  
**function** Compose\_From\_Polar (Modulus, Argument : Real\_Matrix;  
Cycle : Real'Base)  
**return** Complex\_Matrix;
- 100/2      Each function constructs a matrix of Complex results (in Cartesian representation) formed from given matrices of polar components using the corresponding function in numerics.-generic\_complex\_types on matching components of Modulus and Argument. The index ranges of the result are those of Modulus. Constraint\_Error is raised if Modulus'Length(1) is not equal to Argument'Length(1) or Modulus'Length(2) is not equal to Argument'Length(2).
- 101/2      **function** "+" (Right : Complex\_Matrix) **return** Complex\_Matrix;  
**function** "-" (Right : Complex\_Matrix) **return** Complex\_Matrix;
- 102/2      Each operation returns the result of applying the corresponding operation in numerics.-generic\_complex\_types to each component of Right. The index ranges of the result are those of Right.
- 103/2      **function** Conjugate (X : Complex\_Matrix) **return** Complex\_Matrix;
- 104/2      This function returns the result of applying the appropriate function Conjugate in numerics.-generic\_complex\_types to each component of X. The index ranges of the result are those of X.



**function** Transpose (X : Complex\_Matrix) **return** Complex\_Matrix; 105/2

This function returns the transpose of a matrix X. The first and second index ranges of the result are X'Range(2) and X'Range(1) respectively. 106/2

**function** "+" (Left, Right : Complex\_Matrix) **return** Complex\_Matrix; 107/2

**function** "-" (Left, Right : Complex\_Matrix) **return** Complex\_Matrix; 107/2

Each operation returns the result of applying the corresponding operation in numerics.-generic\_complex\_types to each component of Left and the matching component of Right. The index ranges of the result are those of Left. Constraint\_Error is raised if Left'Length(1) is not equal to Right'Length(1) or Left'Length(2) is not equal to Right'Length(2). 108/2

**function** "\*" (Left, Right : Complex\_Matrix) **return** Complex\_Matrix; 109/2

This operation provides the standard mathematical operation for matrix multiplication. The first and second index ranges of the result are Left'Range(1) and Right'Range(2) respectively. Constraint\_Error is raised if Left'Length(2) is not equal to Right'Length(1). This operation involves inner products. 110/2

**function** "\*" (Left, Right : Complex\_Vector) **return** Complex\_Matrix; 111/2

This operation returns the outer product of a (column) vector Left by a (row) vector Right using the appropriate operation "\*" in numerics.generic\_complex\_types for computing the individual components. The first and second index ranges of the result are Left'Range and Right'Range respectively. 112/2

**function** "\*" (Left : Complex\_Vector;  
Right : Complex\_Matrix) **return** Complex\_Vector; 113/2

This operation provides the standard mathematical operation for multiplication of a (row) vector Left by a matrix Right. The index range of the (row) vector result is Right'Range(2). Constraint\_Error is raised if Left'Length is not equal to Right'Length(1). This operation involves inner products. 114/2

**function** "\*" (Left : Complex\_Matrix;  
Right : Complex\_Vector) **return** Complex\_Vector; 115/2

This operation provides the standard mathematical operation for multiplication of a matrix Left by a (column) vector Right. The index range of the (column) vector result is Left'Range(1). Constraint\_Error is raised if Left'Length(2) is not equal to Right'Length. This operation involves inner products. 116/2

**function** "+" (Left : Real\_Matrix;  
Right : Complex\_Matrix) **return** Complex\_Matrix; 117/2

**function** "+" (Left : Complex\_Matrix;  
Right : Real\_Matrix) **return** Complex\_Matrix;

**function** "-" (Left : Real\_Matrix;  
Right : Complex\_Matrix) **return** Complex\_Matrix;

**function** "-" (Left : Complex\_Matrix;  
Right : Real\_Matrix) **return** Complex\_Matrix;

Each operation returns the result of applying the corresponding operation in numerics.-generic\_complex\_types to each component of Left and the matching component of Right. The index ranges of the result are those of Left. Constraint\_Error is raised if Left'Length(1) is not equal to Right'Length(1) or Left'Length(2) is not equal to Right'Length(2). 118/2

```

119/2 function " * " (Left : Real_Matrix;
 Right : Complex_Matrix) return Complex_Matrix;
 function " * " (Left : Complex_Matrix;
 Right : Real_Matrix) return Complex_Matrix;

```

Each operation provides the standard mathematical operation for matrix multiplication. The first and second index ranges of the result are Left'Range(1) and Right'Range(2) respectively. Constraint\_Error is raised if Left'Length(2) is not equal to Right'Length(1). These operations involve inner products.

```

121/2 function "*" (Left : Real_Vector;
 Right : Complex_Vector) return Complex_Matrix;
 function "*" (Left : Complex_Vector;
 Right : Real_Vector) return Complex_Matrix;

```

Each operation returns the outer product of a (column) vector Left by a (row) vector Right using the appropriate operation `"*"` in `numerics.generic_complex_types` for computing the individual components. The first and second index ranges of the result are `Left'Range` and `Right'Range` respectively.

```
123/2 function "*" (Left : Real_Vector;
 Right : Complex_Matrix) return Complex_Vector;
 function "*" (Left : Complex_Vector;
 Right : Real_Matrix) return Complex_Vector;
```

Each operation provides the standard mathematical operation for multiplication of a (row) vector Left by a matrix Right. The index range of the (row) vector result is Right'Range(2). Constraint\_Error is raised if Left'Length is not equal to Right'Length(1). These operations involve inner products.

```

125/2 function "*" (Left : Real_Matrix;
 Right : Complex_Vector) return Complex_Vector;
 function "*" (Left : Complex_Matrix;
 Right : Real_Vector) return Complex_Vector;

```

Each operation provides the standard mathematical operation for multiplication of a matrix Left by a (column) vector Right. The index range of the (column) vector result is Left'Range(1). Constraint\_Error is raised if Left'Length(2) is not equal to Right'Length. These operations involve inner products.

```
127/2 function "*" (Left : Complex; Right : Complex_Matrix) return Complex_Matrix;
```

This operation returns the result of multiplying each component of `Right` by the complex number `Left` using the appropriate operation `"*"` in `numerics.generic_complex_types`. The index ranges of the result are those of `Right`.

```
129/2 function "*" (Left : Complex_Matrix; Right : Complex) return Complex_Matrix;
 function "/" (Left : Complex_Matrix; Right : Complex) return Complex_Matrix;
```

Each operation returns the result of applying the corresponding operation in `numerics-generic_complex_types` to each component of the matrix `Left` and the complex number `Right`. The index ranges of the result are those of `Left`.

```
131/2 function "*" (Left : Real'Base;
 Right : Complex Matrix) return Complex Matrix;
```

This operation returns the result of multiplying each component of *Right* by the real number *Left* using the appropriate operation "\*" in `numerics.generic_complex_types`. The index ranges of the result are those of *Right*.

```

function "*" (Left : Complex_Matrix; 133/2
 Right : Real'Base) return Complex_Matrix;
function "/" (Left : Complex_Matrix;
 Right : Real'Base) return Complex_Matrix;

```

Each operation returns the result of applying the corresponding operation in numerics.- 134/2  
generic\_complex\_types to each component of the matrix Left and the real number Right. The  
index ranges of the result are those of Left.

```

function Solve (A : Complex_Matrix; X : Complex_Vector) return 135/2
Complex_Vector;

```

This function returns a vector Y such that X is (nearly) equal to A \* Y. This is the standard 136/2  
mathematical operation for solving a single set of linear equations. The index range of the result  
is A'Range(2). Constraint\_Error is raised if A'Length(1), A'Length(2), and X'Length are not  
equal. Constraint\_Error is raised if the matrix A is ill-conditioned.

```

function Solve (A, X : Complex_Matrix) return Complex_Matrix; 137/2

```

This function returns a matrix Y such that X is (nearly) equal to A \* Y. This is the standard 138/2  
mathematical operation for solving several sets of linear equations. The index ranges of the  
result are A'Range(2) and X'Range(2). Constraint\_Error is raised if A'Length(1), A'Length(2),  
and X'Length(1) are not equal. Constraint\_Error is raised if the matrix A is ill-conditioned.

```

function Inverse (A : Complex_Matrix) return Complex_Matrix; 139/2

```

This function returns a matrix B such that A \* B is (nearly) equal to the unit matrix. The index 140/2  
ranges of the result are A'Range(2) and A'Range(1). Constraint\_Error is raised if A'Length(1) is  
not equal to A'Length(2). Constraint\_Error is raised if the matrix A is ill-conditioned.

```

function Determinant (A : Complex_Matrix) return Complex; 141/2

```

This function returns the determinant of the matrix A. Constraint\_Error is raised if A'Length(1) 142/2  
is not equal to A'Length(2).

```

function Eigenvalues(A : Complex_Matrix) return Real_Vector; 143/2

```

This function returns the eigenvalues of the Hermitian matrix A as a vector sorted into order 144/2  
with the largest first. Constraint\_Error is raised if A'Length(1) is not equal to A'Length(2). The  
index range of the result is A'Range(1). Argument\_Error is raised if the matrix A is not  
Hermitian.

```

procedure Eigensystem(A : in Complex_Matrix; 145/2
 Values : out Real_Vector;
 Vectors : out Complex_Matrix);

```

This procedure computes both the eigenvalues and eigenvectors of the Hermitian matrix A. The 146/3  
out parameter Values is the same as that obtained by calling the function Eigenvalues. The out  
parameter Vectors is a matrix whose columns are the eigenvectors of the matrix A. The order of  
the columns corresponds to the order of the eigenvalues. The eigenvectors are mutually  
orthonormal, including when there are repeated eigenvalues. Constraint\_Error is raised if  
A'Length(1) is not equal to A'Length(2), or if Values'Range is not equal to A'Range(1), or if the  
index ranges of the parameter Vectors are not equal to those of A. Argument\_Error is raised if  
the matrix A is not Hermitian. Constraint\_Error is also raised in implementation-defined  
circumstances if the algorithm used does not converge quickly enough.

```

147/2 function Unit_Matrix (Order : Positive;
 First_1, First_2 : Integer := 1)
 return Complex_Matrix;

```

148/2        This function returns a square *unit matrix* with Order\*\*2 components and lower bounds of First\_1 and First\_2 (for the first and second index ranges respectively). All components are set to (0.0, 0.0) except for the main diagonal, whose components are set to (1.0, 0.0). Constraint\_Error is raised if First\_1 + Order – 1 > Integer'Last or First\_2 + Order – 1 > Integer'Last.

#### *Implementation Requirements*

149/2    Accuracy requirements for the subprograms Solve, Inverse, Determinant, Eigenvalues and Eigensystem are implementation defined.

150/2    For operations not involving an inner product, the accuracy requirements are those of the corresponding operations of the type Real'Base and Complex in both the strict mode and the relaxed mode (see G.2).

151/2    For operations involving an inner product, no requirements are specified in the relaxed mode. In the strict mode the modulus of the absolute error of the inner product  $X*Y$  shall not exceed  $g*\mathbf{abs}(X)*\mathbf{abs}(Y)$  where  $g$  is defined as

152/2         $g = X'Length * Real'Machine\_Radix**(1 - Real'Model\_Mantissa)$   
for mixed complex and real operands

153/2         $g = \sqrt{2.0} * X'Length * Real'Machine\_Radix**(1 - Real'Model\_Mantissa)$   
for two complex operands

154/2    For the L2-norm, no accuracy requirements are specified in the relaxed mode. In the strict mode the relative error on the norm shall not exceed  $g / 2.0 + 3.0 * Real'Model\_Epsilon$  where  $g$  has the definition appropriate for two complex operands.

#### *Documentation Requirements*

155/2    Implementations shall document any techniques used to reduce cancellation errors such as extended precision arithmetic.

#### *Implementation Permissions*

156/2    The nongeneric equivalent packages may, but need not, be actual instantiations of the generic package for the appropriate predefined type.

157/2    Although many operations are defined in terms of operations from numerics.generic\_complex\_types, they need not be implemented by calling those operations provided that the effect is the same.

#### *Implementation Advice*

158/2    Implementations should implement the Solve and Inverse functions using established techniques. Implementations are recommended to refine the result by performing an iteration on the residuals; if this is done then it should be documented.

159/2    It is not the intention that any special provision should be made to determine whether a matrix is ill-conditioned or not. The naturally occurring overflow (including division by zero) which will result from executing these functions with an ill-conditioned matrix and thus raise Constraint\_Error is sufficient.

160/2    The test that a matrix is Hermitian should use the equality operator to compare the real components and negation followed by equality to compare the imaginary components (see G.2.1).

160.1/3    An implementation should minimize the circumstances under which the algorithm used for Eigenvalues and Eigensystem fails to converge.

Implementations should not perform operations on mixed complex and real operands by first converting the real operand to complex. See G.1.1. 161/2



# Annex H

## (normative)

### High Integrity Systems

This Annex addresses requirements for high integrity systems (including safety-critical systems and security-critical systems). It provides facilities and specifies documentation requirements that relate to several needs: 1/2

- Understanding program execution; 2
- Reviewing object code; 3
- Restricting language constructs whose usage might complicate the demonstration of program correctness 4

Execution understandability is supported by pragma Normalize\_Scalars, and also by requirements for the implementation to document the effect of a program in the presence of a bounded error or where the language rules leave the effect unspecified. 4.1

The pragmas Reviewable and Restrictions relate to the other requirements addressed by this Annex. 5

#### NOTES

- 1 The Valid attribute (see 13.9.2) is also useful in addressing these needs, to avoid problems that could otherwise arise from scalars that have values outside their declared range constraints. 6

## H.1 Pragma Normalize\_Scalars

This pragma ensures that an otherwise uninitialized scalar object is set to a predictable value, but out of range if possible. 1

#### *Syntax*

The form of a pragma Normalize\_Scalars is as follows: 2

**pragma** Normalize\_Scalars; 3

#### *Post-Compilation Rules*

Pragma Normalize\_Scalars is a configuration pragma. It applies to all compilation\_units included in a partition. 4

#### *Documentation Requirements*

If a pragma Normalize\_Scalars applies, the implementation shall document the implicit initial values for scalar subtypes, and shall identify each case in which such a value is used and is not an invalid representation. 5/2

#### *Implementation Advice*

Whenever possible, the implicit initial values for a scalar subtype should be an invalid representation (see 13.9.1). 6/2

#### NOTES

- 2 The initialization requirement applies to uninitialized scalar objects that are subcomponents of composite objects, to allocated objects, and to stand-alone objects. It also applies to scalar **out** parameters. Scalar subcomponents of composite **out** parameters are initialized to the corresponding part of the actual, by virtue of 6.4.1. 7

- 8        3 The initialization requirement does not apply to a scalar for which pragma Import has been specified, since initialization of an imported object is performed solely by the foreign language environment (see B.1).
- 9        4 The use of pragma Normalize\_Scalars in conjunction with Pragma Restrictions(No\_Exceptions) may result in erroneous execution (see H.4).

## H.2 Documentation of Implementation Decisions

### *Documentation Requirements*

- 1        The implementation shall document the range of effects for each situation that the language rules identify as either a bounded error or as having an unspecified effect. If the implementation can constrain the effects of erroneous execution for a given construct, then it shall document such constraints. The documentation might be provided either independently of any compilation unit or partition, or as part of an annotated listing for a given unit or partition. See also 1.1.3, and 1.1.2.

#### NOTES

- 2        5 Among the situations to be documented are the conventions chosen for parameter passing, the methods used for the management of run-time storage, and the method used to evaluate numeric expressions if this involves extended range or extra precision.

## H.3 Reviewable Object Code

- 1        Object code review and validation are supported by pragmas Reviewable and Inspection\_Point.

### H.3.1 Pragma Reviewable

- 1        This pragma directs the implementation to provide information to facilitate analysis and review of a program's object code, in particular to allow determination of execution time and storage usage and to identify the correspondence between the source and object programs.

#### *Syntax*

- 2        The form of a pragma Reviewable is as follows:
- 3        **pragma Reviewable;**

#### *Post-Compilation Rules*

- 4        Pragma Reviewable is a configuration pragma. It applies to all compilation\_units included in a partition.

#### *Implementation Requirements*

- 5        The implementation shall provide the following information for any compilation unit to which such a pragma applies:
- 6        • Where compiler-generated run-time checks remain;
- 7        • An identification of any construct with a language-defined check that is recognized prior to run time as certain to fail if executed (even if the generation of run-time checks has been suppressed);
- 8/2     • For each read of a scalar object, an identification of the read as either “known to be initialized,” or “possibly uninitialized,” independent of whether pragma Normalize\_Scalars applies;
- 9        • Where run-time support routines are implicitly invoked;



- An object code listing, including: 10
  - Machine instructions, with relative offsets; 11
  - Where each data object is stored during its lifetime; 12
  - Correspondence with the source program, including an identification of the code produced per declaration and per statement. 13
- An identification of each construct for which the implementation detects the possibility of erroneous execution; 14
- For each subprogram, block, task, or other construct implemented by reserving and subsequently freeing an area on a run-time stack, an identification of the length of the fixed-size portion of the area and an indication of whether the non-fixed size portion is reserved on the stack or in a dynamically-managed storage region. 15

The implementation shall provide the following information for any partition to which the pragma applies: 16

- An object code listing of the entire partition, including initialization and finalization code as well as run-time system components, and with an identification of those instructions and data that will be relocated at load time; 17
- A description of the run-time model relevant to the partition. 18

The implementation shall provide control- and data-flow information, both within each compilation unit and across the compilation units of the partition. 18.1

#### *Implementation Advice*

The implementation should provide the above information in both a human-readable and machine-readable form, and should document the latter so as to ease further processing by automated tools. 19

Object code listings should be provided both in a symbolic format and also in an appropriate numeric format (such as hexadecimal or octal). 20

#### NOTES

- 6 The order of elaboration of library units will be documented even in the absence of pragma Reviewable (see 10.2). 21

## H.3.2 Pragma Inspection\_Point

An occurrence of a pragma Inspection\_Point identifies a set of objects each of whose values is to be available at the point(s) during program execution corresponding to the position of the pragma in the compilation unit. The purpose of such a pragma is to facilitate code validation. 1

#### *Syntax*

The form of a pragma Inspection\_Point is as follows: 2

**pragma** Inspection\_Point[(*object\_name* {, *object\_name*} )]; 3

#### *Legality Rules*

A pragma Inspection\_Point is allowed wherever a **declarative\_item** or **statement** is allowed. Each *object\_name* shall statically denote the declaration of an object. 4

#### *Static Semantics*

An *inspection point* is a point in the object code corresponding to the occurrence of a pragma Inspection\_Point in the compilation unit. An object is *inspectable* at an inspection point if the corresponding pragma 5/2

Inspection\_Point either has an argument denoting that object, or has no arguments and the declaration of the object is visible at the inspection point.

*Dynamic Semantics*

- 6 Execution of a pragma Inspection\_Point has no effect.

*Implementation Requirements*

- 7 Reaching an inspection point is an external interaction with respect to the values of the inspectable objects at that point (see 1.1.3).

*Documentation Requirements*

- 8 For each inspection point, the implementation shall identify a mapping between each inspectable object and the machine resources (such as memory locations or registers) from which the object's value can be obtained.

NOTES

- 9/2 7 The implementation is not allowed to perform “dead store elimination” on the last assignment to a variable prior to a point where the variable is inspectable. Thus an inspection point has the effect of an implicit read of each of its inspectable objects.
- 10 8 Inspection points are useful in maintaining a correspondence between the state of the program in source code terms, and the machine state during the program's execution. Assertions about the values of program objects can be tested in machine terms at inspection points. Object code between inspection points can be processed by automated tools to verify programs mechanically.
- 11 9 The identification of the mapping from source program objects to machine resources is allowed to be in the form of an annotated object listing, in human-readable or tool-processable form.

## H.4 High Integrity Restrictions

- 1 This clause defines restrictions that can be used with pragma Restrictions (see 13.12); these facilitate the demonstration of program correctness by allowing tailored versions of the run-time system.

*Static Semantics*

2/2 *This paragraph was deleted.*

3/2 The following *restriction\_identifiers* are language defined:

4 **Tasking-related restriction:**

5 No\_Protected\_Types

There are no declarations of protected types or protected objects.

6 **Memory-management related restrictions:**

7 No\_Allocators

There are no occurrences of an *allocator*.

8/1 No\_Local\_Allocators

*Allocators* are prohibited in subprograms, generic subprograms, tasks, and entry bodies.

8.1/3 No\_Anonymous\_Allocators

*Allocators* of anonymous access types are not allowed..

9/2 *This paragraph was deleted.*

|                                                                                                                                                                                                                        |      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| Immediate_Reclamation                                                                                                                                                                                                  | 10   |
| Except for storage occupied by objects created by allocators and not deallocated via unchecked deallocation, any storage reserved at run time for an object is immediately reclaimed when the object no longer exists. |      |
| <b>Exception-related restriction:</b>                                                                                                                                                                                  | 11   |
| No_Exceptions                                                                                                                                                                                                          | 12   |
| Raise_statements and exception_handlers are not allowed. No language-defined run-time checks are generated; however, a run-time check performed automatically by the hardware is permitted.                            |      |
| <b>Other restrictions:</b>                                                                                                                                                                                             | 13   |
| No_Floating_Point                                                                                                                                                                                                      | 14   |
| Uses of predefined floating point types and operations, and declarations of new floating point types, are not allowed.                                                                                                 |      |
| No_Fixed_Point                                                                                                                                                                                                         | 15   |
| Uses of predefined fixed point types and operations, and declarations of new fixed point types, are not allowed.                                                                                                       |      |
| <i>This paragraph was deleted.</i>                                                                                                                                                                                     |      |
| No_Access_Subprograms                                                                                                                                                                                                  | 17   |
| The declaration of access-to-subprogram types is not allowed.                                                                                                                                                          |      |
| No_Unchecked_Access                                                                                                                                                                                                    | 18   |
| The Unchecked_Access attribute is not allowed.                                                                                                                                                                         |      |
| No_Dispatch                                                                                                                                                                                                            | 19   |
| Occurrences of T'Class are not allowed, for any (tagged) subtype T.                                                                                                                                                    |      |
| No_IO                                                                                                                                                                                                                  | 20/2 |
| Semantic dependence on any of the library units Sequential_IO, Direct_IO, Text_IO, Wide_Text_IO, Wide_Wide_Text_IO, or Stream_IO is not allowed.                                                                       |      |
| No_Delay                                                                                                                                                                                                               | 21   |
| Delay_Statements and semantic dependence on package Calendar are not allowed.                                                                                                                                          |      |
| No_Recursion                                                                                                                                                                                                           | 22   |
| As part of the execution of a subprogram, the same subprogram is not invoked.                                                                                                                                          |      |
| No_Reentrancy                                                                                                                                                                                                          | 23   |
| During the execution of a subprogram by a task, no other task invokes the same subprogram.                                                                                                                             |      |

#### Implementation Requirements

|                                                                                                                                |        |
|--------------------------------------------------------------------------------------------------------------------------------|--------|
| An implementation of this Annex shall support:                                                                                 | 23.1/2 |
| • the restrictions defined in this subclause; and                                                                              | 23.2/2 |
| • the following restrictions defined in D.7: No_Task_Hierarchy, No_Abort_Statement, No_Implicit_Heap_Allocation; and           | 23.3/2 |
| • the <b>pragma</b> Profile(Ravenscar); and                                                                                    | 23.4/2 |
| • the following uses of <i>restriction_parameter_identifiers</i> defined in D.7, which are checked prior to program execution: | 23.5/2 |
| • Max_Task_Entries => 0,                                                                                                       | 23.6/2 |
| • Max_Asynchronous_Select_Nesting => 0, and                                                                                    | 23.7/2 |
| • Max_Tasks => 0.                                                                                                              | 23.8/2 |

- 24 If an implementation supports **pragma Restrictions** for a particular argument, then except for the restrictions **No\_Unchecked\_Deallocation**, **No\_Unchecked\_Conversion**, **No\_Access\_Subprograms**, and **No\_Unchecked\_Access**, the associated restriction applies to the run-time system.

*Documentation Requirements*

- 25 If a **pragma Restrictions**(**No\_Exceptions**) is specified, the implementation shall document the effects of all constructs where language-defined checks are still performed automatically (for example, an overflow check performed by the processor).

*Erroneous Execution*

- 26 Program execution is erroneous if **pragma Restrictions**(**No\_Exceptions**) has been specified and the conditions arise under which a generated language-defined run-time check would fail.
- 27 Program execution is erroneous if **pragma Restrictions**(**No\_Recursion**) has been specified and a subprogram is invoked as part of its own execution, or if **pragma Restrictions**(**No\_Reentrancy**) has been specified and during the execution of a subprogram by a task, another task invokes the same subprogram.

NOTES

- 28/2 10 Uses of *restriction\_parameter\_identifier* **No\_Dependence** defined in 13.12.1: **No\_Dependence** => **Ada.Unchecked\_Deallocation** and **No\_Dependence** => **Ada.Unchecked\_Conversion** may be appropriate for high-integrity systems. Other uses of **No\_Dependence** can also be appropriate for high-integrity systems.

## H.5 **Pragma Detect\_Blocking**

- 1/2 The following **pragma** forces an implementation to detect potentially blocking operations within a protected operation.

*Syntax*

- 2/2 The form of a **pragma Detect\_Blocking** is as follows:
- 3/2 **pragma Detect\_Blocking**;

*Post-Compilation Rules*

- 4/2 A **pragma Detect\_Blocking** is a configuration **pragma**.

*Dynamic Semantics*

- 5/2 An implementation is required to detect a potentially blocking operation within a protected operation, and to raise **Program\_Error** (see 9.5.1).

*Implementation Permissions*

- 6/2 An implementation is allowed to reject a **compilation\_unit** if a potentially blocking operation is present directly within an **entry\_body** or the body of a protected subprogram.

NOTES

- 7/2 11 An operation that causes a task to be blocked within a foreign language domain is not defined to be potentially blocking, and need not be detected.

## H.6 Pragma Partition\_Elaboration\_Policy

This clause defines a **pragma** for user control over elaboration policy.

1/2

### *Syntax*

The form of a **pragma** Partition\_Elaboration\_Policy is as follows:

2/2

**pragma** Partition\_Elaboration\_Policy (*policy\_identifier*);

3/2

The *policy\_identifier* shall be either Sequential, Concurrent or an implementation-defined identifier.

4/2

### *Post-Compilation Rules*

A **pragma** Partition\_Elaboration\_Policy is a configuration pragma. It specifies the elaboration policy for a partition. At most one elaboration policy shall be specified for a partition.

5/2

If the Sequential policy is specified for a partition then pragma Restrictions (No\_Task\_Hierarchy) shall also be specified for the partition.

6/2

### *Dynamic Semantics*

Notwithstanding what this International Standard says elsewhere, this **pragma** allows partition elaboration rules concerning task activation and interrupt attachment to be changed. If the *policy\_identifier* is Concurrent, or if there is no pragma Partition\_Elaboration\_Policy defined for the partition, then the rules defined elsewhere in this Standard apply.

7/2

If the partition elaboration policy is Sequential, then task activation and interrupt attachment are performed in the following sequence of steps:

8/2

- The activation of all library-level tasks and the attachment of interrupt handlers are deferred until all library units are elaborated.
- The interrupt handlers are attached by the environment task.
- The environment task is suspended while the library-level tasks are activated.
- The environment task executes the main subprogram (if any) concurrently with these executing tasks.

9/2

10/2

11/2

12/2

If several dynamic interrupt handler attachments for the same interrupt are deferred, then the most recent call of Attach\_Handler or Exchange\_Handler determines which handler is attached.

13/2

If any deferred task activation fails, Tasking\_Error is raised at the beginning of the sequence of statements of the body of the environment task prior to calling the main subprogram.

14/2

### *Implementation Advice*

If the partition elaboration policy is Sequential and the Environment task becomes permanently blocked during elaboration then the partition is deadlocked and it is recommended that the partition be immediately terminated.

15/2

### *Implementation Permissions*

If the partition elaboration policy is Sequential and any task activation fails then an implementation may immediately terminate the active partition to mitigate the hazard posed by continuing to execute with a subset of the tasks being active.

16/2

NOTES

17/2

12 If any deferred task activation fails, the environment task is unable to handle the `Tasking_Error` exception and completes immediately. By contrast, if the partition elaboration policy is `Concurrent`, then this exception could be handled within a library unit.

## Annex J

### (normative)

## Obsolescent Features

This Annex contains descriptions of features of the language whose functionality is largely redundant with other features defined by this International Standard. Use of these features is not recommended in newly written programs. Use of these features can be prevented by using pragma Restrictions (No\_Obsolescent\_Features), see 13.12.1. 1/2

### J.1 Renamings of Library Units

#### *Static Semantics*

The following library\_unit\_renaming\_declarations exist: 1

```

with Ada.Unchecked_Conversion;
generic function Unchecked_Conversion renames Ada.Unchecked_Conversion;
with Ada.Unchecked_Deallocation;
generic procedure Unchecked_Deallocation renames Ada.Unchecked_Deallocation;
with Ada.Sequential_IO;
generic package Sequential_IO renames Ada.Sequential_IO;
with Ada.Direct_IO;
generic package Direct_IO renames Ada.Direct_IO;
with Ada.Text_IO;
package Text_IO renames Ada.Text_IO;
with Ada.IO_Exceptions;
package IO_Exceptions renames Ada.IO_Exceptions;
with Ada.Calendar;
package Calendar renames Ada.Calendar;
with System.Machine_Code;
package Machine_Code renames System.Machine_Code; -- If supported.

```

2  
3  
4  
5  
6  
7  
8  
9

#### *Implementation Requirements*

The implementation shall allow the user to replace these renamings. 10/3

### J.2 Allowed Replacements of Characters

#### *Syntax*

The following replacements are allowed for the vertical line, number sign, and quotation mark characters: 1

- A vertical line character (|) can be replaced by an exclamation mark (!) where used as a delimiter. 2
- The number sign characters (#) of a based\_literal can be replaced by colons (:) provided that the replacement is done for both occurrences. 3
- The quotation marks (") used as string brackets at both ends of a string literal can be replaced by percent signs (%) provided that the enclosed sequence of characters contains no quotation mark, and provided that both string brackets are replaced. Any percent sign within the sequence of characters shall then be doubled and each such doubled percent sign is interpreted as a single percent sign character value. 4

- 5 These replacements do not change the meaning of the program.

## J.3 Reduced Accuracy Subtypes

- 7 A `digits_constraint` may be used to define a floating point subtype with a new value for its requested decimal precision, as reflected by its `Digits` attribute. Similarly, a `delta_constraint` may be used to define an ordinary fixed point subtype with a new value for its *delta*, as reflected by its `Delta` attribute.

### Syntax

- 2 `delta_constraint ::= delta static_expression [range_constraint]`

### Name Resolution Rules

- 3 The expression of a `delta_constraint` is expected to be of any real type.

### Legality Rules

- 4 The expression of a `delta_constraint` shall be static.
- 5 For a `subtype_indication` with a `delta_constraint`, the `subtype_mark` shall denote an ordinary fixed point subtype.
- 6 For a `subtype_indication` with a `digits_constraint`, the `subtype_mark` shall denote either a decimal fixed point subtype or a floating point subtype (notwithstanding the rule given in 3.5.9 that only allows a decimal fixed point subtype).

### Static Semantics

- 7 A `subtype_indication` with a `subtype_mark` that denotes an ordinary fixed point subtype and a `delta_constraint` defines an ordinary fixed point subtype with a *delta* given by the value of the expression of the `delta_constraint`. If the `delta_constraint` includes a `range_constraint`, then the ordinary fixed point subtype is constrained by the `range_constraint`.
- 8 A `subtype_indication` with a `subtype_mark` that denotes a floating point subtype and a `digits_constraint` defines a floating point subtype with a requested decimal precision (as reflected by its `Digits` attribute) given by the value of the expression of the `digits_constraint`. If the `digits_constraint` includes a `range_constraint`, then the floating point subtype is constrained by the `range_constraint`.

### Dynamic Semantics

- 9 A `delta_constraint` is *compatible* with an ordinary fixed point subtype if the value of the expression is no less than the *delta* of the subtype, and the `range_constraint`, if any, is compatible with the subtype.
- 10 A `digits_constraint` is *compatible* with a floating point subtype if the value of the expression is no greater than the requested decimal precision of the subtype, and the `range_constraint`, if any, is compatible with the subtype.
- 11 The elaboration of a `delta_constraint` consists of the elaboration of the `range_constraint`, if any.



## J.4 The Constrained Attribute

### Static Semantics

For every private subtype S, the following attribute is defined:

S'Constrained

Yields the value False if S denotes an unconstrained nonformal private subtype with discriminants; also yields the value False if S denotes a generic formal private subtype, and the associated actual subtype is either an unconstrained subtype with discriminants or an unconstrained array subtype; yields the value True otherwise. The value of this attribute is of the predefined subtype Boolean.

## J.5 ASCII

### Static Semantics

The following declaration exists in the declaration of package Standard:

```

package ASCII is
 -- Control characters:
 NUL : constant Character := nul; SOH : constant Character := soh;
 STX : constant Character := stx; ETX : constant Character := etx;
 EOT : constant Character := eot; ENQ : constant Character := enq;
 ACK : constant Character := ack; BEL : constant Character := bel;
 BS : constant Character := bs; HT : constant Character := ht;
 LF : constant Character := lf; VT : constant Character := vt;
 FF : constant Character := ff; CR : constant Character := cr;
 SO : constant Character := so; SI : constant Character := si;
 DLE : constant Character := dle; DC1 : constant Character := dc1;
 DC2 : constant Character := dc2; DC3 : constant Character := dc3;
 DC4 : constant Character := dc4; NAK : constant Character := nak;
 SYN : constant Character := syn; ETB : constant Character := etb;
 CAN : constant Character := can; EM : constant Character := em;
 SUB : constant Character := sub; ESC : constant Character := esc;
 FS : constant Character := fs; GS : constant Character := gs;
 RS : constant Character := rs; US : constant Character := us;
 DEL : constant Character := del;

 -- Other characters:
 Exclam : constant Character := '!'; Quotation : constant Character := '"';
 Sharp : constant Character := '#'; Dollar : constant Character := '$';
 Percent : constant Character := '%'; Ampersand : constant Character := '&';
 Colon : constant Character := ':'; Semicolon : constant Character := ';';
 Query : constant Character := '?'; At_Sign : constant Character := '@';
 L_Bracket : constant Character := '['; Back_Slash : constant Character := '\';
 R_Bracket : constant Character := ']'; Circumflex : constant Character := '^';
 Underline : constant Character := '_'; Grave : constant Character := '`';
 L_Brace : constant Character := '{'; Bar : constant Character := '|';
 R_Brace : constant Character := '}'; Tilde : constant Character := '~';

 -- Lower case letters:
 LC_A : constant Character := 'a';
 ...
 LC_Z : constant Character := 'z';
end ASCII;
```

## J.6 Numeric\_Error

### Static Semantics

- 1 The following declaration exists in the declaration of package Standard:
- 2     Numeric\_Error : **exception renames** Constraint\_Error;

## J.7 At Clauses

### Syntax

- 1     at\_clause ::= **for** direct\_name **use at** expression;

### Static Semantics

- 2 An **at\_clause** of the form “for *x* use at *y*,” is equivalent to an **attribute\_definition\_clause** of the form “for *x*’Address use *y*,”.

### J.7.1 Interrupt Entries

- 1 Implementations are permitted to allow the attachment of task entries to interrupts via the address clause. Such an entry is referred to as an *interrupt entry*.
- 2 The address of the task entry corresponds to a hardware interrupt in an implementation-defined manner. (See Ada.Interrupts.Reference in C.3.2.)

### Static Semantics

- 3 The following attribute is defined:
- 4 For any task entry X:
- 5 X’Address     For a task entry whose address is specified (an *interrupt entry*), the value refers to the corresponding hardware interrupt. For such an entry, as for any other task entry, the meaning of this value is implementation defined. The value of this attribute is of the type of the subtype System.Address.
- 6             Address may be specified for single entries via an **attribute\_definition\_clause**.

### Dynamic Semantics

- 7 As part of the initialization of a task object, the address clause for an interrupt entry is elaborated, which evaluates the **expression** of the address clause. A check is made that the address specified is associated with some interrupt to which a task entry may be attached. If this check fails, Program\_Error is raised. Otherwise, the interrupt entry is attached to the interrupt associated with the specified address.
- 8 Upon finalization of the task object, the interrupt entry, if any, is detached from the corresponding interrupt and the default treatment is restored.
- 9 While an interrupt entry is attached to an interrupt, the interrupt is reserved (see C.3).
- 10 An interrupt delivered to a task entry acts as a call to the entry issued by a hardware task whose priority is in the System.Interrupt\_Priority range. It is implementation defined whether the call is performed as an ordinary entry call, a timed entry call, or a conditional entry call; which kind of call is performed can depend on the specific interrupt.

*Bounded (Run-Time) Errors*

It is a bounded error to evaluate E'Caller (see C.7.1) in an **accept\_statement** for an interrupt entry. The possible effects are the same as for calling **Current\_Task** from an entry body. 11

*Documentation Requirements*

The implementation shall document to which interrupts a task entry may be attached. 12

The implementation shall document whether the invocation of an interrupt entry has the effect of an ordinary entry call, conditional call, or a timed call, and whether the effect varies in the presence of pending interrupts. 13

*Implementation Permissions*

The support for this subclause is optional. 14

Interrupts to which the implementation allows a task entry to be attached may be designated as reserved for the entire duration of program execution; that is, not just when they have an interrupt entry attached to them. 15

Interrupt entry calls may be implemented by having the hardware execute directly the appropriate **accept\_statement**. Alternatively, the implementation is allowed to provide an internal interrupt handler to simulate the effect of a normal task calling the entry. 16/1

The implementation is allowed to impose restrictions on the specifications and bodies of tasks that have interrupt entries. 17

It is implementation defined whether direct calls (from the program) to interrupt entries are allowed. 18

If a **select\_statement** contains both a **terminate\_alternative** and an **accept\_alternative** for an interrupt entry, then an implementation is allowed to impose further requirements for the selection of the **terminate\_alternative** in addition to those given in 9.3. 19

## NOTES

1 Queued interrupts correspond to ordinary entry calls. Interrupts that are lost if not immediately processed correspond to conditional entry calls. It is a consequence of the priority rules that an **accept\_statement** executed in response to an interrupt can be executed with the active priority at which the hardware generates the interrupt, taking precedence over lower priority tasks, without a scheduling action. 20/1

2 Control information that is supplied upon an interrupt can be passed to an associated interrupt entry as one or more parameters of mode **in**. 21

*Examples*

*Example of an interrupt entry:* 22

```
task Interrupt_Handler is
 entry Done;
 for Done'Address use
 Ada.Interrupts.Reference (Ada.Interrupts.Names.Device_Done);
 end Interrupt_Handler; 23
```

## J.8 Mod Clauses

*Syntax*

1        `mod_clause ::= at mod static_expression;`

*Static Semantics*

2        A `record_representation_clause` of the form:

3        `for r use`  
           `record at mod a`  
           `...`  
           `end record;`

4        is equivalent to:

5        `for r'Alignment use a;`  
           `for r use`  
           `record`  
           `...`  
           `end record;`

## J.9 The Storage\_Size Attribute

*Static Semantics*

1        For any task subtype T, the following attribute is defined:

2        T'`Storage_Size`

Denotes an implementation-defined value of type *universal\_integer* representing the number of storage elements reserved for a task of the subtype T.

3/2        `Storage_Size` may be specified for a task first subtype that is not an interface via an `attribute_definition_clause`.

## J.10 Specific Suppression of Checks

1/2        Pragma `Suppress` can be used to suppress checks on specific entities.

*Syntax*

2/2        The form of a specific `Suppress pragma` is as follows:

3/2        `pragma Suppress(identifier, [On =>] name);`

*Legality Rules*

4/2        The `identifier` shall be the name of a check (see 11.5). The `name` shall statically denote some entity.

5/2        For a specific `Suppress pragma` that is immediately within a `package_specification`, the `name` shall denote an entity (or several overloaded subprograms) declared immediately within the `package_specification`.

*Static Semantics*

6/2        A specific `Suppress pragma` applies to the named check from the place of the `pragma` to the end of the innermost enclosing declarative region, or, if the `pragma` is given in a `package_specification`, to the end of the scope of the named entity. The `pragma` applies only to the named entity, or, for a subtype, on objects and values of its type. A specific `Suppress pragma` suppresses the named check for any entities to

which it applies (see 11.5). Which checks are associated with a specific entity is not defined by this International Standard.

*Implementation Permissions*

An implementation is allowed to place restrictions on specific Suppress pragmas. 7/2

NOTES

3 An implementation may support a similar On parameter on pragma Unsuppress (see 11.5). 8/2

## J.11 The Class Attribute of Untagged Incomplete Types

*Static Semantics*

For the first subtype *S* of a type *T* declared by an `incomplete_type_declaration` that is not tagged, the following attribute is defined: 1/2

|         |                                                                                                                                                                                                                                                             |     |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| S'Class | Denotes the first subtype of the incomplete class-wide type rooted at <i>T</i> . The completion of <i>T</i> shall declare a tagged type. Such an attribute reference shall occur in the same library unit as the <code>incomplete_type_declaration</code> . | 2/2 |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|

## J.12 Pragma Interface

*Syntax*

In addition to an identifier, the reserved word **interface** is allowed as a pragma name, to provide compatibility with a prior edition of this International Standard. 1/2

## J.13 Dependence Restriction Identifiers

The following restrictions involve dependence on specific language-defined units. The more general restriction `No_Dependence` (see 13.12.1) should be used for this purpose. 1/2

*Static Semantics*

The following *restriction\_identifiers* exist: 2/2

|                         |                                                                                                      |     |
|-------------------------|------------------------------------------------------------------------------------------------------|-----|
| No_Asynchronous_Control | Semantic dependence on the predefined package <code>Asynchronous_Task_Control</code> is not allowed. | 3/2 |
|-------------------------|------------------------------------------------------------------------------------------------------|-----|

|                         |                                                                                                          |     |
|-------------------------|----------------------------------------------------------------------------------------------------------|-----|
| No_Unchecked_Conversion | Semantic dependence on the predefined generic function <code>Unchecked_Conversion</code> is not allowed. | 4/2 |
|-------------------------|----------------------------------------------------------------------------------------------------------|-----|

|                           |                                                                                                             |     |
|---------------------------|-------------------------------------------------------------------------------------------------------------|-----|
| No_Unchecked_Deallocation | Semantic dependence on the predefined generic procedure <code>Unchecked_Deallocation</code> is not allowed. | 5/2 |
|---------------------------|-------------------------------------------------------------------------------------------------------------|-----|

## J.14 Character and Wide\_Character Conversion Functions

### *Static Semantics*

1/2 The following declarations exist in the declaration of package Ada.Characters.Handling:

2/2     **function** Is\_Character (Item : **in** Wide\_Character) **return** Boolean  
           **renames** Conversions.Is\_Character;  
    **function** Is\_String     (Item : **in** Wide\_String)     **return** Boolean  
           **renames** Conversions.Is\_String;

3/2     **function** To\_Character (Item           : **in** Wide\_Character;  
                             Substitute : **in** Character := ' ')  
                             **return** Character  
           **renames** Conversions.To\_Character;

4/2     **function** To\_String     (Item           : **in** Wide\_String;  
                             Substitute : **in** Character := ' ')  
                             **return** String  
           **renames** Conversions.To\_String;

5/2     **function** To\_Wide\_Character (Item : **in** Character) **return** Wide\_Character  
           **renames** Conversions.To\_Wide\_Character;

6/2     **function** To\_Wide\_String     (Item : **in** String)     **return** Wide\_String  
           **renames** Conversions.To\_Wide\_String;

## Annex K

### (informative)

## Language-Defined Attributes

|                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                    |        |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| This annex summarizes the definitions given elsewhere of the language-defined attributes. |                                                                                                                                                                                                                                                                                                                                                                                                                                    | 1      |
| P'Access                                                                                  | For a prefix P that denotes a subprogram:                                                                                                                                                                                                                                                                                                                                                                                          | 2      |
|                                                                                           | P'Access yields an access value that designates the subprogram denoted by P. The type of P'Access is an access-to-subprogram type (S), as determined by the expected type. See 3.10.2.                                                                                                                                                                                                                                             | 3      |
| X'Access                                                                                  | For a prefix X that denotes an aliased view of an object:                                                                                                                                                                                                                                                                                                                                                                          | 4      |
|                                                                                           | X'Access yields an access value that designates the object denoted by X. The type of X'Access is an access-to-object type, as determined by the expected type. The expected type shall be a general access type. See 3.10.2.                                                                                                                                                                                                       | 5      |
| X'Address                                                                                 | For a prefix X that denotes an object, program unit, or label:                                                                                                                                                                                                                                                                                                                                                                     | 6/1    |
|                                                                                           | Denotes the address of the first of the storage elements allocated to X. For a program unit or label, this value refers to the machine code associated with the corresponding body or statement. The value of this attribute is of type System.Address. See 13.3.                                                                                                                                                                  | 7      |
| S'Adjacent                                                                                | For every subtype S of a floating point type T:                                                                                                                                                                                                                                                                                                                                                                                    | 8      |
|                                                                                           | S'Adjacent denotes a function with the following specification:                                                                                                                                                                                                                                                                                                                                                                    | 9      |
|                                                                                           | <pre> <b>function</b> S'Adjacent (X, Towards : T) <b>return</b> T </pre>                                                                                                                                                                                                                                                                                                                                                           | 10     |
|                                                                                           | If <i>Towards</i> = X, the function yields X; otherwise, it yields the machine number of the type T adjacent to X in the direction of <i>Towards</i> , if that machine number exists. If the result would be outside the base range of S, Constraint_Error is raised. When T'Signed_Zeros is True, a zero result has the sign of X. When <i>Towards</i> is zero, its sign has no bearing on the result. See A.5.3.                 | 11     |
| S'Aft                                                                                     | For every fixed point subtype S:                                                                                                                                                                                                                                                                                                                                                                                                   | 12     |
|                                                                                           | S'Aft yields the number of decimal digits needed after the decimal point to accommodate the <i>delta</i> of the subtype S, unless the <i>delta</i> of the subtype S is greater than 0.1, in which case the attribute yields the value one. (S'Aft is the smallest positive integer N for which (10**N)*S'Delta is greater than or equal to one.) The value of this attribute is of the type <i>universal_integer</i> . See 3.5.10. | 13     |
| S'Alignment                                                                               | For every subtype S:                                                                                                                                                                                                                                                                                                                                                                                                               | 13.1/2 |
|                                                                                           | The value of this attribute is of type <i>universal_integer</i> , and nonnegative.                                                                                                                                                                                                                                                                                                                                                 | 13.2/2 |
|                                                                                           | For an object X of subtype S, if S'Alignment is not zero, then X'Alignment is a nonzero integral multiple of S'Alignment unless specified otherwise by a representation item. See 13.3.                                                                                                                                                                                                                                            | 13.3/2 |
| X'Alignment                                                                               | For a prefix X that denotes an object:                                                                                                                                                                                                                                                                                                                                                                                             | 14/1   |
|                                                                                           | The value of this attribute is of type <i>universal_integer</i> , and nonnegative; zero means that the object is not necessarily aligned on a storage element boundary. If X'Alignment is not zero, then X is aligned on a storage unit boundary and X'Address is an integral multiple of X'Alignment (that is, the Address modulo the Alignment is zero).                                                                         | 15     |
|                                                                                           | See 13.3.                                                                                                                                                                                                                                                                                                                                                                                                                          | 16/2   |

|      |                  |                                                                                                                                                                                                                                                                                                         |
|------|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 17   | S'Base           | For every scalar subtype S:                                                                                                                                                                                                                                                                             |
| 18   |                  | S'Base denotes an unconstrained subtype of the type of S. This unconstrained subtype is called the <i>base subtype</i> of the type. See 3.5.                                                                                                                                                            |
| 19   | S'Bit_Order      | For every specific record subtype S:                                                                                                                                                                                                                                                                    |
| 20   |                  | Denotes the bit ordering for the type of S. The value of this attribute is of type System.Bit_Order. See 13.5.3.                                                                                                                                                                                        |
| 21/1 | P'Body_Version   | For a prefix P that statically denotes a program unit:                                                                                                                                                                                                                                                  |
| 22   |                  | Yields a value of the predefined type String that identifies the version of the compilation unit that contains the body (but not any subunits) of the program unit. See E.3.                                                                                                                            |
| 23   | T'Callable       | For a prefix T that is of a task type (after any implicit dereference):                                                                                                                                                                                                                                 |
| 24   |                  | Yields the value True when the task denoted by T is <i>callable</i> , and False otherwise; See 9.9.                                                                                                                                                                                                     |
| 25   | E'Caller         | For a prefix E that denotes an <i>entry_declaration</i> :                                                                                                                                                                                                                                               |
| 26   |                  | Yields a value of the type Task_Id that identifies the task whose call is now being serviced. Use of this attribute is allowed only inside an <i>entry_body</i> or <i>accept_statement</i> corresponding to the <i>entry_declaration</i> denoted by E. See C.7.1.                                       |
| 27   | S'Ceiling        | For every subtype S of a floating point type T:                                                                                                                                                                                                                                                         |
| 28   |                  | S'Ceiling denotes a function with the following specification:                                                                                                                                                                                                                                          |
| 29   |                  | <pre>function S'Ceiling (X : T)   return T</pre>                                                                                                                                                                                                                                                        |
| 30   |                  | The function yields the value $\lceil X \rceil$ , i.e., the smallest (most negative) integral value greater than or equal to X. When X is zero, the result has the sign of X; a zero result otherwise has a negative sign when S'Signed_Zeros is True. See A.5.3.                                       |
| 31   | S'Class          | For every subtype S of an untagged private type whose full view is tagged:                                                                                                                                                                                                                              |
| 32   |                  | Denotes the class-wide subtype corresponding to the full view of S. This attribute is allowed only from the beginning of the private part in which the full view is declared, until the declaration of the full view. After the full view, the Class attribute of the full view can be used. See 7.3.1. |
| 33   | S'Class          | For every subtype S of a tagged type T (specific or class-wide):                                                                                                                                                                                                                                        |
| 34   |                  | S'Class denotes a subtype of the class-wide type (called T'Class in this International Standard) for the class rooted at T (or if S already denotes a class-wide subtype, then S'Class is the same as S).                                                                                               |
| 35   |                  | S'Class is unconstrained. However, if S is constrained, then the values of S'Class are only those that when converted to the type T belong to S. See 3.9.                                                                                                                                               |
| 36/1 | X'Component_Size | For a prefix X that denotes an array subtype or array object (after any implicit dereference):                                                                                                                                                                                                          |
| 37   |                  | Denotes the size in bits of components of the type of X. The value of this attribute is of type <i>universal_integer</i> . See 13.3.                                                                                                                                                                    |
| 38   | S'Compose        | For every subtype S of a floating point type T:                                                                                                                                                                                                                                                         |
| 39   |                  | S'Compose denotes a function with the following specification:                                                                                                                                                                                                                                          |
| 40   |                  | <pre>function S'Compose (Fraction : T;                    Exponent  : universal_integer)   return T</pre>                                                                                                                                                                                               |



|               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                      |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
|               | Let $v$ be the value $Fraction \cdot TMachine\_Radix^{Exponent-k}$ , where $k$ is the normalized exponent of $Fraction$ . If $v$ is a machine number of the type $T$ , or if $ v  \geq TModel\_Small$ , the function yields $v$ ; otherwise, it yields either one of the machine numbers of the type $T$ adjacent to $v$ . Constraint_Error is optionally raised if $v$ is outside the base range of $S$ . A zero result has the sign of $Fraction$ when $S'Signed\_Zeros$ is True. See A.5.3.                                                       | 41                   |
| A'Constrained | For a prefix $A$ that is of a discriminated type (after any implicit dereference):<br><br>Yields the value True if $A$ denotes a constant, a value, or a constrained variable, and False otherwise. See 3.7.2.                                                                                                                                                                                                                                                                                                                                       | 42<br>43             |
| S'Copy_Sign   | For every subtype $S$ of a floating point type $T$ :<br><br>S'Copy_Sign denotes a function with the following specification:<br><br><pre>function S'Copy_Sign (Value, Sign : T) return T</pre><br>If the value of $Value$ is nonzero, the function yields a result whose magnitude is that of $Value$ and whose sign is that of $Sign$ ; otherwise, it yields the value zero. Constraint_Error is optionally raised if the result is outside the base range of $S$ . A zero result has the sign of $Sign$ when $S'Signed\_Zeros$ is True. See A.5.3. | 44<br>45<br>46<br>47 |
| E'Count       | For a prefix $E$ that denotes an entry of a task or protected unit:<br><br>Yields the number of calls presently queued on the entry $E$ of the current instance of the unit. The value of this attribute is of the type <i>universal_integer</i> . See 9.9.                                                                                                                                                                                                                                                                                          | 48<br>49             |
| S'Definite    | For a prefix $S$ that denotes a formal indefinite subtype:<br><br>S'Definite yields True if the actual subtype corresponding to $S$ is definite; otherwise it yields False. The value of this attribute is of the predefined type Boolean. See 12.5.1.                                                                                                                                                                                                                                                                                               | 50/1<br>51           |
| S'Delta       | For every fixed point subtype $S$ :<br><br>S'Delta denotes the <i>delta</i> of the fixed point subtype $S$ . The value of this attribute is of the type <i>universal_real</i> . See 3.5.10.                                                                                                                                                                                                                                                                                                                                                          | 52<br>53             |
| S'Denorm      | For every subtype $S$ of a floating point type $T$ :<br><br>Yields the value True if every value expressible in the form<br>$\pm mantissa \cdot TMachine\_Radix^{TMachine\_Emin}$ where <i>mantissa</i> is a nonzero $TMachine\_Mantissa$ -digit fraction in the number base $TMachine\_Radix$ , the first digit of which is zero, is a machine number (see 3.5.7) of the type $T$ ; yields the value False otherwise. The value of this attribute is of the predefined type Boolean. See A.5.3.                                                     | 54<br>55             |
| S'Digits      | For every decimal fixed point subtype $S$ :<br><br>S'Digits denotes the <i>digits</i> of the decimal fixed point subtype $S$ , which corresponds to the number of decimal digits that are representable in objects of the subtype. The value of this attribute is of the type <i>universal_integer</i> . See 3.5.10.                                                                                                                                                                                                                                 | 56<br>57             |
| S'Digits      | For every floating point subtype $S$ :<br><br>S'Digits denotes the requested decimal precision for the subtype $S$ . The value of this attribute is of the type <i>universal_integer</i> . See 3.5.8.                                                                                                                                                                                                                                                                                                                                                | 58<br>59             |
| S'Exponent    | For every subtype $S$ of a floating point type $T$ :<br><br>S'Exponent denotes a function with the following specification:<br><br><pre>function S'Exponent (X : T) return universal_integer</pre>                                                                                                                                                                                                                                                                                                                                                   | 60<br>61<br>62       |

|      |                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 63   |                | The function yields the normalized exponent of $X$ . See A.5.3.                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 64   | S'External_Tag | For every subtype $S$ of a tagged type $T$ (specific or class-wide):                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 65   |                | S'External_Tag denotes an external string representation for S'Tag; it is of the predefined type String. External_Tag may be specified for a specific tagged type via an <i>attribute_definition_clause</i> ; the expression of such a clause shall be static. The default external tag representation is implementation defined. See 13.13.2. See 13.3.                                                                                                                                                                          |
| 66/1 | A'First        | For a <b>prefix</b> $A$ that is of an array type (after any implicit dereference), or denotes a constrained array subtype:                                                                                                                                                                                                                                                                                                                                                                                                        |
| 67   |                | A'First denotes the lower bound of the first index range; its type is the corresponding index type. See 3.6.2.                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 68   | S'First        | For every scalar subtype $S$ :                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 69   |                | S'First denotes the lower bound of the range of $S$ . The value of this attribute is of the type of $S$ . See 3.5.                                                                                                                                                                                                                                                                                                                                                                                                                |
| 70/1 | A'First(N)     | For a <b>prefix</b> $A$ that is of an array type (after any implicit dereference), or denotes a constrained array subtype:                                                                                                                                                                                                                                                                                                                                                                                                        |
| 71   |                | A'First(N) denotes the lower bound of the $N$ -th index range; its type is the corresponding index type. See 3.6.2.                                                                                                                                                                                                                                                                                                                                                                                                               |
| 72   | R.C'First_Bit  | For a component $C$ of a composite, non-array object $R$ :                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 73/2 |                | If the nondefault bit ordering applies to the composite type, and if a <i>component_clause</i> specifies the placement of $C$ , denotes the value given for the <i>first_bit</i> of the <i>component_clause</i> ; otherwise, denotes the offset, from the start of the first of the storage elements occupied by $C$ , of the first bit occupied by $C$ . This offset is measured in bits. The first bit of a storage element is numbered zero. The value of this attribute is of the type <i>universal_integer</i> . See 13.5.2. |
| 74   | S'Floor        | For every subtype $S$ of a floating point type $T$ :                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 75   |                | S'Floor denotes a function with the following specification:                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 76   |                | <pre>function S'Floor (X : T)   return T</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 77   |                | The function yields the value $\lfloor X \rfloor$ , i.e., the largest (most positive) integral value less than or equal to $X$ . When $X$ is zero, the result has the sign of $X$ ; a zero result otherwise has a positive sign. See A.5.3.                                                                                                                                                                                                                                                                                       |
| 78   | S'Fore         | For every fixed point subtype $S$ :                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 79   |                | S'Fore yields the minimum number of characters needed before the decimal point for the decimal representation of any value of the subtype $S$ , assuming that the representation does not include an exponent, but includes a one-character prefix that is either a minus sign or a space. (This minimum number does not include superfluous zeros or underlines, and is at least 2.) The value of this attribute is of the type <i>universal_integer</i> . See 3.5.10.                                                           |
| 80   | S'Fraction     | For every subtype $S$ of a floating point type $T$ :                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 81   |                | S'Fraction denotes a function with the following specification:                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 82   |                | <pre>function S'Fraction (X : T)   return T</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 83   |                | The function yields the value $X \cdot T\text{Machine\_Radix}^{-k}$ , where $k$ is the normalized exponent of $X$ . A zero result, which can only occur when $X$ is zero, has the sign of $X$ . See A.5.3.                                                                                                                                                                                                                                                                                                                        |

|               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |       |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| T'Identity    | For a <b>prefix</b> <i>T</i> that is of a task type (after any implicit dereference):                                                                                                                                                                                                                                                                                                                                                                                                  | 84    |
|               | Yields a value of the type <i>Task_Id</i> that identifies the task denoted by <i>T</i> . See C.7.1.                                                                                                                                                                                                                                                                                                                                                                                    | 85    |
| E'Identity    | For a <b>prefix</b> <i>E</i> that denotes an exception:                                                                                                                                                                                                                                                                                                                                                                                                                                | 86/1  |
|               | <i>E'Identity</i> returns the unique identity of the exception. The type of this attribute is <i>Exception_Id</i> . See 11.4.1.                                                                                                                                                                                                                                                                                                                                                        | 87    |
| S'Image       | For every scalar subtype <i>S</i> :                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 88    |
|               | <i>S'Image</i> denotes a function with the following specification:                                                                                                                                                                                                                                                                                                                                                                                                                    | 89    |
|               | <pre>function S'Image (Arg : S'Base) return String</pre>                                                                                                                                                                                                                                                                                                                                                                                                                               | 90    |
|               | The function returns an image of the value of <i>Arg</i> as a <i>String</i> . See 3.5.                                                                                                                                                                                                                                                                                                                                                                                                 | 91/2  |
| S'Class'Input | For every subtype <i>S'Class</i> of a class-wide type <i>T'Class</i> :                                                                                                                                                                                                                                                                                                                                                                                                                 | 92    |
|               | <i>S'Class'Input</i> denotes a function with the following specification:                                                                                                                                                                                                                                                                                                                                                                                                              | 93    |
|               | <pre>function S'Class'Input ( Stream : not null access Ada.Streams.Root_Stream_Type'Class) return T'Class</pre>                                                                                                                                                                                                                                                                                                                                                                        | 94/2  |
|               | First reads the external tag from <i>Stream</i> and determines the corresponding internal tag (by calling <i>Tags.Descendant_Tag(String'Input(Stream), S'Tag)</i> which might raise <i>Tag_Error</i> — see 3.9) and then dispatches to the subprogram denoted by the <i>Input</i> attribute of the specific type identified by the internal tag; returns that result. If the specific type identified by the internal tag is abstract, <i>Constraint_Error</i> is raised. See 13.13.2. | 95/3  |
| S'Input       | For every subtype <i>S</i> of a specific type <i>T</i> :                                                                                                                                                                                                                                                                                                                                                                                                                               | 96    |
|               | <i>S'Input</i> denotes a function with the following specification:                                                                                                                                                                                                                                                                                                                                                                                                                    | 97    |
|               | <pre>function S'Input ( Stream : not null access Ada.Streams.Root_Stream_Type'Class) return T</pre>                                                                                                                                                                                                                                                                                                                                                                                    | 98/2  |
|               | <i>S'Input</i> reads and returns one value from <i>Stream</i> , using any bounds or discriminants written by a corresponding <i>S'Output</i> to determine how much to read. See 13.13.2.                                                                                                                                                                                                                                                                                               | 99    |
| A'Last        | For a <b>prefix</b> <i>A</i> that is of an array type (after any implicit dereference), or denotes a constrained array subtype:                                                                                                                                                                                                                                                                                                                                                        | 100/1 |
|               | <i>A'Last</i> denotes the upper bound of the first index range; its type is the corresponding index type. See 3.6.2.                                                                                                                                                                                                                                                                                                                                                                   | 101   |
| S'Last        | For every scalar subtype <i>S</i> :                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 102   |
|               | <i>S'Last</i> denotes the upper bound of the range of <i>S</i> . The value of this attribute is of the type of <i>S</i> . See 3.5.                                                                                                                                                                                                                                                                                                                                                     | 103   |
| A'Last(N)     | For a <b>prefix</b> <i>A</i> that is of an array type (after any implicit dereference), or denotes a constrained array subtype:                                                                                                                                                                                                                                                                                                                                                        | 104/1 |
|               | <i>A'Last(N)</i> denotes the upper bound of the <i>N</i> -th index range; its type is the corresponding index type. See 3.6.2.                                                                                                                                                                                                                                                                                                                                                         | 105   |
| R.C'Last_Bit  | For a component <i>C</i> of a composite, non-array object <i>R</i> :                                                                                                                                                                                                                                                                                                                                                                                                                   | 106   |
|               | If the nondefault bit ordering applies to the composite type, and if a <i>component_clause</i> specifies the placement of <i>C</i> , denotes the value given for the <i>last_bit</i> of the <i>component_clause</i> ; otherwise, denotes the offset, from the start of the first of the storage                                                                                                                                                                                        | 107/2 |

elements occupied by  $C$ , of the last bit occupied by  $C$ . This offset is measured in bits. The value of this attribute is of the type *universal\_integer*. See 13.5.2.

- 108 S'Leading\_Part  
For every subtype  $S$  of a floating point type  $T$ :
- 109 S'Leading\_Part denotes a function with the following specification:
- 110 **function** S'Leading\_Part ( $X : T$ ;  
 $Radix\_Digits : universal\_integer$   
**return**  $T$
- 111 Let  $v$  be the value  $TMachine\_Radix^{k-Radix\_Digits}$ , where  $k$  is the normalized exponent of  $X$ . The function yields the value
- 112 •  $\lfloor X/v \rfloor \cdot v$ , when  $X$  is nonnegative and  $Radix\_Digits$  is positive;
- 113 •  $\lceil X/v \rceil \cdot v$ , when  $X$  is negative and  $Radix\_Digits$  is positive.
- 114 Constraint\_Error is raised when  $Radix\_Digits$  is zero or negative. A zero result, which can only occur when  $X$  is zero, has the sign of  $X$ . See A.5.3.
- 115/1 A'Length For a prefix  $A$  that is of an array type (after any implicit dereference), or denotes a constrained array subtype:
- 116 A'Length denotes the number of values of the first index range (zero for a null range); its type is *universal\_integer*. See 3.6.2.
- 117/1 A'Length(N) For a prefix  $A$  that is of an array type (after any implicit dereference), or denotes a constrained array subtype:
- 118 A'Length(N) denotes the number of values of the  $N$ -th index range (zero for a null range); its type is *universal\_integer*. See 3.6.2.
- 119 S'Machine For every subtype  $S$  of a floating point type  $T$ :
- 120 S'Machine denotes a function with the following specification:
- 121 **function** S'Machine ( $X : T$ )  
**return**  $T$
- 122 If  $X$  is a machine number of the type  $T$ , the function yields  $X$ ; otherwise, it yields the value obtained by rounding or truncating  $X$  to either one of the adjacent machine numbers of the type  $T$ . Constraint\_Error is raised if rounding or truncating  $X$  to the precision of the machine numbers results in a value outside the base range of  $S$ . A zero result has the sign of  $X$  when S'Signed\_Zeros is True. See A.5.3.
- 123 S'Machine\_Emax  
For every subtype  $S$  of a floating point type  $T$ :
- 124 Yields the largest (most positive) value of *exponent* such that every value expressible in the canonical form (for the type  $T$ ), having a *mantissa* of  $TMachine\_Mantissa$  digits, is a machine number (see 3.5.7) of the type  $T$ . This attribute yields a value of the type *universal\_integer*. See A.5.3.
- 125 S'Machine\_Emin  
For every subtype  $S$  of a floating point type  $T$ :
- 126 Yields the smallest (most negative) value of *exponent* such that every value expressible in the canonical form (for the type  $T$ ), having a *mantissa* of  $TMachine\_Mantissa$  digits, is a machine number (see 3.5.7) of the type  $T$ . This attribute yields a value of the type *universal\_integer*. See A.5.3.
- 127 S'Machine\_Mantissa  
For every subtype  $S$  of a floating point type  $T$ :

|                     |                                                                                                                                                                                                                                                                                                                                                                              |         |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
|                     | Yields the largest value of $p$ such that every value expressible in the canonical form (for the type $T$ ), having a $p$ -digit <i>mantissa</i> and an <i>exponent</i> between $T$ Machine_Emin and $T$ Machine_Emax, is a machine number (see 3.5.7) of the type $T$ . This attribute yields a value of the type <i>universal_integer</i> . See A.5.3.                     | 128     |
| S'Machine_Overflows | For every subtype S of a fixed point type $T$ :                                                                                                                                                                                                                                                                                                                              | 129     |
|                     | Yields the value True if overflow and divide-by-zero are detected and reported by raising Constraint_Error for every predefined operation that yields a result of the type $T$ ; yields the value False otherwise. The value of this attribute is of the predefined type Boolean. See A.5.4.                                                                                 | 130     |
| S'Machine_Overflows | For every subtype S of a floating point type $T$ :                                                                                                                                                                                                                                                                                                                           | 131     |
|                     | Yields the value True if overflow and divide-by-zero are detected and reported by raising Constraint_Error for every predefined operation that yields a result of the type $T$ ; yields the value False otherwise. The value of this attribute is of the predefined type Boolean. See A.5.3.                                                                                 | 132     |
| S'Machine_Radix     | For every subtype S of a fixed point type $T$ :                                                                                                                                                                                                                                                                                                                              | 133     |
|                     | Yields the radix of the hardware representation of the type $T$ . The value of this attribute is of the type <i>universal_integer</i> . See A.5.4.                                                                                                                                                                                                                           | 134     |
| S'Machine_Radix     | For every subtype S of a floating point type $T$ :                                                                                                                                                                                                                                                                                                                           | 135     |
|                     | Yields the radix of the hardware representation of the type $T$ . The value of this attribute is of the type <i>universal_integer</i> . See A.5.3.                                                                                                                                                                                                                           | 136     |
| S'Machine_Rounding  | For every subtype S of a floating point type $T$ :                                                                                                                                                                                                                                                                                                                           | 136.1/2 |
|                     | S'Machine_Rounding denotes a function with the following specification:                                                                                                                                                                                                                                                                                                      | 136.2/2 |
|                     | <b>function</b> S'Machine_Rounding ( $X : T$ )<br><b>return</b> $T$                                                                                                                                                                                                                                                                                                          | 136.3/2 |
|                     | The function yields the integral value nearest to $X$ . If $X$ lies exactly halfway between two integers, one of those integers is returned, but which of them is returned is unspecified. A zero result has the sign of $X$ when S'Signed_Zeros is True. This function provides access to the rounding behavior which is most efficient on the target processor. See A.5.3. | 136.4/2 |
| S'Machine_Rounds    | For every subtype S of a fixed point type $T$ :                                                                                                                                                                                                                                                                                                                              | 137     |
|                     | Yields the value True if rounding is performed on inexact results of every predefined operation that yields a result of the type $T$ ; yields the value False otherwise. The value of this attribute is of the predefined type Boolean. See A.5.4.                                                                                                                           | 138     |
| S'Machine_Rounds    | For every subtype S of a floating point type $T$ :                                                                                                                                                                                                                                                                                                                           | 139     |
|                     | Yields the value True if rounding is performed on inexact results of every predefined operation that yields a result of the type $T$ ; yields the value False otherwise. The value of this attribute is of the predefined type Boolean. See A.5.3.                                                                                                                           | 140     |
| S'Max               | For every scalar subtype S:                                                                                                                                                                                                                                                                                                                                                  | 141     |
|                     | S'Max denotes a function with the following specification:                                                                                                                                                                                                                                                                                                                   | 142     |

- 143                   **function** S'Max(*Left*, *Right* : S'Base)  
                       **return** S'Base
- 144                   The function returns the greater of the values of the two parameters. See 3.5.
- 144.1/3 S'Max\_Alignment\_For\_Allocation  
           For every subtype S:
- 144.2/3               Denotes the maximum value for Alignment that could be requested by the implementation via Allocate for an access type whose designated subtype is S. The value of this attribute is of type *universal\_integer*. See 13.11.1.
- 145   S'Max\_Size\_In\_Storage\_Elements  
       For every subtype S:
- 146/2               Denotes the maximum value for Size\_In\_Storage\_Elements that could be requested by the implementation via Allocate for an access type whose designated subtype is S. The value of this attribute is of type *universal\_integer*. See 13.11.1.
- 147   S'Min           For every scalar subtype S:
- 148                   S'Min denotes a function with the following specification:
- 149                   **function** S'Min(*Left*, *Right* : S'Base)  
                       **return** S'Base
- 150                   The function returns the lesser of the values of the two parameters. See 3.5.
- 150.1/2 S'Mod        For every modular subtype S:
- 150.2/2               S'Mod denotes a function with the following specification:
- 150.3/2               **function** S'Mod (*Arg* : *universal\_integer*)  
                       **return** S'Base
- 150.4/2               This function returns *Arg* **mod** S'Modulus, as a value of the type of S. See 3.5.4.
- 151   S'Model        For every subtype S of a floating point type *T*:
- 152                   S'Model denotes a function with the following specification:
- 153                   **function** S'Model (*X* : *T*)  
                       **return** *T*
- 154                   If the Numerics Annex is not supported, the meaning of this attribute is implementation defined; see G.2.2 for the definition that applies to implementations supporting the Numerics Annex. See A.5.3.
- 155   S'Model\_Emin   For every subtype S of a floating point type *T*:
- 156                   If the Numerics Annex is not supported, this attribute yields an implementation defined value that is greater than or equal to the value of *T*'Machine\_Emin. See G.2.2 for further requirements that apply to implementations supporting the Numerics Annex. The value of this attribute is of the type *universal\_integer*. See A.5.3.
- 157   S'Model\_Epsilon  
       For every subtype S of a floating point type *T*:
- 158                   Yields the value  $T'Machine\_Radix^{1 - T'Model\_Mantissa}$ . The value of this attribute is of the type *universal\_real*. See A.5.3.
- 159   S'Model\_Mantissa  
       For every subtype S of a floating point type *T*:
- 160                   If the Numerics Annex is not supported, this attribute yields an implementation defined value that is greater than or equal to  $\lceil d \cdot \log(10) / \log(T'Machine\_Radix) \rceil + 1$ , where *d* is

the requested decimal precision of  $T$ , and less than or equal to the value of  $T$ Machine\_Mantissa. See G.2.2 for further requirements that apply to implementations supporting the Numerics Annex. The value of this attribute is of the type *universal\_integer*. See A.5.3.

|                |                                                                                                                                                                                                                                                                                                                                                                                                                     |       |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| S'Model_Small  | For every subtype S of a floating point type $T$ :                                                                                                                                                                                                                                                                                                                                                                  | 161   |
|                | Yields the value $T$ Machine_Radix <sup><math>T</math>Model_Emin - 1</sup> . The value of this attribute is of the type <i>universal_real</i> . See A.5.3.                                                                                                                                                                                                                                                          | 162   |
| S'Modulus      | For every modular subtype S:                                                                                                                                                                                                                                                                                                                                                                                        | 163   |
|                | S'Modulus yields the modulus of the type of S, as a value of the type <i>universal_integer</i> . See 3.5.4.                                                                                                                                                                                                                                                                                                         | 164   |
| S'Class'Output | For every subtype S'Class of a class-wide type $T$ Class:                                                                                                                                                                                                                                                                                                                                                           | 165   |
|                | S'Class'Output denotes a procedure with the following specification:                                                                                                                                                                                                                                                                                                                                                | 166   |
|                | <pre><b>procedure</b> S'Class'Output (<br/>    Stream : <b>not null access</b> Ada.Streams.Root_Stream_Type'Class;<br/>    Item   : <b>in</b> T'Class)</pre>                                                                                                                                                                                                                                                        | 167/2 |
|                | First writes the external tag of <i>Item</i> to <i>Stream</i> (by calling String'Output( <i>Stream</i> , Tags.-External_Tag( <i>Item</i> 'Tag)) — see 3.9) and then dispatches to the subprogram denoted by the Output attribute of the specific type identified by the tag. Tag_Error is raised if the tag of <i>Item</i> identifies a type declared at an accessibility level deeper than that of S. See 13.13.2. | 168/2 |
| S'Output       | For every subtype S of a specific type $T$ :                                                                                                                                                                                                                                                                                                                                                                        | 169   |
|                | S'Output denotes a procedure with the following specification:                                                                                                                                                                                                                                                                                                                                                      | 170   |
|                | <pre><b>procedure</b> S'Output (<br/>    Stream : <b>not null access</b> Ada.Streams.Root_Stream_Type'Class;<br/>    Item   : <b>in</b> T)</pre>                                                                                                                                                                                                                                                                    | 171/2 |
|                | S'Output writes the value of <i>Item</i> to <i>Stream</i> , including any bounds or discriminants. See 13.13.2.                                                                                                                                                                                                                                                                                                     | 172   |
| D'Partition_Id | For a prefix D that denotes a library-level declaration, excepting a declaration of or within a declared-pure library unit:                                                                                                                                                                                                                                                                                         | 173/1 |
|                | Denotes a value of the type <i>universal_integer</i> that identifies the partition in which D was elaborated. If D denotes the declaration of a remote call interface library unit (see E.2.3) the given partition is the one where the body of D was elaborated. See E.1.                                                                                                                                          | 174   |
| S'Pos          | For every discrete subtype S:                                                                                                                                                                                                                                                                                                                                                                                       | 175   |
|                | S'Pos denotes a function with the following specification:                                                                                                                                                                                                                                                                                                                                                          | 176   |
|                | <pre><b>function</b> S'Pos (Arg : S'Base)<br/>    <b>return</b> <i>universal_integer</i></pre>                                                                                                                                                                                                                                                                                                                      | 177   |
|                | This function returns the position number of the value of <i>Arg</i> , as a value of type <i>universal_integer</i> . See 3.5.5.                                                                                                                                                                                                                                                                                     | 178   |
| R.C'Position   | For a component C of a composite, non-array object R:                                                                                                                                                                                                                                                                                                                                                               | 179   |
|                | If the nondefault bit ordering applies to the composite type, and if a <i>component_clause</i> specifies the placement of C, denotes the value given for the <i>position</i> of the <i>component_clause</i> ; otherwise, denotes the same value as R.C'Address - R'Address. The value of this attribute is of the type <i>universal_integer</i> . See 13.5.2.                                                       | 180/2 |

|         |              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|---------|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 181     | S'Pred       | For every scalar subtype <i>S</i> :                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 182     |              | S'Pred denotes a function with the following specification:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 183     |              | <pre><b>function</b> S'Pred(<i>Arg</i> : S'Base)     <b>return</b> S'Base</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 184     |              | For an enumeration type, the function returns the value whose position number is one less than that of the value of <i>Arg</i> ; Constraint_Error is raised if there is no such value of the type. For an integer type, the function returns the result of subtracting one from the value of <i>Arg</i> . For a fixed point type, the function returns the result of subtracting <i>small</i> from the value of <i>Arg</i> . For a floating point type, the function returns the machine number (as defined in 3.5.7) immediately below the value of <i>Arg</i> ; Constraint_Error is raised if there is no such machine number. See 3.5. |
| 184.1/2 | P'Priority   | For a prefix <i>P</i> that denotes a protected object:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 184.2/2 |              | Denotes a non-aliased component of the protected object <i>P</i> . This component is of type System.Any_Priority and its value is the priority of <i>P</i> . P'Priority denotes a variable if and only if <i>P</i> denotes a variable. A reference to this attribute shall appear only within the body of <i>P</i> . See D.5.2.                                                                                                                                                                                                                                                                                                           |
| 185/1   | A'Range      | For a prefix <i>A</i> that is of an array type (after any implicit dereference), or denotes a constrained array subtype:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 186     |              | A'Range is equivalent to the range A'First .. A'Last, except that the prefix <i>A</i> is only evaluated once. See 3.6.2.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 187     | S'Range      | For every scalar subtype <i>S</i> :                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 188     |              | S'Range is equivalent to the range S'First .. S'Last. See 3.5.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 189/1   | A'Range(N)   | For a prefix <i>A</i> that is of an array type (after any implicit dereference), or denotes a constrained array subtype:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 190     |              | A'Range(N) is equivalent to the range A'First(N) .. A'Last(N), except that the prefix <i>A</i> is only evaluated once. See 3.6.2.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 191     | S'Class'Read | For every subtype S'Class of a class-wide type <i>T</i> 'Class:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 192     |              | S'Class'Read denotes a procedure with the following specification:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 193/2   |              | <pre><b>procedure</b> S'Class'Read(     <i>Stream</i> : <b>not null access</b> Ada.Streams.Root_Stream_Type'Class;     <i>Item</i> : <b>out</b> T'Class)</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 194     |              | Dispatches to the subprogram denoted by the Read attribute of the specific type identified by the tag of <i>Item</i> . See 13.13.2.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 195     | S'Read       | For every subtype <i>S</i> of a specific type <i>T</i> :                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 196     |              | S'Read denotes a procedure with the following specification:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 197/2   |              | <pre><b>procedure</b> S'Read(     <i>Stream</i> : <b>not null access</b> Ada.Streams.Root_Stream_Type'Class;     <i>Item</i> : <b>out</b> T)</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 198     |              | S'Read reads the value of <i>Item</i> from <i>Stream</i> . See 13.13.2.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 199     | S'Remainder  | For every subtype <i>S</i> of a floating point type <i>T</i> :                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 200     |              | S'Remainder denotes a function with the following specification:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 201     |              | <pre><b>function</b> S'Remainder(<i>X</i>, <i>Y</i> : T)     <b>return</b> T</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 202     |              | For nonzero <i>Y</i> , let <i>v</i> be the value $X - n \cdot Y$ , where <i>n</i> is the integer nearest to the exact value of $X/Y$ ; if $ n - X/Y  = 1/2$ , then <i>n</i> is chosen to be even. If <i>v</i> is a machine number of the type                                                                                                                                                                                                                                                                                                                                                                                             |



|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |     |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
|                | <i>T</i> , the function yields <i>v</i> ; otherwise, it yields zero. Constraint_Error is raised if <i>Y</i> is zero. A zero result has the sign of <i>X</i> when S'Signed_Zeros is True. See A.5.3.                                                                                                                                                                                                                                                                   |     |
| S'Round        | For every decimal fixed point subtype <i>S</i> :                                                                                                                                                                                                                                                                                                                                                                                                                      | 203 |
|                | S'Round denotes a function with the following specification:                                                                                                                                                                                                                                                                                                                                                                                                          | 204 |
|                | <pre><b>function</b> S'Round (<i>X</i> : <i>universal_real</i>) <b>return</b> S'Base</pre>                                                                                                                                                                                                                                                                                                                                                                            | 205 |
|                | The function returns the value obtained by rounding <i>X</i> (away from 0, if <i>X</i> is midway between two values of the type of <i>S</i> ). See 3.5.10.                                                                                                                                                                                                                                                                                                            | 206 |
| S'Rounding     | For every subtype <i>S</i> of a floating point type <i>T</i> :                                                                                                                                                                                                                                                                                                                                                                                                        | 207 |
|                | S'Rounding denotes a function with the following specification:                                                                                                                                                                                                                                                                                                                                                                                                       | 208 |
|                | <pre><b>function</b> S'Rounding (<i>X</i> : <i>T</i>) <b>return</b> <i>T</i></pre>                                                                                                                                                                                                                                                                                                                                                                                    | 209 |
|                | The function yields the integral value nearest to <i>X</i> , rounding away from zero if <i>X</i> lies exactly halfway between two integers. A zero result has the sign of <i>X</i> when S'Signed_Zeros is True. See A.5.3.                                                                                                                                                                                                                                            | 210 |
| S'Safe_First   | For every subtype <i>S</i> of a floating point type <i>T</i> :                                                                                                                                                                                                                                                                                                                                                                                                        | 211 |
|                | Yields the lower bound of the safe range (see 3.5.7) of the type <i>T</i> . If the Numerics Annex is not supported, the value of this attribute is implementation defined; see G.2.2 for the definition that applies to implementations supporting the Numerics Annex. The value of this attribute is of the type <i>universal_real</i> . See A.5.3.                                                                                                                  | 212 |
| S'Safe_Last    | For every subtype <i>S</i> of a floating point type <i>T</i> :                                                                                                                                                                                                                                                                                                                                                                                                        | 213 |
|                | Yields the upper bound of the safe range (see 3.5.7) of the type <i>T</i> . If the Numerics Annex is not supported, the value of this attribute is implementation defined; see G.2.2 for the definition that applies to implementations supporting the Numerics Annex. The value of this attribute is of the type <i>universal_real</i> . See A.5.3.                                                                                                                  | 214 |
| S'Scale        | For every decimal fixed point subtype <i>S</i> :                                                                                                                                                                                                                                                                                                                                                                                                                      | 215 |
|                | S'Scale denotes the <i>scale</i> of the subtype <i>S</i> , defined as the value <i>N</i> such that S'Delta = 10.0*(- <i>N</i> ). The scale indicates the position of the point relative to the rightmost significant digits of values of subtype <i>S</i> . The value of this attribute is of the type <i>universal_integer</i> . See 3.5.10.                                                                                                                         | 216 |
| S'Scaling      | For every subtype <i>S</i> of a floating point type <i>T</i> :                                                                                                                                                                                                                                                                                                                                                                                                        | 217 |
|                | S'Scaling denotes a function with the following specification:                                                                                                                                                                                                                                                                                                                                                                                                        | 218 |
|                | <pre><b>function</b> S'Scaling (<i>X</i> : <i>T</i>;                     <i>Adjustment</i> : <i>universal_integer</i>) <b>return</b> <i>T</i></pre>                                                                                                                                                                                                                                                                                                                   | 219 |
|                | Let <i>v</i> be the value $X \cdot TMachine\_Radix^{Adjustment}$ . If <i>v</i> is a machine number of the type <i>T</i> , or if $ v  \geq TModel\_Small$ , the function yields <i>v</i> ; otherwise, it yields either one of the machine numbers of the type <i>T</i> adjacent to <i>v</i> . Constraint_Error is optionally raised if <i>v</i> is outside the base range of <i>S</i> . A zero result has the sign of <i>X</i> when S'Signed_Zeros is True. See A.5.3. | 220 |
| S'Signed_Zeros | For every subtype <i>S</i> of a floating point type <i>T</i> :                                                                                                                                                                                                                                                                                                                                                                                                        | 221 |
|                | Yields the value True if the hardware representation for the type <i>T</i> has the capability of representing both positively and negatively signed zeros, these being generated and used by the predefined operations of the type <i>T</i> as specified in IEC 559:1989; yields the value False otherwise. The value of this attribute is of the predefined type Boolean. See A.5.3.                                                                                 | 222 |

|         |                |                                                                                                                                                                                                                                                                                                                                                                                                           |
|---------|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 223     | S'Size         | For every subtype S:                                                                                                                                                                                                                                                                                                                                                                                      |
| 224     |                | If S is definite, denotes the size (in bits) that the implementation would choose for the following objects of subtype S:                                                                                                                                                                                                                                                                                 |
| 225     |                | • A record component of subtype S when the record type is packed.                                                                                                                                                                                                                                                                                                                                         |
| 226     |                | • The formal parameter of an instance of <code>Unchecked_Conversion</code> that converts from subtype S to some other subtype.                                                                                                                                                                                                                                                                            |
| 227     |                | If S is indefinite, the meaning is implementation defined. The value of this attribute is of the type <i>universal_integer</i> . See 13.3.                                                                                                                                                                                                                                                                |
| 228/1   | X'Size         | For a prefix X that denotes an object:                                                                                                                                                                                                                                                                                                                                                                    |
| 229     |                | Denotes the size in bits of the representation of the object. The value of this attribute is of the type <i>universal_integer</i> . See 13.3.                                                                                                                                                                                                                                                             |
| 230     | S'Small        | For every fixed point subtype S:                                                                                                                                                                                                                                                                                                                                                                          |
| 231     |                | S'Small denotes the <i>small</i> of the type of S. The value of this attribute is of the type <i>universal_real</i> . See 3.5.10.                                                                                                                                                                                                                                                                         |
| 232     | S'Storage_Pool | For every access-to-object subtype S:                                                                                                                                                                                                                                                                                                                                                                     |
| 233     |                | Denotes the storage pool of the type of S. The type of this attribute is <code>Root_Storage_Pool'Class</code> . See 13.11.                                                                                                                                                                                                                                                                                |
| 234     | S'Storage_Size | For every access-to-object subtype S:                                                                                                                                                                                                                                                                                                                                                                     |
| 235     |                | Yields the result of calling <code>Storage_Size(S'Storage_Pool)</code> , which is intended to be a measure of the number of storage elements reserved for the pool. The type of this attribute is <i>universal_integer</i> . See 13.11.                                                                                                                                                                   |
| 236/1   | T'Storage_Size | For a prefix T that denotes a task object (after any implicit dereference):                                                                                                                                                                                                                                                                                                                               |
| 237     |                | Denotes the number of storage elements reserved for the task. The value of this attribute is of the type <i>universal_integer</i> . The <code>Storage_Size</code> includes the size of the task's stack, if any. The language does not specify whether or not it includes other storage associated with the task (such as the “task control block” used by some implementations.) See 13.3.               |
| 237.1/2 | S'Stream_Size  | For every subtype S of an elementary type T:                                                                                                                                                                                                                                                                                                                                                              |
| 237.2/2 |                | Denotes the number of bits occupied in a stream by items of subtype S. Hence, the number of stream elements required per item of elementary type T is:                                                                                                                                                                                                                                                    |
| 237.3/2 |                | $T'Stream\_Size / Ada.Streams.Stream\_Element'Size$                                                                                                                                                                                                                                                                                                                                                       |
| 237.4/2 |                | The value of this attribute is of type <i>universal_integer</i> and is a multiple of <code>Stream_Element'Size</code> . See 13.13.2.                                                                                                                                                                                                                                                                      |
| 238     | S'Succ         | For every scalar subtype S:                                                                                                                                                                                                                                                                                                                                                                               |
| 239     |                | S'Succ denotes a function with the following specification:                                                                                                                                                                                                                                                                                                                                               |
| 240     |                | <b>function</b> S'Succ (Arg : S'Base)<br><b>return</b> S'Base                                                                                                                                                                                                                                                                                                                                             |
| 241     |                | For an enumeration type, the function returns the value whose position number is one more than that of the value of Arg; <code>Constraint_Error</code> is raised if there is no such value of the type. For an integer type, the function returns the result of adding one to the value of Arg. For a fixed point type, the function returns the result of adding <i>small</i> to the value of Arg. For a |

|                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                          |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|
|                     | floating point type, the function returns the machine number (as defined in 3.5.7) immediately above the value of <i>Arg</i> ; <i>Constraint_Error</i> is raised if there is no such machine number. See 3.5.                                                                                                                                                                                                                                        |                          |
| X'Tag               | For a prefix <i>X</i> that is of a class-wide tagged type (after any implicit dereference):<br>X'Tag denotes the tag of <i>X</i> . The value of this attribute is of type <i>Tag</i> . See 3.9.                                                                                                                                                                                                                                                      | 242<br>243               |
| S'Tag               | For every subtype <i>S</i> of a tagged type <i>T</i> (specific or class-wide):<br>S'Tag denotes the tag of the type <i>T</i> (or if <i>T</i> is class-wide, the tag of the root type of the corresponding class). The value of this attribute is of type <i>Tag</i> . See 3.9.                                                                                                                                                                       | 244<br>245               |
| T'Terminated        | For a prefix <i>T</i> that is of a task type (after any implicit dereference):<br>Yields the value <i>True</i> if the task denoted by <i>T</i> is terminated, and <i>False</i> otherwise. The value of this attribute is of the predefined type <i>Boolean</i> . See 9.9.                                                                                                                                                                            | 246<br>247               |
| S'Truncation        | For every subtype <i>S</i> of a floating point type <i>T</i> :<br>S'Truncation denotes a function with the following specification:<br><pre>function S'Truncation (X : T)   return T</pre> The function yields the value $\lceil X \rceil$ when <i>X</i> is negative, and $\lfloor X \rfloor$ otherwise. A zero result has the sign of <i>X</i> when S'Signed_Zeros is <i>True</i> . See A.5.3.                                                      | 248<br>249<br>250<br>251 |
| S'Unbiased_Rounding | For every subtype <i>S</i> of a floating point type <i>T</i> :<br>S'Unbiased_Rounding denotes a function with the following specification:<br><pre>function S'Unbiased_Rounding (X : T)   return T</pre> The function yields the integral value nearest to <i>X</i> , rounding toward the even integer if <i>X</i> lies exactly halfway between two integers. A zero result has the sign of <i>X</i> when S'Signed_Zeros is <i>True</i> . See A.5.3. | 252<br>253<br>254<br>255 |
| X'Unchecked_Access  | For a prefix <i>X</i> that denotes an aliased view of an object:<br>All rules and semantics that apply to X'Access (see 3.10.2) apply also to X'Unchecked_Access, except that, for the purposes of accessibility rules and checks, it is as if <i>X</i> were declared immediately within a library package. See 13.10.                                                                                                                               | 256<br>257               |
| S'Val               | For every discrete subtype <i>S</i> :<br>S'Val denotes a function with the following specification:<br><pre>function S'Val (Arg : universal_integer)   return S'Base</pre> This function returns a value of the type of <i>S</i> whose position number equals the value of <i>Arg</i> . See 3.5.5.                                                                                                                                                   | 258<br>259<br>260<br>261 |
| X'Valid             | For a prefix <i>X</i> that denotes a scalar object (after any implicit dereference):<br>Yields <i>True</i> if and only if the object denoted by <i>X</i> is normal and has a valid representation. The value of this attribute is of the predefined type <i>Boolean</i> . See 13.9.2.                                                                                                                                                                | 262<br>263               |
| S'Value             | For every scalar subtype <i>S</i> :<br>S'Value denotes a function with the following specification:<br><pre>function S'Value (Arg : String)   return S'Base</pre>                                                                                                                                                                                                                                                                                    | 264<br>265<br>266        |

|         |                   |                                                                                                                                                                                                                                         |          |
|---------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 267     |                   | This function returns a value given an image of the value as a String, ignoring any leading or trailing spaces. See 3.5.                                                                                                                |          |
| 268/1   | P'Version         | For a prefix P that statically denotes a program unit:                                                                                                                                                                                  |          |
| 269     |                   | Yields a value of the predefined type String that identifies the version of the compilation unit that contains the declaration of the program unit. See E.3.                                                                            |          |
| 270     | S'Wide_Image      | For every scalar subtype S:                                                                                                                                                                                                             |          |
| 271     |                   | S'Wide_Image denotes a function with the following specification:                                                                                                                                                                       |          |
| 272     |                   | <b>function</b> S'Wide_Image( <i>Arg</i> : S'Base)<br><b>return</b> Wide_String                                                                                                                                                         |          |
| 273/2   |                   | The function returns an image of the value of <i>Arg</i> as a Wide_String. See 3.5.                                                                                                                                                     |          |
| 274     | S'Wide_Value      | For every scalar subtype S:                                                                                                                                                                                                             |          |
| 275     |                   | S'Wide_Value denotes a function with the following specification:                                                                                                                                                                       |          |
| 276     |                   | <b>function</b> S'Wide_Value( <i>Arg</i> : Wide_String)<br><b>return</b> S'Base                                                                                                                                                         |          |
| 277     |                   | This function returns a value given an image of the value as a Wide_String, ignoring any leading or trailing spaces. See 3.5.                                                                                                           |          |
| 277.1/2 | S'Wide_Wide_Image | For every scalar subtype S:                                                                                                                                                                                                             |          |
| 277.2/2 |                   | S'Wide_Wide_Image denotes a function with the following specification:                                                                                                                                                                  |          |
| 277.3/2 |                   | <b>function</b> S'Wide_Wide_Image( <i>Arg</i> : S'Base)<br><b>return</b> Wide_Wide_String                                                                                                                                               |          |
| 277.4/2 |                   | The function returns an <i>image</i> of the value of <i>Arg</i> , that is, a sequence of characters representing the value in display form. See 3.5.                                                                                    |          |
| 277.5/2 | S'Wide_Wide_Value | For every scalar subtype S:                                                                                                                                                                                                             |          |
| 277.6/2 |                   | S'Wide_Wide_Value denotes a function with the following specification:                                                                                                                                                                  |          |
| 277.7/2 |                   | <b>function</b> S'Wide_Wide_Value( <i>Arg</i> : Wide_Wide_String)<br><b>return</b> S'Base                                                                                                                                               |          |
| 277.8/2 |                   | This function returns a value given an image of the value as a Wide_Wide_String, ignoring any leading or trailing spaces. See 3.5.                                                                                                      |          |
| 277.9/2 | S'Wide_Wide_Width | For every scalar subtype S:                                                                                                                                                                                                             |          |
|         |                   | S'Wide_Wide_Width denotes the maximum length of a Wide_Wide_String returned by S'Wide_Wide_Image over all values of the subtype S. It denotes zero for a subtype that has a null range. Its type is <i>universal_integer</i> . See 3.5. | 277.10/2 |
| 278     | S'Wide_Width      | For every scalar subtype S:                                                                                                                                                                                                             |          |
| 279     |                   | S'Wide_Width denotes the maximum length of a Wide_String returned by S'Wide_Image over all values of the subtype S. It denotes zero for a subtype that has a null range. Its type is <i>universal_integer</i> . See 3.5.                |          |
| 280     | S'Width           | For every scalar subtype S:                                                                                                                                                                                                             |          |

|               |                                                                                                                                                                                                           |            |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
|               | S'Width denotes the maximum length of a String returned by S'Image over all values of the subtype S. It denotes zero for a subtype that has a null range. Its type is <i>universal_integer</i> . See 3.5. | 281        |
| S'Class'Write | For every subtype S'Class of a class-wide type T'Class:<br>S'Class'Write denotes a procedure with the following specification:                                                                            | 282<br>283 |
|               | <pre>procedure S'Class'Write(<br/>  Stream : not null access Ada.Streams.Root_Stream_Type'Class;<br/>  Item   : in T'Class)</pre>                                                                         | 284/2      |
|               | Dispatches to the subprogram denoted by the Write attribute of the specific type identified by the tag of Item. See 13.13.2.                                                                              | 285        |
| S'Write       | For every subtype S of a specific type T:<br>S'Write denotes a procedure with the following specification:                                                                                                | 286<br>287 |
|               | <pre>procedure S'Write(<br/>  Stream : not null access Ada.Streams.Root_Stream_Type'Class;<br/>  Item   : in T)</pre>                                                                                     | 288/2      |
|               | S'Write writes the value of <i>Item</i> to <i>Stream</i> . See 13.13.2.                                                                                                                                   | 289        |



# Annex L

## (informative)

### Language-Defined Pragmas

|                                                                                                                                                                                                                     |        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| This Annex summarizes the definitions given elsewhere of the language-defined pragmas.                                                                                                                              | 1      |
| <b>pragma</b> All_Calls_Remote( <i>library_unit_name</i> ); — See E.2.3.                                                                                                                                            | 2      |
| <b>pragma</b> Assert([Check =>] <i>boolean_expression</i> [, [Message =>] <i>string_expression</i> ]); — See 11.4.2.                                                                                                | 2.1/2  |
| <b>pragma</b> Assertion_Policy( <i>policy_identifier</i> ); — See 11.4.2.                                                                                                                                           | 2.2/2  |
| <b>pragma</b> Asynchronous( <i>local_name</i> ); — See E.4.1.                                                                                                                                                       | 3      |
| <b>pragma</b> Atomic( <i>local_name</i> ); — See C.6.                                                                                                                                                               | 4      |
| <b>pragma</b> Atomic_Components( <i>array_local_name</i> ); — See C.6.                                                                                                                                              | 5      |
| <b>pragma</b> Attach_Handler( <i>handler_name</i> , <i>expression</i> ); — See C.3.1.                                                                                                                               | 6      |
| <b>pragma</b> Controlled( <i>first_subtype_local_name</i> ); — See 13.11.3.                                                                                                                                         | 7      |
| <b>pragma</b> Convention([Convention =>] <i>convention_identifier</i> , [Entity =>] <i>local_name</i> ); — See B.1.                                                                                                 | 8      |
| <b>pragma</b> Detect_Blocking; — See H.5.                                                                                                                                                                           | 8.1/2  |
| <b>pragma</b> Discard_Names([([On =>] <i>local_name</i> )]); — See C.5.                                                                                                                                             | 9      |
| <b>pragma</b> Elaborate( <i>library_unit_name</i> {, <i>library_unit_name</i> }); — See 10.2.1.                                                                                                                     | 10     |
| <b>pragma</b> Elaborate_All( <i>library_unit_name</i> {, <i>library_unit_name</i> }); — See 10.2.1.                                                                                                                 | 11     |
| <b>pragma</b> Elaborate_Body( <i>library_unit_name</i> ); — See 10.2.1.                                                                                                                                             | 12     |
| <b>pragma</b> Export(<br>[Convention =>] <i>convention_identifier</i> , [Entity =>] <i>local_name</i><br>[, [External_Name =>] <i>string_expression</i> ] [, [Link_Name =>] <i>string_expression</i> ]); — See B.1. | 13     |
| <b>pragma</b> Implemented( <i>procedure_local_name</i> , <i>implementation_kind</i> ); — See 9.5.                                                                                                                   | 13.1/2 |
| <b>pragma</b> Import(<br>[Convention =>] <i>convention_identifier</i> , [Entity =>] <i>local_name</i><br>[, [External_Name =>] <i>string_expression</i> ] [, [Link_Name =>] <i>string_expression</i> ]); — See B.1. | 14     |
| <b>pragma</b> Independent( <i>component_local_name</i> ); — See C.6.                                                                                                                                                | 14.1/3 |
| <b>pragma</b> Independent_Components( <i>local_name</i> ); — See C.6.                                                                                                                                               | 14.2/3 |
| <b>pragma</b> Inline( <i>name</i> {, <i>name</i> }); — See 6.3.2.                                                                                                                                                   | 15     |
| <b>pragma</b> Inspection_Point( <i>object_name</i> {, <i>object_name</i> }); — See H.3.2.                                                                                                                           | 16     |
| <b>pragma</b> Interrupt_Handler( <i>handler_name</i> ); — See C.3.1.                                                                                                                                                | 17     |
| <b>pragma</b> Interrupt_Priority( <i>expression</i> ); — See D.1.                                                                                                                                                   | 18     |
| <b>pragma</b> Linker_Options( <i>string_expression</i> ); — See B.1.                                                                                                                                                | 19     |
| <b>pragma</b> List( <i>identifier</i> ); — See 2.8.                                                                                                                                                                 | 20     |

- 21     **pragma** Locking\_Policy(*policy\_identifier*); — See D.3.
- 21.1/2 **pragma** No\_Return(*procedure\_local\_name*{, *procedure\_local\_name*}); — See 6.5.1.
- 22     **pragma** Normalize\_Scalars; — See H.1.
- 23     **pragma** Optimize(*identifier*); — See 2.8.
- 24     **pragma** Pack(*first\_subtype\_local\_name*); — See 13.2.
- 25     **pragma** Page; — See 2.8.
- 25.1/2 **pragma** Partition\_Elaboration\_Policy (*policy\_identifier*); — See H.6.
- 25.2/2 **pragma** Preelaborable\_Initialization(*direct\_name*); — See 10.2.1.
- 26     **pragma** Preelaborate[(*library\_unit\_name*)]; — See 10.2.1.
- 27     **pragma** Priority(*expression*); — See D.1.
- 27.1/2 **pragma** Priority\_Specific\_Dispatching (  
       *policy\_identifier*, *first\_priority\_expression*, *last\_priority\_expression*); — See D.2.2.
- 27.2/2 **pragma** Profile (*profile\_identifier* {, *profile\_pragma\_argument\_association*}); — See D.13.
- 28     **pragma** Pure[(*library\_unit\_name*)]; — See 10.2.1.
- 29     **pragma** Queuing\_Policy(*policy\_identifier*); — See D.4.
- 29.1/2 **pragma** Relative\_Deadline (*relative\_deadline\_expression*); — See D.2.6.
- 30     **pragma** Remote\_Call\_Interface[(*library\_unit\_name*)]; — See E.2.3.
- 31     **pragma** Remote\_Types[(*library\_unit\_name*)]; — See E.2.2.
- 32     **pragma** Restrictions(*restriction*{, *restriction*}); — See 13.12.
- 33     **pragma** Reviewable; — See H.3.1.
- 34     **pragma** Shared\_Passive[(*library\_unit\_name*)]; — See E.2.1.
- 35     **pragma** Storage\_Size(*expression*); — See 13.3.
- 36     **pragma** Suppress(*identifier*); — See 11.5.
- 37     **pragma** Task\_Dispatching\_Policy(*policy\_identifier*); — See D.2.2.
- 37.1/2 **pragma** Unchecked\_Union (*first\_subtype\_local\_name*); — See B.3.3.
- 37.2/2 **pragma** Unsuppress(*identifier*); — See 11.5.
- 38     **pragma** Volatile(*local\_name*); — See C.6.
- 39     **pragma** Volatile\_Components(*array\_local\_name*); — See C.6.



# Annex M

## (informative)

### Summary of Documentation Requirements

The Ada language allows for certain target machine dependences in a controlled manner. Each Ada implementation must document many characteristics and properties of the target system. This International Standard contains specific documentation requirements. In addition, many characteristics that require documentation are identified throughout this International Standard as being implementation defined. Finally, this International Standard requires documentation of whether implementation advice is followed. The following clauses provide summaries of these documentation requirements.

#### M.1 Specific Documentation Requirements

In addition to implementation-defined characteristics, each Ada implementation must document various properties of the implementation:

- The behavior of implementations in implementation-defined situations shall be documented — see M.2, “Implementation-Defined Characteristics” for a listing. See 1.1.3(19). 2/2
- The set of values that a user-defined Allocate procedure needs to accept for the Alignment parameter. How the standard storage pool is chosen, and how storage is allocated by standard storage pools. See 13.11(22). 3/2
- The algorithm used for random number generation, including a description of its period. See A.5.2(44). 4/2
- The minimum time interval between calls to the time-dependent Reset procedure that is guaranteed to initiate different random number sequences. See A.5.2(45). 5/2
- The conditions under which Io\_Exceptions.Name\_Error, Io\_Exceptions.Use\_Error, and Io\_Exceptions.Device\_Error are propagated. See A.13(15). 6/2
- The behavior of package Environment\_Variables when environment variables are changed by external mechanisms. See A.17(30/2). 7/2
- The overhead of calling machine-code or intrinsic subprograms. See C.1(6). 8/2
- The types and attributes used in machine code insertions. See C.1(7). 9/2
- The subprogram calling conventions for all supported convention identifiers. See C.1(8). 10/2
- The mapping between the Link\_Name or Ada designator and the external link name. See C.1(9). 11/2
- The treatment of interrupts. See C.3(22). 12/2
- The metrics for interrupt handlers. See C.3.1(16). 13/2
- If the Ceiling\_Locking policy is in effect, the default ceiling priority for a protected object that contains an interrupt handler pragma. See C.3.2(24/2). 14/2
- Any circumstances when the elaboration of a preelaborated package causes code to be executed. See C.4(12). 15/2
- Whether a partition can be restarted without reloading. See C.4(13). 16/2
- The effect of calling Current\_Task from an entry body or interrupt handler. See C.7.1(19). 17/2

- 18/2 • For package `Task_Attributes`, limits on the number and size of task attributes, and how to configure any limits. See C.7.2(19).
- 19/2 • The metrics for the `Task_Attributes` package. See C.7.2(27).
- 20/2 • The details of the configuration used to generate the values of all metrics. See D(2).
- 21/2 • The maximum priority inversion a user task can experience from the implementation. See D.2.3(12/2).
- 22/2 • The amount of time that a task can be preempted for processing on behalf of lower-priority tasks. See D.2.3(13/2).
- 23/2 • The quantum values supported for round robin dispatching. See D.2.5(16/2).
- 24/2 • The accuracy of the detection of the exhaustion of the budget of a task for round robin dispatching. See D.2.5(17/2).
- 25/2 • Any conditions that cause the completion of the setting of the deadline of a task to be delayed for a multiprocessor. See D.2.6(32/2).
- 26/2 • Any conditions that cause the completion of the setting of the priority of a task to be delayed for a multiprocessor. See D.5.1(12.1/2).
- 27/2 • The metrics for `Set_Priority`. See D.5.1(14).
- 28/2 • The metrics for setting the priority of a protected object. See D.5.2(10).
- 29/2 • On a multiprocessor, any conditions that cause the completion of an aborted construct to be delayed later than what is specified for a single processor. See D.6(3).
- 30/2 • The metrics for aborts. See D.6(8).
- 31/2 • The values of `Time_First`, `Time_Last`, `Time_Span_First`, `Time_Span_Last`, `Time_Span_Unit`, and `Tick` for package `Real_Time`. See D.8(33).
- 32/2 • The properties of the underlying time base used in package `Real_Time`. See D.8(34).
- 33/2 • Any synchronization of package `Real_Time` with external time references. See D.8(35).
- 34/2 • Any aspects of the external environment that could interfere with package `Real_Time`. See D.8(36/1).
- 35/2 • The metrics for package `Real_Time`. See D.8(45).
- 36/2 • The minimum value of the delay expression of a `delay_relative_statement` that causes a task to actually be blocked. See D.9(7).
- 37/2 • The minimum difference between the value of the delay expression of a `delay_until_statement` and the value of `Real_Time.Clock`, that causes the task to actually be blocked. See D.9(8).
- 38/2 • The metrics for delay statements. See D.9(13).
- 39/2 • The upper bound on the duration of interrupt blocking caused by the implementation. See D.12(5).
- 40/2 • The metrics for entry-less protected objects. See D.12(12).
- 41/2 • The values of `CPU_Time_First`, `CPU_Time_Last`, `CPU_Time_Unit`, and `CPU_Tick` of package `Execution_Time`. See D.14(21/2).
- 42/2 • The properties of the mechanism used to implement package `Execution_Time`. See D.14(22/2).
- 43/2 • The metrics for execution time. See D.14(27).
- 44/2 • The metrics for timing events. See D.15(24).

- Whether the RPC-receiver is invoked from concurrent tasks, and if so, the number of such tasks. See E.5(25). 45/2
- Any techniques used to reduce cancellation errors in Numerics.Generic\_Real\_Arrays shall be documented. See G.3.1(86/2). 46/2
- Any techniques used to reduce cancellation errors in Numerics.Generic\_Complex\_Arrays shall be documented. See G.3.2(155/2). 47/2
- If a `pragma Normalize_Scalars` applies, the implicit initial values of scalar subtypes shall be documented. Such a value should be an invalid representation when possible; any cases when it is not shall be documented. See H.1(5/2). 48/2
- The range of effects for each bounded error and each unspecified effect. If the effects of a given erroneous construct are constrained, the constraints shall be documented. See H.2(1). 49/2
- For each inspection point, a mapping between each inspectable object and the machine resources where the object's value can be obtained shall be provided. See H.3.2(8). 50/2
- If a `pragma Restrictions(No_Exceptions)` is specified, the effects of all constructs where language-defined checks are still performed. See H.4(25). 51/2
- The interrupts to which a task entry may be attached. See J.7.1(12). 52/2
- The type of entry call invoked for an interrupt entry. See J.7.1(13). 53/2

## M.2 Implementation-Defined Characteristics

The Ada language allows for certain machine dependences in a controlled manner. Each Ada implementation must document all implementation-defined characteristics: 1/2

- Whether or not each recommendation given in Implementation Advice is followed — see M.3, “Implementation Advice” for a listing. See 1.1.2(37). 2/2
- Capacity limitations of the implementation. See 1.1.3(3). 3
- Variations from the standard that are impractical to avoid given the implementation's execution environment. See 1.1.3(6). 4
- Which `code_statements` cause external interactions. See 1.1.3(10). 5
- The semantics of an Ada program whose text is not in Normalization Form KC. See 2.1(4.1/2). 5.1/2
- The coded representation for the text of an Ada program. See 2.1(4/3). 6
- *This paragraph was deleted.* 7/2
- The representation for an end of line. See 2.2(2/2). 8
- Maximum supported line length and lexical element length. See 2.2(14). 9
- Implementation-defined pragmas. See 2.8(14). 10
- Effect of `pragma Optimize`. See 2.8(27). 11
- The sequence of characters of the value returned by `S'Wide_Image` when some of the graphic characters of `S'Wide_Wide_Image` are not defined in `Wide_Character`. See 3.5(30/2). 11.1/2
- The sequence of characters of the value returned by `S'Image` when some of the graphic characters of `S'Wide_Wide_Image` are not defined in `Character`. See 3.5(37/2). 12/2
- The predefined integer types declared in Standard. See 3.5.4(25). 13
- Any nonstandard integer types and the operators defined for them. See 3.5.4(26). 14

- 15 • Any nonstandard real types and the operators defined for them. See 3.5.6(8).
- 16 • What combinations of requested decimal precision and range are supported for floating point types. See 3.5.7(7).
- 17 • The predefined floating point types declared in Standard. See 3.5.7(16).
- 18 • The *small* of an ordinary fixed point type. See 3.5.9(8/2).
- 19 • What combinations of *small*, range, and *digits* are supported for fixed point types. See 3.5.9(10).
- 19.1/2 • The sequence of characters of the value returned by Tags.Expanded\_Name (respectively, Tags.Wide\_Expanded\_Name) when some of the graphic characters of Tags.Wide\_Wide\_Expanded\_Name are not defined in Character (respectively, Wide\_Character). See 3.9(10.1/2).
- 20/2 • The result of Tags.Wide\_Wide\_Expanded\_Name for types declared within an unnamed block\_statement. See 3.9(10).
- 21 • Implementation-defined attributes. See 4.1.4(12/1).
- 21.1/2 • Rounding of real static expressions which are exactly half-way between two machine numbers. See 4.9(38/2).
- 22 • Any implementation-defined time types. See 9.6(6).
- 23 • The time base associated with relative delays. See 9.6(20).
- 24 • The time base of the type Calendar.Time. See 9.6(23).
- 25/2 • The time zone used for package Calendar operations. See 9.6(24/2).
- 26 • Any limit on delay\_until\_statements of select\_statements. See 9.6(29).
- 26.1/2 • The result of Calendar.Formatting.Image if its argument represents more than 100 hours. See 9.6.1(86/2).
- 27/3 • *This paragraph was deleted.*
- 28 • The representation for a compilation. See 10.1(2).
- 29 • Any restrictions on compilations that contain multiple compilation\_units. See 10.1(4).
- 29.1/2 • The mechanisms for adding a compilation unit mentioned in a limited\_with\_clause to an environment. See 10.1.4(3/2).
- 30 • The mechanisms for creating an environment and for adding and replacing compilation units. See 10.1.4(3/2).
- 31 • The implementation-defined means, if any, of specifying which compilation units are needed by a given compilation unit. See 10.2(2).
- 32 • The manner of explicitly assigning library units to a partition. See 10.2(2).
- 33 • The manner of designating the main subprogram of a partition. See 10.2(7).
- 34 • The order of elaboration of library\_items. See 10.2(18).
- 35 • Parameter passing and function return for the main subprogram. See 10.2(21).
- 36 • The mechanisms for building and running partitions. See 10.2(24).
- 37 • The details of program execution, including program termination. See 10.2(25).
- 38 • The semantics of any nonactive partitions supported by the implementation. See 10.2(28).
- 39 • The information returned by Exception\_Message. See 11.4.1(10.1/3).

- The sequence of characters of the value returned by `Exceptions.Exception_Name` (respectively, `Exceptions.Wide_Exception_Name`) when some of the graphic characters of `Exceptions.Wide_Wide_Exception_Name` are not defined in `Character` (respectively, `Wide_Character`). See 11.4.1(12.1/2). 39.1/2
- The result of `Exceptions.Wide_Wide_Exception_Name` for exceptions declared within an unnamed `block_statement`. See 11.4.1(12). 40/2
- The information returned by `Exception_Information`. See 11.4.1(13/2). 41
- Implementation-defined *policy\_identifiers* allowed in a `pragma Assertion_Policy`. See 11.4.2(9/2). 41.1/2
- The default assertion policy. See 11.4.2(10/2). 41.2/2
- Existence and meaning of second parameter of `pragma Unsuppress`. See 11.5(27.1/2). 41.3/2
- Implementation-defined check names. See 11.5(27). 42
- The cases that cause conflicts between the representation of the ancestors of a `type_declaration`. See 13.1(13.1/2). 42.1/2
- Any restrictions placed upon representation items. See 13.1(20). 43
- The interpretation of each aspect of representation. See 13.1(20). 44
- The set of machine scalars. See 13.3(8.1/3). 44.1/2
- The meaning of `Size` for indefinite subtypes. See 13.3(48). 45
- The default external representation for a type tag. See 13.3(75/3). 46
- What determines whether a compilation unit is the same in two different partitions. See 13.3(76). 47
- Implementation-defined components. See 13.5.1(15). 48
- If `Word_Size = Storage_Unit`, the default bit ordering. See 13.5.3(5). 49
- The contents of the visible part of package `System`. See 13.7(2). 50/2
- The range of `Storage_Elements.Storage_Offset`, the modulus of `Storage_Elements.Storage_Element`, and the declaration of `Storage_Elements.Integer_Address..`. See 13.7.1(11). 50.1/2
- The contents of the visible part of package `System.Machine_Code`, and the meaning of `code_statements`. See 13.8(7). 51
- The effect of unchecked conversion for instances with nonscalar result types whose effect is not defined by the language. See 13.9(11). 52/2
- The result of unchecked conversion for instances with scalar result types whose result is not defined by the language. See 13.9(11). 52.1/2
- Whether or not the implementation provides user-accessible names for the standard pool type(s). See 13.11(17). 53
- *This paragraph was deleted.* 54/2
- The meaning of `Storage_Size` when neither the `Storage_Size` nor the `Storage_Pool` is specified for an access type. See 13.11(18). 55/2
- *This paragraph was deleted.* 56/2
- The set of restrictions allowed in a `pragma Restrictions`. See 13.12(7/2). 57/2
- The consequences of violating limitations on `Restrictions pragmas`. See 13.12(9). 58

- 59/2 • The contents of the stream elements read and written by the Read and Write attributes of elementary types. See 13.13.2(9).
- 60 • The names and characteristics of the numeric subtypes declared in the visible part of package Standard. See A.1(3).
- 60.1/2 • The values returned by Strings.Hash. See A.4.9(3/2).
- 61 • The accuracy actually achieved by the elementary functions. See A.5.1(1).
- 62 • The sign of a zero result from some of the operators or functions in Numerics.Generic\_Elementary\_Functions, when Float\_Type'Signed\_Zeros is True. See A.5.1(46).
- 63 • The value of Numerics.Discrete\_Random.Max\_Image\_Width. See A.5.2(27).
- 64 • The value of Numerics.Float\_Random.Max\_Image\_Width. See A.5.2(27).
- 65/2 • *This paragraph was deleted.*
- 66 • The string representation of a random number generator's state. See A.5.2(38).
- 67/2 • *This paragraph was deleted.*
- 68 • The values of the Model\_Mantissa, Model\_Emin, Model\_Epsilon, Model\_Safe\_First, and Model\_Safe\_Last attributes, if the Numerics Annex is not supported. See A.5.3(72).
- 69/2 • *This paragraph was deleted.*
- 70 • The value of Buffer\_Size in Storage\_IO. See A.9(10).
- 71/2 • The external files associated with the standard input, standard output, and standard error files. See A.10(5).
- 72 • The accuracy of the value produced by Put. See A.10.9(36).
- 72.1/1 • Current size for a stream file for which positioning is not supported. See A.12.1(1.1/1).
- 73/2 • The meaning of Argument\_Count, Argument, and Command\_Name for package Command\_Line. The bounds of type Command\_Line.Exit\_Status. See A.15(1).
- 73.1/2 • The interpretation of file names and directory names. See A.16(46/2).
- 73.2/2 • The maximum value for a file size in Directories. See A.16(87/2).
- 73.3/2 • The result for Directories.Size for a directory or special file. See A.16(93/2).
- 73.4/2 • The result for Directories.Modification\_Time for a directory or special file. See A.16(95/2).
- 73.5/2 • The interpretation of a non-null search pattern in Directories. See A.16(104/3).
- 73.6/2 • The results of a Directories search if the contents of the directory are altered while a search is in progress. See A.16(110/2).
- 73.7/2 • The definition and meaning of an environment variable. See A.17(1/2).
- 73.8/2 • The circumstances where an environment variable cannot be defined. See A.17(16/2).
- 73.9/2 • Environment names for which Set has the effect of Clear. See A.17(17/2).
- 73.10/2 • The value of Containers.Hash\_Type'Modulus. The value of Containers.Count\_Type'Last. See A.18.1(7/2).
- 74 • Implementation-defined convention names. See B.1(11).
- 75 • The manner of choosing link names when neither the link name nor the address of an imported or exported entity is specified. See B.1(36).

- The meaning of link names. See B.1(36). 76
- The effect of pragma `Linker_Options`. See B.1(37). 77
- The contents of the visible part of package `Interfaces` and its language-defined descendants. See B.2(1). 78
- Implementation-defined children of package `Interfaces`. See B.2(11). 79/2
- The definitions of certain types and constants in `Interfaces.C`. See B.3(41). 79.1/2
- The types `Floating`, `Long_Floating`, `Binary`, `Long_Binary`, `Decimal_Element`, and `COBOL_Character`; and the initializations of the variables `Ada_To_COBOL` and `COBOL_To_Ada`, in `Interfaces.COBOLE`. See B.4(50). 80/1
- The types `Fortran_Integer`, `Real`, `Double_Precision`, and `Character_Set` in `Interfaces.Fortran`. See B.5(17). 80.1/1
- Implementation-defined intrinsic subprograms. See C.1(1). 81/2
- *This paragraph was deleted.* 82/2
- *This paragraph was deleted.* 83/2
- Any restrictions on a protected procedure or its containing type when a pragma `Attach_handler` or `Interrupt_Handler` applies. See C.3.1(17). 83.1/2
- Any other forms of interrupt handler supported by the `Attach_Handler` and `Interrupt_Handler` pragmas. See C.3.1(19). 83.2/2
- *This paragraph was deleted.* 84/2
- The semantics of pragma `Discard_Names`. See C.5(7). 85
- The result of the `Task_Identification.Image` attribute. See C.7.1(7). 86
- The value of `Current_Task` when in a protected entry, interrupt handler, or finalization of a task attribute. See C.7.1(17/3). 87/2
- *This paragraph was deleted.* 88/2
- Granularity of locking for `Task_Attributes`. See C.7.2(16/1). 88.1/1
- *This paragraph was deleted.* 89/2
- *This paragraph was deleted.* 90/2
- The declarations of `Any_Priority` and `Priority`. See D.1(11). 91
- Implementation-defined execution resources. See D.1(15). 92
- Whether, on a multiprocessor, a task that is waiting for access to a protected object keeps its processor busy. See D.2.1(3). 93
- The effect of implementation-defined execution resources on task dispatching. See D.2.1(9/2). 94/2
- *This paragraph was deleted.* 95/2
- *This paragraph was deleted.* 96/2
- Implementation defined task dispatching policies. See D.2.2(18). 97/2
- The value of `Default_Quantum` in `Dispatching.Round_Robin`. See D.2.5(4). 97.1/2
- Implementation-defined *policy\_identifiers* allowed in a pragma `Locking_Policy`. See D.3(4). 98
- The locking policy if no `Locking_Policy` pragma applies to any unit of a partition. See D.3(6). 98.1/2

- 99 • Default ceiling priorities. See D.3(10/2).
- 100 • The ceiling of any protected object used internally by the implementation. See D.3(16).
- 101 • Implementation-defined queuing policies. See D.4(1/1).
- 102/2 • *This paragraph was deleted.*
- 103 • Any operations that implicitly require heap storage allocation. See D.7(8).
- 103.1/2 • When restriction No\_Task\_Termination applies to a partition, what happens when a task terminates. See D.7(15.1/2).
- 103.2/2 • The behavior when restriction Max\_Storage\_At\_Blocking is violated. See D.7(17/1).
- 103.3/2 • The behavior when restriction Max\_Asynchronous\_Select\_Nesting is violated. See D.7(18/1).
- 103.4/2 • The behavior when restriction Max\_Tasks is violated. See D.7(19).
- 104/2 • Whether the use of pragma Restrictions results in a reduction in program code or data size or execution time. See D.7(20).
- 105/2 • *This paragraph was deleted.*
- 106/2 • *This paragraph was deleted.*
- 107/2 • *This paragraph was deleted.*
- 108 • The means for creating and executing distributed programs. See E(5).
- 109 • Any events that can result in a partition becoming inaccessible. See E.1(7).
- 110 • The scheduling policies, treatment of priorities, and management of shared resources between partitions in certain cases. See E.1(11).
- 111/1 • *This paragraph was deleted.*
- 112 • Whether the execution of the remote subprogram is immediately aborted as a result of cancellation. See E.4(13).
- 112.1/2 • The range of type System.RPC.Partition\_Id. See E.5(14).
- 113/2 • *This paragraph was deleted.*
- 114 • Implementation-defined interfaces in the PCS. See E.5(26).
- 115 • The values of named numbers in the package Decimal. See F.2(7).
- 116 • The value of Max\_Picture\_Length in the package Text\_IO.Editing See F.3.3(16).
- 117 • The value of Max\_Picture\_Length in the package Wide\_Text\_IO.Editing See F.3.4(5).
- 117.1/2 • The value of Max\_Picture\_Length in the package Wide\_Wide\_Text\_IO.Editing See F.3.5(5).
- 118 • The accuracy actually achieved by the complex elementary functions and by other complex arithmetic operations. See G.1(1).
- 119 • The sign of a zero result (or a component thereof) from any operator or function in Numerics.Generic\_Complex\_Types, when Real'Signed\_Zeros is True. See G.1.1(53).
- 120 • The sign of a zero result (or a component thereof) from any operator or function in Numerics.Generic\_Complex\_Elementary\_Functions, when Complex\_Types.Real'Signed\_Zeros is True. See G.1.2(45).
- 121 • Whether the strict mode or the relaxed mode is the default. See G.2(2).
- 122 • The result interval in certain cases of fixed-to-float conversion. See G.2.1(10).



- The result of a floating point arithmetic operation in overflow situations, when the Machine\_Overflows attribute of the result type is False. See G.2.1(13). 123
- The result interval for division (or exponentiation by a negative exponent), when the floating point hardware implements division as multiplication by a reciprocal. See G.2.1(16). 124
- The definition of *close result set*, which determines the accuracy of certain fixed point multiplications and divisions. See G.2.3(5). 125
- Conditions on a *universal real* operand of a fixed point multiplication or division for which the result shall be in the *perfect result set*. See G.2.3(22). 126
- The result of a fixed point arithmetic operation in overflow situations, when the Machine\_Overflows attribute of the result type is False. See G.2.3(27). 127
- The result of an elementary function reference in overflow situations, when the Machine\_Overflows attribute of the result type is False. See G.2.4(4). 128
- The accuracy of certain elementary functions for parameters beyond the angle threshold. See G.2.4(10). 129
- The value of the *angle threshold*, within which certain elementary functions, complex arithmetic operations, and complex elementary functions yield results conforming to a maximum relative error bound. See G.2.4(10). 130
- The result of a complex arithmetic operation or complex elementary function reference in overflow situations, when the Machine\_Overflows attribute of the corresponding real type is False. See G.2.6(5). 131
- The accuracy of certain complex arithmetic operations and certain complex elementary functions for parameters (or components thereof) beyond the angle threshold. See G.2.6(8). 132
- The accuracy requirements for the subprograms Solve, Inverse, Determinant, Eigenvalues and Eigensystem for type Real\_Matrix. See G.3.1(81/2). 132.1/2
- The accuracy requirements for the subprograms Solve, Inverse, Determinant, Eigenvalues and Eigensystem for type Complex\_Matrix. See G.3.2(149/2). 132.2/2
- *This paragraph was deleted.* 133/2
- *This paragraph was deleted.* 134/2
- *This paragraph was deleted.* 135/2
- *This paragraph was deleted.* 136/2
- Implementation-defined *policy\_identifiers* allowed in a pragma Partition\_Elaboration\_Policy. See H.6(4/2). 136.1/2

## M.3 Implementation Advice

This International Standard sometimes gives advice about handling certain target machine dependences. Each Ada implementation must document whether that advice is followed: 1/2

- Program\_Error should be raised when an unsupported Specialized Needs Annex feature is used at run time. See 1.1.3(20). 2/2
- Implementation-defined extensions to the functionality of a language-defined library unit should be provided by adding children to the library unit. See 1.1.3(21). 3/2
- If a bounded error or erroneous execution is detected, Program\_Error should be raised. See 1.1.5(12). 4/2

- 5/2 • Implementation-defined pragmas should have no semantic effect for error-free programs. See 2.8(16).
- 6/2 • Implementation-defined pragmas should not make an illegal program legal, unless they complete a declaration or configure the `library_items` in an environment. See 2.8(19).
- 7/2 • `Long_Integer` should be declared in Standard if the target supports 32-bit arithmetic. No other named integer subtypes should be declared in Standard. See 3.5.4(28).
- 8/2 • For a two's complement target, modular types with a binary modulus up to `System.Max_Int*2+2` should be supported. A nonbinary modulus up to `Integer'Last` should be supported. See 3.5.4(29).
- 9/2 • `Program_Error` should be raised for the evaluation of `S'Pos` for an enumeration type, if the value of the operand does not correspond to the internal code for any enumeration literal of the type. See 3.5.5(8).
- 10/2 • `Long_Float` should be declared in Standard if the target supports 11 or more digits of precision. No other named float subtypes should be declared in Standard. See 3.5.7(17).
- 11/2 • Multidimensional arrays should be represented in row-major order, unless the array has convention `Fortran`. See 3.6.2(11).
- 11.1/3 • `Tags.Internal_Tag` should return the tag of a type, if one exists, whose innermost master is a master of the point of the function call.. See 3.9(26.1/3).
- 12/2 • For a real static expression with a non-formal type that is not part of a larger static expression should be rounded the same as the target system. See 4.9(38.1/2).
- 13/2 • The value of `Duration'Small` should be no greater than 100 microseconds. See 9.6(30).
- 14/2 • The time base for `delay_relative_statements` should be monotonic. See 9.6(31).
- 15/2 • Leap seconds should be supported if the target system supports them. Otherwise, operations in `Calendar.Formatting` should return results consistent with no leap seconds. See 9.6.1(89/2).
- 16/2 • When applied to a generic unit, a program unit pragma that is not a library unit pragma should apply to each instance of the generic unit for which there is not an overriding pragma applied directly to the instance. See 10.1.5(10/1).
- 17/2 • A type declared in a preelaborated package should have the same representation in every elaboration of a given version of the package. See 10.2.1(12).
- 18/2 • `Exception_Message` by default should be short, provide information useful for debugging, and should not include the `Exception_Name`. See 11.4.1(19).
- 19/2 • `Exception_Information` should provide information useful for debugging, and should include the `Exception_Name` and `Exception_Message`. See 11.4.1(19).
- 20/2 • Code executed for checks that have been suppressed should be minimized. See 11.5(28).
- 21/2 • The recommended level of support for all representation items should be followed. See 13.1(28/2).
- 22/2 • Storage allocated to objects of a packed type should be minimized. See 13.2(6).
- 23/2 • The recommended level of support for pragma `Pack` should be followed. See 13.2(9).
- 24/2 • For an array `X`, `X'Address` should point at the first component of the array rather than the array bounds. See 13.3(14).
- 25/2 • The recommended level of support for the `Address` attribute should be followed. See 13.3(19).
- 25.1/3 • For any tagged specific subtype `S`, `S'Class'Alignment` should equal `S'Alignment`. See 13.3(28).

- The recommended level of support for the Alignment attribute should be followed. See 13.3(35). 26/2
- The Size of an array object should not include its bounds. See 13.3(41.1/2). 27/2
- If the Size of a subtype allows for efficient independent addressability, then the Size of most objects of the subtype should equal the Size of the subtype. See 13.3(52). 28/2
- A Size clause on a composite subtype should not affect the internal layout of components. See 13.3(53). 29/2
- The recommended level of support for the Size attribute should be followed. See 13.3(56). 30/2
- The recommended level of support for the Component\_Size attribute should be followed. See 13.3(73). 31/2
- The recommended level of support for enumeration\_representation\_clauses should be followed. See 13.4(10). 32/2
- The recommended level of support for record\_representation\_clauses should be followed. See 13.5.1(22). 33/2
- If a component is represented using a pointer to the actual data of the component which is contiguous with the rest of the object, then the storage place attributes should reflect the place of the actual data. If a component is allocated discontinuously from the rest of the object, then a warning should be generated upon reference to one of its storage place attributes. See 13.5.2(5). 34/2
- The recommended level of support for the nondefault bit ordering should be followed. See 13.5.3(8). 35/2
- Type System.Address should be a private type. See 13.7(37). 36/2
- Operations in System and its children should reflect the target environment; operations that do not make sense should raise Program\_Error. See 13.7.1(16). 37/2
- Since the Size of an array object generally does not include its bounds, the bounds should not be part of the converted data in an instance of Unchecked\_Conversion. See 13.9(14/2). 38/2
- There should not be unnecessary run-time checks on the result of an Unchecked\_Conversion; the result should be returned by reference when possible. Restrictions on Unchecked\_Conversions should be avoided. See 13.9(15). 39/2
- The recommended level of support for Unchecked\_Conversion should be followed. See 13.9(17). 40/2
- Any cases in which heap storage is dynamically allocated other than as part of the evaluation of an allocator should be documented. See 13.11(23). 41/2
- A default storage pool for an access-to-constant type should not have overhead to support deallocation of individual objects. See 13.11(24). 42/2
- Usually, a storage pool for an access discriminant or access parameter should be created at the point of an allocator, and be reclaimed when the designated object becomes inaccessible. For other anonymous access types, the pool should be created at the point where the type is elaborated and need not support deallocation of individual objects. See 13.11(25). 43/2
- A call on an instance of Unchecked\_Deallocation with a non-null access value should raise Program\_Error if the actual access type of the instance is a type for which the Storage\_Size has been specified to be zero or is defined by the language to be zero. See 13.11.2(17.1/3). 43.1/3
- For a standard storage pool, an instance of Unchecked\_Deallocation should actually reclaim the storage. See 13.11.2(17). 44/2

- 45/2 • The recommended level of support for the `Stream_Size` attribute should be followed. See 13.13.2(1.8/2).
- 46/2 • If not specified, the value of `Stream_Size` for an elementary type should be the number of bits that corresponds to the minimum number of stream elements required by the first subtype of the type, rounded up to the nearest factor or multiple of the word size that is also a multiple of the stream element size. See 13.13.2(1.6/2).
- 47/2 • If an implementation provides additional named predefined integer types, then the names should end with “Integer”. If an implementation provides additional named predefined floating point types, then the names should end with “Float”. See A.1(52).
- 48/2 • Implementation-defined operations on `Wide_Character`, `Wide_String`, `Wide_Wide_Character`, and `Wide_Wide_String` should be child units of `Wide_Characters` or `Wide_Wide_Characters`. See A.3.1(7/2).
- 49/2 • Bounded string objects should not be implemented by implicit pointers and dynamic allocation. See A.4.4(106).
- 50/2 • `Strings.Hash` should be good a hash function, returning a wide spread of values for different string values, and similar strings should rarely return the same value. See A.4.9(15/2).
- 50.1/3 • If an implementation supports other string encoding schemes, a child of `Ada.Strings` similar to `UTF_Encoding` should be defined. See A.4.11(86/3).
- 51/2 • Any storage associated with an object of type `Generator` of the random number packages should be reclaimed on exit from the scope of the object. See A.5.2(46).
- 52/2 • Each value of `Initiator` passed to `Reset` for the random number packages should initiate a distinct sequence of random numbers, or, if that is not possible, be at least a rapidly varying function of the initiator value. See A.5.2(47).
- 53/2 • `Get_Immediate` should be implemented with unbuffered input; input should be available immediately; line-editing should be disabled. See A.10.7(23).
- 54/2 • `Package Directories.Information` should be provided to retrieve other information about a file. See A.16(124/2).
- 55/2 • `Directories.Start_Search` and `Directories.Search` should raise `Use_Error` for malformed patterns. See A.16(125/2).
- 56/2 • `Directories.Rename` should be supported at least when both `New_Name` and `Old_Name` are simple names and `New_Name` does not identify an existing external file. See A.16(126/2).
- 56.1/3 • `Directories.Hierarchical_File_Names` should be provided for systems with hierarchical file naming, and should not be provided on other systems. See A.16.1(36/3).
- 57/2 • If the execution environment supports subprocesses, the current environment variables should be used to initialize the environment variables of a subprocess. See A.17(32/2).
- 58/2 • Changes to the environment variables made outside the control of `Environment_Variables` should be reflected immediately. See A.17(33/2).
- 59/2 • `Containers.Hash_Type'Modulus` should be at least  $2^{*32}$ . `Containers.Count_Type'Last` should be at least  $2^{*31}-1$ . See A.18.1(8/2).
- 60/2 • The worst-case time complexity of `Element` for `Containers.Vector` should be  $O(\log N)$ . See A.18.2(256/2).
- 61/2 • The worst-case time complexity of `Append` with `Count = 1` when  $N$  is less than the capacity for `Containers.Vector` should be  $O(\log N)$ . See A.18.2(257).

- The worst-case time complexity of Prepend with Count = 1 and Delete\_First with Count=1 for Containers.Vectors should be  $O(N \log N)$ . See A.18.2(258/2). 62/2
- The worst-case time complexity of a call on procedure Sort of an instance of Containers.Vectors.Generic\_Sorting should be  $O(N^{**2})$ , and the average time complexity should be better than  $O(N^{**2})$ . See A.18.2(259/2). 63/2
- Containers.Vectors.Generic\_Sorting.Sort and Containers.Vectors.Generic\_Sorting.Merge should minimize copying of elements. See A.18.2(260/2). 64/2
- Containers.Vectors.Move should not copy elements, and should minimize copying of internal data structures. See A.18.2(261/2). 65/2
- If an exception is propagated from a vector operation, no storage should be lost, nor any elements removed from a vector unless specified by the operation. See A.18.2(262/2). 66/2
- The worst-case time complexity of Element, Insert with Count=1, and Delete with Count=1 for Containers.Doubly\_Linked\_Lists should be  $O(\log N)$ . See A.18.3(160/2). 67/2
- A call on procedure Sort of an instance of Containers.Doubly\_Linked\_Lists.Generic\_Sorting should have an average time complexity better than  $O(N^{**2})$  and worst case no worse than  $O(N^{**2})$ . See A.18.3(161/2). 68/2
- Containers.Doubly\_Linked\_Lists.Move should not copy elements, and should minimize copying of internal data structures. See A.18.3(162/2). 69/2
- If an exception is propagated from a list operation, no storage should be lost, nor any elements removed from a list unless specified by the operation. See A.18.3(163/2). 70/2
- Move for a map should not copy elements, and should minimize copying of internal data structures. See A.18.4(83/2). 71/2
- If an exception is propagated from a map operation, no storage should be lost, nor any elements removed from a map unless specified by the operation. See A.18.4(84/2). 72/2
- The average time complexity of Element, Insert, Include, Replace, Delete, Exclude and Find operations that take a key parameter for Containers.Hashed\_Maps should be  $O(\log N)$ . The average time complexity of the subprograms of Containers.Hashed\_Maps that take a cursor parameter should be  $O(1)$ . The average time complexity of Containers.Hashed\_Maps.Reserve\_Capacity should be  $O(N)$ . See A.18.5(62/2). 73/2
- The worst-case time complexity of Element, Insert, Include, Replace, Delete, Exclude and Find operations that take a key parameter for Containers.Ordered\_Maps should be  $O((\log N)^{**2})$  or better. The worst-case time complexity of the subprograms of Containers.Ordered\_Maps that take a cursor parameter should be  $O(1)$ . See A.18.6(95/2). 74/2
- Move for sets should not copy elements, and should minimize copying of internal data structures. See A.18.7(104/2). 75/2
- If an exception is propagated from a set operation, no storage should be lost, nor any elements removed from a set unless specified by the operation. See A.18.7(105/2). 76/2
- The average time complexity of the Insert, Include, Replace, Delete, Exclude and Find operations of Containers.Hashed\_Sets that take an element parameter should be  $O(\log N)$ . The average time complexity of the subprograms of Containers.Hashed\_Sets that take a cursor parameter should be  $O(1)$ . The average time complexity of Containers.Hashed\_Sets.-Reserve\_Capacity should be  $O(N)$ . See A.18.8(88/2). 77/2
- The worst-case time complexity of the Insert, Include, Replace, Delete, Exclude and Find operations of Containers.Ordered\_Sets that take an element parameter should be  $O((\log N)^{**2})$ . 78/2

The worst-case time complexity of the subprograms of Containers.Ordered\_Sets that take a cursor parameter should be  $O(1)$ . See A.18.9(116/2).

- 78.1/3 • The worst-case time complexity of the Element, Parent, First\_Child, Last\_Child, Insert\_Child with Count=1, and Delete operations of Containers.Multiway\_Trees should be  $O(\log N)$ . See A.18.10(204/3).
- 78.2/3 • Containers.Multiway\_Trees.Move should not copy elements, and should minimize copying of internal data structures. See A.18.10(205/3).
- 78.3/3 • If an exception is propagated from a tree operation, no storage should be lost, nor any elements removed from a tree unless specified by the operation. See A.18.10(206/3).
- 78.4/3 • Bounded vector objects should be implemented without implicit pointers or dynamic allocation. See A.18.19(11/3).
- 78.5/3 • The implementation advice for procedure Move to minimize copying does not apply to bounded vectors. See A.18.19(12/3).
- 78.6/3 • Bounded list objects should be implemented without implicit pointers or dynamic allocation. See A.18.20(15/3).
- 78.7/3 • The implementation advice for procedure Move to minimize copying does not apply to bounded lists. See A.18.20(16/3).
- 78.8/3 • Bounded map objects should be implemented without implicit pointers or dynamic allocation. See A.18.21(16/3).
- 78.9/3 • The implementation advice for procedure Move to minimize copying does not apply to bounded maps. See A.18.21(17/3).
- 78.10/3 • Bounded set objects should be implemented without implicit pointers or dynamic allocation. See A.18.23(16/3).
- 78.11/3 • The implementation advice for procedure Move to minimize copying does not apply to bounded sets. See A.18.23(17/3).
- 78.12/3 • Bounded tree objects should be implemented without implicit pointers or dynamic allocation. See A.18.25(15/3).
- 78.13/3 • The implementation advice for procedure Move to minimize copying does not apply to bounded trees. See A.18.25(16/3).
- 79/2 • Containers.Generic\_Array\_Sort and Containers.Generic\_Constrained\_Array\_Sort should have an average time complexity better than  $O(N**2)$  and worst case no worse than  $O(N**2)$ . See A.18.26(10/2).
- 80/2 • Containers.Generic\_Array\_Sort and Containers.Generic\_Constrained\_Array\_Sort should minimize copying of elements. See A.18.26(11/2).
- 81/2 • If pragma Export is supported for a language, the main program should be able to be written in that language. Subprograms named "adainit" and "adafinal" should be provided for elaboration and finalization of the environment task. See B.1(39).
- 82/2 • Automatic elaboration of preelaborated packages should be provided when pragma Export is supported. See B.1(40).
- 83/2 • For each supported convention *L* other than Intrinsic, pragmas Import and Export should be supported for objects of *L*-compatible types and for subprograms, and pragma Convention should be supported for *L*-eligible types and for subprograms. See B.1(41).
- 84/2 • If an interface to C, COBOL, or Fortran is provided, the corresponding package or packages described in Annex B, "Interface to Other Languages" should also be provided. See B.2(13).

- The constants `nul`, `wide_nul`, `char16_nul`, and `char32_nul` in package `Interfaces.C` should have a representation of zero. See B.3(62.5/2). 85/2
- If C interfacing is supported, the interface correspondences between Ada and C should be supported. See B.3(71). 86/2
- If COBOL interfacing is supported, the interface correspondences between Ada and COBOL should be supported. See B.4(98). 87/2
- If Fortran interfacing is supported, the interface correspondences between Ada and Fortran should be supported. See B.5(26). 88/2
- The machine code or intrinsics support should allow access to all operations normally available to assembly language programmers for the target environment. See C.1(3). 89/2
- Interface to assembler should be supported; the default assembler should be associated with the convention identifier `Assembler`. See C.1(4). 90/2
- If an entity is exported to assembly language, then the implementation should allocate it at an addressable location even if not otherwise referenced from the Ada code. A call to a machine code or assembler subprogram should be treated as if it could read or update every object that is specified as exported. See C.1(5). 91/2
- Little or no overhead should be associated with calling intrinsic and machine-code subprograms. See C.1(10). 92/2
- Intrinsic subprograms should be provided to access any machine operations that provide special capabilities or efficiency not normally available. See C.1(16). 93/2
- If the `Ceiling_Locking` policy is not in effect and the target system allows for finer-grained control of interrupt blocking, a means for the application to specify which interrupts are to be blocked during protected actions should be provided. See C.3(28/2). 94/2
- Interrupt handlers should be called directly by the hardware. See C.3.1(20). 95/2
- Violations of any implementation-defined restrictions on interrupt handlers should be detected before run time. See C.3.1(21). 96/2
- If implementation-defined forms of interrupt handler procedures are supported, then for each such form of a handler, a type analogous to `Parameterless_Handler` should be specified in a child package of `Interrupts`, with the same operations as in the predefined package `Interrupts`. See C.3.2(25). 97/2
- Preelaborated packages should be implemented such that little or no code is executed at run time for the elaboration of entities. See C.4(14). 98/2
- If `pragma Discard_Names` applies to an entity, then the amount of storage used for storing names associated with that entity should be reduced. See C.5(8). 99/2
- A load or store of a volatile object whose size is a multiple of `System.Storage_Unit` and whose alignment is nonzero, should be implemented by accessing exactly the bits of the object and no others. See C.6(22/2). 100/2
- A load or store of an atomic object should be implemented by a single load or store instruction. See C.6(23/2). 101/2
- Finalization of task attributes and reclamation of associated storage should be performed as soon as possible after task termination. See C.7.2(30.1/2). 102/2
- If the target domain requires deterministic memory use at run time, storage for task attributes should be pre-allocated statically and the number of attributes pre-allocated should be documented. See C.7.2(30). 103/2

- 104/2 • Names that end with “\_Locking” should be used for implementation-defined locking policies. See D.3(17).
- 105/2 • Names that end with “\_Queuing” should be used for implementation-defined queuing policies. See D.4(16).
- 106/2 • The `abort_statement` should not require the task executing the statement to block. See D.6(9).
- 107/2 • On a multi-processor, the delay associated with aborting a task on another processor should be bounded. See D.6(10).
- 108/2 • When feasible, specified restrictions should be used to produce a more efficient implementation. See D.7(21).
- 109/2 • When appropriate, mechanisms to change the value of `Tick` should be provided. See D.8(47).
- 110/2 • `Calendar.Clock` and `Real_Time.Clock` should be transformations of the same time base. See D.8(48).
- 111/2 • The “best” time base which exists in the underlying system should be available to the application through `Real_Time.Clock`. See D.8(49).
- 112/2 • When appropriate, implementations should provide configuration mechanisms to change the value of `Execution_Time.CPU_Tick`. See D.14(29/2).
- 113/2 • For a timing event, the handler should be executed directly by the real-time clock interrupt mechanism. See D.15(25).
- 114/2 • The PCS should allow for multiple tasks to call the RPC-receiver. See E.5(28).
- 115/2 • The `System.RPC.Write` operation should raise `Storage_Error` if it runs out of space when writing an item. See E.5(29).
- 116/2 • If COBOL (respectively, C) is supported in the target environment, then interfacing to COBOL (respectively, C) should be supported as specified in Annex B. See F(7).
- 117/2 • Packed decimal should be used as the internal representation for objects of subtype *S* when `S'Machine_Radix = 10`. See F.1(2).
- 118/2 • If Fortran (respectively, C) is supported in the target environment, then interfacing to Fortran (respectively, C) should be supported as specified in Annex B. See G(7).
- 119/2 • Mixed real and complex operations (as well as pure-imaginary and complex operations) should not be performed by converting the real (resp. pure-imaginary) operand to complex. See G.1.1(56).
- 120/2 • If `Real'Signed_Zeros` is true for `Numerics.Generic_Complex_Types`, a rational treatment of the signs of zero results and result components should be provided. See G.1.1(58).
- 121/2 • If `Complex_Types.Real'Signed_Zeros` is true for `Numerics.Generic_Complex_Elementary_-Functions`, a rational treatment of the signs of zero results and result components should be provided. See G.1.2(49).
- 122/2 • For elementary functions, the forward trigonometric functions without a `Cycle` parameter should not be implemented by calling the corresponding version with a `Cycle` parameter. `Log` without a `Base` parameter should not be implemented by calling `Log` with a `Base` parameter. See G.2.4(19).
- 123/2 • For complex arithmetic, the `Compose_From_Polar` function without a `Cycle` parameter should not be implemented by calling `Compose_From_Polar` with a `Cycle` parameter. See G.2.6(15).



- Solve and Inverse for Numerics.Generic\_Real\_Arrays should be implemented using established techniques such as LU decomposition and the result should be refined by an iteration on the residuals. See G.3.1(88/2). 124/2
- The equality operator should be used to test that a matrix in Numerics.Generic\_Real\_Arrays is symmetric. See G.3.1(90/2). 125/2
- An implementation should minimize the circumstances under which the algorithm used for Numerics.Generic\_Real\_Arrays.Eigenvalues and Numerics.Generic\_Real\_Arrays.Eigensystem fails to converge. See G.3.1(91/3). 125.1/3
- Solve and Inverse for Numerics.Generic\_Complex\_Arrays should be implemented using established techniques and the result should be refined by an iteration on the residuals. See G.3.2(158/2). 126/2
- An implementation should minimize the circumstances under which the algorithm used for Numerics.Generic\_Complex\_Arrays.Eigenvalues and Numerics.Generic\_Complex\_Arrays.Eigensystem fails to converge. See G.3.2(160.1/3). 126.1/3
- The equality and negation operators should be used to test that a matrix is Hermitian. See G.3.2(160/2). 127/2
- Mixed real and complex operations should not be performed by converting the real operand to complex. See G.3.2(161/2). 128/2
- The information produced by pragma Reviewable should be provided in both a human-readable and machine-readable form, and the latter form should be documented. See H.3.1(19). 129/2
- Object code listings should be provided both in a symbolic format and in a numeric format. See H.3.1(20). 130/2
- If the partition elaboration policy is Sequential and the Environment task becomes permanently blocked during elaboration then the partition should be immediately terminated. See H.6(15/2). 131/2



## Annex N

### (informative)

## Glossary

|                                                                                                                                                                                                                                                                                                                                                                 |       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| This Annex contains informal descriptions of some of the terms used in this International Standard. The index provides references to more formal definitions of all of the terms used in this International Standard.                                                                                                                                           | 1/2   |
| <b>Abstract type.</b> An abstract type is a tagged type intended for use as an ancestor of other types, but which is not allowed to have objects of its own.                                                                                                                                                                                                    | 1.1/2 |
| <b>Access type.</b> An access type has values that designate aliased objects. Access types correspond to “pointer types” or “reference types” in some other languages.                                                                                                                                                                                          | 2     |
| <b>Aliased.</b> An aliased view of an object is one that can be designated by an access value. Objects allocated by allocators are aliased. Objects can also be explicitly declared as aliased with the reserved word <b>aliased</b> . The Access attribute can be used to create an access value designating an aliased object.                                | 3     |
| <b>Ancestor.</b> An ancestor of a type is the type itself or, in the case of a type derived from other types, its parent type or one of its progenitor types or one of their ancestors. Note that ancestor and descendant are inverse relationships.                                                                                                            | 3.1/2 |
| <b>Array type.</b> An array type is a composite type whose components are all of the same type. Components are selected by indexing.                                                                                                                                                                                                                            | 4     |
| <b>Category (of types).</b> A category of types is a set of types with one or more common properties, such as primitive operations. A category of types that is closed under derivation is also known as a <i>class</i> .                                                                                                                                       | 4.1/2 |
| <b>Character type.</b> A character type is an enumeration type whose values include characters.                                                                                                                                                                                                                                                                 | 5     |
| <b>Class (of types).</b> A class is a set of types that is closed under derivation, which means that if a given type is in the class, then all types derived from that type are also in the class. The set of types of a class share common properties, such as their primitive operations.                                                                     | 6/2   |
| <b>Compilation unit.</b> The text of a program can be submitted to the compiler in one or more compilations. Each compilation is a succession of <code>compilation_units</code> . A <code>compilation_unit</code> contains either the declaration, the body, or a renaming of a program unit.                                                                   | 7     |
| <b>Composite type.</b> A composite type may have components.                                                                                                                                                                                                                                                                                                    | 8/2   |
| <b>Construct.</b> A <i>construct</i> is a piece of text (explicit or implicit) that is an instance of a syntactic category defined under “Syntax”.                                                                                                                                                                                                              | 9     |
| <b>Controlled type.</b> A controlled type supports user-defined assignment and finalization. Objects are always finalized before being destroyed.                                                                                                                                                                                                               | 10    |
| <b>Declaration.</b> A <i>declaration</i> is a language construct that associates a name with (a view of) an entity. A declaration may appear explicitly in the program text (an <i>explicit</i> declaration), or may be supposed to occur at a given place in the text as a consequence of the semantics of another construct (an <i>implicit</i> declaration). | 11    |
| <i>This paragraph was deleted.</i>                                                                                                                                                                                                                                                                                                                              | 12/2  |
| <b>Derived type.</b> A derived type is a type defined in terms of one or more other types given in a derived type definition. The first of those types is the parent type of the derived type and any others are progenitor                                                                                                                                     | 13/2  |

types. Each class containing the parent type or a progenitor type also contains the derived type. The derived type inherits properties such as components and primitive operations from the parent and progenitors. A type together with the types derived from it (directly or indirectly) form a derivation class.

- 13.1/2 **Descendant.** A type is a descendant of itself, its parent and progenitor types, and their ancestors. Note that descendant and ancestor are inverse relationships.
- 14 **Discrete type.** A discrete type is either an integer type or an enumeration type. Discrete types may be used, for example, in `case_statements` and as array indices.
- 15/2 **Discriminant.** A discriminant is a parameter for a composite type. It can control, for example, the bounds of a component of the type if the component is an array. A discriminant for a task type can be used to pass data to a task of the type upon creation.
- 15.1/2 **Elaboration.** The process by which a declaration achieves its run-time effect is called elaboration. Elaboration is one of the forms of execution.
- 16 **Elementary type.** An elementary type does not have components.
- 17 **Enumeration type.** An enumeration type is defined by an enumeration of its values, which may be named by identifiers or character literals.
- 17.1/2 **Evaluation.** The process by which an expression achieves its run-time effect is called evaluation. Evaluation is one of the forms of execution.
- 18 **Exception.** An *exception* represents a kind of exceptional situation; an occurrence of such a situation (at run time) is called an *exception occurrence*. To *raise* an exception is to abandon normal program execution so as to draw attention to the fact that the corresponding situation has arisen. Performing some actions in response to the arising of an exception is called *handling* the exception.
- 19 **Execution.** The process by which a construct achieves its run-time effect is called *execution*. Execution of a declaration is also called *elaboration*. Execution of an expression is also called *evaluation*.
- 19.1/2 **Function.** A function is a form of subprogram that returns a result and can be called as part of an expression.
- 20 **Generic unit.** A generic unit is a template for a (nongeneric) program unit; the template can be parameterized by objects, types, subprograms, and packages. An instance of a generic unit is created by a `generic_instantiation`. The rules of the language are enforced when a generic unit is compiled, using a generic contract model; additional checks are performed upon instantiation to verify the contract is met. That is, the declaration of a generic unit represents a contract between the body of the generic and instances of the generic. Generic units can be used to perform the role that macros sometimes play in other languages.
- 20.1/2 **Incomplete type.** An incomplete type gives a view of a type that reveals only some of its properties. The remaining properties are provided by the full view given elsewhere. Incomplete types can be used for defining recursive data structures.
- 21 **Integer type.** Integer types comprise the signed integer types and the modular types. A signed integer type has a base range that includes both positive and negative numbers, and has operations that may raise an exception when the result is outside the base range. A modular type has a base range whose lower bound is zero, and has operations with “wraparound” semantics. Modular types subsume what are called “unsigned types” in some other languages.

- Interface type.** An interface type is a form of abstract tagged type which has no components or concrete operations except possibly null procedures. Interface types are used for composing other interfaces and tagged types and thereby provide multiple inheritance. Only an interface type can be used as a progenitor of another type. 21.1/2
- Library unit.** A library unit is a separately compiled program unit, and is always a package, subprogram, or generic unit. Library units may have other (logically nested) library units as children, and may have other program units physically nested within them. A root library unit, together with its children and grandchildren and so on, form a *subsystem*. 22
- Limited type.** A limited type is a type for which copying (such as in an `assignment_statement`) is not allowed. A nonlimited type is a type for which copying is allowed. 23/2
- Object.** An object is either a constant or a variable. An object contains a value. An object is created by an `object_declaration` or by an `allocator`. A formal parameter is (a view of) an object. A subcomponent of an object is an object. 24
- Overriding operation.** An overriding operation is one that replaces an inherited primitive operation. Operations may be marked explicitly as overriding or not overriding. 24.1/2
- Package.** Packages are program units that allow the specification of groups of logically related entities. Typically, a package contains the declaration of a type (often a private type or private extension) along with the declarations of primitive subprograms of the type, which can be called from outside the package, while their inner workings remain hidden from outside users. 25
- Parent.** The parent of a derived type is the first type given in the definition of the derived type. The parent can be almost any kind of type, including an interface type. 25.1/2
- Partition.** A *partition* is a part of a program. Each partition consists of a set of library units. Each partition may run in a separate address space, possibly on a separate computer. A program may contain just one partition. A distributed program typically contains multiple partitions, which can execute concurrently. 26
- Pragma.** A pragma is a compiler directive. There are language-defined pragmas that give instructions for optimization, listing control, etc. An implementation may support additional (implementation-defined) pragmas. 27
- Primitive operations.** The primitive operations of a type are the operations (such as subprograms) declared together with the type declaration. They are inherited by other types in the same class of types. For a tagged type, the primitive subprograms are dispatching subprograms, providing run-time polymorphism. A dispatching subprogram may be called with statically tagged operands, in which case the subprogram body invoked is determined at compile time. Alternatively, a dispatching subprogram may be called using a dispatching call, in which case the subprogram body invoked is determined at run time. 28
- Private extension.** A private extension is a type that extends another type, with the additional properties hidden from its clients. 29/2
- Private type.** A private type gives a view of a type that reveals only some of its properties. The remaining properties are provided by the full view given elsewhere. Private types can be used for defining abstractions that hide unnecessary details from their clients. 30/2
- Procedure.** A procedure is a form of subprogram that does not return a result and can only be called by a statement. 30.1/2

- 30.2/2 **Progenitor.** A progenitor of a derived type is one of the types given in the definition of the derived type other than the first. A progenitor is always an interface type. Interfaces, tasks, and protected types may also have progenitors.
- 31 **Program.** A *program* is a set of *partitions*, each of which may execute in a separate address space, possibly on a separate computer. A partition consists of a set of library units.
- 32 **Program unit.** A *program unit* is either a package, a task unit, a protected unit, a protected entry, a generic unit, or an explicitly declared subprogram other than an enumeration literal. Certain kinds of program units can be separately compiled. Alternatively, they can appear physically nested within other program units.
- 33/2 **Protected type.** A protected type is a composite type whose components are accessible only through one of its protected operations which synchronize concurrent access by multiple tasks.
- 34 **Real type.** A real type has values that are approximations of the real numbers. Floating point and fixed point types are real types.
- 35 **Record extension.** A record extension is a type that extends another type by adding additional components.
- 36 **Record type.** A record type is a composite type consisting of zero or more named components, possibly of different types.
- 36.1/2 **Renaming.** A *renaming\_declaration* is a declaration that does not define a new entity, but instead defines a view of an existing entity.
- 37 **Scalar type.** A scalar type is either a discrete type or a real type.
- 37.1/2 **Subprogram.** A subprogram is a section of a program that can be executed in various contexts. It is invoked by a subprogram call that may qualify the effect of the subprogram through the passing of parameters. There are two forms of subprograms: functions, which return values, and procedures, which do not.
- 38/2 **Subtype.** A subtype is a type together with a constraint or null exclusion, which constrains the values of the subtype to satisfy a certain condition. The values of a subtype are a subset of the values of its type.
- 38.1/2 **Synchronized.** A synchronized entity is one that will work safely with multiple tasks at one time. A synchronized interface can be an ancestor of a task or a protected type. Such a task or protected type is called a synchronized tagged type.
- 39 **Tagged type.** The objects of a tagged type have a run-time type tag, which indicates the specific type with which the object was originally created. An operand of a class-wide tagged type can be used in a dispatching call; the tag indicates which subprogram body to invoke. Nondispatching calls, in which the subprogram body to invoke is determined at compile time, are also allowed. Tagged types may be extended with additional components.
- 40/2 **Task type.** A task type is a composite type used to represent active entities which execute concurrently and which can communicate via queued task entries. The top-level task of a partition is called the environment task.
- 41/2 **Type.** Each object has a type. A *type* has an associated set of values, and a set of *primitive operations* which implement the fundamental aspects of its semantics. Types are grouped into *categories*. Most language-defined categories of types are also *classes* of types.

**View.** A view of an entity reveals some or all of the properties of the entity. A single entity may have multiple views. 42/2





## Annex P

### (informative)

## Syntax Summary

This Annex summarizes the complete syntax of the language. See 1.1.4 for a description of the notation used.

```

2.3:
identifier ::=
 identifier_start { identifier_start | identifier_extend }

2.3:
identifier_start ::=
 letter_uppercase
 | letter_lowercase
 | letter_titlecase
 | letter_modifier
 | letter_other
 | number_letter

2.3:
identifier_extend ::=
 mark_non_spacing
 | mark_spacing_combining
 | number_decimal
 | punctuation_connector

2.4:
numeric_literal ::= decimal_literal | based_literal

2.4.1:
decimal_literal ::= numeral [.numeral] [exponent]

2.4.1:
numeral ::= digit {[underline] digit}

2.4.1:
exponent ::= E [+] numeral | E – numeral

2.4.1:
digit ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

2.4.2:
based_literal ::=
 base # based_numeral [.based_numeral] # [exponent]

2.4.2:
base ::= numeral

2.4.2:
based_numeral ::=
 extended_digit {[underline] extended_digit}

2.4.2:
extended_digit ::= digit | A | B | C | D | E | F

2.5:
character_literal ::= 'graphic_character'

2.6:
string_literal ::= "{string_element}"

2.6:
string_element ::= "" | non_quotation_mark_graphic_character

```

2.7:  
comment ::= --{*non\_end\_of\_line\_character*}

2.8:  
pragma ::=  
**pragma** identifier [(pragma\_argument\_association {, pragma\_argument\_association})];

2.8:  
pragma\_argument\_association ::=  
[*pragma\_argument\_identifier* =>] name  
| [*pragma\_argument\_identifier* =>] expression

3.1:  
basic\_declaration ::=  
type\_declaration | subtype\_declaration  
| object\_declaration | number\_declaration  
| subprogram\_declaration | abstract\_subprogram\_declaration  
| null\_procedure\_declaration | package\_declaration  
| renaming\_declaration | exception\_declaration  
| generic\_declaration | generic\_instantiation

3.1:  
defining\_identifier ::= identifier

3.2.1:  
type\_declaration ::= full\_type\_declaration  
| incomplete\_type\_declaration  
| private\_type\_declaration  
| private\_extension\_declaration

3.2.1:  
full\_type\_declaration ::=  
**type** defining\_identifier [known\_discriminant\_part] **is** type\_definition;  
| task\_type\_declaration  
| protected\_type\_declaration

3.2.1:  
type\_definition ::=  
enumeration\_type\_definition | integer\_type\_definition  
| real\_type\_definition | array\_type\_definition  
| record\_type\_definition | access\_type\_definition  
| derived\_type\_definition | interface\_type\_definition

3.2.2:  
subtype\_declaration ::=  
**subtype** defining\_identifier **is** subtype\_indication;

3.2.2:  
subtype\_indication ::= [null\_exclusion] subtype\_mark [constraint]

3.2.2:  
subtype\_mark ::= *subtype\_name*

3.2.2:  
constraint ::= scalar\_constraint | composite\_constraint

3.2.2:  
scalar\_constraint ::=  
range\_constraint | digits\_constraint | delta\_constraint

3.2.2:  
composite\_constraint ::=  
index\_constraint | discriminant\_constraint

3.3.1:  
object\_declaration ::=  
defining\_identifier\_list : [**aliased**] [**constant**] subtype\_indication [:= expression];

```

| defining_identifier_list : [aliased] [constant] access_definition [:= expression];
| defining_identifier_list : [aliased] [constant] array_type_definition [:= expression];
| single_task_declaration
| single_protected_declaration

3.3.1:
defining_identifier_list ::=
 defining_identifier {, defining_identifier}

3.3.2:
number_declaration ::=
 defining_identifier_list : constant := static_expression;

3.4:
derived_type_definition ::=
 [abstract] [limited] new parent_subtype_indication [[and interface_list] record_extension_part]

3.5:
range_constraint ::= range range

3.5:
range ::= range_attribute_reference
 | simple_expression .. simple_expression

3.5.1:
enumeration_type_definition ::=
 (enumeration_literal_specification {, enumeration_literal_specification})

3.5.1:
enumeration_literal_specification ::= defining_identifier | defining_character_literal

3.5.1:
defining_character_literal ::= character_literal

3.5.4:
integer_type_definition ::= signed_integer_type_definition | modular_type_definition

3.5.4:
signed_integer_type_definition ::= range static_simple_expression .. static_simple_expression

3.5.4:
modular_type_definition ::= mod static_expression

3.5.6:
real_type_definition ::=
 floating_point_definition | fixed_point_definition

3.5.7:
floating_point_definition ::=
 digits static_expression [real_range_specification]

3.5.7:
real_range_specification ::=
 range static_simple_expression .. static_simple_expression

3.5.9:
fixed_point_definition ::= ordinary_fixed_point_definition | decimal_fixed_point_definition

3.5.9:
ordinary_fixed_point_definition ::=
 delta static_expression real_range_specification

3.5.9:
decimal_fixed_point_definition ::=
 delta static_expression digits static_expression [real_range_specification]

3.5.9:
digits_constraint ::=
 digits static_expression [range_constraint]

```

```

3.6:
array_type_definition ::=
 unconstrained_array_definition | constrained_array_definition

3.6:
unconstrained_array_definition ::=
 array(index_subtype_definition {, index_subtype_definition}) of component_definition

3.6:
index_subtype_definition ::= subtype_mark range <>

3.6:
constrained_array_definition ::=
 array (discrete_subtype_definition {, discrete_subtype_definition}) of component_definition

3.6:
discrete_subtype_definition ::= discrete_subtype_indication | range

3.6:
component_definition ::=
 [aliased] subtype_indication
 | [aliased] access_definition

3.6.1:
index_constraint ::= (discrete_range {, discrete_range})

3.6.1:
discrete_range ::= discrete_subtype_indication | range

3.7:
discriminant_part ::= unknown_discriminant_part | known_discriminant_part

3.7:
unknown_discriminant_part ::= (<>)

3.7:
known_discriminant_part ::=
 (discriminant_specification {; discriminant_specification})

3.7:
discriminant_specification ::=
 defining_identifier_list : [null_exclusion] subtype_mark [:= default_expression]
 | defining_identifier_list : access_definition [:= default_expression]

3.7:
default_expression ::= expression

3.7.1:
discriminant_constraint ::=
 (discriminant_association {, discriminant_association})

3.7.1:
discriminant_association ::=
 [discriminant_selector_name { | discriminant_selector_name} =>] expression

3.8:
record_type_definition ::= [[abstract] tagged] [limited] record_definition

3.8:
record_definition ::=
 record
 component_list
 end record
 | null record

3.8:
component_list ::=
 component_item {component_item}

```

```

| {component_item} variant_part
| null;

3.8:
component_item ::= component_declaration | aspect_clause

3.8:
component_declaration ::=
 defining_identifier_list : component_definition [:= default_expression];

3.8.1:
variant_part ::=
 case discriminant_direct_name is
 variant
 {variant}
 end case;

3.8.1:
variant ::=
 when discrete_choice_list =>
 component_list

3.8.1:
discrete_choice_list ::= discrete_choice { | discrete_choice }

3.8.1:
discrete_choice ::= expression | discrete_range | others

3.9.1:
record_extension_part ::= with record_definition

3.9.3:
abstract_subprogram_declaration ::=
 [overriding_indicator]
 subprogram_specification is abstract;

3.9.4:
interface_type_definition ::=
 [limited | task | protected | synchronized] interface [and interface_list]

3.9.4:
interface_list ::= interface_subtype_mark { and interface_subtype_mark }

3.10:
access_type_definition ::=
 [null_exclusion] access_to_object_definition
 | [null_exclusion] access_to_subprogram_definition

3.10:
access_to_object_definition ::=
 access [general_access_modifier] subtype_indication

3.10:
general_access_modifier ::= all | constant

3.10:
access_to_subprogram_definition ::=
 access [protected] procedure parameter_profile
 | access [protected] function parameter_and_result_profile

3.10:
null_exclusion ::= not null

3.10:
access_definition ::=
 [null_exclusion] access [constant] subtype_mark
 | [null_exclusion] access [protected] procedure parameter_profile
 | [null_exclusion] access [protected] function parameter_and_result_profile

```

3.10.1:  
incomplete\_type\_declaration ::= **type** defining\_identifier [discriminant\_part] [**is tagged**];

3.11:  
declarative\_part ::= {declarative\_item}

3.11:  
declarative\_item ::=  
    basic\_declarative\_item | body

3.11:  
basic\_declarative\_item ::=  
    basic\_declaration | aspect\_clause | use\_clause

3.11:  
body ::= proper\_body | body\_stub

3.11:  
proper\_body ::=  
    subprogram\_body | package\_body | task\_body | protected\_body

4.1:  
name ::=  
    direct\_name                 | explicit\_dereference  
    | indexed\_component        | slice  
    | selected\_component       | attribute\_reference  
    | type\_conversion           | function\_call  
    | character\_literal         | qualified\_expression

4.1:  
direct\_name ::= identifier | operator\_symbol

4.1:  
prefix ::= name | implicit\_dereference

4.1:  
explicit\_dereference ::= name.all

4.1:  
implicit\_dereference ::= name

4.1.1:  
indexed\_component ::= prefix(expression {, expression})

4.1.2:  
slice ::= prefix(discrete\_range)

4.1.3:  
selected\_component ::= prefix . selector\_name

4.1.3:  
selector\_name ::= identifier | character\_literal | operator\_symbol

4.1.4:  
attribute\_reference ::= prefix'attribute\_designator

4.1.4:  
attribute\_designator ::=  
    identifier[(static\_expression)]  
    | Access | Delta | Digits | Mod

4.1.4:  
range\_attribute\_reference ::= prefix'range\_attribute\_designator

4.1.4:  
range\_attribute\_designator ::= Range[(static\_expression)]

4.3:  
aggregate ::= record\_aggregate | extension\_aggregate | array\_aggregate

4.3.1:  
`record_aggregate ::= (record_component_association_list)`

4.3.1:  
`record_component_association_list ::=`  
`record_component_association {, record_component_association}`  
`| null record`

4.3.1:  
`record_component_association ::=`  
`[component_choice_list =>] expression`  
`| component_choice_list => <>`

4.3.1:  
`component_choice_list ::=`  
`component_selector_name { | component_selector_name}`  
`| others`

4.3.2:  
`extension_aggregate ::=`  
`(ancestor_part with record_component_association_list)`

4.3.2:  
`ancestor_part ::= expression | subtype_mark`

4.3.3:  
`array_aggregate ::=`  
`positional_array_aggregate | named_array_aggregate`

4.3.3:  
`positional_array_aggregate ::=`  
`(expression, expression {, expression})`  
`| (expression {, expression}, others => expression)`  
`| (expression {, expression}, others => <>)`

4.3.3:  
`named_array_aggregate ::=`  
`(array_component_association {, array_component_association})`

4.3.3:  
`array_component_association ::=`  
`discrete_choice_list => expression`  
`| discrete_choice_list => <>`

4.4:  
`expression ::=`  
`relation {and relation}   | relation {and then relation}`  
`| relation {or relation}   | relation {or else relation}`  
`| relation {xor relation}`

4.4:  
`relation ::=`  
`simple_expression [relational_operator simple_expression]`  
`| simple_expression [not] in range`  
`| simple_expression [not] in subtype_mark`

4.4:  
`simple_expression ::= [unary_adding_operator] term {binary_adding_operator term}`

4.4:  
`term ::= factor {multiplying_operator factor}`

4.4:  
`factor ::= primary [** primary] | abs primary | not primary`

4.4:  
`primary ::=`

```

 numeric_literal | null | string_literal | aggregate
 | name | allocator | (expression)
 | (quantified_expression)

4.5:
logical_operator ::= and | or | xor

4.5:
relational_operator ::= = | /= | < | <= | > | >=

4.5:
binary_adding_operator ::= + | - | &

4.5:
unary_adding_operator ::= + | -

4.5:
multiplying_operator ::= * | / | mod | rem

4.5:
highest_precedence_operator ::= ** | abs | not

4.5.7:
quantified_expression ::= for quantifier loop_parameter_specification | predicate

4.5.7:
quantifier ::= all | Some

4.5.7:
predicate ::= boolean_expression

4.6:
type_conversion ::=
 subtype_mark(expression)
 | subtype_mark(name)

4.7:
qualified_expression ::=
 subtype_mark'(expression) | subtype_mark'aggregate

4.8:
allocator ::=
 new subtype_indication | new qualified_expression

5.1:
sequence_of_statements ::= statement {statement} {label}

5.1:
statement ::=
 {label} simple_statement | {label} compound_statement

5.1:
simple_statement ::= null_statement
 | assignment_statement | exit_statement
 | goto_statement | procedure_call_statement
 | simple_return_statement | entry_call_statement
 | requeue_statement | delay_statement
 | abort_statement | raise_statement
 | code_statement

5.1:
compound_statement ::=
 if_statement | case_statement
 | loop_statement | block_statement
 | extended_return_statement |
 | accept_statement | select_statement

```



```

null_statement ::= null;

5.1:
label ::= <<label_statement_identifier>>

5.1:
statement_identifier ::= direct_name

5.2:
assignment_statement ::=
 variable_name := expression;

5.3:
if_statement ::=
 if condition then
 sequence_of_statements
 {elsif condition then
 sequence_of_statements}
 [else
 sequence_of_statements]
 end if;

5.3:
condition ::= boolean_expression

5.4:
case_statement ::=
 case expression is
 case_statement_alternative
 {case_statement_alternative}
 end case;

5.4:
case_statement_alternative ::=
 when discrete_choice_list =>
 sequence_of_statements

5.5:
loop_statement ::=
 [loop_statement_identifier:]
 [iteration_scheme] loop
 sequence_of_statements
 end loop [loop_identifier];

5.5:
iteration_scheme ::= while condition
 | for loop_parameter_specification

5.5:
loop_parameter_specification ::=
 defining_identifier in [reverse] discrete_subtype_definition

5.6:
block_statement ::=
 [block_statement_identifier:]
 [declare
 declarative_part]
 begin
 handled_sequence_of_statements
 end [block_identifier];

5.7:
exit_statement ::=
 exit [loop_name] [when condition];

5.8:

```

```

goto_statement ::= goto label_name;

6.1:
subprogram_declaration ::=
 [overriding_indicator]
 subprogram_specification;

6.1:
subprogram_specification ::=
 procedure_specification
 | function_specification

6.1:
procedure_specification ::= procedure defining_program_unit_name parameter_profile

6.1:
function_specification ::= function defining_designator parameter_and_result_profile

6.1:
designator ::= [parent_unit_name .] identifier | operator_symbol

6.1:
defining_designator ::= defining_program_unit_name | defining_operator_symbol

6.1:
defining_program_unit_name ::= [parent_unit_name .] defining_identifier

6.1:
operator_symbol ::= string_literal

6.1:
defining_operator_symbol ::= operator_symbol

6.1:
parameter_profile ::= [formal_part]

6.1:
parameter_and_result_profile ::=
 [formal_part] return [null_exclusion] subtype_mark
 | [formal_part] return access_definition

6.1:
formal_part ::=
 (parameter_specification { ; parameter_specification})

6.1:
parameter_specification ::=
 defining_identifier_list : mode [null_exclusion] subtype_mark [:= default_expression]
 | defining_identifier_list : access_definition [:= default_expression]

6.1:
mode ::= [in] | in out | out

6.3:
subprogram_body ::=
 [overriding_indicator]
 subprogram_specification is
 declarative_part
 begin
 handled_sequence_of_statements
 end [designator];

6.4:
procedure_call_statement ::=
 procedure_name;
 | procedure_prefix actual_parameter_part;

6.4:

```

```

function_call ::=
 function_name
 | function_prefix actual_parameter_part

6.4:
actual_parameter_part ::=
 (parameter_association {, parameter_association})

6.4:
parameter_association ::=
 [formal_parameter_selector_name =>] explicit_actual_parameter

6.4:
explicit_actual_parameter ::= expression | variable_name

6.5:
simple_return_statement ::= return [expression];

6.5:
extended_return_statement ::=
 return defining_identifier : [constant] return_subtype_indication [:= expression] [do
 handled_sequence_of_statements
 end return];

6.5:
return_subtype_indication ::= subtype_indication | access_definition

6.7:
null_procedure_declaration ::=
 [overriding_indicator]
 procedure_specification is null;

7.1:
package_declaration ::= package_specification;

7.1:
package_specification ::=
 package defining_program_unit_name is
 {basic_declarative_item}
 [private
 {basic_declarative_item}]
 end [[parent_unit_name.]identifier]

7.2:
package_body ::=
 package body defining_program_unit_name is
 declarative_part
 [begin
 handled_sequence_of_statements]
 end [[parent_unit_name.]identifier];

7.3:
private_type_declaration ::=
 type defining_identifier [discriminant_part] is [[abstract] tagged] [limited] private;

7.3:
private_extension_declaration ::=
 type defining_identifier [discriminant_part] is
 [abstract] [limited | synchronized] new ancestor_subtype_indication
 [and interface_list] with private;

8.3.1:
overriding_indicator ::= [not] overriding

8.4:
use_clause ::= use_package_clause | use_type_clause

```

8.4:

```
use_package_clause ::= use package_name {, package_name};
```

8.4:

```
use_type_clause ::= use {all} type subtype_mark {, subtype_mark};
```

8.5:

```
renaming_declaration ::=
 object_renaming_declaration
 | exception_renaming_declaration
 | package_renaming_declaration
 | subprogram_renaming_declaration
 | generic_renaming_declaration
```

8.5.1:

```
object_renaming_declaration ::=
 defining_identifier : [null_exclusion] subtype_mark renames object_name;
 | defining_identifier : access_definition renames object_name;
```

8.5.2:

```
exception_renaming_declaration ::= defining_identifier : exception renames exception_name;
```

8.5.3:

```
package_renaming_declaration ::= package defining_program_unit_name renames package_name;
```

8.5.4:

```
subprogram_renaming_declaration ::=
 [overriding_indicator]
 subprogram_specification renames callable_entity_name;
```

8.5.5:

```
generic_renaming_declaration ::=
 generic package defining_program_unit_name renames generic_package_name;
 | generic procedure defining_program_unit_name renames generic_procedure_name;
 | generic function defining_program_unit_name renames generic_function_name;
```

9.1:

```
task_type_declaration ::=
 task type defining_identifier [known_discriminant_part] [is
 [new interface_list with]
 task_definition];
```

9.1:

```
single_task_declaration ::=
 task defining_identifier [is
 [new interface_list with]
 task_definition];
```

9.1:

```
task_definition ::=
 {task_item}
 [private
 {task_item}]
 end [task_identifier]
```

9.1:

```
task_item ::= entry_declaration | aspect_clause
```

9.1:

```
task_body ::=
 task body defining_identifier is
 declarative_part
 begin
 handled_sequence_of_statements
 end [task_identifier];
```

9.4:  
**protected\_type\_declaration** ::=  
**protected type** defining\_identifier [known\_discriminant\_part] **is**  
  [new interface\_list **with**]  
  protected\_definition;

9.4:  
**single\_protected\_declaration** ::=  
**protected** defining\_identifier **is**  
  [new interface\_list **with**]  
  protected\_definition;

9.4:  
**protected\_definition** ::=  
  { protected\_operation\_declaration }  
**[ private**  
  { protected\_element\_declaration } ]  
**end** [protected\_identifier]

9.4:  
**protected\_operation\_declaration** ::= subprogram\_declaration  
  | entry\_declaration  
  | aspect\_clause

9.4:  
**protected\_element\_declaration** ::= protected\_operation\_declaration  
  | component\_declaration

9.4:  
**protected\_body** ::=  
**protected body** defining\_identifier **is**  
  { protected\_operation\_item }  
**end** [protected\_identifier];

9.4:  
**protected\_operation\_item** ::= subprogram\_declaration  
  | subprogram\_body  
  | entry\_body  
  | aspect\_clause

9.5:  
**implementation\_kind** ::= By\_Entry | By\_Protected\_Procedure | By\_Any

9.5.2:  
**entry\_declaration** ::=  
  [overriding\_indicator]  
  **entry** defining\_identifier [(discrete\_subtype\_definition)] parameter\_profile;

9.5.2:  
**accept\_statement** ::=  
  **accept** entry\_direct\_name [(entry\_index)] parameter\_profile [**do**  
  handled\_sequence\_of\_statements  
  **end** [entry\_identifier]];

9.5.2:  
**entry\_index** ::= expression

9.5.2:  
**entry\_body** ::=  
  **entry** defining\_identifier entry\_body\_formal\_part entry\_barrier **is**  
  declarative\_part  
  **begin**  
  handled\_sequence\_of\_statements  
  **end** [entry\_identifier];

9.5.2:

```

entry_body_formal_part ::= [(entry_index_specification)] parameter_profile

9.5.2:
entry_barrier ::= when condition

9.5.2:
entry_index_specification ::= for defining_identifier in discrete_subtype_definition

9.5.3:
entry_call_statement ::= entry_name [actual_parameter_part];

9.5.4:
requeue_statement ::= requeue procedure_or_entry_name [with abort];

9.6:
delay_statement ::= delay_until_statement | delay_relative_statement

9.6:
delay_until_statement ::= delay until delay_expression;

9.6:
delay_relative_statement ::= delay delay_expression;

9.7:
select_statement ::=
 selective_accept
 | timed_entry_call
 | conditional_entry_call
 | asynchronous_select

9.7.1:
selective_accept ::=
 select
 [guard]
 select_alternative
 { or
 [guard]
 select_alternative }
 [else
 sequence_of_statements]
 end select;

9.7.1:
guard ::= when condition =>

9.7.1:
select_alternative ::=
 accept_alternative
 | delay_alternative
 | terminate_alternative

9.7.1:
accept_alternative ::=
 accept_statement [sequence_of_statements]

9.7.1:
delay_alternative ::=
 delay_statement [sequence_of_statements]

9.7.1:
terminate_alternative ::= terminate;

9.7.2:
timed_entry_call ::=
 select
 entry_call_alternative
 or

```

```

 delay_alternative
 end select;

```

9.7.2:  
 entry\_call\_alternative ::=  
   procedure\_or\_entry\_call [sequence\_of\_statements]

9.7.2:  
 procedure\_or\_entry\_call ::=  
   procedure\_call\_statement | entry\_call\_statement

9.7.3:  
 conditional\_entry\_call ::=  
   **select**  
     entry\_call\_alternative  
   **else**  
     sequence\_of\_statements  
   **end select**;

9.7.4:  
 asynchronous\_select ::=  
   **select**  
     triggering\_alternative  
   **then abort**  
     abortable\_part  
   **end select**;

9.7.4:  
 triggering\_alternative ::= triggering\_statement [sequence\_of\_statements]

9.7.4:  
 triggering\_statement ::= procedure\_or\_entry\_call | delay\_statement

9.7.4:  
 abortable\_part ::= sequence\_of\_statements

9.8:  
 abort\_statement ::= **abort** *task\_name* {, *task\_name*};

10.1.1:  
 compilation ::= {compilation\_unit}

10.1.1:  
 compilation\_unit ::=  
   context\_clause library\_item  
   | context\_clause subunit

10.1.1:  
 library\_item ::= [**private**] library\_unit\_declaration  
   | library\_unit\_body  
   | [**private**] library\_unit\_renaming\_declaration

10.1.1:  
 library\_unit\_declaration ::=  
   subprogram\_declaration   | package\_declaration  
   | generic\_declaration     | generic\_instantiation

10.1.1:  
 library\_unit\_renaming\_declaration ::=  
   package\_renaming\_declaration  
   | generic\_renaming\_declaration  
   | subprogram\_renaming\_declaration

10.1.1:  
 library\_unit\_body ::= subprogram\_body | package\_body

10.1.1:

```

parent_unit_name ::= name

10.1.2:
context_clause ::= {context_item}

10.1.2:
context_item ::= with_clause | use_clause

10.1.2:
with_clause ::= limited_with_clause | nonlimited_with_clause

10.1.2:
limited_with_clause ::= limited [private] with library_unit_name {, library_unit_name};

10.1.2:
nonlimited_with_clause ::= [private] with library_unit_name {, library_unit_name};

10.1.3:
body_stub ::= subprogram_body_stub | package_body_stub | task_body_stub | protected_body_stub

10.1.3:
subprogram_body_stub ::=
 [overriding_indicator]
 subprogram_specification is separate;

10.1.3:
package_body_stub ::= package body defining_identifier is separate;

10.1.3:
task_body_stub ::= task body defining_identifier is separate;

10.1.3:
protected_body_stub ::= protected body defining_identifier is separate;

10.1.3:
subunit ::= separate (parent_unit_name) proper_body

11.1:
exception_declaration ::= defining_identifier_list : exception;

11.2:
handled_sequence_of_statements ::=
 sequence_of_statements
 [exception
 exception_handler
 {exception_handler}]

11.2:
exception_handler ::=
 when [choice_parameter_specification:] exception_choice {| exception_choice} =>
 sequence_of_statements

11.2:
choice_parameter_specification ::= defining_identifier

11.2:
exception_choice ::= exception_name | others

11.3:
raise_statement ::= raise;
 | raise exception_name [with string_expression];

12.1:
generic_declaration ::= generic_subprogram_declaration | generic_package_declaration

12.1:
generic_subprogram_declaration ::=
 generic_formal_part subprogram_specification;

12.1:

```



```

generic_package_declaration ::=
 generic_formal_part package_specification;

12.1:
generic_formal_part ::= generic {generic_formal_parameter_declaration | use_clause}

12.1:
generic_formal_parameter_declaration ::=
 formal_object_declaration
 | formal_type_declaration
 | formal_subprogram_declaration
 | formal_package_declaration

12.3:
generic_instantiation ::=
 package defining_program_unit_name is
 new generic_package_name [generic_actual_part];
 | [overriding_indicator]
 procedure defining_program_unit_name is
 new generic_procedure_name [generic_actual_part];
 | [overriding_indicator]
 function defining_designator is
 new generic_function_name [generic_actual_part];

12.3:
generic_actual_part ::=
 (generic_association {, generic_association})

12.3:
generic_association ::=
 [generic_formal_parameter_selector_name =>] explicit_generic_actual_parameter

12.3:
explicit_generic_actual_parameter ::= expression | variable_name
 | subprogram_name | entry_name | subtype_mark
 | package_instance_name

12.4:
formal_object_declaration ::=
 defining_identifier_list : mode [null_exclusion] subtype_mark [:= default_expression];
 | defining_identifier_list : mode access_definition [:= default_expression];

12.5:
formal_type_declaration ::=
 type defining_identifier[discriminant_part] is formal_type_definition;

12.5:
formal_type_definition ::=
 formal_private_type_definition
 | formal_derived_type_definition
 | formal_discrete_type_definition
 | formal_signed_integer_type_definition
 | formal_modular_type_definition
 | formal_floating_point_definition
 | formal_ordinary_fixed_point_definition
 | formal_decimal_fixed_point_definition
 | formal_array_type_definition
 | formal_access_type_definition
 | formal_interface_type_definition

12.5.1:
formal_private_type_definition ::= [[abstract] tagged] [limited] private

12.5.1:
formal_derived_type_definition ::=

```

[**abstract**] [**limited** | **synchronized**] **new** subtype\_mark [(**and** interface\_list)**with private**]

12.5.2:  
**formal\_discrete\_type\_definition** ::= (<>)

12.5.2:  
**formal\_signed\_integer\_type\_definition** ::= **range** <>

12.5.2:  
**formal\_modular\_type\_definition** ::= **mod** <>

12.5.2:  
**formal\_floating\_point\_definition** ::= **digits** <>

12.5.2:  
**formal\_ordinary\_fixed\_point\_definition** ::= **delta** <>

12.5.2:  
**formal\_decimal\_fixed\_point\_definition** ::= **delta** <> **digits** <>

12.5.3:  
**formal\_array\_type\_definition** ::= **array\_type\_definition**

12.5.4:  
**formal\_access\_type\_definition** ::= **access\_type\_definition**

12.5.5:  
**formal\_interface\_type\_definition** ::= **interface\_type\_definition**

12.6:  
**formal\_subprogram\_declaration** ::= **formal\_concrete\_subprogram\_declaration**  
 | **formal\_abstract\_subprogram\_declaration**

12.6:  
**formal\_concrete\_subprogram\_declaration** ::=  
**with** subprogram\_specification [**is** subprogram\_default];

12.6:  
**formal\_abstract\_subprogram\_declaration** ::=  
**with** subprogram\_specification **is abstract** [subprogram\_default];

12.6:  
**subprogram\_default** ::= default\_name | <> | **null**

12.6:  
**default\_name** ::= name

12.7:  
**formal\_package\_declaration** ::=  
**with package** defining\_identifier **is new** *generic\_package\_name* **formal\_package\_actual\_part**;

12.7:  
**formal\_package\_actual\_part** ::=  
 ([**others** =>] <>)  
 | [**generic\_actual\_part**]  
 | (**formal\_package\_association** {, **formal\_package\_association**} [, **others** => <>])

12.7:  
**formal\_package\_association** ::=  
**generic\_association**  
 | *generic\_formal\_parameter\_selector\_name* => <>

13.1:  
**aspect\_clause** ::= **attribute\_definition\_clause**  
 | **enumeration\_representation\_clause**  
 | **record\_representation\_clause**  
 | **at\_clause**

13.1:

```

local_name ::= direct_name
| direct_name'attribute_designator
| library_unit_name

13.3:
attribute_definition_clause ::=
 for local_name'attribute_designator use expression;
| for local_name'attribute_designator use name;

13.4:
enumeration_representation_clause ::=
 for first_subtype_local_name use enumeration_aggregate;

13.4:
enumeration_aggregate ::= array_aggregate

13.5.1:
record_representation_clause ::=
 for first_subtype_local_name use
 record [mod_clause]
 {component_clause}
 end record;

13.5.1:
component_clause ::=
 component_local_name at position range first_bit .. last_bit;

13.5.1:
position ::= static_expression

13.5.1:
first_bit ::= static_simple_expression

13.5.1:
last_bit ::= static_simple_expression

13.8:
code_statement ::= qualified_expression;

13.12:
restriction ::= restriction_identifier
| restriction_parameter_identifier => restriction_parameter_argument

13.12:
restriction_parameter_argument ::= name | expression

J.3:
delta_constraint ::= delta static_expression [range_constraint]

J.7:
at_clause ::= for direct_name use at expression;

J.8:
mod_clause ::= at mod static_expression;

```

# Syntax Cross Reference

- 1 In the following syntax cross reference, each syntactic category is followed by the clause number where it is defined. In addition, each syntactic category *S* is followed by a list of the categories that use *S* in their definitions. For example, the first listing below shows that `abort_statement` appears in the definition of `simple_statement`.

|                                              |        |                                              |        |
|----------------------------------------------|--------|----------------------------------------------|--------|
| <code>abort_statement</code>                 | 9.8    | <code>aggregate</code>                       | 4.3    |
| <code>simple_statement</code>                | 5.1    | <code>enumeration_aggregate</code>           | 13.4   |
| <code>abortable_part</code>                  | 9.7.4  | <code>array_component_association</code>     | 4.3.3  |
| <code>asynchronous_select</code>             | 9.7.4  | <code>named_array_aggregate</code>           | 4.3.3  |
| <code>abstract_subprogram_declaration</code> | 3.9.3  | <code>array_type_definition</code>           | 3.6    |
| <code>basic_declaration</code>               | 3.1    | <code>formal_array_type_definition</code>    | 12.5.3 |
| <code>accept_alternative</code>              | 9.7.1  | <code>object_declaration</code>              | 3.3.1  |
| <code>select_alternative</code>              | 9.7.1  | <code>type_definition</code>                 | 3.2.1  |
| <code>accept_statement</code>                | 9.5.2  | <code>aspect_clause</code>                   | 13.1   |
| <code>accept_alternative</code>              | 9.7.1  | <code>basic_declarative_item</code>          | 3.11   |
| <code>compound_statement</code>              | 5.1    | <code>component_item</code>                  | 3.8    |
| <code>access_definition</code>               | 3.10   | <code>protected_operation_declaration</code> | 9.4    |
| <code>component_definition</code>            | 3.6    | <code>protected_operation_item</code>        | 9.4    |
| <code>discriminant_specification</code>      | 3.7    | <code>task_item</code>                       | 9.1    |
| <code>formal_object_declaration</code>       | 12.4   | <code>assignment_statement</code>            | 5.2    |
| <code>object_declaration</code>              | 3.3.1  | <code>simple_statement</code>                | 5.1    |
| <code>object_renaming_declaration</code>     | 8.5.1  | <code>asynchronous_select</code>             | 9.7.4  |
| <code>parameter_and_result_profile</code>    | 6.1    | <code>select_statement</code>                | 9.7    |
| <code>parameter_specification</code>         | 6.1    | <code>at_clause</code>                       | 1.7    |
| <code>return_subtype_indication</code>       | 6.5    | <code>aspect_clause</code>                   | 13.1   |
| <code>access_to_object_definition</code>     | 3.10   | <code>attribute_definition_clause</code>     | 13.3   |
| <code>access_type_definition</code>          | 3.10   | <code>aspect_clause</code>                   | 13.1   |
| <code>access_to_subprogram_definition</code> | 3.10   | <code>attribute_designator</code>            | 4.1.4  |
| <code>access_type_definition</code>          | 3.10   | <code>attribute_definition_clause</code>     | 13.3   |
| <code>access_type_definition</code>          | 3.10   | <code>attribute_reference</code>             | 4.1.4  |
| <code>formal_access_type_definition</code>   | 12.5.4 | <code>local_name</code>                      | 13.1   |
| <code>type_definition</code>                 | 3.2.1  | <code>attribute_reference</code>             | 4.1.4  |
| <code>actual_parameter_part</code>           | 6.4    | <code>name</code>                            | 4.1    |
| <code>entry_call_statement</code>            | 9.5.3  | <code>base</code>                            | 2.4.2  |
| <code>function_call</code>                   | 6.4    | <code>based_literal</code>                   | 2.4.2  |
| <code>procedure_call_statement</code>        | 6.4    | <code>based_literal</code>                   | 2.4.2  |
| <code>aggregate</code>                       | 4.3    | <code>based_literai</code>                   | 2.4.2  |
| <code>primary</code>                         | 4.4    | <code>numeric_literai</code>                 | 2.4    |
| <code>qualified_expression</code>            | 4.7    | <code>based_numeral</code>                   | 2.4.2  |
| <code>allocator</code>                       | 4.8    | <code>based_literai</code>                   | 2.4.2  |
| <code>primary</code>                         | 4.4    | <code>basic_declaration</code>               | 3.1    |
| <code>ancestor_part</code>                   | 4.3.2  | <code>basic_declarative_item</code>          | 3.11   |
| <code>extension_aggregate</code>             | 4.3.2  | <code>basic_declarative_item</code>          | 3.11   |
| <code>array_aggregate</code>                 | 4.3.3  | <code>declarative_item</code>                | 3.11   |

|                                |        |                                   |        |
|--------------------------------|--------|-----------------------------------|--------|
| package_specification          | 7.1    | condition                         | 5.3    |
| binary_adding_operator         | 4.5    | entry_barrier                     | 9.5.2  |
| simple_expression              | 4.4    | exit_statement                    | 5.7    |
| block_statement                | 5.6    | guard                             | 9.7.1  |
| compound_statement             | 5.1    | if_statement                      | 5.3    |
|                                |        | iteration_scheme                  | 5.5    |
| body                           | 3.11   | conditional_entry_call            | 9.7.3  |
| declarative_item               | 3.11   | select_statement                  | 9.7    |
| body_stub                      | 10.1.3 | constrained_array_definition      | 3.6    |
| body                           | 3.11   | array_type_definition             | 3.6    |
| case_statement                 | 5.4    | constraint                        | 3.2.2  |
| compound_statement             | 5.1    | subtype_indication                | 3.2.2  |
| case_statement_alternative     | 5.4    | context_clause                    | 10.1.2 |
| case_statement                 | 5.4    | compilation_unit                  | 10.1.1 |
| character                      | 2.1    | context_item                      | 10.1.2 |
| comment                        | 2.7    | context_clause                    | 10.1.2 |
| character_literal              | 2.5    | decimal_fixed_point_definition    | 3.5.9  |
| defining_character_literal     | 3.5.1  | fixed_point_definition            | 3.5.9  |
| name                           | 4.1    | decimal_literal                   | 2.4.1  |
| selector_name                  | 4.1.3  | numeric_literal                   | 2.4    |
| choice_parameter_specification | 11.2   | declarative_item                  | 3.11   |
| exception_handler              | 11.2   | declarative_part                  | 3.11   |
| code_statement                 | 13.8   | declarative_part                  | 3.11   |
| simple_statement               | 5.1    | block_statement                   | 5.6    |
| compilation_unit               | 10.1.1 | entry_body                        | 9.5.2  |
| compilation                    | 10.1.1 | package_body                      | 7.2    |
| component_choice_list          | 4.3.1  | subprogram_body                   | 6.3    |
| record_component_association   | 4.3.1  | task_body                         | 9.1    |
| component_clause               | 13.5.1 | default_expression                | 3.7    |
| record_representation_clause   | 13.5.1 | component_declaration             | 3.8    |
| component_declaration          | 3.8    | discriminant_specification        | 3.7    |
| component_item                 | 3.8    | formal_object_declaration         | 12.4   |
| protected_element_declaration  | 9.4    | parameter_specification           | 6.1    |
| component_definition           | 3.6    | default_name                      | 12.6   |
| component_declaration          | 3.8    | subprogram_default                | 12.6   |
| constrained_array_definition   | 3.6    | defining_character_literal        | 3.5.1  |
| unconstrained_array_definition | 3.6    | enumeration_literal_specification | 3.5.1  |
| component_item                 | 3.8    | defining_designator               | 6.1    |
| component_list                 | 3.8    | function_specification            | 6.1    |
| record_definition              | 3.8    | generic_instantiation             | 12.3   |
| variant                        | 3.8.1  | defining_identifier               | 3.1    |
| composite_constraint           | 3.2.2  | choice_parameter_specification    | 11.2   |
| constraint                     | 3.2.2  | defining_identifier_list          | 3.3.1  |
| compound_statement             | 5.1    | defining_program_unit_name        | 6.1    |
| statement                      | 5.1    | entry_body                        | 9.5.2  |
|                                |        | entry_declaration                 | 9.5.2  |
|                                |        | entry_index_specification         | 9.5.2  |
|                                |        | enumeration_literal_specification | 3.5.1  |
|                                |        | exception_renaming_declaration    | 8.5.2  |

|                               |        |                               |        |
|-------------------------------|--------|-------------------------------|--------|
| extended_return_statement     | 6.5    | designator                    | 6.1    |
| formal_package_declaration    | 12.7   | subprogram_body               | 6.3    |
| formal_type_declaration       | 12.5   |                               |        |
| full_type_declaration         | 3.2.1  | digit 2.4.1                   |        |
| incomplete_type_declaration   | 3.10.1 | extended_digit                | 2.4.2  |
| loop_parameter_specification  | 5.5    | numeral                       | 2.4.1  |
| object_renaming_declaration   | 8.5.1  |                               |        |
| package_body_stub             | 10.1.3 | digits_constraint             | 3.5.9  |
| private_extension_declaration | 7.3    | scalar_constraint             | 3.2.2  |
| private_type_declaration      | 7.3    |                               |        |
| protected_body                | 9.4    | direct_name                   | 4.1    |
| protected_body_stub           | 10.1.3 | accept_statement              | 9.5.2  |
| protected_type_declaration    | 9.4    | at_clause                     | J.7    |
| single_protected_declaration  | 9.4    | local_name                    | 13.1   |
| single_task_declaration       | 9.1    | name                          | 4.1    |
| subtype_declaration           | 3.2.2  | statement_identifier          | 5.1    |
| task_body                     | 9.1    | variant_part                  | 3.8.1  |
| task_body_stub                | 10.1.3 |                               |        |
| task_type_declaration         | 9.1    | discrete_choice               | 3.8.1  |
|                               |        | discrete_choice_list          | 3.8.1  |
| defining_identifier_list      | 3.3.1  |                               |        |
| component_declaration         | 3.8    | discrete_choice_list          | 3.8.1  |
| discriminant_specification    | 3.7    | array_component_association   | 4.3.3  |
| exception_declaration         | 11.1   | case_statement_alternative    | 5.4    |
| formal_object_declaration     | 12.4   | variant                       | 3.8.1  |
| number_declaration            | 3.3.2  |                               |        |
| object_declaration            | 3.3.1  | discrete_range                | 3.6.1  |
| parameter_specification       | 6.1    | discrete_choice               | 3.8.1  |
|                               |        | index_constraint              | 3.6.1  |
| defining_operator_symbol      | 6.1    | slice                         | 4.1.2  |
| defining_designator           | 6.1    |                               |        |
| defining_program_unit_name    | 6.1    | discrete_subtype_definition   | 3.6    |
| defining_designator           | 6.1    | constrained_array_definition  | 3.6    |
| generic_instantiation         | 12.3   | entry_declaration             | 9.5.2  |
| generic_renaming_declaration  | 8.5.5  | entry_index_specification     | 9.5.2  |
| package_body                  | 7.2    | loop_parameter_specification  | 5.5    |
| package_renaming_declaration  | 8.5.3  |                               |        |
| package_specification         | 7.1    | discriminant_association      | 3.7.1  |
| procedure_specification       | 6.1    | discriminant_constraint       | 3.7.1  |
|                               |        |                               |        |
| delay_alternative             | 9.7.1  | discriminant_constraint       | 3.7.1  |
| select_alternative            | 9.7.1  | composite_constraint          | 3.2.2  |
| timed_entry_call              | 9.7.2  |                               |        |
| delay_relative_statement      | 9.6    | discriminant_part             | 3.7    |
| delay_statement               | 9.6    | formal_type_declaration       | 12.5   |
|                               |        | incomplete_type_declaration   | 3.10.1 |
| delay_statement               | 9.6    | private_extension_declaration | 7.3    |
| delay_alternative             | 9.7.1  | private_type_declaration      | 7.3    |
| simple_statement              | 5.1    |                               |        |
| triggering_statement          | 9.7.4  | discriminant_specification    | 3.7    |
|                               |        | known_discriminant_part       | 3.7    |
| delay_until_statement         | 9.6    |                               |        |
| delay_statement               | 9.6    | entry_barrier                 | 9.5.2  |
|                               |        | entry_body                    | 9.5.2  |
| delta_constraint              | J.3    |                               |        |
| scalar_constraint             | 3.2.2  | entry_body                    | 9.5.2  |
|                               |        | protected_operation_item      | 9.4    |
| derived_type_definition       | 3.4    |                               |        |
| type_definition               | 3.2.1  | entry_body_formal_part        | 9.5.2  |
|                               |        | entry_body                    | 9.5.2  |
|                               |        |                               |        |
|                               |        | entry_call_alternative        | 9.7.2  |
|                               |        | conditional_entry_call        | 9.7.3  |

|                                   |       |                                        |        |
|-----------------------------------|-------|----------------------------------------|--------|
| timed_entry_call                  | 9.7.2 | condition                              | 5.3    |
| entry_call_statement              | 9.5.3 | decimal_fixed_point_definition         | 3.5.9  |
| procedure_or_entry_call           | 9.7.2 | default_expression                     | 3.7    |
| simple_statement                  | 5.1   | delay_relative_statement               | 9.6    |
| entry_declaration                 | 9.5.2 | delay_until_statement                  | 9.6    |
| protected_operation_declaration   | 9.4   | delta_constraint                       | J.3    |
| task_item                         | 9.1   | digits_constraint                      | 3.5.9  |
| entry_index                       | 9.5.2 | discrete_choice                        | 3.8.1  |
| accept_statement                  | 9.5.2 | discriminant_association               | 3.7.1  |
| entry_index_specification         | 9.5.2 | entry_index                            | 9.5.2  |
| entry_body_formal_part            | 9.5.2 | explicit_actual_parameter              | 6.4    |
| enumeration_aggregate             | 13.4  | explicit_generic_actual_parameter      | 12.3   |
| enumeration_representation_clause | 13.4  | extended_return_statement              | 6.5    |
| enumeration_literal_specification | 3.5.1 | floating_point_definition              | 3.5.7  |
| enumeration_type_definition       | 3.5.1 | indexed_component                      | 4.1.1  |
| enumeration_representation_clause | 13.4  | mod_clause                             | J.8    |
| aspect_clause                     | 13.1  | modular_type_definition                | 3.5.4  |
| enumeration_type_definition       | 3.5.1 | number_declaration                     | 3.3.2  |
| type_definition                   | 3.2.1 | object_declaration                     | 3.3.1  |
| exception_choice                  | 11.2  | ordinary_fixed_point_definition        | 3.5.9  |
| exception_handler                 | 11.2  | position                               | 13.5.1 |
| exception_declaration             | 11.1  | positional_array_aggregate             | 4.3.3  |
| basic_declaration                 | 3.1   | pragma_argument_association            | 2.8    |
| exception_handler                 | 11.2  | predicate                              | 4.5.7  |
| handled_sequence_of_statements    | 11.2  | primary                                | 4.4    |
| exception_renaming_declaration    | 8.5.2 | qualified_expression                   | 4.7    |
| renaming_declaration              | 8.5   | raise_statement                        | 11.3   |
| exit_statement                    | 5.7   | range_attribute_designator             | 4.1.4  |
| simple_statement                  | 5.1   | record_component_association           | 4.3.1  |
| explicit_actual_parameter         | 6.4   | restriction_parameter_argument         | 13.12  |
| parameter_association             | 6.4   | simple_return_statement                | 6.5    |
| explicit_dereference              | 4.1   | type_conversion                        | 4.6    |
| name                              | 4.1   | extended_digit                         | 2.4.2  |
| explicit_generic_actual_parameter | 12.3  | based_numeral                          | 2.4.2  |
| generic_association               | 12.3  | extended_return_statement              | 6.5    |
| exponent                          | 2.4.1 | compound_statement                     | 5.1    |
| based_literal                     | 2.4.2 | extension_aggregate                    | 4.3.2  |
| decimal_literal                   | 2.4.1 | aggregate                              | 4.3    |
| expression                        | 4.4   | factor                                 | 4.4    |
| ancestor_part                     | 4.3.2 | term                                   | 4.4    |
| array_component_association       | 4.3.3 | first_bit                              | 13.5.1 |
| assignment_statement              | 5.2   | component_clause                       | 13.5.1 |
| at_clause                         | J.7   | fixed_point_definition                 | 3.5.9  |
| attribute_definition_clause       | 13.3  | real_type_definition                   | 3.5.6  |
| attribute_designator              | 4.1.4 | floating_point_definition              | 3.5.7  |
| case_statement                    | 5.4   | real_type_definition                   | 3.5.6  |
|                                   |       | formal_abstract_subprogram_declaration | 12.6   |
|                                   |       | formal_subprogram_declaration          | 12.6   |
|                                   |       | formal_access_type_definition          | 12.5.4 |
|                                   |       | formal_type_definition                 | 12.5   |
|                                   |       | formal_array_type_definition           | 12.5.3 |
|                                   |       | formal_type_definition                 | 12.5   |

|                                        |        |                                      |        |
|----------------------------------------|--------|--------------------------------------|--------|
| formal_concrete_subprogram_declaration | 12.6   | access_to_object_definition          | 3.10   |
| formal_subprogram_declaration          | 12.6   | generic_actual_part                  | 12.3   |
| formal_decimal_fixed_point_definition  | 12.5.2 | formal_package_actual_part           | 12.7   |
| formal_type_definition                 | 12.5   | generic_instantiation                | 12.3   |
| formal_derived_type_definition         | 12.5.1 | generic_association                  | 12.3   |
| formal_type_definition                 | 12.5   | formal_package_association           | 12.7   |
| formal_discrete_type_definition        | 12.5.2 | generic_actual_part                  | 12.3   |
| formal_type_definition                 | 12.5   | generic_declaration                  | 12.1   |
| formal_floating_point_definition       | 12.5.2 | basic_declaration                    | 3.1    |
| formal_type_definition                 | 12.5   | library_unit_declaration             | 10.1.1 |
| formal_interface_type_definition       | 12.5.5 | generic_formal_parameter_declaration | 12.1   |
| formal_type_definition                 | 12.5   | generic_formal_part                  | 12.1   |
| formal_modular_type_definition         | 12.5.2 | generic_formal_part                  | 12.1   |
| formal_type_definition                 | 12.5   | generic_package_declaration          | 12.1   |
| formal_object_declaration              | 12.4   | generic_subprogram_declaration       | 12.1   |
| generic_formal_parameter_declaration   | 12.1   | generic_instantiation                | 12.3   |
| formal_ordinary_fixed_point_definition | 12.5.2 | basic_declaration                    | 3.1    |
| formal_type_definition                 | 12.5   | library_unit_declaration             | 10.1.1 |
| formal_package_actual_part             | 12.7   | generic_package_declaration          | 12.1   |
| formal_package_declaration             | 12.7   | generic_declaration                  | 12.1   |
| formal_package_association             | 12.7   | generic_renaming_declaration         | 8.5.5  |
| formal_package_actual_part             | 12.7   | library_unit_renaming_declaration    | 10.1.1 |
| formal_package_declaration             | 12.7   | renaming_declaration                 | 8.5    |
| generic_formal_parameter_declaration   | 12.1   | generic_subprogram_declaration       | 12.1   |
| formal_part                            | 6.1    | generic_declaration                  | 12.1   |
| parameter_and_result_profile           | 6.1    | goto_statement                       | 5.8    |
| parameter_profile                      | 6.1    | simple_statement                     | 5.1    |
| formal_private_type_definition         | 12.5.1 | graphic_character                    | 2.1    |
| formal_type_definition                 | 12.5   | character_literal                    | 2.5    |
| formal_signed_integer_type_definition  | 12.5.2 | string_element                       | 2.6    |
| formal_type_definition                 | 12.5   | guard                                | 9.7.1  |
| formal_subprogram_declaration          | 12.6   | selective_accept                     | 9.7.1  |
| generic_formal_parameter_declaration   | 12.1   | handled_sequence_of_statements       | 11.2   |
| formal_type_declaration                | 12.5   | accept_statement                     | 9.5.2  |
| generic_formal_parameter_declaration   | 12.1   | block_statement                      | 5.6    |
| formal_type_definition                 | 12.5   | entry_body                           | 9.5.2  |
| formal_type_declaration                | 12.5   | extended_return_statement            | 6.5    |
| full_type_declaration                  | 3.2.1  | package_body                         | 7.2    |
| type_declaration                       | 3.2.1  | subprogram_body                      | 6.3    |
| function_call                          | 6.4    | task_body                            | 9.1    |
| name                                   | 4.1    | identifier                           | 2.3    |
| function_specification                 | 6.1    | accept_statement                     | 9.5.2  |
| subprogram_specification               | 6.1    | attribute_designator                 | 4.1.4  |
| general_access_modifier                | 3.10   | block_statement                      | 5.6    |
|                                        |        | defining_identifier                  | 3.1    |
|                                        |        | designator                           | 6.1    |
|                                        |        | direct_name                          | 4.1    |
|                                        |        | entry_body                           | 9.5.2  |
|                                        |        | loop_statement                       | 5.5    |



|                                  |        |                                   |        |
|----------------------------------|--------|-----------------------------------|--------|
| package_body                     | 7.2    | statement                         | 5.1    |
| package_specification            | 7.1    |                                   |        |
| pragma                           | 2.8    | last_bit                          | 13.5.1 |
| pragma_argument_association      | 2.8    | component_clause                  | 13.5.1 |
| protected_body                   | 9.4    |                                   |        |
| protected_definition             | 9.4    | letter_lowercase                  | ...    |
| restriction                      | 13.12  | identifier_start                  | 2.3    |
| selector_name                    | 4.1.3  |                                   |        |
| task_body                        | 9.1    | letter_modifier                   | ...    |
| task_definition                  | 9.1    | identifier_start                  | 2.3    |
|                                  |        |                                   |        |
| identifier_extend                | 2.3    | letter_other                      | ...    |
| identifier                       | 2.3    | identifier_start                  | 2.3    |
|                                  |        |                                   |        |
| identifier_start                 | 2.3    | letter_titlecase                  | ...    |
| identifier                       | 2.3    | identifier_start                  | 2.3    |
|                                  |        |                                   |        |
| if_statement                     | 5.3    | letter_uppercase                  | ...    |
| compound_statement               | 5.1    | identifier_start                  | 2.3    |
|                                  |        |                                   |        |
| implicit_dereference             | 4.1    | library_item                      | 10.1.1 |
| prefix                           | 4.1    | compilation_unit                  | 10.1.1 |
|                                  |        |                                   |        |
| incomplete_type_declaration      | 3.10.1 | library_unit_body                 | 10.1.1 |
| type_declaration                 | 3.2.1  | library_item                      | 10.1.1 |
|                                  |        |                                   |        |
| index_constraint                 | 3.6.1  | library_unit_declaration          | 10.1.1 |
| composite_constraint             | 3.2.2  | library_item                      | 10.1.1 |
|                                  |        |                                   |        |
| index_subtype_definition         | 3.6    | library_unit_renaming_declaration | 10.1.1 |
| unconstrained_array_definition   | 3.6    | library_item                      | 10.1.1 |
|                                  |        |                                   |        |
| indexed_component                | 4.1.1  | limited_with_clause               | 10.1.2 |
| name                             | 4.1    | with_clause                       | 10.1.2 |
|                                  |        |                                   |        |
| integer_type_definition          | 3.5.4  | local_name                        | 13.1   |
| type_definition                  | 3.2.1  | attribute_definition_clause       | 13.3   |
|                                  |        | component_clause                  | 13.5.1 |
|                                  |        | enumeration_representation_clause | 13.4   |
|                                  |        | record_representation_clause      | 13.5.1 |
| interface_list                   | 3.9.4  |                                   |        |
| derived_type_definition          | 3.4    | loop_parameter_specification      | 5.5    |
| formal_derived_type_definition   | 12.5.1 | iteration_scheme                  | 5.5    |
| interface_type_definition        | 3.9.4  | quantified_expression             | 4.5.7  |
| private_extension_declaration    | 7.3    |                                   |        |
| protected_type_declaration       | 9.4    | loop_statement                    | 5.5    |
| single_protected_declaration     | 9.4    | compound_statement                | 5.1    |
| single_task_declaration          | 9.1    |                                   |        |
| task_type_declaration            | 9.1    | mark_non_spacing                  | ...    |
|                                  |        | identifier_extend                 | 2.3    |
|                                  |        |                                   |        |
| interface_type_definition        | 3.9.4  | mark_spacing_combining            | ...    |
| formal_interface_type_definition | 12.5.5 | identifier_extend                 | 2.3    |
| type_definition                  | 3.2.1  |                                   |        |
|                                  |        | mod_clause                        | 1.8    |
| iteration_scheme                 | 5.5    | record_representation_clause      | 13.5.1 |
| loop_statement                   | 5.5    |                                   |        |
|                                  |        | mode                              | 6.1    |
| known_discriminant_part          | 3.7    | formal_object_declaration         | 12.4   |
| discriminant_part                | 3.7    | parameter_specification           | 6.1    |
| full_type_declaration            | 3.2.1  |                                   |        |
| protected_type_declaration       | 9.4    | modular_type_definition           | 3.5.4  |
| task_type_declaration            | 9.1    | integer_type_definition           | 3.5.4  |
|                                  |        |                                   |        |
| label5.1                         |        |                                   |        |
| sequence_of_statements           | 5.1    |                                   |        |

|                                   |        |                                   |        |
|-----------------------------------|--------|-----------------------------------|--------|
| multiplying_operator              | 4.5    | number_decimal                    | ...    |
| term                              | 4.4    | identifier_extend                 | 2.3    |
| name                              | 4.1    | number_declaration                | 3.3.2  |
| abort_statement                   | 9.8    | basic_declaration                 | 3.1    |
| assignment_statement              | 5.2    | number_letter                     | ...    |
| attribute_definition_clause       | 13.3   | identifier_start                  | 2.3    |
| default_name                      | 12.6   | numeral                           | 2.4.1  |
| entry_call_statement              | 9.5.3  | base                              | 2.4.2  |
| exception_choice                  | 11.2   | decimal_literal                   | 2.4.1  |
| exception_renaming_declaration    | 8.5.2  | exponent                          | 2.4.1  |
| exit_statement                    | 5.7    | numeric_literal                   | 2.4    |
| explicit_actual_parameter         | 6.4    | primary                           | 4.4    |
| explicit_dereference              | 4.1    | object_declaration                | 3.3.1  |
| explicit_generic_actual_parameter | 12.3   | basic_declaration                 | 3.1    |
| formal_package_declaration        | 12.7   | object_renaming_declaration       | 8.5.1  |
| function_call                     | 6.4    | renaming_declaration              | 8.5    |
| generic_instantiation             | 12.3   | operator_symbol                   | 6.1    |
| generic_renaming_declaration      | 8.5.5  | defining_operator_symbol          | 6.1    |
| goto_statement                    | 5.8    | designator                        | 6.1    |
| implicit_dereference              | 4.1    | direct_name                       | 4.1    |
| limited_with_clause               | 10.1.2 | selector_name                     | 4.1.3  |
| local_name                        | 13.1   | ordinary_fixed_point_definition   | 3.5.9  |
| nonlimited_with_clause            | 10.1.2 | fixed_point_definition            | 3.5.9  |
| object_renaming_declaration       | 8.5.1  | overriding_indicator              | 8.3.1  |
| package_renaming_declaration      | 8.5.3  | abstract_subprogram_declaration   | 3.9.3  |
| parent_unit_name                  | 10.1.1 | entry_declaration                 | 9.5.2  |
| pragma_argument_association       | 2.8    | generic_instantiation             | 12.3   |
| prefix                            | 4.1    | null_procedure_declaration        | 6.7    |
| primary                           | 4.4    | subprogram_body                   | 6.3    |
| procedure_call_statement          | 6.4    | subprogram_body_stub              | 10.1.3 |
| raise_statement                   | 11.3   | subprogram_declaration            | 6.1    |
| requeue_statement                 | 9.5.4  | subprogram_renaming_declaration   | 8.5.4  |
| restriction_parameter_argument    | 13.12  | package_body                      | 7.2    |
| subprogram_renaming_declaration   | 8.5.4  | library_unit_body                 | 10.1.1 |
| subtype_mark                      | 3.2.2  | proper_body                       | 3.11   |
| type_conversion                   | 4.6    | package_body_stub                 | 10.1.3 |
| use_package_clause                | 8.4    | body_stub                         | 10.1.3 |
| named_array_aggregate             | 4.3.3  | package_declaration               | 7.1    |
| array_aggregate                   | 4.3.3  | basic_declaration                 | 3.1    |
| nonlimited_with_clause            | 10.1.2 | library_unit_declaration          | 10.1.1 |
| with_clause                       | 10.1.2 | package_renaming_declaration      | 8.5.3  |
| null_exclusion                    | 3.10   | library_unit_renaming_declaration | 10.1.1 |
| access_definition                 | 3.10   | renaming_declaration              | 8.5    |
| access_type_definition            | 3.10   | package_specification             | 7.1    |
| discriminant_specification        | 3.7    | generic_package_declaration       | 12.1   |
| formal_object_declaration         | 12.4   | package_declaration               | 7.1    |
| object_renaming_declaration       | 8.5.1  | parameter_and_result_profile      | 6.1    |
| parameter_and_result_profile      | 6.1    | access_definition                 | 3.10   |
| parameter_specification           | 6.1    |                                   |        |
| subtype_indication                | 3.2.2  |                                   |        |
| null_procedure_declaration        | 6.7    |                                   |        |
| basic_declaration                 | 3.1    |                                   |        |
| null_statement                    | 5.1    |                                   |        |
| simple_statement                  | 5.1    |                                   |        |

|                                 |        |                                 |        |
|---------------------------------|--------|---------------------------------|--------|
| access_to_subprogram_definition | 3.10   | null_procedure_declaration      | 6.7    |
| function_specification          | 6.1    | subprogram_specification        | 6.1    |
| parameter_association           | 6.4    | proper_body                     | 3.11   |
| actual_parameter_part           | 6.4    | body                            | 3.11   |
| parameter_profile               | 6.1    | subunit                         | 10.1.3 |
| accept_statement                | 9.5.2  | protected_body                  | 9.4    |
| access_definition               | 3.10   | proper_body                     | 3.11   |
| access_to_subprogram_definition | 3.10   | protected_body_stub             | 10.1.3 |
| entry_body_formal_part          | 9.5.2  | body_stub                       | 10.1.3 |
| entry_declaration               | 9.5.2  | protected_definition            | 9.4    |
| procedure_specification         | 6.1    | protected_type_declaration      | 9.4    |
| parameter_specification         | 6.1    | single_protected_declaration    | 9.4    |
| formal_part                     | 6.1    | protected_element_declaration   | 9.4    |
| parent_unit_name                | 10.1.1 | protected_definition            | 9.4    |
| defining_program_unit_name      | 6.1    | protected_operation_declaration | 9.4    |
| designator                      | 6.1    | protected_definition            | 9.4    |
| package_body                    | 7.2    | protected_element_declaration   | 9.4    |
| package_specification           | 7.1    | protected_operation_item        | 9.4    |
| subunit                         | 10.1.3 | protected_body                  | 9.4    |
| position                        | 13.5.1 | protected_type_declaration      | 9.4    |
| component_clause                | 13.5.1 | full_type_declaration           | 3.2.1  |
| positional_array_aggregate      | 4.3.3  | punctuation_connector           | ...    |
| array_aggregate                 | 4.3.3  | identifier_extend               | 2.3    |
| pragma_argument_association     | 2.8    | qualified_expression            | 4.7    |
| pragma                          | 2.8    | allocator                       | 4.8    |
| predicate                       | 4.5.7  | code_statement                  | 13.8   |
| quantified_expression           | 4.5.7  | name                            | 4.1    |
| prefix                          | 4.1    | quantified_expression           | 4.5.7  |
| attribute_reference             | 4.1.4  | primary                         | 4.4    |
| function_call                   | 6.4    | quantifier                      | 4.5.7  |
| indexed_component               | 4.1.1  | quantified_expression           | 4.5.7  |
| procedure_call_statement        | 6.4    | raise_statement                 | 11.3   |
| range_attribute_reference       | 4.1.4  | simple_statement                | 5.1    |
| selected_component              | 4.1.3  | range                           | 3.5    |
| slice                           | 4.1.2  | discrete_range                  | 3.6.1  |
| primary                         | 4.4    | discrete_subtype_definition     | 3.6    |
| factor                          | 4.4    | range_constraint                | 3.5    |
| private_extension_declaration   | 7.3    | relation                        | 4.4    |
| type_declaration                | 3.2.1  | range_attribute_designator      | 4.1.4  |
| private_type_declaration        | 7.3    | range_attribute_reference       | 4.1.4  |
| type_declaration                | 3.2.1  | range_attribute_reference       | 4.1.4  |
| procedure_call_statement        | 6.4    | range                           | 3.5    |
| procedure_or_entry_call         | 9.7.2  | range_constraint                | 3.5    |
| simple_statement                | 5.1    | delta_constraint                | J.3    |
| procedure_or_entry_call         | 9.7.2  | digits_constraint               | 3.5.9  |
| entry_call_alternative          | 9.7.2  | scalar_constraint               | 3.2.2  |
| triggering_statement            | 9.7.4  |                                 |        |
| procedure_specification         | 6.1    |                                 |        |

|                                   |        |                                |        |
|-----------------------------------|--------|--------------------------------|--------|
| real_range_specification          | 3.5.7  | selector_name                  | 4.1.3  |
| decimal_fixed_point_definition    | 3.5.9  | component_choice_list          | 4.3.1  |
| floating_point_definition         | 3.5.7  | discriminant_association       | 3.7.1  |
| ordinary_fixed_point_definition   | 3.5.9  | formal_package_association     | 12.7   |
| real_type_definition              | 3.5.6  | generic_association            | 12.3   |
| type_definition                   | 3.2.1  | parameter_association          | 6.4    |
|                                   |        | selected_component             | 4.1.3  |
| record_aggregate                  | 4.3.1  | sequence_of_statements         | 5.1    |
| aggregate                         | 4.3    | abortable_part                 | 9.7.4  |
| record_component_association      | 4.3.1  | accept_alternative             | 9.7.1  |
| record_component_association_list | 4.3.1  | case_statement_alternative     | 5.4    |
| record_component_association_list | 4.3.1  | conditional_entry_call         | 9.7.3  |
| extension_aggregate               | 4.3.2  | delay_alternative              | 9.7.1  |
| record_aggregate                  | 4.3.1  | entry_call_alternative         | 9.7.2  |
| record_definition                 | 3.8    | exception_handler              | 11.2   |
| record_extension_part             | 3.9.1  | handled_sequence_of_statements | 11.2   |
| record_type_definition            | 3.8    | if_statement                   | 5.3    |
| record_extension_part             | 3.9.1  | loop_statement                 | 5.5    |
| derived_type_definition           | 3.4    | selective_accept               | 9.7.1  |
| record_representation_clause      | 13.5.1 | triggering_alternative         | 9.7.4  |
| aspect_clause                     | 13.1   | signed_integer_type_definition | 3.5.4  |
| record_type_definition            | 3.8    | integer_type_definition        | 3.5.4  |
| type_definition                   | 3.2.1  | simple_expression              | 4.4    |
| relation                          | 4.4    | first_bit                      | 13.5.1 |
| expression                        | 4.4    | last_bit                       | 13.5.1 |
| relational_operator               | 4.5    | range                          | 3.5    |
| relation                          | 4.4    | real_range_specification       | 3.5.7  |
| renaming_declaration              | 8.5    | relation                       | 4.4    |
| basic_declaration                 | 3.1    | signed_integer_type_definition | 3.5.4  |
| requeue_statement                 | 9.5.4  | simple_return_statement        | 6.5    |
| simple_statement                  | 5.1    | simple_statement               | 5.1    |
| restriction_parameter_argument    | 13.12  | simple_statement               | 5.1    |
| restriction                       | 13.12  | single_protected_declaration   | 9.4    |
| return_subtype_indication         | 6.5    | object_declaration             | 3.3.1  |
| extended_return_statement         | 6.5    | single_task_declaration        | 9.1    |
| scalar_constraint                 | 3.2.2  | object_declaration             | 3.3.1  |
| constraint                        | 3.2.2  | slice                          | 4.1.2  |
| select_alternative                | 9.7.1  | name                           | 4.1    |
| selective_accept                  | 9.7.1  | statement                      | 5.1    |
| select_statement                  | 9.7    | sequence_of_statements         | 5.1    |
| compound_statement                | 5.1    | statement_identifier           | 5.1    |
| selected_component                | 4.1.3  | block_statement                | 5.6    |
| name                              | 4.1    | label                          | 5.1    |
| selective_accept                  | 9.7.1  | loop_statement                 | 5.5    |
| select_statement                  | 9.7    | string_element                 | 2.6    |
|                                   |        | string_literal                 | 2.6    |
|                                   |        | string_literal                 | 2.6    |
|                                   |        | operator_symbol                | 6.1    |
|                                   |        | primary                        | 4.4    |

|                                        |        |                                |        |
|----------------------------------------|--------|--------------------------------|--------|
| subprogram_body                        | 6.3    | subtype_indication             | 3.2.2  |
| library_unit_body                      | 10.1.1 | type_conversion                | 4.6    |
| proper_body                            | 3.11   | use_type_clause                | 8.4    |
| protected_operation_item               | 9.4    |                                |        |
| subprogram_body_stub                   | 10.1.3 | subunit                        | 10.1.3 |
| body_stub                              | 10.1.3 | compilation_unit               | 10.1.1 |
| subprogram_declaration                 | 6.1    | task_body                      | 9.1    |
| basic_declaration                      | 3.1    | proper_body                    | 3.11   |
| library_unit_declaration               | 10.1.1 | task_body_stub                 | 10.1.3 |
| protected_operation_declaration        | 9.4    | body_stub                      | 10.1.3 |
| protected_operation_item               | 9.4    | task_definition                | 9.1    |
| subprogram_default                     | 12.6   | single_task_declaration        | 9.1    |
| formal_abstract_subprogram_declaration | 12.6   | task_type_declaration          | 9.1    |
| formal_concrete_subprogram_declaration | 12.6   | task_item                      | 9.1    |
| subprogram_renaming_declaration        | 8.5.4  | task_definition                | 9.1    |
| library_unit_renaming_declaration      | 10.1.1 | task_type_declaration          | 9.1    |
| renaming_declaration                   | 8.5    | full_type_declaration          | 3.2.1  |
| subprogram_specification               | 6.1    | term 4.4                       |        |
| abstract_subprogram_declaration        | 3.9.3  | simple_expression              | 4.4    |
| formal_abstract_subprogram_declaration | 12.6   | terminate_alternative          | 9.7.1  |
| formal_concrete_subprogram_declaration | 12.6   | select_alternative             | 9.7.1  |
| generic_subprogram_declaration         | 12.1   | timed_entry_call               | 9.7.2  |
| subprogram_body                        | 6.3    | select_statement               | 9.7    |
| subprogram_body_stub                   | 10.1.3 | triggering_alternative         | 9.7.4  |
| subprogram_declaration                 | 6.1    | asynchronous_select            | 9.7.4  |
| subprogram_renaming_declaration        | 8.5.4  | triggering_statement           | 9.7.4  |
| subtype_declaration                    | 3.2.2  | triggering_alternative         | 9.7.4  |
| basic_declaration                      | 3.1    | type_conversion                | 4.6    |
| subtype_indication                     | 3.2.2  | name                           | 4.1    |
| access_to_object_definition            | 3.10   | type_declaration               | 3.2.1  |
| allocator                              | 4.8    | basic_declaration              | 3.1    |
| component_definition                   | 3.6    | type_definition                | 3.2.1  |
| derived_type_definition                | 3.4    | full_type_declaration          | 3.2.1  |
| discrete_range                         | 3.6.1  | unary_adding_operator          | 4.5    |
| discrete_subtype_definition            | 3.6    | simple_expression              | 4.4    |
| object_declaration                     | 3.3.1  | unconstrained_array_definition | 3.6    |
| private_extension_declaration          | 7.3    | array_type_definition          | 3.6    |
| return_subtype_indication              | 6.5    | underline                      | ...    |
| subtype_declaration                    | 3.2.2  | based_numeral                  | 2.4.2  |
| subtype_mark                           | 3.2.2  | numeral                        | 2.4.1  |
| access_definition                      | 3.10   | unknown_discriminant_part      | 3.7    |
| ancestor_part                          | 4.3.2  | discriminant_part              | 3.7    |
| discriminant_specification             | 3.7    | use_clause                     | 8.4    |
| explicit_generic_actual_parameter      | 12.3   | basic_declarative_item         | 3.11   |
| formal_derived_type_definition         | 12.5.1 | context_item                   | 10.1.2 |
| formal_object_declaration              | 12.4   | generic_formal_part            | 12.1   |
| index_subtype_definition               | 3.6    |                                |        |
| interface_list                         | 3.9.4  |                                |        |
| object_renaming_declaration            | 8.5.1  |                                |        |
| parameter_and_result_profile           | 6.1    |                                |        |
| parameter_specification                | 6.1    |                                |        |
| qualified_expression                   | 4.7    |                                |        |
| relation                               | 4.4    |                                |        |

|                    |       |                |        |
|--------------------|-------|----------------|--------|
| use_package_clause | 8.4   | variant_part   | 3.8.1  |
| use_clause         | 8.4   | variant_part   | 3.8.1  |
| use_type_clause    | 8.4   | component_list | 3.8    |
| use_clause         | 8.4   | with_clause    | 10.1.2 |
| variant            | 3.8.1 | context_item   | 10.1.2 |

# Annex Q

## (informative)

### Language-Defined Entities

This annex lists the language-defined entities of the language. A list of language-defined library units can be found in Annex A, “Predefined Language Environment”. 1/2

## Q.1 Language-Defined Packages

This clause lists all language-defined packages. 1/2

|                                                  |                                                          |
|--------------------------------------------------|----------------------------------------------------------|
| Ada A.2(2)                                       | Containers                                               |
| Address_To_Access_Conversions                    | <i>child of</i> Ada A.18.1(3/2)                          |
| <i>child of</i> System 13.7.2(2)                 | Conversions                                              |
| Arithmetic                                       | <i>child of</i> Ada.Characters A.3.4(2/2)                |
| <i>child of</i> Ada.Calendar 9.6.1(8/2)          | <i>child of</i> Ada.Strings.UTF_Encoding A.4.11(15/3)    |
| ASCII                                            | Decimal                                                  |
| <i>in</i> Standard A.1(36.3/2)                   | <i>child of</i> Ada F.2(2)                               |
| Assertions                                       | Decimal_Conversions                                      |
| <i>child of</i> Ada 11.4.2(12/2)                 | <i>in</i> Interfaces.COBOL B.4(31)                       |
| Asynchronous_Task_Control                        | Decimal_IO                                               |
| <i>child of</i> Ada D.11(3/2)                    | <i>in</i> Ada.Text_IO A.10.1(73)                         |
| Bounded                                          | Decimal_Output                                           |
| <i>child of</i> Ada.Strings A.4.4(3)             | <i>in</i> Ada.Text_IO Editing F.3.3(11)                  |
| Bounded_IO                                       | Direct_IO                                                |
| <i>child of</i> Ada.Text_IO A.10.11(3/2)         | <i>child of</i> Ada A.8.4(2)                             |
| <i>child of</i> Ada.Wide_Text_IO A.11(4/3)       | Directories                                              |
| <i>child of</i> Ada.Wide_Wide_Text_IO A.11(4/3)  | <i>child of</i> Ada A.16(3/2)                            |
| C                                                | Discrete_Random                                          |
| <i>child of</i> Interfaces B.3(4)                | <i>child of</i> Ada.Numerics A.5.2(17)                   |
| Calendar                                         | Dispatching                                              |
| <i>child of</i> Ada 9.6(10)                      | <i>child of</i> Ada D.2.1(1.2/3)                         |
| Characters                                       | Doubly_Linked_Lists                                      |
| <i>child of</i> Ada A.3.1(2)                     | <i>child of</i> Ada.Containers A.18.3(5/3)               |
| COBOL                                            | Dynamic_Priorities                                       |
| <i>child of</i> Interfaces B.4(7)                | <i>child of</i> Ada D.5.1(3/2)                           |
| Command_Line                                     | EDF                                                      |
| <i>child of</i> Ada A.15(3)                      | <i>child of</i> Ada.Dispatching D.2.6(9/2)               |
| Complex_Arrays                                   | <i>child of</i> Ada.Synchronous_Task_Control D.10(5.2/3) |
| <i>child of</i> Ada.Numerics G.3.2(53/2)         | Editing                                                  |
| Complex_Elementary_Functions                     | <i>child of</i> Ada.Text_IO F.3.3(3)                     |
| <i>child of</i> Ada.Numerics G.1.2(9/1)          | <i>child of</i> Ada.Wide_Text_IO F.3.4(1)                |
| Complex_Text_IO                                  | <i>child of</i> Ada.Wide_Wide_Text_IO F.3.5(1/2)         |
| <i>child of</i> Ada G.1.3(9.1/2)                 | Elementary_Functions                                     |
| Complex_Types                                    | <i>child of</i> Ada.Numerics A.5.1(9/1)                  |
| <i>child of</i> Ada.Numerics G.1.1(25/1)         | Enumeration_IO                                           |
| Complex_IO                                       | <i>in</i> Ada.Text_IO A.10.1(79)                         |
| <i>child of</i> Ada.Text_IO G.1.3(3)             | Environment_Variables                                    |
| <i>child of</i> Ada.Wide_Text_IO G.1.4(1)        | <i>child of</i> Ada A.17(3/2)                            |
| <i>child of</i> Ada.Wide_Wide_Text_IO G.1.5(1/2) | Exceptions                                               |
| Constants                                        | <i>child of</i> Ada 11.4.1(2/2)                          |
| <i>child of</i> Ada.Strings.Maps A.4.6(3/2)      | Execution_Time                                           |
|                                                  | <i>child of</i> Ada D.14(3/2)                            |

|                                              |              |                                 |              |
|----------------------------------------------|--------------|---------------------------------|--------------|
| Finalization                                 |              | Indefinite_Ordered_Maps         |              |
| <i>child of</i> Ada                          | 7.6(4/1)     | <i>child of</i> Ada.Containers  | A.18.14(2/3) |
| Fixed                                        |              | Indefinite_Ordered_Sets         |              |
| <i>child of</i> Ada.Strings                  | A.4.3(5)     | <i>child of</i> Ada.Containers  | A.18.16(2/3) |
| Fixed_IO                                     |              | Indefinite_Vectors              |              |
| <i>in</i> Ada.Text_IO                        | A.10.1(68)   | <i>child of</i> Ada.Containers  | A.18.11(2/3) |
| Float_Random                                 |              | Information                     |              |
| <i>child of</i> Ada.Numerics                 | A.5.2(5)     | <i>child of</i> Ada.Directories | A.16(124/2)  |
| Float_Text_IO                                |              | Integer_Text_IO                 |              |
| <i>child of</i> Ada                          | A.10.9(33)   | <i>child of</i> Ada             | A.10.8(21)   |
| Float_Wide_Text_IO                           |              | Integer_Wide_Text_IO            |              |
| <i>child of</i> Ada                          | A.11(2/2)    | <i>child of</i> Ada             | A.11(2/2)    |
| Float_Wide_Wide_Text_IO                      |              | Integer_Wide_Wide_Text_IO       |              |
| <i>child of</i> Ada                          | A.11(3/2)    | <i>child of</i> Ada             | A.11(3/2)    |
| Float_IO                                     |              | Integer_IO                      |              |
| <i>in</i> Ada.Text_IO                        | A.10.1(63)   | <i>in</i> Ada.Text_IO           | A.10.1(52)   |
| Formatting                                   |              | Interfaces                      | B.2(3)       |
| <i>child of</i> Ada.Calendar                 | 9.6.1(15/2)  | Interrupts                      |              |
| Fortran                                      |              | <i>child of</i> Ada             | C.3.2(2)     |
| <i>child of</i> Interfaces                   | B.5(4)       | IO_Exceptions                   |              |
| Generic_Complex_Arrays                       |              | <i>child of</i> Ada             | A.13(3)      |
| <i>child of</i> Ada.Numerics                 | G.3.2(2/2)   | Latin_1                         |              |
| Generic_Complex_Elementary_Functions         |              | <i>child of</i> Ada.Characters  | A.3.3(3)     |
| <i>child of</i> Ada.Numerics                 | G.1.2(2/2)   | Machine_Code                    |              |
| Generic_Complex_Types                        |              | <i>child of</i> System          | 13.8(7)      |
| <i>child of</i> Ada.Numerics                 | G.1.1(2/1)   | Maps                            |              |
| Generic_Dispatching_Constructor              |              | <i>child of</i> Ada.Strings     | A.4.2(3/2)   |
| <i>child of</i> Ada.Tags                     | 3.9(18.2/2)  | Modular_IO                      |              |
| Generic_Elementary_Functions                 |              | <i>in</i> Ada.Text_IO           | A.10.1(57)   |
| <i>child of</i> Ada.Numerics                 | A.5.1(3)     | Multiway_Trees                  |              |
| Generic_Bounded_Length                       |              | <i>child of</i> Ada.Containers  | A.18.10(7/3) |
| <i>in</i> Ada.Strings.Bounded                | A.4.4(4)     | Names                           |              |
| Generic_Keys                                 |              | <i>child of</i> Ada.Interrupts  | C.3.2(12)    |
| <i>in</i> Ada.Containers.Hashtable_Sets      | A.18.8(50/2) | Non_Preemptive                  |              |
| <i>in</i> Ada.Containers.Ordered_Sets        | A.18.9(62/2) | <i>child of</i> Ada.Dispatching | D.2.4(2.2/3) |
| Generic_Real_Arrays                          |              | Numerics                        |              |
| <i>child of</i> Ada.Numerics                 | G.3.1(2/2)   | <i>child of</i> Ada             | A.5(3/2)     |
| Generic_Sorting                              |              | Ordered_Maps                    |              |
| <i>in</i> Ada.Containers.Doubly_Linked_Lists | A.18.3(47/2) | <i>child of</i> Ada.Containers  | A.18.6(2/3)  |
| <i>in</i> Ada.Containers.Vectors             | A.18.2(75/2) | Ordered_Sets                    |              |
| Group_Budgets                                |              | <i>child of</i> Ada.Containers  | A.18.9(2/3)  |
| <i>child of</i> Ada.Execution_Time           | D.14.2(3/2)  | Pointers                        |              |
| Handling                                     |              | <i>child of</i> Interfaces.C    | B.3.2(4)     |
| <i>child of</i> Ada.Characters               | A.3.2(2/2)   | Real_Arrays                     |              |
| Hashed_Maps                                  |              | <i>child of</i> Ada.Numerics    | G.3.1(31/2)  |
| <i>child of</i> Ada.Containers               | A.18.5(2/3)  | Real_Time                       |              |
| Hashed_Sets                                  |              | <i>child of</i> Ada             | D.8(3)       |
| <i>child of</i> Ada.Containers               | A.18.8(2/3)  | Round_Robin                     |              |
| Hierarchical_File_Names                      |              | <i>child of</i> Ada.Dispatching | D.2.5(4/2)   |
| <i>child of</i> Ada.Directories              | A.16.1(3/3)  | RPC                             |              |
| Indefinite_Doubly_Linked_Lists               |              | <i>child of</i> System          | E.5(3)       |
| <i>child of</i> Ada.Containers               | A.18.12(2/3) | Sequential_IO                   |              |
| Indefinite_Hashed_Maps                       |              | <i>child of</i> Ada             | A.8.1(2)     |
| <i>child of</i> Ada.Containers               | A.18.13(2/3) | Single_Precision_Complex_Types  |              |
| Indefinite_Hashed_Sets                       |              | <i>in</i> Interfaces.Fortran    | B.5(8)       |
| <i>child of</i> Ada.Containers               | A.18.15(2/3) | Standard                        | A.1(4)       |
| Indefinite_Holders                           |              | Storage_Elements                |              |
| <i>child of</i> Ada.Containers               | A.18.18(5/3) | <i>child of</i> System          | 13.7.1(2/2)  |
| Indefinite_Multiway_Trees                    |              | Storage_IO                      |              |
| <i>child of</i> Ada.Containers               | A.18.17(2/3) | <i>child of</i> Ada             | A.9(3)       |



Storage\_Pools  
   *child of* System 13.11(5)  
 Stream\_IO  
   *child of* Ada.Streams A.12.1(3)  
 Streams  
   *child of* Ada 13.13.1(2)  
 Strings  
   *child of* Ada A.4.1(3)  
   *child of* Interfaces.C B.3.1(3)  
 Synchronous\_Task\_Control  
   *child of* Ada D.10(3/2)  
 System 13.7(3/2)  
 Tags  
   *child of* Ada 3.9(6/2)  
 Task\_Attributes  
   *child of* Ada C.7.2(2)  
 Task\_Identification  
   *child of* Ada C.7.1(2/2)  
 Task\_Termination  
   *child of* Ada C.7.3(2/2)  
 Text\_Streams  
   *child of* Ada.Text\_IO A.12.2(3)  
   *child of* Ada.Wide\_Text\_IO A.12.3(3)  
   *child of* Ada.Wide\_Wide\_Text\_IO A.12.4(3/2)  
 Text\_IO  
   *child of* Ada A.10.1(2)  
 Time\_Zones  
   *child of* Ada.Calendar 9.6.1(2/2)  
 Timers  
   *child of* Ada.Execution\_Time D.14.1(3/2)  
 Timing\_Events  
   *child of* Ada.Real\_Time D.15(3/2)  
 Unbounded  
   *child of* Ada.Strings A.4.5(3)  
 Unbounded\_IO  
   *child of* Ada.Text\_IO A.10.12(3/2)  
   *child of* Ada.Wide\_Text\_IO A.11(5/3)  
   *child of* Ada.Wide\_Wide\_Text\_IO A.11(5/3)  
 UTF\_Encoding  
   *child of* Ada.Strings A.4.11(3/3)  
 Vectors  
   *child of* Ada.Containers A.18.2(6/3)  
 Wide\_Bounded  
   *child of* Ada.Strings A.4.7(1/2)  
 Wide\_Constants  
   *child of* Ada.Strings.Wide\_Maps A.4.7(1/2), A.4.8(28/2)  
 Wide\_Fixed  
   *child of* Ada.Strings A.4.7(1/2)  
 Wide\_Hash  
   *child of* Ada.Strings A.4.7(1/2)  
 Wide\_Maps  
   *child of* Ada.Strings A.4.7(3)  
 Wide\_Text\_IO  
   *child of* Ada A.11(2/2)  
 Wide\_Unbounded  
   *child of* Ada.Strings A.4.7(1/2)  
 Wide\_Characters  
   *child of* Ada A.3.1(4/2)  
 Wide\_Wide\_Constants  
   *child of* Ada.Strings.Wide\_Wide\_Maps A.4.8(1/2)  
 Wide\_Wide\_Hash  
   *child of* Ada.Strings A.4.8(1/2)  
 Wide\_Wide\_Text\_IO  
   *child of* Ada A.11(3/2)  
 Wide\_Wide\_Bounded  
   *child of* Ada.Strings A.4.8(1/2)  
 Wide\_Wide\_Characters  
   *child of* Ada A.3.1(6/2)  
 Wide\_Wide\_Encoding  
   *child of* Ada.Strings.UTF\_Encoding A.4.11(22/3),  
   A.4.11(30/3)  
 Wide\_Wide\_Fixed  
   *child of* Ada.Strings A.4.8(1/2)  
 Wide\_Wide\_Maps  
   *child of* Ada.Strings A.4.8(3/2)  
 Wide\_Wide\_Unbounded  
   *child of* Ada.Strings A.4.8(1/2)

## Q.2 Language-Defined Types and Subtypes

This clause lists all language-defined types and subtypes.

1/2

Address  
   *in* System 13.7(12)  
 Alignment  
   *in* Ada.Strings A.4.1(6)  
 Alphanumeric  
   *in* Interfaces.COBOL B.4(16)  
 Any\_Priority *subtype of* Integer  
   *in* System 13.7(16)  
 Attribute\_Handle  
   *in* Ada.Task\_Attributes C.7.2(3)  
 Binary  
   *in* Interfaces.COBOL B.4(10)  
 Binary\_Format  
   *in* Interfaces.COBOL B.4(24)  
 Bit\_Order  
   *in* System 13.7(15/2)  
 Boolean  
   *in* Standard A.1(5)  
 Bounded\_String  
   *in* Ada.Strings.Bounded A.4.4(6)  
 Buffer\_Type *subtype of* Storage\_Array  
   *in* Ada.Storage\_IO A.9(4)  
 Byte  
   *in* Interfaces.COBOL B.4(29)

- Byte\_Array
  - in* Interfaces.COBOL B.4(29)
- C\_float
  - in* Interfaces.C B.3(15)
- Cause\_Of\_Termination
  - in* Ada.Task\_Termination C.7.3(3/2)
- char
  - in* Interfaces.C B.3(19)
- char16\_array
  - in* Interfaces.C B.3(39.5/2)
- char16\_t
  - in* Interfaces.C B.3(39.2/2)
- char32\_array
  - in* Interfaces.C B.3(39.14/2)
- char32\_t
  - in* Interfaces.C B.3(39.11/2)
- char\_array
  - in* Interfaces.C B.3(23)
- char\_array\_access
  - in* Interfaces.C.Strings B.3.1(4)
- Character
  - in* Standard A.1(35/3)
- Character\_Mapping
  - in* Ada.Strings.Maps A.4.2(20/2)
- Character\_Mapping\_Function
  - in* Ada.Strings.Maps A.4.2(25)
- Character\_Range
  - in* Ada.Strings.Maps A.4.2(6)
- Character\_Ranges
  - in* Ada.Strings.Maps A.4.2(7)
- Character\_Sequence *subtype of* String
  - in* Ada.Strings.Maps A.4.2(16)
- Character\_Set
  - in* Ada.Strings.Maps A.4.2(4/2)
  - in* Interfaces.Fortran B.5(11)
- chars\_ptr
  - in* Interfaces.C.Strings B.3.1(5/2)
- chars\_ptr\_array
  - in* Interfaces.C.Strings B.3.1(6/2)
- COBOL\_Character
  - in* Interfaces.COBOL B.4(13)
- Complex
  - in* Ada.Numerics.Generic\_Complex\_Types G.1.1(3)
  - in* Interfaces.Fortran B.5(9)
- Complex\_Matrix
  - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(4/2)
- Complex\_Vector
  - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(4/2)
- Controlled
  - in* Ada.Finalization 7.6(5/2)
- Count
  - in* Ada.Direct\_IO A.8.4(4)
  - in* Ada.Streams.Stream\_IO A.12.1(7)
  - in* Ada.Text\_IO A.10.1(5)
- Count\_Type
  - in* Ada.Containers A.18.1(5/2)
- CPU\_Time
  - in* Ada.Execution\_Time D.14(4/2)
- Cursor
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(7/2)
  - in* Ada.Containers.Hashed\_Maps A.18.5(4/2)
  - in* Ada.Containers.Hashed\_Sets A.18.8(4/2)
  - in* Ada.Containers.Multiway\_Trees A.18.10(9/3)
  - in* Ada.Containers.Ordered\_Maps A.18.6(5/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(5/2)
  - in* Ada.Containers.Vectors A.18.2(9/2)
- Day\_Count
  - in* Ada.Calendar.Arithmetic 9.6.1(10/2)
- Day\_Duration *subtype of* Duration
  - in* Ada.Calendar 9.6(11/2)
- Day\_Name
  - in* Ada.Calendar.Formatting 9.6.1(17/2)
- Day\_Number *subtype of* Integer
  - in* Ada.Calendar 9.6(11/2)
- Deadline *subtype of* Time
  - in* Ada.Dispatching.EDF D.2.6(9/2)
- Decimal\_Element
  - in* Interfaces.COBOL B.4(12)
- Direction
  - in* Ada.Strings A.4.1(6)
- Directory\_Entry\_Type
  - in* Ada.Directories A.16(29/2)
- Display\_Format
  - in* Interfaces.COBOL B.4(22)
- double
  - in* Interfaces.C B.3(16)
- Double\_Precision
  - in* Interfaces.Fortran B.5(6)
- Duration
  - in* Standard A.1(43)
- Encoding\_Scheme
  - in* Ada.Strings.UTF\_Encoding A.4.11(4/3)
- Exception\_Id
  - in* Ada.Exceptions 11.4.1(2/2)
- Exception\_Occurrence
  - in* Ada.Exceptions 11.4.1(3/2)
- Exception\_Occurrence\_Access
  - in* Ada.Exceptions 11.4.1(3/2)
- Exit\_Status
  - in* Ada.Command\_Line A.15(7)
- Extended\_Index *subtype of* Index\_Type'Base
  - in* Ada.Containers.Vectors A.18.2(7/2)
- Field *subtype of* Integer
  - in* Ada.Text\_IO A.10.1(6)
- File\_Access
  - in* Ada.Text\_IO A.10.1(18)
- File\_Kind
  - in* Ada.Directories A.16(22/2)
- File\_Mode
  - in* Ada.Direct\_IO A.8.4(4)
  - in* Ada.Sequential\_IO A.8.1(4)
  - in* Ada.Streams.Stream\_IO A.12.1(6)
  - in* Ada.Text\_IO A.10.1(4)
- File\_Size
  - in* Ada.Directories A.16(23/2)
- File\_Type
  - in* Ada.Direct\_IO A.8.4(3)
  - in* Ada.Sequential\_IO A.8.1(3)

- in* Ada.Streams.Stream\_IO A.12.1(5)
- in* Ada.Text\_IO A.10.1(3)
- Filter\_Type
  - in* Ada.Directories A.16(30/2)
- Float
  - in* Standard A.1(21)
- Floating
  - in* Interfaces.COBOL B.4(9)
- Fortran\_Character
  - in* Interfaces.Fortran B.5(12)
- Fortran\_Integer
  - in* Interfaces.Fortran B.5(5)
- Generator
  - in* Ada.Numerics.Discrete\_Random A.5.2(19)
  - in* Ada.Numerics.Float\_Random A.5.2(7)
- Group\_Budget
  - in* Ada.Execution\_Time.Group\_Budgets D.14.2(4/2)
- Group\_Budget\_Handler
  - in* Ada.Execution\_Time.Group\_Budgets D.14.2(5/2)
- Hash\_Type
  - in* Ada.Containers A.18.1(4/2)
- Holder
  - in* Ada.Containers.Indefinite\_Holders A.18.18(6/3)
- Hour\_Number *subtype of* Natural
  - in* Ada.Calendar.Formatting 9.6.1(20/2)
- Imaginary
  - in* Ada.Numerics.Generic\_Complex\_Types G.1.1(4/2)
- Imaginary *subtype of* Imaginary
  - in* Interfaces.Fortran B.5(10)
- int
  - in* Interfaces.C B.3(7)
- Integer
  - in* Standard A.1(12)
- Integer\_Address
  - in* System.Storage\_Elements 13.7.1(10)
- Interrupt\_ID
  - in* Ada.Interrupts C.3.2(2)
- Interrupt\_Priority *subtype of* Any\_Priority
  - in* System 13.7(16)
- ISO\_646 *subtype of* Character
  - in* Ada.Characters.Handling A.3.2(9)
- Leap\_Seconds\_Count *subtype of* Integer
  - in* Ada.Calendar.Arithmetic 9.6.1(11/2)
- Length\_Range *subtype of* Natural
  - in* Ada.Strings.Bounded A.4.4(8)
- Limited\_Controlled
  - in* Ada.Finalization 7.6(7/2)
- List
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(6/2)
- Logical
  - in* Interfaces.Fortran B.5(7)
- long
  - in* Interfaces.C B.3(7)
- Long\_Binary
  - in* Interfaces.COBOL B.4(10)
- long\_double
  - in* Interfaces.C B.3(17)
- Long\_Floating
  - in* Interfaces.COBOL B.4(9)
- Map
  - in* Ada.Containers.Hashed\_Maps A.18.5(3/2)
  - in* Ada.Containers.Ordered\_Maps A.18.6(4/2)
- Membership
  - in* Ada.Strings A.4.1(6)
- Minute\_Number *subtype of* Natural
  - in* Ada.Calendar.Formatting 9.6.1(20/2)
- Month\_Number *subtype of* Integer
  - in* Ada.Calendar 9.6(11/2)
- Name
  - in* System 13.7(4)
- Name\_Case\_Kind
  - in* Ada.Directories A.16(20.1/3)
- Natural *subtype of* Integer
  - in* Standard A.1(13)
- Number\_Base *subtype of* Integer
  - in* Ada.Text\_IO A.10.1(6)
- Numeric
  - in* Interfaces.COBOL B.4(20)
- Packed\_Decimal
  - in* Interfaces.COBOL B.4(12)
- Packed\_Format
  - in* Interfaces.COBOL B.4(26)
- Parameterless\_Handler
  - in* Ada.Interrupts C.3.2(2)
- Params\_Stream\_Type
  - in* System.RPC E.5(6)
- Partition\_Id
  - in* System.RPC E.5(4)
- Picture
  - in* Ada.Text\_IO.Editing F.3.3(4)
- plain\_char
  - in* Interfaces.C B.3(11)
- Pointer
  - in* Interfaces.C.Pointers B.3.2(5)
- Positive *subtype of* Integer
  - in* Standard A.1(13)
- Positive\_Count *subtype of* Count
  - in* Ada.Direct\_IO A.8.4(4)
  - in* Ada.Streams.Stream\_IO A.12.1(7)
  - in* Ada.Text\_IO A.10.1(5)
- Priority *subtype of* Any\_Priority
  - in* System 13.7(16)
- ptrdiff\_t
  - in* Interfaces.C B.3(12)
- Real
  - in* Interfaces.Fortran B.5(6)
- Real\_Matrix
  - in* Ada.Numerics.Generic\_Real\_Arrays G.3.1(4/2)
- Real\_Vector
  - in* Ada.Numerics.Generic\_Real\_Arrays G.3.1(4/2)
- Root\_Storage\_Pool
  - in* System.Storage\_Pools 13.11(6/2)
- Root\_Stream\_Type
  - in* Ada.Streams 13.13.1(3/2)
- RPC\_Receiver
  - in* System.RPC E.5(11)
- Search\_Type
  - in* Ada.Directories A.16(31/2)

Second\_Duration *subtype of* Day\_Duration  
     *in* Ada.Calendar.Formatting 9.6.1(20/2)  
 Second\_Number *subtype of* Natural  
     *in* Ada.Calendar.Formatting 9.6.1(20/2)  
 Seconds\_Count  
     *in* Ada.Real\_Time D.8(15)  
 Set  
     *in* Ada.Containers.Hash\_Sets A.18.8(3/2)  
     *in* Ada.Containers.Ordered\_Sets A.18.9(4/2)  
 short  
     *in* Interfaces.C B.3(7)  
 signed\_char  
     *in* Interfaces.C B.3(8)  
 size\_t  
     *in* Interfaces.C B.3(13)  
 State  
     *in* Ada.Numerics.Discrete\_Random A.5.2(23)  
     *in* Ada.Numerics.Float\_Random A.5.2(11)  
 Storage\_Array  
     *in* System.Storage\_Elements 13.7.1(5)  
 Storage\_Count *subtype of* Storage\_Offset  
     *in* System.Storage\_Elements 13.7.1(4)  
 Storage\_Element  
     *in* System.Storage\_Elements 13.7.1(5)  
 Storage\_Offset  
     *in* System.Storage\_Elements 13.7.1(3)  
 Stream\_Access  
     *in* Ada.Streams.Stream\_IO A.12.1(4)  
     *in* Ada.Text\_IO.Text\_Streams A.12.2(3)  
     *in* Ada.Wide\_Text\_IO.Text\_Streams A.12.3(3)  
     *in* Ada.Wide\_Wide\_Text\_IO.Text\_Streams A.12.4(3/2)  
 Stream\_Element  
     *in* Ada.Streams 13.13.1(4/1)  
 Stream\_Element\_Array  
     *in* Ada.Streams 13.13.1(4/1)  
 Stream\_Element\_Count *subtype of* Stream\_Element\_Offset  
     *in* Ada.Streams 13.13.1(4/1)  
 Stream\_Element\_Offset  
     *in* Ada.Streams 13.13.1(4/1)  
 String  
     *in* Standard A.1(37)  
 String\_Access  
     *in* Ada.Strings.Unbounded A.4.5(7)  
 Suspension\_Object  
     *in* Ada.Synchronous\_Task\_Control D.10(4)  
 Tag  
     *in* Ada.Tags 3.9(6/2)  
 Tag\_Array  
     *in* Ada.Tags 3.9(7.3/2)  
 Task\_Array  
     *in* Ada.Execution\_Time.Group\_Budgets D.14.2(6/2)  
 Task\_Id  
     *in* Ada.Task\_Identification C.7.1(2/2)  
 Termination\_Handler  
     *in* Ada.Task\_Termination C.7.3(4/2)  
 Time  
     *in* Ada.Calendar 9.6(10)  
     *in* Ada.Real\_Time D.8(4)  
 Time\_Offset  
     *in* Ada.Calendar.Time\_Zones 9.6.1(4/2)  
 Time\_Span  
     *in* Ada.Real\_Time D.8(5)  
 Timer  
     *in* Ada.Execution\_Time.Timers D.14.1(4/2)  
 Timer\_Handler  
     *in* Ada.Execution\_Time.Timers D.14.1(5/2)  
 Timing\_Event  
     *in* Ada.Real\_Time.Timing\_Events D.15(4/2)  
 Timing\_Event\_Handler  
     *in* Ada.Real\_Time.Timing\_Events D.15(4/2)  
 Tree  
     *in* Ada.Containers.Multiway\_Trees A.18.10(8/3)  
 Trim\_End  
     *in* Ada.Strings A.4.1(6)  
 Truncation  
     *in* Ada.Strings A.4.1(6)  
 Type\_Set  
     *in* Ada.Text\_IO A.10.1(7)  
 Unbounded\_String  
     *in* Ada.Strings.Unbounded A.4.5(4/2)  
 Uniformly\_Distributed *subtype of* Float  
     *in* Ada.Numerics.Float\_Random A.5.2(8)  
 unsigned  
     *in* Interfaces.C B.3(9)  
 unsigned\_char  
     *in* Interfaces.C B.3(10)  
 unsigned\_long  
     *in* Interfaces.C B.3(9)  
 unsigned\_short  
     *in* Interfaces.C B.3(9)  
 UTF\_16\_Wide\_String *subtype of* Wide\_String  
     *in* Ada.Strings.UTF\_Encoding A.4.11(7/3)  
 UTF\_8\_String *subtype of* String  
     *in* Ada.Strings.UTF\_Encoding A.4.11(6/3)  
 UTF\_String *subtype of* String  
     *in* Ada.Strings.UTF\_Encoding A.4.11(5/3)  
 Vector  
     *in* Ada.Containers.Vectors A.18.2(8/2)  
 wchar\_array  
     *in* Interfaces.C B.3(33)  
 wchar\_t  
     *in* Interfaces.C B.3(30/1)  
 Wide\_Character  
     *in* Standard A.1(36.1/2)  
 Wide\_Character\_Mapping  
     *in* Ada.Strings.Wide\_Maps A.4.7(20/2)  
 Wide\_Character\_Mapping\_Function  
     *in* Ada.Strings.Wide\_Maps A.4.7(26)  
 Wide\_Character\_Range  
     *in* Ada.Strings.Wide\_Maps A.4.7(6)  
 Wide\_Character\_Ranges  
     *in* Ada.Strings.Wide\_Maps A.4.7(7)  
 Wide\_Character\_Sequence *subtype of* Wide\_String  
     *in* Ada.Strings.Wide\_Maps A.4.7(16)  
 Wide\_Character\_Set  
     *in* Ada.Strings.Wide\_Maps A.4.7(4/2)  
 Wide\_String  
     *in* Standard A.1(41)  
 Wide\_Wide\_Character  
     *in* Standard A.1(36.2/2)

Wide\_Wide\_Character\_Mapping  
  *in* Ada.Strings.Wide\_Wide\_Maps A.4.8(20/2)  
Wide\_Wide\_Character\_Mapping\_Function  
  *in* Ada.Strings.Wide\_Wide\_Maps A.4.8(26/2)  
Wide\_Wide\_Character\_Range  
  *in* Ada.Strings.Wide\_Wide\_Maps A.4.8(6/2)  
Wide\_Wide\_Character\_Ranges  
  *in* Ada.Strings.Wide\_Wide\_Maps A.4.8(7/2)

Wide\_Wide\_Character\_Sequence *subtype of*  
  Wide\_Wide\_String  
  *in* Ada.Strings.Wide\_Wide\_Maps A.4.8(16/2)  
Wide\_Wide\_Character\_Set  
  *in* Ada.Strings.Wide\_Wide\_Maps A.4.8(4/2)  
Wide\_Wide\_String  
  *in* Standard A.1(42.1/2)  
Year\_Number *subtype of* Integer  
  *in* Ada.Calendar 9.6(11/2)

## Q.3 Language-Defined Subprograms

1/2 This clause lists all language-defined subprograms.

Abort\_Task *in* Ada.Task\_Identification C.7.1(3/1)  
 Actual\_Quantum  
   *in* Ada.Dispatching.Round\_Robin D.2.5(4/2)  
 Add  
   *in* Ada.Execution\_Time.Group\_Budgets D.14.2(9/2)  
 Add\_Task  
   *in* Ada.Execution\_Time.Group\_Budgets D.14.2(8/2)  
 Adjust *in* Ada.Finalization 7.6(6/2)  
 Allocate *in* System.Storage\_Pools 13.11(7)  
 Ancestor\_Find  
   *in* Ada.Containers.Multiway\_Trees A.18.10(34/3)  
 Append  
   *in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(23/2)  
   *in* Ada.Containers.Vectors A.18.2(46/2), A.18.2(47/2)  
   *in* Ada.Strings.Bounded A.4.4(13), A.4.4(14), A.4.4(15),  
     A.4.4(16), A.4.4(17), A.4.4(18), A.4.4(19), A.4.4(20)  
   *in* Ada.Strings.Unbounded A.4.5(12), A.4.5(13), A.4.5(14)  
 Append\_Child  
   *in* Ada.Containers.Multiway\_Trees A.18.10(45/3)  
 Arccos  
   *in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions  
     G.1.2(5)  
   *in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(6)  
 Arccosh  
   *in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions  
     G.1.2(7)  
   *in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(7)  
 Arccot  
   *in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions  
     G.1.2(5)  
   *in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(6)  
 Arccoth  
   *in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions  
     G.1.2(7)  
   *in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(7)  
 Arcsin  
   *in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions  
     G.1.2(5)  
   *in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(6)  
 Arcsinh  
   *in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions  
     G.1.2(7)  
   *in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(7)  
 Arctan  
   *in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions  
     G.1.2(5)  
   *in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(6)  
 Arctanh  
   *in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions  
     G.1.2(7)  
   *in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(7)  
 Argument  
   *in* Ada.Command\_Line A.15(5)  
   *in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(10/2),  
     G.3.2(31/2)  
   *in* Ada.Numerics.Generic\_Complex\_Types G.1.1(10)  
 Argument\_Count *in* Ada.Command\_Line A.15(4)  
 Assign  
   *in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(17.1/3)  
   *in* Ada.Containers.Hashed\_Maps A.18.5(17.1/3)  
   *in* Ada.Containers.Hashed\_Sets A.18.8(17.1/3)  
   *in* Ada.Containers.Indefinite\_Holders A.18.18(16/3)  
   *in* Ada.Containers.Multiway\_Trees A.18.10(26/3)  
   *in* Ada.Containers.Ordered\_Maps A.18.6(16.1/3)  
   *in* Ada.Containers.Ordered\_Sets A.18.9(16.1/3)  
   *in* Ada.Containers.Vectors A.18.2(34.1/3)  
 Attach\_Handler *in* Ada.Interrupts C.3.2(7)  
 Base\_Name *in* Ada.Directories A.16(19/2)  
 Blank\_When\_Zero  
   *in* Ada.Text\_IO Editing F.3.3(7)  
 Bounded\_Slice *in* Ada.Strings.Bounded A.4.4(28.1/2),  
   A.4.4(28.2/2)  
 Budget\_Has\_Expired  
   *in* Ada.Execution\_Time.Group\_Budgets D.14.2(9/2)  
 Budget\_Remaining  
   *in* Ada.Execution\_Time.Group\_Budgets D.14.2(9/2)  
 Cancel\_Handler  
   *in* Ada.Execution\_Time.Group\_Budgets D.14.2(10/2)  
   *in* Ada.Execution\_Time.Timers D.14.1(7/2)  
   *in* Ada.Real\_Time.Timing\_Events D.15(5/2)  
 Capacity  
   *in* Ada.Containers.Hashed\_Maps A.18.5(8/2)  
   *in* Ada.Containers.Hashed\_Sets A.18.8(10/2)  
   *in* Ada.Containers.Vectors A.18.2(19/2)  
 Ceiling  
   *in* Ada.Containers.Ordered\_Maps A.18.6(41/2)  
   *in* Ada.Containers.Ordered\_Sets A.18.9(51/2), A.18.9(71/2)  
 Child\_Count  
   *in* Ada.Containers.Multiway\_Trees A.18.10(39/3)  
 Child\_Depth  
   *in* Ada.Containers.Multiway\_Trees A.18.10(40/3)  
 Clear  
   *in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(13/2)  
   *in* Ada.Containers.Hashed\_Maps A.18.5(12/2)  
   *in* Ada.Containers.Hashed\_Sets A.18.8(14/2)  
   *in* Ada.Containers.Indefinite\_Holders A.18.18(11/3)  
   *in* Ada.Containers.Multiway\_Trees A.18.10(21/3)  
   *in* Ada.Containers.Ordered\_Maps A.18.6(11/2)  
   *in* Ada.Containers.Ordered\_Sets A.18.9(13/2)  
   *in* Ada.Containers.Vectors A.18.2(24/2)  
   *in* Ada.Environment\_Variables A.17(7/2)  
 Clock  
   *in* Ada.Calendar 9.6(12)  
   *in* Ada.Execution\_Time D.14(5/2)

- in* Ada.Real\_Time D.8(6)
- Close
  - in* Ada.Direct\_IO A.8.4(8)
  - in* Ada.Sequential\_IO A.8.1(8)
  - in* Ada.Streams.Stream\_IO A.12.1(10)
  - in* Ada.Text\_IO A.10.1(11)
- Col *in* Ada.Text\_IO A.10.1(37)
- Command\_Name *in* Ada.Command\_Line A.15(6)
- Compose
  - in* Ada.Directories A.16(20/2)
  - in* Ada.Directories.Hierarchical\_File\_Names A.16.1(14/3)
- Compose\_From\_Cartesian
  - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(9/2), G.3.2(29/2)
  - in* Ada.Numerics.Generic\_Complex\_Types G.1.1(8)
- Compose\_From\_Polar
  - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(11/2), G.3.2(32/2)
  - in* Ada.Numerics.Generic\_Complex\_Types G.1.1(11)
- Conjugate
  - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(13/2), G.3.2(34/2)
  - in* Ada.Numerics.Generic\_Complex\_Types G.1.1(12), G.1.1(15)
- Containing\_Directory
  - in* Ada.Directories A.16(17/2)
  - in* Ada.Directories.Hierarchical\_File\_Names A.16.1(11/3)
- Contains
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(43/2)
  - in* Ada.Containers.Hashed\_Maps A.18.5(32/2)
  - in* Ada.Containers.Hashed\_Sets A.18.8(44/2), A.18.8(57/2)
  - in* Ada.Containers.Multiway\_Trees A.18.10(35/3)
  - in* Ada.Containers.Ordered\_Maps A.18.6(42/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(52/2), A.18.9(72/2)
  - in* Ada.Containers.Vectors A.18.2(71/2)
- Continue
  - in* Ada.Asynchronous\_Task\_Control D.11(3/2)
- Convert
  - in* Ada.Strings.UTF\_Encoding.Conversions A.4.11(16/3), A.4.11(17/3), A.4.11(18/3), A.4.11(19/3), A.4.11(20/3)
- Copy
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(17.2/3)
  - in* Ada.Containers.Hashed\_Maps A.18.5(17.2/3)
  - in* Ada.Containers.Hashed\_Sets A.18.8(17.2/3)
  - in* Ada.Containers.Indefinite\_Holders A.18.18(17/3), A.18.20(10/3), A.18.21(13/3), A.18.22(10/3), A.18.23(13/3), A.18.24(10/3)
  - in* Ada.Containers.Multiway\_Trees A.18.10(27/3)
  - in* Ada.Containers.Ordered\_Maps A.18.6(16.2/3)
  - in* Ada.Containers.Ordered\_Sets A.18.9(16.2/3)
  - in* Ada.Containers.Vectors A.18.2(34.2/3)
- Copy\_Array *in* Interfaces.C.Pointers B.3.2(15)
- Copy\_File *in* Ada.Directories A.16(13/2)
- Copy\_Subtree
  - in* Ada.Containers.Multiway\_Trees A.18.10(47/3)
- Copy\_Terminated\_Array
  - in* Interfaces.C.Pointers B.3.2(14)
- Cos
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(4)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(5)
- Cosh
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(6)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(7)
- Cot
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(4)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(5)
- Coth
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(6)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(7)
- Count
  - in* Ada.Strings.Bounded A.4.4(48), A.4.4(49), A.4.4(50)
  - in* Ada.Strings.Fixed A.4.3(13), A.4.3(14), A.4.3(15)
  - in* Ada.Strings.Unbounded A.4.5(43), A.4.5(44), A.4.5(45)
- Create
  - in* Ada.Direct\_IO A.8.4(6)
  - in* Ada.Sequential\_IO A.8.1(6)
  - in* Ada.Streams.Stream\_IO A.12.1(8)
  - in* Ada.Text\_IO A.10.1(9)
- Create\_Directory *in* Ada.Directories A.16(7/2)
- Create\_Path *in* Ada.Directories A.16(9/2)
- Current\_Directory *in* Ada.Directories A.16(5/2)
- Current\_Error *in* Ada.Text\_IO A.10.1(17), A.10.1(20)
- Current\_Handler
  - in* Ada.Execution\_Time.Group\_Budgets D.14.2(10/2)
  - in* Ada.Execution\_Time.Timers D.14.1(7/2)
  - in* Ada.Interrupts C.3.2(6)
  - in* Ada.Real\_Time.Timing\_Events D.15(5/2)
- Current\_Input *in* Ada.Text\_IO A.10.1(17), A.10.1(20)
- Current\_Output *in* Ada.Text\_IO A.10.1(17), A.10.1(20)
- Current\_State
  - in* Ada.Synchronous\_Task\_Control D.10(4)
- Current\_Task
  - in* Ada.Task\_Identification C.7.1(3/1)
- Current\_Task\_Fallback\_Handler
  - in* Ada.Task\_Termination C.7.3(5/2)
- Day
  - in* Ada.Calendar 9.6(13)
  - in* Ada.Calendar.Formatting 9.6.1(23/2)
- Day\_of\_Week
  - in* Ada.Calendar.Formatting 9.6.1(18/2)
- Deallocate *in* System.Storage\_Pools 13.11(8)
- Decode
  - in* Ada.Strings.UTF\_Encoding.Wide\_Wide\_Encoding A.4.11(26/3), A.4.11(27/3), A.4.11(28/3), A.4.11(34/3), A.4.11(35/3), A.4.11(36/3)
- Decrement *in* Interfaces.C.Pointers B.3.2(11)
- Default\_Modulus
  - in* Ada.Containers.Indefinite\_Holders A.18.21(10/3), A.18.23(10/3)
- Delay\_Until\_And\_Set\_Deadline
  - in* Ada.Dispatching.EDF D.2.6(9/2)

## Delete

*in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(24/2)  
*in* Ada.Containers.Hashed\_Maps A.18.5(25/2), A.18.5(26/2)  
*in* Ada.Containers.Hashed\_Sets A.18.8(24/2), A.18.8(25/2),  
 A.18.8(55/2)  
*in* Ada.Containers.Ordered\_Maps A.18.6(24/2),  
 A.18.6(25/2)  
*in* Ada.Containers.Ordered\_Sets A.18.9(23/2), A.18.9(24/2),  
 A.18.9(68/2)  
*in* Ada.Containers.Vectors A.18.2(50/2), A.18.2(51/2)  
*in* Ada.Direct\_IO A.8.4(8)  
*in* Ada.Sequential\_IO A.8.1(8)  
*in* Ada.Streams.Stream\_IO A.12.1(10)  
*in* Ada.Strings.Bounded A.4.4(64), A.4.4(65)  
*in* Ada.Strings.Fixed A.4.3(29), A.4.3(30)  
*in* Ada.Strings.Unbounded A.4.5(59), A.4.5(60)  
*in* Ada.Text\_IO A.10.1(11)

## Delete\_Children

*in* Ada.Containers.Multiway\_Trees A.18.10(46/3)

Delete\_Directory *in* Ada.Directories A.16(8/2)Delete\_File *in* Ada.Directories A.16(11/2)

## Delete\_First

*in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(25/2)  
*in* Ada.Containers.Ordered\_Maps A.18.6(26/2)  
*in* Ada.Containers.Ordered\_Sets A.18.9(25/2)  
*in* Ada.Containers.Vectors A.18.2(52/2)

## Delete\_Last

*in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(26/2)  
*in* Ada.Containers.Ordered\_Maps A.18.6(27/2)  
*in* Ada.Containers.Ordered\_Sets A.18.9(26/2)  
*in* Ada.Containers.Vectors A.18.2(53/2)

## Delete\_Leaf

*in* Ada.Containers.Multiway\_Trees A.18.10(29/3)

## Delete\_Subtree

*in* Ada.Containers.Multiway\_Trees A.18.10(30/3)

Delete\_Tree *in* Ada.Directories A.16(10/2)

## Depth

*in* Ada.Containers.Multiway\_Trees A.18.10(17/3)

## Dereference\_Error

*in* Interfaces.C.Strings B.3.1(12)

Descendant\_Tag *in* Ada.Tags 3.9(7.1/2)Detach\_Handler *in* Ada.Interrupts C.3.2(9)

## Determinant

*in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(46/2)  
*in* Ada.Numerics.Generic\_Real\_Arrays G.3.1(24/2)

## Difference

*in* Ada.Calendar.Arithmetic 9.6.1(12/2)  
*in* Ada.Containers.Hashed\_Sets A.18.8(32/2), A.18.8(33/2)  
*in* Ada.Containers.Ordered\_Sets A.18.9(33/2), A.18.9(34/2)

Divide *in* Ada.Decimal F.2(6)Do\_APC *in* System.RPC E.5(10)Do\_RPC *in* System.RPC E.5(9)

## Eigensystem

*in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(49/2)  
*in* Ada.Numerics.Generic\_Real\_Arrays G.3.1(27/2)

## Eigenvalues

*in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(48/2)  
*in* Ada.Numerics.Generic\_Real\_Arrays G.3.1(26/2)

## Element

*in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(14/2)  
*in* Ada.Containers.Hashed\_Maps A.18.5(14/2), A.18.5(31/2)  
*in* Ada.Containers.Hashed\_Sets A.18.8(15/2), A.18.8(52/2)  
*in* Ada.Containers.Indefinite\_Holders A.18.18(12/3)  
*in* Ada.Containers.Multiway\_Trees A.18.10(22/3)  
*in* Ada.Containers.Ordered\_Maps A.18.6(13/2),  
 A.18.6(39/2)  
*in* Ada.Containers.Ordered\_Sets A.18.9(14/2), A.18.9(65/2)  
*in* Ada.Containers.Vectors A.18.2(27/2), A.18.2(28/2)  
*in* Ada.Strings.Bounded A.4.4(26)  
*in* Ada.Strings.Unbounded A.4.5(20)

## Encode

*in* Ada.Strings.UTF\_Encoding.Wide\_Wide\_Encoding  
 A.4.11(23/3), A.4.11(24/3), A.4.11(25/3), A.4.11(31/3),  
 A.4.11(32/3), A.4.11(33/3)

Encoding *in* Ada.Strings.UTF\_Encoding A.4.11(13/3)

## End\_Of\_File

*in* Ada.Direct\_IO A.8.4(16)  
*in* Ada.Sequential\_IO A.8.1(13)  
*in* Ada.Streams.Stream\_IO A.12.1(12)  
*in* Ada.Text\_IO A.10.1(34)

End\_Of\_Line *in* Ada.Text\_IO A.10.1(30)End\_Of\_Page *in* Ada.Text\_IO A.10.1(33)End\_Search *in* Ada.Directories A.16(33/2)

## Equal\_Case\_Insensitive

*child of* Ada.Strings A.4.10(2/3)  
*child of* Ada.Strings.Bounded A.4.10(7/3)  
*child of* Ada.Strings.Fixed A.4.10(5/3)  
*child of* Ada.Strings.Unbounded A.4.10(10/3)

## Equal\_Subtree

*in* Ada.Containers.Multiway\_Trees A.18.10(12/3)

## Equivalent\_Elements

*in* Ada.Containers.Hashed\_Sets A.18.8(46/2), A.18.8(47/2),  
 A.18.8(48/2)  
*in* Ada.Containers.Ordered\_Sets A.18.9(3/2)

## Equivalent\_Keys

*in* Ada.Containers.Hashed\_Maps A.18.5(34/2),  
 A.18.5(35/2), A.18.5(36/2)  
*in* Ada.Containers.Ordered\_Maps A.18.6(3/2)  
*in* Ada.Containers.Ordered\_Sets A.18.9(63/2)

## Equivalent\_Sets

*in* Ada.Containers.Hashed\_Sets A.18.8(8/2)  
*in* Ada.Containers.Ordered\_Sets A.18.9(9/2)

Establish\_RPC\_Receiver *in* System.RPC E.5(12)Exception\_Identity *in* Ada.Exceptions 11.4.1(5/2)

## Exception\_Information

*in* Ada.Exceptions 11.4.1(5/2)

Exception\_Message *in* Ada.Exceptions 11.4.1(4/2)Exception\_Name *in* Ada.Exceptions 11.4.1(2/2), 11.4.1(5/2)Exchange\_Handler *in* Ada.Interrupts C.3.2(8)

## Exclude

*in* Ada.Containers.Hashed\_Maps A.18.5(24/2)  
*in* Ada.Containers.Hashed\_Sets A.18.8(23/2), A.18.8(54/2)  
*in* Ada.Containers.Ordered\_Maps A.18.6(23/2)  
*in* Ada.Containers.Ordered\_Sets A.18.9(22/2), A.18.9(67/2)

## Exists

*in* Ada.Directories A.16(24/2)  
*in* Ada.Environment\_Variables A.17(5/2)



- Exp  
*in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(3)  
*in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(4)  
 Expanded\_Name *in* Ada.Tags 3.9(7/2)  
 Extension *in* Ada.Directories A.16(18/2)  
 External\_Tag *in* Ada.Tags 3.9(7/2)  
 Finalize *in* Ada.Finalization 7.6(6/2), 7.6(8/2)  
 Find  
*in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(41/2)  
*in* Ada.Containers.Hashed\_Maps A.18.5(30/2)  
*in* Ada.Containers.Hashed\_Sets A.18.8(43/2), A.18.8(56/2)  
*in* Ada.Containers.Multiway\_Trees A.18.10(32/3)  
*in* Ada.Containers.Ordered\_Maps A.18.6(38/2)  
*in* Ada.Containers.Ordered\_Sets A.18.9(49/2), A.18.9(69/2)  
*in* Ada.Containers.Vectors A.18.2(68/2)  
 Find\_In\_Subtree  
*in* Ada.Containers.Multiway\_Trees A.18.10(33/3)  
 Find\_Index *in* Ada.Containers.Vectors A.18.2(67/2)  
 Find-Token  
*in* Ada.Strings.Bounded A.4.4(50.1/3), A.4.4(51)  
*in* Ada.Strings.Fixed A.4.3(15.1/3), A.4.3(16)  
*in* Ada.Strings.Unbounded A.4.5(45.1/3), A.4.5(46)  
 First  
*in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(33/2)  
*in* Ada.Containers.Hashed\_Maps A.18.5(27/2)  
*in* Ada.Containers.Hashed\_Sets A.18.8(40/2)  
*in* Ada.Containers.Ordered\_Maps A.18.6(28/2)  
*in* Ada.Containers.Ordered\_Sets A.18.9(41/2)  
*in* Ada.Containers.Vectors A.18.2(58/2)  
 First\_Child  
*in* Ada.Containers.Multiway\_Trees A.18.10(53/3)  
 First\_Child\_Element  
*in* Ada.Containers.Multiway\_Trees A.18.10(54/3)  
 First\_Element  
*in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(34/2)  
*in* Ada.Containers.Ordered\_Maps A.18.6(29/2)  
*in* Ada.Containers.Ordered\_Sets A.18.9(42/2)  
*in* Ada.Containers.Vectors A.18.2(59/2)  
 First\_Index *in* Ada.Containers.Vectors A.18.2(57/2)  
 First\_Key  
*in* Ada.Containers.Ordered\_Maps A.18.6(30/2)  
 Floor  
*in* Ada.Containers.Ordered\_Maps A.18.6(40/2)  
*in* Ada.Containers.Ordered\_Sets A.18.9(50/2), A.18.9(70/2)  
 Flush  
*in* Ada.Streams.Stream\_IO A.12.1(25/1)  
*in* Ada.Text\_IO A.10.1(21/1)  
 Form  
*in* Ada.Direct\_IO A.8.4(9)  
*in* Ada.Sequential\_IO A.8.1(9)  
*in* Ada.Streams.Stream\_IO A.12.1(11)  
*in* Ada.Text\_IO A.10.1(12)  
 Free  
*in* Ada.Strings.Unbounded A.4.5(7)  
*in* Interfaces.C.Strings B.3.1(11)  
 Full\_Name *in* Ada.Directories A.16(15/2), A.16(39/2)  
 Generic\_Array\_Sort  
*child of* Ada.Containers A.18.26(3/2)  
 Generic\_Constrained\_Array\_Sort  
*child of* Ada.Containers A.18.26(7/2)  
 Generic\_Sort  
*child of* Ada.Containers A.18.26(9.2/3)  
 Get  
*in* Ada.Text\_IO A.10.1(41), A.10.1(47), A.10.1(54),  
 A.10.1(55), A.10.1(59), A.10.1(60), A.10.1(65), A.10.1(67),  
 A.10.1(70), A.10.1(72), A.10.1(75), A.10.1(77), A.10.1(81),  
 A.10.1(83)  
*in* Ada.Text\_IO.Complex\_IO G.1.3(6), G.1.3(8)  
 Get\_Deadline *in* Ada.Dispatching.EDF D.2.6(9/2)  
 Get\_Immediate *in* Ada.Text\_IO A.10.1(44), A.10.1(45)  
 Get\_Line  
*in* Ada.Text\_IO A.10.1(49), A.10.1(49.1/2)  
*in* Ada.Text\_IO.Bounded\_IO A.10.11(8/2), A.10.11(9/2),  
 A.10.11(10/2), A.10.11(11/2)  
*in* Ada.Text\_IO.Unbounded\_IO A.10.12(8/2), A.10.12(9/2),  
 A.10.12(10/2), A.10.12(11/2)  
 Get\_Next\_Entry *in* Ada.Directories A.16(35/2)  
 Get\_Priority  
*in* Ada.Dynamic\_Priorities D.5.1(5)  
 Has\_Element  
*in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(44/2)  
*in* Ada.Containers.Hashed\_Maps A.18.5(33/2)  
*in* Ada.Containers.Hashed\_Sets A.18.8(45/2)  
*in* Ada.Containers.Multiway\_Trees A.18.10(36/3)  
*in* Ada.Containers.Ordered\_Maps A.18.6(43/2)  
*in* Ada.Containers.Ordered\_Sets A.18.9(53/2)  
*in* Ada.Containers.Vectors A.18.2(72/2)  
 Hash  
*child of* Ada.Strings A.4.9(2/2)  
*child of* Ada.Strings.Bounded A.4.9(7/2)  
*child of* Ada.Strings.Unbounded A.4.9(10/2)  
 Hash\_Case\_Insensitive  
*child of* Ada.Strings A.4.9(13/3)  
*child of* Ada.Strings.Bounded A.4.9(14.4/3)  
*child of* Ada.Strings.Fixed A.4.9(14.2/3)  
*child of* Ada.Strings.Unbounded A.4.9(14.7/3)  
 Head  
*in* Ada.Strings.Bounded A.4.4(70), A.4.4(71)  
*in* Ada.Strings.Fixed A.4.3(35), A.4.3(36)  
*in* Ada.Strings.Unbounded A.4.5(65), A.4.5(66)  
 Hold *in* Ada.Asynchronous\_Task\_Control D.11(3/2)  
 Hour *in* Ada.Calendar.Formatting 9.6.1(24/2)  
 Im  
*in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(7/2),  
 G.3.2(27/2)  
*in* Ada.Numerics.Generic\_Complex\_Types G.1.1(6)  
 Image  
*in* Ada.Calendar.Formatting 9.6.1(35/2), 9.6.1(37/2)  
*in* Ada.Numerics.Discrete\_Random A.5.2(26)  
*in* Ada.Numerics.Float\_Random A.5.2(14)  
*in* Ada.Task\_Identification C.7.1(3/1)  
*in* Ada.Text\_IO.Editing F.3.3(13)  
 Include  
*in* Ada.Containers.Hashed\_Maps A.18.5(22/2)  
*in* Ada.Containers.Hashed\_Sets A.18.8(21/2)  
*in* Ada.Containers.Ordered\_Maps A.18.6(21/2)  
*in* Ada.Containers.Ordered\_Sets A.18.9(20/2)  
 Increment *in* Interfaces.C.Pointers B.3.2(11)

## Index

- in* Ada.Direct\_IO A.8.4(15)
- in* Ada.Streams.Stream\_IO A.12.1(23)
- in* Ada.Strings.Bounded A.4.4(43.2/2), A.4.4(44), A.4.4(45), A.4.4(45.1/2), A.4.4(46)
- in* Ada.Strings.Fixed A.4.3(8.1/2), A.4.3(8.2/2), A.4.3(9), A.4.3(10), A.4.3(10.1/2), A.4.3(11)
- in* Ada.Strings.Unbounded A.4.5(38.1/2), A.4.5(38.2/2), A.4.5(39), A.4.5(40), A.4.5(40.1/2), A.4.5(41)
- Index\_Non\_Blank
  - in* Ada.Strings.Bounded A.4.4(46.1/2), A.4.4(47)
  - in* Ada.Strings.Fixed A.4.3(11.1/2), A.4.3(12)
  - in* Ada.Strings.Unbounded A.4.5(41.1/2), A.4.5(42)
- Initial\_Directory
  - in* Ada.Directories.Hierarchical\_File\_Names A.16.1(12/3)
- Initialize *in* Ada.Finalization 7.6(6/2), 7.6(8/2)
- Insert
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(19/2), A.18.3(20/2), A.18.3(21/2)
  - in* Ada.Containers.Hashed\_Maps A.18.5(19/2), A.18.5(20/2), A.18.5(21/2)
  - in* Ada.Containers.Hashed\_Sets A.18.8(19/2), A.18.8(20/2)
  - in* Ada.Containers.Ordered\_Maps A.18.6(18/2), A.18.6(19/2), A.18.6(20/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(18/2), A.18.9(19/2)
  - in* Ada.Containers.Vectors A.18.2(36/2), A.18.2(37/2), A.18.2(38/2), A.18.2(39/2), A.18.2(40/2), A.18.2(41/2), A.18.2(42/2), A.18.2(43/2)
  - in* Ada.Strings.Bounded A.4.4(60), A.4.4(61)
  - in* Ada.Strings.Fixed A.4.3(25), A.4.3(26)
  - in* Ada.Strings.Unbounded A.4.5(55), A.4.5(56)
- Insert\_Child
  - in* Ada.Containers.Multiway\_Trees A.18.10(41/3), A.18.10(42/3), A.18.10(43/3)
- Insert\_Space
  - in* Ada.Containers.Vectors A.18.2(48/2), A.18.2(49/2)
- Interface\_Anccestor\_Tags *in* Ada.Tags 3.9(7.4/2)
- Internal\_Tag *in* Ada.Tags 3.9(7/2)
- Intersection
  - in* Ada.Containers.Hashed\_Sets A.18.8(29/2), A.18.8(30/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(30/2), A.18.9(31/2)
- Inverse
  - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(46/2)
  - in* Ada.Numerics.Generic\_Real\_Arrays G.3.1(24/2)
- Is\_A\_Group\_Member
  - in* Ada.Execution\_Time.Group\_Budgets D.14.2(8/2)
- Is\_Alphanumeric
  - in* Ada.Characters.Handling A.3.2(4)
- Is\_Attached *in* Ada.Interrupts C.3.2(5)
- Is\_Basic *in* Ada.Characters.Handling A.3.2(4)
- Is\_Callable
  - in* Ada.Task\_Identification C.7.1(4)
- Is\_Character
  - in* Ada.Characters.Conversions A.3.4(3/2)
- Is\_Control *in* Ada.Characters.Handling A.3.2(4)
- Is\_Current\_Directory\_Name
  - in* Ada.Directories.Hierarchical\_File\_Names A.16.1(7/3)
- Is\_Decimal\_Digit
  - in* Ada.Characters.Handling A.3.2(4)
- Is\_Descendant\_At\_Same\_Level
  - in* Ada.Tags 3.9(7.1/2)
- Is\_Digit *in* Ada.Characters.Handling A.3.2(4)
- Is\_Empty
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(12/2)
  - in* Ada.Containers.Hashed\_Maps A.18.5(11/2)
  - in* Ada.Containers.Hashed\_Sets A.18.8(13/2)
  - in* Ada.Containers.Indefinite\_Holders A.18.18(10/3)
  - in* Ada.Containers.Multiway\_Trees A.18.10(14/3)
  - in* Ada.Containers.Ordered\_Maps A.18.6(10/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(12/2)
  - in* Ada.Containers.Vectors A.18.2(23/2)
- Is\_Full\_Name
  - in* Ada.Directories.Hierarchical\_File\_Names A.16.1(8/3)
- Is\_Graphic *in* Ada.Characters.Handling A.3.2(4)
- Is\_Held
  - in* Ada.Asynchronous\_Task\_Control D.11(3/2)
- Is\_Hexadecimal\_Digit
  - in* Ada.Characters.Handling A.3.2(4)
- Is\_In
  - in* Ada.Strings.Maps A.4.2(13)
  - in* Ada.Strings.Wide\_Maps A.4.7(13)
  - in* Ada.Strings.Wide\_Wide\_Maps A.4.8(13/2)
- Is\_ISO\_646 *in* Ada.Characters.Handling A.3.2(10)
- Is\_Leaf
  - in* Ada.Containers.Multiway\_Trees A.18.10(19/3)
- Is\_Letter *in* Ada.Characters.Handling A.3.2(4)
- Is\_Lower *in* Ada.Characters.Handling A.3.2(4)
- Is\_Member
  - in* Ada.Execution\_Time.Group\_Budgets D.14.2(8/2)
- Is\_Nul\_Terminated *in* Interfaces.C B.3(24), B.3(35), B.3(39.16/2), B.3(39.7/2)
- Is\_Open
  - in* Ada.Direct\_IO A.8.4(10)
  - in* Ada.Sequential\_IO A.8.1(10)
  - in* Ada.Streams.Stream\_IO A.12.1(12)
  - in* Ada.Text\_IO A.10.1(13)
- Is\_Parent\_Directory\_Name
  - in* Ada.Directories.Hierarchical\_File\_Names A.16.1(6/3)
- Is\_Relative\_Name
  - in* Ada.Directories.Hierarchical\_File\_Names A.16.1(9/3)
- Is\_Reserved *in* Ada.Interrupts C.3.2(4)
- Is\_Root
  - in* Ada.Containers.Multiway\_Trees A.18.10(18/3)
- Is\_Root\_Directory\_Name
  - in* Ada.Directories.Hierarchical\_File\_Names A.16.1(5/3)
- Is\_Round\_Robin
  - in* Ada.Dispatching.Round\_Robin D.2.5(4/2)
- Is\_Simple\_Name
  - in* Ada.Directories.Hierarchical\_File\_Names A.16.1(4/3)
- Is\_Sorted
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(48/2)
  - in* Ada.Containers.Vectors A.18.2(76/2)
- Is\_Special *in* Ada.Characters.Handling A.3.2(4)
- Is\_String
  - in* Ada.Characters.Conversions A.3.4(3/2)
- Is\_Subset
  - in* Ada.Containers.Hashed\_Sets A.18.8(39/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(40/2)
  - in* Ada.Strings.Maps A.4.2(14)

- in* Ada.Strings.Wide\_Maps A.4.7(14)
- in* Ada.Strings.Wide\_Wide\_Maps A.4.8(14/2)
- Is\_Terminated
  - in* Ada.Task\_Identification C.7.1(4)
- Is\_Upper *in* Ada.Characters.Handling A.3.2(4)
- Is\_Wide\_Character
  - in* Ada.Characters.Conversions A.3.4(3/2)
- Is\_Wide\_String
  - in* Ada.Characters.Conversions A.3.4(3/2)
- Iterate
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(45/2)
  - in* Ada.Containers.Hashed\_Maps A.18.5(37/2)
  - in* Ada.Containers.Hashed\_Sets A.18.8(49/2)
  - in* Ada.Containers.Multiway\_Trees A.18.10(37/3)
  - in* Ada.Containers.Ordered\_Maps A.18.6(50/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(60/2)
  - in* Ada.Containers.Vectors A.18.2(73/2)
  - in* Ada.Environment\_Variables A.17(8/2)
- Iterate\_Children
  - in* Ada.Containers.Multiway\_Trees A.18.10(61/3)
- Iterate\_Subtree
  - in* Ada.Containers.Multiway\_Trees A.18.10(38/3)
- Key
  - in* Ada.Containers.Hashed\_Maps A.18.5(13/2)
  - in* Ada.Containers.Hashed\_Sets A.18.8(51/2)
  - in* Ada.Containers.Ordered\_Maps A.18.6(12/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(64/2)
- Kind *in* Ada.Directories A.16(25/2), A.16(40/2)
- Last
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(35/2)
  - in* Ada.Containers.Ordered\_Maps A.18.6(31/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(43/2)
  - in* Ada.Containers.Vectors A.18.2(61/2)
- Last\_Child
  - in* Ada.Containers.Multiway\_Trees A.18.10(55/3)
- Last\_Child\_Element
  - in* Ada.Containers.Multiway\_Trees A.18.10(56/3)
- Last\_Element
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(36/2)
  - in* Ada.Containers.Ordered\_Maps A.18.6(32/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(44/2)
  - in* Ada.Containers.Vectors A.18.2(62/2)
- Last\_Index *in* Ada.Containers.Vectors A.18.2(60/2)
- Last\_Key
  - in* Ada.Containers.Ordered\_Maps A.18.6(33/2)
- Length
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(11/2)
  - in* Ada.Containers.Hashed\_Maps A.18.5(10/2)
  - in* Ada.Containers.Hashed\_Sets A.18.8(12/2)
  - in* Ada.Containers.Ordered\_Maps A.18.6(9/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(11/2)
  - in* Ada.Containers.Vectors A.18.2(21/2)
  - in* Ada.Strings.Bounded A.4.4(9)
  - in* Ada.Strings.Unbounded A.4.5(6)
  - in* Ada.Text\_IO Editing F.3.3(11)
  - in* Interfaces.COBO B.4(34), B.4(39), B.4(44)
- Less\_Case\_Insensitive
  - child of* Ada.Strings A.4.10(13/3)
  - child of* Ada.Strings.Bounded A.4.10(18/3)
  - child of* Ada.Strings.Fixed A.4.10(16/3)
- child of* Ada.Strings.Unbounded A.4.10(21/3)
- Line *in* Ada.Text\_IO A.10.1(38)
- Line\_Length *in* Ada.Text\_IO A.10.1(25)
- Log
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(3)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(4)
- Look\_Ahead *in* Ada.Text\_IO A.10.1(43)
- Members
  - in* Ada.Execution\_Time.Group\_Budgets D.14.2(8/2)
- Merge
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(50/2)
  - in* Ada.Containers.Vectors A.18.2(78/2)
- Microseconds *in* Ada.Real\_Time D.8(14/2)
- Milliseconds *in* Ada.Real\_Time D.8(14/2)
- Minute *in* Ada.Calendar.Formatting 9.6.1(25/2)
- Minutes *in* Ada.Real\_Time D.8(14/2)
- Mode
  - in* Ada.Direct\_IO A.8.4(9)
  - in* Ada.Sequential\_IO A.8.1(9)
  - in* Ada.Streams.Stream\_IO A.12.1(11)
  - in* Ada.Text\_IO A.10.1(12)
- Modification\_Time *in* Ada.Directories A.16(27/2), A.16(42/2)
- Modulus
  - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(10/2), G.3.2(30/2)
  - in* Ada.Numerics.Generic\_Complex\_Types G.1.1(9)
- Month
  - in* Ada.Calendar 9.6(13)
  - in* Ada.Calendar.Formatting 9.6.1(22/2)
- More\_Entries *in* Ada.Directories A.16(34/2)
- Move
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(18/2)
  - in* Ada.Containers.Hashed\_Maps A.18.5(18/2)
  - in* Ada.Containers.Hashed\_Sets A.18.8(18/2)
  - in* Ada.Containers.Indefinite\_Holders A.18.18(18/3)
  - in* Ada.Containers.Multiway\_Trees A.18.10(28/3)
  - in* Ada.Containers.Ordered\_Maps A.18.6(17/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(17/2)
  - in* Ada.Containers.Vectors A.18.2(35/2)
  - in* Ada.Strings.Fixed A.4.3(7)
- Name
  - in* Ada.Direct\_IO A.8.4(9)
  - in* Ada.Sequential\_IO A.8.1(9)
  - in* Ada.Streams.Stream\_IO A.12.1(11)
  - in* Ada.Text\_IO A.10.1(12)
- Name\_Case\_Equivalence
  - in* Ada.Directories A.16(20.2/3)
- Nanoseconds *in* Ada.Real\_Time D.8(14/2)
- New\_Char\_Array
  - in* Interfaces.C.Strings B.3.1(9)
- New\_Line *in* Ada.Text\_IO A.10.1(28)
- New\_Page *in* Ada.Text\_IO A.10.1(31)
- New\_String *in* Interfaces.C.Strings B.3.1(10)
- Next
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(37/2), A.18.3(39/2)
  - in* Ada.Containers.Hashed\_Maps A.18.5(28/2), A.18.5(29/2)
  - in* Ada.Containers.Hashed\_Sets A.18.8(41/2), A.18.8(42/2)

- in* Ada.Containers.Ordered\_Maps A.18.6(34/2), A.18.6(35/2)
- in* Ada.Containers.Ordered\_Sets A.18.9(45/2), A.18.9(46/2)
- in* Ada.Containers.Vectors A.18.2(63/2), A.18.2(64/2)
- Next\_Sibling
  - in* Ada.Containers.Multiway\_Trees A.18.10(57/3), A.18.10(59/3)
- Node\_Count
  - in* Ada.Containers.Multiway\_Trees A.18.10(15/3)
- Null\_Task\_Id
  - in* Ada.Task\_Identification C.7.1(2/2)
- Open
  - in* Ada.Direct\_IO A.8.4(7)
  - in* Ada.Sequential\_IO A.8.1(7)
  - in* Ada.Streams.Stream\_IO A.12.1(9)
  - in* Ada.Text\_IO A.10.1(10)
- Overlap
  - in* Ada.Containers.Hashed\_Sets A.18.8(38/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(39/2)
- Overwrite
  - in* Ada.Strings.Bounded A.4.4(62), A.4.4(63)
  - in* Ada.Strings.Fixed A.4.3(27), A.4.3(28)
  - in* Ada.Strings.Unbounded A.4.5(57), A.4.5(58)
- Page *in* Ada.Text\_IO A.10.1(39)
- Page\_Length *in* Ada.Text\_IO A.10.1(26)
- Parent
  - in* Ada.Containers.Multiway\_Trees A.18.10(52/3)
- Parent\_Tag *in* Ada.Tags 3.9(7.2/2)
- Pic\_String *in* Ada.Text\_IO.Editing F.3.3(7)
- Prepend
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(22/2)
  - in* Ada.Containers.Vectors A.18.2(44/2), A.18.2(45/2)
- Prepend\_Child
  - in* Ada.Containers.Multiway\_Trees A.18.10(44/3)
- Previous
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(38/2), A.18.3(40/2)
  - in* Ada.Containers.Ordered\_Maps A.18.6(36/2), A.18.6(37/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(47/2), A.18.9(48/2)
  - in* Ada.Containers.Vectors A.18.2(65/2), A.18.2(66/2)
- Previous\_Sibling
  - in* Ada.Containers.Multiway\_Trees A.18.10(58/3), A.18.10(60/3)
- Put
  - in* Ada.Text\_IO A.10.1(42), A.10.1(48), A.10.1(55), A.10.1(60), A.10.1(66), A.10.1(67), A.10.1(71), A.10.1(72), A.10.1(76), A.10.1(77), A.10.1(82), A.10.1(83)
  - in* Ada.Text\_IO.Bounded\_IO A.10.11(4/2), A.10.11(5/2)
  - in* Ada.Text\_IO.Complex\_IO G.1.3(7), G.1.3(8)
  - in* Ada.Text\_IO.Editing F.3.3(14), F.3.3(15), F.3.3(16)
  - in* Ada.Text\_IO.Unbounded\_IO A.10.12(4/2), A.10.12(5/2)
- Put\_Line
  - in* Ada.Text\_IO A.10.1(50)
  - in* Ada.Text\_IO.Bounded\_IO A.10.11(6/2), A.10.11(7/2)
  - in* Ada.Text\_IO.Unbounded\_IO A.10.12(6/2), A.10.12(7/2)
- Query\_Element
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(16/2)
  - in* Ada.Containers.Hashed\_Maps A.18.5(16/2)
  - in* Ada.Containers.Hashed\_Sets A.18.8(17/2)
- in* Ada.Containers.Indefinite\_Holders A.18.18(14/3)
- in* Ada.Containers.Multiway\_Trees A.18.10(24/3)
- in* Ada.Containers.Ordered\_Maps A.18.6(15/2)
- in* Ada.Containers.Ordered\_Sets A.18.9(16/2)
- in* Ada.Containers.Vectors A.18.2(31/2), A.18.2(32/2)
- Raise\_Exception *in* Ada.Exceptions 11.4.1(4/2)
- Random
  - in* Ada.Numerics.Discrete\_Random A.5.2(20)
  - in* Ada.Numerics.Float\_Random A.5.2(8)
- Re
  - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(7/2), G.3.2(27/2)
  - in* Ada.Numerics.Generic\_Complex\_Types G.1.1(6)
- Read
  - in* Ada.Direct\_IO A.8.4(12)
  - in* Ada.Sequential\_IO A.8.1(12)
  - in* Ada.Storage\_IO A.9(6)
  - in* Ada.Streams 13.13.1(5)
  - in* Ada.Streams.Stream\_IO A.12.1(15), A.12.1(16)
  - in* System.RPC E.5(7)
- Reference
  - in* Ada.Interrupts C.3.2(10)
  - in* Ada.Task\_Attributes C.7.2(5)
- Reinitialize *in* Ada.Task\_Attributes C.7.2(6)
- Relative\_Name
  - in* Ada.Directories.Hierarchical\_File\_Names A.16.1(13/3)
- Remove\_Task
  - in* Ada.Execution\_Time.Group\_Budgets D.14.2(8/2)
- Rename *in* Ada.Directories A.16(12/2)
- Replace
  - in* Ada.Containers.Hashed\_Maps A.18.5(23/2)
  - in* Ada.Containers.Hashed\_Sets A.18.8(22/2), A.18.8(53/2)
  - in* Ada.Containers.Ordered\_Maps A.18.6(22/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(21/2), A.18.9(66/2)
- Replace\_Element
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(15/2)
  - in* Ada.Containers.Hashed\_Maps A.18.5(15/2)
  - in* Ada.Containers.Hashed\_Sets A.18.8(16/2)
  - in* Ada.Containers.Indefinite\_Holders A.18.18(13/3)
  - in* Ada.Containers.Multiway\_Trees A.18.10(23/3)
  - in* Ada.Containers.Ordered\_Maps A.18.6(14/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(15/2)
  - in* Ada.Containers.Vectors A.18.2(29/2), A.18.2(30/2)
  - in* Ada.Strings.Bounded A.4.4(27)
  - in* Ada.Strings.Unbounded A.4.5(21)
- Replace\_Slice
  - in* Ada.Strings.Bounded A.4.4(58), A.4.4(59)
  - in* Ada.Strings.Fixed A.4.3(23), A.4.3(24)
  - in* Ada.Strings.Unbounded A.4.5(53), A.4.5(54)
- Replenish
  - in* Ada.Execution\_Time.Group\_Budgets D.14.2(9/2)
- Replicate *in* Ada.Strings.Bounded A.4.4(78), A.4.4(79), A.4.4(80)
- Reraise\_Occurrence *in* Ada.Exceptions 11.4.1(4/2)
- Reserve\_Capacity
  - in* Ada.Containers.Hashed\_Maps A.18.5(9/2)
  - in* Ada.Containers.Hashed\_Sets A.18.8(11/2)
  - in* Ada.Containers.Vectors A.18.2(20/2)
- Reset
  - in* Ada.Direct\_IO A.8.4(8)

- in* Ada.Numerics.Discrete\_Random A.5.2(21), A.5.2(24)
- in* Ada.Numerics.Float\_Random A.5.2(9), A.5.2(12)
- in* Ada.Sequential\_IO A.8.1(8)
- in* Ada.Streams.Stream\_IO A.12.1(10)
- in* Ada.Text\_IO A.10.1(11)
- Reverse\_Elements
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(27/2)
  - in* Ada.Containers.Vectors A.18.2(54/2)
- Reverse\_Find
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(42/2)
  - in* Ada.Containers.Vectors A.18.2(70/2)
- Reverse\_Find\_Index
  - in* Ada.Containers.Vectors A.18.2(69/2)
- Reverse\_Iterate
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(46/2)
  - in* Ada.Containers.Ordered\_Maps A.18.6(51/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(61/2)
  - in* Ada.Containers.Vectors A.18.2(74/2)
- Reverse\_Iterate\_Children
  - in* Ada.Containers.Multiway\_Trees A.18.10(62/3)
- Root *in* Ada.Containers.Multiway\_Trees A.18.10(20/3)
- Save
  - in* Ada.Numerics.Discrete\_Random A.5.2(24)
  - in* Ada.Numerics.Float\_Random A.5.2(12)
- Save\_Occurrence *in* Ada.Exceptions 11.4.1(6/2)
- Second *in* Ada.Calendar.Formatting 9.6.1(26/2)
- Seconds
  - in* Ada.Calendar 9.6(13)
  - in* Ada.Real\_Time D.8(14/2)
- Seconds\_Of *in* Ada.Calendar.Formatting 9.6.1(28/2)
- Set *in* Ada.Environment\_Variables A.17(6/2)
- Set\_Bounded\_String
  - in* Ada.Strings.Bounded A.4.4(12.1/2)
- Set\_Col *in* Ada.Text\_IO A.10.1(35)
- Set\_Deadline *in* Ada.Dispatching.EDF D.2.6(9/2)
- Set\_Dependents\_Fallback\_Handler
  - in* Ada.Task\_Termination C.7.3(5/2)
- Set\_Directory *in* Ada.Directories A.16(6/2)
- Set\_Error *in* Ada.Text\_IO A.10.1(15)
- Set\_Exit\_Status *in* Ada.Command\_Line A.15(9)
- Set\_False
  - in* Ada.Synchronous\_Task\_Control D.10(4)
- Set\_Handler
  - in* Ada.Execution\_Time.Group\_Budgets D.14.2(10/2)
  - in* Ada.Execution\_Time.Timers D.14.1(7/2)
  - in* Ada.Real\_Time.Timing\_Events D.15(5/2)
- Set\_Im
  - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(8/2), G.3.2(28/2)
  - in* Ada.Numerics.Generic\_Complex\_Types G.1.1(7)
- Set\_Index
  - in* Ada.Direct\_IO A.8.4(14)
  - in* Ada.Streams.Stream\_IO A.12.1(22)
- Set\_Input *in* Ada.Text\_IO A.10.1(15)
- Set\_Length *in* Ada.Containers.Vectors A.18.2(22/2)
- Set\_Line *in* Ada.Text\_IO A.10.1(36)
- Set\_Line\_Length *in* Ada.Text\_IO A.10.1(23)
- Set\_Mode *in* Ada.Streams.Stream\_IO A.12.1(24)
- Set\_Output *in* Ada.Text\_IO A.10.1(15)
- Set\_Page\_Length *in* Ada.Text\_IO A.10.1(24)
- Set\_Priority
  - in* Ada.Dynamic\_Priorities D.5.1(4)
- Set\_Quantum
  - in* Ada.Dispatching.Round\_Robin D.2.5(4/2)
- Set\_Re
  - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(8/2), G.3.2(28/2)
  - in* Ada.Numerics.Generic\_Complex\_Types G.1.1(7)
- Set\_Specific\_Handler
  - in* Ada.Task\_Termination C.7.3(6/2)
- Set\_True
  - in* Ada.Synchronous\_Task\_Control D.10(4)
- Set\_Unbounded\_String
  - in* Ada.Strings.Unbounded A.4.5(11.1/2)
- Set\_Value *in* Ada.Task\_Attributes C.7.2(6)
- Simple\_Name
  - in* Ada.Directories A.16(16/2), A.16(38/2)
  - in* Ada.Directories.Hierarchical\_File\_Names A.16.1(10/3)
- Sin
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(4)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(5)
- Sinh
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(6)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(7)
- Size
  - in* Ada.Direct\_IO A.8.4(15)
  - in* Ada.Directories A.16(26/2), A.16(41/2)
  - in* Ada.Streams.Stream\_IO A.12.1(23)
- Skip\_Line *in* Ada.Text\_IO A.10.1(29)
- Skip\_Page *in* Ada.Text\_IO A.10.1(32)
- Slice
  - in* Ada.Strings.Bounded A.4.4(28)
  - in* Ada.Strings.Unbounded A.4.5(22)
- Solve
  - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(46/2)
  - in* Ada.Numerics.Generic\_Real\_Arrays G.3.1(24/2)
- Sort
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(49/2)
  - in* Ada.Containers.Vectors A.18.2(77/2)
- Specific\_Handler
  - in* Ada.Task\_Termination C.7.3(6/2)
- Splice
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(30/2), A.18.3(31/2), A.18.3(32/2)
- Splice\_Children
  - in* Ada.Containers.Multiway\_Trees A.18.10(50/3), A.18.10(51/3)
- Splice\_Subtree
  - in* Ada.Containers.Multiway\_Trees A.18.10(48/3), A.18.10(49/3)
- Split
  - in* Ada.Calendar 9.6(14)
  - in* Ada.Calendar.Formatting 9.6.1(29/2), 9.6.1(32/2), 9.6.1(33/2), 9.6.1(34/2)
  - in* Ada.Execution\_Time D.14(8/2)
  - in* Ada.Real\_Time D.8(16)

- Sqrt**  
*in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(3)  
*in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(4)
- Standard\_Error** *in* Ada.Text\_IO A.10.1(16), A.10.1(19)
- Standard\_Input** *in* Ada.Text\_IO A.10.1(16), A.10.1(19)
- Standard\_Output** *in* Ada.Text\_IO A.10.1(16), A.10.1(19)
- Start\_Search** *in* Ada.Directories A.16(32/2)
- Storage\_Size** *in* System.Storage\_Pools 13.11(9)
- Stream**  
*in* Ada.Streams.Stream\_IO A.12.1(13)  
*in* Ada.Text\_IO.Text\_Streams A.12.2(4)  
*in* Ada.Wide\_Text\_IO.Text\_Streams A.12.3(4)  
*in* Ada.Wide\_Wide\_Text\_IO.Text\_Streams A.12.4(4/2)
- Strlen** *in* Interfaces.C.Strings B.3.1(17)
- Sub\_Second** *in* Ada.Calendar.Formatting 9.6.1(27/2)
- Subtree\_Node\_Count**  
*in* Ada.Containers.Multiway\_Trees A.18.10(16/3)
- Suspend\_Until\_True**  
*in* Ada.Synchronous\_Task\_Control D.10(4)
- Suspend\_Until\_True\_And\_Set\_Deadline**  
*in* Ada.Synchronous\_Task\_Control.EDF D.10(5.2/3)
- Swap**  
*in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(28/2)  
*in* Ada.Containers.Multiway\_Trees A.18.10(31/3)  
*in* Ada.Containers.Vectors A.18.2(55/2), A.18.2(56/2)
- Swap\_Links**  
*in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(29/2)
- Symmetric\_Difference**  
*in* Ada.Containers.Hashed\_Sets A.18.8(35/2), A.18.8(36/2)  
*in* Ada.Containers.Ordered\_Sets A.18.9(36/2), A.18.9(37/2)
- Tail**  
*in* Ada.Strings.Bounded A.4.4(72), A.4.4(73)  
*in* Ada.Strings.Fixed A.4.3(37), A.4.3(38)  
*in* Ada.Strings.Unbounded A.4.5(67), A.4.5(68)
- Tan**  
*in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(4)  
*in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(5)
- Tanh**  
*in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(6)  
*in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(7)
- Time\_Of**  
*in* Ada.Calendar 9.6(15)  
*in* Ada.Calendar.Formatting 9.6.1(30/2), 9.6.1(31/2)  
*in* Ada.Execution\_Time D.14(9/2)  
*in* Ada.Real\_Time D.8(16)
- Time\_Of\_Event**  
*in* Ada.Real\_Time.Timing\_Events D.15(6/2)
- Time\_Remaining**  
*in* Ada.Execution\_Time.Timers D.14.1(8/2)
- To\_Ada**  
*in* Interfaces.C B.3(22), B.3(26), B.3(28), B.3(32), B.3(37), B.3(39), B.3(39.10/2), B.3(39.13/2), B.3(39.17/2), B.3(39.19/2), B.3(39.4/2), B.3(39.8/2)  
*in* Interfaces.COBOL B.4(17), B.4(19)  
*in* Interfaces.Fortran B.5(13), B.5(14), B.5(16)
- To\_Address**  
*in* System.Address\_To\_Access\_Conversions 13.7.2(3)  
*in* System.Storage\_Elements 13.7.1(10)
- To\_Basic** *in* Ada.Characters.Handling A.3.2(6), A.3.2(7)
- To\_Binary** *in* Interfaces.COBOL B.4(45), B.4(48)
- To\_Bounded\_String**  
*in* Ada.Strings.Bounded A.4.4(11)
- To\_C** *in* Interfaces.C B.3(21), B.3(25), B.3(27), B.3(32), B.3(36), B.3(38), B.3(39.13/2), B.3(39.16/2), B.3(39.18/2), B.3(39.4/2), B.3(39.7/2), B.3(39.9/2)
- To\_Character**  
*in* Ada.Characters.Conversions A.3.4(5/2)
- To\_Chars\_Ptr** *in* Interfaces.C.Strings B.3.1(8)
- To\_COBOL** *in* Interfaces.COBOL B.4(17), B.4(18)
- To\_Cursor** *in* Ada.Containers.Vectors A.18.2(25/2)
- To\_Decimal** *in* Interfaces.COBOL B.4(35), B.4(40), B.4(44), B.4(47)
- To\_Display** *in* Interfaces.COBOL B.4(36)
- To\_Domain**  
*in* Ada.Strings.Maps A.4.2(24)  
*in* Ada.Strings.Wide\_Maps A.4.7(24)  
*in* Ada.Strings.Wide\_Wide\_Maps A.4.8(24/2)
- To\_Duration** *in* Ada.Real\_Time D.8(13)
- To\_Fortran** *in* Interfaces.Fortran B.5(13), B.5(14), B.5(15)
- To\_Holder**  
*in* Ada.Containers.Indefinite\_Holders A.18.18(9/3)
- To\_Index** *in* Ada.Containers.Vectors A.18.2(26/2)
- To\_Integer** *in* System.Storage\_Elements 13.7.1(10)
- To\_ISO\_646** *in* Ada.Characters.Handling A.3.2(11), A.3.2(12)
- To\_Long\_Binary** *in* Interfaces.COBOL B.4(48)
- To\_Lower** *in* Ada.Characters.Handling A.3.2(6), A.3.2(7)
- To\_Mapping**  
*in* Ada.Strings.Maps A.4.2(23)  
*in* Ada.Strings.Wide\_Maps A.4.7(23)  
*in* Ada.Strings.Wide\_Wide\_Maps A.4.8(23/2)
- To\_Packed** *in* Interfaces.COBOL B.4(41)
- To\_Picture** *in* Ada.Text\_IO.Editing F.3.3(6)
- To\_Pointer**  
*in* System.Address\_To\_Access\_Conversions 13.7.2(3)
- To\_Range**  
*in* Ada.Strings.Maps A.4.2(24)  
*in* Ada.Strings.Wide\_Maps A.4.7(25)  
*in* Ada.Strings.Wide\_Wide\_Maps A.4.8(25/2)
- To\_Ranges**  
*in* Ada.Strings.Maps A.4.2(10)  
*in* Ada.Strings.Wide\_Maps A.4.7(10)  
*in* Ada.Strings.Wide\_Wide\_Maps A.4.8(10/2)
- To\_Sequence**  
*in* Ada.Strings.Maps A.4.2(19)  
*in* Ada.Strings.Wide\_Maps A.4.7(19)  
*in* Ada.Strings.Wide\_Wide\_Maps A.4.8(19/2)
- To\_Set**  
*in* Ada.Containers.Hashed\_Sets A.18.8(9/2)  
*in* Ada.Containers.Ordered\_Sets A.18.9(10/2)  
*in* Ada.Strings.Maps A.4.2(8), A.4.2(9), A.4.2(17), A.4.2(18)  
*in* Ada.Strings.Wide\_Maps A.4.7(8), A.4.7(9), A.4.7(17), A.4.7(18)  
*in* Ada.Strings.Wide\_Wide\_Maps A.4.8(8/2), A.4.8(9/2), A.4.8(17/2), A.4.8(18/2)

- To\_String
  - in* Ada.Characters.Conversions A.3.4(5/2)
  - in* Ada.Strings.Bounded A.4.4(12)
  - in* Ada.Strings.Unbounded A.4.5(11)
- To\_Time\_Span *in* Ada.Real\_Time D.8(13)
- To\_Unbounded\_String
  - in* Ada.Strings.Unbounded A.4.5(9), A.4.5(10)
- To\_Upper *in* Ada.Characters.Handling A.3.2(6), A.3.2(7)
- To\_Vector *in* Ada.Containers.Vectors A.18.2(13/2), A.18.2(14/2)
- To\_Wide\_Character
  - in* Ada.Characters.Conversions A.3.4(4/2), A.3.4(5/2)
- To\_Wide\_String
  - in* Ada.Characters.Conversions A.3.4(4/2), A.3.4(5/2)
- To\_Wide\_Wide\_Character
  - in* Ada.Characters.Conversions A.3.4(4/2)
- To\_Wide\_Wide\_String
  - in* Ada.Characters.Conversions A.3.4(4/2)
- Translate
  - in* Ada.Strings.Bounded A.4.4(53), A.4.4(54), A.4.4(55), A.4.4(56)
  - in* Ada.Strings.Fixed A.4.3(18), A.4.3(19), A.4.3(20), A.4.3(21)
  - in* Ada.Strings.Unbounded A.4.5(48), A.4.5(49), A.4.5(50), A.4.5(51)
- Transpose
  - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(34/2)
  - in* Ada.Numerics.Generic\_Real\_Arrays G.3.1(17/2)
- Trim
  - in* Ada.Strings.Bounded A.4.4(67), A.4.4(68), A.4.4(69)
  - in* Ada.Strings.Fixed A.4.3(31), A.4.3(32), A.4.3(33), A.4.3(34)
  - in* Ada.Strings.Unbounded A.4.5(61), A.4.5(62), A.4.5(63), A.4.5(64)
- Unbounded\_Slice
  - in* Ada.Strings.Unbounded A.4.5(22.1/2), A.4.5(22.2/2)
- Unchecked\_Conversion
  - child of* Ada 13.9(3)
- Unchecked\_Deallocation
  - child of* Ada 13.11.2(3)
- Union
  - in* Ada.Containers.Hash\_Sets A.18.8(26/2), A.18.8(27/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(27/2), A.18.9(28/2)
- Unit\_Matrix
  - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(51/2)
  - in* Ada.Numerics.Generic\_Real\_Arrays G.3.1(29/2)
- Unit\_Vector
  - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(24/2)
  - in* Ada.Numerics.Generic\_Real\_Arrays G.3.1(14/2)
- Update *in* Interfaces.C.Strings B.3.1(18), B.3.1(19)
- Update\_Element
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(17/2)
  - in* Ada.Containers.Hash\_Sets A.18.5(17/2)
  - in* Ada.Containers.Indefinite\_Holders A.18.18(15/3)
  - in* Ada.Containers.Multiway\_Trees A.18.10(25/3)
  - in* Ada.Containers.Ordered\_Maps A.18.6(16/2)
  - in* Ada.Containers.Vectors A.18.2(33/2), A.18.2(34/2)
- Update\_Element\_Preserving\_Key
  - in* Ada.Containers.Hash\_Sets A.18.8(58/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(73/2)
- Update\_Error *in* Interfaces.C.Strings B.3.1(20)
- UTC\_Time\_Offset
  - in* Ada.Calendar.Time\_Zones 9.6.1(6/2)
- Valid
  - in* Ada.Text\_IO.Editing F.3.3(5), F.3.3(12)
  - in* Interfaces.COBO\_L B.4(33), B.4(38), B.4(43)
- Value
  - in* Ada.Calendar.Formatting 9.6.1(36/2), 9.6.1(38/2)
  - in* Ada.Environment\_Variables A.17(4/2)
  - in* Ada.Numerics.Discrete\_Random A.5.2(26)
  - in* Ada.Numerics.Float\_Random A.5.2(14)
  - in* Ada.Strings.Maps A.4.2(21)
  - in* Ada.Strings.Wide\_Maps A.4.7(21)
  - in* Ada.Strings.Wide\_Wide\_Maps A.4.8(21/2)
  - in* Ada.Task\_Attributes C.7.2(4)
  - in* Interfaces.C.Pointers B.3.2(6), B.3.2(7)
  - in* Interfaces.C.Strings B.3.1(13), B.3.1(14), B.3.1(15), B.3.1(16)
- Virtual\_Length
  - in* Interfaces.C.Pointers B.3.2(13)
- Wide\_Hash
  - child of* Ada.Strings.Wide\_Bounded A.4.7(1/2)
  - child of* Ada.Strings.Wide\_Fixed A.4.7(1/2)
  - child of* Ada.Strings.Wide\_Unbounded A.4.7(1/2)
- Wide\_Exception\_Name *in* Ada.Exceptions 11.4.1(2/2), 11.4.1(5/2)
- Wide\_Expanded\_Name *in* Ada.Tags 3.9(7/2)
- Wide\_Wide\_Hash
  - child of* Ada.Strings.Wide\_Wide\_Bounded A.4.8(1/2)
  - child of* Ada.Strings.Wide\_Wide\_Fixed A.4.8(1/2)
  - child of* Ada.Strings.Wide\_Wide\_Unbounded A.4.8(1/2)
- Wide\_Wide\_Exception\_Name
  - in* Ada.Exceptions 11.4.1(2/2), 11.4.1(5/2)
- Wide\_Wide\_Expanded\_Name *in* Ada.Tags 3.9(7/2)
- Write
  - in* Ada.Direct\_IO A.8.4(13)
  - in* Ada.Sequential\_IO A.8.1(12)
  - in* Ada.Storage\_IO A.9(7)
  - in* Ada.Streams 13.13.1(6)
  - in* Ada.Streams.Stream\_IO A.12.1(18), A.12.1(19)
  - in* System.RPC E.5(8)
- Year
  - in* Ada.Calendar 9.6(13)
  - in* Ada.Calendar.Formatting 9.6.1(21/2)
- Yield *in* Ada.Dispatching D.2.1(1.3/3)
- Yield\_To\_Higher
  - in* Ada.Dispatching.Non\_Preemptive D.2.4(2.2/3)
- Yield\_To\_Same\_Or\_Higher
  - in* Ada.Dispatching.Non\_Preemptive D.2.4(2.2/3)

## Q.4 Language-Defined Exceptions

1/2 This clause lists all language-defined exceptions.

|                                                         |                                            |                                                 |
|---------------------------------------------------------|--------------------------------------------|-------------------------------------------------|
| Argument_Error                                          | <i>in</i> Ada.Numerics A.5(3/2)            | <i>in</i> Ada.Streams.Stream_IO A.12.1(26)      |
| Capacity_Error                                          | <i>in</i> Ada.Containers A.18.1(5.1/3)     | <i>in</i> Ada.Text_IO A.10.1(85)                |
| Communication_Error                                     | <i>in</i> System.RPC E.5(5)                | Name_Error                                      |
| Constraint_Error                                        | <i>in</i> Standard A.1(46)                 | <i>in</i> Ada.Direct_IO A.8.4(18)               |
| Conversion_Error                                        | <i>in</i> Interfaces.COBOL B.4(30)         | <i>in</i> Ada.Directories A.16(43/2)            |
| Data_Error                                              | <i>in</i> Ada.Direct_IO A.8.4(18)          | <i>in</i> Ada.IO_Exceptions A.13(4)             |
| <i>in</i> Ada.IO_Exceptions A.13(4)                     | <i>in</i> Ada.Sequential_IO A.8.1(15)      | <i>in</i> Ada.Sequential_IO A.8.1(15)           |
| <i>in</i> Ada.Storage_IO A.9(9)                         | <i>in</i> Ada.Streams.Stream_IO A.12.1(26) | <i>in</i> Ada.Streams.Stream_IO A.12.1(26)      |
| <i>in</i> Ada.Text_IO A.10.1(85)                        |                                            | <i>in</i> Ada.Text_IO A.10.1(85)                |
| Device_Error                                            |                                            | Pattern_Error                                   |
| <i>in</i> Ada.Direct_IO A.8.4(18)                       |                                            | <i>in</i> Ada.Strings A.4.1(5)                  |
| <i>in</i> Ada.Directories A.16(43/2)                    |                                            | Picture_Error                                   |
| <i>in</i> Ada.IO_Exceptions A.13(4)                     |                                            | <i>in</i> Ada.Text_IO.Editing F.3.3(9)          |
| <i>in</i> Ada.Sequential_IO A.8.1(15)                   |                                            | Pointer_Error                                   |
| <i>in</i> Ada.Streams.Stream_IO A.12.1(26)              |                                            | <i>in</i> Interfaces.C.Pointers B.3.2(8)        |
| <i>in</i> Ada.Text_IO A.10.1(85)                        |                                            | Program_Error                                   |
| Dispatching_Policy_Error                                |                                            | <i>in</i> Standard A.1(46)                      |
| <i>in</i> Ada.Dispatching D.2.1(1.4/3)                  |                                            | Status_Error                                    |
| Encoding_Error                                          |                                            | <i>in</i> Ada.Direct_IO A.8.4(18)               |
| <i>in</i> Ada.Strings.UTF_Encoding A.4.11(8/3)          |                                            | <i>in</i> Ada.Directories A.16(43/2)            |
| End_Error                                               |                                            | <i>in</i> Ada.IO_Exceptions A.13(4)             |
| <i>in</i> Ada.Direct_IO A.8.4(18)                       |                                            | <i>in</i> Ada.Sequential_IO A.8.1(15)           |
| <i>in</i> Ada.IO_Exceptions A.13(4)                     |                                            | <i>in</i> Ada.Streams.Stream_IO A.12.1(26)      |
| <i>in</i> Ada.Sequential_IO A.8.1(15)                   |                                            | <i>in</i> Ada.Text_IO A.10.1(85)                |
| <i>in</i> Ada.Streams.Stream_IO A.12.1(26)              |                                            | Storage_Error                                   |
| <i>in</i> Ada.Text_IO A.10.1(85)                        |                                            | <i>in</i> Standard A.1(46)                      |
| Group_Budget_Error                                      |                                            | Tag_Error                                       |
| <i>in</i> Ada.Execution_Time.Group_Budgets D.14.2(11/2) |                                            | <i>in</i> Ada.Tags 3.9(8)                       |
| Index_Error                                             |                                            | Tasking_Error                                   |
| <i>in</i> Ada.Strings A.4.1(5)                          |                                            | <i>in</i> Standard A.1(46)                      |
| Layout_Error                                            |                                            | Terminator_Error                                |
| <i>in</i> Ada.IO_Exceptions A.13(4)                     |                                            | <i>in</i> Interfaces.C B.3(40)                  |
| <i>in</i> Ada.Text_IO A.10.1(85)                        |                                            | Time_Error                                      |
| Length_Error                                            |                                            | <i>in</i> Ada.Calendar 9.6(18)                  |
| <i>in</i> Ada.Strings A.4.1(5)                          |                                            | Timer_Resource_Error                            |
| Mode_Error                                              |                                            | <i>in</i> Ada.Execution_Time.Timers D.14.1(9/2) |
| <i>in</i> Ada.Direct_IO A.8.4(18)                       |                                            | Translation_Error                               |
| <i>in</i> Ada.IO_Exceptions A.13(4)                     |                                            | <i>in</i> Ada.Strings A.4.1(5)                  |
| <i>in</i> Ada.Sequential_IO A.8.1(15)                   |                                            | Unknown_Zone_Error                              |
|                                                         |                                            | <i>in</i> Ada.Calendar.Time_Zones 9.6.1(5/2)    |
|                                                         |                                            | Use_Error                                       |
|                                                         |                                            | <i>in</i> Ada.Direct_IO A.8.4(18)               |
|                                                         |                                            | <i>in</i> Ada.Directories A.16(43/2)            |
|                                                         |                                            | <i>in</i> Ada.IO_Exceptions A.13(4)             |
|                                                         |                                            | <i>in</i> Ada.Sequential_IO A.8.1(15)           |
|                                                         |                                            | <i>in</i> Ada.Streams.Stream_IO A.12.1(26)      |
|                                                         |                                            | <i>in</i> Ada.Text_IO A.10.1(85)                |



## Q.5 Language-Defined Objects

This clause lists all language-defined constants, variables, named numbers, and enumeration literals.

1/2

ACK *in* Ada.Characters.Latin\_1 A.3.3(5)  
 Acute *in* Ada.Characters.Latin\_1 A.3.3(22)  
 Ada\_To\_COBOL *in* Interfaces.COBOL B.4(14)  
 Alphanumeric\_Set  
   *in* Ada.Strings.Maps.Constants A.4.6(4)  
 Ampersand *in* Ada.Characters.Latin\_1 A.3.3(8)  
 APC *in* Ada.Characters.Latin\_1 A.3.3(19)  
 Apostrophe *in* Ada.Characters.Latin\_1 A.3.3(8)  
 Asterisk *in* Ada.Characters.Latin\_1 A.3.3(8)  
 Basic\_Map  
   *in* Ada.Strings.Maps.Constants A.4.6(5)  
 Basic\_Set  
   *in* Ada.Strings.Maps.Constants A.4.6(4)  
 BEL *in* Ada.Characters.Latin\_1 A.3.3(5)  
 BOM\_16 *in* Ada.Strings.UTF\_Encoding A.4.11(12/3)  
 BOM\_16BE *in* Ada.Strings.UTF\_Encoding A.4.11(10/3)  
 BOM\_16LE *in* Ada.Strings.UTF\_Encoding A.4.11(11/3)  
 BOM\_8 *in* Ada.Strings.UTF\_Encoding A.4.11(9/3)  
 BPH *in* Ada.Characters.Latin\_1 A.3.3(17)  
 Broken\_Bar *in* Ada.Characters.Latin\_1 A.3.3(21/3)  
 BS *in* Ada.Characters.Latin\_1 A.3.3(5)  
 Buffer\_Size *in* Ada.Storage\_IO A.9(4)  
 CAN *in* Ada.Characters.Latin\_1 A.3.3(6)  
 CCH *in* Ada.Characters.Latin\_1 A.3.3(18)  
 Cedilla *in* Ada.Characters.Latin\_1 A.3.3(22)  
 Cent\_Sign *in* Ada.Characters.Latin\_1 A.3.3(21/3)  
 char16\_nul *in* Interfaces.C B.3(39/3/2)  
 char32\_nul *in* Interfaces.C B.3(39/12/2)  
 CHAR\_BIT *in* Interfaces.C B.3(6)  
 Character\_Set  
   *in* Ada.Strings.Wide\_Maps A.4.7(46/2)  
   *in* Ada.Strings.Wide\_Maps.Wide\_Constants A.4.8(48/2)  
 Circumflex *in* Ada.Characters.Latin\_1 A.3.3(12)  
 COBOL\_To\_Ada *in* Interfaces.COBOL B.4(15)  
 Colon *in* Ada.Characters.Latin\_1 A.3.3(10)  
 Comma *in* Ada.Characters.Latin\_1 A.3.3(8)  
 Commercial\_At  
   *in* Ada.Characters.Latin\_1 A.3.3(10)  
 Control\_Set  
   *in* Ada.Strings.Maps.Constants A.4.6(4)  
 Copyright\_Sign  
   *in* Ada.Characters.Latin\_1 A.3.3(21/3)  
 CPU\_Tick *in* Ada.Execution\_Time D.14(4/2)  
 CPU\_Time\_First *in* Ada.Execution\_Time D.14(4/2)  
 CPU\_Time\_Last *in* Ada.Execution\_Time D.14(4/2)  
 CPU\_Time\_Unit *in* Ada.Execution\_Time D.14(4/2)  
 CR *in* Ada.Characters.Latin\_1 A.3.3(5)  
 CSI *in* Ada.Characters.Latin\_1 A.3.3(19)  
 Currency\_Sign  
   *in* Ada.Characters.Latin\_1 A.3.3(21/3)  
 DC1 *in* Ada.Characters.Latin\_1 A.3.3(6)  
 DC2 *in* Ada.Characters.Latin\_1 A.3.3(6)  
 DC3 *in* Ada.Characters.Latin\_1 A.3.3(6)  
 DC4 *in* Ada.Characters.Latin\_1 A.3.3(6)  
 DCS *in* Ada.Characters.Latin\_1 A.3.3(18)  
 Decimal\_Digit\_Set  
   *in* Ada.Strings.Maps.Constants A.4.6(4)  
 Default\_Aft  
   *in* Ada.Text\_IO A.10.1(64), A.10.1(69), A.10.1(74)  
   *in* Ada.Text\_IO.Complex\_IO G.1.3(5)  
 Default\_Base *in* Ada.Text\_IO A.10.1(53), A.10.1(58)  
 Default\_Bit\_Order *in* System 13.7(15/2)  
 Default\_Currency  
   *in* Ada.Text\_IO Editing F.3.3(10)  
 Default\_Deadline  
   *in* Ada.Dispatching.EDF D.2.6(9/2)  
 Default\_Exp  
   *in* Ada.Text\_IO A.10.1(64), A.10.1(69), A.10.1(74)  
   *in* Ada.Text\_IO.Complex\_IO G.1.3(5)  
 Default\_Fill *in* Ada.Text\_IO Editing F.3.3(10)  
 Default\_Fore  
   *in* Ada.Text\_IO A.10.1(64), A.10.1(69), A.10.1(74)  
   *in* Ada.Text\_IO.Complex\_IO G.1.3(5)  
 Default\_Priority *in* System 13.7(17)  
 Default\_Quantum  
   *in* Ada.Dispatching.Round\_Robin D.2.5(4/2)  
 Default\_Radix\_Mark  
   *in* Ada.Text\_IO Editing F.3.3(10)  
 Default\_Separator  
   *in* Ada.Text\_IO Editing F.3.3(10)  
 Default\_Setting *in* Ada.Text\_IO A.10.1(80)  
 Default\_Width *in* Ada.Text\_IO A.10.1(53), A.10.1(58),  
   A.10.1(80)  
 Degree\_Sign *in* Ada.Characters.Latin\_1 A.3.3(22)  
 DEL *in* Ada.Characters.Latin\_1 A.3.3(14)  
 Diaeresis *in* Ada.Characters.Latin\_1 A.3.3(21/3)  
 Division\_Sign  
   *in* Ada.Characters.Latin\_1 A.3.3(26)  
 DLE *in* Ada.Characters.Latin\_1 A.3.3(6)  
 Dollar\_Sign *in* Ada.Characters.Latin\_1 A.3.3(8)  
 e *in* Ada.Numerics A.5(3/2)  
 EM *in* Ada.Characters.Latin\_1 A.3.3(6)  
 Empty\_Holder  
   *in* Ada.Containers.Indefinite\_Holders A.18.18(7/3)  
 Empty\_List  
   *in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(8/2)  
 Empty\_Map  
   *in* Ada.Containers.Hashed\_Maps A.18.5(5/2)  
   *in* Ada.Containers.Ordered\_Maps A.18.6(6/2)  
 Empty\_Set  
   *in* Ada.Containers.Hashed\_Sets A.18.8(5/2)  
   *in* Ada.Containers.Ordered\_Sets A.18.9(6/2)  
 Empty\_Tree  
   *in* Ada.Containers.Multiway\_Trees A.18.10(10/3)

- Empty\_Vector
  - in* Ada.Containers.Vectors A.18.2(10/2)
- ENQ *in* Ada.Characters.Latin\_1 A.3.3(5)
- EOT *in* Ada.Characters.Latin\_1 A.3.3(5)
- EPA *in* Ada.Characters.Latin\_1 A.3.3(18)
- Equals\_Sign *in* Ada.Characters.Latin\_1 A.3.3(10)
- ESA *in* Ada.Characters.Latin\_1 A.3.3(17)
- ESC *in* Ada.Characters.Latin\_1 A.3.3(6)
- ETB *in* Ada.Characters.Latin\_1 A.3.3(6)
- ETX *in* Ada.Characters.Latin\_1 A.3.3(5)
- Exclamation *in* Ada.Characters.Latin\_1 A.3.3(8)
- Failure *in* Ada.Command\_Line A.15(8)
- Feminine\_Ordinal\_Indicator
  - in* Ada.Characters.Latin\_1 A.3.3(21/3)
- FF *in* Ada.Characters.Latin\_1 A.3.3(5)
- Fine\_Delta *in* System 13.7(9)
- Fraction\_One\_Half
  - in* Ada.Characters.Latin\_1 A.3.3(22)
- Fraction\_One\_Quarter
  - in* Ada.Characters.Latin\_1 A.3.3(22)
- Fraction\_Three\_Quarters
  - in* Ada.Characters.Latin\_1 A.3.3(22)
- Friday *in* Ada.Calendar.Formatting 9.6.1(17/2)
- FS *in* Ada.Characters.Latin\_1 A.3.3(6)
- Full\_Stop *in* Ada.Characters.Latin\_1 A.3.3(8)
- Graphic\_Set
  - in* Ada.Strings.Maps.Constants A.4.6(4)
- Grave *in* Ada.Characters.Latin\_1 A.3.3(13)
- Greater\_Than\_Sign
  - in* Ada.Characters.Latin\_1 A.3.3(10)
- GS *in* Ada.Characters.Latin\_1 A.3.3(6)
- Hexadecimal\_Digit\_Set
  - in* Ada.Strings.Maps.Constants A.4.6(4)
- High\_Order\_First
  - in* Interfaces.COBOL B.4(25)
  - in* System 13.7(15/2)
- HT *in* Ada.Characters.Latin\_1 A.3.3(5)
- HTJ *in* Ada.Characters.Latin\_1 A.3.3(17)
- HTS *in* Ada.Characters.Latin\_1 A.3.3(17)
- Hyphen *in* Ada.Characters.Latin\_1 A.3.3(8)
- i
  - in* Ada.Numerics.Generic\_Complex\_Types G.1.1(5)
  - in* Interfaces.Fortran B.5(10)
- Identity
  - in* Ada.Strings.Maps A.4.2(22)
  - in* Ada.Strings.Wide\_Maps A.4.7(22)
  - in* Ada.Strings.Wide\_Wide\_Maps A.4.8(22/2)
- Inverted\_Exclamation
  - in* Ada.Characters.Latin\_1 A.3.3(21/3)
- Inverted\_Question
  - in* Ada.Characters.Latin\_1 A.3.3(22)
- IS1 *in* Ada.Characters.Latin\_1 A.3.3(16)
- IS2 *in* Ada.Characters.Latin\_1 A.3.3(16)
- IS3 *in* Ada.Characters.Latin\_1 A.3.3(16)
- IS4 *in* Ada.Characters.Latin\_1 A.3.3(16)
- ISO\_646\_Set
  - in* Ada.Strings.Maps.Constants A.4.6(4)
- j
  - in* Ada.Numerics.Generic\_Complex\_Types G.1.1(5)
  - in* Interfaces.Fortran B.5(10)
- LC\_A *in* Ada.Characters.Latin\_1 A.3.3(13)
- LC\_A\_Acute *in* Ada.Characters.Latin\_1 A.3.3(25)
- LC\_A\_Circumflex
  - in* Ada.Characters.Latin\_1 A.3.3(25)
- LC\_A\_Diaeresis
  - in* Ada.Characters.Latin\_1 A.3.3(25)
- LC\_A\_Grave *in* Ada.Characters.Latin\_1 A.3.3(25)
- LC\_A\_Ring *in* Ada.Characters.Latin\_1 A.3.3(25)
- LC\_A\_Tilde *in* Ada.Characters.Latin\_1 A.3.3(25)
- LC\_AE\_Diphthong
  - in* Ada.Characters.Latin\_1 A.3.3(25)
- LC\_B *in* Ada.Characters.Latin\_1 A.3.3(13)
- LC\_C *in* Ada.Characters.Latin\_1 A.3.3(13)
- LC\_C\_Cedilla
  - in* Ada.Characters.Latin\_1 A.3.3(25)
- LC\_D *in* Ada.Characters.Latin\_1 A.3.3(13)
- LC\_E *in* Ada.Characters.Latin\_1 A.3.3(13)
- LC\_E\_Acute *in* Ada.Characters.Latin\_1 A.3.3(25)
- LC\_E\_Circumflex
  - in* Ada.Characters.Latin\_1 A.3.3(25)
- LC\_E\_Diaeresis
  - in* Ada.Characters.Latin\_1 A.3.3(25)
- LC\_E\_Grave *in* Ada.Characters.Latin\_1 A.3.3(25)
- LC\_F *in* Ada.Characters.Latin\_1 A.3.3(13)
- LC\_G *in* Ada.Characters.Latin\_1 A.3.3(13)
- LC\_German\_Sharp\_S
  - in* Ada.Characters.Latin\_1 A.3.3(24)
- LC\_H *in* Ada.Characters.Latin\_1 A.3.3(13)
- LC\_I *in* Ada.Characters.Latin\_1 A.3.3(13)
- LC\_I\_Acute *in* Ada.Characters.Latin\_1 A.3.3(25)
- LC\_I\_Circumflex
  - in* Ada.Characters.Latin\_1 A.3.3(25)
- LC\_I\_Diaeresis
  - in* Ada.Characters.Latin\_1 A.3.3(25)
- LC\_I\_Grave *in* Ada.Characters.Latin\_1 A.3.3(25)
- LC\_Icelandic\_Eth
  - in* Ada.Characters.Latin\_1 A.3.3(26)
- LC\_Icelandic\_Thorn
  - in* Ada.Characters.Latin\_1 A.3.3(26)
- LC\_J *in* Ada.Characters.Latin\_1 A.3.3(13)
- LC\_K *in* Ada.Characters.Latin\_1 A.3.3(13)
- LC\_L *in* Ada.Characters.Latin\_1 A.3.3(13)
- LC\_M *in* Ada.Characters.Latin\_1 A.3.3(13)
- LC\_N *in* Ada.Characters.Latin\_1 A.3.3(13)
- LC\_N\_Tilde *in* Ada.Characters.Latin\_1 A.3.3(26)
- LC\_O *in* Ada.Characters.Latin\_1 A.3.3(13)
- LC\_O\_Acute *in* Ada.Characters.Latin\_1 A.3.3(26)
- LC\_O\_Circumflex
  - in* Ada.Characters.Latin\_1 A.3.3(26)
- LC\_O\_Diaeresis
  - in* Ada.Characters.Latin\_1 A.3.3(26)
- LC\_O\_Grave *in* Ada.Characters.Latin\_1 A.3.3(26)
- LC\_O\_Oblique\_Stroke
  - in* Ada.Characters.Latin\_1 A.3.3(26)
- LC\_O\_Tilde *in* Ada.Characters.Latin\_1 A.3.3(26)
- LC\_P *in* Ada.Characters.Latin\_1 A.3.3(14)
- LC\_Q *in* Ada.Characters.Latin\_1 A.3.3(14)
- LC\_R *in* Ada.Characters.Latin\_1 A.3.3(14)
- LC\_S *in* Ada.Characters.Latin\_1 A.3.3(14)
- LC\_T *in* Ada.Characters.Latin\_1 A.3.3(14)

LC\_U *in* Ada.Characters.Latin\_1 A.3.3(14)  
 LC\_U\_Acute *in* Ada.Characters.Latin\_1 A.3.3(26)  
 LC\_U\_Circumflex  
     *in* Ada.Characters.Latin\_1 A.3.3(26)  
 LC\_U\_Diaeresis  
     *in* Ada.Characters.Latin\_1 A.3.3(26)  
 LC\_U\_Grave *in* Ada.Characters.Latin\_1 A.3.3(26)  
 LC\_V *in* Ada.Characters.Latin\_1 A.3.3(14)  
 LC\_W *in* Ada.Characters.Latin\_1 A.3.3(14)  
 LC\_X *in* Ada.Characters.Latin\_1 A.3.3(14)  
 LC\_Y *in* Ada.Characters.Latin\_1 A.3.3(14)  
 LC\_Y\_Acute *in* Ada.Characters.Latin\_1 A.3.3(26)  
 LC\_Y\_Diaeresis  
     *in* Ada.Characters.Latin\_1 A.3.3(26)  
 LC\_Z *in* Ada.Characters.Latin\_1 A.3.3(14)  
 Leading\_Nonseparate  
     *in* Interfaces.COBOL B.4(23)  
 Leading\_Separate *in* Interfaces.COBOL B.4(23)  
 Left\_Angle\_Quotation  
     *in* Ada.Characters.Latin\_1 A.3.3(21/3)  
 Left\_Curly\_Bracket  
     *in* Ada.Characters.Latin\_1 A.3.3(14)  
 Left\_Parenthesis  
     *in* Ada.Characters.Latin\_1 A.3.3(8)  
 Left\_Square\_Bracket  
     *in* Ada.Characters.Latin\_1 A.3.3(12)  
 Less\_Than\_Sign  
     *in* Ada.Characters.Latin\_1 A.3.3(10)  
 Letter\_Set  
     *in* Ada.Strings.Maps.Constants A.4.6(4)  
 LF *in* Ada.Characters.Latin\_1 A.3.3(5)  
 Low\_Line *in* Ada.Characters.Latin\_1 A.3.3(12)  
 Low\_Order\_First  
     *in* Interfaces.COBOL B.4(25)  
     *in* System 13.7(15/2)  
 Lower\_Case\_Map  
     *in* Ada.Strings.Maps.Constants A.4.6(5)  
 Lower\_Set  
     *in* Ada.Strings.Maps.Constants A.4.6(4)  
 Macron *in* Ada.Characters.Latin\_1 A.3.3(21/3)  
 Masculine\_Ordinal\_Indicator  
     *in* Ada.Characters.Latin\_1 A.3.3(22)  
 Max\_Base\_Digits *in* System 13.7(8)  
 Max\_Binary\_Modulus *in* System 13.7(7)  
 Max\_Decimal\_Digits *in* Ada.Decimal F.2(5)  
 Max\_Delta *in* Ada.Decimal F.2(4)  
 Max\_Digits *in* System 13.7(8)  
 Max\_Digits\_Binary *in* Interfaces.COBOL B.4(11)  
 Max\_Digits\_Long\_Binary  
     *in* Interfaces.COBOL B.4(11)  
 Max\_Image\_Width  
     *in* Ada.Numerics.Discrete\_Random A.5.2(25)  
     *in* Ada.Numerics.Float\_Random A.5.2(13)  
 Max\_Int *in* System 13.7(6)  
 Max\_Length *in* Ada.Strings.Bounded A.4.4(5)  
 Max\_Mantissa *in* System 13.7(9)  
 Max\_Nonbinary\_Modulus *in* System 13.7(7)  
 Max\_Picture\_Length  
     *in* Ada.Text\_IO.Editing F.3.3(8)  
 Max\_Scale *in* Ada.Decimal F.2(3)  
 Memory\_Size *in* System 13.7(13)  
 Micro\_Sign *in* Ada.Characters.Latin\_1 A.3.3(22)  
 Middle\_Dot *in* Ada.Characters.Latin\_1 A.3.3(22)  
 Min\_Delta *in* Ada.Decimal F.2(4)  
 Min\_Handler\_Ceiling  
     *in* Ada.Execution\_Time.Group\_Budgets D.14.2(7/2)  
     *in* Ada.Execution\_Time.Timers D.14.1(6/2)  
 Min\_Int *in* System 13.7(6)  
 Min\_Scale *in* Ada.Decimal F.2(3)  
 Minus\_Sign *in* Ada.Characters.Latin\_1 A.3.3(8)  
 Monday *in* Ada.Calendar.Formatting 9.6.1(17/2)  
 Multiplication\_Sign  
     *in* Ada.Characters.Latin\_1 A.3.3(24)  
 MW *in* Ada.Characters.Latin\_1 A.3.3(18)  
 NAK *in* Ada.Characters.Latin\_1 A.3.3(6)  
 Native\_Binary *in* Interfaces.COBOL B.4(25)  
 NBH *in* Ada.Characters.Latin\_1 A.3.3(17)  
 NBSP *in* Ada.Characters.Latin\_1 A.3.3(21/3)  
 NEL *in* Ada.Characters.Latin\_1 A.3.3(17)  
 No\_Break\_Space  
     *in* Ada.Characters.Latin\_1 A.3.3(21/3)  
 No\_Element  
     *in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(9/2)  
     *in* Ada.Containers.Hashed\_Maps A.18.5(6/2)  
     *in* Ada.Containers.Hashed\_Sets A.18.8(6/2)  
     *in* Ada.Containers.Multiway\_Trees A.18.10(11/3)  
     *in* Ada.Containers.Ordered\_Maps A.18.6(7/2)  
     *in* Ada.Containers.Ordered\_Sets A.18.9(7/2)  
     *in* Ada.Containers.Vectors A.18.2(11/2)  
 No\_Index *in* Ada.Containers.Vectors A.18.2(7/2)  
 No\_Tag *in* Ada.Tags 3.9(6.1/2)  
 Not\_Sign *in* Ada.Characters.Latin\_1 A.3.3(21/3)  
 NUL  
     *in* Ada.Characters.Latin\_1 A.3.3(5)  
     *in* Interfaces.C B.3(20/1)  
 Null\_Address *in* System 13.7(12)  
 Null\_Bounded\_String  
     *in* Ada.Strings.Bounded A.4.4(7)  
 Null\_Id *in* Ada.Exceptions 11.4.1(2/2)  
 Null\_Occurrence *in* Ada.Exceptions 11.4.1(3/2)  
 Null\_Ptr *in* Interfaces.C.Strings B.3.1(7)  
 Null\_Set  
     *in* Ada.Strings.Maps A.4.2(5)  
     *in* Ada.Strings.Wide\_Maps A.4.7(5)  
     *in* Ada.Strings.Wide\_Wide\_Maps A.4.8(5/2)  
 Null\_Unbounded\_String  
     *in* Ada.Strings.Unbounded A.4.5(5)  
 Number\_Sign *in* Ada.Characters.Latin\_1 A.3.3(8)  
 OSC *in* Ada.Characters.Latin\_1 A.3.3(19)  
 Packed\_Signed *in* Interfaces.COBOL B.4(27)  
 Packed\_Unsigned *in* Interfaces.COBOL B.4(27)  
 Paragraph\_Sign  
     *in* Ada.Characters.Latin\_1 A.3.3(22)  
 Percent\_Sign  
     *in* Ada.Characters.Latin\_1 A.3.3(8)  
 Pi *in* Ada.Numerics A.5(3/2)  
 Pilcrow\_Sign  
     *in* Ada.Characters.Latin\_1 A.3.3(22)  
 PLD *in* Ada.Characters.Latin\_1 A.3.3(17)  
 PLU *in* Ada.Characters.Latin\_1 A.3.3(17)

|                                                                            |                                                                  |
|----------------------------------------------------------------------------|------------------------------------------------------------------|
| Plus_Minus_Sign<br><i>in</i> Ada.Characters.Latin_1 A.3.3(22)              | Sunday <i>in</i> Ada.Calendar.Formatting 9.6.1(17/2)             |
| Plus_Sign <i>in</i> Ada.Characters.Latin_1 A.3.3(8)                        | Superscript_One<br><i>in</i> Ada.Characters.Latin_1 A.3.3(22)    |
| PM <i>in</i> Ada.Characters.Latin_1 A.3.3(19)                              | Superscript_Three<br><i>in</i> Ada.Characters.Latin_1 A.3.3(22)  |
| Pound_Sign <i>in</i> Ada.Characters.Latin_1 A.3.3(21/3)                    | Superscript_Two<br><i>in</i> Ada.Characters.Latin_1 A.3.3(22)    |
| PU1 <i>in</i> Ada.Characters.Latin_1 A.3.3(18)                             | SYN <i>in</i> Ada.Characters.Latin_1 A.3.3(6)                    |
| PU2 <i>in</i> Ada.Characters.Latin_1 A.3.3(18)                             | System_Name <i>in</i> System 13.7(4)                             |
| Question <i>in</i> Ada.Characters.Latin_1 A.3.3(10)                        | Thursday <i>in</i> Ada.Calendar.Formatting 9.6.1(17/2)           |
| Quotation <i>in</i> Ada.Characters.Latin_1 A.3.3(8)                        | Tick<br><i>in</i> Ada.Real_Time D.8(6)                           |
| Registered_Trade_Mark_Sign<br><i>in</i> Ada.Characters.Latin_1 A.3.3(21/3) | <i>in</i> System 13.7(10)                                        |
| Reserved_128<br><i>in</i> Ada.Characters.Latin_1 A.3.3(17)                 | Tilde <i>in</i> Ada.Characters.Latin_1 A.3.3(14)                 |
| Reserved_129<br><i>in</i> Ada.Characters.Latin_1 A.3.3(17)                 | Time_First <i>in</i> Ada.Real_Time D.8(4)                        |
| Reserved_132<br><i>in</i> Ada.Characters.Latin_1 A.3.3(17)                 | Time_Last <i>in</i> Ada.Real_Time D.8(4)                         |
| Reserved_153<br><i>in</i> Ada.Characters.Latin_1 A.3.3(19)                 | Time_Span_First <i>in</i> Ada.Real_Time D.8(5)                   |
| Reverse_Solidus<br><i>in</i> Ada.Characters.Latin_1 A.3.3(12)              | Time_Span_Last <i>in</i> Ada.Real_Time D.8(5)                    |
| RI <i>in</i> Ada.Characters.Latin_1 A.3.3(17)                              | Time_Span_Unit <i>in</i> Ada.Real_Time D.8(5)                    |
| Right_Angle_Quotation<br><i>in</i> Ada.Characters.Latin_1 A.3.3(22)        | Time_Span_Zero <i>in</i> Ada.Real_Time D.8(5)                    |
| Right_Curly_Bracket<br><i>in</i> Ada.Characters.Latin_1 A.3.3(14)          | Time_Unit <i>in</i> Ada.Real_Time D.8(4)                         |
| Right_Parenthesis<br><i>in</i> Ada.Characters.Latin_1 A.3.3(8)             | Trailing_Nonseparate<br><i>in</i> Interfaces.COBOL B.4(23)       |
| Right_Square_Bracket<br><i>in</i> Ada.Characters.Latin_1 A.3.3(12)         | Trailing_Separate <i>in</i> Interfaces.COBOL B.4(23)             |
| Ring_Above <i>in</i> Ada.Characters.Latin_1 A.3.3(22)                      | Tuesday <i>in</i> Ada.Calendar.Formatting 9.6.1(17/2)            |
| RS <i>in</i> Ada.Characters.Latin_1 A.3.3(6)                               | UC_A_Acute <i>in</i> Ada.Characters.Latin_1 A.3.3(23)            |
| Saturday <i>in</i> Ada.Calendar.Formatting 9.6.1(17/2)                     | UC_A_Circumflex<br><i>in</i> Ada.Characters.Latin_1 A.3.3(23)    |
| SCHAR_MAX <i>in</i> Interfaces.C B.3(6)                                    | UC_A_Diaeresis<br><i>in</i> Ada.Characters.Latin_1 A.3.3(23)     |
| SCHAR_MIN <i>in</i> Interfaces.C B.3(6)                                    | UC_A_Grave <i>in</i> Ada.Characters.Latin_1 A.3.3(23)            |
| SCI <i>in</i> Ada.Characters.Latin_1 A.3.3(19)                             | UC_A_Ring <i>in</i> Ada.Characters.Latin_1 A.3.3(23)             |
| Section_Sign<br><i>in</i> Ada.Characters.Latin_1 A.3.3(21/3)               | UC_A_Tilde <i>in</i> Ada.Characters.Latin_1 A.3.3(23)            |
| Semicolon <i>in</i> Ada.Characters.Latin_1 A.3.3(10)                       | UC_AE_Diphthong<br><i>in</i> Ada.Characters.Latin_1 A.3.3(23)    |
| SI <i>in</i> Ada.Characters.Latin_1 A.3.3(5)                               | UC_C_Cedilla<br><i>in</i> Ada.Characters.Latin_1 A.3.3(23)       |
| SO <i>in</i> Ada.Characters.Latin_1 A.3.3(5)                               | UC_E_Acute <i>in</i> Ada.Characters.Latin_1 A.3.3(23)            |
| Soft_Hyphen <i>in</i> Ada.Characters.Latin_1 A.3.3(21/3)                   | UC_E_Circumflex<br><i>in</i> Ada.Characters.Latin_1 A.3.3(23)    |
| SOH <i>in</i> Ada.Characters.Latin_1 A.3.3(5)                              | UC_E_Diaeresis<br><i>in</i> Ada.Characters.Latin_1 A.3.3(23)     |
| Solidus <i>in</i> Ada.Characters.Latin_1 A.3.3(8)                          | UC_E_Grave <i>in</i> Ada.Characters.Latin_1 A.3.3(23)            |
| SOS <i>in</i> Ada.Characters.Latin_1 A.3.3(19)                             | UC_I_Acute <i>in</i> Ada.Characters.Latin_1 A.3.3(23)            |
| SPA <i>in</i> Ada.Characters.Latin_1 A.3.3(18)                             | UC_I_Circumflex<br><i>in</i> Ada.Characters.Latin_1 A.3.3(23)    |
| Space<br><i>in</i> Ada.Characters.Latin_1 A.3.3(8)                         | UC_I_Diaeresis<br><i>in</i> Ada.Characters.Latin_1 A.3.3(23)     |
| <i>in</i> Ada.Strings A.4.1(4/2)                                           | UC_I_Grave <i>in</i> Ada.Characters.Latin_1 A.3.3(23)            |
| Special_Set<br><i>in</i> Ada.Strings.Maps.Constants A.4.6(4)               | UC_Icelandic_Eth<br><i>in</i> Ada.Characters.Latin_1 A.3.3(24)   |
| SS2 <i>in</i> Ada.Characters.Latin_1 A.3.3(17)                             | UC_Icelandic_Thorn<br><i>in</i> Ada.Characters.Latin_1 A.3.3(24) |
| SS3 <i>in</i> Ada.Characters.Latin_1 A.3.3(17)                             | UC_N_Tilde <i>in</i> Ada.Characters.Latin_1 A.3.3(24)            |
| SSA <i>in</i> Ada.Characters.Latin_1 A.3.3(17)                             | UC_O_Acute <i>in</i> Ada.Characters.Latin_1 A.3.3(24)            |
| ST <i>in</i> Ada.Characters.Latin_1 A.3.3(19)                              | UC_O_Circumflex<br><i>in</i> Ada.Characters.Latin_1 A.3.3(24)    |
| Storage_Unit <i>in</i> System 13.7(13)                                     | UC_O_Diaeresis<br><i>in</i> Ada.Characters.Latin_1 A.3.3(24)     |
| STS <i>in</i> Ada.Characters.Latin_1 A.3.3(18)                             | UC_O_Grave <i>in</i> Ada.Characters.Latin_1 A.3.3(24)            |
| STX <i>in</i> Ada.Characters.Latin_1 A.3.3(5)                              |                                                                  |
| SUB <i>in</i> Ada.Characters.Latin_1 A.3.3(6)                              |                                                                  |
| Success <i>in</i> Ada.Command_Line A.15(8)                                 |                                                                  |

UC\_O\_Oblique\_Stroke  
     *in* Ada.Characters.Latin\_1 A.3.3(24)  
 UC\_O\_Tilde *in* Ada.Characters.Latin\_1 A.3.3(24)  
 UC\_U\_Acute *in* Ada.Characters.Latin\_1 A.3.3(24)  
 UC\_U\_Circumflex  
     *in* Ada.Characters.Latin\_1 A.3.3(24)  
 UC\_U\_Diaeresis  
     *in* Ada.Characters.Latin\_1 A.3.3(24)  
 UC\_U\_Grave *in* Ada.Characters.Latin\_1 A.3.3(24)  
 UC\_Y\_Acute *in* Ada.Characters.Latin\_1 A.3.3(24)  
 UCHAR\_MAX *in* Interfaces.C B.3(6)  
 Unbounded *in* Ada.Text\_IO A.10.1(5)  
 Unsigned *in* Interfaces.COBOL B.4(23)  
 Upper\_Case\_Map  
     *in* Ada.Strings.Maps.Constants A.4.6(5)

Upper\_Set  
     *in* Ada.Strings.Maps.Constants A.4.6(4)  
 US *in* Ada.Characters.Latin\_1 A.3.3(6)  
 Vertical\_Line  
     *in* Ada.Characters.Latin\_1 A.3.3(14)  
 VT *in* Ada.Characters.Latin\_1 A.3.3(5)  
 VTS *in* Ada.Characters.Latin\_1 A.3.3(17)  
 Wednesday *in* Ada.Calendar.Formatting 9.6.1(17/2)  
 Wide\_Character\_Set  
     *in* Ada.Strings.Wide\_Maps.Wide\_Constants A.4.8(48/2)  
 wide\_nul *in* Interfaces.C B.3(31/1)  
 Wide\_Space *in* Ada.Strings A.4.1(4/2)  
 Wide\_Wide\_Space *in* Ada.Strings A.4.1(4/2)  
 Word\_Size *in* System 13.7(13)  
 Yen\_Sign *in* Ada.Characters.Latin\_1 A.3.3(21/3)



# Index

Index entries are given by paragraph number.

- & operator 4.4(1), 4.5.3(3)
- \* operator 4.4(1), 4.5.5(1)
- \*\* operator 4.4(1), 4.5.6(7)
- + operator 4.4(1), 4.5.3(1), 4.5.4(1)
- operator 4.4(1), 4.5.3(1), 4.5.4(1)
- / operator 4.4(1), 4.5.5(1)
- /= operator 4.4(1), 4.5.2(1), 6.6(6/3)
- 10646:2003, ISO/IEC standard 1.2(8/2)
- 14882:2003, ISO/IEC standard 1.2(9/2)
- 1539-1:2004, ISO/IEC standard 1.2(3/2)
- 19769:2004, ISO/IEC technical report 1.2(10/2)
- 1989:2002, ISO standard 1.2(4/2)
- 6429:1992, ISO/IEC standard 1.2(5)
- 646:1991, ISO/IEC standard 1.2(2)
- 8859-1:1987, ISO/IEC standard 1.2(6)
- 9899:1999, ISO/IEC standard 1.2(7/2)
- < operator 4.4(1), 4.5.2(1)
- <= operator 4.4(1), 4.5.2(1)
- = operator 4.4(1), 4.5.2(1)
- > operator 4.4(1), 4.5.2(1)
- >= operator 4.4(1), 4.5.2(1)
- A**
- AARM 0.3(5/2)
- abnormal completion 7.6.1(2/2)
- abnormal state of an object 13.9.1(4)
  - [*partial*] 9.8(21), 11.6(6), A.13(17)
- abnormal task 9.8(4)
- abort
  - of a partition E.1(7)
  - of a task 9.8(4)
  - of the execution of a construct 9.8(5)
- abort completion point 9.8(15)
- abort-deferred operation 9.8(5)
- abort\_statement 9.8(2)
  - used* 5.1(4/2), P
- Abort\_Task
  - in* Ada.Task\_Identification C.7.1(3/1)
- abortable\_part 9.7.4(5)
  - used* 9.7.4(2), P
- abs operator 4.4(1), 4.5.6(1)
- absolute value 4.4(1), 4.5.6(1)
- abstract data type (ADT)
  - See* private types and private extensions 7.3(1)
  - See also* abstract type 3.9.3(1/2)
- abstract subprogram 3.9.3(1/2), 3.9.3(3/2)
- abstract type 3.9.3(1.2/2), 3.9.3(1/2), N(1.1/2)
- abstract\_subprogram\_declaration
  - 3.9.3(1.1/2)
  - used* 3.1(3/2), P
- accept\_alternative 9.7.1(5)
  - used* 9.7.1(4), P
- accept\_statement 9.5.2(3)
  - used* 5.1(5/2), 9.7.1(5), P
- acceptable interpretation 8.6(14)
- Access attribute 3.10.2(24/1), 3.10.2(32/2)
  - See also* Unchecked\_Access attribute 13.10(3)
- access discriminant 3.7(9/2)
- access parameter 6.1(24/2)
- access paths
  - distinct 6.2(12)
- access result type 6.1(24/2)
- access type 3.10(1), N(2)
- access types
  - input-output unspecified A.7(6)
- access value 3.10(1)
- access-to-constant type 3.10(10)
- access-to-object type 3.10(7/1)
- access-to-subprogram type 3.10(7/1), 3.10(11)
- access-to-variable type 3.10(10)
- Access\_Check 11.5(11/2)
  - [*partial*] 4.1(13), 4.6(51/2)
- access\_definition 3.10(6/2)
  - used* 3.3.1(2/2), 3.6(7/2), 3.7(5/2), 6.1(13/2), 6.1(15/2), 6.5(2.2/2), 8.5.1(2/2), 12.4(2/3), P
- access\_to\_object\_definition 3.10(3)
  - used* 3.10(2/2), P
- access\_to\_subprogram\_definition 3.10(5)
  - used* 3.10(2/2), P
- access\_type\_definition 3.10(2/2)
  - used* 3.2.1(4/2), 12.5.4(2), P
- accessibility
  - from shared passive library units E.2.1(8)
- accessibility level 3.10.2(3/2)
- accessibility rule
  - Access attribute 3.10.2(28/3), 3.10.2(32/2)
  - request statement 9.5.4(6/3)
  - type conversion 4.6(24.17/3), 4.6(24.21/2)
  - type conversion, array components 4.6(24.6/2)
- Accessibility\_Check 11.5(19.1/2)
  - [*partial*] 3.10.2(29), 4.6(39.1/2), 4.6(48/3), 4.8(10.1/3), 6.5(8/3), 6.5(21/2), E.4(18/1)
- accessible partition E.1(7)
- accuracy 4.6(32), G.2(1)
- ACK
  - in* Ada.Characters.Latin\_1 A.3.3(5)
- acquire
  - execution resource associated with protected object 9.5.1(5)
- activation
  - of a task 9.2(1)
- activation failure 9.2(1)
- activator
  - of a task 9.2(5)
- active partition 10.2(28), E.1(2)
- active priority D.1(15)
- actual 12.3(7/3)
- actual duration D.9(12)
- actual parameter
  - for a formal parameter 6.4.1(3)
- actual subtype 3.3(23), 12.5(4)
  - of an object 3.3.1(9/2)
- actual type 12.5(4)
- actual\_parameter\_part 6.4(4)
  - used* 6.4(2), 6.4(3), 9.5.3(2), P
- Actual\_Quantum
  - in* Ada.Dispatching.Round\_Robin D.2.5(4/2)
- Acute
  - in* Ada.Characters.Latin\_1 A.3.3(22)
- Ada A.2(2)
- Ada calling convention 6.3.1(3)
- Ada.Assertions 11.4.2(12/2)
- Ada.Asynchronous\_Task\_Control D.11(3/2)
- Ada.Calendar 9.6(10)
- Ada.Calendar.Arithmetic 9.6.1(8/2)
- Ada.Calendar.Formatting 9.6.1(15/2)

|                                               |                |                                                   |               |                                         |              |
|-----------------------------------------------|----------------|---------------------------------------------------|---------------|-----------------------------------------|--------------|
| Ada.Calendar.Time_Zones                       | 9.6.1(2/2)     | Ada.Execution_Time.Timers                         | D.14.1(3/2)   | Ada.Strings.Fixed.Less_Case_Insensitive |              |
| Ada.Characters                                | A.3.1(2)       | Ada.Finalization                                  | 7.6(4/1)      | Ada.Strings.Fixed.Less_Case_Insensitive | A.4.10(16/3) |
| Ada.Characters.Conversions                    | A.3.4(2/2)     | Ada.Float_Text_IO                                 | A.10.9(33)    | Ada.Strings.Hash                        | A.4.9(2/2)   |
| Ada.Characters.Handling                       | A.3.2(2/2)     | Ada.Float_Wide_Text_IO                            | A.11(2/2)     | Ada.Strings.Hash_Case_Insensitive       | A.4.9(13/3)  |
| Ada.Characters.Latin_1                        | A.3.3(3)       | Ada.Float_Wide_Wide_Text_IO                       |               | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Command_Line                              | A.15(3)        |                                                   | A.11(3/2)     | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Complex_Text_IO                           | G.1.3(9.1/2)   | Ada.Integer_Text_IO                               | A.10.8(21)    | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers                                | A.18.1(3/2)    | Ada.Integer_Wide_Text_IO                          | A.11(2/2)     | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Doubly_Linked_Lists            |                | Ada.Integer_Wide_Wide_Text_IO                     |               | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Doubly_Linked_Lists            | A.18.3(5/3)    |                                                   | A.11(3/2)     | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Generic_Array_Sort             |                | Ada.Interrupts                                    | C.3.2(2)      | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Generic_Array_Sort             | A.18.26(3/2)   | Ada.Interrupts.Names                              | C.3.2(12)     | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Generic_Constrained_Array_Sort | A.18.26(7/2)   | Ada.IO_Exceptions                                 | A.13(3)       | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Generic_Sort                   |                | Ada.Numerics                                      | A.5(3/2)      | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Generic_Sort                   | A.18.26(9.2/3) | Ada.Numerics.Complex_Arrays                       |               | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Hashed_Maps                    |                | Ada.Numerics.Complex_Arrays                       | G.3.2(53/2)   | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Hashed_Maps                    | A.18.5(2/3)    | Ada.Numerics.Complex_Elementary_Functions         | G.1.2(9/1)    | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Hashed_Sets                    | A.18.8(2/3)    | Ada.Numerics.Complex_Elementary_Functions         | G.1.2(9/1)    | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Hashed_Sets                    | A.18.8(2/3)    | Ada.Numerics.Complex_Types                        |               | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Indefinite_Doubly_Linked_Lists | A.18.12(2/3)   | Ada.Numerics.Complex_Types                        | G.1.1(25/1)   | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Indefinite_Hashed_Maps         |                | Ada.Numerics.Discrete_Random                      |               | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Indefinite_Hashed_Maps         | A.18.13(2/3)   | Ada.Numerics.Discrete_Random                      | A.5.2(17)     | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Indefinite_Hashed_Sets         |                | Ada.Numerics.Elementary_Functions                 |               | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Indefinite_Hashed_Sets         | A.18.15(2/3)   | Ada.Numerics.Elementary_Functions                 | A.5.1(9/1)    | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Indefinite_Holders             |                | Ada.Numerics.Float_Random                         | A.5.2(5)      | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Indefinite_Holders             | A.18.18(5/3)   | Ada.Numerics.Generic_Complex_Arrays               |               | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Indefinite_Holders             | A.18.18(5/3)   | Ada.Numerics.Generic_Complex_Arrays               | G.3.2(2/2)    | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Indefinite_Multiway_Trees      |                | Ada.Numerics.Generic_Complex_Elementary_Functions | G.1.2(2/2)    | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Indefinite_Multiway_Trees      | A.18.17(2/3)   | Ada.Numerics.Generic_Complex_Elementary_Functions | G.1.2(2/2)    | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Indefinite_Ordered_Maps        |                | Ada.Numerics.Generic_Complex_Types                |               | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Indefinite_Ordered_Maps        | A.18.14(2/3)   | Ada.Numerics.Generic_Complex_Types                | G.1.1(2/1)    | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Indefinite_Ordered_Sets        |                | Ada.Numerics.Generic_Elementary_Functions         | A.5.1(3)      | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Indefinite_Ordered_Sets        | A.18.16(2/3)   | Ada.Numerics.Generic_Elementary_Functions         | A.5.1(3)      | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Indefinite_Vectors             |                | Ada.Numerics.Generic_Real_Arrays                  |               | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Indefinite_Vectors             | A.18.11(2/3)   | Ada.Numerics.Generic_Real_Arrays                  | G.3.1(2/2)    | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Multiway_Trees                 |                | Ada.Numerics.Real_Arrays                          | G.3.1(31/2)   | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Multiway_Trees                 | A.18.10(7/3)   | Ada.Real_Time                                     | D.8(3)        | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Ordered_Maps                   |                | Ada.Real_Time.Timing_Events                       |               | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Ordered_Maps                   | A.18.6(2/3)    | Ada.Real_Time.Timing_Events                       | D.15(3/2)     | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Ordered_Sets                   |                | Ada.Sequential_IO                                 | A.8.1(2)      | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Ordered_Sets                   | A.18.9(2/3)    | Ada.Storage_IO                                    | A.9(3)        | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Containers.Vectors                        | A.18.2(6/3)    | Ada.Streams                                       | 13.13.1(2)    | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Decimal                                   | F.2(2)         | Ada.Streams.Stream_IO                             | A.12.1(3)     | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Direct_IO                                 | A.8.4(2)       | Ada.Strings                                       | A.4.1(3)      | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Directories                               | A.16(3/2)      | Ada.Strings.Bounded                               | A.4.4(3)      | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Directories.Hierarchical_File_Names       |                | Ada.Strings.Bounded.Equal_Case_Insensitive        |               | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Directories.Hierarchical_File_Names       | A.16.1(3/3)    | Ada.Strings.Bounded.Equal_Case_Insensitive        | A.4.10(7/3)   | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Directories.Information                   | A.16(124/2)    | Ada.Strings.Bounded.Hash                          | A.4.9(7/2)    | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Dispatching                               | D.2.1(1.2/3)   | Ada.Strings.Bounded.Hash_Case_Insensitive         |               | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Dispatching.EDF                           | D.2.6(9/2)     | Ada.Strings.Bounded.Hash_Case_Insensitive         | A.4.9(14.4/3) | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Dispatching.Non_Preemptive                |                | Ada.Strings.Bounded.Less_Case_Insensitive         |               | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Dispatching.Non_Preemptive                | D.2.4(2.2/3)   | Ada.Strings.Bounded.Less_Case_Insensitive         | A.4.10(18/3)  | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Dispatching.Round_Robin                   |                | Ada.Strings.Bounded.Less_Case_Insensitive         |               | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Dispatching.Round_Robin                   | D.2.5(4/2)     | Ada.Strings.Bounded.Less_Case_Insensitive         | A.4.10(2/3)   | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Dynamic_Priorities                        | D.5.1(3/2)     | Ada.Strings.Fixed                                 | A.4.3(5)      | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Environment_Variables                     | A.17(3/2)      | Ada.Strings.Fixed.Equal_Case_Insensitive          |               | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Exceptions                                | 11.4.1(2/2)    | Ada.Strings.Fixed.Equal_Case_Insensitive          | A.4.10(5/3)   | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Execution_Time                            | D.14(3/2)      | Ada.Strings.Fixed.Hash_Case_Insensitive           |               | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Execution_Time.Group_Budgets              |                | Ada.Strings.Fixed.Hash_Case_Insensitive           | A.4.9(14.2/3) | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |
| Ada.Execution_Time.Group_Budgets              | D.14.2(3/2)    | Ada.Strings.Fixed.Hash_Case_Insensitive           | A.4.9(14.2/3) | Ada.Strings.Less_Case_Insensitive       | A.4.10(13/3) |



- Ada.Synchronous\_Task\_Control.EDF D.10(5.2/3)
- Ada.Tags 3.9(6/2)
- Ada.Tags.Generic\_Dispatching\_-Constructor 3.9(18.2/2)
- Ada.Task\_Attributes C.7.2(2)
- Ada.Task\_Identification C.7.1(2/2)
- Ada.Task\_Termination C.7.3(2/2)
- Ada.Text\_IO A.10.1(2)
- Ada.Text\_IO.Bounded\_IO A.10.11(3/2)
- Ada.Text\_IO.Complex\_IO G.1.3(3)
- Ada.Text\_IO.Editing F.3.3(3)
- Ada.Text\_IO.Text\_Streams A.12.2(3)
- Ada.Text\_IO.Unbounded\_IO A.10.12(3/2)
- Ada.Unchecked\_Conversion 13.9(3)
- Ada.Unchecked\_Deallocation 13.11.2(3)
- Ada.Wide\_Characters A.3.1(4/2)
- Ada.Wide\_Text\_IO A.11(2/2)
- Ada.Wide\_Text\_IO.Bounded\_IO A.11(4/3)
- Ada.Wide\_Text\_IO.Complex\_IO G.1.4(1)
- Ada.Wide\_Text\_IO.Editing F.3.4(1)
- Ada.Wide\_Text\_IO.Text\_Streams A.12.3(3)
- Ada.Wide\_Text\_IO.Unbounded\_IO A.11(5/3)
- Ada.Wide\_Wide\_Characters A.3.1(6/2)
- Ada.Wide\_Wide\_Text\_IO A.11(3/2)
- Ada.Wide\_Wide\_Text\_IO.Bounded\_IO A.11(4/3)
- Ada.Wide\_Wide\_Text\_IO.Complex\_IO G.1.5(1/2)
- Ada.Wide\_Wide\_Text\_IO.Editing F.3.5(1/2)
- Ada.Wide\_Wide\_Text\_IO.Text\_Streams A.12.4(3/2)
- Ada.Wide\_Wide\_-Text\_IO.Unbounded\_IO A.11(5/3)
- Ada\_To\_COBOL *in* Interfaces.COBOL B.4(14)
- adafinal B.1(39)
- adainit B.1(39)
- Add *in* Ada.Execution\_Time.Group\_Budgets D.14.2(9/2)
- Add\_Task *in* Ada.Execution\_Time.Group\_Budgets D.14.2(8/2)
- address
  - arithmetic 13.7.1(6)
  - comparison 13.7(14)
  - in* System 13.7(12)
- Address attribute 13.3(11), J.7.1(5)
- Address clause 13.3(7/2), 13.3(12)
- Address\_To\_Access\_Conversions *child of* System 13.7.2(2)
- Adjacent attribute A.5.3(48)
- Adjust 7.6(2)
  - in* Ada.Finalization 7.6(6/2)
- adjusting the value of an object 7.6(15), 7.6(16/3)
- adjustment 7.6(15), 7.6(16/3)
  - as part of assignment 5.2(14)
- ADT (abstract data type)
  - See* private types and private extensions 7.3(1)
  - See also* abstract type 3.9.3(1/2)
- advice 1.1.2(37)
- Aft attribute 3.5.10(5)
- aggregate 4.3(1), 4.3(2)
  - used* 4.4(7/3), 4.7(2), P
  - See also* composite type 3.2(2/2)
- aliasing 3.10(9/3), N(3)
- aliasing
  - See* distinct access paths 6.2(12)
- Alignment
  - in* Ada.Strings A.4.1(6)
- Alignment attribute 13.3(23/2), 13.3(26.2/2)
- Alignment clause 13.3(7/2), 13.3(25/2), 13.3(26.4/2)
- All\_Calls\_Remote pragma E.2.3(5), L(2)
- All\_Checks 11.5(25)
- Allocate
  - in* System.Storage\_Pools 13.11(7)
- Allocation\_Check 11.5(19.2/2)
  - [*partial*] 4.8(10.2/2), 4.8(10.3/2)
- allocator 4.8(2)
  - used* 4.4(7/3), P
- Alphanumeric
  - in* Interfaces.COBOL B.4(16)
- alphanumeric character
  - a category of Character A.3.2(31)
- Alphanumeric\_Set
  - in* Ada.Strings.Maps.Constants A.4.6(4)
- ambiguous 8.6(30)
- ambiguous cursor
  - of a vector A.18.2(240/2)
- ampersand 2.1(15/2)
  - in* Ada.Characters.Latin\_1 A.3.3(8)
- ampersand operator 4.4(1), 4.5.3(3)
- ancestor N(3.1/2)
  - of a library unit 10.1.1(11)
  - of a tree node A.18.10(4/3)
  - of a type 3.4.1(10/2)
  - ultimate 3.4.1(10/2)
- ancestor subtype
  - of a formal derived type 12.5.1(5/2)
  - of a private *extension\_declaration* 7.3(8)
- Ancestor\_Find
  - in* Ada.Containers.Multiway\_Trees A.18.10(34/3)
- ancestor\_part 4.3.2(3)
  - used* 4.3.2(2), P
- and operator 4.4(1), 4.5.1(2)
- and then (short-circuit control form) 4.4(1), 4.5.1(1)
- angle threshold G.2.4(10)
- Annex
  - informative 1.1.2(18)
  - normative 1.1.2(14)
  - Specialized Needs 1.1.2(7)
- Annotated Ada Reference Manual 0.3(5/2)
- anonymous access type 3.10(12/2)
- anonymous array type 3.3.1(1)
- anonymous protected type 3.3.1(1)
- anonymous task type 3.3.1(1)
- anonymous type 3.2.1(7/2)
- Any\_Priority *subtype of* Integer *in* System 13.7(16)
- APC
  - in* Ada.Characters.Latin\_1 A.3.3(19)
- apostrophe 2.1(15/2)
  - in* Ada.Characters.Latin\_1 A.3.3(8)
- Append
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(23/2)
  - in* Ada.Containers.Vectors A.18.2(46/2), A.18.2(47/2)
  - in* Ada.Strings.Bounded A.4.4(13), A.4.4(14), A.4.4(15), A.4.4(16), A.4.4(17), A.4.4(18), A.4.4(19), A.4.4(20)
  - in* Ada.Strings.Unbounded A.4.5(12), A.4.5(13), A.4.5(14)
- Append\_Child
  - in* Ada.Containers.Multiway\_Trees A.18.10(45/3)
- applicable index constraint 4.3.3(10)
- application areas 1.1.2(7)
- apply
  - to a callable construct by a return statement 6.5(4/2)
  - to a loop\_statement by an exit statement 5.7(4)
  - to a program unit by a program unit pragma 10.1.5(2)
- arbitrary order 1.1.4(18)
- Arccos
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(5)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(6)
- Arccosh
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(7)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(7)
- Arccot
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(5)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(6)

- Arccoth
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(7)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(7)
- Arcsin
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(5)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(6)
- Arcsinh
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(7)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(7)
- Arctan
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(5)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(6)
- Arctanh
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(7)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(7)
- Argument
  - in* Ada.Command\_Line A.15(5)
  - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(10/2), G.3.2(31/2)
  - in* Ada.Numerics.Generic\_Complex\_Types G.1.1(10)
- argument of a pragma 2.8(9)
- Argument\_Count
  - in* Ada.Command\_Line A.15(4)
- Argument\_Error
  - in* Ada.Numerics A.5(3/2)
- Arithmetic
  - child of* Ada.Calendar 9.6.1(8/2)
- array 3.6(1)
- array component expression 4.3.3(6)
- array indexing
  - See* indexed\_component 4.1.1(1)
- array slice 4.1.2(1)
- array type 3.6(1), N(4)
- array\_aggregate 4.3.3(2)
  - used* 4.3(2), 13.4(3), P
- array\_component\_association 4.3.3(5/2)
  - used* 4.3.3(4), P
- array\_type\_definition 3.6(2)
  - used* 3.2.1(4/2), 3.3.1(2/2), 12.5.3(2), P
- ASCII
  - package physically nested within the declaration of Standard A.1(36.3/2)
  - in* Standard A.1(36.3/2)
- aspect of representation 13.1(8/3)
  - coding 13.4(7)
  - controlled 13.11.3(5)
  - convention, calling convention B.1(28)
  - exported B.1(28)
  - imported B.1(28)
  - layout 13.5(1)
  - packing 13.2(5)
  - record layout 13.5(1)
  - specifiable attributes 13.3(5/1)
  - storage place 13.5(1)
  - aspect\_clause 13.1(2/1)
    - used* 3.8(5/1), 3.11(4/1), 9.1(5/1), 9.4(5/1), 9.4(8/1), P
  - assembly language C.1(4)
  - Assert pragma 11.4.2(3/2), L(2.1/2)
  - assertion policy 11.4.2(18/2)
  - Assertion\_Policy pragma 11.4.2(6/2), L(2.2/2)
  - Assertions 11.4.2(1/2)
    - child of* Ada 11.4.2(12/2)
  - Assign
    - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(17.1/3)
    - in* Ada.Containers.Hashed\_Maps A.18.5(17.1/3)
    - in* Ada.Containers.Hashed\_Sets A.18.8(17.1/3)
    - in* Ada.Containers.Indefinite\_Holders A.18.18(16/3)
    - in* Ada.Containers.Multiway\_Trees A.18.10(26/3)
    - in* Ada.Containers.Ordered\_Maps A.18.6(16.1/3)
    - in* Ada.Containers.Ordered\_Sets A.18.9(16.1/3)
    - in* Ada.Containers.Vectors A.18.2(34.1/3)
  - See* assignment operation 5.2(3)
  - assigning back of parameters 6.4.1(17)
  - assignment
    - user-defined 7.6(1)
  - assignment operation 5.2(3), 5.2(12), 7.6(13)
    - during elaboration of an object\_declaration 3.3.1(18/2)
    - during evaluation of a generic\_association for a formal object of mode in 12.4(11)
    - during evaluation of a parameter\_association 6.4.1(11)
    - during evaluation of an aggregate 4.3(5)
    - during evaluation of an initialized allocator 4.8(7/2)
    - during evaluation of an uninitialized allocator 4.8(9/2)
    - during evaluation of concatenation 4.5.3(10)
    - during execution of a for loop 5.5(9)
    - during execution of an assignment\_statement 5.2(12)
    - during parameter copy back 6.4.1(17)
    - assignment\_statement 5.2(2)
      - used* 5.1(4/2), P
  - associated components
    - of a record\_component\_association 4.3.1(10)
  - associated discriminants
    - of a named\_discriminant\_association 3.7.1(5)
    - of a positional\_discriminant\_association 3.7.1(5)
  - associated object
    - of a value of a by-reference type 6.2(10)
  - asterisk 2.1(15/2)
    - in* Ada.Characters.Latin\_1 A.3.3(8)
  - asynchronous
    - remote procedure call E.4.1(9)
  - Asynchronous pragma E.4.1(3), L(3)
  - asynchronous remote procedure call E.4(1)
  - asynchronous\_select 9.7.4(2)
    - used* 9.7(2), P
  - Asynchronous\_Task\_Control
    - child of* Ada D.11(3/2)
  - at-most-once execution E.4(11)
  - at\_clause J.7(1)
    - used* 13.1(2/1), P
  - atomic C.6(7/2)
  - Atomic pragma C.6(3), L(4)
  - Atomic\_Components pragma C.6(5), L(5)
  - Attach\_Handler
    - in* Ada.Interrupts C.3.2(7)
  - Attach\_Handler pragma C.3.1(4), L(6)
  - attaching
    - to an interrupt C.3(2)
  - attribute 4.1.4(1), K(1)
    - representation 13.3(1/1)
    - specifiable 13.3(5/1)
    - specifying 13.3(1/1)
  - attribute\_definition\_clause 13.3(2)
    - used* 13.1(2/1), P
  - attribute\_designator 4.1.4(3/2)
    - used* 4.1.4(2), 13.1(3), 13.3(2), P
  - Attribute\_Handle
    - in* Ada.Task\_Attributes C.7.2(3)
  - attribute\_reference 4.1.4(2)
    - used* 4.1(2/3), P
  - attributes
    - Access 3.10.2(24/1), 3.10.2(32/2)
    - Address 13.3(11), J.7.1(5)
    - Adjacent A.5.3(48)
    - Aft 3.5.10(5)
    - Alignment 13.3(23/2), 13.3(26.2/2)
    - Base 3.5(15)
    - Bit\_Order 13.5.3(4)
    - Body\_Version E.3(4)
    - Callable 9.9(2)
    - Caller C.7.1(14)
    - Ceiling A.5.3(33)
    - Class 3.9(14), 7.3.1(9), J.11(2/2)
    - Component\_Size 13.3(69)

Compose A.5.3(24)  
 Constrained 3.7.2(3), J.4(2)  
 Copy\_Sign A.5.3(51)  
 Count 9.9(5)  
 Definite 12.5.1(23)  
 Delta 3.5.10(3)  
 Denorm A.5.3(9)  
 Digits 3.5.8(2/1), 3.5.10(7)  
 Exponent A.5.3(18)  
 External\_Tag 13.3(75/3)  
 First 3.5(12), 3.6.2(3)  
 First(N) 3.6.2(4)  
 First\_Bit 13.5.2(3/2)  
 Floor A.5.3(30)  
 Fore 3.5.10(4)  
 Fraction A.5.3(21)  
 Identity 11.4.1(9), C.7.1(12)  
 Image 3.5(35)  
 Input 13.13.2(22), 13.13.2(32)  
 Last 3.5(13), 3.6.2(5)  
 Last(N) 3.6.2(6)  
 Last\_Bit 13.5.2(4/2)  
 Leading\_Part A.5.3(54)  
 Length 3.6.2(9)  
 Length(N) 3.6.2(10)  
 Machine A.5.3(60)  
 Machine\_Emax A.5.3(8)  
 Machine\_Emin A.5.3(7)  
 Machine\_Mantissa A.5.3(6)  
 Machine\_Overflows A.5.3(12),  
     A.5.4(4)  
 Machine\_Radix A.5.3(2), A.5.4(2)  
 Machine\_Rounding A.5.3(41.1/2)  
 Machine\_Rounds A.5.3(11), A.5.4(3)  
 Max 3.5(19)  
 Max\_Alignment\_For\_Allocation  
     13.11.1(3/1/3)  
 Max\_Size\_In\_Storage\_Elements  
     13.11.1(3/2)  
 Min 3.5(16)  
 Mod 3.5.4(16.1/2)  
 Model A.5.3(68), G.2.2(7)  
 Model\_Emin A.5.3(65), G.2.2(4)  
 Model\_Epsilon A.5.3(66)  
 Model\_Mantissa A.5.3(64), G.2.2(3/2)  
 Model\_Small A.5.3(67)  
 Modulus 3.5.4(17)  
 Output 13.13.2(19), 13.13.2(29)  
 Partition\_Id E.1(9)  
 Pos 3.5.5(2)  
 Position 13.5.2(2/2)  
 Pred 3.5(25)  
 Priority D.5.2(3/2)  
 Range 3.5(14), 3.6.2(7)  
 Range(N) 3.6.2(8)  
 Read 13.13.2(6), 13.13.2(14)  
 Remainder A.5.3(45)  
 Round 3.5.10(12)  
 Rounding A.5.3(36)  
 Safe\_First A.5.3(71), G.2.2(5)

Safe\_Last A.5.3(72), G.2.2(6)  
 Scale 3.5.10(11)  
 Scaling A.5.3(27)  
 Signed\_Zeros A.5.3(13)  
 Size 13.3(40), 13.3(45)  
 Small 3.5.10(2/1)  
 Storage\_Pool 13.11(13)  
 Storage\_Size 13.3(60), 13.11(14),  
     J.9(2)  
 Stream\_Size 13.13.2(1.2/2)  
 Succ 3.5(22)  
 Tag 3.9(16), 3.9(18)  
 Terminated 9.9(3)  
 Truncation A.5.3(42)  
 Unbiased\_Rounding A.5.3(39)  
 Unchecked\_Access 13.10(3), H.4(18)  
 Val 3.5.5(5)  
 Valid 13.9.2(3), H(6)  
 Value 3.5(52)  
 Version E.3(3)  
 Wide\_Image 3.5(28)  
 Wide\_Value 3.5(40)  
 Wide\_Wide\_Image 3.5(27.1/2)  
 Wide\_Wide\_Value 3.5(39.1/2)  
 Wide\_Wide\_Width 3.5(37.1/2)  
 Wide\_Width 3.5(38)  
 Width 3.5(39)  
 Write 13.13.2(3), 13.13.2(11)  
 available  
     stream attribute 13.13.2(39/2)

## B

Backus-Naur Form (BNF)  
     complete listing P  
     cross reference P  
     notation 1.1.4(3)  
     under Syntax heading 1.1.2(25)  
 base 2.4.2(3), 2.4.2(6)  
     *used* 2.4.2(2), P  
 base 16 literal 2.4.2(1)  
 base 2 literal 2.4.2(1)  
 base 8 literal 2.4.2(1)  
 Base attribute 3.5(15)  
 base decimal precision  
     of a floating point type 3.5.7(9)  
     of a floating point type 3.5.7(10)  
 base priority D.1(15)  
 base range  
     of a decimal fixed point type 3.5.9(16)  
     of a fixed point type 3.5.9(12)  
     of a floating point type 3.5.7(8),  
         3.5.7(10)  
     of a modular type 3.5.4(10)  
     of a scalar type 3.5(6)  
     of a signed integer type 3.5.4(9)  
     of an ordinary fixed point type  
         3.5.9(13)  
 base subtype  
     of a type 3.5(15)

Base\_Name  
     *in* Ada.Directories A.16(19/2)  
 based\_literal 2.4.2(2)  
     *used* 2.4(2), P  
 based\_numeral 2.4.2(4)  
     *used* 2.4.2(2), P  
 basic letter  
     a category of Character A.3.2(27)  
 basic\_declaration 3.1(3/2)  
     *used* 3.11(4/1), P  
 basic\_declarative\_item 3.11(4/1)  
     *used* 3.11(3), 7.1(3), P  
 Basic\_Map  
     *in* Ada.Strings.Maps.Constants  
         A.4.6(5)  
 Basic\_Set  
     *in* Ada.Strings.Maps.Constants  
         A.4.6(4)  
 become nonlimited 7.3.1(5/1), 7.5(16)  
 BEL  
     *in* Ada.Characters.Latin\_1 A.3.3(5)  
 belong  
     to a range 3.5(4)  
     to a subtype 3.2(8/2)  
 bibliography 1.2(1)  
 big endian 13.5.3(2)  
 binary  
     literal 2.4.2(1)  
     *in* Interfaces.COBOL B.4(10)  
 binary adding operator 4.5.3(1)  
 binary literal 2.4.2(1)  
 binary operator 4.5(9)  
 binary\_adding\_operator 4.5(4)  
     *used* 4.4(4), P  
 Binary\_Format  
     *in* Interfaces.COBOL B.4(24)  
 bit field  
     *See* record\_representation\_clause  
         13.5.1(1)  
 bit ordering 13.5.3(2)  
 bit string  
     *See* logical operators on boolean arrays  
         4.5.1(2)  
 Bit\_Order  
     *in* System 13.7(15/2)  
 Bit\_Order attribute 13.5.3(4)  
 Bit\_Order clause 13.3(7/2), 13.5.3(4)  
 blank  
     in text input for enumeration and  
         numeric types A.10.6(5/2)  
 Blank\_When\_Zero  
     *in* Ada.Text\_IO Editing F.3.3(7)  
 block\_statement 5.6(2)  
     *used* 5.1(5/2), P  
 blocked  
     [*partial*] D.2.1(11)  
     a task state 9(10)  
     during an entry call 9.5.3(19)  
     execution of a selective\_accept  
         9.7.1(16)

- on a delay\_statement 9.6(21)
- on an accept\_statement 9.5.2(24)
- waiting for activations to complete 9.2(5)
- waiting for dependents to terminate 9.3(5)
- blocked interrupt C.3(2)
- blocking, potentially 9.5.1(8)
  - Abort\_Task C.7.1(16)
  - delay\_statement 9.6(34), D.9(5)
  - remote subprogram call E.4(17)
  - RPC operations E.5(23)
  - Suspend\_Until\_True D.10(10)
- BMP 3.5.2(2/3), 3.5.2(3.1/2), 3.5.2(3/2)
- BNF (Backus-Naur Form)
  - complete listing P
  - cross reference P
  - notation 1.1.4(3)
  - under Syntax heading 1.1.2(25)
- body 3.11(5), 3.11.1(1/1)
  - used 3.11(3), P
- body\_stub 10.1.3(2)
  - used 3.11(5), P
- Body\_Version attribute E.3(4)
- BOM\_16
  - in Ada.Strings.UTF\_Encoding A.4.11(12/3)
- BOM\_16BE
  - in Ada.Strings.UTF\_Encoding A.4.11(10/3)
- BOM\_16LE
  - in Ada.Strings.UTF\_Encoding A.4.11(11/3)
- BOM\_8
  - in Ada.Strings.UTF\_Encoding A.4.11(9/3)
- Boolean 3.5.3(1)
  - in Standard A.1(5)
- boolean type 3.5.3(1)
- Bounded
  - child of Ada.Strings A.4.4(3)
- bounded error 1.1.2(31), 1.1.5(8)
- cause 4.8(11.1/2), 6.2(12), 7.6.1(14/1), 9.4(20.1/2), 9.5.1(8), 9.8(20), 10.2(26), 13.9.1(9), 13.11.2(11), A.17(25/2), A.18.2(238/2), A.18.2(239/2), A.18.2(243/2), A.18.3(152.1/3), A.18.3(152.2/3), A.18.3(152/2), A.18.4(75.1/3), A.18.4(75.2/3), A.18.7(96.1/3), A.18.7(96.2/3), A.18.10(194/3), A.18.10(195/3), A.18.18(53/3), A.18.18(54/3), A.18.19(10/3), A.18.20(14/3), A.18.21(15/3), A.18.22(12/3), A.18.23(15/3), A.18.24(12/3), A.18.25(14/3), C.7.1(17/3), C.7.2(13.2/1), D.2.6(30/2), D.3(13.1/2), E.1(10/2), E.3(6), J.7.1(11)
- Bounded\_IO
  - child of Ada.Text\_IO A.10.11(3/2)
  - child of Ada.Wide\_Text\_IO A.11(4/3)
  - child of Ada.Wide\_Wide\_Text\_IO A.11(4/3)
- Bounded\_Slice
  - in Ada.Strings.Bounded A.4.4(28.1/2), A.4.4(28.2/2)
- Bounded\_String
  - in Ada.Strings.Bounded A.4.4(6)
- bounds
  - of a discrete\_range 3.6.1(6)
  - of an array 3.6(13)
  - of the index range of an array\_aggregate 4.3.3(24)
- box
  - compound delimiter 3.6(15)
- BPH
  - in Ada.Characters.Latin\_1 A.3.3(17)
- broadcast signal
  - See protected object 9.4(1)
  - See queue 9.5.4(1)
- Broken\_Bar
  - in Ada.Characters.Latin\_1 A.3.3(21/3)
- BS
  - in Ada.Characters.Latin\_1 A.3.3(5)
- budget D.14.2(14/2)
- Budget\_Has\_Expired
  - in Ada.Execution\_Time.Group\_Budgets D.14.2(9/2)
- Budget\_Remaining
  - in Ada.Execution\_Time.Group\_Budgets D.14.2(9/2)
- Buffer\_Size
  - in Ada.Storage\_IO A.9(4)
- Buffer\_Type subtype of Storage\_Array
  - in Ada.Storage\_IO A.9(4)
- built in place 7.6(17.1/3)
- by copy parameter passing 6.2(2)
- by reference parameter passing 6.2(2)
- by-copy type 6.2(3)
- by-reference type 6.2(4)
  - atomic or volatile C.6(18)
- Byte
  - in Interfaces.COBOL B.4(29)
  - See storage element 13.3(8)
- byte sex
  - See ordering of storage elements in a word 13.5.3(5)
- Byte\_Array
  - in Interfaces.COBOL B.4(29)
- C**
- C
  - child of Interfaces B.3(4)
- C interface B.3(1/2)
- C standard 1.2(7/2)
- C++ standard 1.2(9/2)
- C\_float
  - in Interfaces.C B.3(15)
- Calendar
  - child of Ada 9.6(10)
- call 6(2)
- call on a dispatching operation 3.9.2(2/3)
- callable 9.9(2)
- Callable attribute 9.9(2)
- callable construct 6(2)
- callable entity 6(2)
- called partition E.4(1)
- Caller attribute C.7.1(14)
- calling convention 6.3.1(2/1), B.1(11)
  - Ada 6.3.1(3)
  - associated with a designated profile 3.10(11)
  - entry 6.3.1(13)
  - Intrinsic 6.3.1(4)
  - protected 6.3.1(12)
- calling partition E.4(1)
- calling stub E.4(10)
- CAN
  - in Ada.Characters.Latin\_1 A.3.3(6)
- Cancel\_Handler
  - in Ada.Execution\_Time.Group\_Budgets D.14.2(10/2)
  - in Ada.Execution\_Time.Timers D.14.1(7/2)
  - in Ada.Real\_Time.Timing\_Events D.15(5/2)
- cancellation
  - of a delay\_statement 9.6(22/3)
  - of an entry call 9.5.3(20)
- cancellation of a remote subprogram call E.4(13)
- canonical form A.5.3(3)
- canonical semantics 11.6(2)
- canonical-form representation A.5.3(10)
- capacity
  - of a hashed map A.18.5(41/2)
  - of a hashed set A.18.8(63/2)
  - of a vector A.18.2(2/2)
  - in Ada.Containers.Hashed\_Maps A.18.5(8/2)
  - in Ada.Containers.Hashed\_Sets A.18.8(10/2)
  - in Ada.Containers.Vectors A.18.2(19/2)
- Capacity\_Error
  - in Ada.Containers A.18.1(5.1/3)
- case insensitive 2.3(5/3)
- case\_statement 5.4(2)
  - used 5.1(5/2), P
- case\_statement\_alternative 5.4(3)
  - used 5.4(2), P
- cast
  - See type conversion 4.6(1)
  - See unchecked type conversion 13.9(1)

- catch (an exception)
    - See* handle 11(1)
  - categorization pragma E.2(2)
    - Remote\_Call\_Interface E.2.3(2)
    - Remote\_Types E.2.2(2)
    - Shared\_Passive E.2.1(2)
  - categorized library unit E.2(2)
  - category
    - of types 3.2(2/2), 3.4(1.1/2)
  - category (of types) N(4.1/2)
  - category determined for a formal type 12.5(6/2)
  - catenation operator
    - See* concatenation operator 4.4(1)
    - See* concatenation operator 4.5.3(3)
  - Cause\_Of\_Termination
    - in* Ada.Task\_Termination C.7.3(3/2)
  - CCH
    - in* Ada.Characters.Latin\_1 A.3.3(18)
  - cease to exist
    - object 7.6.1(11/2), 13.11.2(10/2)
    - type 7.6.1(11/2)
  - Cedilla
    - in* Ada.Characters.Latin\_1 A.3.3(22)
  - Ceiling
    - in* Ada.Containers.Ordered\_Maps A.18.6(41/2)
    - in* Ada.Containers.Ordered\_Sets A.18.9(51/2), A.18.9(71/2)
  - Ceiling attribute A.5.3(33)
  - ceiling priority
    - of a protected object D.3(8/2)
  - Ceiling\_Check
    - [*partial*] C.3.1(11/2), D.3(13)
  - Ceiling\_Locking locking policy D.3(7)
  - Cent\_Sign
    - in* Ada.Characters.Latin\_1 A.3.3(21/3)
  - change of representation 13.6(1)
  - char
    - in* Interfaces.C B.3(19)
  - char16\_array
    - in* Interfaces.C B.3(39.5/2)
  - char16\_nul
    - in* Interfaces.C B.3(39.3/2)
  - char16\_t
    - in* Interfaces.C B.3(39.2/2)
  - char32\_array
    - in* Interfaces.C B.3(39.14/2)
  - char32\_nul
    - in* Interfaces.C B.3(39.12/2)
  - char32\_t
    - in* Interfaces.C B.3(39.11/2)
  - char\_array
    - in* Interfaces.C B.3(23)
  - char\_array\_access
    - in* Interfaces.C.Strings B.3.1(4)
  - CHAR\_BIT
    - in* Interfaces.C B.3(6)
  - Character 3.5.2(2/3)
    - used* 2.7(2), P
  - in* Standard A.1(35/3)
  - character encoding A.4.11(38/3)
  - character plane 2.1(1/2)
  - character set 2.1(1/2)
  - character set standard
    - 16 and 32-bit 1.2(8/2)
    - 7-bit 1.2(2)
    - 8-bit 1.2(6)
    - control functions 1.2(5)
  - character type 3.5.2(1), N(5)
  - character\_literal 2.5(2)
    - used* 3.5.1(4), 4.1(2/3), 4.1.3(3), P
  - Character\_Mapping
    - in* Ada.Strings.Maps A.4.2(20/2)
  - Character\_Mapping\_Function
    - in* Ada.Strings.Maps A.4.2(25)
  - Character\_Range
    - in* Ada.Strings.Maps A.4.2(6)
  - Character\_Ranges
    - in* Ada.Strings.Maps A.4.2(7)
  - Character\_Sequence *subtype of* String
    - in* Ada.Strings.Maps A.4.2(16)
  - Character\_Set
    - in* Ada.Strings.Maps A.4.2(4/2)
  - in* Ada.Strings.Wide\_Maps A.4.7(46/2)
  - in* Ada.Strings.Wide\_Maps.Wide\_Constants A.4.8(48/2)
  - in* Interfaces.Fortran B.5(11)
- characteristics 7.3(15)
- Characters
  - child of* Ada A.3.1(2)
- chars\_ptr
  - in* Interfaces.C.Strings B.3.1(5/2)
- chars\_ptr\_array
  - in* Interfaces.C.Strings B.3.1(6/2)
- check
  - language-defined 11.5(2), 11.6(1)
- check, language-defined
  - Access\_Check 4.1(13), 4.6(51/2)
  - Accessibility\_Check 3.10.2(29), 4.6(39.1/2), 4.6(48/3), 4.8(10.1/3), 6.5(8/3), 6.5(21/2), E.4(18/1)
  - Allocation\_Check 4.8(10.2/2), 4.8(10.3/2)
  - Ceiling\_Check C.3.1(11/2), D.3(13)
  - Discriminant\_Check 4.1.3(15), 4.3(6), 4.3.2(8), 4.6(43), 4.6(45), 4.6(51/2), 4.6(52), 4.7(4), 4.8(10/2), 6.5(5.9/3)
  - Division\_Check 3.5.4(20), 4.5.5(22), A.5.1(28), A.5.3(47), G.1.1(40), G.1.2(28), K(202)
  - Elaboration\_Check 3.11(9)
  - Index\_Check 4.1.1(7), 4.1.2(7), 4.3.3(29/3), 4.3.3(30), 4.5.3(8), 4.6(51/2), 4.7(4), 4.8(10/2)
  - Length\_Check 4.5.1(8), 4.6(37), 4.6(52)
  - Overflow\_Check 3.5.4(20), 4.4(11), 5.4(13), G.2.1(11), G.2.2(7), G.2.3(25), G.2.4(2), G.2.6(3)
- Partition\_Check E.4(19)
- Range\_Check 3.2.2(11), 3.5(24), 3.5(27), 3.5(39.12/2), 3.5(39.4/2), 3.5(39.5/2), 3.5(43/2), 3.5(55/2), 3.5.5(7), 3.5.9(19), 4.2(11), 4.3.3(28), 4.5.1(8), 4.5.6(6), 4.5.6(13), 4.6(28), 4.6(38), 4.6(46), 4.6(51/2), 4.7(4), 13.13.2(35/2), A.5.2(39), A.5.3(26), A.5.3(29), A.5.3(50), A.5.3(53), A.5.3(59), A.5.3(62), K(11), K(114), K(122), K(184), K(220), K(241), K(41), K(47)
- Reserved\_Check C.3.1(10)
- Storage\_Check 11.1(6), 13.3(67), 13.11(17), D.7(17/1), D.7(18/1), D.7(19/1)
- Tag\_Check 3.9.2(16), 4.6(42), 4.6(52), 5.2(10), 6.5(8.1/3)
- Checking pragmas 11.5(1/2)
- child
  - of a library unit 10.1.1(1)
- Child\_Count
  - in* Ada.Containers.Multiway\_Trees A.18.10(39/3)
- Child\_Depth
  - in* Ada.Containers.Multiway\_Trees A.18.10(40/3)
- choice parameter 11.2(9)
- choice\_parameter\_specification 11.2(4)
  - used* 11.2(3), P
- Circumflex
  - in* Ada.Characters.Latin\_1 A.3.3(12)
- class
  - of types 3.2(2/2), 3.4(1.1/2)
  - See also* package 7(1)
  - See also* tag 3.9(3)
- class (of types) N(6/2)
- Class attribute 3.9(14), 7.3.1(9), J.11(2/2)
- class factory 3.9(30.1/2)
- class-wide type 3.4.1(4), 3.7(26)
- cleanup
  - See* finalization 7.6.1(1)
- clear
  - execution timer object D.14.1(12/2)
  - group budget object D.14.2(15/2)
  - timing event object D.15(9/2)
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(13/2)
  - in* Ada.Containers.Hashed\_Maps A.18.5(12/2)
  - in* Ada.Containers.Hashed\_Sets A.18.8(14/2)
  - in* Ada.Containers.Indefinite\_Holders A.18.18(11/3)
  - in* Ada.Containers.Multiway\_Trees A.18.10(21/3)
  - in* Ada.Containers.Ordered\_Maps A.18.6(11/2)

- in* Ada.Containers.Ordered\_Sets A.18.9(13/2)
- in* Ada.Containers.Vectors A.18.2(24/2)
- in* Ada.Environment\_Variables A.17(7/2)
- cleared
  - termination handler C.7.3(9/2)
- clock 9.6(6)
  - in* Ada.Calendar 9.6(12)
  - in* Ada.Execution\_Time D.14(5/2)
  - in* Ada.Real\_Time D.8(6)
- clock jump D.8(32)
- clock tick D.8(23)
- Close
  - in* Ada.Direct\_IO A.8.4(8)
  - in* Ada.Sequential\_IO A.8.1(8)
  - in* Ada.Streams.Stream\_IO A.12.1(10)
  - in* Ada.Text\_IO A.10.1(11)
- close result set G.2.3(5)
- closed entry 9.5.3(5)
  - of a protected object 9.5.3(7)
  - of a task 9.5.3(6)
- closed under derivation 3.4(28), N(6/2)
- closure
  - downward 3.10.2(37/2)
- COBOL
  - child of* Interfaces B.4(7)
- COBOL interface B.4(1)
- COBOL standard 1.2(4/2)
- COBOL\_Character
  - in* Interfaces.COBOL B.4(13)
- COBOL\_To\_Ada
  - in* Interfaces.COBOL B.4(15)
- code\_statement 13.8(2)
  - used* 5.1(4/2), P
- coding
  - aspect of representation 13.4(7)
- coextension
  - of an object 3.10.2(14.4/3)
- Col
  - in* Ada.Text\_IO A.10.1(37)
- collection
  - finalization of 7.6.1(11/2)
- colon 2.1(15/2)
  - in* Ada.Characters.Latin\_1 A.3.3(10)
- column number A.10(9)
- comma 2.1(15/2)
  - in* Ada.Characters.Latin\_1 A.3.3(8)
- Command\_Line
  - child of* Ada A.15(3)
- Command\_Name
  - in* Ada.Command\_Line A.15(6)
- comment 2.7(2)
- comments, instructions for submission 0.3(58/1)
- Commercial\_At
  - in* Ada.Characters.Latin\_1 A.3.3(10)
- Communication\_Error
  - in* System.RPC E.5(5)
- comparison operator
  - See* relational operator 4.5.2(1)
- compatibility
- composite\_constraint with an access subtype 3.10(15/2)
- constraint with a subtype 3.2.2(12)
- delta\_constraint with an ordinary fixed point subtype J.3(9)
- digits\_constraint with a decimal fixed point subtype 3.5.9(18)
- digits\_constraint with a floating point subtype J.3(10)
- discriminant constraint with a subtype 3.7.1(10)
- index constraint with a subtype 3.6.1(7)
- range with a scalar subtype 3.5(8)
- range\_constraint with a scalar subtype 3.5(8)
- compatible
  - a type, with a convention B.1(12)
- compilation 10.1.1(2)
  - separate 10.1(1)
- Compilation unit 10.1(2), 10.1.1(9), N(7)
- compilation units needed
  - by a compilation unit 10.2(2)
  - remote call interface E.2.3(18)
  - shared passive library unit E.2.1(11)
- compilation\_unit 10.1.1(3)
  - used* 10.1.1(2), P
- compile-time error 1.1.2(27), 1.1.5(4)
- compile-time semantics 1.1.2(28)
- complete context 8.6(4)
- completely defined 3.11.1(8)
- completion
  - abnormal 7.6.1(2/2)
  - compile-time concept 3.11.1(1/1)
  - normal 7.6.1(2/2)
  - run-time concept 7.6.1(2/2)
- completion and leaving (completed and left) 7.6.1(2/2)
- completion legality
  - [*partial*] 3.10.1(13)
  - entry\_body 9.5.2(16)
- Complex
  - in* Ada.Numerics.Generic\_Complex\_-Types G.1.1(3)
  - in* Interfaces.Fortran B.5(9)
- Complex\_Arrays
  - child of* Ada.Numerics G.3.2(53/2)
- Complex\_Elementary\_Functions
  - child of* Ada.Numerics G.1.2(9/1)
- Complex\_IO
  - child of* Ada.Text\_IO G.1.3(3)
  - child of* Ada.Wide\_Text\_IO G.1.4(1)
  - child of* Ada.Wide\_Wide\_Text\_IO G.1.5(1/2)
- Complex\_Matrix
  - in* Ada.Numerics.Generic\_Complex\_-Arrays G.3.2(4/2)
- Complex\_Text\_IO
  - child of* Ada G.1.3(9.1/2)
- Complex\_Types
  - child of* Ada.Numerics G.1.1(25/1)
- Complex\_Vector
  - in* Ada.Numerics.Generic\_Complex\_-Arrays G.3.2(4/2)
- component 3.2(2/2)
- component subtype 3.6(10)
- component\_choice\_list 4.3.1(5)
  - used* 4.3.1(4/2), P
- component\_clause 13.5.1(3)
  - used* 13.5.1(2), P
- component\_declaration 3.8(6)
  - used* 3.8(5/1), 9.4(6), P
- component\_definition 3.6(7/2)
  - used* 3.6(3), 3.6(5), 3.8(6), P
- component\_item 3.8(5/1)
  - used* 3.8(4), P
- component\_list 3.8(4)
  - used* 3.8(3), 3.8.1(3), P
- Component\_Size attribute 13.3(69)
- Component\_Size clause 13.3(7/2), 13.3(70)
- components
  - of a record type 3.8(9/2)
- Compose
  - in* Ada.Directories A.16(20/2)
  - in*
    - Ada.Directories.Hierarchical\_File\_Names A.16.1(14/3)
- Compose attribute A.5.3(24)
- Compose\_From\_Cartesian
  - in* Ada.Numerics.Generic\_Complex\_-Arrays G.3.2(9/2), G.3.2(29/2)
  - in* Ada.Numerics.Generic\_Complex\_-Types G.1.1(8)
- Compose\_From\_Polar
  - in* Ada.Numerics.Generic\_Complex\_-Arrays G.3.2(11/2), G.3.2(32/2)
  - in* Ada.Numerics.Generic\_Complex\_-Types G.1.1(11)
- composite type 3.2(2/2), N(8/2)
- composite\_constraint 3.2.2(7)
  - used* 3.2.2(5), P
- compound delimiter 2.2(10)
- compound\_statement 5.1(5/2)
  - used* 5.1(3), P
- concatenation operator 4.4(1), 4.5.3(3)
- concrete subprogram
  - See* nonabstract subprogram 3.9.3(1/2)
- concrete type
  - See* nonabstract type 3.9.3(1/2)
- concurrent processing
  - See* task 9(1)
- condition 5.3(3)
  - used* 5.3(2), 5.5(3), 5.7(2), 9.5.2(7), 9.7.1(3), P
  - See also* exception 11(1)
- conditional\_entry\_call 9.7.3(2)

- used* 9.7(2), P
- configuration
  - of the partitions of a program E(4)
- configuration pragma 10.1.5(8)
  - Assertion\_Policy 11.4.2(7/2)
  - Detect\_Blocking H.5(4/2)
  - Discard\_Names C.5(4)
  - Locking\_Policy D.3(5)
  - Normalize\_Scalars H.1(4)
  - Partition\_Elaboration\_Policy H.6(5/2)
  - Priority\_Specific\_Dispatching D.2.2(4/2)
  - Profile D.13(6/2)
  - Queuing\_Policy D.4(5)
  - Restrictions 13.12(8/3)
  - Reviewable H.3.1(4)
  - Suppress 11.5(5/2)
  - Task\_Dispatching\_Policy D.2.2(4/2)
  - Unsuppress 11.5(5/2)
- confirming
  - representation item 13.1(18.2/2)
- conformance 6.3.1(1)
  - of an implementation with the Standard 1.1.3(1)
  - See also* full conformance, mode conformance, subtype conformance, type conformance
- Conjugate
  - in* Ada.Numerics.Generic\_Complex - Arrays G.3.2(13/2), G.3.2(34/2)
  - in* Ada.Numerics.Generic\_Complex - Types G.1.1(12), G.1.1(15)
- consistency
  - among compilation units 10.1.4(5)
- constant 3.3(13/3)
  - result of a function call 6.4(12/2)
  - See also* literal 4.2(1)
  - See also* static 4.9(1)
- constant object 3.3(13/3)
- constant view 3.3(13/3)
- Constants
  - child of* Ada.Strings.Maps A.4.6(3/2)
- constituent
  - of a construct 1.1.4(17)
- constrained 3.2(9)
  - object 3.3.1(9/2)
  - object 6.4.1(16)
  - subtype 3.2(9), 3.4(6), 3.5(7), 3.5.1(10), 3.5.4(9), 3.5.4(10), 3.5.7(11), 3.5.9(13), 3.5.9(16), 3.6(15), 3.6(16), 3.7(26), 3.9(15)
  - subtype 3.10(14/1)
  - subtype K(35)
- Constrained attribute 3.7.2(3), J.4(2)
- constrained by its initial value 3.3.1(9/2)
  - [*partial*] 4.8(6/3), 6.5(5.9/3)
- constrained\_array\_definition 3.6(5)
  - used* 3.6(2), P
- constraint 3.2.2(5)
  - [*partial*] 3.2(7/2)
- of a first array subtype 3.6(16)
- of a subtype 3.2(8/2)
- of an object 3.3.1(9/2)
- used* 3.2.2(3/2), P
- Constraint\_Error
  - raised by failure of run-time check 3.2.2(12), 3.5(24), 3.5(27), 3.5(39.12/2), 3.5(39.4/2), 3.5(39.5/2), 3.5(43/2), 3.5(55/2), 3.5.4(20), 3.5.5(7), 3.5.9(19), 3.9.2(16), 4.1(13), 4.1.1(7), 4.1.2(7), 4.1.3(15), 4.2(11), 4.3(6), 4.3.2(8), 4.3.3(31), 4.4(11), 4.5(10), 4.5(11), 4.5(12), 4.5.1(8), 4.5.3(8), 4.5.5(22), 4.5.6(6), 4.5.6(12), 4.5.6(13), 4.6(28), 4.6(57), 4.6(60), 4.7(4), 4.8(10/2), 5.2(10), 5.4(13), 6.5(5.9/3), 6.5(8.1/3), 11.1(4), 11.4.1(14/2), 11.5(10), 13.9.1(9), 13.13.2(35/2), A.4.3(109), A.4.7(47), A.4.8(51/2), A.5.1(28), A.5.1(34), A.5.2(39), A.5.2(40.1/1), A.5.3(26), A.5.3(29), A.5.3(47), A.5.3(50), A.5.3(53), A.5.3(59), A.5.3(62), A.15(14), B.3(53), B.3(54), B.4(58), E.4(19), G.1.1(40), G.1.2(28), G.2.1(12), G.2.2(7), G.2.3(26), G.2.4(3), G.2.6(4), K(11), K(114), K(122), K(184), K(202), K(220), K(241), K(261), K(41), K(47)
  - in* Standard A.1(46)
- Construct 1.1.4(16), N(9)
- constructor
  - See* initialization 3.3.1(18/2)
  - See* initialization 7.6(1)
  - See* initialization expression 3.3.1(4)
  - See* Initialize 7.6(1)
  - See* initialized allocator 4.8(4)
- container
  - cursor A.18.2(2)
  - list A.18.3(1/2)
  - map A.18.4(1/2)
  - set A.18.7(1/2)
  - vector A.18.2(1/2)
- Containers
  - child of* Ada A.18.1(3/2)
- Containing\_Directory
  - in* Ada.Directories A.16(17/2)
  - in* Ada.Directories.Hierarchical\_File\_Names A.16.1(11/3)
- Contains
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(43/2)
  - in* Ada.Containers.Hashed\_Maps A.18.5(32/2)
  - in* Ada.Containers.Hashed\_Sets A.18.8(44/2), A.18.8(57/2)
  - in* Ada.Containers.Multiway\_Trees A.18.10(35/3)
- in* Ada.Containers.Ordered\_Maps A.18.6(42/2)
- in* Ada.Containers.Ordered\_Sets A.18.9(52/2), A.18.9(72/2)
- in* Ada.Containers.Vectors A.18.2(71/2)
- context free grammar
- complete listing P
- cross reference P
- notation 1.1.4(3)
  - under Syntax heading 1.1.2(25)
- context\_clause 10.1.2(2)
  - used* 10.1.1(3), P
- context\_item 10.1.2(3)
  - used* 10.1.2(2), P
- contiguous representation
  - [*partial*] 13.5.2(5), 13.7.1(12), 13.9(9), 13.9(17), 13.11(21.6/3)
- Continue
  - in* Ada.Asynchronous\_Task\_Control D.11(3/2)
- control character
  - a category of Character A.3.2(22)
  - a category of Character A.3.3(4), A.3.3(15)
  - See also* format\_effector 2.1(13/2)
- Control\_Set
  - in* Ada.Strings.Maps.Constants A.4.6(4)
- controlled
  - aspect of representation 13.11.3(5)
  - in* Ada.Finalization 7.6(5/2)
- Controlled pragma 13.11.3(3), L(7)
- controlled type 7.6(2), 7.6(9/2), N(10)
- controlling access result 3.9.2(2/3)
- controlling formal parameter 3.9.2(2/3)
- controlling operand 3.9.2(2/3)
- controlling result 3.9.2(2/3)
- controlling tag
  - for a call on a dispatching operation 3.9.2(1/2)
- controlling tag value 3.9.2(14)
  - for the expression in an assignment\_statement 5.2(9)
- controlling type
  - of a formal\_abstract\_subprogram\_declaration on 12.6(8.4/2)
- convention 6.3.1(2/1), B.1(11)
  - aspect of representation B.1(28)
- Convention pragma B.1(7), L(8)
- conversion 4.6(1), 4.6(28)
  - access 4.6(24.11/2), 4.6(24.18/2), 4.6(24.19/2), 4.6(47)
  - arbitrary order 1.1.4(18)
  - array 4.6(24.2/2), 4.6(36)
  - composite (non-array) 4.6(21/2), 4.6(40)
  - enumeration 4.6(21.1/2), 4.6(34)
  - numeric 4.6(24.1/2), 4.6(29)

- unchecked 13.9(1)
- value 4.6(5/2)
- view 4.6(5/2)
- Conversion\_Error
  - in* Interfaces.COBOL B.4(30)
- Conversions
  - child of* Ada.Characters A.3.4(2/2)
  - child of* Ada.Strings.UTF\_Encoding A.4.11(15/3)
- Convert
  - in*
    - Ada.Strings.UTF\_Encoding.Conversion ns A.4.11(16/3), A.4.11(17/3), A.4.11(18/3), A.4.11(19/3), A.4.11(20/3)
- convertible 4.6(4)
  - required 4.6(24.13/2), 4.6(24.4/2), 8.6(27.1/3)
- Copy
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(17.2/3)
  - in* Ada.Containers.Hashed\_Maps A.18.5(17.2/3)
  - in* Ada.Containers.Hashed\_Sets A.18.8(17.2/3)
  - in* Ada.Containers.Indefinite\_Holders A.18.18(17/3), A.18.20(10/3), A.18.21(13/3), A.18.22(10/3), A.18.23(13/3), A.18.24(10/3)
  - in* Ada.Containers.Multiway\_Trees A.18.10(27/3)
  - in* Ada.Containers.Ordered\_Maps A.18.6(16.2/3)
  - in* Ada.Containers.Ordered\_Sets A.18.9(16.2/3)
  - in* Ada.Containers.Vectors A.18.2(34.2/3)
- copy back of parameters 6.4.1(17)
- copy parameter passing 6.2(2)
- Copy\_Array
  - in* Interfaces.C.Pointers B.3.2(15)
- Copy\_File
  - in* Ada.Directories A.16(13/2)
- Copy\_Sign attribute A.5.3(51)
- Copy\_Subtree
  - in* Ada.Containers.Multiway\_Trees A.18.10(47/3)
- Copy\_Terminated\_Array
  - in* Interfaces.C.Pointers B.3.2(14)
- Copyright\_Sign
  - in* Ada.Characters.Latin\_1 A.3.3(21/3)
- core language 1.1.2(2)
- corresponding constraint 3.4(6)
- corresponding discriminants 3.7(18)
- corresponding index
  - for an array\_aggregate 4.3.3(8)
- corresponding subtype 3.4(18/3)
- corresponding value
  - of the target type of a conversion 4.6(28)
- Cos
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(4)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(5)
- Cosh
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(6)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(7)
- Cot
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(4)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(5)
- Coth
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(6)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(7)
- Count
  - in* Ada.Direct\_IO A.8.4(4)
  - in* Ada.Streams.Stream\_IO A.12.1(7)
  - in* Ada.Strings.Bounded A.4.4(48), A.4.4(49), A.4.4(50)
  - in* Ada.Strings.Fixed A.4.3(13), A.4.3(14), A.4.3(15)
  - in* Ada.Strings.Unbounded A.4.5(43), A.4.5(44), A.4.5(45)
  - in* Ada.Text\_IO A.10.1(5)
- Count attribute 9.9(5)
- Count\_Type
  - in* Ada.Containers A.18.1(5/2)
- cover
  - a type 3.4.1(9)
  - of a choice and an exception 11.2(6)
- cover a value
  - by a discrete\_choice 3.8.1(9)
  - by a discrete\_choice\_list 3.8.1(13)
- CPU clock tick D.14(15/2)
- CPU time
  - of a task D.14(11/2)
- CPU\_Tick
  - in* Ada.Execution\_Time D.14(4/2)
- CPU\_Time
  - in* Ada.Execution\_Time D.14(4/2)
- CPU\_Time\_First
  - in* Ada.Execution\_Time D.14(4/2)
- CPU\_Time\_Last
  - in* Ada.Execution\_Time D.14(4/2)
- CPU\_Time\_Unit
  - in* Ada.Execution\_Time D.14(4/2)
- CR
  - in* Ada.Characters.Latin\_1 A.3.3(5)
- create 3.1(12)
  - in* Ada.Direct\_IO A.8.4(6)
  - in* Ada.Sequential\_IO A.8.1(6)
  - in* Ada.Streams.Stream\_IO A.12.1(8)
  - in* Ada.Text\_IO A.10.1(9)
- Create\_Directory
  - in* Ada.Directories A.16(7/2)
- Create\_Path
  - in* Ada.Directories A.16(9/2)
- creation
  - of a protected object C.3.1(10)
  - of a return object 6.5(5.9/3)
  - of a tag 13.14(20/2)
  - of a task object D.1(17)
  - of an object 3.3(1)
- critical section
  - See* intertask communication 9.5(1)
- CSI
  - in* Ada.Characters.Latin\_1 A.3.3(19)
- Currency\_Sign
  - in* Ada.Characters.Latin\_1 A.3.3(21/3)
- current column number A.10(9)
- current index
  - of an open direct file A.8(4)
  - of an open stream file A.12.1(1.1/1)
- current instance
  - of a generic unit 8.6(18)
  - of a type 8.6(17/2)
- current line number A.10(9)
- current mode
  - of an open file A.7(7)
- current page number A.10(9)
- Current size
  - of a stream file A.12.1(1.1/1)
  - of an external file A.8(3)
- Current\_Directory
  - in* Ada.Directories A.16(5/2)
- Current\_Error
  - in* Ada.Text\_IO A.10.1(17), A.10.1(20)
- Current\_Handler
  - in*
    - Ada.Execution\_Time.Group\_Budgets D.14.2(10/2)
  - in* Ada.Execution\_Time.Timers D.14.1(7/2)
  - in* Ada.Interrupts C.3.2(6)
  - in* Ada.Real\_Time.Timing\_Events D.15(5/2)
- Current\_Input
  - in* Ada.Text\_IO A.10.1(17), A.10.1(20)
- Current\_Output
  - in* Ada.Text\_IO A.10.1(17), A.10.1(20)
- Current\_State
  - in* Ada.Synchronous\_Task\_Control D.10(4)
- Current\_Task
  - in* Ada.Task\_Identification C.7.1(3/1)
- Current\_Task\_Fallback\_Handler
  - in* Ada.Task\_Termination C.7.3(5/2)
- cursor
  - ambiguous A.18.2(240/2)
  - for a container A.18(2/2)
  - invalid A.18.2(248/2), A.18.3(153/2), A.18.4(76/2), A.18.7(97/2), A.18.10(196/3)



- in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(7/2)
- in* Ada.Containers.Hashed\_Maps A.18.5(4/2)
- in* Ada.Containers.Hashed\_Sets A.18.8(4/2)
- in* Ada.Containers.Multiway\_Trees A.18.10(9/3)
- in* Ada.Containers.Ordered\_Maps A.18.6(5/2)
- in* Ada.Containers.Ordered\_Sets A.18.9(5/2)
- in* Ada.Containers.Vectors A.18.2(9/2)
- D**
- dangling references
  - prevention via accessibility rules 3.10.2(3/2)
- Data\_Error
  - in* Ada.Direct\_IO A.8.4(18)
  - in* Ada.IO\_Exceptions A.13(4)
  - in* Ada.Sequential\_IO A.8.1(15)
  - in* Ada.Storage\_IO A.9(9)
  - in* Ada.Streams.Stream\_IO A.12.1(26)
  - in* Ada.Text\_IO A.10.1(85)
- date and time formatting standard 1.2(5.1/2)
- Day
  - in* Ada.Calendar 9.6(13)
  - in* Ada.Calendar.Formatting 9.6.1(23/2)
- Day\_Count
  - in* Ada.Calendar.Arithmetic 9.6.1(10/2)
- Day\_Duration *subtype* of Duration
  - in* Ada.Calendar 9.6(11/2)
- Day\_Name
  - in* Ada.Calendar.Formatting 9.6.1(17/2)
- Day\_Number *subtype* of Integer
  - in* Ada.Calendar 9.6(11/2)
- Day\_of\_Week
  - in* Ada.Calendar.Formatting 9.6.1(18/2)
- DC1
  - in* Ada.Characters.Latin\_1 A.3.3(6)
- DC2
  - in* Ada.Characters.Latin\_1 A.3.3(6)
- DC3
  - in* Ada.Characters.Latin\_1 A.3.3(6)
- DC4
  - in* Ada.Characters.Latin\_1 A.3.3(6)
- DCS
  - in* Ada.Characters.Latin\_1 A.3.3(18)
- Deadline *subtype* of Time
  - in* Ada.Dispatching.EDF D.2.6(9/2)
- Dealocate
  - in* System.Storage\_Pools 13.11(8)
- deallocation of storage 13.11.2(1)
- Decimal
  - child of* Ada F.2(2)
- decimal digit
  - a category of Character A.3.2(28)
- decimal fixed point type 3.5.9(1), 3.5.9(6)
- Decimal\_Conversions
  - in* Interfaces.COBOl B.4(31)
- Decimal\_Digit\_Set
  - in* Ada.Strings.Maps.Constants A.4.6(4)
- Decimal\_Element
  - in* Interfaces.COBOl B.4(12)
- decimal\_fixed\_point\_definition 3.5.9(4)
  - used* 3.5.9(2), P
- Decimal\_IO
  - in* Ada.Text\_IO A.10.1(73)
- decimal\_literal 2.4.1(2)
  - used* 2.4(2), P
- Decimal\_Output
  - in* Ada.Text\_IO.Editing F.3.3(11)
- Declaration 3.1(5), 3.1(6/2), N(11)
- declaration list
  - declarative\_part 3.11(6.1/2)
  - package\_specification 7.1(6/2)
- declarative region
  - of a construct 8.1(1)
- declarative\_item 3.11(3)
  - used* 3.11(2), P
- declarative\_part 3.11(2)
  - used* 5.6(2), 6.3(2/2), 7.2(2), 9.1(6), 9.5.2(5), P
- declare 3.1(8), 3.1(12)
- declared pure 10.2.1(17/3)
- Decode
  - in* Ada.Strings.UTF\_Encoding.Wide\_Wi de\_Encoding A.4.11(26/3), A.4.11(27/3), A.4.11(28/3), A.4.11(34/3), A.4.11(35/3), A.4.11(36/3)
- Decrement
  - in* Interfaces.C.Pointers B.3.2(11)
- deeper
  - accessibility level 3.10.2(3/2)
  - statically 3.10.2(4), 3.10.2(17)
- default directory A.16(48/2)
- default entry queuing policy 9.5.3(17)
- default treatment C.3(5)
- Default\_Aft
  - in* Ada.Text\_IO A.10.1(64), A.10.1(69), A.10.1(74)
  - in* Ada.Text\_IO.Complex\_IO G.1.3(5)
- Default\_Base
  - in* Ada.Text\_IO A.10.1(53), A.10.1(58)
- Default\_Bit\_Order
  - in* System 13.7(15/2)
- Default\_Currency
  - in* Ada.Text\_IO.Editing F.3.3(10)
- Default\_Deadline
  - in* Ada.Dispatching.EDF D.2.6(9/2)
- Default\_Exp
  - in* Ada.Text\_IO A.10.1(64), A.10.1(69), A.10.1(74)
  - in* Ada.Text\_IO.Complex\_IO G.1.3(5)
- default\_expression 3.7(6)
  - used* 3.7(5/2), 3.8(6), 6.1(15/2), 12.4(2/3), P
- Default\_Fill
  - in* Ada.Text\_IO.Editing F.3.3(10)
- Default\_Fore
  - in* Ada.Text\_IO A.10.1(64), A.10.1(69), A.10.1(74)
  - in* Ada.Text\_IO.Complex\_IO G.1.3(5)
- Default\_Modulus
  - in* Ada.Containers.Indefinite\_Holders A.18.21(10/3), A.18.23(10/3)
- default\_name 12.6(4)
  - used* 12.6(3/2), P
- Default\_Priority
  - in* System 13.7(17)
- Default\_Quantum
  - in* Ada.Dispatching.Round\_Robin D.2.5(4/2)
- Default\_Radix\_Mark
  - in* Ada.Text\_IO.Editing F.3.3(10)
- Default\_Separator
  - in* Ada.Text\_IO.Editing F.3.3(10)
- Default\_Setting
  - in* Ada.Text\_IO A.10.1(80)
- Default\_Width
  - in* Ada.Text\_IO A.10.1(53), A.10.1(58), A.10.1(80)
- deferred constant 7.4(2)
- deferred constant declaration 3.3.1(6), 7.4(2)
- defining name 3.1(10)
- defining\_character\_literal 3.5.1(4)
  - used* 3.5.1(3), P
- defining\_designator 6.1(6)
  - used* 6.1(4.2/2), 12.3(2/2), P
- defining\_identifier 3.1(4)
  - used* 3.2.1(3), 3.2.2(2), 3.3.1(3), 3.5.1(3), 3.10.1(2/2), 5.5(4), 6.1(7), 6.5(2.1/3), 7.3(2), 7.3(3/2), 8.5.1(2/2), 8.5.2(2), 9.1(2/2), 9.1(3/2), 9.1(6), 9.4(2/2), 9.4(3/2), 9.4(7), 9.5.2(2/2), 9.5.2(5), 9.5.2(8), 10.1.3(4), 10.1.3(5), 10.1.3(6), 11.2(4), 12.5(2), 12.7(2), P
- defining\_identifier\_list 3.3.1(3)
  - used* 3.3.1(2/2), 3.3.2(2), 3.7(5/2), 3.8(6), 6.1(15/2), 11.1(2), 12.4(2/3), P
- defining\_operator\_symbol 6.1(11)
  - used* 6.1(6), P
- defining\_program\_unit\_name 6.1(7)
  - used* 6.1(4.1/2), 6.1(6), 7.1(3), 7.2(2), 8.5.3(2), 8.5.5(2), 12.3(2/2), P
- Definite attribute 12.5.1(23)
- definite subtype 3.3(23)

- definition 3.1(7), N(12/2)
- Degree\_Sign
  - in* Ada.Characters.Latin\_1 A.3.3(22)
- DEL
  - in* Ada.Characters.Latin\_1 A.3.3(14)
- delay\_alternative 9.7.1(6)
  - used* 9.7.1(4), 9.7.2(2), P
- delay\_relative\_statement 9.6(4)
  - used* 9.6(2), P
- delay\_statement 9.6(2)
  - used* 5.1(4/2), 9.7.1(6), 9.7.4(4/2), P
- Delay\_Until\_And\_Set\_Deadline
  - in* Ada.Dispatching.EDF D.2.6(9/2)
- delay\_until\_statement 9.6(3)
  - used* 9.6(2), P
- Delete
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(24/2)
  - in* Ada.Containers.Hashed\_Maps A.18.5(25/2), A.18.5(26/2)
  - in* Ada.Containers.Hashed\_Sets A.18.8(24/2), A.18.8(25/2), A.18.8(55/2)
  - in* Ada.Containers.Ordered\_Maps A.18.6(24/2), A.18.6(25/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(23/2), A.18.9(24/2), A.18.9(68/2)
  - in* Ada.Containers.Vectors A.18.2(50/2), A.18.2(51/2)
  - in* Ada.Direct\_IO A.8.4(8)
  - in* Ada.Sequential\_IO A.8.1(8)
  - in* Ada.Streams.Stream\_IO A.12.1(10)
  - in* Ada.Strings.Bounded A.4.4(64), A.4.4(65)
  - in* Ada.Strings.Fixed A.4.3(29), A.4.3(30)
  - in* Ada.Strings.Unbounded A.4.5(59), A.4.5(60)
  - in* Ada.Text\_IO A.10.1(11)
- Delete\_Children
  - in* Ada.Containers.Multiway\_Trees A.18.10(46/3)
- Delete\_Directory
  - in* Ada.Directories A.16(8/2)
- Delete\_File
  - in* Ada.Directories A.16(11/2)
- Delete\_First
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(25/2)
  - in* Ada.Containers.Ordered\_Maps A.18.6(26/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(25/2)
  - in* Ada.Containers.Vectors A.18.2(52/2)
- Delete\_Last
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(26/2)
  - in* Ada.Containers.Ordered\_Maps A.18.6(27/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(26/2)
  - in* Ada.Containers.Vectors A.18.2(53/2)
- Delete\_Leaf
  - in* Ada.Containers.Multiway\_Trees A.18.10(29/3)
- Delete\_Subtree
  - in* Ada.Containers.Multiway\_Trees A.18.10(30/3)
- Delete\_Tree
  - in* Ada.Directories A.16(10/2)
- delimiter 2.2(8/2)
- delivery
  - of an interrupt C.3(2)
- delta
  - of a fixed point type 3.5.9(1)
- Delta attribute 3.5.10(3)
- delta\_constraint J.3(2)
  - used* 3.2.2(6), P
- Denorm attribute A.5.3(9)
- denormalized number A.5.3(10)
- denote 8.6(16)
  - informal definition 3.1(8)
  - name used as a pragma argument 8.6(32)
- depend on a discriminant
  - for a component 3.7(20)
  - for a constraint or component\_definition 3.7(19)
- dependence
  - elaboration 10.2(9)
  - of a task on a master 9.3(1)
  - of a task on another task 9.3(4)
  - semantic 10.1.1(26/2)
- depth
  - accessibility level 3.10.2(3/2)
  - in* Ada.Containers.Multiway\_Trees A.18.10(17/3)
- depth-first order A.18.10(5/3)
- dereference 4.1(8)
- Dereference\_Error
  - in* Interfaces.C.Strings B.3.1(12)
- derivation class
  - for a type 3.4.1(2/2)
- derived from
  - directly or indirectly 3.4.1(2/2)
- derived type 3.4.1(1/2), N(13/2)
  - [*partial*] 3.4(24)
- derived\_type\_definition 3.4(2/2)
  - used* 3.2.1(4/2), P
- descendant 10.1.1(11), N(13.1/2)
  - of a tree node A.18.10(4/3)
  - of a type 3.4.1(10/2)
  - relationship with scope 8.2(4)
- Descendant\_Tag
  - in* Ada.Tags 3.9(7.1/2)
- designate 3.10(1)
- designated profile
  - of an access-to-subprogram type 3.10(11)
  - of an anonymous access type 3.10(12/2)
- designated subtype
  - of a named access type 3.10(10)
  - of an anonymous access type 3.10(12/2)
- designated type
  - of a named access type 3.10(10)
  - of an anonymous access type 3.10(12/2)
- designator 6.1(5)
  - used* 6.3(2/2), P
- destructor
  - See* finalization 7.6(1)
  - See* finalization 7.6.1(1)
- Detach\_Handler
  - in* Ada.Interrupts C.3.2(9)
- Detect\_Blocking pragma H.5(3/2), L(8.1/2)
- Determinant
  - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(46/2)
  - in* Ada.Numerics.Generic\_Real\_Arrays G.3.1(24/2)
- determined category for a formal type 12.5(6/2)
- determines
  - a type by a subtype\_mark 3.2.2(8)
- Device\_Error
  - in* Ada.Direct\_IO A.8.4(18)
  - in* Ada.Directories A.16(43/2)
  - in* Ada.IO\_Exceptions A.13(4)
  - in* Ada.Sequential\_IO A.8.1(15)
  - in* Ada.Streams.Stream\_IO A.12.1(26)
  - in* Ada.Text\_IO A.10.1(85)
- Diaeresis
  - in* Ada.Characters.Latin\_1 A.3.3(21/3)
- Difference
  - in* Ada.Calendar.Arithmetic 9.6.1(12/2)
  - in* Ada.Containers.Hashed\_Sets A.18.8(32/2), A.18.8(33/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(33/2), A.18.9(34/2)
- digit 2.4.1(4.1/2)
  - used* 2.4.1(3), 2.4.2(5), P
- digits
  - of a decimal fixed point subtype 3.5.9(6), 3.5.10(7)
- Digits attribute 3.5.8(2/1), 3.5.10(7)
- digits\_constraint 3.5.9(5)
  - used* 3.2.2(6), P
- dimensionality
  - of an array 3.6(12)
- direct access A.8(3)

- direct file A.8(1/2)
  - Direct\_IO
    - child of* Ada A.8.4(2)
  - direct\_name 4.1(3)
    - used* 3.8.1(2), 4.1(2/3), 5.1(8), 9.5.2(3), 10.2.1(4.2/2), 13.1(3), J.7(1), L(25.2/2), P
  - Direction
    - in* Ada.Strings A.4.1(6)
  - directly specified
    - of an aspect of representation of an entity 13.1(8/3)
    - of an operational aspect of an entity 13.1(8.1/1)
  - directly visible 8.3(2), 8.3(21)
    - within a pragma in a context\_clause 10.1.6(3)
    - within a pragma that appears at the place of a compilation unit 10.1.6(5)
    - within a use\_clause in a context\_clause 10.1.6(3)
    - within a with\_clause 10.1.6(2/2)
    - within the parent\_unit\_name of a library unit 10.1.6(2/2)
    - within the parent\_unit\_name of a subunit 10.1.6(4)
  - Directories
    - child of* Ada A.16(3/2)
  - directory A.16(45/2)
  - directory entry A.16(49/2)
  - directory name A.16(46/2)
  - Directory\_Entry\_Type
    - in* Ada.Directories A.16(29/2)
  - Discard\_Names pragma C.5(3), L(9)
  - discontiguous representation
    - [*partial*] 13.5.2(5), 13.7.1(12), 13.9(9), 13.9(17), 13.11(21.6/3)
  - discrete array type 4.5.2(1)
  - discrete type 3.2(3), 3.5(1), N(14)
  - discrete\_choice 3.8.1(5)
    - used* 3.8.1(4), P
  - discrete\_choice\_list 3.8.1(4)
    - used* 3.8.1(3), 4.3.3(5/2), 5.4(3), P
  - Discrete\_Random
    - child of* Ada.Numerics A.5.2(17)
  - discrete\_range 3.6.1(3)
    - used* 3.6.1(2), 3.8.1(5), 4.1.2(2), P
  - discrete\_subtype\_definition 3.6(6)
    - used* 3.6(5), 5.5(4), 9.5.2(2/2), 9.5.2(8), P
  - discriminant 3.2(5/2), 3.7(1/2), N(15/2)
    - of a variant\_part 3.8.1(6)
    - use in a record definition 3.8(12)
  - discriminant\_association 3.7.1(3)
    - used* 3.7.1(2), P
  - Discriminant\_Check 11.5(12)
    - [*partial*] 4.1.3(15), 4.3(6), 4.3.2(8), 4.6(43), 4.6(45), 4.6(51/2), 4.6(52), 4.7(4), 4.8(10/2), 6.5(5.9/3)
  - discriminant\_constraint 3.7.1(2)
    - used* 3.2.2(7), P
  - discriminant\_part 3.7(2/2)
    - used* 3.10.1(2/2), 7.3(2), 7.3(3/2), 12.5(2), P
  - discriminant\_specification 3.7(5/2)
    - used* 3.7(4), P
  - discriminants
    - known 3.7(26)
    - unknown 3.7(26)
  - discriminated type 3.7(8/2)
  - dispatching 3.9(3)
    - child of* Ada D.2.1(1.2/3)
  - dispatching call
    - on a dispatching operation 3.9.2(1/2)
  - dispatching operation 3.9.2(1/2), 3.9.2(2/3)
    - [*partial*] 3.9(1)
  - dispatching point D.2.1(4/2)
    - [*partial*] D.2.3(8/2), D.2.4(9/3)
  - dispatching policy for tasks
    - [*partial*] D.2.1(5/2)
  - dispatching, task D.2.1(4/2)
  - Dispatching\_Policy\_Error
    - in* Ada.Dispatching D.2.1(1.4/3)
  - Display\_Format
    - in* Interfaces.COBOL B.4(22)
  - displayed magnitude (of a decimal value) F.3.2(14)
  - disruption of an assignment 9.8(21), 13.9.1(5)
    - [*partial*] 11.6(6)
  - distinct access paths 6.2(12)
  - distributed program E(3)
  - distributed system E(2)
  - distributed systems C(1)
  - divide 2.1(15/2)
    - in* Ada.Decimal F.2(6)
  - divide operator 4.4(1), 4.5.5(1)
  - Division\_Check 11.5(13/2)
    - [*partial*] 3.5.4(20), 4.5.5(22), A.5.1(28), A.5.3(47), G.1.1(40), G.1.2(28), K(202)
  - Division\_Sign
    - in* Ada.Characters.Latin\_1 A.3.3(26)
  - DLE
    - in* Ada.Characters.Latin\_1 A.3.3(6)
  - Do\_APC
    - in* System.RPC E.5(10)
  - Do\_RPC
    - in* System.RPC E.5(9)
  - documentation (required of an implementation) 1.1.3(18), M.1(1/2), M.2(1/2), M.3(1/2)
  - documentation requirements 1.1.2(34), M(1/2)
    - summary of requirements M.1(1/2)
  - Dollar\_Sign
    - in* Ada.Characters.Latin\_1 A.3.3(8)
  - dot 2.1(15/2)
  - dot selection
    - See* selected\_component 4.1.3(1)
  - double
    - in* Interfaces.C B.3(16)
  - Double\_Precision
    - in* Interfaces.Fortran B.5(6)
  - Doubly\_Linked\_Lists
    - child of* Ada.Containers A.18.3(5/3)
  - downward closure 3.10.2(37/2)
  - drift rate D.8(41)
  - Duration
    - in* Standard A.1(43)
  - dynamic binding
    - See* dispatching operation 3.9(1)
  - dynamic semantics 1.1.2(30)
  - Dynamic\_Priorities
    - child of* Ada D.5.1(3/2)
  - dynamically determined tag 3.9.2(1/2)
  - dynamically enclosing
    - of one execution by another 11.4(2)
  - dynamically tagged 3.9.2(5/2)
- E**
- e
    - in* Ada.Numerics A.5(3/2)
  - EDF
    - child of* Ada.Dispatching D.2.6(9/2)
    - child of*
      - Ada.Synchronous\_Task\_Control D.10(5.2/3)
  - EDF\_Across\_Priorities task dispatching policy D.2.6(7/2)
  - edited output F.3(1/2)
  - Editing
    - child of* Ada.Text\_IO F.3.3(3)
    - child of* Ada.Wide\_Text\_IO F.3.4(1)
    - child of* Ada.Wide\_Wide\_Text\_IO F.3.5(1/2)
  - effect
    - external 1.1.3(8)
  - efficiency 11.5(29), 11.6(1)
  - Eigensystem
    - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(49/2)
    - in* Ada.Numerics.Generic\_Real\_Arrays G.3.1(27/2)
  - Eigenvalues
    - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(48/2)
    - in* Ada.Numerics.Generic\_Real\_Arrays G.3.1(26/2)
  - Elaborate pragma 10.2.1(20), L(10)
  - Elaborate\_All pragma 10.2.1(21), L(11)
  - Elaborate\_Body pragma 10.2.1(22), L(12)
  - elaborated 3.11(8)
  - elaboration 3.1(11), N(15.1/2), N(19)
    - abstract\_subprogram\_declaration 3.9.3(11.1/2)

- access\_definition 3.10(17/2)
- access\_type\_definition 3.10(16)
- array\_type\_definition 3.6(21)
- aspect\_clause 13.1(19/1)
- choice\_parameter\_specification 11.4(7)
- component\_declaration 3.8(17)
- component\_definition 3.6(22/2), 3.8(18/2)
- component\_list 3.8(17)
- declaration named by a pragma Import B.1(38)
- declarative\_part 3.11(7)
- deferred constant declaration 7.4(10/3)
- delta\_constraint J.3(11)
- derived\_type\_definition 3.4(26)
- digits\_constraint 3.5.9(19)
- discrete\_subtype\_definition 3.6(22/2)
- discriminant\_constraint 3.7.1(12)
- entry\_declaration 9.5.2(22/1)
- enumeration\_type\_definition 3.5.1(10)
- exception\_declaration 11.1(5)
- fixed\_point\_definition 3.5.9(17)
- floating\_point\_definition 3.5.7(13)
- full type definition 3.2.1(11)
- full\_type\_declaration 3.2.1(11)
- generic body 12.2(2)
- generic\_declaration 12.1(10)
- generic\_instantiation 12.3(20)
- incomplete\_type\_declaration 3.10.1(12)
- index\_constraint 3.6.1(8)
- integer\_type\_definition 3.5.4(18)
- loop\_parameter\_specification 5.5(9)
- non-generic subprogram\_body 6.3(6)
- nongeneric package\_body 7.2(6)
- null\_procedure\_declaration 6.7(5/2)
- number\_declaration 3.3.2(7)
- object\_declaration 3.3.1(15)
- of library units for a foreign language main subprogram B.1(39)
- package\_body of Standard A.1(50)
- package\_declaration 7.1(8)
- partition E.1(6)
- partition E.5(21)
- per-object constraint 3.8(18.1/1)
- pragma 2.8(12)
- private\_extension\_declaration 7.3(17)
- private\_type\_declaration 7.3(17)
- protected\_declaration 9.4(12)
- protected\_body 9.4(15)
- protected\_definition 9.4(13)
- range\_constraint 3.5(9)
- real\_type\_definition 3.5.6(5)
- record\_definition 3.8(16)
- record\_extension\_part 3.9.1(5)
- record\_type\_definition 3.8(16)
- renaming\_declaration 8.5(3)
- single\_protected\_declaration 9.4(12)
- single\_task\_declaration 9.1(10)
- Storage\_Size pragma 13.3(66)
- subprogram\_declaration 6.1(31/2)
- subtype\_declaration 3.2.2(9)
- subtype\_indication 3.2.2(9)
- task\_declaration 9.1(10)
- task\_body 9.1(13)
- task\_definition 9.1(11)
- use\_clause 8.4(12)
- variant\_part 3.8.1(22)
- elaboration control 10.2.1(1)
- elaboration dependence
  - library\_item on another 10.2(9)
- Elaboration\_Check 11.5(20)
  - [*partial*] 3.11(9)
- element
  - of a storage pool 13.11(11)
  - in Ada.Containers.Doubly\_Linked\_Lists A.18.3(14/2)
  - in Ada.Containers.Hashed\_Maps A.18.5(14/2), A.18.5(31/2)
  - in Ada.Containers.Hashed\_Sets A.18.8(15/2), A.18.8(52/2)
  - in Ada.Containers.Indefinite\_Holders A.18.18(12/3)
  - in Ada.Containers.Multiway\_Trees A.18.10(22/3)
  - in Ada.Containers.Ordered\_Maps A.18.6(13/2), A.18.6(39/2)
  - in Ada.Containers.Ordered\_Sets A.18.9(14/2), A.18.9(65/2)
  - in Ada.Containers.Vectors A.18.2(27/2), A.18.2(28/2)
  - in Ada.Strings.Bounded A.4.4(26)
  - in Ada.Strings.Unbounded A.4.5(20)
- elementary type 3.2(2/2), N(16)
- Elementary\_Functions
  - child of* Ada.Numerics A.5.1(9/1)
- eligible
  - a type, for a convention B.1(14)
- else part
  - of a selective\_accept 9.7.1(11)
- EM
  - in Ada.Characters.Latin\_1 A.3.3(6)
- embedded systems C(1), D(1)
- empty element
  - of a vector A.18.2(4/2)
- empty holder A.18.18(3/3)
- Empty\_Holder
  - in Ada.Containers.Indefinite\_Holders A.18.18(7/3)
- Empty\_List
  - in Ada.Containers.Doubly\_Linked\_Lists A.18.3(8/2)
- Empty\_Map
  - in Ada.Containers.Hashed\_Maps A.18.5(5/2)
  - in Ada.Containers.Ordered\_Maps A.18.6(6/2)
- Empty\_Set
  - in Ada.Containers.Hashed\_Sets A.18.8(5/2)
  - in Ada.Containers.Ordered\_Sets A.18.9(6/2)
- Empty\_Tree
  - in Ada.Containers.Multiway\_Trees A.18.10(10/3)
- Empty\_Vector
  - in Ada.Containers.Vectors A.18.2(10/2)
- encapsulation
  - See package 7(1)
- enclosing
  - immediately 8.1(13)
- Encode
  - in Ada.Strings.UTF\_Encoding.Wide\_Wide\_Encoding A.4.11(23/3), A.4.11(24/3), A.4.11(25/3), A.4.11(31/3), A.4.11(32/3), A.4.11(33/3)
- Encoding
  - in Ada.Strings.UTF\_Encoding A.4.11(13/3)
- encoding scheme A.4.11(38/3)
- Encoding\_Error
  - in Ada.Strings.UTF\_Encoding A.4.11(8/3)
- Encoding\_Scheme
  - in Ada.Strings.UTF\_Encoding A.4.11(4/3)
- end of a line 2.2(2/2)
- End\_Error
  - raised by failure of run-time check 13.13.2(37/1)
  - in Ada.Direct\_IO A.8.4(18)
  - in Ada.IO\_Exceptions A.13(4)
  - in Ada.Sequential\_IO A.8.1(15)
  - in Ada.Streams.Stream\_IO A.12.1(26)
  - in Ada.Text\_IO A.10.1(85)
- End\_Of\_File
  - in Ada.Direct\_IO A.8.4(16)
  - in Ada.Sequential\_IO A.8.1(13)
  - in Ada.Streams.Stream\_IO A.12.1(12)
  - in Ada.Text\_IO A.10.1(34)
- End\_Of\_Line
  - in Ada.Text\_IO A.10.1(30)
- End\_Of\_Page
  - in Ada.Text\_IO A.10.1(33)
- End\_Search
  - in Ada.Directories A.16(33/2)
- endian
  - big 13.5.3(2)
  - little 13.5.3(2)
- ENQ
  - in Ada.Characters.Latin\_1 A.3.3(5)
- entity
  - [*partial*] 3.1(1)

- entry
  - closed 9.5.3(5)
  - open 9.5.3(5)
  - single 9.5.2(20)
- entry call 9.5.3(1)
  - simple 9.5.3(1)
- entry calling convention 6.3.1(13)
- entry family 9.5.2(20)
- entry index subtype 3.8(18/2), 9.5.2(20)
- entry queue 9.5.3(12)
- entry queuing policy 9.5.3(17)
  - default policy 9.5.3(17)
- entry\_barrier 9.5.2(7)
  - used 9.5.2(5), P
- entry\_body 9.5.2(5)
  - used 9.4(8/1), P
- entry\_body\_formal\_part 9.5.2(6)
  - used 9.5.2(5), P
- entry\_call\_alternative 9.7.2(3/2)
  - used 9.7.2(2), 9.7.3(2), P
- entry\_call\_statement 9.5.3(2)
  - used 5.1(4/2), 9.7.2(3.1/2), P
- entry\_declaration 9.5.2(2/2)
  - used 9.1(5/1), 9.4(5/1), P
- entry\_index 9.5.2(4)
  - used 9.5.2(3), P
- entry\_index\_specification 9.5.2(8)
  - used 9.5.2(6), P
- enumeration literal 3.5.1(6/3)
- enumeration type 3.2(3), 3.5.1(1), N(17)
- enumeration\_aggregate 13.4(3)
  - used 13.4(2), P
- Enumeration\_IO
  - in Ada.Text\_IO A.10.1(79)
- enumeration\_literal\_specification 3.5.1(3)
  - used 3.5.1(2), P
- enumeration\_representation\_clause 13.4(2)
  - used 13.1(2/1), P
- enumeration\_type\_definition 3.5.1(2)
  - used 3.2.1(4/2), P
- environment 10.1.4(1)
- environment\_declarative\_part 10.1.4(1)
  - for the environment task of a partition 10.2(13)
- environment task 10.2(8)
- environment variable A.17(1/2)
- Environment\_Variables
  - child of Ada A.17(3/2)
- EOT
  - in Ada.Characters.Latin\_1 A.3.3(5)
- EPA
  - in Ada.Characters.Latin\_1 A.3.3(18)
- epoch D.8(19)
- equal operator 4.4(1), 4.5.2(1)
- Equal\_Case\_Insensitive
  - child of Ada.Strings A.4.10(2/3)
  - child of Ada.Strings.Bounded A.4.10(7/3)
- child of Ada.Strings.Fixed A.4.10(5/3)
- child of Ada.Strings.Unbounded A.4.10(10/3)
- Equal\_Subtree
  - in Ada.Containers.Multiway\_Trees A.18.10(12/3)
- equality operator 4.5.2(1)
  - special inheritance rule for tagged types 3.4(17/2), 4.5.2(14/3)
- equals sign 2.1(15/2)
- Equals\_Sign
  - in Ada.Characters.Latin\_1 A.3.3(10)
- equivalent element
  - of a hashed set A.18.8(64/2)
  - of an ordered set A.18.9(78/2)
- equivalent key
  - of a hashed map A.18.5(42/2)
  - of an ordered map A.18.6(55/2)
- Equivalent\_Elements
  - in Ada.Containers.Hashed\_Sets A.18.8(46/2), A.18.8(47/2), A.18.8(48/2)
  - in Ada.Containers.Ordered\_Sets A.18.9(3/2)
- Equivalent\_Keys
  - in Ada.Containers.Hashed\_Maps A.18.5(34/2), A.18.5(35/2), A.18.5(36/2)
  - in Ada.Containers.Ordered\_Maps A.18.6(3/2)
  - in Ada.Containers.Ordered\_Sets A.18.9(63/2)
- Equivalent\_Sets
  - in Ada.Containers.Hashed\_Sets A.18.8(8/2)
  - in Ada.Containers.Ordered\_Sets A.18.9(9/2)
- erroneous execution 1.1.2(32), 1.1.5(10)
  - cause 3.7.2(4), 3.9(25.3/2), 6.4.1(17.1/3), 9.8(21), 9.10(11), 11.5(26), 13.3(13/3), 13.3(27), 13.3(28/2), 13.9.1(8), 13.9.1(12/2), 13.9.1(13/3), 13.11(21), 13.11.2(16/3), 13.13.2(53/2), A.10.3(22/1), A.12.1(36.1/1), A.13(17), A.17(28/2), A.18.2(252/2), A.18.3(157/2), A.18.4(80/2), A.18.7(101/2), B.1(38.1/2), B.3.1(51), B.3.1(55), B.3.1(56), B.3.1(57), B.3.2(35), B.3.2(36), B.3.2(37), B.3.2(38), B.3.2(39), B.3.2(42), C.3.1(14), C.3.1(14.1/1), C.7.1(18), C.7.2(14), C.7.2(15), C.7.2(15.1/2), D.2.6(31/2), D.5.1(12), D.11(9), D.14(19/2), D.14.1(25/2), D.14.2(35/2), H.4(26), H.4(27)
- error
  - compile-time 1.1.2(27), 1.1.5(4)
  - link-time 1.1.2(29), 1.1.5(4)
- run-time 1.1.2(30), 1.1.5(6), 11.5(2), 11.6(1)
  - See also bounded error, erroneous execution
- ESA
  - in Ada.Characters.Latin\_1 A.3.3(17)
- ESC
  - in Ada.Characters.Latin\_1 A.3.3(6)
- Establish\_RPC\_Receiver
  - in System.RPC E.5(12)
- ETB
  - in Ada.Characters.Latin\_1 A.3.3(6)
- ETX
  - in Ada.Characters.Latin\_1 A.3.3(5)
- evaluation 3.1(11), N(17.1/2), N(19)
  - aggregate 4.3(5)
  - allocator 4.8(7/2)
  - array\_aggregate 4.3.3(21)
  - attribute\_reference 4.1.4(11)
  - concatenation 4.5.3(5)
  - dereference 4.1(13)
  - discrete\_range 3.6.1(8)
  - extension\_aggregate 4.3.2(7)
  - generic\_association 12.3(21)
  - generic\_association for a formal object of mode in 12.4(11)
  - indexed\_component 4.1.1(7)
  - initialized\_allocator 4.8(7/2)
  - membership test 4.5.2(27)
  - name 4.1(11/2)
  - name that has a prefix 4.1(12)
  - null literal 4.2(9)
  - numeric literal 4.2(9)
  - parameter\_association 6.4.1(7)
  - prefix 4.1(12)
  - primary that is a name 4.4(10)
  - qualified\_expression 4.7(4)
  - range 3.5(9)
  - range\_attribute\_reference 4.1.4(11)
  - record\_aggregate 4.3.1(18)
  - record\_component\_association\_list 4.3.1(19)
  - selected\_component 4.1.3(14)
  - short-circuit control form 4.5.1(7)
  - slice 4.1.2(7)
  - string\_literal 4.2(10)
  - uninitialized\_allocator 4.8(8)
  - Val 3.5.5(7), K(261)
  - Value 3.5(55/2)
  - value conversion 4.6(28)
  - view conversion 4.6(52)
  - Wide\_Value 3.5(43/2)
  - Wide\_Wide\_Value 3.5(39.4/2)
- Exception 11(1), 11.1(1), N(18)
- exception occurrence 11(1)
- exception\_choice 11.2(5)
  - used 11.2(3), P
- exception\_declaration 11.1(2)
  - used 3.1(3/2), P
- exception\_handler 11.2(3)

- used* 11.2(2), P
- Exception\_Id
  - in* Ada.Exceptions 11.4.1(2/2)
- Exception\_Identity
  - in* Ada.Exceptions 11.4.1(5/2)
- Exception\_Information
  - in* Ada.Exceptions 11.4.1(5/2)
- Exception\_Message
  - in* Ada.Exceptions 11.4.1(4/2)
- Exception\_Name
  - in* Ada.Exceptions 11.4.1(2/2), 11.4.1(5/2)
- Exception\_Occurrence
  - in* Ada.Exceptions 11.4.1(3/2)
- Exception\_Occurrence\_Access
  - in* Ada.Exceptions 11.4.1(3/2)
- exception\_renaming\_declaration 8.5.2(2)
  - used* 8.5(2), P
- Exceptions
  - child of* Ada 11.4.1(2/2)
- Exchange\_Handler
  - in* Ada.Interrupts C.3.2(8)
- Exclamation
  - in* Ada.Characters.Latin\_1 A.3.3(8)
- exclamation point 2.1(15/2)
- Exclude
  - in* Ada.Containers.Hashed\_Maps A.18.5(24/2)
  - in* Ada.Containers.Hashed\_Sets A.18.8(23/2), A.18.8(54/2)
  - in* Ada.Containers.Ordered\_Maps A.18.6(23/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(22/2), A.18.9(67/2)
- excludes null
  - subtype 3.10(13.1/2)
- execution 3.1(11), N(19)
  - abort\_statement 9.8(4)
  - aborting the execution of a construct 9.8(5)
  - accept\_statement 9.5.2(24)
  - Ada program 9(1)
  - assignment\_statement 5.2(7), 7.6(17), 7.6.1(12/2)
  - asynchronous\_select with a delay\_statement\_trigger 9.7.4(7)
  - asynchronous\_select with a procedure call\_trigger 9.7.4(6/2)
  - asynchronous\_select with an entry call\_trigger 9.7.4(6/2)
  - block\_statement 5.6(5)
  - call on a dispatching operation 3.9.2(14)
  - call on an inherited subprogram 3.4(27/2)
  - case\_statement 5.4(11)
  - conditional\_entry\_call 9.7.3(3)
  - delay\_statement 9.6(20)
  - dynamically enclosing 11.4(2)
  - entry\_body 9.5.2(26)
  - entry\_call\_statement 9.5.3(8)
  - exit\_statement 5.7(5)
  - extended\_return\_statement 6.5(5.9/3)
  - goto\_statement 5.8(5)
  - handled\_sequence\_of\_statements 11.2(10)
  - handler 11.4(7)
  - if\_statement 5.3(5)
  - instance of Unchecked\_Deallocation 7.6.1(10)
  - loop\_statement 5.5(7)
  - loop\_statement with a for iteration\_scheme 5.5(9)
  - loop\_statement with a while iteration\_scheme 5.5(8)
  - null\_statement 5.1(13)
  - partition 10.2(25)
  - pragma 2.8(12)
  - program 10.2(25)
  - protected\_subprogram\_call 9.5.1(3)
  - raise\_statement with an exception\_name 11.3(4/2)
  - re-raise statement 11.3(4/2)
  - remote\_subprogram\_call E.4(9)
  - requeue\_protected\_entry 9.5.4(9)
  - requeue\_task\_entry 9.5.4(8)
  - requeue\_statement 9.5.4(7/3)
  - selective\_accept 9.7.1(15)
  - sequence\_of\_statements 5.1(15)
  - simple\_return\_statement 6.5(6/2)
  - subprogram\_call 6.4(10/2)
  - subprogram\_body 6.3(7)
  - task 9.2(1)
  - task\_body 9.2(1)
  - timed\_entry\_call 9.7.2(4/2)
- execution resource
  - associated with a protected object 9.4(18)
  - required for a task to run 9(10)
- execution time
  - of a task D.14(11/2)
- Execution\_Time
  - child of* Ada D.14(3/2)
- exhaust
  - a budget D.14.2(14/2)
- exist
  - cease to 7.6.1(11/2), 13.11.2(10/2)
- Exists
  - in* Ada.Directories A.16(24/2)
  - in* Ada.Environment\_Variables A.17(5/2)
- exit\_statement 5.7(2)
  - used* 5.1(4/2), P
- Exit\_Status
  - in* Ada.Command\_Line A.15(7)
- Exp
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(3)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(4)
- expanded name 4.1.3(4)
- Expanded\_Name
  - in* Ada.Tags 3.9(7/2)
- expected profile 8.6(26)
  - accept\_statement entry\_direct\_name 9.5.2(11)
- Access attribute\_reference prefix 3.10.2(2.3/2)
- attribute\_definition\_clause name 13.3(4)
- character\_literal 4.2(3)
- formal\_subprogram\_actual 12.6(6)
- formal\_subprogram\_default\_name 12.6(5)
- subprogram\_renaming\_declaration 8.5.4(3)
- expected type 8.6(20/2)
  - abort\_statement task\_name 9.8(3)
  - access attribute\_reference 3.10.2(2/2)
- Access attribute\_reference prefix 3.10.2(2.3/2)
- actual parameter 6.4.1(3)
- aggregate 4.3(3/2)
- allocator 4.8(3/1)
- array\_aggregate 4.3.3(7/2)
- array\_aggregate component expression 4.3.3(7/2)
- array\_aggregate discrete\_choice 4.3.3(8)
- assignment\_statement expression 5.2(4/2)
- assignment\_statement variable\_name 5.2(4/2)
- attribute\_definition\_clause expression or name 13.3(4)
- attribute\_designator expression 4.1.4(7)
- case expression 5.4(4)
- case\_statement\_alternative discrete\_choice 5.4(4)
- character\_literal 4.2(3)
- code\_statement 13.8(4)
- component\_clause expressions 13.5.1(7)
- component\_declaration
  - default\_expression 3.8(7)
- condition 5.3(4)
- decimal fixed point type digits 3.5.9(6)
- delay\_relative\_statement expression 9.6(5)
- delay\_until\_statement expression 9.6(5)
- delta\_constraint expression J.3(3)
- dereference name 4.1(8)
- discrete\_subtype\_definition range 3.6(8)
- discriminant default\_expression 3.7(7)
- discriminant\_association expression 3.7.1(6)
- entry\_index 9.5.2(11)

- enumeration\_representation\_clause
    - expressions 13.4(4)
  - expression of
    - extended\_return\_statement 6.5(3/2)
  - expression of simple\_return\_statement 6.5(3/2)
  - extension\_aggregate 4.3.2(4/2)
  - extension\_aggregate\_ancestor
    - expression 4.3.2(4/2)
  - first\_bit 13.5.1(7)
  - fixed point type delta 3.5.9(6)
  - generic formal in object actual 12.4(4)
  - generic formal object
    - default\_expression 12.4(3)
  - index\_constraint discrete\_range 3.6.1(4)
  - indexed\_component expression 4.1.1(4)
  - Interrupt\_Priority pragma argument D.1(6)
  - last\_bit 13.5.1(7)
  - link\_name B.1(10)
  - membership test simple\_expression 4.5.2(3/2)
  - modular\_type\_definition expression 3.5.4(5)
  - number\_declaration expression 3.3.2(3)
  - object\_declaration initialization
    - expression 3.3.1(4)
  - parameter default\_expression 6.1(17)
  - position 13.5.1(7)
  - Priority pragma argument D.1(6)
  - range simple\_expressions 3.5(5)
  - range\_attribute\_designator expression 4.1.4(7)
  - range\_constraint range 3.5(5)
  - real\_range\_specification bounds 3.5.7(5)
  - record\_aggregate 4.3.1(8/2)
  - record\_component\_association
    - expression 4.3.1(10)
  - requested decimal precision 3.5.7(4)
  - restriction parameter expression 13.12(5)
  - short-circuit control form relation 4.5.1(1)
  - signed\_integer\_type\_definition
    - simple\_expression 3.5.4(5)
  - slice discrete\_range 4.1.2(4)
  - Storage\_Size pragma argument 13.3(65)
  - string\_literal 4.2(4)
  - type\_conversion operand 4.6(6)
  - variant\_part discrete\_choice 3.8.1(6)
  - expiration time
    - [partial] 9.6(1)
    - for a delay\_relative\_statement 9.6(20)
    - for a delay\_until\_statement 9.6(20)
  - expires
    - execution timer D.14.1(15/2)
  - explicit declaration 3.1(5), N(11)
  - explicit initial value 3.3.1(1)
  - explicit\_actual\_parameter 6.4(6)
    - used 6.4(5), P
  - explicit\_dereference 4.1(5)
    - used 4.1(2/3), P
  - explicit\_generic\_actual\_parameter 12.3(5)
    - used 12.3(4), P
  - explicitly assign 10.2(2)
  - explicitly limited record 3.8(13.1/3)
  - exponent 2.4.1(4), 4.5.6(11/3)
    - used 2.4.1(2), 2.4.2(2), P
  - Exponent attribute A.5.3(18)
  - exponentiation operator 4.4(1), 4.5.6(7)
  - Export pragma B.1(6), L(13)
  - exported
    - aspect of representation B.1(28)
  - exported entity B.1(23)
  - expression 4.4(1), 4.4(2)
    - used 2.8(3), 3.3.1(2/2), 3.3.2(2), 3.5.4(4), 3.5.7(2), 3.5.9(3), 3.5.9(4), 3.5.9(5), 3.7(6), 3.7.1(3), 3.8.1(5), 4.1.1(2), 4.1.4(3/2), 4.1.4(5), 4.3.1(4/2), 4.3.2(3), 4.3.3(3/2), 4.3.3(5/2), 4.4(7/3), 4.5.7(3/3), 4.6(2), 4.7(2), 5.2(2), 5.3(3), 5.4(2), 6.4(6), 6.5(2.1/3), 6.5(2/2), 9.5.2(4), 9.6(3), 9.6(4), 11.3(2/2), 11.4.2(3/2), 12.3(5), 13.3(2), 13.3(63), 13.5.1(4), 13.12(4.1/2), B.1(5), B.1(6), B.1(8), B.1(10), C.3.1(4), D.1(3), D.1(5), D.2.2(2.2/2), D.2.6(4/2), J.3(2), J.7(1), J.8(1), L(2.1/2), L(6), L(13), L(14), L(18), L(19), L(27), L(27.1/2), L(29.1/2), L(35), P
  - extended\_digit 2.4.2(5)
    - used 2.4.2(4), P
  - Extended\_Index\_subtype of
    - Index\_Type'Base
      - in Ada.Containers.Vectors A.18.2(7/2)
  - extended\_return\_statement 6.5(2.1/3)
    - used 5.1(5/2), P
  - extension
    - of a private type 3.9(2.1/2), 3.9.1(1/2)
    - of a record type 3.9(2.1/2), 3.9.1(1/2)
    - of a type 3.9(2/2), 3.9.1(1/2)
    - in Ada.Directories A.16(18/2)
  - extension\_aggregate 4.3.2(2)
    - used 4.3(2), P
  - external call 9.5(4)
  - external effect
    - of the execution of an Ada program 1.1.3(8)
    - volatile/atomic objects C.6(20)
  - external file A.7(1)
  - external interaction 1.1.3(8)
  - external name B.1(34)
  - external\_requeue 9.5(7)
  - external streaming
    - type supports 13.13.2(52/3)
  - External\_Tag
    - in Ada.Tags 3.9(7/2)
  - External\_Tag attribute 13.3(75/3)
  - External\_Tag clause 13.3(7/2), 13.3(75/3), K(65)
  - extra permission to avoid raising exceptions 11.6(5)
  - extra permission to reorder actions 11.6(6)
- F**
- factor 4.4(6)
    - used 4.4(5), P
  - factory 3.9(30.1/2)
  - failure
    - of a language-defined check 11.5(2)
    - in Ada.Command\_Line A.15(8)
  - fall-back handler C.7.3(9/2)
  - False 3.5.3(1)
  - family
    - entry 9.5.2(20)
  - Feminine\_Ordinal\_Indicator
    - in Ada.Characters.Latin\_1 A.3.3(21/3)
  - FF
    - in Ada.Characters.Latin\_1 A.3.3(5)
  - Field\_subtype of Integer
    - in Ada.Text\_IO A.10.1(6)
  - FIFO\_Queueing queueing policy D.4(7/2)
  - FIFO\_Within\_Priorities task dispatching policy D.2.3(2/2)
  - file
    - as file object A.7(2)
    - file name A.16(46/2)
    - file terminator A.10(7)
  - File\_Access
    - in Ada.Text\_IO A.10.1(18)
  - File\_Kind
    - in Ada.Directories A.16(22/2)
  - File\_Mode
    - in Ada.Direct\_IO A.8.4(4)
    - in Ada.Sequential\_IO A.8.1(4)
    - in Ada.Streams.Stream\_IO A.12.1(6)
    - in Ada.Text\_IO A.10.1(4)
  - File\_Size
    - in Ada.Directories A.16(23/2)
  - File\_Type
    - in Ada.Direct\_IO A.8.4(3)
    - in Ada.Sequential\_IO A.8.1(3)
    - in Ada.Streams.Stream\_IO A.12.1(5)
    - in Ada.Text\_IO A.10.1(3)
  - Filter\_Type
    - in Ada.Directories A.16(30/2)
  - finalization
    - of a master 7.6.1(4)
    - of a protected object 9.4(20)
    - of a protected object C.3.1(12/1)

- of a task object J.7.1(8)
- of an object 7.6.1(5)
- of environment task for a foreign language main subprogram B.1(39)
- child of Ada* 7.6(4/1)
- finalization of the collection 7.6.1(11/2)
- Finalize 7.6(2)
  - in Ada.Finalization* 7.6(6/2), 7.6(8/2)
- Find
  - in Ada.Containers.Doubly\_Linked\_Lists* A.18.3(41/2)
  - in Ada.Containers.Hashed\_Maps* A.18.5(30/2)
  - in Ada.Containers.Hashed\_Sets* A.18.8(43/2), A.18.8(56/2)
  - in Ada.Containers.Multiway\_Trees* A.18.10(32/3)
  - in Ada.Containers.Ordered\_Maps* A.18.6(38/2)
  - in Ada.Containers.Ordered\_Sets* A.18.9(49/2), A.18.9(69/2)
  - in Ada.Containers.Vectors* A.18.2(68/2)
- Find\_In\_Subtree
  - in Ada.Containers.Multiway\_Trees* A.18.10(33/3)
- Find\_Index
  - in Ada.Containers.Vectors* A.18.2(67/2)
- Find\_Token
  - in Ada.Strings.Bounded* A.4.4(50.1/3), A.4.4(51)
  - in Ada.Strings.Fixed* A.4.3(15.1/3), A.4.3(16)
  - in Ada.Strings.Unbounded* A.4.5(45.1/3), A.4.5(46)
- Fine\_Delta
  - in System* 13.7(9)
- First
  - in Ada.Containers.Doubly\_Linked\_Lists* A.18.3(33/2)
  - in Ada.Containers.Hashed\_Maps* A.18.5(27/2)
  - in Ada.Containers.Hashed\_Sets* A.18.8(40/2)
  - in Ada.Containers.Ordered\_Maps* A.18.6(28/2)
  - in Ada.Containers.Ordered\_Sets* A.18.9(41/2)
  - in Ada.Containers.Vectors* A.18.2(58/2)
- First attribute 3.5(12), 3.6.2(3)
- first element
  - of a hashed set A.18.8(68/2)
  - of an ordered set A.18.9(81/2)
  - of a set A.18.7(6/2)
- first node
  - of a hashed map A.18.5(46/2)
  - of a map A.18.4(6/2)
  - of an ordered map A.18.6(58/2)
- first subtype 3.2.1(6), 3.4.1(5)
- First(N) attribute 3.6.2(4)
- first\_bit 13.5.1(5)
  - used* 13.5.1(3), P
- First\_Bit attribute 13.5.2(3/2)
- First\_Child
  - in Ada.Containers.Multiway\_Trees* A.18.10(53/3)
- First\_Child\_Element
  - in Ada.Containers.Multiway\_Trees* A.18.10(54/3)
- First\_Element
  - in Ada.Containers.Doubly\_Linked\_Lists* A.18.3(34/2)
  - in Ada.Containers.Ordered\_Maps* A.18.6(29/2)
  - in Ada.Containers.Ordered\_Sets* A.18.9(42/2)
  - in Ada.Containers.Vectors* A.18.2(59/2)
- First\_Index
  - in Ada.Containers.Vectors* A.18.2(57/2)
- First\_Key
  - in Ada.Containers.Ordered\_Maps* A.18.6(30/2)
- Fixed
  - child of Ada.Strings* A.4.3(5)
- fixed point type 3.5.9(1)
- Fixed\_IO
  - in Ada.Text\_IO* A.10.1(68)
- fixed\_point\_definition 3.5.9(2)
  - used* 3.5.6(2), P
- Float 3.5.7(12), 3.5.7(14)
  - in Standard* A.1(21)
- Float\_IO
  - in Ada.Text\_IO* A.10.1(63)
- Float\_Random
  - child of Ada.Numerics* A.5.2(5)
- Float\_Text\_IO
  - child of Ada* A.10.9(33)
- Float\_Wide\_Text\_IO
  - child of Ada* A.11(2/2)
- Float\_Wide\_Wide\_Text\_IO
  - child of Ada* A.11(3/2)
- Floating
  - in Interfaces.COBOL* B.4(9)
- floating point type 3.5.7(1)
- floating\_point\_definition 3.5.7(2)
  - used* 3.5.6(2), P
- Floor
  - in Ada.Containers.Ordered\_Maps* A.18.6(40/2)
  - in Ada.Containers.Ordered\_Sets* A.18.9(50/2), A.18.9(70/2)
- Floor attribute A.5.3(30)
- Flush
  - in Ada.Streams.Stream\_IO* A.12.1(25/1)
  - in Ada.Text\_IO* A.10.1(21/1)
- Fore attribute 3.5.10(4)
- form
  - of an external file A.7(1)
  - in Ada.Direct\_IO* A.8.4(9)
  - in Ada.Sequential\_IO* A.8.1(9)
  - in Ada.Streams.Stream\_IO* A.12.1(11)
  - in Ada.Text\_IO* A.10.1(12)
- formal object, generic 12.4(1)
- formal package, generic 12.7(1)
- formal parameter
  - of a subprogram 6.1(17)
- formal subprogram, generic 12.6(1)
- formal subtype 12.5(5)
- formal type 12.5(5)
- formal\_abstract\_subprogram\_declaration 12.6(2.2/2)
  - used* 12.6(2/2), P
- formal\_access\_type\_definition 12.5.4(2)
  - used* 12.5(3/2), P
- formal\_array\_type\_definition 12.5.3(2)
  - used* 12.5(3/2), P
- formal\_concrete\_subprogram\_declaration 12.6(2.1/2)
  - used* 12.6(2/2), P
- formal\_decimal\_fixed\_point\_definition 12.5.2(7)
  - used* 12.5(3/2), P
- formal\_derived\_type\_definition 12.5.1(3/2)
  - used* 12.5(3/2), P
- formal\_discrete\_type\_definition 12.5.2(2)
  - used* 12.5(3/2), P
- formal\_floating\_point\_definition 12.5.2(5)
  - used* 12.5(3/2), P
- formal\_interface\_type\_definition 12.5.5(2/2)
  - used* 12.5(3/2), P
- formal\_modular\_type\_definition 12.5.2(4)
  - used* 12.5(3/2), P
- formal\_object\_declaration 12.4(2/3)
  - used* 12.1(6), P
- formal\_ordinary\_fixed\_point\_definition 12.5.2(6)
  - used* 12.5(3/2), P
- formal\_package\_actual\_part 12.7(3/2)
  - used* 12.7(2), P
- formal\_package\_association 12.7(3.1/2)
  - used* 12.7(3/2), P
- formal\_package\_declaration 12.7(2)
  - used* 12.1(6), P
- formal\_part 6.1(14)
  - used* 6.1(12), 6.1(13/2), P
- formal\_private\_type\_definition 12.5.1(2)
  - used* 12.5(3/2), P
- formal\_signed\_integer\_type\_definition 12.5.2(3)
  - used* 12.5(3/2), P



- formal\_subprogram\_declaration 12.6(2/2)  
   *used* 12.1(6), P  
 formal\_type\_declaration 12.5(2)  
   *used* 12.1(6), P  
 formal\_type\_definition 12.5(3/2)  
   *used* 12.5(2), P  
 format\_effector 2.1(13/2)  
 Formatting  
   *child of* Ada.Calendar 9.6.1(15/2)  
 Fortran  
   *child of* Interfaces B.5(4)  
 Fortran interface B.5(1)  
 Fortran standard 1.2(3/2)  
 Fortran\_Character  
   *in* Interfaces.Fortran B.5(12)  
 Fortran\_Integer  
   *in* Interfaces.Fortran B.5(5)  
 Fraction attribute A.5.3(21)  
 Fraction\_One\_Half  
   *in* Ada.Characters.Latin\_1 A.3.3(22)  
 Fraction\_One\_Quarter  
   *in* Ada.Characters.Latin\_1 A.3.3(22)  
 Fraction\_Three\_Quarters  
   *in* Ada.Characters.Latin\_1 A.3.3(22)  
 Free  
   *in* Ada.Strings.Unbounded A.4.5(7)  
   *in* Interfaces.C.Strings B.3.1(11)  
 freed  
   *See* nonexistent 13.11.2(10/2)  
 freeing storage 13.11.2(1)  
 freezing  
   by a constituent of a construct 13.14(4/1)  
   by an expression 13.14(8/1)  
   by an implicit call 13.14(8.1/3)  
   by an object name 13.14(8/1)  
   class-wide type caused by the freezing of the specific type 13.14(15)  
   constituents of a full type definition 13.14(15)  
   designated subtype caused by an allocator 13.14(13)  
   entity 13.14(2)  
   entity caused by a body 13.14(3/3)  
   entity caused by a construct 13.14(4/1)  
   entity caused by a name 13.14(11)  
   entity caused by the end of an enclosing construct 13.14(3/3)  
   first subtype caused by the freezing of the type 13.14(15)  
   generic\_instantiation 13.14(5)  
   nominal subtype caused by a name 13.14(11)  
   object\_declaration 13.14(6)  
   profile 13.14(2.1/3)  
   profile of a callable entity by an instantiation 13.14(10.2/3)  
   profile of a function call 13.14(10.1/3)  
   specific type caused by the freezing of the class-wide type 13.14(15)  
   subtype caused by a record extension 13.14(7)  
   subtype caused by an implicit conversion 13.14(8.2/1)  
   subtype caused by an implicit dereference 13.14(11.1/1)  
   subtypes of the profile of a callable entity 13.14(14/3)  
   type caused by a range 13.14(12)  
   type caused by an expression 13.14(10)  
   type caused by the freezing of a subtype 13.14(15)  
 freezing points  
   entity 13.14(2)  
 Friday  
   *in* Ada.Calendar.Formatting 9.6.1(17/2)  
 FS  
   *in* Ada.Characters.Latin\_1 A.3.3(6)  
 full conformance  
   for discrete\_subtype\_definitions 6.3.1(24)  
   for expressions 6.3.1(19)  
   for known\_discriminant\_parts 6.3.1(23)  
   for profiles 6.3.1(18/3)  
   required 3.10.1(4/3), 6.3(4), 7.3(9), 8.3(12.3/2), 8.5.4(5/1), 9.5.2(14), 9.5.2(16), 9.5.2(17), 10.1.3(11), 10.1.3(12)  
   full constant declaration 3.3.1(6)  
   corresponding to a formal object of mode in 12.4(10/2)  
   full declaration 7.4(2)  
   full name  
     of a file A.16(47/2)  
   full stop 2.1(15/2)  
   full type 3.2.1(8/2)  
   full type definition 3.2.1(8/2)  
   full view  
     of a type 3.2.1(8/2)  
 Full\_Name  
   *in* Ada.Directories A.16(15/2), A.16(39/2)  
 Full\_Stop  
   *in* Ada.Characters.Latin\_1 A.3.3(8)  
 full\_type\_declaration 3.2.1(3)  
   *used* 3.2.1(2), P  
 function 6(1), N(19.1/2)  
   with a controlling access result 3.9.2(2/3)  
   with a controlling result 3.9.2(2/3)  
 function instance 12.3(13)  
 function\_call 6.4(3)  
   *used* 4.1(2/3), P  
 function\_specification 6.1(4.2/2)  
   *used* 6.1(4/2), P
- G**  
 garbage collection 13.11.3(6)  
 general access type 3.10(7/1), 3.10(8)  
 general\_access\_modifier 3.10(4)  
   *used* 3.10(3), P  
 generation  
   of an interrupt C.3(2)  
 Generator  
   *in* Ada.Numerics.Discrete\_Random A.5.2(19)  
   *in* Ada.Numerics.Float\_Random A.5.2(7)  
 generic actual 12.3(7/3)  
 generic actual parameter 12.3(7/3)  
 generic actual subtype 12.5(4)  
 generic actual type 12.5(4)  
 generic body 12.2(1)  
 generic contract issue 10.2.1(10/2) [partial] 3.4(5.1/3), 3.7(10/3), 3.7.1(7/3), 3.9.1(3/2), 3.9.4(17/2), 3.10.2(28.1/3), 3.10.2(32/2), 4.5.2(9.8/3), 4.6(24.17/3), 4.6(24.21/2), 4.8(5.5/3), 4.9(37/2), 6.5.1(6/2), 7.3(8), 8.3(26/2), 8.3.1(7/2), 8.5.1(4.6/2), 8.5.1(5/3), 8.5.4(4.3/2), 9.1(9.9/2), 9.4(11.13/2), 9.4(11.8/2), 9.5(17/3), 9.5.2(13.4/2), 10.2.1(11.7/3), 10.2.1(11/3), 10.2.1(17/3), 12.4(8.5/2), 12.6(8.3/2), 13.11.2(3.1/3), B.3.3(10/3), C.3.1(7/3)  
 generic formal 12.1(9)  
 generic formal object 12.4(1)  
 generic formal package 12.7(1)  
 generic formal subprogram 12.6(1)  
 generic formal subtype 12.5(5)  
 generic formal type 12.5(5)  
 generic function 12.1(8/2)  
 generic package 12.1(8/2)  
 generic procedure 12.1(8/2)  
 generic subprogram 12.1(8/2)  
 generic unit 12(1), N(20)  
   *See also* dispatching operation 3.9(1)  
 generic\_actual\_part 12.3(3)  
   *used* 12.3(2/2), 12.7(3/2), P  
 Generic\_Array\_Sort  
   *child of* Ada.Containers A.18.26(3/2)  
 generic\_association 12.3(4)  
   *used* 12.3(3), 12.7(3.1/2), P  
 Generic\_Bounded\_Length  
   *in* Ada.Strings.Bounded A.4.4(4)  
 Generic\_Complex\_Arrays  
   *child of* Ada.Numerics G.3.2(2/2)  
 Generic\_Complex\_Elementary\_Functions  
   *child of* Ada.Numerics G.1.2(2/2)  
 Generic\_Complex\_Types  
   *child of* Ada.Numerics G.1.1(2/1)  
 Generic\_Constrained\_Array\_Sort  
   *child of* Ada.Containers A.18.26(7/2)  
 generic\_declaration 12.1(2)

*used* 3.1(3/2), 10.1.1(5), P  
 Generic\_Dispatching\_Constructor  
   *child of* Ada.Tags 3.9(18.2/2)  
 Generic\_Elementary\_Functions  
   *child of* Ada.Numerics A.5.1(3)  
 generic\_formal\_parameter\_declaration  
   12.1(6)  
   *used* 12.1(5), P  
 generic\_formal\_part 12.1(5)  
   *used* 12.1(3), 12.1(4), P  
 generic\_instantiation 12.3(2/2)  
   *used* 3.1(3/2), 10.1.1(5), P  
 Generic\_Keys  
   *in* Ada.Containers.Hashed\_Sets  
     A.18.8(50/2)  
   *in* Ada.Containers.Ordered\_Sets  
     A.18.9(62/2)  
 generic\_package\_declaration 12.1(4)  
   *used* 12.1(2), P  
 Generic\_Real\_Arrays  
   *child of* Ada.Numerics G.3.1(2/2)  
 generic\_renaming\_declaration 8.5.5(2)  
   *used* 8.5(2), 10.1.1(6), P  
 Generic\_Sort  
   *child of* Ada.Containers A.18.26(9.2/3)  
 Generic\_Sorting  
   *in* Ada.Containers.Doubly\_Linked\_Lists  
     A.18.3(47/2)  
   *in* Ada.Containers.Vectors  
     A.18.2(75/2)  
 generic\_subprogram\_declaration 12.1(3)  
   *used* 12.1(2), P  
 Get  
   *in* Ada.Text\_IO A.10.1(41),  
     A.10.1(47), A.10.1(54), A.10.1(55),  
     A.10.1(59), A.10.1(60), A.10.1(65),  
     A.10.1(67), A.10.1(70), A.10.1(72),  
     A.10.1(75), A.10.1(77), A.10.1(81),  
     A.10.1(83)  
   *in* Ada.Text\_IO.Complex\_IO G.1.3(6),  
     G.1.3(8)  
 Get\_Deadline  
   *in* Ada.Dispatching.EDF D.2.6(9/2)  
 Get\_Immediate  
   *in* Ada.Text\_IO A.10.1(44), A.10.1(45)  
 Get\_Line  
   *in* Ada.Text\_IO A.10.1(49),  
     A.10.1(49.1/2)  
   *in* Ada.Text\_IO.Bounded\_IO  
     A.10.11(8/2), A.10.11(9/2),  
     A.10.11(10/2), A.10.11(11/2)  
   *in* Ada.Text\_IO.Unbounded\_IO  
     A.10.12(8/2), A.10.12(9/2),  
     A.10.12(10/2), A.10.12(11/2)  
 Get\_Next\_Entry  
   *in* Ada.Directories A.16(35/2)  
 Get\_Priority  
   *in* Ada.Dynamic\_Priorities D.5.1(5)  
 global to 8.1(15)  
 Glossary N(1/2)

goto\_statement 5.8(2)  
   *used* 5.1(4/2), P  
 govern a variant 3.8.1(20)  
 govern a variant\_part 3.8.1(20)  
 grammar  
   complete listing P  
   cross reference P  
   notation 1.1.4(3)  
   resolution of ambiguity 8.6(3)  
   under Syntax heading 1.1.2(25)  
 graphic character  
   a category of Character A.3.2(23)  
 graphic\_character 2.1(14/2)  
   *used* 2.5(2), 2.6(3), P  
 Graphic\_Set  
   *in* Ada.Strings.Maps.Constants  
     A.4.6(4)  
 Grave  
   *in* Ada.Characters.Latin\_1 A.3.3(13)  
 greater than operator 4.4(1), 4.5.2(1)  
 greater than or equal operator 4.4(1),  
   4.5.2(1)  
 greater-than sign 2.1(15/2)  
 Greater\_Than\_Sign  
   *in* Ada.Characters.Latin\_1 A.3.3(10)  
 Group\_Budget  
   *in*  
     Ada.Execution\_Time.Group\_Budgets  
       D.14.2(4/2)  
 Group\_Budget\_Error  
   *in*  
     Ada.Execution\_Time.Group\_Budgets  
       D.14.2(11/2)  
 Group\_Budget\_Handler  
   *in*  
     Ada.Execution\_Time.Group\_Budgets  
       D.14.2(5/2)  
 Group\_Budgets  
   *child of* Ada.Execution\_Time  
     D.14.2(3/2)  
 GS  
   *in* Ada.Characters.Latin\_1 A.3.3(6)  
 guard 9.7.1(3)  
   *used* 9.7.1(2), P

## H

handle  
   an exception 11(1), N(18)  
   an exception occurrence 11.4(1),  
     11.4(7)  
 handled\_sequence\_of\_statements 11.2(2)  
   *used* 5.6(2), 6.3(2/2), 6.5(2.1/3), 7.2(2),  
     9.1(6), 9.5.2(3), 9.5.2(5), P  
 handler  
   execution timer D.14.1(13/2)  
   group budget D.14.2(14/2)  
   interrupt C.3(2)  
   termination C.7.3(8/2)  
   timing event D.15(10/2)

Handling  
   *child of* Ada.Characters A.3.2(2/2)  
 Has\_Element  
   *in* Ada.Containers.Doubly\_Linked\_Lists  
     A.18.3(44/2)  
   *in* Ada.Containers.Hashed\_Maps  
     A.18.5(33/2)  
   *in* Ada.Containers.Hashed\_Sets  
     A.18.8(45/2)  
   *in* Ada.Containers.Multiway\_Trees  
     A.18.10(36/3)  
   *in* Ada.Containers.Ordered\_Maps  
     A.18.6(43/2)  
   *in* Ada.Containers.Ordered\_Sets  
     A.18.9(53/2)  
   *in* Ada.Containers.Vectors  
     A.18.2(72/2)  
 Hash  
   *child of* Ada.Strings A.4.9(2/2)  
   *child of* Ada.Strings.Bounded  
     A.4.9(7/2)  
   *child of* Ada.Strings.Unbounded  
     A.4.9(10/2)  
 Hash\_Case\_Insensitive  
   *child of* Ada.Strings A.4.9(13/3)  
   *child of* Ada.Strings.Bounded  
     A.4.9(14.4/3)  
   *child of* Ada.Strings.Fixed  
     A.4.9(14.2/3)  
   *child of* Ada.Strings.Unbounded  
     A.4.9(14.7/3)  
 Hash\_Type  
   *in* Ada.Containers A.18.1(4/2)  
 Hashed\_Maps  
   *child of* Ada.Containers A.18.5(2/3)  
 Hashed\_Sets  
   *child of* Ada.Containers A.18.8(2/3)  
 Head  
   *in* Ada.Strings.Bounded A.4.4(70),  
     A.4.4(71)  
   *in* Ada.Strings.Fixed A.4.3(35),  
     A.4.3(36)  
   *in* Ada.Strings.Unbounded A.4.5(65),  
     A.4.5(66)  
 head (of a queue) D.2.1(5/2)  
 heap management  
   user-defined 13.11(1)  
   *See also* allocator 4.8(1)  
 held priority D.11(4/2)  
 heterogeneous input-output A.12.1(1)  
 hexadecimal  
   literal 2.4.2(1)  
   hexadecimal digit  
     a category of Character A.3.2(30)  
   hexadecimal literal 2.4.2(1)  
 Hexadecimal\_Digit\_Set  
   *in* Ada.Strings.Maps.Constants  
     A.4.6(4)  
 hidden from all visibility 8.3(5), 8.3(14)  
   by lack of a with\_clause 8.3(20/2)

- for a declaration completed by a subsequent declaration 8.3(19)
  - for overridden declaration 8.3(15)
  - within the declaration itself 8.3(16)
  - hidden from direct visibility 8.3(5), 8.3(21)
  - by an inner homograph 8.3(22)
  - where hidden from all visibility 8.3(23)
  - hiding 8.3(5)
  - Hierarchical\_File\_Names
    - child of* Ada.Directories A.16.1(3/3)
  - High\_Order\_First 13.5.3(2)
    - in* Interfaces.COBOL B.4(25)
    - in* System 13.7(15/2)
  - highest precedence operator 4.5.6(1)
  - highest\_precedence\_operator 4.5(7)
  - Hold
    - in* Ada.Asynchronous\_Task\_Control D.11(3/2)
  - Holder
    - in* Ada.Containers.Indefinite\_Holders A.18.18(6/3)
  - homograph 8.3(8)
  - Hour
    - in* Ada.Calendar.Formatting 9.6.1(24/2)
  - Hour\_Number *subtype of* Natural
    - in* Ada.Calendar.Formatting 9.6.1(20/2)
  - HT
    - in* Ada.Characters.Latin\_1 A.3.3(5)
  - HTJ
    - in* Ada.Characters.Latin\_1 A.3.3(17)
  - HTS
    - in* Ada.Characters.Latin\_1 A.3.3(17)
  - Hyphen
    - in* Ada.Characters.Latin\_1 A.3.3(8)
  - hyphen-minus 2.1(15/2)
- I**
- i
    - in* Ada.Numerics.Generic\_Complex\_ Types G.1.1(5)
    - in* Interfaces.Fortran B.5(10)
  - identifier 2.3(2/2)
    - used* 2.8(2), 2.8(3), 2.8(21), 2.8(23), 3.1(4), 4.1(3), 4.1.3(3), 4.1.4(3/2), 5.5(2), 5.6(2), 6.1(5), 7.1(3), 7.2(2), 9.1(4), 9.1(6), 9.4(4), 9.4(7), 9.5.2(3), 9.5.2(5), 11.4.2(6/2), 11.5(4.1/2), 11.5(4/2), 13.12(4/2), B.1(5), B.1(6), B.1(7), D.2.2(2), D.2.2(2.2/2), D.3(3), D.3(4), D.4(3), D.4(4), D.13(3/2), H.6(3/2), J.10(3/2), L(2.2/2), L(8), L(13), L(14), L(20), L(21), L(23), L(25.1/2), L(27.1/2), L(27.2/2), L(29), L(36), L(37), L(37.2/2), M.2(98), P
  - identifier specific to a pragma 2.8(10)
  - identifier\_extend 2.3(3.1/3)
    - used* 2.3(2/2), P
  - identifier\_start 2.3(3/2)
    - used* 2.3(2/2), P
  - Identity
    - in* Ada.Strings.Maps A.4.2(22)
    - in* Ada.Strings.Wide\_Maps A.4.7(22)
    - in* Ada.Strings.Wide\_Wide\_Maps A.4.8(22/2)
  - Identity attribute 11.4.1(9), C.7.1(12)
  - idle task D.11(4/2)
  - if\_statement 5.3(2)
    - used* 5.1(5/2), P
  - illegal
    - construct 1.1.2(27)
    - partition 1.1.2(29)
  - Im
    - in* Ada.Numerics.Generic\_Complex\_ Arrays G.3.2(7/2), G.3.2(27/2)
    - in* Ada.Numerics.Generic\_Complex\_ Types G.1.1(6)
  - image
    - of a value 3.5(27.3/2), 3.5(30/2), K(273/2), K(277.4/2)
    - in* Ada.Calendar.Formatting 9.6.1(35/2), 9.6.1(37/2)
    - in* Ada.Numerics.Discrete\_Random A.5.2(26)
    - in* Ada.Numerics.Float\_Random A.5.2(14)
    - in* Ada.Task\_Identification C.7.1(3/1)
    - in* Ada.Text\_IO Editing F.3.3(13)
  - Image attribute 3.5(35)
  - Imaginary
    - in* Ada.Numerics.Generic\_Complex\_ Types G.1.1(4/2)
  - Imaginary *subtype of* Imaginary
    - in* Interfaces.Fortran B.5(10)
  - immediate scope
    - of (a view of) an entity 8.2(11)
    - of a declaration 8.2(2)
  - Immediate\_Reclamation restriction H.4(10)
  - immediately enclosing 8.1(13)
  - immediately visible 8.3(4), 8.3(21)
  - immediately within 8.1(13)
  - immutably limited 7.5(8.1/3)
  - implementation advice 1.1.2(37)
    - summary of advice M.3(1/2)
  - implementation defined 1.1.3(18)
    - summary of characteristics M.2(1/2)
  - implementation permissions 1.1.2(36)
  - implementation requirements 1.1.2(33)
  - implementation-dependent
    - See* unspecified 1.1.3(18)
  - implementation\_kind 9.5(12/3)
    - used* 9.5(11/3), L(13.1/2)
  - implemented
    - by a protected entry 9.4(11.1/3)
    - by a protected subprogram 9.4(11.1/3)
    - by a task entry 9.1(9.2/3)
  - Implemented pragma 9.5(11/3), L(13.1/2)
  - implicit conversion
    - legality 8.6(27.1/3)
  - implicit declaration 3.1(5), N(11)
  - implicit initial values
    - for a subtype 3.3.1(10)
  - implicit subtype conversion 4.6(59), 4.6(60)
    - Access attribute 3.10.2(30)
    - access discriminant 3.7(27/2)
    - array bounds 4.6(38)
    - array index 4.1.1(7)
    - assignment to view conversion 4.6(55)
    - assignment\_statement 5.2(11)
    - bounds of a decimal fixed point type 3.5.9(16)
    - bounds of a fixed point type 3.5.9(14)
    - bounds of a range 3.5(9), 3.6(18)
    - choices of aggregate 4.3.3(22)
    - component defaults 3.3.1(13)
    - delay expression 9.6(20)
    - derived type discriminants 3.4(21)
    - discriminant values 3.7.1(12)
    - entry index 9.5.2(24)
    - expressions in aggregate 4.3.1(19)
    - expressions of aggregate 4.3.3(23)
    - function return 6.5(5.9/3), 6.5(6/2)
    - generic formal object of mode in 12.4(11)
    - inherited enumeration literal 3.4(29)
    - initialization expression 3.3.1(17)
    - initialization expression of allocator 4.8(7/2)
    - named number value 3.3.2(6)
    - operand of concatenation 4.5.3(9)
    - parameter passing 6.4.1(10), 6.4.1(11), 6.4.1(17)
    - pragma Interrupt\_Priority D.1(17), D.3(6.1/2)
    - pragma Priority D.1(17), D.3(6.1/2)
    - qualified\_expression 4.7(4)
    - reading a view conversion 4.6(56)
    - result of inherited function 3.4(27/2)
  - implicit\_dereference 4.1(6)
    - used* 4.1(4), P
  - Import pragma B.1(5), L(14)
  - imported
    - aspect of representation B.1(28)
  - imported entity B.1(23)
  - in (membership test) 4.4(1), 4.5.2(2)
  - inaccessible partition E.1(7)
  - inactive
    - a task state 9(10)
  - Include
    - in* Ada.Containers.Hashed\_Maps A.18.5(22/2)
    - in* Ada.Containers.Hashed\_Sets A.18.8(21/2)

- in* Ada.Containers.Ordered\_Maps A.18.6(21/2)
- in* Ada.Containers.Ordered\_Sets A.18.9(20/2)
- included
  - one range in another 3.5(4)
- incomplete type 3.2(4.1/2), 3.10.1(2.1/2), N(20.1/2)
- incomplete view 3.10.1(2.1/2)
  - tagged 3.10.1(2.1/2)
- incomplete\_type\_declaration 3.10.1(2/2)
  - used* 3.2.1(2), P
- Increment
  - in* Interfaces.C.Pointers B.3.2(11)
- indefinite subtype 3.3(23), 3.7(26)
- Indefinite\_Doubly\_Linked\_Lists
  - child of* Ada.Containers A.18.12(2/3)
- Indefinite\_Hashed\_Maps
  - child of* Ada.Containers A.18.13(2/3)
- Indefinite\_Hashed\_Sets
  - child of* Ada.Containers A.18.15(2/3)
- Indefinite\_Holders
  - child of* Ada.Containers A.18.18(5/3)
- Indefinite\_Multiway\_Trees
  - child of* Ada.Containers A.18.17(2/3)
- Indefinite\_Ordered\_Maps
  - child of* Ada.Containers A.18.14(2/3)
- Indefinite\_Ordered\_Sets
  - child of* Ada.Containers A.18.16(2/3)
- Indefinite\_Vectors
  - child of* Ada.Containers A.18.11(2/3)
- Independent pragma C.6(4.1/3), L(14.1/3)
- independent subprogram 11.6(6)
- Independent\_Components pragma C.6(6.1/3), L(14.2/3)
- independently addressable 9.10(1/3)
  - specified C.6(14.1/3)
- index
  - of an element of an open direct file A.8(3)
  - in* Ada.Direct\_IO A.8.4(15)
  - in* Ada.Streams.Stream\_IO A.12.1(23)
  - in* Ada.Strings.Bounded A.4.4(43.1/2), A.4.4(43.2/2), A.4.4(44), A.4.4(45), A.4.4(45.1/2), A.4.4(46)
  - in* Ada.Strings.Fixed A.4.3(8.1/2), A.4.3(8.2/2), A.4.3(9), A.4.3(10), A.4.3(10.1/2), A.4.3(11)
  - in* Ada.Strings.Unbounded A.4.5(38.1/2), A.4.5(38.2/2), A.4.5(39), A.4.5(40), A.4.5(40.1/2), A.4.5(41)
- index range 3.6(13)
- index subtype 3.6(9)
- index type 3.6(9)
- Index\_Check 11.5(14)
  - [*partial*] 4.1.1(7), 4.1.2(7), 4.3.3(29/3), 4.3.3(30), 4.5.3(8), 4.6(51/2), 4.7(4), 4.8(10/2)
- index\_constraint 3.6.1(2)
  - used* 3.2.2(7), P
- Index\_Error
  - in* Ada.Strings A.4.1(5)
- Index\_Non\_Blank
  - in* Ada.Strings.Bounded A.4.4(46.1/2), A.4.4(47)
  - in* Ada.Strings.Fixed A.4.3(11.1/2), A.4.3(12)
  - in* Ada.Strings.Unbounded A.4.5(41.1/2), A.4.5(42)
- index\_subtype\_definition 3.6(4)
  - used* 3.6(3), P
- indexed\_component 4.1.1(2)
  - used* 4.1(2/3), P
- indivisible C.6(10)
- inferable discriminants B.3.3(20/2)
- Information
  - child of* Ada.Directories A.16(124/2)
- information hiding
  - See* package 7(1)
  - See* private types and private extensions 7.3(1)
- information systems C(1), F(1)
- informative 1.1.2(18)
- inherently mutable object 3.3(13/3)
- inheritance
  - See* derived types and classes 3.4(1/2)
  - See also* tagged types and type extension 3.9(1)
- inherited
  - from an ancestor type 3.4.1(11)
- inherited component 3.4(11), 3.4(12)
- inherited discriminant 3.4(11)
- inherited entry 3.4(12)
- inherited protected subprogram 3.4(12)
- inherited subprogram 3.4(17/2)
- Initial\_Directory
  - in* Ada.Directories.Hierarchical\_File\_Names A.16.1(12/3)
- initialization
  - of a protected object 9.4(14)
  - of a protected object C.3.1(10), C.3.1(11/2)
  - of a task object 9.1(12/1), J.7.1(7)
  - of an object 3.3.1(18/2)
- initialization expression 3.3.1(1), 3.3.1(4)
- Initialize 7.6(2)
  - in* Ada.Finalization 7.6(6/2), 7.6(8/2)
- initialized allocator 4.8(4)
- initialized by default 3.3.1(18/2)
- Inline pragma 6.3.2(3), L(15)
- innermost dynamically enclosing 11.4(2)
- input A.6(1/2)
- Input attribute 13.13.2(22), 13.13.2(32)
- Input clause 13.3(7/2), 13.13.2(38/3)
- input-output
  - unspecified for access types A.7(6)
- Insert
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(19/2), A.18.3(20/2), A.18.3(21/2)
  - in* Ada.Containers.Hashed\_Maps A.18.5(19/2), A.18.5(20/2), A.18.5(21/2)
  - in* Ada.Containers.Hashed\_Sets A.18.8(19/2), A.18.8(20/2)
  - in* Ada.Containers.Ordered\_Maps A.18.6(18/2), A.18.6(19/2), A.18.6(20/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(18/2), A.18.9(19/2)
  - in* Ada.Containers.Vectors A.18.2(36/2), A.18.2(37/2), A.18.2(38/2), A.18.2(39/2), A.18.2(40/2), A.18.2(41/2), A.18.2(42/2), A.18.2(43/2)
  - in* Ada.Strings.Bounded A.4.4(60), A.4.4(61)
  - in* Ada.Strings.Fixed A.4.3(25), A.4.3(26)
  - in* Ada.Strings.Unbounded A.4.5(55), A.4.5(56)
- Insert\_Child
  - in* Ada.Containers.Multiway\_Trees A.18.10(41/3), A.18.10(42/3), A.18.10(43/3)
- Insert\_Space
  - in* Ada.Containers.Vectors A.18.2(48/2), A.18.2(49/2)
- inspectable object H.3.2(5/2)
- inspection point H.3.2(5/2)
- Inspection\_Point pragma H.3.2(3), L(16)
- instance
  - of a generic function 12.3(13)
  - of a generic package 12.3(13)
  - of a generic procedure 12.3(13)
  - of a generic subprogram 12.3(13)
  - of a generic unit 12.3(1)
- instructions for comment submission 0.3(58/1)
- int
  - in* Interfaces.C B.3(7)
- Integer 3.5.4(11), 3.5.4(21)
  - in* Standard A.1(12)
- integer literal 2.4(1)
- integer literals 3.5.4(14), 3.5.4(30)
- integer type 3.5.4(1), N(21)
- Integer\_Address
  - in* System.Storage\_Elements 13.7.1(10)
- Integer\_IO
  - in* Ada.Text\_IO A.10.1(52)
- Integer\_Text\_IO
  - child of* Ada A.10.8(21)
- integer\_type\_definition 3.5.4(2)
  - used* 3.2.1(4/2), P
- Integer\_Wide\_Text\_IO
  - child of* Ada A.11(2/2)

- Integer\_Wide\_Wide\_Text\_IO  
*child of* Ada A.11(3/2)
- interaction  
 between tasks 9(1)
- interface 3.9.4(4/2)  
 limited 3.9.4(5/2)  
 nonlimited 3.9.4(5/2)  
 protected 3.9.4(5/2)  
 synchronized 3.9.4(5/2)  
 task 3.9.4(5/2)  
 type 3.9.4(4/2)
- interface to assembly language C.1(4)
- interface to C B.3(1/2)
- interface to COBOL B.4(1)
- interface to Fortran B.5(1)
- interface to other languages B(1)
- interface type N(21.1/2)
- Interface\_Ancessor\_Tags  
*in* Ada.Tags 3.9(7.4/2)
- interface\_list 3.9.4(3/2)  
*used* 3.4(2/2), 3.9.4(2/2), 7.3(3/2),  
 9.1(2/2), 9.1(3/2), 9.4(2/2), 9.4(3/2),  
 12.5.1(3/2), P
- interface\_type\_definition 3.9.4(2/2)  
*used* 3.2.1(4/2), 12.5.5(2/2), P
- Interfaces B.2(3)
- Interfaces.C B.3(4)
- Interfaces.C.Pointers B.3.2(4)
- Interfaces.C.Strings B.3.1(3)
- Interfaces.COBOLE B.4(7)
- Interfaces.Fortran B.5(4)
- interfacing pragma B.1(4)  
 Convention B.1(4)  
 Export B.1(4)  
 Import B.1(4)
- internal call 9.5(3)
- internal code 13.4(7)
- internal requeue 9.5(7)
- Internal\_Tag  
*in* Ada.Tags 3.9(7/2)
- interpretation  
 of a complete context 8.6(10)  
 of a constituent of a complete context  
 8.6(15)  
 overload resolution 8.6(14)
- interrupt C.3(2)  
 example using asynchronous\_select  
 9.7.4(10), 9.7.4(12)
- interrupt entry J.7.1(5)
- interrupt handler C.3(2)
- Interrupt\_Handler pragma C.3.1(2),  
 L(17)
- Interrupt\_ID  
*in* Ada.Interrupts C.3.2(2)
- Interrupt\_Priority pragma D.1(5), L(18)
- Interrupt\_Priority\_subtype of Any\_Priority  
*in* System 13.7(16)
- Interrupts  
*child of* Ada C.3.2(2)
- Intersection  
*in* Ada.Containers.Hashted\_Sets  
 A.18.8(29/2), A.18.8(30/2)  
*in* Ada.Containers.Ordered\_Sets  
 A.18.9(30/2), A.18.9(31/2)
- intertask communication 9.5(1)  
*See also* task 9(1)
- Intrinsic calling convention 6.3.1(4)
- invalid cursor  
 of a list container A.18.3(153/2)  
 of a map A.18.4(76/2)  
 of a set A.18.7(97/2)  
 of a tree A.18.10(196/3)  
 of a vector A.18.2(248/2)
- invalid representation 13.9.1(9)
- Inverse  
*in* Ada.Numerics.Generic\_Complex\_-  
 Arrays G.3.2(46/2)  
*in* Ada.Numerics.Generic\_Real\_Arrays  
 G.3.1(24/2)
- Inverted\_Exclamation  
*in* Ada.Characters.Latin\_1 A.3.3(21/3)
- Inverted\_Question  
*in* Ada.Characters.Latin\_1 A.3.3(22)
- involve an inner product  
 complex G.3.2(56/2)  
 real G.3.1(34/2)
- IO\_Exceptions  
*child of* Ada A.13(3)
- IS1  
*in* Ada.Characters.Latin\_1 A.3.3(16)
- IS2  
*in* Ada.Characters.Latin\_1 A.3.3(16)
- IS3  
*in* Ada.Characters.Latin\_1 A.3.3(16)
- IS4  
*in* Ada.Characters.Latin\_1 A.3.3(16)
- Is\_A\_Group\_Member  
*in*  
 Ada.Execution\_Time.Group\_Budgets  
 D.14.2(8/2)
- Is\_Alphanumeric  
*in* Ada.Characters.Handling A.3.2(4)
- Is\_Attached  
*in* Ada.Interrupts C.3.2(5)
- Is\_Basic  
*in* Ada.Characters.Handling A.3.2(4)
- Is\_Callable  
*in* Ada.Task\_Identification C.7.1(4)
- Is\_Character  
*in* Ada.Characters.Conversions  
 A.3.4(3/2)
- Is\_Control  
*in* Ada.Characters.Handling A.3.2(4)
- Is\_Current\_Directory\_Name  
*in*  
 Ada.Directories.Hierarchical\_File\_Na-  
 mes A.16.1(7/3)
- Is\_Decimal\_Digit  
*in* Ada.Characters.Handling A.3.2(4)
- Is\_Descendant\_At\_Same\_Level  
*in* Ada.Tags 3.9(7.1/2)
- Is\_Digit  
*in* Ada.Characters.Handling A.3.2(4)
- Is\_Empty  
*in* Ada.Containers.Doubly\_Linked\_-  
 Lists A.18.3(12/2)  
*in* Ada.Containers.Hashted\_Maps  
 A.18.5(11/2)  
*in* Ada.Containers.Hashted\_Sets  
 A.18.8(13/2)  
*in* Ada.Containers.Indefinite\_Holders  
 A.18.18(10/3)  
*in* Ada.Containers.Multiway\_Trees  
 A.18.10(14/3)  
*in* Ada.Containers.Ordered\_Maps  
 A.18.6(10/2)  
*in* Ada.Containers.Ordered\_Sets  
 A.18.9(12/2)  
*in* Ada.Containers.Vectors  
 A.18.2(23/2)
- Is\_Full\_Name  
*in*  
 Ada.Directories.Hierarchical\_File\_Na-  
 mes A.16.1(8/3)
- Is\_Graphic  
*in* Ada.Characters.Handling A.3.2(4)
- Is\_Held  
*in* Ada.Asynchronous\_Task\_Control  
 D.11(3/2)
- Is\_Hexadecimal\_Digit  
*in* Ada.Characters.Handling A.3.2(4)
- Is\_In  
*in* Ada.Strings.Maps A.4.2(13)  
*in* Ada.Strings.Wide\_Maps A.4.7(13)  
*in* Ada.Strings.Wide\_Wide\_Maps  
 A.4.8(13/2)
- Is\_ISO\_646  
*in* Ada.Characters.Handling A.3.2(10)
- Is\_Leaf  
*in* Ada.Containers.Multiway\_Trees  
 A.18.10(19/3)
- Is\_Letter  
*in* Ada.Characters.Handling A.3.2(4)
- Is\_Lower  
*in* Ada.Characters.Handling A.3.2(4)
- Is\_Member  
*in*  
 Ada.Execution\_Time.Group\_Budgets  
 D.14.2(8/2)
- Is\_Nul\_Terminated  
*in* Interfaces.C B.3(24), B.3(35),  
 B.3(39.16/2), B.3(39.7/2)
- Is\_Open  
*in* Ada.Direct\_IO A.8.4(10)  
*in* Ada.Sequential\_IO A.8.1(10)  
*in* Ada.Streams.Stream\_IO A.12.1(12)  
*in* Ada.Text\_IO A.10.1(13)

|                                                           |                                                           |                                                           |
|-----------------------------------------------------------|-----------------------------------------------------------|-----------------------------------------------------------|
| Is_Parent_Directory_Name                                  | ISO/IEC 646:1991 1.2(2)                                   | <i>in</i> Ada.Containers.Ordered_Sets                     |
| <i>in</i>                                                 | ISO/IEC 8859-1:1987 1.2(6)                                | A.18.9(64/2)                                              |
| Ada.Directories.Hierarchical_File_Names A.16.1(6/3)       | ISO/IEC 9899:1999 1.2(7/2)                                | Kind                                                      |
| Is_Relative_Name                                          | ISO/IEC TR 19769:2004 1.2(10/2)                           | <i>in</i> Ada.Directories A.16(25/2),                     |
| <i>in</i>                                                 | ISO 646 <i>subtype of</i> Character                       | A.16(40/2)                                                |
| Ada.Directories.Hierarchical_File_Names A.16.1(9/3)       | <i>in</i> Ada.Characters.Handling A.3.2(9)                | known discriminants 3.7(26)                               |
| Is_Reserved                                               | ISO 646_Set                                               | known to be unvarying 6.4.1(6.7/3)                        |
| <i>in</i> Ada.Interrupts C.3.2(4)                         | <i>in</i> Ada.Strings.Maps.Constants                      | known to denote the same object                           |
| Is_Root                                                   | A.4.6(4)                                                  | 6.4.1(6/3)                                                |
| <i>in</i> Ada.Containers.Multiway_Trees                   | issue                                                     | known to refer to the same object                         |
| A.18.10(18/3)                                             | an entry call 9.5.3(8)                                    | 6.4.1(6.12/3)                                             |
| Is_Root_Directory_Name                                    | italics                                                   | known_discriminant_part 3.7(4)                            |
| <i>in</i>                                                 | nongraphic characters 3.5.2(2/3)                          | <i>used</i> 3.2.1(3), 3.7(2/2), 9.1(2/2),                 |
| Ada.Directories.Hierarchical_File_Names A.16.1(5/3)       | pseudo-names of anonymous types                           | 9.4(2/2), P                                               |
| Is_Round_Robin                                            | 3.2.1(7/2), A.1(2)                                        |                                                           |
| <i>in</i> Ada.Dispatching.Round_Robin                     | syntax rules 1.1.4(14)                                    | <b>L</b>                                                  |
| D.2.5(4/2)                                                | terms introduced or defined 1.3(1/2)                      | label 5.1(7)                                              |
| Is_Simple_Name                                            | Iterate                                                   | <i>used</i> 5.1(2/3), 5.1(3), P                           |
| <i>in</i>                                                 | <i>in</i> Ada.Containers.Doubly_Linked_Lists A.18.3(45/2) | Landau symbol O(X) A.18(3/2)                              |
| Ada.Directories.Hierarchical_File_Names A.16.1(4/3)       | <i>in</i> Ada.Containers.Hashed_Maps                      | language                                                  |
| Is_Sorted                                                 | A.18.5(37/2)                                              | interface to assembly C.1(4)                              |
| <i>in</i> Ada.Containers.Doubly_Linked_Lists A.18.3(48/2) | <i>in</i> Ada.Containers.Hashed_Sets                      | interface to non-Ada B(1)                                 |
| <i>in</i> Ada.Containers.Vectors                          | A.18.8(49/2)                                              | language-defined categories                               |
| A.18.2(76/2)                                              | <i>in</i> Ada.Containers.Multiway_Trees                   | [ <i>partial</i> ] 3.2(10/2)                              |
| Is_Special                                                | A.18.10(37/3)                                             | language-defined category                                 |
| <i>in</i> Ada.Characters.Handling A.3.2(4)                | <i>in</i> Ada.Containers.Ordered_Maps                     | of types 3.2(2/2)                                         |
| Is_String                                                 | A.18.6(50/2)                                              | language-defined check 11.5(2), 11.6(1)                   |
| <i>in</i> Ada.Characters.Conversions                      | <i>in</i> Ada.Containers.Ordered_Sets                     | language-defined class                                    |
| A.3.4(3/2)                                                | A.18.9(60/2)                                              | [ <i>partial</i> ] 3.2(10/2)                              |
| Is_Subset                                                 | <i>in</i> Ada.Containers.Vectors                          | of types 3.2(2/2)                                         |
| <i>in</i> Ada.Containers.Hashed_Sets                      | A.18.2(73/2)                                              | Language-defined constants Q.5(1/2)                       |
| A.18.8(39/2)                                              | <i>in</i> Ada.Environment_Variables                       | Language-defined exceptions Q.4(1/2)                      |
| <i>in</i> Ada.Containers.Ordered_Sets                     | A.17(8/2)                                                 | Language-Defined Library Units A(1)                       |
| A.18.9(40/2)                                              | Iterate_Children                                          | Language-defined objects Q.5(1/2)                         |
| <i>in</i> Ada.Strings.Maps A.4.2(14)                      | <i>in</i> Ada.Containers.Multiway_Trees                   | Language-defined packages Q.1(1/2)                        |
| <i>in</i> Ada.Strings.Wide_Maps A.4.7(14)                 | A.18.10(61/3)                                             | Language-defined subprograms Q.3(1/2)                     |
| <i>in</i> Ada.Strings.Wide_Wide_Maps                      | Iterate_Subtree                                           | Language-defined subtypes Q.2(1/2)                        |
| A.4.8(14/2)                                               | <i>in</i> Ada.Containers.Multiway_Trees                   | Language-defined types Q.2(1/2)                           |
| Is_Terminated                                             | A.18.10(38/3)                                             | Language-defined values Q.5(1/2)                          |
| <i>in</i> Ada.Task_Identification C.7.1(4)                | iteration_scheme 5.5(3)                                   | Last                                                      |
| Is_Upper                                                  | <i>used</i> 5.5(2), P                                     | <i>in</i> Ada.Containers.Doubly_Linked_Lists A.18.3(35/2) |
| <i>in</i> Ada.Characters.Handling A.3.2(4)                |                                                           | <i>in</i> Ada.Containers.Ordered_Maps                     |
| Is_Wide_Character                                         | <b>J</b>                                                  | A.18.6(31/2)                                              |
| <i>in</i> Ada.Characters.Conversions                      | j                                                         | <i>in</i> Ada.Containers.Ordered_Sets                     |
| A.3.4(3/2)                                                | <i>in</i> Ada.Numerics.Generic_Complex_Types G.1.1(5)     | A.18.9(43/2)                                              |
| Is_Wide_String                                            | <i>in</i> Interfaces.Fortran B.5(10)                      | <i>in</i> Ada.Containers.Vectors                          |
| <i>in</i> Ada.Characters.Conversions                      |                                                           | A.18.2(61/2)                                              |
| A.3.4(3/2)                                                | <b>K</b>                                                  | Last attribute 3.5(13), 3.6.2(5)                          |
| ISO 1989:2002 1.2(4/2)                                    | Key                                                       | last element                                              |
| ISO 8601:2004 1.2(5.1/2)                                  | <i>in</i> Ada.Containers.Hashed_Maps                      | of a hashed set A.18.8(68/2)                              |
| ISO/IEC 10646:2003 1.2(8/2),                              | A.18.5(13/2)                                              | of an ordered set A.18.9(81/2)                            |
| 3.5.2(2/3), 3.5.2(3.1/2), 3.5.2(3/2)                      | <i>in</i> Ada.Containers.Hashed_Sets                      | of a set A.18.7(6/2)                                      |
| ISO/IEC 14882:2003 1.2(9/2)                               | A.18.8(51/2)                                              | last node                                                 |
| ISO/IEC 1539-1:2004 1.2(3/2)                              | <i>in</i> Ada.Containers.Ordered_Maps                     | of a hashed map A.18.5(46/2)                              |
| ISO/IEC 6429:1992 1.2(5)                                  | A.18.6(12/2)                                              | of a map A.18.4(6/2)                                      |
|                                                           |                                                           | of an ordered map A.18.6(58/2)                            |
|                                                           |                                                           | Last(N) attribute 3.6.2(6)                                |
|                                                           |                                                           | last_bit 13.5.1(6)                                        |

|                                            |                                            |                                               |
|--------------------------------------------|--------------------------------------------|-----------------------------------------------|
| <i>used</i> 13.5.1(3), P                   | LC_E_Circumflex                            | LC_S                                          |
| Last_Bit attribute 13.5.2(4/2)             | <i>in</i> Ada.Characters.Latin_1 A.3.3(25) | <i>in</i> Ada.Characters.Latin_1 A.3.3(14)    |
| Last_Child                                 | LC_E_Diaeresis                             | LC_T                                          |
| <i>in</i> Ada.Containers.Multiway_Trees    | <i>in</i> Ada.Characters.Latin_1 A.3.3(25) | <i>in</i> Ada.Characters.Latin_1 A.3.3(14)    |
| A.18.10(55/3)                              | LC_E_Grave                                 | LC_U                                          |
| Last_Child_Element                         | <i>in</i> Ada.Characters.Latin_1 A.3.3(25) | <i>in</i> Ada.Characters.Latin_1 A.3.3(14)    |
| <i>in</i> Ada.Containers.Multiway_Trees    | LC_F                                       | LC_U_Acute                                    |
| A.18.10(56/3)                              | <i>in</i> Ada.Characters.Latin_1 A.3.3(13) | <i>in</i> Ada.Characters.Latin_1 A.3.3(26)    |
| Last_Element                               | LC_G                                       | LC_U_Circumflex                               |
| <i>in</i> Ada.Containers.Doubly_Linked_    | <i>in</i> Ada.Characters.Latin_1 A.3.3(13) | <i>in</i> Ada.Characters.Latin_1 A.3.3(26)    |
| Lists A.18.3(36/2)                         | LC_German_Sharp_S                          | LC_U_Diaeresis                                |
| <i>in</i> Ada.Containers.Ordered_Maps      | <i>in</i> Ada.Characters.Latin_1 A.3.3(24) | <i>in</i> Ada.Characters.Latin_1 A.3.3(26)    |
| A.18.6(32/2)                               | LC_H                                       | LC_U_Grave                                    |
| <i>in</i> Ada.Containers.Ordered_Sets      | <i>in</i> Ada.Characters.Latin_1 A.3.3(13) | <i>in</i> Ada.Characters.Latin_1 A.3.3(26)    |
| A.18.9(44/2)                               | LC_I                                       | LC_V                                          |
| <i>in</i> Ada.Containers.Vectors           | <i>in</i> Ada.Characters.Latin_1 A.3.3(13) | <i>in</i> Ada.Characters.Latin_1 A.3.3(14)    |
| A.18.2(62/2)                               | LC_I_Acute                                 | LC_W                                          |
| Last_Index                                 | <i>in</i> Ada.Characters.Latin_1 A.3.3(25) | <i>in</i> Ada.Characters.Latin_1 A.3.3(14)    |
| <i>in</i> Ada.Containers.Vectors           | LC_I_Circumflex                            | LC_X                                          |
| A.18.2(60/2)                               | <i>in</i> Ada.Characters.Latin_1 A.3.3(25) | <i>in</i> Ada.Characters.Latin_1 A.3.3(14)    |
| Last_Key                                   | LC_I_Diaeresis                             | LC_Y                                          |
| <i>in</i> Ada.Containers.Ordered_Maps      | <i>in</i> Ada.Characters.Latin_1 A.3.3(25) | <i>in</i> Ada.Characters.Latin_1 A.3.3(14)    |
| A.18.6(33/2)                               | LC_I_Grave                                 | LC_Y_Acute                                    |
| lateness D.9(12)                           | <i>in</i> Ada.Characters.Latin_1 A.3.3(25) | <i>in</i> Ada.Characters.Latin_1 A.3.3(26)    |
| Latin-1 3.5.2(2/3)                         | LC_Icelandic_Eth                           | LC_Y_Diaeresis                                |
| Latin_1                                    | <i>in</i> Ada.Characters.Latin_1 A.3.3(26) | <i>in</i> Ada.Characters.Latin_1 A.3.3(26)    |
| <i>child of</i> Ada.Characters A.3.3(3)    | LC_Icelandic_Thorn                         | LC_Z                                          |
| layout                                     | <i>in</i> Ada.Characters.Latin_1 A.3.3(26) | <i>in</i> Ada.Characters.Latin_1 A.3.3(14)    |
| aspect of representation 13.5(1)           | LC_J                                       | Leading_Nonseparate                           |
| Layout_Error                               | <i>in</i> Ada.Characters.Latin_1 A.3.3(13) | <i>in</i> Interfaces.COBOL B.4(23)            |
| <i>in</i> Ada.IO_Exceptions A.13(4)        | LC_K                                       | Leading_Part attribute A.5.3(54)              |
| <i>in</i> Ada.Text_IO A.10.1(85)           | <i>in</i> Ada.Characters.Latin_1 A.3.3(13) | Leading_Separate                              |
| LC_A                                       | LC_L                                       | <i>in</i> Interfaces.COBOL B.4(23)            |
| <i>in</i> Ada.Characters.Latin_1 A.3.3(13) | <i>in</i> Ada.Characters.Latin_1 A.3.3(13) | leaf node                                     |
| LC_A_Acute                                 | LC_M                                       | of a tree A.18.10(4/3)                        |
| <i>in</i> Ada.Characters.Latin_1 A.3.3(25) | <i>in</i> Ada.Characters.Latin_1 A.3.3(13) | Leap_Seconds_Count <i>subtype of</i> Integer  |
| LC_A_Circumflex                            | LC_N                                       | <i>in</i> Ada.Calendar.Arithmetic 9.6.1(11/2) |
| <i>in</i> Ada.Characters.Latin_1 A.3.3(25) | <i>in</i> Ada.Characters.Latin_1 A.3.3(13) | leaving 7.6.1(3/2)                            |
| LC_A_Diaeresis                             | LC_N_Tilde                                 | left 7.6.1(3/2)                               |
| <i>in</i> Ada.Characters.Latin_1 A.3.3(25) | <i>in</i> Ada.Characters.Latin_1 A.3.3(26) | left parenthesis 2.1(15/2)                    |
| LC_A_Grave                                 | LC_O                                       | Left_Angle_Quotation                          |
| <i>in</i> Ada.Characters.Latin_1 A.3.3(25) | <i>in</i> Ada.Characters.Latin_1 A.3.3(13) | <i>in</i> Ada.Characters.Latin_1 A.3.3(21/3)  |
| LC_A_Ring                                  | LC_O_Acute                                 | Left_Curly_Bracket                            |
| <i>in</i> Ada.Characters.Latin_1 A.3.3(25) | <i>in</i> Ada.Characters.Latin_1 A.3.3(26) | <i>in</i> Ada.Characters.Latin_1 A.3.3(14)    |
| LC_A_Tilde                                 | LC_O_Circumflex                            | Left_Parenthesis                              |
| <i>in</i> Ada.Characters.Latin_1 A.3.3(25) | <i>in</i> Ada.Characters.Latin_1 A.3.3(26) | <i>in</i> Ada.Characters.Latin_1 A.3.3(8)     |
| LC_AE_Diphthong                            | LC_O_Diaeresis                             | Left_Square_Bracket                           |
| <i>in</i> Ada.Characters.Latin_1 A.3.3(25) | <i>in</i> Ada.Characters.Latin_1 A.3.3(26) | <i>in</i> Ada.Characters.Latin_1 A.3.3(12)    |
| LC_B                                       | LC_O_Grave                                 | legal                                         |
| <i>in</i> Ada.Characters.Latin_1 A.3.3(13) | <i>in</i> Ada.Characters.Latin_1 A.3.3(26) | construct 1.1.2(27)                           |
| LC_C                                       | LC_O_Oblique_Stroke                        | partition 1.1.2(29)                           |
| <i>in</i> Ada.Characters.Latin_1 A.3.3(13) | <i>in</i> Ada.Characters.Latin_1 A.3.3(26) | legality rules 1.1.2(27)                      |
| LC_C_Cedilla                               | LC_O_Tilde                                 | length                                        |
| <i>in</i> Ada.Characters.Latin_1 A.3.3(25) | <i>in</i> Ada.Characters.Latin_1 A.3.3(26) | of a dimension of an array 3.6(13)            |
| LC_D                                       | LC_P                                       | of a list container A.18.3(3/2)               |
| <i>in</i> Ada.Characters.Latin_1 A.3.3(13) | <i>in</i> Ada.Characters.Latin_1 A.3.3(14) | of a map A.18.4(5/2)                          |
| LC_E                                       | LC_Q                                       | of a one-dimensional array 3.6(13)            |
| <i>in</i> Ada.Characters.Latin_1 A.3.3(13) | <i>in</i> Ada.Characters.Latin_1 A.3.3(14) | of a set A.18.7(5/2)                          |
| LC_E_Acute                                 | LC_R                                       | of a vector container A.18.2(2/2)             |
| <i>in</i> Ada.Characters.Latin_1 A.3.3(25) | <i>in</i> Ada.Characters.Latin_1 A.3.3(14) |                                               |

- in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(11/2)
- in* Ada.Containers.Hashed\_Maps A.18.5(10/2)
- in* Ada.Containers.Hashed\_Sets A.18.8(12/2)
- in* Ada.Containers.Ordered\_Maps A.18.6(9/2)
- in* Ada.Containers.Ordered\_Sets A.18.9(11/2)
- in* Ada.Containers.Vectors A.18.2(21/2)
- in* Ada.Strings.Bounded A.4.4(9)
- in* Ada.Strings.Unbounded A.4.5(6)
- in* Ada.Text\_IO.Editing F.3.3(11)
- in* Interfaces.COBOL B.4(34), B.4(39), B.4(44)
- Length attribute 3.6.2(9)
- Length(N) attribute 3.6.2(10)
- Length\_Check 11.5(15)
  - [partial]* 4.5.1(8), 4.6(37), 4.6(52)
- Length\_Error
  - in* Ada.Strings A.4.1(5)
- Length\_Range *subtype of* Natural
  - in* Ada.Strings.Bounded A.4.4(8)
- less than operator 4.4(1), 4.5.2(1)
- less than or equal operator 4.4(1), 4.5.2(1)
- less-than sign 2.1(15/2)
- Less\_Case\_Insensitive
  - child of* Ada.Strings A.4.10(13/3)
  - child of* Ada.Strings.Bounded A.4.10(18/3)
  - child of* Ada.Strings.Fixed A.4.10(16/3)
  - child of* Ada.Strings.Unbounded A.4.10(21/3)
- Less\_Than\_Sign
  - in* Ada.Characters.Latin\_1 A.3.3(10)
- letter
  - a category of Character A.3.2(24)
- letter\_lowercase 2.1(9/2)
  - used* 2.3(3/2), P
- letter\_modifier 2.1(9.2/2)
  - used* 2.3(3/2), P
- letter\_other 2.1(9.3/2)
  - used* 2.3(3/2), P
- Letter\_Set
  - in* Ada.Strings.Maps.Constants A.4.6(4)
- letter\_titlecase 2.1(9.1/2)
  - used* 2.3(3/2), P
- letter\_uppercase 2.1(8/2)
  - used* 2.3(3/2), P
- level
  - accessibility 3.10.2(3/2)
  - library 3.10.2(22)
- lexical element 2.2(1)
- lexicographic order 4.5.2(26)
- LF
  - in* Ada.Characters.Latin\_1 A.3.3(5)
- library 10.1.4(9)
  - [partial]* 10.1.1(9)
  - informal introduction 10(2)
  - See also* library level, library unit, library\_item
  - library level 3.10.2(22)
  - Library unit 10.1(3), 10.1.1(9), N(22)
    - informal introduction 10(2)
    - See also* language-defined library units
  - library unit pragma 10.1.5(7/3)
    - All\_Calls\_Remote E.2.3(6)
    - categorization pragmas E.2(2)
    - Elaborate\_Body 10.2.1(24)
    - Prelaborate 10.2.1(4)
    - Pure 10.2.1(15)
  - library\_item 10.1.1(4)
    - informal introduction 10(2)
    - used* 10.1.1(3), P
  - library\_unit\_body 10.1.1(7)
    - used* 10.1.1(4), P
  - library\_unit\_declaration 10.1.1(5)
    - used* 10.1.1(4), P
  - library\_unit\_renaming\_declaration 10.1.1(6)
    - used* 10.1.1(4), P
  - lifetime 3.10.2(3/2)
  - limited interface 3.9.4(5/2)
  - limited type 7.5(3/3), N(23/2)
    - becoming nonlimited 7.3.1(5/1), 7.5(16)
    - immutable 7.5(8.1/3)
  - limited view 10.1.1(12.1/2)
  - Limited\_Controlled
    - in* Ada.Finalization 7.6(7/2)
  - limited\_with\_clause 10.1.2(4.1/2)
    - used* 10.1.2(4/2), P
  - line 2.2(2/2)
    - in* Ada.Text\_IO A.10.1(38)
  - line terminator A.10(7)
  - Line\_Length
    - in* Ada.Text\_IO A.10.1(25)
  - link name B.1(35)
  - link-time error
    - See* post-compilation error 1.1.2(29)
    - See* post-compilation error 1.1.5(4)
  - Linker\_Options pragma B.1(8), L(19)
  - linking
    - See* partition building 10.2(2)
  - List
    - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(6/2)
  - list container A.18.3(1/2)
  - List pragma 2.8(21), L(20)
  - literal 4.2(1)
    - based 2.4.2(1)
    - decimal 2.4.1(1)
    - numeric 2.4(1)
    - See also* aggregate 4.3(1)
  - little endian 13.5.3(2)
  - load time C.4(3)
  - local to 8.1(14)
  - local\_name 13.1(3)
    - used* 6.5.1(3/2), 9.5(11/3), 13.2(3), 13.3(2), 13.4(2), 13.5.1(2), 13.5.1(3), 13.11.3(3), B.1(5), B.1(6), B.1(7), B.3.3(3/2), C.5(3), C.6(3), C.6(4), C.6(4.1/3), C.6(5), C.6(6), C.6(6.1/3), E.4.1(3), L(3), L(4), L(5), L(7), L(8), L(9), L(13), L(13.1/2), L(14), L(14.1/3), L(14.2/3), L(21.1/2), L(24), L(37.1/2), L(38), L(39), P
  - locking policy D.3(6/2)
    - Ceiling\_Locking D.3(7)
  - Locking\_Policy pragma D.3(3), L(21)
  - Log
    - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(3)
    - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(4)
  - Logical
    - in* Interfaces.Fortran B.5(7)
  - logical operator 4.5.1(2)
    - See also* not operator 4.5.6(3)
  - logical\_operator 4.5(2)
  - long
    - in* Interfaces.C B.3(7)
  - Long\_Binary
    - in* Interfaces.COBOL B.4(10)
  - long\_double
    - in* Interfaces.C B.3(17)
  - Long\_Float 3.5.7(15), 3.5.7(16), 3.5.7(17)
  - Long\_Floating
    - in* Interfaces.COBOL B.4(9)
  - Long\_Integer 3.5.4(22), 3.5.4(25), 3.5.4(28)
  - Look\_Ahead
    - in* Ada.Text\_IO A.10.1(43)
  - loop parameter 5.5(6)
  - loop\_parameter\_specification 5.5(4)
    - used* 4.5.7(1/3), 5.5(3), P
  - loop\_statement 5.5(2)
    - used* 5.1(5/2), P
  - low line 2.1(15/2)
  - low-level programming C(1)
  - Low\_Line
    - in* Ada.Characters.Latin\_1 A.3.3(12)
  - Low\_Order\_First 13.5.3(2)
    - in* Interfaces.COBOL B.4(25)
    - in* System 13.7(15/2)
  - lower bound
    - of a range 3.5(4)
  - lower-case letter
    - a category of Character A.3.2(25)
  - Lower\_Case\_Map
    - in* Ada.Strings.Maps.Constants A.4.6(5)



Lower\_Set  
*in* Ada.Strings.Maps.Constants  
 A.4.6(4)

## M

Machine attribute A.5.3(60)  
 machine code insertion 13.8(1), C.1(2)  
 machine numbers  
   of a fixed point type 3.5.9(8/2)  
   of a floating point type 3.5.7(8)  
 machine scalar 13.3(8.1/3)  
 Machine\_Code  
   *child of* System 13.8(7)  
 Machine\_Emax attribute A.5.3(8)  
 Machine\_Emin attribute A.5.3(7)  
 Machine\_Mantissa attribute A.5.3(6)  
 Machine\_Overflows attribute A.5.3(12),  
   A.5.4(4)  
 Machine\_Radix attribute A.5.3(2),  
   A.5.4(2)  
 Machine\_Radix clause 13.3(7/2), F.1(1)  
 Machine\_Rounding attribute  
   A.5.3(41.1/2)  
 Machine\_Rounds attribute A.5.3(11),  
   A.5.4(3)  
 macro  
   *See* generic unit 12(1)  
 Macron  
   *in* Ada.Characters.Latin\_1 A.3.3(21/3)  
 main subprogram  
   for a partition 10.2(7)  
 malloc  
   *See* allocator 4.8(1)  
 Map  
   *in* Ada.Containers.Hashed\_Maps  
     A.18.5(3/2)  
   *in* Ada.Containers.Ordered\_Maps  
     A.18.6(4/2)  
 map container A.18.4(1/2)  
 Maps  
   *child of* Ada.Strings A.4.2(3/2)  
 mark\_non\_spacing 2.1(9.4/2), 2.1(9.5/2)  
   *used* 2.3(3.1/3), P  
 mark\_spacing\_combining  
   *used* 2.3(3.1/3), P  
 marshalling E.4(9)  
 Masculine\_Ordinal\_Indicator  
   *in* Ada.Characters.Latin\_1 A.3.3(22)  
 master 7.6.1(3/2)  
 match  
   a character to a pattern character  
     A.4.2(54)  
   a character to a pattern character, with  
     respect to a character mapping  
     function A.4.2(64)  
   a string to a pattern string A.4.2(54)  
 matching components 4.5.2(16)  
 Max attribute 3.5(19)

Max\_Alignment\_For\_Allocation  
   attribute 13.11.1(3.1/3)  
 Max\_Asynchronous\_Select\_Nesting  
   restriction D.7(18/1)  
 Max\_Base\_Digits 3.5.7(6)  
   *in* System 13.7(8)  
 Max\_Binary\_Modulus 3.5.4(7)  
   *in* System 13.7(7)  
 Max\_Decimal\_Digits  
   *in* Ada.Decimal F.2(5)  
 Max\_Delta  
   *in* Ada.Decimal F.2(4)  
 Max\_Digits 3.5.7(6)  
   *in* System 13.7(8)  
 Max\_Digits\_Binary  
   *in* Interfaces.COBOL B.4(11)  
 Max\_Digits\_Long\_Binary  
   *in* Interfaces.COBOL B.4(11)  
 Max\_Entry\_Queue\_Length restriction  
   D.7(19.1/2)  
 Max\_Image\_Width  
   *in* Ada.Numerics.Discrete\_Random  
     A.5.2(25)  
   *in* Ada.Numerics.Float\_Random  
     A.5.2(13)  
 Max\_Int 3.5.4(14)  
   *in* System 13.7(6)  
 Max\_Length  
   *in* Ada.Strings.Bounded A.4.4(5)  
 Max\_Mantissa  
   *in* System 13.7(9)  
 Max\_Nonbinary\_Modulus 3.5.4(7)  
   *in* System 13.7(7)  
 Max\_Picture\_Length  
   *in* Ada.Text\_IO.Editing F.3.3(8)  
 Max\_Protected\_Entries restriction  
   D.7(14)  
 Max\_Scale  
   *in* Ada.Decimal F.2(3)  
 Max\_Select\_Alternatives restriction  
   D.7(12)  
 Max\_Size\_In\_Storage\_Elements  
   attribute 13.11.1(3/2)  
 Max\_Storage\_At\_Blocking restriction  
   D.7(17/1)  
 Max\_Task\_Entries restriction D.7(13)  
 Max\_Tasks restriction D.7(19/1)  
 maximum box error  
   for a component of the result of  
   evaluating a complex function  
     G.2.6(3)  
 maximum line length A.10(11)  
 maximum page length A.10(11)  
 maximum relative error  
   for a component of the result of  
   evaluating a complex function  
     G.2.6(3)  
   for the evaluation of an elementary  
   function G.2.4(2)

Members  
   *in*  
     Ada.Execution\_Time.Group\_Budgets  
       D.14.2(8/2)  
 Membership  
   *in* Ada.Strings A.4.1(6)  
 membership test 4.5.2(2)  
 Memory\_Size  
   *in* System 13.7(13)  
 mentioned  
   *in* a with\_clause 10.1.2(6/2)  
 Merge  
   *in* Ada.Containers.Doubly\_Linked\_ -  
     Lists A.18.3(50/2)  
   *in* Ada.Containers.Vectors  
     A.18.2(78/2)  
 message  
   *See* dispatching call 3.9.2(1/2)  
 method  
   *See* dispatching subprogram 3.9.2(1/2)  
 metrics 1.1.2(35)  
 Micro\_Sign  
   *in* Ada.Characters.Latin\_1 A.3.3(22)  
 Microseconds  
   *in* Ada.Real\_Time D.8(14/2)  
 Middle\_Dot  
   *in* Ada.Characters.Latin\_1 A.3.3(22)  
 Milliseconds  
   *in* Ada.Real\_Time D.8(14/2)  
 Min attribute 3.5(16)  
 Min\_Delta  
   *in* Ada.Decimal F.2(4)  
 Min\_Handler\_Ceiling  
   *in*  
     Ada.Execution\_Time.Group\_Budgets  
       D.14.2(7/2)  
   *in* Ada.Execution\_Time.Timers  
     D.14.1(6/2)  
 Min\_Int 3.5.4(14)  
   *in* System 13.7(6)  
 Min\_Scale  
   *in* Ada.Decimal F.2(3)  
 minus 2.1(15/2)  
 minus operator 4.4(1), 4.5.3(1), 4.5.4(1)  
 Minus\_Sign  
   *in* Ada.Characters.Latin\_1 A.3.3(8)  
 Minute  
   *in* Ada.Calendar.Formatting  
     9.6.1(25/2)  
 Minute\_Number *subtype of* Natural  
   *in* Ada.Calendar.Formatting  
     9.6.1(20/2)  
 Minutes  
   *in* Ada.Real\_Time D.8(14/2)  
 mixed-language programs B(1), C.1(4)  
 Mod attribute 3.5.4(16.1/2)  
 mod operator 4.4(1), 4.5.5(1)  
 mod\_clause J.8(1)  
   *used* 13.5.1(2), P  
 mode 6.1(16)

- used* 6.1(15/2), 12.4(2/3), P
- in* Ada.Direct\_IO A.8.4(9)
- in* Ada.Sequential\_IO A.8.1(9)
- in* Ada.Streams.Stream\_IO A.12.1(11)
- in* Ada.Text\_IO A.10.1(12)
- mode conformance 6.3.1(16/3)
  - required 8.5.4(4), 8.5.4(5/1), 12.5.4(5), 12.6(7), 12.6(8), 13.3(6)
- mode of operation
  - nonstandard 1.1.5(11)
  - standard 1.1.5(11)
- Mode\_Error
  - in* Ada.Direct\_IO A.8.4(18)
  - in* Ada.IO\_Exceptions A.13(4)
  - in* Ada.Sequential\_IO A.8.1(15)
  - in* Ada.Streams.Stream\_IO A.12.1(26)
  - in* Ada.Text\_IO A.10.1(85)
- Model attribute A.5.3(68), G.2.2(7)
- model interval G.2.1(4)
  - associated with a value G.2.1(4)
- model number G.2.1(3)
- model-oriented attributes
  - of a floating point subtype A.5.3(63)
- Model\_Emin attribute A.5.3(65), G.2.2(4)
- Model\_Epsilon attribute A.5.3(66)
- Model\_Mantissa attribute A.5.3(64), G.2.2(3/2)
- Model\_Small attribute A.5.3(67)
- Modification\_Time
  - in* Ada.Directories A.16(27/2), A.16(42/2)
- modular type 3.5.4(1)
- Modular\_IO
  - in* Ada.Text\_IO A.10.1(57)
- modular\_type\_definition 3.5.4(4)
  - used* 3.5.4(2), P
- module
  - See* package 7(1)
- modulus
  - of a modular type 3.5.4(7)
  - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(10/2), G.3.2(30/2)
  - in* Ada.Numerics.Generic\_Complex\_Types G.1.1(9)
- Modulus attribute 3.5.4(17)
- Monday
  - in* Ada.Calendar.Formatting 9.6.1(17/2)
- Month
  - in* Ada.Calendar 9.6(13)
  - in* Ada.Calendar.Formatting 9.6.1(22/2)
- Month\_Number *subtype of* Integer
  - in* Ada.Calendar 9.6(11/2)
- More\_Entries
  - in* Ada.Directories A.16(34/2)
- Move
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(18/2)
  - in* Ada.Containers.Hashed\_Maps A.18.5(18/2)
  - in* Ada.Containers.Hashed\_Sets A.18.8(18/2)
  - in* Ada.Containers.Indefinite\_Holders A.18.18(18/3)
  - in* Ada.Containers.Multiway\_Trees A.18.10(28/3)
  - in* Ada.Containers.Ordered\_Maps A.18.6(17/2)
  - in* Ada.Containers.Ordered\_Sets A.18.9(17/2)
  - in* Ada.Containers.Vectors A.18.2(35/2)
  - in* Ada.Strings.Fixed A.4.3(7)
- multi-dimensional array 3.6(12)
- Multiplication\_Sign
  - in* Ada.Characters.Latin\_1 A.3.3(24)
- multiply 2.1(15/2)
- multiply operator 4.4(1), 4.5.5(1)
- multiplying operator 4.5.5(1)
- multiplying\_operator 4.5(6)
  - used* 4.4(5), P
- Multiway\_Trees
  - child of* Ada.Containers A.18.10(7/3)
- MW
  - in* Ada.Characters.Latin\_1 A.3.3(18)
- N**
- n-dimensional array\_aggregate 4.3.3(6)
- NAK
  - in* Ada.Characters.Latin\_1 A.3.3(6)
- name 4.1(2/3)
  - [*partial*] 3.1(1)
  - of (a view of) an entity 3.1(8)
  - of a pragma 2.8(9)
  - of an external file A.7(1)
  - used* 2.8(3), 3.2.2(4), 4.1(4), 4.1(5), 4.1(6), 4.4(7/3), 4.6(2), 5.2(2), 5.7(2), 5.8(2), 6.3.2(3), 6.4(2), 6.4(3), 6.4(6), 8.4(3), 8.5.1(2/2), 8.5.2(2), 8.5.3(2), 8.5.4(2/2), 8.5.5(2), 9.5.3(2), 9.5.4(2/3), 9.8(2), 10.1.1(8), 10.1.2(4.1/2), 10.1.2(4.2/2), 10.2.1(3), 10.2.1(14), 10.2.1(20), 10.2.1(21), 10.2.1(22), 11.2(5), 11.3(2/2), 12.3(2/2), 12.3(5), 12.6(4), 12.7(2), 13.1(3), 13.3(2), 13.12(4.1/2), C.3.1(2), C.3.1(4), E.2.1(3), E.2.2(3), E.2.3(3), E.2.3(5), H.3.2(3), J.10(3/2), L(2), L(6), L(10), L(11), L(12), L(15), L(16), L(17), L(26), L(28), L(30), L(31), L(34), P
  - in* Ada.Direct\_IO A.8.4(9)
  - in* Ada.Sequential\_IO A.8.1(9)
  - in* Ada.Streams.Stream\_IO A.12.1(11)
  - in* Ada.Text\_IO A.10.1(12)
  - in* System 13.7(4)
- name resolution rules 1.1.2(26)
- Name\_Case\_Equivalence
  - in* Ada.Directories A.16(20.2/3)
- Name\_Case\_Kind
  - in* Ada.Directories A.16(20.1/3)
- Name\_Error
  - in* Ada.Direct\_IO A.8.4(18)
  - in* Ada.Directories A.16(43/2)
  - in* Ada.IO\_Exceptions A.13(4)
  - in* Ada.Sequential\_IO A.8.1(15)
  - in* Ada.Streams.Stream\_IO A.12.1(26)
  - in* Ada.Text\_IO A.10.1(85)
- named
  - in* a use clause 8.4(7.1/2)
  - in* a with\_clause 10.1.2(6/2)
- named association 6.4(7), 6.4.1(2/3), 12.3(6)
- named component association 4.3.1(6)
- named discriminant association 3.7.1(4)
- named entry index 9.5.2(21)
- named number 3.3(24)
- named parameter association 6.4.1(2/3)
- named type 3.2.1(7/2)
- named\_array\_aggregate 4.3.3(4)
  - used* 4.3.3(2), P
- Names
  - child of* Ada.Interrupts C.3.2(12)
- Nanoseconds
  - in* Ada.Real\_Time D.8(14/2)
- Native\_Binary
  - in* Interfaces.COBOL B.4(25)
- Natural 3.5.4(12)
- Natural *subtype of* Integer
  - in* Standard A.1(13)
- NBH
  - in* Ada.Characters.Latin\_1 A.3.3(17)
- NBSP
  - in* Ada.Characters.Latin\_1 A.3.3(21/3)
- needed
  - of a compilation unit by another 10.2(2)
  - remote call interface E.2.3(18)
  - shared passive library unit E.2.1(11)
- needed component
  - extension\_aggregate
    - record\_component\_association\_list 4.3.2(6)
  - record\_aggregate
    - record\_component\_association\_list 4.3.1(9)
- needs finalization 7.6(9.1/2)

|                                                                                                                                                                                                                                                     |                                                                  |                                                                                                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| language-defined type A.4.5(72.1/2), A.5.2(15.1/2), A.5.2(27.1/2), A.8.1(17/2), A.8.4(20/2), A.10.1(86/2), A.12.1(27.1/2), A.16(102/2), A.18.2(84/2), A.18.3(56/2), A.18.4(4/2), A.18.7(4/2), A.18.10(65/3), A.18.18(23/3), D.14.2(13/2), D.15(8/2) | <i>in</i> Ada.Containers.Ordered_Maps A.18.6(7/2)                | associated with an indexed_component 4.1.1(5)                                                                           |
| NEL                                                                                                                                                                                                                                                 | <i>in</i> Ada.Containers.Ordered_Sets A.18.9(7/2)                | of a component 3.6(20)                                                                                                  |
| <i>new</i>                                                                                                                                                                                                                                          | <i>in</i> Ada.Containers.Vectors A.18.2(11/2)                    | of a formal parameter 6.1(23/2)                                                                                         |
| <i>See</i> allocator 4.8(1)                                                                                                                                                                                                                         | No_Exceptions restriction H.4(12)                                | of a function result 6.1(23/2)                                                                                          |
| New_Char_Array                                                                                                                                                                                                                                      | No_Fixed_Point restriction H.4(15)                               | of a generic formal object 12.4(9/2)                                                                                    |
| <i>in</i> Interfaces.C.Strings B.3.1(9)                                                                                                                                                                                                             | No_Floating_Point restriction H.4(14)                            | of a record component 3.8(14)                                                                                           |
| New_Line                                                                                                                                                                                                                                            | No_Implementation_Pragmas restriction 13.12.1(2/2), 13.12.1(3/2) | of the result of a function_call 6.4(12/2)                                                                              |
| <i>in</i> Ada.Text_IO A.10.1(28)                                                                                                                                                                                                                    | No_Implicit_Heap_Allocations restriction D.7(8)                  | non-normative                                                                                                           |
| New_Page                                                                                                                                                                                                                                            | No_Index                                                         | <i>See</i> informative 1.1.2(18)                                                                                        |
| <i>in</i> Ada.Text_IO A.10.1(31)                                                                                                                                                                                                                    | <i>in</i> Ada.Containers.Vectors A.18.2(7/2)                     | non-returning 6.5.1(4/2)                                                                                                |
| New_String                                                                                                                                                                                                                                          | No_IO restriction H.4(20/2)                                      | Non_Preemptive                                                                                                          |
| <i>in</i> Interfaces.C.Strings B.3.1(10)                                                                                                                                                                                                            | No_Local_Allocators restriction H.4(8/1)                         | <i>child of</i> Ada.Dispatching D.2.4(2.2/3)                                                                            |
| Next                                                                                                                                                                                                                                                | No_Local_Protected_Objects restriction D.7(10.1/3)               | Non_Preemptive_FIFO_Within_Priorities                                                                                   |
| <i>in</i> Ada.Containers.Doubly_Linked_Lists A.18.3(37/2), A.18.3(39/2)                                                                                                                                                                             | No_Local_Timing_Events restriction D.7(10.2/3)                   | task disp. policy D.2.4(2/2)                                                                                            |
| <i>in</i> Ada.Containers.Hashing_Maps A.18.5(28/2), A.18.5(29/2)                                                                                                                                                                                    | No_Nested_Finalization restriction D.7(4/3)                      | nondispatching call                                                                                                     |
| <i>in</i> Ada.Containers.Hashing_Sets A.18.8(41/2), A.18.8(42/2)                                                                                                                                                                                    | No_Obsolete_Features restriction 13.12.1(4/2)                    | on a dispatching operation 3.9.2(1/2)                                                                                   |
| <i>in</i> Ada.Containers.Ordered_Maps A.18.6(34/2), A.18.6(35/2)                                                                                                                                                                                    | No_Protected_Type_Allocators restriction D.7(10.3/2)             | nonexistent 13.11.2(10/2), 13.11.2(16/3)                                                                                |
| <i>in</i> Ada.Containers.Ordered_Sets A.18.9(45/2), A.18.9(46/2)                                                                                                                                                                                    | No_Protected_Types restriction H.4(5)                            | nongraphic character 3.5(27.5/2)                                                                                        |
| <i>in</i> Ada.Containers.Vectors A.18.2(63/2), A.18.2(64/2)                                                                                                                                                                                         | No_Recursion restriction H.4(22)                                 | nonlimited interface 3.9.4(5/2)                                                                                         |
| Next_Sibling                                                                                                                                                                                                                                        | No_Reentrancy restriction H.4(23)                                | nonlimited type 7.5(7)                                                                                                  |
| <i>in</i> Ada.Containers.Multiway_Trees A.18.10(57/3), A.18.10(59/3)                                                                                                                                                                                | No_Relative_Delay restriction D.7(10.4/2)                        | becoming nonlimited 7.3.1(5/1), 7.5(16)                                                                                 |
| No_Abort_Statements restriction D.7(5)                                                                                                                                                                                                              | No_Resume_Statements restriction D.7(10.5/2)                     | nonlimited_with_clause 10.1.2(4.2/2)                                                                                    |
| No_Access_Subprograms restriction H.4(17)                                                                                                                                                                                                           | No_Return pragma 6.5.1(3/2), L(21.1/2)                           | <i>used</i> 10.1.2(4/2), P                                                                                              |
| No_Allocators restriction H.4(7)                                                                                                                                                                                                                    | No_Select_Statements restriction D.7(10.6/2)                     | nonstandard integer type 3.5.4(26)                                                                                      |
| No_Anonymous_Allocators restriction H.4(8.1/3)                                                                                                                                                                                                      | No_Specific_Termination_Handlers restriction D.7(10.7/2)         | nonstandard mode 1.1.5(11)                                                                                              |
| No_Break_Space                                                                                                                                                                                                                                      | No_Tag                                                           | nonstandard real type 3.5.6(8)                                                                                          |
| <i>in</i> Ada.Characters.Latin_1 A.3.3(21/3)                                                                                                                                                                                                        | <i>in</i> Ada.Tags 3.9(6.1/2)                                    | normal completion 7.6.1(2/2)                                                                                            |
| No_Delay restriction H.4(21)                                                                                                                                                                                                                        | No_Task_Allocators restriction D.7(7)                            | normal library unit E.2(4/1)                                                                                            |
| No_Dispatch restriction H.4(19)                                                                                                                                                                                                                     | No_Task_Hierarchy restriction D.7(3/3)                           | normal state of an object 11.6(6), 13.9.1(4)                                                                            |
| No_Dynamic_Attachment restriction D.7(10/3)                                                                                                                                                                                                         | No_Task_Termination restriction D.7(15.1/2)                      | [ <i>partial</i> ] 9.8(21), A.13(17)                                                                                    |
| No_Dynamic_Priorities restriction D.7(9/2)                                                                                                                                                                                                          | No_Terminate_Alternatives restriction D.7(6)                     | Normalize_Scalars pragma H.1(3), L(22)                                                                                  |
| No_Element                                                                                                                                                                                                                                          | No_Unchecked_Access restriction H.4(18)                          | normalized exponent A.5.3(14)                                                                                           |
| <i>in</i> Ada.Containers.Doubly_Linked_Lists A.18.3(9/2)                                                                                                                                                                                            | node                                                             | normalized number A.5.3(10)                                                                                             |
| <i>in</i> Ada.Containers.Hashing_Maps A.18.5(6/2)                                                                                                                                                                                                   | of a list A.18.3(2/2)                                            | normative 1.1.2(14)                                                                                                     |
| <i>in</i> Ada.Containers.Hashing_Sets A.18.8(6/2)                                                                                                                                                                                                   | of a map A.18.4(5/2)                                             | not equal operator 4.4(1), 4.5.2(1)                                                                                     |
| <i>in</i> Ada.Containers.Multiway_Trees A.18.10(11/3)                                                                                                                                                                                               | of a tree A.18.10(2/3)                                           | not in (membership test) 4.4(1), 4.5.2(2)                                                                               |
|                                                                                                                                                                                                                                                     | Node_Count                                                       | not operator 4.4(1), 4.5.6(3)                                                                                           |
|                                                                                                                                                                                                                                                     | <i>in</i> Ada.Containers.Multiway_Trees A.18.10(15/3)            | Not_Sign                                                                                                                |
|                                                                                                                                                                                                                                                     | nominal subtype 3.3(23), 3.3.1(8/2)                              | <i>in</i> Ada.Characters.Latin_1 A.3.3(21/3)                                                                            |
|                                                                                                                                                                                                                                                     | associated with a dereference 4.1(9/3)                           | notes 1.1.2(38)                                                                                                         |
|                                                                                                                                                                                                                                                     | associated with a type_conversion 4.6(27)                        | notwithstanding 7.6(17.5/3), 10.1.6(6/2), B.1(22), B.1(38), C.3.1(19), E.2.1(8), E.2.1(11), E.2.3(18), H.6(7/2), J.3(6) |
|                                                                                                                                                                                                                                                     |                                                                  | NUL                                                                                                                     |
|                                                                                                                                                                                                                                                     |                                                                  | <i>in</i> Ada.Characters.Latin_1 A.3.3(5)                                                                               |
|                                                                                                                                                                                                                                                     |                                                                  | <i>in</i> Interfaces.C B.3(20/1)                                                                                        |
|                                                                                                                                                                                                                                                     |                                                                  | null access value 4.2(9)                                                                                                |
|                                                                                                                                                                                                                                                     |                                                                  | null array 3.6.1(7)                                                                                                     |
|                                                                                                                                                                                                                                                     |                                                                  | null constraint 3.2(7/2)                                                                                                |
|                                                                                                                                                                                                                                                     |                                                                  | null extension 3.9.1(4.1/2)                                                                                             |
|                                                                                                                                                                                                                                                     |                                                                  | null pointer                                                                                                            |
|                                                                                                                                                                                                                                                     |                                                                  | <i>See</i> null access value 4.2(9)                                                                                     |
|                                                                                                                                                                                                                                                     |                                                                  | null procedure 6.7(3/2)                                                                                                 |
|                                                                                                                                                                                                                                                     |                                                                  | null range 3.5(4)                                                                                                       |
|                                                                                                                                                                                                                                                     |                                                                  | null record 3.8(15)                                                                                                     |
|                                                                                                                                                                                                                                                     |                                                                  | null slice 4.1.2(7)                                                                                                     |

- null string literal 2.6(6)
    - null value
      - of an access type 3.10(13/2)
  - Null\_Address
    - in System 13.7(12)
  - Null\_Bounded\_String
    - in Ada.Strings.Bounded A.4.4(7)
  - null\_exclusion 3.10(5.1/2)
    - used 3.2.2(3/2), 3.7(5/2), 3.10(2/2), 3.10(6/2), 6.1(13/2), 6.1(15/2), 8.5.1(2/2), 12.4(2/3), P
  - Null\_Id
    - in Ada.Exceptions 11.4.1(2/2)
  - Null\_Occurrence
    - in Ada.Exceptions 11.4.1(3/2)
  - null\_procedure\_declaration 6.7(2/2)
    - used 3.1(3/2), P
  - Null\_Ptr
    - in Interfaces.C.Strings B.3.1(7)
  - Null\_Set
    - in Ada.Strings.Maps A.4.2(5)
    - in Ada.Strings.Wide\_Maps A.4.7(5)
    - in Ada.Strings.Wide\_Wide\_Maps A.4.8(5/2)
  - null\_statement 5.1(6)
    - used 5.1(4/2), P
  - Null\_Task\_Id
    - in Ada.Task\_Identification C.7.1(2/2)
  - Null\_Unbounded\_String
    - in Ada.Strings.Unbounded A.4.5(5)
  - number sign 2.1(15/2)
  - Number\_Base *subtype of* Integer
    - in Ada.Text\_IO A.10.1(6)
  - number\_decimal 2.1(10/2)
    - used 2.3(3.1/3), P
  - number\_declaration 3.3.2(2)
    - used 3.1(3/2), P
  - number\_letter 2.1(10.1/2)
    - used 2.3(3/2), P
  - Number\_Sign
    - in Ada.Characters.Latin\_1 A.3.3(8)
  - numeral 2.4.1(3)
    - used 2.4.1(2), 2.4.1(4), 2.4.2(3), P
  - Numeric
    - in Interfaces.COBOL B.4(20)
  - numeric type 3.5(1)
  - numeric\_literal 2.4(2)
    - used 4.4(7/3), P
  - numerics G(1)
    - child of* Ada A.5(3/2)
- O**
- O(f(N)) A.18(3/2)
  - object 3.3(2), N(24)
    - [*partial*] 3.2(1)
  - object-oriented programming (OOP)
    - See* dispatching operations of tagged types 3.9.2(1/2)
    - See* tagged types and type extensions 3.9(1)
  - object\_declaration 3.3.1(2/2)
    - used 3.1(3/2), P
  - object\_renaming\_declaration 8.5.1(2/2)
    - used 8.5(2), P
  - obsolescent feature J(1/2)
  - occur immediately within 8.1(13)
  - occurrence
    - of an interrupt C.3(2)
  - octal
    - literal 2.4.2(1)
  - octal literal 2.4.2(1)
  - one's complement
    - modular types 3.5.4(27)
  - one-dimensional array 3.6(12)
  - only as a completion
    - entry\_body 9.5.2(16)
  - OOP (object-oriented programming)
    - See* dispatching operations of tagged types 3.9.2(1/2)
    - See* tagged types and type extensions 3.9(1)
  - opaque type
    - See* private types and private extensions 7.3(1)
  - Open
    - in Ada.Direct\_IO A.8.4(7)
    - in Ada.Sequential\_IO A.8.1(7)
    - in Ada.Streams.Stream\_IO A.12.1(9)
    - in Ada.Text\_IO A.10.1(10)
  - open alternative 9.7.1(14)
  - open entry 9.5.3(5)
    - of a protected object 9.5.3(7)
    - of a task 9.5.3(6)
  - operand
    - of a qualified\_expression 4.7(3)
    - of a type\_conversion 4.6(3)
  - operand interval G.2.1(6)
  - operand type
    - of a type\_conversion 4.6(3)
  - operates on a type 3.2.3(1/2)
  - operational aspect 13.1(8.1/1)
    - specifiable attributes 13.3(5/1)
  - operational item 13.1(1.1/1)
  - operator 6.6(1)
    - & 4.4(1), 4.5.3(3)
    - \* 4.4(1), 4.5.5(1)
    - \*\* 4.4(1), 4.5.6(7)
    - + 4.4(1), 4.5.3(1), 4.5.4(1)
    - 4.4(1), 4.5.3(1), 4.5.4(1)
    - / 4.4(1), 4.5.5(1)
    - /= 4.4(1), 4.5.2(1)
    - < 4.4(1), 4.5.2(1)
    - <= 4.4(1), 4.5.2(1)
    - = 4.4(1), 4.5.2(1)
    - > 4.4(1), 4.5.2(1)
    - >= 4.4(1), 4.5.2(1)
  - abs 4.4(1), 4.5.6(1)
  - ampersand 4.4(1), 4.5.3(3)
  - and 4.4(1), 4.5.1(2)
  - binary 4.5(9)
  - binary adding 4.5.3(1)
  - concatenation 4.4(1), 4.5.3(3)
  - divide 4.4(1), 4.5.5(1)
  - equal 4.4(1), 4.5.2(1)
  - equality 4.5.2(1)
  - exponentiation 4.4(1), 4.5.6(7)
  - greater than 4.4(1), 4.5.2(1)
  - greater than or equal 4.4(1), 4.5.2(1)
  - highest precedence 4.5.6(1)
  - less than 4.4(1), 4.5.2(1)
  - less than or equal 4.4(1), 4.5.2(1)
  - logical 4.5.1(2)
  - minus 4.4(1), 4.5.3(1), 4.5.4(1)
  - mod 4.4(1), 4.5.5(1)
  - multiply 4.4(1), 4.5.5(1)
  - multiplying 4.5.5(1)
  - not 4.4(1), 4.5.6(3)
  - not equal 4.4(1), 4.5.2(1)
  - or 4.4(1), 4.5.1(2)
  - ordering 4.5.2(1)
  - plus 4.4(1), 4.5.3(1), 4.5.4(1)
  - predefined 4.5(9)
  - relational 4.5.2(1)
  - rem 4.4(1), 4.5.5(1)
  - times 4.4(1), 4.5.5(1)
  - unary 4.5(9)
  - unary adding 4.5.4(1)
  - user-defined 6.6(1)
  - xor 4.4(1), 4.5.1(2)
  - operator precedence 4.5(1)
  - operator\_symbol 6.1(9)
    - used 4.1(3), 4.1.3(3), 6.1(5), 6.1(11), P
  - optimization 11.5(29), 11.6(1)
  - Optimize pragma 2.8(23), L(23)
  - or else (short-circuit control form) 4.4(1), 4.5.1(1)
  - or operator 4.4(1), 4.5.1(2)
  - Ordered\_Maps
    - child of* Ada.Containers A.18.6(2/3)
  - Ordered\_Sets
    - child of* Ada.Containers A.18.9(2/3)
  - ordering operator 4.5.2(1)
  - ordinary file A.16(45/2)
  - ordinary fixed point type 3.5.9(1), 3.5.9(8/2)
  - ordinary\_fixed\_point\_definition 3.5.9(3)
    - used 3.5.9(2), P
  - OSC
    - in Ada.Characters.Latin\_1 A.3.3(19)
  - other\_control 2.1(13.1/2)
  - other\_format 2.1(10.3/2)
  - other\_private\_use 2.1(13.2/2)
  - other\_surrogate 2.1(13.3/2)
  - output A.6(1/2)
  - Output attribute 13.13.2(19), 13.13.2(29)

Output clause 13.3(7/2), 13.13.2(38/3)  
 overall interpretation  
   of a complete context 8.6(10)  
 Overflow\_Check 11.5(16)  
   [*partial*] 3.5.4(20), 4.4(11), 5.4(13),  
   G.2.1(11), G.2.2(7), G.2.3(25),  
   G.2.4(2), G.2.6(3)  
 Overlap  
   *in* Ada.Containers.Hash\_Sets  
     A.18.8(38/2)  
   *in* Ada.Containers.Ordered\_Sets  
     A.18.9(39/2)  
 overload resolution 8.6(1)  
 overloadable 8.3(7)  
 overloaded 8.3(6)  
   enumeration literal 3.5.1(9)  
 overloading rules 1.1.2(26), 8.6(2)  
 overridable 8.3(9/1)  
 override 8.3(9/1), 12.3(17)  
   a primitive subprogram 3.2.3(7/2)  
   when implemented by 9.1(9.2/3),  
     9.4(11.1/3)  
 overriding operation N(24.1/2)  
 overriding\_indicator 8.3.1(2/2)  
   *used* 3.9.3(1.1/2), 6.1(2/2), 6.3(2/2),  
   6.7(2/2), 8.5.4(2/2), 9.5.2(2/2),  
   10.1.3(3/2), 12.3(2/2), P  
 Overwrite  
   *in* Ada.Strings.Bounded A.4.4(62),  
   A.4.4(63)  
   *in* Ada.Strings.Fixed A.4.3(27),  
   A.4.3(28)  
   *in* Ada.Strings.Unbounded A.4.5(57),  
   A.4.5(58)

## P

Pack pragma 13.2(3), L(24)  
 Package 7(1), N(25)  
 package instance 12.3(13)  
 package\_body 7.2(2)  
   *used* 3.11(6), 10.1.1(7), P  
 package\_body\_stub 10.1.3(4)  
   *used* 10.1.3(2), P  
 package\_declaration 7.1(2)  
   *used* 3.1(3/2), 10.1.1(5), P  
 package\_renaming\_declaration 8.5.3(2)  
   *used* 8.5(2), 10.1.1(6), P  
 package\_specification 7.1(3)  
   *used* 7.1(2), 12.1(4), P  
 packed 13.2(5)  
 Packed\_Decimal  
   *in* Interfaces.COBOL B.4(12)  
 Packed\_Format  
   *in* Interfaces.COBOL B.4(26)  
 Packed\_Signed  
   *in* Interfaces.COBOL B.4(27)  
 Packed\_Unsigned  
   *in* Interfaces.COBOL B.4(27)

packing  
   aspect of representation 13.2(5)  
 padding bits 13.1(7/2)  
 Page  
   *in* Ada.Text\_IO A.10.1(39)  
 Page pragma 2.8(22), L(25)  
 page terminator A.10(7)  
 Page\_Length  
   *in* Ada.Text\_IO A.10.1(26)  
 Paragraph\_Sign  
   *in* Ada.Characters.Latin\_1 A.3.3(22)  
 parallel processing  
   *See* task 9(1)  
 parameter  
   *See* formal parameter 6.1(17)  
   *See* generic formal parameter 12(1)  
   *See also* discriminant 3.7(1/2)  
   *See also* loop parameter 5.5(6)  
 parameter assigning back 6.4.1(17)  
 parameter copy back 6.4.1(17)  
 parameter mode 6.1(18/3)  
 parameter passing 6.4.1(1)  
 parameter\_and\_result\_profile 6.1(13/2)  
   *used* 3.10(5), 3.10(6/2), 6.1(4.2/2), P  
 parameter\_association 6.4(5)  
   *used* 6.4(4), P  
 parameter\_profile 6.1(12)  
   *used* 3.10(5), 3.10(6/2), 6.1(4.1/2),  
   9.5.2(2/2), 9.5.2(3), 9.5.2(6), P  
 parameter\_specification 6.1(15/2)  
   *used* 6.1(14), P  
 Parameterless\_Handler  
   *in* Ada.Interrupts C.3.2(2)  
 Params\_Stream\_Type  
   *in* System.RPC E.5(6)  
 parent N(25.1/2)  
   *in* Ada.Containers.Multiway\_Trees  
     A.18.10(52/3)  
 parent body  
   of a subunit 10.1.3(8/2)  
 parent\_declaration  
   of a library unit 10.1.1(10)  
   of a library\_item 10.1.1(10)  
 parent\_subtype 3.4(3/2)  
 parent\_type 3.4(3/2)  
 parent\_unit  
   of a library unit 10.1.1(10)  
 Parent\_Tag  
   *in* Ada.Tags 3.9(7.2/2)  
 parent\_unit\_name 10.1.1(8)  
   *used* 6.1(5), 6.1(7), 7.1(3), 7.2(2),  
   10.1.3(7), P  
 part  
   of an object or value 3.2(6/2)  
 partial view  
   of a type 7.3(4)  
 partition 10.2(2), N(26)  
 partition\_building 10.2(2)  
 partition\_communication\_subsystem  
   (PCS) E.5(1/2)

Partition\_Check  
   [*partial*] E.4(19)  
 Partition\_Elaboration\_Policy pragma  
   H.6(3/2), L(25.1/2)  
 Partition\_Id  
   *in* System.RPC E.5(4)  
 Partition\_Id attribute E.1(9)  
 pass by copy 6.2(2)  
 pass by reference 6.2(2)  
 passive partition E.1(2)  
 Pattern\_Error  
   *in* Ada.Strings A.4.1(5)  
 PCS (partition communication  
   subsystem) E.5(1/2)  
 pending interrupt occurrence C.3(2)  
 per-object constraint 3.8(18/2)  
 per-object expression 3.8(18/2)  
 percent sign 2.1(15/2)  
 Percent\_Sign  
   *in* Ada.Characters.Latin\_1 A.3.3(8)  
 perfect result set G.2.3(5)  
 periodic task  
   example 9.6(39)  
   *See* delay\_until\_statement 9.6(39)  
 Pi  
   *in* Ada.Numerics A.5(3/2)  
 Pic\_String  
   *in* Ada.Text\_IO Editing F.3.3(7)  
 Picture  
   *in* Ada.Text\_IO Editing F.3.3(4)  
 picture String  
   for edited output F.3.1(1)  
 Picture\_Error  
   *in* Ada.Text\_IO Editing F.3.3(9)  
 Pilcrow\_Sign  
   *in* Ada.Characters.Latin\_1 A.3.3(22)  
 plain\_char  
   *in* Interfaces.C B.3(11)  
 plane  
   character 2.1(1/2)  
 PLD  
   *in* Ada.Characters.Latin\_1 A.3.3(17)  
 PLU  
   *in* Ada.Characters.Latin\_1 A.3.3(17)  
 plus operator 4.4(1), 4.5.3(1), 4.5.4(1)  
 plus sign 2.1(15/2)  
 Plus\_Minus\_Sign  
   *in* Ada.Characters.Latin\_1 A.3.3(22)  
 Plus\_Sign  
   *in* Ada.Characters.Latin\_1 A.3.3(8)  
 PM  
   *in* Ada.Characters.Latin\_1 A.3.3(19)  
 point 2.1(15/2)  
 Pointer  
   *in* Interfaces.C.Pointers B.3.2(5)  
   *See* access value 3.10(1)  
   *See* type System.Address 13.7(34/2)  
 pointer type  
   *See* access type 3.10(1)

- Pointer\_Error
  - in* Interfaces.C.Pointers B.3.2(8)
- Pointers
  - child of* Interfaces.C B.3.2(4)
- polymorphism 3.9(1), 3.9.2(1/2)
- pool element 3.10(7/1), 13.11(11)
- pool type 13.11(11)
- pool-specific access type 3.10(7/1), 3.10(8)
- Pos attribute 3.5.5(2)
- position 13.5.1(4)
  - used* 13.5.1(3), P
- Position attribute 13.5.2(2/2)
- position number 3.5(1)
  - of an enumeration value 3.5.1(7)
  - of an integer value 3.5.4(15)
- positional association 6.4(7), 6.4.1(2/3), 12.3(6)
- positional component association 4.3.1(6)
- positional discriminant association 3.7.1(4)
- positional parameter association 6.4.1(2/3)
- positional\_array\_aggregate 4.3.3(3/2)
  - used* 4.3.3(2), P
- Positive 3.5.4(12)
- Positive *subtype of* Integer
  - in* Standard A.1(13)
- Positive\_Count *subtype of* Count
  - in* Ada.Direct\_IO A.8.4(4)
  - in* Ada.Streams.Stream\_IO A.12.1(7)
  - in* Ada.Text\_IO A.10.1(5)
- possible interpretation 8.6(14)
  - for direct\_names 8.3(24)
  - for selector\_names 8.3(24)
- post-compilation error 1.1.2(29)
- post-compilation rules 1.1.2(29)
- potentially blocking operation 9.5.1(8)
  - Abort\_Task C.7.1(16)
  - delay\_statement 9.6(34), D.9(5)
  - remote subprogram call E.4(17)
  - RPC operations E.5(23)
  - Suspend\_Until\_True D.10(10)
- potentially use-visible 8.4(8/3)
- Pound\_Sign
  - in* Ada.Characters.Latin\_1 A.3.3(21/3)
- Pragma 2.8(1), 2.8(2), L(1), N(27)
- pragma argument 2.8(9)
- pragma name 2.8(9)
- pragma, categorization E.2(2)
  - Remote\_Call\_Interface E.2.3(2)
  - Remote\_Types E.2.2(2)
  - Shared\_Passive E.2.1(2)
- pragma, configuration 10.1.5(8)
  - Assertion\_Policy 11.4.2(7/2)
  - Detect\_Blocking H.5(4/2)
  - Discard\_Names C.5(4)
  - Locking\_Policy D.3(5)
  - Normalize\_Scalars H.1(4)
- Partition\_Elaboration\_Policy H.6(5/2)
- Priority\_Specific\_Dispatching
  - D.2.2(4/2)
- Profile D.13(6/2)
- Queuing\_Policy D.4(5)
- Restrictions 13.12(8/3)
- Reviewable H.3.1(4)
- Suppress 11.5(5/2)
- Task\_Dispatching\_Policy D.2.2(4/2)
- Unsuppress 11.5(5/2)
- pragma, identifier specific to 2.8(10)
- pragma, interfacing
  - Convention B.1(4)
  - Export B.1(4)
  - Import B.1(4)
  - Linker\_Options B.1(4)
- pragma, library unit 10.1.5(7/3)
  - All\_Calls\_Remote E.2.3(6)
  - categorization pragmas E.2(2)
  - Elaborate\_Body 10.2.1(24)
  - Preelaborate 10.2.1(4)
  - Pure 10.2.1(15)
- pragma, program unit 10.1.5(2)
  - Convention B.1(29)
  - Export B.1(29)
  - Import B.1(29)
  - Inline 6.3.2(2)
  - library unit pragmas 10.1.5(7/3)
- pragma, representation 13.1(1/1)
  - Asynchronous E.4.1(8)
  - Atomic C.6(14)
  - Atomic\_Components C.6(14)
  - Controlled 13.11.3(5)
  - Convention B.1(28)
  - Discard\_Names C.5(6)
  - Export B.1(28)
  - Import B.1(28)
  - Independent C.6(14)
  - Independent\_Components C.6(14)
  - Pack 13.2(5)
  - Volatile C.6(14)
  - Volatile\_Components C.6(14)
- pragma\_argument\_association 2.8(3)
  - used* 2.8(2), D.13(3/2), L(27.2/2), P
- pragmas
  - All\_Calls\_Remote E.2.3(5), L(2)
  - Assert 11.4.2(3/2), L(2.1/2)
  - Assertion\_Policy 11.4.2(6/2), L(2.2/2)
  - Asynchronous E.4.1(3), L(3)
  - Atomic C.6(3), L(4)
  - Atomic\_Components C.6(5), L(5)
  - Attach\_Handler C.3.1(4), L(6)
  - Controlled 13.11.3(3), L(7)
  - Convention B.1(7), L(8)
  - Detect\_Blocking H.5(3/2), L(8.1/2)
  - Discard\_Names C.5(3), L(9)
  - Elaborate 10.2.1(20), L(10)
  - Elaborate\_All 10.2.1(21), L(11)
  - Elaborate\_Body 10.2.1(22), L(12)
  - Export B.1(6), L(13)
- Implemented 9.5(11/3), L(13.1/2)
- Import B.1(5), L(14)
- Independent C.6(4.1/3), L(14.1/3)
- Independent\_Components C.6(6.1/3), L(14.2/3)
- Inline 6.3.2(3), L(15)
- Inspection\_Point H.3.2(3), L(16)
- Interrupt\_Handler C.3.1(2), L(17)
- Interrupt\_Priority D.1(5), L(18)
- Linker\_Options B.1(8), L(19)
- List 2.8(21), L(20)
- Locking\_Policy D.3(3), L(21)
- No\_Return 6.5.1(3/2), L(21.1/2)
- Normalize\_Scalars H.1(3), L(22)
- Optimize 2.8(23), L(23)
- Pack 13.2(3), L(24)
- Page 2.8(22), L(25)
- Partition\_Elaboration\_Policy H.6(3/2), L(25.1/2)
- Preelaborable\_Initialization
  - 10.2.1(4.2/2), L(25.2/2)
- Preelaborate 10.2.1(3), L(26)
- Priority D.1(3), L(27)
- Priority\_Specific\_Dispatching
  - D.2.2(2.2/2), L(27.1/2)
- Profile D.13(3/2), L(27.2/2)
- Pure 10.2.1(14), L(28)
- Queuing\_Policy D.4(3), L(29)
- Relative\_Deadline D.2.6(4/2), L(29.1/2)
- Remote\_Call\_Interface E.2.3(3), L(30)
- Remote\_Types E.2.2(3), L(31)
- Restrictions 13.12(3), L(32)
- Reviewable H.3.1(3), L(33)
- Shared\_Passive E.2.1(3), L(34)
- Storage\_Size 13.3(63), L(35)
- Suppress 11.5(4/2), J.10(3/2), L(36)
- Task\_Dispatching\_Policy D.2.2(2), L(37)
- Unchecked\_Union B.3.3(3/2), L(37.1/2)
- Unsuppress 11.5(4.1/2), L(37.2/2)
- Volatile C.6(4), L(38)
- Volatile\_Components C.6(6), L(39)
- precedence of operators 4.5(1)
- Pred attribute 3.5(25)
- predefined environment A(1)
- predefined exception 11.1(4)
- predefined library unit
  - See* language-defined library units
- predefined operation
  - of a type 3.2.3(1/2)
- predefined operations
  - of a discrete type 3.5.5(10)
  - of a fixed point type 3.5.10(17)
  - of a floating point type 3.5.8(3)
  - of a record type 3.8(24)
  - of an access type 3.10.2(34/2)
  - of an array type 3.6.2(15)
- predefined operator 4.5(9)

- [*partial*] 3.2.1(9)
- predefined type 3.2.1(10)
  - See language-defined types
- predicate 4.5.7(3/3)
  - used 4.5.7(1/3), P
- preelaborable
  - of an elaborable construct 10.2.1(5)
- preelaborable initialization
  - 10.2.1(11.1/2)
- Preelaborable\_Initialization pragma
  - 10.2.1(4.2/2), L(25.2/2)
- Preelaborate pragma 10.2.1(3), L(26)
- preelaborated 10.2.1(11/3)
  - [*partial*] 10.2.1(11/3), E.2.1(9)
- preempt
  - a running task D.2.3(9/2)
- preference
  - for root numeric operators and ranges 8.6(29)
  - for universal access equality operators 8.6(29.1/3)
- preference control
  - See requeue 9.5.4(1)
- prefix 4.1(4)
  - of a prefixed view 4.1.3(9.2/3)
  - used 4.1.1(2), 4.1.2(2), 4.1.3(2), 4.1.4(2), 4.1.4(4), 6.4(2), 6.4(3), P
- prefixed view 4.1.3(9.2/3)
- prefixed view profile 6.3.1(24.1/2)
- Prepend
  - in Ada.Containers.Doubly\_Linked\_Lists A.18.3(22/2)
  - in Ada.Containers.Vectors A.18.2(44/2), A.18.2(45/2)
- Prepend\_Child
  - in Ada.Containers.Multiway\_Trees A.18.10(44/3)
- prescribed result
  - for the evaluation of a complex arithmetic operation G.1.1(42)
  - for the evaluation of a complex elementary function G.1.2(35)
  - for the evaluation of an elementary function A.5.1(37)
- Previous
  - in Ada.Containers.Doubly\_Linked\_Lists A.18.3(38/2), A.18.3(40/2)
  - in Ada.Containers.Ordered\_Maps A.18.6(36/2), A.18.6(37/2)
  - in Ada.Containers.Ordered\_Sets A.18.9(47/2), A.18.9(48/2)
  - in Ada.Containers.Vectors A.18.2(65/2), A.18.2(66/2)
- Previous\_Sibling
  - in Ada.Containers.Multiway\_Trees A.18.10(58/3), A.18.10(60/3)
- primary 4.4(7/3)
  - used 4.4(6), P
- primitive function A.5.3(17)
- primitive operation
  - [*partial*] 3.2(1)
- primitive operations N(28)
  - of a type 3.2.3(1/2)
- primitive operator
  - of a type 3.2.3(8)
- primitive subprograms
  - of a type 3.2.3(2)
- priority D.1(15)
  - of a protected object D.3(6/2)
- Priority attribute D.5.2(3/2)
- priority inheritance D.1(15)
- priority inversion D.2.3(11/2)
- priority of an entry call D.4(9)
- Priority pragma D.1(3), L(27)
- Priority\_subtype of Any\_Priority in System 13.7(16)
- Priority\_Queueing queueing policy D.4(8)
- Priority\_Specific\_Dispatching pragma D.2.2(2.2/2), L(27.1/2)
- private declaration of a library unit 10.1.1(12)
- private descendant
  - of a library unit 10.1.1(12)
- private extension 3.2(4.1/2), 3.9(2.1/2), 3.9.1(1/2), N(29/2)
  - [*partial*] 7.3(14), 12.5.1(5/2)
- private library unit 10.1.1(12)
- private operations 7.3.1(1)
- private part 8.2(5)
  - of a package 7.1(6/2)
  - of a protected unit 9.4(11/2)
  - of a task unit 9.1(9)
- private type 3.2(4.1/2), N(30/2)
  - [*partial*] 7.3(14)
- private types and private extensions 7.3(1)
- private\_extension\_declaration 7.3(3/2)
  - used 3.2.1(2), P
- private\_type\_declaration 7.3(2)
  - used 3.2.1(2), P
- procedure 6(1), N(30.1/2)
  - null 6.7(3/2)
- procedure instance 12.3(13)
- procedure\_call\_statement 6.4(2)
  - used 5.1(4/2), 9.7.2(3.1/2), P
- procedure\_or\_entry\_call 9.7.2(3.1/2)
  - used 9.7.2(3/2), 9.7.4(4/2), P
- procedure\_specification 6.1(4.1/2)
  - used 6.1(4/2), 6.7(2/2), P
- processing node E(2)
- profile 6.1(22)
  - associated with a dereference 4.1(10)
  - fully conformant 6.3.1(18/3)
  - mode conformant 6.3.1(16/3)
  - subtype conformant 6.3.1(17)
  - type conformant 6.3.1(15/2)
- Profile pragma D.13(3/2), L(27.2/2)
- profile resolution rule
  - name with a given expected profile 8.6(26)
- progenitor N(30.2/2)
- progenitor subtype 3.9.4(9/2)
- progenitor type 3.9.4(9/2)
- program 10.2(1), N(31)
- program execution 10.2(1)
- program library
  - See library 10(2)
  - See library 10.1.4(9)
- Program unit 10.1(1), N(32)
- program unit pragma 10.1.5(2)
  - Convention B.1(29)
  - Export B.1(29)
  - Import B.1(29)
  - Inline 6.3.2(2)
  - library unit pragmas 10.1.5(7/3)
- Program\_Error
  - raised by failure of run-time check 1.1.3(20), 1.1.5(8), 1.1.5(12), 3.5.5(8), 3.10.2(29), 3.11(14), 4.6(57), 4.8(10.1/3), 4.8(10.2/2), 4.8(10.3/2), 6.2(12), 6.4(11/2), 6.5(8/3), 6.5(21/2), 6.5.1(9/2), 7.6.1(15), 7.6.1(16/2), 7.6.1(17), 7.6.1(17.2/1), 7.6.1(18/2), 8.5.4(8.1/1), 9.4(20), 9.5.1(17), 9.5.3(7), 9.7.1(21), 9.8(20), 10.2(26), 11.1(4), 11.5(19), 12.5.1(23.3/2), 13.7.1(16), 13.9.1(9), 13.11.2(13), 13.11.2(14), A.5.2(40.1/1), A.7(14), B.3.3(22/2), C.3.1(10), C.3.1(11/2), C.3.2(17), C.3.2(20), C.3.2(21), C.3.2(22/2), C.7.1(15), C.7.1(17/3), C.7.2(13), D.3(13), D.3(13.2/2), D.3(13.4/2), D.5.1(9), D.5.2(6/2), D.7(19.1/2), D.10(10), D.11(8), E.1(10/2), E.3(6), E.4(18/1), J.7.1(7) in Standard A.1(46)
- propagate 11.4(1)
  - an exception occurrence by an execution, to a dynamically enclosing execution 11.4(6)
- proper\_body 3.11(6)
  - used 3.11(5), 10.1.3(7), P
- protected action 9.5.1(4)
  - complete 9.5.1(6)
  - start 9.5.1(5)
- protected calling convention 6.3.1(12)
- protected declaration 9.4(1)
- protected entry 9.4(1)
- protected function 9.5.1(1)
- protected interface 3.9.4(5/2)
- protected object 9(3), 9.4(1)
- protected operation 9.4(1)
- protected procedure 9.5.1(1)
- protected subprogram 9.4(1), 9.5.1(1)
- protected tagged type 3.9.4(6/2)
- protected type N(33/2)
- protected unit 9.4(1)

- protected\_body 9.4(7)  
*used* 3.11(6), P
- protected\_body\_stub 10.1.3(6)  
*used* 10.1.3(2), P
- protected\_definition 9.4(4)  
*used* 9.4(2/2), 9.4(3/2), P
- protected\_element\_declaration 9.4(6)  
*used* 9.4(4), P
- protected\_operation\_declaration 9.4(5/1)  
*used* 9.4(4), 9.4(6), P
- protected\_operation\_item 9.4(8/1)  
*used* 9.4(7), P
- protected\_type\_declaration 9.4(2/2)  
*used* 3.2.1(3), P
- ptrdiff\_t  
*in* Interfaces.C B.3(12)
- PU1  
*in* Ada.Characters.Latin\_1 A.3.3(18)
- PU2  
*in* Ada.Characters.Latin\_1 A.3.3(18)
- public declaration of a library unit  
 10.1.1(12)
- public descendant  
 of a library unit 10.1.1(12)
- public library unit 10.1.1(12)
- punctuation\_connector 2.1(10.2/2)  
*used* 2.3(3.1/3), P
- pure 10.2.1(15.1/3)
- Pure pragma 10.2.1(14), L(28)
- Put  
*in* Ada.Text\_IO A.10.1(42),  
 A.10.1(48), A.10.1(55), A.10.1(60),  
 A.10.1(66), A.10.1(67), A.10.1(71),  
 A.10.1(72), A.10.1(76), A.10.1(77),  
 A.10.1(82), A.10.1(83)
- in* Ada.Text\_IO.Bounded\_IO  
 A.10.11(4/2), A.10.11(5/2)
- in* Ada.Text\_IO.Complex\_IO G.1.3(7),  
 G.1.3(8)
- in* Ada.Text\_IO Editing F.3.3(14),  
 F.3.3(15), F.3.3(16)
- in* Ada.Text\_IO.Unbounded\_IO  
 A.10.12(4/2), A.10.12(5/2)
- Put\_Line  
*in* Ada.Text\_IO A.10.1(50)
- in* Ada.Text\_IO.Bounded\_IO  
 A.10.11(6/2), A.10.11(7/2)
- in* Ada.Text\_IO.Unbounded\_IO  
 A.10.12(6/2), A.10.12(7/2)
- Q**
- qualified\_expression 4.7(2)  
*used* 4.1(2/3), 4.8(2), 13.8(2), P
- quantified expressions 4.5.7(5/3)
- quantified\_expression 4.5.7(1/3)  
*used* 4.4(7/3), P
- quantifier 4.5.7(2/3)  
*used* 4.5.7(1/3), P
- Query\_Element  
*in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(16/2)
- in* Ada.Containers.Hashed\_Maps  
 A.18.5(16/2)
- in* Ada.Containers.Hashed\_Sets  
 A.18.8(17/2)
- in* Ada.Containers.Indefinite\_Holders  
 A.18.18(14/3)
- in* Ada.Containers.Multiway\_Trees  
 A.18.10(24/3)
- in* Ada.Containers.Ordered\_Maps  
 A.18.6(15/2)
- in* Ada.Containers.Ordered\_Sets  
 A.18.9(16/2)
- in* Ada.Containers.Vectors  
 A.18.2(31/2), A.18.2(32/2)
- Question  
*in* Ada.Characters.Latin\_1 A.3.3(10)
- queuing policy D.4(1/1), D.4(6)
- FIFO\_Queueing D.4(7/2)
- Priority\_Queueing D.4(8)
- Queueing\_Policy pragma D.4(3), L(29)
- Quotation  
*in* Ada.Characters.Latin\_1 A.3.3(8)
- quotation mark 2.1(15/2)
- quoted string  
*See* string\_literal 2.6(1)
- R**
- raise  
 an exception 11(1)  
 an exception 11.3(4/2)  
 an exception N(18)  
 an exception occurrence 11.4(3)
- Raise\_Exception  
*in* Ada.Exceptions 11.4.1(4/2)
- raise\_statement 11.3(2/2)  
*used* 5.1(4/2), P
- Random  
*in* Ada.Numerics.Discrete\_Random  
 A.5.2(20)
- in* Ada.Numerics.Float\_Random  
 A.5.2(8)
- random number A.5.2(1)
- range 3.5(3), 3.5(4)  
 of a scalar subtype 3.5(7)  
*used* 3.5(2), 3.6(6), 3.6.1(3), 4.4(3), P
- Range attribute 3.5(14), 3.6.2(7)
- Range(N) attribute 3.6.2(8)
- range\_attribute\_designator 4.1.4(5)  
*used* 4.1.4(4), P
- range\_attribute\_reference 4.1.4(4)  
*used* 3.5(3), P
- Range\_Check 11.5(17)
- [*partial*] 3.2.2(11), 3.5(24), 3.5(27),  
 3.5(39.12/2), 3.5(39.4/2), 3.5(39.5/2),  
 3.5(43/2), 3.5(55/2), 3.5.5(7),  
 3.5.9(19), 4.2(11), 4.3.3(28), 4.5.1(8),  
 4.5.6(6), 4.5.6(13), 4.6(28), 4.6(38),  
 4.6(46), 4.6(51/2), 4.7(4),  
 13.13.2(35/2), A.5.2(39), A.5.3(26),  
 A.5.3(29), A.5.3(50), A.5.3(53),  
 A.5.3(59), A.5.3(62), K(11), K(114),  
 K(122), K(184), K(220), K(241),  
 K(41), K(47)
- range\_constraint 3.5(2)  
*used* 3.2.2(6), 3.5.9(5), J.3(2), P
- Ravenscar D.13.1(1/2)
- RCI  
 generic E.2.3(7/1)  
 library unit E.2.3(7/1)  
 package E.2.3(7/1)
- Re  
*in* Ada.Numerics.Generic\_Complex\_Arrays  
 G.3.2(7/2), G.3.2(27/2)
- in* Ada.Numerics.Generic\_Complex\_Types  
 G.1.1(6)
- re-raise statement 11.3(3)
- read  
 the value of an object 3.3(14)  
*in* Ada.Direct\_IO A.8.4(12)
- in* Ada.Sequential\_IO A.8.1(12)
- in* Ada.Storage\_IO A.9(6)
- in* Ada.Streams 13.13.1(5)
- in* Ada.Streams.Stream\_IO A.12.1(15),  
 A.12.1(16)
- in* System.RPC E.5(7)
- Read attribute 13.13.2(6), 13.13.2(14)
- Read clause 13.3(7/2), 13.13.2(38/3)
- ready  
 a task state 9(10)
- ready queue D.2.1(5/2)
- ready task D.2.1(5/2)
- Real  
*in* Interfaces.Fortran B.5(6)
- real literal 2.4(1)
- real literals 3.5.6(4)
- real time D.8(18)
- real type 3.2(3), 3.5.6(1), N(34)
- real-time systems C(1), D(1)
- Real\_Arrays  
*child of* Ada.Numerics G.3.1(31/2)
- Real\_Matrix  
*in* Ada.Numerics.Generic\_Real\_Arrays  
 G.3.1(4/2)
- real\_range\_specification 3.5.7(3)  
*used* 3.5.7(2), 3.5.9(3), 3.5.9(4), P
- Real\_Time  
*child of* Ada D.8(3)
- real\_type\_definition 3.5.6(2)  
*used* 3.2.1(4/2), P
- Real\_Vector  
*in* Ada.Numerics.Generic\_Real\_Arrays  
 G.3.1(4/2)



|                                                   |                                                                     |                                                                                      |
|---------------------------------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| receiving stub E.4(10)                            | Relative_Deadline pragma D.2.6(4/2),<br>L(29.1/2)                   | <i>in</i> Ada.Containers.Indefinite_Holders<br>A.18.18(13/3)                         |
| reclamation of storage 13.11.2(1)                 | Relative_Name                                                       | <i>in</i> Ada.Containers.Multiway_Trees<br>A.18.10(23/3)                             |
| recommended level of support 13.1(20)             | <i>in</i><br>Ada.Directories.Hierarchical_File_Names A.16.1(13/3)   | <i>in</i> Ada.Containers.Ordered_Maps<br>A.18.6(14/2)                                |
| Address attribute 13.3(15)                        | relaxed mode G.2(1)                                                 | <i>in</i> Ada.Containers.Ordered_Sets<br>A.18.9(15/2)                                |
| Alignment attribute for objects 13.3(33)          | release                                                             | <i>in</i> Ada.Containers.Vectors<br>A.18.2(29/2), A.18.2(30/2)                       |
| Alignment attribute for subtypes 13.3(29)         | execution resource associated with<br>protected object 9.5.1(6)     | <i>in</i> Ada.Strings.Bounded A.4.4(27)<br><i>in</i> Ada.Strings.Unbounded A.4.5(21) |
| bit ordering 13.5.3(7)                            | rem operator 4.4(1), 4.5.5(1)                                       | Replace_Slice<br><i>in</i> Ada.Strings.Bounded A.4.4(58),<br>A.4.4(59)               |
| Component_Size attribute 13.3(71)                 | Remainder attribute A.5.3(45)                                       | <i>in</i> Ada.Strings.Fixed A.4.3(23),<br>A.4.3(24)                                  |
| enumeration_representation_clause 13.4(9)         | remote access E.1(5)                                                | <i>in</i> Ada.Strings.Unbounded A.4.5(53),<br>A.4.5(54)                              |
| pragma Pack 13.2(7)                               | remote access type E.2.2(9/3)                                       | Replenish<br><i>in</i><br>Ada.Execution_Time.Group_Budgets<br>D.14.2(9/2)            |
| record_representation_clause 13.5.1(17)           | remote access-to-class-wide type<br>E.2.2(9/3)                      | Replicate<br><i>in</i> Ada.Strings.Bounded A.4.4(78),<br>A.4.4(79), A.4.4(80)        |
| required in Systems Programming<br>Annex C.2(2)   | remote access-to-subprogram type<br>E.2.2(9/3)                      | representation<br>change of 13.6(1)                                                  |
| Size attribute 13.3(42/2), 13.3(54)               | remote call interface E.2(4/1), E.2.3(7/1)                          | representation aspect 13.1(8/3)                                                      |
| Stream_Size attribute 13.13.2(1.7/2)              | remote procedure call                                               | representation attribute 13.3(1/1)                                                   |
| unchecked conversion 13.9(16)                     | asynchronous E.4.1(9)                                               | representation item 13.1(1/1)                                                        |
| with respect to nonstatic expressions<br>13.1(21) | remote subprogram E.2.3(7/1)                                        | representation of an object 13.1(7/2)                                                |
| record 3.8(1)                                     | remote subprogram binding E.4(1)                                    | representation pragma 13.1(1/1)                                                      |
| explicitly limited 3.8(13.1/3)                    | remote subprogram call E.4(1)                                       | Asynchronous E.4.1(8)                                                                |
| record extension 3.4(5/2), 3.9.1(1/2),<br>N(35)   | remote types library unit E.2(4/1),<br>E.2.2(4)                     | Atomic C.6(14)                                                                       |
| record layout                                     | Remote_Call_Interface pragma E.2.3(3),<br>L(30)                     | Atomic_Components C.6(14)                                                            |
| aspect of representation 13.5(1)                  | Remote_Types pragma E.2.2(3), L(31)                                 | Controlled 13.11.3(5)                                                                |
| record type 3.8(1), N(36)                         | Remove_Task                                                         | Convention B.1(28)                                                                   |
| record_aggregate 4.3.1(2)                         | <i>in</i><br>Ada.Execution_Time.Group_Budgets<br>D.14.2(8/2)        | Discard_Names C.5(6)                                                                 |
| <i>used</i> 4.3(2), P                             | Rename                                                              | Export B.1(28)                                                                       |
| record_component_association 4.3.1(4/2)           | <i>in</i> Ada.Directories A.16(12/2)                                | Import B.1(28)                                                                       |
| <i>used</i> 4.3.1(3), P                           | renamed entity 8.5(3)                                               | Independent C.6(14)                                                                  |
| record_component_association_list<br>4.3.1(3)     | renamed view 8.5(3)                                                 | Independent_Components C.6(14)                                                       |
| <i>used</i> 4.3.1(2), 4.3.2(2), P                 | renaming N(36.1/2)                                                  | Pack 13.2(5)                                                                         |
| record_definition 3.8(3)                          | renaming-as-body 8.5.4(1)                                           | Volatile C.6(14)                                                                     |
| <i>used</i> 3.8(2), 3.9.1(2), P                   | renaming-as-declaration 8.5.4(1)                                    | Volatile_Components C.6(14)                                                          |
| record_extension_part 3.9.1(2)                    | renaming_declaration 8.5(2)                                         | representation-oriented attributes<br>of a fixed point subtype A.5.4(1)              |
| <i>used</i> 3.4(2/2), P                           | <i>used</i> 3.1(3/2), P                                             | of a floating point subtype A.5.3(1)                                                 |
| record_representation_clause 13.5.1(2)            | rendezvous 9.5.2(25)                                                | representation_clause<br><i>See</i> aspect_clause 13.1(4/1)                          |
| <i>used</i> 13.1(2/1), P                          | Replace                                                             | represented in canonical form A.5.3(10)                                              |
| record_type_definition 3.8(2)                     | <i>in</i> Ada.Containers.Hashed_Maps<br>A.18.5(23/2)                | requested decimal precision<br>of a floating point type 3.5.7(4)                     |
| <i>used</i> 3.2.1(4/2), P                         | <i>in</i> Ada.Containers.Hashed_Sets<br>A.18.8(22/2), A.18.8(53/2)  | requeue 9.5.4(1)                                                                     |
| reentrant A(3/2)                                  | <i>in</i> Ada.Containers.Ordered_Maps<br>A.18.6(22/2)               | requeue target 9.5.4(3/3)                                                            |
| Reference                                         | <i>in</i> Ada.Containers.Ordered_Sets<br>A.18.9(21/2), A.18.9(66/2) | requeue-with-abort 9.5.4(13)                                                         |
| <i>in</i> Ada.Interrupts C.3.2(10)                | Replace_Element                                                     | requeue_statement 9.5.4(2/3)<br><i>used</i> 5.1(4/2), P                              |
| <i>in</i> Ada.Task_Attributes C.7.2(5)            | <i>in</i> Ada.Containers.Doubly_Linked_<br>Lists A.18.3(15/2)       | require overriding 3.9.3(6/2)                                                        |
| reference parameter passing 6.2(2)                | <i>in</i> Ada.Containers.Hashed_Maps<br>A.18.5(15/2)                |                                                                                      |
| references 1.2(1)                                 | <i>in</i> Ada.Containers.Hashed_Sets<br>A.18.8(16/2)                |                                                                                      |
| Registered_Trade_Mark_Sign                        |                                                                     |                                                                                      |
| <i>in</i> Ada.Characters.Latin_1 A.3.3(21/3)      |                                                                     |                                                                                      |
| Reinitialize                                      |                                                                     |                                                                                      |
| <i>in</i> Ada.Task_Attributes C.7.2(6)            |                                                                     |                                                                                      |
| relation 4.4(3)                                   |                                                                     |                                                                                      |
| <i>used</i> 4.4(2), P                             |                                                                     |                                                                                      |
| relational operator 4.5.2(1)                      |                                                                     |                                                                                      |
| relational_operator 4.5(3)                        |                                                                     |                                                                                      |
| <i>used</i> 4.4(3), P                             |                                                                     |                                                                                      |

requires a completion 3.11.1(1/1), 3.11.1(6)  
 declaration of a partial view 7.3(4)  
 declaration to which a pragma  
   Elaborate\_Body applies 10.2.1(25)  
 deferred constant declaration 7.4(2)  
 generic\_package\_declaration 7.1(5/2)  
 generic\_subprogram\_declaration 6.1(20/2)  
 incomplete\_type\_declaration 3.10.1(3/3)  
 package\_declaration 7.1(5/2)  
 protected entry\_declaration 9.5.2(16)  
 protected\_declaration 9.4(11.2/2)  
 subprogram\_declaration 6.1(20/2)  
 task\_declaration 9.1(9.3/2)  
 requires late initialization 3.3.1(8.1/2)  
 Raise\_Occurrence  
   in Ada.Exceptions 11.4.1(4/2)  
 Reserve\_Capacity  
   in Ada.Containers.Hashed\_Maps A.18.5(9/2)  
   in Ada.Containers.Hashed\_Sets A.18.8(11/2)  
   in Ada.Containers.Vectors A.18.2(20/2)  
 reserved interrupt C.3(2)  
 reserved word 2.9(2/3)  
 Reserved\_128  
   in Ada.Characters.Latin\_1 A.3.3(17)  
 Reserved\_129  
   in Ada.Characters.Latin\_1 A.3.3(17)  
 Reserved\_132  
   in Ada.Characters.Latin\_1 A.3.3(17)  
 Reserved\_153  
   in Ada.Characters.Latin\_1 A.3.3(19)  
 Reserved\_Check  
   [*partial*] C.3.1(10)  
 Reset  
   in Ada.Direct\_IO A.8.4(8)  
   in Ada.Numerics.Discrete\_Random A.5.2(21), A.5.2(24)  
   in Ada.Numerics.Float\_Random A.5.2(9), A.5.2(12)  
   in Ada.Sequential\_IO A.8.1(8)  
   in Ada.Streams.Stream\_IO A.12.1(10)  
   in Ada.Text\_IO A.10.1(11)  
 resolution rules 1.1.2(26)  
 resolve  
   overload resolution 8.6(14)  
 restriction 13.12(4/2)  
   *used* 13.12(3), L(32)  
 restriction\_parameter\_argument 13.12(4.1/2)  
   *used* 13.12(4/2), P  
 restrictions  
   Immediate\_Reclamation H.4(10)  
   Max\_Asynchronous\_Select\_Nesting D.7(18/1)  
   Max\_Entry\_Queue\_Length D.7(19.1/2)  
   Max\_Protected\_Entries D.7(14)  
   Max\_Select\_Alternatives D.7(12)  
   Max\_Storage\_At\_Blocking D.7(17/1)  
   Max\_Task\_Entries D.7(13)  
   Max\_Tasks D.7(19/1)  
   No\_Abort\_Statements D.7(5)  
   No\_Access\_Subprograms H.4(17)  
   No\_Allocators H.4(7)  
   No\_Anonymous\_Allocators H.4(8.1/3)  
   No\_Asynchronous\_Control J.13(3/2)  
   No\_Delay H.4(21)  
   No\_Dependence 13.12.1(6/2)  
   No\_Dispatch H.4(19)  
   No\_Dynamic\_Attachment D.7(10/3)  
   No\_Dynamic\_Priorities D.7(9/2)  
   No\_Exceptions H.4(12)  
   No\_Fixed\_Point H.4(15)  
   No\_Floating\_Point H.4(14)  
   No\_Implementation\_Attributes 13.12.1(2/2)  
   No\_Implementation\_Pragmas 13.12.1(3/2)  
   No\_Implicit\_Heap\_Allocations D.7(8)  
   No\_IO H.4(20/2)  
   No\_Local\_Allocators H.4(8/1)  
   No\_Local\_Protected\_Objects D.7(10.1/3)  
   No\_Local\_Timing\_Events D.7(10.2/3)  
   No\_Nested\_Finalization D.7(4/3)  
   No\_Obsolescent\_Features 13.12.1(4/2)  
   No\_Protected\_Type\_Allocators D.7(10.3/2)  
   No\_Protected\_Types H.4(5)  
   No\_Recursion H.4(22)  
   No\_Reentrancy H.4(23)  
   No\_Relative\_Delay D.7(10.4/2)  
   No\_Requeue\_Statements D.7(10.5/2)  
   No\_Select\_Statements D.7(10.6/2)  
   No\_Specific\_Termination\_Handlers D.7(10.7/2)  
   No\_Task\_Allocators D.7(7)  
   No\_Task\_Hierarchy D.7(3/3)  
   No\_Task\_Termination D.7(15.1/2)  
   No\_Terminate\_Alternatives D.7(6)  
   No\_Unchecked\_Access H.4(18)  
   No\_Unchecked\_Conversion J.13(4/2)  
   No\_Unchecked\_Deallocation J.13(5/2)  
   Simple\_Barriers D.7(10.8/3)  
 Restrictions pragma 13.12(3), L(32)  
 result interval  
   for a component of the result of evaluating a complex function G.2.6(3)  
   for the evaluation of a predefined arithmetic operation G.2.1(8)  
   for the evaluation of an elementary function G.2.4(2)  
 result subtype  
   of a function 6.5(3/2)  
 return object  
   extended\_return\_statement 6.5(5.8/3)  
   simple\_return\_statement 6.5(6/2)  
 return statement 6.5(1/2)  
 return\_subtype\_indication 6.5(2.2/2)  
   *used* 6.5(2.1/3), P  
 Reverse\_Elements  
   in Ada.Containers.Doubly\_Linked\_Lists A.18.3(27/2)  
   in Ada.Containers.Vectors A.18.2(54/2)  
 Reverse\_Find  
   in Ada.Containers.Doubly\_Linked\_Lists A.18.3(42/2)  
   in Ada.Containers.Vectors A.18.2(70/2)  
 Reverse\_Find\_Index  
   in Ada.Containers.Vectors A.18.2(69/2)  
 Reverse\_Iterate  
   in Ada.Containers.Doubly\_Linked\_Lists A.18.3(46/2)  
   in Ada.Containers.Ordered\_Maps A.18.6(51/2)  
   in Ada.Containers.Ordered\_Sets A.18.9(61/2)  
   in Ada.Containers.Vectors A.18.2(74/2)  
 Reverse\_Iterate\_Children  
   in Ada.Containers.Multiway\_Trees A.18.10(62/3)  
 Reverse\_Solidus  
   in Ada.Characters.Latin\_1 A.3.3(12)  
 Reviewable pragma H.3.1(3), L(33)  
 RI  
   in Ada.Characters.Latin\_1 A.3.3(17)  
 right parenthesis 2.1(15/2)  
 Right\_Angle\_Quotation  
   in Ada.Characters.Latin\_1 A.3.3(22)  
 Right\_Curly\_Bracket  
   in Ada.Characters.Latin\_1 A.3.3(14)  
 Right\_Parenthesis  
   in Ada.Characters.Latin\_1 A.3.3(8)  
 Right\_Square\_Bracket  
   in Ada.Characters.Latin\_1 A.3.3(12)  
 Ring\_Above  
   in Ada.Characters.Latin\_1 A.3.3(22)  
 root  
   of a tree A.18.10(3/3)  
   in Ada.Containers.Multiway\_Trees A.18.10(20/3)  
 root library unit 10.1.1(10)  
 root node  
   of a tree A.18.10(3/3)  
 root type  
   of a class 3.4.1(2/2)  
 root\_integer 3.5.4(14)  
   [*partial*] 3.4.1(8)  
 root\_real 3.5.6(3)  
   [*partial*] 3.4.1(8)

Root\_Storage\_Pool  
   *in* System.Storage\_Pools 13.11(6/2)  
 Root\_Stream\_Type  
   *in* Ada.Streams 13.13.1(3/2)  
 rooted at a type 3.4.1(2/2)  
 roots the subtree A.18.10(3/3)  
 rotate B.2(9)  
 Round attribute 3.5.10(12)  
 Round\_Robin  
   *child of* Ada.Dispatching D.2.5(4/2)  
 Round\_Robin\_Within\_Priorities task  
   dispatching policy D.2.5(2/2)  
 Rounding attribute A.5.3(36)  
 RPC  
   *child of* System E.5(3)  
 RPC-receiver E.5(21)  
 RPC\_Receiver  
   *in* System.RPC E.5(11)  
 RS  
   *in* Ada.Characters.Latin\_1 A.3.3(6)  
 run-time check  
   *See* language-defined check 11.5(2)  
 run-time error 1.1.2(30), 1.1.5(6),  
   11.5(2), 11.6(1)  
 run-time polymorphism 3.9.2(1/2)  
 run-time semantics 1.1.2(30)  
 run-time type  
   *See* tag 3.9(3)  
 running a program  
   *See* program execution 10.2(1)  
 running task D.2.1(6/2)

## S

safe range  
   of a floating point type 3.5.7(9)  
   of a floating point type 3.5.7(10)  
 Safe\_First attribute A.5.3(71), G.2.2(5)  
 Safe\_Last attribute A.5.3(72), G.2.2(6)  
 safety-critical systems H(1/2)  
 satisfies  
   a discriminant constraint 3.7.1(11)  
   a range constraint 3.5(4)  
   an index constraint 3.6.1(7)  
   for an access value 3.10(15/2)  
 Saturday  
   *in* Ada.Calendar.Formatting  
   9.6.1(17/2)  
 Save  
   *in* Ada.Numerics.Discrete\_Random  
   A.5.2(24)  
   *in* Ada.Numerics.Float\_Random  
   A.5.2(12)  
 Save\_Occurrence  
   *in* Ada.Exceptions 11.4.1(6/2)  
 scalar type 3.2(3), 3.5(1), N(37)  
 scalar\_constraint 3.2.2(6)  
   *used* 3.2.2(5), P

scale  
   of a decimal fixed point subtype  
   3.5.10(11), K(216)  
 Scale attribute 3.5.10(11)  
 Scaling attribute A.5.3(27)  
 SCHAR\_MAX  
   *in* Interfaces.C B.3(6)  
 SCHAR\_MIN  
   *in* Interfaces.C B.3(6)  
 SCI  
   *in* Ada.Characters.Latin\_1 A.3.3(19)  
 scope  
   informal definition 3.1(8)  
   of (a view of) an entity 8.2(11)  
   of a declaration 8.2(10)  
   of a use\_clause 8.4(6)  
   of a with\_clause 10.1.2(5)  
   of an attribute\_definition\_clause  
   8.2(10.1/2)  
 Search\_Type  
   *in* Ada.Directories A.16(31/2)  
 Second  
   *in* Ada.Calendar.Formatting  
   9.6.1(26/2)  
 Second\_Duration subtype of  
   Day\_Duration  
   *in* Ada.Calendar.Formatting  
   9.6.1(20/2)  
 Second\_Number subtype of Natural  
   *in* Ada.Calendar.Formatting  
   9.6.1(20/2)  
 Seconds  
   *in* Ada.Calendar 9.6(13)  
   *in* Ada.Real\_Time D.8(14/2)  
 Seconds\_Count  
   *in* Ada.Real\_Time D.8(15)  
 Seconds\_Of  
   *in* Ada.Calendar.Formatting  
   9.6.1(28/2)  
 Section\_Sign  
   *in* Ada.Characters.Latin\_1 A.3.3(21/3)  
 secure systems H(1/2)  
 select an entry call  
   from an entry queue 9.5.3(13),  
   9.5.3(16)  
   immediately 9.5.3(8)  
 select\_alternative 9.7.1(4)  
   *used* 9.7.1(2), P  
 select\_statement 9.7(2)  
   *used* 5.1(5/2), P  
 selected\_component 4.1.3(2)  
   *used* 4.1(2/3), P  
 selection  
   of an entry caller 9.5.2(24)  
 selective\_accept 9.7.1(2)  
   *used* 9.7(2), P  
 selector\_name 4.1.3(3)  
   *used* 3.7.1(3), 4.1.3(2), 4.3.1(5), 6.4(5),  
   12.3(4), 12.7(3.1/2), P

semantic dependence  
   of one compilation unit upon another  
   10.1.1(26/2)  
 semicolon 2.1(15/2)  
   *in* Ada.Characters.Latin\_1 A.3.3(10)  
 separate compilation 10.1(1)  
 separator 2.2(3/2)  
 separator\_line 2.1(12/2)  
 separator\_paragraph 2.1(12.1/2)  
 separator\_space 2.1(11/2)  
 sequence of characters  
   of a string\_literal 2.6(5)  
 sequence\_of\_statements 5.1(2/3)  
   *used* 5.3(2), 5.4(3), 5.5(2), 9.7.1(2),  
   9.7.1(5), 9.7.1(6), 9.7.2(3/2), 9.7.3(2),  
   9.7.4(3), 9.7.4(5), 11.2(2), 11.2(3), P  
 sequential  
   actions 9.10(11), C.6(17)  
 sequential access A.8(2)  
 sequential file A.8(1/2)  
 Sequential\_IO  
   *child of* Ada A.8.1(2)  
 service  
   an entry queue 9.5.3(13)  
 set  
   execution timer object D.14.1(12/2)  
   group budget object D.14.2(15/2)  
   termination handler C.7.3(9/2)  
   timing event object D.15(9/2)  
   *in* Ada.Containers.Hashed\_Sets  
   A.18.8(3/2)  
   *in* Ada.Containers.Ordered\_Sets  
   A.18.9(4/2)  
   *in* Ada.Environment\_Variables  
   A.17(6/2)  
 set container A.18.7(1/2)  
 Set\_Bounded\_String  
   *in* Ada.Strings.Bounded A.4.4(12.1/2)  
 Set\_Col  
   *in* Ada.Text\_IO A.10.1(35)  
 Set\_Deadline  
   *in* Ada.Dispatching.EDF D.2.6(9/2)  
 Set\_Dependents\_Fallback\_Handler  
   *in* Ada.Task\_Termination C.7.3(5/2)  
 Set\_Directory  
   *in* Ada.Directories A.16(6/2)  
 Set\_Error  
   *in* Ada.Text\_IO A.10.1(15)  
 Set\_Exit\_Status  
   *in* Ada.Command\_Line A.15(9)  
 Set\_False  
   *in* Ada.Synchronous\_Task\_Control  
   D.10(4)  
 Set\_Handler  
   *in*  
   Ada.Execution\_Time.Group\_Budgets  
   D.14.2(10/2)  
   *in* Ada.Execution\_Time.Timers  
   D.14.1(7/2)

- in* Ada.Real\_Time.Timing\_Events D.15(5/2)
- Set\_Im
  - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(8/2), G.3.2(28/2)
  - in* Ada.Numerics.Generic\_Complex\_Types G.1.1(7)
- Set\_Index
  - in* Ada.Direct\_IO A.8.4(14)
  - in* Ada.Streams.Stream\_IO A.12.1(22)
- Set\_Input
  - in* Ada.Text\_IO A.10.1(15)
- Set\_Length
  - in* Ada.Containers.Vectors A.18.2(22/2)
- Set\_Line
  - in* Ada.Text\_IO A.10.1(36)
- Set\_Line\_Length
  - in* Ada.Text\_IO A.10.1(23)
- Set\_Mode
  - in* Ada.Streams.Stream\_IO A.12.1(24)
- Set\_Output
  - in* Ada.Text\_IO A.10.1(15)
- Set\_Page\_Length
  - in* Ada.Text\_IO A.10.1(24)
- Set\_Priority
  - in* Ada.Dynamic\_Priorities D.5.1(4)
- Set\_Quantum
  - in* Ada.Dispatching.Round\_Robin D.2.5(4/2)
- Set\_Re
  - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(8/2), G.3.2(28/2)
  - in* Ada.Numerics.Generic\_Complex\_Types G.1.1(7)
- Set\_Specific\_Handler
  - in* Ada.Task\_Termination C.7.3(6/2)
- Set\_True
  - in* Ada.Synchronous\_Task\_Control D.10(4)
- Set\_Unbounded\_String
  - in* Ada.Strings.Unbounded A.4.5(11.1/2)
- Set\_Value
  - in* Ada.Task\_Attributes C.7.2(6)
- shared passive library unit E.2(4/1), E.2.1(4)
- shared variable
  - protection of 9.10(1/3)
- Shared\_Passive pragma E.2.1(3), L(34)
- shift B.2(9)
- short
  - in* Interfaces.C B.3(7)
- short-circuit control form 4.5.1(1)
- Short\_Float 3.5.7(16)
- Short\_Integer 3.5.4(25)
- SI
  - in* Ada.Characters.Latin\_1 A.3.3(5)
- signal
  - as defined between actions 9.10(2)
  - See* interrupt C.3(1)
- signal (an exception)
  - See* raise 11(1)
- signal handling
  - example 9.7.4(10)
- signed integer type 3.5.4(1)
- signed\_char
  - in* Interfaces.C B.3(8)
- signed\_integer\_type\_definition 3.5.4(3)
  - used* 3.5.4(2), P
- Signed\_Zeros attribute A.5.3(13)
- simple entry call 9.5.3(1)
- simple name
  - of a file A.16(47/2)
- Simple\_Barriers restriction D.7(10.8/3)
- simple\_expression 4.4(4)
  - used* 3.5(3), 3.5.4(3), 3.5.7(3), 4.4(3), 13.5.1(5), 13.5.1(6), P
- Simple\_Name
  - in* Ada.Directories A.16(16/2), A.16(38/2)
  - in*
    - Ada.Directories.Hierarchical\_File\_Names A.16.1(10/3)
- simple\_return\_statement 6.5(2/2)
  - used* 5.1(4/2), P
- simple\_statement 5.1(4/2)
  - used* 5.1(3), P
- Sin
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(4)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(5)
- single
  - class expected type 8.6(27/2)
- single entry 9.5.2(20)
- Single\_Precision\_Complex\_Types
  - in* Interfaces.Fortran B.5(8)
- single\_protected\_declaration 9.4(3/2)
  - used* 3.3.1(2/2), P
- single\_task\_declaration 9.1(3/2)
  - used* 3.3.1(2/2), P
- Sinh
  - in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(6)
  - in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(7)
- size
  - of an object 13.1(7/2)
  - in* Ada.Direct\_IO A.8.4(15)
  - in* Ada.Directories A.16(26/2), A.16(41/2)
  - in* Ada.Streams.Stream\_IO A.12.1(23)
- Size attribute 13.3(40), 13.3(45)
- Size clause 13.3(7/2), 13.3(41), 13.3(48)
- size\_t
  - in* Interfaces.C B.3(13)
- Skip\_Line
  - in* Ada.Text\_IO A.10.1(29)
- Skip\_Page
  - in* Ada.Text\_IO A.10.1(32)
- slice 4.1.2(2)
  - used* 4.1(2/3), P
- in* Ada.Strings.Bounded A.4.4(28)
- in* Ada.Strings.Unbounded A.4.5(22)
- small
  - of a fixed point type 3.5.9(8/2)
- Small attribute 3.5.10(2/1)
- Small clause 3.5.10(2/1), 13.3(7/2)
- SO
  - in* Ada.Characters.Latin\_1 A.3.3(5)
- Soft\_Hyphen
  - in* Ada.Characters.Latin\_1 A.3.3(21/3)
- SOH
  - in* Ada.Characters.Latin\_1 A.3.3(5)
- solidus 2.1(15/2)
  - in* Ada.Characters.Latin\_1 A.3.3(8)
- Solve
  - in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(46/2)
  - in* Ada.Numerics.Generic\_Real\_Arrays G.3.1(24/2)
- Sort
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(49/2)
  - in* Ada.Containers.Vectors A.18.2(77/2)
- SOS
  - in* Ada.Characters.Latin\_1 A.3.3(19)
- SPA
  - in* Ada.Characters.Latin\_1 A.3.3(18)
- Space
  - in* Ada.Characters.Latin\_1 A.3.3(8)
  - in* Ada.Strings A.4.1(4/2)
- special file A.16(45/2)
- special graphic character
  - a category of Character A.3.2(32)
- Special\_Set
  - in* Ada.Strings.Maps.Constants A.4.6(4)
- Specialized Needs Annexes 1.1.2(7)
- specifiable
  - of Address for entries J.7.1(6)
  - of Address for stand-alone objects and for program units 13.3(12)
  - of Alignment for first subtypes 13.3(26.4/2)
  - of Alignment for objects 13.3(25/2)
  - of Bit\_Order for record types and record extensions 13.5.3(4)
  - of Component\_Size for array types 13.3(70)
  - of External\_Tag for a tagged type 13.3(75/3), K(65)
  - of Input for a type 13.13.2(38/3)
  - of Machine\_Radix for decimal first subtypes F.1(1)

|                                                                                       |                                                                        |                                                                                                                                                                                                          |
|---------------------------------------------------------------------------------------|------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| of Output for a type 13.13.2(38/3)                                                    | corresponding to a formal object of mode in 12.4(10/2)                 | statically matching effect on subtype-specific aspects 13.1(14)                                                                                                                                          |
| of Read for a type 13.13.2(38/3)                                                      | stand-alone object 3.3.1(1)                                            | for constraints 4.9.1(1/2)                                                                                                                                                                               |
| of Size for first subtypes 13.3(48)                                                   | <i>[partial]</i> 12.4(10/2)                                            | for ranges 4.9.1(3)                                                                                                                                                                                      |
| of Size for stand-alone objects 13.3(41)                                              | stand-alone variable 3.3.1(23/3)                                       | for subtypes 4.9.1(2/2)                                                                                                                                                                                  |
| of Small for fixed point types 3.5.10(2/1)                                            | Standard A.1(4)                                                        | required 3.9.2(10/2), 3.10.2(27.1/2), 4.6(24.15/2), 4.6(24.5/2), 6.3.1(16/3), 6.3.1(17), 6.3.1(23), 6.5(5.2/3), 7.3(13), 8.5.1(4.2/2), 12.4(8.1/2), 12.5.1(14), 12.5.3(6), 12.5.3(7), 12.5.4(3), 12.7(7) |
| of Storage_Pool for a non-derived access-to-object type 13.11(15)                     | standard error file A.10(6)                                            | statically tagged 3.9.2(4/2)                                                                                                                                                                             |
| of Storage_Size for a non-derived access-to-object type 13.11(15)                     | standard input file A.10(5)                                            | Status_Error                                                                                                                                                                                             |
| of Storage_Size for a task first subtype J.9(3/2)                                     | standard mode 1.1.5(11)                                                | <i>in</i> Ada.Direct_IO A.8.4(18)                                                                                                                                                                        |
| of Write for a type 13.13.2(38/3)                                                     | standard output file A.10(5)                                           | <i>in</i> Ada.Directories A.16(43/2)                                                                                                                                                                     |
| specifiable (of an attribute and for an entity) 13.3(5/1)                             | standard storage pool 13.11(17)                                        | <i>in</i> Ada.IO_Exceptions A.13(4)                                                                                                                                                                      |
| specific handler C.7.3(9/2)                                                           | Standard_Error                                                         | <i>in</i> Ada.Sequential_IO A.8.1(15)                                                                                                                                                                    |
| specific type 3.4.1(3/2)                                                              | <i>in</i> Ada.Text_IO A.10.1(16), A.10.1(19)                           | <i>in</i> Ada.Streams.Stream_IO A.12.1(26)                                                                                                                                                               |
| Specific_Handler                                                                      | Standard_Input                                                         | <i>in</i> Ada.Text_IO A.10.1(85)                                                                                                                                                                         |
| <i>in</i> Ada.Task_Termination C.7.3(6/2)                                             | <i>in</i> Ada.Text_IO A.10.1(16), A.10.1(19)                           | storage deallocation unchecked 13.11.2(1)                                                                                                                                                                |
| specified                                                                             | Standard_Output                                                        | storage element 13.3(8)                                                                                                                                                                                  |
| of an aspect of representation of an entity 13.1(17)                                  | <i>in</i> Ada.Text_IO A.10.1(16), A.10.1(19)                           | storage management user-defined 13.11(1)                                                                                                                                                                 |
| of an operational aspect of an entity 13.1(18.1/1)                                    | Start_Search                                                           | storage node E(2)                                                                                                                                                                                        |
| specified (not!) 1.1.3(18)                                                            | <i>in</i> Ada.Directories A.16(32/2)                                   | storage place of a component 13.5(1)                                                                                                                                                                     |
| specified as independently addressable C.6(14.1/3)                                    | State                                                                  | storage place attributes of a component 13.5.2(1)                                                                                                                                                        |
| specified discriminant 3.7(18)                                                        | <i>in</i> Ada.Numerics.Discrete_Random A.5.2(23)                       | storage pool 3.10(7/1)                                                                                                                                                                                   |
| Splice                                                                                | <i>in</i> Ada.Numerics.Float_Random A.5.2(11)                          | storage pool element 13.11(11)                                                                                                                                                                           |
| <i>in</i> Ada.Containers.Doubly_Linked_Lists A.18.3(30/2), A.18.3(31/2), A.18.3(32/2) | statement 5.1(3)                                                       | storage pool type 13.11(11)                                                                                                                                                                              |
| Splice_Children                                                                       | <i>used</i> 5.1(2/3), P                                                | Storage_Array                                                                                                                                                                                            |
| <i>in</i> Ada.Containers.Multiway_Trees A.18.10(50/3), A.18.10(51/3)                  | statement_identifier 5.1(8)                                            | <i>in</i> System.Storage_Elements 13.7.1(5)                                                                                                                                                              |
| Splice_Subtree                                                                        | <i>used</i> 5.1(7), 5.5(2), 5.6(2), P                                  | Storage_Check 11.5(23)                                                                                                                                                                                   |
| <i>in</i> Ada.Containers.Multiway_Trees A.18.10(48/3), A.18.10(49/3)                  | static 4.9(1)                                                          | <i>[partial]</i> 11.1(6), 13.3(67), 13.11(17), D.7(17/1), D.7(18/1), D.7(19/1)                                                                                                                           |
| Split                                                                                 | constant 4.9(24)                                                       | Storage_Count <i>subtype of</i> Storage_Offset                                                                                                                                                           |
| <i>in</i> Ada.Calendar 9.6(14)                                                        | constraint 4.9(27)                                                     | <i>in</i> System.Storage_Elements 13.7.1(4)                                                                                                                                                              |
| <i>in</i> Ada.Calendar.Formatting 9.6.1(29/2), 9.6.1(32/2), 9.6.1(33/2), 9.6.1(34/2)  | delta constraint 4.9(29)                                               | Storage_Element                                                                                                                                                                                          |
| <i>in</i> Ada.Execution_Time D.14(8/2)                                                | digits constraint 4.9(29)                                              | <i>in</i> System.Storage_Elements 13.7.1(5)                                                                                                                                                              |
| <i>in</i> Ada.Real_Time D.8(16)                                                       | discrete_range 4.9(25)                                                 | Storage_Elements                                                                                                                                                                                         |
| Sqrt                                                                                  | discriminant constraint 4.9(31)                                        | <i>child of</i> System 13.7.1(2/2)                                                                                                                                                                       |
| <i>in</i> Ada.Numerics.Generic_Complex_Elementary_Functions G.1.2(3)                  | expression 4.9(2)                                                      | Storage_Error                                                                                                                                                                                            |
| <i>in</i> Ada.Numerics.Generic_Elementary_Functions A.5.1(4)                          | function 4.9(18)                                                       | raised by failure of run-time check 4.8(14), 8.5.4(8.1/1), 11.1(4), 11.1(6), 11.5(23), 13.3(67), 13.11(17), 13.11(18), A.7(14), D.7(17/1), D.7(18/1), D.7(19/1)                                          |
| SS2                                                                                   | index constraint 4.9(30)                                               | <i>in</i> Standard A.1(46)                                                                                                                                                                               |
| <i>in</i> Ada.Characters.Latin_1 A.3.3(17)                                            | range 4.9(25)                                                          | Storage_IO                                                                                                                                                                                               |
| SS3                                                                                   | range constraint 4.9(29)                                               | <i>child of</i> Ada A.9(3)                                                                                                                                                                               |
| <i>in</i> Ada.Characters.Latin_1 A.3.3(17)                                            | range constraint 4.9(26/2)                                             | Storage_Offset                                                                                                                                                                                           |
| SSA                                                                                   | scalar subtype 4.9(26/2)                                               | <i>in</i> System.Storage_Elements 13.7.1(3)                                                                                                                                                              |
| <i>in</i> Ada.Characters.Latin_1 A.3.3(17)                                            | string subtype 4.9(26/2)                                               | Storage_Pool attribute 13.11(13)                                                                                                                                                                         |
| ST                                                                                    | subtype 4.9(26/2)                                                      | Storage_Pool clause 13.3(7/2), 13.11(15)                                                                                                                                                                 |
| <i>in</i> Ada.Characters.Latin_1 A.3.3(19)                                            | subtype 12.4(9/2)                                                      | Storage_Pools                                                                                                                                                                                            |
| stand-alone constant 3.3.1(23/3)                                                      | static semantics 1.1.2(28)                                             | <i>child of</i> System 13.11(5)                                                                                                                                                                          |
|                                                                                       | statically                                                             |                                                                                                                                                                                                          |
|                                                                                       | constrained 4.9(32)                                                    |                                                                                                                                                                                                          |
|                                                                                       | denote 4.9(14)                                                         |                                                                                                                                                                                                          |
|                                                                                       | statically compatible for a constraint and a scalar subtype 4.9.1(4/3) |                                                                                                                                                                                                          |
|                                                                                       | for a constraint and an access or composite subtype 4.9.1(4/3)         |                                                                                                                                                                                                          |
|                                                                                       | for two subtypes 4.9.1(4/3)                                            |                                                                                                                                                                                                          |
|                                                                                       | statically deeper 3.10.2(4), 3.10.2(17)                                |                                                                                                                                                                                                          |
|                                                                                       | statically determined tag 3.9.2(1/2)                                   |                                                                                                                                                                                                          |
|                                                                                       | <i>[partial]</i> 3.9.2(15), 3.9.2(19)                                  |                                                                                                                                                                                                          |

- Storage\_Size
  - in* System.Storage\_Pools 13.11(9)
- Storage\_Size attribute 13.3(60), 13.11(14), J.9(2)
- Storage\_Size clause 13.3(7/2), 13.11(15)
  - See also* pragma Storage\_Size 13.3(61)
- Storage\_Size pragma 13.3(63), L(35)
- Storage\_Unit
  - in* System 13.7(13)
- stream 13.13(1)
  - in* Ada.Streams.Stream\_IO A.12.1(13)
  - in* Ada.Text\_IO.Text\_Streams A.12.2(4)
  - in* Ada.Wide\_Text\_IO.Text\_Streams A.12.3(4)
  - in* Ada.Wide\_Wide\_Text\_IO.Text\_Streams A.12.4(4/2)
- stream file A.8(1/2)
- stream type 13.13(1)
- Stream\_Access
  - in* Ada.Streams.Stream\_IO A.12.1(4)
  - in* Ada.Text\_IO.Text\_Streams A.12.2(3)
  - in* Ada.Wide\_Text\_IO.Text\_Streams A.12.3(3)
  - in* Ada.Wide\_Wide\_Text\_IO.Text\_Streams A.12.4(3/2)
- Stream\_Element
  - in* Ada.Streams 13.13.1(4/1)
- Stream\_Element\_Array
  - in* Ada.Streams 13.13.1(4/1)
- Stream\_Element\_Count *subtype of* Stream\_Element\_Offset
  - in* Ada.Streams 13.13.1(4/1)
- Stream\_Element\_Offset
  - in* Ada.Streams 13.13.1(4/1)
- Stream\_IO
  - child of* Ada.Streams A.12.1(3)
- Stream\_Size attribute 13.13.2(1.2/2)
- Stream\_Size clause 13.3(7/2)
- Streams
  - child of* Ada 13.13.1(2)
- strict mode G.2(1)
- strict weak ordering A.18(4.1/3)
- String
  - in* Standard A.1(37)
- string type 3.6.3(1)
- String\_Access
  - in* Ada.Strings.Unbounded A.4.5(7)
- string\_element 2.6(3)
  - used* 2.6(2), P
- string\_literal 2.6(2)
  - used* 4.4(7/3), 6.1(9), P
- Strings
  - child of* Ada A.4.1(3)
  - child of* Interfaces.C B.3.1(3)
- Strlen
  - in* Interfaces.C.Strings B.3.1(17)
- structure
  - See* record type 3.8(1)
- STS
  - in* Ada.Characters.Latin\_1 A.3.3(18)
- STX
  - in* Ada.Characters.Latin\_1 A.3.3(5)
- SUB
  - in* Ada.Characters.Latin\_1 A.3.3(6)
- Sub\_Second
  - in* Ada.Calendar.Formatting 9.6.1(27/2)
- subaggregate
  - of an array\_aggregate 4.3.3(6)
- subcomponent 3.2(6/2)
- subprogram 6(1), N(37.1/2)
  - abstract 3.9.3(3/2)
  - subprogram call 6.4(1)
  - subprogram instance 12.3(13)
  - subprogram\_body 6.3(2/2)
    - used* 3.11(6), 9.4(8/1), 10.1.1(7), P
  - subprogram\_body\_stub 10.1.3(3/2)
    - used* 10.1.3(2), P
  - subprogram\_declaration 6.1(2/2)
    - used* 3.1(3/2), 9.4(5/1), 9.4(8/1), 10.1.1(5), P
  - subprogram\_default 12.6(3/2)
    - used* 12.6(2.1/2), 12.6(2.2/2), P
  - subprogram\_renaming\_declaration 8.5.4(2/2)
    - used* 8.5(2), 10.1.1(6), P
  - subprogram\_specification 6.1(4/2)
    - used* 3.9.3(1.1/2), 6.1(2/2), 6.3(2/2), 8.5.4(2/2), 10.1.3(3/2), 12.1(3), 12.6(2.1/2), 12.6(2.2/2), P
- subsystem 10.1(3), N(22)
- subtree
  - node which roots A.18.10(3/3)
  - of a tree A.18.10(3/3)
- Subtree\_Node\_Count
  - in* Ada.Containers.Multiway\_Trees A.18.10(16/3)
- subtype 3.2(8/2), N(38/2)
  - constraint of 3.2(8/2)
  - type of 3.2(8/2)
  - values belonging to 3.2(8/2)
- subtype (of an object)
  - See* actual subtype of an object 3.3(23)
  - See* actual subtype of an object 3.3.1(9/2)
- subtype conformance 6.3.1(17)
  - [*partial*] 3.10.2(34/2), 9.5.4(17)
  - required 3.9.2(10/2), 3.10.2(32/2), 4.6(24.20/2), 8.5.1(4.3/2), 8.5.4(5/1), 9.1(9.7/2), 9.1(9.8/2), 9.4(11.6/2), 9.4(11.7/2), 9.5.4(5/3), 12.4(8.2/2)
- subtype conversion
  - See* type conversion 4.6(1)
  - See also* implicit subtype conversion 4.6(1)
- subtype-specific
  - of a representation item 13.1(8/3)
  - of an aspect 13.1(8/3)
- subtype\_declaration 3.2.2(2)
  - used* 3.1(3/2), P
- subtype\_indication 3.2.2(3/2)
  - used* 3.2.2(2), 3.3.1(2/2), 3.4(2/2), 3.6(6), 3.6(7/2), 3.6.1(3), 3.10(3), 4.8(2), 6.5(2.2/2), 7.3(3/2), P
- subtype\_mark 3.2.2(4)
  - used* 3.2.2(3/2), 3.6(4), 3.7(5/2), 3.9.4(3/2), 3.10(6/2), 4.3.2(3), 4.4(3), 4.6(2), 4.7(2), 6.1(13/2), 6.1(15/2), 8.4(4/3), 8.5.1(2/2), 12.3(5), 12.4(2/3), 12.5.1(3/2), P
- subtypes
  - of a profile 6.1(25)
- subunit 10.1.3(7), 10.1.3(8/2)
  - of a program unit 10.1.3(8/2)
    - used* 10.1.1(3), P
- Succ attribute 3.5(22)
- Success
  - in* Ada.Command\_Line A.15(8)
- successor element
  - of a hashed set A.18.8(68/2)
  - of an ordered set A.18.9(81/2)
  - of a set A.18.7(6/2)
- successor node
  - of a hashed map A.18.5(46/2)
  - of a map A.18.4(6/2)
  - of an ordered map A.18.6(58/2)
- Sunday
  - in* Ada.Calendar.Formatting 9.6.1(17/2)
- super
  - See* view conversion 4.6(5/2)
- Superscript\_One
  - in* Ada.Characters.Latin\_1 A.3.3(22)
- Superscript\_Three
  - in* Ada.Characters.Latin\_1 A.3.3(22)
- Superscript\_Two
  - in* Ada.Characters.Latin\_1 A.3.3(22)
- support external streaming 13.13.2(52/3)
- Suppress pragma 11.5(4/2), J.10(3/2), L(36)
- suppressed check 11.5(8/2)
- Suspend\_Until\_True
  - in* Ada.Synchronous\_Task\_Control D.10(4)
- Suspend\_Until\_True\_And\_Set\_Deadline
  - in* Ada.Synchronous\_Task\_Control.EDF D.10(5.2/3)
- Suspension\_Object
  - in* Ada.Synchronous\_Task\_Control D.10(4)
- Swap
  - in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(28/2)
  - in* Ada.Containers.Multiway\_Trees A.18.10(31/3)
  - in* Ada.Containers.Vectors A.18.2(55/2), A.18.2(56/2)

Swap\_Links  
*in* Ada.Containers.Doubly\_Linked\_Lists A.18.3(29/2)

Symmetric\_Difference  
*in* Ada.Containers.Hashed\_Sets A.18.8(35/2), A.18.8(36/2)  
*in* Ada.Containers.Ordered\_Sets A.18.9(36/2), A.18.9(37/2)

SYN  
*in* Ada.Characters.Latin\_1 A.3.3(6)

synchronization 9(1)

synchronized N(38.1/2)

synchronized interface 3.9.4(5/2)

synchronized tagged type 3.9.4(6/2)

Synchronous\_Task\_Control  
*child of* Ada D.10(3/2)

syntactic category 1.1.4(15)

syntax  
 complete listing P(1)  
 cross reference P(1)  
 notation 1.1.4(3)  
 under Syntax heading 1.1.2(25)

System 13.7(3/2)

System.Address\_To\_Access\_Conversions 13.7.2(2)

System.Machine\_Code 13.8(7)

System.RPC E.5(3)

System.Storage\_Elements 13.7.1(2/2)

System.Storage\_Pools 13.11(5)

System\_Name  
*in* System 13.7(4)

systems programming C(1)

## T

Tag  
*in* Ada.Tags 3.9(6/2)

Tag attribute 3.9(16), 3.9(18)

tag indeterminate 3.9.2(6/2)

tag of an object 3.9(3)  
 class-wide object 3.9(22)  
 object created by an allocator 3.9(21)  
 preserved by type conversion and parameter passing 3.9(25)  
 returned by a function 3.9(23), 3.9(24/2)  
 stand-alone object, component, or aggregate 3.9(20)

Tag\_Array  
*in* Ada.Tags 3.9(7.3/2)

Tag\_Check 11.5(18)  
 [partial] 3.9.2(16), 4.6(42), 4.6(52), 5.2(10), 6.5(8.1/3)

Tag\_Error  
*in* Ada.Tags 3.9(8)

tagged incomplete view 3.10.1(2.1/2)

tagged type 3.9(2/2), N(39)  
 protected 3.9.4(6/2)  
 synchronized 3.9.4(6/2)  
 task 3.9.4(6/2)

Tags  
*child of* Ada 3.9(6/2)

Tail  
*in* Ada.Strings.Bounded A.4.4(72), A.4.4(73)  
*in* Ada.Strings.Fixed A.4.3(37), A.4.3(38)  
*in* Ada.Strings.Unbounded A.4.5(67), A.4.5(68)

tail (of a queue) D.2.1(5/2)

tamper with cursors  
 of a list A.18.3(62/2)  
 of a map A.18.4(8/2)  
 of a set A.18.7(8/2)  
 of a tree A.18.10(72/3)  
 of a vector A.18.2(91/2)

tamper with elements  
 of a holder A.18.18(26/3)  
 of a list A.18.3(67/2)  
 of a map A.18.4(13/2)  
 of a set A.18.7(13/2)  
 of a tree A.18.10(78/3)  
 of a vector A.18.2(95/2)

Tan  
*in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(4)  
*in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(5)

Tanh  
*in* Ada.Numerics.Generic\_Complex\_Elementary\_Functions G.1.2(6)  
*in* Ada.Numerics.Generic\_Elementary\_Functions A.5.1(7)

target  
 of an assignment operation 5.2(3)  
 of an assignment\_statement 5.2(3)

target object  
 of a call on an entry or a protected subprogram 9.5(2)  
 of a requeue\_statement 9.5(7)

target statement  
 of a goto\_statement 5.8(3)

target subtype  
 of a type\_conversion 4.6(3)

task 9(1)  
 activation 9.2(1)  
 completion 9.3(1)  
 dependence 9.3(1)  
 execution 9.2(1)  
 termination 9.3(1)

task declaration 9.1(1)

task dispatching D.2.1(4/2)

task dispatching point D.2.1(4/2)  
 [partial] D.2.3(8/2), D.2.4(9/3)

task dispatching policy D.2.2(6/2)  
 [partial] D.2.1(5/2)  
 EDF\_Across\_Priorities D.2.6(7/2)  
 FIFO\_Within\_Priorities D.2.3(2/2)  
 Non\_Preemptive\_FIFO\_Within\_Priorities D.2.4(2/2)

Round\_Robin\_Within\_Priorities D.2.5(2/2)

task interface 3.9.4(5/2)

task priority D.1(15)

task state  
 abnormal 9.8(4)  
 blocked 9(10)  
 callable 9.9(2)  
 held D.11(4/2)  
 inactive 9(10)  
 ready 9(10)  
 terminated 9(10)

task tagged type 3.9.4(6/2)

task type N(40/2)

task unit 9(9)

Task\_Array  
*in* Ada.Execution\_Time.Group\_Budgets D.14.2(6/2)

Task\_Attributes  
*child of* Ada C.7.2(2)

task\_body 9.1(6)  
*used* 3.11(6), P

task\_body\_stub 10.1.3(5)  
*used* 10.1.3(2), P

task\_definition 9.1(4)  
*used* 9.1(2/2), 9.1(3/2), P

Task\_Dispatching\_Policy pragma D.2.2(2), L(37)

Task\_Id  
*in* Ada.Task\_Identification C.7.1(2/2)

Task\_Identification  
*child of* Ada C.7.1(2/2)

task\_item 9.1(5/1)  
*used* 9.1(4), P

Task\_Termination  
*child of* Ada C.7.3(2/2)

task\_type\_declaration 9.1(2/2)  
*used* 3.2.1(3), P

Tasking\_Error  
 raised by failure of run-time check 9.2(5), 9.5.3(21), 11.1(4), 13.11.2(13), 13.11.2(14), C.7.2(13), D.5.1(8), D.11(8)  
*in* Standard A.1(46)

template 12(1)  
 for a formal package 12.7(4)  
*See* generic unit 12(1)

term 4.4(5)  
*used* 4.4(4), P

terminal interrupt  
 example 9.7.4(10)

terminate\_alternative 9.7.1(7)  
*used* 9.7.1(4), P

terminated  
 a task state 9(10)

Terminated attribute 9.9(3)

termination  
 of a partition E.1(7)

termination handler C.7.3(8/2)

- fall-back C.7.3(9/2)
- specific C.7.3(9/2)
- Termination\_Handler
  - in* Ada.Task\_Termination C.7.3(4/2)
- Terminator\_Error
  - in* Interfaces.C B.3(40)
- tested type
  - of a membership test 4.5.2(3/2)
- text of a program 2.2(1)
- Text\_IO
  - child of* Ada A.10.1(2)
- Text\_Streams
  - child of* Ada.Text\_IO A.12.2(3)
  - child of* Ada.Wide\_Text\_IO A.12.3(3)
  - child of* Ada.Wide\_Wide\_Text\_IO A.12.4(3/2)
- throw (an exception)
  - See* raise 11(1)
- Thursday
  - in* Ada.Calendar.Formatting 9.6.1(17/2)
- tick 2.1(15/2)
  - in* Ada.Real\_Time D.8(6)
  - in* System 13.7(10)
- Tilde
  - in* Ada.Characters.Latin\_1 A.3.3(14)
- Time
  - in* Ada.Calendar 9.6(10)
  - in* Ada.Real\_Time D.8(4)
- time base 9.6(6)
- time limit
  - example 9.7.4(12)
- time type 9.6(6)
- Time-dependent Reset procedure
  - of the random number generator A.5.2(34)
- time-out
  - example 9.7.4(12)
  - See* asynchronous\_select 9.7.4(12)
  - See* selective\_accept 9.7.1(1)
  - See* timed\_entry\_call 9.7.2(1/2)
- Time\_Error
  - in* Ada.Calendar 9.6(18)
- Time\_First
  - in* Ada.Real\_Time D.8(4)
- Time\_Last
  - in* Ada.Real\_Time D.8(4)
- Time\_Of
  - in* Ada.Calendar 9.6(15)
  - in* Ada.Calendar.Formatting 9.6.1(30/2), 9.6.1(31/2)
  - in* Ada.Execution\_Time D.14(9/2)
  - in* Ada.Real\_Time D.8(16)
- Time\_Of\_Event
  - in* Ada.Real\_Time.Timing\_Events D.15(6/2)
- Time\_Offset
  - in* Ada.Calendar.Time\_Zones 9.6.1(4/2)
- Time\_Remaining
  - in* Ada.Execution\_Time.Timers D.14.1(8/2)
- Time\_Span
  - in* Ada.Real\_Time D.8(5)
- Time\_Span\_First
  - in* Ada.Real\_Time D.8(5)
- Time\_Span\_Last
  - in* Ada.Real\_Time D.8(5)
- Time\_Span\_Unit
  - in* Ada.Real\_Time D.8(5)
- Time\_Span\_Zero
  - in* Ada.Real\_Time D.8(5)
- Time\_Unit
  - in* Ada.Real\_Time D.8(4)
- Time\_Zones
  - child of* Ada.Calendar 9.6.1(2/2)
- timed\_entry\_call 9.7.2(2)
  - used* 9.7(2), P
- Timer
  - in* Ada.Execution\_Time.Timers D.14.1(4/2)
- timer interrupt
  - example 9.7.4(12)
- Timer\_Handler
  - in* Ada.Execution\_Time.Timers D.14.1(5/2)
- Timer\_Resource\_Error
  - in* Ada.Execution\_Time.Timers D.14.1(9/2)
- Timers
  - child of* Ada.Execution\_Time D.14.1(3/2)
- times operator 4.4(1), 4.5.5(1)
- timing
  - See* delay\_statement 9.6(1)
- Timing\_Event
  - in* Ada.Real\_Time.Timing\_Events D.15(4/2)
- Timing\_Event\_Handler
  - in* Ada.Real\_Time.Timing\_Events D.15(4/2)
- Timing\_Events
  - child of* Ada.Real\_Time D.15(3/2)
- To\_Ada
  - in* Interfaces.C B.3(22), B.3(26), B.3(28), B.3(32), B.3(37), B.3(39), B.3(39.10/2), B.3(39.13/2), B.3(39.17/2), B.3(39.19/2), B.3(39.4/2), B.3(39.8/2)
  - in* Interfaces.COBOL B.4(17), B.4(19)
  - in* Interfaces.Fortran B.5(13), B.5(14), B.5(16)
- To\_Address
  - in* System.Address\_To\_Access\_Conversions 13.7.2(3)
  - in* System.Storage\_Elements 13.7.1(10)
- To\_Basic
  - in* Ada.Characters.Handling A.3.2(6), A.3.2(7)
- To\_Binary
  - in* Interfaces.COBOL B.4(45), B.4(48)
- To\_Bounded\_String
  - in* Ada.Strings.Bounded A.4.4(11)
- To\_C
  - in* Interfaces.C B.3(21), B.3(25), B.3(27), B.3(32), B.3(36), B.3(38), B.3(39.13/2), B.3(39.16/2), B.3(39.18/2), B.3(39.4/2), B.3(39.7/2), B.3(39.9/2)
- To\_Character
  - in* Ada.Characters.Conversions A.3.4(5/2)
- To\_Chars\_Ptr
  - in* Interfaces.C.Strings B.3.1(8)
- To\_COBOL
  - in* Interfaces.COBOL B.4(17), B.4(18)
- To\_Cursor
  - in* Ada.Containers.Vectors A.18.2(25/2)
- To\_Decimal
  - in* Interfaces.COBOL B.4(35), B.4(40), B.4(44), B.4(47)
- To\_Display
  - in* Interfaces.COBOL B.4(36)
- To\_Domain
  - in* Ada.Strings.Maps A.4.2(24)
  - in* Ada.Strings.Wide\_Maps A.4.7(24)
  - in* Ada.Strings.Wide\_Wide\_Maps A.4.8(24/2)
- To\_Duration
  - in* Ada.Real\_Time D.8(13)
- To\_Fortran
  - in* Interfaces.Fortran B.5(13), B.5(14), B.5(15)
- To\_Holder
  - in* Ada.Containers.Indefinite\_Holders A.18.18(9/3)
- To\_Index
  - in* Ada.Containers.Vectors A.18.2(26/2)
- To\_Integer
  - in* System.Storage\_Elements 13.7.1(10)
- To\_ISO\_646
  - in* Ada.Characters.Handling A.3.2(11), A.3.2(12)
- To\_Long\_Binary
  - in* Interfaces.COBOL B.4(48)
- To\_Lower
  - in* Ada.Characters.Handling A.3.2(6), A.3.2(7)
- To\_Mapping
  - in* Ada.Strings.Maps A.4.2(23)
  - in* Ada.Strings.Wide\_Maps A.4.7(23)
  - in* Ada.Strings.Wide\_Wide\_Maps A.4.8(23/2)
- To\_Packed
  - in* Interfaces.COBOL B.4(41)
- To\_Picture
  - in* Ada.Text\_IO.Editing F.3.3(6)



- To\_Pointer**  
*in* System.Address\_To\_Access\_Conversions 13.7.2(3)
- To\_Range**  
*in* Ada.Strings.Maps A.4.2(24)  
*in* Ada.Strings.Wide\_Maps A.4.7(25)  
*in* Ada.Strings.Wide\_Wide\_Maps A.4.8(25/2)
- To\_Ranges**  
*in* Ada.Strings.Maps A.4.2(10)  
*in* Ada.Strings.Wide\_Maps A.4.7(10)  
*in* Ada.Strings.Wide\_Wide\_Maps A.4.8(10/2)
- To\_Sequence**  
*in* Ada.Strings.Maps A.4.2(19)  
*in* Ada.Strings.Wide\_Maps A.4.7(19)  
*in* Ada.Strings.Wide\_Wide\_Maps A.4.8(19/2)
- To\_Set**  
*in* Ada.Containers.Hashed\_Sets A.18.8(9/2)  
*in* Ada.Containers.Ordered\_Sets A.18.9(10/2)  
*in* Ada.Strings.Maps A.4.2(8), A.4.2(9), A.4.2(17), A.4.2(18)  
*in* Ada.Strings.Wide\_Maps A.4.7(8), A.4.7(9), A.4.7(17), A.4.7(18)  
*in* Ada.Strings.Wide\_Wide\_Maps A.4.8(8/2), A.4.8(9/2), A.4.8(17/2), A.4.8(18/2)
- To\_String**  
*in* Ada.Characters.Conversions A.3.4(5/2)  
*in* Ada.Strings.Bounded A.4.4(12)  
*in* Ada.Strings.Unbounded A.4.5(11)
- To\_Time\_Span**  
*in* Ada.Real\_Time D.8(13)
- To\_Unbounded\_String**  
*in* Ada.Strings.Unbounded A.4.5(9), A.4.5(10)
- To\_Upper**  
*in* Ada.Characters.Handling A.3.2(6), A.3.2(7)
- To\_Vector**  
*in* Ada.Containers.Vectors A.18.2(13/2), A.18.2(14/2)
- To\_Wide\_Character**  
*in* Ada.Characters.Conversions A.3.4(4/2), A.3.4(5/2)
- To\_Wide\_String**  
*in* Ada.Characters.Conversions A.3.4(4/2), A.3.4(5/2)
- To\_Wide\_Wide\_Character**  
*in* Ada.Characters.Conversions A.3.4(4/2)
- To\_Wide\_Wide\_String**  
*in* Ada.Characters.Conversions A.3.4(4/2)
- token**  
*See* lexical element 2.2(1)
- Trailing\_Nonseparate**  
*in* Interfaces.COBOL B.4(23)
- Trailing\_Separate**  
*in* Interfaces.COBOL B.4(23)
- transfer of control** 5.1(14/2)
- Translate**  
*in* Ada.Strings.Bounded A.4.4(53), A.4.4(54), A.4.4(55), A.4.4(56)  
*in* Ada.Strings.Fixed A.4.3(18), A.4.3(19), A.4.3(20), A.4.3(21)  
*in* Ada.Strings.Unbounded A.4.5(48), A.4.5(49), A.4.5(50), A.4.5(51)
- Translation\_Error**  
*in* Ada.Strings A.4.1(5)
- Transpose**  
*in* Ada.Numerics.Generic\_Complex\_Arrays G.3.2(34/2)  
*in* Ada.Numerics.Generic\_Real\_Arrays G.3.1(17/2)
- Tree**  
*in* Ada.Containers.Multiway\_Trees A.18.10(8/3)
- triggering\_alternative** 9.7.4(3)  
*used* 9.7.4(2), P
- triggering\_statement** 9.7.4(4/2)  
*used* 9.7.4(3), P
- Trim**  
*in* Ada.Strings.Bounded A.4.4(67), A.4.4(68), A.4.4(69)  
*in* Ada.Strings.Fixed A.4.3(31), A.4.3(32), A.4.3(33), A.4.3(34)  
*in* Ada.Strings.Unbounded A.4.5(61), A.4.5(62), A.4.5(63), A.4.5(64)
- Trim\_End**  
*in* Ada.Strings A.4.1(6)
- True** 3.5.3(1)
- Truncation**  
*in* Ada.Strings A.4.1(6)
- Truncation attribute** A.5.3(42)
- Tuesday**  
*in* Ada.Calendar.Formatting 9.6.1(17/2)
- two's complement**  
 modular types 3.5.4(29)
- type** 3.2(1), N(41/2)  
 abstract 3.9.3(1.2/2)  
 needs finalization 7.6(9.1/2)  
 of a subtype 3.2(8/2)  
 synchronized tagged 3.9.4(6/2)  
*See also* tag 3.9(3)  
*See also* language-defined types
- type conformance** 6.3.1(15/2)  
 [partial] 3.4(17/2), 8.3(8), 8.3(26/2), 10.1.4(4/1)  
 required 3.11.1(5), 4.1.4(14/2), 8.6(26), 9.1(9.2/3), 9.1(9.5/3), 9.4(11.1/3), 9.4(11.4/3), 9.5.4(3/3), 12.4(5/2)
- type conversion** 4.6(1)  
 access 4.6(24.11/2), 4.6(24.18/2), 4.6(24.19/2), 4.6(47)  
 arbitrary order 1.1.4(18)  
 array 4.6(24.2/2), 4.6(36)  
 composite (non-array) 4.6(21/2), 4.6(40)  
 enumeration 4.6(21.1/2), 4.6(34)  
 numeric 4.6(24.1/2), 4.6(29)  
 unchecked 13.9(1)  
*See also* qualified expression 4.7(1)
- type conversion, implicit**  
*See* implicit subtype conversion 4.6(1)
- type extension** 3.9(2/2), 3.9.1(1/2)
- type of a discrete\_range** 3.6.1(4)
- type of a range** 3.5(4)
- type parameter**  
*See* discriminant 3.7(1/2)
- type profile**  
*See* profile, type conformant 6.3.1(15/2)
- type resolution rules** 8.6(20/2)  
 if any type in a specified class of types is expected 8.6(21)  
 if expected type is specific 8.6(22)  
 if expected type is universal or class-wide 8.6(21)
- type tag**  
*See* tag 3.9(3)
- type-related**  
 aspect 13.1(8.1/1)  
 aspect 13.1(8/3)  
 operational item 13.1(8.1/1)  
 representation item 13.1(8/3)
- type\_conversion** 4.6(2)  
*used* 4.1(2/3), P  
*See also* unchecked type conversion 13.9(1)
- type\_declaration** 3.2.1(2)  
*used* 3.1(3/2), P
- type\_definition** 3.2.1(4/2)  
*used* 3.2.1(3), P
- Type\_Set**  
*in* Ada.Text\_IO A.10.1(7)
- types**  
 of a profile 6.1(29)
- U**
- UC\_A\_Acute**  
*in* Ada.Characters.Latin\_1 A.3.3(23)
- UC\_A\_Circumflex**  
*in* Ada.Characters.Latin\_1 A.3.3(23)
- UC\_A\_Diaeresis**  
*in* Ada.Characters.Latin\_1 A.3.3(23)
- UC\_A\_Grave**  
*in* Ada.Characters.Latin\_1 A.3.3(23)
- UC\_A\_Ring**  
*in* Ada.Characters.Latin\_1 A.3.3(23)
- UC\_A\_Tilde**  
*in* Ada.Characters.Latin\_1 A.3.3(23)
- UC\_AE\_Diphthong**  
*in* Ada.Characters.Latin\_1 A.3.3(23)

- UC\_C\_Cedilla  
  *in* Ada.Characters.Latin\_1 A.3.3(23)
- UC\_E\_Acute  
  *in* Ada.Characters.Latin\_1 A.3.3(23)
- UC\_E\_Circumflex  
  *in* Ada.Characters.Latin\_1 A.3.3(23)
- UC\_E\_Diaeresis  
  *in* Ada.Characters.Latin\_1 A.3.3(23)
- UC\_E\_Grave  
  *in* Ada.Characters.Latin\_1 A.3.3(23)
- UC\_I\_Acute  
  *in* Ada.Characters.Latin\_1 A.3.3(23)
- UC\_I\_Circumflex  
  *in* Ada.Characters.Latin\_1 A.3.3(23)
- UC\_I\_Diaeresis  
  *in* Ada.Characters.Latin\_1 A.3.3(23)
- UC\_I\_Grave  
  *in* Ada.Characters.Latin\_1 A.3.3(23)
- UC\_Icelandic\_Eth  
  *in* Ada.Characters.Latin\_1 A.3.3(24)
- UC\_Icelandic\_Thorn  
  *in* Ada.Characters.Latin\_1 A.3.3(24)
- UC\_N\_Tilde  
  *in* Ada.Characters.Latin\_1 A.3.3(24)
- UC\_O\_Acute  
  *in* Ada.Characters.Latin\_1 A.3.3(24)
- UC\_O\_Circumflex  
  *in* Ada.Characters.Latin\_1 A.3.3(24)
- UC\_O\_Diaeresis  
  *in* Ada.Characters.Latin\_1 A.3.3(24)
- UC\_O\_Grave  
  *in* Ada.Characters.Latin\_1 A.3.3(24)
- UC\_O\_Oblique\_Stroke  
  *in* Ada.Characters.Latin\_1 A.3.3(24)
- UC\_O\_Tilde  
  *in* Ada.Characters.Latin\_1 A.3.3(24)
- UC\_U\_Acute  
  *in* Ada.Characters.Latin\_1 A.3.3(24)
- UC\_U\_Circumflex  
  *in* Ada.Characters.Latin\_1 A.3.3(24)
- UC\_U\_Diaeresis  
  *in* Ada.Characters.Latin\_1 A.3.3(24)
- UC\_U\_Grave  
  *in* Ada.Characters.Latin\_1 A.3.3(24)
- UC\_Y\_Acute  
  *in* Ada.Characters.Latin\_1 A.3.3(24)
- UCHAR\_MAX  
  *in* Interfaces.C B.3(6)
- ultimate ancestor  
  of a type 3.4.1(10/2)
- unary adding operator 4.5.4(1)
- unary operator 4.5(9)
- unary\_adding\_operator 4.5(5)  
  *used* 4.4(4), P
- Unbiased\_Rounding attribute A.5.3(39)
- Unbounded  
  *child of* Ada.Strings A.4.5(3)  
  *in* Ada.Text\_IO A.10.1(5)
- Unbounded\_IO  
  *child of* Ada.Text\_IO A.10.12(3/2)  
  *child of* Ada.Wide\_Text\_IO A.11(5/3)  
  *child of* Ada.Wide\_Wide\_Text\_IO A.11(5/3)
- Unbounded\_Slice  
  *in* Ada.Strings.Unbounded A.4.5(22.1/2), A.4.5(22.2/2)
- Unbounded\_String  
  *in* Ada.Strings.Unbounded A.4.5(4/2)
- unchecked storage deallocation  
  13.11.2(1)
- unchecked type conversion 13.9(1)
- unchecked union object B.3.3(6/2)
- unchecked union subtype B.3.3(6/2)
- unchecked union type B.3.3(6/2)
- Unchecked\_Access attribute 13.10(3),  
  H.4(18)  
  *See also* Access attribute 3.10.2(24/1)
- Unchecked\_Conversion  
  *child of* Ada 13.9(3)
- Unchecked\_Deallocation  
  *child of* Ada 13.11.2(3)
- Unchecked\_Union pragma B.3.3(3/2),  
  L(37.1/2)
- unconstrained 3.2(9)
- object 3.3.1(9/2)
- object 6.4.1(16)
- subtype 3.2(9), 3.4(6), 3.5(7),  
    3.5.1(10), 3.5.4(9), 3.5.4(10),  
    3.5.7(11), 3.5.9(13), 3.5.9(16),  
    3.6(15), 3.6(16), 3.7(26), 3.9(15)
- subtype 3.10(14/1)
- subtype K(35)
- unconstrained\_array\_definition 3.6(3)  
  *used* 3.6(2), P
- undefined result 11.6(5)
- underline 2.1(15/2)  
  *used* 2.4.1(3), 2.4.2(4), P
- Uniformly\_Distributed\_subtype of Float  
  *in* Ada.Numerics.Float\_Random A.5.2(8)
- uninitialized allocator 4.8(4)
- uninitialized variables 13.9.1(2)  
  [*partial*] 3.3.1(21)
- union  
  C B.3.3(1/2)  
  *in* Ada.Containers.Hashed\_Sets A.18.8(26/2), A.18.8(27/2)  
  *in* Ada.Containers.Ordered\_Sets A.18.9(27/2), A.18.9(28/2)
- unit consistency E.3(6)
- unit matrix  
  complex matrix G.3.2(148/2)
- real matrix G.3.1(80/2)
- unit vector  
  complex vector G.3.2(90/2)  
  real vector G.3.1(48/2)
- Unit\_Matrix  
  *in* Ada.Numerics.Generic\_Complex\_ Arrays G.3.2(51/2)
- in* Ada.Numerics.Generic\_Real\_Arrays G.3.1(29/2)
- Unit\_Vector  
  *in* Ada.Numerics.Generic\_Complex\_ Arrays G.3.2(24/2)
- in* Ada.Numerics.Generic\_Real\_Arrays G.3.1(14/2)
- universal type 3.4.1(6/2)
- universal access  
  [*partial*] 3.4.1(6/2), 4.2(8/2)
- universal\_fixed  
  [*partial*] 3.4.1(6/2), 3.5.6(4)
- universal\_integer  
  [*partial*] 3.4.1(6/2), 3.5.4(14),  
    3.5.4(30), 4.2(8/2)
- universal\_real  
  [*partial*] 3.4.1(6/2), 3.5.6(4), 4.2(8/2)
- unknown discriminants 3.7(26)
- unknown\_discriminant\_part 3.7(3)  
  *used* 3.7(2/2), P
- Unknown\_Zone\_Error  
  *in* Ada.Calendar.Time\_Zones 9.6.1(5/2)
- unmarshalling E.4(9)
- unpolluted 13.13.1(2)
- unsigned  
  *in* Interfaces.C B.3(9)  
  *in* Interfaces.COBOL B.4(23)
- unsigned type  
  *See* modular type 3.5.4(1)
- unsigned\_char  
  *in* Interfaces.C B.3(10)
- unsigned\_long  
  *in* Interfaces.C B.3(9)
- unsigned\_short  
  *in* Interfaces.C B.3(9)
- unspecified 1.1.3(18)

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [ <i>partial</i> ] 2.1(5/2), 3.9(4/2), 3.9(12.4/2), upper-case letter 4.5.2(13), 4.5.2(24.2/1), 4.5.5(21), 6.2(11), 7.2(5), 7.6(17.4/3), 9.8(14), 9.10(1/3), 10.2(26), 11.1(6), 11.4.1(10.1/3), 11.5(27/2), 13.1(18), 13.7.2(5/2), 13.9.1(7), 13.11(20), 13.11(21.6/3), 13.13.2(36/2), A.1(1), A.5.1(34), A.5.2(28), A.5.2(34), A.5.3(41.3/2), A.7(6), A.10(8), A.10.7(8/3), A.10.7(12/3), A.10.7(17.3/2), A.10.7(19), A.14(1), A.18.2(231/3), A.18.2(252/2), A.18.2(83/2), A.18.3(145/3), A.18.3(157/2), A.18.3(55/2), A.18.4(3/2), A.18.4(80/2), A.18.5(43/2), A.18.5(44/2), A.18.5(45/2), A.18.5(46/2), A.18.6(56/3), A.18.6(57/2), A.18.7(3/2), A.18.7(101/2), A.18.7(87/2), A.18.7(88/2), A.18.8(65/2), A.18.8(66.1/3), A.18.8(66/2), A.18.8(67/2), A.18.8(68/2), A.18.8(86/2), A.18.8(87/2), A.18.9(114/2), A.18.9(79.1/3), A.18.9(79/3), A.18.9(80/2), A.18.10(201/3), A.18.10(64/3), A.18.26(5/3), A.18.26(9.4/3), A.18.26(9/3), D.2.2(6.1/2), D.8(19), E.3(5/1), G.1.1(40), G.1.2(33), G.1.2(48), H(4.1), H.2(1), K(136.4/2) | Upper_Case_Map<br><i>in</i> Ada.Strings.Maps.Constants A.4.6(5)<br>Upper_Set<br><i>in</i> Ada.Strings.Maps.Constants A.4.6(4)<br>US<br><i>in</i> Ada.Characters.Latin_1 A.3.3(6)<br>usage name 3.1(10)<br>use-visible 8.3(4), 8.4(9)<br>use_clause 8.4(2)<br><i>used</i> 3.11(4/1), 10.1.2(3), 12.1(5), P<br>Use_Error<br><i>in</i> Ada.Direct_IO A.8.4(18)<br><i>in</i> Ada.Directories A.16(43/2)<br><i>in</i> Ada.IO_Exceptions A.13(4)<br><i>in</i> Ada.Sequential_IO A.8.1(15)<br><i>in</i> Ada.Streams.Stream_IO A.12.1(26)<br><i>in</i> Ada.Text_IO A.10.1(85)<br>use_package_clause 8.4(3)<br><i>used</i> 8.4(2), P<br>use_type_clause 8.4(4/3)<br><i>used</i> 8.4(2), P<br>user-defined assignment 7.6(1)<br>user-defined heap management 13.11(1)<br>user-defined operator 6.6(1)<br>user-defined storage management 13.11(1)<br>UTC_Time_Offset<br><i>in</i> Ada.Calendar.Time_Zones 9.6.1(6/2)<br>UTF-16 A.4.11(38/3)<br>UTF-8 A.4.11(38/3)<br>UTF_16_Wide_String <i>subtype of</i> Wide_String<br><i>in</i> Ada.Strings.UTF_Encoding A.4.11(7/3)<br>UTF_8_String <i>subtype of</i> String<br><i>in</i> Ada.Strings.UTF_Encoding A.4.11(6/3)<br>UTF_Encoding<br><i>child of</i> Ada.Strings A.4.11(3/3)<br>UTF_String <i>subtype of</i> String<br><i>in</i> Ada.Strings.UTF_Encoding A.4.11(5/3) | Value<br><i>in</i> Ada.Calendar.Formatting 9.6.1(36/2), 9.6.1(38/2)<br><i>in</i> Ada.Environment_Variables A.17(4/2)<br><i>in</i> Ada.Numerics.Discrete_Random A.5.2(26)<br><i>in</i> Ada.Numerics.Float_Random A.5.2(14)<br><i>in</i> Ada.Strings.Maps A.4.2(21)<br><i>in</i> Ada.Strings.Wide_Maps A.4.7(21)<br><i>in</i> Ada.Strings.Wide_Wide_Maps A.4.8(21/2)<br><i>in</i> Ada.Task_Attributes C.7.2(4)<br><i>in</i> Interfaces.C.Pointers B.3.2(6), B.3.2(7)<br><i>in</i> Interfaces.C.Strings B.3.1(13), B.3.1(14), B.3.1(15), B.3.1(16)<br>Value attribute 3.5(52)<br>value conversion 4.6(5/2)<br>values<br>belonging to a subtype 3.2(8/2)<br>variable 3.3(13/3)<br>variable object 3.3(13/3)<br>variable view 3.3(13/3)<br>variant 3.8.1(3)<br><i>used</i> 3.8.1(2), P<br><i>See also</i> tagged type 3.9(1)<br>variant_part 3.8.1(2)<br><i>used</i> 3.8(4), P<br>Vector<br><i>in</i> Ada.Containers.Vectors A.18.2(8/2)<br>vector container A.18.2(1/2)<br>Vectors<br><i>child of</i> Ada.Containers A.18.2(6/3)<br>version<br>of a compilation unit E.3(5/1)<br>Version attribute E.3(3)<br>vertical line 2.1(15/2)<br>Vertical_Line<br><i>in</i> Ada.Characters.Latin_1 A.3.3(14)<br>view 3.1(7), N(42/2)<br>of a subtype (implied) 3.1(7.1/3)<br>of a type (implied) 3.1(7.1/3)<br>of an object (implied) 3.1(7.1/3)<br>view conversion 4.6(5/2)<br>virtual function<br><i>See</i> dispatching subprogram 3.9.2(1/2)<br>Virtual_Length<br><i>in</i> Interfaces.C.Pointers B.3.2(13)<br>visibility<br>direct 8.3(2), 8.3(21)<br>immediate 8.3(4), 8.3(21)<br>use clause 8.3(4), 8.4(9)<br>visibility rules 8.3(1)<br>visible 8.3(2), 8.3(14)<br>attribute_definition_clause 8.3(23.1/2)<br>within a pragma in a context_clause 10.1.6(3) |
| Unsuppress pragma 11.5(4.1/2), L(37.2/2)<br>update<br>the value of an object 3.3(14)<br><i>in</i> Interfaces.C.Strings B.3.1(18), B.3.1(19)<br>Update_Element<br><i>in</i> Ada.Containers.Doubly_Linked_Lists A.18.3(17/2)<br><i>in</i> Ada.Containers.Hashed_Maps A.18.5(17/2)<br><i>in</i> Ada.Containers.Indefinite_Holders A.18.18(15/3)<br><i>in</i> Ada.Containers.Multiway_Trees A.18.10(25/3)<br><i>in</i> Ada.Containers.Ordered_Maps A.18.6(16/2)<br><i>in</i> Ada.Containers.Vectors A.18.2(33/2), A.18.2(34/2)<br>Update_Element_Preserving_Key<br><i>in</i> Ada.Containers.Hashed_Sets A.18.8(58/2)<br><i>in</i> Ada.Containers.Ordered_Sets A.18.9(73/2)<br>Update_Error<br><i>in</i> Interfaces.C.Strings B.3.1(20)<br>upper bound<br>of a range 3.5(4)                                                                                                                                                                                                                                                                | <b>V</b><br>Val attribute 3.5.5(5)<br>Valid<br><i>in</i> Ada.Text_IO.Editing F.3.3(5), F.3.3(12)<br><i>in</i> Interfaces.COBOL B.4(33), B.4(38), B.4(43)<br>Valid attribute 13.9.2(3), H(6)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

|                                                                 |                                  |                                            |                                       |                                         |                                             |
|-----------------------------------------------------------------|----------------------------------|--------------------------------------------|---------------------------------------|-----------------------------------------|---------------------------------------------|
| within a pragma that appears at the place of a compilation unit | 10.1.6(5)                        | Wide_Constants                             | <i>child of</i> Ada.Strings.Wide_Maps | Wide_Wide_Constants                     | <i>child of</i> Ada.Strings.Wide_Wide_Maps  |
| within a use_clause in a context_clause                         | 10.1.6(3)                        | A.4.7(1/2), A.4.8(28/2)                    | Wide_Exception_Name                   | A.4.8(1/2)                              |                                             |
| within a with_clause                                            | 10.1.6(2/2)                      | in Ada.Exceptions                          | 11.4.1(2/2),                          | Wide_Wide_Encoding                      | <i>child of</i> Ada.Strings.UTF_Encoding    |
| within the parent_unit_name of a library unit                   | 10.1.6(2/2)                      | 11.4.1(5/2)                                | Wide_Expanded_Name                    | A.4.11(22/3), A.4.11(30/3)              |                                             |
| within the parent_unit_name of a subunit                        | 10.1.6(4)                        | in Ada.Tags                                | 3.9(7/2)                              | Wide_Wide_Exception_Name                | <i>in</i> Ada.Exceptions                    |
| visible part                                                    | 8.2(5)                           | Wide_Fixed                                 | <i>child of</i> Ada.Strings           | 11.4.1(2/2),                            | 11.4.1(5/2)                                 |
| of a formal package                                             | 12.7(10/2)                       | Wide_Hash                                  | A.4.7(1/2)                            | Wide_Wide_Expanded_Name                 | <i>in</i> Ada.Tags                          |
| of a generic unit                                               | 8.2(8)                           | <i>child of</i> Ada.Strings                | A.4.7(1/2)                            | Wide_Wide_Fixed                         | <i>child of</i> Ada.Strings                 |
| of a package (other than a generic formal package)              | 7.1(6/2)                         | <i>child of</i> Ada.Strings.Wide_Bounded   | A.4.7(1/2)                            | A.4.8(1/2)                              |                                             |
| of a protected unit                                             | 9.4(11/2)                        | <i>child of</i> Ada.Strings.Wide_Fixed     | A.4.7(1/2)                            | Wide_Wide_Hash                          | <i>child of</i> Ada.Strings                 |
| of a task unit                                                  | 9.1(9)                           | <i>child of</i> Ada.Strings.Wide_Unbounded | A.4.7(1/2)                            | <i>child of</i> Ada.Strings.Wide_Wide - | Bounded                                     |
| of a view of a callable entity                                  | 8.2(6)                           | A.4.7(1/2)                                 | Wide_Image attribute                  | A.4.8(1/2)                              | <i>child of</i> Ada.Strings.Wide_Wide_Fixed |
| of a view of a composite type                                   | 8.2(7)                           | Wide_Map                                   | 3.5(28)                               | A.4.8(1/2)                              | <i>child of</i> Ada.Strings.Wide_Wide -     |
| volatile                                                        | C.6(8)                           | Wide_Maps                                  | <i>child of</i> Ada.Strings           | A.4.8(1/2)                              | Unbounded                                   |
| Volatile pragma                                                 | C.6(4), L(38)                    | wide_nul                                   | A.4.7(3)                              | Wide_Wide_Image attribute               | 3.5(27.1/2)                                 |
| Volatile_Components pragma                                      | C.6(6), L(39)                    | <i>in</i> Interfaces.C                     | B.3(31/1)                             | Wide_Wide_Maps                          | <i>child of</i> Ada.Strings                 |
| VT                                                              | <i>in</i> Ada.Characters.Latin_1 | Wide_Space                                 | A.4.1(4/2)                            | A.4.8(3/2)                              |                                             |
| VTs                                                             | <i>in</i> Ada.Characters.Latin_1 | Wide_String                                | A.1(41)                               | Wide_Wide_Space                         | <i>in</i> Ada.Strings                       |
|                                                                 |                                  | Wide_Text_IO                               | A.11(2/2)                             | A.4.1(4/2)                              |                                             |
|                                                                 |                                  | <i>child of</i> Ada                        | A.11(2/2)                             | Wide_Wide_String                        | <i>in</i> Standard                          |
|                                                                 |                                  | Wide_Unbounded                             | <i>child of</i> Ada.Strings           | A.1(42.1/2)                             |                                             |
|                                                                 |                                  | Wide_Value attribute                       | 3.5(40)                               | Wide_Wide_Text_IO                       | <i>child of</i> Ada                         |
|                                                                 |                                  | Wide_Wide_Bounded                          | <i>child of</i> Ada.Strings           | A.11(3/2)                               |                                             |
|                                                                 |                                  | Wide_Wide_Character                        | 3.5.2(3.1/2)                          | Wide_Wide_Unbounded                     | <i>child of</i> Ada.Strings                 |
|                                                                 |                                  | <i>in</i> Standard                         | A.1(36.2/2)                           | A.4.8(1/2)                              |                                             |
|                                                                 |                                  | Wide_Wide_Character_Mapping                | <i>in</i> Ada.Strings.Wide_Wide_Maps  | Wide_Wide_Value attribute               | 3.5(39.1/2)                                 |
|                                                                 |                                  | A.4.8(20/2)                                |                                       | Wide_Wide_Width attribute               | 3.5(37.1/2)                                 |
|                                                                 |                                  | Wide_Wide_Character_Mapping_Function       | <i>in</i> Ada.Strings.Wide_Wide_Maps  | Wide_Width attribute                    | 3.5(38)                                     |
|                                                                 |                                  | <i>in</i> Ada.Strings.Wide_Wide_Maps       | A.4.8(26/2)                           | Width attribute                         | 3.5(39)                                     |
|                                                                 |                                  | Wide_Wide_Character_Range                  | <i>in</i> Ada.Strings.Wide_Wide_Maps  | with_clause                             | 10.1.2(4/2)                                 |
|                                                                 |                                  | A.4.8(6/2)                                 |                                       | mentioned in                            | 10.1.2(6/2)                                 |
|                                                                 |                                  | Wide_Wide_Character_Ranges                 | <i>in</i> Ada.Strings.Wide_Wide_Maps  | named in                                | 10.1.2(6/2)                                 |
|                                                                 |                                  | <i>in</i> Ada.Strings.Wide_Wide_Maps       | A.4.8(7/2)                            | used                                    | 10.1.2(3), P                                |
|                                                                 |                                  | Wide_Wide_Character_Sequence subtype       | <i>of</i> Wide_Wide_String            | within                                  | immediately                                 |
|                                                                 |                                  | <i>in</i> Ada.Strings.Wide_Wide_Maps       | A.4.8(16/2)                           | word                                    | 13.3(8)                                     |
|                                                                 |                                  | Wide_Wide_Character_Set                    | <i>in</i> Ada.Strings.Wide_Wide_Maps  | Word_Size                               | <i>in</i> System                            |
|                                                                 |                                  | <i>in</i> Ada.Strings.Wide_Wide_Maps       | A.4.8(4/2)                            | 13.7(13)                                |                                             |
|                                                                 |                                  | Wide_Wide_Characters                       | <i>child of</i> Ada                   | Write                                   | <i>in</i> Ada.Direct_IO                     |
|                                                                 |                                  | A.3.1(6/2)                                 |                                       | <i>in</i> Ada.Sequential_IO             | A.8.4(13)                                   |
|                                                                 |                                  |                                            |                                       | <i>in</i> Ada.Storage_IO                | A.8.1(12)                                   |
|                                                                 |                                  |                                            |                                       | <i>in</i> Ada.Streams                   | A.9(7)                                      |
|                                                                 |                                  |                                            |                                       | <i>in</i> Ada.Streams.Stream_IO         | A.12.1(18),                                 |
|                                                                 |                                  |                                            |                                       | A.12.1(19)                              |                                             |
|                                                                 |                                  |                                            |                                       | <i>in</i> System.RPC                    | E.5(8)                                      |
|                                                                 |                                  |                                            |                                       | Write attribute                         | 13.13.2(3), 13.13.2(11)                     |
|                                                                 |                                  |                                            |                                       | Write clause                            | 13.3(7/2), 13.13.2(38/3)                    |
|                                                                 |                                  |                                            |                                       | <b>X</b>                                |                                             |
|                                                                 |                                  |                                            |                                       | xor operator                            | 4.4(1), 4.5.1(2)                            |

**Y**

Year

in Ada.Calendar 9.6(13)

in Ada.Calendar.Formatting

9.6.1(21/2)

Year\_Number *subtype of* Integer

in Ada.Calendar 9.6(11/2)

Yen\_Sign

in Ada.Characters.Latin\_1 A.3.3(21/3)

Yield

in Ada.Dispatching D.2.1(1.3/3)

Yield\_To\_Higher

in Ada.Dispatching.Non\_Preemptive

D.2.4(2.2/3)

Yield\_To\_Same\_Or\_Higher

in Ada.Dispatching.Non\_Preemptive

D.2.4(2.2/3)